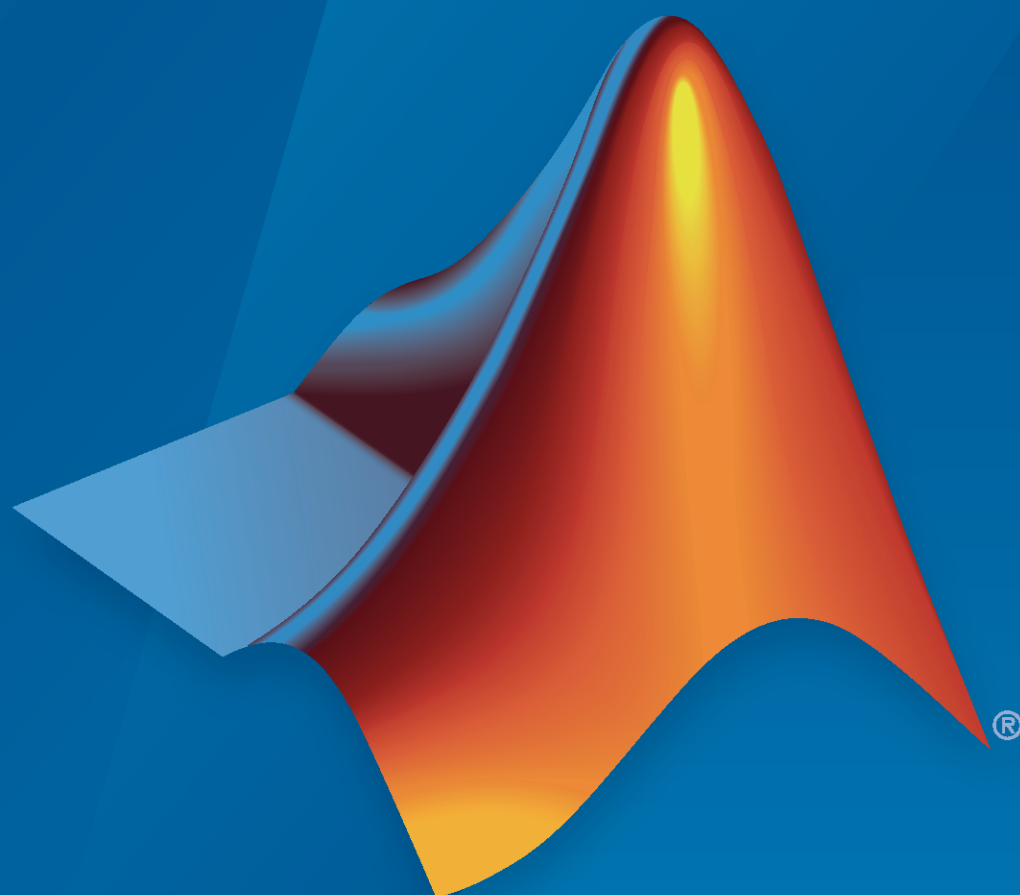


Reinforcement Learning Toolbox™

User's Guide



MATLAB®

R2026a



How to Contact MathWorks



Latest news: www.mathworks.com

Sales and services: www.mathworks.com/sales_and_services

User community: www.mathworks.com/matlabcentral

Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Reinforcement Learning Toolbox™ User's Guide

© COPYRIGHT 2019–2026 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

March 2019	Online only	New for Version 1.0 (Release 2019a)
September 2019	Online only	Revised for Version 1.1 (Release 2019b)
March 2020	Online only	Revised for Version 1.2 (Release 2020a)
September 2020	Online only	Revised for Version 1.3 (Release 2020b)
March 2021	Online only	Revised for Version 2.0 (Release 2021a)
September 2021	Online only	Revised for Version 2.1 (Release 2021b)
March 2022	Online only	Revised for Version 2.2 (Release 2022a)
September 2022	Online only	Revised for Version 2.3 (Release 2022b)
March 2023	Online only	Revised for Version 2.4 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Revised for Version 25.2 (R2025b)
March 2026	Online only	Revised for Version 26.1 (R2026a)

1

Getting Started

Reinforcement Learning Toolbox Product Description	1-2
What Is Reinforcement Learning?	1-3
Reinforcement Learning Workflow	1-5
Reinforcement Learning for Control Systems Applications	1-7
Train Reinforcement Learning Agent in MDP Environment	1-10
Train Reinforcement Learning Agent in Basic Grid World	1-16

2

Create Environments

Reinforcement Learning Environments	2-2
Environment Objects	2-4
Markov Decision Process (MDP) Environments	2-4
Predefined Control System Environments	2-5
Custom Environments	2-5
Neural Network Environments	2-7
Use Predefined Grid World Environments	2-9
Basic Grid World	2-9
Deterministic Waterfall Grid World	2-14
Stochastic Waterfall Grid World	2-20
Create Custom Grid World Environments	2-28
Create Grid World Object	2-29
Configure Grid World Object	2-36
Create Grid World Environment from Grid World Object	2-36
Use Predefined Control System Environments	2-42
MATLAB Double-Integrator Environment	2-43
MATLAB Cart-Pole Environment	2-49
MATLAB Simple Pendulum Swing-Up with Image Observation Environment	2-55
Simulink Simple Pendulum Swing-up Environment	2-62
Simscape Cart-Pole Swing-up Environment	2-69

Use Predefined Multiagent Environments	2-77
MATLAB Pusher Environment	2-77
Simulink Pusher Environment	2-85
Create Custom Environment Using Step and Reset Functions	2-94
Create Custom Environment from Class Template	2-103
Create Template Class	2-103
Environment Properties	2-103
Required Functions	2-104
Optional Functions	2-106
Environment Visualization	2-107
Instantiate Custom Environment	2-108
Create Custom Simulink Environments	2-110
Algebraic Loops Between Environment and Agent	2-111
Reset Function for Simulink Environments	2-112
Create and Simulate Same Environment in Both MATLAB and Simulink	2-114
Define Observation and Reward Signals in Custom Environments	2-127
Strict Causality and Time-Discretization	2-127
Observation	2-128
Reward	2-129
Water Tank Custom Simulink Environment for Reinforcement Learning	2-132
Load MATLAB Environments in Reinforcement Learning Designer ...	2-138
Load Simulink Environments in Reinforcement Learning Designer ...	2-142

Create Agents

3

Reinforcement Learning Agents	3-2
Agent Objects	3-3
Built-In Agents	3-3
Choosing Agent Type	3-6
Model-Based Policy Optimization	3-8
Extract Policy Objects from Agents	3-9
Custom Agents	3-9
Create Agents Using Reinforcement Learning Designer	3-11
Create Agent	3-11
Import Agent	3-13
Edit Agent Options	3-13
Import Agent Options	3-15
Import Actor and Critic	3-15
Edit Actor or Critic Network	3-17
Import Actor or Critic Network	3-18

Export Agents and Agent Components	3-19
Q-Learning Agent	3-22
Critic Used by the Q-Learning Agent	3-22
Q-Learning Agent Creation	3-22
Q-Learning Agent Initialization	3-23
Q-Learning Algorithm	3-23
SARSA Agent	3-25
Critic Used by the SARSA Agent	3-25
SARSA Agent Creation	3-25
SARSA Agent Initialization	3-26
SARSA Training Algorithm	3-26
LSPI Agent	3-28
Critic Used by LSPI Agent	3-28
LSPI Agent Creation	3-29
LSPI Agent Initialization	3-29
LSPI Training Algorithm	3-29
Deep Q-Network (DQN) Agent	3-32
Critics Used by the DQN Agent	3-32
DQN Agent Creation	3-33
DQN Agent Initialization	3-33
DQN Training Algorithm	3-34
REINFORCE Policy Gradient (PG) Agent	3-37
Actor and Critic Used by the PG Agent	3-38
PG Agent Creation	3-38
PG Agent Initialization	3-39
REINFORCE Training Algorithm	3-39
Actor-Critic (AC) Agent	3-44
Actor and Critic Used by the AC Agent	3-45
AC Agent Creation	3-45
AC Agent Initialization	3-46
AC Training Algorithm	3-46
Proximal Policy Optimization (PPO) Agent	3-50
Actor and Critic Used by the PPO Agent	3-51
PPO Agent Creation	3-51
PPO Agent Initialization	3-52
PPO Training Algorithm	3-53
Trust Region Policy Optimization (TRPO) Agent	3-58
Actor and Critic Used by the TRPO Agent	3-59
TRPO Agent Creation	3-59
TRPO Agent Initialization	3-60
Trust Region Policy Optimization	3-60
TRPO Training Algorithm	3-62
Deep Deterministic Policy Gradient (DDPG) Agent	3-67
Actor and Critics Used by the DDPG Agent	3-67
DDPG Agent Creation	3-68
DDPG Agent Initialization	3-68

DDPG Training Algorithm	3-69
Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent	3-72
Actor and Critics Used by the TD3 Agent	3-73
TD3 Agent Creation	3-73
TD3 Agent Initialization	3-74
TD3 Training Algorithm	3-74
Soft Actor-Critic (SAC) Agent	3-78
Actor and Critics Used by the SAC Agent	3-79
SAC Agent Creation	3-80
SAC Agent Initialization	3-81
SAC Training Algorithm	3-82
Model-Based Policy Optimization (MBPO) Agent	3-86
MBPO Agent Initialization	3-87
MBPO Training Algorithm	3-88
Tips	3-90
Create Custom Reinforcement Learning Agents	3-92
Create Template Class	3-92
Agent Properties	3-92
Constructor Function	3-93
Actor and Critic	3-94
Required Functions	3-94
Optional Functions	3-97
Create Custom Agent	3-97

Actors, Critics, and Policy Objects

4

Create Actors, Critics, and Policy Objects	4-2
Actors and Critic Objects	4-2
Policy Objects	4-5
Table Approximation Models	4-7
Custom Basis Function Approximation Models	4-7
Neural Network Approximation Models	4-8
Create Agent from Actor and Critic	4-15
Import Neural Network Models Using ONNX	4-16
Import Actor and Critic for Image Observation Application	4-16

Train and Simulate Agents

5

Train Reinforcement Learning Agents	5-3
Training Algorithm	5-4
Reinforcement Learning Training Monitor	5-5
Save Candidate Agents During Training	5-6

Validate Trained Agent	5-6
Address Memory Issues During Training	5-7
Visualize Environment State During Training	5-7
Evaluate Agents During Training	5-7
Train Agent Offline from a Data Set	5-8
Use Evolutionary Strategies	5-8
Multiagent Training	5-8
Why Is My Training Not Converging?	5-9
Results Reproducibility	5-10
Design and Train Agent Using Reinforcement Learning Designer	5-13
Specify Training Options in Reinforcement Learning Designer	5-29
Specify Basic Options	5-29
Specify Parallel Training Options	5-29
Specify Agent Evaluation Options	5-32
Specify Hyperparameter Tuning Options	5-35
Specify Additional Options	5-48
Specify Simulation Options in Reinforcement Learning Designer	5-50
Specify Simulation Options in Reinforcement Learning Designer	5-50
Specify Basic Options	5-50
Specify Parallel Simulation Options	5-50
Tune Hyperparameters Using Reinforcement Learning Designer	5-53
Train Default DQN Agent to Balance Discrete Cart-Pole	5-64
Train Default DQN Agent to Swing Up and Balance Discrete Pendulum	5-70
Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum	5-78
Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole	5-85
Train Default PPO Agent for Discrete Lander Vehicle	5-93
Train LSPI Agent to Balance Discrete Cart-Pole	5-102
Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole	5-109
Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator	5-115
Train DDPG Agent with Custom Networks Using Image Observation .	5-122
Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle	5-132
Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal	5-141

Train Agents Using Parallel Computing and GPUs	5-151
Using Multiple Processes	5-151
Agent-Specific Parallel Training Considerations	5-153
Using GPUs	5-154
Using Both Multiple Processes and GPUs	5-155
Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing	5-157
Train PPO Agent with Curriculum Learning for a Lane Keeping Application	5-164
Train DQN Agent Using Hindsight Experience Replay	5-176
Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum	5-184
Train Biped Robot to Walk Using Evolution Strategy-Reinforcement Learning Agents	5-196
Transfer Learning: Fine-Tune DQN Agent for Pendulum Swing-Up from Earth to Mars	5-207
Create DQN Agent Using Deep Network Designer and Train Using Image Observations	5-217
Log Training Data to Disk	5-231
Train Agent or Tune Environment Parameters Using Parameter Sweeping	5-240
Tune Hyperparameters Using Bayesian Optimization	5-250
Configure Exploration for Reinforcement Learning Agents	5-259
Train Reinforcement Learning Policy Using Custom Training Loop ...	5-273
Custom Training Loop with Simulink Action Noise	5-282
Custom PPO Training Loop with Random Network Distillation	5-298
Custom DQN Training Loop with LSTM Network	5-317
Create and Train Custom PG Agent	5-334
Create and Train Custom LQR Agent	5-345
Train MBPO Agent to Balance Continuous Cart-Pole System	5-356
Model-Based Reinforcement Learning Using Custom Training Loop ..	5-368

Generate Code from Trained Reinforcement Learning Policies	6-2
Generate Code Using GPU Coder	6-2
Generate Code Using MATLAB Coder	6-3
Generate Policy Block for Deployment	6-7
Deploy Trained Reinforcement Learning Policy as Microservice Docker Image	6-16
Run SIL and PIL Verification for Reinforcement Learning	6-17
Examine Approaches to Fine Tune a Deployed Policy	6-28
Agent Runs on Desktop and Interacts with Real World	6-28
Agent Runs on Dedicated Hardware Board	6-28
Desktop Algorithm Trains Deployed Policy	6-29
Summary: Advantages and Disadvantages	6-29
Train Policy Deployed on Raspberry Pi	6-31
Control a Quanser QUBE Pendulum with a Raspberry Pi using Reinforcement Learning	6-50

Compare Agents on Basic Grid World	7-2
Compare Agents on Deterministic Waterfall Grid World	7-36
Compare Agents on Stochastic Waterfall Grid World	7-70
Compare Agents on Discrete Double-Integrator	7-104
Compare Agents on Discrete Cart-Pole	7-121
Compare Agents on the Discrete Pendulum Swing-Up	7-138
Compare Agents on Discrete Simscape Cart-Pole Swing-Up	7-157
Compare Agents on Discrete Pendulum Swing-Up with Image	7-174
Compare Agents on Continuous Double-Integrator	7-189
Compare Agents on Continuous Cart Pole	7-208
Compare Agents on Continuous Pendulum Swing-Up	7-228

Compare Agents on Continuous Cart Pole Swing-Up	7-251
Compare Agents on Continuous Pendulum Swing-Up with Image	7-268
Compare DDPG Agent to LQR Controller	7-285

Application Examples

8

Control Water Level in a Tank Using a DDPG Agent	8-2
Train Default TD3 Agent to Control Quanser QUBE Pendulum	8-14
Train DDPG Agent to Control Two-Thruster Sliding Vehicle	8-22
Train Multiple Agents to Perform Collaborative Task	8-31
Train Multiple Agents for Area Coverage	8-44
Train Multiple Agents for Path Following Control	8-57
Train DDPG Agent for Adaptive Cruise Control	8-68
Train DQN Agent for Lane Keeping Assist	8-76
Train PPO Agent for Automatic Parking Valet	8-85
Train DDPG Agent for Path-Following Control	8-97
Train Hybrid SAC Agent for Path-Following Control	8-109
Train Hybrid-Action PPO Agent for Path-Following Control	8-120
Train DQN Agent for Lane Keeping Assist Using Parallel Computing .	8-134
Train Biped Robot to Walk Using Reinforcement Learning Agents . . .	8-144
Quadruped Robot Locomotion Using DDPG Agent	8-158
Add Safety Constraint to Simulate Two-Link Robot with SAC Agent . .	8-168
Train SAC Agent for Ball Balance Control	8-181
Automatic Parking Valet with Unreal Engine Simulation	8-197
Generate Reward Function from a Model Predictive Controller for a Servomotor	8-214
Generate Reward Function from a Model Verification Block for a Water Tank System	8-227

Train TD3 Agent for PMSM Control	8-242
Imitate MPC Controller for Lane Keeping Assist	8-256
Train DDPG Agent with Pretrained Actor Network	8-265
Imitate Nonlinear MPC Controller for Sliding Robot	8-275
Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning	8-284
Train DQN Agent with LSTM Network to Control House Heating System	8-299
Train Reinforcement Learning Agent for Simple Contextual Bandit Problem	8-313
Why Solving Regression Using Reinforcement Learning is Not Recommended	8-327
Why Solving Classification Using Reinforcement Learning Is Not Recommended	8-337
Train Agent to Play Turn-Based Game	8-348
Water Distribution System Scheduling Using Reinforcement Learning	8-361
Optimizing Queue Selection Strategies Using Reinforcement Learning	8-372
Identify Vulnerabilities in DC Microgrids	8-383

Getting Started

- “Reinforcement Learning Toolbox Product Description” on page 1-2
- “What Is Reinforcement Learning?” on page 1-3
- “Reinforcement Learning Workflow” on page 1-5
- “Reinforcement Learning for Control Systems Applications” on page 1-7
- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

Reinforcement Learning Toolbox Product Description

Design and train policies using reinforcement learning

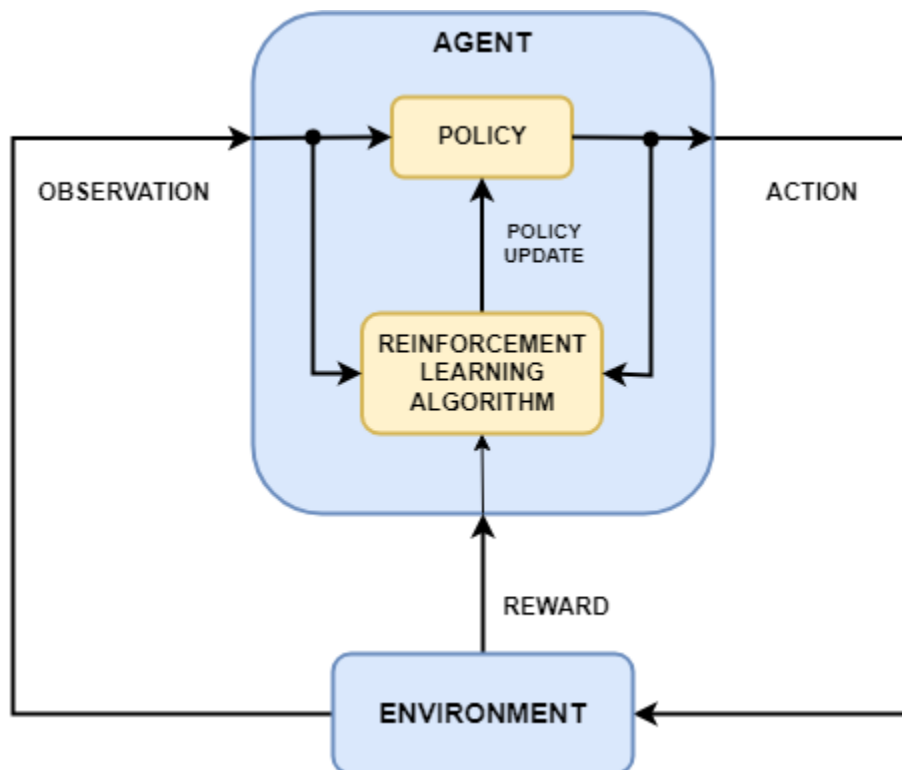
Reinforcement Learning Toolbox™ provides an app, functions, and a Simulink® block for training policies using reinforcement learning algorithms, including DQN, PPO, SAC, and DDPG. You can use these policies to implement controllers and decision-making algorithms for complex applications such as resource allocation, robotics, and autonomous systems.

The toolbox lets you represent policies and value functions using deep neural networks or look-up tables and train them through interactions with environments modeled in MATLAB® or Simulink. You can evaluate the single- or multi-agent reinforcement learning algorithms provided in the toolbox or develop your own. You can experiment with hyperparameter settings, monitor training progress, and simulate trained agents either interactively through the app or programmatically. To improve training performance, simulations can be run in parallel on multiple CPUs, GPUs, computer clusters, and the cloud (with Parallel Computing Toolbox™ and MATLAB Parallel Server™).

Through the ONNX™ model format, existing policies can be imported from deep learning frameworks such as TensorFlow™ Keras and PyTorch (with Deep Learning Toolbox™). You can generate optimized C, C++, and CUDA® code to deploy trained policies on microcontrollers and GPUs. The toolbox includes reference examples to help you get started.

What Is Reinforcement Learning?

Reinforcement learning is a goal-directed computational approach in which an agent learns to perform a task by interacting with a dynamic environment. This learning approach enables the agent to make a series of decisions to maximize the cumulative reward for the task without human intervention and without explicitly programming the agent to achieve a goal. The following diagram shows a general representation of a reinforcement learning scenario.



The agent contains two components: a *policy* and a *learning algorithm*.

- The policy is a mapping that selects actions based on the observations from the environment. Typically, the policy is a function approximator with tunable parameters, such as a deep neural network.
- The learning algorithm continuously updates the policy parameters based on the actions, observations, and reward. The goal of the learning algorithm is to find an optimal policy that maximizes the cumulative reward received during the task.

The reinforcement learning algorithm trains the *agent* to complete a task within the *environment* (which is unknown to the agent). The agent receives *observations* and a *reward* from the environment and sends *actions* to the environment. The reward is a measure of how successful an action is with respect to eventually completing the task goal.

In other words, reinforcement learning involves an agent learning the optimal behavior through repeated trial-and-error interactions with the environment without human involvement.

As an example, consider the task of parking a vehicle using an automated driving system. The goal of this task is for the vehicle computer (**agent**) to park the vehicle in the correct position and

orientation. To do so, the controller uses readings from cameras, accelerometers, gyroscopes, a GPS receiver, and lidar (**observations**) to generate steering, braking, and acceleration commands (**actions**). The action commands are sent to the actuators that control the vehicle. The resulting observations depend on the actuators, sensors, vehicle dynamics, road surface, wind, and many other less-important factors. All these factors, that is, everything that is not the agent, make up the **environment** in reinforcement learning.

To learn how to generate the correct actions from the observations, the computer repeatedly tries to park the vehicle using a trial-and-error process. To guide the learning process, you provide a signal that is one when the car successfully reaches the desired position and orientation and zero otherwise (**reward**). During each trial, the computer selects actions using a mapping (**policy**) initialized with some default values. After each trial, the computer updates the mapping to maximize the reward (**learning algorithm**). This process continues until the computer learns an optimal mapping that successfully parks the car.

For an introduction to the typical workflow used in reinforcement learning, see “Reinforcement Learning Workflow” on page 1-5.

References

- [1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Reinforcement Learning for Control Systems Applications” on page 1-7

Reinforcement Learning Workflow

The general workflow for training an agent using reinforcement learning includes the following steps.



- 1 **Formulate problem** — Define the task for the agent to learn, including how the agent interacts with the environment and any primary and secondary goals the agent must achieve.
- 2 **Create environment** — Define the environment within which the agent operates, including the interface between agent and environment and the environment dynamic model. For more information, see “Reinforcement Learning Environments” on page 2-2.
- 3 **Define reward** — Specify the reward signal that the agent uses to measure its performance against the task goals and how to calculate this signal from the environment. For more information, see “Define Observation and Reward Signals in Custom Environments” on page 2-127.
- 4 **Create agent** — Create the agent. You can create a default built-in agent, or a built-in agent with custom actor and critic objects. For more information, see “Create Actors, Critics, and Policy Objects” on page 4-2 and “Reinforcement Learning Agents” on page 3-2.
- 5 **Train agent** — Train the agent approximators using the defined environment, reward, and agent learning algorithm. For more information, see “Train Reinforcement Learning Agents” on page 5-3.
- 6 **Simulate agent** — Evaluate the performance of the trained agent by simulating the agent and environment together. For more information, see “Train Reinforcement Learning Agents” on page 5-3.
- 7 **Deploy policy** — Deploy the trained policy approximator using, for example, generated GPU code. For more information, see “Generate Code from Trained Reinforcement Learning Policies” on page 6-2.

Training an agent using reinforcement learning is an iterative process. Decisions and results in later stages can require you to return to an earlier stage in the learning workflow. For example, if the training process does not converge to an optimal policy within a reasonable amount of time, you might have to update some of the following before retraining the agent:

- Training settings
- Learning algorithm configuration
- Policy and value function (actor and critic) approximators
- Reward signal definition
- Action and observation signals
- Environment dynamics

See Also

Related Examples

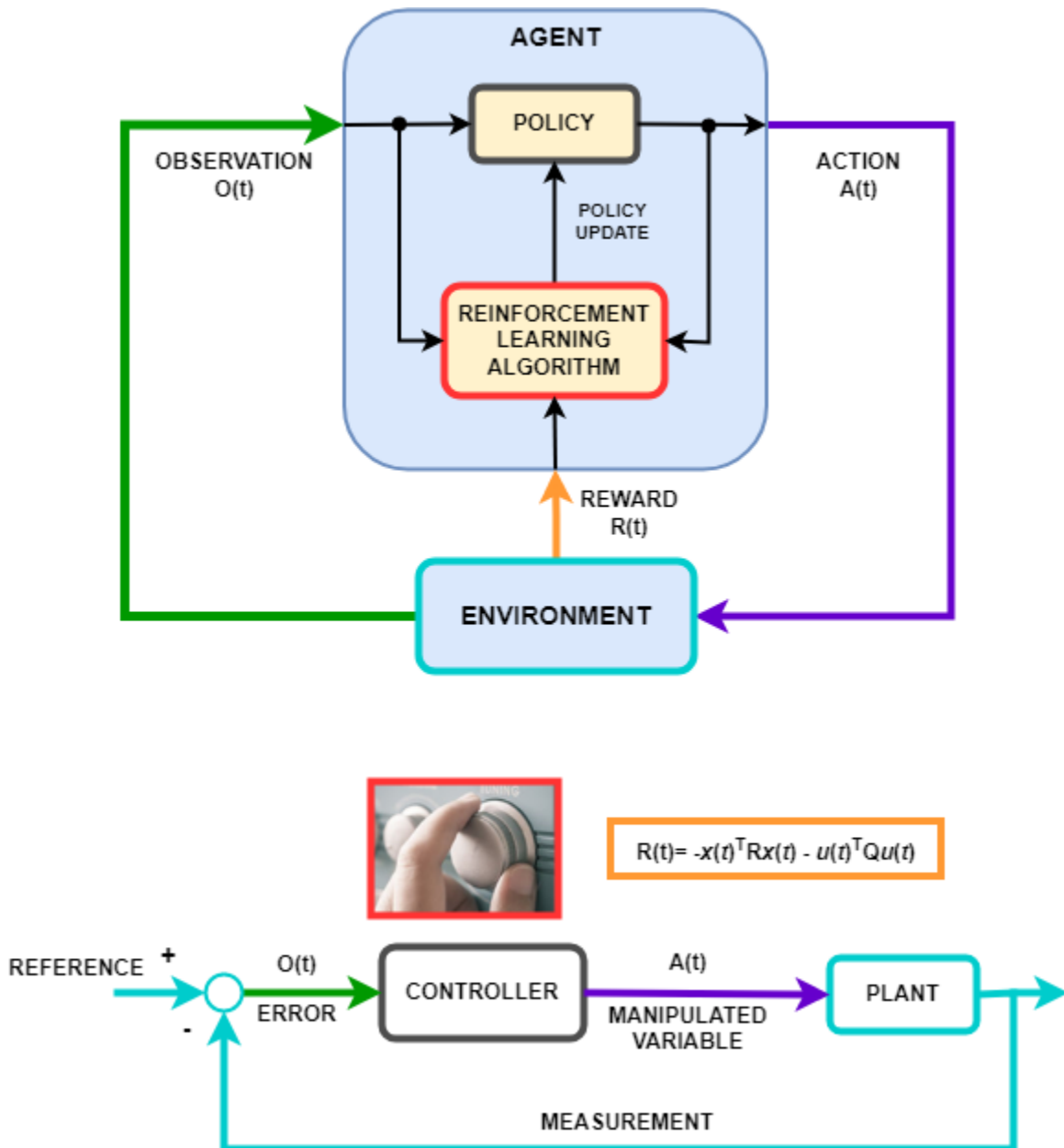
- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Control Water Level in a Tank Using a DDPG Agent” on page 8-2

More About

- “What Is Reinforcement Learning?” on page 1-3
- “Reinforcement Learning Environments” on page 2-2
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3
- “Reinforcement Learning for Control Systems Applications” on page 1-7

Reinforcement Learning for Control Systems Applications

The behavior of a reinforcement learning policy—that is, how the policy observes the environment and generates actions to complete a task in an optimal manner—is similar to the operation of a controller in a control system. Reinforcement learning can be translated to a control system representation using the following mapping.



Reinforcement Learning	Control Systems
Environment	Everything that is not the controller — In the preceding diagram, the environment includes the plant, the reference signal, and the calculation of the error. In general, the environment can also include additional elements, such as: • Measurement noise • Disturbance signals • Filters • Analog-to-digital and digital-to-analog converters
Observation	Any measurable value from the environment that is visible to the agent — In the preceding diagram, the controller can see the error signal from the environment. You can also create agents that observe, for example, the reference signal, measurement signal, and measurement signal rate of change.
Action	Manipulated variables or control actions
Reward	Function of the measurement, error signal, or some other performance metric — For example, you can implement reward functions that minimize the steady-state error while minimizing control effort. When control specifications such as cost and constraint functions are available, you can use the <code>generateRewardFunction</code> function to generate a reward function from an MPC object or model verification blocks. You can then use the generated reward function as a starting point for reward design, for example, by changing the weights or penalty functions.
Learning Algorithm	Adaptation mechanism of an adaptive controller

Many control problems encountered in areas such as robotics and automated driving require complex, nonlinear control architectures. Techniques such as gain scheduling, robust control, and nonlinear model predictive control (MPC) can be used for these problems, but often require significant domain expertise from the control engineer. For example, gains and parameters are difficult to tune. The resulting controllers can pose implementation challenges, such as the computational intensity of nonlinear MPC.

You can use deep neural networks, trained using reinforcement learning, to implement such complex controllers. These systems can be self-taught without intervention from an expert control engineer. Also, once the system is trained, you can deploy the reinforcement learning policy in a computationally efficient way.

You can also use reinforcement learning to create an end-to-end controller that generates actions directly from raw data, such as images. This approach is attractive for video-intensive applications, such as automated driving, since you do not have to manually define and select image features.

See Also

Related Examples

- “Compare DDPG Agent to LQR Controller” on page 7-285

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning” on page 8-284
- “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14

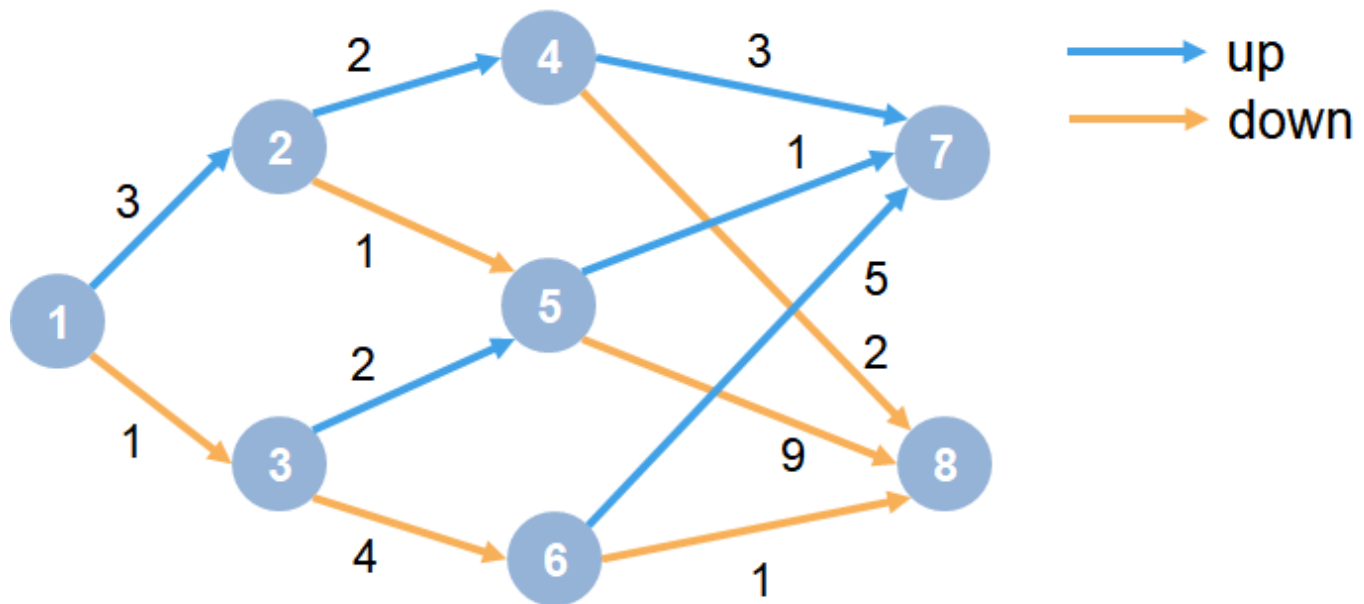
More About

- “What Is Reinforcement Learning?” on page 1-3
- “Reinforcement Learning Workflow” on page 1-5
- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Simulink Environments” on page 2-110
- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Train Reinforcement Learning Agent in MDP Environment

This example shows how to train a Q-learning agent to solve a generic Markov decision process (MDP) environment. For more information on these agents, see “Q-Learning Agent” on page 3-22.

The MDP environment has the following graph.



Here:

- 1 Each circle represents a state.
- 2 At each state there is a decision to go up or down.
- 3 The agent begins from state 1.
- 4 The agent receives a reward equal to the value on each transition in the graph.
- 5 The training goal is to collect the maximum cumulative reward.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create MDP Environment

Create an MDP model with eight states and two actions ("up" and "down").

```
mdp = createMDP(8, ["up"; "down"]);
```

To model the transitions from the above graph, modify the state transition matrix and reward matrix of the MDP. By default, these matrices contain zeros. For more information on creating an MDP model and the properties of an MDP object, see `createMDP`.

Specify the state transition and reward matrices for `mdp`. For example, in the following commands:

- The first two lines specify the transition from state 1 to state 2 by taking action 1 ("up") and a reward of +3 for this transition.
- The next two lines specify the transition from state 1 to state 3 by taking action 2 ("down") and a reward of +1 for this transition.

```
mdp.T(1,2,1) = 1;
mdp.R(1,2,1) = 3;
mdp.T(1,3,2) = 1;
mdp.R(1,3,2) = 1;
```

Similarly, specify the state transitions and rewards for the remaining rules in the graph.

```
% State 2 transition and reward
mdp.T(2,4,1) = 1;
mdp.R(2,4,1) = 2;
mdp.T(2,5,2) = 1;
mdp.R(2,5,2) = 1;
% State 3 transition and reward
mdp.T(3,5,1) = 1;
mdp.R(3,5,1) = 2;
mdp.T(3,6,2) = 1;
mdp.R(3,6,2) = 4;
% State 4 transition and reward
mdp.T(4,7,1) = 1;
mdp.R(4,7,1) = 3;
mdp.T(4,8,2) = 1;
mdp.R(4,8,2) = 2;
% State 5 transition and reward
mdp.T(5,7,1) = 1;
mdp.R(5,7,1) = 1;
mdp.T(5,8,2) = 1;
mdp.R(5,8,2) = 9;
% State 6 transition and reward
mdp.T(6,7,1) = 1;
mdp.R(6,7,1) = 5;
mdp.T(6,8,2) = 1;
mdp.R(6,8,2) = 1;
% State 7 transition and reward
mdp.T(7,7,1) = 1;
mdp.R(7,7,1) = 0;
mdp.T(7,7,2) = 1;
mdp.R(7,7,2) = 0;
% State 8 transition and reward
mdp.T(8,8,1) = 1;
mdp.R(8,8,1) = 0;
```

```
mdp.T(8,8,2) = 1;  
mdp.R(8,8,2) = 0;
```

Specify states "s7" and "s8" as terminal states of the MDP.

```
mdp.TerminalStates = ["s7","s8"];
```

Create the reinforcement learning environment env from mdp.

```
env = rlMDPEnv(mdp);
```

To specify that the initial state of the agent is always state 1, specify a reset function that returns the initial agent state. This function is called at the start of each training or simulation episode. Create an anonymous function handle that sets the initial state to 1.

```
env.ResetFcn = @() 1;
```

Create Q-Learning Agent

To create a Q-learning agent, first create a Q table model using the observation and action specifications from the MDP environment. Set the learning rate of the table model to 0.1.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);  
qTable = rlTable(obsInfo, actInfo);
```

Next create a Q-value critic function from the table model.

```
qFunction = rlQValueFunction(qTable, obsInfo, actInfo);
```

Finally, create the Q-learning agent using this critic function. For this training:

- Specify a discount factor of one to promote undiscounted long term rewards.
- Specify the initial epsilon value 0.9 for the agent's epsilon greedy exploration model.
- Specify a decay rate of 1e-3 and the minimum value of 0.1 for the epsilon parameter. Decaying the exploration gradually enables the agent to exploit its greedy policy toward the latter stages of training.
- Use the stochastic gradient descent with momentum (sgdm) algorithm to update the table model with the learning rate of 0.1.
- Using the L2 regularization factor 0. For this example disabling regularization helps in better estimating the long term undiscounted rewards.

```
agentOpts = rlQAgentOptions;  
agentOpts.DiscountFactor = 1;  
agentOpts.EpsilonGreedyExploration.Epsilon = 0.9;  
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-3;  
agentOpts.EpsilonGreedyExploration.EpsilonMin = 0.1;  
agentOpts.CriticOptimizerOptions = rlOptimizerOptions( ...  
    Algorithm="sgdm", ...  
    LearnRate=0.1, ...  
    L2RegularizationFactor=0);  
qAgent = rlQAgent(qFunction,agentOpts);
```

For more information on creating Q-learning agents, see `rlQAgent` and `rlQAgentOptions`.

Train Q-Learning Agent

To train the agent, first specify the training options. For this example, use the following options:

- Train for 400 episodes, with each episode lasting a maximum of 50 time steps.
- Specify a window length of 30 for averaging the episode rewards.

For more information on training options, see `rlTrainingOptions`.

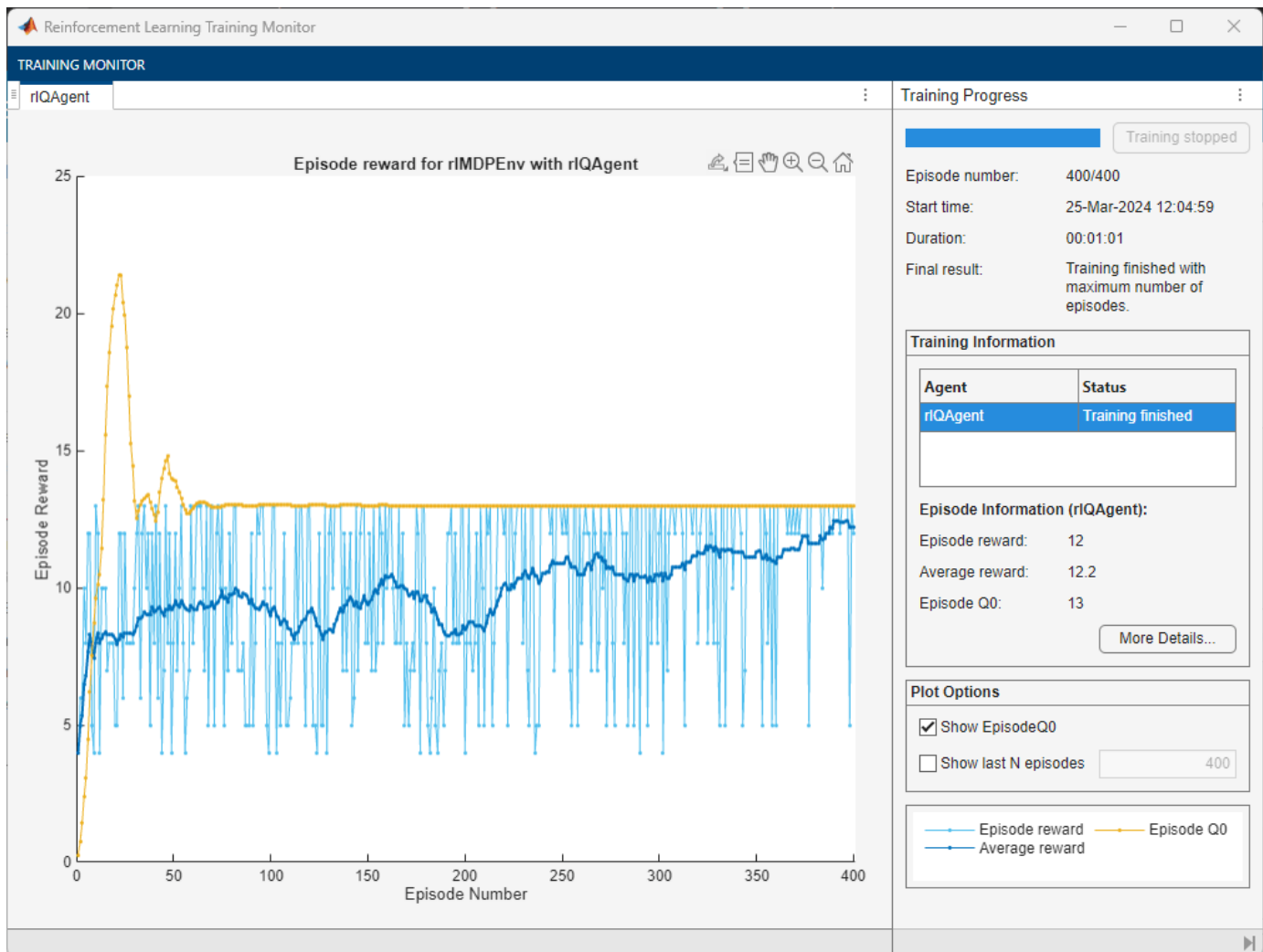
```
trainOpts = rlTrainingOptions;
trainOpts.MaxStepsPerEpisode = 50;
trainOpts.MaxEpisodes = 400;
trainOpts.ScoreAveragingWindowLength = 30;
trainOpts.StopTrainingCriteria = "none";
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. This might take several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(qAgent, env, trainOpts); %#ok<UNRCH>
else
    % Load pretrained agent for the example.
    load("genericMDPQAgent.mat", "qAgent");
end
```



Validate Q-Learning Results

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To validate the training results, simulate the agent in the training environment using the `sim` function. The agent successfully finds the optimal path which results in cumulative reward of 13.

```
Data = sim(qAgent, env);
cumulativeReward = sum(Data.Reward)
```

```
cumulativeReward =
13
```

Because the discount factor is set to 1, the values in the Q table of the trained agent are consistent with the undiscounted returns of the environment.

```
QTable = getLearnableParameters(getCritic(qAgent));
QTable{1}
```

```
ans = 8x2 single matrix
```

```
13.0000    12.0000
 5.0000    10.0000
11.0000     9.0000
 3.0000     2.0000
 1.0000     9.0000
 5.0000     1.0000
      0         0
      0         0
```

Display the true table values for comparison.

```
TrueTableValues = [13,12;5,10;11,9;3,2;1,9;5,1;0,0;0,0]
```

```
TrueTableValues = 8x2
```

```
13    12
 5    10
11     9
 3     2
 1     9
 5     1
 0     0
 0     0
```

The Q table learned from the agent (`QTable{1}`) contains the true value function (`TrueTableValues`). This shows that the Q-Learning agent has learned the correct value function for the problem.

See Also

Functions

`createMDP` | `train` | `sim`

Objects

`rlMDPEnv` | `rlQAgent` | `rlQAgentOptions` | `rlTrainingOptions`

Related Examples

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

More About

- “What Is Reinforcement Learning?” on page 1-3
- “Reinforcement Learning Environments” on page 2-2
- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Q-Learning Agent” on page 3-22
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Train Reinforcement Learning Agent in Basic Grid World

This example shows how to solve a grid world environment using reinforcement learning by training Q-learning and SARSA agents. For more information on these agents, see “Q-Learning Agent” on page 3-22 and “SARSA Agent” on page 3-25.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

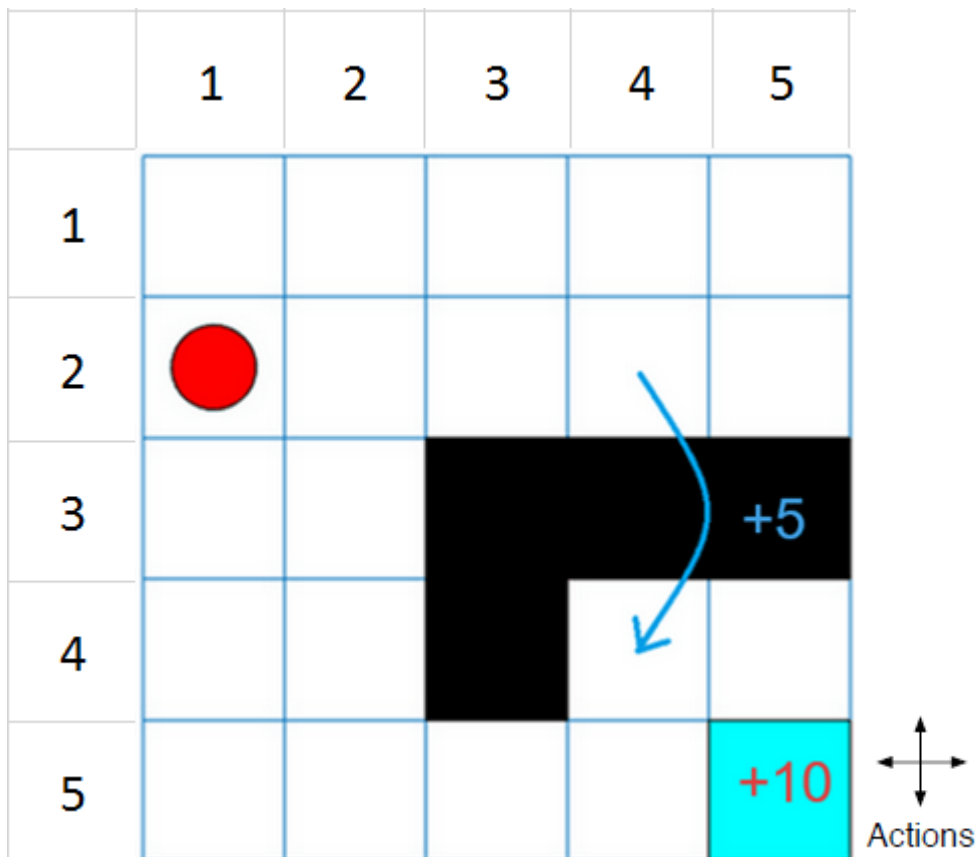
Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

This grid world environment has the following configuration and rules:

- 1 The grid world is 5-by-5 and bounded by borders, with four possible actions (North = 1, South = 2, East = 3, West = 4).
- 2 The agent begins from cell [2,1] (second row, first column).
- 3 The agent receives a reward +10 if it reaches the terminal state at cell [5,5] (blue).
- 4 The environment contains a special jump from cell [2,4] to cell [4,4] with a reward of +5.
- 5 The agent is blocked by obstacles (black cells).
- 6 All other actions result in -1 reward.



Create Grid World Environment

Create the basic grid world environment.

```
env = rlPredefinedEnv("BasicGridWorld");
```

To specify that the initial state of the agent is always [2,1], create a reset function that returns the state number for the initial agent state. This function is called at the start of each training episode and simulation. States are numbered starting at position [1,1]. The state number increases as you move down the first column and then down each subsequent column. Therefore, create an anonymous function handle that sets the initial state to 2.

```
env.ResetFcn = @( ) 2;
```

Use the `getActionInfo` and `getObservationInfo` functions to extract the action and observation specification objects from the environment.

```
actInfo = getActionInfo(env);
obsInfo = getObservationInfo(env);
```

Create Q-Learning Agent

To create a Q-learning agent, first create a Q table using the observation and action specifications from the grid world environment. Set the learning rate of the optimizer to 0.01.

```
qTable = rlTable(obsInfo,actInfo);
```

To approximate the Q-value function within the agent, create a `rlQValueFunction` approximator object, using the table and the environment information.

```
qFcnAppx = rlQValueFunction(qTable,obsInfo,actInfo);
```

Next, create a Q-learning agent using the Q-value function.

```
qAgent = rlQAgent(qFcnAppx);
```

Configure agent options such as the epsilon-greedy exploration and the learning rate for the function approximator.

```
qAgent.AgentOptions.EpsilonGreedyExploration.Epsilon = .04;  
qAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 0.01;
```

For more information on creating Q-learning agents, see `rlQAgent` and `rlQAgentOptions`.

Train Q-Learning Agent

To train the agent, first specify the training options. For this example, use the following options:

- Train for a maximum of 200 episodes. Specify that each episode lasts for most 50 time steps.
- Stop the training when the agent receives an average cumulative reward of 11 over 30 consecutive episodes.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions;  
trainOpts.MaxStepsPerEpisode = 50;  
trainOpts.MaxEpisodes= 200;  
trainOpts.StopTrainingCriteria = "AverageReward";  
trainOpts.StopTrainingValue = 11;  
trainOpts.ScoreAveragingWindowLength = 30;
```

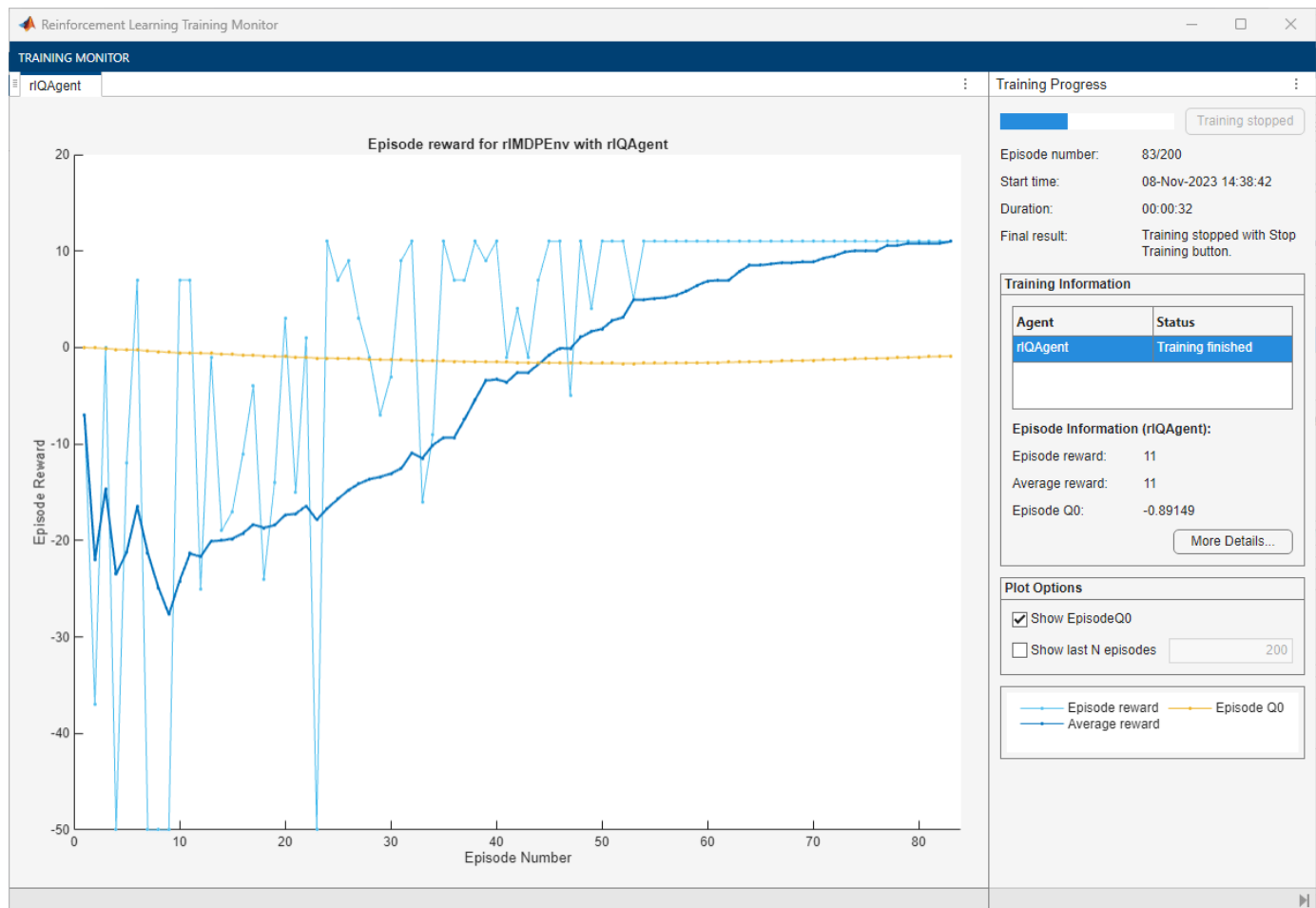
Fix the random stream for reproducibility.

```
rng(0,"twister");
```

Train the Q-learning agent using the `train` function. Training can take several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    qTrainingStats = train(qAgent,env,trainOpts);  
else  
    % Load the pretrained agent for the example.  
    load("basicGWQAgent.mat","qAgent")  
end
```

The **Reinforcement Learning Training Monitor** window opens and displays the training progress.



Validate Q-Learning Results

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

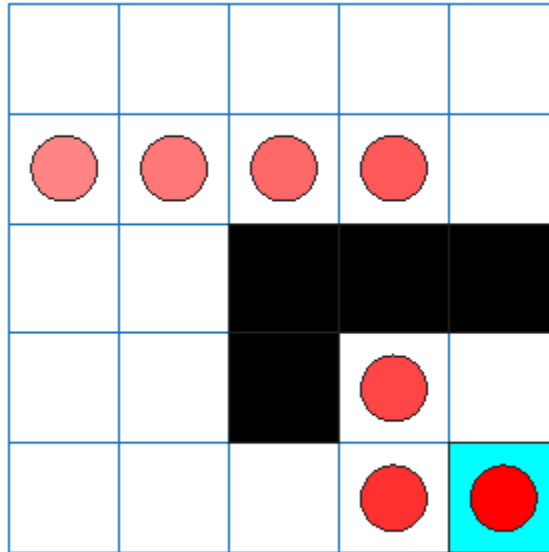
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Before running the simulation, visualize the environment, configure the visualization to maintain a trace of the agent states, and clear any previously existing trace.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

Simulate the agent in the environment using the `sim` function.

```
sim(qAgent, env)
```



The agent trace shows that the agent successfully finds the jump from cell [2,4] to cell [4,4].

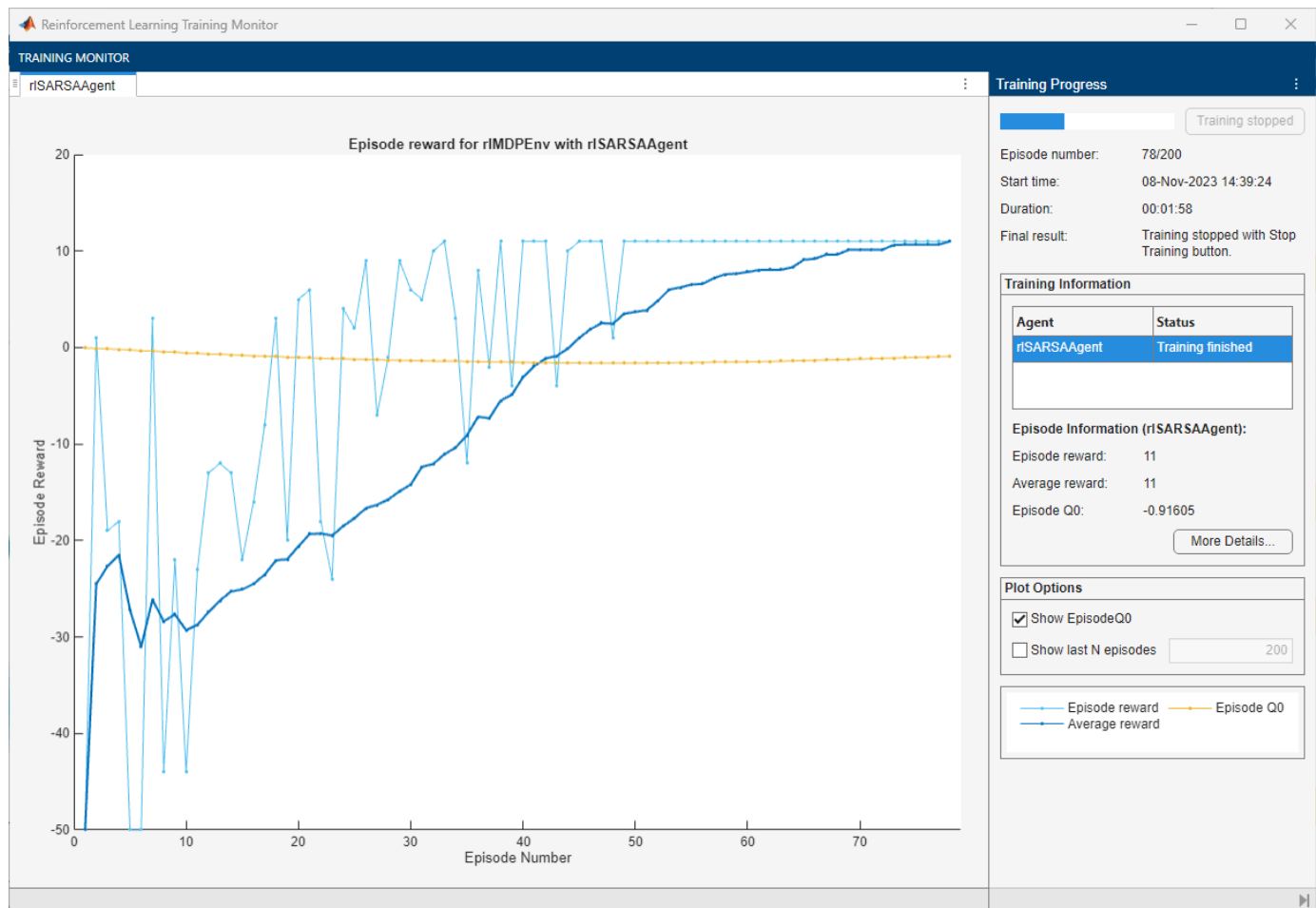
Create and Train SARSA Agent

To create a SARSA agent, use the same Q value function and epsilon-greedy configuration as for the Q-learning agent. For more information on creating SARSA agents, see `rlSARSAAgent` and `rlSARSAAgentOptions`.

```
sarsaAgent = rlSARSAAgent(qFcnAppx);  
sarsaAgent.AgentOptions.EpsilonGreedyExploration.Epsilon = .04;  
sarsaAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 0.01;
```

Train the SARSA agent using the `train` function. Training can take several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    sarsaTrainingStats = train(sarsaAgent,env,trainOpts);  
else  
    % Load the pretrained agent for the example.  
    load("basicGWSarsaAgent.mat","sarsaAgent")  
end
```



Validate SARSA Training

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

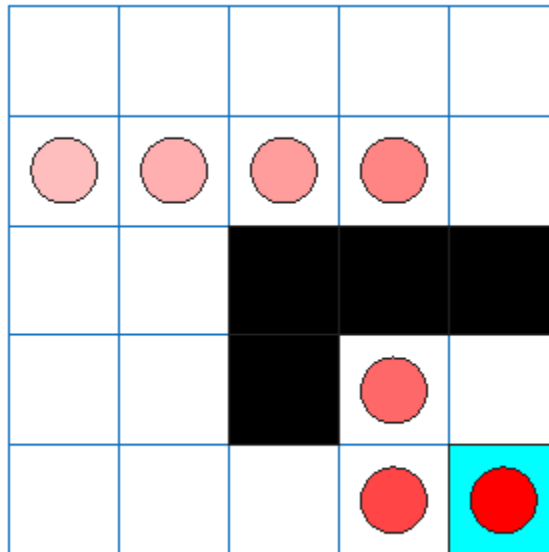
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Before running the simulation, visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

Simulate the agent in the environment.

```
sim(sarsaAgent, env)
```



The SARSA agent finds the same grid world solution as the Q-learning agent.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`createGridWorld` | `sim` | `train`

Objects

`rlSARSAAgentOptions` | `rlSARSAAgent` | `rlMDPEnv` | `rlTrainingOptions` | `rlQAgent` | `rlQAgentOptions`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10

More About

- “Q-Learning Agent” on page 3-22
- “SARSA Agent” on page 3-25
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Create Environments

- “Reinforcement Learning Environments” on page 2-2
- “Use Predefined Grid World Environments” on page 2-9
- “Create Custom Grid World Environments” on page 2-28
- “Use Predefined Control System Environments” on page 2-42
- “Use Predefined Multiagent Environments” on page 2-77
- “Create Custom Environment Using Step and Reset Functions” on page 2-94
- “Create Custom Environment from Class Template” on page 2-103
- “Create Custom Simulink Environments” on page 2-110
- “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114
- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132
- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142

Reinforcement Learning Environments

In a reinforcement learning scenario, where you train an agent to complete a task, the *environment* models the external system (that is, the world) with which the agent interacts. A *multiagent* environment interacts with more than one agent at the same time.

In control systems applications, this external system is often referred to as *the plant*. Any reference signal that might need to be tracked by some of the environment variables is also included in the environment.

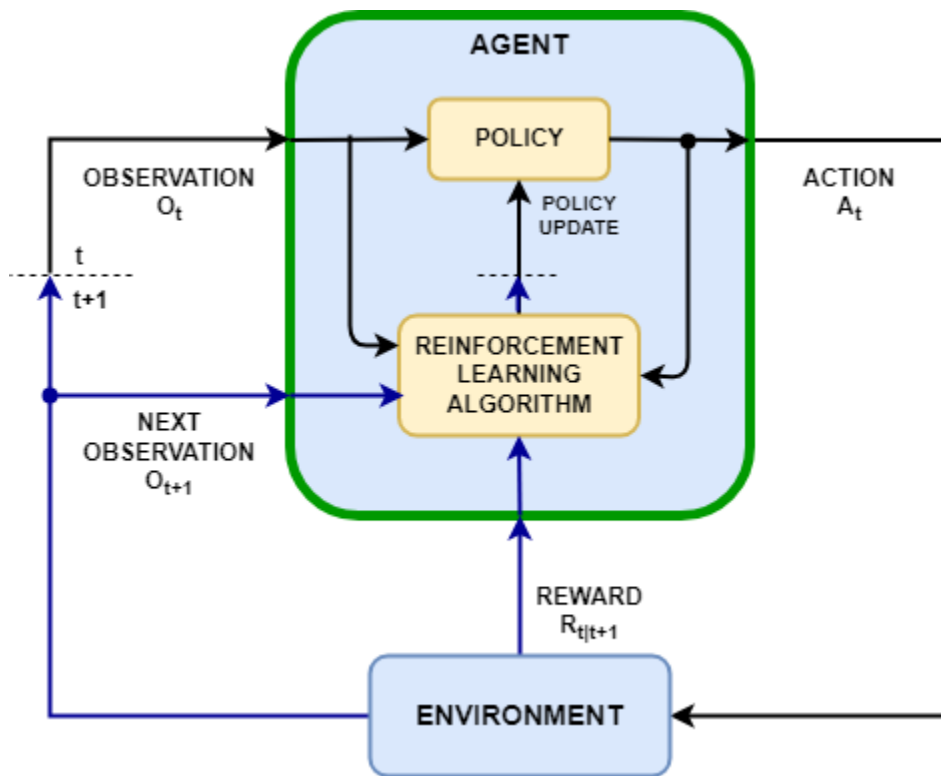
The agent and the environment interact at each of a sequence of *discrete* time steps:

- 1 At a given time step t , the environment is in a state $S(t)$, which results in the observation $O(t)$.

Based on $O(t)$ and its internal policy function, the agent calculates an action $A(t)$.

- 2 Based on both the state $S(t)$ and the action $A(t)$, and according to its internal dynamics, the environment updates its state to $S(t+1)$, which results in the next observation $O(t+1)$.
- 3 Based on $S(t)$, $A(t)$, and $S(t+1)$, the environment also calculates a scalar reward $R(t+1)$. The reward is an immediate measure of how good the action $A(t)$ is. Note that neither the next observation $O(t+1)$ nor the reward $R(t+1)$ depend on the next action $A(t+1)$. In other words, there is *no direct feedthrough* between action and either observation or reward.
- 4 At the next time step $t+1$ the agent receives the observation $O(t+1)$ and the reward $R(t+1)$.
- 5 Based on the history of observations and rewards received, the learning algorithm updates the agent policy parameters in an attempt to improve the policy function. The parameter update might occur at each step or after a subsequence of steps.
- 6 Based on $O(t+1)$ and on its policy function, the agent calculates the next action $A(t+1)$, and the process is repeated.

Starting from time $t=1$ and using subscripts to indicate time, the causal sequence of events, often also called *trajectory*, can be summarized as O_1, A_1, R_2, O_2, A_2 . The interaction between the environment and the agent is also illustrated in the following figure, where the dashed lines represent a delay of one step.



By convention, the observation or action can be divided into one or more *channels*, each of which carries a group of single elements all belonging to either a numeric (infinite and continuous) set or a finite (discrete) set. Each group can be organized according to any number of dimensions (for example a vector or a matrix), and is defined by a specification object. The specification object can be either a `rlNumericSpec` object (for channels carrying continuous signals) or an `rlFiniteSetSpec` object (for channels carrying discrete signals).

For example, an agent tasked with controlling a rover might receive from the environment an observation composed of four channels such as a continuous channel carrying acceleration measurements from accelerometers, another continuous channel carrying angular velocity estimates from an inertial measurement unit, a third continuous channel carrying a 100 by 100 pixels image, where each pixel is represented by an `uint8` value, and a fourth discrete channel carrying a logical value indicating whether the collision sensor is detecting a collision. In this case, the observation is specified by a vector containing three `rlNumericSpec` object followed by one `rlFiniteSetSpec` object. If this agent uses neural networks as the underlying approximator model, the networks tasked to process the observations must have an input layer for each corresponding observation channel.

For non-hybrid environments (that is, environments with an action space that is either discrete or continuous but not both), only one channel is allowed for the action. The reward must be a numeric scalar. For more information on specification objects for groups of actions and observations, see `rlNumericSpec` and `rlFiniteSetSpec`.

Multiagent environments are environments in which you can train and simulate multiple agents together.

Environment Objects

Reinforcement Learning Toolbox represents environments with MATLAB objects. Such objects interact with agents using object functions (methods) such as `step` or `reset`. Specifically, at the beginning of each training or simulation episode, the `reset` function is called (by a built-in training or simulation function) to set the environment initial condition. Then at each training or simulation time step, the `step` function is called to update the state of the environment and return the next state along with a reward.

Note Environment objects are handle objects, so if you copy environment into a new variable, the new variable will also always point to the most recent environment version with updated states. For more information about handle objects, see “Handle Object Behavior”.

After you create an environment object in the MATLAB workspace, you can extract the observation and action specifications from the environment object, and use these specifications to create an agent object that works within your environment.

You can then use both the environment and agent objects as arguments for the built-in functions `train` and `sim`, which train and simulate the agent within the environment, respectively. Alternatively, with some exception explained below, you can create your custom training or simulation loop that calls the environment `reset` and `step` functions directly.

Environments that rely on an underlying Simulink model for the calculation of the state transition, reward, and observation are called Simulink environments. These environments act as an interface so that the reinforcement learning simulation or training function calls the (compiled) Simulink model to generate experiences for the agents. Simulink environments do not support using the `reset` and `step` functions.

Environments that instead rely on MATLAB functions or objects for the calculation of the state transition, reward, and observation, are referred to as MATLAB environments.

The following sections summarize the different types of environment provided by the software.

Markov Decision Process (MDP) Environments

A Markov decision process (MDP) is a discrete time stochastic control process. It provides a mathematical framework for modeling decision making in situations where outcomes are partly random and partly under the control of the decision maker. Reinforcement Learning Toolbox represents MDPs using `rLMDPEnv` objects.

MDP environments are MATLAB environments based on `rLMDPEnv` objects. In these environments, which are very useful for studying optimization problems that can be solved using reinforcement learning, state and observation belong to finite spaces, and state transitions are in general governed by stochastic rules.

Grid world environments are a special case of MDP environments. Here, the state represents a position in a two-dimensional grid, while the action represents a move from the current position to the next, which an agent might attempt. Grid world environments are often used in many introductory reinforcement learning examples.

You can use three types of MDP environment.

- **Predefined grid world environments**

Reinforcement Learning Toolbox provides three predefined grid world environment object types. For predefined environments, all states, actions, and rewards are already defined. You can use them to learn basic reinforcement learning concepts and gain familiarity with Reinforcement Learning Toolbox software features. For an introduction to predefined grid world environments, see “Use Predefined Grid World Environments” on page 2-9.

- **Custom grid world environments**

You can create custom grid worlds of any size with your own custom rewards, state transitions, and obstacle configurations. To create a custom grid world environment, you typically use the `createGridWorld` function to create a `GridWorld` object. You can then modify some of this object properties and pass it to `rlMDPEnv` to create an environment that agents can interact with for training and simulation.

For an introduction to custom grid worlds, see “Create Custom Grid World Environments” on page 2-28.

- **Custom Markov Decision Process (MDP) environments**

You can also create custom generic MDP environments by supplying your own state and action sets. To create generic MDP environments, you typically use the `createMDP` function to create a `GenericMDP` object. You can then modify some of this object properties and pass it to `rlMDPEnv` to create an environment that agents can interact with for training and simulation. For an example, see “Train Reinforcement Learning Agent in MDP Environment” on page 1-10.

Predefined Control System Environments

Control system environments are environments that represent dynamic systems in which state and observation typically belong to infinite (and uncountable) numerical vector spaces. Here, the state transition laws are deterministic and often derived by discretizing the dynamics of an underlying physical system that you want to model. Note that in these environments the action can still belong to a finite set.

Reinforcement Learning Toolbox provides several predefined control system environment objects that model dynamic systems such a double-integrator or cart-pole system. In general each predefined environment comes in two versions, one with a discrete (finite) action space and the other with a continuous (infinite and uncountable) action space.

Some of the predefined control system environments are Simulink environments, and some are multiagent environments.

You can use predefined control system environments to learn how to apply reinforcement learning to the control of physical systems, gain familiarity with Reinforcement Learning Toolbox software features, or test your own agents. For an introduction to predefined control system environments, see “Use Predefined Control System Environments” on page 2-42.

Custom Environments

You can create different types of custom environments. Once you create a custom environment, you can train and simulate agents as with any other environment.

For critical considerations on defining reward and observation signals in custom environments, see “Define Observation and Reward Signals in Custom Environments” on page 2-127.

You can create three different types of custom environment.

- **Custom function environments**

Custom function environments rely (for the calculation of the state transition, reward, observation, and initial state) on custom `step` and `reset` MATLAB functions.

For single-agent environments, once you define your action and observation specifications and write your custom `step` and `reset` functions, you use `rlFunctionEnv` to return an environment object that can interact with your agent in the same way any other environment does.

For an example on custom functions environments, see “Create Custom Environment Using Step and Reset Functions” on page 2-94.

You can also create two different kinds of custom *multiagent* function environments:

- Multiagent environments with universal sample time, in which all agents execute in the same step.
- Turn-based function environments, in which agents execute in turns. Specifically, the environment assigns execution to only one group of agents at a time, and the group executes when it is its turn to do so. For an example, see “Train Agent to Play Turn-Based Game” on page 8-348.

For both kinds of multiagent environments, the observation and action specifications are cell arrays of specification objects in which each element corresponds to one agent. For example, for an environment with three agents, the observation is then specified by a cell with three elements. Each element can be, for example, a vector of specification objects representing the observation channels for the corresponding agent.

For custom multiagent function environments with universal sample time, use `rlMultiAgentFunctionEnv` to return an environment object. For custom turn-based multiagent function environments, use `rlTurnBasedFunctionEnv`.

To specify options for training agents in multiagent environments, create and configure a `rlMultiAgentTrainingOptions` object. Doing so allows you to specify, for example, whether different groups of agents are trained in a decentralized or centralized manner. In a group of agents subject to decentralized training, each agent collects its own set of experiences and learns from its own set of experiences. In a group of agents subject to centralized training, each agent shares its experiences with the other agents in the group and each agent in the group learns from the collective shared experiences.

You can train and simulate your agents within a multiagent environment using `train` and `sim`, respectively. You can visualize the training progress of all the agents using the Reinforcement Learning Training Manager.

For more information on training multiagent environments, see “Multiagent Training” on page 5-8.

For more information on predefined multiagent environments, see “Use Predefined Multiagent Environments” on page 2-77.

- **Custom template environments**

Custom template environments are based on a modified class template.

To create a custom template environment, you use the `rlCreateEnvTemplate` function to open a MATLAB script that contains a template class for an environment, then modify the template, specifying environment properties, required environment functions, and optional environment functions.

While this process is more elaborate than just writing custom `step` and `reset` functions, it gives you more flexibility in adding properties or methods that might be needed for your application. For example, you can write a custom plot method to plot a visual representation of the environment at a given time.

For an introduction to creating environments using a template, see “Create Custom Environment from Class Template” on page 2-103.

- **Custom Simulink environments**

Custom Simulink environments are based on a Simulink model that you create.

You can also use Simulink to design multiagent environments. In particular, Simulink allows you to model environments with multi-rate execution, in which each agent might have its own execution rates.

For an introduction to creating custom Simulink environments, see “Create Custom Simulink Environments” on page 2-110. For an example that illustrates some differences between MATLAB and Simulink environments, highlighting the role of delays between action and observations, see “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114.

Neural Network Environments

Neural network environments are custom environments that rely on a neural network for the calculation of the state transition. Here, state and observation belong to continuous spaces and the state transitions laws can be deterministic or stochastic.

Neural network environments are typically used within model-based reinforcement learning agents, such as “Model-Based Policy Optimization (MBPO) Agent” on page 3-86. However, you could extract the environment from a *trained* `rlMBPOAgent` agent and use it as a (potentially less computational demanding, but less accurate) approximation of the original environment in which the model based agent was trained in.

For more information on how to create neural network environments, see `rlNeuralNetworkEnvironment`.

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim` |
`rlCreateEnvTemplate` | `validateEnvironment` | `rlSimulinkEnv` | `bus2RLSpec` |
`createIntegratedEnv`

Objects

`rlNumericSpec` | `rlFiniteSetSpec` | `rlMDPEnv` | `rlFunctionEnv` |
`rlMultiAgentFunctionEnv` | `rlTurnBasedFunctionEnv` | `SimulinkEnvWithAgent` |
`rlNeuralNetworkEnvironment`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Create Custom Environment Using Step and Reset Functions” on page 2-94
- “Create Custom Environment from Class Template” on page 2-103

More About

- “What Is Reinforcement Learning?” on page 1-3
- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Use Predefined Grid World Environments” on page 2-9
- “Use Predefined Control System Environments” on page 2-42
- “Create Custom Simulink Environments” on page 2-110

Use Predefined Grid World Environments

Reinforcement Learning Toolbox software provides several predefined grid world environments for which the actions, observations, rewards, and dynamics are already defined. You can use these environments to:

- Learn reinforcement learning concepts.
- Gain familiarity with grid world environments and Reinforcement Learning Toolbox software features.
- Test your own discrete-action-space reinforcement learning agents.

To create your own custom grid world environments instead, see “Create Custom Grid World Environments” on page 2-28.

To create an object that implements one of the following predefined MATLAB grid world environments, use the `rlPredefinedEnv` function. For example, at the MATLAB command line, type: `env = rlPredefinedEnv("BasicGridWorld")`.

Environment	Agent Task
“Basic Grid World” on page 2-9	Move from a starting location to a target location on a two-dimensional grid by selecting moves from the discrete action space {N, S, E, W}.
“Deterministic Waterfall Grid World” on page 2-14	Move from a starting location to a target location on a larger two-dimensional grid with unknown deterministic dynamics.
“Stochastic Waterfall Grid World” on page 2-20	Move from a starting location to a target location on a larger two-dimensional grid with unknown stochastic dynamics.

Reinforcement Learning Toolbox implements these grid world environments as `rlMDPEnv` objects. For more information on the properties of grid world environments, see “Create Custom Grid World Environments” on page 2-28.

You can also create objects that implement predefined MATLAB control system environments. For more information, see “Use Predefined Control System Environments” on page 2-42.

Basic Grid World

The basic grid world environment is a two-dimensional 5-by-5 grid with a starting location, a terminal location, and obstacles. The environment also contains a special jump from state [2,4] to state [4,4]. The goal of the agent is to move from the starting location to the terminal location while avoiding obstacles and maximizing the total reward.

To create a basic grid world environment, use the `rlPredefinedEnv` function. When you use the keyword “BasicGridWorld” this function returns an `rlMDPEnv` object representing the grid world.

```
env = rlPredefinedEnv("BasicGridWorld")
```

```
env =
```

```
rlMDPEnv with properties:
```

```
    Model: [1x1 rl.env.GridWorld]
  ResetFcn: []
```

Here, the `Model` property of `env` is a `GridWorld` object that encodes all the environment properties including its observation dynamics, the rewards returned to the agent for each move, and the possible presence of obstacles and terminals states. For more information on `GridWorld` objects, see “Create Custom Grid World Environments” on page 2-28.

You can use the `ResetFcn` property to specify your own reset function, instead of the default function that returns a random state. A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode. For more information on how to set the reset function of this environment, see “Reset Function” on page 2-12.

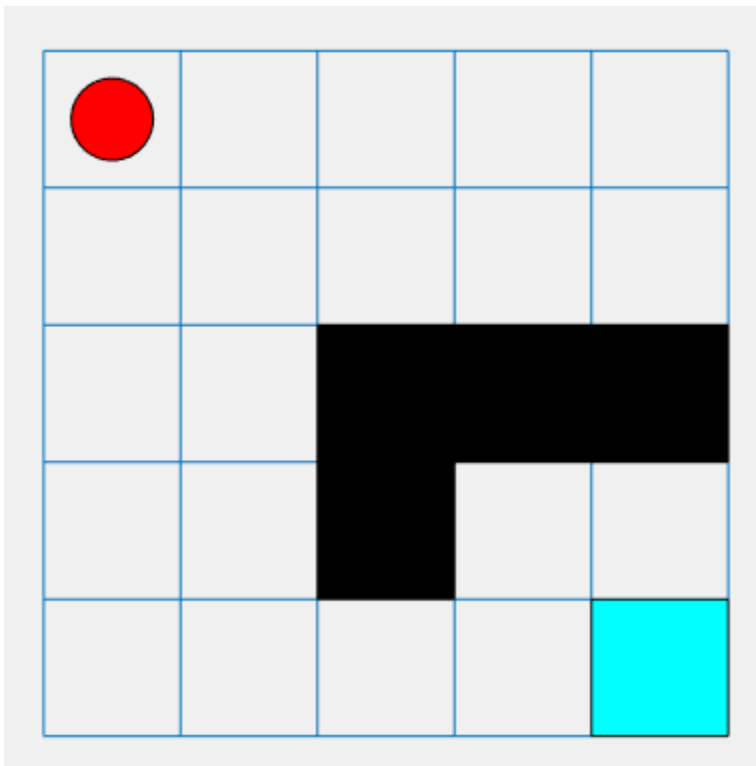
For an example that compares discrete state space agents on this environment, see “Compare Agents on Basic Grid World” on page 7-2.

Environment Visualization

You can visualize the grid world environment using the `plot` function.

- The agent location is a red circle. In this case, the agent location is [1,1], corresponding to the environment state number 1.
- The terminal location is a light blue square.
- The obstacles are black squares.

```
plot(env)
```



Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Actions

For this environment, the action channel carries a scalar integer from 1 to 4 that indicates an attempted move in one of four possible directions: north, south, east, or west, respectively. Therefore the agent has an action channel which is specified by an `rlFiniteSetSpec` object. To extract the action specification, use the `getActionInfo` function.

```
actInfo = getActionInfo(env)

actInfo =

    rlFiniteSetSpec with properties:

        Elements: [4×1 double]
        Name: "MDP Actions"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Observations

The observation is composed by a single channel carrying a scalar integer from 1 to 25. This integer indicates the current agent location (that is, the environment state) in column-wise fashion. For example, the observation 5 corresponds to the agent position [5,1] on the grid, the observation 6 corresponds to the position [1,2], and so on. Therefore the observation specification is an `rlFiniteSetSpec` object. To extract the observation specification, use the `getObservationInfo` function.

```
obsInfo = getObservationInfo(env)

obsInfo =

    rlFiniteSetSpec with properties:

        Elements: [25×1 double]
        Name: "MDP Observations"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Dynamics

If the agent attempts an illegal move, such as an action that moves it out of the grid or into an obstacle, the resulting position of the agent remains unchanged, otherwise, the environment updates the position of the agent according to the action. For example, if the agent is in the position 5, the action 3 (an attempted move eastward) moves the agent to the position 10 at the next time step. The action 4 (an attempted move westward) instead results in the agent maintaining position 5 at the next time step.

You can customize the default dynamics by modifying the transition matrix, and the number and position of the obstacle and terminal states in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Rewards

The action $A(t)$ results in the transition from the current state $S(t)$ to the following state $S(t+1)$, which in turn results in these rewards or penalties from the environment to the agent, represented by the scalar $R(t+1)$:

- +10 reward for reaching the terminal state at [5,5]
- +5 reward for jumping from state [2,4] to state [4,4]
- -1 penalty for every other action

You can customize the default rewards by modifying the reward matrix in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Reset Function

The default reset function for this environment sets the initial environment state randomly, while avoiding the obstacle and the target cells.

```
x0 = reset(env)
```

```
x0 =
```

```
11
```

You can write your own reset function to specify a different initial state. For example, to specify that the initial state of the environment (that is, the initial position of the agent on the grid) is always 2, create a reset function that always returns 2, and set the `ResetFcn` property to the handle of the function.

```
env.ResetFcn = @( ) 2;
```

A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode.

Create an Agent for This Environment

Using default agents for tabular problems is not recommended. This is because default agents rely on neural network approximation models for their actors and critics, and these models generally do not approximate tables satisfactorily, leading to poor overall performance.

In general Q, SARSA, and DQN agents that use a table-based critic perform well on tabular problems. For an example comparing discrete state space agents on this environment, see “Compare Agents on Basic Grid World” on page 7-2.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether the environment reaches a final state. For example, reset the basic grid world environment and call the step function with an action input of 3 (move east).

First, reset the environment.

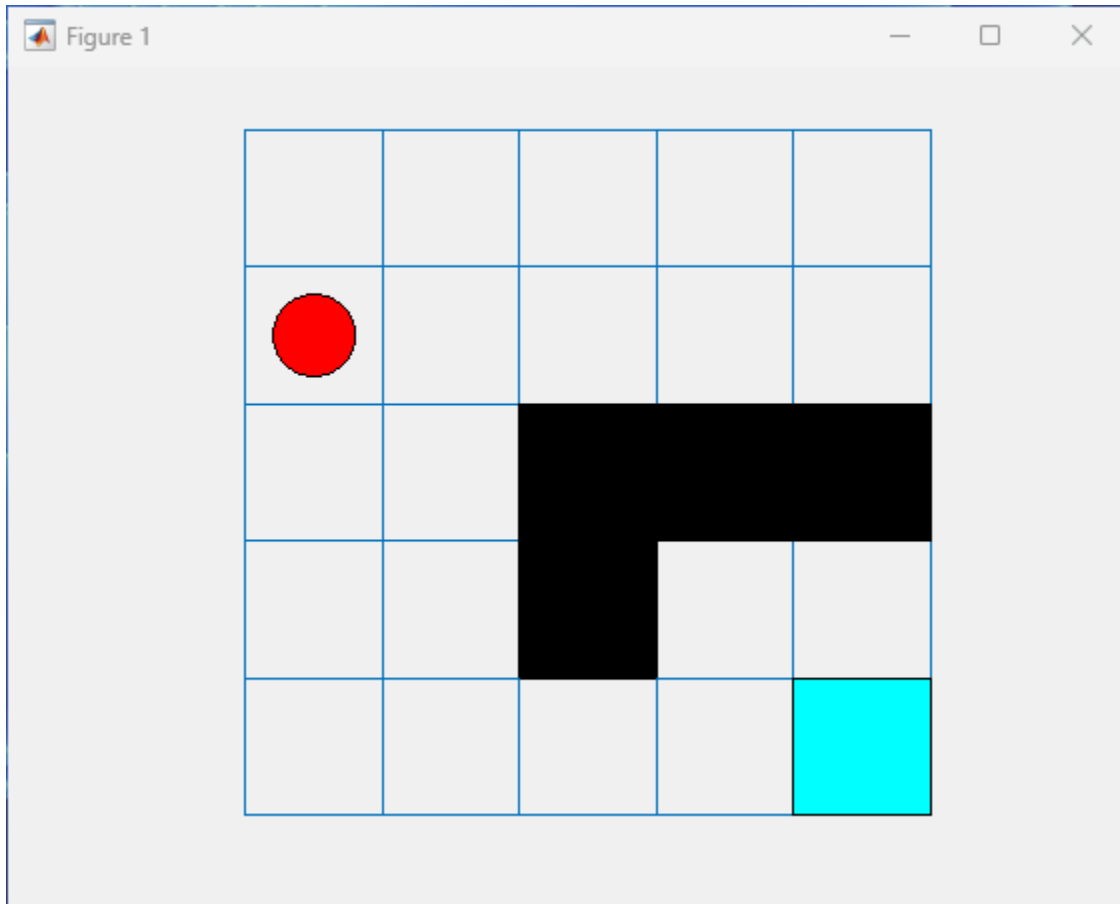
```
x0 = reset(env)
```

```
x0 =
```

```
2
```

Visualize the environment.

```
plot(env)
```



Move the agent eastward.

```
[xn,rn,id]=step(env,3)
```

```
7
```

```
rn =
```

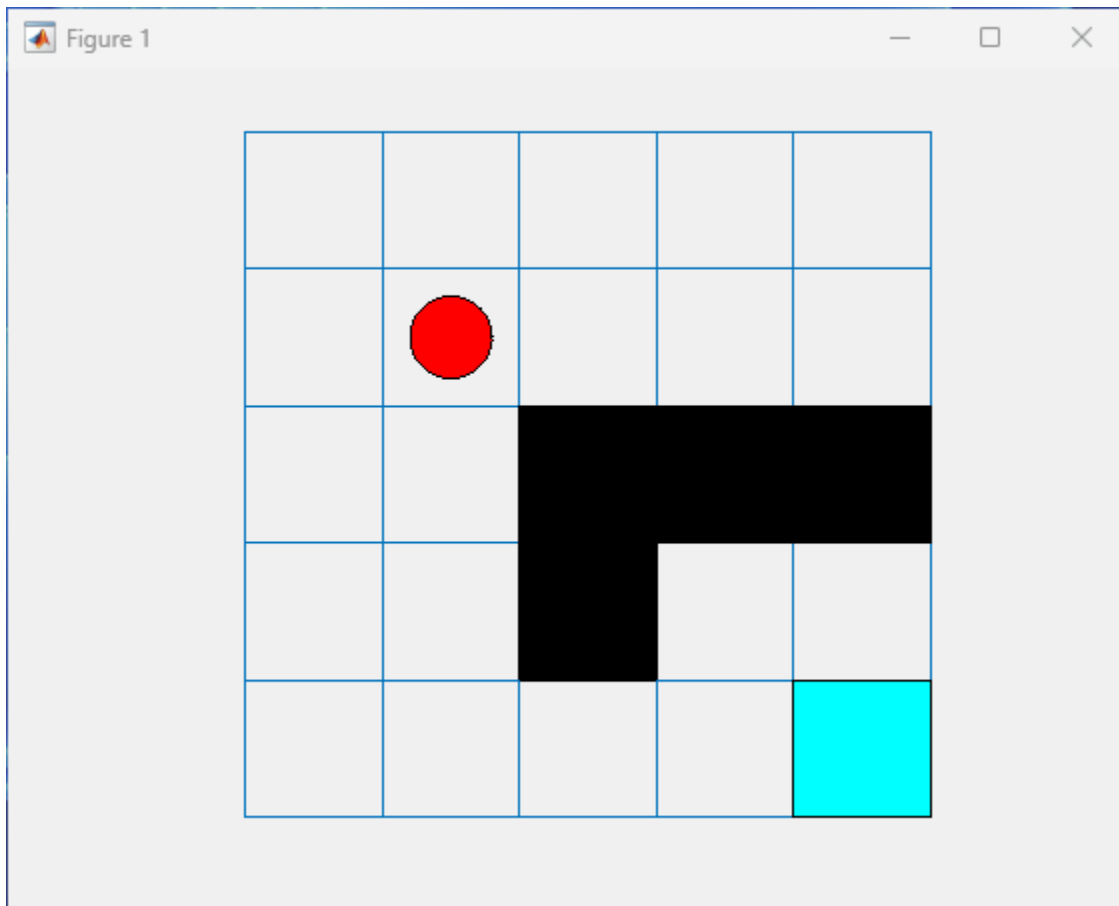
```
-1
```

```
id =
```

```
logical
```

```
0
```

After the step function executes, the environment plot, if present, visualizes the new position of the agent on the grid.



As described in the “Observations” on page 2-11 and “Rewards” on page 2-12 sections, an attempted move either outside the grid or into a position where an obstacle is present results in the agent staying in the same position, while still collecting a reward of -1.

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the function that returns this environment, at the MATLAB command line, type:

```
edit rl.env.BasicGridWorld
```

Deterministic Waterfall Grid World

The deterministic waterfall grid world environment is a two-dimensional 8-by-7 grid with a starting location and a terminal location. The environment includes a waterfall that pushes the agent toward the bottom of the grid. The goal of the agent is to move from the starting location to the terminal location while maximizing the total reward.

To create a deterministic waterfall grid world, use the `rlPredefinedEnv` function. This function creates an `rlMDPEnv` object representing the grid world.

```
env = rlPredefinedEnv("WaterFallGridWorld-Deterministic")
```

```
env =
```

`rlMDPEnv` with properties:

```
Model: [1x1 rl.env.GridWorld]  
ResetFcn: []
```

Here, the `Model` property of `env` is a `GridWorld` object that encodes all the environment properties including its observation dynamics, the rewards returned to the agent for each move, and the possible presence of obstacles and terminals states.

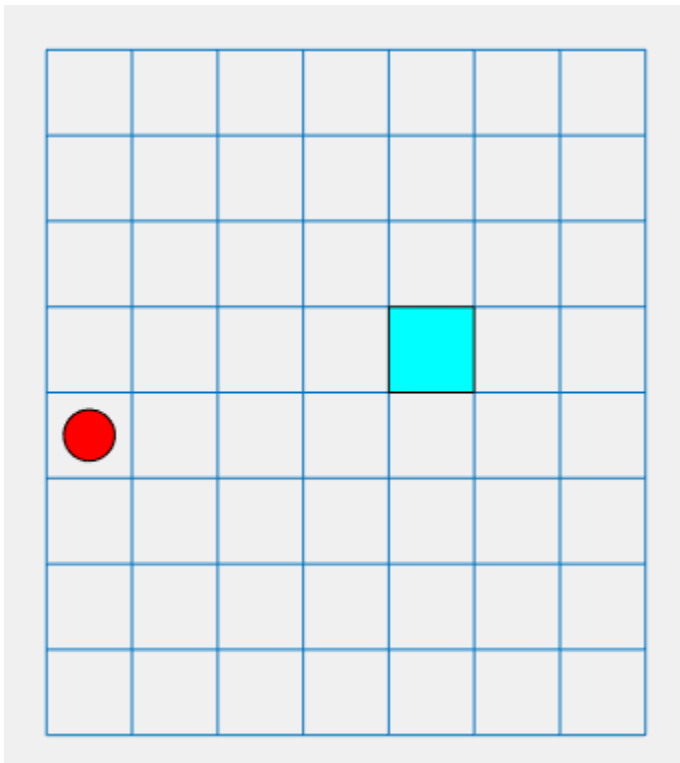
You can use the `ResetFcn` property to specify your own reset function, instead of the default function that which returns a random state. The reset function is called (by a training or simulation function) at the beginning of each training or simulation episode. For more information on how to set the reset function of this environment, see “Reset Function” on page 2-18.

For an example that compares discrete state space agents on this environment, see “Compare Agents on Deterministic Waterfall Grid World” on page 7-36.

Environment Visualization

As with any grid world environment, you can visualize the environment using the `plot` function.

```
plot(env)
```



Here the agent position is represented with a red circle and the terminal location is represented with light blue squares.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Actions

As in all other predefined grid world environments, in a deterministic waterfall environment the action channel carries a scalar integer from 1 to 4, which indicates an attempted move in one of four possible directions: north, south, east, or west, respectively. Therefore, the action specification is an `rlFiniteSetSpec` object. To extract the action specification, use the `getActionInfo` function.

```
actInfo = getActionInfo(env)

actInfo =

    rlFiniteSetSpec with properties:
        Elements: [4×1 double]
        Name: "MDP Actions"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Observations

The observation has a single channel carrying a scalar integer from 1 to 56. This integer indicates the current agent location (that is, the environment state) in columnwise fashion. For example, the observation 8 corresponds to the agent position [8,1] on the grid, the observation 9 corresponds to the position [1,2], and so on. Therefore the observation specification is an `rlFiniteSetSpec` object. To extract the observation specification, use the `getObservationInfo` function.

```
obsInfo = getObservationInfo(env)

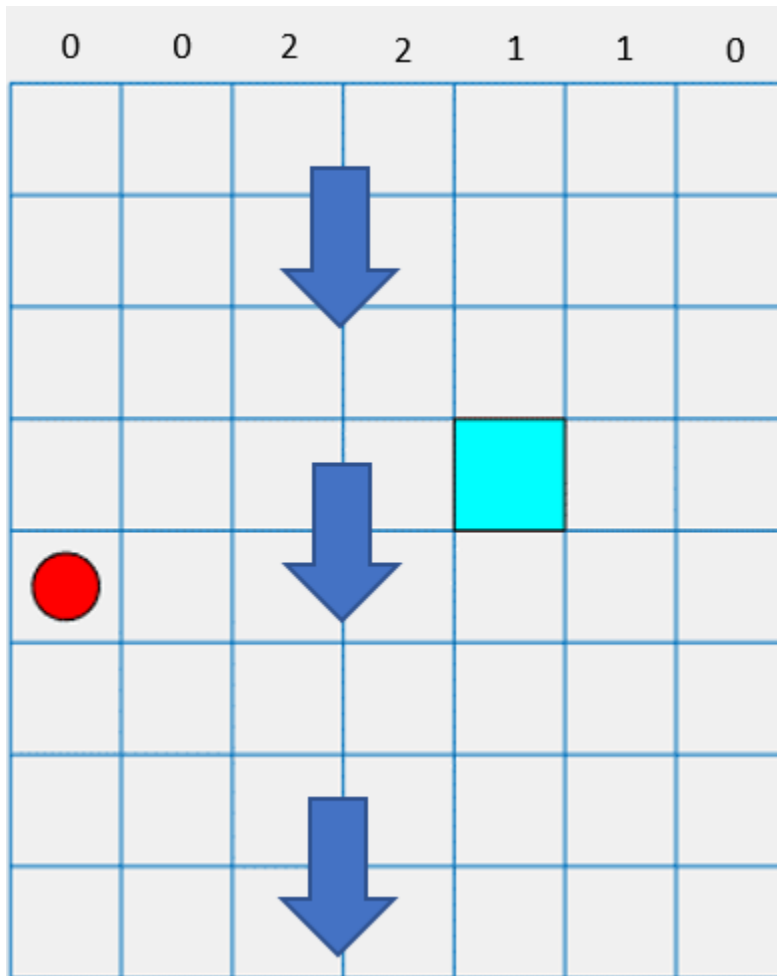
obsInfo =

    rlFiniteSetSpec with properties:
        Elements: [56×1 double]
        Name: "MDP Observations"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Dynamics

If the agent attempts an illegal move, such as an action that moves it out of the grid, the resulting position remains unchanged, otherwise, the environment updates the agent position according to the action. For example, if the agent is in the position 8, the action 3 (an attempted move eastward) moves the agent to the position 10 at the next time step. The action 4 (an attempted move westward) results in the agent maintaining the position 8 at the next time step.

In a deterministic waterfall grid world environment, a waterfall pushes the agent toward the bottom of the grid.



The intensity of the waterfall varies between the columns, as shown at the top of the figure. When the agent moves into a column with a nonzero intensity, the waterfall pushes it downward by the indicated number of squares. For example, if the agent attempts to move east from the position [5,2] (environment state number 13), it reaches position [7,3] (environment state number 23).

You can customize the default dynamics by modifying the transition matrix, and the number and position of the obstacle and terminal states in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Rewards

The action $A(t)$ results in the transition from the current state $S(t)$ to the following state $S(t+1)$, which in turn results in these rewards or penalties from the environment to the agent, represented by the scalar $R(t+1)$:

- +10 reward for reaching the terminal position at [4,5]
- -1 penalty for every other action

You can customize the default rewards by modifying the reward matrix in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Reset Function

The default reset function for this environment sets the initial environment state (that is, the initial position of the agent on the grid) randomly, while avoiding the target cell.

```
x0 = reset(env)

x0 =

    12
```

You can write your own reset function to specify a different initial state. For example, to specify that the initial state of the environment is always 5, create a reset function that always returns 5, and set the `ResetFcn` property to the handle of the function.

```
env.ResetFcn = @() 5;
```

A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode.

Create an Agent for This Environment

Using default agents for tabular problems is not recommended. This is because default agents rely on neural network approximation models for their actors and critics, and these models generally do not approximate tables satisfactorily, leading to poor overall performance.

In general Q, SARSA, and DQN agents that use a table-based critic perform well on tabular problems. For an example comparing discrete state space agents on this environment, see “Compare Agents on Deterministic Waterfall Grid World” on page 7-36.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

The environment observation and action specifications allow you to create an agent that works with your environment. You can then use both the environment and agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

You can also call the environment step function to return the next observation, reward and an `is-done` scalar indicating whether the environment reaches a final state.

For example, first define a reset function so that the initial position of the agent on the grid is always [5,2], encoded by the environment state number 13.

```
env.ResetFcn = @() 13;
```

Reset the environment.

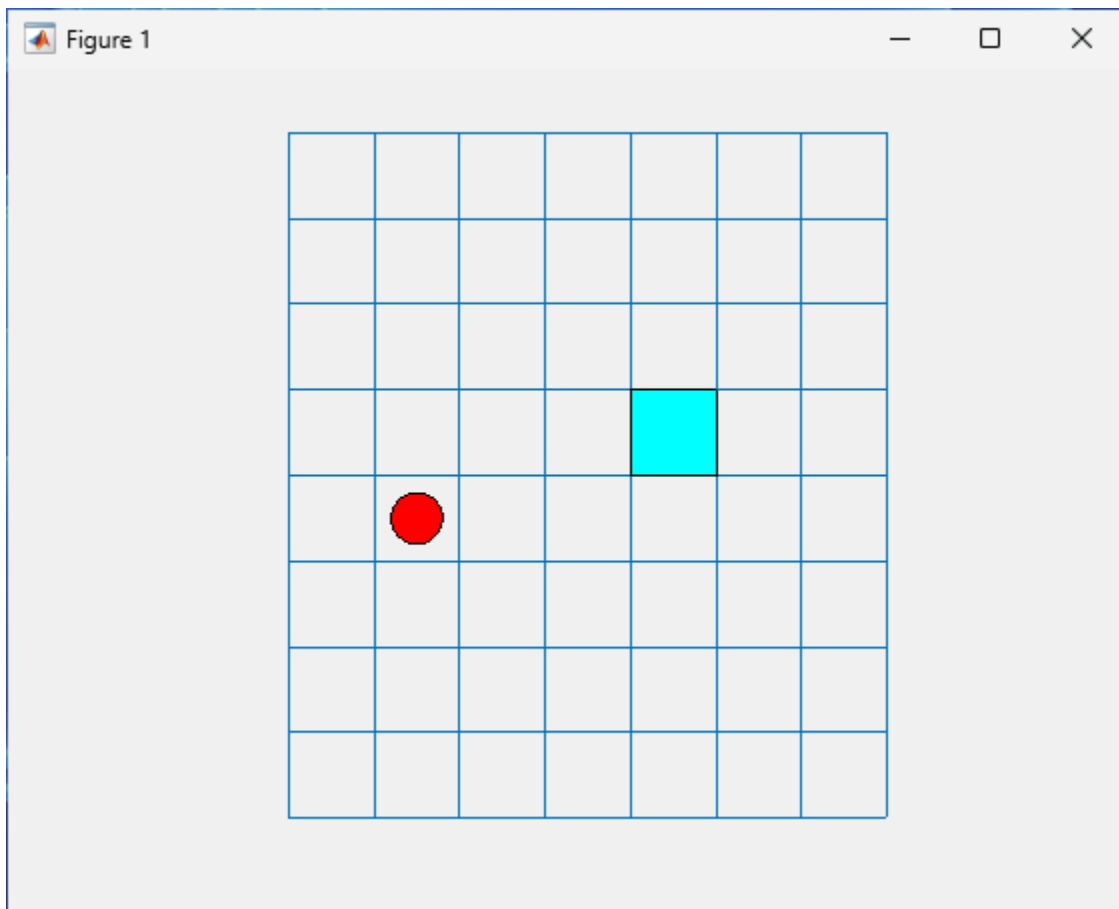
```
reset(env)

ans =

    13
```

Plot the environment.

```
plot(env)
```

Call the step function with an action input of 3 (move east).

```
[xn,rn,id]=step(env,3)
```

```
xn =
```

```
23
```

```
rn =
```

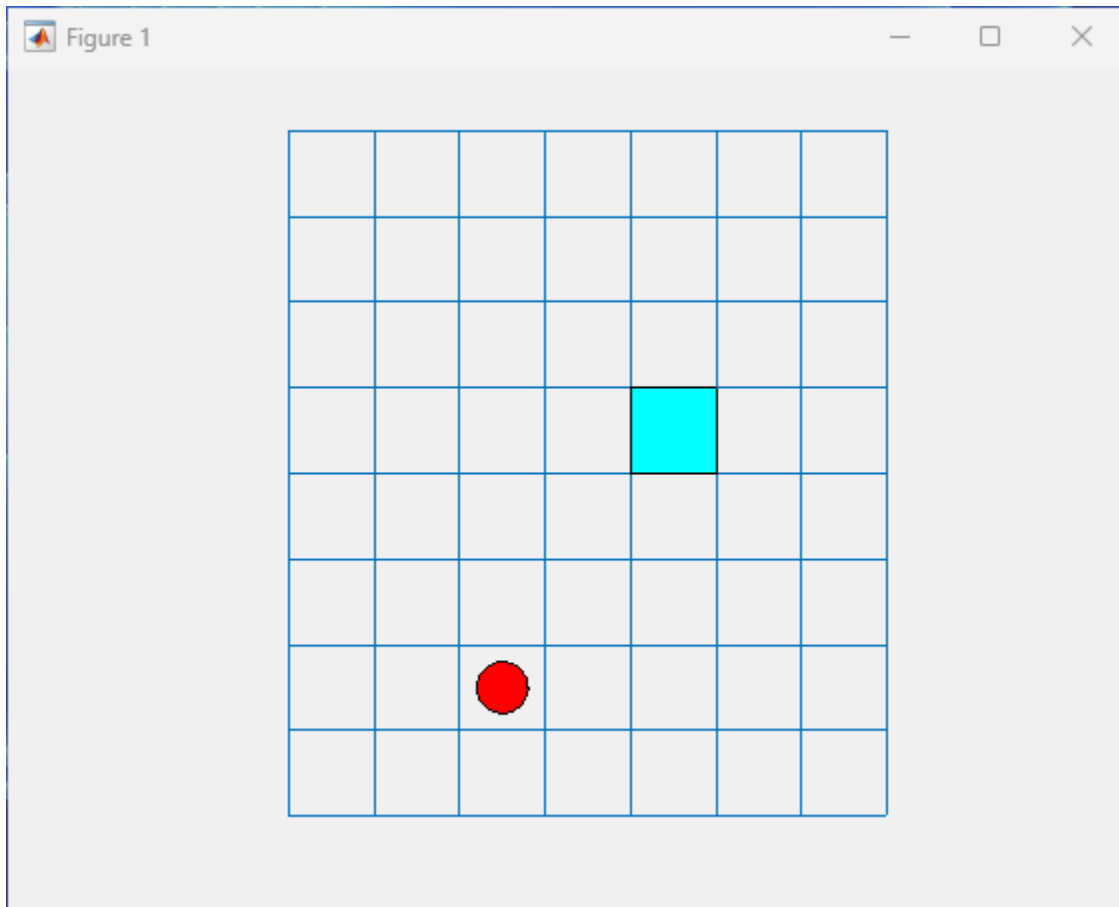
```
-1
```

```
id =
```

```
logical
```

```
0
```

The environment plot visualizes the new position of the agent on the grid.



Due to the downward waterfall, the agent moves considerably southward to the position [5,3], directly east from the position [5,2].

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the function that returns this environment, at the MATLAB command line, type:

```
edit rl.env.WaterFallDeterministicGridWorld
```

Stochastic Waterfall Grid World

The stochastic waterfall grid world environment is a two-dimensional 8-by-7 grid with a starting location and eight terminal locations. The environment includes a waterfall that pushes the agent toward the bottom of the grid with a stochastic intensity. The goal of the agent is to move from the starting location to the target terminal location while avoiding the penalty terminal states along the bottom of the grid and maximizing the total reward.

To create a stochastic waterfall grid world, use the `rlPredefinedEnv` function. This function creates an `rlMDPEnv` object representing the grid world.

```
env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
```

```
env =
```

```
rlMDPEnv with properties:
```

```
Model: [1x1 rl.env.GridWorld]  
ResetFcn: []
```

Here, the `Model` property of `env` is a `GridWorld` object that encodes all the environment properties including its observation dynamics, the rewards returned to the agent for each move, and the possible presence of obstacles and terminals states. For more information on `GridWorld` objects, see “Create Custom Grid World Environments” on page 2-28.

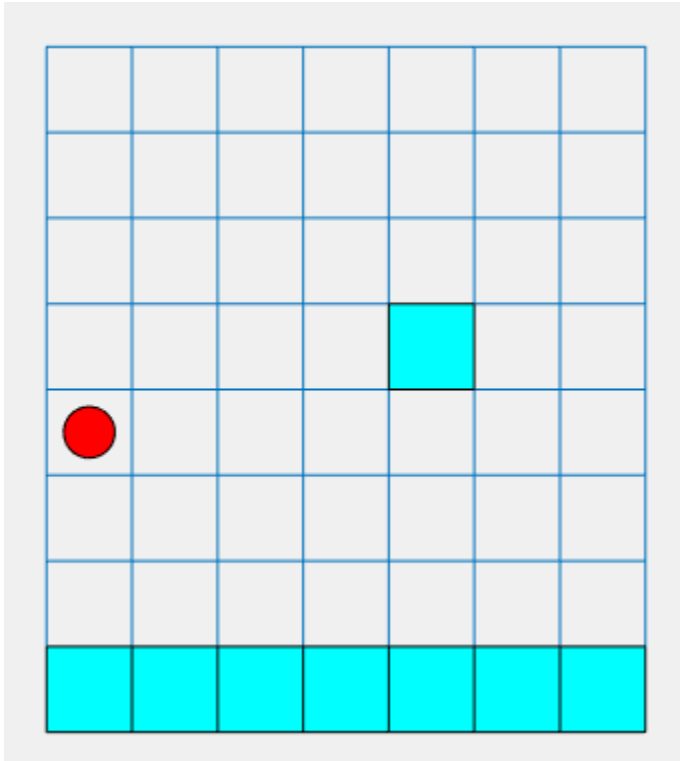
You can use the `ResetFcn` property to specify your own reset function, instead of the default one (which returns a random state). A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode. For more information on how to set the reset function of this environment, see “Reset Function” on page 2-24.

For an example comparing discrete state space agents on this environment, see “Compare Agents on Stochastic Waterfall Grid World” on page 7-70.

Environment Visualization

As with any grid world environment, you can visualize the environment using the `plot` function.

```
plot(env)
```



Here the agent position is represented with a red circle and the terminal locations are represented with light blue squares.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Actions

As in all predefined grid world environments, in a stochastic waterfall environment the action channel carries a scalar integer from 1 to 4 that indicates an attempted move in one of four possible directions: north, south, east, or west, respectively. Therefore, the action specification is an `rlFiniteSetSpec` object. To extract the action specification, use the `getActionInfo` function.

```
actInfo = getActionInfo(env)

actInfo =

    rlFiniteSetSpec with properties:

        Elements: [4×1 double]
        Name: "MDP Actions"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Observations

The observation has a single channel carrying a scalar integer from 1 to 56. This integer indicates the current agent location (that is, the environment state) in columnwise fashion. For example, the observation 8 corresponds to the agent position [8,1] on the grid, the observation 9 corresponds to the position [1,2], and so on. Therefore the observation specification is an `rlFiniteSetSpec` object. To extract the observation specification, use the `getObservationInfo` function.

```
obsInfo = getObservationInfo(env)

obsInfo =

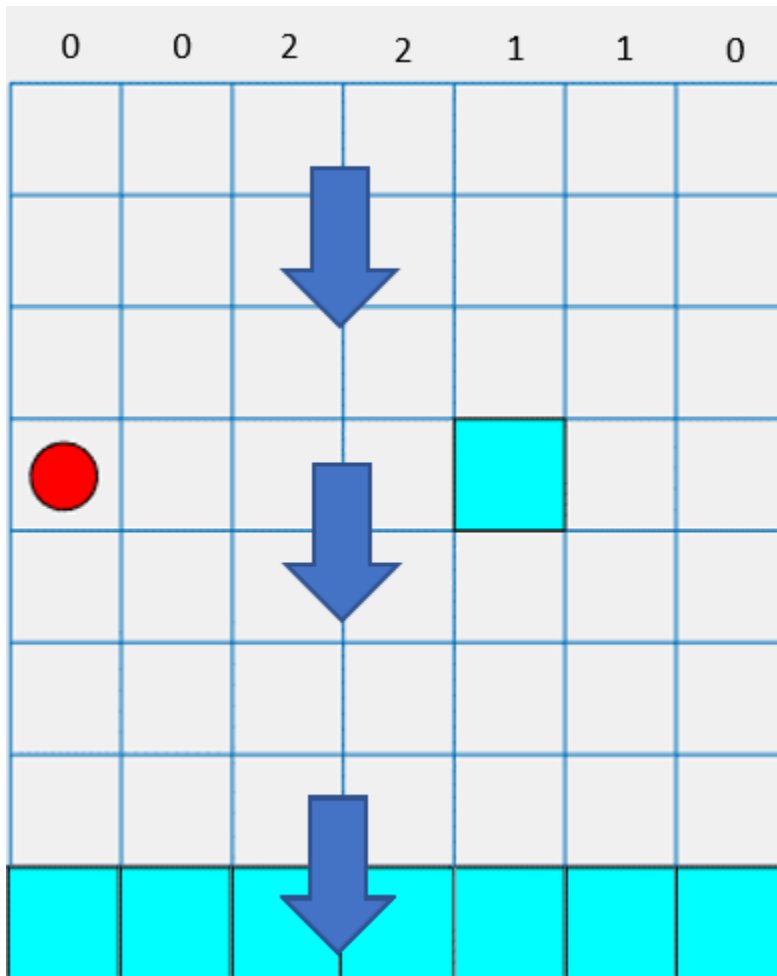
    rlFiniteSetSpec with properties:

        Elements: [56×1 double]
        Name: "MDP Observations"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Dynamics

If the agent attempts an illegal move, such as an action that moves the agent out of the grid, the resulting position remains unchanged, otherwise, the environment updates the agent position according to the action. For example, if the agent is in the position 8, the action 3 (an attempted move eastward) moves the agent to the position 10 at the next time step. The action 4 (an attempted move westward) results in the agent still keeping position 8 at the next time step.

In a stochastic waterfall grid world, a waterfall pushes the agent toward the bottom of the grid with a stochastic intensity.



The baseline intensity matches the intensity of the deterministic waterfall environment. However, in the stochastic waterfall environment, the agent has an equal chance of experiencing the indicated intensity, one level above that intensity, or one level below that intensity. For example, if the agent moves east from state [5,2], it has an equal chance of reaching state [6,3], [7,3], or [8,3].

You can customize the default dynamics by modifying the transition matrix and the number and position of the obstacle and terminal states in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Rewards

The action $A(t)$ results in the transition from the current state $S(t)$ to the following state $S(t+1)$, which in turn results in these rewards or penalties from the environment to the agent, represented by the scalar $R(t+1)$:

- +10 reward for reaching the terminal state at [4,5]
- -10 penalty for reaching any terminal state in the bottom row of the grid
- -1 penalty for every other action

You can customize the default rewards by modifying the reward matrix in the underlying `GridWorld` object. For more information, see “Create Grid World Object” on page 2-29.

Reset Function

The default reset function for this environment sets the initial environment state (that is, the initial position of the agent on the grid) randomly, while avoiding the target cells.

```
x0 = reset(env)
```

```
x0 =
```

```
12
```

You can write your own reset function to specify a different initial state. For example, to specify that the initial state of the environment is always 5, create a reset function that always returns 5, and set the `ResetFcn` property to the handle of the function.

```
env.ResetFcn = @() 5;
```

A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode.

Create an Agent for This Environment

Using default agents for tabular problems is not recommended. This is because default agents rely on neural network approximation models for their actors and critics, and these models generally do not approximate tables satisfactorily, leading to poor overall performance.

In general Q, SARSA, and DQN agents that use a table-based critic perform well on tabular problems. For an example comparing discrete state space agents on this environment, see “Compare Agents on Stochastic Waterfall Grid World” on page 7-70.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether the environment reaches a final state.

For example, first define a reset function so that the initial position of the agent on the grid is always [5,2], encoded by the environment state number 13.

```
env.ResetFcn = @() 13;
```

Reset the environment.

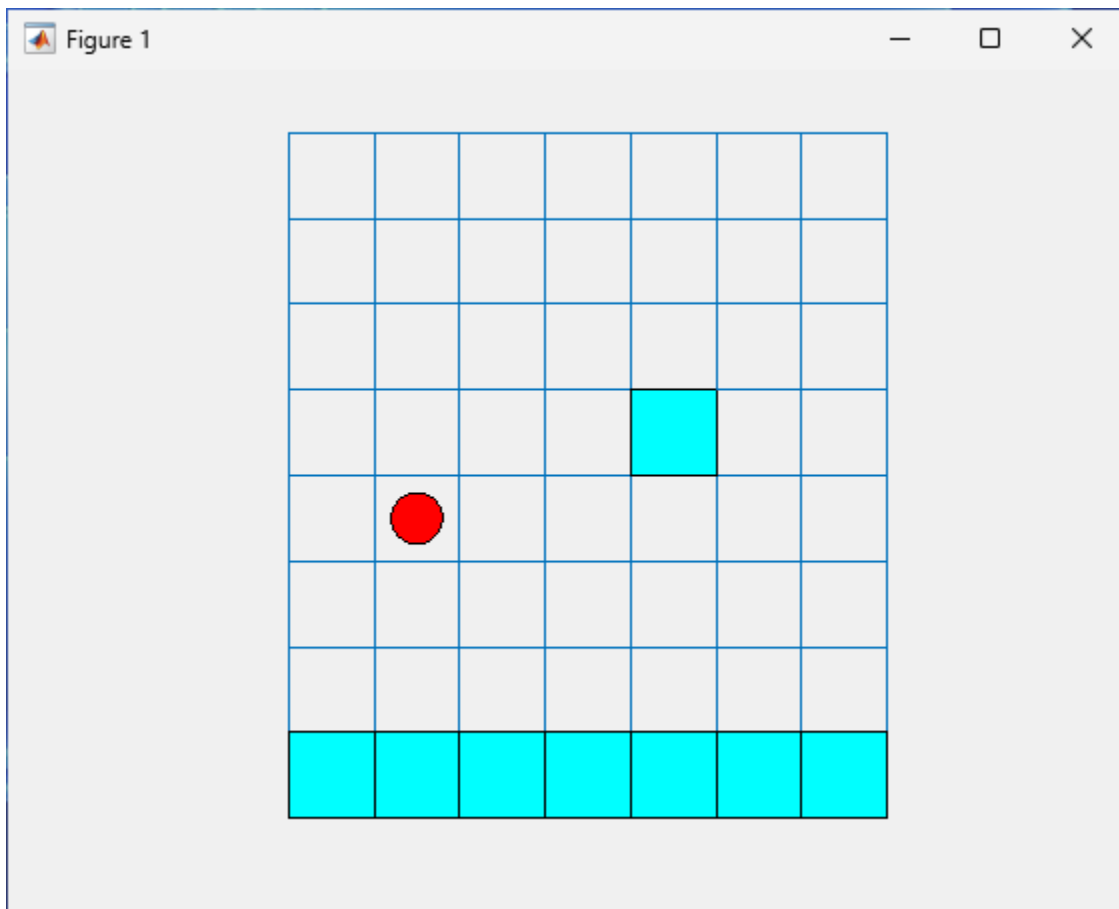
```
reset(env)
```

```
ans =
```

```
13
```

Plot the environment.

```
plot(env)
```



Call the step function with an action input of 3 (move east).

```
[xn,rn,id]=step(env,3)
```

```
xn =
```

```
23
```

```
rn =
```

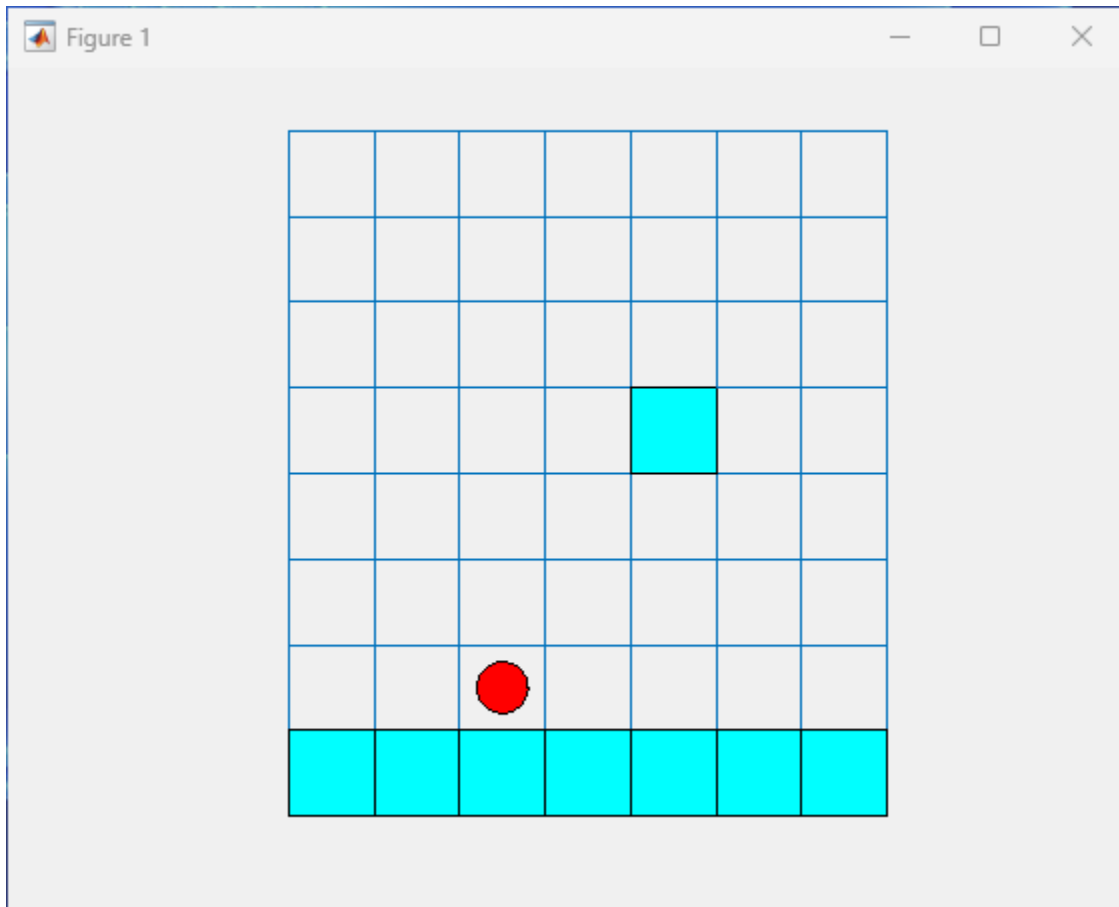
```
-1
```

```
id =
```

```
logical
```

```
0
```

The environment plot visualizes the new position of the agent on the grid.



Due to the downward waterfall, the agent moves considerably southward to the position [5,3], directly east from the position [5,2].

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the function that returns this environment, at the MATLAB command line, type:

```
edit rl.env.WaterFallStochasticGridWorld
```

See Also

Functions

`rlPredefinedEnv` | `getObservationInfo` | `getActionInfo` | `train` | `sim`

Objects

`rlMDPEnv` | `rlNumericSpec` | `rlFiniteSetSpec`

Related Examples

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

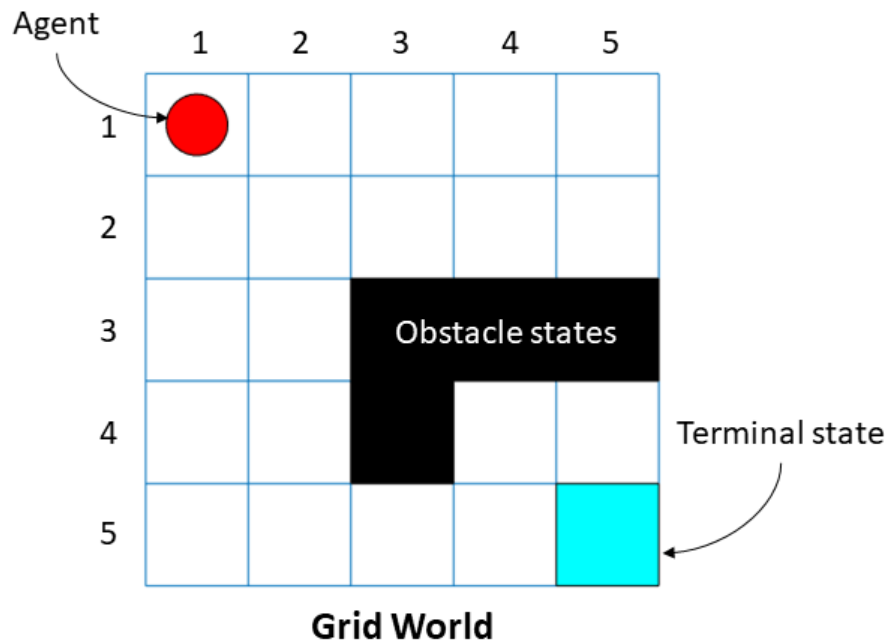
More About

- “Reinforcement Learning Environments” on page 2-2

Create Custom Grid World Environments

A custom grid world environment is a MATLAB environment featuring a generic two-dimensional grid with actions, observations, rewards, dynamics, and optional obstacles and terminal states that are mostly left for you to define. As in any grid world environment, the goal of the agent is to move in a way to maximize its expected discounted cumulative long-term reward.

Grid world environments are a special case of Markov Decision Process (MDP) environments. An MDP is a discrete time stochastic control process. It provides a mathematical framework for modeling decision making in situations where outcomes are partly random and partly under the control of the decision maker. In a grid world environment, the state represents a position in a two-dimensional grid, while the action represents a move from the current position to the next, which an agent might attempt. To create a custom MDP environment, see “Markov Decision Process (MDP) Environments” on page 2-4, `createMDP`, and `rlMDPEnv`.



You can use a custom grid world environment to analyze the behavior of different discrete-time agents on custom grid worlds, and to explore reinforcement learning concepts. For example, many common benchmark reinforcement learning problems are grid world problems, and you can study them with Reinforcement Learning Toolbox by creating a custom grid world environments.

To create a custom grid world environment:

- 1 Create the grid world object.
- 2 Configure the grid world object.
- 3 Use the grid world object to create your environment.

To load a grid world environment with predefined actions, observations, rewards, and dynamics, see “Use Predefined Grid World Environments” on page 2-9.

Create Grid World Object

You can create your own grid world model using the `createGridWorld` function. Specify the grid size when creating the `GridWorld` object.

For example, at the MATLAB command line, type:

```
GW = createGridWorld(6,6,"Standard")
```

```
GW =
```

```
GridWorld with properties:
```

```
    GridSize: [6 6]
   CurrentState: "[1,1]"
        States: [36×1 string]
        Actions: [4×1 string]
             T: [36×36×4 double]
             R: [36×36×4 double]
  ObstacleStates: [0×1 string]
  TerminalStates: [0×1 string]
ProbabilityTolerance: 8.8818e-16
```

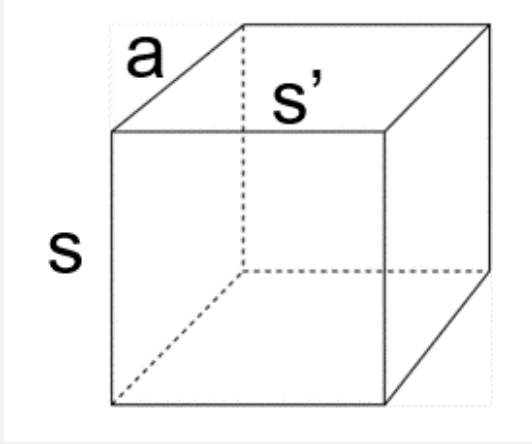
Note The grid world model `GW` is a `GridWorld` object, not an environment object. You must later create an `rLMDPEnv` environment object from `GW`.

The `GridWorld` object has these properties.

Property	Read-Only	Description
<code>GridSize</code>	Yes	Dimensions of the grid world, displayed as a row vector containing two positive integers. The first integer <code>m</code> is the number of rows in the grid, and the second integer <code>n</code> is the number of columns in the grid.

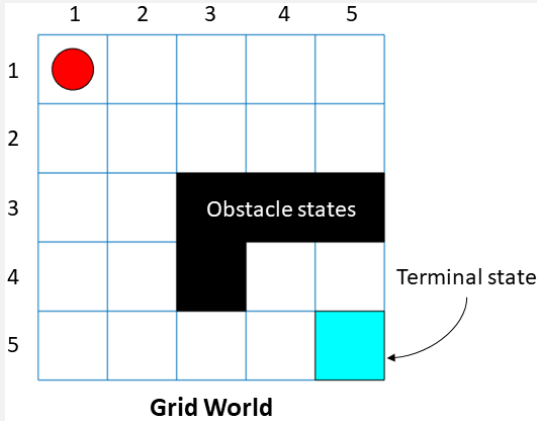
Property	Read-Only	Description
CurrentState	No	Name of the current state of the environment. This name corresponds to the current agent position in the grid, and it is specified as a string or character vector such as "[a,b]". Here, a and b are two positive integers less than m and n, respectively, that indicate the row (a) and the column (b) corresponding to the agent position on the grid. Specifying this property in any other format results in an error when the environment step function is executed on an environment built using GW. By default, this property is set to the string "[1,1]". You can use this property to set the initial state of the environment. For example, the command <code>GW.CurrentState = "[2,4]";</code> sets the current position of the agent in the cell located in the second row and the fourth column of the grid. On an 8-by-7 grid this position is encoded as environment state number 26, using the formula $8*(4-1) + 2 = 26$. If you call the step function on an environment built using GW, the environment executes the function from the state indicated by CurrentState. Note that every time the environment reset function is called, the environment state is reset according to the specific code in the reset function. For more information on step and reset functions, see "Create Custom Environment Using Step and Reset Functions" on page 2-94.
States	Yes	A string vector containing the state names of the grid world, as specified in the CurrentState property. For example, for a 2-by-2 grid world model GW, you can specify the state names as follows. <pre>GW.States = ["[1,1]"; "[2,1]"; "[1,2]"; "[2,2]"];</pre> You can use the <code>state2idx</code> function to obtain the state index associated with a state name.

Property	Read-Only	Description						
Actions	Yes	A string vector containing the list of possible actions that the agent can execute in the grid world environment. You can set the actions when you create the grid world model by using the <code>moves</code> argument. For example, at the MATLAB command line, type: <pre>GW = createGridWorld(m,n,moves)</pre> Here, <code>m</code> and <code>n</code> are integers as specified in <code>GridSize</code>, and <code>moves</code> is a string that can be either "Standard" or "Kings". <table><tr><th>moves</th><th>GW.Actions</th></tr><tr><td>"Standard"</td><td>["N"; "S"; "E"; "W"], indicates that the agent can attempt to move north, south, east, and west from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 4. For example, <code>step(env,3)</code> indicates that the agent attempts to move east from its current position.</td></tr><tr><td>"Kings"</td><td>["N"; "S"; "E"; "W"; "NE"; "NW"; "SE"; "SW"] indicating that the agent can attempt to move north, south, east, west, northeast, northwest, southeast and southwest, respectively, from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 8, so that, for example, <code>step(env,8)</code> indicates that the agent attempts to move southwest from its current position.</td></tr></table> You can use the <code>action2idx</code> function to obtain the action index associated with a state name.	moves	GW.Actions	"Standard"	["N"; "S"; "E"; "W"], indicates that the agent can attempt to move north, south, east, and west from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 4. For example, <code>step(env,3)</code> indicates that the agent attempts to move east from its current position.	"Kings"	["N"; "S"; "E"; "W"; "NE"; "NW"; "SE"; "SW"] indicating that the agent can attempt to move north, south, east, west, northeast, northwest, southeast and southwest, respectively, from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 8, so that, for example, <code>step(env,8)</code> indicates that the agent attempts to move southwest from its current position.
moves	GW.Actions							
"Standard"	["N"; "S"; "E"; "W"], indicates that the agent can attempt to move north, south, east, and west from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 4. For example, <code>step(env,3)</code> indicates that the agent attempts to move east from its current position.							
"Kings"	["N"; "S"; "E"; "W"; "NE"; "NW"; "SE"; "SW"] indicating that the agent can attempt to move north, south, east, west, northeast, northwest, southeast and southwest, respectively, from its current grid position. The step function of the environment built using <code>GW</code> encodes these moves using integers from 1 to 8, so that, for example, <code>step(env,8)</code> indicates that the agent attempts to move southwest from its current position.							

Property	Read-Only	Description
T	No	State transition matrix, specified as a 3-D array in which every row of each page contains nonnegative numbers that must add up to one. The state transition matrix T is a probability matrix in which each entry indicates the likelihood of the agent moving from the current state s to any possible next state s' by performing action a.  T can be denoted as $T(s, s', a) = \text{probability}(s' s, a)$ T is: - A K-by-K-by-4 array, if moves is specified as "Standard". Here, $K = m*n$. - A K-by-K-by-8 array, if moves is specified as "Kings". When you create a grid world object, the default transition matrix contains standard deterministic transitions corresponding to the four or eight actions that the agent can execute. Specifically, the default transition matrix is such that any attempted move in any direction results in the agent moving one cell in that direction with probability of one, except for any attempted move outside the grid, which results in the agent keeping its current position. For example, consider a 5-by-5 deterministic grid world object GW with the agent in cell $[3, 1]$. View the state transition matrix for the north direction. <pre>northStateTransition = GW.T(:, :, 1)</pre> Here, the number 1 in the last dimension encodes the attempted move north, as specified in the <code>Actions</code> property.

Property	Read-Only	Description
		<pre>northStateTransition = 25x25 "[1,1]" "[2,1]" "[3,1]" "[1,1]" 1 0 0 "[2,1]" 1 0 0 "[3,1]" 0 1 0 "[4,1]" 0 0 1 "[5,1]" 0 0 0 "[1,2]" 0 0 0</pre> In this figure, the value of <code>northStateTransition(3,2)</code> is 1. This value indicates that when the agent is in the position <code>[3,1]</code>, following the action 'N', the agent moves to the position <code>[2,1]</code> with a probability of 1 (and with a probability of 0 to the other cells specified in the same row). Note Because each number in a row represents the probability of moving from the cell indexed by the column into the cell indexed by the row, all the numbers along a row must always add to either one or zero (within the tolerance specified in <code>ProbabilityTolerance</code>). To set transition probabilities, first, set an entire row to zero, then set the non-zero probabilities all at once. For an example, see <code>createGridWorld</code> or <code>createMDP</code>. Alternatively, copy the transition matrix into a variable, modify the variable, and then assign it back as transition matrix of your grid world object. To index specific states and actions in the transition matrix, you can use the <code>state2idx</code> and <code>action2idx</code> functions. For example: <pre>GW.T(state2idx(GW,"[3,4]"),state2idx(GW,"[4,4]"),action2idx("S"))</pre> For an example on how to set the transition matrix, see <code>createGridWorld</code>.

Property	Read-Only	Description
R	No	Reward transition matrix, specified as a 3-D array. R determines how much reward the agent receives after performing an action in the environment. Each entry of the reward transition matrix specifies the reward that the agent obtains when moving from the current state s to any possible next state s' by performing action a: $r = R(s, s', a).$ R has the same shape and size as the state transition matrix T. Specifically, R is: - A K-by-K-by-4 array, if moves is specified as "Standard". Here, $K = m*n$. - A K-by-K-by-8 array, if moves is specified as "Kings". When you create a grid world object, the reward matrix is zero. Set up R so that there is a reward to the agent after every action. For example, you can set up a positive reward if the agent transitions over obstacle states and when it reaches the terminal state. You can also set up a reward of -1 for any action the agent takes, independent of the current state and next state. To index specific states and actions in the reward matrix, you can use the <code>state2idx</code> and <code>action2idx</code> functions. For example: <pre>GW.R(state2idx(GW,"[3,4]"),state2idx(GW,"[4,4]"),action2idx("S"))</pre> For an example on how to set the reward matrix, see <code>createGridWorld</code>.

Property	Read-Only	Description
ObstacleStates	No	ObstacleStates are states that cannot be reached in the grid world, specified as a string vector. Consider this 5-by-5 grid world object GW.  This syntax specifies the obstacle states, represented by black squares in the figure. <pre>GW.ObstacleStates = ["[3,3]"; "[3,4]"; "[3,5]"; "[4,3]"];</pre> When you set obstacle states, the transition matrix T automatically updates so that if the agent attempts to move into an obstacle, its resulting position is the same as its current position, with a probability of one. For an example on how to set obstacles, see <code>createGridWorld</code>.
TerminalStates	No	TerminalStates are the final states in the grid world, specified as a string vector. Consider the picture of the previous 5-by-5 grid world model GW. The blue cell is the terminal state and you can specify it with this command. <pre>GW.TerminalStates = "[5,5]";</pre> When you set terminal states, the transition and reward matrices of GW automatically update so that both the probability of moving out of the terminal state and the reward for staying in the terminal state is zero. Additionally, for any environment <code>env</code> created from GW, the <code>step</code> function returns an <code>is-done</code> value of <code>true</code> as its third output argument when the agent moves in a terminal state. As a result, when you use the <code>train</code> or <code>sim</code> functions, the training or simulation episode stops when the agent reaches the terminal state. For an example on how to set terminal states, see <code>createGridWorld</code>.

Property	Read-Only	Description
Probability Tolerance	No	This property is the tolerance for the sum of probabilities along a row of the transition matrix. Because the sum of numbers along a row of the transition matrix represents the probability of moving into the state indexed by the row number, all the numbers along a row must add to either one or zero, within the tolerance specified in <code>ProbabilityTolerance</code>. If this condition is not verified, an error is thrown. To set transition probabilities, first, set an entire row to zero, then set the non-zero probabilities all at once. For an example, see <code>createGridWorld</code> or <code>createMDP</code>. Alternatively, copy the transition matrix into a variable, modify the variable, and then assign it back as transition matrix of your grid world object.

Configure Grid World Object

After creating your `GridWorld` object, you need to configure its transition matrix, to make sure it represents your desired dynamics. You also need to configure its reward matrix to make sure the agent gets the appropriate rewards for its moves.

Because each row of each page of the transition matrix must always sum to one, you cannot modify the transition matrix entries in place one at a time. Instead, assign the default matrix to a temporary variable, in the workspace, modify the variable entries appropriately, and then reassign the modified variable to the transition matrix of your `GridWorld` object.

For example, create a `GridWorld` object with five rows and five columns.

```
gw = createGridWorld(5,5);
```

Extract the default transition matrix, which already contains the standard transition dynamics.

```
T = gw.T;
```

Modify the temporary matrix so that from state 6 any action leads to state 10.

```
% Zero out the existing transitions from state 6.
T(6, :, :) = 0;
% For any action, set the probability of reaching state 10 to 1.
T(6, 10, 1) = 1; T(6, 10, 2) = 1; T(6, 10, 3) = 1; T(6, 10, 4) = 1;
```

Update the transition matrix of your `GridWorld` object.

```
gw.T = T;
```

Create Grid World Environment from Grid World Object

After configuring your `GridWorld` object, use it to create an MDP environment using `rLMDPEnv`. This step is necessary because the `GridWorld` object is not an environment object.

For example, if you have the `GridWorld` object `gw` in the MATLAB workspace, at the command line, type:

```
env = rlMDPEnv(gw)
```

```
env =
```

```
rlMDPEnv with properties:
```

```
    Model: [1x1 rl.env.GridWorld]
  ResetFcn: []
```

This command creates the environment `env` that contains your `GridWorld` object.

```
env.Model
```

```
ans =
```

```
GridWorld with properties:
```

```
    GridSize: [5 5]
  CurrentState: "[1,1]"
        States: [25x1 string]
        Actions: [4x1 string]
           T: [25x25x4 double]
           R: [25x25x4 double]
  ObstacleStates: [0x1 string]
  TerminalStates: [0x1 string]
  ProbabilityTolerance: 8.8818e-16
```

If necessary, you can set your own reset function. For example, to make sure the agent always starts from state number 2, set the `ResetFcn` environment property to the handle of an anonymous function that always returns 2.

```
env.ResetFcn = @() 2
```

```
env =
```

```
rlMDPEnv with properties:
```

```
    Model: [1x1 rl.env.GridWorld]
  ResetFcn: @()2
```

For more information on the reset function, see “Reset Function” on page 2-39.

Environment Visualization

As with other grid world environments, you can visualize the environment using the `plot` function. A red circle represents the current agent position, that is, the environment state. If present, the terminal locations and obstacles are represented by blue and black squares, respectively.

```
plot(env)
```

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Actions

Depending on the `Actions` property of the underlying `GridWorld` model, the action channel carries a scalar integer ranging from either 1 to 4 or 1 to 8.

- When `Actions` is set to "Standard", the integer indicates an (attempted) move in the directions north, south, east, or west, respectively.
- When `Actions` is set to "Kings", the integer indicates an (attempted) move in the directions north, south, east, west, northeast, northwest, southeast and southwest, respectively.

For more information, see "Create Grid World Object" on page 2-29.

In either case, the action specification is an `rlFiniteSetSpec` object. To extract the action specification, use the `getActionInfo` function.

```
actInfo = getActionInfo(env)

actInfo =

    rlFiniteSetSpec with properties:
        Elements: [4×1 double]
        Name: "MDP Actions"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Observations

As in all grid world environments, the environment observation has a single channel carrying a scalar integer from 1 to the number of environment states. The observation indicates the current agent location (that is, the environment state) in column-wise fashion. So, the observation specification is an `rlFiniteSetSpec` object. To extract the observation specification, use the `getObservationInfo` function.

```
obsInfo = getObservationInfo(env)

obsInfo =

    rlFiniteSetSpec with properties:
        Elements: [56×1 double]
        Name: "MDP Observations"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

Grid World Dynamics

As for all grid world environments, the transition matrix property `T` of the underlying `GridWorld` object determines the dynamics.

The default transition matrix is such that any attempted move in any direction results in the agent moving one cell in that direction with a probability of one, except for any attempted move outside the grid, which results in the agent maintaining its current position.

For more information, see "Create Grid World Object" on page 2-29 and "Configure Grid World Object" on page 2-36.

Rewards

As for all grid world environments, the reward matrix property `R` of the underlying `GridWorld` object determines the reward.

The default reward matrix contains only zeroes.

For more information, see “Create Grid World Object” on page 2-29 and “Configure Grid World Object” on page 2-36.

Reset Function

The state of a custom grid world environment is initially set to 1, which is equivalent to the string “[1,1]”, representing the most northwestern position of the grid. The default reset function for a custom grid world environment then sets the initial environment state (that is, the initial position of the agent on the grid), randomly.

```
x0 = reset(env)
```

```
x0 =
```

```
12
```

You can write your own reset function to specify a different initial state. For example, to specify that the initial state of the environment is always 5, create a reset function that always returns 3, and set the `ResetFcn` property to the handle of the function.

```
env.ResetFcn = @() 3;
```

A training or simulation function automatically calls the reset function at the beginning of each training or simulation episode.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent (with discrete action space) that works with your environment. For example, create a default AC agent.

```
acAgent = rlACAgent(obsInfo,actInfo)
```

```
acAgent =
```

```
rlACAgent with properties:
```

```
    AgentOptions: [1x1 rl.option.rlACAgentOptions]
  UseExplorationPolicy: 1
    ObservationInfo: [1x1 rl.util.rlFiniteSetSpec]
      ActionInfo: [1x1 rl.util.rlFiniteSetSpec]
      SampleTime: 1
```

If needed, modify the agent options using dot notation.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

As in other MATLAB environments, you can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether the environment reaches a final state.

For example, call the step function with an action input of 2 to move the agent south.

```
[xn,rn,id]=step(env,3)
```

```
xn =
```

```
4
```

```
rn =
```

```
0
```

```
id =
```

```
logical
```

```
0
```

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

References

- [1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Functions

`createMDP` | `createGridWorld` | `rlPredefinedEnv` | `getObservationInfo` | `getActionInfo` | `train` | `sim`

Objects

`rlMDPEnv` | `rlNumericSpec` | `rlFiniteSetSpec` | `rlFunctionEnv` | `rlMultiAgentFunctionEnv` | `rlTurnBasedFunctionEnv` | `SimulinkEnvWithAgent`

Related Examples

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

More About

- “Reinforcement Learning Environments” on page 2-2

Use Predefined Control System Environments

Reinforcement Learning Toolbox software provides several predefined environments representing dynamic systems that are often used as benchmark cases for control systems design.

You can use predefined control system environments to learn how to apply reinforcement learning to the control of physical systems, gain familiarity with Reinforcement Learning Toolbox software features, or test your own agents. You can also use the predefined control system environments as a starting point for developing your own control system environment.

To create your own custom environments instead, see “Custom Environments” on page 2-5.

In the predefined control system environments, the state and observation are predefined and belong to nonfinite numerical vector spaces, while the action, also predefined, can still belong to a finite set. The (deterministic) state transitions laws are derived by discretizing the dynamics of an underlying physical system.

Simulink environments are environments that rely on an underlying model for the calculation of the state transition, reward, and observation. These environments do not support using `reset` and `step` functions. Some of the predefined control system environments are Simulink environments.

By contrast, environments that instead rely on MATLAB functions or objects for the calculation of the state transition, reward, and observation, are referred to as MATLAB environments.

Multiagent environments are environments in which you can train and simulate multiple agents together. Some of the predefined MATLAB and Simulink control system environments are multiagent environments. For more information on predefined multiagent environments, see “Use Predefined Multiagent Environments” on page 2-77.

To create an object that implements a predefined MATLAB control system environment listed in the following table, use the `rlPredefinedEnv` function. Each of these predefined environment is available in two versions, one with a discrete action space, and the other with a continuous action space.

Environment	Agent Task
“MATLAB Double-Integrator Environment” on page 2-43	Control a second-order dynamic system using either a discrete or continuous action space.
“MATLAB Cart-Pole Environment” on page 2-49	Balance a pole on a moving cart by applying forces to the cart using either a discrete or continuous action space.
“MATLAB Simple Pendulum Swing-Up with Image Observation Environment” on page 2-55	Swing up and balance a simple pendulum using the pendulum image as an observation, with either a discrete or continuous action space.

To create an object that implements a predefined Simulink environment listed in the following table, use the `rlPredefinedEnv` function. For these environments, `rlPredefinedEnv` creates a `SimulinkEnvWithAgent` object. Each of these predefined environment is also available in two versions, one with a discrete action space, and the other with a continuous action space.

Environment	Agent Task
"Simulink Simple Pendulum Swing-up Environment" on page 2-62	Swing up and balance a simple pendulum by applying a torque to the pendulum, using either a discrete or continuous action space.
"Simscape Cart-Pole Swing-up Environment" on page 2-69	Balance a pole on a moving cart by applying forces to the cart using either a discrete or continuous action space. This environment relies on a Simulink model that uses Simscape™ blocks to model the system.

You can also create objects that implement predefined grid world environments. For more information, see "Use Predefined Grid World Environments" on page 2-9.

To learn how to create your own custom environment, see "Create Custom Environment Using Step and Reset Functions" on page 2-94, "Create Custom Simulink Environments" on page 2-110, and "Create Custom Environment from Class Template" on page 2-103.

MATLAB Double-Integrator Environment

The double-integrator environment is a predefined MATLAB environment in which the goal of the agent is to control the position (right-positive) of a mass in a frictionless mono-dimensional space by applying a force input. The system has a second-order dynamics that can be represented by a double integrator, that is, two integrators in series.

In this environment, a training or simulation episode ends when either of these events occurs:

- The mass moves beyond a given threshold (by default, 5 meters) from the origin.
- The norm of the state vector is less than a given threshold (by default, 0.01).

There are two double-integrator environment variants, which differ by the agent action space:

- Discrete — The agent can apply a force of either F_{max} , zero, or $-F_{max}$ to the cart, where F_{max} is the `MaxForce` property of the environment.
- Continuous — The agent can apply any force within the range $[-F_{max}, F_{max}]$.

To create a double-integrator environment, use the `rlPredefinedEnv` function as follows, depending on the action space:

- Discrete action space:

```
env = rlPredefinedEnv("DoubleIntegrator-Discrete");
```

```
env =
```

```
DoubleIntegratorDiscreteAction with properties:
```

```

    Gain: 1
    Ts: 0.1000
    MaxDistance: 5
    GoalThreshold: 0.0100
    Q: [2x2 double]
    R: 0.0100
    MaxForce: 2
    State: [2x1 double]
```

For examples showing how to train an agent to control a discrete double-integrator environment, see “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115. For an example comparing all default discrete action space agents on this environment, see “Compare Agents on Discrete Double-Integrator” on page 7-104.

- Continuous action space:

```
env = rlPredefinedEnv("DoubleIntegrator-Continuous")
```

```
env =
```

DoubleIntegratorContinuousAction with properties:

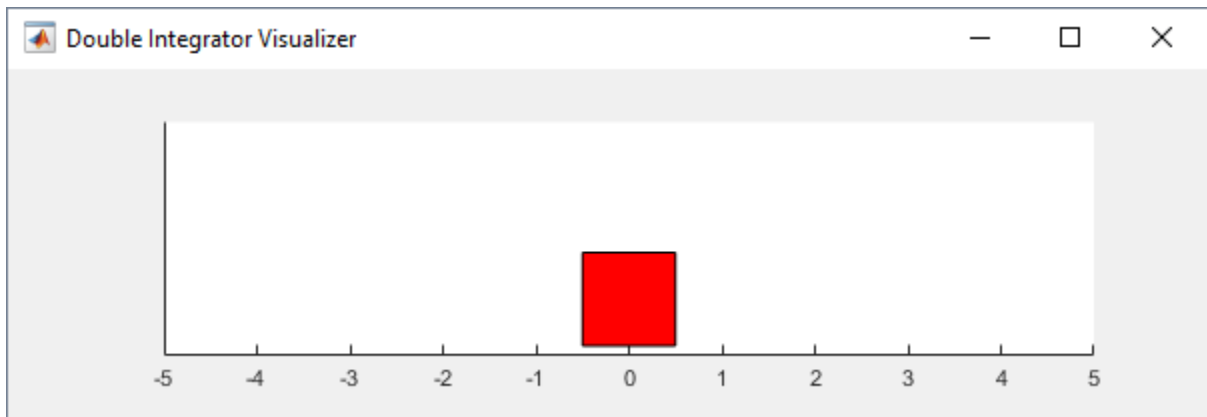
```
Gain: 1
Ts: 0.1000
MaxDistance: 5
GoalThreshold: 0.0100
Q: [2×2 double]
R: 0.0100
MaxForce: Inf
State: [2×1 double]
```

For an example showing how to train an agent to control a continuous double-integrator environment, see “Compare DDPG Agent to LQR Controller” on page 7-285. For an example comparing all default continuous action space agents on this environment, see “Compare Agents on Continuous Double-Integrator” on page 7-189.

Environment Visualization

You can visualize the double-integrator environment using the `plot` function. The plot displays the mass as a red rectangle.

```
plot(env)
```



To visualize the environment during training, call `plot` before training and keep the visualization figure open.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Environment Properties

All the environment properties listed in this table are writable. You can change them to customize the double-integrator environment to your needs.

Property	Description	Default
Gain	Gain for the double integrator. This gain multiplies the force applied to the mass.	1
Ts	Sample time in seconds. The software uses the sample time to discretize the dynamics of the underlying continuous-time system. The discrete-time dynamics is then used to calculate the next state as a function of the current state and of the applied action.	0.1
MaxDistance	Distance magnitude threshold in meters. The episode terminates if the distance is greater than this threshold.	5
GoalThreshold	State norm threshold. The episode terminates if the state is greater than this threshold.	0.01
Q	Weight matrix for the observation component of the reward signal	[10 0; 0 1]
R	Weight matrix for the action component of the reward signal	0.01
MaxForce	Maximum input force in newtons. If the magnitude of the input force exceeds this threshold, the magnitude of the force that is actually applied to the mass is limited to MaxForce.	Discrete: 2 Continuous: Inf
State	Environment state, specified as a column vector with the following state variables: - Mass position - Time derivative of mass position	[0 0]'

Dynamics

The continuous-time dynamics of this environment are described by this equation.

$$\ddot{p}(t) = kF(t)$$

Here, p is the position of the mass, F is the force applied by the agent, and k is the value of the Gain property. The discrete-time dynamics is then obtained using the zero-order hold approximation with sample time Ts. For more information on the used discretization method, see `lqrd`.

Actions

The agent interacts with the environment using an action channel carrying a single scalar, the force applied to the mass. The action space specification is as follows:

- For the environment with discrete action space, the action channel specification is an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)
actInfo =
  rlFiniteSetSpec with properties:
    Elements: [-2 0 2]
    Name: "force"
    Description: [0×0 string]
    Dimension: [1 1]
    DataType: "double"
```

- For the environment with a continuous action space, the action channel specification is an `rlNumericSpec` object.

```
actInfo = getActionInfo(env)
actInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "force"
    Description: [0×0 string]
    Dimension: [1 1]
    DataType: "double"
```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In the double-integrator system, the agent can observe both of the environment state variables stored in `env.State`. The observation has a single channel, specified by an `rlNumericSpec` object, because both states are continuous.

```
obsInfo = getObservationInfo(env)
obsInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "states"
    Description: "x, dx"
    Dimension: [2 1]
    DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Rewards

The reward signal r_{t+1} , provided at every time step, is a discretization of $r(t)$:

$$r(t) = -(x(t)' Q x(t) + u(t)' R u(t))$$

where:

- x is the state vector of the system.
- u is the force applied to the mass.
- Q is the weight matrix on the control performance, with default value $Q = [10\ 0; 0\ 1]$.
- R is the weight matrix on the control effort, with default value $R = 0.01$.

In some control system applications, the reward signal r_k is also referred to as the negative of the cost at *stage* k .

Note that $r(t)$ is the instantaneous equivalent of this cumulative continuous-time reward, which is analogous to the cost function of an LQR controller:

$$\text{reward} = - \int (x'Qx + u'Ru) dt$$

For more information on the discretization of an LQR cost function, see `lqrd`.

Reset Function

The reset function randomly sets the initial state of the environment so that the mass is still and situated at 80% of `MaxDistance` in either the positive or negative direction.

```
x0 = reset(env)
```

```
x0 =
```

```
    4  
    0
```

Unlike other environments, you cannot set your own reset function for a double-integrator environment.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the discrete action space version of this environment, create a default PG agent.

```
pgAgent = rlPGAgent(obsInfo,actInfo)
```

```
pgAgent =
```

```
    rlPGAgent with properties:
```

```
        AgentOptions: [1x1 rl.option.rlPGAgentOptions]  
    UseExplorationPolicy: 1  
        ObservationInfo: [1x1 rl.util.rlNumericSpec]  
        ActionInfo: [1x1 rl.util.rlFiniteSetSpec]  
        SampleTime: 1
```

If needed, modify the agent options using dot notation.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

For examples comparing all default agents on these environments, see “Compare Agents on Discrete Double-Integrator” on page 7-104 and “Compare Agents on Continuous Double-Integrator” on page 7-189.

For an example showing how to train an agent with custom approximators to control a continuous double-integrator environment, see “Compare DDPG Agent to LQR Controller” on page 7-285

You also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an is-done scalar indicating whether a final state has been reached.

For example, call the step function with an input force of -10 N.

```
[xn,rn,id] = step(env,-10)
```

```
xn =
```

```
    3.9500  
   -1.0000
```

```
rn =
```

```
   -16.0005
```

```
id =
```

```
logical  
  
    0
```

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the functions that returns the discrete action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.DoubleIntegratorDiscreteAction
```

To access the function that returns the continuous action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.DoubleIntegratorContinuousAction
```

To access the abstract class that contains the step function, at the MATLAB command line, type:

```
edit rl.env.DoubleIntegratorAbstract
```

MATLAB Cart-Pole Environment

A MATLAB cart-pole environment is a predefined environment in which the goal of the agent is to balance a pole on a moving cart by applying a horizontal force to the cart. The pole is considered successfully balanced if both of these conditions are satisfied at the same time:

- The pole angle (clockwise positive) remains within a given threshold of the vertical position, where the vertical position is zero radians.
- The magnitude of the cart position (right positive) remains below a given threshold.

In this environment, a training or simulation episode ends when either of these events occurs:

- The magnitude of the pole angle with respect to the vertical axis is more than a given threshold (by default, 12 degrees).
- The cart moves more than a predefined threshold (by default, 2.4 meters) from the original position.

There are two cart-pole environment variants, which differ by the agent action space:

- Discrete — The agent can apply a force of either F_{max} or $-F_{max}$ to the cart, where F_{max} is the `MaxForce` property of the environment (by default, 10 newtons).
- Continuous — The agent can apply any force within the range $[-F_{max}, F_{max}]$.

To create a cart-pole environment, use the `rlPredefinedEnv` function as follows, depending on the desired action space:

- Discrete action space:

```
env = rlPredefinedEnv("CartPole-Discrete")
```

CartPoleDiscreteAction with properties:

```

    Gravity: 9.8000
    MassCart: 1
    MassPole: 0.1000
    Length: 0.5000
    MaxForce: 10
    Ts: 0.0200
    ThetaThresholdRadians: 0.2094
    XThreshold: 2.4000
    RewardForNotFalling: 1
    PenaltyForFalling: -5
    State: [4x1 double]
```

For examples showing how to train an agent in this environments, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64 and “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109. For an example comparing tall default discrete action space agents on this environment, see “Compare Agents on Discrete Cart-Pole” on page 7-121.

- Continuous action space:

```
env = rlPredefinedEnv("CartPole-Continuous")
```

CartPoleContinuousAction with properties:

```
Gravity: 9.8000
```

```

    MassCart: 1
    MassPole: 0.1000
    Length: 0.5000
    MaxForce: 10
    Ts: 0.0200
    ThetaThresholdRadians: 0.2094
    XThreshold: 2.4000
    RewardForNotFalling: 1
    PenaltyForFalling: -50
    State: [4x1 double]

```

For an example showing how to train an agent in this environments, see “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356. For an example comparing all default continuous action space agents on this environment, see “Compare Agents on Continuous Cart Pole” on page 7-208.

Environment Properties

All the environment properties listed below are writable. You can change them to customize the cart-pole environment to your needs.

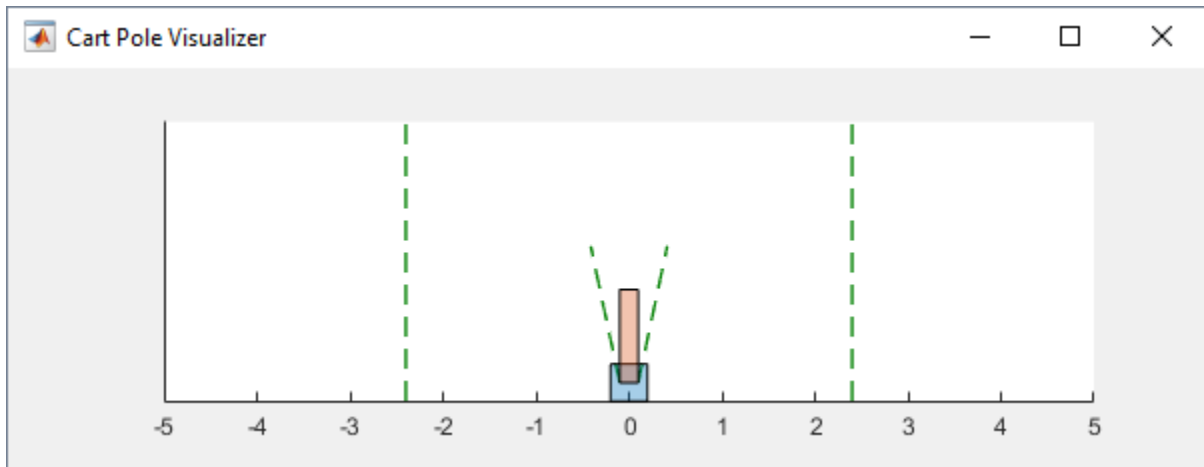
Property	Description	Default
Gravity	Acceleration due to gravity in meters per second squared	9.8
MassCart	Mass of the cart in kilograms	1
MassPole	Mass of the pole in kilograms	0.1
Length	Distance from the pole hinge to the cart center of mass. If the pole density is uniform, this is half the length of the pole.	0.5
MaxForce	Maximum horizontal force magnitude in newtons. If the magnitude of the input force exceeds this threshold, the magnitude of the force that is actually applied to the cart is limited to MaxForce.	10
Ts	Sample time in seconds. The software uses the sample time to discretize the dynamics of the underlying continuous-time system. The discrete-time dynamics is then used to calculate the next state as a function of the current state and of the applied action.	0.02
ThetaThresholdRadians	Pole angle threshold in radians. The episode terminates if the pole angle is greater than this threshold.	0.2094
XThreshold	Cart position threshold in meters. The episode terminates if the cart position is greater than this threshold.	2.4
RewardForNotFalling	Reward for each time step the pole is balanced	1

Property	Description	Default
PenaltyForFalling	Reward penalty for failing to balance the pole	Discrete — -5 Continuous — -50
State	Environment state, specified as a column vector with these state variables: • Cart position (right-positive, meters) • Derivative of cart position (meters per second) • Pole angle (clockwise-positive, radians) • Derivative of pole angle (radians per second)	[0 0 0 0]'

Visualization

You can visualize the cart-pole environment using the `plot` function. The plot displays the cart as a blue square and the pole as a red rectangle.

```
plot(env)
```



To visualize the environment during training, call `plot` before training and keep the visualization figure open.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Dynamics

The cart is subjected to an external input force and can move horizontally (the horizontal axis points right). Neither the cart nor the pole is subject to friction. The pole can rotate about the hinge on the upper surface of the cart (clockwise rotations are positive). Gravity acts vertically downwards. The center of gravity of the cart is at its geometrical center, and the center of gravity of the pole is at its half length. The continuous-time equations used to model this system are described in the **Examples of Cart-Pole System in Reinforcement Learning Toolbox** section of the example “Derive Equations of Motion and Simulate Cart-Pole System” (Symbolic Math Toolbox) and the reference [2]

therein. The discrete-time dynamics of the environment are then obtained using the Euler integration method.

Actions

The agent interacts with the environment using an action channel carrying a single scalar, the horizontal force applied to the cart. The action space specification is as follows:

- For an environment with a discrete action space, the action channel is specification is an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)

rlFiniteSetSpec with properties:
    Elements: [-10 10]
    Name: "CartPole Action"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"
```

- For an environment with a continuous action space, the action specification is an `rlNumericSpec` object.

```
actInfo = getActionInfo(env)

rlNumericSpec with properties:
    LowerLimit: -10
    UpperLimit: 10
    Name: "CartPole Action"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"
```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In the cart-pole system, the agent can observe all the environment state variables stored in `env.State`. The observation has a single channel specified by an `rlNumericSpec` object, because both signals are continuous.

```
obsInfo = getObservationInfo(env)

obsInfo =

rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "CartPole States"
    Description: "x, dx, theta, dtheta"
    Dimension: [4 1]
    DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Reward

The reward signal for this environment is either:

- A positive reward for each time step that the pole is balanced, that is, when the cart and pole both remain within their specified threshold ranges. This reward accumulates over the entire training episode. To control the size of this reward, use the `RewardForNotFalling` property of the environment.
- A one-time negative penalty if either the pole or cart move outside of their threshold range. At this point, the training episode stops. To control the size of this penalty, use the `PenaltyForFalling` property of the environment.

Reset Function

The reset function sets the initial state of the environment so that the cart position and velocity is zero, the pole angle is a random number between -0.05 and 0.05 radians, and the pole angular velocity is zero.

```
x0 = reset(env)
```

```
x0 =
```

```
    0
    0
0.0315
    0
```

Unlike other environments, you cannot set your own reset function for a cart-pole environment.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the continuous action space version of this environment, create a default AC agent.

```
acAgent = rlACAgent(obsInfo,actInfo)
```

```
acAgent =
```

```
    rlACAgent with properties:
```

```
        AgentOptions: [1x1 rl.option.rlACAgentOptions]
    UseExplorationPolicy: 1
        ObservationInfo: [1x1 rl.util.rlNumericSpec]
            ActionInfo: [1x1 rl.util.rlNumericSpec]
            SampleTime: 1
```

If needed, modify the agent options using dot notation.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

For examples comparing all default agents on these environments, see “Compare Agents on Discrete Cart-Pole” on page 7-121 and “Compare Agents on Continuous Cart Pole” on page 7-208.

For other examples showing how to train default agents for these environments, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64 and “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356.

For examples showing how to train agents with custom approximators for the discrete action space version of this environment, see “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109, “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157 and “Train LSPI Agent to Balance Discrete Cart-Pole” on page 5-102.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether a final state has been reached.

For example, call the step function with an input force of 10 newtons.

```
[xn,rn,id] = step(env,10)
```

```
xn =
```

```
    0  
  0.1947  
  0.0315  
 -0.2826
```

```
rn =
```

```
    1
```

```
id =
```

```
logical  
    0
```

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the functions that returns the discrete action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.CartPoleDiscreteAction
```

To access the function that returns the continuous action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.CartPoleContinuousAction
```

To access the abstract class that contains the step function, at the MATLAB command line, type:

```
edit rl.env.CartPoleAbstract
```

MATLAB Simple Pendulum Swing-Up with Image Observation Environment

A simple pendulum swing-up with image observation environment is a predefined MATLAB environment featuring a simple pendulum that initially hangs still in a downward position, that is, the initial angle of the pendulum is π and its initial angular speed is zero. The goal of the agent is to swing up the pendulum by applying an input torque and then keep the pendulum upright, using minimal control effort.

In this environment, the full state of the system (the angular position and the velocity of the pendulum) is not entirely available to the agent as an observation. Specifically, while the angular velocity is available in the second observation channel, the first observation channel carries a 50-by-50 pixel image representing the position of the pendulum center of mass. So, the agent also has to learn to estimate the pendulum angle from the image, as well as learning the right torque to apply to swing up the pendulum and keep it upright. As a consequence, agents built for this task normally feature one or more convolution networks, and training an agent in this environment is typically more computationally demanding than training an agent in simpler predefined environments.

There are two simple pendulum with image environment variants, which differ by the agent action space:

- Discrete — The agent can apply a torque of -2, -1, 0, 1, or 2 to the pendulum.
- Continuous — The agent can apply any torque within the range [-2,2].

To create a simple pendulum environment, use the `rlPredefinedEnv` function as follows, depending on the desired action space:

- Discrete action space:

```
env = rlPredefinedEnv("SimplePendulumWithImage-Discrete")
```

```
env =
```

SimplePendulumWithImageDiscreteAction with properties:

```

    Mass: 1
    RodLength: 1
    RodInertia: 0
    Gravity: 9.8100
    DampingRatio: 0
    MaximumTorque: 2
    Ts: 0.0500
    State: [2x1 double]
    Q: [2x2 double]
    R: 1.0000e-03
```

For an example showing how to train an agent in this environments, see “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217. For an example

comparing all default discrete action space agents on this environment, see “Compare Agents on Discrete Pendulum Swing-Up with Image” on page 7-174.

- Continuous action space:

```
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous")
```

```
env =
```

```
SimplePendulumWithImageContinuousAction with properties:
```

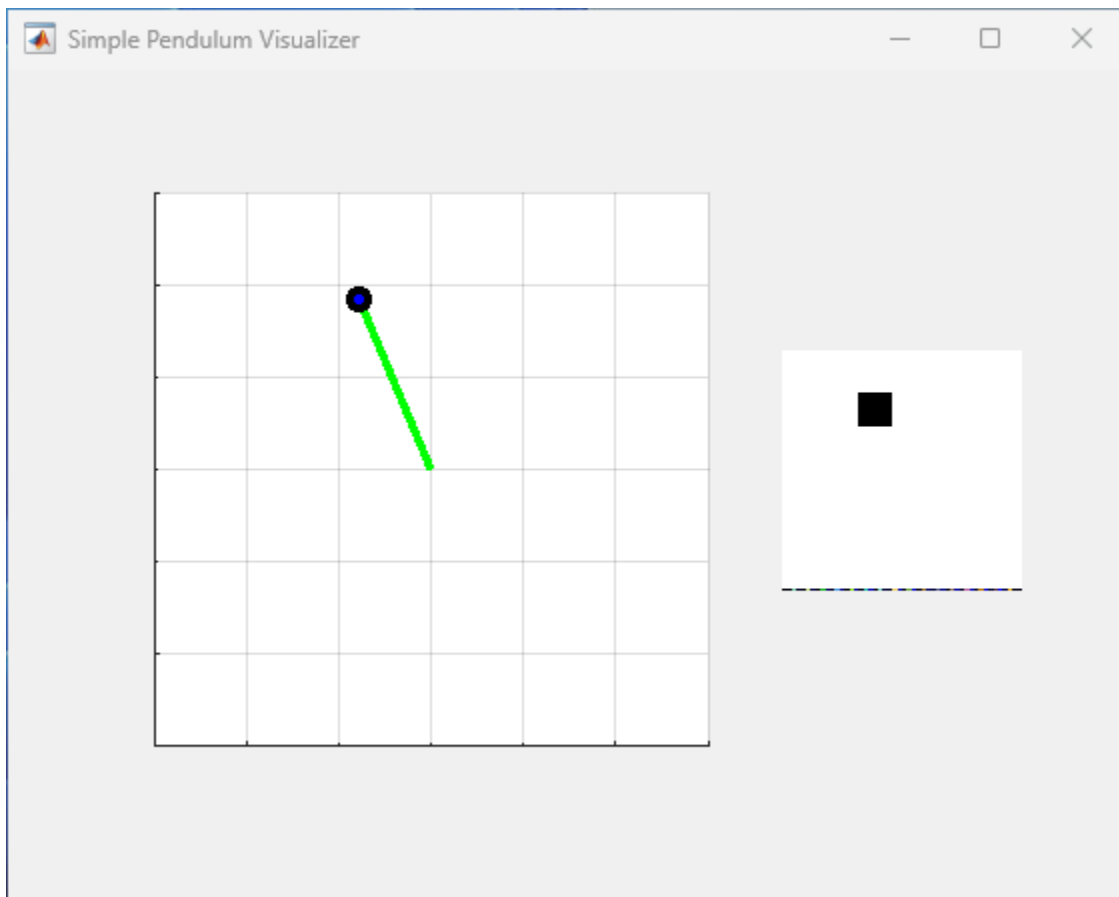
```
    Mass: 1
    RodLength: 1
    RodInertia: 0
    Gravity: 9.8100
    DampingRatio: 0
    MaximumTorque: 2
    Ts: 0.0500
    State: [2×1 double]
    Q: [2×2 double]
    R: 1.0000e-03
```

For an example showing how to train an agent in this environments, see “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122. For an example comparing all default continuous action space agents on this environment, see “Compare Agents on Continuous Pendulum Swing-Up with Image” on page 7-268.

Environment Visualization

You can visualize the simple pendulum with image environment using the `plot` function. The main figure on the left side of the plot displays the rod as a green line, and a black circle surrounding a blue dot indicates the position of the rod center of mass. The plot also contains a smaller figure on the right visualizing the content of the first observation channel, which is a 50-by-50 pixel image with a black square representing the position of the pendulum center of mass.

```
plot(env)
```



To visualize the environment during training, call `plot` before training and keep the visualization figure open.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Environment Properties

All the environment properties listed in this table are writable. You can change them to customize the simple pendulum with image environment to your needs.

Property	Description	Default
Mass	Pendulum mass in kilograms	1
RodLength	Distance from the hinge to the pendulum center of mass in meters. For uniform rods, this distance is the half-length.	1
RodInertia	Inertia moment of the rod (pendulum) about its center of mass in kilograms per meter squared	0
Gravity	Acceleration due to gravity in meters per second squared	9.81

Property	Description	Default
DampingRatio	Damping on pendulum motion (viscous friction) in newtons per meter per second.	0
MaximumTorque	Maximum input torque in newtons per meter. If the magnitude of the input torque exceeds this threshold, the magnitude of the torque that is actually applied to the pendulum is limited to MaximumTorque	2
Ts	Sample time in seconds. The software uses the sample time to discretize the dynamics of the underlying continuous-time system. The discrete-time dynamics is then used to calculate the next state as a function of the current state and of the applied action.	0.05
State	Environment state, specified as a column vector with these state variables: - Pendulum angle (counterclockwise-positive, radians) - Pendulum angular velocity (radians per second)	[0 0]'
Q	Weight matrix for the observation component of reward signal	[1 0;0 0.1]
R	Weight matrix for the action component of reward signal	1e-3

Dynamics

The pendulum is a rigid rod subjected to an input torque (counterclockwise positive) and might be subjected to a viscous friction that counteracts the torque. The angle of rotation is counterclockwise positive and gravity acts vertically downwards. The center of gravity of the pendulum is at its half-length. The continuous-time equations used to model this system are described in the **Simple Pendulum Equation** section of the example “Derive Equations of Motion and Simulate Cart-Pole System” (Symbolic Math Toolbox). The discrete-time dynamics of the environment are then obtained using a trapezoidal integration method.

Actions

The agent interacts with the environment using an action channel carrying a single scalar, the torque applied at the base of the pendulum. The action space specification is as follows:

- For environments with a discrete action space, the specification is an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)

actInfo =

rlFiniteSetSpec with properties:

    Elements: [-2 -1 0 1 2]
    Name: "torque"
    Description: [0x0 string]
```



```

    Dimension: [1 1]
    DataType: "double"

```

- For environments with a continuous action space, the specification is an `rlNumericSpec` object.

```

actInfo = getActionInfo(env)

actInfo =

rlNumericSpec with properties:

    LowerLimit: -2
    UpperLimit: 2
    Name: "torque"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"

```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In a simple pendulum environment with image environment, the agent receives the following observation signals:

- 50-by-50 grayscale image of the pendulum position
- Derivative of the pendulum angle

The observation has two channels, each specified by an `rlNumericSpec` object, because both signals are continuous.

```

obsInfo = getObservationInfo(env)

1x2 rlNumericSpec array with properties:

    LowerLimit
    UpperLimit
    Name
    Description
    Dimension
    DataType

```

```

obsInfo(1)

rlNumericSpec with properties:

    LowerLimit: 0
    UpperLimit: 1
    Name: "pendImage"
    Description: [0x0 string]
    Dimension: [50 50]
    DataType: "double"

```

```

obsInfo(2)

rlNumericSpec with properties:

    LowerLimit: -Inf
    UpperLimit: Inf

```

```
        Name: "angularRate"
    Description: [0×0 string]
    Dimension: [1 1]
    DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Reward

The reward signal for this environment, at time t , is:

$$r_t = -\left(\theta_t^2 + 0.1 \dot{\theta}_t^2 + 0.001 u_{t-1}^2\right)$$

Here:

- θ_t is the pendulum angle of displacement from the upright position (counterclockwise-positive, radians).
- $\dot{\theta}_t$ is the derivative of the pendulum angle with respect to time.
- u_{t-1} is the control effort from the previous time step.

Reset Function

The reset function sets the initial state of the environment so that the pendulum is hanging downward and still.

```
reset(env);
env.State
```

```
ans =

    3.1416
         0
```

Unlike other environments, you cannot set your own reset function for a simple pendulum with image environment.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the discrete action space version of this environment, create a default DQN agent.

```
dqnAgent = rlDQNAgent(obsInfo,actInfo)

dqnAgent =

    rlDQNAgent with properties:
        ExperienceBuffer: [1×1 rl.replay.rlReplayMemory]
        AgentOptions: [1×1 rl.option.rlDQNAgentOptions]
        UseExplorationPolicy: 0
        ObservationInfo: [1×2 rl.util.rlNumericSpec]
        ActionInfo: [1×1 rl.util.rlFiniteSetSpec]
        SampleTime: 1
```

If needed, modify the agent options using dot notation.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

For examples comparing all default agents on these environments, see “Compare Agents on Discrete Pendulum Swing-Up with Image” on page 7-174 and “Compare Agents on Continuous Pendulum Swing-Up with Image” on page 7-268.

For other examples showing how to train agents with custom networks for these environments, see “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217 and “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether a final state has been reached.

For example, call the step function with an input torque of 1 newton per meter.

```
[xn,rn,id]=step(env,1)

xn =

    1×2 cell array

    {50×50 double}    {[0.0500]}

rn =

    -9.8630

id =

    logical

    0
```

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the functions that returns the discrete action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.SimplePendulumWithImageDiscreteAction
```

To access the function that returns the continuous action space version of this environment, at the MATLAB command line, type:

```
edit rl.env.SimplePendulumWithImageContinuousAction
```

To access the abstract class that contains the step function, at the MATLAB command line, type:

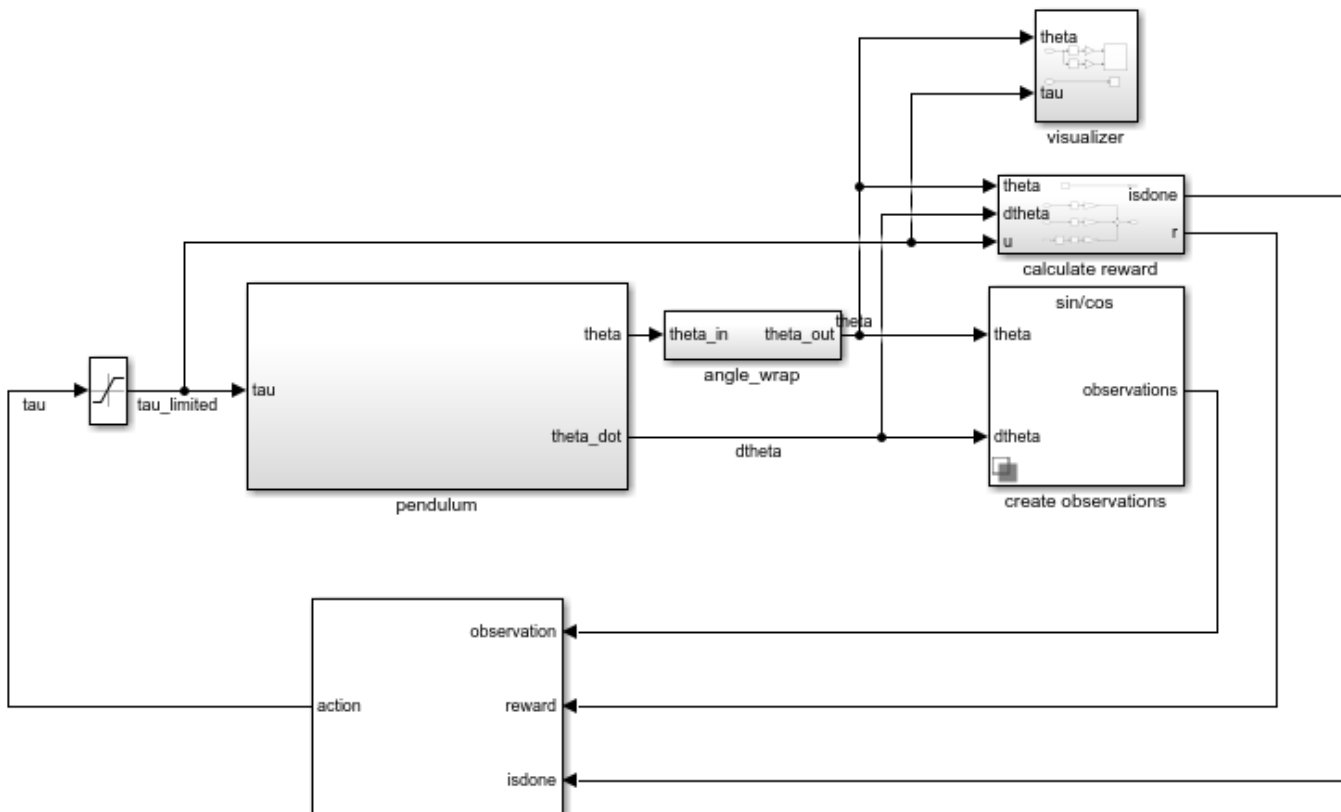
```
edit rl.env.AbstractSimplePendulumWithImage
```

Simulink Simple Pendulum Swing-up Environment

A simple pendulum swing-up environment is a predefined Simulink environment featuring a simple pendulum that initially hangs still in a downward position (that is, the initial angle of the pendulum is π and its initial angular speed is zero). The goal of the agent is to swing up the pendulum by applying an input torque and then keep the pendulum upright using minimal control effort.

The model for this environment is defined in the `rlSimplePendulumModel` Simulink model.

```
open_system("rlSimplePendulumModel")
```



There are two simple pendulum environment variants, which differ by the agent action space.

- Discrete — The agent can apply a torque of either T_{max} , 0 , or $-T_{max}$ to the pendulum.
- Continuous — The agent can apply any torque within the range $[-T_{max}, T_{max}]$.

Here, T_{max} is the variable `max_tau`, which you can change by accessing the workspace of the `rlSimplePendulumModel` model. For more information on other variables that you can change, see “Dynamics” on page 2-64. For more information on Simulink model workspaces, see “Model Workspaces” (Simulink) and “Change Model Workspace Data” (Simulink).

To create a simple pendulum environment, use the `rlPredefinedEnv` as follows, depending on the desired action space:

- Discrete action space:

```
env = rlPredefinedEnv("SimplePendulumModel-Discrete")
```

```
env =
```

SimulinkEnvWithAgent with properties:

```
Model : rlSimplePendulumModel
AgentBlock : rlSimplePendulumModel/RL Agent
ResetFcn : []
UseFastRestart : on
```

For an example of how to that train an agent in this environment, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70. For an example comparing default discrete action space agents on this environment, see “Compare Agents on the Discrete Pendulum Swing-Up” on page 7-138.

- Continuous action space:

```
env = rlPredefinedEnv("SimplePendulumModel-Continuous")
```

```
env =
```

SimulinkEnvWithAgent with properties:

```
Model : rlSimplePendulumModel
AgentBlock : rlSimplePendulumModel/RL Agent
ResetFcn : []
UseFastRestart : on
```

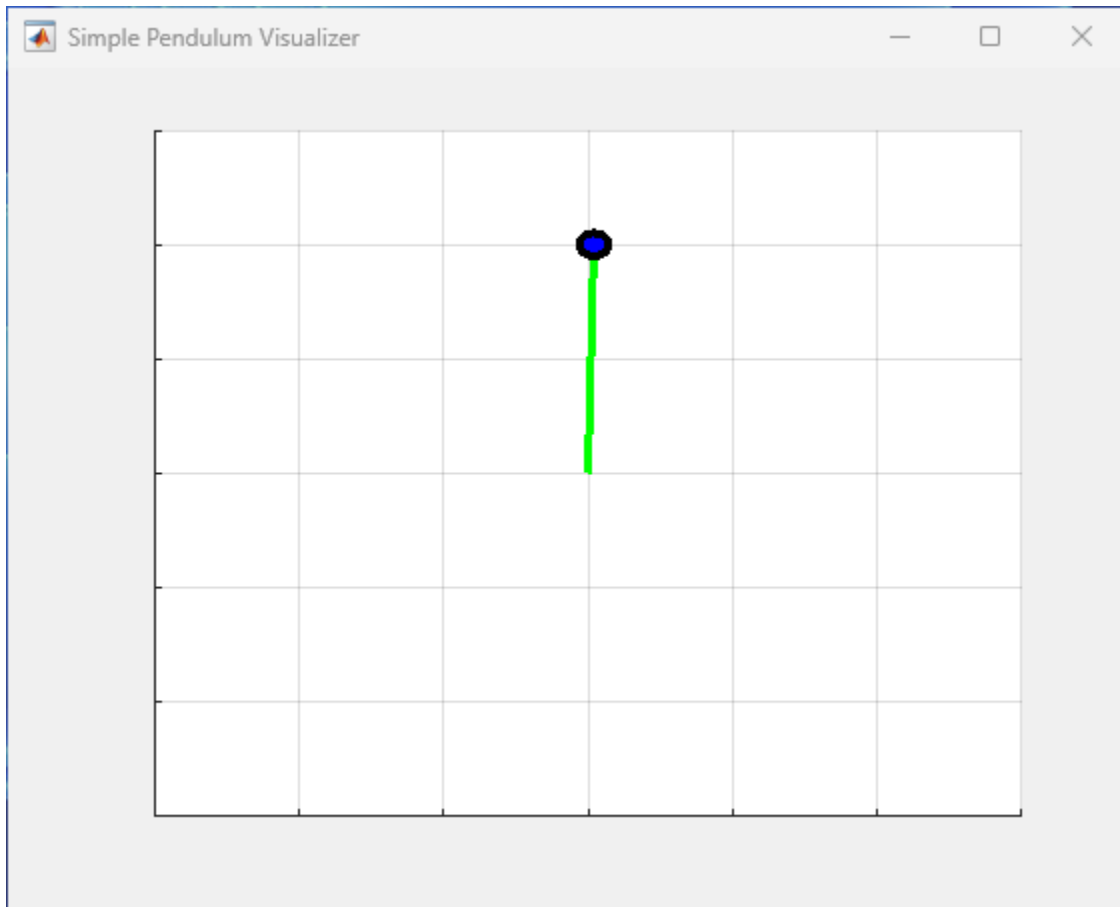
For an example of how to that train an agent in this environment, see “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78. For an example comparing default continuous action space agents on this environment, see “Compare Agents on Continuous Pendulum Swing-Up” on page 7-228.

Note When training or simulating an agent within a Simulink environment, to ensure that the RL Agent block executes at the desired sample time, set the `SampleTime` property of the agent object appropriately.

For more information on Simulink environments, see `SimulinkEnvWithAgent` and “Create Custom Simulink Environments” on page 2-110.

Environment Visualization

The environment is visualized with a plot that displays the pendulum as a green line and a black circle surrounding a blue dot indicating the position of the rod center of mass. The plot is displayed automatically by the Simulink model when it is executed by the simulation or training function.



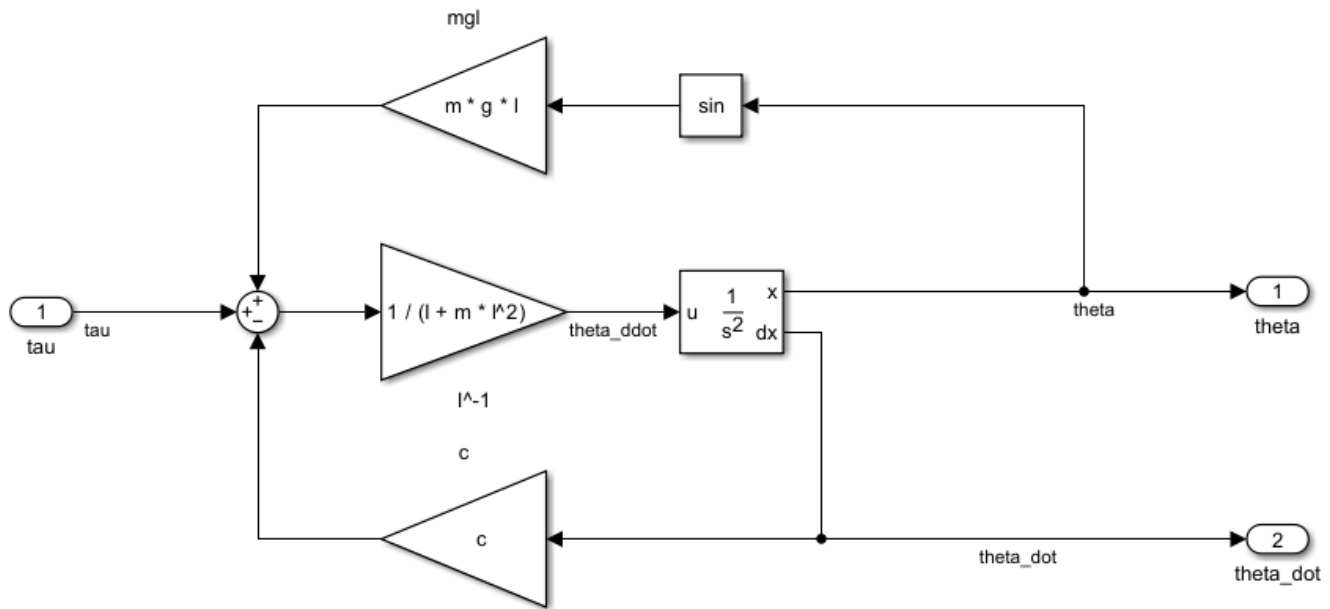
Environment Properties

The environment properties that you can access using dot notation are described in `SimulinkEnvWithAgent`.

Dynamics

The pendulum is a rigid rod subjected to an input counterclockwise-positive torque. The pendulum might be subjected to a viscous friction that counteracts the torque. The angle of rotation is counterclockwise positive and gravity acts vertically downward. The center of gravity of the pendulum is at its half-length. For the exact continuous-time equations used to model this system, see the **Simple Pendulum Equation** section in the example "Derive Equations of Motion and Simulate Cart-Pole System" (Symbolic Math Toolbox). During simulation, the Simulink solver then integrates the dynamics according to the selected solver, while the RL Agent block executes at discrete intervals according to the sample time specified in the `SampleTime` property of the agent.

The pendulum dynamics are represented in the pendulum subsystem of the `rlSimplePendulumModel` model, which calculates the pendulum angular position and velocity as a function of the input force and of the previous state:



$$(I + ml^2) \theta_{ddot} = \tau - c \theta_{dot} + mgl \sin(\theta)$$

The parameters that define the pendulum dynamics are stored in the workspace of the `rlSimplePendulumModel` model.

Variable	Description	Default
I	Inertia moment of the rod (pendulum) about its center of mass in kilograms per meter squared	0
c	Torque damping on pendulum motion (viscous friction) in newtons per meter per second.	0
g	Acceleration due to gravity in meters per second squared	9.81
l	Distance from the hinge to the pendulum center of mass in meters. For uniform rods, this distance is the half-length.	1
m	Pendulum mass in kilograms	1
max_tau	Maximum input torque in newtons per meter. If the magnitude of the input torque exceeds this threshold, the magnitude of the torque that is actually applied to the pendulum is limited to <code>MaximumTorque</code> .	2
theta0	Initial value of the pendulum angle (counterclockwise-positive). The reset function normally sets this value to <code>topi</code> .	0
theta_dot0	Initial value of the pendulum angular speed. The reset function normally sets this value to 0.	0

For more information on Simulink model workspaces, see “Model Workspaces” (Simulink) and “Change Model Workspace Data” (Simulink).

Actions

The agent interacts with the environment using an action channel carrying a single scalar, the torque applied at the base of the pendulum. The action space specification is as follows:

- For an environment with a discrete action space, the specification is an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)
actInfo =
    rlFiniteSetSpec with properties:
        Elements: [3×1 double]
        Name: "torque"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
actInfo.Elements'
ans =
    -2     0     2
```

- For an environment with a continuous action space, the specification is an `rlNumericSpec` object.

- `actInfo = getActionInfo(env)`

```
actInfo =
    rlNumericSpec with properties:
        LowerLimit: -2
        UpperLimit: 2
        Name: "torque"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In a simple pendulum swing-up with Simulink environment, the observation is calculated within the create observations subsystem of the `rlSimplePendulumModel` model.

The agent receives these observation signals:

- Sine of the pendulum angle
- Cosine of the pendulum angle
- Derivative of the pendulum angle with respect to time

The observation has a single channel specified by an `rlNumericSpec` object, because the three observation signals are continuous.


```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
```

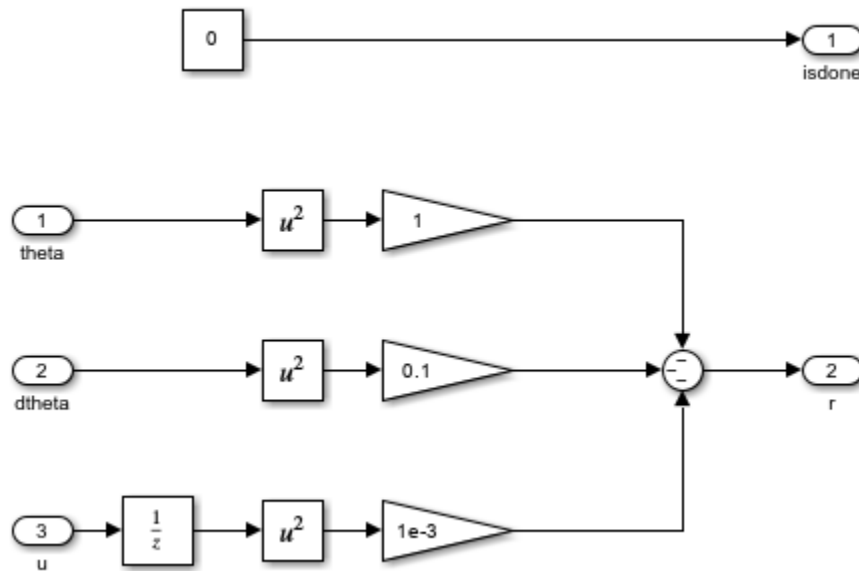
```
rlNumericSpec with properties:
```

```
LowerLimit: -Inf
UpperLimit: Inf
Name: "observations"
Description: [0×0 string]
Dimension: [3 1]
DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Reward

In this environment, the reward is calculated within the calculate reward subsystem of the `rlSimplePendulumModel` model.



The reward is

$$r_t = -\left(\theta_t^2 + 0.1 \dot{\theta}_t^2 + 0.001 u_{t-1}^2\right)$$

Here:

- θ_t is the pendulum angle of displacement (counterclockwise-positive) from the upright position.
- $\dot{\theta}_t$ is the derivative of the pendulum angle with respect to time.
- u_{t-1} is the control effort from the previous time step.

Reset Function

The environment has no default `reset` function. However, you can set your own `reset` function to set the initial state of the pendulum every time a simulation starts to train or simulate the environment.

For example, to set the initial condition of the pendulum as still and hanging downward, specify the handle of an anonymous `reset` function that sets the model workspace variable `theta0` to `pi`.

```
env.ResetFcn = @(in)setVariable(in,  
    "theta0",pi, ...  
    "Workspace","rlSimplePendulumModel");
```

Here, you use `setVariable` to set the variable `theta0` to a random value in the `Simulink.SimulationInput` object `in`. The value of `theta0` that you specify overrides the existing `theta0` value in the model workspace for the duration of the simulation or training. The value of `theta0` then reverts to its original value when the simulation or training completes. For more information, see “Reset Function for Simulink Environments” on page 2-112.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the continuous action space version of this environment, create a default PPO agent.

```
agent = rlPPOAgent(obsInfo,actInfo)
```

```
agent =
```

```
    rlPPOAgent with properties:
```

```
        AgentOptions: [1x1 rl.option.rlPPOAgentOptions]  
    UseExplorationPolicy: 1  
        ObservationInfo: [1x1 rl.util.rlNumericSpec]  
          ActionInfo: [1x1 rl.util.rlNumericSpec]  
        SampleTime: 1
```

Set the agent sample time so that the RL Agent block executes at the desired rate.

```
agent.SampleTime = 0.05;
```

If needed, modify the agent options using dot notation.

```
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

For examples comparing all default agents on these environments, see “Compare Agents on the Discrete Pendulum Swing-Up” on page 7-138 and “Compare Agents on Continuous Pendulum Swing-Up” on page 7-228.

For other examples showing how to train default agents for these environments, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70 and “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

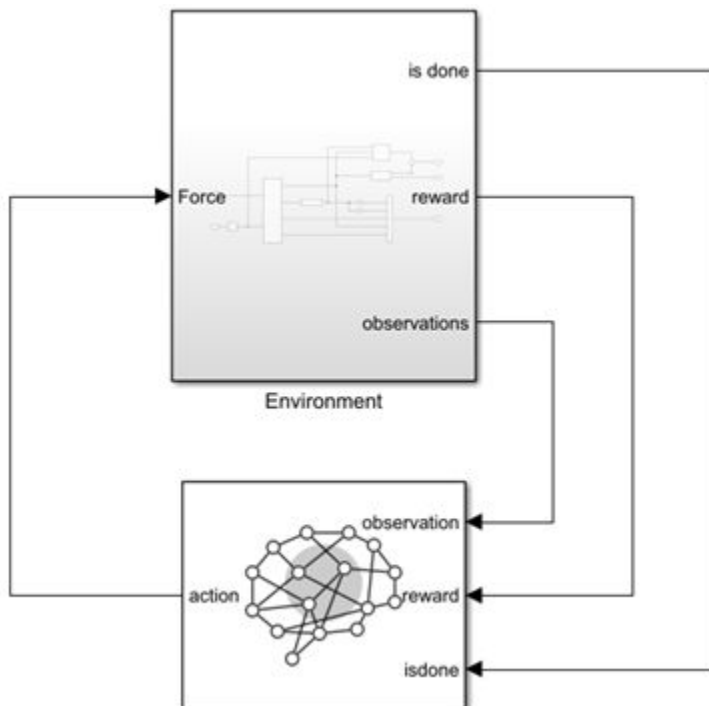
Because this is a Simulink environment, calling the `step` function is not supported. For an example on how to build a custom training loop using a Simulink environment, see “Custom Training Loop with Simulink Action Noise” on page 5-282.

Simscape Cart-Pole Swing-up Environment

A Simscape cart-pole swing-up environment is a predefined Simulink environment featuring a cart-pole system modeled using Simscape Multibody™. The pole initially hangs still in a downward position (that is, the initial angle is π radians and the initial angular speed is zero). The goal of the agent is to swing up the pole by applying an input force on the cart and then maintain the pole upright using minimal control effort. The episode terminates if the cart moves more than 3.5 meters from the original position.

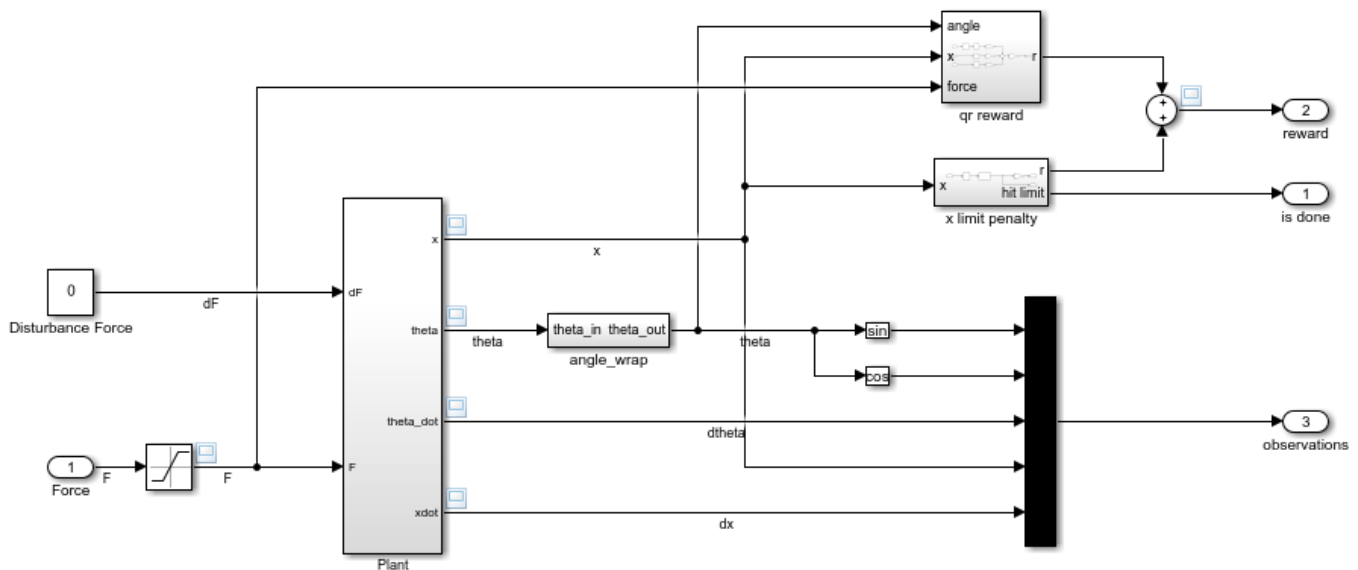
The model for this environment is defined in the `rlCartPoleSimscapeModel` Simulink model.

```
open_system("rlCartPoleSimscapeModel")
```



In the `Environment` subsystem, the model dynamics are defined within the `Plant` subsystem, using Simscape components. The reward and is-done signals are defined in the `qr reward` and `x limit` penalty subsystems, respectively, using Simulink blocks. The black multiplexer block aggregates the observations into a single observation signal.

```
open_system("rlCartPoleSimscapeModel/Environment")
```



There are two variants of this environment, which differ by the agent action space:

- Discrete — The agent can apply a force of 15, 0, or -15 newtons to the cart.
- Continuous — The agent can apply any force within the range [-15,15] newtons.

To create a cart-pole environment, use the `rlPredefinedEnv` function as follows, depending on the desired action space:

- Discrete action space:

```
env = rlPredefinedEnv("CartPoleSimscapeModel-Discrete")
```

```
env =
```

SimulinkEnvWithAgent with properties:

```
Model : rlCartPoleSimscapeModel
AgentBlock : rlCartPoleSimscapeModel/RL Agent
ResetFcn : []
UseFastRestart : on
```

For an example comparing default discrete action space agents on this environment, see “Compare Agents on Discrete Simscape Cart-Pole Swing-Up” on page 7-157.

- Continuous action space:

```
env = rlPredefinedEnv("CartPoleSimscapeModel-Continuous")
```

```
env =
```

SimulinkEnvWithAgent with properties:

```
Model : rlCartPoleSimscapeModel
AgentBlock : rlCartPoleSimscapeModel/RL Agent
```

```
ResetFcn : []
UseFastRestart : on
```

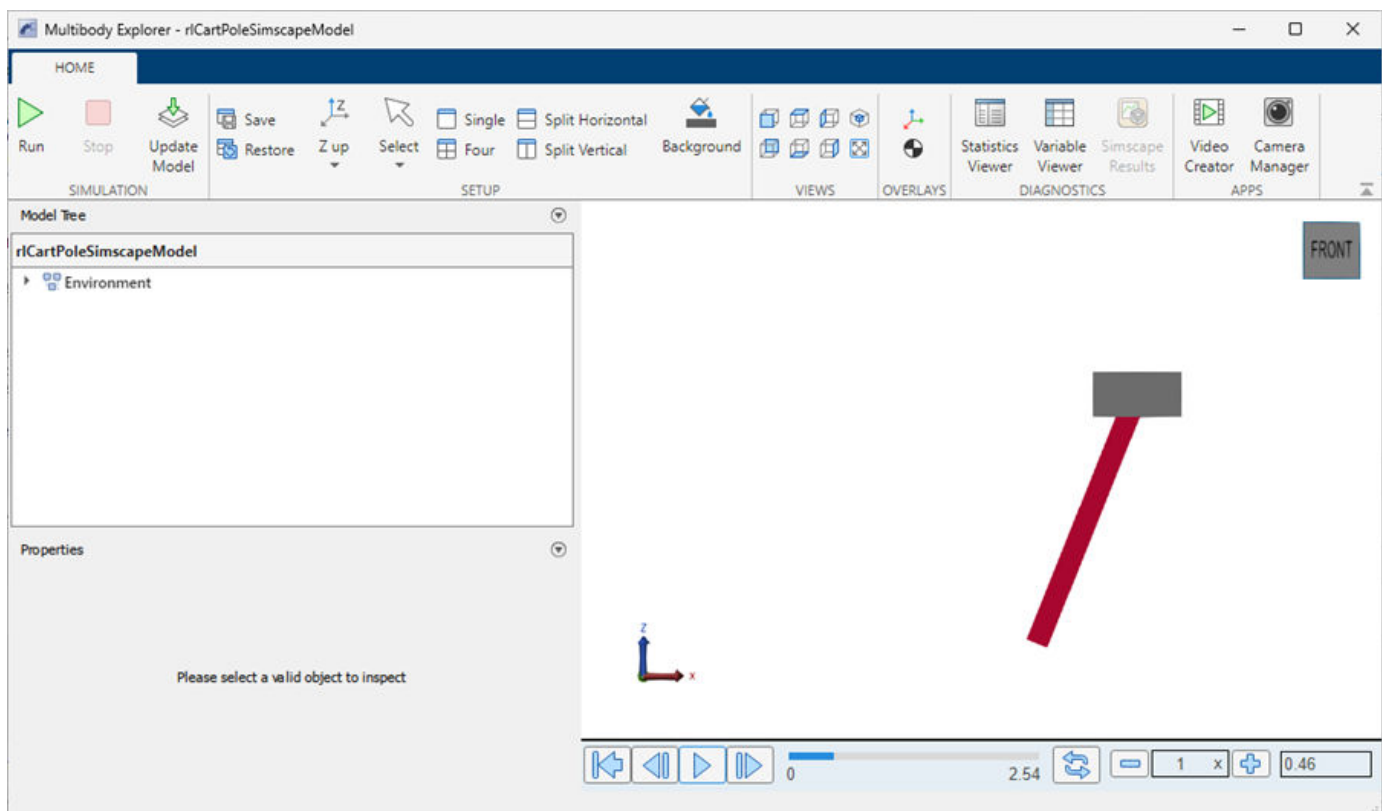
For an example that trains an agent in this cart-pole environment, see “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85. For an example comparing default continuous action space agents on this environment, see “Compare Agents on Continuous Cart Pole Swing-Up” on page 7-251.

Note When training or simulating an agent within a Simulink environment, to ensure that the RL Agent block executes at the desired sample time, set the `SampleTime` property of the agent object appropriately.

For more information on Simulink environments, see `SimulinkEnvWithAgent` and “Create Custom Simulink Environments” on page 2-110.

Environment Visualization

The environment visualization shows the cart and the pole as gray and red rectangles, respectively, and it is displayed automatically in the Multibody Explorer by the Simulink model when it is executed by the simulation or training function.



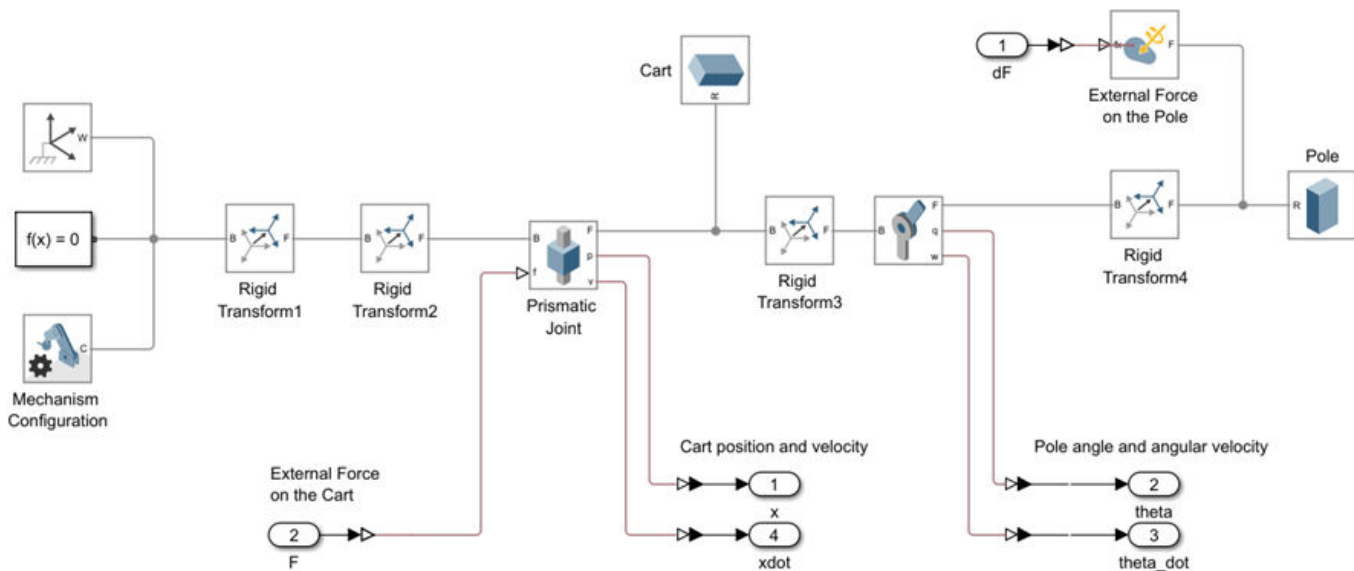
Environment Properties

The environment properties that you can access using dot notation are described in `SimulinkEnvWithAgent`.

Dynamics

The cart is subjected to an external input force as well as viscous friction and can move horizontally (the horizontal axis points right). The pole is not subject to friction and can rotate about the hinge on the cart (counterclockwise rotations are positive). Gravity acts vertically downwards. For the exact continuous-time equations used to model this system, see the **Examples of Cart-Pole System in Simscape Multibody** section in the example “Derive Equations of Motion and Simulate Cart-Pole System” (Symbolic Math Toolbox). During simulation, the Simulink solver integrates the dynamics according to the selected solver, while the RL Agent block executes at discrete intervals according to the sample time specified in the SampleTime property of the agent.

The continuous-time system dynamics are represented using Simscape Multibody in the Plant subsystem of the `rlCartPoleSimscapeModel` model. This subsystem calculates the linear and angular positions and velocities of the cart and pole, respectively, as a function of the input force on the cart and of the previous state:



The parameters that define the system dynamics are stored in the workspace of the `rlCartPoleSimscapeModel` model.

Variable	Description	Default
<code>mCart</code>	Mass of the cart in kilograms	1
<code>mPend</code>	Mass of the pole in kilograms	1
<code>Gravity</code>	Acceleration due to gravity in meters per second squared	9.8
<code>Length</code>	Distance from the pole hinge to the cart center of mass. If the pole density is uniform, this distance is half the length of the pole.	0.5
<code>theta0</code>	Initial value of the pendulum angle (counterclockwise-positive) in degrees.	180

Variable	Description	Default
Kd	Force damping on cart motion (viscous friction) in newtons per second over meter.	10

For more information on Simulink model workspaces, see “Model Workspaces” (Simulink) and “Change Model Workspace Data” (Simulink).

Actions

The agent interacts with the environment using an action channel carrying a single scalar, the horizontal force applied to the cart. The action space specification is as follows:

- Discrete action space, the channel is specified by an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)
actInfo =
    rlFiniteSetSpec with properties:
        Elements: [3×1 double]
        Name: "force"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
actInfo.Elements'
ans =
    -15     0    15
```

- Continuous action space, the channel is specified by an `rlNumericSpec` object.

```
actInfo = getActionInfo(env)
actInfo =
    rlNumericSpec with properties:
        LowerLimit: -15
        UpperLimit: 15
        Name: "force"
        Description: [0×0 string]
        Dimension: [1 1]
        DataType: "double"
```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In this environment, the observations are aggregated using a multiplexer block within the Environment subsystem of the `rlCartPoleSimscapeModel` model. The agent receives the following five observation signals.

- Sine of the pole angle
- Cosine of the pole angle

- Derivative of the pendulum angle
- Cart position
- Derivative the of cart position

The observation has a single channel, specified by an `rlNumericSpec` object, because the five observation signals are continuous,

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
```

```
rlNumericSpec with properties:
```

```
LowerLimit: -Inf
UpperLimit: Inf
Name: "observations"
Description: [0x0 string]
Dimension: [5 1]
DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Reward

In this environment, the reward is calculated within the `qr` reward subsystem.

The reward signal for this environment is the sum of two components ($r = r_{qr} + r_n + r_p$):

- A quadratic regulator control reward, constructed in the `Environment/qr` reward subsystem:

$$r_{qr} = -\left(0.1 * x^2 + 0.5 * \theta^2 + 0.005 * u_{t-1}^2\right)$$

- A cart limit penalty, constructed in the `Environment/x limit penalty` subsystem. This subsystem generates a negative reward when the magnitude of the cart position exceeds a given threshold:

$$r_p = -100 * (|x| \geq 3.5)$$

Here:

- x is the cart position.
- θ is the pole angle of displacement from the upright position.
- u_{t-1} is the control effort from the previous time step.

Reset Function

The environment has no default `reset` function. However, you can set your own `reset` function to set the initial state of the system every time a simulation starts to train or simulate the environment.

For example, to set the initial pole to a random angle between 180 and -180 degrees, instead of the default value of 180 degrees, specify the handle of an anonymous reset function that sets the model workspace variable `theta0` to `360*rand-180`.


```
env.ResetFcn = @(in)setVariable(in, ...
    "theta0",360*rand-180, ...
    "Workspace","rlCartPoleSimscapeModel");
```

Here, you use `setVariable` to set the variable `theta0` to a random value in the `Simulink.SimulationInput` object `in`. The value of `theta0` that you specify overrides the existing `theta0` value in the model workspace for the duration of the simulation or training. The value of `theta0` then reverts to its original value when the simulation or training completes. For more information, see “Reset Function for Simulink Environments” on page 2-112.

Create a Default Agent for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the discrete action space version of this environment, create a default SAC agent.

```
agent = rlSACAgent(obsInfo,actInfo)
```

```
agent =
```

```
rlSACAgent with properties:
```

```
ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]
AgentOptions: [1x1 rl.option.rlSACAgentOptions]
UseExplorationPolicy: 1
ObservationInfo: [1x1 rl.util.rlNumericSpec]
ActionInfo: [1x1 rl.util.rlFiniteSetSpec]
SampleTime: 1
```

Set the agent sample time so that the RL Agent block executes at the desired rate.

```
agent.SampleTime = 0.02;
```

If needed, modify the agent options using dot notation.

```
agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

For examples comparing all default agents on these environments, see “Compare Agents on Discrete Simscape Cart-Pole Swing-Up” on page 7-157 and “Compare Agents on Continuous Cart Pole Swing-Up” on page 7-251.

For another example that shows how to train a default agent for the continuous action space version of this environment, see “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

Because this is a Simulink environment, calling the `step` function is not supported. For an example on how to build a custom training loop using a Simulink environment, see “Custom Training Loop with Simulink Action Noise” on page 5-282.

See Also

Functions

`rlPredefinedEnv` | `getObservationInfo` | `getActionInfo` | `train` | `sim` | `reset`

Objects

`rlNumericSpec` | `rlFiniteSetSpec` | `SimulinkEnvWithAgent`

Blocks

RL Agent

Related Examples

- “Train Reinforcement Learning Agents” on page 5-3

More About

- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Simulink Environments” on page 2-110
- “Create Custom Environment from Class Template” on page 2-103
- “Create Custom Environment Using Step and Reset Functions” on page 2-94

Use Predefined Multiagent Environments

Reinforcement Learning Toolbox software provides two predefined environments in which two agents interact with each other to collaboratively push a larger object outside a circular boundary. You can use these environments to learn how to apply reinforcement learning to multiagent systems, to test your own agents, or as a starting point for developing your own multiagent environment.

To create your own custom multiagent environments instead, use `rlTurnBasedFunctionEnv` and `rlMultiAgentFunctionEnv`. For more information on training multiagent environments, see “Multiagent Training” on page 5-8.

The two predefined multiagent environments are derived by discretizing the dynamics of the same physical system as they rely on the same internal function to calculate the dynamics. The environments have the same inputs, outputs, internal states, and parameters, but one is a MATLAB environment and the other is a Simulink environment. When designing your own multiagent environment, you can use as a starting point the implementation that makes most sense for your application.

Environment	Implementation
Pusher	MATLAB
PusherModel	Simulink

In these environments, the state and observations belong to continuous numerical vector spaces, while the action of each agent can belong to either a continuous or finite set. Specifically, each of these predefined environments is available in two versions:

- In the discrete version both agents have a discrete action space.
- In the continuous version both agents have a continuous action space.

To create an object that implements a predefined multiagent environments, use the `rlPredefinedEnv` function.

MATLAB Pusher Environment

A MATLAB pusher environment is a predefined MATLAB environment featuring two agents, referred to as agents *A* and *B*. Each agent exerts a planar force on a two-dimensional disk that as a result can slide on the plane, according to Newton's laws of motion. The two disks guided by the agents can collide with each other and with a third disk. The goal of the agents is to push the third disk completely outside a circular boundary using minimal control effort.

This environment is equivalent to the predefined Simulink `PusherModel` environment, with some differences in the number of accessible properties, and the availability of the `plot` function. You can use the implementation that you are most comfortable with as a starting point when designing and developing your own multiagent environment.

There are two variants of the predefined MATLAB pusher environment, which differ by the agent action space:

- Discrete — The agent can apply a force which is quantized along the horizontal and vertical dimensions in five points consisting of `-MaxForce`, `-MaxForce/2`, `0`, `MaxForce/2`, and `MaxForce`.

- Continuous — The agent can apply any torque within the range $[-\text{MaxForce}, \text{MaxForce}]$.

Here, `MaxForce` is an environment property that you can change using dot notation. For more information, see “Environment Properties” on page 2-79.

You can set the `UseContinuousAction` property of this environment to establish whether each agent must have a continuous or discrete action space. For example, you can set the action spaces of the two agents differently, so that one agent has a discrete action space and the other agent has a continuous action space.

To create a predefined MATLAB pusher environment, use the `rlPredefinedEnv` function as follows, depending on the desired action space:

- Discrete action space:

```
env = rlPredefinedEnv("Pusher-Discrete")
```

```
env =
```

PusherEnvironment with properties:

```
Width: 20
Height: 20
MaxForce: 5
TaskRadius: 8
ParticleRadius: [0.2500 0.2500 0.7500]
ParticleMass: [1 1 10]
VelocityDampingCoefficient: [0.1000 0.1000 10]
ContactStiffnessCoefficient: 1000
UseContinuousAction: [0 0]
Ts: 0.0200
```

- Continuous action space:

```
env = rlPredefinedEnv("Pusher-Continuous")
```

```
env =
```

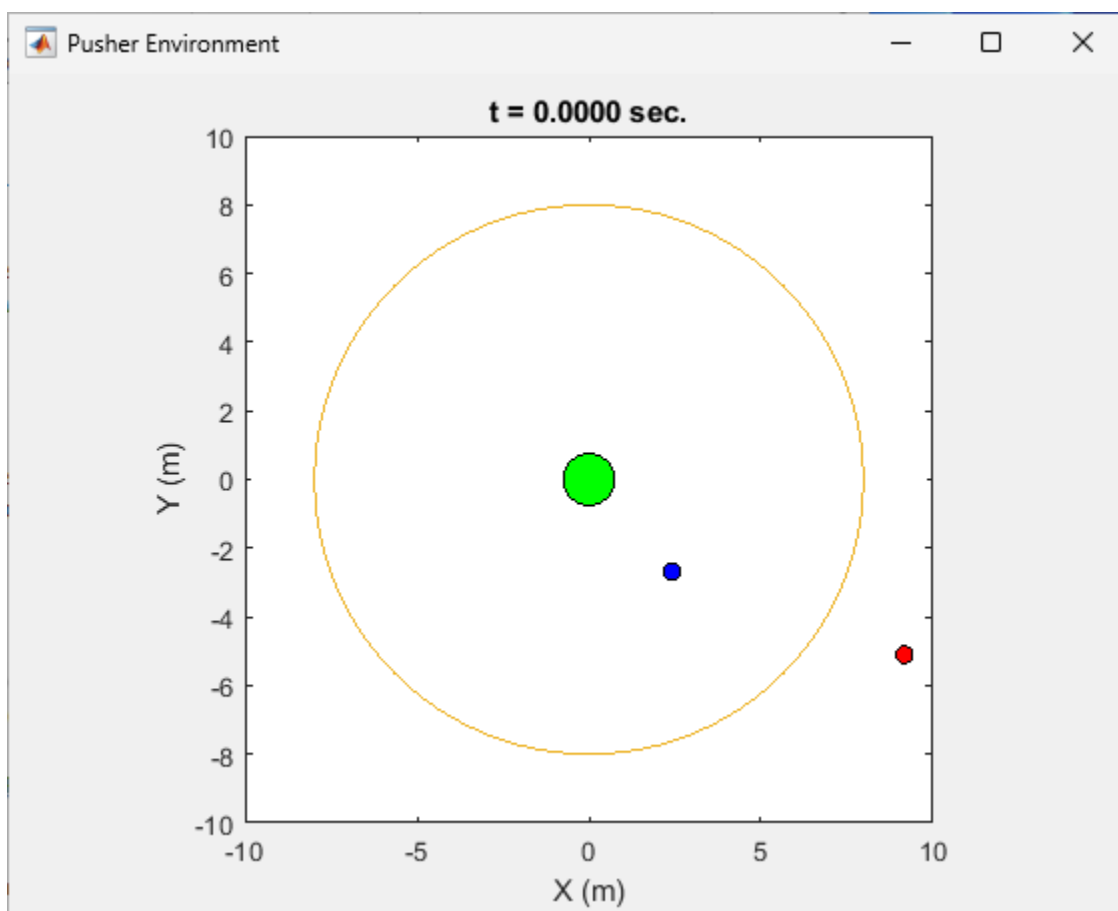
PusherEnvironment with properties:

```
Width: 20
Height: 20
MaxForce: 5
TaskRadius: 8
ParticleRadius: [0.2500 0.2500 0.7500]
ParticleMass: [1 1 10]
VelocityDampingCoefficient: [0.1000 0.1000 10]
ContactStiffnessCoefficient: 1000
UseContinuousAction: [1 1]
Ts: 0.0200
```

Environment Visualization

You can visualize the predefined MATLAB pusher environment using the `plot` function. The green circle at the origin represents disk *C*, which the agents have to move out of the task boundary, represented by the orange circle. The smaller blue and red circles represent the disks guided by agents *A* and *B*, respectively.

```
plot(env)
```



To visualize the environment during training, call `plot` before training and keep the visualization figure open.

Note Visualizing the environment during training can provide insight, but it tends to increase training time. For faster training, keep the environment plot closed during training.

Environment Properties

All the environment properties listed in this table are writable. You can change them to customize the pusher environment to your needs.

Property	Description	Default
Width	Width in meters of the rectangular region in which the disks must stay. Disks bounce off the boundary of this region when attempting to move through it.	20
Height	Height in meters of the rectangular region in the which disks move. Disks bounce off the boundary of this region when attempting to move through it.	20

Property	Description	Default
MaxForce	Maximum force in newtons that an agent can apply to its disk	5
TaskRadius	Radius in meters of the circular boundary out of which the agents have to push the third disk	8
ParticleRadius	Radii of the disks in meters, specified as $[r_A, r_B, r_C]$, where r_A and r_B are the radii of the disks guided by agents A and B , respectively, and r_C is the radii of the third disk.	[0.25 0.25 0.75]
ParticleMass	Masses of the disks in kilograms, specified as $[m_A, m_B, m_C]$, where m_A and m_B are the masses of the disks guided by agents A and B , respectively, and m_C is the mass of the third disk.	[1 1 10]
VelocityDampingCoefficient	Damping on disks motion (viscous friction), in newtons per second over meters, specified as $[d_A, d_B, d_C]$, where d_C and d_B are the damping coefficients for the disks guided by agents A and B , respectively, and d_C is the damping coefficient for the third disk.	[0.1 0.1 10]
ContactStiffnessCoefficient	Coefficient for computing contact forces (contact forces are modeled as static spring forces). For more information, see “Dynamics” on page 2-80.	1000
UseContinuousAction	Option to enable continuous action space for agents A and B , respectively, specified as a vector containing two logical values.	[0 0]
Ts	Sample time in seconds. The software uses the sample time to discretize the dynamics of the underlying continuous-time system. The discrete-time dynamics is then used to calculate the next state as a function of the current state and of the applied action.	0.05

Dynamics

The two agents A and B exert forces u_A and u_B on their respective disks. Per Newton's second law, the disks, which are subjected to viscous friction, move on a planar horizontal surface. The disks guided by the agents can collide with each other, with a third disk initially positioned at the origin, and with the boundary of the larger rectangular area within which all three disks are confined. Collisions are modeled as a mass-spring system. These equations represent the continuous-time dynamics of the environment.

$$\begin{aligned}
m_A \ddot{p}_A + d_A \dot{p}_A &= u_A + F_A(p_A, p_B, p_C) \\
m_B \ddot{p}_B + d_B \dot{p}_B &= u_B + F_B(p_A, p_B, p_C) \\
m_C \ddot{p}_C + d_C \dot{p}_C &= F_C(p_A, p_B, p_C)
\end{aligned}$$

Here, using an index i to denote subscripts A , B , and C :

- m_i are the masses of the three disks.

- p_i are the positions of the three disks on the plane. Each position has scalar components x_i (positive-right) and y_i (positive-up). Each dot above p_i denotes a derivative with respect to time.
- d_i are the damping coefficients used to calculate the viscous friction force on the three disks.
- F_i is the force exerted on the disk i as a consequence of the collision with another disk or with the rectangular boundary wall.

Specifically,

- **No collision** — When p_i is within the rectangular boundary and D_{ij} is greater than R_{ij} , then F_i is zero.
- **Collision with another disk** — when p_i is within the rectangular boundary and D_{ij} is less than R_{ij} , then F_i is equal to $-k_i$ multiplied by the transition distance $D_{ij}-R_{ij}$.
- **Collision with a wall** — when p_i is outside the rectangular boundary, then F_i is equal to $-k_i$ multiplied by the distance from the disk surface to the wall: p_i+r_i-W .

The software then uses a Runge-Kutta integration method to obtain the discrete-time dynamics of the environment.

Actions

In the predefined MATLAB pusher environment, each agent uses its own action channel. Each action channel carries a two-element vector containing the (normalized) forces applied by the agent to its disk in the x (positive-right) and y (positive-up) directions. The action space is specified as follows:

- Discrete action spaces — by default, the specification for the action channel of each agent is an `rlFiniteSetSpec` object.

```
actInfo = getActionInfo(env)

actInfo =

1x2 cell array

    {1x1 rl.util.rlFiniteSetSpec}    {1x1 rl.util.rlFiniteSetSpec}
```

If you set the `UseContinuousAction` property of the environment so that an agent uses a continuous action space, then the action channel of that agent is an `rlNumericSpec` object.

Display the specification object for the action channel of the second agent.

```
a2 = actInfo{2}

a2 =

rlFiniteSetSpec with properties:

    Elements: {9x1 cell}
        Name: "action"
    Description: "force"
    Dimension: [2 1]
    DataType: "double"
```

Display all the possible elements that the discrete action channel can carry. To obtain the actual force vector applied by the second agent to its disk, multiply each element by `MaxForce`. For

example, when `MaxForce` has the default value of 5 newtons, the second element represents a force of 5 newtons applied in the y-axis to disk *B*. By convention a positive value means the force is applied from bottom to top.

```
[a2.Elements{:}]'
```

```
ans =
```

```
      0      0
      0  1.0000
  1.0000      0
  1.0000  1.0000
  0.5000  0.5000
      0  0.5000
  0.5000      0
  1.0000  0.5000
  0.5000  1.0000
```

- Continuous action spaces — by default, the specification for the action channel of each agent is an `rlNumericSpec` object.

```
actInfo = getActionInfo(env)
```

```
actInfo =
```

```
1×2 cell array
```

```
{1×1 rl.util.rlNumericSpec} {1×1 rl.util.rlNumericSpec}
```

If you set the `UseContinuousAction` property of the environment so that an agent uses a discrete action space, then the action channel of that agent is an `rlFiniteSetSpec` object.

Display the specification object for the action channel of the first agent.

```
a1 = actInfo{1}
```

```
a1 =
```

```
rlNumericSpec with properties:
```

```
LowerLimit: -1
UpperLimit: 1
      Name: "action"
Description: "force"
  Dimension: [2 1]
    DataType: "double"
```

For more information on obtaining action specifications from an environment, see `getActionInfo`.

Observations

In the predefined MATLAB pusher environment, each agent receives these observations:

- Position and velocity of its disk
- Position and velocity of disk *C*
- A flag indicating whether its disk is undergoing a collision with disk *C*

The observations are stacked in a single observation channel for each agent, and the channel is specified by an `rlNumericSpec` object.

```
obsInfo = getObservationInfo(env)
```

```
1x2 cell array
```

```
{1x1 rl.util.rlNumericSpec} {1x1 rl.util.rlNumericSpec}
```

Display the specification object for the observation channel of the first agent.

```
o1 = obsInfo{1}
```

```
o1 =
```

```
rlNumericSpec with properties:
```

```
LowerLimit: -Inf
```

```
UpperLimit: Inf
```

```
Name: "observation"
```

```
Description: "self and object position and velocity, collision flag"
```

```
Dimension: [9 1]
```

```
DataType: "double"
```

For more information on obtaining observation specifications from an environment, see `getObservationInfo`.

Rewards

The reward signal that this environment gives to the agents is composed of several parts:

- Reward to both agents for solving the task. The task is solved when disk *C* is completely outside the origin-centered circle or radius `TaskRadius`.
- Reward to both agents for colliding with disk *C*.
- Reward to both agents proportional to the squared distance of *C* from the origin.
- Penalty to each agent proportional to its individual squared distance from *C*.
- Constant penalty to both agents at each time step.

Specifically, at time *t*, the reward signals for the two agents are:

$$r_a = 10 * C_A + 0.05 * p_C^T p_C - 0.01 * (p_A^A (p_A - p_C) - 0.1 + 100 * S$$

$$r_b = 10 * C_B + 0.05 * p_C^T p_C - 0.01 * (p_B^B (p_B - p_C) - 0.1 + 100 * S$$

Here:

- C_A and C_B are flags indicating (when their value is 1) that the agent *A* and *B*, respectively, are undergoing a collision with disk *C*.
- S is a flag indicating (when its value is 1) when the task is solved.

Reset Function

The reset function sets the initial state of the environment so that disk *C* is in the origin, while the agents disks *A* and *B* are in a random position inside the rectangular boundary.

Reset the environment and display the initial observations for the two agents.

```
obs0 = reset(env);
[obs0{1} obs0{2}]

ans =

    2.4486   -4.0978
   -7.4455    0.8673
         0         0
         0         0
         0         0
         0         0
         0         0
         0         0
```

Create Default Agents for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the version of this environment in which both agents have a continuous action space, create default TD3 and SAC agents.

```
agentA = rlTD3Agent(obsInfo{1},actInfo{1})
agentB = rlSACAgent(obsInfo{2},actInfo{2})

agentA =

    rlTD3Agent with properties:

        ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]
        AgentOptions: [1x1 rl.option.rlTD3AgentOptions]
        UseExplorationPolicy: 0
        ObservationInfo: [1x1 rl.util.rlNumericSpec]
        ActionInfo: [1x1 rl.util.rlNumericSpec]
        SampleTime: 1

agentB =

    rlSACAgent with properties:

        ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]
        AgentOptions: [1x1 rl.option.rlSACAgentOptions]
        UseExplorationPolicy: 1
        ObservationInfo: [1x1 rl.util.rlNumericSpec]
        ActionInfo: [1x1 rl.util.rlNumericSpec]
        SampleTime: 1
```

If needed, modify the agent options using dot notation.

```
agentA.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
agentA.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
agentA.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agentB.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
agentB.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
agentB.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

You can also call the environment step function to return the next observation, reward, and an `is-done` scalar indicating whether a final state has been reached.

For example, call the step function applying the maximum force on the second disk pointing to the left and down, respectively.

```
[xn,rn,id] = step(env,{[0 0]',[-1 -1]'})
```

```
xn =
```

```
1x2 cell array
```

```
{9x1 double} {9x1 double}
```

```
rn =
```

```
-0.7143 -0.2755
```

```
id =
```

```
logical
```

```
0
```

The environment `step` and `reset` functions allow you to create a custom training or simulation loop. For more information on custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Environment Code

To access the functions that returns this environment, at the MATLAB command line, type:

```
edit rl.env.pusher.PusherEnvironment
```

To access the function that calculates the dynamics, at the MATLAB command line, type:

```
edit rl.env.pusher.pusherDynamics
```

Simulink Pusher Environment

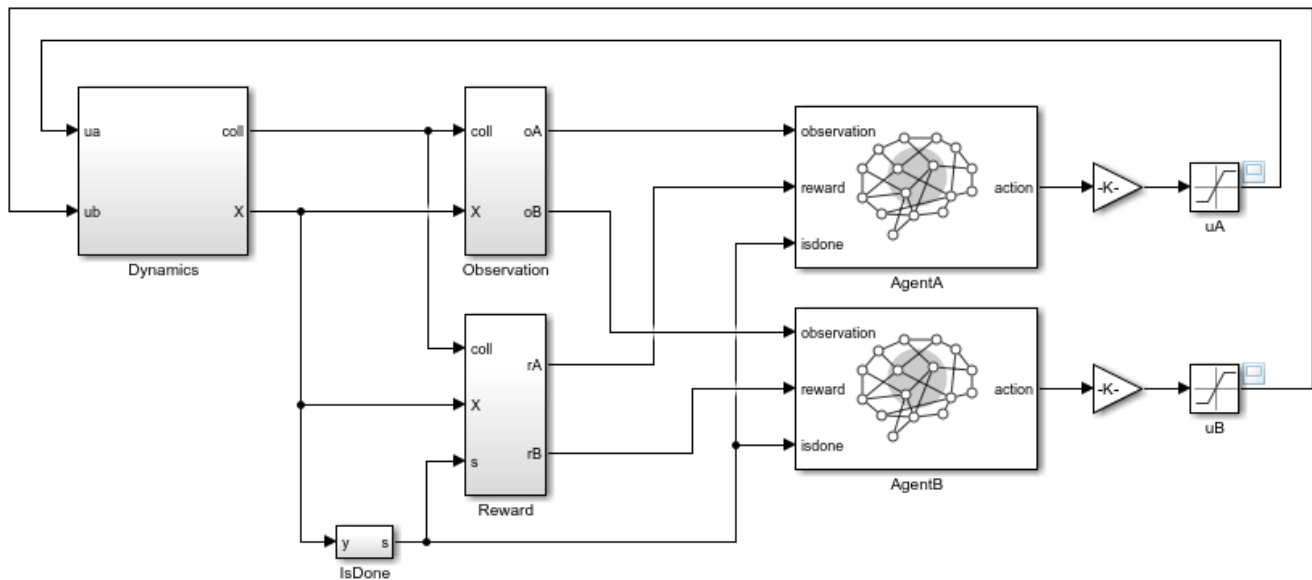
A Simulink pusher environment is a predefined Simulink environment featuring two agents, referred to as agents *A* and *B*. Each agent exerts a planar force on a two-dimensional disk that as a result can slide on the plane, according to Newton's laws of motion. The two disks guided by the agents can

collide with each other and with a third disk. The goal of the agents is to push the third disk completely outside a circular boundary using minimal control effort.

This environment is equivalent to the predefined MATLAB pusher environment, with some differences in the number of accessible properties, the used solver, and the management of the `plot` function. You can use the implementation that you are most comfortable with as a starting point when designing and developing your own multiagent environment.

The model for this environment is defined in the `rlPusherModel` Simulink model.

```
open_system("rlPusherModel")
```



There are two variants of the predefined Simulink pusher environment, which differ by the agent action space.

- Discrete — The agent can apply a force which is quantized along the horizontal and vertical dimensions in five points consisting of $-\text{MaxForce}$, $-\text{MaxForce}/2$, 0 , $\text{MaxForce}/2$, and MaxForce .
- Continuous — The agent can apply any torque within the range $[-\text{MaxForce}, \text{MaxForce}]$.

Here, `MaxForce` is an environment property that you can change by accessing the workspace of the `rlPusherModel` model. For more information on other variables that you can change, see “Dynamics” on page 2-88. For more information on Simulink model workspaces, see “Model Workspaces” (Simulink) and “Change Model Workspace Data” (Simulink).

To create a predefined Simulink pusher environment, use the `rlPredefinedEnv` function as follows, depending on the desired action space:

- Discrete action space


```
env = rlPredefinedEnv("PusherModel-Discrete")
env =
```

SimulinkEnvWithAgent with properties:

```

        Model : rlPusherModel
    AgentBlock : [
                    rlPusherModel/AgentA
                    rlPusherModel/AgentB
                ]
        ResetFcn : []
    UseFastRestart : on

```

- Continuous action space

```
env = rlPredefinedEnv("PusherModel-Continuous")
```

```
env =
```

SimulinkEnvWithAgent with properties:

```

        Model : rlPusherModel
    AgentBlock : [
                    rlPusherModel/AgentA
                    rlPusherModel/AgentB
                ]
        ResetFcn : []
    UseFastRestart : on

```

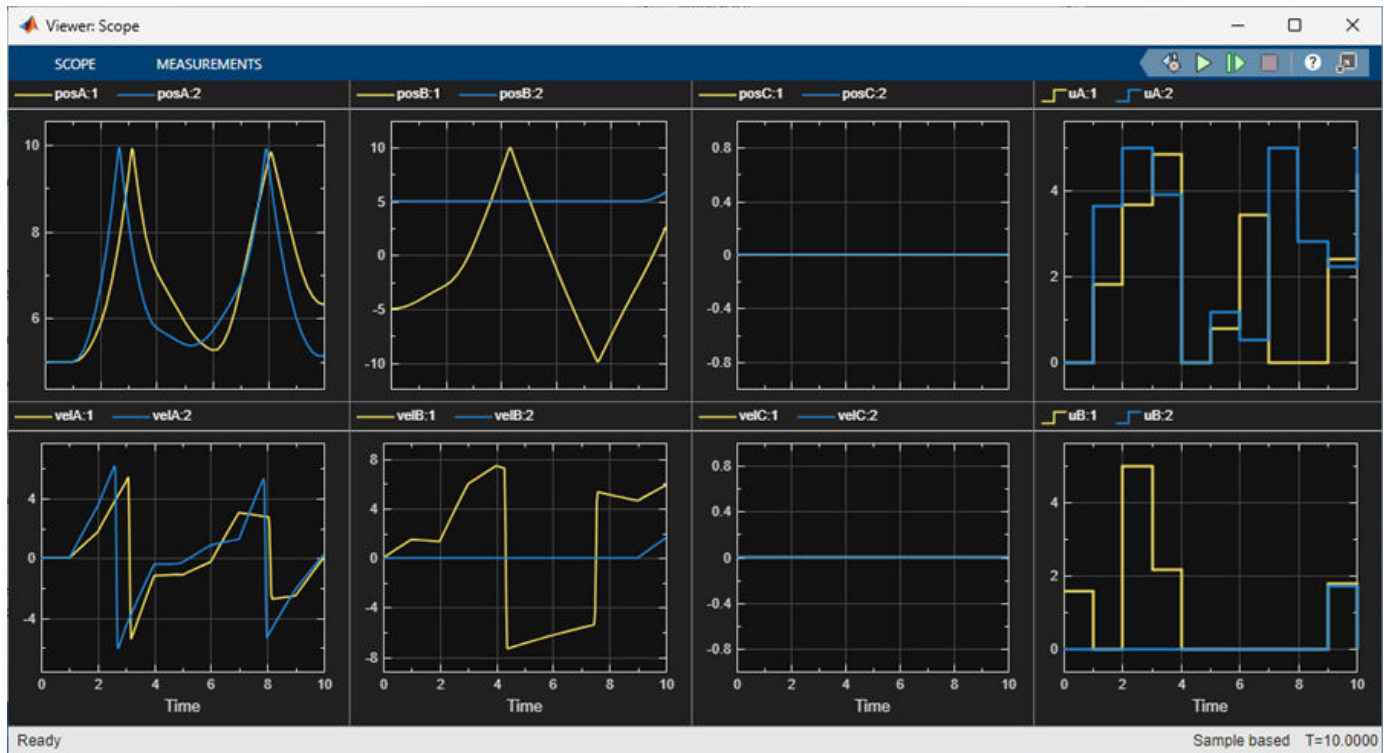
Note When training or simulating an agent within a Simulink environment, to ensure that the RL Agent block executes at the desired sample time, set the `SampleTime` property of the agent object appropriately.

For more information on Simulink environments, see `SimulinkEnvWithAgent` and “Create Custom Simulink Environments” on page 2-110.

Environment Visualization

Unlike the MATLAB pusher environment, for the Simulink pusher environment you do not have access to a `plot` function. However, the same plot described in the **Environment Visualization** section of the “MATLAB Pusher Environment” on page 2-77, is displayed automatically by the Simulink model when it is executed by the simulation or training function.

An additional visualization, from a Simulink Floating Scope and Scope Viewer block, automatically displays when the model is executed. This visualization shows how the positions and velocities of the three disks, and the forces applied by the agents, evolve during the current episode.



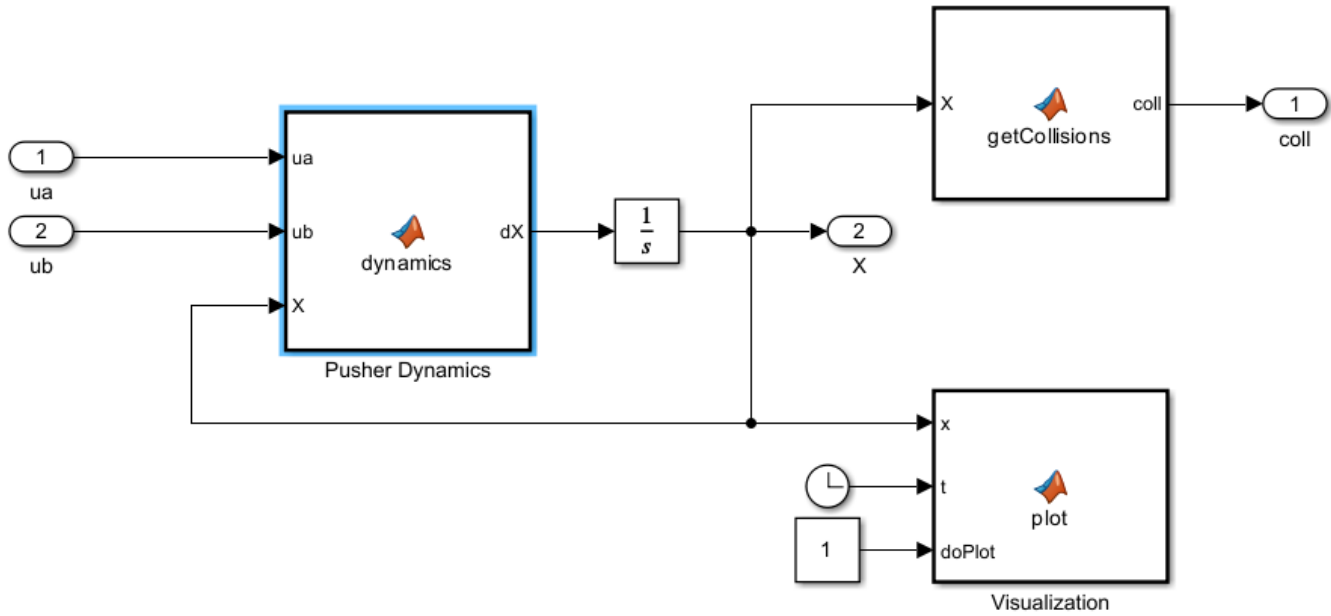
Environment Properties

The environment properties that you can access using dot notation are described in `SimulinkEnvWithAgent`.

Dynamics

The continuous-time environment dynamics are described in the **Pusher Environment Dynamics** section of “MATLAB Pusher Environment” on page 2-77. During simulation, the Simulink solver integrates the dynamics according to the selected solver, while the RL Agent block executes at discrete intervals according to the sample time specified in the `SampleTime` property of the agent.

The Dynamics subsystem of the `r1PusherModel` model relies on three MATLAB Function (Simulink) blocks to call the same code used for the “MATLAB Pusher Environment” on page 2-77 environment to calculate the dynamics and the collision flags and to visualize the disks on the plane.



The parameters that define the system dynamics are stored in the model workspace.

Variable	Description	Default
ContactStiffnessCoefficient	Coefficient for computing contact forces (contact forces are modeled as static spring forces). For more information, see “Dynamics” on page 2-80.	1000
Height	Height in meters of the rectangular region in which disks move. Disks bounce off the boundary of this region when attempting to move through it.	20
MaxForce	Maximum force in newtons that an agent can apply to its disk	5
ParticleMass	Masses of the disks in kilograms, specified as $[m_A, m_B, m_C]$, and m_A and m_B are the masses of the disks guided by agents <i>A</i> and <i>B</i> , respectively, and m_C is the mass of the third disk.	[1 1 10]
ParticleRadius	Radii of the disks in meters, specified as $[r_A, r_B, r_C]$ where r_A and r_B are the radii of the disks guided by agents <i>A</i> and <i>B</i> , respectively, and r_C is the radii of the third disk.	[0.25 0.25 0.75]
TaskRadius	Radius of the circular boundary in meters	8

Variable	Description	Default
VelocityDampingCoefficient	Damping on disks motion (viscous friction) in newtons per second over meters, specified as $[d_A, d_B, d_C]$, where d_C and d_B are the damping coefficients for the masses guided by agents A and B, respectively, and d_C is the damping coefficient for the disk that the agents have to move out of the task circle.	[0.1 0.1 10]
Width	Width in meters of the rectangular region in which disks must stay. Disks bounce off the boundary of this region when attempting to move through it.	20
X0	Initial value of the environment state (positions and velocities of the three disks)	[5 5 -5 5 0 0 0 0 0 0 0 0]'

For more information on Simulink model workspaces, see “Model Workspaces” (Simulink) and “Change Model Workspace Data” (Simulink).

Actions

Like the predefined MATLAB pusher environment, each agent uses its own action channel, which carries a two-element vector containing the (normalized) forces applied by the agent to its disk in the x (positive-right) and y (positive-up) directions.

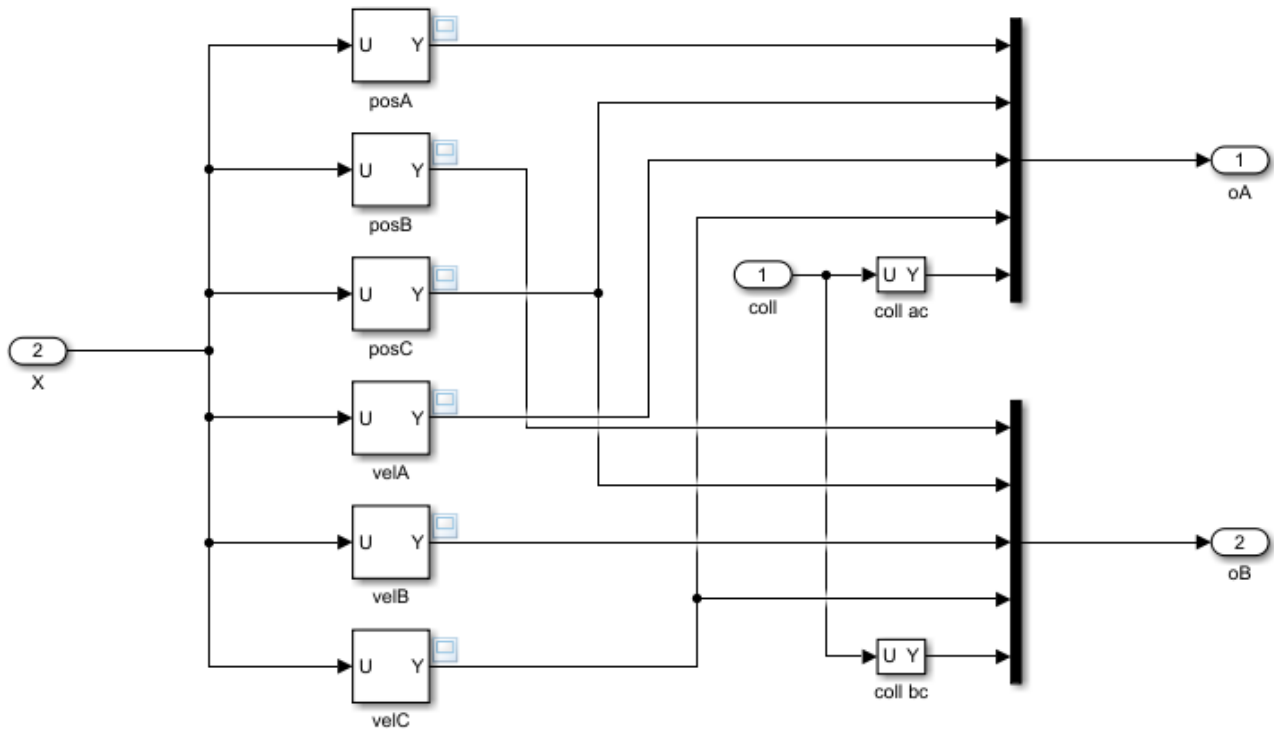
For the detailed description of the action spaces for the discrete and continuous versions of this environment, see the **Actions** section of “MATLAB Pusher Environment” on page 2-77.

Unlike the MATLAB pusher environment, the action space of the agent is fixed. Both agents either have a discrete action space in the discrete environment or they have a continuous action space in the continuous environment. You cannot set the type of an agent action space after you create the environment.

Observations

As for the predefined MATLAB pusher environment, each agent has its own single observation channel, which carries a nine-element vector containing the position and velocity of its disk, the position and velocity of disk C, and a flag indicating whether a collision with disk C is occurring.

In this environment, the observation signal for each agent is calculated within the Observation subsystem of the `rlPusherModel` model.

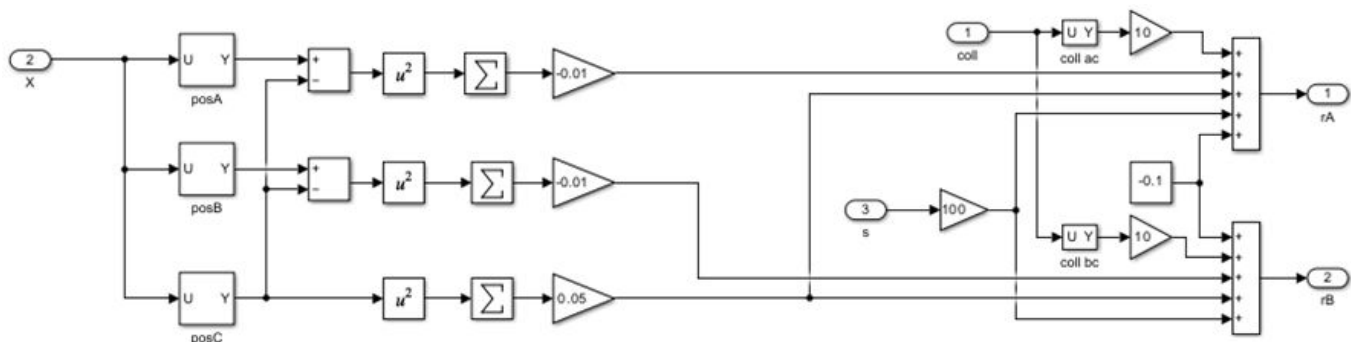


For the detailed description of the agents observation spaces, see the **Observations** section of “MATLAB Pusher Environment” on page 2-77.

Reward

The reward signal for each agent is described in the **Rewards** section of “MATLAB Pusher Environment” on page 2-77.

In this environment, the reward signal for each agent is calculated within the Reward subsystem of the `rlPusherModel` model.



Reset Function

The environment has no default `reset` function. However, you can set your own `reset` function to set the initial state of the system every time a simulation is started to train or simulate the agents.

For example, set the initial state so that the agent disks stand still near the upper corners of the default allowed rectangle.

```
env.ResetFcn = @(in)setVariable(in, ...  
    "X0",[7 7 -7 7 zeros(1,8)]', ...  
    "Workspace","rlPusherModel");
```

Here, you use `setVariable` to set the variable `X0` to `[7 7 -7 7 zeros(1,8)]'` in the `Simulink.SimulationInput` object `in`. The value of `X0` that you specify overrides the existing `X0` value in the model workspace for the duration of the simulation or training. The value of `X0` then automatically reverts to its original value when the simulation or training completes.

Create Default Agents for this Environment

The environment observation and action specifications allow you to create an agent that works with your environment. For example, for the version of this environment in which both agents have a continuous action space, create default PPO and SAC agents.

```
agentA = rlPP0Agent(obsInfo{1},actInfo{1})  
agentB = rlSACAgent(obsInfo{2},actInfo{2})
```

```
agentA =
```

```
    rlPP0Agent with properties:
```

```
        AgentOptions: [1x1 rl.option.rlPP0AgentOptions]  
    UseExplorationPolicy: 1  
        ObservationInfo: [1x1 rl.util.rlNumericSpec]  
            ActionInfo: [1x1 rl.util.rlFiniteSetSpec]  
            SampleTime: 1
```

```
agentB =
```

```
    rlSACAgent with properties:
```

```
        ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]  
        AgentOptions: [1x1 rl.option.rlSACAgentOptions]  
    UseExplorationPolicy: 1  
        ObservationInfo: [1x1 rl.util.rlNumericSpec]  
            ActionInfo: [1x1 rl.util.rlFiniteSetSpec]  
            SampleTime: 1
```

Set the agents sample time so that the RL Agent blocks execute at the desired rate.

```
agentA.SampleTime = 0.05;  
agentB.SampleTime = 0.05;
```

If needed, modify the agent options using dot notation.

```
agentA.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
agentA.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
agentA.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
agentB.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
agentB.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
agentB.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
```

You can now use both the environment and the agent as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

You can also create and train agents for this environment interactively using the Reinforcement Learning Designer app. For an example, see “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2.

Step Function

Because this is a Simulink environment, calling the `step` function is not supported. For an example on how to build a custom training loop using a Simulink environment, see “Custom Training Loop with Simulink Action Noise” on page 5-282.

See Also

Functions

`rlPredefinedEnv` | `getObservationInfo` | `getActionInfo` | `train` | `sim` | `reset`

Objects

`rlNumericSpec` | `rlFiniteSetSpec` | `SimulinkEnvWithAgent`

Blocks

RL Agent

Related Examples

- “Train Reinforcement Learning Agents” on page 5-3
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31
- “Train Multiple Agents for Area Coverage” on page 8-44
- “Train Multiple Agents for Path Following Control” on page 8-57

More About

- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Simulink Environments” on page 2-110
- “Create Custom Environment from Class Template” on page 2-103
- “Create Custom Environment Using Step and Reset Functions” on page 2-94

Create Custom Environment Using Step and Reset Functions

This example shows how to create a custom environment by writing your own MATLAB® `step` and `reset` functions.

Using the `rlFunctionEnv` function, you can create a MATLAB reinforcement learning environment from an observation specification, an action specification, and `step` and `reset` functions that you supply. You can then train a reinforcement learning agent in this environment. For this example, the necessary `step` and `reset` functions are already defined.

Creating an environment using custom functions is especially useful for simpler environments that do not need many helper functions and have no special visualization requirements. For more complex environments, you can create an environment object using a template class. For more information, see “Create Custom Environment from Class Template” on page 2-103.

For more information on creating reinforcement learning environments, see “Reinforcement Learning Environments” on page 2-2 and “Create Custom Simulink Environments” on page 2-110.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

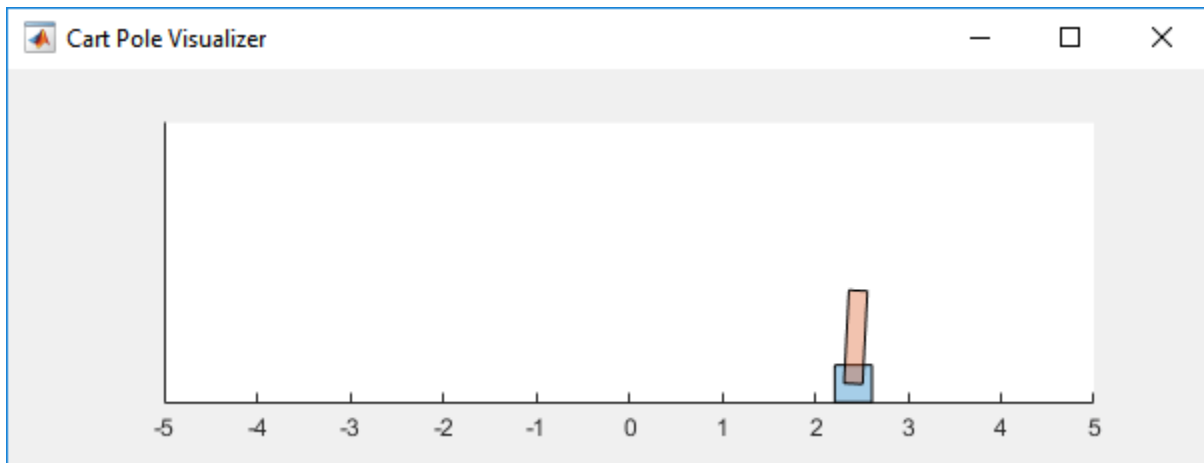
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Cart-Pole MATLAB Environment

The cart-pole environment is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to make the pendulum stand upright.



For this environment:

- The balanced, upright pendulum position is zero radians and the downward hanging pendulum position is π radians.
- The pendulum starts upright with an initial angle that is between -0.05 radians and 0.05 radians.
- The force action signal from the agent to the environment is either -10 N or 10 N.
- The observations from the environment are the cart position, cart velocity, pendulum angle, and pendulum angular velocity.
- The episode terminates if the pole is more than 12 degrees from vertical or the cart moves more than 2.4 m from the original position.
- A reward of $+1$ is provided for every time step that the pole remains upright. A penalty of -10 is applied when the pendulum falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

For this example, instead of loading the predefined cart-pole environment using `rlPredefinedEnv`, you implement a basic version of this environment by supplying your `step` and `reset` functions.

Observation and Action Specifications

The observations from the environment are the cart position, cart velocity, pendulum angle, and pendulum angle derivative.

```
ObsInfo = rlNumericSpec([4 1]);
ObsInfo.Name = "CartPole States";
ObsInfo.Description = 'x, dx, theta, dtheta';
```

The environment has a discrete action space where the agent can apply one of two possible force values to the cart: -10 or 10 N.

```
ActInfo = rlFiniteSetSpec([-10 10]);
ActInfo.Name = "CartPole Action";
```

For more information on specifying environment actions and observations, see `rlNumericSpec` and `rlFiniteSetSpec`.

Define Environment Reset and Step Functions

To define a custom environment, first specify the custom `step` and `reset` functions. These functions must be in your current working folder or on the MATLAB path.

The `reset` function sets the initial state of the environment. This function must have the following signature.

```
[InitialObservation,Info] = myResetFunction()
```

The first output argument is the initial observation. The second output argument can be any useful environment information that you want to pass from one step to the next, such as for example, the environment state, or a structure containing state and parameters.

At the beginning of the training (or simulation) episode, `train` (or `sim`) calls your `reset` function and uses its second output argument to initialize the `Info` property of your custom environment. During a training (or simulation) step, `train` (or `sim`) supplies the current value of `Info` as the second input

argument of `StepFcn`, and then uses the fourth output argument returned by `StepFcn` to update the value of `Info`.

For this example, use the second argument to store the initial states of the cart-pole environment: the position and velocity of the cart, the pendulum angle (clockwise-positive), and the pendulum angle derivative. The `reset` function sets the cart angle to a random value each time the environment is reset.

For this example, use the custom reset function defined in `myResetFunction.m`.

type `myResetFunction.m`

```
function [InitialObservation, InitialState] = myResetFunction()
% Reset function to place custom cart-pole environment into a random
% initial state.

% Theta (randomize)
T0 = 2 * 0.05 * rand() - 0.05;
% Thetadot
Td0 = 0;
% X
X0 = 0;
% Xdot
Xd0 = 0;

% Return initial environment state variables as logged signals.
InitialState = [X0;Xd0;T0;Td0];
InitialObservation = InitialState;

end
```

The `step` function specifies how the environment advances to the next state based on a given action. This function must have the following signature.

```
[NextObservation,Reward,IsDone,UpdatedInfo] = myStepFunction(Action,Info)
```

To calculate the new state, the step function applies the dynamic equation to the current state stored in `Info`. The function then returns the updated state in `UpdatedInfo`. At the next training (or simulation) step, `train` (or `sim`) takes the fourth output argument obtained during the previous step, `UpdatedInfo`, and supplies it to the step function as the second input argument, `Info`. Note that `Action` affects only `NextObservation` and the associated `Reward`, but does not affect the current observation. In other words, there is no *direct feedthrough* between action and observation.

For this example, use the custom step function defined in `myStepFunction.m`. For implementation simplicity, this function redefines physical constants, such as the cart mass, every time `step` is executed. An alternative is to define the physical constants in the reset function, define `Info` as a structure containing both state and parameters, and therefore use `Info` to store the physical constants as well as the environment states. This alternative implementation lets you easily change some of the parameters during the simulation or training if you need to.

type `myStepFunction.m`

```
function [NextObs,Reward,IsDone,NextState] = myStepFunction(Action,State)
% Custom step function to construct cart-pole environment for the function
% name case.
%
% This function applies the given action to the environment and evaluates
```

```

% the system dynamics for one simulation step.

% Define the environment constants.

% Acceleration due to gravity in m/s^2
Gravity = 9.8;
% Mass of the cart
CartMass = 1.0;
% Mass of the pole
PoleMass = 0.1;
% Half the length of the pole
HalfPoleLength = 0.5;
% Max force the input can apply
MaxForce = 10;
% Sample time
Ts = 0.02;
% Pole angle at which to fail the episode
AngleThreshold = 12 * pi/180;
% Cart distance at which to fail the episode
DisplacementThreshold = 2.4;
% Reward each time step the cart-pole is balanced
RewardForNotFalling = 1;
% Penalty when the cart-pole fails to balance
PenaltyForFalling = -10;

% Check if the given action is valid.
if ~ismember(Action,[-MaxForce MaxForce])
    error('Action must be %g for going left and %g for going right.',...
        -MaxForce,MaxForce);
end
Force = Action;

% Unpack the state vector from the logged signals.
XDot = State(2);
Theta = State(3);
ThetaDot = State(4);

% Cache to avoid recomputation.
CosTheta = cos(Theta);
SinTheta = sin(Theta);
SystemMass = CartMass + PoleMass;
temp = (Force + PoleMass*HalfPoleLength*ThetaDot*ThetaDot*SINTheta)/SystemMass;

% Apply motion equations.
ThetaDotDot = (Gravity*SINTheta - CosTheta*temp) / ...
    (HalfPoleLength*(4.0/3.0 - PoleMass*COSTheta*COSTheta/SystemMass));
XDotDot = temp - PoleMass*HalfPoleLength*ThetaDotDot*COSTheta/SystemMass;

% Perform Euler integration to calculate next state.
NextState = State + Ts.*[XDot;XDotDot;ThetaDot;ThetaDotDot];

% Copy next state to next observation.
NextObs = NextState;

% Check terminal condition.
X = NextObs(1);
Theta = NextObs(3);
IsDone = abs(X) > DisplacementThreshold || abs(Theta) > AngleThreshold;

```

```
% Calculate reward.
if ~IsDone
    Reward = RewardForNotFalling;
else
    Reward = PenaltyForFalling;
end

end
```

Use `rlFunctionEnv` to create a custom environment object using the observation and action specification, and the names of your `step` and `reset` functions.

```
env = rlFunctionEnv(ObsInfo,ActInfo,"myStepFunction","myResetFunction");
```

To verify the operation of your environment, `rlFunctionEnv` automatically calls `validateEnvironment` after creating the environment.

Pass Additional Arguments Using Anonymous Functions

While the custom reset and step functions that you must pass to `rlFunctionEnv` must have exactly zero and two arguments, respectively, you can partially avoid this limitation by using anonymous functions. Specifically, you define the `reset` and `step` functions to be passed to `rlFunctionEnv` as anonymous functions (with zero and two arguments, respectively). In turn, these anonymous functions, call your custom functions that have additional arguments.

For example, to pass the additional arguments `arg1` and `arg2` to both the `step` and `reset` function, you can write the following functions.

```
[InitialObservation,Info] = myResetFunction(arg1,arg2)
[Observation,Reward,IsDone,Info] = myStepFunction(Action,State,arg1,arg2)
```

Then, with `arg1` and `arg2` in the MATLAB workspace, define the following handles to anonymous reset and step functions with zero and two arguments, respectively.

```
ResetHandle = @() myResetFunction(arg1,arg2);
StepHandle = @(Action,Info) myStepFunction(Action,State,arg1,arg2);
```

If `arg1` and `arg2` are available at the time that `ResetHandle` and `StepHandle` are created, the workspaces of both anonymous functions include those values. The values persist within the function workspaces even if you clear the variables from the MATLAB workspace. When `ResetHandle` and `StepHandle` are evaluated, they invoke `myResetFunction` and `myStepFunction` and pass them a copy of the values that `arg1` and `arg2` had when the reset function was defined. For more information, see “Anonymous Functions”.

Using additional input arguments can create a more efficient environment implementation. For example, `myStepFunction2.m` is a custom step function that takes the environment constants as its third input argument (`envPars`). By doing so, this function avoids redefining the environment constants at each step.

```
type myStepFunction2.m
```

```
function [NextObs,Reward,IsDone,NextState] = myStepFunction2(Action,State,EnvPars)
% Custom step function to construct cart-pole environment for the function
% handle case.
%
% This function applies the given action to the environment and evaluates
```



```

% the system dynamics for one simulation step.

% Check if the given action is valid.
if ~ismember(Action,[-EnvPars.MaxForce EnvPars.MaxForce])
    error('Action must be %g for going left and %g for going right.',...
        -EnvPars.MaxForce,EnvPars.MaxForce);
end
Force = Action;

% Unpack the state vector from the logged signals.
XDot = State(2);
Theta = State(3);
ThetaDot = State(4);

% Cache to avoid recomputation.
CosTheta = cos(Theta);
SinTheta = sin(Theta);
MassPole = EnvPars.MassPole;
HalfLen = EnvPars.HalfLength;
SystemMass = EnvPars.MassCart + MassPole;
temp = (Force + MassPole*HalfLen*ThetaDot*ThetaDot*SinTheta)/SystemMass;

% Apply motion equations.
ThetaDotDot = (EnvPars.Gravity*SinTheta - CosTheta*temp)...
    / (HalfLen*(4.0/3.0 - MassPole*CosTheta*CosTheta/SystemMass));
XDotDot = temp - MassPole*HalfLen*ThetaDotDot*CosTheta/SystemMass;

% Perform Euler integration.
NextState = State + EnvPars.Ts.*[XDot;XDotDot;ThetaDot;ThetaDotDot];

% Copy next state to next observation.
NextObs = NextState;

% Check terminal condition.
X = NextObs(1);
Theta = NextObs(3);
IsDone = abs(X) > EnvPars.XThreshold || ...
    abs(Theta) > EnvPars.ThetaThresholdRadians;

% Calculate reward.
if ~IsDone
    Reward = EnvPars.RewardForNotFalling;
else
    Reward = EnvPars.PenaltyForFalling;
end
end
end

```

Create the structure that contains the environment parameters.

```

% Acceleration due to gravity in m/s^2
envPars.Gravity = 9.8;
% Mass of the cart
envPars.MassCart = 1.0;
% Mass of the pole
envPars.MassPole = 0.1;
% Half the length of the pole
envPars.HalfLength = 0.5;

```

```
% Max force the input can apply
envPars.MaxForce = 10;
% Sample time
envPars.Ts = 0.02;
% Angle at which to fail the episode
envPars.ThetaThresholdRadians = 12 * pi/180;
% Distance at which to fail the episode
envPars.XThreshold = 2.4;
% Reward each time step the cart-pole is balanced
envPars.RewardForNotFalling = 1;
% Penalty when the cart-pole fails to balance
envPars.PenaltyForFalling = -5;
```

Create an anonymous function that calls your custom step function, passing `envPars` as an additional input argument.

```
StepHandle = @(Action,Info) myStepFunction2(Action,Info,envPars);
```

Because `envPars` is available at the time that `StepHandle` is created, the anonymous function workspace includes a copy of `envPars`. When `StepHandle` is evaluated, it calls `myStepFunction2` passing its copy of `envPars`.

Use the same `reset` function, specifying it as a function handle rather than by using its name.

```
ResetHandle = @() myResetFunction;
```

Create the environment using the handles to the anonymous functions.

```
env2 = rlFunctionEnv(ObsInfo,ActInfo,StepHandle,ResetHandle);
```

Visually Inspect Outputs of Custom Functions

While `rlFunctionEnv` automatically calls `validateEnvironment` after creating the environment, it might be useful to visually inspect the output of your functions to further confirm that their behavior conforms to your expectations. To do so, initialize your environment using the `reset` function and run one simulation step using the `step` function. For reproducibility, set the random generator seed before validation.

Validate the environment created using function names. Fix the random number stream so that the `reset` function always returns the same initial observation.

```
rng(0,"twister");
InitialObs = reset(env)
```

```
InitialObs = 4×1
```

```
    0
    0
0.0315
    0
```

```
[NextObs,Reward,IsDone,Info] = step(env,10);
NextObs
```

```
NextObs = 4×1
```

```
    0
```

```

0.1947
0.0315
-0.2826

```

Validate the environment created using function handles. Fix the random number stream so that the reset function always returns the same initial observation.

```

rng(0,"twister");
InitialObs2 = reset(env2)

```

```

InitialObs2 = 4×1

```

```

    0
    0
0.0315
    0

```

```

[NextObs2,Reward2,IsDone2,Info2] = step(env2,10);
NextObs2

```

```

NextObs2 = 4×1

```

```

    0
0.1947
0.0315
-0.2826

```

Both environments initialize and simulate successfully, producing the same state values in `NextObs`.

Visualize the Environment in Reset and Step Functions

You can visualize the environment status at each step during training or simulation by plotting some environment features within your custom step and reset functions.

For example, you can plot the environment in its initial state in the reset function, and use the `Info` output argument to pass along the relevant line and axes objects that need to be updated as the environment state evolves. Then, in your step function, you can extract and update the line and axes objects (if they are still valid) according to the current environment state.

Alternatively, both the reset and step function can call a single, more complex, plotting function that takes the whole environment as input argument. The first time this function is called, (by your reset function), it plots the initial state of the environment and saves the relevant line and axis objects as persistent variables in its workspace. When the plotting function is subsequently called by your step function, it retrieves the axis and line objects from its workspace, and updates them according to the current environment state. For an example on how to do this, see “Train Agent to Play Turn-Based Game” on page 8-348.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlFiniteSetSpec` | `rlNumericSpec`

Objects

`rlFunctionEnv`

More About

- “Anonymous Functions”
- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Environment from Class Template” on page 2-103

Create Custom Environment from Class Template

You can define a custom reinforcement learning environment by creating and modifying a template environment class. You can use a custom template environment to:

- Implement more complex environment dynamics.
- Add custom visualizations to your environment.
- Create an interface to third-party libraries defined in languages such as C++, Java®, or Python®. For more information, see “External Language Interfaces”.

For more information about creating MATLAB classes, see “User-Defined Classes”.

You can create less complex custom reinforcement learning environments using custom functions, as described in “Create Custom Environment Using Step and Reset Functions” on page 2-94.

Create Template Class

To define your custom environment, first create the template class file, specifying the name of the class. For this example, name the class `MyEnvironment`.

```
rlCreateEnvTemplate("MyEnvironment")
```

The function `rlCreateEnvTemplate` creates and opens the template class file. The template class is a subclass of the `rl.env.MATLABEnvironment` abstract class, as shown in the class definition at the start of the template file. This abstract class is the same one used by the other MATLAB reinforcement learning environment objects.

```
classdef MyEnvironment < rl.env.MATLABEnvironment
```

By default, the template class implements a simple cart-pole balancing model similar to the cart-pole predefined environments described in “Use Predefined Control System Environments” on page 2-42.

To define your environment dynamics, save the file as `MyEnvironment.m`. Then modify the template class by specifying the following:

- Environment properties
- Required environment methods
- Optional environment methods

Environment Properties

In the `properties` section of the template, specify any parameters necessary for creating and simulating the environment. These parameters can include:

- Physical constants — The sample environment defines the acceleration due to gravity (`Gravity`).
- Environment geometry — The sample environment defines the cart and pole masses (`CartMass` and `PoleMass`) and the half-length of the pole (`HalfPoleLength`).
- Environment constraints — The sample environment defines the pole angle and cart distance thresholds (`AngleThreshold` and `DisplacementThreshold`). The environment uses these values to detect when a training episode is finished.

- Variables required for evaluating the environment — The sample environment defines the state vector (`State`) and a flag for indicating when an episode is finished (`IsDone`).
- Constants for defining the actions or observation spaces — The sample environment defines the maximum force for the action space (`MaxForce`).
- Constants for calculating the reward signal — The sample environment defines the constants `RewardForNotFalling` and `PenaltyForFalling`.

```

properties
    % Specify and initialize the necessary properties of the environment
    % Acceleration due to gravity in m/s^2
    Gravity = 9.8

    % Mass of the cart
    CartMass = 1.0

    % Mass of the pole
    PoleMass = 0.1

    % Half the length of the pole
    HalfPoleLength = 0.5

    % Max force the input can apply
    MaxForce = 10

    % Sample time
    Ts = 0.02

    % Angle at which to fail the episode (radians)
    AngleThreshold = 12 * pi/180

    % Distance at which to fail the episode
    DisplacementThreshold = 2.4

    % Reward each time step the cart-pole is balanced
    RewardForNotFalling = 1

    % Penalty when the cart-pole fails to balance
    PenaltyForFalling = -10
end

properties
    % Initialize system state [x,dx,theta,dtheta]'
    State = zeros(4,1)
end

properties(Access = protected)
    % Initialize internal flag to indicate episode termination
    IsDone = false
end

```

Required Functions

A reinforcement learning environment requires the following functions to be defined. The `getObservationInfo`, `getActionInfo`, `sim`, and `validateEnvironment` functions are already defined in the base abstract class. To create your environment, you must define the constructor, `reset`, and `step` functions.

Function	Description
<code>getObservationInfo</code>	Return information about the environment observations
<code>getActionInfo</code>	Return information about the environment actions
<code>sim</code>	Simulate the environment with an agent
<code>validateEnvironment</code>	Validate the environment by calling the <code>reset</code> function and simulating the environment for one time step using <code>step</code>
<code>reset</code>	Initialize the environment state and clean up any visualization

Function	Description
step	Apply an action, simulate the environment for one step, and output the observations and rewards; also, set a flag indicating whether the episode is complete
Constructor function	A function with the same name as the class that creates an instance of the class

Sample Constructor Function

The sample cart-pole constructor function creates the environment by:

- Defining the action and observation specifications. For more information about creating these specifications, see `rlNumericSpec` and `rlFiniteSetSpec`.
- Calling the constructor of the base abstract class.

```
function this = MyEnvironment()
% Initialize observation settings
ObservationInfo = rlNumericSpec([4 1]);
ObservationInfo.Name = 'CartPole States';
ObservationInfo.Description = 'x, dx, theta, dtheta';

% Initialize action settings
ActionInfo = rlFiniteSetSpec([-1 1]);
ActionInfo.Name = 'CartPole Action';

% The following line implements built-in functions of the RL environment
this = this@rl.env.MATLABEnvironment(ObservationInfo,ActionInfo);

% Initialize property values and precompute necessary values
updateActionInfo(this);
end
```

This sample constructor function does not include any input arguments. However, you can add input arguments for your custom constructor.

Sample reset Function

The sample cart-pole reset function sets the initial condition of the model and returns the initial values of the observations. It also generates a notification that the environment has been updated by calling the `envUpdatedCallback` function, which is useful for updating the environment visualization.

```
% Reset environment to initial state and return initial observation
function InitialObservation = reset(this)
% Theta (+- .05 rad)
T0 = 2 * 0.05 * rand - 0.05;
% Thetadot
Td0 = 0;
% X
X0 = 0;
% Xdot
Xd0 = 0;

InitialObservation = [X0;Xd0;T0;Td0];
this.State = InitialObservation;

% (Optional) Use notifyEnvUpdated to signal that the
% environment is updated (for example, to update the visualization)
notifyEnvUpdated(this);
end
```

Sample step Function

The sample cart-pole step function:

- Processes the input action.
- Evaluates the environment dynamic equations for one time step.
- Computes and returns the updated observations.
- Computes and returns the reward signal.
- Checks if the episode is complete and returns the IsDone signal as appropriate.
- Generates a notification that the environment has been updated.

```
function [Observation,Reward,IsDone,Info] = step(this,Action)
    Info = [];

    % Get action
    Force = getForce(this,Action);

    % Unpack state vector
    XDot = this.State(2);
    Theta = this.State(3);
    ThetaDot = this.State(4);

    % Cache to avoid recomputation
    CosTheta = cos(Theta);
    SinTheta = sin(Theta);
    SystemMass = this.CartMass + this.PoleMass;
    temp = (Force + this.PoleMass*this.HalfPoleLength*ThetaDot^2*SinTheta)...
        /SystemMass;

    % Apply motion equations
    ThetaDotDot = (this.Gravity*SinTheta - CosTheta*temp)...
        / (this.HalfPoleLength*(4.0/3.0 - this.PoleMass*CosTheta*CosTheta/SystemMass));
    XDotDot = temp - this.PoleMass*this.HalfPoleLength*ThetaDotDot*CosTheta/SystemMass;

    % Euler integration
    Observation = this.State + this.Ts.*[XDot;XDotDot;ThetaDot;ThetaDotDot];

    % Update system states
    this.State = Observation;

    % Check terminal condition
    X = Observation(1);
    Theta = Observation(3);
    IsDone = abs(X) > this.DisplacementThreshold || abs(Theta) > this.AngleThreshold;
    this.IsDone = IsDone;

    % Get reward
    Reward = getReward(this);

    % (Optional) Use notifyEnvUpdated to signal that the
    % environment has been updated (for example, to update the visualization)
    notifyEnvUpdated(this);
end
```

Optional Functions

You can define any other functions in your template class as required. For example, you can create helper functions that are called by either `step` or `reset`. The cart-pole template model implements a `getReward` function for computing the reward at each time step.

```
function Reward = getReward(this)
    if ~this.IsDone
        Reward = this.RewardForNotFalling;
```



```

else
    Reward = this.PenaltyForFalling;
end
end

```

Environment Visualization

You can add a visualization to your custom environment by implementing the `plot` function. In the `plot` function:

- Create a figure or an instance of a visualizer class of your own implementation. For this example, you create a figure and store a handle to the figure within the environment object.
- Call the `envUpdatedCallback` function.

```

function plot(this)
    % Initiate the visualization
    this.Figure = figure('Visible','on','HandleVisibility','off');
    ha = gca(this.Figure);
    ha.XLimMode = 'manual';
    ha.YLimMode = 'manual';
    ha.XLim = [-3 3];
    ha.YLim = [-1 2];
    hold(ha,'on');
    % Update the visualization
    envUpdatedCallback(this)
end

```

For this example, store the handle to the figure as a protected property of the environment object.

```

properties(Access = protected)
    % Initialize internal flag to indicate episode termination
    IsDone = false

    % Handle to figure
    Figure
end

```

In the `envUpdatedCallback`, plot the visualization to the figure or use your custom visualizer object. For example, check if the figure handle has been set. If it has, then plot the visualization.

```

function envUpdatedCallback(this)
    if ~isempty(this.Figure) && isvalid(this.Figure)
        % Set visualization figure as the current figure
        ha = gca(this.Figure);

        % Extract the cart position and pole angle
        x = this.State(1);
        theta = this.State(3);

        cartplot = findobj(ha,'Tag','cartplot');
        poleplot = findobj(ha,'Tag','poleplot');
        if isempty(cartplot) || ~isvalid(cartplot) ...
            || isempty(poleplot) || ~isvalid(poleplot)
            % Initialize the cart plot
            cartpoly = polyshape([-0.25 -0.25 0.25 0.25],[-0.125 0.125 0.125 -0.125]);
            cartpoly = translate(cartpoly,[x 0]);
            cartplot = plot(ha,cardpoly,'FaceColor',[0.8500 0.3250 0.0980]);
            cartplot.Tag = 'cartplot';

            % Initialize the pole plot
            L = this.HalfPoleLength*2;

```

```
polepoly = polyshape([-0.1 -0.1 0.1 0.1],[0 L L 0]);
polepoly = translate(polepoly,[x,0]);
polepoly = rotate(polepoly,rad2deg(theta),[x,0]);
poleplot = plot(ha,polepoly,'FaceColor',[0 0.4470 0.7410]);
poleplot.Tag = 'poleplot';
else
    cartpoly = cartplot.Shape;
    polepoly = poleplot.Shape;
end

% Compute the new cart and pole position
[cartposx,~] = centroid(cartpoly);
[poleposx,poleposy] = centroid(polepoly);
dx = x - cartposx;
dtheta = theta - atan2(cartposx-poleposx,poleposy-0.25/2);
cartpoly = translate(cartpoly,[dx,0]);
polepoly = translate(polepoly,[dx,0]);
polepoly = rotate(polepoly,rad2deg(dtheta),[x,0.25/2]);

% Update the cart and pole positions on the plot
cartplot.Shape = cartpoly;
poleplot.Shape = polepoly;

% Refresh rendering in the figure window
drawnow();
end
end
```

The environment calls the `envUpdatedCallback` function, and therefore updates the visualization, whenever the environment is updated.

Instantiate Custom Environment

After you define your custom environment class, create an instance of it in the MATLAB workspace. At the command line, type the following.

```
env = MyEnvironment;
```

If your constructor has input arguments, specify them after the class name. For example, `MyEnvironment(arg1,arg2)`.

After you create your environment, the best practice is to validate the environment dynamics. To do so, use the `validateEnvironment` function, which prints an error to the command window if your environment implementation has any issues.

```
validateEnvironment(env)
```

After validating the environment object, you can use it to train a reinforcement learning agent. For more information on training agents, see “Train Reinforcement Learning Agents” on page 5-3.

See Also

Functions

`rlCreateEnvTemplate` | `train`

Objects

`rlNumericSpec` | `rlFiniteSetSpec`

More About

- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Environment Using Step and Reset Functions” on page 2-94
- “Define Observation and Reward Signals in Custom Environments” on page 2-127

Create Custom Simulink Environments

To create a custom Simulink environment, first create a Simulink environment model that represents the world as seen from the agent. Such a system is often referred to as *plant* or *open loop* system, while the whole (integrated) system that includes both agent and environment is often referred to as the *closed loop* system.

Your environment model must have an input signal, the action, which influences (through some discrete, continuous or mixed dynamics) its next internal state and its outputs, which are the observation, the reward and the is-done signals. The is-done signal is a scalar that indicates the termination of an episode, causing the simulation to stop when its value is `true`.

To prevent the waste of computational resources, update the observation, reward and is-done signals at the same execution rate used by the Agent block.

Note A reinforcement learning environment is normally assumed to be strictly causal from the current action to the current observation. That is, it is assumed that the current observation does not depend on the current action (while the next state generally does). In other words, there must be no direct feedthrough between the current action and the current observation.

Note The reward signal at time t must be the one corresponding to the transition between the observation output at time $t-1$ and the observation output at time t .

If your observation contains multiple channels, group the signals carried by the channels into a single observation bus. Similarly, for a hybrid environment, your action must be a two-element bus containing both the discrete (first) and the continuous (second) action channel. For more information about bus signals, see “Simulink Bus Capabilities” (Simulink).

For critical considerations on defining reward and observation signals in custom environments, see “Define Observation and Reward Signals in Custom Environments” on page 2-127.

Once you have created the Simulink model that represents the environment, you must add the RL Agent block to it. You can do so automatically or manually.

- To automatically create a new closed-loop Simulink model that contains an RL Agent block and references your environment model from its Environment block, use the `createIntegratedEnv` function, specifying the names of both your existing environment model and of the new, to be created, closed-loop model that contains the agent.

You can specify as input arguments the names of the action, observation, is-done, and reward ports in your environment model. If your action or observation space is finite, you can also specify its possible values (otherwise the signals are assumed to be continuous).

This function returns an environment object as well as the block path of the agent and the environment observation and action specifications. For more information on model referencing, see “Model Reference Behavior and Capabilities” (Simulink).

- To manually add the agent to your model, drag and drop the RL Agent block from the Reinforcement Learning Simulink library. Connect the action, observation, reward and is-done signals to the appropriate output and input ports of the block.

Unless you already have an agent object for this environment in the MATLAB workspace, you must create specification objects for the action and observation signals using `rlNumericSpec` (for

continuous signals) or `rlFiniteSetSpec` (for discrete signals). For bus signals, create specifications using `bus2RLSpec`.

Once you connect the blocks, create an environment object using `rlSimulinkEnv`, specifying the model filename, the block path to the RL Agent within the model, and the specification objects for the observation and the action channels, respectively. If your agent block already references an agent object in the MATLAB workspace, you do not need to supply the specification objects as input arguments.

For an example, see “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132 and “Control Water Level in a Tank Using a DDPG Agent” on page 8-2.

Both `rlSimulinkEnv` and `createIntegratedEnv` return a custom Simulink environment as a `SimulinkEnvWithAgent` object. This environment object acts as an interface so that when you call `sim` or `train`, these functions in turn call the (compiled) Simulink model associated with the object to generate experiences for the agents. You can use this object to train and simulate agents in the same way as with any other environment.

Note Before training or simulating an agent within a Simulink environment, to make sure that the RL Agent block runs at the intended sample time, set the `SampleTime` property of your agent object appropriately.

You can also create a multiagent Simulink environment. To do so, create a Simulink model that has one action input and one set of outputs (observation, reward and is-done) for every agent. Then manually add an agent block for each agent. Once you connect the blocks, create an environment object using `rlSimulinkEnv`. Unless each agent block already references an agent object in the MATLAB workspace, you must supply to `rlSimulinkEnv` two cell arrays containing the observation action specification objects, respectively, as input arguments. For an example, see “Train Multiple Agents to Perform Collaborative Task” on page 8-31.

Your environment can also include third-party functionality. For more information, see “Integrate Components from External Tools” (Simulink).

Algebraic Loops Between Environment and Agent

To avoid (potentially unsolvable) algebraic loops, you must avoid any direct feedthrough (that is, any direct dependency in the same time step) from the action to the observation output signal. This is because in the Simulink implementation of the agent block, the action at a given time step depends on the observation at the same time step. In other words, the agent block has a direct feedthrough from its observation input to its action output (similarly to an output feedback controller).

Avoiding a direct feedthrough from the action to the observation output signal is also in line with the fact that the standard formulation of a reinforcement learning environment as a Markov Decision Process is strictly causal from the current action to the current observation, since the current state does not depend on the current action (while the next state generally does).

However, note that for models created using `createIntegratedEnv` the environment block is a referenced subsystem. Referenced subsystems are normally treated as a direct feedthrough block (including the path from action to observation) unless the **Minimize artificial algebraic loop occurrences** parameter in the *referenced subsystem* is enabled. When the referenced subsystem has no direct feedthrough from an input port that participates in an artificial algebraic loop to *any* of its

outputs ports, enabling this parameter can remove artificial algebraic loops involving the referenced subsystem.

In general, adding a Delay or Memory block to the action signal between the agent block and environment block removes the algebraic loop. When you add an action delay, make sure that your reset function, which is called at the beginning of each training or simulation episode, initializes the delay to a feasible value.

Alternatively you can add delay blocks to all the environment output signals after the environment block. If you do so, make sure that your reset function initializes the delay to a feasible value which is also consistent with the initial state of the environment.

Note In general, adding delays to solve algebraic loops should be done with extreme care, as it involves a modification of the loop dynamics.

If you have separate state and output functions (instead of a single step function), you can call them using separate MATLAB Function blocks, using a delay to represent the environment state. If you do so, your reset function only needs to initialize the state.

For more information on algebraic loops and how to remove some of them, see “Algebraic Loop Concepts” (Simulink) and “Remove Algebraic Loops” (Simulink). For a related example about using delays in a reinforcement learning loop implemented in Simulink, see “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114.

Reset Function for Simulink Environments

When you create a Simulink environment, you also typically create a custom `reset` function that can set the parameters or initial state appropriately every time a simulation starts to train or simulate the environment. For example, you can create a reset function that randomizes certain block states such that each training episode begins from different initial conditions.

To specify your reset function, assign the `ResetFcn` property of the environment to a function handle or anonymous function handle. The function must have a single `Simulink.SimulationInput` input argument and a single `Simulink.SimulationInput` output argument. The output object specifies temporary changes applied to model for the duration of the simulation of training episode. For more information about Simulink simulation input objects, see `Simulink.SimulationInput`.

For example, if you have an existing reset function `myResetFunction` on the MATLAB path, you can set `ResetFcn` to an anonymous function that in turn calls `myResetFunction`.

```
env.ResetFcn = @(in)myResetFunction(in);
```

Note that you can use the anonymous function to pass additional input arguments whose value is available when the function is defined. For example, you can pass the additional arguments `arg1` and `arg2` as follows.

```
env.ResetFcn = @(in)myResetFunction(in,arg1,arg2);
```

Here, when `env.ResetFcn` is created, its workspace include the values of `arg1` and `arg2`. These values persist within the reset function workspace even if you clear the variables from the MATLAB workspace. When `env.ResetFcn` is evaluated, it invokes `myResetFunction` and passes it a copy of the values that `arg1` and `arg2` had when the reset function was defined. For more information, see “Anonymous Functions”.

If your reset behavior is simple, you can implement it without writing the separate function `myResetFunction`. For example, the following code uses `setVariable` to set the variable `x0` to a random value in the model workspace, and return the change in the `Simulink.SimulationInput` object `in`.

```
env.ResetFcn = @(in) setVariable(in,'x0',rand());
```

Here, the value of `x0` that you specify overrides the existing `x0` value in the model workspace for the duration of the simulation or training. The value of `x0` is then reverted to the original when the simulation or training completes.

If you call the reset function of a `SimulinkEnvWithAgent` object that has its `ResetFcn` property empty, a `Simulink.SimulationInput` for the unmodified Simulink model is returned.

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. To make sure that the reset function always uses the same random initial values when it is called the first time in your script, fix the random stream generator before calling `train` or `sim`. For example, fix the random number stream with seed 0 and random number algorithm Mersenne Twister as follows.

```
previousRngState = rng(0,"twister");
```

For more information on controlling the seed used for random number generation, see `rng`. For more information on result reproducibility, see “Results Reproducibility” on page 5-10.

For more information on reset functions for Simulink environments, see the `ResetFcn` property of `SimulinkEnvWithAgent`. For examples, see `rlSimulinkEnv`, “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132, and “Train Multiple Agents to Perform Collaborative Task” on page 8-31.

See Also

Functions

`rlSimulinkEnv` | `createIntegratedEnv` | `setVariable` | `train` | `sim`

Objects

`SimulinkEnvWithAgent` | `Simulink.SimulationInput`

Related Examples

- “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132
- “Control Water Level in a Tank Using a DDPG Agent” on page 8-2
- “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114

More About

- “Algebraic Loop Concepts” (Simulink)
- “Reinforcement Learning Environments” on page 2-2
- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142
- “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114

Create and Simulate Same Environment in Both MATLAB and Simulink

In this example, you create a MATLAB® implementation of an environment and compare it with different Simulink® implementations of the same environment. This comparison highlights important differences in the agent-environment interaction between MATLAB and Simulink.

This example also shows that, in general, when implementing an environment in Simulink, you have two options:

- 1 Output the *next observation* signal (that is, the output function of the next state). This is the approach you follow when calling the step function of a MATLAB environment using a MATLAB Function block. The resulting implementation is equivalent to the way a MATLAB environment works, but requires you to place a Unit Delay block either on each input or on the output of the RL Agent block.
- 2 Output the *current observation* signal (that is, the output function of the current state) without using any Unit Delay block.

Although both approaches are equivalent, it is often simpler to use the second one, especially when the next state signal is not available, such as for continuous-time environments.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Using MATLAB Functions

To implement an environment, first define the environment observation and action specifications. For this example, both action and observation are a simple numerical scalar.

```
oinfo = rlNumericSpec([1 1]);
ainfo = rlNumericSpec([1 1]);
```

Use the custom step and reset functions in the supporting files to define the behavior of a simple discrete nonlinear system. For this example, set the reward equal to the observation.

Display the step function, `dsStepFcn.m`.

```
type dsStepFcn.m

function [NextObs,Reward,IsDone,NextState] = dsStepFcn(Action,State)
% Advances environment to next state and calculates outputs
```



```
NextState = 0.9*State + cos(Action);
NextObs = NextState^2;
```

```
Reward = NextObs;
IsDone = 0;
```

```
end
```

It is important to notice that although there is a direct feedthrough between the action and the *next* observation, there is no direct feedthrough between the action and the *current* observation. In other words, the environment is strictly causal from action to observation.

The reset function, `dsResetFcn.m`, initializes the environment state to 5 and the corresponding initial observation to 25. Display the reset function.

```
type dsResetFcn.m
```

```
function [InitialObservation, InitialState] = dsResetFcn()
% Resets environment to an initial state
```

```
InitialState = 5;
InitialObservation = InitialState^2;
```

```
end
```

Use `rlFunctionEnv` to create a custom environment object.

```
menv = rlFunctionEnv(oinfo,ainfo,@dsStepFcn,@dsResetFcn);
```

Create Default Actor-Critic Agent

Create a default actor-critic agent using the environment specifications. To ensure consistency between different simulations, prevent the agent from exploring by setting the `UseExplorationPolicy` option to `false`. When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0,"twister");
agentObj = rlACAgent(oinfo,ainfo);
agentObj.UseExplorationPolicy = false;
```

Simulate Agent Using MATLAB Environment

To set the maximum number of simulation steps to six, create an `rlSimulationOptions` object and set its `MaxSteps` property to 6.

```
simopts = rlSimulationOptions(MaxSteps=6);
```

Then simulate the environment using `sim`, and collect the experience in the variable `mtraj`.

```
mtraj = sim(menv,agentObj,simopts);
```

Display the action data.

```
mtraj.Action.act1.Data(:)'
```

```
ans = 1×6
```

```
2.5540    1.3743    1.2488    1.2252    1.2199    1.2186
```

Display the observation data.

```
mtraj.Observation.obs1.Data(:)'
```

```
ans = 1×7
```

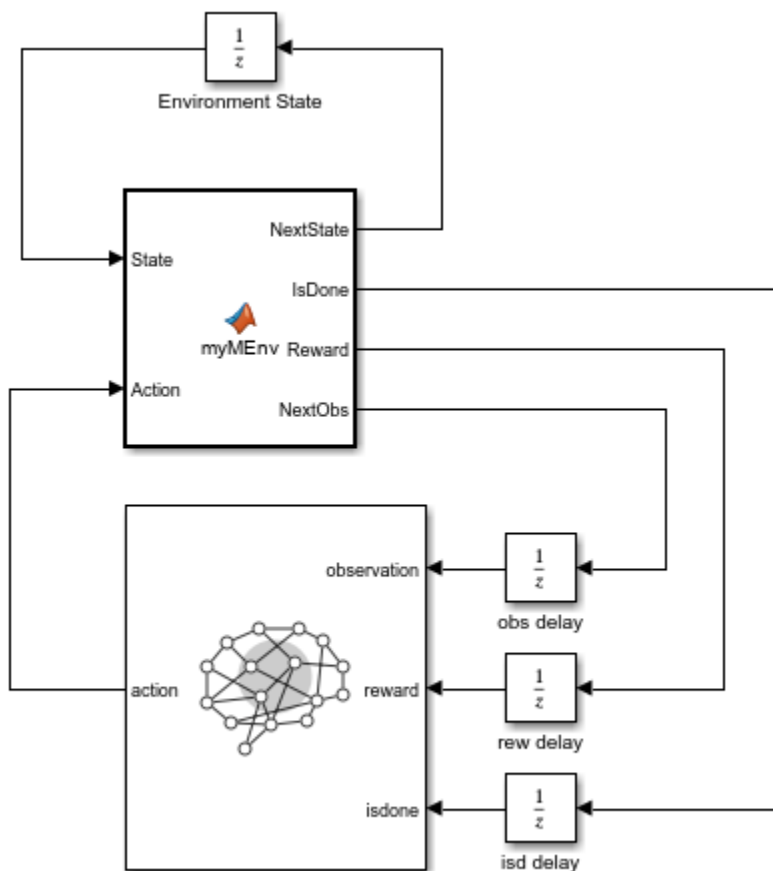
```
25.0000    13.4520    12.2233    11.9928    11.9405    11.9282    11.9253
```

Call MATLAB Environment from Simulink Using Delays on Observation and Reward

You can use a MATLAB Function (Simulink) block to call the environment step function from Simulink. Assume that both the environment and the agent have a sample time of one second.

Load the Simulink model in memory.

```
mdl = "dsLoopMLF3";  
load_system(mdl);
```



In this model, the RL Agent block is set to use `agentObj` as an agent. The MATLAB Function block calls the environment step function. In the step function, the order of inputs and outputs are rearranged so that the respective signal lines do not cross each other in Simulink.

```
function [NextObs,IsDone,Reward,NextState] = myMEnv(Action,State)

% Call environment step function
[NextObs,Reward,IsDone,NextState] = dsStepFcn(Action,State);
```

In the model, a Unit Delay block with an initial state set to 5 stores the environment state. This implementation avoids using permanent variables to implement states in the MATLAB Function block, which is subject to several limitations. For more information on these limitations, see the tables in “Speed Up Simulation Workflows by Using Model Operating Points” (Simulink) and “How Stepping Through Simulations Works” (Simulink).

The RL Agent block expects the *current* observation as the input. Therefore, the model uses a delay block, with the **Initial condition** parameter set to 25, to obtain the current observation. If you instead connect the agent observation port directly to the **NextObs** environment output, the action signal would be anticipated one step with respect to the MATLAB implementation, which would be incorrect and can also lead to an algebraic loop. For more information on how to deal with algebraic loops see “Create Custom Simulink Environments” on page 2-110 and “Algebraic Loop Concepts” (Simulink).

Because the reward and is-done signal received by the agent need to be synchronized with the observation signal, the model uses Unit Delay blocks also on the reward and isdone signals to synchronize these signals with the observation signal. The **Initial condition** parameters of these Unit Delay blocks are set to 25 and 0, respectively.

Create a Simulink environment from the closed-loop Simulink model. First, define block path to the RL Agent block.

```
blk = mdl + "/RL Agent";
```

Use `rlSimulinkEnv` to create a Simulink environment.

```
slenv = rlSimulinkEnv(mdl,blk,oinfo,ainfo);
```

Use `sim` to simulate the environment. Collect the resulting experience in the variable `straj`.

```
straj = sim(slenv,agentObj,simopts);
```

Display the action data.

```
straj.Action.act1.Data(:)'
```

```
ans = 1×6
```

```
2.5540    1.3743    1.2488    1.2252    1.2199    1.2186
```

Display the observation data.

```
straj.Observation.obs1.Data(:)'
```

```
ans = 1×7
```

```
25.0000    13.4520    12.2233    11.9928    11.9405    11.9282    11.9253
```

The trajectory is identical to the one obtained from the simulation of the MATLAB environment.

Note that, when you use this approach to calling a MATLAB environment from Simulink, you have to initialize the Unit Delay blocks in a way that is consistent with the initial state of the environment.

Call MATLAB Environment from Simulink Using Delay on Action

You can also call the environment step function by placing a single Unit Delay block on the action signal instead of placing three Unit Delay blocks on the RL Agent block inputs. However, this design results in a closed-loop system that generally starts from a different initial condition, and therefore generates a different trajectory.

For the first episode of training or simulation, you can set the initial condition of the Unit Delay block on the action signal so that the resulting trajectory is the same (just anticipated one step) as the one generated by a MATLAB environment. However, in general, any trajectory that starts from a feasible initial condition is equally valid to train or simulate the agent.

Get the initial value of the action when the observation is equal to 5^2 .

```
a0 = getAction(agentObj,{5^2});
```

Display the value of the initial action.

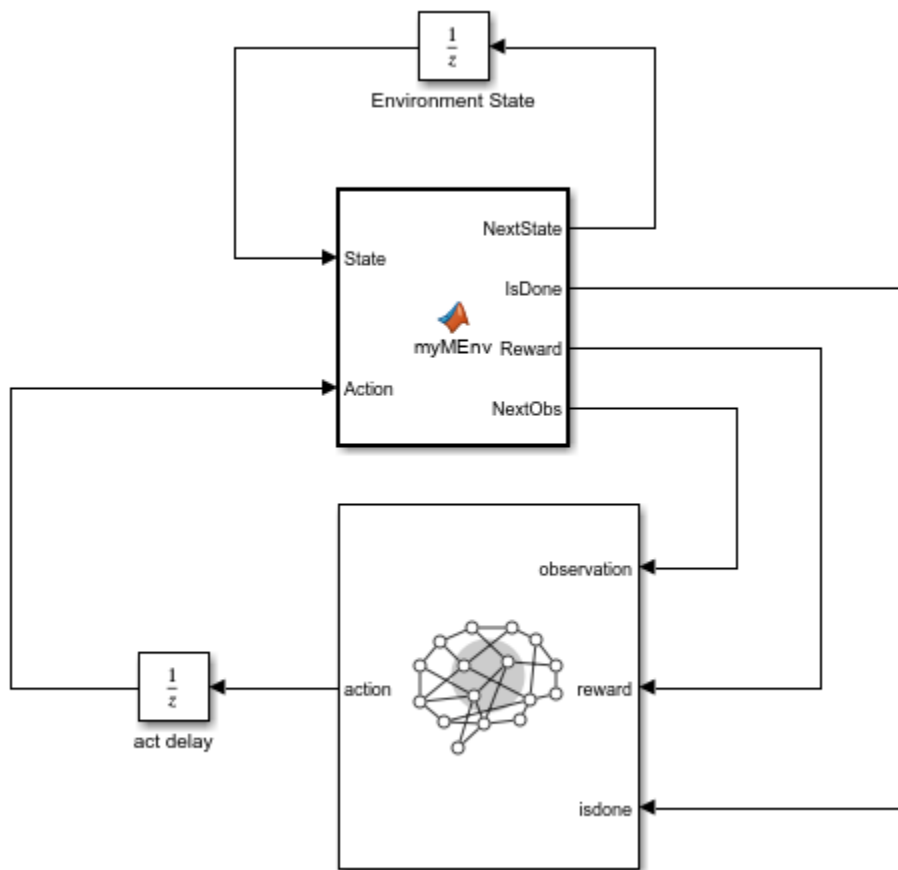
```
a0{1}
```

```
ans =  
2.5540
```

In the following model, `dsLoopMLF1.slx`, the **Initial condition** parameter of the Unit Delay block on the action signal is already set to `a0{1}`.

Load the Simulink model in memory.

```
mdl = "dsLoopMLF1";  
load_system(mdl);
```



Define the block path to the RL Agent block.

```
blk = mdl + "/RL Agent";
```

Use `rlSimulinkEnv` to create a Simulink environment.

```
slenv = rlSimulinkEnv(mdl,blk,oinfo,ainfo);
```

Use `sim` to simulate the environment. Collect the resulting experience in the variable `straj`.

```
straj = sim(slenv,agentObj,simopts);
```

Display the action data.

```
straj.Action.act1.Data(:)'
```

```
ans = 1x6
```

```
1.3743    1.2488    1.2252    1.2199    1.2186    1.2183
```

Display the observation data.

```
straj.Observation.obs1.Data(:)'
```

```
ans = 1×7  
    13.4520    12.2233    11.9928    11.9405    11.9282    11.9253    11.9246
```

The trajectory is one time step ahead of the one obtained from the simulation of the MATLAB environment.

Call Separate Environment State and Output Functions from Simulink

If you have separate state and output functions (instead of a single step function), you can call them using separate MATLAB Function (Simulink) blocks, and use an Unit Delay block to represent the environment state.

Display the state function, `dsStateFcn.m`.

```
type dsStateFcn.m  
  
function NextState = dsStateFcn(Action,State)  
% Advances environment to next state  
  
NextState = 0.9*State + cos(Action);  
  
end
```

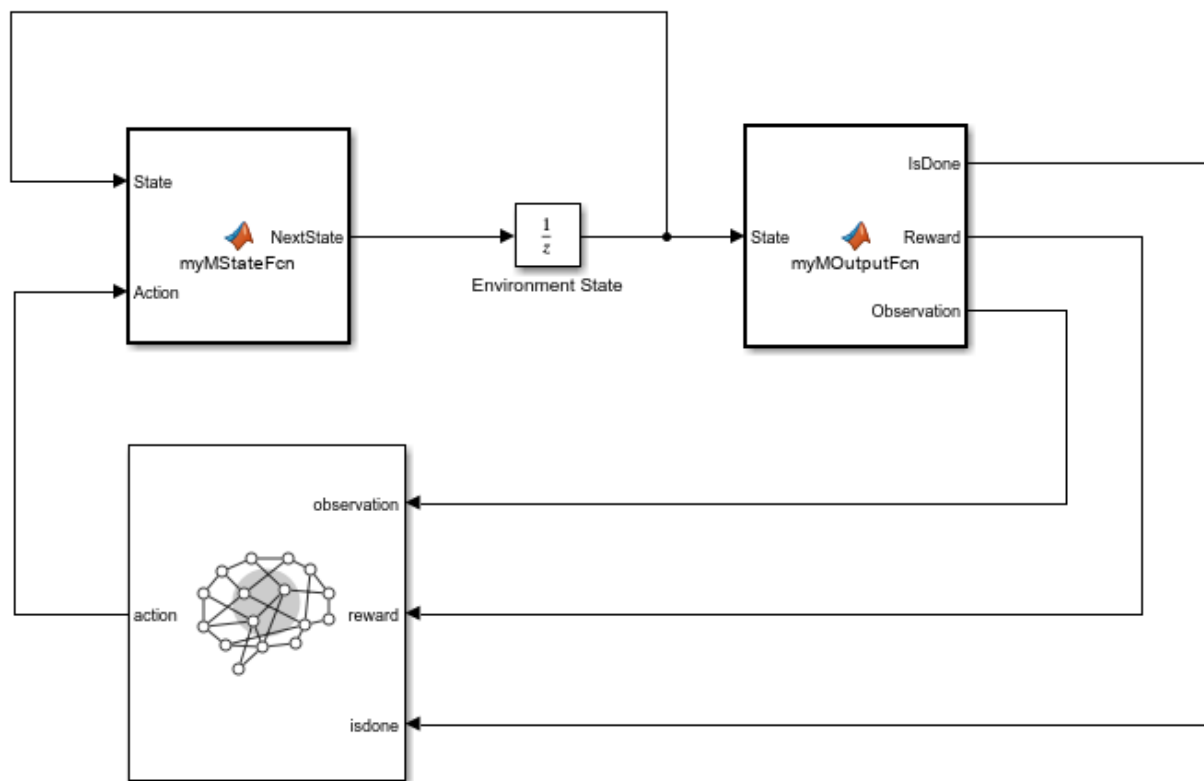
Display the output function, `dsOutputFcn.m`.

```
type dsOutputFcn.m  
  
function [Observation,Reward,IsDone] = dsOutputFcn(State)  
% Calculates outputs  
  
Observation = State^2;  
Reward = Observation;  
IsDone = 0;  
  
end
```

Note that the output does not depend on the current action, because the environment is assumed to be strictly causal.

Load the Simulink model in memory.

```
mdl = "dsLoopMLF2";  
load_system(mdl);
```



This model uses MATLAB Function (Simulink) blocks to call both functions, and a single Unit Delay block to represent the environment state.

In the first model of this example, you fed the *delayed NextObs* output signal to the observation input port of the agent block. In this model, instead, the corresponding signal is the delayed output function (in this case, the square) of the **NextState** output signal. This signal is identical, from the agent perspective, to the output function of the *current* state signal, which is, in turn, exactly the current observation signal.

In other words, in this model, there is no Unit Delay block on the observation signal because the output function already receives in input the current state instead of the next state (and therefore calculates the current observation instead of the next observation).

Define the block path to the RL Agent block.

```
blk = mdl + "/RL Agent";
```

Use `rlSimulinkEnv` to create a Simulink environment.

```
slenv = rlSimulinkEnv(mdl,blk,oinfo,ainfo);
```

Use `sim` to simulate the environment. Collect the resulting experience in the variable `straj`.

```
straj = sim(slenv,agentObj,simopts);
```

Display the action data.

```
straj.Action.act1.Data(:)'
```

```
ans = 1×6
```

```
2.5540 1.3743 1.2488 1.2252 1.2199 1.2186
```

Display the observation data.

```
straj.Observation.obs1.Data(:)'
```

```
ans = 1×7
```

```
25.0000 13.4520 12.2233 11.9928 11.9405 11.9282 11.9253
```

The trajectory is identical to the one obtained from the simulation of the MATLAB environment.

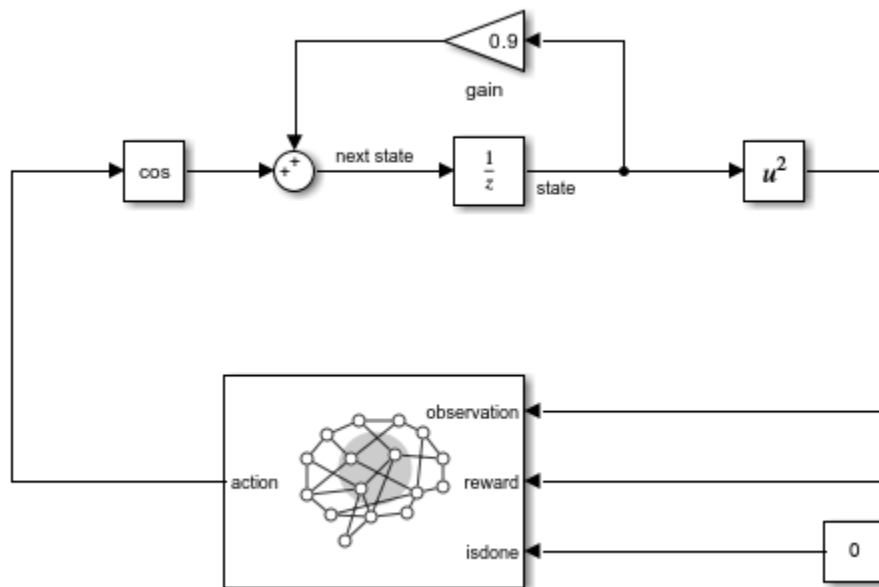
For an example in which a MATLAB environment is called from Simulink using this approach, see “Train Multiple Agents for Area Coverage” on page 8-44.

Implement Same Environment Using Built-In Simulink Blocks

Instead of calling existing MATLAB environment functions, the `dsLoopDSS.slx` model directly implements, using built-in Simulink blocks, the discrete-time system defined in the `dsStepFcn.m` file.

Load the Simulink model in memory.

```
mdl = "dsLoopSDS";  
load_system(mdl);
```



The delayed signal from the **NextObs** output port in the first example corresponds to delayed square of the next **state** signal in this model. From the perspective of the agent, this signal is identical to the square of the **state** signal.

Define the block path to the RL Agent block, and create the environment object.

```
blk = mdl + "/RL Agent";
slenv = rlSimulinkEnv(mdl,blk,oinfo,ainfo);
```

Simulate the environment, and collect the resulting experience in the variable **straj**.

```
straj = sim(slenv,agentObj,simopts);
```

Display the action data.

```
straj.Action.act1.Data(:)'
```

```
ans = 1×6
```

```
2.5540 1.3743 1.2488 1.2252 1.2199 1.2186
```

Display the observation data.

```
straj.Observation.obs1.Data(:)'
```

```
ans = 1×7
```

```
25.0000 13.4520 12.2233 11.9928 11.9405 11.9282 11.9253
```

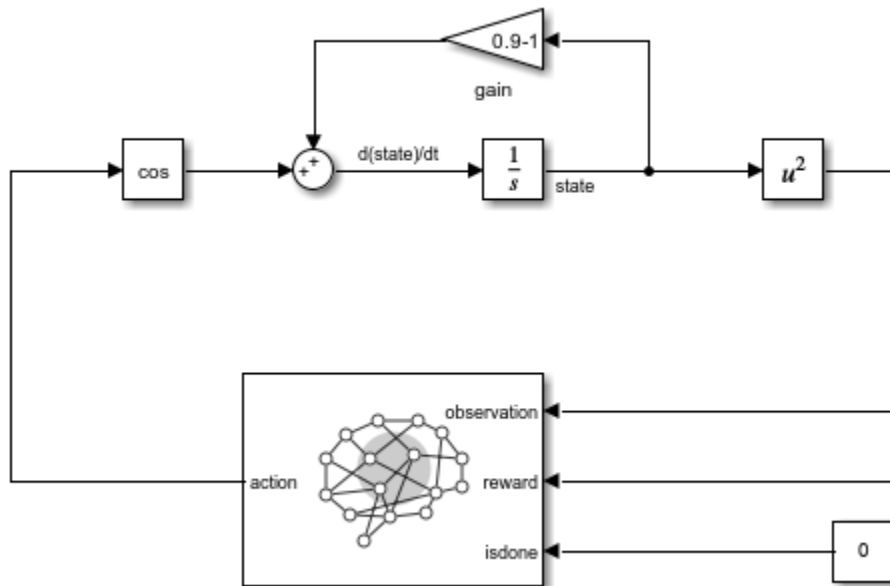
As expected, the trajectory is identical to the previous ones.

Implement Same Environment in Continuous-Time

In the **csLoop.slx** model, the inner linear discrete-time system featured in the **dsLoopDSS.slx** model is replaced with its continuous-time equivalent.

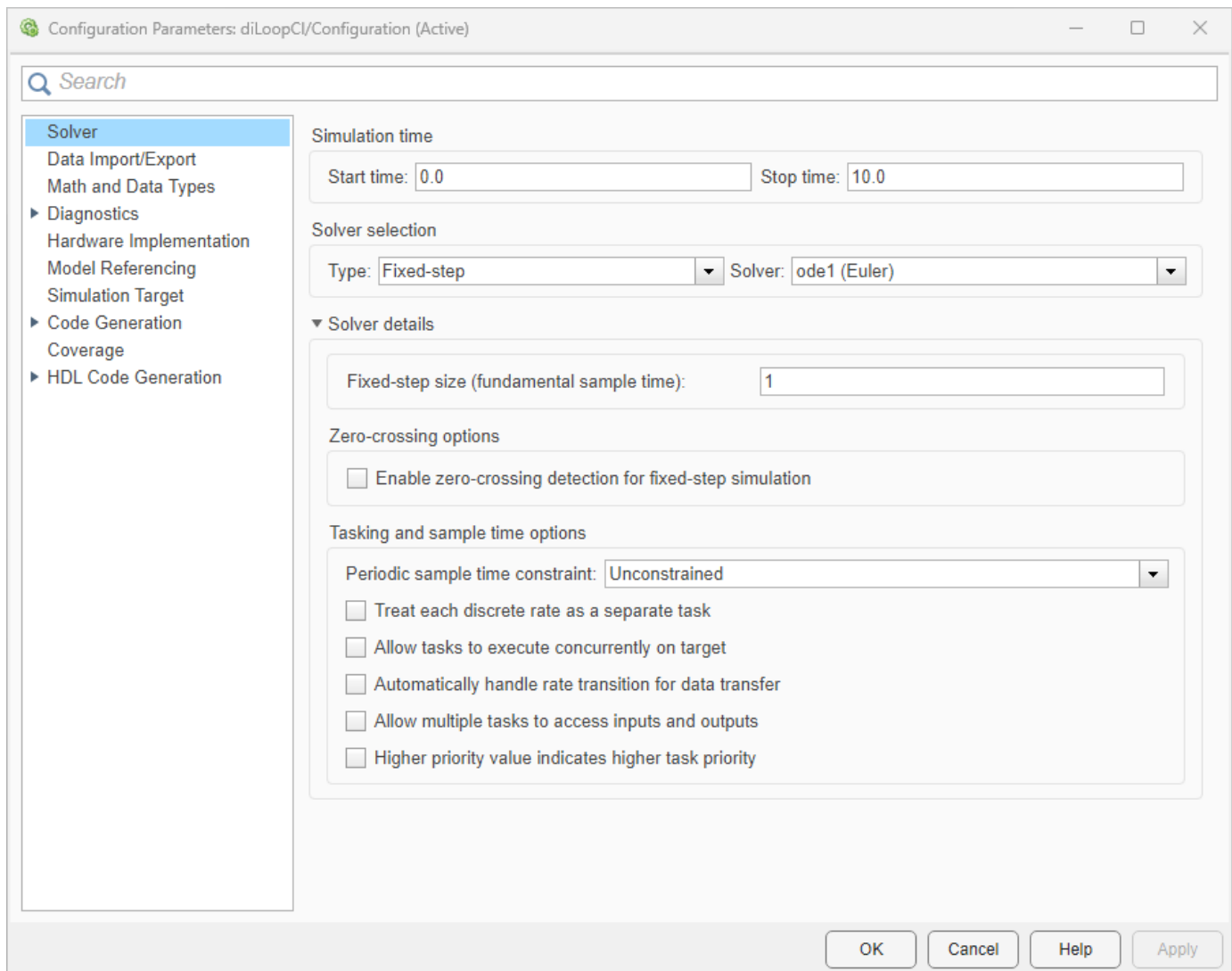
Load and open the Simulink model with the continuous integrator.

```
mdl = "csLoop";
load_system(mdl);
```



Here, you do not have access to the next state and next observation signals.

Because this is a continuous-time model, it relies on the Simulink solver to perform the integration step. For this example, the fixed-step ODE1 (Euler) solver is selected.



Create the environment object

```
blk = mdl + "/RL Agent";
slenv = rlSimulinkEnv(mdl,blk,oinfo,ainfo);
```

Simulate the environment, and collect the resulting experience in the variable `straj`.

```
straj = sim(slenv,agentObj,simopts);
```

Display the action data.

```
straj.Action.act1.Data(:)'
```

```
ans = 1×6
```

```
2.5540 1.3743 1.2488 1.2252 1.2199 1.2186
```

Display the observation data.

```
straj.Observation.obs1.Data(:) '  
ans = 1×7  
    25.0000    13.4520    12.2233    11.9928    11.9405    11.9282    11.9253
```

The trajectory is identical to the previous ones.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Blocks

MATLAB Function

Functions

`rlSimulinkEnv` | `sim`

Related Examples

- “Train Multiple Agents for Area Coverage” on page 8-44

More About

- “Create Custom Simulink Environments” on page 2-110
- “Algebraic Loop Concepts” (Simulink)

Define Observation and Reward Signals in Custom Environments

When designing a reinforcement learning environment, the choice of observation and reward signals is crucial because they directly influence the agent's learning efficiency and its ability to generalize and perform well in the task. Poorly designed observation and reward signals can lead to suboptimal learning, convergence to unintended behaviors, or failure to learn altogether.

While the observation signals might be mostly determined by the underlying physical systems and the available sensors, a good observation signal should provide the agent with sufficient and relevant information about the environment's state, enabling it to make informed decisions.

You typically have more flexibility to design an appropriate reward signal. A good reward signal should accurately reflect the agent's performance in achieving the desired outcome, encouraging behaviors that lead to success while discouraging those that do not. It should be neither too sparse nor too dense, providing timely feedback to guide the agent effectively.

An important consideration is that a reinforcement learning environment is normally assumed to be strictly causal from the current action to the current observation. That is, it is assumed that the current observation does not depend on the current action (while the next state generally does). In other words, there must be no direct feedthrough between the current action and the current observation.

Strict Causality and Time-Discretization

Reinforcement Learning Toolbox environments must be strictly causal. This means that the current observation must not depend on the current action, or, in other words, there must not be a direct feedthrough between action and observation. For a step-by-step explanation of how the environment interacts with an agent, see “Reinforcement Learning Environments” on page 2-2.

When you implement a custom MATLAB environment using custom step and reset functions or a class template, the training or simulation function interprets the first and second outputs of your step function as the *next* observation $O(t+1)$ and its *associated* reward $R(t+1)$. While they are typically functions of the current action $A(t)$, they cannot be a function of the next action $A(t+1)$, which is not yet available to your step function. Therefore, strict causality is enforced by the software.

When you implement a custom Simulink environment, you must make sure that there is no direct feedthrough between action and observation, to avoid potentially unsolvable algebraic loops. For more information, see “Create Custom Simulink Environments” on page 2-110.

If your MATLAB environment relies on continuous-time equations to represent the dynamics of the underlying system, you must use an integration technique such as the Euler or the trapezoidal method to calculate the next state $x(t+1)$, the next observation $O(t+1)$, and its associated reward $R(t+1)$ as functions of the current state $x(t)$. Here, you typically assume that the action stays constant in the interval between t and $t+1$.

Simulink environments can also directly rely on continuous-time equations to represent the system dynamics. In this case, during simulation, the Simulink solver integrates the continuous-time equations. The RL Agent block always executes at discrete intervals according to the sample time specified in the `SampleTime` property of the agent object.

For an example on different ways to implement the same environment in Simulink, see “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114.

Observation

Choosing the right observations for the agent is crucial for effective training and performance in reinforcement learning. The observations should provide sufficient information about the current environment state (independently on past environment states) for the agent to make informed decisions.

For example, for control system applications, while the observation depends on your application and on the measurements that are available for the underlying physical system, the integrals (and sometimes derivatives) of error signals are often useful observations. Also, for reference-tracking applications, having a time-varying reference signal as an observation is important.

If the environment state is low-dimensional, and all the states are available for measurement, it is best practice to include all the available environment states in the observation vector. This ensures that all the necessary information about the environment can be captured by the agent. Failure to do so can lead to situations in which different environment states result in the same observation. For such states, the agent policy (assuming it is a static function of the observation) can only return the same action. Such a policy is typically unsuccessful, because it is normally the case that a successful policy needs to react to different environment states by returning different actions.

For example, an image observation of a swinging pendulum has position information but does not have enough information, by itself, to determine the pendulum velocity. In this case, a static policy that cannot sense the velocity would not be able to stabilize the pendulum. But if the velocity can be measured or estimated, adding it as an additional entry in the observation vector will provide a static policy with enough information to stabilize the pendulum.

When not all states are available as observation signals (for example because it would be unrealistic to measure them), a possible workaround is to use an estimator (as a part of the environment) that estimates the values of the unmeasured states, and makes such estimates available to the agent as observations. Alternatively, you can use recurrent networks such as an LSTM in your policy. Doing so results in a policy that has states, and that might therefore be able to use its state as an internal representation of the environment state. Such a policy can consequently return different actions (based on different values of its internal state) even when there is not enough information to reconstruct the correct environment state from the current observation.

When the state or action space is large, it becomes challenging for the agent to explore and learn effectively. The curse of dimensionality can make it difficult to find an optimal policy within a reasonable time frame. Therefore, if the observation space is high dimensional (for example containing images) it might be necessary to preprocess the observations. When images are used as observations, techniques like image resizing, cropping, or converting to grayscale can help reduce dimensionality and focus on relevant features.

Depending on the complexity of the task, you might need to perform feature engineering to extract relevant information from the observations. This can involve transforming or combining the raw observations to create more informative features. In general, it is best practice to leverage any domain knowledge or insights you have about the task to guide the selection of observations. Understanding the underlying environment dynamics and relevant factors can help you identify the most informative observations for the agent.

The selection of observation (and action) signals has an important effect on the agent training and its convergence. For more information, see the last section of “Train Reinforcement Learning Agents” on page 5-3.

Reward

To guide the learning process, reinforcement learning uses a scalar reward signal generated from the environment.

This signal measures the immediate performance of the agent with respect to the task goals. Specifically, the reward $R(t+1)$ measures the immediate effectiveness of the transition from the observation $O(t)$ to the next observation $O(t+1)$ caused by the agent action $A(t)$. Note that the reward $R(t+1)$ is synchronized with the *next* observation $O(t+1)$, and cannot depend on $A(t+1)$. In other words, the reward is strictly causal with respect to the action.

During training, an agent updates its policy based on the rewards received for different state-action combinations. For an introduction to different types of agents and how they use the reward signal during training, see “Reinforcement Learning Agents” on page 3-2.

In general, you provide a positive reward to encourage certain agent actions and a negative reward (penalty) to discourage other actions. A well-designed reward signal guides the agent to maximize the expectation of the (possibly discounted) cumulative long-term reward. What constitutes a well-designed reward depends on your application and the agent goals.

For example, when an agent must perform a task for as long as possible, a common strategy is to provide a small positive reward for each time step that the agent successfully performs the task and a large penalty when the agent fails. This approach encourages longer training episodes while heavily discouraging actions that lead to episodes in which the agent fails. For an example that uses this approach, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64.

If your reward function incorporates multiple signals, such as position, velocity, and control effort, you must consider the relative sizes of the signals and scale their contributions to the reward signal accordingly.

You can specify either continuous or discrete reward signals. In either case, you must provide a reward signal that provides rich information when the action and observation signals change.

For control system applications in which cost functions and constraints are already available, you can also use built-in functions to generate rewards from such specifications.

Continuous Reward Functions

A continuous reward function varies continuously with changes in the environment observations and actions. In general, continuous reward signals improve convergence during training and can lead to simpler network structures.

An example of a continuous reward is the quadratic regulator (QR) cost function, where the cumulative long-term reward can be expressed as:

$$J_i = - \left(s_i^T Q_i s_i + \sum_{j=i}^{\tau} s_j^T Q_j s_j + a_j^T R_j a_j + 2s_j^T N_j a_j \right)$$

Here, Q_τ , Q , R , and N are the weight matrices. Q_τ is the terminal weight matrix, applied only at the end of the episode. Also, s is the observation vector, a is the action vector, and τ is the terminal iteration of the episode. The (instantaneous) reward for this cost function is

$$r_i = s_i^T Q_i s_i + a_i^T R_i a_i + 2s_i^T N_i a_i$$

This QR reward structure encourages an agent to drive s to zero with minimal action effort. A QR-based reward structure is a good reward to choose for regulation or stationary point problems, such as pendulum swing-up or regulating the position of the double integrator. For training examples that use a QR reward, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70 and “Compare DDPG Agent to LQR Controller” on page 7-285.

Smooth continuous rewards, such as the QR regulator, are good for fine-tuning parameters and can provide policies similar to optimal controllers (LQR/MPC).

Discrete Reward Functions

A discrete reward function varies discontinuously with changes in the environment observations or actions. These types of reward signals can make convergence slower and can require more complex network structures. Discrete rewards are usually implemented as *events* that occur in the environment—for example, when an agent receives a positive reward if it exceeds some target value or a penalty when it violates some performance constraint.

While discrete rewards can slow down convergence, they can also guide the agent toward better reward regions in the state space of the environment. For example, a region-based reward, such as a fixed reward when the agent is near a target location, can emulate final-state constraints. Also, a region-based penalty can encourage an agent to avoid certain areas of the state space.

Mixed Reward Functions

In many cases, providing a mixed reward signal that has a combination of continuous and discrete reward components is beneficial. The discrete reward signal can be used to drive the system away from bad states, and the continuous reward signal can improve convergence by providing a smooth reward near target states. For example, in “Train DDPG Agent to Control Two-Thruster Sliding Vehicle” on page 8-22, the reward function has three components: r_1 , r_2 , and r_3 .

$$\begin{aligned} r_1 &= 10 \left((x_t^2 + y_t^2 + \theta_t^2) < 0.5 \right) \\ r_2 &= -100 (|x_t| \geq 20 \mid |y_t| \geq 20) \\ r_3 &= - \left(0.2(R_{t-1} + L_{t-1})^2 + 0.3(R_{t-1} - L_{t-1})^2 + 0.03x_t^2 + 0.03y_t^2 + 0.02\theta_t^2 \right) \\ r &= r_1 + r_2 + r_3 \end{aligned}$$

Here:

- r_1 is a region-based continuous reward that applies only near the target location of the robot.
- r_2 is a discrete signal that provides a large penalty when the robot moves far from the target location.
- r_3 is a continuous QR penalty that applies for all robot states.

Reward Generation from Control Specifications

For applications where a working control system already exists, specifications such as cost functions or constraints might already be available. In these cases, you can use `generateRewardFunction` to automatically generate a reward function, coded in MATLAB, that can be used as a starting point for reward design. This function allows you to generate reward functions from:

- Cost and constraint specifications defined in an `mpc` or `nlmpc` controller object. This feature requires Model Predictive Control Toolbox™ software.

- Performance constraints defined in Simulink Design Optimization™ model verification blocks.

In both cases, when constraints are violated, a negative reward is calculated using penalty functions such as `exteriorPenalty` (default), `hyperbolicPenalty` or `barrierPenalty` functions.

Starting from the generated reward function, you can tune the cost and penalty weights, use a different penalty function, and then use the resulting reward function within an environment to train an agent.

See Also

Functions

`generateRewardFunction` | `exteriorPenalty` | `hyperbolicPenalty` | `barrierPenalty` | `rlSimulinkEnv` | `rlCreateEnvTemplate` | `validateEnvironment`

Objects

`rlFunctionEnv` | `SimulinkEnvWithAgent`

Related Examples

- “Generate Reward Function from a Model Predictive Controller for a Servomotor” on page 8-214
- “Generate Reward Function from a Model Verification Block for a Water Tank System” on page 8-227

More About

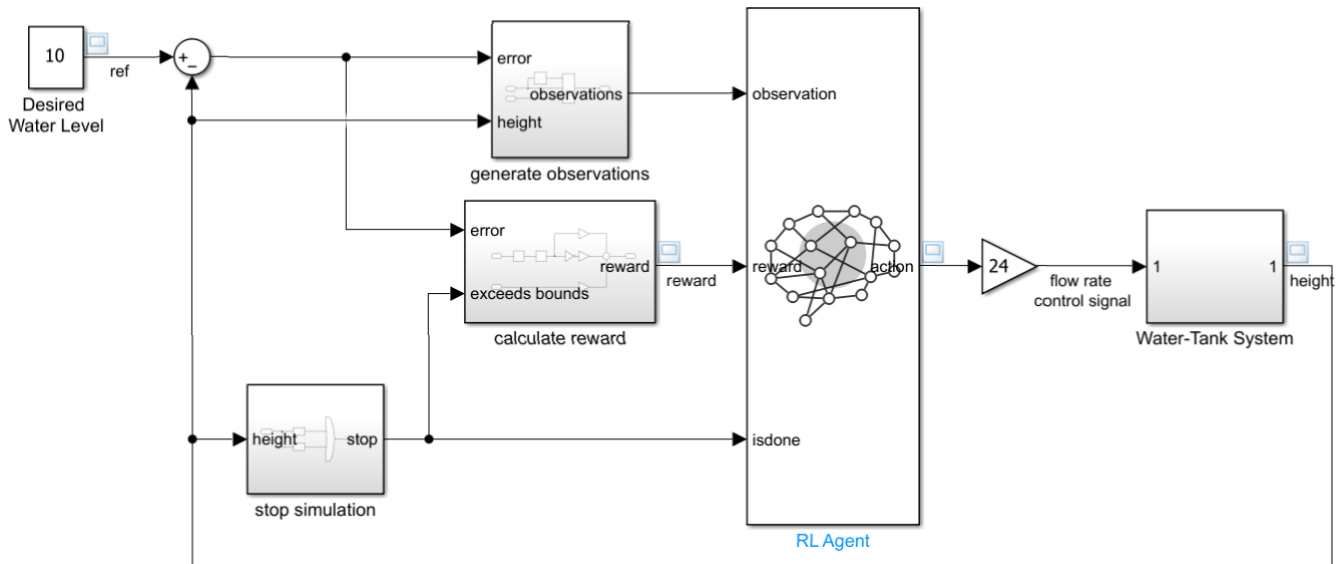
- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Simulink Environments” on page 2-110

Water Tank Custom Simulink Environment for Reinforcement Learning

This example illustrates the water tank Simulink® environment. This environment is based on the `rlwatertank` Simulink model, which contains an RL Agent block (instead of a controller) to control the water level in a tank.

To simulate this model, you must create an agent and specify that agent in the RL Agent block. For an example that trains an agent using this environment, see “Control Water Level in a Tank Using a DDPG Agent” on page 8-2.

```
mdl = "rlwatertank";
open_system(mdl)
```



This model already contains an RL Agent block, which connects to the following signals:

- Scalar action output signal
- Vector of observation input signals
- Scalar reward input signal
- Logical input signal for stopping the simulation

Action Specification

A reinforcement learning environment receives action signals from the agent and generates observation signals in response to these actions. To create a custom Simulink environment, first create action and observation specification objects. For more information about custom Simulink environments, see “Create Custom Simulink Environments” on page 2-110.

The action signal for this environment is the flow rate control signal that is sent to the plant, which is a continuous variable. To create a specification object for an action channel carrying a continuous signal, use `rlNumericSpec`. The flow-rate control signal is normalized and nonnegative, with a lower

limit of 0 and an upper limit of 1. This signal is scaled before being sent to the Water Tank System subsystem.

```
actInfo = rlNumericSpec([1 1], LowerLimit=0, UpperLimit=1);
actInfo.Name = "flow rate control signal";
```

If the action signal instead takes one of a discrete set of possible values, create the specification using the `rlFiniteSetSpec`.

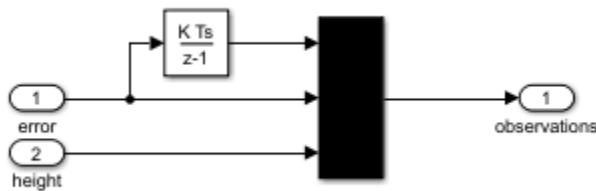
Observation Specification

For this environment, there are three observation signals sent to the agent, specified as a vector signal. The observation vector is $[f e dt e h]^T$, where:

- h is the height of the water in the tank.
- $e = r - h$, where r is the reference value for the water height.

Compute the observation signals in the generate observations subsystem.

```
open_system mdl + "/generate observations")
```



Create a three-element vector of observation specifications. Specify a lower bound of 0 for the water height, leaving the other observation signals unbounded.

```
obsInfo = rlNumericSpec([3 1], ...
    LowerLimit=[-inf -inf 0]', ...
    UpperLimit=[inf inf inf]');
obsInfo.Name = "observations";
obsInfo.Description = "integrated error, error, and measured height";
```

If the actions or observations are represented by bus signals, create specifications using the `bus2RLSpec` function.

Reward Signal

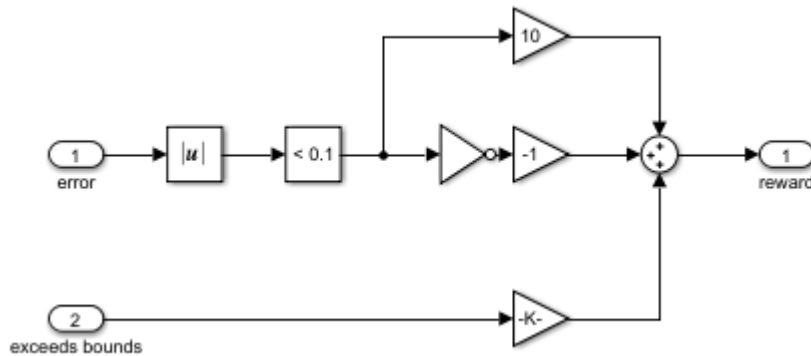
Construct a scalar reward signal. For this example, specify the following reward.

$$\text{reward} = 10(|e| < 0.1) - 1(|e| \geq 0.1) - 100(h \leq 0 || h \geq 20)$$

The reward is positive when the error is below 0.1 and negative otherwise. Also, there is a large reward penalty when the water height is outside the 0 to 20 range.

Construct this reward in the calculate reward subsystem.

```
open_system mdl + "/calculate reward")
```

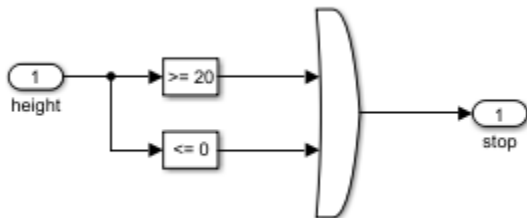


Stop Signal

To terminate training episodes and simulations, specify a logical signal to the `isdone` input port of the block. For this example, terminate the episode if $h \leq 0$ or $h \geq 20$.

Compute this signal in the stop simulation subsystem.

```
open_system mdl + "/stop simulation")
```



Create Environment Object

Create an environment object for the Simulink model.

```
env = rlSimulinkEnv mdl, mdl + "/RL Agent", obsInfo, actInfo);
```

Add Reset Function

You can also create a custom reset function that randomizes parameters, variables, or states of the model. In this example, the reset function `localResetFcn` on page 2-137 defined at the end of the example randomizes the reference signal and the initial water height and sets the corresponding block parameters.

```
env.ResetFcn = @(in) localResetFcn(in);
```

To make sure that the reset function always returns the same random initial value when it is called the first time in your script, fix the random stream generator before calling `train` or `sim`. For example, fix the random number stream with seed 0 and random number algorithm Mersenne Twister:

```
previousRngState = rng(0, "twister");
```

For more information on controlling the seed used for random number generation, see `rng`. For more information on result reproducibility, see “Results Reproducibility” on page 5-10.

Load Trained Agent and Simulate Environment

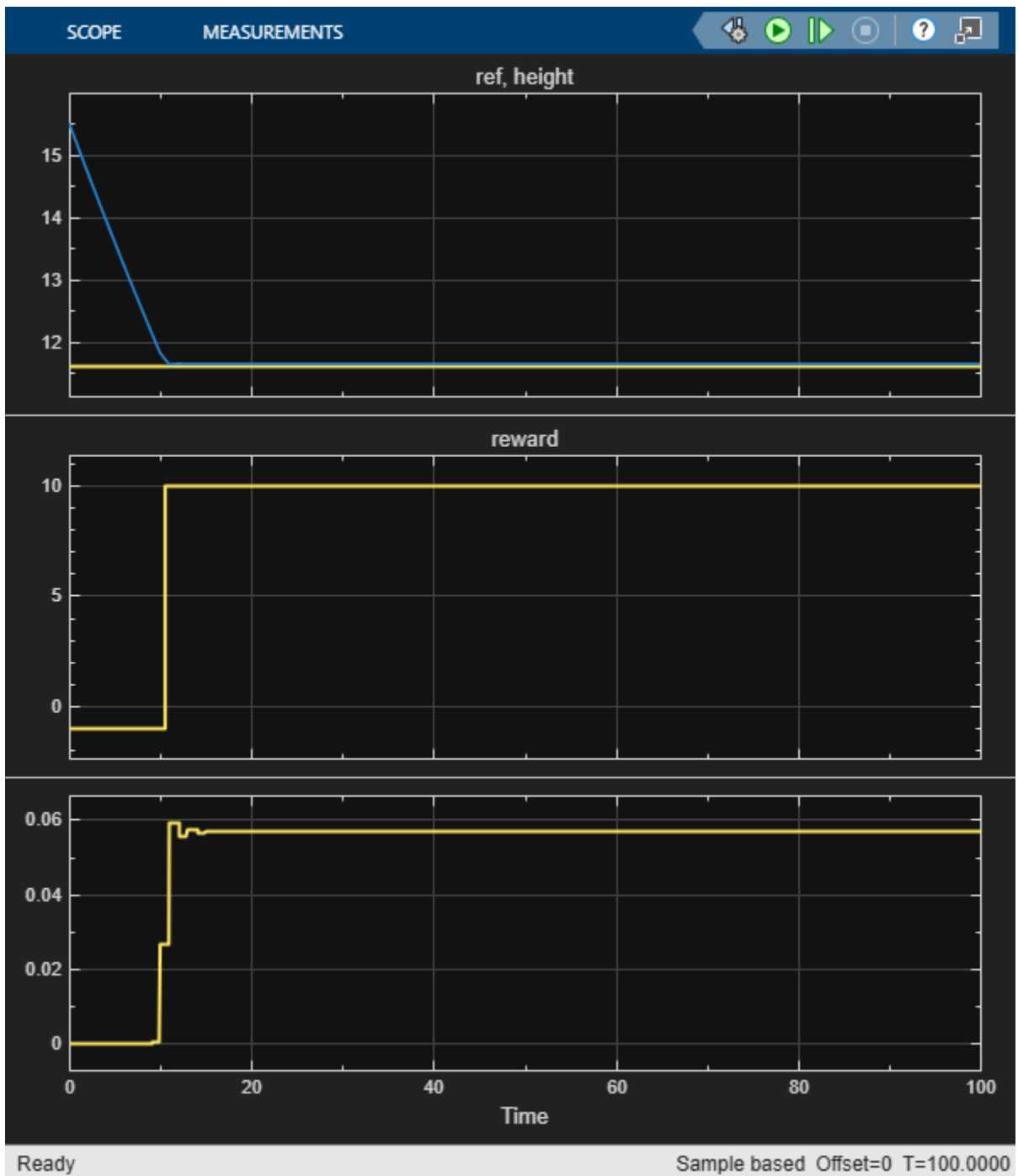
Load the DDPG agent trained in the example “Control Water Level in a Tank Using a DDPG Agent” on page 8-2.

```
load("WaterTankDDPG.mat","agent")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the agent within the environment for 100 steps, and return the experience as output. For more information, see `rlSimulationOptions` and `sim`.

```
simOpts = rlSimulationOptions(MaxSteps=100,StopOnError="on");  
experience = sim(env,agent,simOpts);
```



Display the total reward that the agent collects in the simulation.

```
sum(experience.Reward)
```

```
ans =  
890
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the reset function uses `setBlockParameter` (Simulink) to specify random values of the reference signal and the initial water height. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)

% Randomize reference signal
blk = sprintf("rlwatertank/Desired \nWater Level");
h = 3*randn + 10;
while h <= 0 || h >= 20
    h = 3*randn + 10;
end
in = setBlockParameter(in,blk,Value=num2str(h));

% Randomize initial water height
h = 3*randn + 10;
while h <= 0 || h >= 20
    h = 3*randn + 10;
end
blk = "rlwatertank/Water-Tank System/H";
in = setBlockParameter(in,blk,InitialCondition=num2str(h));

end
```

See Also

Functions

`rlSimulinkEnv`

Related Examples

- “Control Water Level in a Tank Using a DDPG Agent” on page 8-2

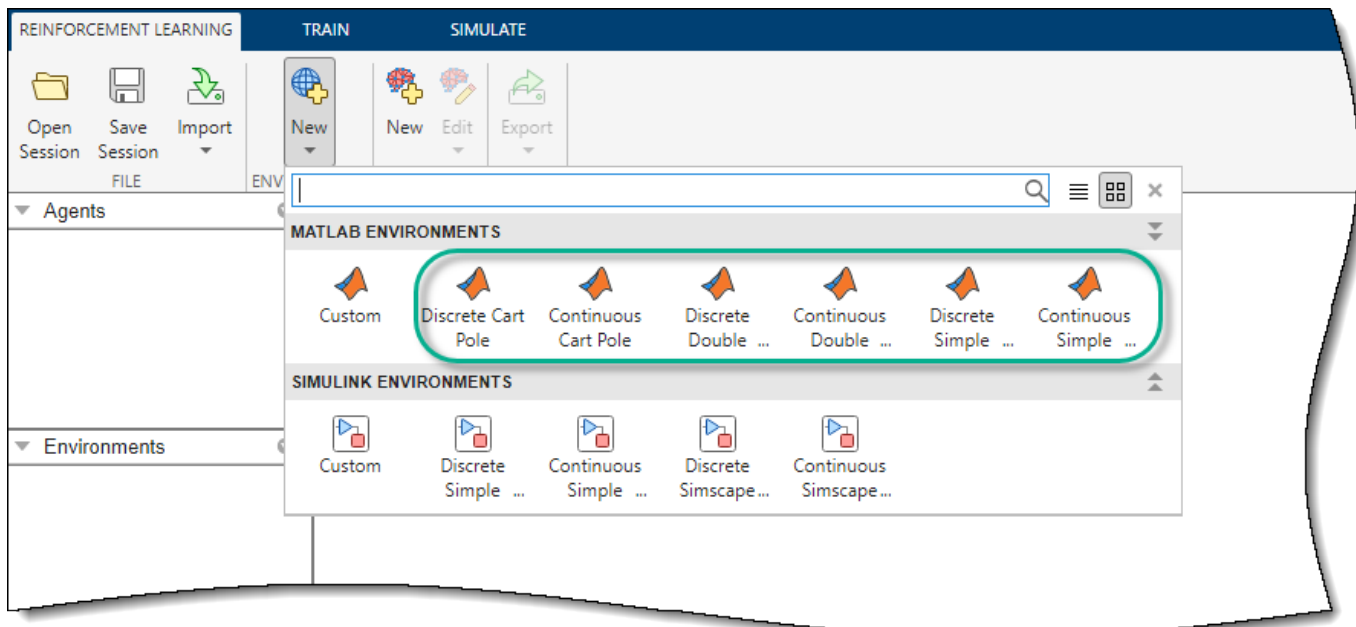
More About

- “Create Custom Simulink Environments” on page 2-110

Load MATLAB Environments in Reinforcement Learning Designer

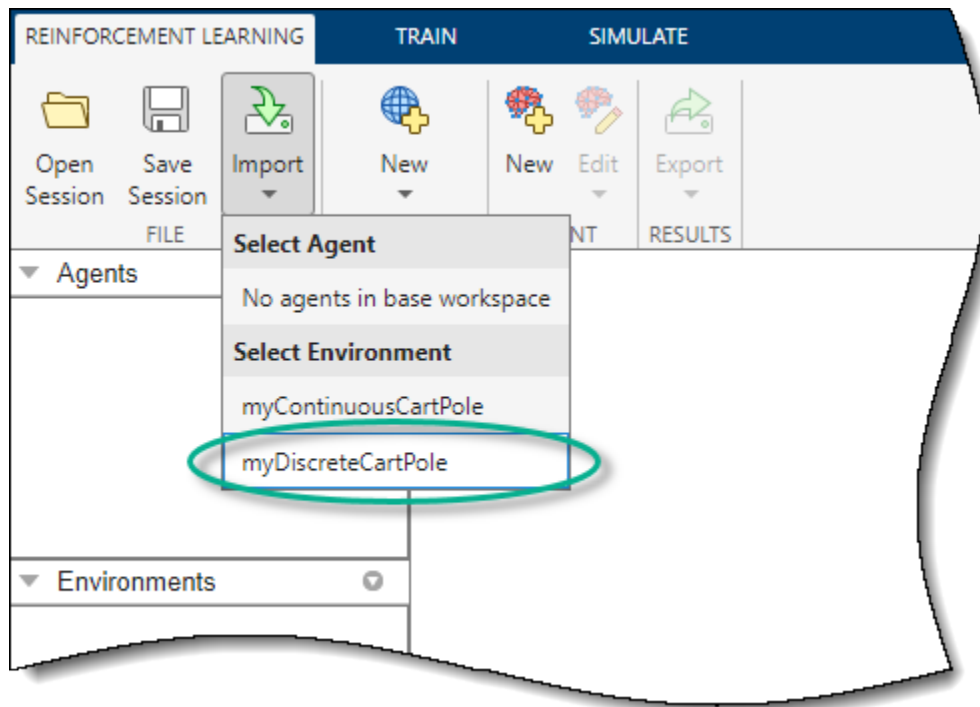
When training an agent using the Reinforcement Learning Designer app, you can create a predefined environment object from within the app or import a custom environment object from the MATLAB workspace.

To create a predefined environment object, on the **Reinforcement Learning** tab, in the **Environment** section, click **New**. Then, under **MATLAB Environments**, select one of the predefined environments.

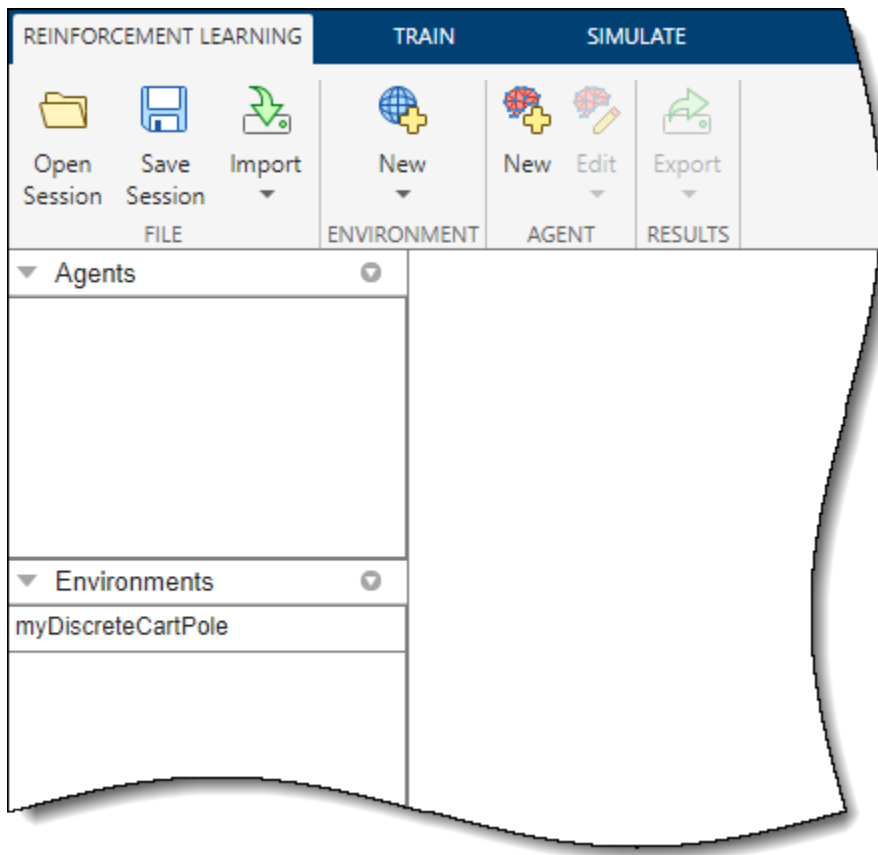


To use a custom environment, you must first create the environment object at the MATLAB command line and then import the object into Reinforcement Learning Designer. For an introduction to creating a custom environment, see “Reinforcement Learning Environments” on page 2-2.

Once you create a custom environment using one of the available methods, import the environment into Reinforcement Learning Designer. On the **Reinforcement Learning** tab, click **Import**. Then, under **Select Environment**, select the environment object.



Once you have created or imported an environment object, the app adds the environment to the **Environments** pane.



Once you have created an environment, you can create an agent for that environment. For more information on creating agents within Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

See Also

Functions

`rlPredefinedEnv` | `rlCreateEnvTemplate` | `validateEnvironment`

Objects

`rlFunctionEnv`

Apps

Reinforcement Learning Designer

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13

More About

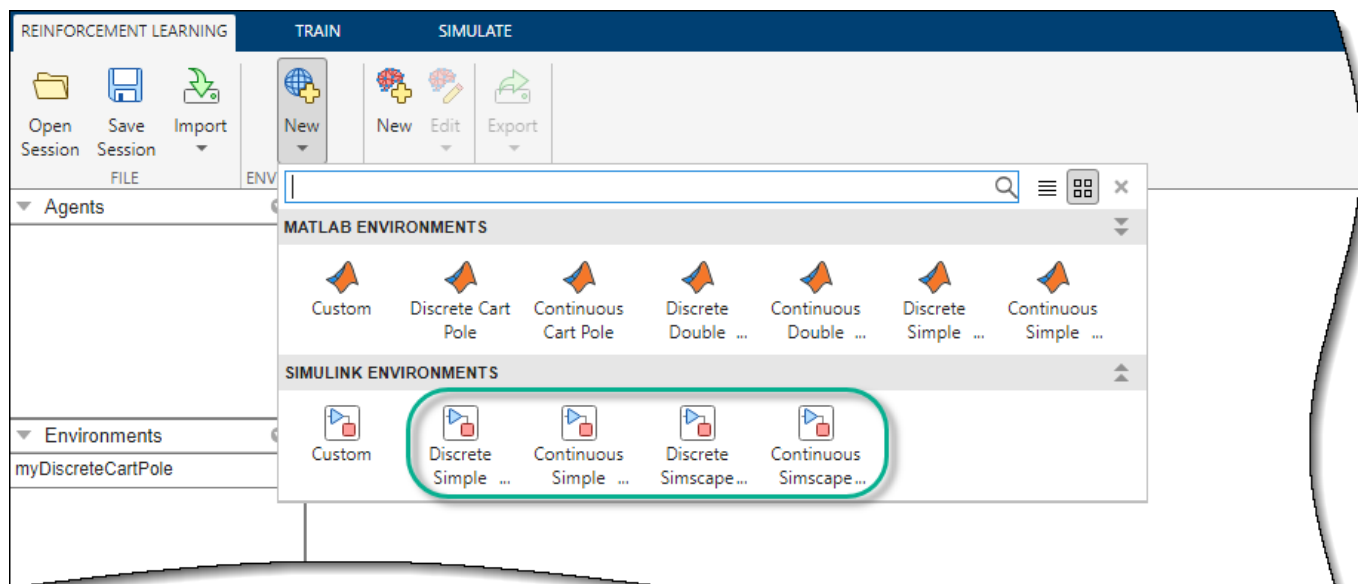
- “Reinforcement Learning Environments” on page 2-2
- “Use Predefined Control System Environments” on page 2-42

- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142
- “Create Agents Using Reinforcement Learning Designer” on page 3-11

Load Simulink Environments in Reinforcement Learning Designer

When training an agent using the Reinforcement Learning Designer app, you can create a predefined Simulink environment object from within the app or import a custom Simulink environment object from the MATLAB workspace.

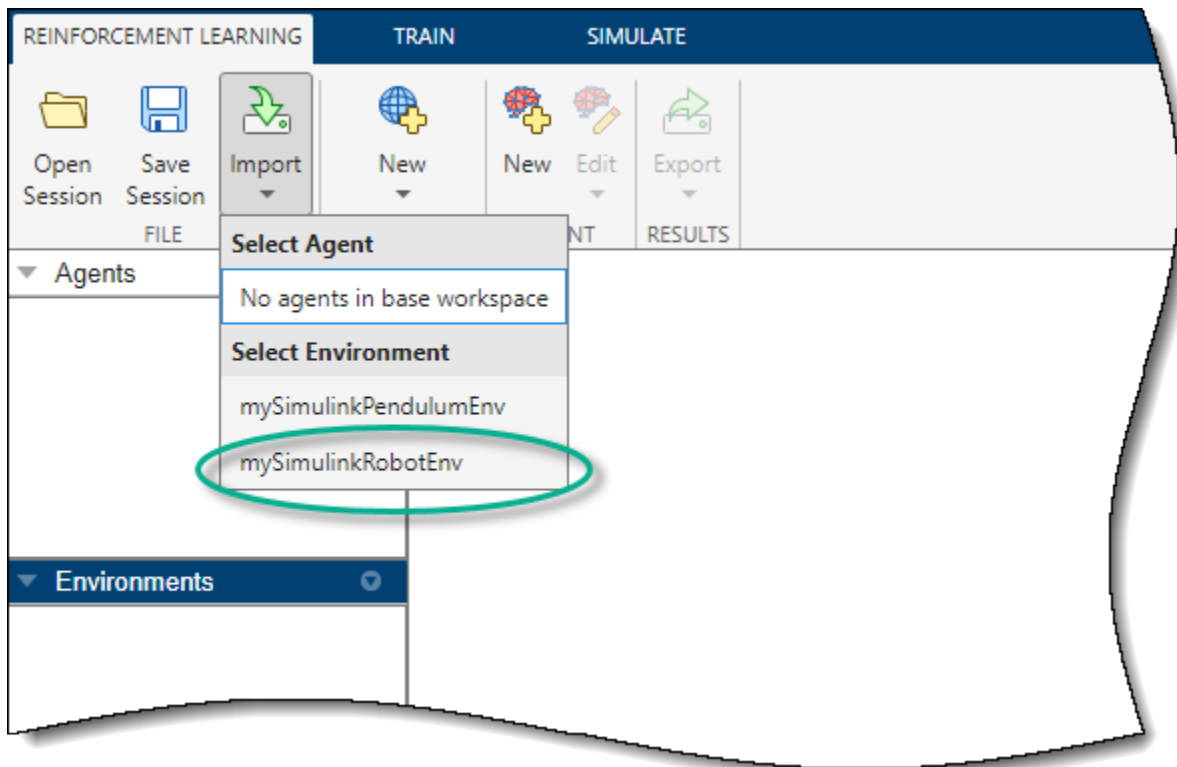
To create a predefined environment, on the **Reinforcement Learning** tab, in the **Environment** section, click **New**. Then, under **Simulink Environments**, select one of the predefined environments.



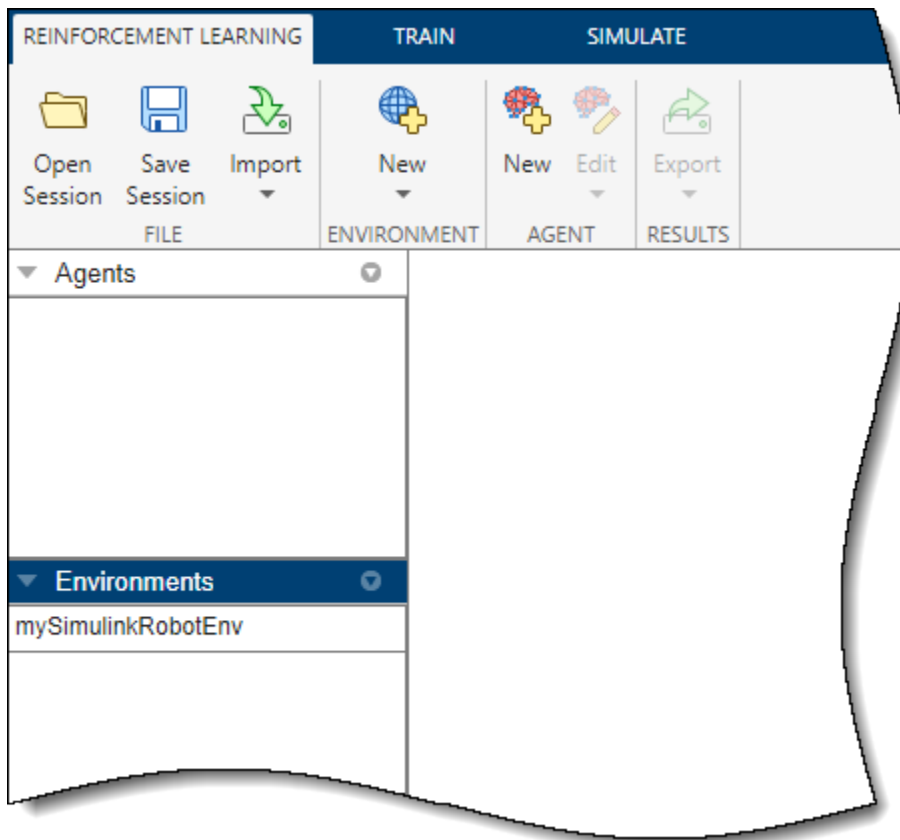
To use a custom environment, you must first create the environment at the MATLAB command line and then import the environment into Reinforcement Learning Designer. For more information on creating a Simulink environment, see “Create Custom Simulink Environments” on page 2-110.

For training and simulating Simulink environments, you must define all variables necessary for running the Simulink model in the MATLAB workspace.

Once you create a custom environment using one of the methods described in the preceding section, import the environment into Reinforcement Learning Designer. On the **Reinforcement Learning** tab, click **Import**. Then, under **Select Environment**, select the environment.



Once you have created or imported an environment, the app adds the environment to the **Environments** pane.



Once you have created an environment, you can create an agent to train in that environment. For more information on creating agent from within Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

See Also

Functions

`rlPredefinedEnv` | `rlSimulinkEnv` | `createIntegratedEnv` | `bus2RLSpec` | `validateEnvironment`

Objects

`SimulinkEnvWithAgent`

Apps

Reinforcement Learning Designer

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13

More About

- “Reinforcement Learning Workflow” on page 1-5

- “Reinforcement Learning Environments” on page 2-2
- “Create Custom Simulink Environments” on page 2-110
- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Create Agents Using Reinforcement Learning Designer” on page 3-11

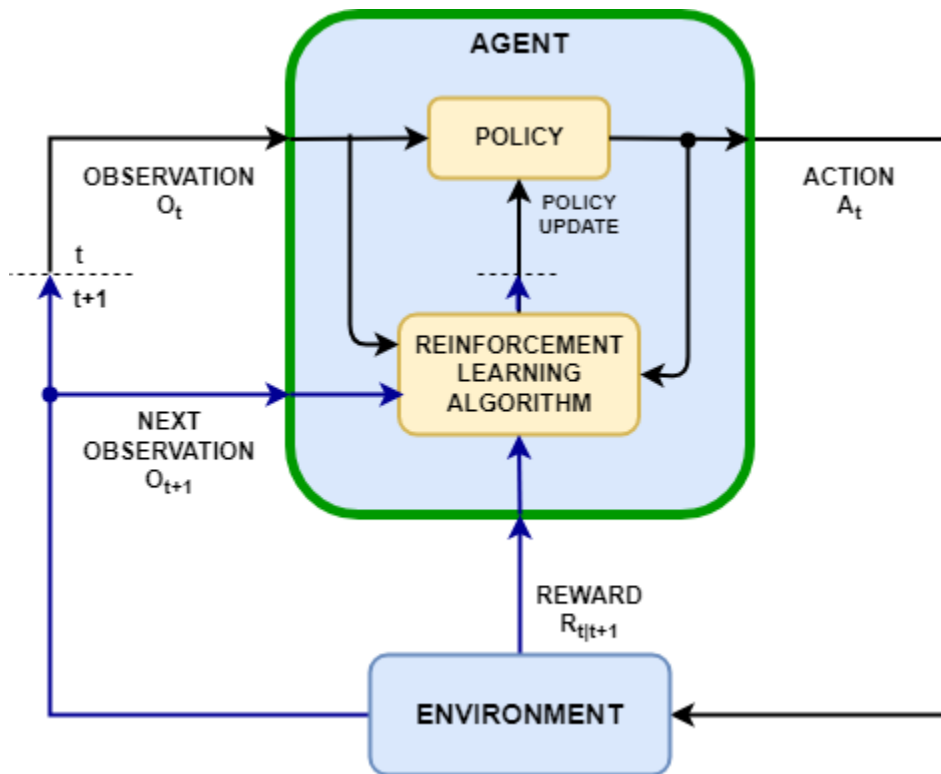
Create Agents

- “Reinforcement Learning Agents” on page 3-2
- “Create Agents Using Reinforcement Learning Designer” on page 3-11
- “Q-Learning Agent” on page 3-22
- “SARSA Agent” on page 3-25
- “LSPI Agent” on page 3-28
- “Deep Q-Network (DQN) Agent” on page 3-32
- “REINFORCE Policy Gradient (PG) Agent” on page 3-37
- “Actor-Critic (AC) Agent” on page 3-44
- “Proximal Policy Optimization (PPO) Agent” on page 3-50
- “Trust Region Policy Optimization (TRPO) Agent” on page 3-58
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Model-Based Policy Optimization (MBPO) Agent” on page 3-86
- “Create Custom Reinforcement Learning Agents” on page 3-92

Reinforcement Learning Agents

The goal of reinforcement learning is to train an agent to complete a task within an uncertain environment. At each time interval, the agent receives observations and a reward from the environment and sends an action to the environment. The reward is an immediate measure of how successful the previous action (taken from the previous state) was with respect to completing the task goal.

The agent and the environment interact at each of a sequence of *discrete* time steps as described in “Reinforcement Learning Environments” on page 2-2 and illustrated in the following figure.



By convention, the observation can be divided into one or more *channels* each of which carries a group of single elements all belonging to either a numeric (infinite and continuous) set or to a finite (discrete) set. Each group can be organized according to any number of dimensions (for example a vector or a matrix). Note that only one channel is allowed for the action, while the reward must be a numeric scalar. For more information on specification objects for actions and observations, see `rlFiniteSetSpec` and `rlNumericSpec`.

The agent contains two components: a policy and a learning algorithm.

- The policy is a mapping from the current environment observation to a probability distribution of the actions to be taken. Within an agent, the policy is implemented by a function approximator with tunable parameters and a specific approximation model, such as a deep neural network.
- The learning algorithm continuously updates the policy learnable parameters based on the actions, observations, and rewards. The goal of the learning algorithm is to find an optimal policy that maximizes the expected discounted cumulative long-term reward received during the task.

Depending on the agent, the learning algorithm operates on one or more parameterized function approximators that learn the policy. Approximators can be used in two ways.

- **Critics** — For a given observation and action, a critic returns an approximation of the policy value (that is, the policy expected discounted cumulative long-term reward).
- **Actor** — For a given observation, an actor returns the action that (often) maximizes the policy value.

Agents that use only critics to select their actions rely on an *indirect policy representation*. These agents are also referred to as *value-based*, and they use an approximator to represent a value function (value as a function of the observation) or Q-value function (value as a function of observation and action). In general, these agents work better with discrete action spaces but can become computationally expensive for continuous action spaces.

Agents that use only actors to select their actions rely on a *direct policy representation*. These agents are also referred to as *policy-based*. The policy can be either deterministic or stochastic. In general, these agents are simpler and can handle continuous action spaces, though the training algorithm can be sensitive to noisy measurement and can converge on local minima.

Agents that use both an actor and a critic are referred to as *actor-critic* agents. In these agents, during training, the actor learns the best action to take using feedback from the critic (instead of using the reward directly). At the same time, the critic learns the value function from the rewards so that it can properly criticize the actor. In general, these agents can handle both discrete and continuous action spaces.

For more information on actors, critics, and their theoretical underpinnings, see [1] and the references therein.

Agent Objects

Reinforcement Learning Toolbox represents agents with MATLAB objects. Such objects interact with environments using object functions (methods) such as `getAction`, which returns an action as output given an environment observation.

Note Agent objects are handle objects, so if you copy the agent into a new variable, the new variable will also always point to the most recent agent version with updated states and parameters. For more information about handle objects, see “Handle Object Behavior”.

After you create an agent object for a given environment in the MATLAB workspace, you can then use both the environment and agent variables as arguments for the built-in functions `train` and `sim`, which train or simulate the agent within the environment, respectively.

The software provides different types of built-in agents. For each of these you can manually configure its approximation objects (such as actor or critic) and models (such as neural networks or custom basis function). For most built-in agents you can also use a default network configuration. Alternatively, you can also create your custom agent object.

Built-In Agents

The following tables summarize the types, action spaces, and used approximators for all the built-in agents provided with Reinforcement Learning Toolbox software.

On-policy agents attempt to evaluate or improve the policy that they are using to make decisions, whereas off-policy agents evaluate or improve a policy that can be different from the one that they are using to make decisions, (or from the one that has been used to generate data). For each agent, the observation space can be discrete, continuous or mixed.

On-Policy Built-In Agents: Type and Action Space

Agent	Type	Action Space
"SARSA Agent" on page 3-25	Value-Based	Discrete
"REINFORCE Policy Gradient (PG) Agent" on page 3-37	Policy-Based	Discrete or continuous
"Actor-Critic (AC) Agent" on page 3-44	Actor-Critic	Discrete or continuous
"Trust Region Policy Optimization (TRPO) Agent" on page 3-58	Actor-Critic	Discrete or continuous
"Proximal Policy Optimization (PPO) Agent" on page 3-50	Actor-Critic	Discrete or continuous

Off-Policy Built-In Agents: Type and Action Space

Agent	Type	Action Space
"Q-Learning Agent" on page 3-22	Value-Based	Discrete
"LSPI Agent" on page 3-28	Value-Based	Discrete
"Deep Q-Network (DQN) Agent" on page 3-32	Value-Based	Discrete
"Deep Deterministic Policy Gradient (DDPG) Agent" on page 3-67	Actor-Critic	Continuous
"Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent" on page 3-72	Actor-Critic	Continuous
"Soft Actor-Critic (SAC) Agent" on page 3-78	Actor-Critic	Discrete, Continuous, or Hybrid
"Model-Based Policy Optimization (MBPO) Agent" on page 3-86	Actor-Critic	Discrete or continuous

Built-In Agents: Critics Used by Each Agent

Critic	Q, SARSA, DQN	PG	AC, PPO, TRPO	SAC	DDPG, TD3
Value function critic $V(S)$, which you can create using <code>rlValueFunction</code>		X (if baseline is used)	X		
Q-value function critic $Q(S,A)$, which you can create using <code>rlQValueFunction</code>	X			X (continuous action space SAC)	X
Multi-output Q-value function critic $Q(S)$, for discrete action spaces, which you can create using <code>rlVectorQValueFunction</code>	X			X (discrete and hybrid action space SAC)	

The LSPI agent must use a linear-in-the-parameters custom basis function, which you can create using `rlQValueFunction`.

Built-In Agents: Actors Used by Each Agent

Actor	Q, DQN, SARSA	PG	AC, PPO, TRPO	SAC	DDPG, TD3
Deterministic policy actor $\pi(S)$, which you can create using <code>rlContinuousDeterministicActor</code>					X
Stochastic (Multinoulli) policy actor $\pi(S)$, for discrete action spaces, which you can create using <code>rlDiscreteCategoricalActor</code>		X	X	X	
Stochastic (Gaussian) policy actor $\pi(S)$, for continuous action spaces, which you can create using <code>rlContinuousGaussianActor</code>		X	X		
Stochastic policy actor $\pi(S)$, for hybrid action spaces, which you can create using <code>rlHybridStochasticActor</code>			X	X	

Default Built-In Agents — A default built-in agent is an agent with default approximators and options, that is, an agent in which the actor and critics use default networks and have default options. You can create default agents of any type (except Q-learning, LSPI and SARSA) using just the

observation and action specifications from the environment. To do so, at the MATLAB command line, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options by creating an options object set for the specific agent. This option object in turn includes `rlOptimizerOptions` objects that specify optimization objects for the agent actor or critic.
- 5 Create the agent using the corresponding agent constructor function. The resulting agent contains the appropriate actor and critics listed in the previous table. The actor and critic use default agent-specific deep neural networks as internal approximators. The agent and approximator options are set to their default values.

Built-In Agents with Custom Approximators —Alternatively, you can create built-in agents using custom actor and critic approximators (typically approximators relying on custom neural networks). To do so, first you create your own approximators, and use them to create your actor and critic objects. Then, you use these actor and critic objects to create the agent. For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

You can also use the Reinforcement Learning Designer app to import an existing environment and interactively design DQN, DDPG, PPO, TRPO, TD3 and SAC agents. The app allows you to train and simulate the agent within your environment, analyze the simulation results, refine the agent parameters, and export the agent to the MATLAB workspace for further use and deployment. For more information, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

Choosing Agent Type

When choosing an agent, best practice is to start with a simpler (and faster to train) algorithm that is compatible with your action and observation spaces. You can then try progressively more complicated algorithms if the simpler ones do not perform as desired.

Note that PG and AC agents are earlier (and simpler) implementation of policy gradient and actor-critic concepts. While these agents might be relatively easier to tune and memory efficient, in general they have very little functional advantages with respect to PPO, SAC, TD3 and DQN, and are provided mostly for educational purposes.

Similarly, PPO generally performs better than TRPO along all dimensions (with TRPO also being particularly hard to tune). Furthermore, TRPO only supports actors and critics with deep networks for which higher order derivatives can be calculated (therefore, you cannot use actors or critics with recurrent networks, custom basis functions or tables within a TRPO agent).

- **Discrete action spaces** — For deterministic problems in which a satisfying linear-in-the-parameters approximation of a Q value function might exist (given a suitable set of features), the LSPI agent can learn a good policy quickly and in a relatively stable way, even when the state space is large.

For simple environments with a relatively small discrete action spaces, using a tabular approximator can be viable (provided that the observation space is also discrete). In these cases,

the Q-learning and SARSA agents are the simplest compatible agent, with Q-learning usually performing better in terms of training speed and SARSA being slightly more robust.

Because the number of state-action pairs increases exponentially with the number of states and actions, tabular approximation does not scale well to environments with large state and action spaces, because it requires increasing memory and training time as the state space grows larger. In these cases, or when part of your observation space is continuous, unless a custom basis function suffices, use a network approximator.

The agents with discrete action space that support neural approximators are compared in the following picture. The off-policy agents (DQN and SAC) are on top and the four on-policy agents (PPO, TRPO, AC and PG) are on the bottom.

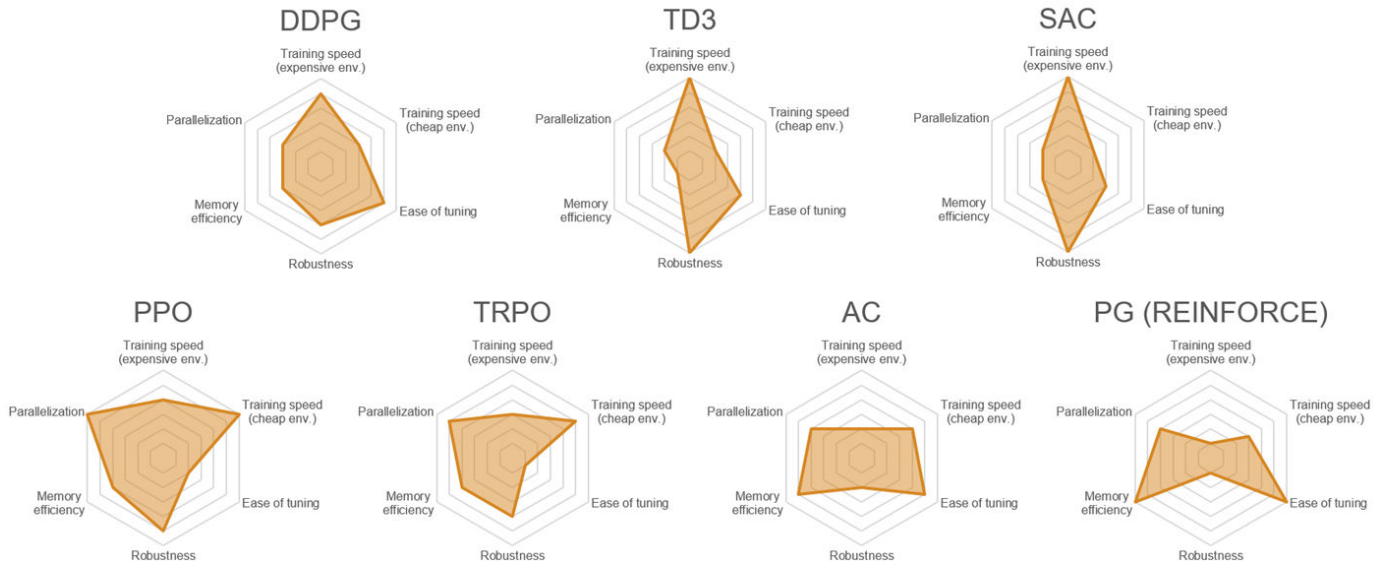


The agents are compared using six different metrics. There are two training speed metrics, for computationally expensive and computationally cheap environments, respectively. Ease of tuning is higher for agents that have less hyperparameters, or hyperparameters that are easier to understand and tune. Robustness is inversely related to the sensitivity to hyperparameters, weights, and initial conditions. Memory efficiency is inversely related to memory usage during training, and parallelization indicates how well the agent learning algorithm scales with the number of parallel workers.

The picture can be summarized as follows:

- DQN and PPO exhibit good performance overall.
- DQN is generally easier to tune (hence potentially a good starting point) and a relatively good choice for computationally expensive environments.
- PPO and SAC have been developed recently, as indicated by the red symbol.
- PPO tends to perform better in terms of parallelization and training speed for computationally cheap environments.
- SAC tends to perform better in terms of robustness and training speed for computationally expensive environments.

- **Continuous action spaces** — The agents with continuous action space that support neural approximators are compared in the following picture, with the three off-policy agents on the top and the four on-policy agents on the bottom.



As for the previous picture, the first two metrics are the training speed for computationally expensive and computationally cheap environments, respectively. The ease of tuning is higher for agents that have less hyperparameters, or hyperparameters that are easier to understand and tune. Robustness is inversely related to the sensitivity to hyperparameters, weights, and initial conditions. Memory efficiency is inversely related to memory usage during training, and parallelization indicates how well the agent learning algorithm scales with the number of parallel workers.

The picture for agents that support continuous action spaces can be summarized as follows:

- DDPG is the easiest to tune, followed by TD3, SAC and PPO.
- DDPG is a good starting point, and tends to perform well overall.
- Both TD3 and SAC are improved, more complex, and robust versions of DDPG, and are excellent choices for computationally expensive environments.
- PPO is harder to tune but generally performs well overall. In particular, it is an excellent choice for computationally cheap environments, and is highly parallelizable.
- SAC, which generates stochastic policies that can be useful for exploration, is slightly harder to tune but can be more memory efficient. This agent tends to perform very well for many environments.
- **Hybrid action spaces** — Only SAC supports hybrid action spaces (that is, action spaces containing both a discrete and a continuous part). Hybrid action spaces are specified by two action channels, a discrete one and a continuous one. You can define these channels using a vector containing one `rlFiniteSetSpec` object followed by one `rlNumericSpec` object.

Model-Based Policy Optimization

If you are using an off-policy agent different than Q-learning, (DQN, DDPG, TD3, SAC), you can consider using a model-based policy optimization (MBPO) agent to improve your sample efficiency

during training. An MBPO agent contains an internal model of the environment, which it uses to generate additional experiences without interacting with the environment.

During training, the MBPO agent generates real experiences by interacting with the environment. These experiences are used to train the internal environment model, which is used to generate additional experiences. The training algorithm then uses both the real and generated experiences to update the agent policy.

An MBPO agent can be more sample efficient than model-free agents because the model can generate large sets of diverse experiences. However, MBPO agents require much more computational time than model-free agents, because they must train the environment model and generate samples in addition to training the base agent.

For more information, see “Model-Based Policy Optimization (MBPO) Agent” on page 3-86.

Extract Policy Objects from Agents

You can extract a policy object from an agent and then use the `getAction` function to generate deterministic or stochastic actions from the policy, given an input observation. Working with policy objects can be useful for application deployment or custom training purposes. For more information, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Custom Agents

You can also train policies using other learning algorithms by creating a custom agent. Creating a custom agent allows you to use built-in functions `train` and `sim`, which can train or simulate your agent. To do so, you create a subclass of a custom agent class, and define the agent behavior using a set of required and optional methods. For more information, see “Create Custom Reinforcement Learning Agents” on page 3-92.

Alternatively, to implement a custom learning algorithm which does not rely on `train` or `sim`, you can create a custom training loop. For more information about custom training loops, see “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

References

- [1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Objects

`rlQAgent` | `rlSARSAAgent` | `rlLSPIAgent` | `rlDQNAgent` | `rlPGAgent` | `rlDDPGAgent` | `rlTD3Agent` | `rlACAgent` | `rlSACAgent` | `rlPPOAgent` | `rlTRPOAgent` | `rlMBPOAgent`

Related Examples

- “Train Reinforcement Learning Agents” on page 5-3

More About

- “What Is Reinforcement Learning?” on page 1-3
- “Reinforcement Learning Workflow” on page 1-5
- “Create Agents Using Reinforcement Learning Designer” on page 3-11

Create Agents Using Reinforcement Learning Designer

The Reinforcement Learning Designer app supports the following types of agents.

- “Deep Q-Network (DQN) Agent” on page 3-32 (DQN)
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67 (DDPG)
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72 (TD3)
- “Proximal Policy Optimization (PPO) Agent” on page 3-50 (PPO)
- “Trust Region Policy Optimization (TRPO) Agent” on page 3-58 (TRPO)
- “Soft Actor-Critic (SAC) Agent” on page 3-78 (SAC)

To train an agent using Reinforcement Learning Designer, you must first create or import an environment. For more information, see “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138 and “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142.

Create Agent

To create an agent, on the **Reinforcement Learning** tab, in the **Agent** section, click **New**.

Create an agent compatible with the specified environment. The agent is initialized with a default neural network structure, which you can modify after creation.

Agent name:

Environment:

Compatible algorithm:

Neural network parameterizations

Number of hidden units:

☐ Use recurrent neural network

Diagram illustrating the interaction between the RL Agent and the Environment:

```

graph TD
    subgraph RL_Agent [RL Agent]
        Policy[Policy]
        Algorithm[Algorithm]
        Policy -- "Policy Update" --> Algorithm
    end
    Environment[Environment]
    Environment -- "Observations" --> Policy
    Policy -- "Actions" --> Environment
    Environment -- "Reward" --> Algorithm
  
```

Buttons: Help, OK, Cancel

In the Create agent dialog box, specify the following information.

- **Agent name** — Specify the name of your agent.
- **Environment** — Select an environment that you previously created or imported.
- **Compatible algorithm** — Select an agent training algorithm. This list contains only algorithms that are compatible with the environment you select.

The Reinforcement Learning Designer app creates agents with actors and critics based on default deep neural network. You can specify the following options for the default networks.

- **Number of hidden units** — Specify number of units in each fully-connected or LSTM layer of the actor and critic networks.
- **Use recurrent neural network** — Select this option to create actor and critic with recurrent neural networks that contain an LSTM layer.

To create the agent, click **OK**.

The app adds the new default agent to the **Agents** pane and opens a document for editing the agent options.

The screenshot shows the Reinforcement Learning Designer app interface. The top menu bar includes REINFORCEMENT LEARNING, TRAIN, SIMULATE, and DQN AGENT. The left sidebar contains panes for Agents, Environments, Results, and Preview. The main area displays the configuration for 'agent1' under the 'DQN AGENT' tab.

Hyperparameters

Agent Options		Critic Optimizer Options	
Sample time	1	Learn rate	0.01
Discount factor	0.99	Gradient threshold	Inf
Execution environment	☒ CPU ☐ GPU	More Options	
Batch size	64		
Experience buffer length	1e+04		

Exploration

Epsilon Greedy Exploration Options	
Initial epsilon	1
Epsilon decay	0.005
Epsilon min	0.01

Plot Options

X-axis limit	1000
--------------	------

Epsilon Decay Plot

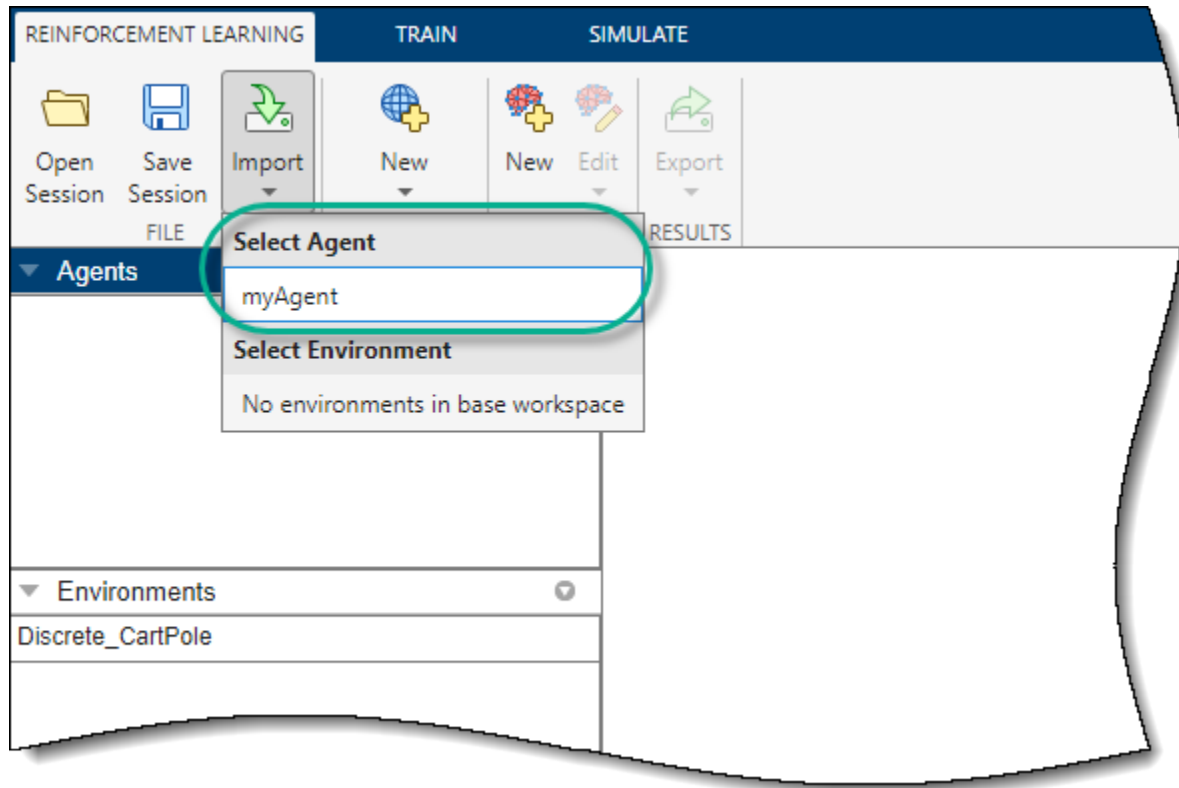
The plot shows the decay of epsilon over 1000 steps. The y-axis is 'Value' (0 to 1) and the x-axis is 'Steps' (0 to 1000). A blue line represents 'Epsilon' and a yellow line represents 'Epsilon min'.

Steps	Epsilon	Epsilon min
0	1.0	0.01
100	0.6	0.01
200	0.4	0.01
300	0.3	0.01
400	0.25	0.01
500	0.2	0.01
600	0.18	0.01
700	0.16	0.01
800	0.15	0.01
900	0.14	0.01
1000	0.13	0.01

Training result deleted: "trainStats1".

Import Agent

You can also import an agent from the MATLAB workspace into Reinforcement Learning Designer. To do so, on the **Reinforcement Learning** tab, click **Import**. Then, under **Select Agent**, select the agent to import.



The app adds the new imported agent to the **Agents** pane and opens a document for editing the agent options.

Edit Agent Options

In Reinforcement Learning Designer, you can edit agent options in the corresponding agent document.

The screenshot shows a web-based configuration interface for two agents, 'agent1' and 'agent2'. The 'agent2' tab is active. The interface is divided into three main sections: 'Agent Options', 'Actor Optimizer Options', and 'Critic Optimizer Options'. Each section contains various parameters that can be edited via text inputs, dropdowns, or checkboxes. At the bottom, a status bar indicates 'Agent opened: "agent2"'.
Agent Options:
 Sample time: 1
 Discount factor: 0.99
 Execution environment: ☒ CPU ☐ GPU
 Batch size: 128
 Experience horizon: 512
 Entropy loss weight: 0.01
More Options:
☒ Use exploration policy
 Clip factor: 0.2
 Number of epochs: 3
 Advantage estimation method: ☒ GAE ☐ Finite horizon
 GAE factor: 0.95
 Advantage normalization method: none
Actor Optimizer Options:
 Learn rate: 0.01
 Gradient threshold: Inf
More Options:
 Optimizer: adam
 Denominator offset: 1e-08
 Gradient decay: 0.9
 Squared gradient decay: 0.999
 Gradient threshold method: l2norm
 L2 regularization: 0.0001
Critic Optimizer Options:
 Learn rate: 0.01
 Gradient threshold: Inf
More Options:

You can edit the following options for each agent.

- **Agent Options** — Agent options, such as the sample time and discount factor. Specify these options for all supported agent types.
- **Exploration Model** — Exploration model options. PPO agents do not have an exploration model.
- **Target Policy Smoothing Model** — Options for target policy smoothing, which is supported for only TD3 agents.

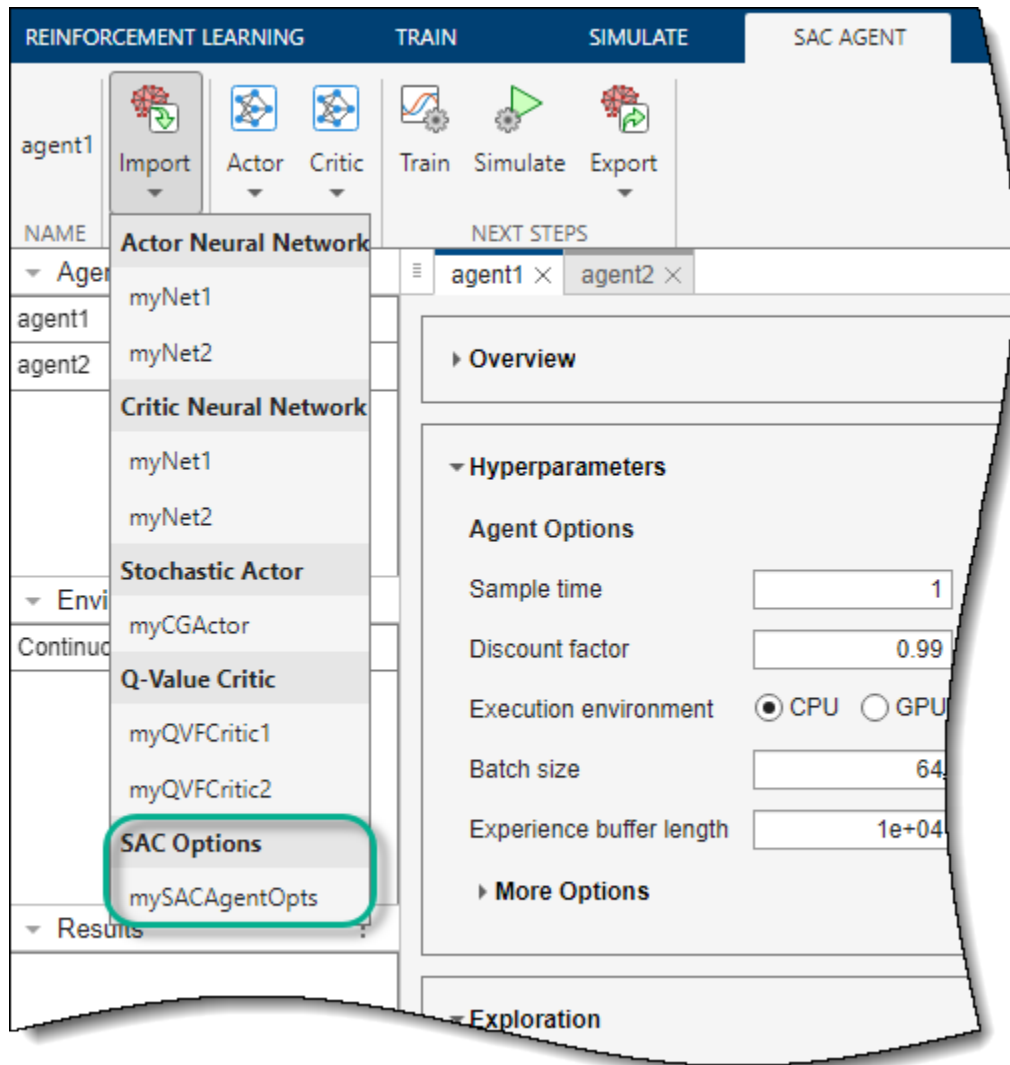
For more information on these options, see the corresponding agent options object.

- `r1DQNAgentOptions` — DQN agent options
- `r1PPOAgentOptions` — PPO agent options
- `r1TRPOAgentOptions` — TRPO agent options
- `r1DDPGAgentOptions` — DDPG agent options
- `r1TD3AgentOptions` — TD3 agent options
- `r1SACAgentOptions` — SAC agent options

Import Agent Options

You can import agent options from the MATLAB workspace. To create options for each type of agent, use one of the preceding objects. You can also import options that you previously exported from the Reinforcement Learning Designer app.

To import the options, on the corresponding **Agent** tab, click **Import**. Then, under **Options**, select an options object. The app lists only compatible options objects from the MATLAB workspace.



The app configures the agent options to match those in the selected options object.

Import Actor and Critic

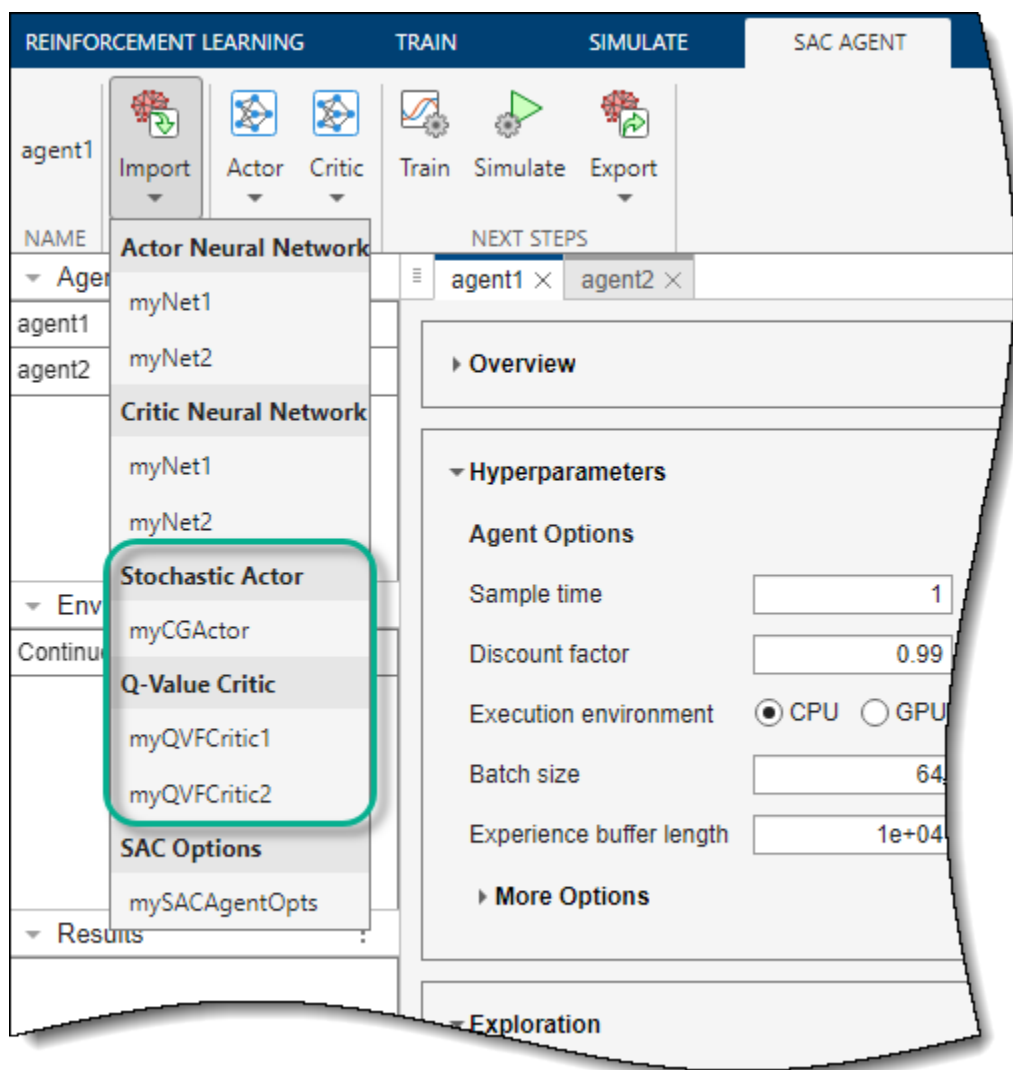
You can import actors and critics from the MATLAB workspace. You can also import actors and critics that you previously exported from the Reinforcement Learning Designer app.

- DQN agents have one critic.

- DDPG, PPO and TRPO agents have an actor and a critic.
- TD3 and SAC agents have an actor two critics. When you modify the critic options, the changes apply to both critics.

For more information on creating actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

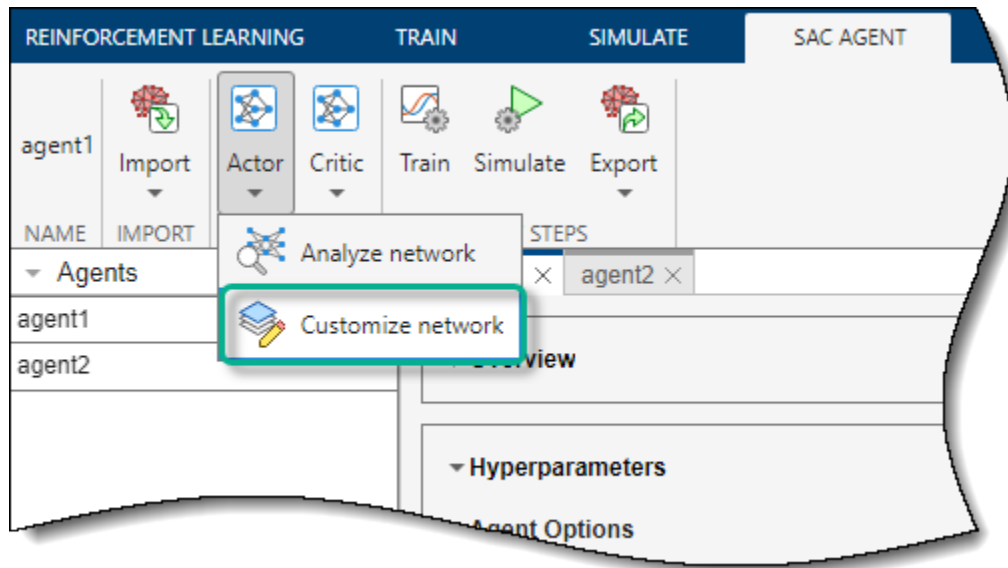
To import an actor or critic, on the corresponding **Agent** tab, click **Import**. Then, under either **Actor** or **Critic**, select an actor or critic object with action and observation specifications that are compatible with the specifications of the agent.



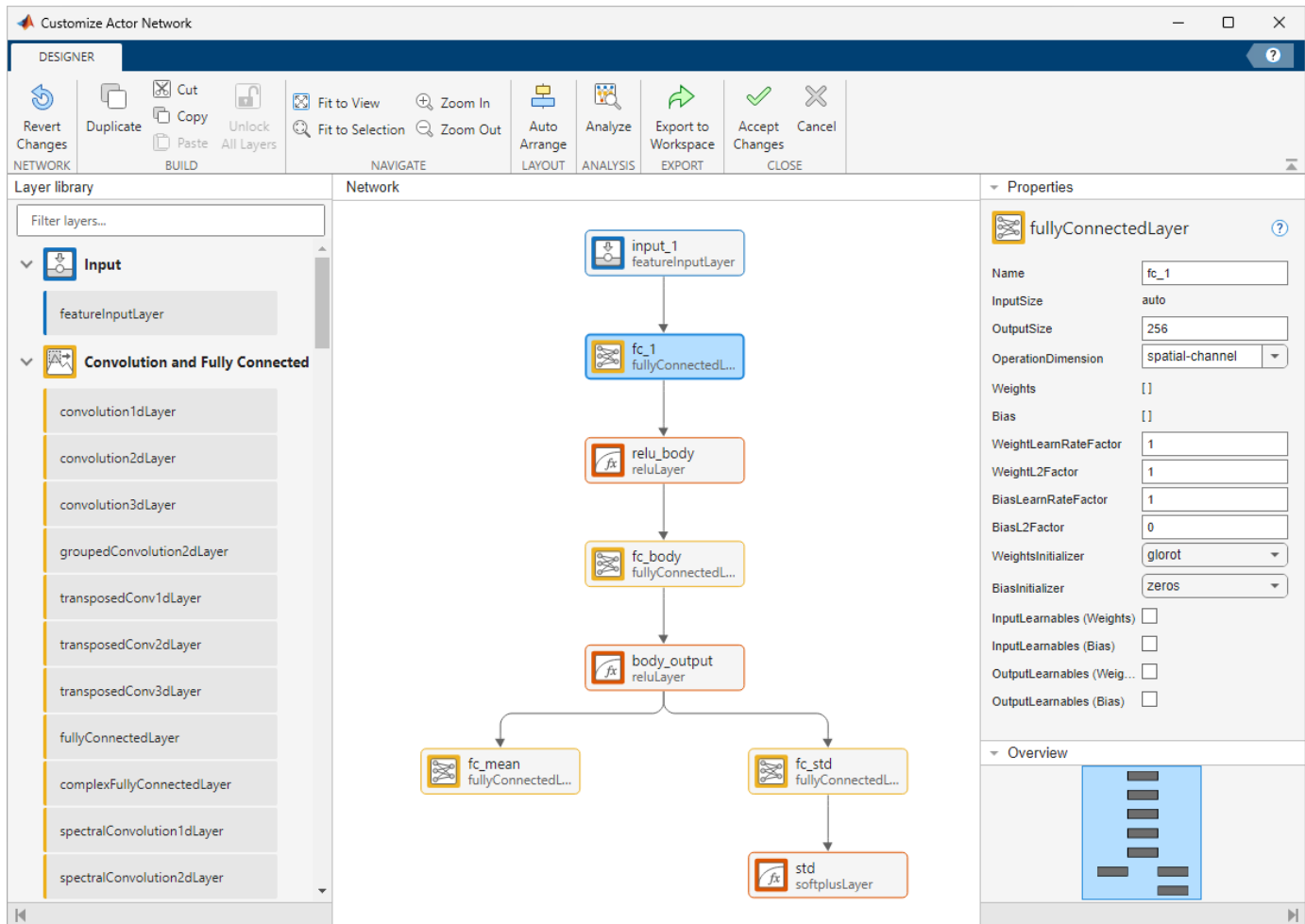
The app replaces the existing actor or critic in the agent with the selected one. If you import a critic for a SAC or TD3 agent, the app replaces the network for both critics.

Edit Actor or Critic Network

You can modify an actor or critic network, on the corresponding **Agent** tab, under **Actor** or **Critic**, select **Customize Network**.



The Deep Network Designer opens and displays the network, which you can modify.



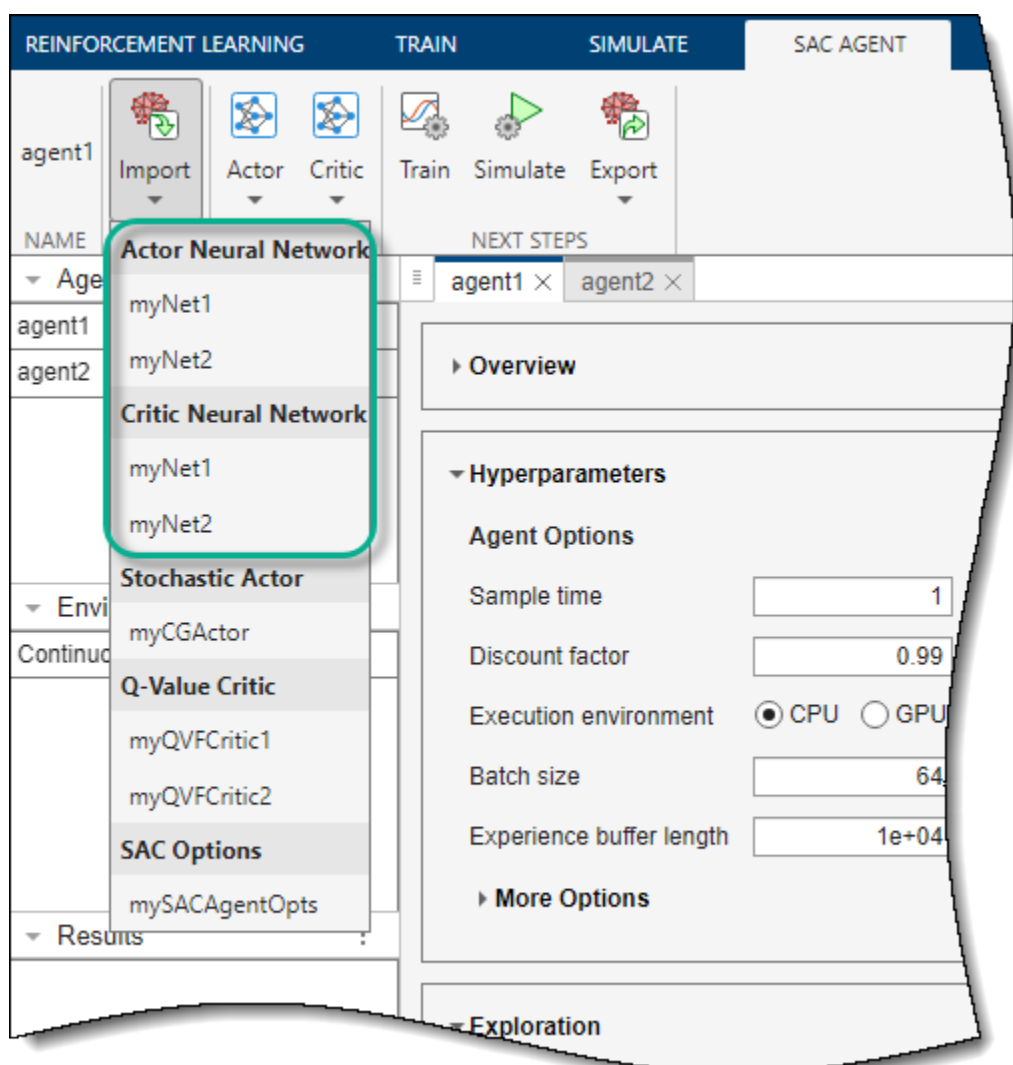
Click on **Accept Changes** to save the network and close the Deep Network Designer. The app then updates the deep neural network in the corresponding actor or critic.

Another strategy to customize your actor or critic network is to export the agent default network to the MATLAB workspace, modify it using the Deep Network Designer app (outside the Reinforcement Learning Designer app), and then import it back into Reinforcement Learning Designer.

For more information on creating deep neural networks for actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Import Actor or Critic Network

You can import an actor or critic network from the MATLAB workspace. To do so, on the corresponding **Agent** tab, click **Import**. Then, under either **Actor Neural Network** or **Critic Neural Network**, select a network with input and output layers that are compatible with the observation and action specifications of the agent.



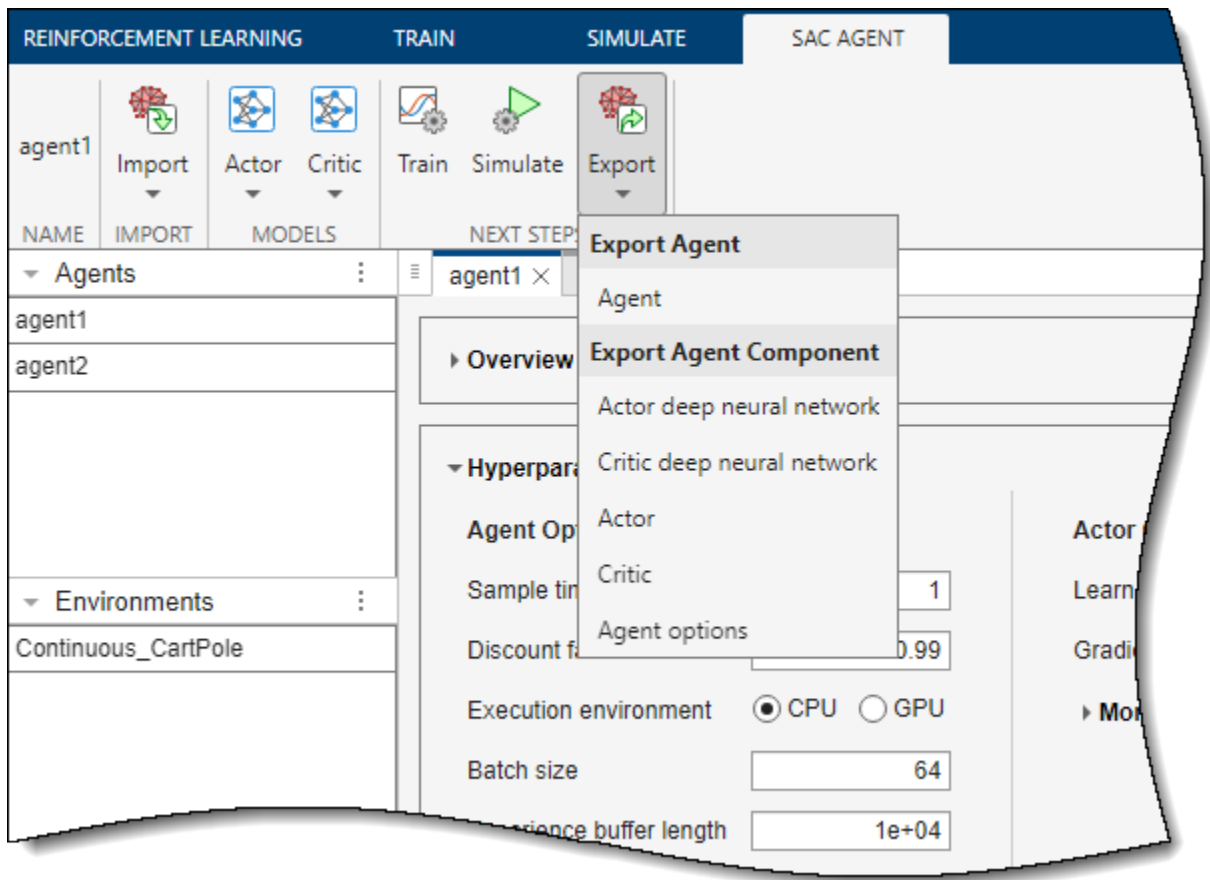
The app replaces the deep neural network in the corresponding actor or agent. If you import a critic network for a TD3 agent, the app replaces the network for both critics.

Export Agents and Agent Components

For a given agent, you can export any of the following to the MATLAB workspace.

- Agent
- Agent options
- Actor or critic
- Deep neural network in the actor or critic

To export an agent or agent component, on the corresponding **Agent** tab, click **Export**. Then, select the item to export.



The app saves a copy of the agent or agent component in the MATLAB workspace.

See Also

Apps

Reinforcement Learning Designer

Functions

analyzeNetwork

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53

More About

- “Reinforcement Learning Agents” on page 3-2
- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142
- “Specify Training Options in Reinforcement Learning Designer” on page 5-29

- “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50
- “Deep Q-Network (DQN) Agent” on page 3-32

Q-Learning Agent

The Q-learning algorithm is an off-policy reinforcement learning method for environments with a discrete action space. A Q-learning agent trains a Q-value function critic to estimate the value of the optimal policy, while following an epsilon-greedy policy based on the value estimated by the critic (it does not try to directly learn an optimal policy). For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a Q-learning agent is implemented by an `rlQAgent` object.

Note Q-learning agents do not support recurrent networks.

Q-learning agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete

Q agents use the following critic.

Critic	Actor
Q-value function critic $Q(S,A)$, which you create using <code>rlQValueFunction</code> or <code>rlVectorQValueFunction</code>	Q agents do not use an actor

During training, the agent explores the action space using epsilon-greedy exploration. During each control interval the agent selects a random action with probability ϵ , otherwise it selects the action for which the action-value function is greatest with probability $1-\epsilon$.

Critic Used by the Q-Learning Agent

To estimate the value of the optimal policy, a Q-learning agent uses a critic. The critic is a function approximator object that implements the parameterized action-value function $Q(S,A;\phi)$, using parameters ϕ . For a given observation S and action A , the critic stores the corresponding estimate of the expected discounted cumulative long-term reward when following the optimal policy (this is the value of the optimal policy). During training, the critic tunes the parameters in ϕ to improve its action-value function estimation. After training, the parameters remain at their tuned values in the critic internal to the trained agent.

For critics that use table-based value functions, the parameters in ϕ are the actual $Q(S,A)$ values in the table.

For more information on critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Q-Learning Agent Creation

To create a Q-learning agent:

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.

- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your critic. Depending on the type of problem and on the specific critic you use in the next step, this model can be an `rlTable` object (only for discrete observation spaces), a custom basis function with initial parameter values, or a neural network object. The inputs and outputs of the model you create depend on the type of critic you use in the next step.
- 4 Create a critic using `rlQValueFunction` or `rlVectorQValueFunction`. Use the model you created in the previous step as a first input argument.
- 5 Specify agent options using an `rlQAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 6 Create the agent using `rlQAgent`.

Q-Learning Agent Initialization

When you create a Q-Learning agent, the critic $Q(S,A;\phi)$ uses random parameter values in ϕ .

The agent uses this initial critic parameters at the beginning of the first training session. For each subsequent training session, the critic retains the parameters from the previous session.

Q-Learning Algorithm

The Q-learning agent uses the following training algorithm. To configure the training algorithm, specify options using an `rlQAgentOptions` object.

- For each training episode:
 - 1 At the beginning of the episode, get the initial observation from the environment.
 - 2 Repeat the following operations for each step of the episode until S is a terminal state.
 - a For the current observation S , select a random action A with probability ϵ . Otherwise, select the action for which the critic value function is greatest.

$$A = \underset{A}{\operatorname{argmax}} Q(S, A; \phi)$$

To specify ϵ and its decay rate, use the `EpsilonGreedyExploration` option.

- b Execute action A . Observe the reward R and the next observation S' .
- c If S' is a terminal state, set the value function target y to R . Otherwise, set it to

$$y = R + \gamma \underset{A}{\operatorname{max}} Q(S', A; \phi)$$

To set the discount factor γ , use the `DiscountFactor` option.

- d Compute the difference ΔQ between the value function target and the current $Q(S,A;\phi)$ value.

$$\Delta Q = y - Q(S, A; \phi)$$

- e Update the critic using the learning rate α . Specify the learning rate when you create the critic by setting the `LearnRate` option in the `rlCriticOptimizerOptions` property within the agent options object.

- For table-based critics, update the corresponding $Q(S,A)$ value in the table.

$$Q(S, A) = Q(S, A; \phi) + \alpha \cdot \Delta Q$$

- For all other types of critics, compute the gradients $\Delta\phi$ of the loss function with respect to the parameters ϕ . Then, update the parameters based on the computed gradients. In this case, the loss function is the square of ΔQ .

$$\Delta\phi = \frac{1}{2} \nabla_{\phi} (\Delta Q)^2$$

$$\phi = \phi + \alpha \cdot \Delta\phi$$

- f** If ϵ is greater than its minimum value, perform the decay operation as described in `EpsilonGreedyExploration`.
- g** Set the observation S to S' .

References

- [1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Objects

`rlQAgent` | `rlQAgentOptions` | `rlQValueFunction` | `rlVectorQValueFunction` | `rlSARSAgent` | `rlLSPIAgent` | `rlDQNAgent`

Related Examples

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Compare Agents on Deterministic Waterfall Grid World” on page 7-36

More About

- “Reinforcement Learning Agents” on page 3-2
- “SARSA Agent” on page 3-25
- “LSPI Agent” on page 3-28
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

SARSA Agent

The SARSA algorithm is an on-policy reinforcement learning method for environments with a discrete action space. A SARSA agent trains a value function based critic to estimate the expected discounted cumulative long-term reward of the current policy. Therefore, SARSA is the on-policy version of Q-learning. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a SARSA agent is implemented by an `rlSARSAAgent` object.

Note SARSA agents do not support recurrent networks.

SARSA agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete

SARSA agents use the following critic.

Critic	Actor
Q-value function critic $Q(S,A)$, which you create using <code>rlQValueFunction</code> or <code>rlVectorQValueFunction</code>	SARSA agents do not use an actor.

During training, the agent explores the action space using epsilon-greedy exploration. During each control interval the agent selects a random action with probability ϵ , otherwise it selects the action for which the action-value function is greatest with probability $1-\epsilon$.

Critic Used by the SARSA Agent

To estimate the value of the current policy, a SARSA agent uses a critic. The critic is a function approximator object that implements the parameterized action-value function $Q(S,A;\phi)$, using parameters ϕ . For a given observation S and action A , the critic stores the corresponding estimate of the expected discounted cumulative long-term reward given the current policy. During training, the critic tunes the parameters in ϕ to improve its action-value function estimation. After training, the parameters remain at their tuned values in the critic internal to the trained agent.

For critics that use table-based value functions, the parameters in ϕ are the actual $Q(S,A)$ values in the table.

For more information on creating critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

SARSA Agent Creation

To create a SARSA agent:

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.

- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your critic. Depending on the type of problem and on the specific critic you use in the next step, this model can be an `rlTable` object (only for discrete observation spaces), a custom basis function with initial parameter values, or a neural network object. The inputs and outputs of the model you create depend on the type of critic you use in the next step.
- 4 Create a critic using `rlQValueFunction` or `rlVectorQValueFunction`. Use the model you created in the previous step as a first input argument.
- 5 Specify agent options using an `rlSARSAAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 6 Create the agent using `rlSARSAAgent`.

SARSA Agent Initialization

When you create a SARSA agent, the critic $Q(S,A;\phi)$ uses random parameter values in ϕ .

The agent uses this initial critic parameters at the beginning of the first training session. For each subsequent training session, the critic retains the parameters from the previous session.

SARSA Training Algorithm

The SARSA agent uses the following training algorithm. To configure the training algorithm, specify options using an `rlSARSAAgentOptions` object.

- For each training episode:
 - 1 At the beginning of the episode, get the initial observation from the environment.
 - 2 For the current observation S , select a random action A with probability ϵ . Otherwise, select the action for which the critic value function is greatest.

$$A = \underset{A}{\operatorname{argmax}} Q(S, A; \phi)$$

To specify ϵ and its decay rate, use the `EpsilonGreedyExploration` option.

- 3 Repeat the following operations for each step of the episode until S is a terminal state:
 - a Execute action A . Observe the reward R and the next observation S' .
 - b For the current observation S' , select a random action A' with probability ϵ . Otherwise, select the action for which the critic value function is greatest.

$$A' = \underset{A'}{\operatorname{argmax}} Q(S', A'; \phi)$$

- c If S' is a terminal state, set the value function target y to R . Otherwise, set it to

$$y = R + \gamma Q(S', A'; \phi)$$

To set the discount factor γ , use the `DiscountFactor` option.

- d Compute the difference ΔQ between the value function target and the current $Q(S,A;\phi)$ value.

$$\Delta Q = y - Q(S, A; \phi)$$

- e Update the critic using the learning rate α . Specify the learning rate when you create the critic by setting the `LearnRate` option in the `rlCriticOptimizerOptions` property within the agent options object.

- For table-based critics, update the corresponding $Q(S,A)$ value in the table.

$$Q(S, A) = Q(S, A; \phi) + \alpha \cdot \Delta Q$$

- For all other types of critics, compute the gradients $\Delta\phi$ of the loss function with respect to the parameters ϕ . Then, update the parameters based on the computed gradients. In this case, the loss function is the square of ΔQ .

$$\Delta\phi = \frac{1}{2} \nabla_{\phi} (\Delta Q)^2$$

$$\phi = \phi + \alpha \cdot \Delta\phi$$

- f If ϵ is greater than its minimum value, perform the decay operation as described in `EpsilonGreedyExploration`.
- g Set the observation S to S' .
- h Set the action A to A' .

References

- [1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Objects

`rlSARSAAgent` | `rlSARSAAgentOptions` | `rlQValueFunction` | `rlVectorQValueFunction` | `rlQAgent` | `rlLSPIAgent` | `rlDQNAgent`

Related Examples

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Compare Agents on Deterministic Waterfall Grid World” on page 7-36

More About

- “Reinforcement Learning Agents” on page 3-2
- “Q-Learning Agent” on page 3-22
- “LSPI Agent” on page 3-28
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

LSPI Agent

The least square policy iteration (LSPI) algorithm is an off-policy reinforcement learning method for environments with a discrete action space. This algorithm trains a Q-value function critic to estimate the value of the optimal policy, while following an epsilon-greedy policy based on the value estimated by the critic.

The LSPI algorithm uses a least-squares approach to directly approximate the Q-value function. This approach is unlike gradient-based algorithms such as deep-Q network (DQN) and policy gradient (PG), which update the policy parameters using gradient descent to minimize a loss function. The approximation model used by critic must be a linear-in-the-parameters custom basis function. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, an LSPI agent is implemented by an `rlLSPIAgent` object. This implementation uses an online version of the LSPI algorithm that is designed to update the policy iteratively as new data becomes available, rather than relying on a fixed batch of data.

LSPI agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete

LSPI agents use the following critic.

Critic	Actor
Q-value function critic $Q(S,A)$ that uses a custom basis function approximation model. You create this critic using <code>rlQValueFunction</code> .	LSPI agents do not use an actor

During training, the agent:

- Updates the critic learnable parameters at a certain frequency specified by the `LearningFrequency` agent option.
- Explores the action space using epsilon-greedy exploration. During each control interval the agent selects a random action with probability ϵ , otherwise, it selects the action for which the action-value function is the greatest, with probability $1-\epsilon$.

Critic Used by LSPI Agent

The LSPI agent uses a critic to estimate the value of the optimal policy. The critic is a function approximator object that implements the parameterized action-value function $Q(S,A;W)$, using parameters W . For LSPI agents, Q must be linear in W . So, you must implement the critic using a custom basis function $\phi(S,A)$, instead of a neural network or a table, to implement the critic. For a given observation S and action A , the critic stores the corresponding estimate of the expected discounted cumulative long-term reward when following the optimal policy. This is the value of the optimal policy. During training, the critic tunes the parameters in W to improve its action-value function estimation. After training, the parameters remain at their tuned values in the critic internal to the trained agent.

For more information on critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

LSPI Agent Creation

To create an LSPI agent object, follow these steps:

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using the `getObservationInfo` function.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using the `getActionInfo` function.
- 3 Create a custom basis function that takes an observation and an action as inputs, and returns a vector as output. Each element of the vector also referred to as a feature. Ideally, there is a linear combination of the features that approximates the value function of your reinforcement learning problem with reasonable accuracy. Functions defined in a separate file or anonymous functions are recommended over local functions.
- 4 Initialize a parameter (weights) vector W . This vector must contain as many elements as the number of features returned by your custom basis function.
- 5 Create an `rlQValueFunction` critic object, passing as a first input argument a cell containing both a handle to your custom basis function and the initial weight vector.
- 6 Specify agent options using an `rlLSPIAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 7 Create the agent using `rlLSPIAgent`.

LSPI Agent Initialization

When you create an LSPI agent, it initializes the following parameters.

- The feature matrix F is set to an identity matrix multiplied by a scalar δ . F has as many rows and columns as the number of features supplied by your custom basis function. To specify δ , use the `FeatureMatrixInitializationConstant` option.
- The target vector b is set to zero. The vector b has as many elements as the number of rows of F .

The agent uses these initial values for F and b at the beginning of the first training session. For each subsequent training session, F and b retain the values from the previous session.

LSPI Training Algorithm

LSPI agents use the following training algorithm. To configure the training algorithm, specify options using an `rlLSPIAgentOptions` object.

- For each training episode, perform these operations:
 - 1 At the beginning of each episode, get the initial observation from the environment.
 - 2 For the current observation S , select a random action A with probability ϵ . Otherwise, select the action for which the critic value function is greatest.

$$A = \underset{A}{\operatorname{argmax}} Q(S, A; W) = \underset{A}{\operatorname{argmax}} W\phi(S, A)$$

To specify ϵ and its decay rate, use the `EpsilonGreedyExploration` option.

- 3 Execute action A . Observe the reward R and the next observation S' .

- 4 Store the experience (S, A, R, S') .
- 5 If ϵ is greater than its minimum value, perform the decay operation as described in `EpsilonGreedyExploration`.
- 6 Every M time steps (to specify M use the `LearningFrequency` option) perform these learning operations:
 - a Assemble the matrices Φ' and Φ using the M stored experiences:

$$\Phi = [\phi(S_1, A_1) \ \phi(S_2, A_2) \ \cdots \ \phi(S_M, A_M)]$$

$$\Phi' = [\phi(S'_1, A'_1) \ \phi(S'_2, A'_2) \ \cdots \ \phi(S'_M, A'_M)]$$

Here, A'_i is the action for which the critic value function calculated in S'_i is the greatest:

$$A'_i = \underset{A}{\operatorname{argmax}} W\phi(S'_i, A)$$

If $S'_i = S_{i+1}$ is a terminal state, the vector $\phi(S'_i, A'_i)$ is set to zero.

- b Compute the matrix F_{new} and the vector b_{new} :

$$F_{new} = \Phi(\Phi - \gamma\Phi')^T$$

$$b_{new} = \Phi[R_1 \ R_2 \ \cdots \ R_M]^T$$

To set the discount factor γ , use the `DiscountFactor` option.

- c Update the feature matrix F and the target vector b using a running mean formula:

$$N = N + M$$

$$F = \frac{N - M}{N}F + \frac{1}{N}F_{new}$$

$$b = \frac{N - M}{N}b + \frac{1}{N}b_{new}$$

Here, N is the total number of samples collected from the beginning.

To set the discount factor γ , use the `DiscountFactor` option.

- d Compute the new weights matrix W :

$$W = F^{-1}b$$

References

- [1] Lagoudakis, Michail G., and Ronald Parr. "Least-Squares Policy Iteration." *Journal of Machine Learning Research* 4, no. Dec (2003): 1107-49. <https://www.jmlr.org/papers/v4/lagoudakis03a.html>.
- [2] Busoniu, L., D Ernst, B. De Schutter, and R. Babuska. "Online Least-Squares Policy Iteration for Reinforcement Learning Control." In *Proceedings of the 2010 American Control Conference*, 486-91. Baltimore, MD: IEEE, 2010. <https://doi.org/10.1109/ACC.2010.5530856>.

See Also

Functions

`getObservationInfo` | `getActionInfo`

Objects

`rlLSPIAgent` | `rlLSPIAgentOptions` | `rlQAgentOptions` | `rlQValueFunction` | `rlQAgent` | `rlSARSAgent` | `rlDQNAgent`

Related Examples

- “Train LSPI Agent to Balance Discrete Cart-Pole” on page 5-102
- “Compare Agents on Basic Grid World” on page 7-2

More About

- “Reinforcement Learning Agents” on page 3-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Deep Q-Network (DQN) Agent

The deep Q-network (DQN) algorithm is an off-policy reinforcement learning method for environments with discrete action spaces. A DQN agent trains a Q-value function to estimate the expected discounted cumulative long-term reward when following the optimal policy. DQN is a variant of Q-learning that features a target critic and an experience buffer. The DQN agent supports offline training (training from saved data, without an environment). For more information on Q-learning, see “Q-Learning Agent” on page 3-22. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a DQN agent is implemented by an `rlDQNAgent` object.

DQN agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete

DQN agents use the following critic.

Critic	Actor
Q-value function critic $Q(S,A)$, which you create using <code>rlQValueFunction</code> or <code>rlVectorQValueFunction</code>	DQN agents do not use an actor.

During training, the agent:

- Updates the critic learnable parameters at each time step during learning.
- Explores the action space using epsilon-greedy exploration. During each control interval, the agent either selects a random action with probability ϵ or selects an action greedily with respect to the action-value function with probability $1-\epsilon$. The greedy action is the action for which the action-value function is greatest.
- Stores past experiences using a circular experience buffer. The agent updates the critic based on a mini-batch of experiences randomly sampled from the buffer.

Critics Used by the DQN Agent

To estimate the value of the optimal policy, a DQN agent uses two parametrized action-value functions, each maintained by a corresponding critic.

- Critic $Q(S,A;\phi)$ — Given observation S and action A this critic stores the corresponding estimate of the expected discounted cumulative long-term reward when following the optimal policy (this is the value of the optimal policy).
- Target critic $Q_t(S,A;\phi_t)$ — To improve the stability of the optimization, the agent periodically updates the target critic learnable parameters ϕ_t using the latest critic parameter values.

Both $Q(S,A;\phi)$ and $Q_t(S,A;\phi_t)$ are implemented by function approximator objects having the same structure and parameterization. During training, the training algorithm tunes the critics parameter values to improve their action-value function estimation. After training, the parameters remain at their tuned values in the critics internal to the trained agent.

For more information on critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

DQN Agent Creation

You can create and train DQN agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a default DQN agent based on the observation and action specifications from the environment. A default DQN agent uses function default approximators that rely on a deep neural network model. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer (the default is 256 neurons) or whether to use an LSTM layer (by default no LSTM layer is used). To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options using an `rlDQNAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlDQNAgent`.

Alternatively, you can create a critic and use it to create your agent. In this case, ensure that the dimensions of the observation and action layers in the critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your critic. Depending on the type of problem and on the specific critic you use in the next step, this model can be an `rlTable` object (only for discrete observation spaces), a custom basis function with initial parameter values, or a neural network object. The inputs and outputs of the model you create depend on the type of critic you use in the next step.
- 4 Create a critic using `rlQValueFunction` or `rlVectorQValueFunction`. Use the model you created in the previous step as a first input argument.
- 5 Specify agent options using an `rlDQNAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 6 Create the agent using `rlDQNAgent`.

DQN agents support critics that use recurrent deep neural networks as functions approximators.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

DQN Agent Initialization

When you create a DQN agent, the critic $Q(S,A;\phi)$ uses random parameter values in ϕ . The agent then initializes the target critic parameters ϕ_t with the same values.

The agent uses this initial critic parameters at the beginning of the first training session. For each subsequent training session, the critics retain the parameters from the previous session.

DQN Training Algorithm

DQN agents use the following training algorithm, in which they update their critic model at each time step. To configure the training algorithm, specify options using an `rldQNAgentOptions` object.

- Perform a warm start by taking a sequence of actions following an epsilon-greedy policy:
 - 1 At the beginning of each episode, get the initial observation from the environment.
 - 2 For the current observation S , select a random action A with probability ϵ . Otherwise, select the action for which the critic value function is greatest.

$$A = \underset{A}{\operatorname{argmax}} Q(S, A; \phi)$$

To specify ϵ and its decay rate, use the `EpsilonGreedyExploration` option.

- 3 Execute action A . Observe the reward R and the next observation S' .
- 4 Store the experience (S, A, R, S') in the experience buffer.
- 5 If ϵ is greater than its minimum value, perform the decay operation as described in `EpsilonGreedyExploration`.

To specify the size of the experience buffer, use the `ExperienceBufferLength` option in the agent `rldQNAgentOptions` object. To specify the number of warm up actions, use the `NumWarmStartSteps` option.

- After the warm start procedure, for each training time step:
 - 1 Execute the five operations described in the warm start procedure.
 - 2 Every D_C time steps (to specify D_C use the `LearningFrequency` option) perform the following two operations for `NumEpoch` times:
 - a Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i, A_i, R_i, S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.

- b For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-34.

When `LearningFrequency` has the default value of -1, the mini-batches creation (described in point a) and the learning operations (described in point b) are executed after each episode is finished.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1 Update the probability threshold ϵ for selecting a random action based on the decay rate you specify in the `EpsilonGreedyExploration` option.
- 2 Update the critic parameters by one-step minimization of the loss L_k across all sampled experiences.

$$L_k = \frac{1}{2M} \sum_{i=1}^M (y_i - Q_k(S_i, A_i; \phi_k))^2$$

To specify the optimizer options used to minimize L_k , use the options contained in the `CriticOptimizerOptions` option (which in turn contains an `rlOptimizerOptions` object).

If the agent contains recurrent neural networks, each element of the sum over the batch elements is itself a sum over the time (sequence) dimension.

If S'_i is a terminal state, set the value function target y_i to R_i . Otherwise, set it to

$$A_{\max} = \underset{A'}{\operatorname{argmax}} Q(S'_i, A'; \phi) \quad (\text{double DQN})$$

$$y_i = R_i + \gamma Q_t(S'_i, A_{\max}; \phi_t)$$

$$y_i = R_i + \gamma \underset{A'}{\operatorname{max}} Q_t(S'_i, A'; \phi_t) \quad (\text{DQN})$$

Here, the normal DQN algorithm selects the action that maximizes the action-value function maintained by the target critic, while the double DQN selects the action that maximizes the action-value function maintained by the base critic.

To set the discount factor γ , use the `DiscountFactor` option. To use double DQN, set the `UseDoubledDQN` option to `true`.

If you specify a value of `NumStepsToLookAhead` equal to N , then the N -step return (which adds the rewards of the following N steps and the discounted estimated value of the state that caused the N -th reward) is used to calculate the target y_i .

- 3 At every `TargetUpdateFrequency` critic updates, update the target critic parameters depending on the target update method. For more information, see “Target Update Methods” on page 3-35.

Target Update Methods

DQN agents update their target critic parameters using one of the following target update methods.

- **Smoothing** — Update the target parameters at every time step using smoothing factor τ . To specify the smoothing factor, use the `TargetSmoothFactor` option.

$$\phi_t = \tau \phi + (1 - \tau) \phi_t$$
- **Periodic** — Update the target parameters periodically without smoothing (`TargetSmoothFactor` = 1). To specify the update period, use the `TargetUpdateFrequency` parameter.
- **Periodic Smoothing** — Update the target parameters periodically with smoothing.

To configure the target update method, create a `rlDQNAgentOptions` object, and set the `TargetUpdateFrequency` and `TargetSmoothFactor` parameters as shown in the following table.

Update Method	TargetUpdateFrequency	TargetSmoothFactor
Smoothing (default)	1	Less than 1
Periodic	Greater than 1	1
Periodic smoothing	Greater than 1	Less than 1

References

- [1] Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. “Playing Atari with Deep Reinforcement Learning.” arXiv, December 19, 2013. <https://doi.org/10.48550/arXiv.1312.5602>.

See Also

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlQValueFunction` | `rlVectorQValueFunction` | `rlQAgent` | `rlSARSAAgent` | `rlLSPIAgent`

Related Examples

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train DQN Agent Using Hindsight Experience Replay” on page 5-176
- “Train DQN Agent for Lane Keeping Assist” on page 8-76

More About

- “Reinforcement Learning Agents” on page 3-2
- “Q-Learning Agent” on page 3-22
- “LSPI Agent” on page 3-28
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

REINFORCE Policy Gradient (PG) Agent

The REINFORCE policy gradient (PG) algorithm is an on-policy reinforcement learning method for environments with a discrete or continuous action space. The REINFORCE policy gradient agent (also referred to sometimes as Monte Carlo policy gradient or vanilla policy gradient) is a policy-based reinforcement learning agent that uses the REINFORCE algorithm to search for an optimal stochastic policy (a stochastic policy that maximizes the expected discounted cumulative long-term reward). As this algorithm belongs to the class of Monte Carlo methods, the agent does not learn during an episode but only after an episode is finished.

To reduce the variance of the parameter updates you can use a baseline value function critic that estimates the expected discounted cumulative long-term reward. Note that such baseline does not fully act as a critic as it is not used for bootstrapping (that it is not used to update the value estimate of a state based on the value estimates of subsequent states).

Note The PG agent does not generally have any functional advantage with respect to more recent agents such as PPO and is provided mostly for educational purposes.

For continuous action spaces and with custom networks, this agent does not automatically enforce the constraints set by the action specification. In this case, you must enforce action space constraints within the environment. The default agent for continuous action spaces automatically enforces constraints on the action.

For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a REINFORCE actor-critic agent is implemented by an `rlPGAgent` object.

Policy gradient agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete or continuous

Policy gradient agents use the following actor and critic.

Critic (if a baseline is used)	Actor
Value function critic $V(S)$, which you create using <code>rlValueFunction</code>	Stochastic policy actor $\pi(S)$, which you create using <code>rlDiscreteCategoricalActor</code> (for a discrete action space) or <code>rlContinuousGaussianActor</code> (for a continuous action space).

During training, a PG agent:

- Estimates probabilities of taking each action in the action space and randomly selects actions based on the probability distribution.
- Completes a full training episode using the current policy before learning from the experience and updating the policy parameters.

If the `UseExplorationPolicy` option of the agent is set to `false`, the action with maximum likelihood is always used in `sim` and `generatePolicyFunction`. As a result, the simulated agent and generated policy behave deterministically.

If the `UseExplorationPolicy` is set to `true` the agent selects its actions by sampling its probability distribution. As a result the policy is stochastic and the agent explores its observation space.

Note The `UseExplorationPolicy` option affects only simulation and deployment; it does not affect training. When you train an agent using `train`, the agent always uses its exploration policy independently of the value of this property.

Actor and Critic Used by the PG Agent

Policy gradient agents represent the policy using an actor function approximator $\pi(A|S;\theta)$ with parameters θ . The actor outputs the conditional probability of taking each action A when in state S as one of the following:

- Discrete action space — A vector containing the probabilities of taking each discrete action. The sum of these probabilities across all actions is 1.
- Continuous action space — Two vectors containing the means and standard deviations, respectively, of the Gaussian distribution for each continuous action.

The agent samples these probability distributions to obtain the action vector.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critic (if used) tunes the parameter values in ϕ to improve its value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

To reduce the variance of the parameter updates during gradient estimation, REINFORCE policy gradient agents can use a baseline value function, which is estimated using a critic function approximator, $V(S;\phi)$ with parameters ϕ . The critic computes the value function for a given observation state.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

PG Agent Creation

You can create a REINFORCE policy gradient agent with default actor and critic based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.

- 4 If needed, specify agent options using an `rlPGAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlPGAgent`.

Alternatively, you can create actor and critic and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. For continuous action spaces, this model must be a neural network object. For discrete action spaces, you also have the option of using a custom basis function with initial parameter values.
- 4 Create an actor using `rlDiscreteCategoricalActor` (for discrete action spaces) or `rlContinuousGaussianActor` (for continuous action spaces). Use the model you created in the previous step as a first input argument.
- 5 If you are using a baseline function, create an approximation model for your critic using either a custom basis function with initial parameter values or a neural network object.
- 6 If you are using a baseline function, create a critic using `rlValueFunction`. Use the model you created in the previous step as a first input argument.
- 7 Specify agent options using the `rlPGAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlPGAgent`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

PG Agent Initialization

When you create an agent, the actor $\pi(A|S;\theta)$ uses random parameter values in θ .

If used, the baseline $V(S;\phi)$ uses random parameter values in ϕ .

The agent uses this initial actor and baseline parameters at the beginning of the first training session. For each subsequent training session, the actor and baseline retain the parameters from the previous session.

REINFORCE Training Algorithm

PG agents use the REINFORCE (also known as Monte Carlo policy gradient) algorithm either with or without a baseline. To configure the training algorithm, specify options using an `rlPGAgentOptions` object.

REINFORCE Algorithm

- 1 For each training episode, generate the episode experience by following the current policy $\pi(S)$:
 - a At the beginning of each episode, get the initial observation S_1 from the environment.

- b** For the current observation S_t , select the action A_t using the policy in $\pi(A|S;\theta)$.
- c** Execute action A_t . Observe the reward R_{t+1} and the next observation S_{t+1} .
- d** Store the experience $(S_t, A_t, R_{t+1}, S_{t+1})$.

The agent takes actions until it reaches the terminal state corresponding to observation S_T . The episode experience consists of the sequence

$$S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$$

- 2** For each state in the episode sequence, that is, for $t = 1, 2, \dots, T-1$, calculate the return G_t , which is the discounted future reward.

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_k$$

- 3** Accumulate the gradients for the actor network by following the gradient of the policy to maximize the expected discounted cumulative long-term reward. If the `EntropyLossWeight` option is greater than zero, then additional gradients are accumulated to maximize the entropy loss function.

$$d\theta = \sum_{t=1}^{T-1} G_t \nabla_{\theta} (\ln \pi(S_t; \theta) + w \mathcal{H}_t(\theta, S_t))$$

Here,

- $\mathcal{H}_t(\theta, S_t)$ is the entropy loss and w is the entropy loss weight factor, specified using the `EntropyLossWeight` option. For more information on entropy loss, see “Entropy Loss” on page 3-41.
- $\pi(A_i|S_i; \theta)$ indicates one of the following quantities:
 - For a discrete action space — The probability of taking the discrete action A_i , given observation S_i , and parameters θ .
 - For a continuous action space — The value of the joint probability density function of the action, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S; \theta) = \prod_{j=1}^N f_j(A_{j,i}|S_i; \theta)$$

Here, f_j is the probability density function for the dimension j of the action vector, and $A_{i,j}$ is the j th element of A_i , which has dimension N .

- 4** Update the actor parameters by applying the gradients.

$$\theta = \theta + \alpha d\theta$$

Here, α is the learning rate of the actor. Specify the learning rate when you create the actor by setting the `LearnRate` option in the `rlActorOptimizerOptions` property within the agent options object. For simplicity, this step shows a gradient update using basic stochastic gradient descent. The actual gradient update method depends on the optimizer you specify in the `rlOptimizerOptions` object assigned to the `rlActorOptimizerOptions` property.

- 5** Repeat steps 2 through 5 for each training episode until training is complete.

REINFORCE with Baseline Algorithm

- 1 For each training episode, generate the episode experience by following the current policy $\pi(A|S; \theta)$, as described in the “REINFORCE Algorithm” on page 3-39 section, until a terminal state is reached. The episode experience consists of the sequence

$$S_0, A_0, R_1, S_1, \dots, S_{T-1}, A_{T-1}, R_T, S_T$$

- 2 For $t = 1, 2, \dots, T$:

- Calculate the return G_t , which is the discounted future reward.

$$G_t = \sum_{k=t}^T \gamma^{k-t} R_k$$

- Compute the advantage function δ_t using the baseline value function estimate from the critic.

$$\delta_t = G_t - V(S_t; \phi)$$

- 3 Accumulate the gradients for the critic network.

$$d\phi = \sum_{t=1}^{T-1} \delta_t \nabla_{\phi} V(S_t; \phi)$$

- 4 Accumulate the gradients for the actor network. If the `EntropyLossWeight` option is greater than zero, then additional gradients are accumulated to maximize the entropy loss function.

$$d\theta = \sum_{t=1}^{T-1} \delta_t \nabla_{\theta} (\ln \pi(S_t; \theta) + w \mathcal{H}_t(\theta, S_t))$$

Here, $\mathcal{H}_t(\theta, S_t)$ is the entropy loss and w is the entropy loss weight factor, specified using the `EntropyLossWeight` option. For more information on entropy loss, see “Entropy Loss” on page 3-41.

- 5 Update the critic parameters ϕ .

$$\phi = \phi + \beta d\phi$$

Here, β is the learning rate of the critic. Specify the learning rate when you create the critic by setting the `LearnRate` option in the `rlCriticOptimizerOptions` property within the agent options object.

- 6 Update the actor parameters θ .

$$\theta = \theta + \alpha d\theta$$

- 7 Repeat steps 3 through 8 for each training episode until training is complete.

For simplicity, the actor and critic updates in this algorithm show a gradient update using basic stochastic gradient descent. The actual gradient update method depends on the optimizer you specify in the `rlOptimizerOptions` object assigned to the `rlCriticOptimizerOptions` property.

Entropy Loss

To promote agent exploration, you can subtract an entropy loss term $w \mathcal{H}_i(\theta, S_i)$ from the actor loss function, where w is the entropy loss weight and $\mathcal{H}_i(\theta, S_i)$ is the entropy.

The entropy value is higher when the agent is more uncertain about which action to take next. Therefore, maximizing the entropy loss term (minimizing the negative entropy loss) increases the

agent uncertainty, thus encouraging exploration. To promote additional exploration, which can help the agent move out of local optima, you can specify a larger entropy loss weight.

For a discrete action space, the agent uses the following entropy value. In this case, the actor outputs the probability of taking each possible discrete action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^d(\theta, S_i) = - \sum_{k=1}^P \pi(A_k|S_i; \theta) \ln \pi(A_k|S_i; \theta)$$

Here:

- P is the number of possible discrete actions.
- $\pi(A_k|S_i; \theta)$ is the probability of taking action A_k when in state S_i following the current policy.

For a continuous action space, the agent uses the following entropy value. In this case, the actor outputs the mean and standard deviation of the Gaussian distribution for each continuous action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^c(\theta, S_i) = \frac{1}{2} \sum_{k=1}^{N_C} \ln(2\pi \cdot e \cdot \sigma_{k,i}^2)$$

Here:

- C is the number of continuous actions output by the actor.
- $\sigma_{k,i}$ is the standard deviation for action k when in state S_i following the current policy.

References

- [1] Williams, Ronald J. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8, no. 3-4 (May 1992): 229-56. <https://doi.org/10.1007/BF00992696>.
- [2] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Objects

rlPGAgent | rlPGAgentOptions

Related Examples

- "Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole" on page 5-109
- "Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator" on page 5-115

More About

- "Reinforcement Learning Agents" on page 3-2
- "Train Reinforcement Learning Agents" on page 5-3

- “Create Actors, Critics, and Policy Objects” on page 4-2

Actor-Critic (AC) Agent

Actor-critic (AC) agents implement actor-critic algorithms such as A2C and A3C, which are on-policy policy-gradient reinforcement learning methods for environments with a discrete or continuous action space. The actor-critic agent directly optimizes a stochastic policy and uses a value function critic to estimate the value of the policy [1]. The AC agent is similar to a REINFORCE policy gradient agent with a baseline, with the difference that AC uses a fully bootstrapping critic (that is, it used to update the value estimate of a state based on the value estimates of subsequent states). You can use the, which uses to implement actor-critic algorithms, such as A2C and A3C.

For continuous action spaces and with custom networks, this agent does not automatically enforce the constraints set by the action specification. In this case, you must enforce action space constraints within the environment. The default agent for continuous action spaces automatically enforces constraints on the action.

In Reinforcement Learning Toolbox, a REINFORCE actor-critic agent is implemented by an `rlACAgent` object.

Note AC agents do not generally have functional advantages with respect to more recent agents such as PPO and are provided mostly for educational purposes.

For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

Actor-critic agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete or continuous

Actor-critic agents use the following actor and critics.

Critic	Actor
Value function critic $V(S)$, which you create using <code>rlValueFunction</code>	Stochastic policy actor $\pi(S)$, which you create using <code>rlDiscreteCategoricalActor</code> (for discrete action spaces) or <code>rlContinuousGaussianActor</code> (for continuous action spaces)

During training, an actor-critic agent:

- Estimates probabilities of taking each action in the action space and randomly selects actions based on the probability distribution.
- Interacts with the environment for multiple steps using the current policy before updating the actor and critic properties.

If the `UseExplorationPolicy` option of the agent is set to `false` the action with maximum likelihood is always used in `sim` and `generatePolicyFunction`. As a result, the simulated agent and generated policy behave deterministically.

If the `UseExplorationPolicy` is set to `true` the agent selects its actions by sampling its probability distribution. As a result the policy is stochastic and the agent explores its observation space.

Note The `UseExplorationPolicy` option affects only simulation and deployment; it does not affect training. When you train an agent using `train`, the agent always uses its exploration policy independently of the value of this property.

Actor and Critic Used by the AC Agent

To estimate the policy and value function, an actor-critic agent maintains two function approximators.

- Stochastic actor $\pi(A|S;\theta)$ — The actor, with parameters θ , outputs the conditional probability of taking each action A when in state S as one of the following:
 - Discrete action space — A vector containing the probabilities of taking each discrete action. The sum of these probabilities across all actions is 1.
 - Continuous action space — Two vectors containing the means and standard deviations, respectively, of the Gaussian distribution for each continuous action.

The agent samples these probability distributions to obtain the action vector.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critic tunes the parameter values in ϕ to improve its value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

AC Agent Creation

You can create an actor-critic agent with default actor and critics based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options using an `rlACAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlACAgent`.

Alternatively, you can create actor and critic and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. For continuous action spaces, this model must be a neural network object. For discrete action spaces, you also have the option of using a custom basis function with initial parameter values.
- 4 Create an actor using `rlDiscreteCategoricalActor` (for discrete action spaces) or `rlContinuousGaussianActor` (for continuous action spaces). Use the model you created in the previous step as a first input argument.
- 5 Create an approximation model for your critic using either a custom basis function with initial parameter values or a neural network object.
- 6 Create a critic using `rlValueFunction`. Use the model you created in the previous step as a first input argument.
- 7 Specify agent options using an `rlACAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlACAgent`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

AC Agent Initialization

When you create an AC agent, the actor and critic initialize as follows.

- The actor $\pi(A|S;\theta)$ uses random parameter values in θ .
- The critic $V(S;\phi)$ uses parameter values in ϕ .

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

AC Training Algorithm

Actor-critic agents use the following training algorithm. To configure the training algorithm, specify options using an `rlACAgentOptions` object.

- 1 Generate up to $N+1$ experiences (within the current episode) by following the current policy.
 - a At the beginning of each episode, get the initial observation S_t from the environment.
 - b For the current observation S_t , select the action A_t using the policy in $\pi(A|S;\theta)$.
 - c Execute action A_t . Observe the reward R_{t+1} and the next observation S_{t+1} .
 - d Store the experience $(S_t, A_t, R_{t+1}, S_{t+1})$.

The episode experience sequence is

$$S_{ts}, A_{ts}, R_{ts+1}, S_{ts+1}, \dots, S_{ts+N-1}, A_{ts+N-1}, R_{ts+N}, S_{ts+N}$$

Here, ts is the starting time step of the current set of N experiences. At the beginning of the training episode, $ts = 1$. For each subsequent set of N experiences in the same training episode, $ts = ts + N$.

For each training episode that does not contain a terminal state, N is equal to the value of the `NumStepsToLookAhead` option. Otherwise, N is less than `NumStepsToLookAhead` and S_N is the terminal state.

- 2 For each episode step $t = ts, ts+1, \dots, ts+N$, compute the return G_t , which is the sum of the reward for that step and the discounted future reward. If S_{ts+N} is not a terminal state, the discounted future reward includes the discounted state value function, computed using the critic network V .

$$G_t = \sum_{k=t+1}^{ts+N} (\gamma^{k-t-1} R_k) + b\gamma^{ts+N-t} V(S_{ts+N}; \phi)$$

Here, b is 0 if S_{ts+N} is a terminal state and 1 otherwise.

To specify the discount factor γ , use the `DiscountFactor` option.

- 3 Compute the advantage function D_t .

$$D_t = G_t - V(S_t; \phi)$$

- 4 Accumulate the gradients for the actor network by following the policy gradient to maximize the expected discounted cumulative long-term reward.

$$d\theta = \sum_{t=1}^N \nabla_{\theta} \ln \pi(A|S_t; \theta) \cdot D_t$$

Here,

- $\mathcal{H}_t(\theta, S_t)$ is the entropy loss and w is the entropy loss weight factor, specified using the `EntropyLossWeight` option. For more information on entropy loss, see “Entropy Loss” on page 3-48.
- $\pi(A_i|S_i; \theta)$ indicates one of the following quantities:
 - For a discrete action space — The probability of taking the discrete action A_i , given observation S_i , and parameters θ .
 - For a continuous action space — The value of the joint probability density function of the action, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S; \theta) = \prod_{j=1}^N f_j(A_{j,i}|S_i; \theta)$$

Here, f_j is the probability density function for the dimension j of the action vector, and $A_{i,j}$ is the j th element of A_i , which has dimension N .

- 5 Accumulate the gradients for the critic network by minimizing the mean squared error loss between the estimated value function $V(S_t; \phi)$ and the computed target return G_t across all N experiences.

$$d\phi = \sum_{t=1}^N \nabla_{\phi} (G_t - V(S_t; \phi))^2$$

- 6 Update the actor parameters by applying the gradients.

$$\theta = \theta + \alpha d\theta$$

Here, α is the learning rate of the actor. Specify the learning rate when you create the actor by setting the `LearnRate` option in the `rlActorOptimizerOptions` property within the agent options object.

- 7 Update the critic parameters by applying the gradients.

$$\phi = \phi + \beta d\phi$$

Here, β is the learning rate of the critic. Specify the learning rate when you create the critic by setting the `LearnRate` option in the `rlCriticOptimizerOptions` property within the agent options object.

- 8 Repeat steps 3 through 9 for each training episode until training is complete.

For simplicity, the actor and critic updates in this algorithm description show a gradient update using basic stochastic gradient descent. The actual gradient update method depends on the optimizer you specify in the `rlOptimizerOptions` object assigned to the `rlCriticOptimizerOptions` property.

Entropy Loss

To promote agent exploration, you can subtract an entropy loss term $w\mathcal{H}_i(\theta, S_i)$ from the actor loss function, where w is the entropy loss weight and $\mathcal{H}_i(\theta, S_i)$ is the entropy.

The entropy value is higher when the agent is more uncertain about which action to take next. Therefore, maximizing the entropy loss term (minimizing the negative entropy loss) increases the agent uncertainty, thus encouraging exploration. To promote additional exploration, which can help the agent move out of local optima, you can specify a larger entropy loss weight.

For a discrete action space, the agent uses the following entropy value. In this case, the actor outputs the probability of taking each possible discrete action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^d(\theta, S_i) = - \sum_{k=1}^P \pi(A_k|S_i; \theta) \ln \pi(A_k|S_i; \theta)$$

Here:

- P is the number of possible discrete actions.
- $\pi(A_k|S_i; \theta)$ is the probability of taking action A_k when in state S_i following the current policy.

For a continuous action space, the agent uses the following entropy value. In this case, the actor outputs the mean and standard deviation of the Gaussian distribution for each continuous action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^c(\theta, S_i) = \frac{1}{2} \sum_{k=1}^{N_C} \ln(2\pi \cdot e \cdot \sigma_{k,i}^2)$$

Here:

- C is the number of continuous actions output by the actor.
- $\sigma_{k,i}$ is the standard deviation for action k when in state S_i following the current policy.

References

- [1] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. “Asynchronous Methods for Deep Reinforcement Learning.” arXiv, June 16, 2016. <https://doi.org/10.48550/arXiv.1602.01783>.
- [2] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning. Cambridge, Mass: The MIT Press, 2018.

See Also

Objects

`rlACAgent` | `rlACAgentOptions`

Related Examples

- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115

More About

- “Reinforcement Learning Agents” on page 3-2
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Proximal Policy Optimization (PPO) Agent

Proximal policy optimization (PPO) is an on-policy, policy gradient reinforcement learning method for environments with a discrete, continuous or hybrid action space. It directly estimates a stochastic policy and uses a value function critic to estimate the value of the policy. This algorithm alternates between sampling data through environmental interaction and optimizing a clipped surrogate objective function using stochastic gradient descent. The clipped surrogate objective function improves training stability by limiting the size of the policy change at each step [1].

For continuous action spaces and with custom networks, this agent does not automatically enforce the constraints set by the action specification. In this case, you must enforce action space constraints within the environment. The default agent for continuous action spaces automatically enforces constraints on the action.

PPO is a simplified version of TRPO. Specifically, PPO has less hyperparameters and therefore is easier to tune, and less computationally expensive than TRPO. For more information on TRPO agents, see “Trust Region Policy Optimization (TRPO) Agent” on page 3-58. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a proximal policy optimization agent is implemented by an `rlPPOAgent` object.

Proximal policy optimization agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete, continuous, or hybrid.

Proximal policy optimization agents use the following actor and critics. In the most general case, for hybrid action spaces, the action A has a discrete part A^d and a continuous part A^c .

Critic	Actor
Value function critic $V(S)$, which you create using <code>rlValueFunction</code>	Stochastic policy actor $\pi(A S)$, which you create using <code>rlDiscreteCategoricalActor</code> (for discrete action spaces), <code>rlContinuousGaussianActor</code> (for continuous action spaces) or <code>rlHybridStochasticActor</code> for hybrid action spaces

During training, a Proximal policy optimization agent:

- Estimates probabilities of taking each action in the action space and randomly selects actions based on the probability distribution.
- Interacts with the environment for multiple steps using the current policy before using mini-batches to update the actor and critic properties over multiple epochs.

If the `UseExplorationPolicy` option of the agent is set to `false` the action with maximum likelihood is always used in `sim` and `generatePolicyFunction`. As a result, the simulated agent and generated policy behave deterministically.

If the `UseExplorationPolicy` is set to `true` the agent selects its actions by sampling its probability distribution. As a result the policy is stochastic and the agent explores its observation space.

Note The `UseExplorationPolicy` option affects only simulation and deployment; it does not affect training. When you train an agent using `train`, the agent always uses its exploration policy independently of the value of this property.

Actor and Critic Used by the PPO Agent

To estimate the policy and value function, a proximal policy optimization agent maintains two function approximators.

- Stochastic actor $\pi(A|S;\theta)$ — The actor, with parameters θ , outputs the conditional probability of taking each action A when in state S as one of the following:
 - Discrete action space — A vector containing the probability of taking each discrete action. The sum of these probabilities across all actions is 1.
 - Continuous action space — Two vectors containing the means and standard deviations, respectively, of the Gaussian distribution for each continuous action.
 - Hybrid action space — A vector of probabilities for each discrete action, and two vectors representing mean and standard deviation for each continuous action.

The agent samples these probability distributions to obtain the action vector.

- Critic $V(S;\phi)$ — The critic, with parameters ϕ , takes observation S and returns the corresponding expectation of the discounted long-term reward.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critic tunes the parameter values in ϕ to improve its value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

PPO Agent Creation

You can create and train proximal policy optimization agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a PPO agent with default actor and critic based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.

- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 Specify agent options using an `rlPPOAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlPPOAgent`.

Alternatively, you can create actor and critic and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. For continuous action spaces, this model must be a neural network object. For discrete action spaces, you also have the option of using a custom basis function with initial parameter values.
- 4 Create a stochastic actor using `rlContinuousGaussianActor` (for continuous action spaces), `rlDiscreteCategoricalActor` (for discrete action spaces), or `rlHybridStochasticActor` (for hybrid action spaces). Use the model you created in the previous step as a first input argument.
- 5 Create an approximation model for your critic using either a custom basis function with initial parameter values or a neural network object.
- 6 Create a critic using `rlValueFunction`. Use the model you created in the previous step as a first input argument.
- 7 If needed, specify agent options using an `rlPPOAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlPPOAgent`.

PPO agents support actors and critics that use recurrent deep neural networks as function approximators.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

PPO Agent Initialization

When you create a PPO agent, the actor and critic initialize as follows.

- The actor $\pi(A|S;\theta)$ uses random parameter values in θ .
- The critic $V(S;\phi)$ uses parameter values in ϕ .

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

PPO Training Algorithm

Proximal policy optimization agents use the following training algorithm. To configure the training algorithm, specify options using an `rlPPOAgentOptions` object.

- 1 Generate E experiences (possibly running multiple episodes) by following the current policy. Specifically:
 - a At the beginning of each episode, get the initial observation from the environment.
 - b For the current observation S , select the action A using the policy in $\pi(A|S;\theta)$.
 - c Execute action A . Observe the reward R and the next observation S' .
 - d Store the experience (S,A,R,S') .

To specify E use the `LearningFrequency` option.

- 2 Divide the E experiences into sequences each having N experiences, numbered from t_s to t_s+N . For each experience sequence that does not contain a terminal state, N is equal to the `ExperienceHorizon` option value. Otherwise, N is less than `ExperienceHorizon` and S_N is the terminal state.
- 3 For each step $t = t_s, t_s+1, \dots, t_s+N-1$, of each sequence, compute the return and advantage function using the method specified by the `AdvantageEstimateMethod` option.
 - **Finite Horizon** (`AdvantageEstimateMethod` = "finite-horizon") — Compute the return G_t , which is the sum of the reward for that step and the discounted future reward [2].

$$G_t = \sum_{k=t+1}^{t_s+N} (\gamma^{k-t-1} R_k) + b\gamma^{t_s+N-t} V(S_{t_s+N}; \phi)$$

Here, b is 0 if S_{t_s+N} is a terminal state and 1 otherwise. That is, if S_{t_s+N} is not a terminal state, the discounted future reward includes the discounted state value function, computed using the critic approximator V .

Compute the advantage function D_t .

$$D_t = G_t - V(S_t; \phi)$$

- **Generalized Advantage Estimator** (`AdvantageEstimateMethod` = "gae") — Compute the advantage function D_t , which is the discounted sum of temporal difference errors [3].

$$D_t = \sum_{k=t}^{t_s+N-1} (\gamma\lambda)^{k-t} \delta_k$$

$$\delta_k = R_{k+1} + b\gamma V(S_{k+1}; \phi) - V(S_k; \phi)$$

Here, b is 0 if S_{t_s+N} is a terminal state and 1 otherwise. λ is a smoothing factor specified using the `GAEFactor` option.

Compute the return G_t .

$$G_t = D_t + V(S_t; \phi)$$

To specify the discount factor γ for either method, use the `DiscountFactor` option.

- 4 Perform the following two operations for `NumEpoch` times:

- a Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i, A_i, R_i, S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.

- b For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-54.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1 Update the critic parameters by minimizing the loss L_{critic} across all sampled mini-batch data.

$$L_{critic}(\phi) = \frac{1}{2M} \sum_{i=1}^M (G_i - V(S_i; \phi))^2$$

- 2 Normalize the advantage values D_i based on recent unnormalized advantage values.

- If the `NormalizedAdvantageMethod` option is "none", do not normalize the advantage values.

$$\hat{D}_i \leftarrow D_i$$

- If the `NormalizedAdvantageMethod` option is "current", normalize the advantage values based on the unnormalized advantages in the current mini-batch.

$$\hat{D}_i \leftarrow \frac{D_i - \text{mean}(D_1, D_2, \dots, D_M)}{\text{std}(D_1, D_2, \dots, D_M)}$$

- If the `NormalizedAdvantageMethod` option is "moving", normalize the advantage values based on the unnormalized advantages for the N most recent advantages, including the current advantage value. To specify the window size N , use the `AdvantageNormalizingWindow` option.

$$\hat{D}_i \leftarrow \frac{D_i - \text{mean}(D_1, D_2, \dots, D_N)}{\text{std}(D_1, D_2, \dots, D_N)}$$

- 3 Update the actor parameters by minimizing the actor loss function L_{actor} across all sampled mini-batch data.

$$L_{actor}(\theta) = \frac{1}{M} \sum_{i=1}^M \left(-\min(r_i(\theta) \cdot \hat{D}_i, c_i(\theta) \cdot \hat{D}_i) - w \mathcal{H}_i(\theta, S_i) \right)$$

$$r_i(\theta) = \frac{\pi(A_i|S_i; \theta)}{\pi(A_i|S_i; \theta_{old})}$$

$$c_i(\theta) = \max(\min(r_i(\theta), 1 + \varepsilon), 1 - \varepsilon)$$

Here:

- D_i and G_i are the advantage function and return value for the i th element of the mini-batch, respectively.

- $\mathcal{H}_i(\theta, S_i)$ is the entropy loss and w is the entropy loss weight factor, specified using the `EntropyLossWeight` option. For more information on entropy loss, see “Entropy Loss” on page 3-55.
- $\pi(A_i|S_i; \theta)$ indicates one of the following quantities:
 - For a discrete action space — The probability of taking the discrete action A_i , given observation S_i , and parameters θ .
 - For a continuous action space — The value of the joint probability density function of the action, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S_i; \theta) = \prod_{j=1}^N f_j(A_{j,i}|S_i; \theta)$$

Here, f_j is the probability density function for the dimension j of the action vector, and $A_{j,i}$ is the j th element of A_i , which has dimension N .

- For a Hybrid action space — The value of the joint probability density function of both the discrete and continuous actions, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S_i; \theta) = p(A_i^d|S_i; \theta) \prod_{j=1}^{N_C} f_{j,i}(A_{j,i}^c|S_i; \theta)$$

Here, $p(A_i^d|S_i; \theta)$ is the probability of taking the discrete action A_i^d , given observation S_i and parameters θ , $f_{j,i}$ is the probability density function for the dimension j of the continuous part of the action vector A_i^c , and $A_{j,i}^c$ is the j th element of A_i^c , which has dimension N_C .

- $\pi(A_i|S_i; \theta_{old})$ is the same as $\pi(A_i|S_i; \theta)$, except that the previous policy parameters θ_{old} from before the current learning epoch are used instead of the current parameters θ .
- ϵ is the clip factor specified using the `ClipFactor` option.

Entropy Loss

To promote agent exploration, you can subtract an entropy loss term $w\mathcal{H}_i(\theta, S_i)$ from the actor loss function, where w is the entropy loss weight and $\mathcal{H}_i(\theta, S_i)$ is the entropy.

The entropy value is higher when the agent is more uncertain about which action to take next. Therefore, maximizing the entropy loss term (minimizing the negative entropy loss) increases the agent uncertainty, thus encouraging exploration. To promote additional exploration, which can help the agent move out of local optima, you can specify a larger entropy loss weight. For more information on entropy weights, see the `EntropyLossWeight` property in `rlPPOAgentOptions`.

For a discrete action space, the agent uses the following entropy value. In this case, the actor outputs the probability of taking each possible discrete action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^d(\theta, S_i) = - \sum_{k=1}^P \pi(A_k|S_i; \theta) \ln \pi(A_k|S_i; \theta)$$

Here:

- P is the number of possible discrete actions.

- $\pi(A_k|S_i;\theta)$ is the probability of taking action A_k when in state S_i following the current policy.

For a continuous action space, the agent uses the following entropy value. In this case, the actor outputs the mean and standard deviation of the Gaussian distribution for each continuous action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^c(\theta, S_i) = \frac{1}{2} \sum_{k=1}^{N_C} \ln(2\pi \cdot e \cdot \sigma_{k,i}^2)$$

Here:

- N_C is the dimension of the continuous part of the action vector.
- $\sigma_{k,i}$ is the standard deviation for action k when in state S_i following the current policy.

For an hybrid action space, the entropy is a two-element column vector containing both the discrete and a continuous entropy values, respectively.

$$\mathcal{H}_i(\theta, S_i) = \begin{bmatrix} \mathcal{H}_i^d(\theta, S_i) \\ \mathcal{H}_i^c(\theta, S_i) \end{bmatrix}$$

In this case, the entropy weight is a row vector containing weights for the discrete and continuous entropy values, respectively.

References

- [1] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. "Proximal Policy Optimization Algorithms." arXiv, August 28, 2017. <https://doi.org/10.48550/arXiv.1707.06347>.
- [2] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. "Asynchronous Methods for Deep Reinforcement Learning." arXiv, June 16, 2016. <https://doi.org/10.48550/arXiv.1602.01783>.
- [3] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." arXiv, October 20, 2018. <https://doi.org/10.48550/arXiv.1506.02438>.

See Also

Objects

`rlPPOAgent` | `rlPPOAgentOptions`

Related Examples

- "Train Default PPO Agent for Discrete Lander Vehicle" on page 5-93
- "Train Hybrid-Action PPO Agent for Path-Following Control" on page 8-120
- "Train PPO Agent with Curriculum Learning for a Lane Keeping Application" on page 5-164

More About

- "Reinforcement Learning Agents" on page 3-2
- "Create Actors, Critics, and Policy Objects" on page 4-2

- “Train Reinforcement Learning Agents” on page 5-3

Trust Region Policy Optimization (TRPO) Agent

Trust region policy optimization (TRPO) is an on-policy, policy gradient reinforcement learning method for environments with a discrete or continuous action space. It directly estimates a stochastic policy and uses a value function critic to estimate the value of the policy. The KL-divergence between the old policy and the new policy is used as a constraint during optimization. As a result, this algorithm prevents significant performance drops compared to standard policy gradient methods by keeping the updated policy within a trust region close to the current policy [1].

For continuous action spaces and with custom networks, this agent does not automatically enforce the constraints set by the action specification. In this case, you must enforce action space constraints within the environment. The default agent for continuous action spaces automatically enforces constraints on the action.

In Reinforcement Learning Toolbox, a TRPO agent is implemented by an `rlTRPOAgent` object.

Note For TRPO agents, you can only use actors or critics with deep network that support calculating higher order derivatives. Actors and critics that use recurrent networks, custom basis functions, or tables are not supported. To implement a table using a neural network, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

PPO is a simplified version of TRPO. Specifically, PPO has less hyperparameters and therefore is easier to tune, and less computationally expensive than TRPO. For more information on PPO agents, see “Proximal Policy Optimization (PPO) Agent” on page 3-50. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

TRPO agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete or continuous

TRPO agents use the following actor and critic.

Critic	Actor
Value function critic $V(S)$, which you create using <code>rlValueFunction</code>	Stochastic policy actor $\pi(S)$, which you create using <code>rlDiscreteCategoricalActor</code> (for discrete action spaces) or <code>rlContinuousGaussianActor</code> (for continuous action spaces)

During training, a TRPO agent:

- Estimates probabilities of taking each action in the action space and randomly selects actions based on the probability distribution.
- Interacts with the environment for multiple steps using the current policy before using mini-batches to update the actor and critic properties over multiple epochs.

If the `UseExplorationPolicy` option of the agent is set to `false` the action with maximum likelihood is always used in `sim` and `generatePolicyFunction`. As a result, the simulated agent and generated policy behave deterministically.

If the `UseExplorationPolicy` is set to `true` the agent selects its actions by sampling its probability distribution. As a result the policy is stochastic and the agent explores its observation space.

Note The `UseExplorationPolicy` option affects only simulation and deployment; it does not affect training. When you train an agent using `train`, the agent always uses its exploration policy independently of the value of this property.

Actor and Critic Used by the TRPO Agent

To estimate the policy and value function, a TRPO agent maintains two function approximators.

- Stochastic actor $\pi(A|S;\theta)$ — The actor, with parameters θ , outputs the conditional probability of taking each action A when in state S as one of the following:
 - Discrete action space — A vector containing the probabilities of taking each discrete action. The sum of these probabilities across all actions is 1.
 - Continuous action space — Two vectors containing the means and standard deviations, respectively, of the Gaussian distribution for each continuous action.

The agent samples these probability distributions to obtain the action vector.

- Critic $V(S;\phi)$ — The critic, with parameters ϕ , takes observation S and returns the corresponding expectation of the discounted long-term reward.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critic tunes the parameter values in ϕ to improve its value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

TRPO Agent Creation

You can create and train TRPO agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a TRPO agent with default actor and critic based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer. To do so, create an agent initialization options object using `rlAgentInitializationOptions`.
- 4 Specify agent options using an `rlTRPOAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.

5 Create the agent using `rlTRPOAgent`.

Alternatively, you can create actor and critic and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. For continuous action spaces, this model must be a neural network object. For discrete action spaces, you also have the option of using a custom basis function with initial parameter values.
- 4 Create an actor using `rlDiscreteCategoricalActor` (for discrete action spaces) or `rlContinuousGaussianActor` (for continuous action spaces). Use the model you created in the previous step as a first input argument.
- 5 Create an approximation model for your critic using either a custom basis function with initial parameter values or a neural network object.
- 6 Create a critic using `rlValueFunction`. Use the model you created in the previous step as a first input argument.
- 7 If needed, specify agent options using an `rlTRPOAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlTRPOAgent`.

TRPO agents do not support actors and critics that use recurrent deep neural networks as function approximators. TRPO agents also do not support deep neural networks that use a `quadraticLayer`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

TRPO Agent Initialization

When you create a TRPO agent, the actor and critic initialize as follows.

- The actor $\pi(A|S;\theta)$ uses random parameter values in θ .
- The critic $V(S;\phi)$ uses parameter values in ϕ .

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

Trust Region Policy Optimization

Trust region policy optimization finds the actor parameters that minimize the following actor loss function.

$$L_{actor}(\theta) = -\frac{1}{M} \sum_{i=1}^M \left(\frac{\pi(A_i|S_i;\theta)}{\pi(A_i|S_i;\theta_{old})} D_i + w \mathcal{H}_i(\theta, S_i) \right)$$

Here:

- M is the mini-batch size.
- D_i is an advantage function.
- $\pi(A_i|S_i;\theta)$ indicates one of the following quantities:
 - For a discrete action space — The probability of taking the discrete action A_i , given observation S_i , and parameters θ .
 - For a continuous action space — The value of the joint probability density function of the action, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S;\theta) = \prod_{j=1}^N f_j(A_{j,i}|S_i;\theta)$$

Here, f_j is the probability density function for the dimension j of the action vector, and $A_{i,j}$ is the j th element of A_i , which has dimension N .

- $\pi(A_i|S_i;\theta_{old})$ is the same as $\pi(A_i|S_i;\theta)$, except that the previous policy parameters θ_{old} from before the current learning epoch are used instead of the current parameters θ .
- $w\mathcal{H}_i(\theta, S_i)$ is an entropy loss term, where w is the entropy loss weight and $\mathcal{H}_i(\theta, S_i)$ is the entropy. For more information, see “Entropy Loss” on page 3-65.

This minimization is subject to the following constraint.

$$\frac{1}{M} \sum_{i=1}^M D_{KL}(\theta_{old}, \theta, S_i) \leq \delta$$

Here:

- $D_{KL}(\theta_{old}, \theta, S_i)$ is the Kullback-Leibler (KL) divergence between the old policy $\pi(A|S_i;\theta_{old})$ and current policy $\pi(A|S_i;\theta)$. D_{KL} measures how much the probability distributions of the old and new policies differ. D_{KL} is zero when the two distributions are identical.
- δ is the limit for D_{KL} and controls how much the new policy can deviate from the old policy.

For agents with discrete action spaces, D_{KL} is computed as follows, where P is the number of actions.

$$D_{KL}(\theta_{old}, \theta, S_i) = \sum_{k=1}^P \pi(A_k|S_i; \theta_{old}) \ln \left(\frac{\pi(A_k|S_i; \theta_{old})}{\pi(A_k|S_i; \theta)} \right)$$

For agents with continuous action spaces, D_{KL} is computed as follows.

$$D_{KL}(\theta_{old}, \theta, S_i) = \frac{1}{P} \sum_{k=1}^P \left(\ln(\sigma_{\theta,k}) - \ln(\sigma_{\theta_{old},k}) + \frac{\sigma_{\theta_{old},k}^2 + (\mu_{\theta_{old},k} - \mu_{\theta,k})^2}{2\sigma_{\theta,k}^2} - 0.5 \right)$$

Here:

- $\mu_{\theta,k}$ and $\sigma_{\theta,k}$ are the mean and standard deviation for the k th action output by the current actor policy $\pi(A_k|S_i;\theta)$.
- $\mu_{\theta_{old},k}$ and $\sigma_{\theta_{old},k}$ are the mean and standard deviation for the k th action output by the old policy $\pi(A_k|S_i;\theta_{old})$.

To approximate this optimization problem, the TRPO agent uses a linear approximation of $L_{actor}(\theta)$ and a quadratic approximation of $D_{KL}(\theta_{old}, \theta, S_i)$. The approximations are computed by taking the Taylor series expansion around θ .

$$\begin{aligned} \min_{\theta} L_{actor}(\theta) &\approx g(\theta - \theta_{old}) = \nabla_{\theta} L_{actor}(\theta)|_{\theta = \theta_{old}} \cdot (\theta - \theta_{old}) \\ \text{subject to } &\frac{1}{2}(\theta_{old} - \theta)^T H(\theta_{old} - \theta) \leq \delta \\ \text{where } H &= \nabla_{\theta}^2 \frac{1}{M} \sum_{i=1}^M D_{KL}(\theta_{old}, \theta, S_i) \Big|_{\theta = \theta_{old}} \end{aligned}$$

The analytical solution to this approximate optimization problem is as follows.

$$\theta = \theta_{old} + \alpha \sqrt{\frac{2\delta}{x^T H^{-1} x}} x$$

Here, $x = H^{-1}g$ and α is a coefficient for ensuring that the policy improves and satisfies the constraint.

TRPO Training Algorithm

TRPO agents use the following training algorithm. To configure the training algorithm, specify options using an `rlTRPOAgentOptions` object.

- 1 Generate E experiences (possibly running multiple episodes) by following the current policy.
 - a At the beginning of each episode, get the initial observation from the environment.
 - b For the current observation S , select the action A using the policy in $\pi(A|S;\theta)$.
 - c Execute action A . Observe the reward R and the next observation S' .
 - d Store the experience (S, A, R, S') .

To specify E use the `LearningFrequency` option.

- 2 Divide the E experiences into sequences each having N experiences, numbered from t_s to $t_s + N$. For each experience sequence that does not contain a terminal state, N is equal to the `ExperienceHorizon` option value. Otherwise, N is less than `ExperienceHorizon` and S_N is the terminal state.
- 3 For each step $t = t_s, t_s + 1, \dots, t_s + N - 1$, of each sequence, compute the return and advantage function using the method specified by the `AdvantageEstimateMethod` option.
 - **Finite Horizon** (`AdvantageEstimateMethod = "finite-horizon"`) — Compute the return G_t , which is the sum of the reward for that step and the discounted future reward [2].

$$G_t = \sum_{k=t+1}^{t_s+N} (\gamma^{k-t-1} R_k) + b \gamma^{t_s+N-t} V(S_{t_s+N}; \phi)$$

Here, b is 0 if S_{t_s+N} is a terminal state and 1 otherwise. That is, if S_{t_s+N} is not a terminal state, the discounted future reward includes the discounted state value function, computed using the critic network V .

Compute the advantage function D_t .

$$D_t = G_t - V(S_t; \phi)$$

- **Generalized Advantage Estimator** (`AdvantageEstimateMethod = "gae"`) — Compute the advantage function D_t , which is the discounted sum of temporal difference errors [3].

$$D_t = \sum_{k=t}^{ts+N-1} (\gamma\lambda)^{k-t} \delta_k$$

$$\delta_k = R_{k+1} + b\gamma V(S_{k+1}; \phi) - V(S_k; \phi)$$

Here, b is 0 if S_{ts+N} is a terminal state and 1 otherwise. λ is a smoothing factor specified using the `GAEFactor` option.

Compute the return G_t .

$$G_t = D_t + V(S_t; \phi)$$

To specify the discount factor γ for either method, use the `DiscountFactor` option.

4 Perform the following two operations for NumEpoch times:

- a** Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i, A_i, R_i, S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.

- b** For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-63.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1** Update the critic parameters by minimizing the loss L_{critic} across all sampled mini-batch data.

$$L_{critic}(\phi) = \frac{1}{2M} \sum_{i=1}^M (G_i - V(S_i; \phi))^2$$

- 2** Normalize the advantage values D_i based on recent unnormalized advantage values.

- If the `NormalizedAdvantageMethod` option is "none", do not normalize the advantage values.

$$\hat{D}_i \leftarrow D_i$$

- If the `NormalizedAdvantageMethod` option is "current", normalize the advantage values based on the unnormalized advantages in the current mini-batch.

$$\hat{D}_i \leftarrow \frac{D_i - \text{mean}(D_1, D_2, \dots, D_M)}{\text{std}(D_1, D_2, \dots, D_M)}$$

- If the `NormalizedAdvantageMethod` option is "moving", normalize the advantage values based on the unnormalized advantages for the N most recent advantages, including the current advantage value. To specify the window size N , use the `AdvantageNormalizingWindow` option.

$$\hat{D}_i \leftarrow \frac{D_i - \text{mean}(D_1, D_2, \dots, D_N)}{\text{std}(D_1, D_2, \dots, D_N)}$$

3 a Update the actor parameters by solving the constrained optimization problem.

i Compute the policy gradient.

$$g = \nabla_{\theta} L_{actor}(\theta) = \nabla_{\theta} - \frac{1}{M} \sum_{i=1}^M \left(\frac{\pi(A_i|S_i; \theta)}{\pi(A_i|S_i; \theta_{old})} \widehat{D}_i + w \mathcal{H}_i(\theta, S_i) \right)$$

Here:

- $\mathcal{H}_i(\theta, S_i)$ is the entropy loss and w is the entropy loss weight factor, specified using the `EntropyLossWeight` option. For more information on entropy loss, see “Entropy Loss” on page 3-65.
- $\pi(A_i|S_i; \theta)$ indicates one of the following quantities:
 - For a discrete action space — The probability of taking the discrete action A_i , given observation S_i , and parameters θ .
 - For a continuous action space — The value of the joint probability density function of the action, calculated for the action vector A_i and the observation S_i , given parameters θ :

$$\pi(A_i|S; \theta) = \prod_{j=1}^N f_j(A_{j,i}|S_i; \theta)$$

Here, f_j is the probability density function for the dimension j of the action vector, and $A_{j,i}$ is the j th element of A_i , which has dimension N .

- $\pi(A_i|S_i; \theta_{old})$ is the same as $\pi(A_i|S_i; \theta)$, except that the previous policy parameters θ_{old} from before the current learning epoch are used instead of the current parameters θ .
- ii Apply the conjugate gradient (CG) method to find an approximate solution to the following equation, where H is the Hessian of the KL-divergence between the old and new policies.

$$x \approx H^{-1}g$$

To configure the termination conditions for the CG algorithm, use the `NumIterationsConjugateGradient` and `ConjugateGradientResidualTolerance` options. To stabilize the numerical computation for the CG algorithm, use the `ConjugateGradientDamping` option.

iii Using a line search algorithm, find the largest α that satisfies the following constraints.

$$\theta = \theta_{old} + \alpha \sqrt{\frac{2\delta}{x^T H^{-1} x}} x$$

$$L_{actor}(\theta) - L_{actor}(\theta_{old}) < 0$$

$$\frac{1}{M} \sum_{i=1}^M D_{KL}(\theta_{old}, \theta, S_i) \leq \delta$$

$$\alpha \in \left\{ 1, \frac{1}{2}, \frac{1}{2^2}, \dots, \frac{1}{2^{n-1}} \right\}$$

Here, δ is the KL-divergence limit, which you set using the `KLDivergenceLimit` option. n is the number of line search iterations, which you set using the `NumIterationsLineSearch` option.

- iv If a valid value of α exists, update the parameters of the actor network to θ . If a valid value of α does not exist, do not update the actor parameters.

Entropy Loss

To promote agent exploration, you can subtract an entropy loss term $w\mathcal{H}_i(\theta, S_i)$ from the actor loss function, where w is the entropy loss weight and $\mathcal{H}_i(\theta, S_i)$ is the entropy.

The entropy value is higher when the agent is more uncertain about which action to take next. Therefore, maximizing the entropy loss term (minimizing the negative entropy loss) increases the agent uncertainty, thus encouraging exploration. To promote additional exploration, which can help the agent move out of local optima, you can specify a larger entropy loss weight.

For a discrete action space, the agent uses the following entropy value. In this case, the actor outputs the probability of taking each possible discrete action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^d(\theta, S_i) = - \sum_{k=1}^P \pi(A_k|S_i; \theta) \ln \pi(A_k|S_i; \theta)$$

Here:

- P is the number of possible discrete actions.
- $\pi(A_k|S_i; \theta)$ is the probability of taking action A_k when in state S_i following the current policy.

For a continuous action space, the agent uses the following entropy value. In this case, the actor outputs the mean and standard deviation of the Gaussian distribution for each continuous action.

$$\mathcal{H}_i(\theta, S_i) = \mathcal{H}_i^c(\theta, S_i) = \frac{1}{2} \sum_{k=1}^{N_C} \ln(2\pi \cdot e \cdot \sigma_{k,i}^2)$$

Here:

- C is the number of continuous actions output by the actor.
- $\sigma_{k,i}$ is the standard deviation for action k when in state S_i following the current policy.

References

- [1] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. "Trust Region Policy Optimization." *Proceedings of the 32nd International Conference on Machine Learning*, pp. 1889-1897. 2015.
- [2] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. "Asynchronous Methods for Deep Reinforcement Learning." arXiv, June 16, 2016. <https://doi.org/10.48550/arXiv.1602.01783>.
- [3] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." arXiv, October 20, 2018. <https://doi.org/10.48550/arXiv.1506.02438>.

See Also

Objects

`rlTRPOAgent` | `rlTRPOAgentOptions`

Related Examples

- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93
- “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”

More About

- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3
- “Create Actors, Critics, and Policy Objects” on page 4-2

Deep Deterministic Policy Gradient (DDPG) Agent

The deep deterministic policy gradient (DDPG) algorithm is an off-policy actor-critic method for environments with a continuous action-space. A DDPG agent learns a deterministic policy while also using a Q-value function critic to estimate the value of the optimal policy. It features a target actor and critic as well as an experience buffer. DDPG agents supports offline training (training from saved data, without an environment). For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a deep deterministic policy gradient agent is implemented by an `rlDDPGAgent` object.

DDPG agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Continuous

DDPG agents use the following actor and critic.

Critic	Actor
Q-value function critic $Q(S,A)$, which you create using <code>rlQValueFunction</code>	Deterministic policy actor $\pi(S)$, which you create using <code>rlContinuousDeterministicActor</code>

During training, a DDPG agent:

- Updates the actor and critic learnable parameters at each time step during learning.
- Stores past experiences using a circular experience buffer. The agent updates the actor and critic using a mini-batch of experiences randomly sampled from the buffer.
- Perturbs the action chosen by the policy using a stochastic noise model at each training step.

Actor and Critics Used by the DDPG Agent

To estimate the policy and value function, a DDPG agent maintains four function approximators:

- Actor $\pi(S;\theta)$ — The actor, with parameters θ , takes observation S and returns the corresponding action that maximizes the long-term reward. Note that π here does not represent a probability distribution, but a function that returns an action.
- Target actor $\pi_t(S;\theta_t)$ — To improve the stability of the optimization, the agent periodically updates the target actor learnable parameters θ_t using the latest actor parameter values.
- Critic $Q(S,A;\phi)$ — The critic, with parameters ϕ , takes observation S and action A as inputs and returns the corresponding expectation of the long-term reward.
- Target critic $Q_t(S,A;\phi_t)$ — To improve the stability of the optimization, the agent periodically updates the target critic learnable parameters ϕ_t using the latest critic parameter values.

Both $Q(S,A;\phi)$ and $Q_t(S,A;\phi_t)$ have the same structure and parameterization, and both $\pi(S;\theta)$ and $\pi_t(S;\theta_t)$ have the same structure and parameterization.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critic tunes the parameter values in ϕ to improve its action-value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

DDPG Agent Creation

You can create and train DDPG agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a DDPG agent with default actor and critic based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options using an `rlDDPGAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlDDPGAgent`.

Alternatively, you can create a custom actor and critic and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. You can use a custom basis function with initial parameter values or a neural network object.
- 4 Create an actor using `rlContinuousDeterministicActor`. Use the model you created in the previous step as a first input argument.
- 5 Create an approximation model for your critic. You can use a custom basis function with initial parameter values, or a neural network object.
- 6 Create a critic using `rlQValueFunction`. Use the model you created in the previous step as a first input argument.
- 7 Specify agent options using an `rlDDPGAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlDDPGAgent`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

DDPG Agent Initialization

When you create a DDPG agent, the actor and critics initialize as follows.

- The critic $Q(S,A;\phi)$ uses random parameter values in ϕ . The agent then initializes the target critic parameters ϕ_t with the same values: $\phi_t = \phi$.
- The actor $\pi(S;\theta)$ uses random parameter values in θ . The agent then initializes the target actor parameters θ_t with the same values: $\theta_t = \theta$.

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

DDPG Training Algorithm

DDPG agents use the following training algorithm, in which they update their actor and critic models at each time step. To configure the training algorithm, specify options using an `rlDDPGAgentOptions` object.

- Perform a warm start by taking a sequence of actions following the initial policy represented by $\pi(S;\theta)$.

- 1 At the beginning of each episode, get the initial observation from the environment.
- 2 For the current observation, select action $A = \pi(S;\theta) + N$, where N is stochastic noise from the noise model. To configure the noise model, use the `NoiseOptions` option.
- 3 Execute action A . Observe the reward R and the next observation S' .
- 4 Store the experience (S,A,R,S') in the experience buffer.

To specify the size of the experience buffer, use the `ExperienceBufferLength` option in the agent `rlDDPGAgentOptions` object. To specify the number of warm up actions, use the `NumWarmStartSteps` option.

- After the warm start procedure, for each training time step:
 - 1 Execute the four operations described in the warm start procedure.
 - 2 Every D_C time steps (to specify D_C use the `LearningFrequency` option), perform the following two operations for `NumEpoch` times:
 - a Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i,A_i,R_i,S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.

- b For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-70.

When `LearningFrequency` is the default value of -1, the creation of the mini-batches (described in point a) and the learning operations (described in point b) are executed after each episode is finished.

For simplicity, the actor and critic updates in this algorithm show a gradient update using basic stochastic gradient descent. The actual gradient update method depends on the optimizer you specify in the `rlOptimizerOptions` object assigned to the `rlCriticOptimizerOptions` property.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1 If S'_i is a terminal state, set the value function target y_i to R_i . Otherwise, set it to

$$y_i = R_i + \gamma Q_t(S'_i, \pi_t(S'_i; \theta_t); \phi_t)$$

The value function target is the sum of the experience reward R_i and the discounted future reward. To specify the discount factor γ , use the `DiscountFactor` option.

To compute the cumulative reward, the agent first computes a next action by passing the next observation S'_i from the sampled experience to the target actor. The agent finds the cumulative reward by passing the next action to the target critic.

If you specify a value of `NumStepsToLookAhead` equal to N , then the N -step return (which adds the rewards of the following N steps and the discounted estimated value of the state that caused the N -th reward) is used to calculate the target y_i .

- 2 Update the critic parameters by minimizing the loss L_k across all sampled experiences.

$$L_k = \frac{1}{2M} \sum_{i=1}^M (y_i - Q_k(S_i, A_i; \phi_k))^2$$

- 3 Every D_A critic updates (to set D_A , use the `PolicyUpdateFrequency` option), update the actor parameters using the following sampled policy gradient to maximize the expected discounted cumulative long-term reward.

$$\nabla_{\theta} J \approx \frac{1}{M} \sum_{i=1}^M G_{ai} G_{\pi i}$$

$$G_{ai} = \nabla_A Q(S_i, A; \phi) \quad \text{where } A = \pi(S_i; \theta)$$

$$G_{\pi i} = \nabla_{\theta} \pi(S_i; \theta)$$

Here, G_{ai} is the gradient of the critic output with respect to the action computed by the actor network, and $G_{\pi i}$ is the gradient of the actor output with respect to the actor parameters. Both gradients are evaluated for observation S_i .

- 4 At every `TargetUpdateFrequency` critic updates, update the target actor and critics depending on the target update method. For more information, see “Target Update Methods” on page 3-70.

Target Update Methods

DDPG agents update their target actor and critic parameters using one of the following target update methods.

- **Smoothing** — Update the target parameters at every time step using smoothing factor τ . To specify the smoothing factor, use the `TargetSmoothFactor` option.

$$\phi_t = \tau \phi + (1 - \tau) \phi_t \quad (\text{critic parameters})$$

$$\theta_t = \tau \theta + (1 - \tau) \theta_t \quad (\text{actor parameters})$$

- **Periodic** — Update the target parameters periodically without smoothing (`TargetSmoothFactor = 1`). To specify the update period, use the `TargetUpdateFrequency` parameter.
- **Periodic Smoothing** — Update the target parameters periodically with smoothing.

To configure the target update method, create a `rlDDPGAgentOptions` object, and set the `TargetUpdateFrequency` and `TargetSmoothFactor` parameters as shown in the following table.

Update Method	TargetUpdateFrequency	TargetSmoothFactor
Smoothing (default)	1	Less than 1
Periodic	Greater than 1	1
Periodic smoothing	Greater than 1	Less than 1

References

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous Control with Deep Reinforcement Learning." arXiv, July 5, 2019. <https://doi.org/10.48550/arXiv.1509.02971>.

[1]

See Also

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions`

Related Examples

- "Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole" on page 5-85
- "Control Water Level in a Tank Using a DDPG Agent" on page 8-2
- "Train DDPG Agent to Control Two-Thruster Sliding Vehicle" on page 8-22
- "Compare DDPG Agent to LQR Controller" on page 7-285

More About

- "Reinforcement Learning Agents" on page 3-2
- "Create Actors, Critics, and Policy Objects" on page 4-2
- "Train Reinforcement Learning Agents" on page 5-3

Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent

The twin-delayed deep deterministic (TD3) policy gradient algorithm is an off-policy actor-critic method for environments with a continuous action-space. A TD3 agent learns a deterministic policy while also using two Q-value function critics to estimate the value of the optimal policy. It features a target actor and target critics as well as an experience buffer. TD3 agents supports offline training (training from saved data, without an environment).

The TD3 algorithm is an extension of the DDPG algorithm. DDPG agents can overestimate value functions, which can produce suboptimal policies. To reduce value function overestimation, the TD3 algorithm includes the following modifications of the DDPG algorithm.

- 1 A TD3 agent learns two Q-value functions and uses the minimum value function estimate during policy updates.
- 2 A TD3 agent updates the policy and targets less frequently than the Q functions.
- 3 When updating the policy, a TD3 agent adds noise to the target action, which makes the policy less likely to exploit actions with high Q-value estimates.

You can use a TD3 agent to implement one of the following training algorithms, depending on the number of critics you specify.

- TD3 — Train the agent with two Q-value functions. This algorithm implements all three of the preceding modifications.
- Delayed DDPG — Train the agent with a single Q-value function. This algorithm trains a DDPG agent with target policy smoothing and delayed policy and target updates.

For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a TD3 agent is implemented by an `rlTD3Agent` object.

TD3 agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Continuous or discrete	Continuous

TD3 agents use the following actor and critics.

Critics	Actor
One or more Q-value function critics $Q(S,A)$, which you create using <code>rlQValueFunction</code>	Deterministic policy actor $\pi(S)$, which you create using <code>rlContinuousDeterministicActor</code>

During training, a TD3 agent:

- Updates the actor and critic learnable parameters at each time step during learning.
- Stores past experiences using a circular experience buffer. The agent updates the actor and critic using a mini-batch of experiences randomly sampled from the buffer.
- Perturbs the action chosen by the policy using a stochastic noise model at each training step.

Actor and Critics Used by the TD3 Agent

To estimate the policy and value function, a TD3 agent maintains the following function approximators:

- Deterministic actor $\pi(S;\theta)$ — The actor, with parameters θ , takes observation S and returns the corresponding action that maximizes the long-term reward. Note that π here does not represent a probability distribution, but a function that returns an action.
- Target actor $\pi_t(S;\theta_t)$ — To improve the stability of the optimization, the agent periodically updates the target actor learnable parameters θ_t using the latest actor parameter values.
- One or two Q-value critics $Q_k(S,A;\phi_k)$ — The critics, each with different parameters ϕ_k , take observation S and action A as inputs and returns the corresponding expectation of the long-term reward.
- One or two target critics $Q_{tk}(S,A;\phi_{tk})$ — To improve the stability of the optimization, the agent periodically updates the target critic learnable parameters ϕ_{tk} using the latest corresponding critic parameter values. The number of target critics matches the number of critics.

Both $\pi(S;\theta)$ and $\pi_t(S;\theta_t)$ have the same structure and parameterization.

For each critic, $Q_k(S,A;\phi_k)$ and $Q_{tk}(S,A;\phi_{tk})$ have the same structure and parameterization.

When using two critics, $Q_1(S,A;\phi_1)$ and $Q_2(S,A;\phi_2)$, each critic can have a different structure, though TD3 works best when the critics have the same structure. When the critics have the same structure, they must have different initial parameter values.

During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critics tune their parameter values to improve their action-value function estimation. After training, the parameters remain at their tuned values in the actor and critics internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

TD3 Agent Creation

You can create and train TD3 agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a TD3 agent with default actor and critics based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use an LSTM layer. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options using an `rlTD3AgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.

5 Create the agent using `rlTD3Agent`.

Alternatively, you can create custom actor and critics and use these objects to create your agent. In this case, ensure that the input and output dimensions of the actor and critics match the corresponding action and observation specifications of the environment.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. You can use a custom basis function with initial parameter values or a neural network object.
- 4 Create an actor using `rlContinuousDeterministicActor`. Use the model you created in the previous step as a first input argument.
- 5 Create an approximation model for your critic. You can use a custom basis function with initial parameter values, or a neural network object.
- 6 Create one or two critics using `rlQValueFunction`. Use the model you created in the previous step as a first input argument. If the critics have the same structure, they must have different initial parameter values.
- 7 Specify agent options using an `rlTD3AgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlTD3Agent`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

TD3 Agent Initialization

When you create a TD3 agent, the actor and critics initialize as follows.

- Each critic $Q_k(S,A;\phi_k)$ uses different random parameter values ϕ_k . The agent then initializes each target critic with the same random parameter values of the corresponding critic, $\phi_{tk} = \phi_k$.
- The actor $\pi(S;\theta)$ uses random parameter values θ . The agent then initializes the target actor with the same parameter values: $\theta_t = \theta$.

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

TD3 Training Algorithm

TD3 agents use the following training algorithm, in which they update their actor and critic models at each time step. To configure the training algorithm, specify options using an `rlTD3AgentOptions` object. Here, $K = 2$ is the number of critics and k is the critic index.

- Perform a warm start by taking a sequence of actions following the initial policy represented by $\pi(S;\theta)$.
 - 1 At the beginning of each episode, get the initial observation from the environment.

- 2 For the current observation, select action $A = \pi(S; \theta) + N$, where N is stochastic noise from the noise model. To configure the noise model, use the `ExplorationModel` option.
- 3 Execute action A . Observe the reward R and the next observation S' .
- 4 Store the experience (S, A, R, S') in the experience buffer.

To specify the size of the experience buffer, use the `ExperienceBufferLength` option in the agent `rlTD3AgentOptions` object. To specify the number of warm up actions, use the `NumWarmStartSteps` option.

- After the warm start procedure, for each training time step:
 - 1 Execute the four operations described in the warm start procedure.
 - 2 Every D_C time steps (to specify D_C use the `LearningFrequency` option), perform the following two operations for `NumEpoch` times:
 - a Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i, A_i, R_i, S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.
 - b For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-75.

When `LearningFrequency` is the default value of -1, the creation of the mini-batches (described in point a) and the learning operations (described in point b) are executed after each episode is finished.

For simplicity, the actor and critic updates in this algorithm show a gradient update using basic stochastic gradient descent. The actual gradient update method depends on the optimizer you specify in the `rlOptimizerOptions` object assigned to the `rlCriticOptimizerOptions` property.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1 If S'_i is a terminal state, set the value function target y_i to R_i . Otherwise, set it to

$$y_i = R_i + \gamma * \min_k (Q_{tk}(S'_i, \text{clip}(\pi_t(S'_i; \theta_t) + \varepsilon); \phi_{tk}))$$

The value function target is the sum of the experience reward R_i and the minimum discounted future reward from the critics. To specify the discount factor γ , use the `DiscountFactor` option.

To compute the cumulative reward, the agent first computes a next action by passing the next observation S'_i from the sampled experience to the target actor. Then, the agent adds noise ε to the computed action using the `TargetPolicySmoothModel`, and clips the action based on the upper and lower noise limits. The agent finds the cumulative rewards by passing the next action to the target critics.

If you specify a value of `NumStepsToLookAhead` equal to N , then the N -step return (which adds the rewards of the following N steps and the discounted estimated value of the state that caused the N -th reward) is used to calculate the target y_i .

- 2 Update the parameters of each critic by minimizing the loss L_k across all sampled experiences.

$$L_k = \frac{1}{2M} \sum_{i=1}^M (y_i - Q_k(S_i, A_i; \phi_k))^2$$

- 3 Every D_A critic updates (to set D_A , use the `PolicyUpdateFrequency` option), update the actor parameters using the following sampled policy gradient to maximize the expected discounted cumulative long-term reward.

$$\nabla_{\theta} J \approx \frac{1}{M} \sum_{i=1}^M G_{ai} G_{mi}$$

$$G_{ai} = \nabla_A \min_k (Q_k(S_i, A; \phi)) \quad \text{where } A = \pi(S_i; \theta)$$

$$G_{mi} = \nabla_{\theta} \pi(S_i; \theta)$$

Here, G_{ai} is the gradient of the minimum critic output with respect to the action computed by the actor network, and G_{mi} is the gradient of the actor output with respect to the actor parameters. Both gradients are evaluated for observation S_i .

- 4 Every `TargetUpdateFrequency` critic updates, update the target actor and critics depending on the target update method. For more information, see “Target Update Methods” on page 3-76.

Target Update Methods

TD3 agents update their target actor and critic parameters using one of the following target update methods.

- **Smoothing** — Update the target parameters at every time step using smoothing factor τ . To specify the smoothing factor, use the `TargetSmoothFactor` option.

$$\phi_{tk} = \tau \phi_k + (1 - \tau) \phi_{tk} \quad (\text{critic parameters})$$

$$\theta_t = \tau \theta + (1 - \tau) \theta_t \quad (\text{actor parameters})$$

- **Periodic** — Update the target parameters periodically without smoothing (`TargetSmoothFactor` = 1). To specify the update period, use the `TargetUpdateFrequency` parameter.

$$\phi_{tk} = \phi_k$$

$$\theta_t = \theta$$

- **Periodic Smoothing** — Update the target parameters periodically with smoothing.

To configure the target update method, create a `rlTD3AgentOptions` object, and set the `TargetUpdateFrequency` and `TargetSmoothFactor` parameters as shown in the following table.

Update Method	TargetUpdateFrequency	TargetSmoothFactor
Smoothing (default)	1	Less than 1
Periodic	Greater than 1	1
Periodic smoothing	Greater than 1	Less than 1

References

Fujimoto, Scott, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods." arXiv, October 22, 2018. <https://doi.org/10.48550/arXiv.1802.09477>

[1] .

See Also

Objects

`rlTD3Agent` | `rlTD3AgentOptions`

Related Examples

- "Train TD3 Agent for PMSM Control" on page 8-242
- "Identify Vulnerabilities in DC Microgrids" on page 8-383
- "Train Biped Robot to Walk Using Reinforcement Learning Agents" on page 8-144
- "Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum" on page 5-184

More About

- "Reinforcement Learning Agents" on page 3-2
- "Create Actors, Critics, and Policy Objects" on page 4-2
- "Train Reinforcement Learning Agents" on page 5-3

Soft Actor-Critic (SAC) Agent

The soft actor-critic (SAC) algorithm is an off-policy actor-critic method for environments with discrete, continuous, and hybrid action-spaces. The SAC algorithm attempts to learn the stochastic policy that maximizes a combination of the policy value and its entropy. The policy entropy is a measure of policy uncertainty given the state. A higher entropy value promotes more exploration. Maximizing both the expected discounted cumulative long-term reward and the entropy balances exploration and exploitation of the environment. A soft actor-critic agent uses two critics to estimate the value of the optimal policy, while also featuring target critics and an experience buffer. SAC agents support offline training (training from saved data, without an environment). For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, a soft actor-critic agent is implemented by an `rlSACAgent` object. This implementation uses two Q-value function critics, which prevents overestimation of the value function. Other implementations of the soft actor-critic algorithm use an additional value function critic.

Soft actor-critic agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Discrete, continuous, or hybrid.	Discrete, continuous or hybrid

Note Soft actor-critic agents with a hybrid action space do not support training with an evolutionary strategy. Also, they cannot be used to build model based agents. Finally, while you can train offline (from existing data) any SAC agent, only SAC agents with continuous action space support batch data regularizer options.

Soft actor-critic agents use the following actor and critic. In the most general case, for hybrid action spaces, the action A has a discrete part A^d and a continuous part A^c .

Critics	Actor
Q-value function critics $Q(S,A)$, which you create using <code>rlQValueFunction</code> (for continuous action spaces) or <code>rlVectorQValueFunction</code> (for discrete action spaces)	Stochastic policy actor $\pi(A S)$, which you create using <code>rlDiscreteCategoricalActor</code> (for discrete action spaces), <code>rlContinuousGaussianActor</code> (for continuous action spaces) or <code>rlHybridStochasticActor</code> for hybrid action spaces

During training, a soft actor-critic agent:

- Updates the actor and critic learnable parameters at regular intervals during learning.
- Estimates the probability distribution of the action and randomly selects an action based on the distribution.
- Updates an entropy weight term to reduce the difference between entropy and target entropy.
- Stores past experience using a circular experience buffer. The agent updates the actor and critic using a mini-batch of experiences randomly sampled from the buffer.

If the `UseExplorationPolicy` option of the agent is set to `false`, the action with maximum likelihood is always used in `sim` and `generatePolicyFunction`. As a result, the simulated agent and generated policy behave deterministically.

If the `UseExplorationPolicy` is set to `true`, the agent selects its actions by sampling its probability distribution. As a result, the policy is stochastic and the agent explores its observation space.

Note The `UseExplorationPolicy` option affects only simulation and deployment; it does not affect training. When you train an agent using `train`, the agent always uses its exploration policy independently of the value of this property.

Actor and Critics Used by the SAC Agent

To estimate the policy and value function, a soft actor-critic agent maintains the following function approximators.

- Stochastic actor $\pi(A|S;\theta)$ — The actor, with parameters θ , outputs the conditional probability of taking each action A when in state S as one of the following:
 - Discrete action space — A vector containing the probabilities of taking each discrete action. The sum of these probabilities across all actions is 1.
 - Continuous action space — Two vectors containing the means and standard deviations, respectively, of the Gaussian distribution for each continuous action. Note that the SAC algorithm bounds the continuous action selected from these two vectors.
 - Hybrid action space — A vector of probabilities for each discrete action, and two vectors representing mean and standard deviation for each continuous action.

The agent samples these probability distributions to obtain the action vector.

- One or two Q-value (or vector Q-value) critics $Q_k(S, A^c; \phi_k)$ — The critics, each with parameters ϕ_k , take observation S and the continuous part of the action A^c (if present) as inputs and return the corresponding value function (for continuous action spaces), or the value of each possible discrete action A^d (for discrete or hybrid action spaces). The value function is calculated including the entropy of the policy as well as its expected discounted cumulative long-term reward.
- One or two target critics $Q_{tk}(S, A^c; \phi_{tk})$ — To improve the stability of the optimization, the agent periodically sets the target critic parameters ϕ_{tk} to the latest corresponding critic parameter values. The number of target critics matches the number of critics.

When you use two critics, $Q_1(S, A^c; \phi_1)$ and $Q_2(S, A^c; \phi_2)$, each critic can have different structures. When the critics have the same structure, they must have different initial parameter values.

Each critic $Q_k(S, A^c; \phi_k)$ and corresponding target critic $Q_{tk}(S, A^c; \phi_{tk})$ must have the same structure and parameterization.

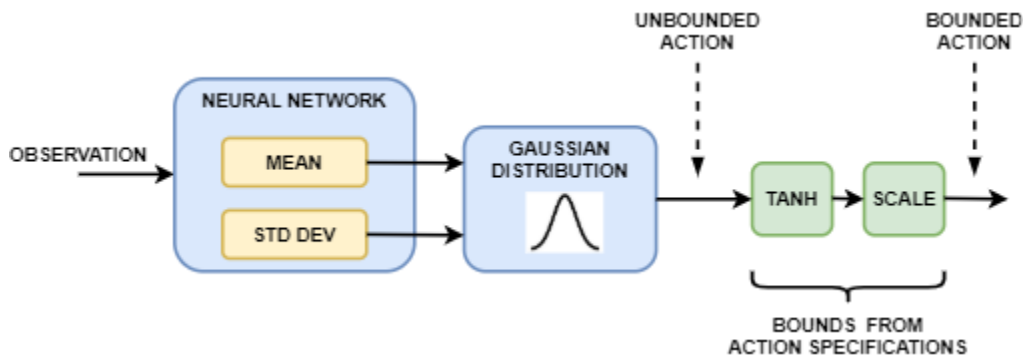
During training, the actor tunes the parameter values in θ to improve the policy. Similarly, during training, the critics tune their parameter values to improve their action-value function estimation. After training, the parameters remain at their tuned values in the actor and critic internal to the trained agent.

For more information on actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Continuous Action Generation

In a continuous action space soft actor-critic agent, the neural network in the actor takes the current observation and generates two outputs, one for the mean and the other for the standard deviation. To select an action, the actor randomly selects an unbounded action from this Gaussian distribution. If the soft actor-critic agent needs to generate bounded actions, the actor applies *tanh* and *scaling* operations to the action sampled from the Gaussian distribution.

During training, the agent uses the unbounded Gaussian distribution to calculate the entropy of the policy for the given observation.



Discrete Action Generation

In a discrete action space soft actor-critic agent, the actor takes the current observation and generates a categorical distribution, in which each possible action is associated with a probability. Because each action that belongs to the finite set is already assumed feasible, no bounding is needed.

During training, the agent uses the categorical distribution to calculate the entropy of the policy for the given observation.

Hybrid Action Generation

In a hybrid action space soft actor-critic agent, the actor takes the current observation and generates both a categorical and a Gaussian distribution, which are both used to calculate the entropy of the policy during training.

A discrete action is then sampled from the categorical distribution, and a continuous action is sampled from the Gaussian distribution. If needed, the continuous action is then also automatically bounded as for continuous action generation.

The discrete and continuous actions are then returned to the environment using two different action channels.

SAC Agent Creation

You can create and train soft actor-critic agents at the MATLAB command line or using the Reinforcement Learning Designer app. For more information on creating agents using Reinforcement Learning Designer, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.

At the command line, you can create a soft actor-critic agent with default actor and critic based on the observation and action specifications from the environment. To do so, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 If needed, specify the number of neurons in each learnable layer of the default network or whether to use a recurrent default network. To do so, create an agent initialization option object using `rlAgentInitializationOptions`.
- 4 If needed, specify agent options using an `rlSACAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 5 Create the agent using `rlSACAgent`.

Alternatively, you can create custom actor and critic objects and use them to create your agent. In this case, ensure that the input and output dimensions of the actor and critic match the corresponding action and observation specifications of the environment. To create an agent using your custom actor and critic objects, perform the following steps.

- 1 Create observation specifications for your environment. If you already have an environment object, you can obtain these specifications using `getObservationInfo`.
- 2 Create action specifications for your environment. If you already have an environment object, you can obtain these specifications using `getActionInfo`.
- 3 Create an approximation model for your actor. For continuous action spaces, this model must be a neural network object. For discrete action spaces, you also have the option of using a custom basis function with initial parameter values.
- 4 Create a stochastic actor using `rlContinuousGaussianActor` (for continuous action spaces), `rlDiscreteCategoricalActor` (for discrete action spaces), or `rlHybridStochasticActor` (for hybrid action spaces). Use the model you created in the previous step as a first input argument. For soft actor-critic agents with continuous or hybrid action spaces, the actor network must *not* contain a `tanhLayer` and `scalingLayer` as last two layers in the output path for the mean values, since the scaling already occurs automatically. However, to ensure that the standard deviation values are not negative, the actor network must contain a `reluLayer` as the last layer in the output path for the standard deviation values.
- 5 Create an approximation model for your critic. You can use a custom basis function with initial parameter values, or a neural network object. The inputs and outputs of the model you create depend on the type of critic you use in the next step.
- 6 Create one or two critics using `rlQValueFunction` (for continuous action spaces) or using `rlVectorQValueFunction` (for hybrid or discrete action spaces). Use the model you created in the previous step as a first input argument. For hybrid action spaces, the critics must take as inputs both the observation and the continuous action. If the critics have the same structure, they must have different initial parameter values.
- 7 Specify agent options using an `rlSACAgentOptions` object. Alternatively, you can skip this step and modify the agent options later using dot notation.
- 8 Create the agent using `rlSACAgent`.

For more information on creating actors and critics for function approximation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

SAC Agent Initialization

When you create a SAC agent, the actor and critics initialize as follows.

- Each critic $Q_k(S,A;\phi_k)$ uses random parameter values ϕ_k . The agent then initializes each target critic with the same random parameter values of the corresponding critic, $\phi_{tk} = \phi_k$.
- The actor $\pi(A|S;\theta)$ uses random parameter values θ .

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

SAC Training Algorithm

The soft actor-critic agent uses the following training algorithm, in which it periodically updates the actor and critic models and entropy weight. To configure the training algorithm, specify options using an `rlSACAgentOptions` object. Here, $K = 2$ is the number of critics and k is the critic index.

- Perform a warm start by taking a sequence of actions following the initial random policy in $\pi(A|S)$.
 - 1 At the beginning of each episode, get the initial observation from the environment.
 - 2 For the current observation S , select the action A (with its continuous part bounded) using the policy in $\pi(A|S;\theta)$.
 - 3 Execute action A . Observe the reward R and the next observation S' .
 - 4 Store the experience (S,A,R,S') in the experience buffer.

To specify the size of the experience buffer, use the `ExperienceBufferLength` option in the agent `rlSACAgentOptions` object. To specify the number of warm up actions, use the `NumWarmStartSteps` option.

- After the warm start procedure, for each training time step:
 - 1 Execute the four operations described in the warm start procedure.
 - 2 Every D_C time steps (to specify D_C use the `LearningFrequency` option), perform the following two operations for `NumEpoch` times:
 - a Using all the collected experiences, create a maximum of B different mini-batches. To specify B , use the `MaxMiniBatchPerEpoch` option. Each mini-batch contains M different (typically nonconsecutive) experiences (S_i,A_i,R_i,S'_i) that are randomly sampled from the experience buffer (each experience can only be part of one mini-batch). To specify M , use the `MiniBatchSize` option.

If the agent contains recurrent neural networks, each mini-batch contains M different sequences. Each sequence contains K consecutive experiences (starting from a randomly sampled experience). To specify K , use the `SequenceLength` option.
 - b For each (randomly selected) mini-batch, perform the learning operations described in “Mini-Batch Learning Operations” on page 3-82.

When `LearningFrequency` is the default value of -1, the creation of the mini-batches (described in point a) and the learning operations (described in point b) are executed after each episode is finished.

Mini-Batch Learning Operations

Operations performed for each mini-batch.

- 1 Update the parameters of each critic by minimizing the loss L_k across all sampled experiences.

$$L_k = \frac{1}{2M} \sum_{i=1}^M (y_i - Q_k(S_i, A_i; \phi_k))^2$$

To specify the optimizer options used to minimize L_k , use the options contained in the `CriticOptimizerOptions` option (which in turn contains an `rlOptimizerOptions` object).

If the agent contains recurrent neural networks, each element of the sum over the batch elements is itself a sum over the time (sequence) dimension.

If S'_i is a terminal state, the value function target y_i is set equal to the experience reward R_i . Otherwise, the value function target is the sum of R_i , the minimum discounted future reward from the critics, and the weighted entropy. The following formulas show the value function target in discrete, continuous, and hybrid action spaces, respectively.

$$y_i^d = R_i + \gamma \min_k \left(\sum_{j=1}^{N^d} \pi^d(A_j^{d'} | S'_i; \theta) Q_{tk}(S'_i, A_j^{d'}; \phi_{tk}) \right) - \alpha^d \sum_{j=1}^{N^d} \pi^d(A_j^{d'} | S'_i; \theta) \ln \pi^d(A_j^{d'} | S'_i; \theta)$$

$$y_i^c = R_i + \gamma \min_k (Q_{tk}(S'_i, A_i^{c'}; \phi_{tk})) - \alpha^c \ln \pi^c(A_i^{c'} | S'_i; \theta)$$

$$y_i^h = R_i + \gamma \min_k \left(\sum_{j=1}^{N^d} \pi^d(A_j^{d'} | S'_i; \theta) Q_{tk}(S'_i, A_j^{d'}, A_i^{c'}; \phi_{tk}) \right) - \alpha^d \sum_{j=1}^{N^d} \pi^d(A_j^{d'} | S'_i; \theta) \ln \pi^d(A_j^{d'} | S'_i; \theta) - \alpha^c \ln \pi^c(A_i^{c'} | S'_i; \theta)$$

Here:

- The superscripts d , c , and h indicate the quantity in the discrete, continuous, and hybrid cases, respectively. N^d is the number of possible discrete actions, and $A_j^{d'}$ indicates the j th action belonging to the discrete action set.
- γ is the discount factor, which you specify in the `DiscountFactor` option.
- The last two terms of the target equation for the hybrid case, (or the last term in the other cases) represent the weighted policy entropy for the output of the actor when in state S . α^d and α^c are the entropy loss weights for the discrete and continuous action spaces, which you specify by setting the `EntropyWeight` option of the respective `EntropyWeightOptions` property. To specify the other optimizer options used to tune one of the entropy term, use the other properties of the `EntropyWeightOptions` agent option.

If you specify a value of `NumStepsToLookAhead` equal to N , then the N -step return (which adds the rewards of the following N steps and the discounted estimated value of the state that caused the N -th reward) is used to calculate the target y_i .

- 2 Every D_A critic updates (to set D_A , use the `PolicyUpdateFrequency` option), perform the following two operations:
 - a Update the parameters of the actor by minimizing the following objective function across all sampled experiences. The following formulas show the objective function in discrete, continuous, and hybrid action spaces, respectively.

$$\begin{aligned}
J_{\pi}^d &= \frac{1}{M} \sum_{i=1}^M \left(-\min_k \left(\sum_{j=1}^{N^d} \pi^d(A_j^d | S_i; \theta) Q_{tk}(S_i, A_j^d; \phi_{tk}) \right) + \alpha^d \sum_{j=1}^{N^d} \pi^d(A_j^d | S_i; \theta) \ln \pi^d(A_j^d | S_i; \theta) \right) \\
J_{\pi}^c &= \frac{1}{M} \sum_{i=1}^M \left(-\min_k (Q_k(S_i, A_i^c; \phi_k)) + \alpha^c \ln \pi^c(A_i^c | S_i; \theta) \right) \\
J_{\pi}^h &= \frac{1}{M} \sum_{i=1}^M \left(-\min_k \left(\sum_{j=1}^{N^d} \pi^d(A_j^d | S_i; \theta) Q_{tk}(S_i, A_j^d, A_i^c; \phi_{tk}) \right) + \alpha^d \sum_{j=1}^{N^d} \pi^d(A_j^d | S_i; \theta) \ln \pi^d(A_j^d | S_i; \theta) + \alpha^c \ln \pi^c(A_i^c | S_i; \theta) \right)
\end{aligned}$$

To specify the optimizer options used to minimize J_{π} , use the options contained in the `ActorOptimizerOptions` option (which in turn contains an `rlOptimizerOptions` object).

If the agent contains recurrent neural networks, each element of the sum over the mini-batch elements is itself a sum over the time (sequence) dimension.

- b** Update the entropy weights by minimizing the following loss functions. When the action space is discrete or continuous, only the respective entropy weight is minimized. When the action space is hybrid, both weights are updated by minimizing both functions.

$$\begin{aligned}
L_{\alpha}^d &= \frac{1}{M} \sum_{i=1}^M \left(-\alpha^d \sum_{j=1}^{N^d} \pi^d(A_j^d | S_i; \theta) \ln \pi^d(A_j^d | S_i; \theta) - \alpha^d \mathcal{H}^d \right) \\
L_{\alpha}^c &= \frac{1}{M} \sum_{i=1}^M \left(-\alpha^c \ln \pi^c(A_i^c | S_i; \theta) - \alpha^c \mathcal{H}^c \right)
\end{aligned}$$

Here, $\mathcal{H}^d$ and $\mathcal{H}^c$ are the target entropies for the discrete and continuous cases, which you specify in the `TargetEntropy` property of the corresponding `EntropyWeightOptions.TargetEntropy` option.

- 3** Every `TargetUpdateFrequency` critic updates, update the target critics depending on the target update method. For more information, see “Target Update Methods” on page 3-84.

Target Update Methods

Soft actor-critic agents update their target critic parameters using one of the following target update methods.

- Smoothing — Update the target critic parameters using smoothing factor τ . To specify the smoothing factor, use the `TargetSmoothFactor` option.

$$\phi_{tk} = \tau \phi_k + (1 - \tau) \phi_{tk}$$

- Periodic — Update the target critic parameters periodically without smoothing (`TargetSmoothFactor = 1`). To specify the update period, use the `TargetUpdateFrequency` parameter.

$$\phi_{tk} = \phi_k$$

- Periodic smoothing — Update the target parameters periodically with smoothing.

To configure the target update method, set the `TargetUpdateFrequency` and `TargetSmoothFactor` parameters as shown in the following table.

Update Method	TargetUpdateFrequency	TargetSmoothFactor
Smoothing (default)	1	Less than 1
Periodic	Greater than 1	1
Periodic smoothing	Greater than 1	Less than 1

References

- [1] Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. “Soft Actor-Critic Algorithms and Applications.” arXiv, January 29, 2019. <https://doi.org/10.48550/arXiv.1812.05905>.
- [2] Christodoulou, Petros. “Soft Actor-Critic for Discrete Action Settings.” arXiv, October 18, 2019. <https://doi.org/10.48550/arXiv.1910.07207>.
- [3] Delalleau, Olivier, Maxim Peter, Eloi Alonso, and Adrien Logut. “Discrete and Continuous Action Representation for Practical RL in Video Games.” arXiv, December 23, 2019. <https://doi.org/10.48550/arXiv.1912.11077>

See Also

Functions

`getActionInfo` | `getObservationInfo` | `train` | `trainFromData` | `sim`

Objects

`rlSACAgent` | `rlSACAgentOptions` | `rlQValueFunction` | `rlVectorQValueFunction` | `rlContinuousGaussianActor` | `rlHybridStochasticActor` | `rlDDPGAgent` | `rlTD3Agent` | `rlACAgent` | `rlPPOAgent`

Related Examples

- “Train SAC Agent for Ball Balance Control” on page 8-181
- “Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle” on page 5-132
- “Train Hybrid SAC Agent for Path-Following Control” on page 8-109

More About

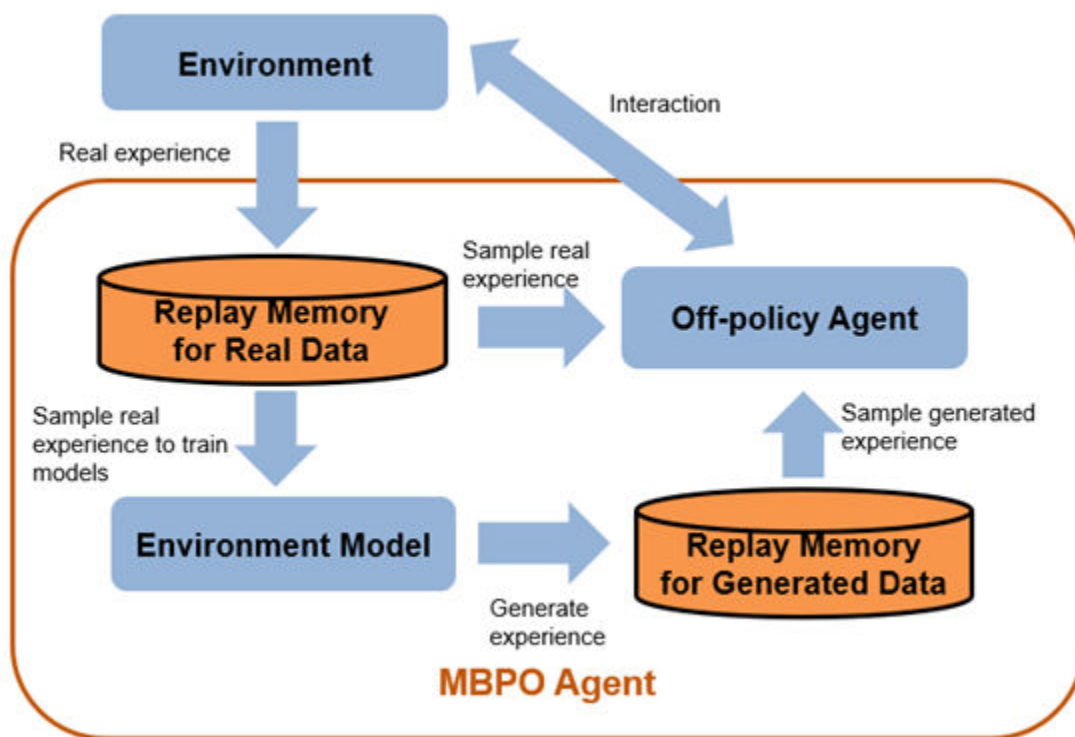
- “Reinforcement Learning Agents” on page 3-2
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Model-Based Policy Optimization (MBPO) Agent

Model-based policy optimization (MBPO) is a model-based, off-policy reinforcement learning algorithm for environments with a discrete or continuous action space. An MBPO agent contains an internal model of the environment, which it uses to generate additional experiences without interacting with the environment. Specifically, during training, the MBPO agent generates real experiences by interacting with the environment. These experiences are used to train the internal environment model, which is used to generate additional experiences. The training algorithm then uses both the real and generated experiences to update the agent policy. For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

In Reinforcement Learning Toolbox, an MBPO agent is implemented by an `rlMBPOAgent` object.

The following figure shows the components and behavior of an MBPO agent. The agent samples real experience data through environmental interaction and trains a model of the environment using this experience. Then, the agent updates the policy learnable parameters of its base agent using the real experience data and experience generated from the environment model.



Note MBPO agents do not support recurrent networks.

MBPO agents can be trained in environments with the following observation and action spaces.

Observation Space	Action Space
Continuous	Discrete or continuous

You can use the following off-policy agents as the base agent in an MBPO agent.

Action Space	Base Off-Policy Agent
Discrete	- DQN agent — “Deep Q-Network (DQN) Agent” on page 3-32 - SAC agent with a discrete action space — “Soft Actor-Critic (SAC) Agent” on page 3-78
Continuous	- DDPG agent — “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67 - TD3 agent — “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72 - SAC agent with a continuous action space — “Soft Actor-Critic (SAC) Agent” on page 3-78

Note Soft actor-critic agents with a hybrid action space cannot be used to build model based agents.

MBPO agents use an environment model that you define using an `rlNeuralNetworkEnvironment` object, which contains the following components. In general, these components use a deep neural network to learn the environment behavior during training.

- One or more transition functions that predict the next observation based on the current observation and action. You can define deterministic transition functions using `rlContinuousDeterministicTransitionFunction` objects or stochastic transition functions using `rlContinuousGaussianTransitionFunction` objects.
- A reward function that predicts the reward from the environment based on a combination of the current observation, current action, and next observation. You can define a deterministic reward function using an `rlContinuousDeterministicRewardFunction` object or a stochastic reward function using an `rlContinuousGaussianRewardFunction` object. You can also define a known reward function using a custom function.
- An is-done function that predicts the termination signal based on a combination of the current observation, current action, and next observation. You can also define a known termination signal using a custom function.

During training, an MBPO agent:

- Updates the environment model at the beginning of each episode by training the transition functions, reward function, and is-done function
- Generates samples using the trained environment model and stores the samples in a circular experience buffer
- Stores real samples from the interaction between the agent and the environment using a separate circular experience buffer within the base agent
- Updates the actor and critic of the base agent using a mini-batch of experiences randomly sampled from both the generated experience buffer and the real experience buffer

MBPO Agent Initialization

When you create an MBPO agent, the agent performs these initialization steps.

- 1 Initialize the actor and critics of the base agent.
- 2 Initialize the transition functions, reward function, and is-done function in the environment model.

The agent uses this initial actor and critic parameters at the beginning of the first training session. For each subsequent training session, the actor and critic retain the parameters from the previous session.

MBPO Training Algorithm

MBPO agents use the following training algorithm, in which they periodically update the environment model and the base off-policy agent. To configure the training algorithm, specify options using an `rlMBPOAgentOptions` object.

- 1 At the beginning of each training episode, get the initial observation from the environment, and perform the following steps:
 - a For each model-training epoch, perform the following steps. To specify the number of epochs, use the `NumEpochForTrainingModel` option.
 - i Train the transition functions. If the corresponding `LearnRate` optimizer option is 0, skip this step.
 - Use a half-mean loss for an `rlContinuousDeterministicTransitionFunction` object and a maximum likelihood loss for an `rlContinuousStochasticTransitionFunction` object.
 - To make each observation channel equally important, first compute the loss for each observation channel. Then, divide each loss by the number of elements in its corresponding observation specification.

$$Loss = \sum_{i=1}^{N_o} \frac{1}{M_{oi}} Loss_{oi}$$

For example, if the observation specification for the environment is defined by `[rlNumericSpec([10,1]) rlNumericSpec([4,1])]`, then N_o is 2, M_{o1} is 10, and M_{o2} is 4.

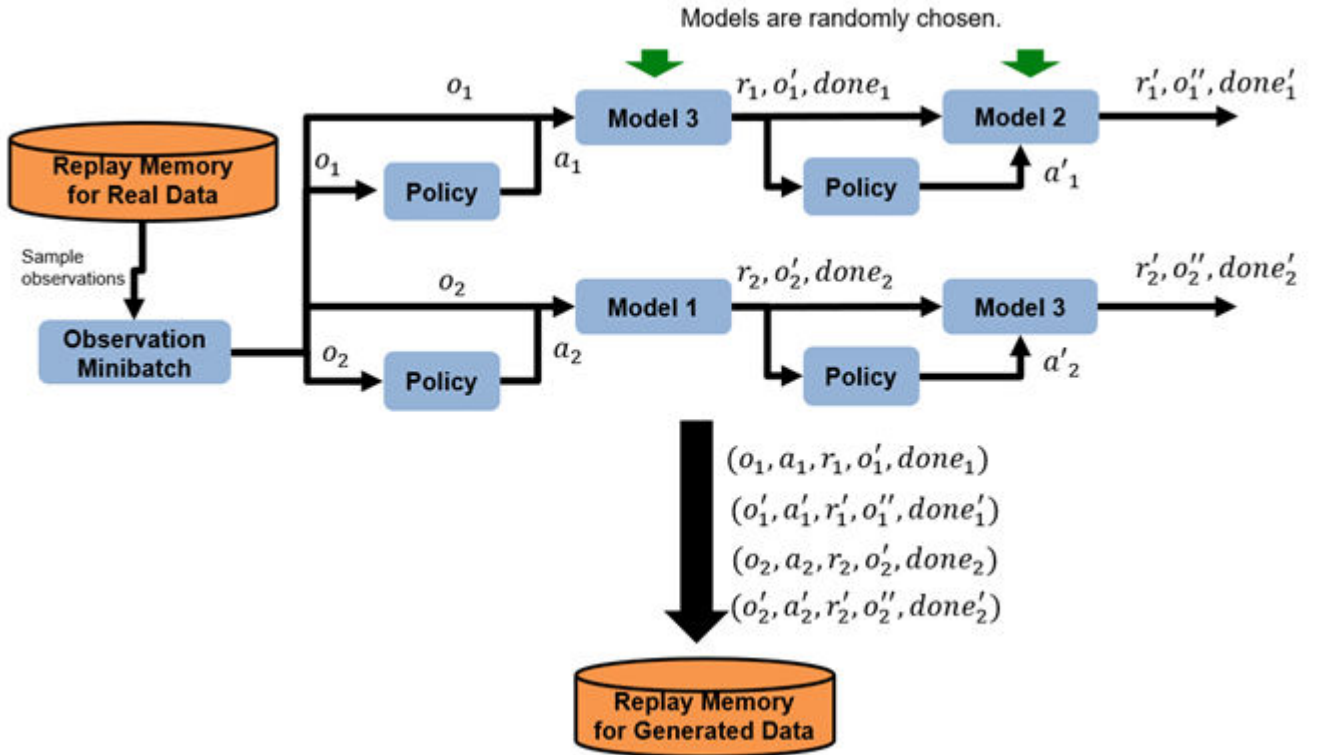
- ii Train the reward function. If the corresponding `LearnRate` optimizer option is 0 or a ground-truth custom reward function is defined, skip this step.
 - Use a half-mean loss for an `rlContinuousDeterministicRewardFunction` object and a maximum likelihood loss for an `rlContinuousStochasticRewardFunction` object.
- iii Train the is-done function. If the corresponding `LearnRate` optimizer option is 0 or a ground-truth custom is-done function is defined, skip this step.
 - Use a weighted cross-entropy loss function. In general, the terminal conditions (`isdone = 1`) occur much less frequently than nonterminal conditions (`isdone = 0`). To deal with the heavily imbalanced data, use the following weights and loss function.

$$w_0 = \frac{1}{\sum_{i=1}^M (1 - T_i)}, \quad w_1 = \frac{1}{\sum_{i=1}^M T_i}$$

$$Loss = \frac{-1}{M} \sum_{i=1}^M (w_0 T_i \ln Y_i + w_1 (1 - T_i) \ln (1 - Y_i))$$

Here, M is the mini-batch size, T_i is a target, and Y_i is the output from the reward network for the i th sample in the batch. $T_i = 1$ when `isdone` is 1 and $T_i = 0$ when `isdone` is 0.

- b Generate samples using the trained environment model. The following figure shows an example of two roll-out trajectories with a horizon of two.



- i Increase the horizon based on the horizon update settings defined in the `ModelRolloutOptions` object.
- ii Randomly sample a batch of N_R observations from the real experience buffer. To specify N_R , use the `ModelRolloutOptions.NumRollout` option.
- iii For each horizon step:
 - Randomly divide the observations into N_M groups, where N_M is the number of transition models, and assign each group to a transition model.
 - For each observation o_i , generate an action a_i using the exploration policy defined by the `ModelRolloutOptions.NoiseOptions` object. If `ModelRolloutOptions.NoiseOptions` is empty, use the exploration policy of the base agent.
 - For each observation-action pair, predict the next observation o'_2 using the corresponding transition model.

- Using the environment model reward function, predict the reward value r_i based on the observation, action, and next observation.
 - Using the environment model is-done function, predict the termination signal $done_i$ based on the observation, action, and next observation.
 - Add the experience $(o_i, a_i, r_i, o'_i, done_i)$ to the generated experience buffer.
 - For the next horizon step, substitute each observation with the predicted next observation.
- 2 For each step in each training episode:
- a Sample a mini-batch of M total experiences from the real experience buffer and the generated experience buffer. To specify M , use the `MiniBatchSize` option.
 - Sample $N_{real} = \lceil M \cdot R \rceil$ samples from the real experience buffer. To specify R , use the `RealRatio` option.
 - $N_{model} = M - N_{real}$ samples from the generated experience buffer.
 - b Train the base agent using the sampled mini-batch of data by following the update rule of the base agent. For more information, see the corresponding SAC on page 3-78, TD3 on page 3-72, DDPG on page 3-67, or DQN on page 3-32 training algorithm.

Tips

- MBPO agents can be more sample-efficient than model-free agents because the model can generate large sets of diverse experiences. However, MBPO agents require much more computational time than model-free agents, because they must train the environment model and generate samples in addition to training the base agent.
- To overcome modeling uncertainty, best practice is to use multiple environment transition models.
- If they are available, it is best to use known ground-truth reward and is-done functions.
- It is better to generate a large number of trajectories (thousands or tens of thousands). Doing so generates many samples, which reduces the likelihood of selecting the same sample multiple times in a training episode.
- Because modeling errors can accumulate, it is better to use a shorter horizon when generating samples. A shorter horizon is usually enough to generate diverse experiences.
- In general, an agent created using `rLMBPOAgent` is not suitable for environments with image observations.
- When using a SAC base agent, taking more gradient steps (defined by the `NumGradientStepsPerUpdate` SAC agent option) makes the MBPO agent more sample-efficient. However, doing so increases the computational time.
- The MBPO implementation in `rLMBPOAgent` is based on the algorithm in the original MBPO paper [1] but with the differences shown in the following table.

Original Paper	rLMBPOAgent
Generates samples at each environment step	Generates samples at the beginning of each training episode
Trains actor and critic using only generated samples	Trains actor and critic using both real data and generated data

Original Paper	rMBPOAgent
Uses stochastic environment models	Uses either stochastic or deterministic environment models
Uses SAC agents	Can use SAC, DQN, DDPG, and TD3 agents

References

- [1] Janner, Michael, Justin Fu, Marvin Zhang, and Sergey Levine. “When to Trust Your Model: Model-Based Policy Optimization.” In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 12519–30. 1122. Red Hook, NY, USA: Curran Associates Inc., 2019.

See Also

Objects

`rMBPOAgent` | `rMBPOAgentOptions` | `rlNeuralNetworkEnvironment`

Related Examples

- “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356

Create Custom Reinforcement Learning Agents

To implement your own custom reinforcement learning algorithms, you can create a custom agent by creating a subclass of a custom agent class. You can then train and simulate this agent in MATLAB and Simulink environments. For more information about creating classes in MATLAB, see “User-Defined Classes”.

Create Template Class

To define your custom agent, first create a class that is a subclass of the `rl.agent.CustomAgent` class. As an example, this topic describes the custom LQR agent trained in “Create and Train Custom LQR Agent” on page 5-345. As a starting point for your own agent, you can open and modify this custom agent class. To download the example files in a local folder and open the main example live script, at the MATLAB command line, type the following code.

```
openExample('rl/TrainCustomLQRAgentExample')
```

Close the `TrainCustomLQRAgentExample.mlx` file and open the `LQRCustomAgent.m` class file.

```
edit LQRCustomAgent.m
```

After saving the class to your own working folder, you can remove the example files and the local folder in which they were downloaded.

The class defined in `LQRCustomAgent.m` has the following class definition, which indicates the agent class name and the associated abstract agent.

```
classdef LQRCustomAgent < rl.agent.CustomAgent
```

To define your agent, you must specify the following:

- Agent properties
- Constructor function
- A critic that estimates the discounted long-term reward (if required for learning)
- An actor that selects an action based on the current observation (if required for learning)
- Required agent methods
- Optional agent methods

Agent Properties

In the `properties` section of the class file, specify any parameters necessary for creating and training the agent. These parameters can include:

- Discount factor for discounting future rewards
- Configuration parameters for exploration models, such as noise models or epsilon-greedy exploration
- Experience buffers for using replay memory
- Mini-batch sizes for sampling from the experience buffer
- Number of steps to look ahead during training

For more information on potential agent properties, see the option objects for the built-in Reinforcement Learning Toolbox agents.

The `rl.Agent.CustomAgent` class already includes properties for the agent sample time (`SampleTime`) and the action and observation specifications (`ActionInfo` and `ObservationInfo`, respectively).

The custom LQR agent defines the following agent properties.

```
properties
    % Q
    Q

    % R
    R

    % Feedback gain
    K

    % Discount factor
    Gamma = 0.95

    % Critic
    Critic

    % Buffer for K
    KBuffer
    % Number of updates for K
    KUpdate = 1

    % Number for estimator update
    EstimateNum = 10
end

properties (Access = private)
    Counter = 1
    YBuffer
    HBuffer
end
```

Constructor Function

To create your custom agent, you must define a constructor function that:

- Defines the action and observation specifications. For more information about creating these specifications, see `rlNumericSpec` and `rlFiniteSetSpec`.
- Creates actor and critic as required by your training algorithm. For more information, see “Create Actors, Critics, and Policy Objects” on page 4-2.
- Configures agent properties.
- Calls the constructor of the base abstract class.

For example, the `LQRCustomAgent` constructor defines continuous action and observation spaces and creates a critic. The `createCritic` function is an optional helper function that defines the critic.

```
function obj = LQRCustomAgent(Q,R,InitialK)
    % Check the number of input arguments
```

```
narginchk(3,3);

% Call the abstract class constructor
obj = obj@rl.agent.CustomAgent();

% Set the Q and R matrices
obj.Q = Q;
obj.R = R;

% Define the observation and action spaces
obj.ObservationInfo = rlNumericSpec([size(Q,1),1]);
obj.ActionInfo = rlNumericSpec([size(R,1),1]);

% Create the critic
obj.Critic = createCritic(obj);

% Initialize the gain matrix
obj.K = InitialK;

% Initialize the experience buffers
obj.YBuffer = zeros(obj.EstimateNum,1);
num = size(Q,1) + size(R,1);
obj.HBuffer = zeros(obj.EstimateNum,0.5*num*(num+1));
obj.KBuffer = cell(1,1000);
obj.KBuffer{1} = obj.K;
end
```

Actor and Critic

If your learning algorithm uses a critic to estimate the long-term reward, an actor for selecting an action, or both, you must add these as agent properties. You must then create these objects when you create your agent; that is, in the constructor function. For more information on creating actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

For example, the custom LQR agent uses a critic, stored in its `Critic` property, and no actor. The critic creation is implemented in the `createCritic` helper function, which is called from the `LQRCustomAgent` constructor.

```
function critic = createCritic(obj)
    nQ = size(obj.Q,1);
    nR = size(obj.R,1);
    n = nQ+nR;
    w0 = 0.1*ones(0.5*(n+1)*n,1);
    critic = rlQValueFunction(@(x,u) computeQuadraticBasis(x,u,n),w0, ...
        getObservationInfo(obj),getActionInfo(obj));
    critic.Options.GradientThreshold = 1;
end
```

In this case, the critic is an `rlQValueFunction` object. To create this object, you must specify the handle to a custom basis function, in this case the `computeQuadraticBasis` function. For more information, see “Create and Train Custom LQR Agent” on page 5-345.

Required Functions

To create a custom reinforcement learning agent you must define the following implementation functions. To call these functions in your own code, use the wrapper methods from the abstract base class. For example, to call `getActionImpl`, use `getAction`. The wrapper methods have the same input and output arguments as the implementation methods.

Function	Description
<code>getActionImpl</code>	Selects an action by evaluating the agent policy for a given observation
<code>getActionWithExplorationImpl</code>	Selects an action using the exploration model of the agent
<code>learnImpl</code>	Learns from the current experiences and returns an action with exploration

Within your implementation functions, to evaluate your actor and critic, you can use the `getValue`, `getAction`, and `getMaxQValue` functions.

- To evaluate an `rlValueFunction` critic, you need only the observation input, and you can obtain the value of the current observation V using the following syntax.

```
V = getValue(Critic, Observation);
```

- To evaluate an `rlQValueFunction` critic you need both observation and action inputs, and you can obtain the value of the current state-action Q using the following syntax.

```
Q = getValue(Critic, [Observation, Action]);
```

- To evaluate an `rlVectorQValueFunction` critic you need only the observation input, and you can obtain the value of the current observation Q for all possible discrete actions using the following syntax.

```
Q = getValue(Critic, Observation);
```

- For a discrete action space `rlQValueFunction` critic, obtain the maximum Q state-action value function Q for all possible discrete actions using the following syntax.

```
[MaxQ, MaxActionIndex] = getMaxQValue(Critic, Observation);
```

- To evaluate an actor, obtain the action A using the following syntax.

```
A = getAction(Actor, Observation);
```

For each of these cases, if your actor or critic network uses a recurrent neural network, the functions can also return the current values of the network state after obtaining the corresponding network output.

getActionImpl Function

The `getActionImpl` function evaluates the policy of your agent and selects an action. This function must have the following signature, where `obj` is the agent object, `Observation` is the current observation, and `action` is the selected action.

```
function action = getActionImpl(obj, Observation)
```

For the custom LQR agent, you select an action by applying the $u = -Kx$ control law.

```
function action = getActionImpl(obj, Observation)
    % Given the current state of the system, return an action
    action = -obj.K*Observation{:};
end
```

getActionWithExplorationImpl Function

The `getActionWithExplorationImpl` function selects an action using the exploration model of your agent. Using this function you can implement algorithms such as epsilon-greedy exploration.

This function must have the following signature, where `obj` is the agent object, `Observation` is the current observation, and `action` is the selected action.

```
function action = getActionWithExplorationImpl(obj,Observation)
```

For the custom LQR agent, the `getActionWithExplorationImpl` function adds random white noise to an action selected using the current agent policy.

```
function action = getActionWithExplorationImpl(obj,Observation)
    % Given the current observation, select an action
    action = getAction(obj,Observation);

    % Add random noise to the action
    num = size(obj.R,1);
    action = action + 0.1*randn(num,1);
end
```

learnImpl Function

The `learnImpl` function defines how the agent learns from the current experience. This function implements the custom learning algorithm of your agent by updating the policy parameters and selecting an action with exploration. This function must have the following signature, where `obj` is the agent object, `exp` is the current agent experience, and `action` is the selected action.

```
function action = learnImpl(obj,exp)
```

The agent experience is the cell array `exp = {state,action,reward,nextstate,isdone}`.

- `state` is the current observation.
- `action` is the current action.
- `reward` is the current reward.
- `nextState` is the next observation.
- `isDone` is a logical flag indicating that the training episode is complete.

For the custom LQR agent, the critic parameters are updated every `N` steps.

```
function action = learnImpl(obj,exp)
    % Parse the experience input
    x = exp{1}{1};
    u = exp{2}{1};
    dx = exp{4}{1};
    y = (x'*obj.Q*x + u'*obj.R*u);
    num = size(obj.Q,1) + size(obj.R,1);

    % Wait N steps before updating the critic parameters
    N = obj.EstimateNum;
    h1 = computeQuadraticBasis(x,u,num);
    h2 = computeQuadraticBasis(dx,-obj.K*dx,num);
    H = h1 - obj.Gamma* h2;
    if obj.Counter<=N
        obj.YBuffer(obj.Counter) = y;
        obj.HBuffer(obj.Counter,:) = H;
        obj.Counter = obj.Counter + 1;
    else
        % Update the critic parameters based on the batch of
        % experiences
```



```

H_buf = obj.HBuffer;
y_buf = obj.YBuffer;
theta = (H_buf'*H_buf)\H_buf'*y_buf;
obj.Critic = setLearnableParameters(obj.Critic,{theta});

% Derive a new gain matrix based on the new critic parameters
obj.K = getNewK(obj);

% Reset the experience buffers
obj.Counter = 1;
obj.YBuffer = zeros(N,1);
obj.HBuffer = zeros(N,0.5*num*(num+1));
obj.KUpdate = obj.KUpdate + 1;
obj.KBuffer{obj.KUpdate} = obj.K;
end

% Find and return an action with exploration
action = getActionWithExploration(obj,exp{4});
end

```

Optional Functions

Optionally, you can define how your agent is reset at the start of training by specifying a `resetImpl` function with the following function signature, where `obj` is the agent object. Using this function, you can set the agent into a known or random condition before training.

```
function resetImpl(obj)
```

Also, you can define any other helper functions in your custom agent class as required. For example, the custom LQR agent defines a `createCritic` function for creating the critic and a `getNewK` function that derives the feedback gain matrix from the trained critic parameters.

Create Custom Agent

After you define your custom agent class, create an instance of it in the MATLAB workspace. For example, to create the custom LQR agent, define the `Q`, `R`, and `InitialK` values and call the constructor function.

```

Q = [10,3,1;3,5,4;1,4,9];
R = 0.5*eye(3);
K0 = place(A,B,[0.4,0.8,0.5]);
agent = LQRCustomAgent(Q,R,K0);

```

After validating the environment object, you can use it to train a reinforcement learning agent. For an example that trains the custom LQR agent, see “Create and Train Custom LQR Agent” on page 5-345.

See Also

Functions

`train`

Related Examples

- “Create and Train Custom PG Agent” on page 5-334

- “Create and Train Custom LQR Agent” on page 5-345
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273

More About

- “Reinforcement Learning Agents” on page 3-2
- “Create Actors, Critics, and Policy Objects” on page 4-2

Actors, Critics, and Policy Objects

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Import Neural Network Models Using ONNX” on page 4-16

Create Actors, Critics, and Policy Objects

A reinforcement learning policy is a mapping from an environment observation to a probability distribution of the actions to be taken (starting from the state corresponding to the observation). A value (or Q-value) function is a mapping from an environment observation (or observation-action pair) to the value of a policy. The value of a policy is defined as its expected discounted cumulative long-term reward.

Reinforcement learning agents use *parameterized* policies and value functions, which are implemented by function approximators called actors and critics, respectively. During training, the actor learns the policy that selects the best action to take. It does so by tuning its parameters to assign larger probability to actions that yield the greater values. The critic learns the value (or Q-value) function that estimates the value of the current policy. It does so by tuning its parameters so that the predicted rewards approximate the observed ones.

Before creating a non-default agent, you must create the actor and critic using approximation models such as deep neural networks, linear basis functions, or lookup tables. The type of function approximator and model you can use depends on the type of agent that you want to create.

You can also create policy objects from agents, actors, or critics. You can train these objects using custom loops and deploy them in applications.

The following sections are an introduction to actor, critic, and policy objects, as well as their internal approximation models. For an introduction to agents, see “Reinforcement Learning Agents” on page 3-2.

Actors and Critic Objects

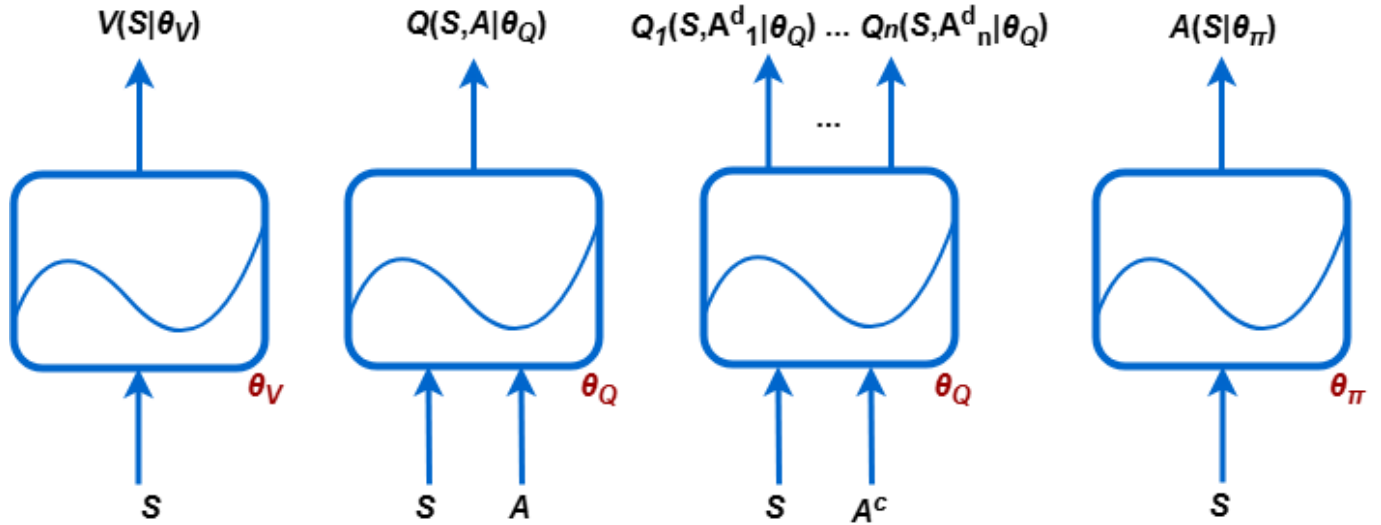
Reinforcement Learning Toolbox software supports the following types of actors and critics:

- $V(S;\theta_V)$ — Critics that estimate the expected discounted cumulative long-term reward of a policy based on a given observation S . You can create these critics using `rlValueFunction`.
- $Q(S,A;\theta_Q)$ — Critics that estimate the expected discounted cumulative long-term reward of a policy for a given discrete action A and a given observation S . You can create these critics using `rlQValueFunction`.
- $Q_i(S,A^c,A^d_i|\theta_Q)$ — Vector (also referred to as multi-outputs) critics that estimate the expected discounted cumulative long-term reward of a policy for all possible discrete actions A^d_i given the observation S and (if present) the continuous action A^c . You can create these critics using `rlVectorQValueFunction`.
- $\pi(A|S;\theta_\pi)$ — Deterministic actors with a continuous action space that select an action based on a given observation S , thereby implementing a deterministic policy. You can create these actors using `rlContinuousDeterministicActor`.
- $\pi(A|S;\theta_\pi)$ — Stochastic actors with a discrete, continuous, or hybrid action space that select an action by sampling a probability distribution based on a given observation S , thereby implementing a stochastic policy. You can create these actors using `rlDiscreteCategoricalActor` (for discrete action spaces), `rlContinuousGaussianActor` (for continuous action spaces), or `rlHybridStochasticActor`, (for hybrid action spaces).

Note that, unless otherwise specified, when the notation $\pi(A|S;\theta)$ is used in formulas, it indicates, (in the more general case in which you have an hybrid action space), the value of the joint probability density function of both the discrete and continuous actions, calculated for the action vector A and

the observation S , given parameters θ . This quantity reduces itself to the joint probability density function of actions for continuous-only action spaces and to the probability of taking a discrete action for discrete-only action spaces.

Each function approximator (actor or critic) uses a set of parameters $(\theta_v, \theta_Q, \theta_\pi)$, which are computed during the learning process.



For systems with a limited number of discrete observations and discrete actions, you can store value functions in a lookup table. For systems that have many discrete observations and actions and for observation and action spaces that are continuous, storing the observations and actions is impractical or impossible. For such systems, you can represent your actors and critics using deep neural networks or custom (linear in the parameters) basis functions.

The following table summarizes the way in which you can use the six approximator objects available with Reinforcement Learning Toolbox software, depending on the action and observation spaces of your environment, and on the approximation model and agent that you want to use.

Critics Used in Agents

Critic	Supported Model	Observation Space	Action Space	Supported Agents
Value function critic $V(S)$, which you create using <code>rlValueFunction</code>	Table	Discrete	Not applicable	PG, AC, PPO
	Deep neural network or custom basis function	Discrete or continuous	Not applicable	PG, AC, PPO
	Deep neural network	Discrete or continuous	Not applicable	TRPO
Q-value function critic, $Q(S,A)$, which you create using <code>rlQValueFunction</code>	Table	Discrete	Discrete	Q, SARSA, DQN
	Custom basis function	Discrete or continuous	Discrete	LSPI
	Deep neural network or custom basis function	Discrete or continuous	Discrete	Q, SARSA, DQN
			Continuous	DDPG, TD3, SAC
Multi-output Q-value function critic with a hybrid or discrete action space $Q(S,A)$, which you create using <code>rlVectorQValueFunction</code>	Deep neural network or custom basis function	Discrete or continuous	Hybrid or discrete	Q, SARSA, DQN, SAC

Actors Used in Agents

Actor	Supported Model	Observation Space	Action Space	Supported Agents
Deterministic policy actor with a continuous action space $\pi(S)$, which you create using <code>rlContinuousDeterministicActor</code>	Deep neural network or custom basis function	Discrete or continuous	Continuous	DDPG, TD3
Stochastic policy actor with a discrete action space $\pi(S)$, which you create using <code>rlDiscreteCategoricalActor</code>	Deep neural network or custom basis function	Discrete or continuous	Discrete	PG, AC, PPO, SAC
	Deep neural network	Discrete or continuous	Discrete	TRPO
Stochastic policy actor with a continuous action space $\pi(S)$, which you create using <code>rlContinuousGaussianActor</code>	Deep neural network	Discrete or continuous	Continuous	PG, AC, PPO, SAC, TRPO
Stochastic policy actor with a hybrid action space $\pi(S)$, which you create using <code>rlHybridStochasticActor</code>	Deep neural network	Discrete or continuous	Hybrid (discrete plus continuous)	SAC, PPO

Note `rlDiscreteCategoricalActor`, which is used in agents such as PG, AC, PPO and TRPO when the action space is discrete, does not support tables as approximation model.

You can configure the actor and critic optimization options using the `rlOptimizerOptions` object within an agent option object.

Specifically, you can create an agent options object and set its `CriticOptimizerOptions` and `ActorOptimizerOptions` properties to appropriate `rlOptimizerOptions` objects. Then you pass the agent options object to the function that creates the agent.

Alternatively, you can create the agent and then use dot notation to access the optimization options for the agent actor and critic, for example:

```
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 0.1;
```

For an example on how to create an approximator with a mixed (discrete and continuous) observation space, see the last examples in `rlValueFunction` and `rlQValueFunction`.

For more information on agents, see “Reinforcement Learning Agents” on page 3-2.

Policy Objects

You can extract a policy object from an agent using `getGreedyPolicy` or `getExplorationPolicy`, or you can create a policy object from an actor or critic.

Once you have the policy object, you can then use the `getAction` function to generate deterministic or stochastic actions from it, given an input observation. Differently from function approximator objects like actors and critics, policy objects do not have functions that you can use to easily calculate gradients with respect to parameters. Therefore, policy objects are more tailored toward application deployment, rather than training. The following table describes the available policy objects.

Policy Objects

Policy Object and <code>getAction</code> Behavior	Distribution and Exploration	Action Space	Approximator Objects Used for Creation	Agents Needed for Extraction
<code>rlMaxQPolicy</code> Generates actions that maximize a discrete action-space Q-value function	Deterministic (no exploration) and greedy.	Discrete	<code>rlQValueFunction</code> or <code>rlVectorQValueFunction</code>	Q, SARSA, LSPI, DQN
<code>rlEpsilonGreedyPolicy</code> Generates either actions that maximize a discrete action-space Q-value function with probability $1 - \text{Epsilon}$ or random actions otherwise	Default: Stochastic (random actions help exploration)	Discrete	<code>rlQValueFunction</code> or <code>rlVectorQValueFunction</code>	Q, SARSA, LSPI, DQN
<code>rlDeterministicActorPolicy</code> Generates continuous deterministic actions	Deterministic (no exploration) and greedy.	Continuous	<code>rlContinuousDeterministicActor</code>	DDPG, TD3
<code>rlAdditiveNoisePolicy</code> Generates continuous deterministic actions with added noise according to an internal noise model	Default: Stochastic (noise helps exploration)	Continuous	<code>rlContinuousDeterministicActor</code>	DDPG, TD3
<code>rlStochasticActorPolicy</code> Generates stochastic actions according to a probability distribution	Default: Stochastic (random actions help exploration)	Discrete	<code>rlDiscreteCategoricalActor</code>	PG, AC, PPO, TRPO, SAC
		Continuous	<code>rlContinuousGaussianActor</code>	PG, AC, PPO, TRPO, SAC
<code>rlHybridStochasticActorPolicy</code> Generates a hybrid stochastic actions according to a discrete and a continuous probability distributions	Default: Stochastic (random actions help exploration)	Hybrid	<code>rlHybridStochasticActor</code>	SAC, PPO

Each one of the stochastic policy objects has an option to enable deterministic behavior, thereby disabling exploration. Except for `rlEpsilonGreedyPolicy` and `rlAdditiveNoisePolicy`, you can use the `generatePolicyBlock` and `generatePolicyFunction` functions to generate a Simulink block or a function that evaluates the policy, returning an action, for a given observation input. You can then use the generated function or block to generate code for application deployment. For more information, see “Generate Code from Trained Reinforcement Learning Policies” on page 6-2.

Table Approximation Models

Value function approximators (critics) based on lookup tables models are appropriate for environments with a limited number of *discrete* observations and actions. You can create two types of lookup tables:

- Value tables, which store rewards for corresponding observations
- Q-tables, which store rewards for corresponding observation-action pairs

To create a table based critic, first create a value table or Q-table using the `rlTable` function. Then use the table object as input argument for either `rlValueFunction` or `rlQValueFunction` to create the approximator object.

Some discrete action space agents do not support table models in their actors or critics. While in general for tabular problems with a small observation and action spaces agents like Q, SARSA and DQN suffice, you can still use more complex agents by using a custom basis function to implement a table. For an examples on how to do so, see “Compare Agents on Basic Grid World” on page 7-2. Alternatively, you could also implement a table using a neural network with a custom table layer. For an example on how to do this, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Custom Basis Function Approximation Models

Custom (linear in the parameters) basis function approximation models have the form $f = W'B$, where W is a weight array and B is the column vector output of a custom basis function that you must create. The learnable parameters of a linear basis function are the elements of W .

For value function critics, (such as the ones used in AC, PG or PPO agents), f is a scalar value, so W must be a column vector with the same length as B , and B must be a function of the observation. For more information and examples, see `rlValueFunction`.

For single-output Q-value function critics, (such as the ones used in Q, SARSA, LSPI, DQN, DDPG, TD3, and continuous SAC agents), f is a scalar value, so W must be a column vector with the same length as B , and B must be a function of both the observation and action. For more information and examples, see `rlQValueFunction`.

For multi-output Q-value function critics with discrete action spaces, (such as those used in Q, SARSA, DQN and discrete SAC agents), f is a vector with as many elements as the number of possible actions. Therefore W must be a matrix with as many columns as the number of possible actions and as many rows as the length of B . B must be only a function of the observation. For more information and examples, see `rlVectorQValueFunction`.

- For deterministic actors with a continuous action space (such as the ones in DDPG, and TD3 agents), the dimensions of f must match the dimensions of the agent action specification, which is

either a scalar or a column vector. For more information and examples, see `rlContinuousDeterministicActor`.

- For stochastic actors with a discrete action space (such as the ones in PG, AC, PPO and discrete SAC agents), `f` must be column vector with length equal to the number of possible discrete actions. The output of the actor is `softmax(f)`, which represents the probability of selecting each possible action. For more information and examples, see `rlDiscreteCategoricalActor`.
- Stochastic actors with continuous action spaces cannot rely on custom basis functions (they can only use neural network approximators, due to the need to enforce positivity for the standard deviations). For more information and examples, see `rlContinuousGaussianActor`.

For any actor, `W` must have as many columns as the number of elements in `f`, and as many rows as the number of elements in `B`. `B` must be only a function of the observation.

Note Using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

For an example that trains a custom agent that uses a linear basis function, see “Create and Train Custom LQR Agent” on page 5-345.

Neural Network Approximation Models

You can create actor and critic function approximators using deep neural networks models. Doing so uses Deep Learning Toolbox software features.

Network Input and Output Dimensions

The dimensions of the network input and output layers for your actor and critic must match the dimension of the corresponding environment observation and action channels, respectively. To obtain the action and observation specifications from the environment `env`, use the `getActionInfo` and `getObservationInfo` functions, respectively.

```
actInfo = getActionInfo(env);  
obsInfo = getObservationInfo(env);
```

Access the `Dimensions` property of each channel. For example, get the size of the first environment and action channel:

```
actSize = actInfo(1).Dimensions;  
obsSize = obsInfo(1).Dimensions;
```

In general `actSize` and `obsSize` are row vectors whose elements are the lengths of the corresponding dimensions. For example, if the first observation channel is a 256-by-256 RGB image, `actSize` is the vector `[256 256 3]`. To calculate the total number of dimension of the channel, use `prod`. For example, assuming the environment has only one observation channel:

```
obsDimensions = prod(obsInfo.Dimensions);
```

For `rlVectorQValueFunction` critics and `rlDiscreteCategoricalActor` actors, you need to obtain the number of possible elements of the action set. You can do so by accessing the `Elements` property of the action channel. For example, assuming the environment has only one action channel:

```
actNumElements = numel(actInfo.Elements);
```

Networks for value function critics (such as the ones used in AC, PG, PPO or TRPO agents) must take only observations as inputs and must have a single scalar output. For these networks, the dimensions of the input layers must match the dimensions of the environment observation channels. For more information, see `rlValueFunction`.

Networks for single-output Q-value function critics (such as the ones used in Q, DQN, SARSA, DDPG, TD3, and SAC agents) must take both observations and actions as inputs, and must have a single scalar output. For these networks, the dimensions of the input layers must match the dimensions of the environment channels for both observations and actions. For more information, see `rlQValueFunction`.

Networks for multi-output Q-value function critics (such as those used in Q, DQN, and SARSA agents) take only observations as inputs and must have a single output layer with output size equal to the number of possible discrete actions. For these networks the dimensions of the input layers must match the dimensions of the environment observations channels. For more information, see `rlVectorQValueFunction`.

For actor networks, the dimensions of the input layers must match the dimensions of the environment observation channels and the dimension of the output layer must be as follows.

- Networks used in actors with a discrete action space (such as the ones in PG, AC, and PPO agents) must have a single output layer with an output size equal to the number of possible discrete actions. For more information, see `rlDiscreteCategoricalActor`.
- Networks used in deterministic actors with a continuous action space (such as the ones in DDPG and TD3 agents) must have a single output layer with an output size matching the dimension of the action space defined in the environment action specification. For more information, see `rlContinuousDeterministicActor`.
- Networks used in stochastic actors with a continuous action space (such as the ones in PG, AC, PPO, and SAC agents) must have a two output layers each with as many elements as the dimension of the action space, as defined in the environment specification. One output layer must produce the mean values (which must be scaled to the output range of the action), and the other must produce the standard deviations of the actions (which must be non-negative). For more information, see `rlContinuousGaussianActor`.

Deep Neural Networks

Deep neural networks consist of a series of interconnected layers. You can specify a deep neural network as one of the following:

- Array of Layer objects
- `dlnetwork` object
- `SeriesNetwork` object
- `layerGraph` object
- `DAGNetwork` object

Note Among the different network objects, `dlnetwork` is preferred, since it has built-in validation checks and supports automatic differentiation. If you pass another network object as an input argument, it is internally converted to a `dlnetwork` object. However, best practice is to convert other network objects to `dlnetwork` explicitly *before* using it to create a critic or an actor for a

reinforcement learning agent. You can do so using `dlnet=dlnetwork(net)`, where `net` is any neural network object from the Deep Learning Toolbox. The resulting `dlnet` is the `dlnetwork` object that you use for your critic or actor. This practice allows a greater level of insight and control for cases in which the conversion is not straightforward and might require additional specifications.

Typically, you build your neural network by stacking together a number of layers in an array of `Layer` objects, possibly adding these arrays to a `layerGraph` object, and then converting the final result to a `dlnetwork` object.

For agents that need multiple input or output layers, you create an array of `Layer` objects for each input path (observations or actions) and for each output path (estimated rewards or actions). You then add these arrays to a `layerGraph` object and connect the paths together using the `connectLayers` function.

You can also create your deep neural network using the Deep Network Designer app. For an example, see “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217.

The following table lists some common deep learning layers used in reinforcement learning applications. For a full list of available layers, see “List of Deep Learning Layers”.

Deep Learning Toolbox Layer	Description
<code>featureInputLayer</code>	Inputs feature data. Normalization is not supported.
<code>imageInputLayer</code>	Inputs vectors and 2-D images. Normalization is not supported.
<code>sequenceInputLayer</code>	Provides inputs sequence data to a network. Normalization is not supported.
<code>fullyConnectedLayer</code>	Multiplies the input vector by a weight matrix, and add a bias vector.
<code>convolution2dLayer</code>	Applies sliding convolutional filters to the input.
<code>concatenationLayer</code>	Concatenates inputs along a specified dimension.
<code>additionLayer</code>	Adds the outputs of multiple layers together.
<code>reluLayer</code>	Sets any input values that are less than zero to zero.
<code>sigmoidLayer</code>	Applies a sigmoid function to the input such that the output is bounded in the interval (0,1).
<code>tanhLayer</code>	Applies a hyperbolic tangent activation layer to the input.
<code>softmaxLayer</code>	Applies a softmax function layer to the input, normalizing it to a probability distribution.
<code>lstmLayer</code>	Applies a Long Short-Term Memory layer to the input. Supported for DQN and PPO agents.
<code>scalingLayer</code>	Applies a linear scale and offset (bias) to an input array. This layer is useful for scaling and shifting the outputs of nonlinear layers, such as <code>tanhLayer</code> and <code>sigmoidLayer</code> .
<code>softplusLayer</code>	Implements the softplus activation $Y = \log(1 + e^X)$, which ensures that the output is always positive. This function is a smoothed version of the rectified linear unit (ReLU).

Deep Learning Toolbox Layer	Description
quadraticLayer	Creates a vector of quadratic monomials constructed from the elements of the input array. This layer is useful when you need an output that is some quadratic function of its inputs, such as for an LQR controller.

Note The `biLstmLayer` and `batchNormalizationLayer` layers are not supported for reinforcement learning. Normalization in any of the input layers is also not supported.

You can also create your own custom layers. For more information, see “Define Custom Deep Learning Layers”.

When you create a deep neural network, it is good practice to specify names for the first layer of each input path and the final layer of the output path. These names allow you to connect network paths and then later explicitly associate each network input layer with its appropriate environment channel.

The following code creates and connects the following input and output paths:

- An observation input path, `observationPath`, with the first layer named “`obsInputLayer`”.
- An action input path, `actionPath`, with the first layer named “`actInputLayer`”.
- An estimated value function output path, `commonPath`, which takes the outputs of `observationPath` and `actionPath` as inputs and joins them together along the first dimension using `concatenationLayer`. The final layer of this path is named “`QValueOutputLayer`”.

Note Instead of using `concatenationLayer` to join the paths, you can also add the outputs of the previous paths together using `additionLayer`. While concatenation and addition both successfully merge paths, the concatenation layer - everything else equal - uses a larger dimensionality signal after the merging and thereby involves more learnable parameters and allows more degrees of freedom for the network to learn the underlying function. Indeed, while an addition layer might need some preprocessing of its inputs to approximate a desired function, when using a concatenation layer you can potentially move much of the processing after the concatenation.

```
% Observation path: array of layer objects
observationPath = [
    featureInputLayer(4,Name="obsInputLayer")
    fullyConnectedLayer(24)
    reluLayer
    fullyConnectedLayer(24,Name="ObsFC2")
];

% Action path: array of layer objects
actionPath = [
    featureInputLayer(1,Name="actInputLayer")
    fullyConnectedLayer(24,Name="ActFC1")
];

% Common path: array of layer objects
commonPath = [
    concatenationLayer(1,2,Name="cct")
    reluLayer
    fullyConnectedLayer(1,Name="QValueOutputLayer")
];

% Assemble dlnetwork object
criticNetwork = dlnetwork();
```

```
criticNetwork = addLayers(criticNetwork,observationPath);
criticNetwork = addLayers(criticNetwork,actionPath);
criticNetwork = addLayers(criticNetwork,commonPath);

% Connect layers
criticNetwork = connectLayers(criticNetwork,"ObsFC2","cct/in1");
criticNetwork = connectLayers(criticNetwork,"ActFC1","cct/in2");

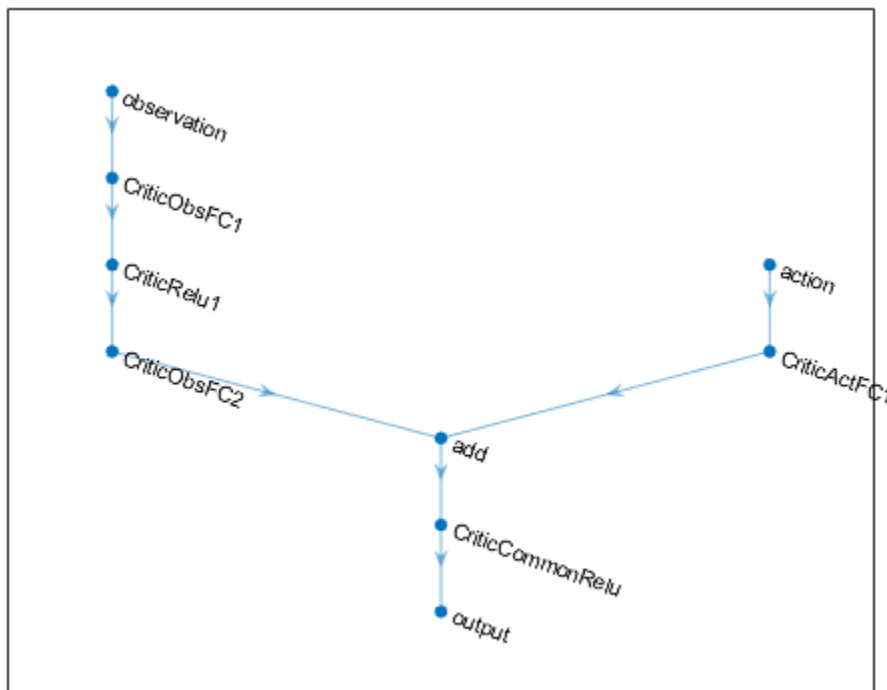
% Initialize dlnetwork object
criticNetwork = initialize(criticNetwork);

% Display the number of learnable parameters
summary(criticNetwork)
```

For all observation and action input paths, you must specify a `featureInputLayer` as the first layer in the path, with a number of input neurons equal to the number of dimensions of the corresponding environment channel.

You can view the structure of your deep neural network using the `plot` function.

```
plot(criticNetwork)
```



Because the output of a network in an `rlDiscreteCategoricalActor` actors must represent the probability of executing each possible action, the software automatically adds a `softmaxLayer` as a final output layer if you do not specify it explicitly. When computing the action, the actor then randomly samples the distribution to return an action.

Determining the number, type, and size of layers for your deep neural network can be difficult and is application dependent. However, the most critical component in deciding the characteristics of the function approximator is whether it is able to approximate the optimal policy or discounted value function for your application, that is, whether it has layers that can correctly learn the features of your observation, action, and reward signals.

Consider the following tips when constructing your network.

- For continuous action spaces, bound actions with a `tanhLayer` followed by a `ScalingLayer` to scale the action to desired values, if necessary.
- Deep dense networks with `reluLayer` layers can be fairly good at approximating many different functions. Therefore, they are often a good first choice.
- Start with the smallest possible network that you think can approximate the optimal policy or value function.
- When you approximate strong nonlinearities or systems with algebraic constraints, adding more layers is often better than increasing the number of outputs per layer. In general, the ability of the approximator to represent more complex (compositional) functions grows only polynomially in the size of the layers, but grows exponentially with the number of layers. In other words, more layers allow approximating more complex and nonlinear compositional functions, although this generally requires more data and longer training times. Given a total number of neurons and comparable approximation tasks, networks with fewer layers can require exponentially more units to successfully approximate the same class of functions, and might fail to learn and generalize correctly.
- For on-policy agents (the ones that learn only from experience collected while following the current policy), such as AC and PG agents, parallel training works better if your networks are large (for example, a network with two hidden layers with 32 nodes each, which has a few hundred parameters). On-policy parallel updates assume each worker updates a different part of the network, such as when they explore different areas of the observation space. If the network is small, the worker updates can correlate with each other and make training unstable.

Create and Configure Actors and Critics from a Neural Network

To create a critic from your deep neural network, use an `rlValueFunction`, `rlQValueFunction` or (whenever possible) an `rlVectorQValueFunction` object. To create a deterministic actor for a continuous action space from your deep neural network, use an `rlContinuousDeterministicActor` object. To create a stochastic actor from your deep neural network use either an `rlDiscreteCategoricalActor` or an `rlContinuousGaussianActor` object. To configure the learning rate and optimization used by the actor or critic, use an optimizer object within an agent option object.

For example, create a Q-value function critic using the neural network `criticNetwork` and the environment action and observation specifications. Pass as additional arguments also the names of the network input layers to be connected with the observation and action channels, respectively.

```
critic = rlQValueFunction(criticNetwork, obsInfo, actInfo, ...
    ObservationInputNames={"obsInputLayer"}, ...
    ActionInputNames="actInputLayer");
```

To specify training options for the critic, use `rlOptimizerOptions` to create the critic optimizer object `criticOpts`, specifying a learning rate of 0.02 and a gradient threshold of 1.

```
criticOpts = rlOptimizerOptions(LearnRate=0.02, ...
    GradientThreshold=1);
```

Then create an agent option object, and set the `CriticOptimizerOptions` property of the agent option object to `criticOpts`. When finally you create the agent, pass the agent option object as a last input argument to the agent constructor function. Alternatively, you can create the agent first, and then access its option object, and modify the options, using dot notation.

When you create your deep neural network and configure your actor or critic, consider using the following approach as a starting point.

- 1 Start with the smallest possible network and a high learning rate (0.01). Train this initial network to see if the agent converges quickly to a poor policy or acts in a random manner. If either of these issues occur, rescale the network by adding more layers or more outputs on each layer. Your goal is to find a network structure that is just big enough, does not learn too fast, and shows signs of learning (an improving trajectory of the reward graph) after an initial training period.
- 2 Once you settle on a good network architecture, a low initial learning rate can allow you to see if the agent is on the right track, and help you check that your network architecture is satisfactory for the problem. A low learning rate makes tuning parameters easier, especially for difficult problems.

Also, consider the following tips when configuring your deep neural network agent.

- Be patient with DDPG and DQN agents, since they might not learn anything for some time during the early episodes, and they typically show a dip in cumulative reward early in the training process. Eventually, they can show signs of learning after the first few thousand episodes.
- For DDPG and DQN agents, promoting exploration of the agent is critical.
- For agents with both actor and critic networks, set the initial learning rates of both actor and critic to the same value. However, for some problems, setting the critic learning rate to a higher value than that of the actor can improve learning results.

Recurrent Neural Networks

When creating actors or critics for use with any agent except Q, SARSA, TRPO and MBPO, you can use recurrent neural networks (RNN). These networks are deep neural networks with a `sequenceInputLayer` input layer and at least one layer that has hidden state information, such as an `lstmLayer`. They can be especially useful when the environment has states that cannot be included in the observation vector.

For agents that have both actor and critic, you must either use an RNN for both of them, or not use an RNN for any of them. You cannot use an RNN only for the critic or only for the actor.

Note Code generation is not supported for continuous action space PG, AC, PPO agents, and SAC agents using a recurrent neural network (RNN), or for any agent having multiple input paths and containing an RNN in any of the paths.

When using PG agents, the learning trajectory length (that is, the sequence of input data that the network uses for learning) for the RNN is the whole episode. For an AC agent, the `NumStepsToLookAhead` property of its options object is treated as the training trajectory length (except when training in parallel, in which case `NumStepsToLookAhead` is ignored and the whole episode is used as trajectory length). For a PPO agent, the trajectory length is the `MiniBatchSize` property of its options object.

For DQN, DDPG, SAC and TD3 agents, you must specify the trajectory length as an integer greater than one in the `SequenceLength` property of their options object. These learning algorithms randomly sample `MiniBatchSize` experience points from all the available episodes. For each experience point, a sequence of `SequenceLength` consecutive experiences is used for learning. If the number of available consecutive experiences is shorter than `SequenceLength` (this can happen for example, when an episode terminates prematurely), then the available experiences are padded to complete the sequence. The sequence is also appropriately masked so that the padded data does not affect the gradient computation.

For more information and examples on policies and value functions, see `rlValueFunction`, `rlQValueFunction`, `rlVectorQValueFunction`, `rlContinuousDeterministicActor`, `rlDiscreteCategoricalActor`, and `rlContinuousGaussianActor`.

Create Agent from Actor and Critic

Once you create your actor and critic, you can create a built-in reinforcement learning agent that uses them. For example, create a PG agent using a given actor and critic (baseline) network.

```
agentOpts = rlPGAgentOptions(UseBaseline=true);
agent = rlPGAgent(actor,baseline,agentOpts);
```

For more information on the different types of reinforcement learning agents, see “Reinforcement Learning Agents” on page 3-2.

You can obtain the actor and critic from an existing agent using `getActor` and `getCritic`, respectively.

You can also set the actor and critic of an existing agent using `setActor` and `setCritic`, respectively. The input and output layers of the actor and critic must match the observation and action specifications of the original agent.

See Also

Functions

`getValue` | `getModel` | `setModel` | `getLearnableParameters` | `setLearnableParameters` | `evaluate`

Objects

`rlTable` | `rlValueFunction` | `rlQValueFunction` | `rlVectorQValueFunction` | `rlContinuousDeterministicActor` | `rlDiscreteCategoricalActor` | `rlContinuousGaussianActor` | `rlMaxQPolicy` | `rlEpsilonGreedyPolicy` | `rlDeterministicActorPolicy` | `rlAdditiveNoisePolicy` | `rlStochasticActorPolicy`

Related Examples

- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115
- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle” on page 5-132
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Reinforcement Learning Agents” on page 3-2
- “Import Neural Network Models Using ONNX” on page 4-16

Import Neural Network Models Using ONNX

To create function approximators for reinforcement learning, you can import pre-trained deep neural networks or deep neural network layer architectures using the Deep Learning Toolbox network import functionality. You can import:

- Open Neural Network Exchange (ONNX) models, which require the Deep Learning Toolbox Converter for ONNX Model Format support package software. For more information, see `importNetworkFromONNX`.
- TensorFlow-Keras networks, which require Deep Learning Toolbox Converter for TensorFlow Models support package software. For more information, see `importNetworkFromTensorFlow`.

After you import a deep neural network, you can create an actor or critic object, such as `rlValueFunction` or `rlDiscreteCategoricalActor`.

When you import deep neural network architectures, consider the following.

- The dimensions of the input and output layers of the imported network architecture must match the dimensions of the environment action, observation, or reward channels that need to be connected to those layers.
- After importing the network architecture, it is best practice to specify the names of the input and output layers to be connected to the corresponding environment action and observation channels.

For more information on the deep neural network architectures supported for reinforcement learning, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Import Actor and Critic for Image Observation Application

As an example, assume that you have an environment with a 50-by-50 grayscale image observation signal and a continuous action space. To train a policy gradient (PG) agent, you require the following function approximators, both of which must have a single 50-by-50 image input observation layer and a single scalar output value.

- **Actor** — Selects an action based on the current observation.
- **Critic** — Estimates the expected discounted cumulative long-term reward based on the current observation.

Also, assume that you have the following network architectures to import:

- A deep neural network architecture for the critic with a 50-by-50 image input layer and a scalar output layer, which is saved in the ONNX format (`criticNetwork.onnx`).
- A deep neural network architecture for the actor with a 50-by-50 image input layer and a scalar output layer, which is saved in the ONNX format (`actorNetwork.onnx`).

To import the critic and actor networks, use the `importNetworkFromONNX` function.

```
criticNetwork = importNetworkFromONNX("criticNetwork.onnx");  
actorNetwork = importNetworkFromONNX("actorNetwork.onnx");
```

After you import the network, if you already have an appropriate agent for your environment you can use the `getActor` and `getCritic` functions to extract the actor and critic function approximators for the agent, then the `setModel` function to set the imported networks as the approximation models

of the actor and critic, and then the `setActor` and `setCritic` functions to set the actor and critic with the imported network into your agent.

Alternatively, create new actor and critic function approximators that use the imported networks. To do so, first obtain the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Create the critic. PG agents use an `rlValueFunction` approximator.

```
critic = rlValueFunction(criticNetwork,obsInfo);
```

If your critic has more than one input channel (for example because your environment has more than one output channel or because you are using a Q-value function critic, which also needs an action input), it is good practice to specify the names of the input layer that need to be connected, in sequential order, with each critic input channel. For an example, see “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122.

Create the actor. PG agents use an `rlContinuousDeterministicActor` approximator.

```
actor = rlContinuousDeterministicActor(actorNetwork,obsInfo,actInfo);
```

As for the critic, if your actor has more than one input channel (because your environment has more than one output channel), it is good practice to specify the name of the input layer that needs to be connected with each actor input channel.

To verify that your actor and critic work properly, use the `getAction` and `getValue` functions to return an action (for the actor) and the value (for the critic) corresponding to a random observation, using the current network weights.

After you have the actor and critic with the imported networks, you can then:

- Create an agent using this actor and critic. For more information, see “Reinforcement Learning Agents” on page 3-2.
- Set the actor and critic in an existing agent using `setActor` and `setCritic`, respectively.

See Also

Functions

`importNetworkFromONNX` | `importNetworkFromTensorFlow` | `importCaffeLayers` | `getActionInfo` | `getObservationInfo` | `getAction` | `getValue` | `setActor` | `setCritic` | `setModel`

Objects

`rlValueFunction` | `rlContinuousDeterministicActor` | `rlPGAgent`

Related Examples

- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Reinforcement Learning Agents” on page 3-2

- “Interoperability Between Deep Learning Toolbox, TensorFlow, PyTorch, and ONNX”
- “Tips on Importing Models from TensorFlow, PyTorch, and ONNX”

Train and Simulate Agents

- “Train Reinforcement Learning Agents” on page 5-3
- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Specify Training Options in Reinforcement Learning Designer” on page 5-29
- “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50
- “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53
- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85
- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93
- “Train LSPI Agent to Balance Discrete Cart-Pole” on page 5-102
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122
- “Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle” on page 5-132
- “Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal” on page 5-141
- “Train Agents Using Parallel Computing and GPUs” on page 5-151
- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train PPO Agent with Curriculum Learning for a Lane Keeping Application” on page 5-164
- “Train DQN Agent Using Hindsight Experience Replay” on page 5-176
- “Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum” on page 5-184
- “Train Biped Robot to Walk Using Evolution Strategy-Reinforcement Learning Agents” on page 5-196
- “Transfer Learning: Fine-Tune DQN Agent for Pendulum Swing-Up from Earth to Mars” on page 5-207
- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217
- “Log Training Data to Disk” on page 5-231
- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240
- “Tune Hyperparameters Using Bayesian Optimization” on page 5-250
- “Configure Exploration for Reinforcement Learning Agents” on page 5-259
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Custom Training Loop with Simulink Action Noise” on page 5-282
- “Custom PPO Training Loop with Random Network Distillation” on page 5-298
- “Custom DQN Training Loop with LSTM Network” on page 5-317

- “Create and Train Custom PG Agent” on page 5-334
- “Create and Train Custom LQR Agent” on page 5-345
- “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356
- “Model-Based Reinforcement Learning Using Custom Training Loop” on page 5-368

Train Reinforcement Learning Agents

Once you have created an environment and reinforcement learning agent, you can train the agent in the environment using the `train` function. To configure your training, use an `rlTrainingOptions` object. For example, create a training option object `opt`.

```
opt = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=1000, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=480);
```

Then train agent `agent` in environment `env`, using the training options object `opt`.

```
trainResults = train(agent,env,opt);
```

If `env` is a multiagent environment specify the `agent` argument as an array. The order of the agents in the array must match the agent order used to create `env`. For multiagent training, use `rlMultiAgentTrainingOptions` instead of `rlTrainingOptions`. Using `rlMultiAgentTrainingOptions` gives you access to training options that are specific to multiagent training. For more information about multiagent training, see “Multiagent Training” on page 5-8.

Note Calling `clear all` negatively affects performance. Use `clear` instead.

For more information on creating agents, see “Reinforcement Learning Agents” on page 3-2. For more information on creating environments, see “Reinforcement Learning Environments” on page 2-2 and “Create Custom Simulink Environments” on page 2-110.

Note The `train` function updates the input argument `agent` as training progresses. This is possible because agents are handle objects. To preserve the original agent parameters for later use, save the agent to a MAT file:

```
save("initialAgent.mat","agent")
```

If you copy the agent into a new variable, the new variable will also always point to the most recent agent version with updated parameters. For more information about handle objects, see “Handle Object Behavior”.

Training terminates automatically when the conditions you specify in the `StopTrainingCriteria` and `StopTrainingValue` options of your `rlTrainingOptions` object are satisfied. You can also terminate training before any termination condition is reached by clicking **Stop Training** in Reinforcement Learning Training Monitor.

When training terminates the training statistics and results are stored in the `trainResults` object (unless you interrupt training by pressing **Ctrl-C** at the MATLAB command line).

Because `train` updates the agent at the end of each episode, and because `trainResults` stores the last training results along with data to correctly recreate the training scenario and update Reinforcement Learning Training Monitor, you can later resume training from the exact point at which it stopped. To do so, at the command line, type:

```
trainResults = train(agent,env,trainResults);
```

This starts the training from the last values of the agent parameters and training results object obtained after the previous `train` call.

The `TrainingOptions` property of `trainResults` contains the `rlTrainingOptions` object that specifies the training option set. Therefore, to restart the training with updated training options, first change the training options in `trainResults` using dot notation. If the maximum number of episodes was already reached in the previous training session, you must increase the maximum number of episodes.

For example, disable displaying the training progress on Reinforcement Learning Training Monitor, enable the `Verbose` option to display training progress at the command line, change the maximum number of episodes to 2000, and then restart the training, returning a new `trainResults` object as output.

```
trainResults.TrainingOptions.MaxEpisodes = 2000;  
trainResults.TrainingOptions.Plots = "none";  
trainResults.TrainingOptions.Verbose = 1;  
trainResultsNew = train(agent,env,trainResults);
```

Note When training terminates, `agent` reflects the state of each agent at the end of the final training episode. The rewards obtained by the final agents are not necessarily the highest achieved during the training process, due to continuous exploration. To save agents during training, create an `rlTrainingOptions` object specifying the `SaveAgentCriteria` and `SaveAgentValue` properties and pass it to `train` as a `trainOpts` argument. For information on how to evaluate agent, see “Evaluate Agents During Training” on page 5-7.

Training Algorithm

In general, training performs the following steps.

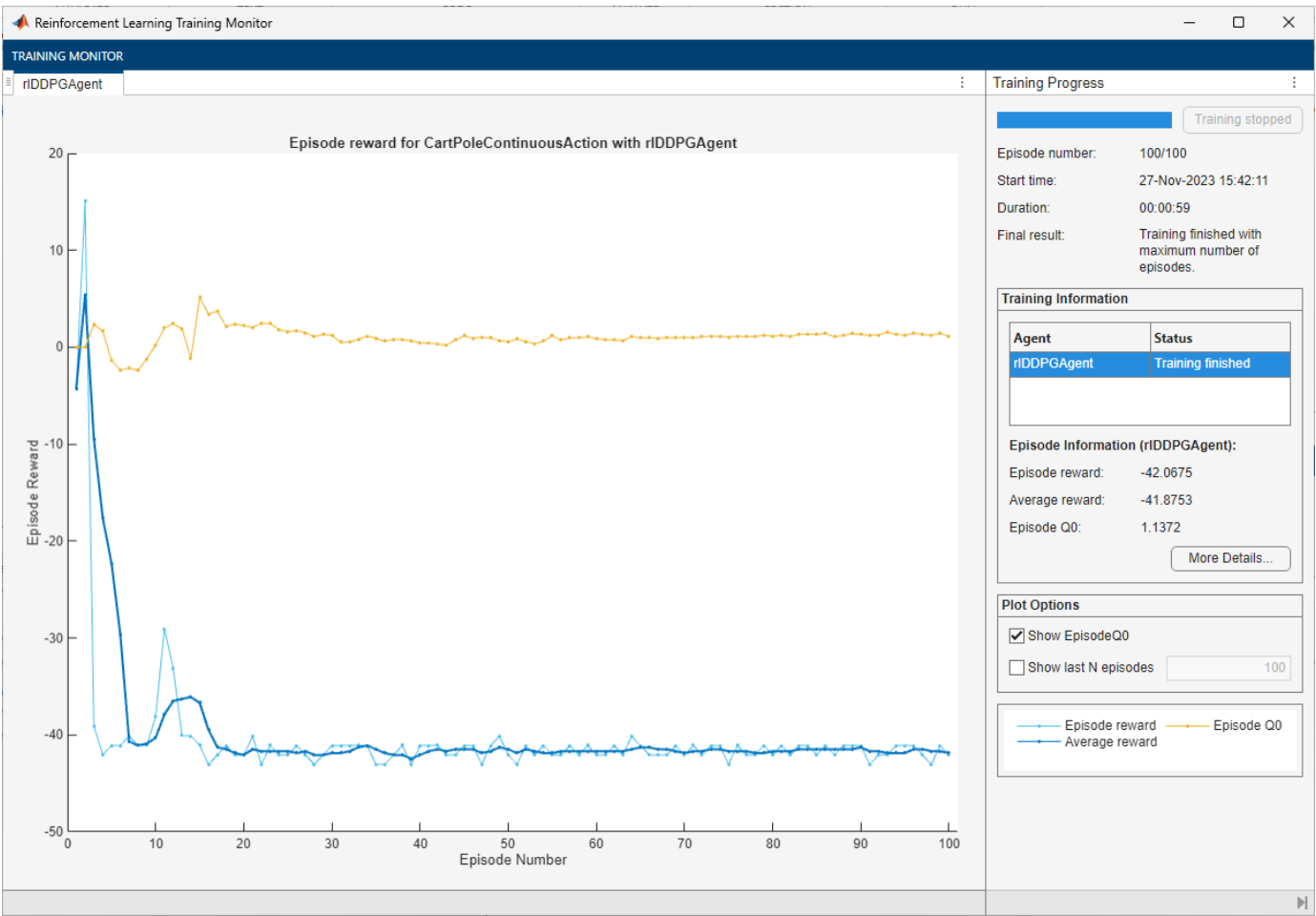
- 1 Initialize the agent.
- 2 For each episode:
 - a Reset the environment.
 - b Get the initial observation S_0 from the environment.
 - c Compute the initial action $A_0 = \mu(S_0)$, where $\mu(S)$ is an action selected according to the current policy $\pi(A|S)$.
 - d Set the current action to the initial action ($A \leftarrow A_0$), and set the current observation to the initial observation ($S \leftarrow S_0$).
 - e While the episode is not finished or terminated, perform the following steps.
 - i Apply action A to the environment and obtain the next observation S' and the reward R .
 - ii Learn from the experience set (S, A, R, S') .
 - iii Compute the next action $A' = \mu(S')$.
 - iv Update the current action with the next action ($A \leftarrow A'$) and update the current observation with the next observation ($S \leftarrow S'$).
 - v Terminate the episode if the termination conditions defined in the environment are met.

- 3 If the training termination condition is met, terminate training. Otherwise, begin the next episode.

The specifics of how the software performs these steps depend on the configuration of the agent and environment. For instance, resetting the environment at the start of each episode can include randomizing initial state values, if you configure your environment to do so. For more information on agents and their training algorithms, see “Reinforcement Learning Agents” on page 3-2. To use parallel processing and GPUs to speed up training, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

Reinforcement Learning Training Monitor

By default, calling the `train` function opens the Reinforcement Learning Training Monitor, which lets you visualize the training progress.



The Reinforcement Learning Training Monitor plot shows the reward for each episode (**EpisodeReward**) and a running average reward value (**AverageReward**).

For agents with a critic, **Episode Q0** is the estimate of the discounted long-term reward at the start of each episode, given the initial observation of the environment. As training progresses, if the critic is well designed and learns successfully, **Episode Q0** approaches in average the true discounted long-

term reward, which might be offset from the **EpisodeReward** value because of discounting. For a well designed critic using an undiscounted reward (**DiscountFactor** is equal to 1), then on average **Episode Q0** approaches the true episode reward, as shown in the preceding figure.

The Reinforcement Learning Training Monitor also displays several episode and training statistics. You can also use the `train` function to return episode and training information. To prevent displaying the training information with the Reinforcement Learning Training Monitor, set the **Plots** option of `rlTrainingOptions` to "none".

Save Candidate Agents During Training

During training, you can save candidate agents that meet conditions you specify in the **SaveAgentCriteria** and **SaveAgentValue** options of your `rlTrainingOptions` object. For instance, you can save any agent whose episode reward exceeds a certain value, even if the overall condition for terminating training is not yet satisfied. For example, save agents when the episode reward is greater than 100.

```
opt = rlTrainingOptions(SaveAgentCriteria="EpisodeReward",SaveAgentValue=100);
```

`train` stores saved agents in a MAT file in the folder you specify using the **SaveAgentDirectory** option of `rlTrainingOptions`. Saved agents can be useful, for instance, to test candidate agents generated during a long-running training process. For details about saving criteria and saving location, see `rlTrainingOptions`.

After training is complete, you can save the final trained agent from the MATLAB workspace using the `save` function. For example, save the agent `myAgent` to the file `finalAgent.mat` in the current working directory.

```
save(opt.SaveAgentDirectory + "/finalAgent.mat",'agent')
```

By default, when DDPG and DQN agents are saved, the experience buffer data is not saved. If you plan to further train your saved agent, you can start training with the previous experience buffer as a starting point. In this case, set the **SaveExperienceBufferWithAgent** option to `true`. For some agents, such as those with large experience buffers and image-based observations, the memory required for saving the experience buffer is large. In these cases, you must ensure that enough memory is available for the saved agents.

Validate Trained Agent

To validate your trained agent, you can simulate the agent within the training environment using the `sim` function. To configure the simulation, use `rlSimulationOptions`.

When validating your agent, consider checking how your agent handles the following:

- Changes to simulation initial conditions — To change the model initial conditions, modify the reset function for the environment. For example, reset functions, see “Create Custom Environment Using Step and Reset Functions” on page 2-94, “Create Custom Environment from Class Template” on page 2-103, and “Create Custom Simulink Environments” on page 2-110.
- Mismatches between the training and simulation environment dynamics — To check such mismatches, create test environments in the same way that you created the training environment, modifying the environment behavior.

As with parallel training, if you have Parallel Computing Toolbox software, you can run multiple parallel simulations on multicore computers. If you have MATLAB Parallel Server software, you can

run multiple parallel simulations on computer clusters or cloud resources. For more information on configuring your simulation to use parallel computing, see `UseParallel` and `ParallelizationOptions` in `rlSimulationOptions`.

Address Memory Issues During Training

Some Simulink environments save a considerable amount of data when running. Specifically, by default, the software saves anything that appears as the output of a `sim` command. This can cause out-of-memory issues when training or simulating an agent in this kind of environment. You can use three training (or simulation) options to prevent memory-related problems:

- **SimulationStorageType** — This option specifies the type of storage used for data generated during training or simulation by a Simulink environment. The default value is `"memory"`, indicating that data is stored in memory. To store environment data to disk instead, set this option to `"file"`. When this option is set to `"none"` environment data is not stored.
- **SaveSimulationDirectory** — This option specifies the directory to save environment data when `SimulationStorageType` is set to `"file"`. The default value is `"savedSims"`.
- **SaveFileVersion** — This option specifies the MAT file version for environment data. The default is `"-v7"`. The other possible options are `"-v7.3"` and `"-v6"`.

For more information, see `rlTrainingOptions`, `rlMultiAgentTrainingOptions`, `rlEvolutionStrategyTrainingOptions`, and `rlSimulationOptions`.

Visualize Environment State During Training

If your MATLAB-based environment implements the `plot` method, you can visualize the environment behavior during training and simulation. If you call `plot(env)` before training or simulation, where `env` is your environment object, then the visualization updates during training to allow you to visualize the progress of each episode or simulation.

Environment visualization is not supported when training or simulating your agent using parallel computing.

Note Environment visualization or training progress visualization increases training time. Therefore, for faster training, do not enable any visualization.

For custom MATLAB-based environments, you must implement your own `plot` method. For more information on creating a custom environments with a `plot` function, see “Create Custom Environment from Class Template” on page 2-103.

Evaluate Agents During Training

You can automatically evaluate your agent at regular intervals during training. Doing so allows you to observe the actual training progress and automatically stop the training or save the agent when some pre-specified conditions are met.

To configure evaluation options for your agents, first create an evaluator object using `rlEvaluator`. You can specify properties such as the type of evaluation statistic, the frequency at which evaluation episodes occur, or whether exploration is allowed during an evaluation episode.

To train the agents and evaluate them during training, pass this object to `train`.

You can also create a custom evaluator object, which uses a custom evaluation function that you supply. To do so, use `rlCustomEvaluator`.

For more information, see also the `EvaluationStatistic` of `train`.

Train Agent Offline from a Data Set

You can train off-policy agents (DQN, SAC, DDPG, and TD3) offline, using an existing dataset, instead of an environment.

To train your agent from an existing dataset, first, use `rlTrainingFromDataOptions` to create a training from data option object. Then pass this option object (along with the environment and agent) to `trainFromData` to train your agent.

To deal with possible differences between the probability distribution of the dataset and the one generated by the environment, use the batch data regularization options provided for off-policy agents. For more information, see the new `BatchDataRegularizerOptions` property of the off-policy agents options objects, as well as the new `rlBehaviorCloningRegularizerOptions` and `rlConservativeQLearningOptions` options objects.

Use Evolutionary Strategies

You can train DDPG, TD3 and SAC agents using an evolutionary algorithm.

Evolutionary reinforcement learning adaptation strategies update the weights of your agents using a selection process inspired by biological evolution. Compared to gradient-based approaches, evolutionary algorithms are less reliant on backpropagation, are easily parallelizable, and have a reduced sensitivity to local minima. They also generally display good (nonlocal) exploration and robustness, especially in complex scenarios where data is incomplete or noisy and rewards are sparse or conflicting.

To train your agent using an evolutionary algorithm, first, use `rlEvolutionStrategyTrainingOptions` to create an evolution strategy training option object. Then pass this option object (along with the environment and agent) to `trainWithEvolutionStrategy` to train your agent.

For an example, see “Train Biped Robot to Walk Using Evolution Strategy-Reinforcement Learning Agents” on page 5-196.

Multiagent Training

You can train multiple agents that work together in the same environment. For more information on loading predefined multiagent environments, see “Use Predefined Multiagent Environments” on page 2-77.

You can also create a custom multiagent environment. Specifically, using MATLAB functions, you can create two different kinds of custom multiagent environments:

- Multiagent environments with universal sample time, in which all agents execute in the same step. You can create these environments by supplying your own reset and step functions, as well as observations and action specifications, to `rlMultiAgentFunctionEnv`.

- Turn-based function environments, in which agents execute in turns. Specifically, the environment assigns execution to only one group of agents at a time, and the group executes when it is its turn to do so. You can create these environments by supplying your own reset and step functions, as well as observations and action specifications, to `rlTurnBasedFunctionEnv`. For an example, see “Train Agent to Play Turn-Based Game” on page 8-348.

For both kinds of multiagent environments, the observation and action specifications are cell arrays of specification objects in which each element corresponds to one agent.

You can also create a custom multiagent Simulink environment (this option allows you to model environments with multi-rate execution, in which each agent might have its own execution rates).

To create a custom multiagent Simulink, first, create a Simulink model that has one action input and one set of outputs (observation, reward and is-done) for every agent. Then manually add an agent block for each agent. Once you connect the blocks, create an environment object using `rlSimulinkEnv`. Unless each agent block already references an agent object in the MATLAB workspace, you must supply to `rlSimulinkEnv` two cell arrays containing the observation action specification objects, respectively, as input arguments. For an example, see “Train Multiple Agents to Perform Collaborative Task” on page 8-31.

Once you have created your multiagent environment, specify options for multiagent training by creating and configuring an `rlMultiAgentTrainingOptions` object. Doing so allows you to specify, for example, whether different groups of agents are trained in a decentralized or centralized manner. In a group of agents subject to decentralized training, each agent collects its own set of experiences and learns from its own set of experiences. In a group of agents subject to centralized training, each agent shares its experiences with the other agents in the group and each agent in the group learns from the collective shared experiences. In general, centralized learning boosts exploration and facilitates learning in applications where the agents perform a collaborative (or the same) task.

Once you have your multiagent environment and multiagent training options object, you can train and simulate your agents with `train` and `sim`, respectively. You can visualize the training progress of all the agents using the Reinforcement Learning Training Manager.

For more examples on training multiple agents, see also “Train Multiple Agents for Area Coverage” on page 8-44, and “Train Multiple Agents for Path Following Control” on page 8-57.

Why Is My Training Not Converging?

In general, reinforcement learning algorithms cannot be guaranteed to converge even to a vicinity of a local optimum, unless many assumptions on the environment, the algorithm, and the function approximator are verified. Typically such assumption involve availability of the environment state, finite state and action spaces, known dynamics, tabular or linear function approximation, appropriate (and decaying) exploration and learning rate.

In practice, there might be many reasons why training does not converge to a satisfying solution.

- Insufficient exploration: If the agent does not explore different actions and states adequately, it might not discover the optimal policy. Insufficient exploration can lead to suboptimal results or getting stuck in local optima.
- High-dimensional state or action spaces: When the state or action space is large, it becomes challenging for the agent to explore and learn effectively. The curse of dimensionality can make it difficult to find an optimal policy within a reasonable time frame.

- **Function approximation limitations:** It is important to consider whether your function approximator can properly approximate your value functions or policy, while at the same time being able to properly generalize from experience, otherwise, convergence issues can arise. Note that unavoidable approximation errors arise when observation signals are not adequately selected. For more information on selecting good observation signals, see the observation section of “Define Observation and Reward Signals in Custom Environments” on page 2-127.
- **Improperly scaled observation or action signals:** Signals with widely different ranges can skew the learning process, making it hard for your approximator to successfully learn the important features. To prevent scaling problems, normalize the observations to a consistent scale. Common normalization techniques include scaling the observations to the range [0, 1] or standardizing them with zero mean and unit variance.
- **Poorly designed reward signal:** The reward signal plays a crucial role in reinforcement learning. If the rewards are not properly designed or do not reflect the desired behavior, the agent might struggle to learn the optimal policy. Inconsistent or sparse rewards can make the learning process unstable or slow. For more information on selecting good reward signals, see the rewards section of “Define Observation and Reward Signals in Custom Environments” on page 2-127.
- **Inadequate training time or resources:** Reinforcement learning algorithms often require extensive training time and computational resources to converge. If the training time or resources are limited, the algorithm might not have sufficient iterations to converge to the optimal policy. For more information on using parallel computing to train agents, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.
- **Inappropriate algorithm or hyperparameters:** Choosing an inappropriate algorithm or setting incorrect hyperparameters can significantly impact convergence. Different algorithms and hyperparameters might be more suitable for specific problem domains, and selecting the wrong ones can hinder convergence. For more information on selecting agents, see the last section of “Reinforcement Learning Agents” on page 3-2.
- **Non-stationary environments:** If the environment in which the agent operates changes over time, the learning algorithm might struggle to adapt. Non-stationary environments can introduce additional challenges, requiring the algorithm to continuously update its policy to account for the changes.

To address these convergence issues, you can try to adjust the exploration-exploitation tradeoff, use better reward shaping, fine-tune hyperparameters, use an experience replay, or rely on more advanced algorithms.

Note Choosing the right observation and action sets is crucial for effective training and performance in reinforcement learning. The observations should provide sufficient information about the current environment state for the agent to make informed decisions, and the actions should be able to adequately steer the environment behavior. For more information on selecting good observation and reward signals, see “Define Observation and Reward Signals in Custom Environments” on page 2-127.

Results Reproducibility

The results you obtain when training or simulating an agent in a reinforcement learning environment can be difficult to reproduce exactly. This happens because reinforcement learning algorithms are affected (to a greater extent than machine learning algorithms) by several arbitrary factors, which can typically be categorized as follows.

- **Randomness** — Reinforcement learning algorithms typically depend on the values of random variables, which might be inconsistent across multiple runs unless the random seed is controlled explicitly before training. For example, the weights of the actor and critic networks and the state of the environments are typically initialized randomly. Reset functions called at the beginning of training and simulation often initialize the environment state randomly. Probabilistic environments, stochastic or exploration actions, random mini-batch sampling, are also sources of randomness. To learn about random number generation in MATLAB see “Controlling Random Number Generation”.
- **Hardware configuration** — Numerical values might differ slightly between different platform or hardware settings. Differences in CPU/GPU architecture might also introduce discrepancies. Small numerical differences might add up to significant differences in training behavior. Furthermore, the closed-loop nature of the interaction between agent and environment in reinforcement learning means that when such closed loop is unstable small numerical differences might be amplified during training or simulation.
- **Software configuration** — Changes made to Reinforcement Learning Toolbox software (as well as in component dependencies such as Deep Learning Toolbox) might result in numerical differences from one release to the next. For example, improving an algorithm might require generating new random numbers or a calculating gradients in a more robust (but different) way, which might ultimately affect results. Thus, reproducibility across MATLAB releases cannot be guaranteed.

Additionally, asynchronous parallel training is generally not reproducible, because data is processed in a non-deterministic order, depending on the specific processors workload during training.

To enhance the extent to which your results can be reproduced on the same hardware and software configuration, you can adopt the following guidelines.

- Set the random seed and generator type at the beginning of your script using the `rng` command. Best practice is to directly specify the random number generation algorithm as in `rng(s, "twister")`, where `s`, the seed, is a nonnegative integer. Avoid using the default generator (as in `rng(s)`, or `rng(s, "default")`) because the default seed and generator depend on the particular configuration set in the MATLAB Settings Window window.
- For synchronous parallel training, ensure that `trainOpts.ParallelizationOptions.WorkerRandomSeeds` is not set to `-2`, because in this case the random stream of parallel workers is not initialized. Leave this property to the default `-1` (unique seed per worker) or specify a vector of unique nonnegative integers for best results.
- If you need to execute different sections of a script independently, set both the random number generator algorithm and the seed again at the beginning of each section. This practice preserves the random number sequence in the section every time you run it.

For examples on how to structure a training script for reproducibility, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64 and, for parallel training, “Quadruped Robot Locomotion Using DDPG Agent” on page 8-158.

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim` | `rlSimulinkEnv` | `trainFromData` | `trainWithEvolutionStrategy`

Objects

`rlTrainingOptions` | `rlSimulationOptions` | `rlEvaluator` | `rlCustomEvaluator` |
`rlTrainingFromDataOptions` | `rlEvolutionStrategyTrainingOptions` |
`rlMultiAgentFunctionEnv` | `rlTurnBasedFunctionEnv`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31
- “Train Agent to Play Turn-Based Game” on page 8-348

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Reinforcement Learning Agents” on page 3-2
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

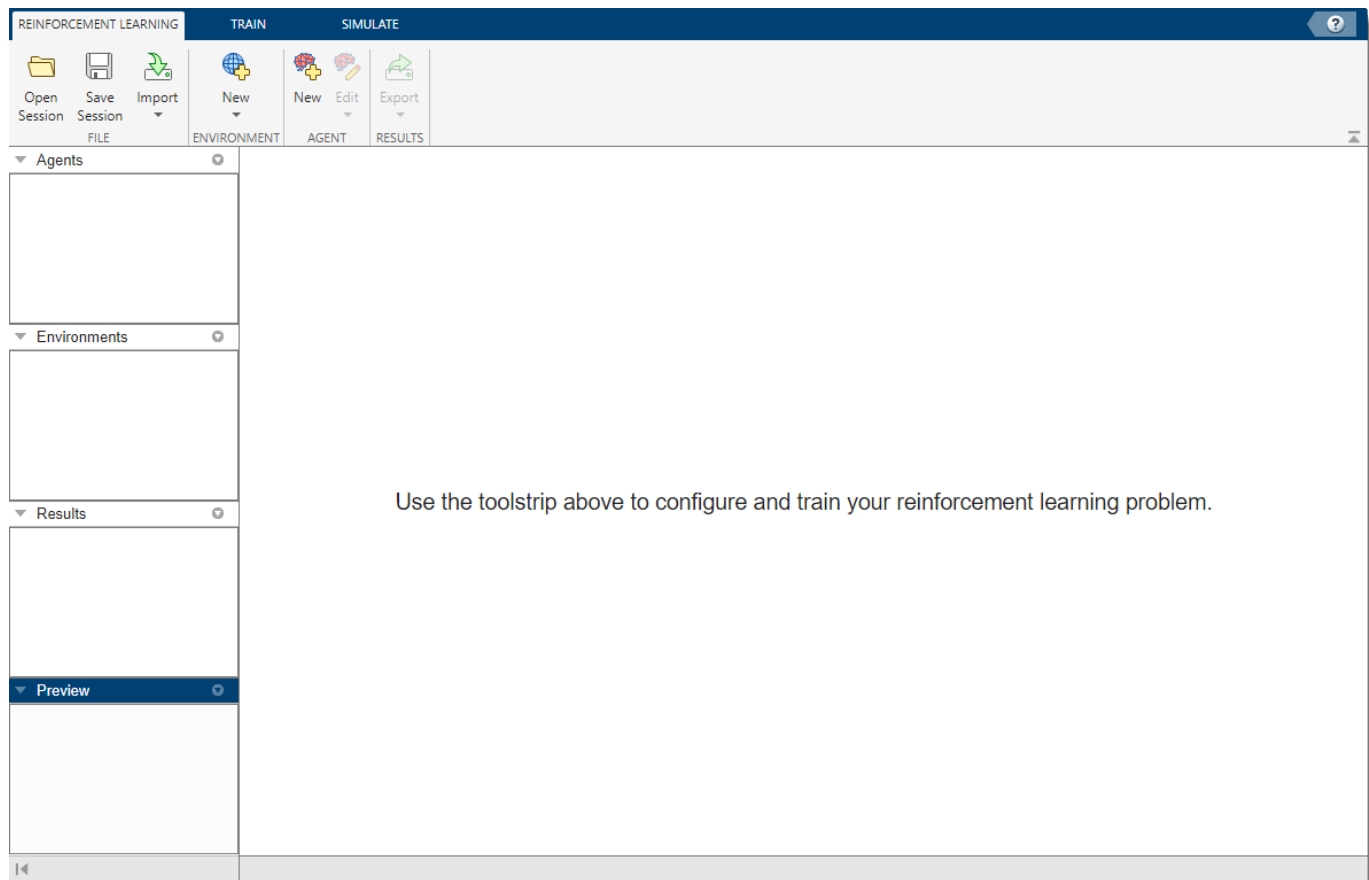
Design and Train Agent Using Reinforcement Learning Designer

This example shows how to design and train a DQN agent for an environment with a discrete action space using Reinforcement Learning Designer.

Open the Reinforcement Learning Designer App

Open the Reinforcement Learning Designer app.

reinforcementLearningDesigner

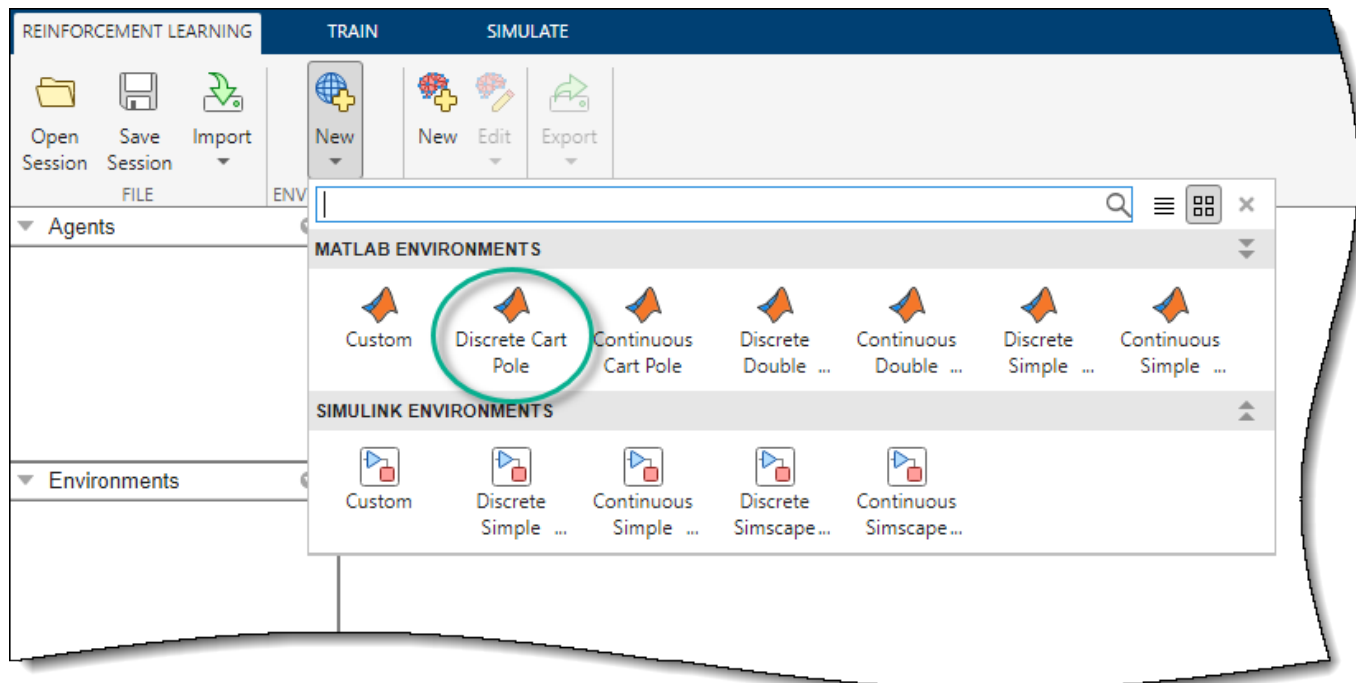


Initially, no agents or environments are loaded in the app.

Import Cart-Pole Environment

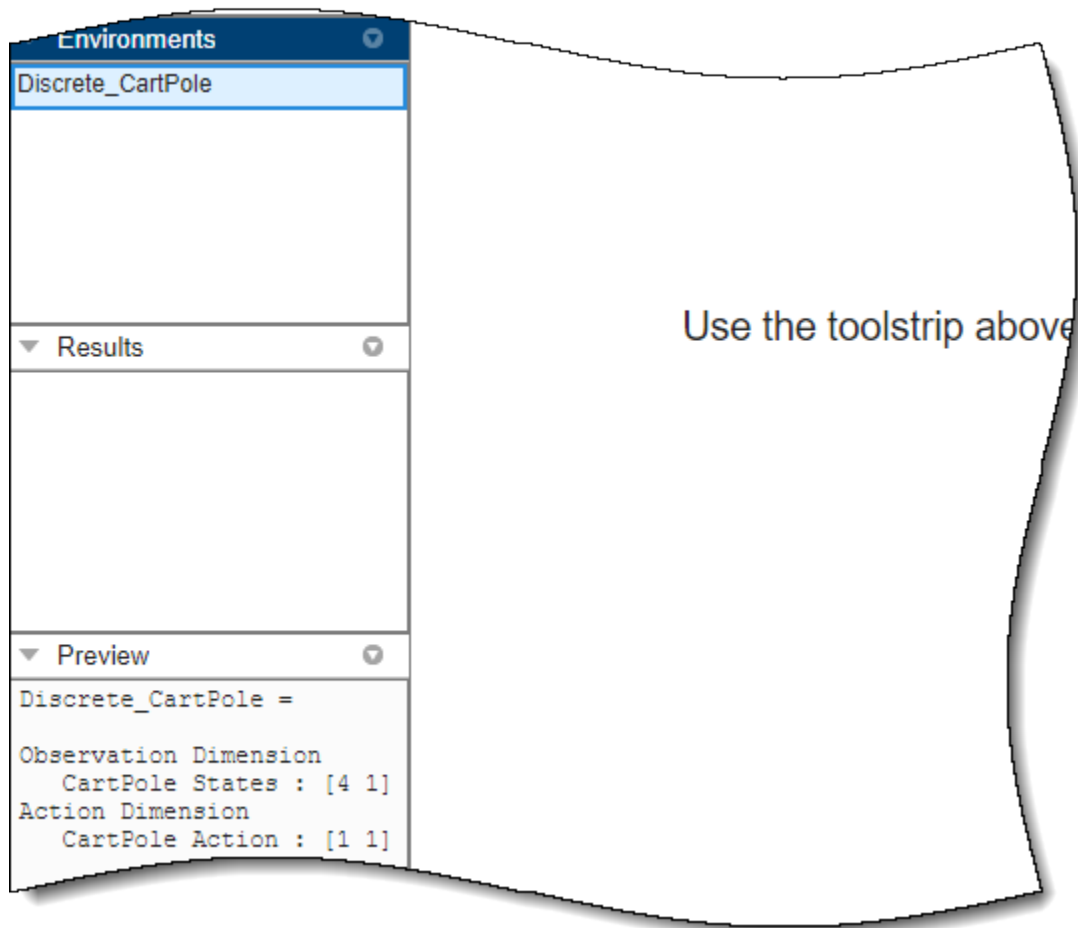
When using the Reinforcement Learning Designer, you can import an environment from the MATLAB workspace or create a predefined environment. For more information, see “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138 and “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142.

For this example, use the predefined discrete cart-pole MATLAB environment. To import this environment, on the **Reinforcement Learning** tab, in the **Environments** section, select **New > Discrete Cart-Pole**.



In the **Environments** pane, the app adds the imported **Discrete CartPole** environment. To rename the environment, click the environment text. You can also import multiple environments in the session.

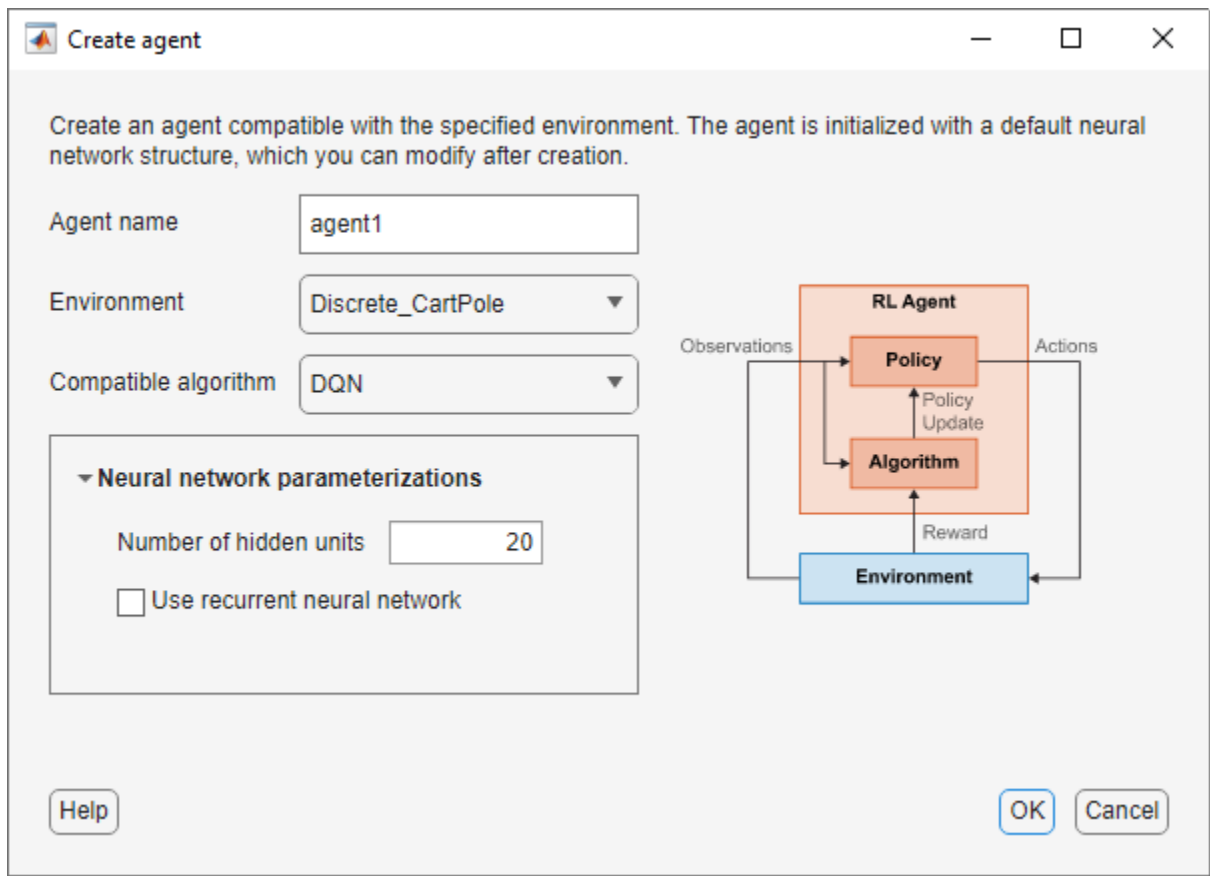
To view the dimensions of the observation and action space, click the environment text. The app shows the dimensions in the **Preview** pane.



This environment has a continuous four-dimensional observation space (the positions and velocities of both the cart and pole) and a discrete one-dimensional action space consisting of two possible forces, -10 N or 10 N. This environment is used in the “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64 example. For more information on predefined control system environments, see “Use Predefined Control System Environments” on page 2-42.

Create DQN Agent for Imported Environment

To create an agent, on the **Reinforcement Learning** tab, in the **Agent** section, click **New**. In the Create agent dialog box, specify the agent name, the environment, and the training algorithm. The default agent configuration uses the imported environment and the DQN algorithm. For this example, change the number of hidden units from 256 to 20. For more information on creating agents, see “Create Agents Using Reinforcement Learning Designer” on page 3-11.



Click **OK**.

The app adds the new agent to the **Agents** pane and opens a corresponding **agent1** document.

In the **Hyperparameter** section, under **Critic Optimizer Options** set **Learn rate** to 0.0001.

Reinforcement Learning Designer

REINFORCEMENT LEARNING TRAIN SIMULATE DQN AGENT

agent1 Import Critic Train Simulate Export

NAME IMPORT MODEL NEXT STEPS

Agents agent1

agent1

Environments Discrete_CartPole

Results

Preview

agent1 =

Type DQN

Observation Dimension

CartPole States : [4 1]

Action Dimension

CartPole Action : [1 1]

Critic Learn Rate 0.0001

Overview

Hyperparameters

Agent Options

Sample time 1

Discount factor 0.99

Execution environment ☒ CPU ☐ GPU

Batch size 64

Experience buffer length 1e+04

Critic Optimizer Options

Learn rate 0.0001

Gradient threshold Inf

More Options

Exploration

Epsilon Greedy Exploration Options

Initial epsilon 1

Epsilon decay 0.005

Epsilon min 0.01

Plot Options

X-axis limit 1000

Epsilon Decay

Value

Steps

Epsilon

Epsilon min

Agent opened: "agent1"

For a brief summary of DQN agent features and to view the observation and action specifications for the agent, click **Overview**.

Train Simulate Export

NEXT STEPS

agent1

Overview

A DQN agent is a value-based reinforcement learning agent that trains a critic to estimate the return or future rewards.

Learn More

Observation Specification

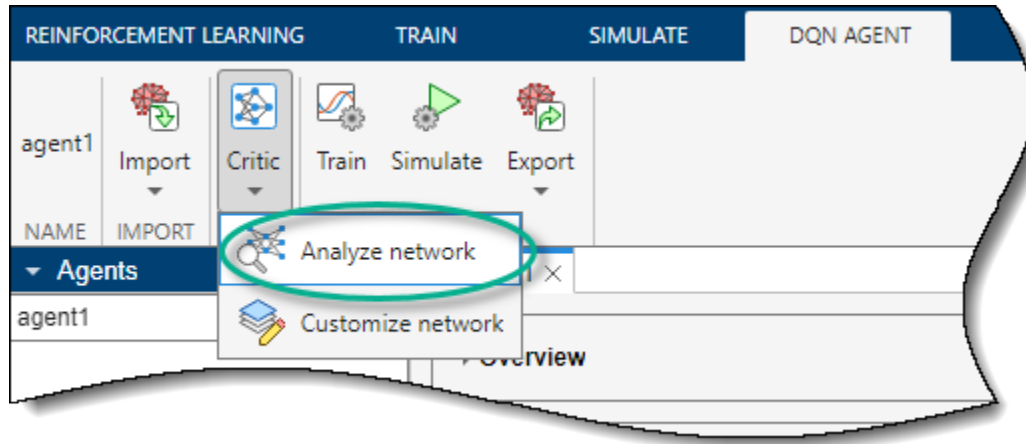
Observation Name	Domain	Dimension	Data Type
CartPole States	continuous	[4 1]	double

Action Specification

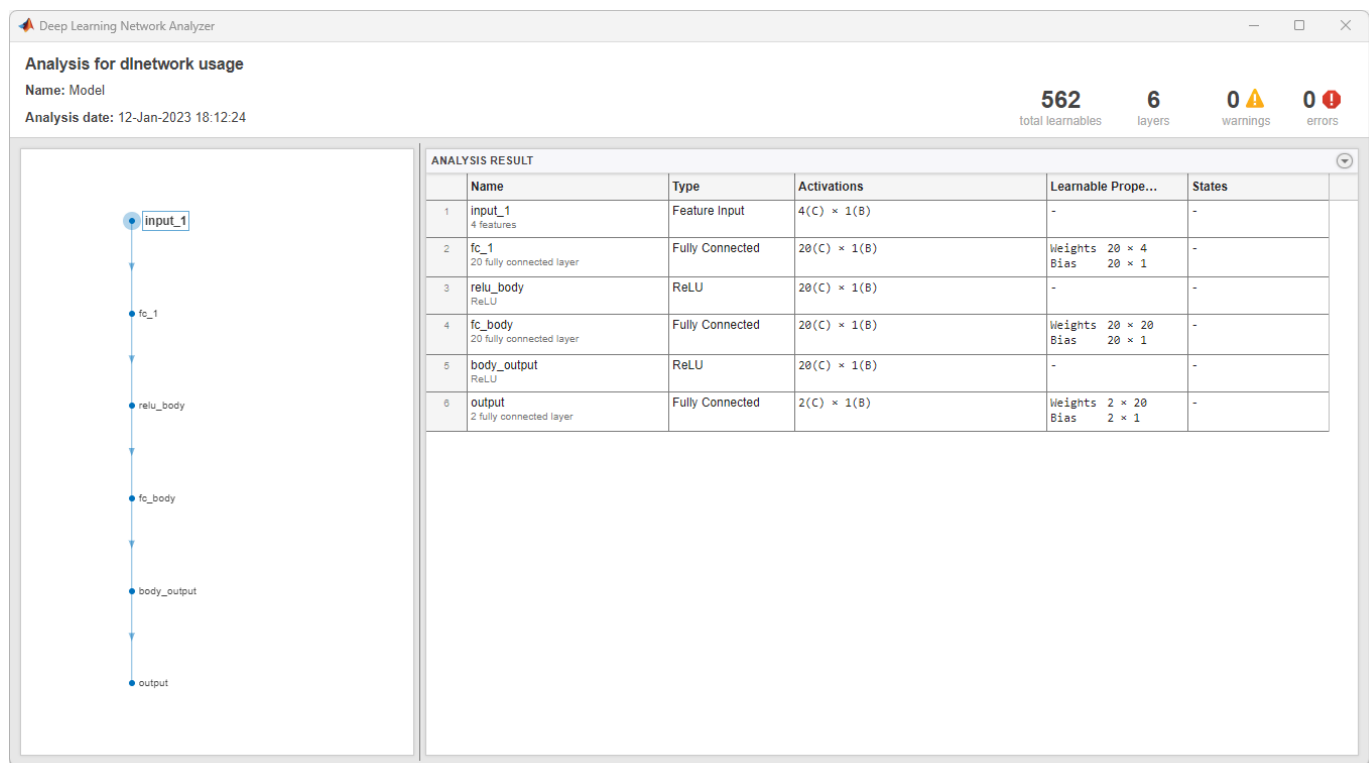
Action Name	Domain	Dimension	Data Type
CartPole Action	discrete	[1 1]	double

Hyperparameters

When you create a DQN agent in Reinforcement Learning Designer, the agent uses a default deep neural network structure for its critic. To view the critic network, on the **DQN Agent** tab, under **Critic**, select **Analyze Network**.

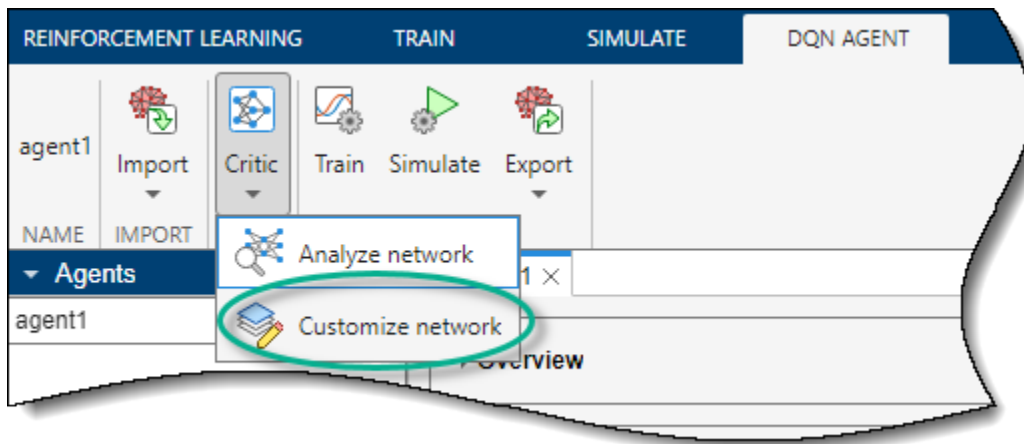


The Deep Learning Network Analyzer opens and displays the critic structure.

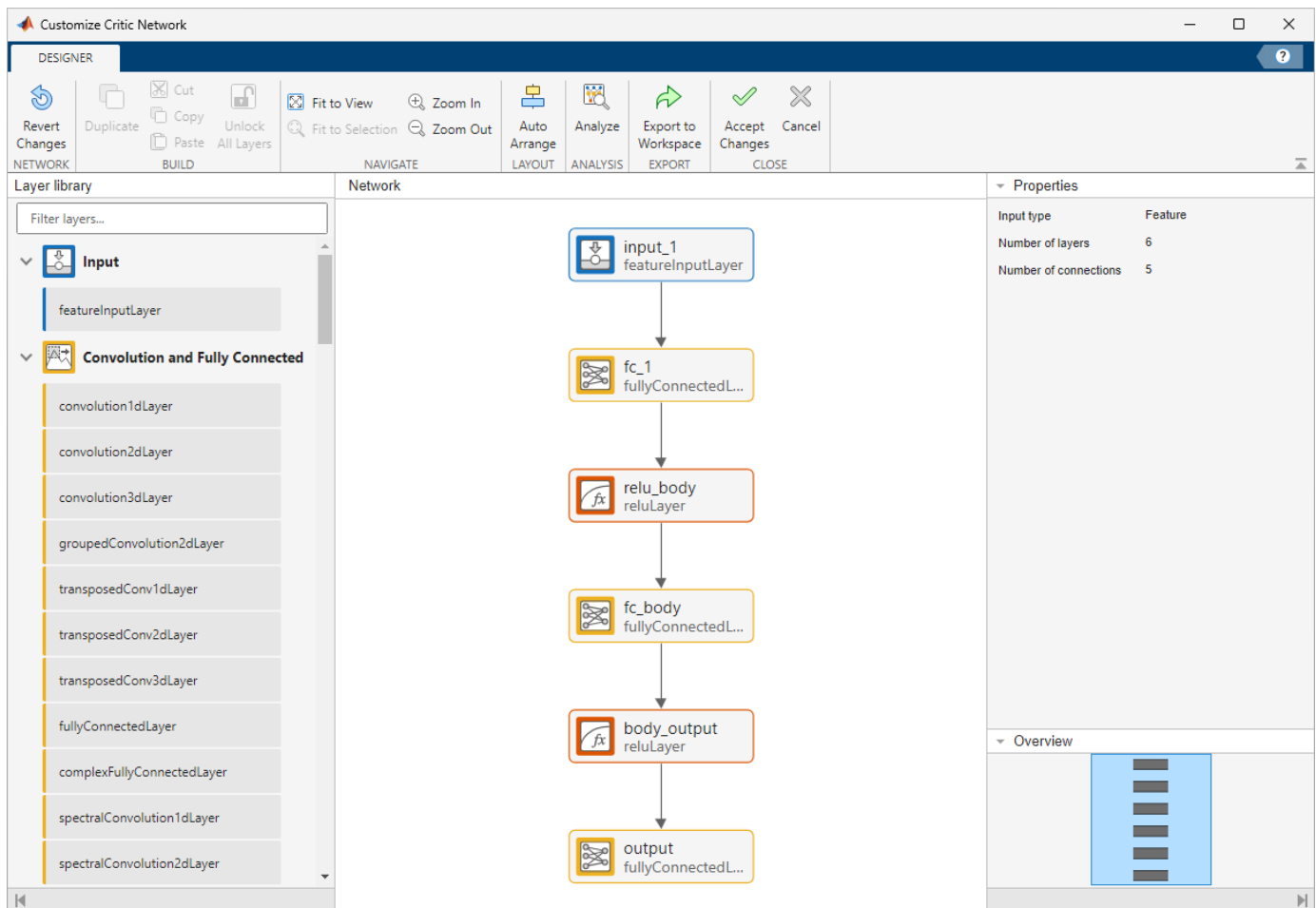


Close the Deep Learning Network Analyzer.

To modify the existing network instead, on the **DQN Agent** tab, under **Critic**, select **Customize Network**.



The Deep Network Designer opens and displays the critic structure, which you can modify.

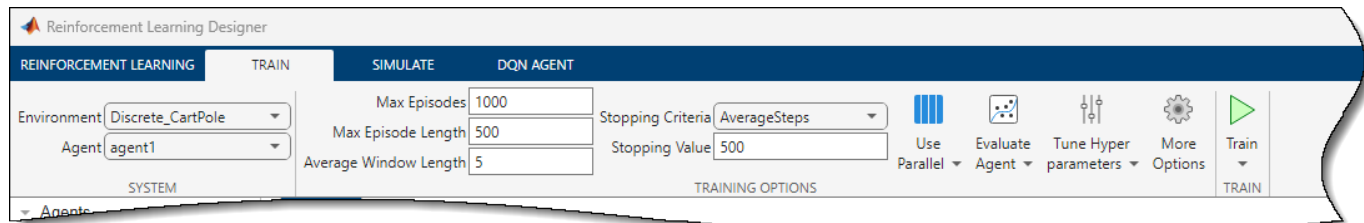


Close the Deep Network Designer.

Train Agent

To train your agent, on the **Train** tab, first specify options for training the agent. For information on specifying training options, see “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50.

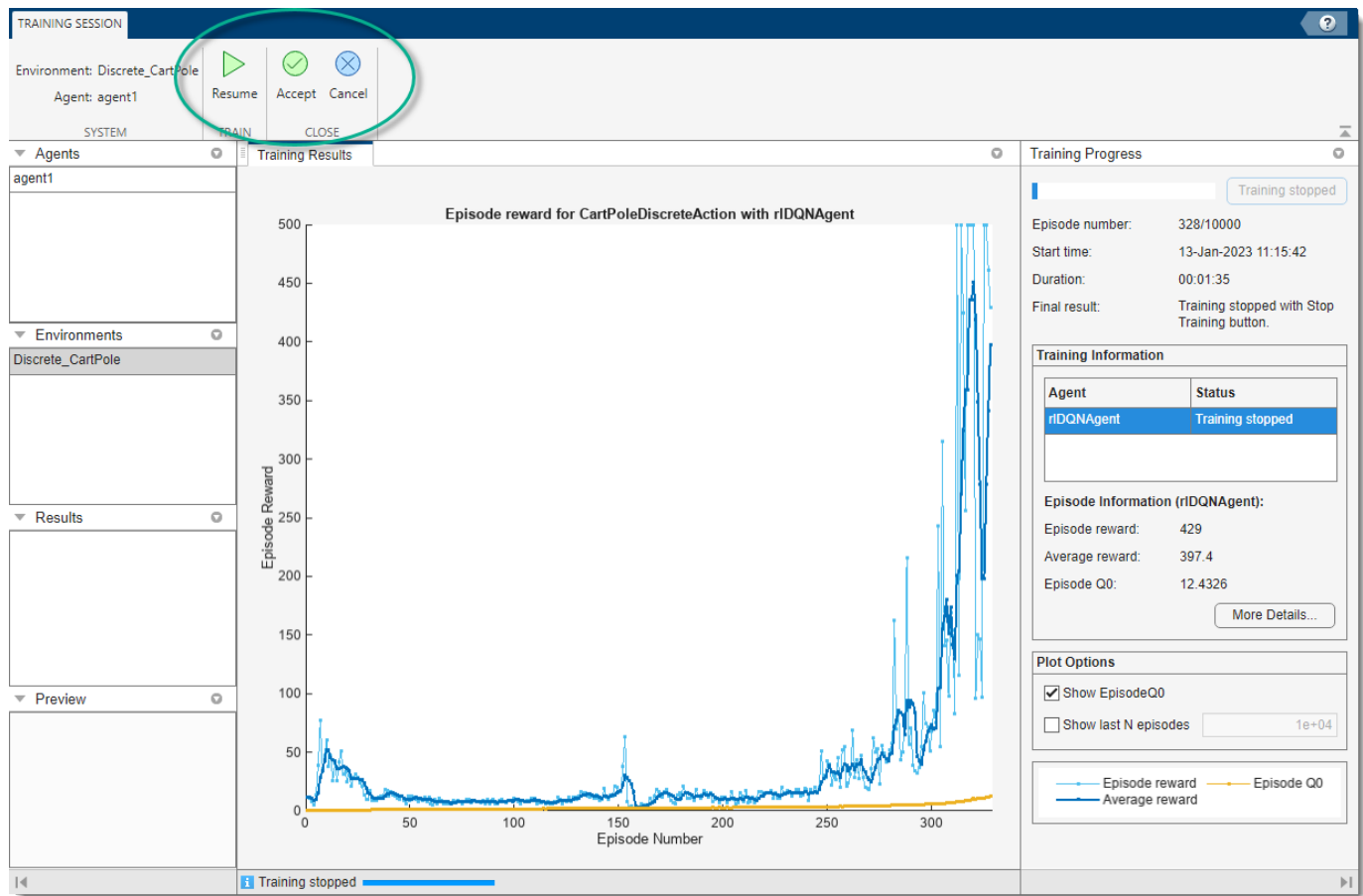
For this example, specify the maximum number of training episodes by setting **Max Episodes** to **1000**. For the other training options, use their default values. The default criteria for stopping is when the average number of steps per episode (over the last 5 episodes) is greater than **500**.



To start training, click **Train**.

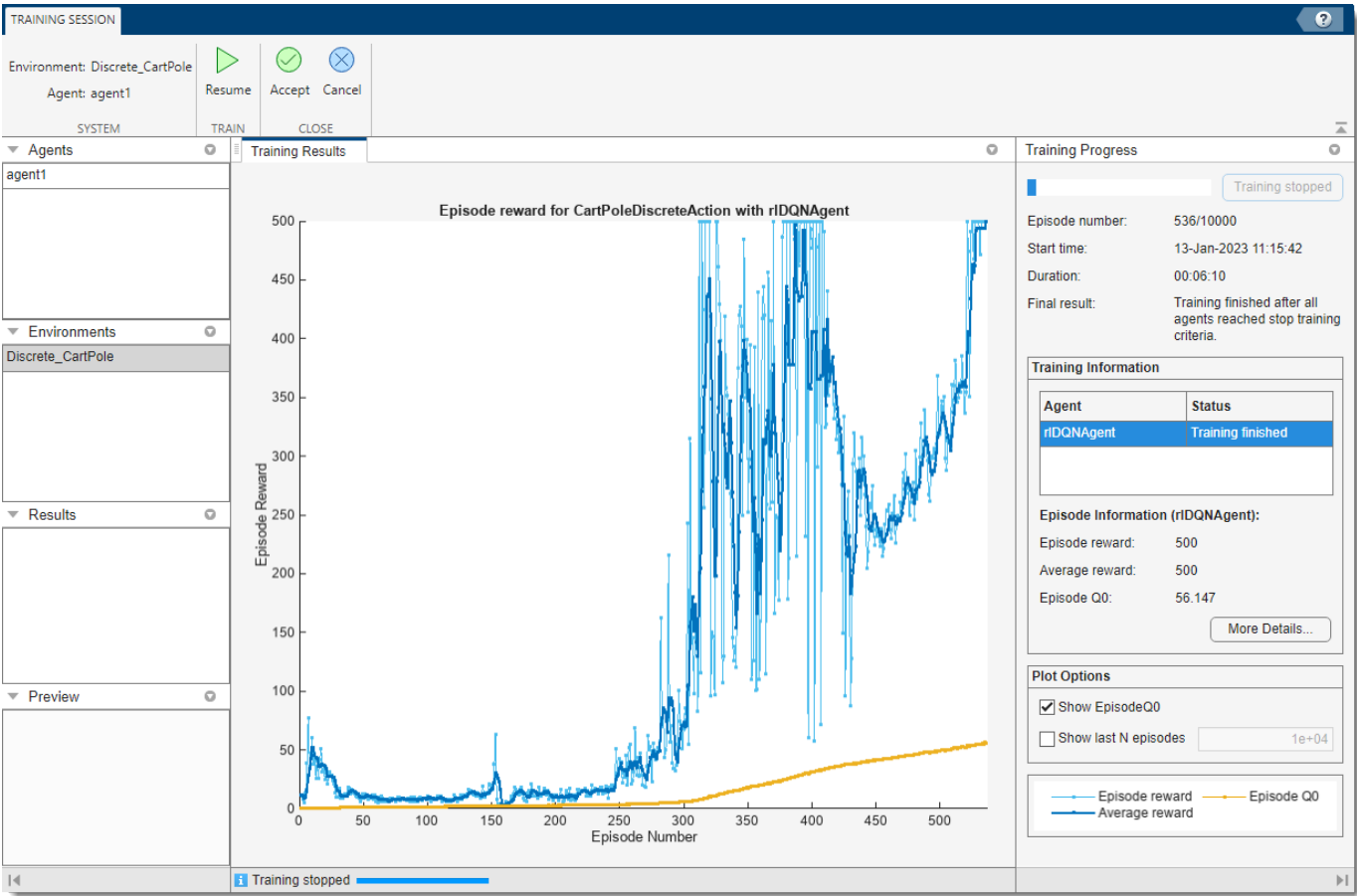
During training, the app opens the **Training Session** tab and displays the training progress in the **Training Results** document.

At any time during training, you can click on the **Stop** or **Stop Training** buttons to interrupt training and perform other operations on the command line.



At this point the **Resume**, **Accept**, and **Cancel** buttons in the **Training Session** tab give you the option to resume the training, accept the training results (which stores the training results and the trained agent in the app) or cancel the training altogether, respectively.

To resume training click **Resume**.



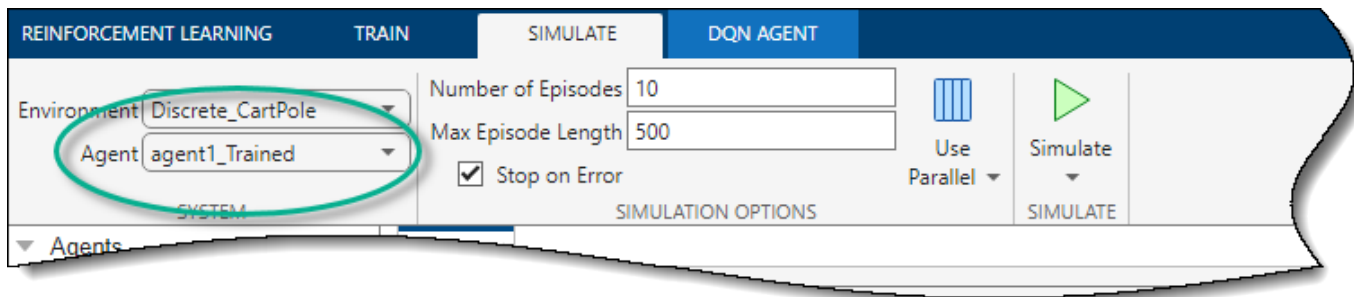
Here, the training stops when the average number of steps per episode is 500.

Note The training history and results depend on the specific configuration of your hardware, software, and random number generator, and are generally not exactly reproducible. For more information, see “Results Reproducibility” on page 5-10.

To accept the training results click **Accept**. In the **Agents** pane, the app adds the trained agent, agent1_Trained.

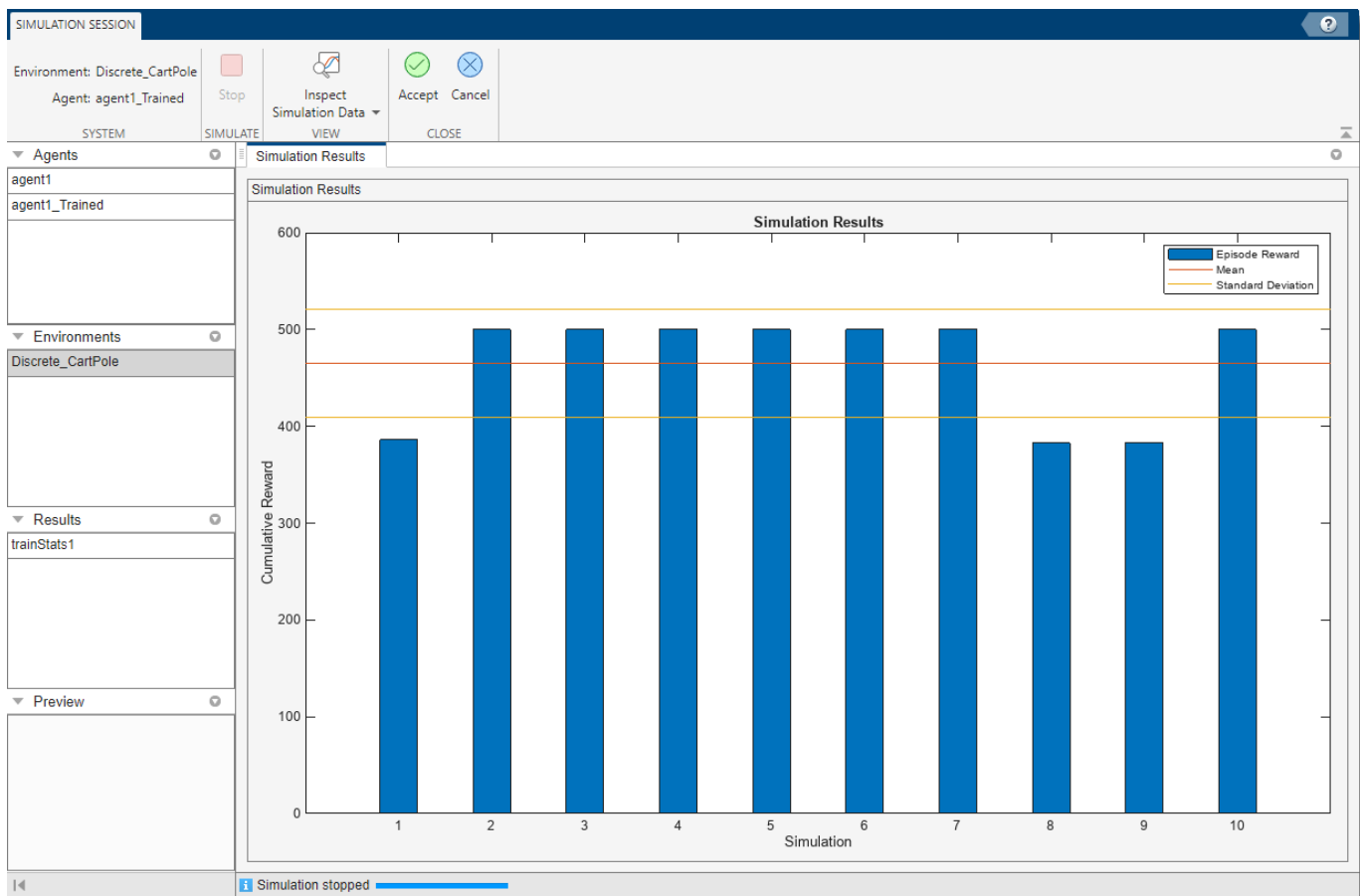
Simulate Agent and Inspect Simulation Results

To simulate the trained agent, on the **Simulate** tab, first select agent1_Trained in the **Agent** drop-down list, then configure the simulation options. For this example, use the default number of episodes (10) and maximum episode length (500). For more information on specifying simulation options, see “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50.



To simulate the agent, click **Simulate**.

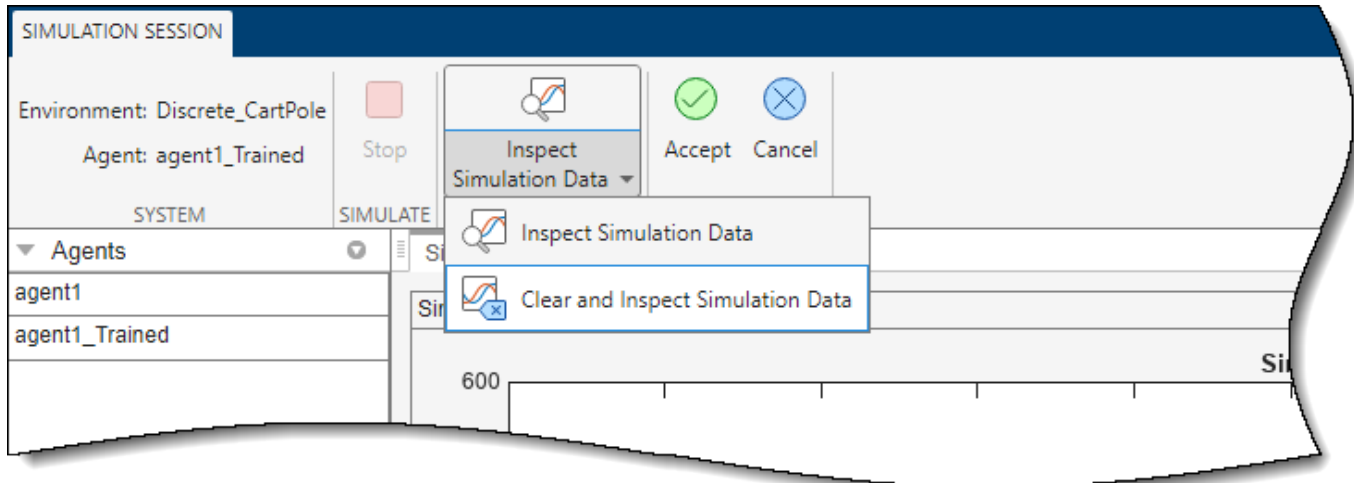
The app opens the **Simulation Session** tab. After the simulation is completed, the **Simulation Results** document shows the reward for each episode as well as the reward mean and standard deviation.



For three episodes the agent was not able to reach the maximum reward of 500. This suggests that the robustness of the trained agent to different initial conditions might be improved. In this case, training the agent longer, for example, by selecting an **Average Window Length** of 10 instead of 5, yields better robustness. You can also modify some DQN agent options such as **BatchSize** and **TargetUpdateFrequency** to promote faster and more robust learning.

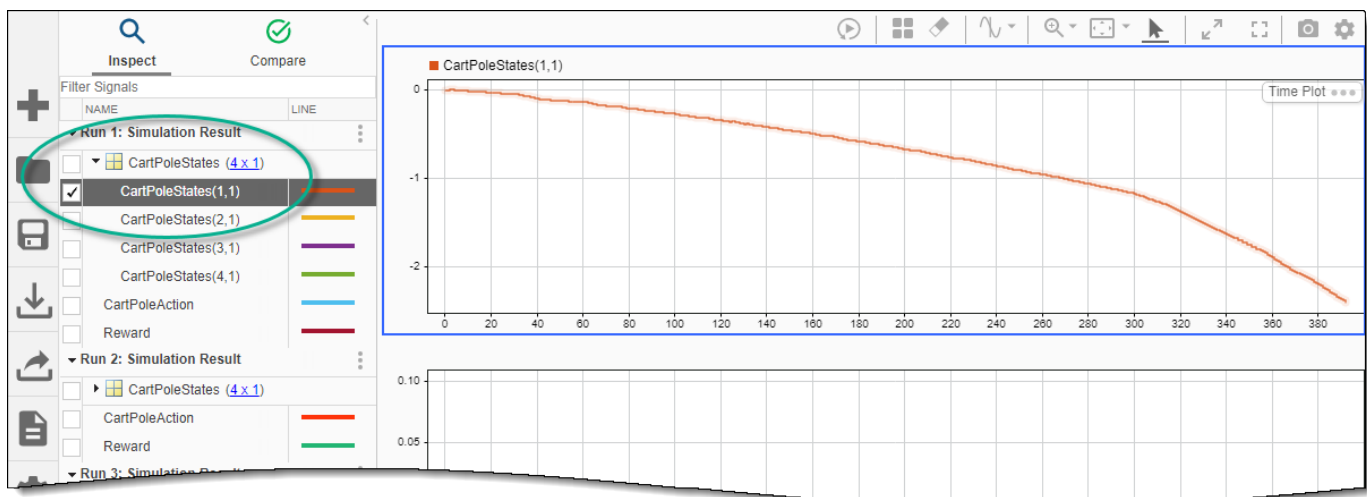
To analyze the simulation results, click **Inspect Simulation Data**. This opens the Simulation Data Inspector. For more information, see Simulation Data Inspector (Simulink).

You also have the option to preemptively clear from the Simulation Data Inspector any data that you might have loaded in a previous session. To do so, under **Inspect Simulation Data**, select **Clear and Inspect Simulation Data**.



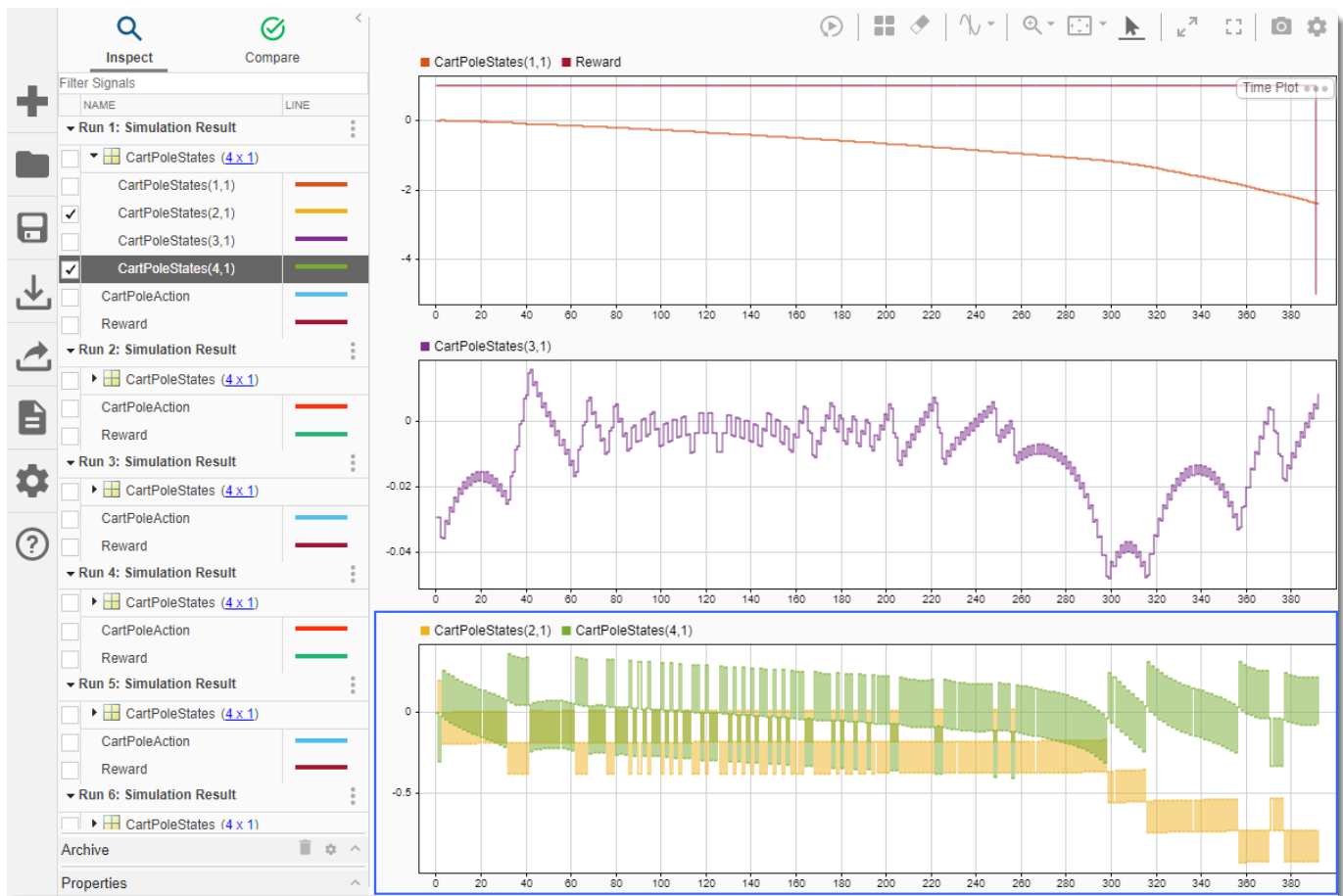
In the Simulation Data Inspector you can view the saved signals for each simulation episode.

By default, the upper plot area is selected. To show the first state (the cart position), during the first episode, under **Run 1: Simulation Result**, open the **CartPoleStates** variable, and select **CartPoleStates(1,1)**. The cart goes outside the boundary after about 390 seconds, causing the simulation to terminate.



To also show the reward in the upper plot area, select the **Reward** variable. Note that the units on the vertical axis change accordingly.

Click the middle plot area, and select the third state (pole angle). Then click the bottom area and select the second and fourth state (cart velocity and pole angle derivative).



For a related example, in which a DQN agent is trained on the same environment, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64.

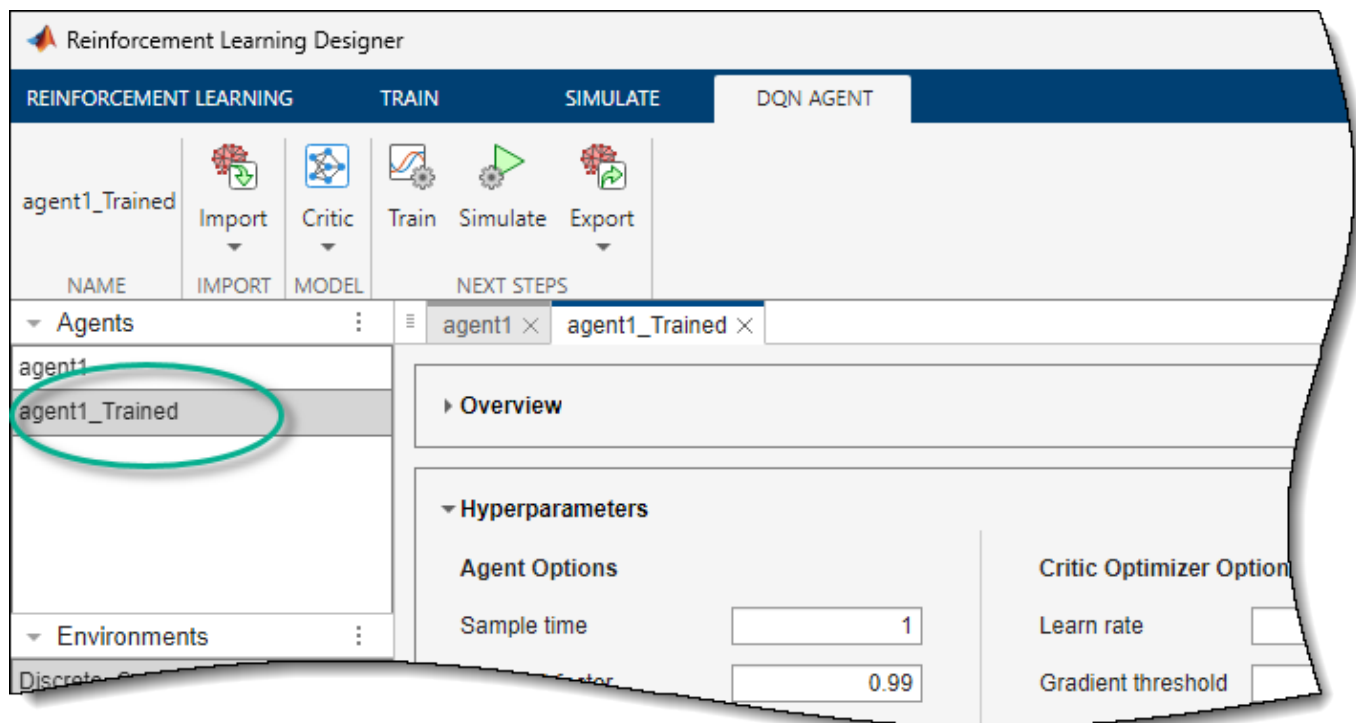
Close the Simulation Data Inspector.

To accept the simulation results, on the **Simulation Session** tab, click **Accept**.

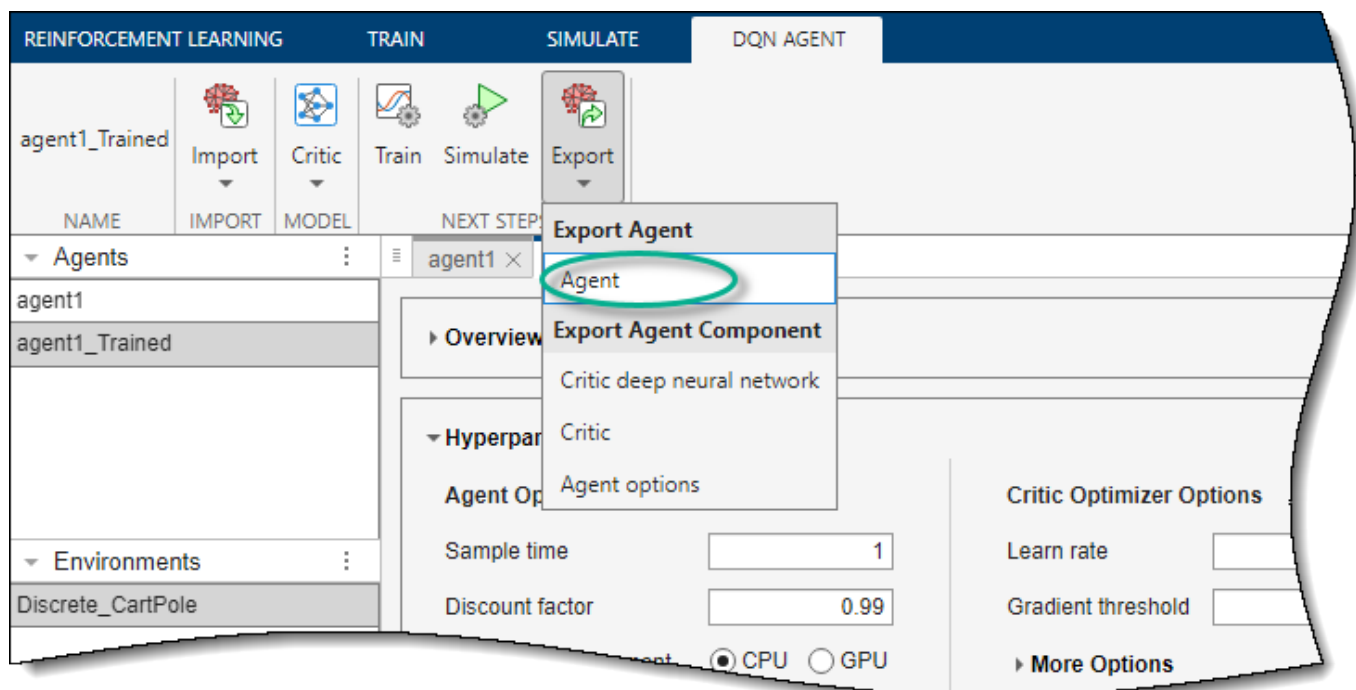
In the **Results** pane, the app adds the simulation results structure, `experience1`.

Export Agent and Save Session

To select the trained agent and open the corresponding **agent1_Trained** document, under the **Agents** pane, double click on `agent1_Trained`.



Then, to export the trained agent to the MATLAB workspace, on the **Reinforcement Learning** tab, under **Export**, select the trained agent.



To save the app session, on the **Reinforcement Learning** tab, click **Save Session**. In the future, to resume your work where you left off, you can open the session in Reinforcement Learning Designer.

Simulate Agent at the Command Line

To simulate the agent at the MATLAB command line, first load the cart-pole environment.

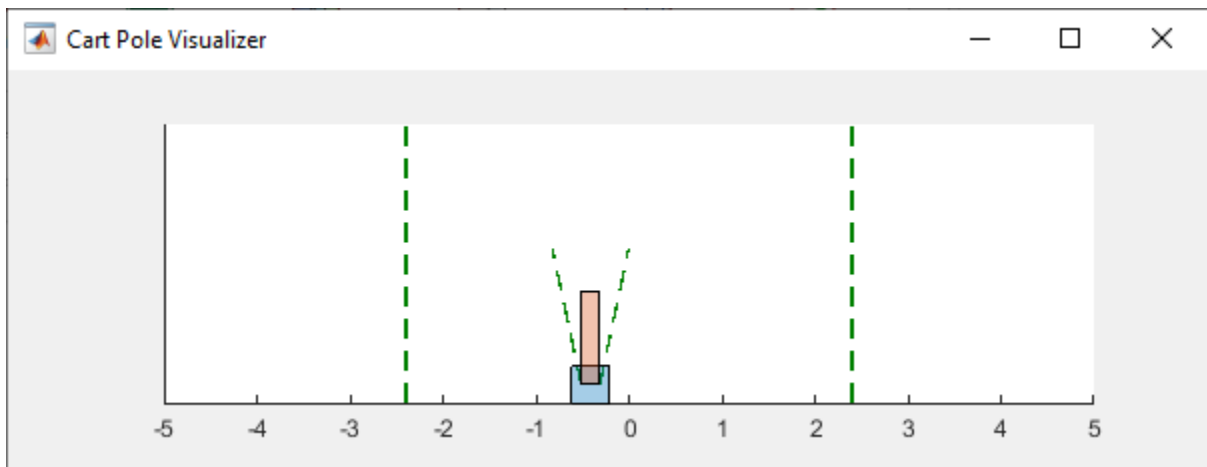
```
env = rlPredefinedEnv("CartPole-Discrete");
```

The cart-pole environment has an environment visualizer that allows you to see how the system behaves during simulation and training.

Plot the environment and perform a simulation using the trained agent that you previously exported from the app.

```
plot(env)
xpr2 = sim(env,agent1_Trained);
```

During the simulation, the visualizer shows the movement of the cart and pole. In this simulation, the trained agent is able to stabilize the system.



Finally, display the cumulative reward for the simulation.

```
sum(xpr2.Reward)
```

```
env =  
500
```

As expected, the cumulative reward is 500.

See Also

Apps

Reinforcement Learning Designer

Functions

train | analyzeNetwork

Related Examples

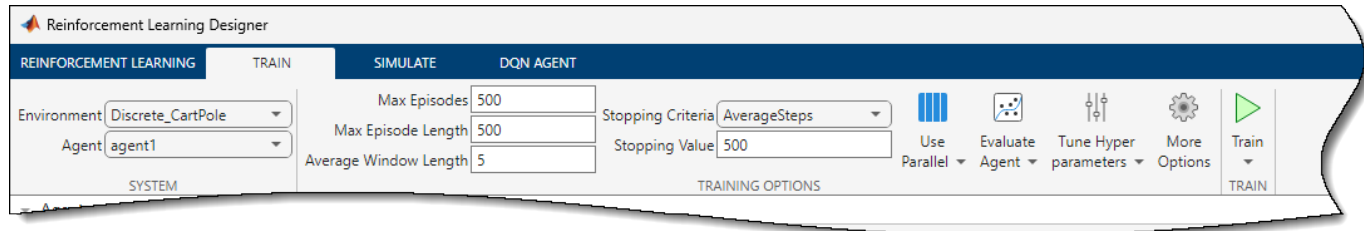
- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53
- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217

More About

- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142
- “Create Agents Using Reinforcement Learning Designer” on page 3-11
- “Specify Training Options in Reinforcement Learning Designer” on page 5-29
- “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3

Specify Training Options in Reinforcement Learning Designer

To configure the training of an agent in the Reinforcement Learning Designer app, specify training options on the **Train** tab.



Specify Basic Options

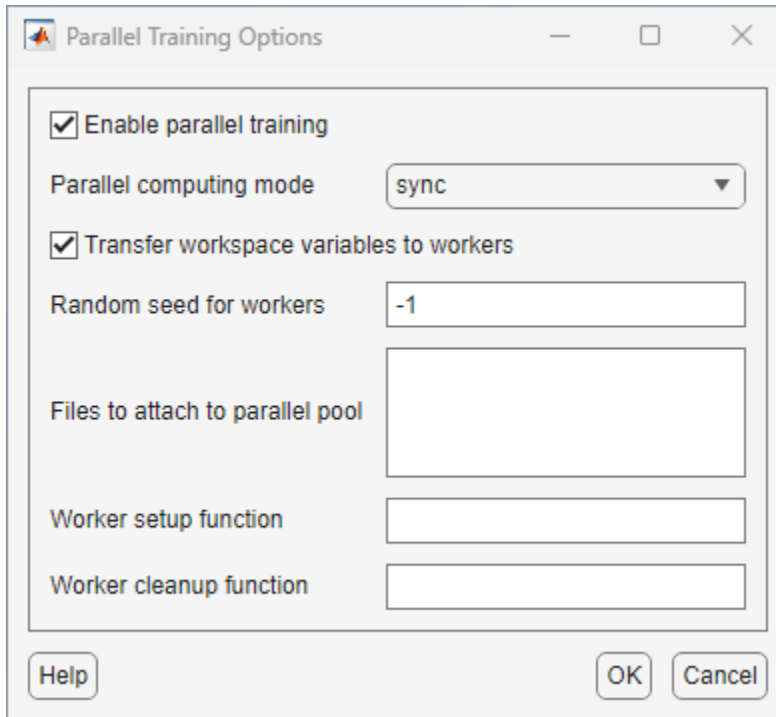
On the **Train** tab, you can specify these basic training options.

Option	Description
Max Episodes	Maximum number of episodes to train the agent, specified as a positive integer.
Max Episode Length	Maximum number of steps to run per episode, specified as a positive integer.
Stopping Criteria	Training termination condition, specified as one of the following values: AverageSteps — Stop training when the running average number of steps per episode equals or exceeds the critical value specified by Stopping Value. AverageReward — Stop training when the running average reward equals or exceeds the critical value. EpisodeReward — Stop training when the reward in the current episode equals or exceeds the critical value. GlobalStepCount — Stop training when the total number of steps in all episodes, that is, when the total number of times the agent is invoked, equals or exceeds the critical value. EpisodeCount — Stop training when the number of training episodes equals or exceeds the critical value.
Stopping Value	Critical value of the training termination condition in Stopping Criteria , specified as a scalar.
Average Window Length	Window length for averaging the scores, rewards, and number of steps for the agent when either Stopping Criteria or Save agent criteria specify an averaging condition.

Specify Parallel Training Options

To enable the use of multiple processes for training, on the **Train** tab, click the **Use Parallel** button . Training agents using parallel computing requires Parallel Computing Toolbox software. For more information, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

To specify options for parallel training, select **Use Parallel > Parallel training options**.



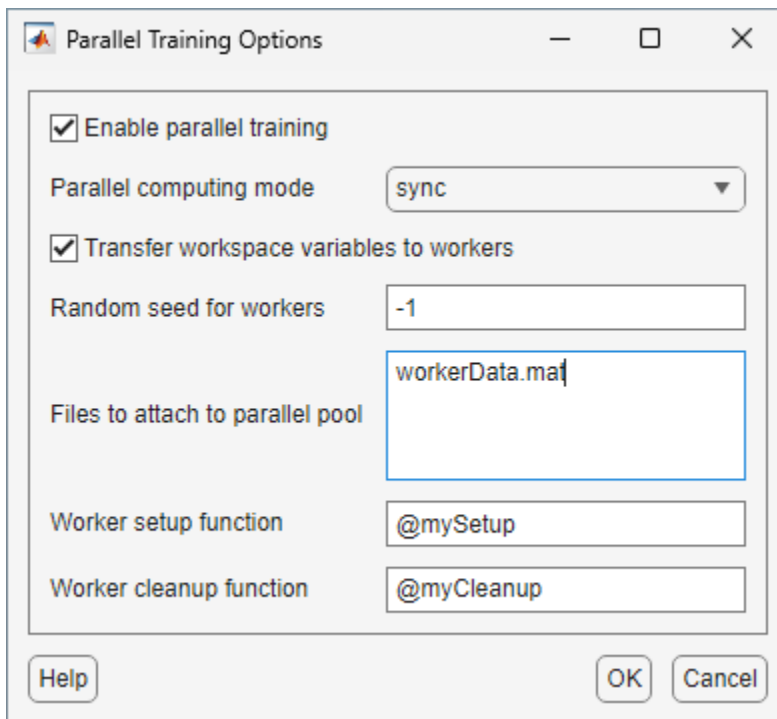
In the Parallel Training Options dialog box, you can specify these training options.

Option	Description
Enable parallel training	Enables using multiple processes to perform environment simulations during training. Select this option by clicking the Use Parallel button  .
Parallel computing mode	Parallel computing mode, specified as one of these values: sync — Use <code>parpool</code> to run synchronous training on the available workers. The parallel pool client (the process that starts the training) updates the parameters of its actor and critic, based on the results from all the workers, and sends the updated parameters to all workers. When you select this option, workers must pause execution until all workers are finished. As a result, the training only advances as fast as the slowest worker allows. async — Use <code>parpool</code> to run asynchronous training on the available workers. In this case, workers send their data back to the client as soon as they finish and receive updated parameters from the client. The workers then continue with their task.

Option	Description
Transfer workspace variables to workers	Select this option to send model and workspace variables to parallel workers. When you select this option, the parallel pool client sends variables used in models and defined in the MATLAB workspace to the workers.
Random seed for workers	Randomizer initialization for workers, specified as one of these values: –1 — Assign a unique random seed to each worker. The value of the seed is the worker ID. –2 — Do not assign a random seed to the workers. - Vector — Manually specify the random seed for each worker. The number of elements in the vector must match the number of workers.
Files to attach to parallel pool	Additional files to attach to the parallel pool. Specify names of files in the current working directory, with one name on each line.
Worker setup function	Function to run before training starts, specified as a handle to a function having no input arguments. This function runs once for each worker before training begins. Write this function to perform any processing that you need prior to training.
Worker cleanup function	Function to run after training ends, specified as a handle to a function having no input arguments. You can write this function to clean up the workspace or perform other processing after training terminates.

This figure shows an example parallel training configuration for these files and functions:

- Data file attached to the parallel pool — `workerData.mat`
- Worker setup function — `mySetup.m`
- Worker cleanup function — `myCleanup.m`

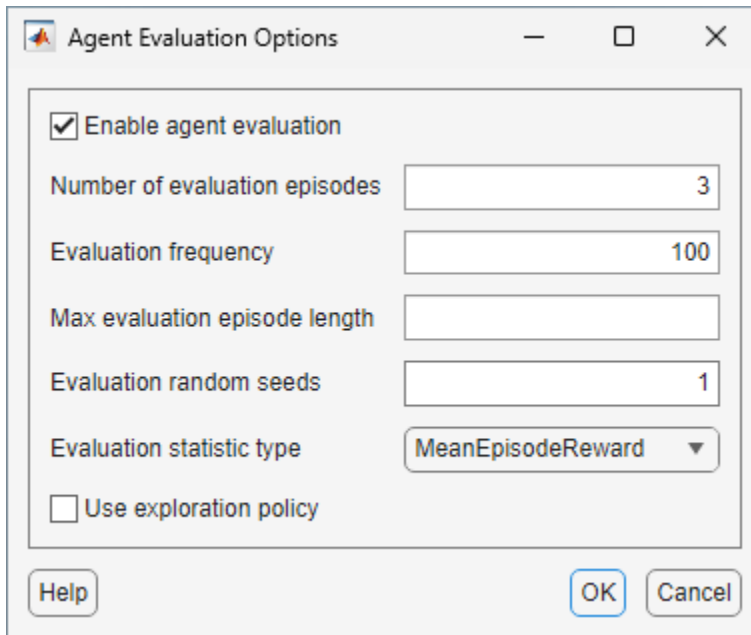


For more information on parallel training options, see the `UseParallel` and `ParallelizationOptions` properties in `rlTrainingOptions`. For more information on parallel training, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

Specify Agent Evaluation Options

To enable agent evaluation at regular intervals during training, on the **Train** tab, click .

To specify agent evaluation options, select **Evaluate Agent > Agent evaluation options**.



The dialog box titled "Agent Evaluation Options" contains the following settings:

- ☒ Enable agent evaluation
- Number of evaluation episodes: 3
- Evaluation frequency: 100
- Max evaluation episode length: (empty)
- Evaluation random seeds: 1
- Evaluation statistic type: MeanEpisodeReward (dropdown menu)
- ☐ Use exploration policy

Buttons at the bottom: Help, OK, Cancel.

In the Agent Evaluation Options dialog box, you can specify these training options.

Option	Description
Enable agent evaluation	Enables periodic agent evaluation during training. You can select this option by clicking the Evaluate Agent button  .
Number of evaluation episodes	Number of consecutive evaluation episodes, specified as a positive integer. After running the number of consecutive training episodes specified in Evaluation frequency, the software runs the number of evaluation episodes specified in this field, consecutively. For example, if you specify 100 in the Evaluation frequency field, and 3 in this field, then three evaluation episodes run, consecutively, after 100 the training episodes. These three evaluation episodes are used to calculate a single statistic, specified by the Evaluation statistic type field. The statistic is returned as the 100th element the of training result object instantiated after training. After 200 training episodes, three new evaluation episodes run, with their statistic returned in the 200th element of the training results object, and so on. The default is value 3.
Evaluation frequency	Evaluation period, specified as a positive integer. This value is the number of consecutive training episodes after which the number of consecutive evaluation episodes specified in the Number of evaluation episodes field run. For example, if you specify 100 in this field and 3 in the Number of evaluation episodes field, three evaluation episodes run, consecutively, after the 100 training episodes. The default is value 100.

Option	Description
Max evaluation episode length	Maximum number of steps to run for an evaluation episode, specified as a positive integer. This value is the maximum number of steps to run for an evaluation episode if other termination conditions are not met. To accurately assess the agent stability and performance, it is often useful to specify a larger number of steps for an evaluation episode than for a training episode. If you leave this field empty (default), the maximum number of steps per episode specified in the Max Episode Length field is used.
Evaluation random seeds	Random seeds used for evaluation episodes, specified as follows: • Empty field — The random seed is not initialized before an evaluation episode. • Nonnegative integer — The random seed is reinitialized to the specified value before every first evaluation episode that occurs after the number of consecutive training episodes specified in the Evaluation frequency field. This is the default behavior, with the seed initialized to 1. • Vector of nonnegative integers — with the same number of elements as the number of evaluation episodes specified in the Number of evaluation episodes field — Before each episode of an evaluation sequence, the random seed is reinitialized to the corresponding element of the specified vector. This reinitialization guarantees that the <i>i</i>th episode of each evaluation sequence always runs with the same random seed, which helps when comparing evaluation episodes that occur at different stages of training. The current random seed used for training is stored before the first episode of an evaluation sequence and reset as the current seed after the evaluation sequence. This behavior ensures that the training results with evaluation are the same as the results without evaluation.
Evaluation statistic type	Type of evaluation statistic for each group of consecutive evaluation episodes, specified as one of these strings: • "MeanEpisodeReward" — Mean value of the evaluation episodes rewards. This setting is the default. • "MedianEpisodeReward" — Median value of the evaluation episodes rewards. • "MaxEpisodeReward" — Maximum value of the evaluation episodes rewards. • "MinEpisodeReward" — Minimum value of the evaluation episodes rewards. This value is returned, in the training result object, as the element of the EvaluationStatistics vector corresponding to the training episode that precedes the group of consecutive evaluation episodes.

Option	Description
Use exploration policy	Option to use exploration policy during evaluation episodes. When this option is disabled (default) the agent uses its base greedy policy when selecting actions during an evaluation episode. When you enable this option, the agent uses its base exploration policy when selecting actions during an evaluation episode.

For more information on evaluation options, see `rlEvaluator`.

Specify Hyperparameter Tuning Options

Hyperparameter tuning consists in a series of training experiments, also referred to as trials, designed to select the best combination of hyperparameters. To enable a hyperparameter tuning

session, on the **Train** tab, click on the **Tune Hyperparameters**  button.

To specify hyperparameter tuning options, select **Tune Hyperparameters > Hyperparameter tuning options**.

☐ Enable hyperparameter tuning

Tuning algorithm: Bayesian optimization

Hyperparameter selection

	Optimize	Min	Max	LogScaling
ActorLearnRate	☒	1e-06	0.1	☒
CriticLearnRate	☒	1e-06	0.1	☒
MiniBatchSize	☒	64	1024	☐
StandardDeviation	☒	0.1	0.5	☐
StandardDeviationDecayRate	☐	1e-06	0.01	☒
DiscountFactor	☐	0.1	1	☒

Restore Defaults

Tuning goal: Learning performance Random seed: 0

Max trial evaluations: 100 ☒ Save artifacts to disk

Max time (hours): Inf Save directory: Trials

Help OK Cancel

In the Tuning Options dialog box, you can specify these training options:

Option	Description
Enable hyperparameter tuning	Enables hyperparameter tuning. This option gets selected also when you click on the Tune Hyperparameters  button.
Tuning algorithm	Tuning algorithm, specified as one of these values: Bayesian optimization — Use Bayesian optimization. This option requires Statistics and Machine Learning Toolbox™. Bayesian optimization performs an adaptive search on the hyperparameter space. For more information, see “Bayesian Optimization Algorithm” (Statistics and Machine Learning Toolbox). Grid search — Use grid search. Grid search performs an exhaustive sweep over the hyperparameter space.

Option	Description
Hyperparameter selection	The Hyperparameter selection section allows you to select up to six parameters to be tuned, depending on the agent type. You can tune the learning rates used by the critic and, if available, the actor, mini-batch size, discount factor. You can also tune exploration parameters such as the action noise standard deviation and its decay rate or the epsilon decay rate and its decay rate. You can enable the tuning of a hyperparameter by selecting the corresponding Optimize check box. You can then adjust the hyperparameter search range using the corresponding Min and Max fields. Check Log Scaling to space the grid points logarithmically for the search dimension corresponding to the hyperparameter. For more information about the hyperparameter that you can tune in the app and their default range, see “Hyperparameter Selection” on page 5-37.
Restore Defaults	Click this button to reset the hyperparameter configuration.
Tuning goal	Tuning goal, specified as one of these values: • Learning performance — When you specify this option, the score computed for each trial (that is, for each training) is the final average episodic reward for that trial. • Grid search — Specify this option to perform an exhaustive sweep over all the points in the hyperparameter grid.
Random seed	Initial random seeds used for each hyperparameter tuning training run, specified as a nonnegative integer.
Max trial evaluations	Maximum number of trials (training experiments). When the number of completed trials reaches this value, the software selects the combination of hyperparameters that yields the best score.
Max time (hours)	Maximum time in hours to perform tuning. When the time elapsed from the beginning of the hyperparameter tuning session reaches this value, the software does not perform additional training experiments. The software then selects the combination of hyperparameters that yield the best score.
Save artifacts to disk	Enable or disable saving artifacts, (that is, the tuned agent and training result) to disk.
Save directory	Directory in which the software saves the artifacts.

Hyperparameter Selection

The hyperparameter selection section of the hyperparameter Tuning Options dialog box allows you to tune these hyperparameters.

y p e r p a r a m e t e r	Description
t o r L e a r n R a t e	Actor learning rate, available for all agents except DQN and TRPO. For more information, see the optimizer options object of the actor used by the agent.

Hyperparameter	Description
Critic Learning Rate	Critic learning rate, available for all agents. For more information, see the optimizer options object of the critic used by the agent.

y p e r p a r a m e t e r	Description	B o o l e a n ()
p e r i e n c e H o r i z o n	Number of steps used to calculate the advantage value, available for PPO and TRPO agents. For more information, see ExperienceHorizon.	I n t e g e r ()

Parameter	Description
Mini-Batch Size	Size of the mini-batch, available for all agents. For more information, see the agent options object.

y p e r p a r a m e t e r	Description
t r o p y L o s s W e i g h t	Weight of the entropy loss, available for PPO and TRPO agents. For more information, see EntropyLossWeight .

Description		
Discount factor	Discount factor, available for all agents. For more information, see the agent options.	
Epsilon	Value of the epsilon variable, for epsilon-greedy exploration, available for DQN agents. For more information, see <code>EpsilonGreedyExploration</code> .	

y p e r p a r a m e t e r	Description
s i l o n D e c a y	Decay rate of the epsilon variable, for epsilon-greedy exploration, available for DQN agents. For more information, see EpsilonGreedyExploration.

Hyperparameter	Description
Mean Attraction Constant	Value of the standard deviation for the agent noise model, available for DDPG agents. For more information, see <code>NoiseOptions</code> .

Hyperparameter	Description
Standard Deviation	Value of the standard deviation for the agent noise model, available for DDPG and TD3 agents. For more information, see <code>NoiseOptions</code>.

Description	Hyperparameter
Decay rate of the standard deviation for the agent noise model, available for DDPG and TD3 agents. For more information, see NoiseOptions .	Standard Deviation Decay Rate

Note You can tune a set of hyperparameters that is different from the set allowed in the Hyperparameter tuning options dialog box using the MATLAB code generated by the app. To do so, generate code by selecting **Train > Generate MATLAB function for hyperparameter tuning**. Then, customize the generated code to include the hyperparameters to tune.

For an example on how to tune hyperparameters in Reinforcement Learning Designer, see “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53. For an example on how to perform parameter sweeping using the Experiment Manager app, see “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240. For an general example on how to tune a custom set of hyperparameters programmatically, using Bayesian optimization, see “Tune Hyperparameters Using Bayesian Optimization” on page 5-250.

Specify Additional Options

To specify additional training options, on the **Train** tab, click **More Options**.

In the More Training Options dialog box, you can specify these options.

Option	Description
Save agent criteria	Condition for saving agents during training, specified as one of these values: none — Do not save any agents during training. AverageSteps — Save the agent when the running average number of steps per episode equals or exceeds the critical value specified by Save agent value. AverageReward — Save the agent when the running average reward equals or exceeds the critical value. EpisodeReward — Save the agent when the reward in the current episode equals or exceeds the critical value. GlobalStepCount — Save the agent when the total number of steps in all episodes, that is, the total number of times the agent is invoked, equals or exceeds the critical value. EpisodeCount — Save the agent when the number of training episodes equals or exceeds the critical value.
Save agent value	Critical value of the save agent condition in Save agent criteria , specified as a scalar or "none".
Save directory	Folder for saved agents. If you specify a folder name and that folder does not exist, the app creates the folder in the current working directory. To interactively select a folder, click Browse .
Show verbose output	Select this option to display the training progress at the command line.
Stop on error	Select this option to stop training when an error occurs during an episode.

For more information training options, see `rlTrainingOptions`.

See Also

Apps

Reinforcement Learning Designer

Functions

`train`

Objects

`rlTrainingOptions` | `rlEvaluator`

Related Examples

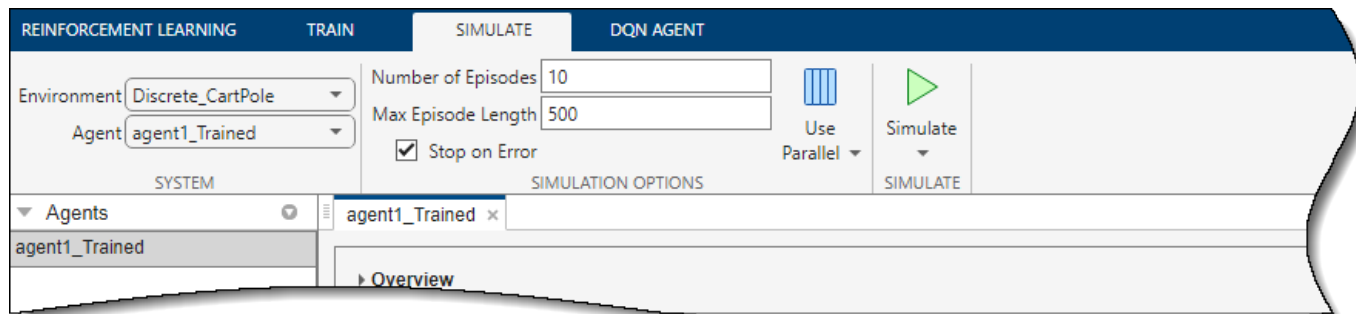
- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50

More About

- “Create Agents Using Reinforcement Learning Designer” on page 3-11
- “Reinforcement Learning Agents” on page 3-2
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Specify Simulation Options in Reinforcement Learning Designer

To configure the simulation of an agent in the Reinforcement Learning Designer app, specify simulation options on the **Simulate** tab.



Specify Simulation Options in Reinforcement Learning Designer

Specify Basic Options

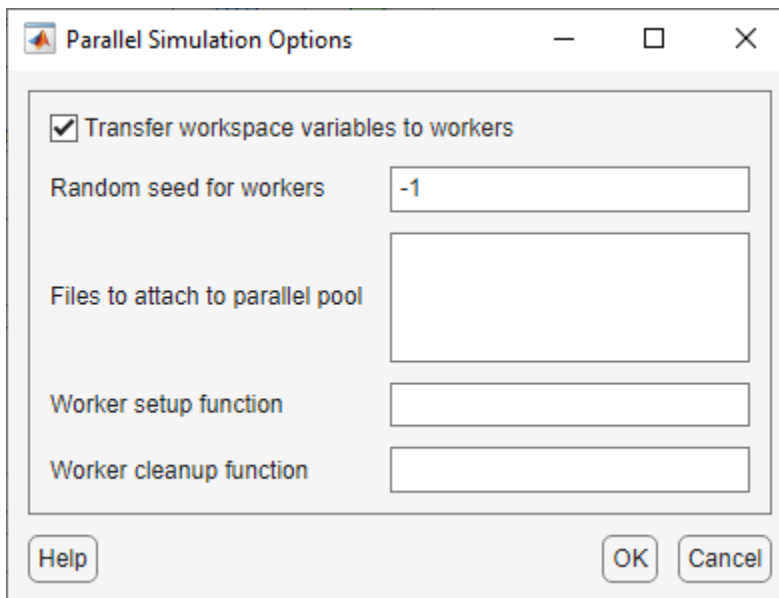
On the **Simulate** tab, you can specify the following basic simulation options.

Option	Description
Number of Episodes	Number of episodes to simulate the agent, specified as a positive integer. At the start of each simulation episode, the app resets the environment.
Max Episode Length	Number of steps to run the simulation, specified as a positive integer. In general, you define episode termination conditions in the environment. This value is the maximum number of steps to run in the simulation if those termination conditions are not met.
Stop on Error	Select this option to stop simulation when an error occurs during an episode.

Specify Parallel Simulation Options

To simulate your agent using parallel computing, on the **Simulate** tab, click . Simulating agents using parallel computing requires Parallel Computing Toolbox software. For more information, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

To specify options for parallel simulation, select **Use Parallel > Parallel training options**.



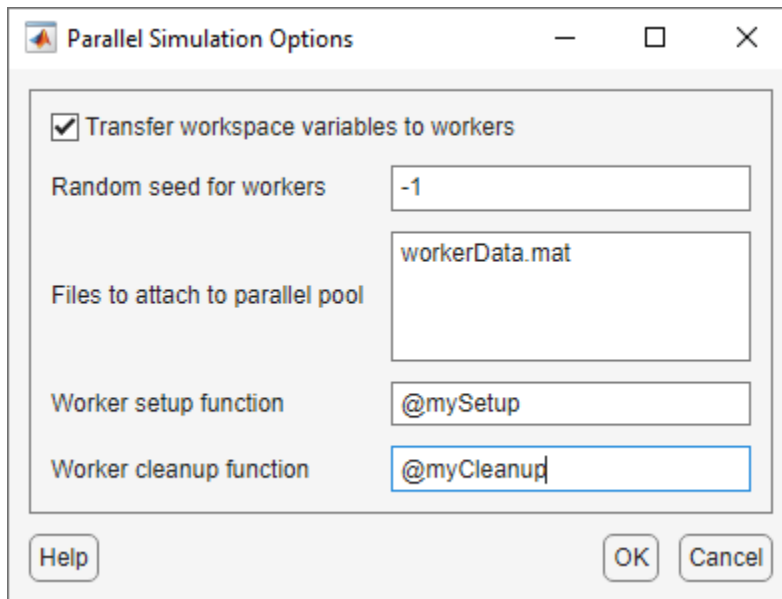
In the Parallel Simulation Options dialog box, you can specify the following training options.

Option	Description
Transfer workspace variables to workers	Select this option to send model and workspace variables to parallel workers. When you select this option, the parallel pool client (the process that starts the training) sends variables used in models and defined in the MATLAB workspace to the workers.
Random seed for workers	Randomizer initialization for workers, specified as one of the following values. –1 — Assign a unique random seed to each worker. The value of the seed is the worker ID. –2 — Do not assign a random seed to the workers. - Vector — Manually specify the random seed for each worker. The number of elements in the vector must match the number of workers.
Files to attach to parallel pool	Additional files to attach to the parallel pool. Specify names of files in the current working directory, with one name on each line.
Worker setup function	Function to run before simulation starts, specified as the name of a function having no input arguments. This function is run once per worker before simulation begins. Write this function to perform any processing that you need prior to training.
Worker cleanup function	Function to run after simulation ends, specified as the name of a function having no input arguments. You can write this function to clean up the workspace or perform other processing after training terminates.

The following figure shows an example parallel training configuration the following files and functions.

- Data file attached to the parallel pool — `workerData.mat`
- Worker setup function — `mySetup.m`

- Worker cleanup function — `myCleanup.m`



See Also

Apps

Reinforcement Learning Designer

Functions

`sim`

Objects

`rlSimulationOptions`

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Specify Training Options in Reinforcement Learning Designer” on page 5-29

More About

- “Create Agents Using Reinforcement Learning Designer” on page 3-11
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Tune Hyperparameters Using Reinforcement Learning Designer

This example shows how to tune agent hyperparameters using Reinforcement Learning Designer.

Introduction

The behavior of reinforcement learning algorithms depends on the set of hyperparameters you choose. The optimal set of hyperparameters varies depending on the application you use. The default set of hyperparameters often is not sufficient for the agent to learn the desired policy. When the default set of hyperparameters is not sufficient for training the agent, you can tune the hyperparameters. In this example, you tune the hyperparameters of a deep Q-network (DQN) agent for a discrete cart-pole environment using Reinforcement Learning Designer.

Open Reinforcement Learning Designer

Open the Reinforcement Learning Designer app.

You can open the app as follows:

- From the MATLAB® Toolstrip — On the Apps tab, under Machine Learning and Deep Learning, click the Reinforcement Learning Designer icon.
- At the MATLAB command prompt — Enter `reinforcementLearningDesigner`.

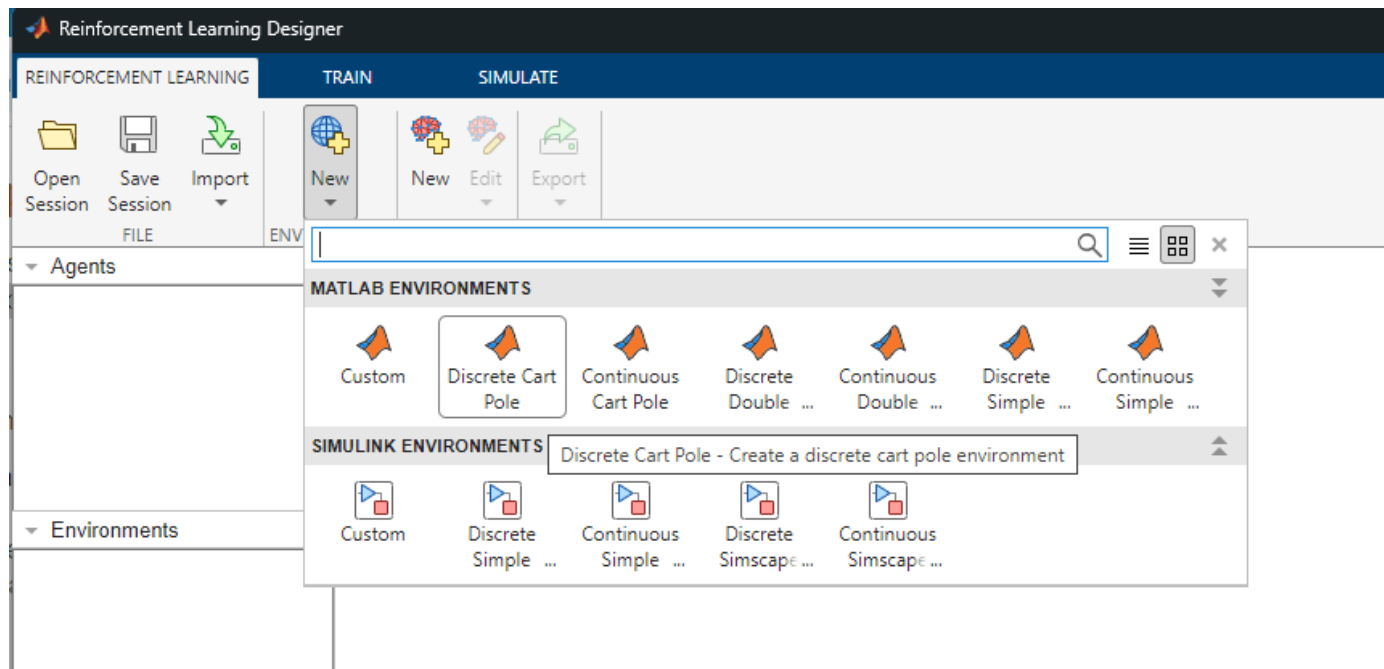
For this example, use the MATLAB command prompt.

```
reinforcementLearningDesigner
```

Load Environment and Create Agent

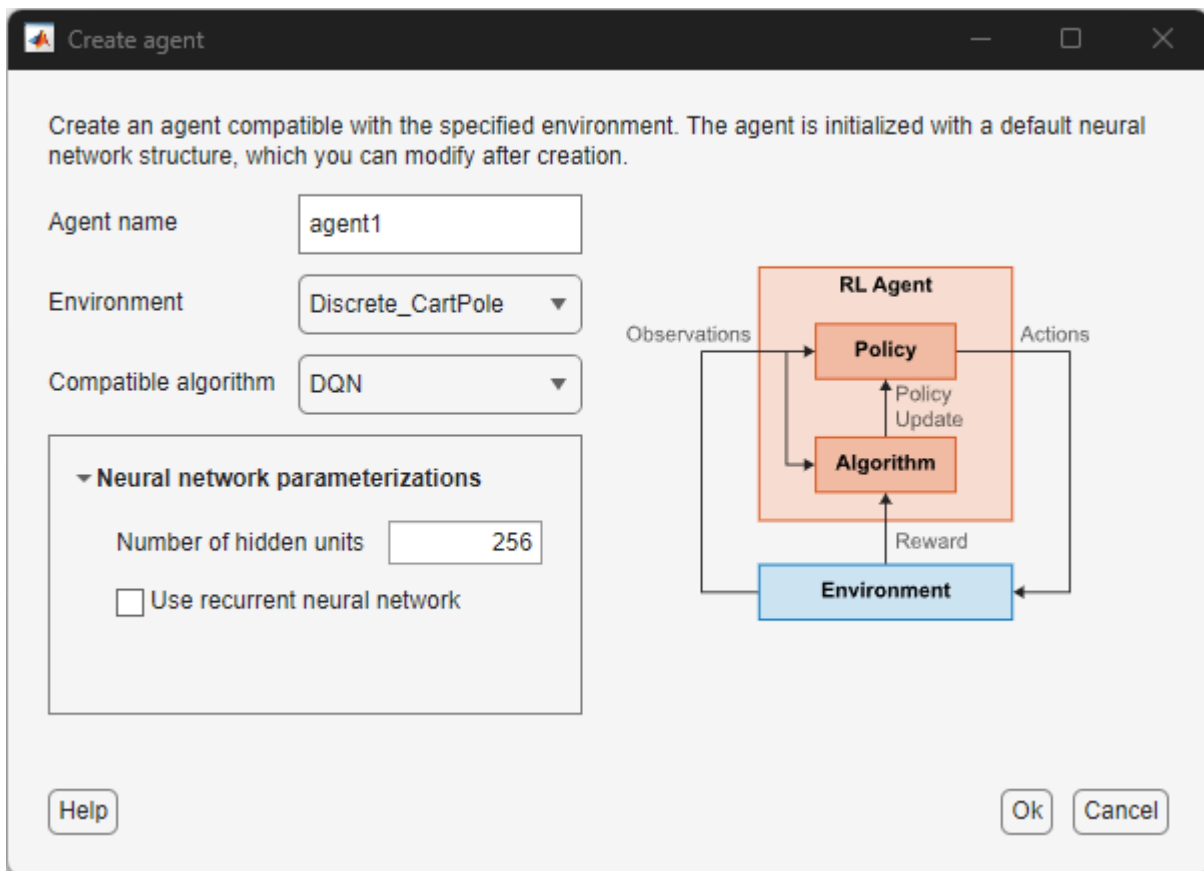
To create a discrete action space cart-pole environment:

- 1 Navigate to the **Reinforcement Learning** tab and select **New**, in the **Environment** section.
- 2 Select **Discrete Cart Pole** from the list to create the environment in the app workspace.



To create a DQN agent:

- Select **New**, in the **Agent** section, to open the **Create agent** dialog box.
- Select **DQN** as a compatible agent.
- Press **Ok** to create the agent in the app workspace and to open the agent editor document.



Alternatively, select **Import** from the **Reinforcement Learning** tab to import your reinforcement learning environment and agent from the MATLAB workspace.

Tune Hyperparameters

You can tune the hyperparameters of your reinforcement learning agent with an exhaustive grid-based search approach or with a Bayesian optimization adaptive search approach. The Bayesian optimization approach requires the Statistics and Machine Learning Toolbox™. For detailed information about these approaches, see these examples:

- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240
- “Tune Hyperparameters Using Bayesian Optimization” on page 5-250

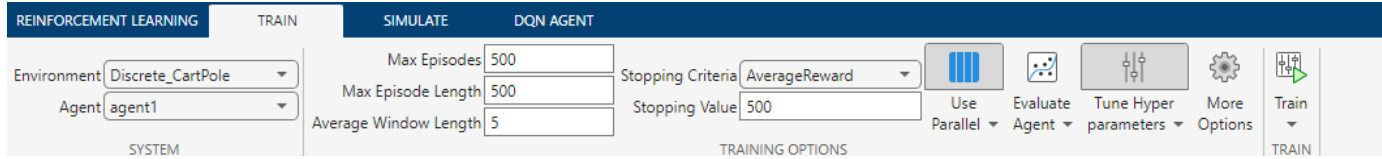
The Reinforcement Learning Designer app automates several steps in these examples. You can tune hyperparameters with default settings or configure the settings before tuning.

In this section, you tune hyperparameters using default hyperparameter tuning settings. To configure hyperparameter tuning settings instead, see the Configure Tuning Options on page 5-58 section.

To tune using default hyperparameter settings, follow these steps:

1. Navigate to the **Train** tab and specify the appropriate values for training options. For this example, only set **Stopping Criteria** to **Average Reward**.
2. Select **Tune Hyperparameters** to enable hyperparameter tuning.

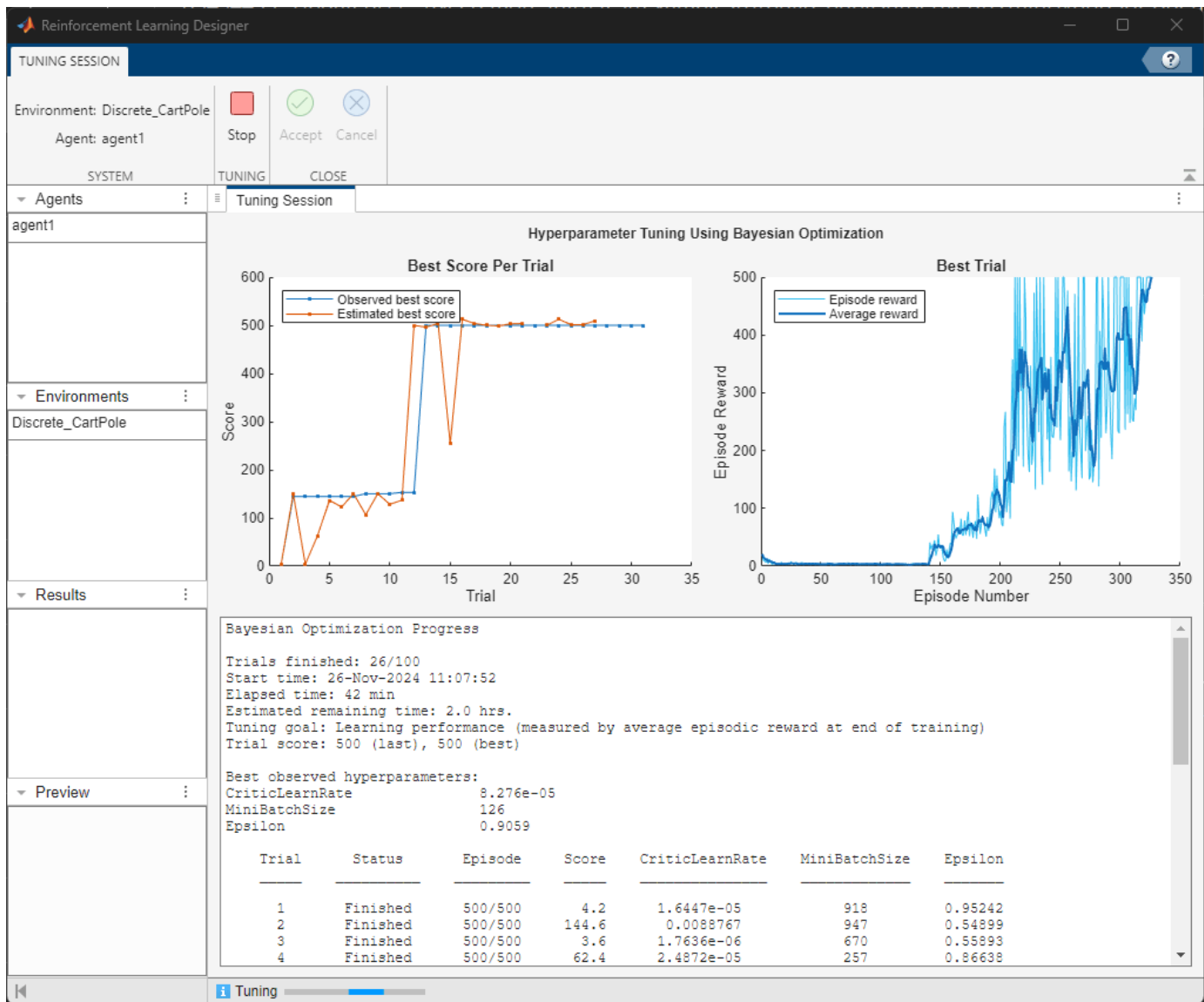
- 3 If you have Parallel Computing Toolbox™ installed, select **Use Parallel** to speed up the tuning process using parallel workers. This step is optional.
- 4 Click **Train** to start tuning.



View the progress in the **Tuning Session** progress window:

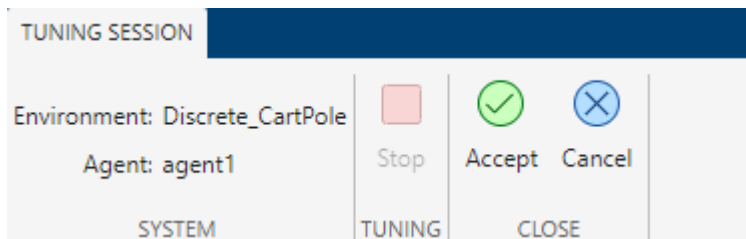
- The **Best Score Per Trial** plot shows the best observed scores (for this example, the average episodic reward) at the end of each trial. A trial is a training experiment for a particular combination of hyperparameters. Bayesian optimization uses a Gaussian process model to estimate the trial scores. This plot also shows the scores estimated by the model.
- The **Best Trial** plot displays the episodic rewards from the best performing trial.
- The verbose output panel displays the detailed tuning progress.
- The trained agent and training result from each trial is saved in the **Trials** folder in the current directory.
- The tuning stops after the software runs the maximum number of trials. To manually stop the tuning at any time, select **Stop** from the **Tuning Session** tab. The software saves the best performing agent and tuning result in the app workspace.

The figure shows a snapshot of tuning progress using Bayesian optimization.



To validate the performance of the agent, follow these steps after the software finishes the tuning process:

- 1 Select **Accept** to store the optimized agent and tuning results in the app workspace.
- 2 Open the agent **Agent1_Tuned1** from the **Agents** panel and view the optimized hyperparameters.
- 3 Open **tuningResult1** from the **Results** panel to view details of the tuning experiment.
- 4 Validate the performance of the agent from the **Simulate** tab. For more information, see “Simulate Agent and Inspect Simulation Results” on page 5-0 .



▼ Agents	:
agent1	
agent1_Tuned1	
▼ Environments	:
Discrete_CartPole	
▼ Results	:
tuningResult1	

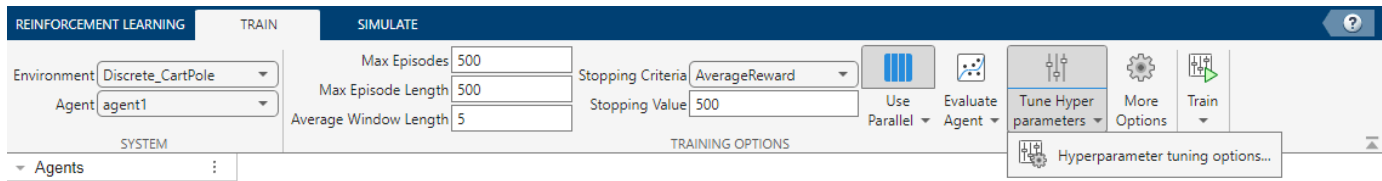
Note:

- The tuning process is computationally expensive and typically takes many hours to complete.
- Due to randomness in the training process, your results might vary from the results shown in this example.
- The algorithm automatically selects hyperparameters to tune and configures the search space appropriately.
- By default, the software uses the “Bayesian Optimization Algorithm” (Statistics and Machine Learning Toolbox) to search for the optimal hyperparameter set with the maximum average episodic reward at the end of training. Using this algorithm requires the Statistics and Machine Learning Toolbox. If you do not have the toolbox, the algorithm uses an exhaustive grid-based search approach for optimizing the hyperparameters.

Configure Tuning Options

The default tuning settings might be sufficient for optimizing hyperparameters. However, you can configure the tuning settings to have more control over the tuning process. Follow these steps to configure the tuning settings:

- 1 From the **Train** tab, in the **Tune Hyperparameters** list, select **Hyperparameter tuning options** to display the **Tuning Options** dialog box.



- 1 In the dialog box, under **Tuning algorithm**, select either **Bayesian optimization** or **Grid search**. Bayesian optimization performs an adaptive search on the hyperparameters space by updating a Gaussian process model. Grid search performs an exhaustive sweep over the hyperparameter space.
- 2 Under **Hyperparameter selection**, select or clear the check boxes in the **Optimize** column to configure which hyperparameters to optimize. For Bayesian optimization, configure the **Min** and **Max** values of the search space and enable or disable log scaling. For grid search, specify an expression for the hyperparameter search space. For example, to discretize the search space of the critic learning rate from $1e-3$ to $1e-1$ with 5 points and log space scaling, specify `logspace(-3, -1, 5)` in the **Values** column. The expression must evaluate to a numeric array.
- 3 Under **Tuning goal**, select **Learning performance** or **Policy performance**. If you select **Learning performance**, the score of the trial is the average episodic reward from the final episode of the training. If you select **Policy performance**, the score of the trial is the maximum agent evaluation statistic value over all training episodes. The evaluation statistic is the mean episodic reward from 5 simulations periodically computed by an agent evaluator during the training.
- 4 Under **Number of trials**, specify the maximum number of trials to perform.
- 5 Under **Max time**, specify the maximum number of hours for the tuning experiment.
- 6 Under **Random seed**, specify a positive integer to control the random seed at the beginning of each trial.
- 7 To save tuning artifacts (that is, the trained agent and training result from each trial), select **Save artifacts to disk**. Specify the folder name under **Save directory**.
- 8 Select **Ok** to confirm your choices or **Cancel** to discard your changes.

These figures show the default settings for Bayesian optimization and grid search configurations.

Tuning Options

☒ Enable hyperparameter tuning

Tuning algorithm: Bayesian optimization

Hyperparameter selection

	Optimize	Min	Max	LogScaling
CriticLearnRate	☒	1e-06	0.1	☒
MiniBatchSize	☒	64	1024	☐
Epsilon	☒	0.5	1	☐
EpsilonDecay	☐	1e-06	0.01	☒
DiscountFactor	☐	0.1	1	☒

Restore Defaults

Tuning goal: Learning performance Random seed: 0

Number of trials: 100 ☒ Save artifacts to disk

Max time (hours): Inf Save directory: Trials

Help Ok Cancel

Tuning Options

☒ Enable hyperparameter tuning

Tuning algorithm: Grid search

Hyperparameter selection

	Optimize	Values
CriticLearnRate	☒	logspace(-3,-1,5)
MiniBatchSize	☒	64*2.^(0:4)
Epsilon	☐	linspace(0.5,1,0,5)
EpsilonDecay	☐	logspace(-6,-2,5)
DiscountFactor	☐	logspace(-1,0,5)

Restore Defaults

Tuning goal: Learning performance Random seed: 0

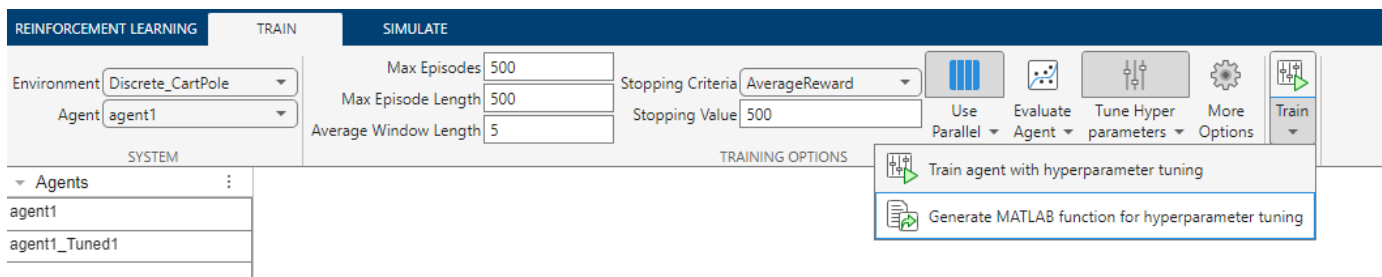
For more information about hyperparameter tuning options, see “Specify Hyperparameter Tuning Options” on page 5-35.

Generate Code

You can generate MATLAB code for the hyperparameter tuning process. The generated code can serve as a starting point to further customize the hyperparameter tuning process.

To generate code:

- Optionally configure the hyperparameter tuning options using the instructions from the section above.
- From the **Train** tab, select the environment and agent.
- Select **Tune Hyperparameters** to enable tuning.
- From the **Train** dropdown button list select **Generate MATLAB function for hyperparameter tuning**.



Here is a sample of the generated code.

```
function optimResults = runBayesoptTuningExperiment(agent,env)
% Run a Bayesian hyperparameter tuning experiment.

% Reinforcement Learning Toolbox
% Generated on: 26-Nov-2024 13:18:47

arguments
    agent rl.agent.rlDQNAgent
    env rl.env.CartPoleDiscreteAction
end

% Define training options.
% Here you can specify the options to train
% your reinforcement learning agent.
topts = rlTrainingOptions( ...
    MaxEpisodes=500, ...
    MaxStepsPerEpisode=500, ...
    Verbose=false, ...
    Plots="none", ...
    SimulationStorageType="none", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=500, ...
    StopOnError="on");

% Specify the hyperparameters to tune here.
% For each hyperparameter, create an optimizableVariable object
% with the appropriate range, transform and type,
```

```
% and store it in the optimvars variable.
% Then, in the objectiveFunction function at the end of this script,
% configure the agent with the new hyperparameters.

% A few hyperparameters are already configured for reference.
optimvars(1) = optimizableVariable('CriticLearnRate',[1e-06 0.1], ...
    Transform='log',Type='real');
optimvars(2) = optimizableVariable('MiniBatchSize',[64 1024], ...
    Transform='none',Type='integer');
optimvars(3) = optimizableVariable('Epsilon',[0.5 1], ...
    Transform='none',Type='real');

% Run a Bayesian optimization experiment.
objfun = @(params) objectiveFunction(params,agent,env,topts);
optimResults = bayesopt(objfun,optimvars, ...
    MaxObjectiveEvaluations=100, ...
    MaxTime=Inf, ...
    Verbose=1, ...
    UseParallel=true);

end

%% Define local functions.
function [objective,constraint,userData] = ...
    objectiveFunction(params,agent,env,topts)
% Hyperparameter tuning objective function.

% Fix random seed for reproducibility.
rng(0,"twister");

% Copy the agent so that the trials start with the same learnables.
ag = copy(agent);

% Apply here the previously defined hyperparameters.
% The params structure contains one combination of
% the previously defined hyperparameters.
for i = 1:numel(ag.AgentOptions.CriticOptimizerOptions)
    ag.AgentOptions.CriticOptimizerOptions(i).LearnRate = ...
        params.CriticLearnRate;
end
ag.AgentOptions.MiniBatchSize = params.MiniBatchSize;
ag.AgentOptions.EpsilonGreedyExploration.Epsilon = params.Epsilon;

% Run the training.
result = train(ag,env,topts);

% Compute the objective value.
objective = -result.AverageReward(end);

% Define constraint.
constraint = [];

% Store the trial result.
userData.Agent = ag;
userData.TrainingResult = result;

end
```

Conclusion

The example demonstrates how you can tune agent hyperparameters using Reinforcement Learning Designer.

See Also

Apps

Reinforcement Learning Designer

Functions

`train`

Objects

`rlTrainingOptions` | `rlEvaluator`

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Tune Hyperparameters Using Bayesian Optimization” on page 5-250
- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240
- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64

More About

- “Load MATLAB Environments in Reinforcement Learning Designer” on page 2-138
- “Load Simulink Environments in Reinforcement Learning Designer” on page 2-142
- “Specify Training Options in Reinforcement Learning Designer” on page 5-29
- “Specify Simulation Options in Reinforcement Learning Designer” on page 5-50
- “Create Agents Using Reinforcement Learning Designer” on page 3-11
- “Train Reinforcement Learning Agents” on page 5-3

Train Default DQN Agent to Balance Discrete Cart-Pole

This example shows how to create and train a default deep Q-learning network (DQN) agent to balance a discrete action space cart-pole system modeled in MATLAB®.

For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32. For an example that shows how to train a default DQN agent in a Simulink® environment, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

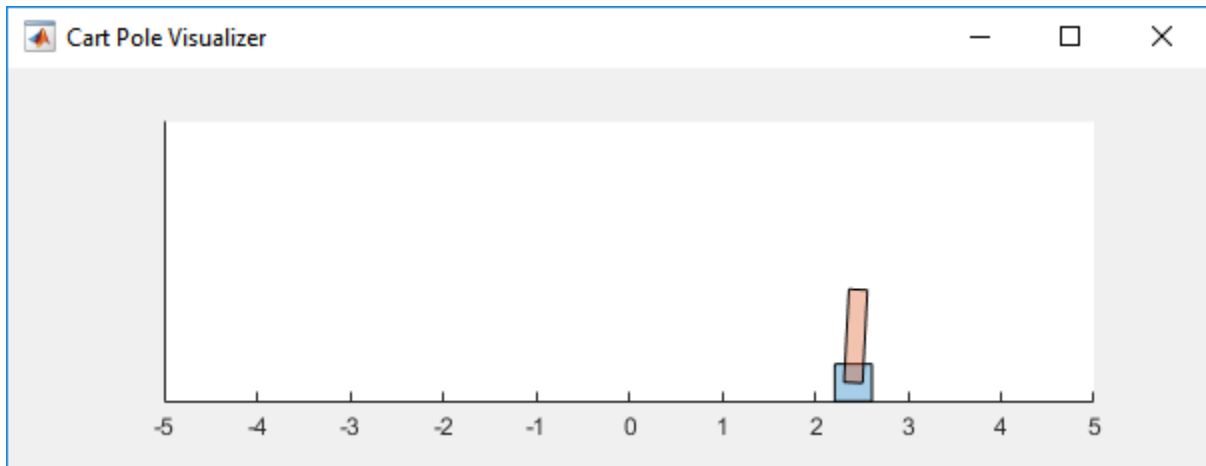
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Cart-Pole MATLAB Environment

The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to make the pole stand upright.



For this environment:

- The upright pole angle is zero radians. Initially, the pole hangs downward (with an angle of π radians) without moving.
- The pole starts upright with an initial angle between -0.05 and 0.05 radians.
- The force action signal from the agent to the environment is either -10 or 10 N.
- The observations from the environment are the position and velocity of the cart, the pole angle, and the pole angle derivative.

- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of +1 is provided for every time step that the pole remains upright. A penalty of -5 is applied when the pole falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined cart-pole environment object.

```
env = rlPredefinedEnv("CartPole-Discrete")
```

```
env =
  CartPoleDiscreteAction with properties:
        Gravity: 9.8000
        MassCart: 1
        MassPole: 0.1000
        Length: 0.5000
        MaxForce: 10
        Ts: 0.0200
        ThetaThresholdRadians: 0.2094
        XThreshold: 2.4000
        RewardForNotFalling: 1
        PenaltyForFalling: -5
        State: [4x1 double]
```

The object has a discrete action space where the agent can apply one of two possible force values to the cart, -10 or 10 N.

Get the observation and action specification information.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
        LowerLimit: -Inf
        UpperLimit: Inf
        Name: "CartPole States"
        Description: "x, dx, theta, dtheta"
        Dimension: [4 1]
        DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlFiniteSetSpec with properties:
        Elements: [-10 10]
        Name: "CartPole Action"
        Description: [0x0 string]
        Dimension: [1 1]
```

```
DataType: "double"
```

Create Default DQN Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Create an agent initialization object to initialize the critic network with the hidden layer size 20.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=20);
```

Specify the agent options for training using `rlDQNAgentOptions` and `rlOptimizerOptions` objects. For this training:

- Use mini-batches of 256 experiences. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Update the target critic network every 4 learning iterations with a smoothing factor of 1.
- Do not use the double-DQN algorithm.

```
agentOpts = rlDQNAgentOptions( ...  
    MiniBatchSize      = 256, ...  
    TargetSmoothFactor  = 1, ...  
    TargetUpdateFrequency = 4, ...  
    UseDoubleDQN       = false);
```

A DQN agent uses the epsilon-greedy algorithm to explore the action space during training. Specify a decay rate of 1e-3 for the epsilon value to gradually decay during training. This promotes exploration toward the beginning when the agent does not have a good policy, and exploitation toward the end when the agent has learned the optimal policy.

```
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-3;
```

Create the DQN agent using the observation and action input specifications, initialization options and agent options. When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
agent = rlDQNAgent(obsInfo, actInfo, initOpts, agentOpts);
```

For more information, see `rlDQNAgent`.

Check the agent's actions with a random observation input.

```
getAction(agent, {rand(obsInfo.Dimension)})  
  
ans = 1x1 cell array  
    {[10]}
```

Train Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To train the agent, first specify the training options. For this example, use the following options:

- Run the training for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor window (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the performance of the greedy policy every 20 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score reaches 500. At this point, the agent can balance the cart-pole system in the upright position.

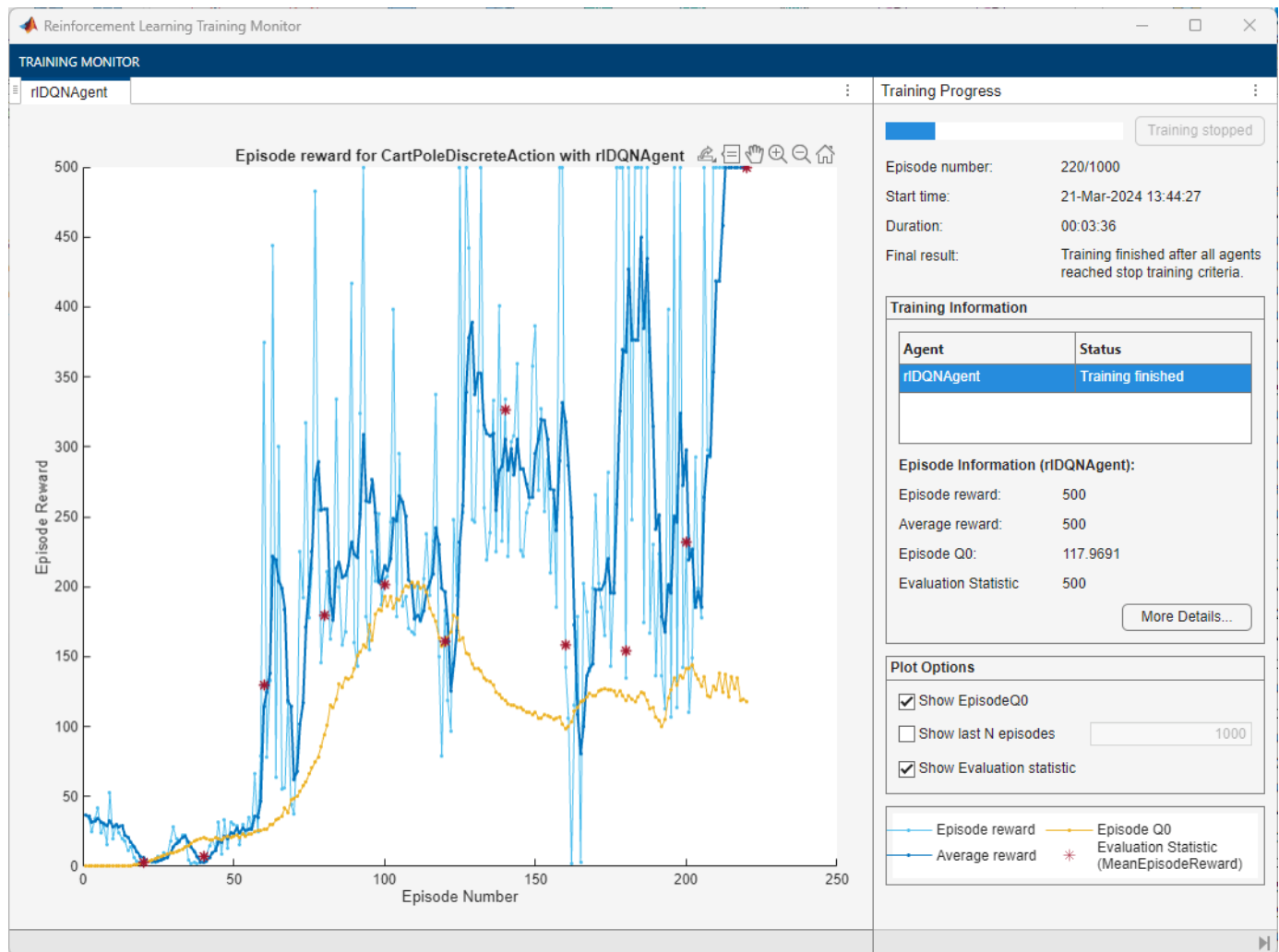
```
% training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=500, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=500);

% agent evaluator
evl = rlEvaluator(EvaluationFrequency=20, NumEpisodes=5);
```

For more information on training options, see `rlTrainingOptions` and `rlEvaluator`.

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("MATLABCartpoleDQNMuti.mat","agent")
end
```



Simulate Agent

Fix the random stream for reproducibility.

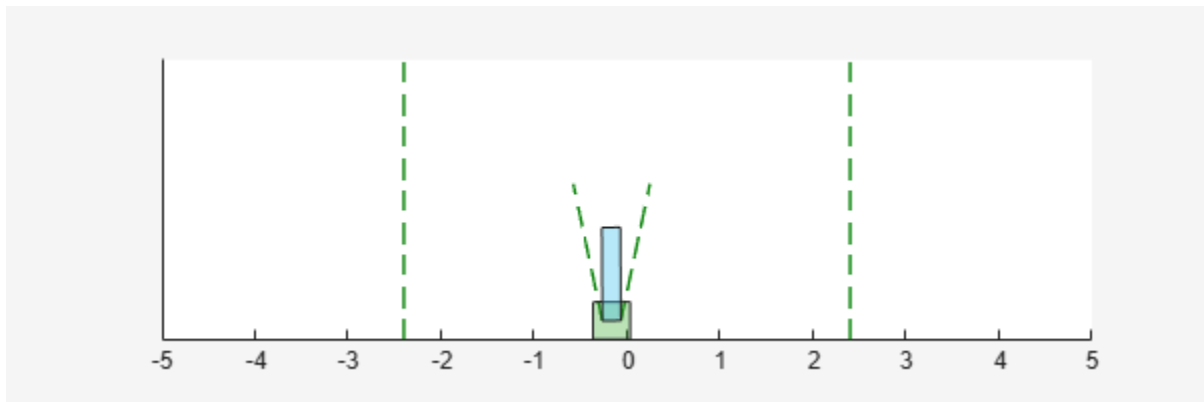
```
rng(0, "twister");
```

You can visualize the cart-pole system by using the `plot` function.

```
plot(env)
```

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```

```
totalReward = sum(experience.Reward)
```

```
totalReward =  
500
```

The agent can balance the cart-pole system.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions` | `rlEvaluator`

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Train Reinforcement Learning Agents” on page 5-3

More About

- “Use Predefined Control System Environments” on page 2-42
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3

Train Default DQN Agent to Swing Up and Balance Discrete Pendulum

This example shows how to train a deep Q-learning network (DQN) agent to swing up and balance a pendulum modeled in Simulink®.

For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32. For an example that shows how to train a default DQN agent in a MATLAB® environment, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

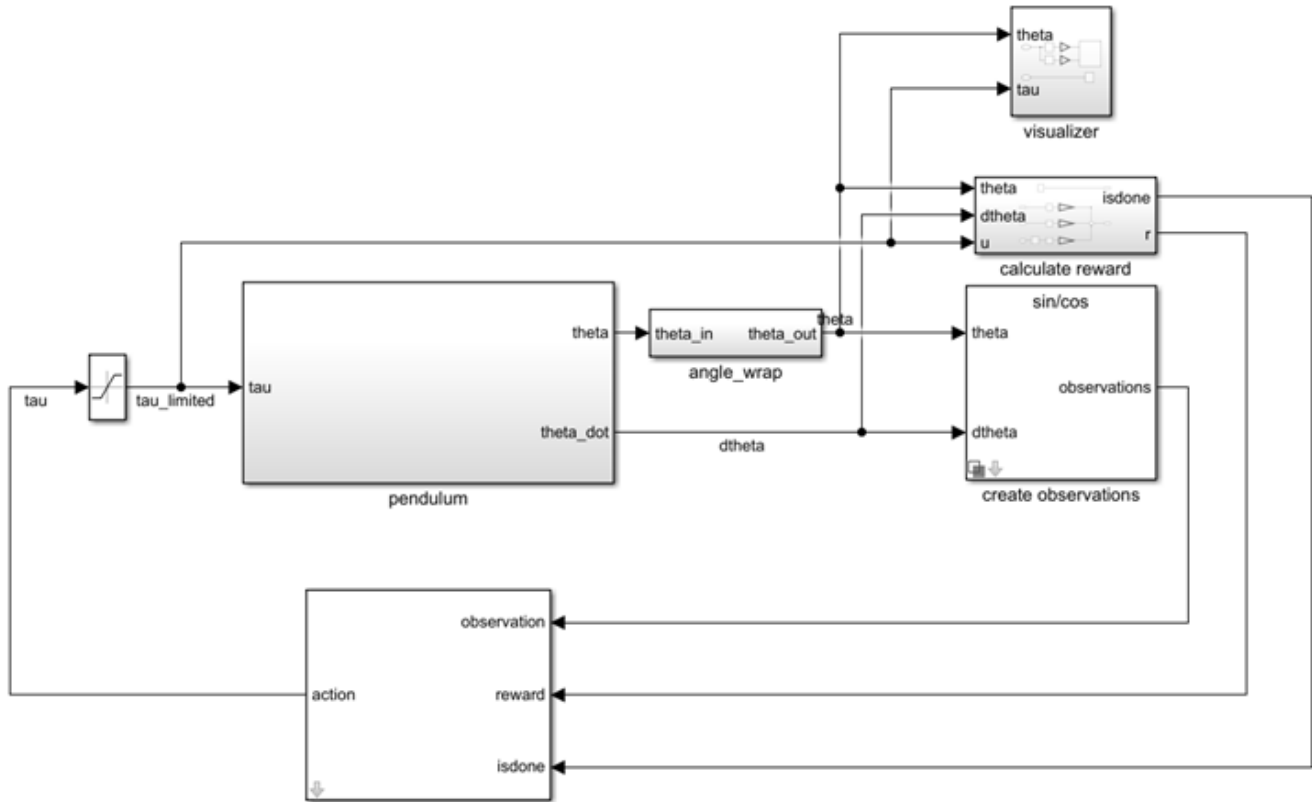
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Pendulum Swing-Up Environment Model

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

Open the model.

```
mdl = "rlSimplePendulumModel";  
open_system(mdl)
```



In this model:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment is from -2 to 2 N·m.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("SimplePendulumModel-Discrete")
```

```
env =  
SimulinkEnvWithAgent with properties:
```

```
    Model : rlSimplePendulumModel  
    AgentBlock : rlSimplePendulumModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

The object has a discrete action space where the agent can apply one of three possible torque values to the pendulum: -2, 0, or 2 N·m.

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function uses `setVariable` (Simulink) set the model workspace variable `theta0` to `pi`. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in)setVariable(in,"theta0",pi,"Workspace",mdl);
```

Get the observation and action specification information from the environment.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
rlNumericSpec with properties:
```

```
    LowerLimit: -Inf  
    UpperLimit: Inf  
    Name: "observations"  
    Description: [0×0 string]  
    Dimension: [3 1]  
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =  
rlFiniteSetSpec with properties:
```

```
    Elements: [3×1 double]  
    Name: "torque"  
    Description: [0×0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;  
Tf = 20;
```

Create Default DQN Agent

When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Create an agent initialization object to initialize the critic network with the hidden layer size 256.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=256);
```

Specify the agent options for training using `rlDQNAgentOptions` and `rlOptimizerOptions` objects. For this training:

- Specify the critic learning rate to be $5e-3$. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Use a gradient threshold of 5 to clip the gradients. Clipping the gradients can improve training stability.
- Use mini-batches of 256 experiences. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Update the target critic network every 4 learning iterations with a smoothing factor of $5e-3$.
- Do not use the double-DQN algorithm.

```
optimOpts = rlOptimizerOptions(LearnRate=5e-3, GradientThreshold=5);
agentOptions = rlDQNAgentOptions( ...
    SampleTime=Ts, ...
    CriticOptimizerOptions=optimOpts, ...
    ExperienceBufferLength=1e5, ...
    TargetSmoothFactor=5e-3, ...
    UseDoubleDQN=false, ...
    MiniBatchSize=256);
```

A DQN agent uses the epsilon-greedy algorithm to explore the action space during training. Specify a decay rate of $5e-5$ for the epsilon value to gradually decay during training until the minimum value of 0.1 is reached. This promotes exploration toward the beginning when the agent does not have a good policy, and exploitation toward the end when the agent has learned the optimal policy.

```
agentOptions.EpsilonGreedyExploration.EpsilonDecay = 5e-5;
agentOptions.EpsilonGreedyExploration.EpsilonMin = 0.1;
```

Create the DQN agent using the observation and action input specifications, initialization options and agent options.

```
agent = rlDQNAgent(obsInfo, actInfo, initOpts, agentOptions);
```

For more information, see `rlDQNAgent`.

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run each training for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the performance of the greedy policy every 10 training episodes.

- Stop the training when the agent receives an average cumulative reward greater than -1100 when evaluating the deterministic policy. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.
- Save a copy of the agent for each episode where the cumulative reward is greater than -1100.

For more information on training options, see `rlTrainingOptions`.

```
% training options
trainingOptions = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=500, ...
    ScoreAveragingWindowLength=5, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-1100, ...
    SaveAgentCriteria="EvaluationStatistic", ...
    SaveAgentValue=-1100);

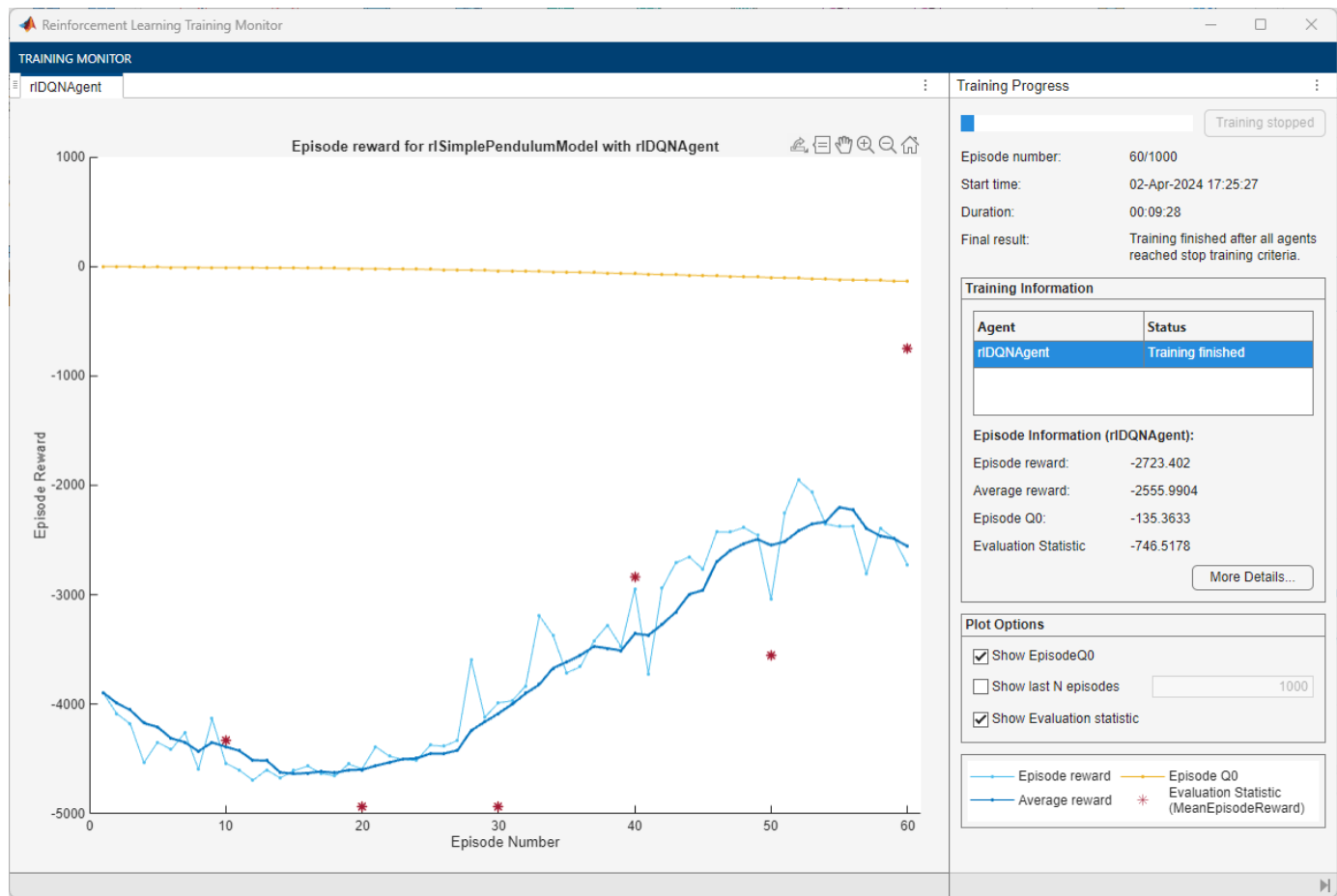
% agent evaluator
evl = rlEvaluator(EvaluationFrequency=10, NumEpisodes=1);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent, env, trainingOptions, ...
        Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("SimulinkPendulumDQNMulti.mat", "agent");
end
```



Simulate DQN Agent

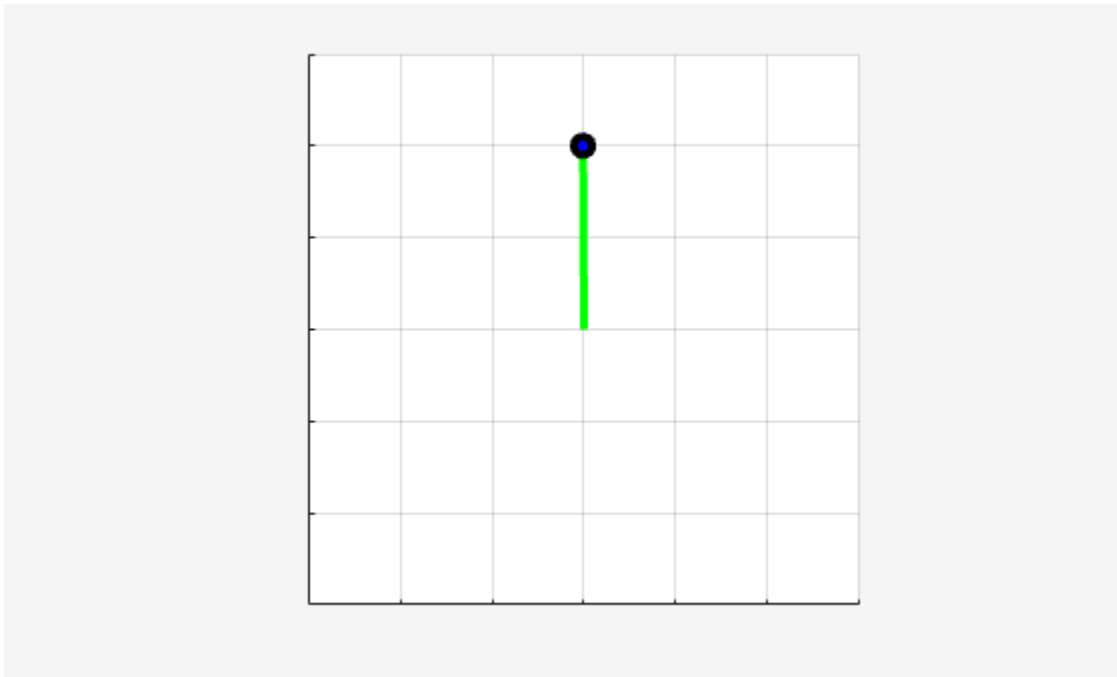
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To validate the performance of the trained agent, simulate it within the pendulum environment. For more information on agent simulation, see `rLSimulationOptions` and `sim`.

```
simOptions = rLSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```



Display the total reward.

```
totalRwd = sum(experience.Reward)
```

```
totalRwd =  
-746.5178
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlVectorQValueFunction` | `rlTrainingOptions` | `rlSimulationOptions` | `rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal” on page 5-141

- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217

More About

- “Use Predefined Control System Environments” on page 2-42
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Create Custom Simulink Environments” on page 2-110
- “Train Reinforcement Learning Agents” on page 5-3

Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum

This example shows how to train a deep deterministic policy gradient (DDPG) agent to swing up and balance a pendulum modeled in Simulink®.

For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67. For an example that shows how to train a DDPG agent in a MATLAB® environment, see “Compare DDPG Agent to LQR Controller” on page 7-285.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

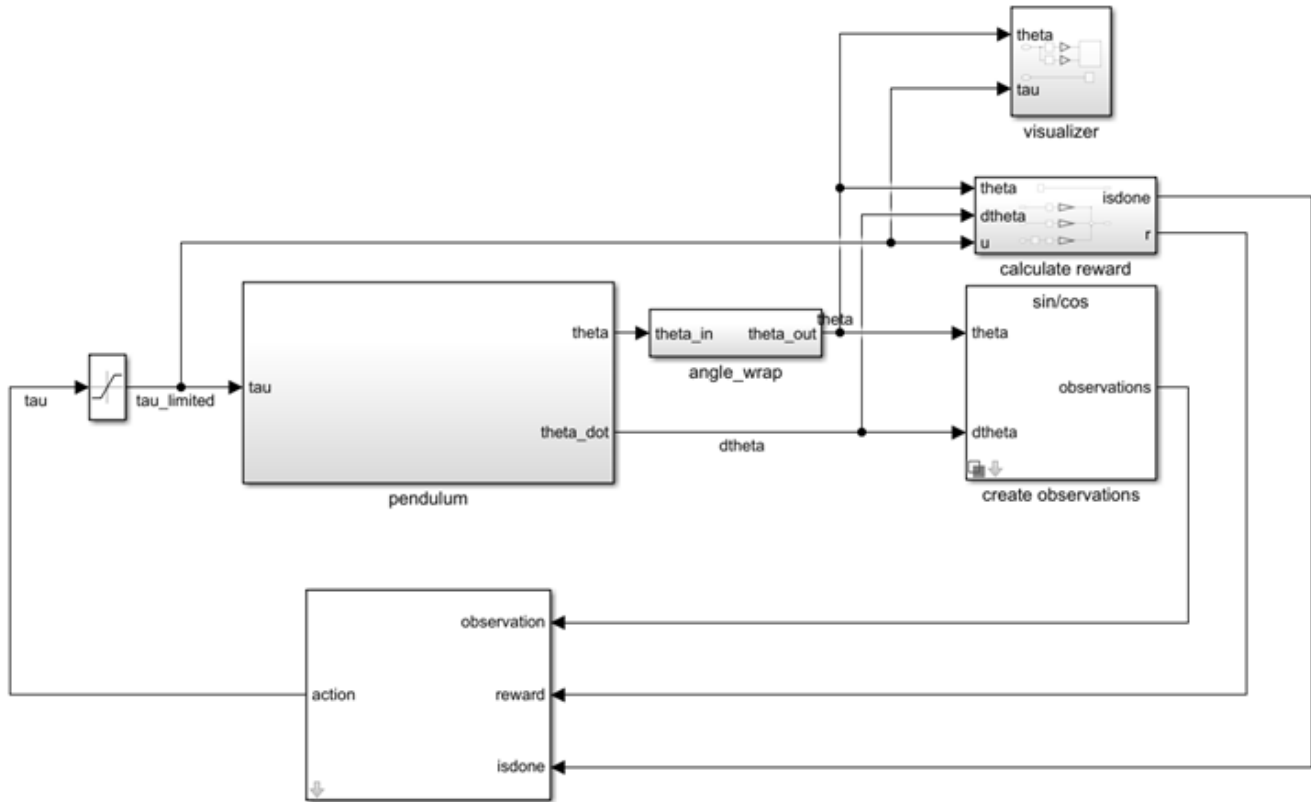
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Pendulum Swing-Up Environment Model

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

Open the model.

```
mdl = "rlSimplePendulumModel";  
open_system(mdl)
```



In this model:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment is from -2 to 2 N·m.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("SimplePendulumModel-Continuous")
```

```
env =  
SimulinkEnvWithAgent with properties:
```

```
    Model : rlSimplePendulumModel  
    AgentBlock : rlSimplePendulumModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

```
obsInfo = getObservationInfo(env);
```

The object has a continuous action space where the agent can apply torque values between -2 to 2 N·m to the pendulum.

```
actInfo = getActionInfo(env)
```

```
actInfo =  
rlNumericSpec with properties:
```

```
    LowerLimit: -2  
    UpperLimit: 2  
    Name: "torque"  
    Description: [0×0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

Set the observations of the environment to be the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.

```
set_param( ...  
    "rlSimplePendulumModel/create_observations", ...  
    "ThetaObservationHandling","sincos");
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function uses the `setVariable` (Simulink) function to set the model workspace variable `theta0` to `pi`. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in)setVariable(in,"theta0",pi,"Workspace",mdl);
```

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;  
Tf = 20;
```

Create Default DDPG Agent

When you create the agent, the actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlDDPGAgent` object using the environment specification objects.

```
agent = rlDDPGAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of the agent.

```
agent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Increase the length of the experience buffer and the size of the mini buffer.

```
agent.AgentOptions.ExperienceBufferLength = 1e5;
agent.AgentOptions.MiniBatchSize = 128;
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run training for a maximum of 5000 episodes, with each episode lasting a maximum of `ceil(Tf/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the agent receives an average cumulative reward greater than -740 when evaluating the deterministic policy. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.
- Save a copy of the agent for each episode where the cumulative reward is greater than -740.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 5000;
maxsteps = ceil(Tf/Ts);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    ScoreAveragingWindowLength=5, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-740, ...
    SaveAgentCriteria="EvaluationStatistic", ...
    SaveAgentValue=-740);
```

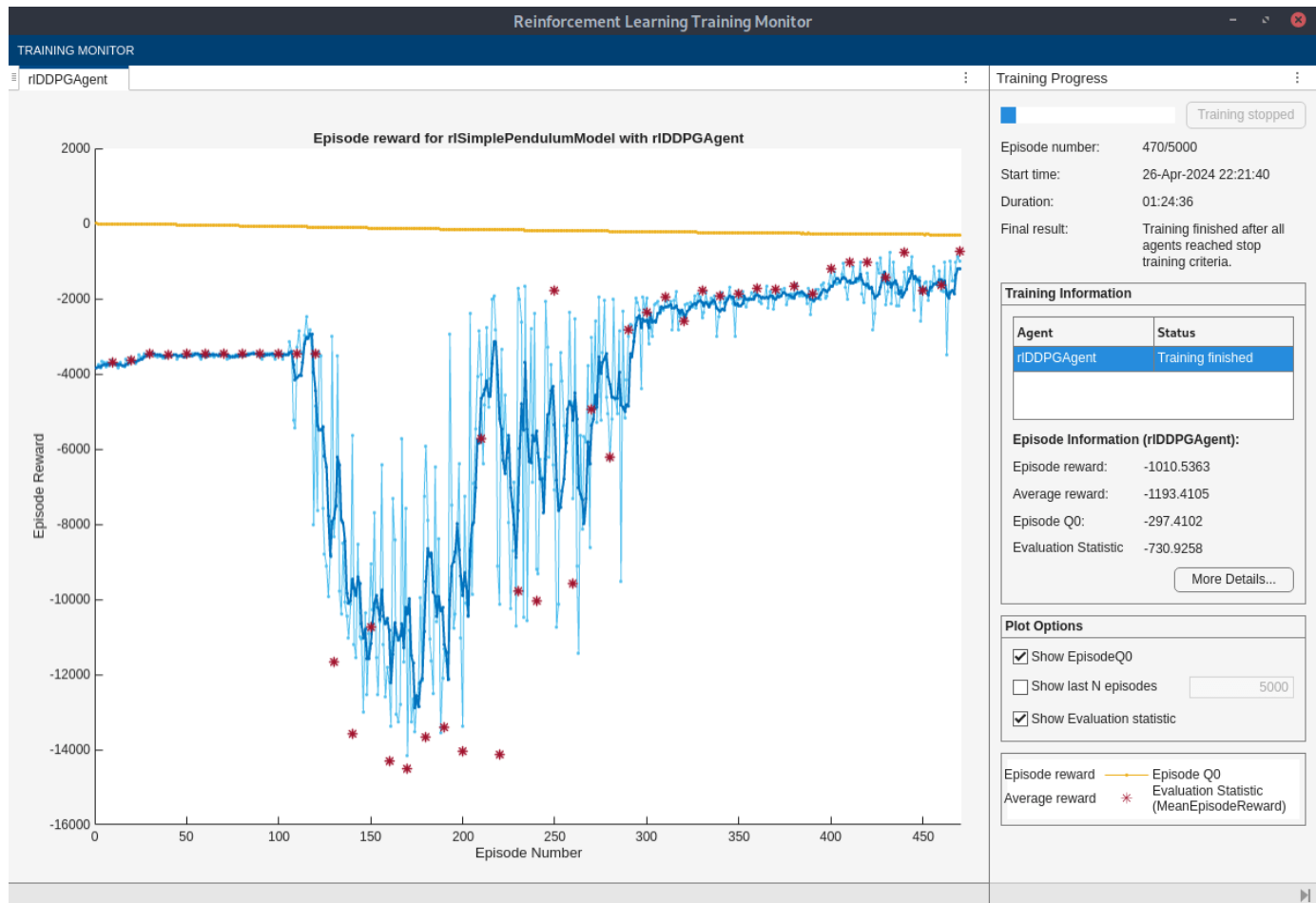
Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Use the rlEvaluator object to measure policy performance every 10
    % episodes
    evaluator = rlEvaluator( ...
```

```

        NumEpisodes=1, ...
        EvaluationFrequency=10);
    % Train the agent.
    trainingResults = train(agent,env,trainOpts,Evaluator=evaluator);
else
    % Load the pretrained agent for the example.
    load("SimulinkPendulumDDPG.mat","agent")
end

```



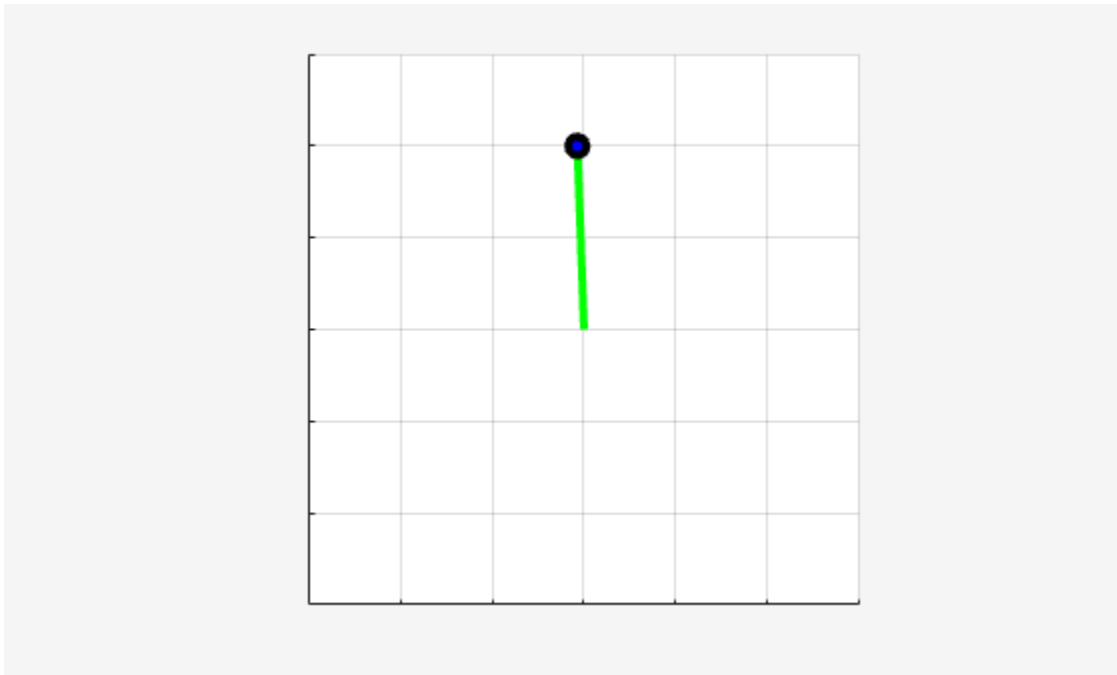
Simulate DDPG Agent

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

To validate the performance of the trained agent, simulate it within the pendulum environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
```



```
totalReward = sum(experience.Reward)
```

```
totalReward =  
-731.2350
```

The simulation shows that the agent stabilizes the pendulum in the vertical position.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85
- “Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal” on page 5-141
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122

More About

- “Create Custom Simulink Environments” on page 2-110
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole

This example shows how to train a deep deterministic policy gradient (DDPG) agent to swing up and balance a cart-pole system with continuous action space modeled in Simscape™ Multibody™.

For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67. For an example that shows how to train a DDPG agent in a MATLAB® environment, see “Compare DDPG Agent to LQR Controller” on page 7-285.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

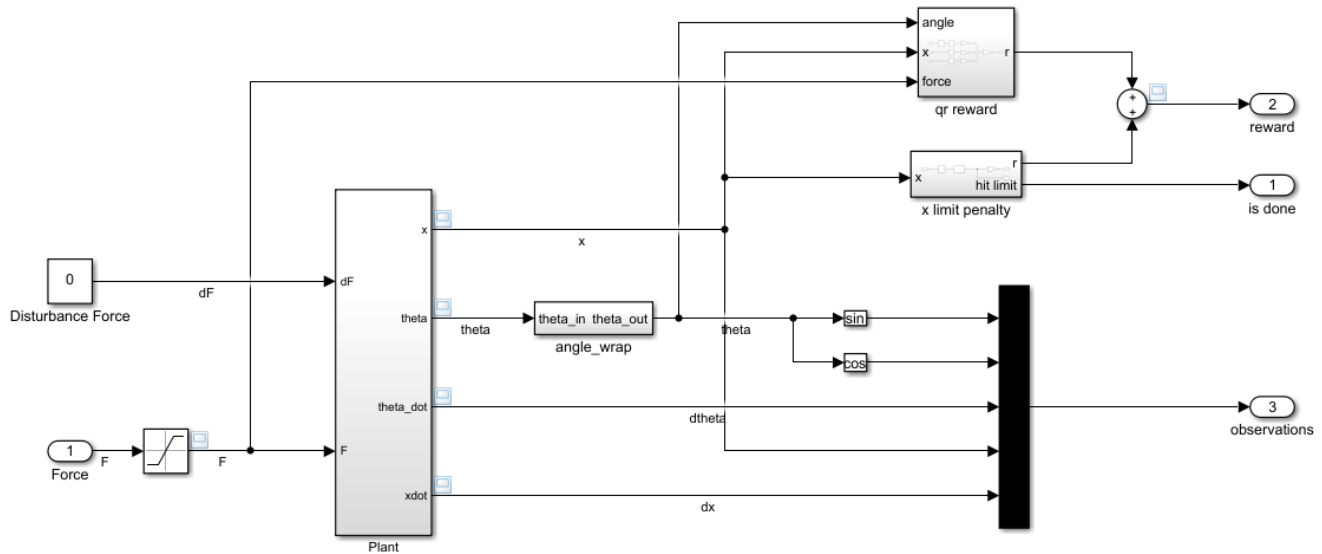
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Cart-Pole Simscape Environment Model

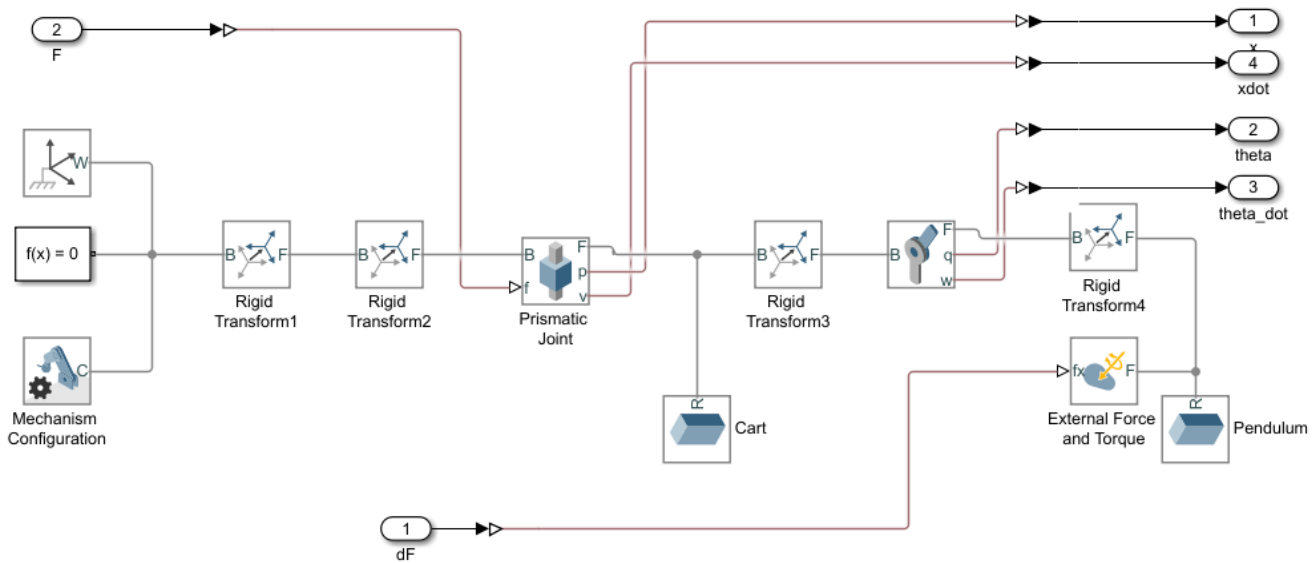
The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to make the pole stand upright using minimal control effort.

Open the model.

```
mdl = "rlCartPoleSimscapeModel";  
open_system(mdl)
```



The cart-pole system is modeled using Simscape Multibody.



In this model:

- The upright pole angle is θ radians, and the downward hanging pendulum position is π radians.
- The force (rightward-positive) action signal from the agent to the environment is from -15 to 15 N.
- The observations from the environment are the position and velocity of the cart, and the sine, cosine, and derivative of the pole angle.
- The episode terminates if the cart moves more than 3.5 m from the original position.

- The reward r_t , provided at every time step, is

$$r_t = -0.1(5\theta_t^2 + x_t^2 + 0.05u_{t-1}^2) - 100B$$

Here:

- θ_t is the angle of displacement from the upright position of the pole (counterclockwise-positive).
- x_t is the position displacement from the center position of the cart (rightward-positive).
- u_{t-1} is the control effort from the previous time step.
- B is a flag (1 or 0) that indicates whether the cart is out of bounds.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pole.

```
env = rlPredefinedEnv("CartPoleSimscapeModel-Continuous")
```

```
env =  
SimulinkEnvWithAgent with properties:
```

```
    Model : rlCartPoleSimscapeModel  
    AgentBlock : rlCartPoleSimscapeModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

The environment has a continuous action space where the agent can apply a horizontal force (from -15 to 15 N) to the cart.

Obtain the observation and action information from the environment object.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

Specify the agent sample time T_s and the simulation time T_f in seconds.

```
 $T_s$  = 0.02;  
 $T_f$  = 25;
```

Create Default DDPG Agent

You will create and train a DDPG agent in this example. The agent uses:

- A Q-value function critic to estimate the value of the policy. The critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated discounted cumulative long-term reward given the action from the state corresponding to the current observation, and following the policy thereafter).
- A deterministic policy over a continuous action space, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

The actor and critic functions are approximated using neural network representations. Create an agent initialization object to initialize the networks with the hidden layer size 200.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=200);
```

Configure the hyperparameters for training using `rlDDPGAgentOptions`.

- Specify the actor and critic learning rates as $5e-3$. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Specify a gradient threshold value of 1. Clipping the gradient improves learning stability.
- Specify the experience buffer capacity to be $1e5$. A large capacity enables storing a diverse set of experiences.
- Specify a smoothing factor of $5e-3$ to update the target critic network.
- Specify the sample time of the agent as T_s .
- Learn with a maximum of 200 mini batches per training epoch.

```
criticOptions = rlOptimizerOptions( ...  
    LearnRate=5e-03, ...  
    GradientThreshold=1);  
actorOptions = rlOptimizerOptions( ...  
    LearnRate=5e-03, ...  
    GradientThreshold=1);  
agentOptions = rlDDPGAgentOptions( ...  
    SampleTime=Ts, ...  
    ActorOptimizerOptions=actorOptions, ...  
    CriticOptimizerOptions=criticOptions, ...  
    ExperienceBufferLength=1e5, ...  
    TargetSmoothFactor=5e-3, ...  
    MaxMiniBatchPerEpoch=200);
```

Alternatively, you can also create the agent first, and then access its option object and modify the options using dot notation.

The DDPG agent in this example uses an Ornstein-Uhlenbeck (OU) noise model for exploration. Specify the following noise options using dot notation:

- Specify a mean attraction value of $0.1/T_s$. A larger mean attraction value keeps the noise values close to the mean.
- Specify a standard deviation value of $1/\sqrt{T_s}$ to improve exploration during training.

```
agentOptions.NoiseOptions.StandardDeviation = 1.0/sqrt(Ts);  
agentOptions.NoiseOptions.MeanAttractionConstant = 0.1/Ts;
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Then, create the agent using the input specification and agent options objects. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(obsInfo, actInfo, initOpts, agentOptions);
```

Train DDPG Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 2000 episodes, with each episode lasting a maximum of `ceil(Tf/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the performance of the greedy policy by running one simulation every 50 training episodes.
- Stop the training when the greedy policy evaluation exceeds -390. At this point, the agent can quickly balance the pole in the upright position using minimal control effort.

For more information on training options, see `rlTrainingOptions` and `rlEvaluator`.

```
% training options
maxepisodes = 2000;
maxsteps = ceil(Tf/Ts);
trainingOptions = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    ScoreAveragingWindowLength=5, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-390);

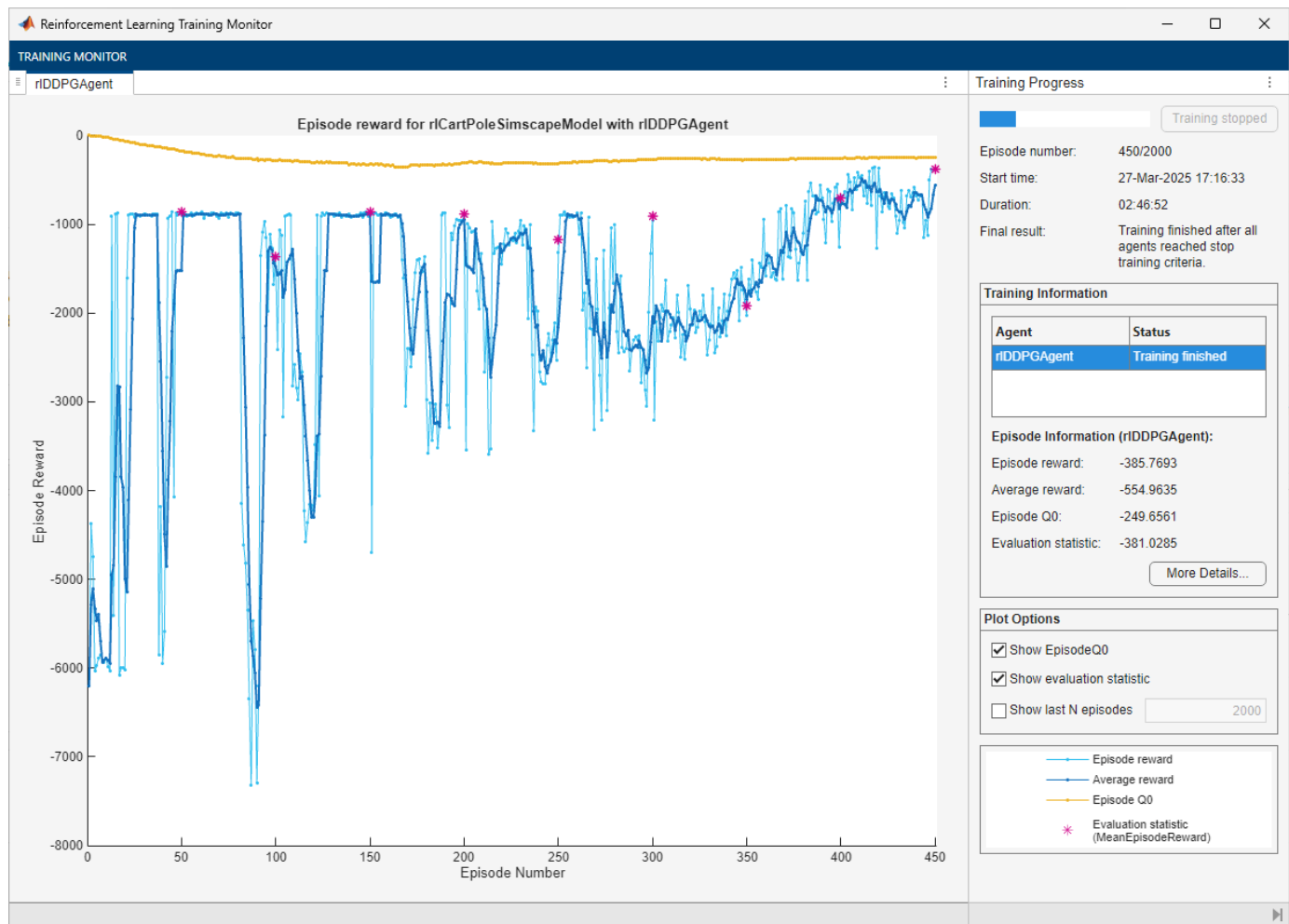
% agent evaluation
evl = rlEvaluator(NumEpisodes=1,EvaluationFrequency=50);
```

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

Train the agent using the `train` function. Training this agent process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainingOptions,Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("SimscapeCartPoleDDPG.mat","agent")
end
```



The training converges after 450 episodes. You can simulate the trained agent in the environment.

Simulate DDPG Agent

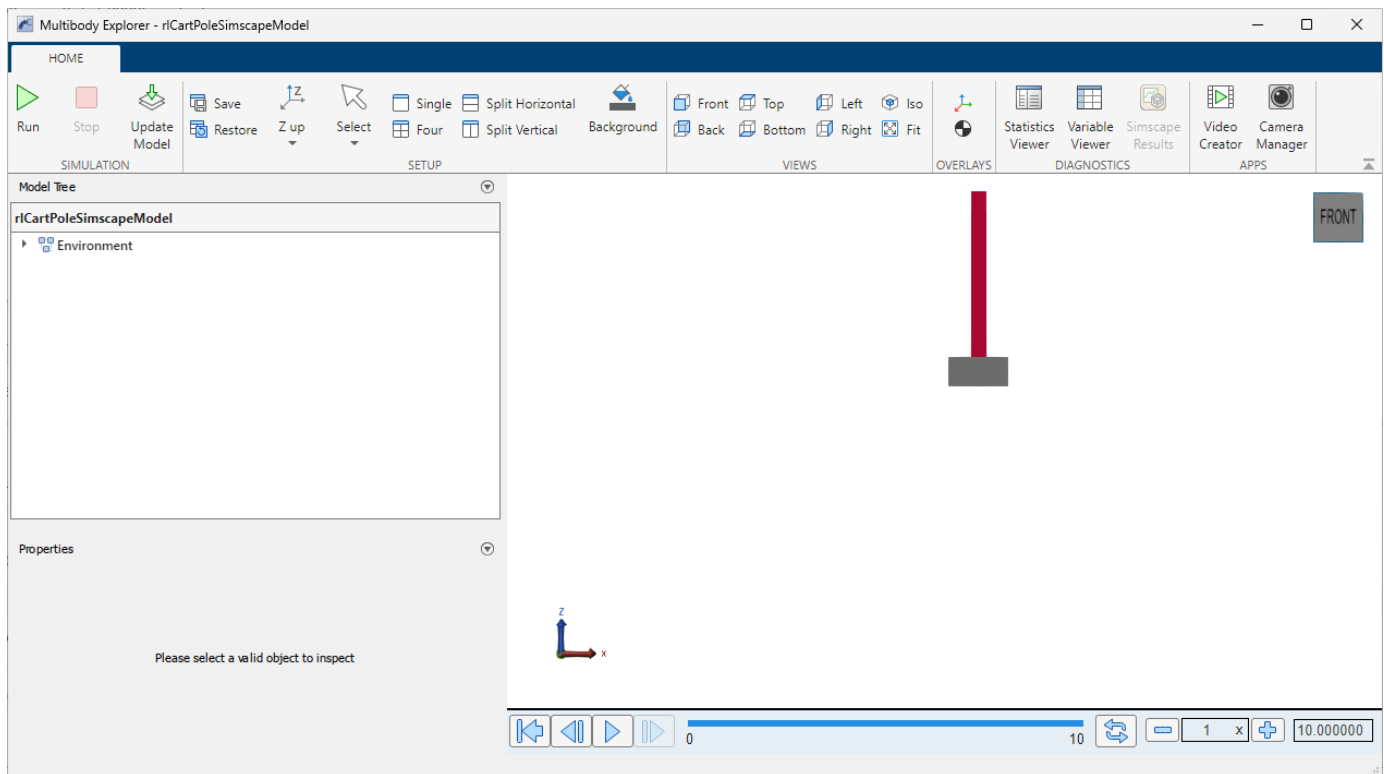
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```



Display the total reward.

```
sum(experience.Reward)
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

```
ans =  
-379.9958
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal” on page 5-141
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train Default PPO Agent for Discrete Lander Vehicle

This example shows how to train a proximal policy optimization (PPO) agent with a discrete action space to land an airborne vehicle on the ground. For more information on PPO agents, see “Proximal Policy Optimization (PPO) Agent” on page 3-50. For an example that shows how to train a SAC agent for this environment, see “Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle” on page 5-132.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

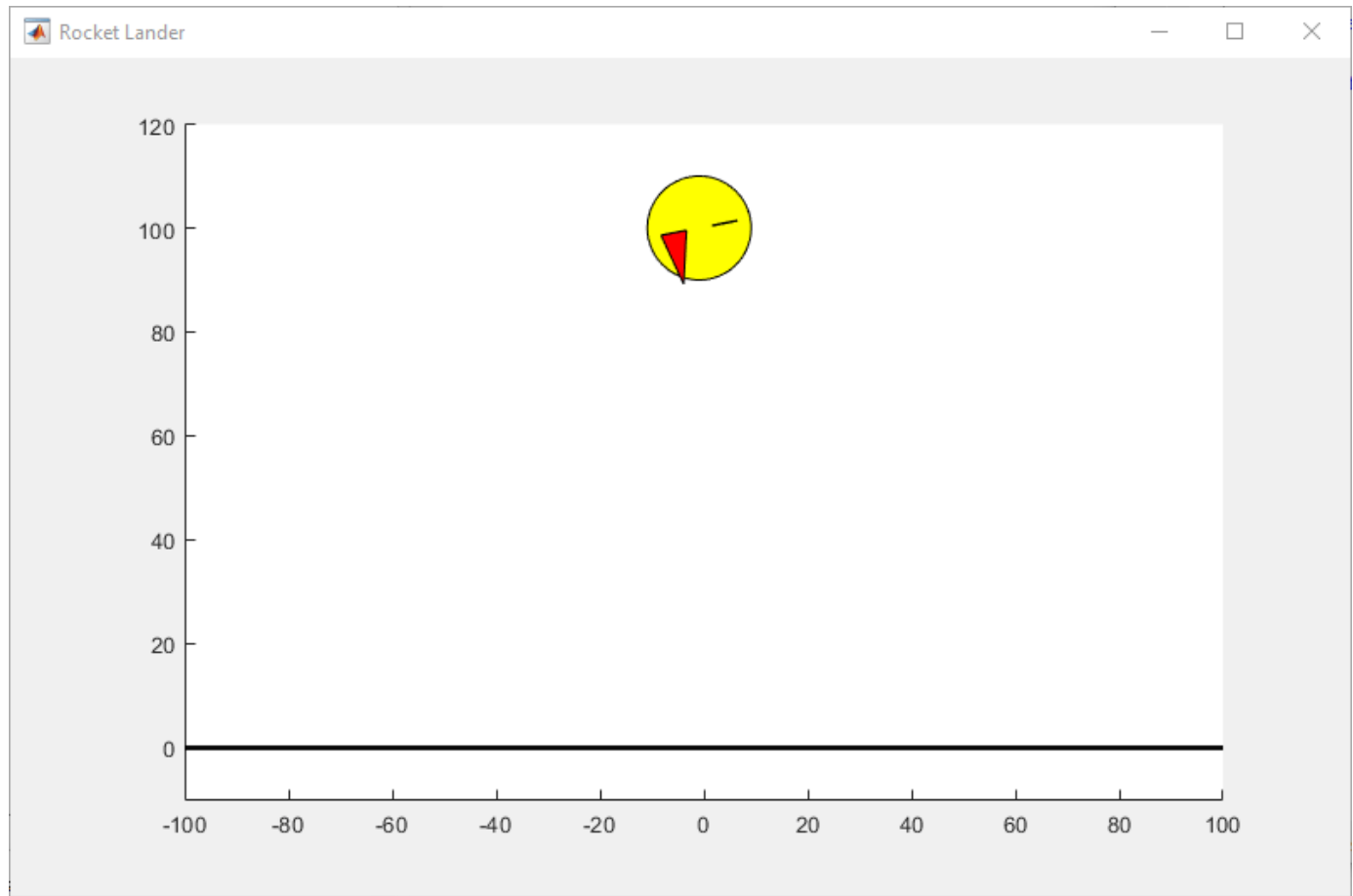
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

The environment in this example is a lander vehicle represented by a 3-DOF circular disc with mass. The vehicle has two thrusters for forward and rotational motion. Gravity acts vertically downwards, and there are no aerodynamic drag forces. The training goal is to make the vehicle land on the ground at a specified location.



For this environment:

- Motion of the lander vehicle is bounded in X (horizontal axis) from -100 to 100 meters and Y (vertical axis) from 0 to 120 meters.
- The goal position is at (0,0) meters and the goal orientation is zero radians.
- The maximum thrust applied by each thruster is 8.5 N.
- The sample time is 0.1 seconds.
- The observations from the environment are the vehicle's position (x, y) , orientation (θ) , velocity $(\dot{x}, \dot{y})$, angular velocity $(\dot{\theta})$, and a sensor reading that detects rough landing (-1), soft landing (1) or airborne (0) condition. The observations are normalized between -1 and 1.
- The environment has a discrete action space. At every time step, the agent selects one of the following nine discrete action pairs:

L, L – do nothing
 L, M – fire right (med)
 L, H – fire right (high)
 M, L – fire left (med)
 M, M – fire left (med) + right (med)
 M, H – fire left (med) + right (high)
 H, L – fire left (high)
 H, M – fire left (high) + right (med)
 H, H – fire left (high) + right (high)

Here, $L = 0.0$, $M = 0.5$ and $H = 1.0$ are normalized thrust values for each thruster. The environment `step` function scales these values to determine the actual thrust values.

- At the beginning of every episode, the vehicle starts from a random initial x position and orientation. The altitude is always reset to 100 meters.
- The reward r_t provided at the time step t is as follows.

$$r_t = (s_t - s_{t-1}) - 0.1\theta_t^2 - 0.01(L_t^2 + R_t^2) + 500c$$

$$s_t = 1 - \left(\sqrt{\hat{d}_t} + \frac{\sqrt{\hat{v}_t}}{2} \right)$$

$$c = (y_t \leq 0) \ \&\& \ (\dot{y}_t \geq -0.5 \ \&\& \ |\dot{x}_t| \leq 0.5)$$

Here:

- $x_t, y_t, \dot{x}_t$, and $\dot{y}_t$ are the positions and velocities of the lander vehicle along the x and y axes.
- $\hat{d}_t = \sqrt{x_t^2 + y_t^2} / d_{\max}$ is the normalized distance of the lander vehicle from the goal position.
- $\hat{v}_t = \sqrt{\dot{x}_t^2 + \dot{y}_t^2} / v_{\max}$ is the normalized speed of the lander vehicle.
- $d_{\max}$ and $v_{\max}$ are the maximum distances and speeds.
- θ_t is the orientation with respect to the vertical axis.
- L_t and R_t are the action values for the left and right thrusters.
- c is a sparse reward for soft-landing with horizontal and vertical velocities less than 0.5 m/s.

Create a MATLAB® environment using the `LanderVehicle` class provided in the example folder.

```
env = LanderVehicle()
```

```
env =
```

```
  LanderVehicle with properties:
```

```

    Mass: 1
    L1: 10
    L2: 5
    Gravity: 9.8060
    ThrustLimits: [0 8.5000]
    Ts: 0.1000
    State: [6x1 double]
    LastAction: [2x1 double]
    LastShaping: 0
  
```

```
DistanceIntegral: 0
VelocityIntegral: 0
TimeCount: 0
```

To view the implementation of the `LanderVehicle` class, open the class file.

```
open("LanderVehicle.m")
```

Obtain the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "states"
    Description: [0x0 string]
    Dimension: [7 1]
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlFiniteSetSpec with properties:
    Elements: {9x1 cell}
    Name: "thrusts"
    Description: [0x0 string]
    Dimension: [2 1]
    DataType: "double"
```

Create Default PPO Agent

You will create and train a PPO agent in this example. The agent uses:

- A value function critic to estimate the value of the policy. The critic takes the current observation as input and returns a single scalar as output (the estimated discounted cumulative long-term reward for following the policy from the state corresponding to the current observation).
- A stochastic actor for computing actions. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

The actor and critic functions are approximated using neural network representations. Specify a hidden layer size of 400 for the networks.

```
hlsz = 400;
```

Create an agent initialization object to initialize the networks with the specified layer size.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=hlsz);
```

Specify hyperparameters for training using the `rlPPOAgentOptions` object:

- Specify an experience horizon of 900 steps. A large experience horizon can improve the stability of the training.
- Train with 3 epochs and mini-batches of length 300. Smaller mini-batches are computationally efficient but might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.
- Specify a learning rate of $5e-4$ for the actor and $9e-3$ for the critic. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Specify an objective function clip factor of 0.04 for improving training stability.
- Specify a discount factor value of 0.998 to promote long term rewards.
- Specify an entropy loss weight factor of 0.02 to enhance exploration during training.

```
agentOpts = rlPPOAgentOptions( ...
    ExperienceHorizon    = 900, ...
    ClipFactor           = 0.04, ...
    EntropyLossWeight    = 0.02, ...
    ActorOptimizerOptions = rlOptimizerOptions(LearnRate=5e-4), ...
    CriticOptimizerOptions = rlOptimizerOptions(LearnRate=9e-3), ...
    MiniBatchSize        = 300, ...
    NumEpoch            = 3, ...
    SampleTime           = env.Ts, ...
    DiscountFactor        = 0.998);
```

Create the PPO agent object. When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
agent = rlPPOAgent(obsInfo, actInfo, initOpts, agentOpts);
```

Train Agent

To train the PPO agent, specify the following training options.

- Run the training for a maximum of 10000 episodes, with each episode lasting a maximum of 600 time steps.
- Specify an averaging window length of 100 for the episode reward.
- Stop the training when the average reward reaches 450 for 100 consecutive episodes.

```
% training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=10000, ...
    MaxStepsPerEpisode=600, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=450, ...
    ScoreAveragingWindowLength=100);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

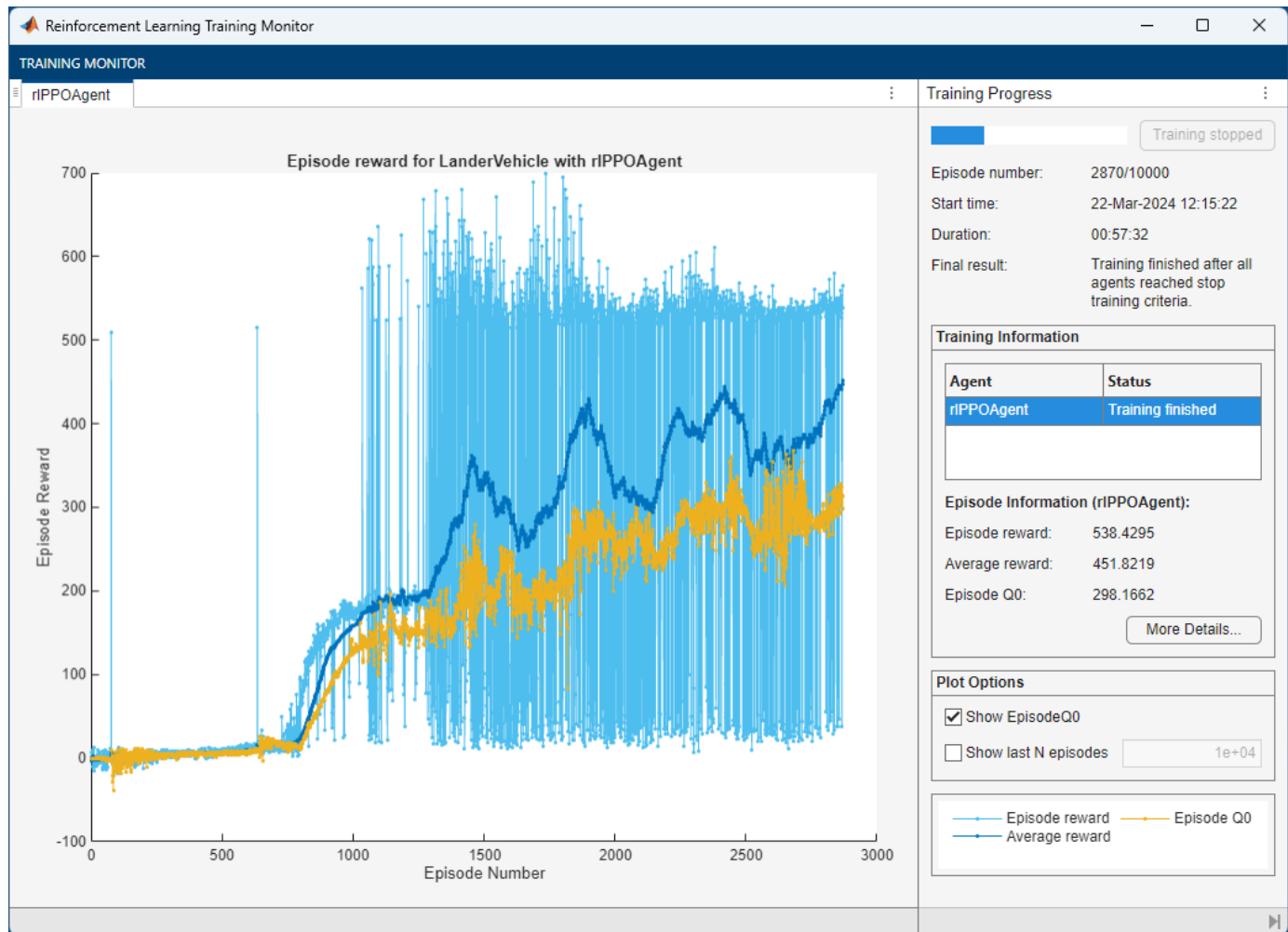
Train the agent using the `train` function. Due to the complexity of the environment, training process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`.

```

doTraining = ;
if doTraining
    trainingStats = train(agent, env, trainOpts);
else
    load("landerVehicleAgent.mat");
end

```

An example of the training is shown below. The actual results might vary because of randomness in the training process.



Simulate Trained Agent

Plot the environment first to create a visualization for the lander vehicle.

```
plot(env)
```

Set up simulation options to perform 5 simulations. For more information, see `rlSimulationOptions`.

```

simOptions = rlSimulationOptions(MaxSteps=600);
simOptions.NumSimulations = 5;

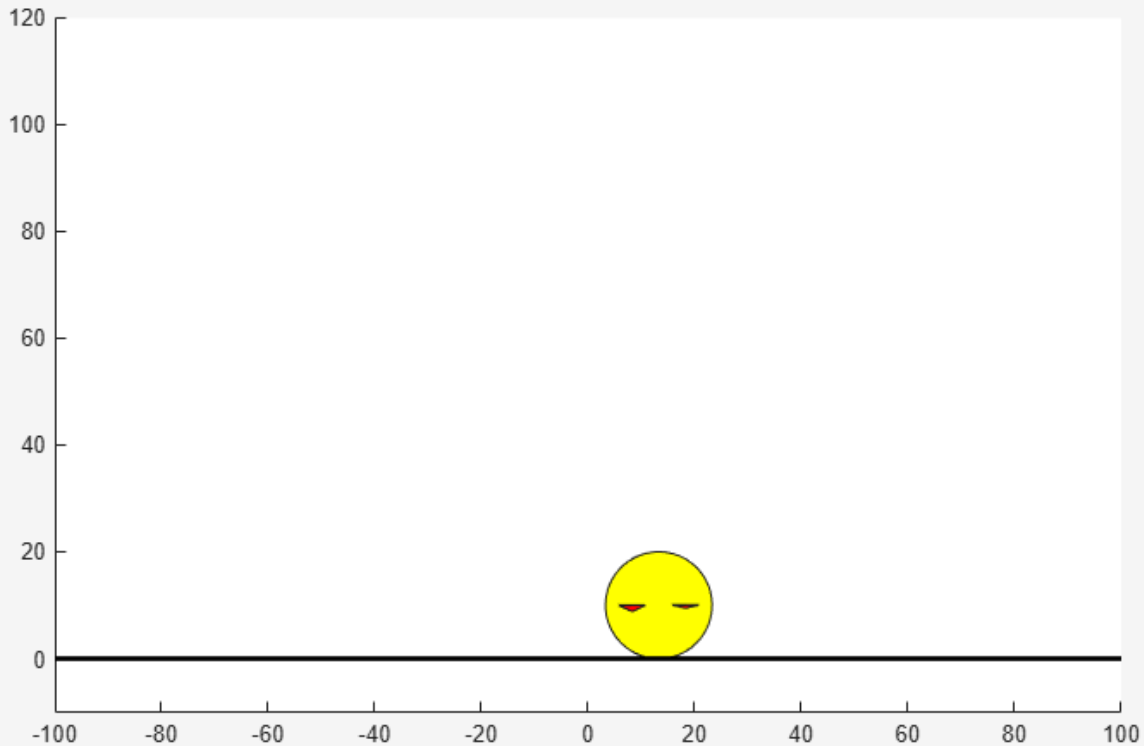
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Simulate the trained agent within the environment. For more information, see `sim`.

```
experience = sim(env, agent, simOptions);
```



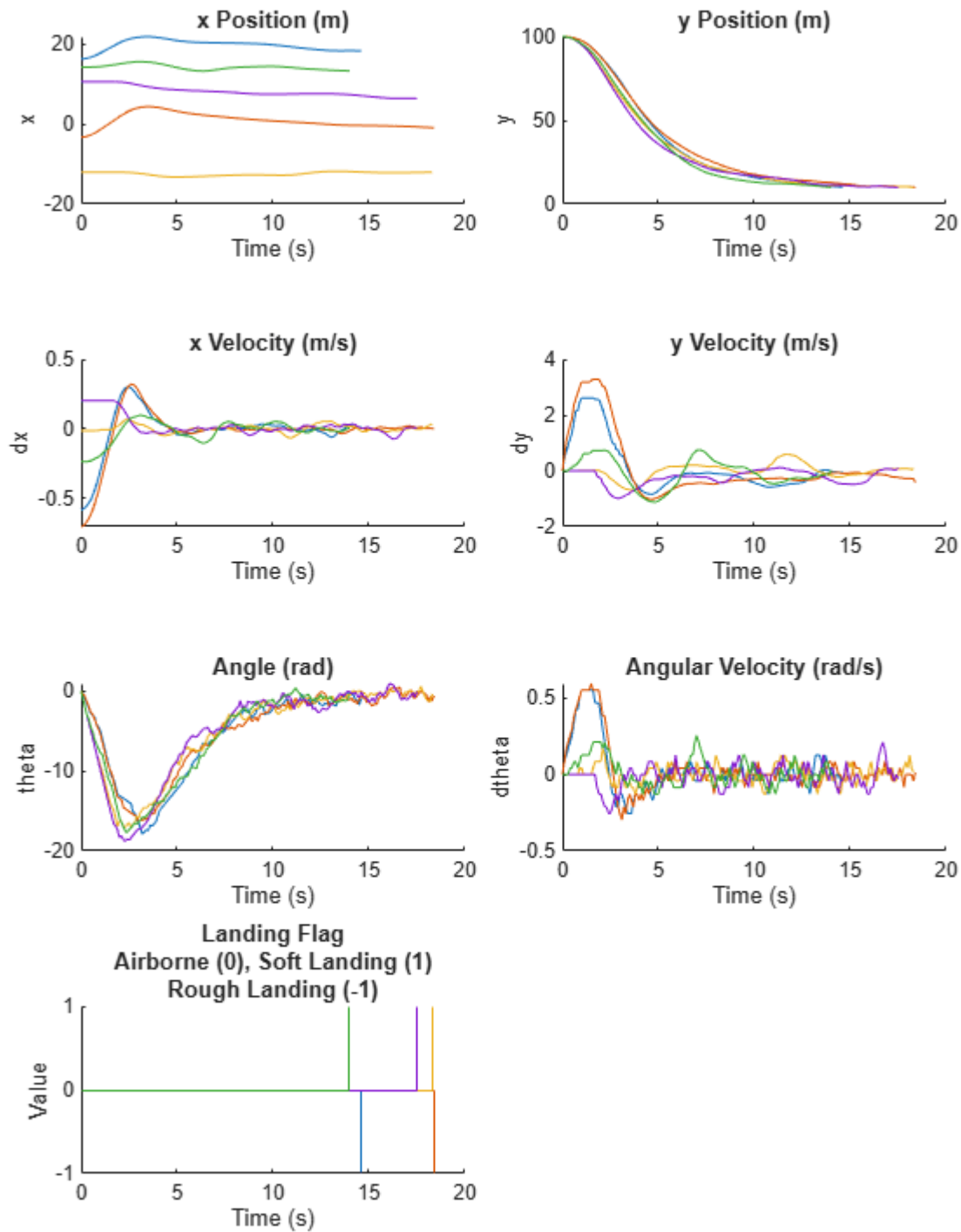
Plot the time history of the states for all simulations using the helper function `plotLanderVehicleTrajectory` provided in the example folder.

```
% Observations to plot
obsToPlot = ["x", "y", "dx", "dy", "theta", "dtheta", "landing"];

% Create a figure
f = figure();
f.Position(3:4) = [800, 1000];

% Create a tiled layout for the plots
t = tiledlayout(f, 4, 2, TileSpacing="compact");

% Plot the data
for ct = 1:numel(obsToPlot)
    ax = nexttile(t);
    plotLanderVehicleTrajectory(ax, experience, env, obsToPlot(ct));
end
```



Restore the random number stream using the information stored in `previousRngState`.


```
rng(previousRngState);
```

See Also

Functions

`train` | `sim`

Objects

`rlPPOAgent` | `rlPPOAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle” on page 5-132
- “Train DDPG Agent to Control Two-Thruster Sliding Vehicle” on page 8-22
- “Train PPO Agent for Automatic Parking Valet” on page 8-85
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31

More About

- “Reinforcement Learning Environments” on page 2-2
- “Proximal Policy Optimization (PPO) Agent” on page 3-50
- “Train Reinforcement Learning Agents” on page 5-3

Train LSPI Agent to Balance Discrete Cart-Pole

This example shows how to train an online least-squares policy iteration (LSPI) agent to balance a discrete action space cart-pole system modeled in MATLAB®. For more information on LSPI agents, see “LSPI Agent” on page 3-28.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

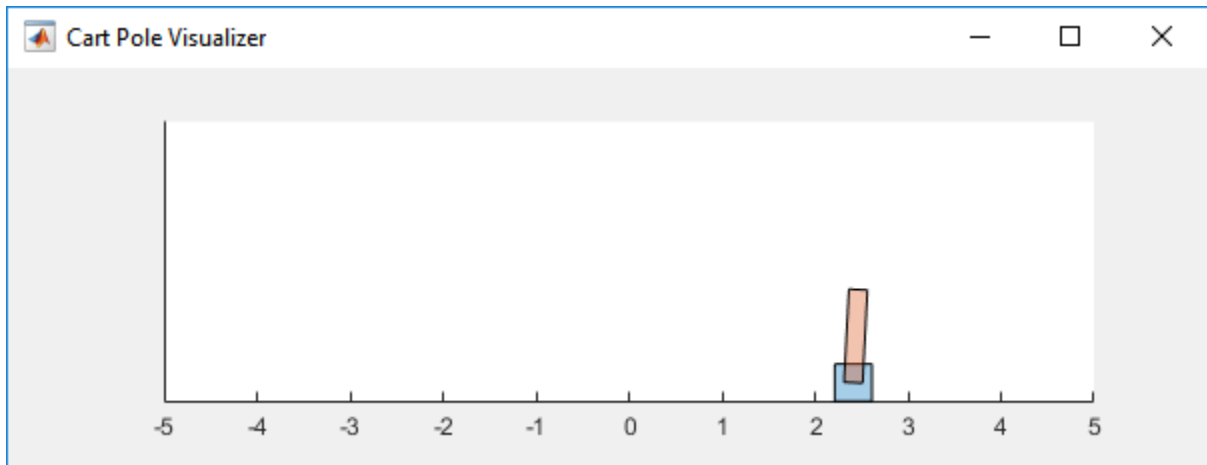
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Cart-Pole MATLAB Environment

The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart that moves along a frictionless track. The training goal is to make the pole stand upright.



For this environment:

- The upward balanced pole position is 0 radians, and the downward hanging position is π radians.
- The pole starts upright with an initial angle between -0.05 and 0.05 radians.
- The force action signal from the agent to the environment is either -10 or 10 N.
- The observations from the environment are the position and velocity of the cart, pole angle, and pole angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.

- A reward of +1 is provided for every time step that the pole remains upright. A penalty of -5 is applied when the pole falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined cart-pole environment object.

```
env = rlPredefinedEnv("CartPole-Discrete")
```

```
env =
  CartPoleDiscreteAction with properties:
      Gravity: 9.8000
      MassCart: 1
      MassPole: 0.1000
      Length: 0.5000
      MaxForce: 10
      Ts: 0.0200
      ThetaThresholdRadians: 0.2094
      XThreshold: 2.4000
      RewardForNotFalling: 1
      PenaltyForFalling: -5
      State: [4x1 double]
```

The interface has a discrete action space where the agent can apply one of two possible force values to the cart, -10 or 10 N.

Get the observation and action specification information.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
      LowerLimit: -Inf
      UpperLimit: Inf
      Name: "CartPole States"
      Description: "x, dx, theta, dtheta"
      Dimension: [4 1]
      DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlFiniteSetSpec with properties:
      Elements: [-10 10]
      Name: "CartPole Action"
      Description: [0x0 string]
      Dimension: [1 1]
      DataType: "double"
```

Create LSPI Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

LSPI agents use Q-value functions with basis functions as critics. This Q-value function represents the expected cumulative long-term reward for taking the corresponding discrete action from the state indicated by the observation inputs. For more information on creating value functions, see “Create Actors, Critics, and Policy Objects” on page 4-2. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

This examples uses `myCartpoleBasisFcn` as a basis function. The file containing this function is provided in the example folder.

```
type myCartPoleBasisFcn.m
```

```
function feature = myCartPoleBasisFcn(obs,act)
    % A basis function for a cartpole problem.
```

```
    % Copyright 2024 The MathWorks Inc.
```

```
    act = act/10;
    feature = [
        obs
        act
        obs.^2
        act.^2
        obs(1,1,:).*obs(2,1,:)
        obs(1,1,:).*obs(3,1,:)
        obs(1,1,:).*obs(4,1,:)
        obs(2,1,:).*obs(3,1,:)
        obs(2,1,:).*obs(4,1,:)
        obs(3,1,:).*obs(4,1,:)
        obs(1,1,:).*obs(2,1,:).*act
        obs(1,1,:).*obs(3,1,:).*act
        obs(1,1,:).*obs(4,1,:).*act
        obs(2,1,:).*obs(3,1,:).*act
        obs(2,1,:).*obs(4,1,:).*act
        obs(3,1,:).*obs(4,1,:).*act
        obs.*act];
```

```
end
```

This choice approximates the Q-value function as a polynomial. The linear, quadratic, and cross-product monomials capture curvature and smooth nonlinear relationships between the cart-pole's state and action. Other choices are possible, such as radial basis functions, and might produce an even better approximation of the optimal Q-value function.

Construct a Q-value function using `myCartpoleBasisFcn`.

```
% Compute the number of features.
obs = zeros(4,1);
act = 10;
temp = myCartPoleBasisFcn(obs, act);
```

```
numFeatures = numel(temp);
```

```
% Initialize weights.
```

```
w0 = rand(numFeatures, 1);
```

```
% Create a critic object.
```

```
critic = rlQValueFunction(@myCartPoleBasisFcn,w0,obsInfo, actInfo);
```

Specify the agent options for training using `rlLSPIAgentOptions` and `rlOptimizerOptions` objects. For this training, set `LearningFrequency` to 10 to update the critic every 10 samples collected.

```
agentOpts = rlLSPIAgentOptions(LearningFrequency=10);
```

LSPI agents use the epsilon-greedy algorithm to explore the action space during training. Specify a decay rate of $2e-4$ for the epsilon value to gradually decay during training. This decay rate promotes exploration toward the beginning when the agent does not have a good policy, and exploitation toward the end when the agent has learned the optimal policy.

```
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 2e-4;
```

```
agentOpts.EpsilonGreedyExploration.EpsilonMin = 0.05;
```

Create the LSPI agent using the observation and action input specifications, initialization options and agent options.

```
rng(0, "twister");
```

```
agent = rlLSPIAgent(critic,agentOpts);
```

For more information, see `rlLSPIAgent`.

Check the action of the agent with a random observation input.

```
getAction(agent,{rand(obsInfo.Dimension)}))
```

```
ans = 1x1 cell array
    {[10]}
```

Train LSPI Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To train the agent, first specify the training options. For this example, use these options:

- Run the training for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor window (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the performance of the greedy policy every 50 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score reaches 500. At this point, the agent can balance the cart-pole system in the upright position.

```
% Training options
```

```
trainOpts = rlTrainingOptions( ...
```

```
MaxEpisodes=3000, ...  
MaxStepsPerEpisode=500, ...  
Verbose=false, ...  
Plots="training-progress", ...  
StopTrainingCriteria="EvaluationStatistic", ...  
StopTrainingValue=500);
```

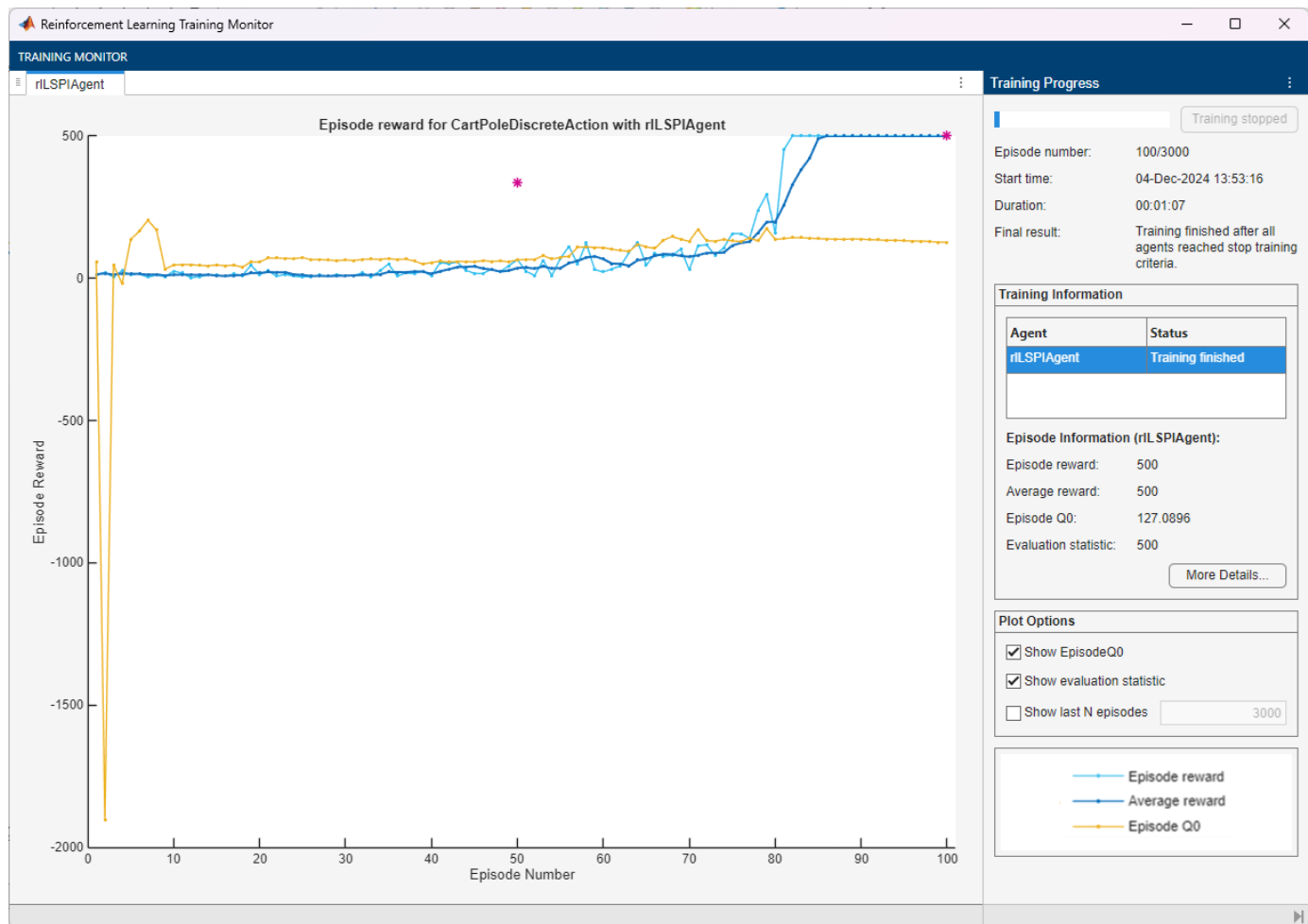
```
% Agent evaluator
```

```
evl = rlEvaluator(EvaluationFrequency=50,NumEpisodes=5,RandomSeeds=1:5);
```

For more information, see `rlTrainingOptions` and `rlEvaluator`.

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainingStats = train(agent,env,trainOpts,Evaluator=evl);  
else  
    % Load the pretrained agent for the example.  
    load("MATLABCartpoleLSPI.mat","agent")  
end
```



Simulate LSPI Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

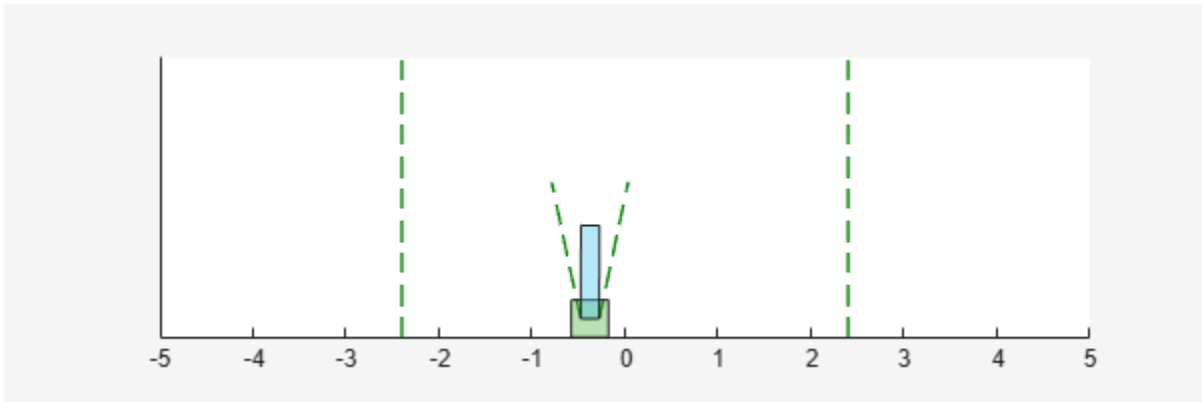
You can visualize the cart-pole system by using the `plot` function.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```



```
totalReward = sum(experience.Reward)
```

```
totalReward =  
500
```

The agent can balance the cart-pole system.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`train` | `sim`

Objects

`rlLSPIAgent` | `rlLSPIAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions` | `rlEvaluator`

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Train Reinforcement Learning Agents” on page 5-3

More About

- “Use Predefined Control System Environments” on page 2-42
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “LSPI Agent” on page 3-28
- “Train Reinforcement Learning Agents” on page 5-3

Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole

This example shows how to train a policy gradient (PG) agent to balance a discrete action space cart-pole system modeled in MATLAB®. For more information on PG agents, see “REINFORCE Policy Gradient (PG) Agent” on page 3-37.

For an example that trains a PG agent with a baseline and custom networks, see “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115.

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

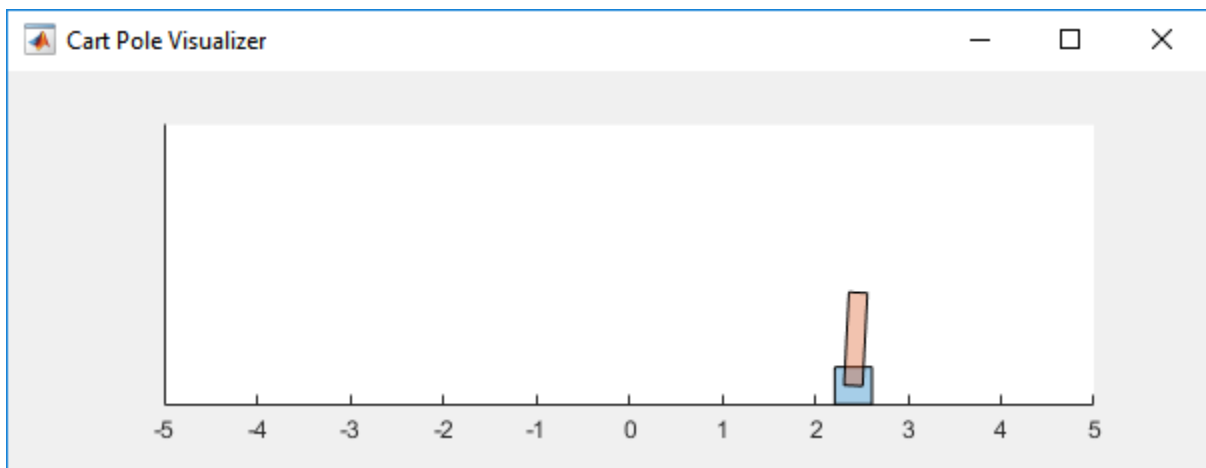
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Cart-Pole MATLAB Environment

The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to make the pendulum stand upright.



For this environment:

- The balanced, upright pendulum position is 0 radians, and the downward hanging pendulum position is π radians.
- The pendulum starts upright with an initial angle between -0.05 and 0.05 radians.
- The force action signal from the agent to the environment is either -10 or 10 N.
- The observations from the environment are the position and velocity of the cart, the pendulum angle (clockwise-positive), and the pendulum angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of +1 is provided for every time step that the pole remains upright. A penalty of -5 is applied when the pendulum falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("CartPole-Discrete")
```

```
env =  
    CartPoleDiscreteAction with properties:  
  
        Gravity: 9.8000  
        MassCart: 1  
        MassPole: 0.1000  
        Length: 0.5000  
        MaxForce: 10  
        Ts: 0.0200  
        ThetaThresholdRadians: 0.2094  
        XThreshold: 2.4000  
        RewardForNotFalling: 1  
        PenaltyForFalling: -5  
        State: [4×1 double]
```

The object has a discrete action space where the agent can apply one of two possible force values to the cart, -10 or 10 N.

Obtain the observation and action information from the environment object.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

Create PG Agent with Custom Actor Network

For policy gradient agents, the actor executes a stochastic policy, which for discrete action spaces is approximated by a discrete categorical actor. This actor must take the observation signal as input and return a probability for each action.

As a simple parameterized policy is all you need to stabilize the pole on the cart, define a custom PG agent without any baseline. To implement the parameterized policy within the actor, use a simple neural network with just one hidden layer containing 10 neurons.

Define the network as an array of layer objects, and get the dimension of the observation space and the number of possible actions from the environment specification objects. When you create the

network, the initial parameters are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
actorNet = [
    featureInputLayer(prod(obsInfo.Dimension))
    fullyConnectedLayer(10)
    reluLayer
    fullyConnectedLayer(numel(actInfo.Elements))
    softmaxLayer
];
```

For more information on creating a deep neural network policy representation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Convert to `dlnetwork` and display the number of weights.

```
actorNet = dlnetwork(actorNet);
summary(actorNet)
```

```
Initialized: true

Number of learnables: 72

Inputs:
    1 'input'    4 features
```

Create the actor representation using the specified deep neural network and the environment specification objects. For more information, see `rlDiscreteCategoricalActor`.

```
actor = rlDiscreteCategoricalActor(actorNet, obsInfo, actInfo);
```

To return the probability distribution of the possible actions as a function of a random observation, and given the current network weights, use `evaluate`.

```
prb = evaluate(actor, {rand(obsInfo.Dimension)});
prb{1}
```

```
ans = 2×1 single column vector
```

```
    0.7229
    0.2771
```

Create the agent using the actor. For more information, see `rlPGAgent`.

```
agent = rlPGAgent(actor);
```

Check the agent with a random observation input.

```
getAction(agent, {rand(obsInfo.Dimension)})
```

```
ans = 1×1 cell array
    {-10}
```

Specify training options for the actor. Alternatively, you can use `rlPGAgentOptions` and `rlOptimizerOptions` objects.

For this training:

- Specify the actor learning rate to be 5e-3. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Use a gradient threshold of 1 to clip the gradients. Clipping the gradients can improve training stability.

```
agent.AgentOptions.ActorOptimizerOptions = ...  
    rlOptimizerOptions(LearnRate=5e-3, ...  
        GradientThreshold=1);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the Plots option) and disable the command line display (set the Verbose option to false).
- Evaluate the performance of the greedy policy every 50 training episodes, averaging the cumulative reward of 10 simulations.
- Stop the training when the evaluation score reaches 500. At this point, the agent can balance the cart-pole system in the upright position.

For more information on training options, see `rlTrainingOptions`.

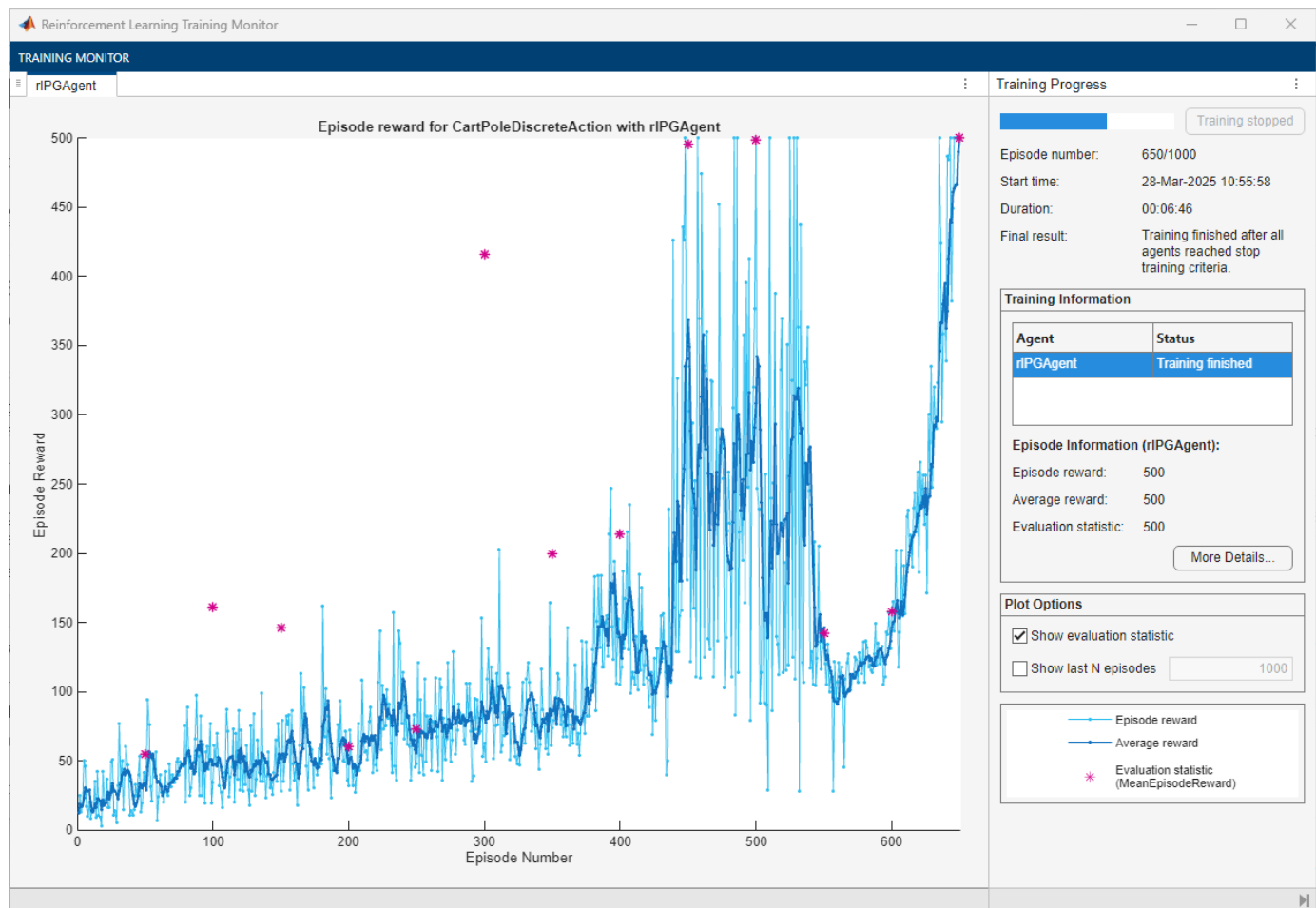
```
% training options  
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=1000, ...  
    MaxStepsPerEpisode=500, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=500);  
  
% agent evaluator  
evl = rlEvaluator(EvaluationFrequency=50, NumEpisodes=10);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainingStats = train(agent, env, trainOpts, Evaluator=evl);  
else  
    % Load the pretrained agent for the example.  
    load("MATLABCartpolePG.mat", "agent");  
end
```



The training converges after 650 episodes. You can simulate the trained agent in the environment.

Simulate PG Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

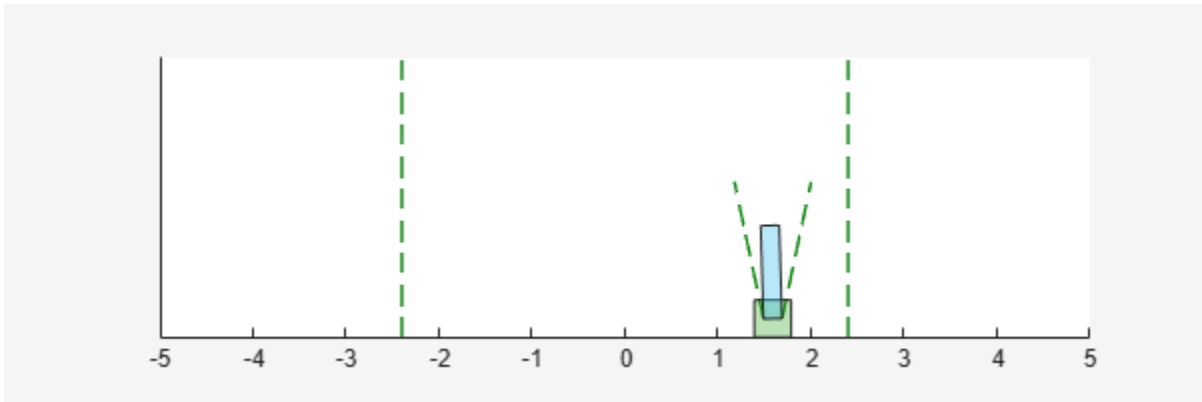
You can visualize the cart-pole system by using the `plot` function.

```
plot(env)
```

Use a deterministic policy for simulation.

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```



The agent can balance the cart-pole system. Display the total reward obtained during the simulation.

```
totalReward = sum(experience.Reward)
```

```
totalReward =  
500
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlTrainingOptions` | `rlSimulationOptions` | `rlPGAgent` | `rlPGAgentOptions`

Related Examples

- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115
- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93

More About

- “Use Predefined Control System Environments” on page 2-42
- “REINFORCE Policy Gradient (PG) Agent” on page 3-37
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator

This example shows how to train a policy gradient (PG) agent with baseline to control a discrete action space second-order dynamic system modeled in MATLAB®.

For more information on the basic PG agent with no baseline, see the example “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

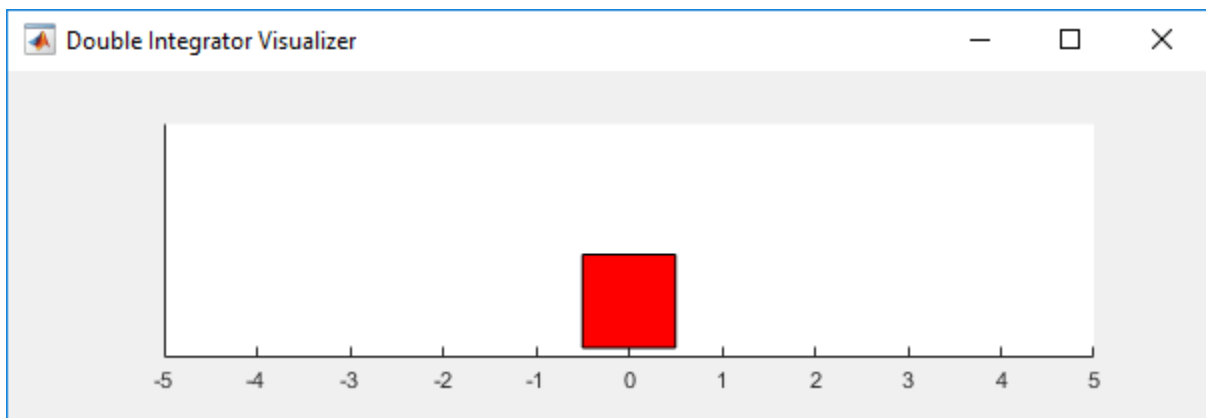
Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Double-Integrator MATLAB Environment

The reinforcement learning environment for this example is a second-order double-integrator system with a gain and a discrete action space. The training goal is to control the position of a mass in the second-order system by applying a force input.



For this environment:

- The mass starts at an initial position between -2 and 2 m.
- The agent can apply one of three possible force values to the mass: -2, 0, or 2 N.
- The observations from the environment are the position and velocity of the mass.
- The episode terminates if the mass moves more than 5 m from the original position or if $|x| < 0.01$.

- The reward r_t , provided at every time step, is a discretization of $r(t)$:

$$r(t) = - (x(t)' Q x(t) + u(t)' R u(t))$$

Here:

- x is the state vector of the mass.
- u is the force applied to the mass.
- Q is the weight on the state deviation from zero; $Q = [10 \ 0; 0 \ 1]$.
- R is the weight on the control effort; $R = 0.01$.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("DoubleIntegrator-Discrete")
```

```
env =  
    DoubleIntegratorDiscreteAction with properties:
```

```
        Gain: 1  
        Ts: 0.1000  
    MaxDistance: 5  
    GoalThreshold: 0.0100  
        Q: [2x2 double]  
        R: 0.0100  
    MaxForce: 2  
    State: [2x1 double]
```

Obtain the observation and action information from the environment object.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

Create Actor with Custom Network

The actor network of the PG agent are initialized randomly. Fix the random number stream.

```
rng(0, "twister");
```

For policy gradient agents, the actor executes a stochastic policy, which for discrete action spaces is approximated by a discrete categorical actor. This actor must take the observation signal as input and return a probability for each action.

To approximate the policy within the actor, use a neural network. Define the network as an array of layer objects with one input (the observation) and one output (the action), and get the dimension of the observation space and the number of possible actions from the environment specification objects. For more information on creating a deep neural network value function representation, see “Create Actors, Critics, and Policy Objects” on page 4-2.

```
actorNet = [  
    featureInputLayer(obsInfo.Dimension(1))
```



```
fullyConnectedLayer(numel(actInfo.Elements))
];
```

Convert to `dlnetwork` and display the number of weights.

```
actorNet = dlnetwork(actorNet);
summary(actorNet)
```

```
Initialized: true
```

```
Number of learnables: 9
```

```
Inputs:
    1 'input'    2 features
```

Create the actor representation using the neural network and the environment specification objects. For more information, see `rlDiscreteCategoricalActor`.

```
actor = rlDiscreteCategoricalActor(actorNet,obsInfo,actInfo);
```

To return the probability distribution of the possible actions as a function of a random observation, and given the current network weights, use `evaluate`.

```
prb = evaluate(actor,{rand(obsInfo.Dimension)}))
```

```
prb = 1x1 cell array
      {3x1 single}
```

```
prb{1}
```

```
ans = 3x1 single column vector
```

```
0.4994
0.3770
0.1235
```

Create Baseline with Custom Network

The baseline network of the PG agent are initialized randomly. Fix the random number stream.

```
rng(0, "twister");
```

The PG Agent algorithm, (also known as REINFORCE) returns can be compared to a baseline that depends on the state. Using a baseline can reduce the variance of the expected value of the update and thus improve the speed of learning. A possible choice for the baseline is an estimate of the state value function [1].

A value-function approximator object must accept an observation as input and return a single scalar (the estimated discounted cumulative long-term reward) as output. Use a neural network as approximation model. Define the network as an array of layer objects, and get the dimension of the observation space and the number of possible actions from the environment specification objects.

```
baselineNet = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(8)
    reluLayer
```

```
fullyConnectedLayer(1)
];
```

Convert to `dlnetwork` and display the number of weights.

```
baselineNet = dlnetwork(baselineNet);
```

Create the baseline value function approximator `baselineNet`, and the observation specification. For more information, see `rlValueFunction`.

```
baseline = rlValueFunction(baselineNet,obsInfo);
```

Check the baseline with a random observation input.

```
getValue(baseline,{rand(obsInfo.Dimension)}))
```

```
ans = single
```

```
1.5300
```

Configure Agent Options and Create Agent

Specify options for the actor. For more information, see `rlOptimizerOptions`. Alternatively, you can change agent (including actor and critic) options using dot notation after the agent is created.

```
actorOpts = rlOptimizerOptions( ...
    LearnRate=5e-3, ...
    GradientThreshold=1);
```

Specify some options for the baseline.

```
baselineOpts = rlOptimizerOptions( ...
    LearnRate=5e-3, ...
    GradientThreshold=1);
```

Specify the PG agent options using `rlPGAgentOptions` and set the `UseBaseline` option set to `true`.

```
agentOpts = rlPGAgentOptions( ...
    UseBaseline=true, ...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=baselineOpts);
```

Create the agent using the specified actor representation, baseline representation, and agent options. For more information, see `rlPGAgent`.

```
agent = rlPGAgent(actor,baseline,agentOpts);
```

Check the agent with a random observation input.

```
getAction(agent,{rand(obsInfo.Dimension)}))
```

```
ans = 1x1 cell array
    {-2}
```

Configure Training Options

Set up an evaluator object to evaluate the agent five times without exploration every 50 training episodes.

```
evl = rlEvaluator(NumEpisodes=5,EvaluationFrequency=50);
```

Configure the training options. For this example, use the following options.

- Run a maximum of 1000 episodes, with each episode lasting a maximum of 200 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the average reward in the evaluation episodes is greater than -40. At this point, the agent can control the position of the mass using minimal control effort.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=200, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-40);
```

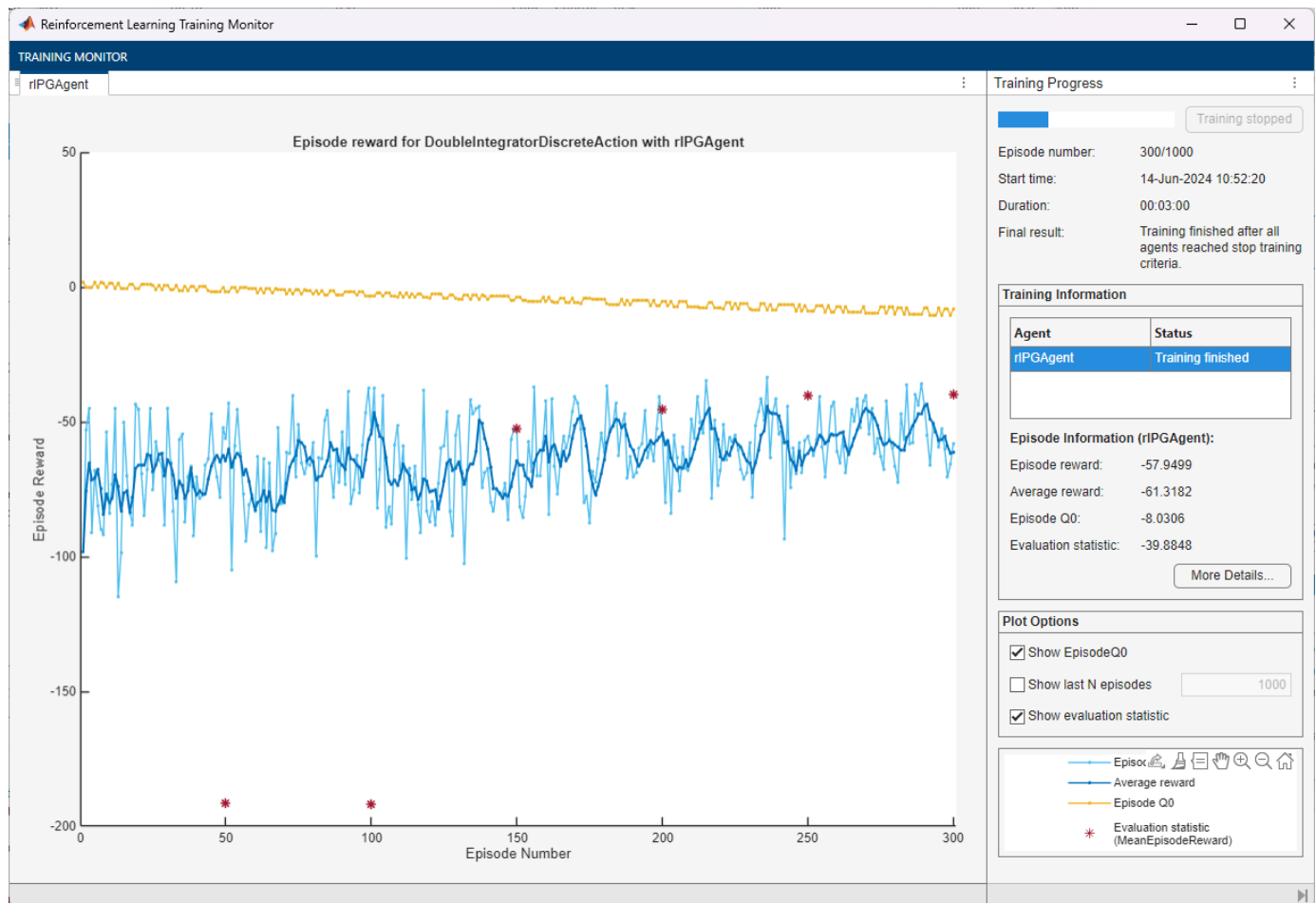
Train Agent

Fix the random number stream.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=evl);
else
    % Load the pretrained parameters for the example.
    load("DoubleIntegPGBaseline.mat");
end
```



Simulate Agent

Fix the random number stream.

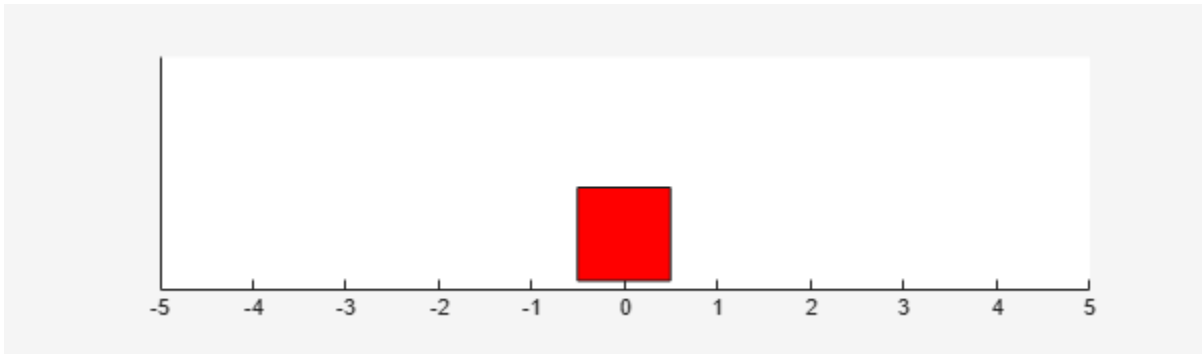
```
rng(0, "twister");
```

You can visualize the double-integrator system using the `plot` function during training or simulation.

```
plot(env)
```

To validate the performance of the trained agent, simulate it within the double-integrator environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
```



```
totalReward = sum(experience.Reward)
```

```
totalReward =  
-63.8540
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState)
```

References

[1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning Series. Cambridge, MA: The MIT Press, 2018.

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlPGAgent` | `rlPGAgentOptions` | `rlTrainingOptions`

Related Examples

- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109
- “Compare DDPG Agent to LQR Controller” on page 7-285

More About

- “Use Predefined Control System Environments” on page 2-42
- “REINFORCE Policy Gradient (PG) Agent” on page 3-37
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Train DDPG Agent with Custom Networks Using Image Observation

This example shows how to train a deep deterministic policy gradient (DDPG) agent to swing up and balance a pendulum with an image observation modeled in MATLAB®.

For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

Simple Pendulum with Image MATLAB Environment

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

For this environment:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment is from -2 to 2 N·m (counterclockwise positive).
- The observations from the environment are an image indicating the location of the pendulum mass and the pendulum angular velocity.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position (counterclockwise positive).
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous")
```

```
env =  
    SimplePendulumWithImageContinuousAction with properties:
```

```
        Mass: 1  
    RodLength: 1  
    RodInertia: 0  
        Gravity: 9.8100  
    DampingRatio: 0  
    MaximumTorque: 2
```

```

Ts: 0.0500
State: [2×1 double]
Q: [2×2 double]
R: 1.0000e-03

```

The object has a continuous action space where the agent can apply a torque between -2 to 2 N·m.

Obtain the observation and action specification from the environment object.

```

obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);

```

Fix the random generator seed for reproducibility.

```
rng(0)
```

Create DDPG Agent with Custom Networks

DDPG agents use a parameterized Q-value function approximator to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output. The output is the estimated value of the policy, that is the estimated discounted cumulative long-term reward when the agent starts from the state corresponding to the current observation, takes the action, and follows the policy thereafter.

To model the parameterized Q-value function within the critic, use a convolutional neural network (CNN) with three input layers (one for each observation channel, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value).

Define each network path as an array of layer objects, and assign names to the input and output layers of each path, as well as to the addition and concatenation layers. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel. For more information on creating representations, see “Create Actors, Critics, and Policy Objects” on page 4-2.

```

hiddenLayerSize1 = 256;
hiddenLayerSize2 = 256;

% Image input path
imgPath = [
    imageInputLayer(obsInfo(1).Dimension, ...
        Name="imgInLyr")
    convolution2dLayer(5,8,Stride=3,Padding=0)
    reluLayer
    convolution2dLayer(5,8,Stride=3,Padding=0)
    reluLayer
    fullyConnectedLayer(32)
    concatenationLayer(1,2,Name="cat1")
    fullyConnectedLayer(hiddenLayerSize1)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize2)
    additionLayer(2,Name="add")
    reluLayer
    fullyConnectedLayer(1,Name="fc4")
];

```

```

% d(theta)/dt input path
dthPath = [
    featureInputLayer(prod(obsInfo(2).Dimension), ...
        Name="dthInLyr")
    fullyConnectedLayer(1,Name="fc5", ...
        BiasLearnRateFactor=0, ...
        Bias=0)
];

% Action path
actPath =[
    featureInputLayer(prod(obsInfo(2).Dimension), ...
        Name="actInLyr")
    fullyConnectedLayer(hiddenLayerSize2, ...
        Name="fc6", ...
        BiasLearnRateFactor=0, ...
        Bias=zeros(hiddenLayerSize2,1))
];

```

Assemble dlnetwork object.

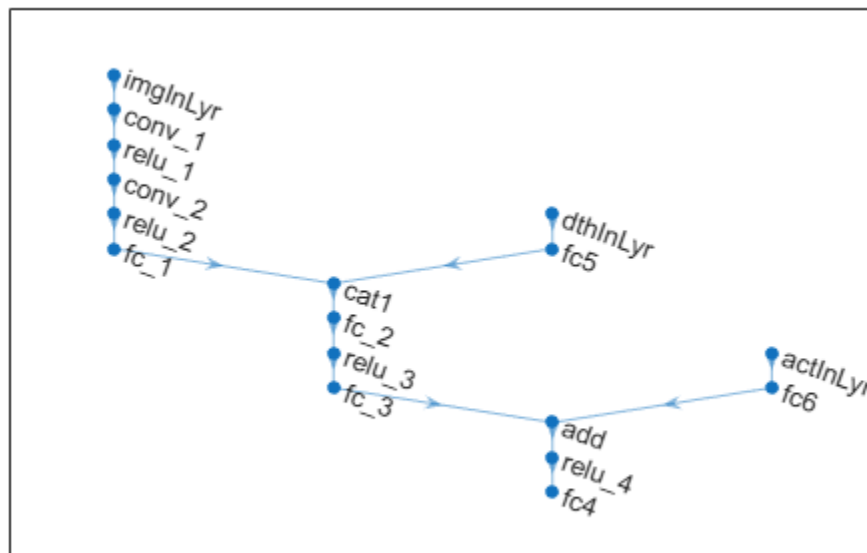
```

criticNetwork = dlnetwork();
criticNetwork = addLayers(criticNetwork,imgPath);
criticNetwork = addLayers(criticNetwork,dthPath);
criticNetwork = addLayers(criticNetwork,actPath);
criticNetwork = connectLayers(criticNetwork,"fc5","cat1/in2");
criticNetwork = connectLayers(criticNetwork,"fc6","add/in2");

```

View the critic network configuration and display the number of parameters.

```
plot(criticNetwork)
```



Initialize the network and display the number of parameters.


```
criticNetwork = initialize(criticNetwork);
summary(criticNetwork)
```

```
Initialized: true
```

```
Number of learnables: 81.2k
```

```
Inputs:
  1  'imgInLyr'   50×50×1 images
  2  'dthInLyr'   1 features
  3  'actInLyr'   1 features
```

Create the critic using the specified neural network and the environment action and observation specifications. Pass as additional arguments also the names of the network layers to be connected with the observation and action channels. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(criticNetwork, ...
    obsInfo, actInfo, ...
    ObservationInputNames=["imgInLyr", "dthInLyr"], ...
    ActionInputNames="actInLyr");
```

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is implemented by a continuous deterministic actor. A continuous deterministic actor implements a parameterized deterministic policy for a continuous action space. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with two input layers (receiving the content of the two environment observation channels, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`).

Define the network as an array of layer objects.

```
% Image input path
imgPath = [
    imageInputLayer(obsInfo(1).Dimension, ...
        Name="imgInLyr")
    convolution2dLayer(5,8,Stride=3,Padding=0)
    reluLayer
    convolution2dLayer(5,8,Stride=3,Padding=0)
    reluLayer
    fullyConnectedLayer(32)
    concatenationLayer(1,2,Name="cat1")
    fullyConnectedLayer(hiddenLayerSize1)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize2)
    reluLayer
    fullyConnectedLayer(1)
    tanhLayer
    scalingLayer(Name="scale1", ...
        Scale=max(actInfo.UpperLimit))
];

% d(theta)/dt input layer
dthPath = [
    featureInputLayer(prod(obsInfo(2).Dimension), ...
```

```

        Name="dthInLyr")
    fullyConnectedLayer(1, ...
        Name="fc5", ...
        BiasLearnRateFactor=0, ...
        Bias=0)
];

```

Assemble `dlnetwork` object.

```

actorNetwork = dlnetwork();
actorNetwork = addLayers(actorNetwork,imgPath);
actorNetwork = addLayers(actorNetwork,dthPath);
actorNetwork = connectLayers(actorNetwork,"fc5","cat1/in2");

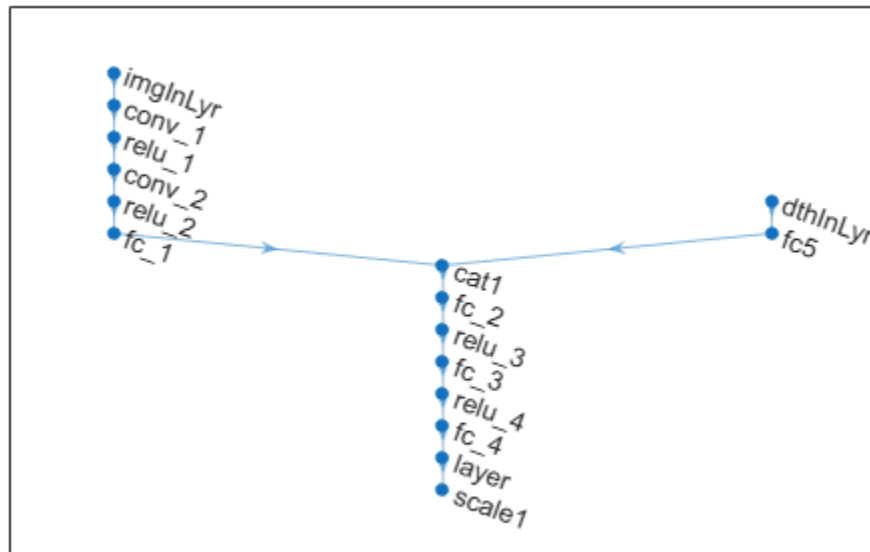
```

View the actor network configuration and display the number of weights.

```

figure
plot(actorNetwork)

```



Initialize the network and display the number of parameters.

```

actorNetwork = initialize(actorNetwork);
summary(actorNetwork)

```

```

    Initialized: true

```

```

    Number of learnables: 80.7k

```

```

    Inputs:

```

```

        1  'imgInLyr'   50x50x1 images
        2  'dthInLyr'   1 features

```

Create the actor using the specified neural network. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNetwork, ...
    obsInfo,actInfo, ...
    ObservationInputNames=["imgInLyr","dthInLyr"]);
```

Specify options for the actor and critic using `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-03, ...
    GradientThreshold=1);
actorOptions = rlOptimizerOptions( ...
    LearnRate=1e-04, ...
    GradientThreshold=1);
```

Training performance using the GPU is impacted by the batch size, network structure, and the hardware itself. Therefore, using a GPU does not always guarantee a better training performance. For more information on supported GPUs, see “GPU Computing Requirements” (Parallel Computing Toolbox).

Set `UseGPUCritic` to true to train the critic using a GPU.

```
UseGPUCritic = false;
if canUseGPU && UseGPUCritic
    critic.UseDevice = "gpu";
end
```

Set `UseGPUActor` to true to train the actor using a GPU.

```
UseGPUActor = false;
if canUseGPU && UseGPUActor
    actor.UseDevice = "gpu";
end
```

Fix the random generator seed used on the GPU for reproducibility.

```
if canUseGPU && (UseGPUCritic || UseGPUActor)
    gpurng(0)
end
```

Specify the DDPG agent options using `rlDDPGAgentOptions`.

```
agentOptions = rlDDPGAgentOptions( ...
    SampleTime=env.Ts, ...
    TargetSmoothFactor=1e-3, ...
    ExperienceBufferLength=1e6, ...
    DiscountFactor=0.99, ...
    MiniBatchSize=128);
```

You can also specify options using dot notation.

```
agentOptions.NoiseOptions.StandardDeviation = 0.6;
agentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-6;
agentOptions.NoiseOptions.StandardDeviationMin = 0.1;
```

Specify the training options for the function approximator objects.

```
agentOptions.CriticOptimizerOptions = criticOptions;
agentOptions.ActorOptimizerOptions = actorOptions;
```

Then create the agent using the specified actor, critic, and agent options. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic,agentOptions);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run each training for a maximum of 5000 episodes, with each episode lasting a maximum of 400 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the Plots option).
- Stop the training when the agent receives an evaluation statistic greater than -740. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.

For more information on training options, see `rlTrainingOptions`.

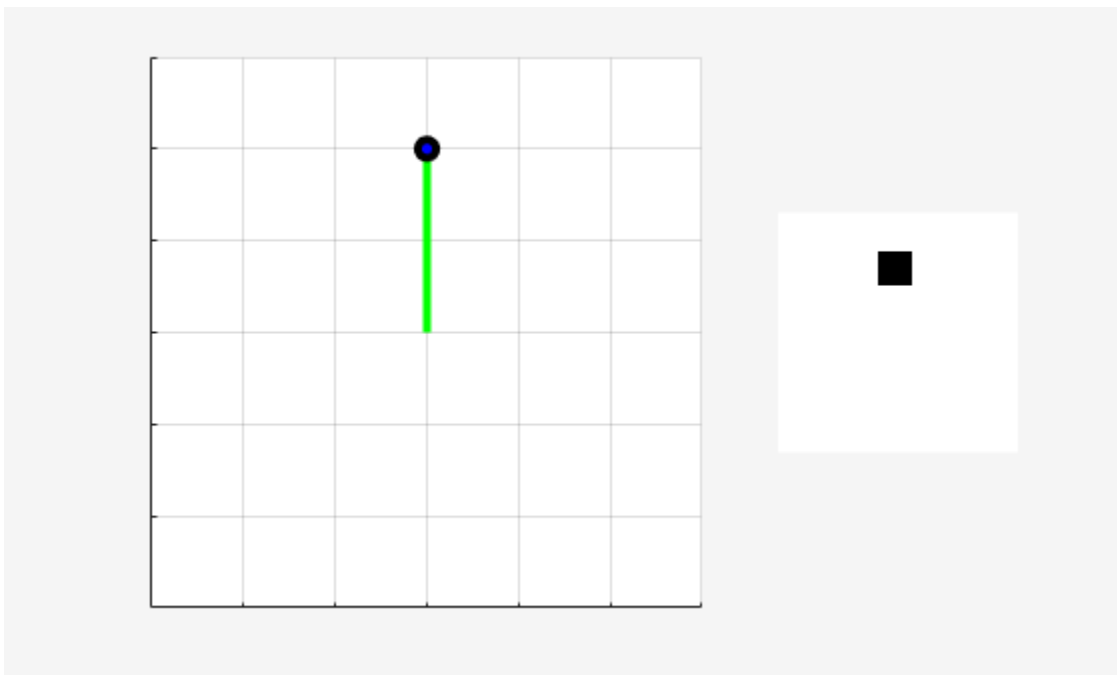
```
maxepisodes = 5000;
maxsteps = 400;
trainingOptions = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-740);
```

Create an evaluator to evaluate the agent at every 50 training episodes.

```
evl = rlEvaluator(EvaluationFrequency=50, NumEpisodes=1);
```

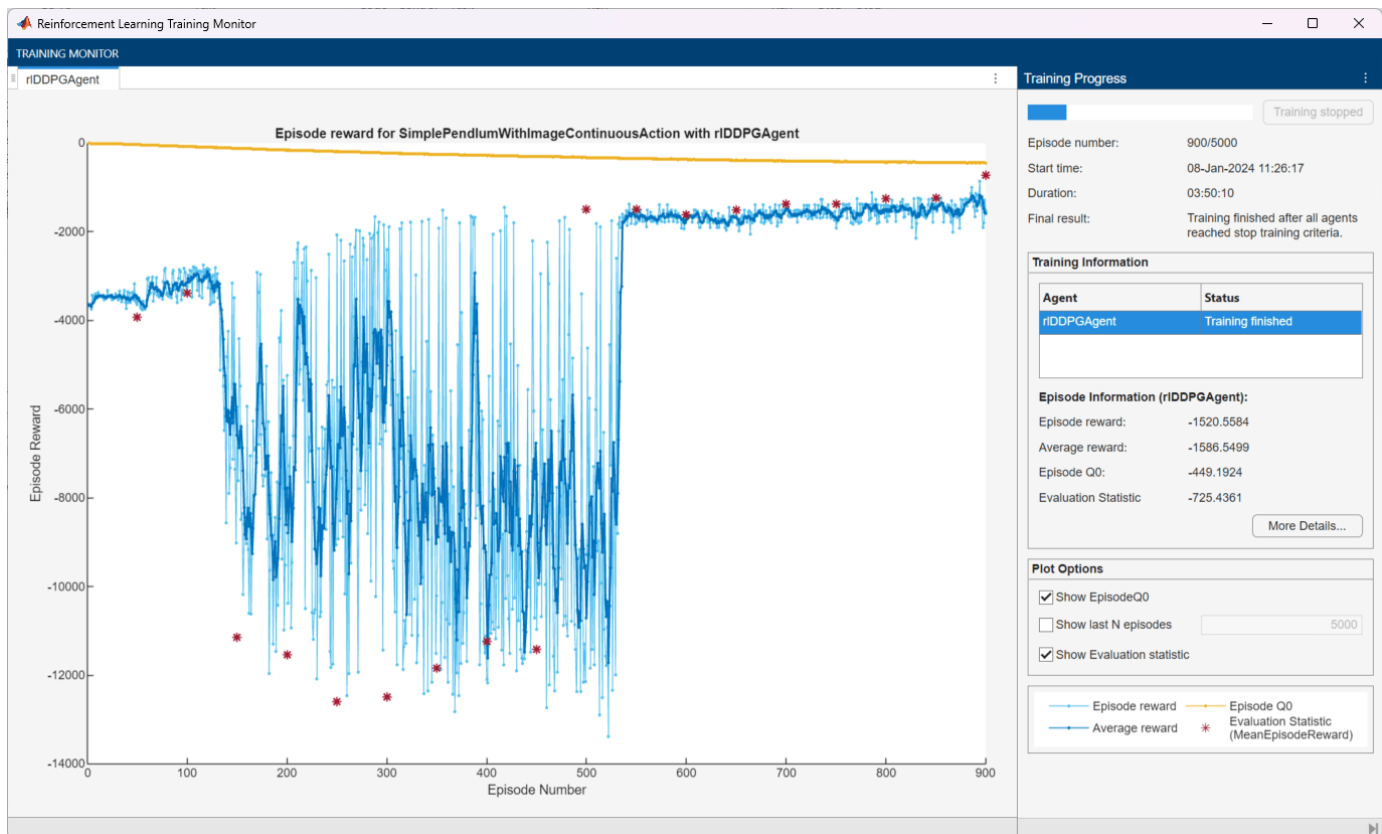
You can visualize the pendulum by using the `plot` function during training or simulation.

```
plot(env)
```



Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainingOptions, Evaluator=evl);
else
    % Load pretrained agent for the example.
    load("SimplePendulumWithImageDDPG.mat","agent")
end
```



Simulate DDPG Agent

To validate the performance of the trained agent, simulate it within the pendulum environment.

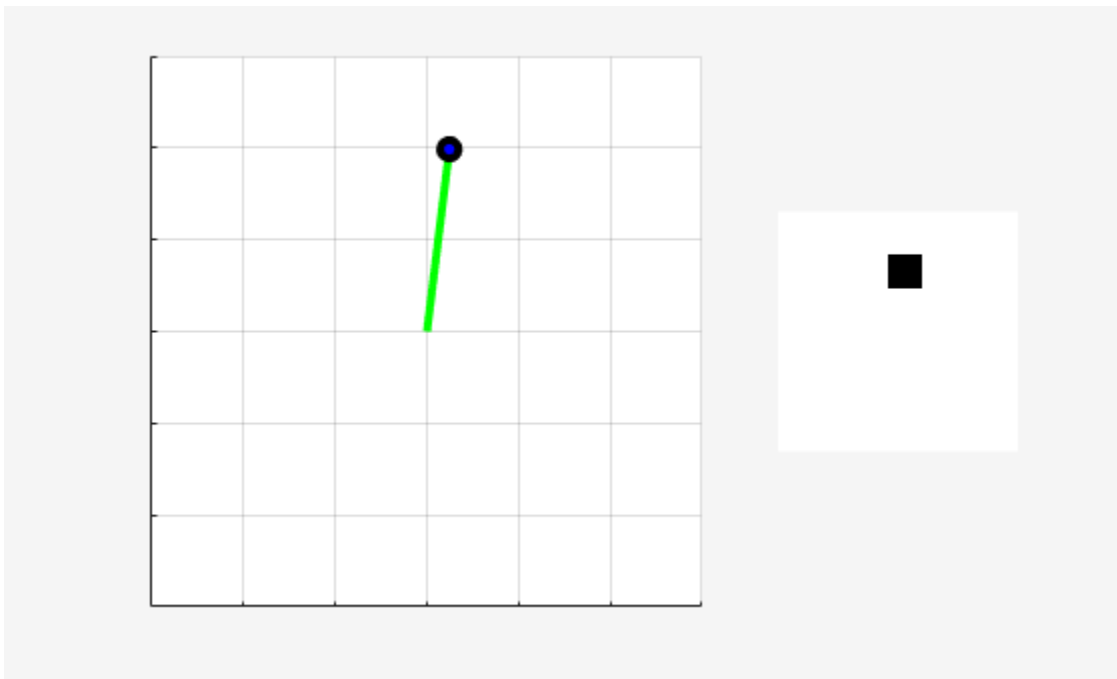
Fix the random number stream for reproducibility.

```
rng(0,"twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Run a simulation episode with 500 steps. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);  
experience = sim(env,agent,simOptions);
```



Display the total reward.

```
totalRwd = sum(experience.Reward)
```

```
totalRwd =  
-727.1931
```

See Also

Functions

`train` | `sim`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal” on page 5-141

More About

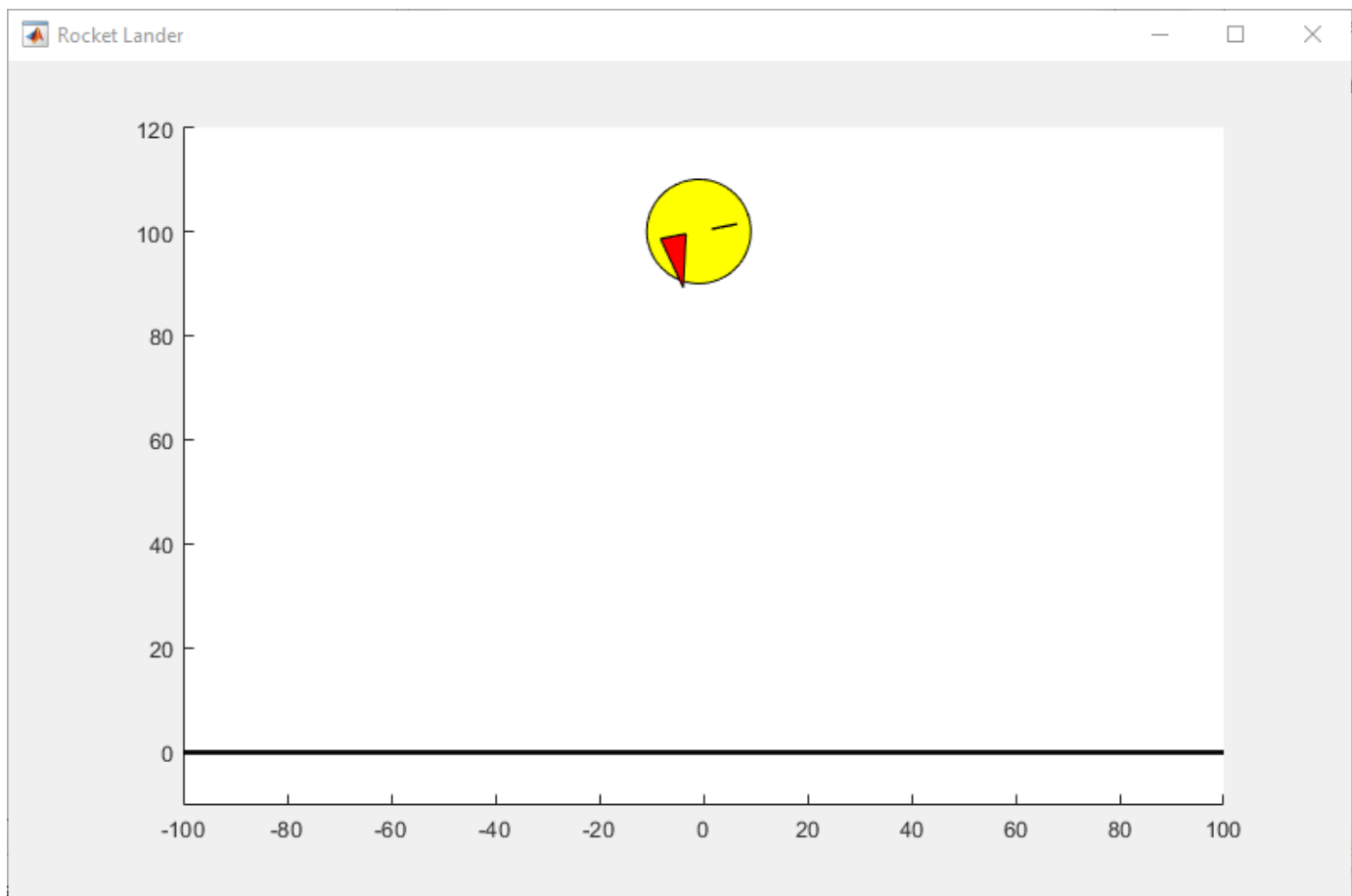
- “Use Predefined Control System Environments” on page 2-42
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Train Soft Actor Critic Agent with Custom Networks for Discrete Lander Vehicle

This example shows how to train a discrete soft actor critic (DSAC) agent with a discrete action space to land an airborne vehicle on the ground. For more information on DSAC agents, see “Soft Actor-Critic (SAC) Agent” on page 3-78.

Overview

The soft actor critic (SAC) is a stochastic off-policy reinforcement learning algorithm [1] on page 5-140 and [2] on page 5-140. Because SAC is an off-policy algorithm, it can learn from the data stored over the episodes, which results in better sample efficiency and robustness. In this example, you train a SAC agent with discrete action space to solve a challenging lander environment. For more details on the lander environment, see “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93.



Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with a seed 0, using the Mersenne Twister random number algorithm. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

Create the environment object using the `LanderVehicle` class provided in the example folder.

```
env = LanderVehicle

env =
    LanderVehicle with properties:

        Mass: 1
        L1: 10
        L2: 5
        Gravity: 9.8060
        ThrustLimits: [0 8.5000]
        Ts: 0.1000
        State: [6×1 double]
        LastAction: [2×1 double]
        LastShaping: 0
        DistanceIntegral: 0
        VelocityIntegral: 0
        TimeCount: 0
```

Get the observation and action specifications for the environment.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Create Discrete SAC Agent with Custom Networks

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Discrete SAC agents use one or two vector Q-value function critics. This example uses two critics to mitigate overestimation in the critic target.

A discrete vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible discrete actions. The value of each output element represents the expected discounted cumulative long-term reward for taking the action corresponding to the element number, from the state corresponding to the current observation, and following the policy afterwards.

To model the parameterized vector Q-value function within the critics, use a neural network with one input layer (receiving the content of the observation channel, as specified by `obsInfo`) and one output layer (returning the vector of values for all the possible actions, as specified by `actInfo`).

Define the network for the critics as an array of layer objects. Note that `numel(actInfo.Elements)` returns the number of elements of the discrete action space.

```
hiddenLayerSize_Critic = [400, 300];
criticNetLayers = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(hiddenLayerSize_Critic(1))
    reluLayer
    fullyConnectedLayer(hiddenLayerSize_Critic(2))
    reluLayer
    fullyConnectedLayer(numel(actInfo.Elements))
];
```

Create the `dlnetwork` object `criticNet1` and display the number of weights.

```
criticNet1 = dlnetwork(criticNetLayers);
summary(criticNet1)
```

```
Initialized: true

Number of learnables: 126.2k

Inputs:
    1  'input'    7 features
```

Create the first critic approximator object using `criticNet1`, the observation specification and the action specification. For more information on Q-value function approximator, see `rlVectorQValueFunction`.

```
critic1 = rlVectorQValueFunction(criticNet1,obsInfo,actInfo);
```

Create the `dlnetwork` object `criticNet2`.

```
criticNet2 = dlnetwork(criticNetLayers);
```

Create the second critic approximator object using `criticNet2`, the observation specification and the action specification.

```
critic2 = rlVectorQValueFunction(criticNet2,obsInfo,actInfo);
```

The discrete SAC agents use a parameterized stochastic policy, which for discrete action spaces is implemented by a discrete categorical actor. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer. The output layer must return a vector of probabilities for each possible actions, as specified by `actInfo`.

Define the network as an array of layer objects.

```
hiddenLayerSize_Actor = [400, 300];
actorNetLayers = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(hiddenLayerSize_Actor(1))
    reluLayer
    fullyConnectedLayer(hiddenLayerSize_Actor(2))
    reluLayer
    fullyConnectedLayer(numel(actInfo.Elements))
];
```

Convert the array of layers to a `dlnetwork` object and display the number of weights.

```
actorNet = dlnetwork(actorNetLayers);
summary(actorNet)
```

```
Initialized: true

Number of learnables: 126.2k

Inputs:
  1  'input'  7 features
```

Create the actor using `actorNet` and the observation and action specifications. For more information on discrete categorical actors, see `rlDiscreteCategoricalActor`.

```
actor = rlDiscreteCategoricalActor(actorNet,obsInfo,actInfo);
```

Update the actor and critic neural networks using the Adam algorithm with a learning rate of $1e-3$ and gradient threshold of 1. To create optimizer options for the actor and critic, use `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions( ...
    Optimizer="adam", ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=2e-4);

actorOptions = rlOptimizerOptions( ...
    Optimizer="adam", ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-5);
```

Use a buffer of size $1e6$ to store experiences, and a mini-batch of size 300 to update the critics and actor networks. The discount factor and target smoothing factor are to 0.99 and $1e-3$, respectively. For a list of available DSAC agent hyperparameters, see `rlSACAgentOptions`.

Set the agent options.

```
agentOptions = rlSACAgentOptions;
agentOptions.SampleTime = env.Ts;
agentOptions.DiscountFactor = 0.99;
agentOptions.TargetSmoothFactor = 1e-3;
agentOptions.ExperienceBufferLength = 1e6;
agentOptions.MiniBatchSize = 300;
agentOptions.EntropyWeightOptions.EntropyWeight = 0.05;
agentOptions.EntropyWeightOptions.TargetEntropy = 0.5;
```

Set optimizer options for the actor and critic.

```
agentOptions.CriticOptimizerOptions = criticOptions;
agentOptions.ActorOptimizerOptions = actorOptions;
```

Create the discrete SAC agent.

```
agent = rlSACAgent(actor,[critic1 critic2],agentOptions);
```

Train Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Create the evaluator object to run 5 evaluation episodes every 20 training episodes. For more information, see `rlEvaluator`.

```
evl = rlEvaluator(EvaluationFrequency=20, NumEpisodes=5);
```

To train the agent, first specify the training options using `rlTrainingOptions`. For this example, use the following options:

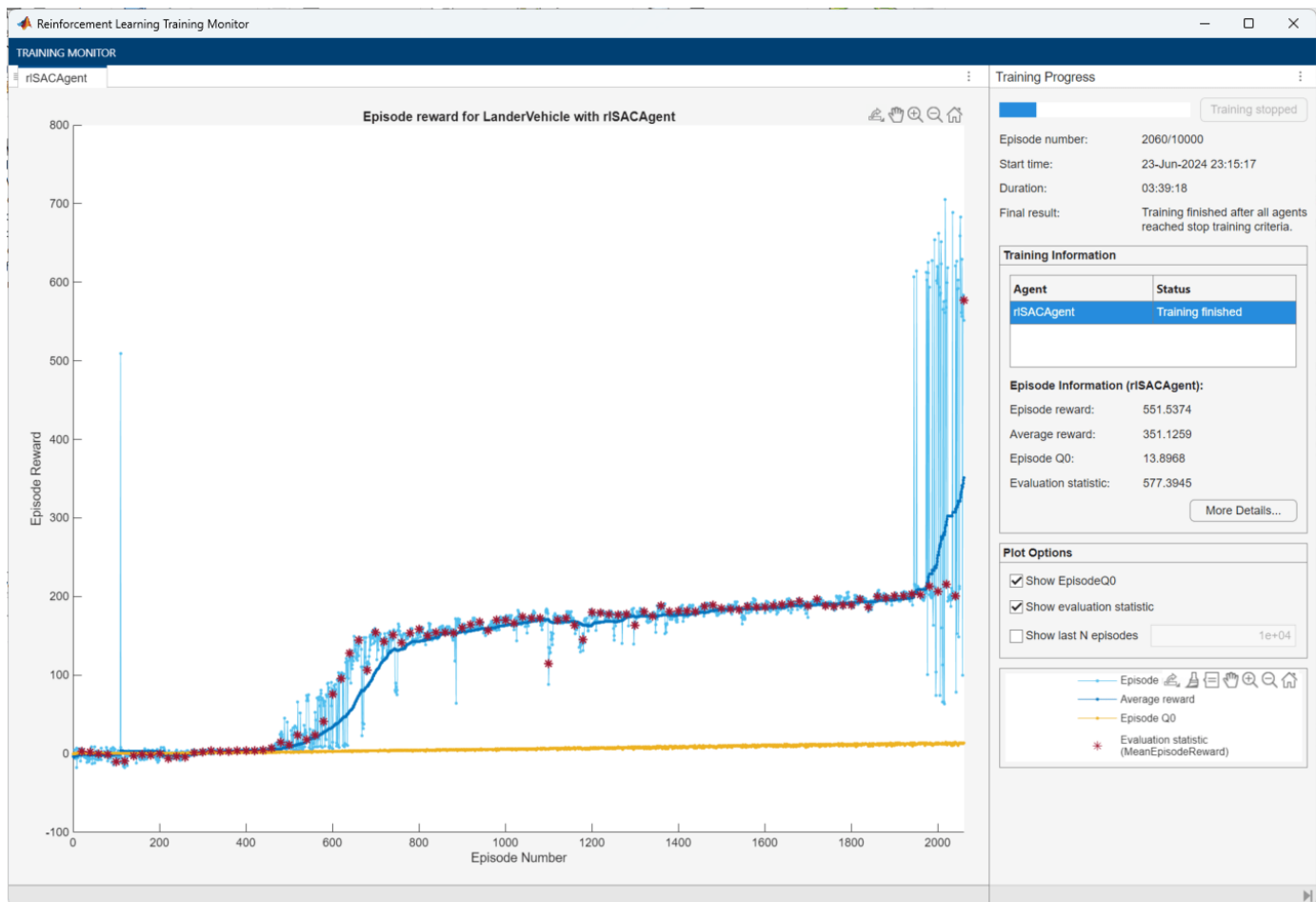
- Run each training for a maximum of 5000 episodes, with each episode lasting at most 600 time steps.
- To speed up training, set the `UseParallel` option to `true`, which requires Parallel Computing Toolbox™ software. If you do not have the software installed set the option to `false`.
- Stop the training when the average cumulative reward over the evaluation episodes reaches 450.

```
trainOpts = rlTrainingOptions;  
trainOpts.Plots = 'training-progress';  
trainOpts.MaxEpisodes = 10000;  
trainOpts.MaxStepsPerEpisode = 600;  
trainOpts.Verbose = 1;  
trainOpts.StopTrainingCriteria = 'EvaluationStatistic';  
trainOpts.StopTrainingValue = 450;  
trainOpts.UseParallel = true;  
trainOpts.ScoreAveragingWindowLength = 100;
```

Train the agent. Due to the complexity of the environment, the training process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. For more information on training agents, see `train` and “Train Reinforcement Learning Agents” on page 5-3.

```
doTraining = ;  
if doTraining  
    trainingStats = train(agent, env, trainOpts, Evaluator=evl);  
else  
    load("landerVehicleDSACAgent.mat");  
end
```

An example training session is shown below. Your results can differ because of randomness in the training process.



Simulate Trained Agent

Set the random seed for simulation reproducibility.

```
rng(0, "twister");
```

Plot the environment first to create a visualization for the lander vehicle.

```
plot(env)
```

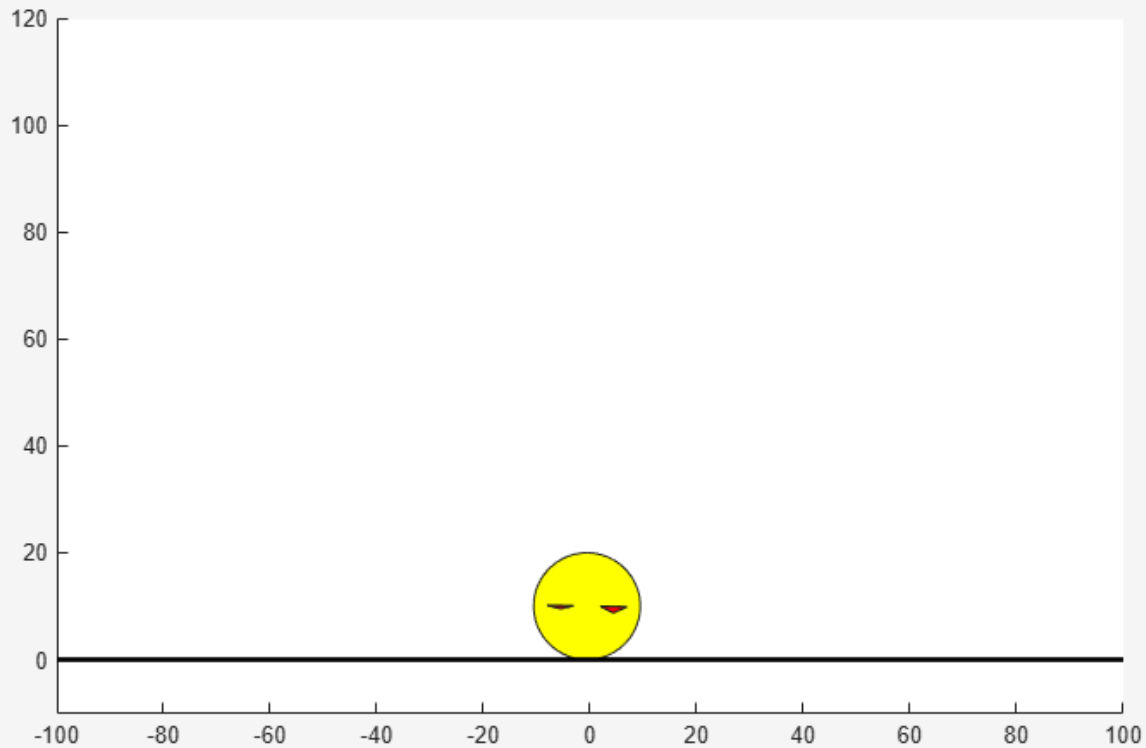
Set up the number of steps in simulation and the number of simulations in one episode. For more information, see `rlSimulationOptions`.

```
simOptions = rlSimulationOptions(MaxSteps=600);
simOptions.NumSimulations = 10;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to true.

Simulate the trained agent within the environment. For more information, see `sim`.

```
experience = sim(env, agent, simOptions);
```



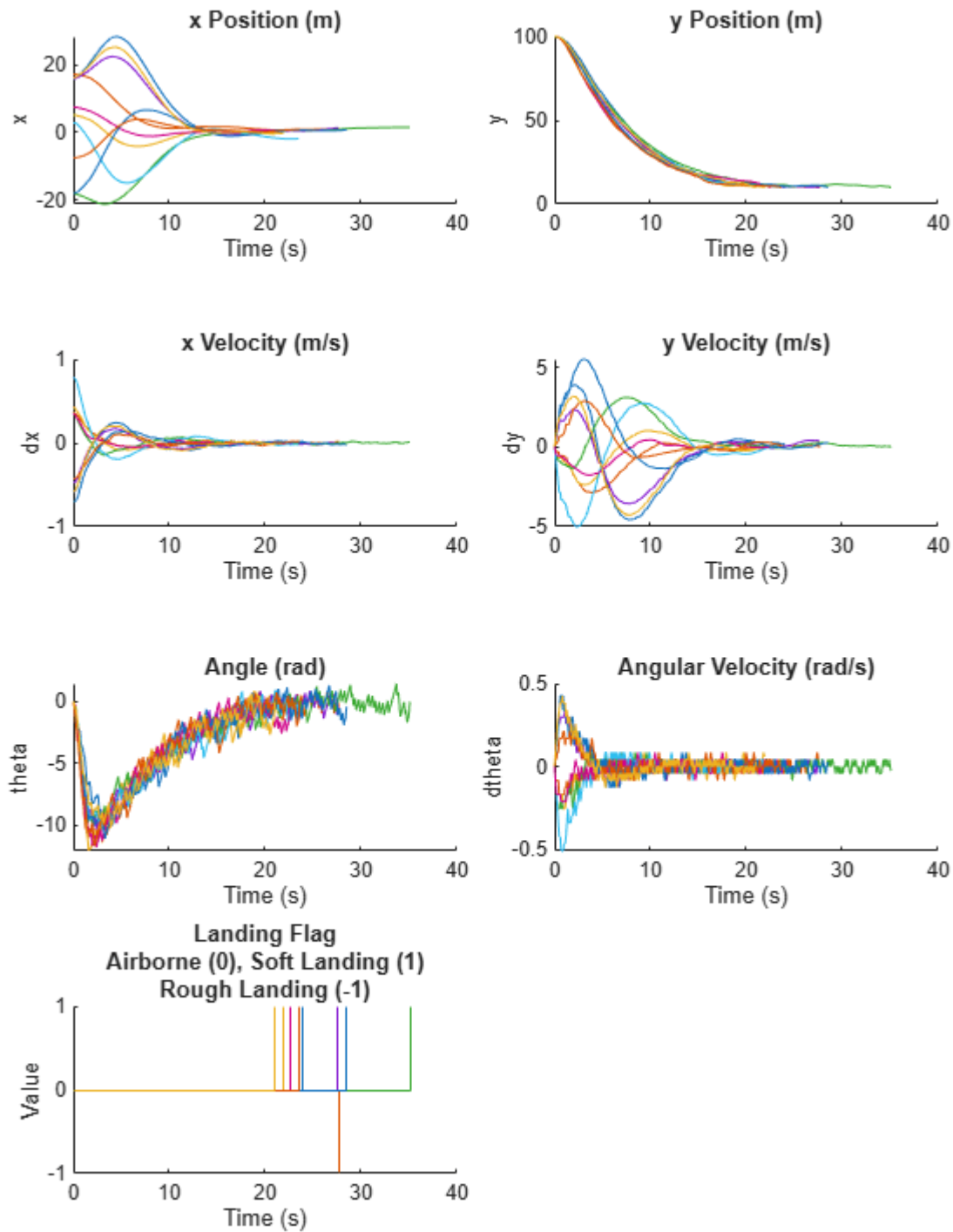
Plot the time history of the states for all simulations using the helper function `plotLanderVehicleTrajectory`, provided in the example folder.

```
% Observations to plot
obsToPlot = ["x","y","dx","dy","theta","dtheta","landing"];

% Create a figure
f = figure();
f.Position(3:4) = [800,1000];

% Create a tiled layout for the plots
t = tiledlayout(f,4,2,TileSpacing="compact");

% Plot the data
for ct = 1:numel(obsToPlot)
    ax = nexttile(t);
    plotLanderVehicleTrajectory(ax,experience,env,obsToPlot(ct));
end
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

[1] Haarnoja, Tuomas, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, et al. "Soft Actor-Critic Algorithms and Application." Preprint, submitted January 29, 2019. <https://arxiv.org/abs/1812.05905>.

[2] Christodoulou, Petros. "Soft Actor-Critic for Discrete Action Settings." Preprint, submitted October 18, 2019. <https://arxiv.org/abs/1910.07207>.

See Also

Functions

`train` | `sim`

Objects

`rlVectorQValueFunction` | `rlDiscreteCategoricalActor` | `rlSACAgent` |
`rlSACAgentOptions` | `rlEvaluator` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- "Train Default PPO Agent for Discrete Lander Vehicle" on page 5-93
- "Train SAC Agent for Ball Balance Control" on page 8-181
- "Train DDPG Agent to Control Two-Thruster Sliding Vehicle" on page 8-22
- "Train Multiple Agents to Perform Collaborative Task" on page 8-31

More About

- "Reinforcement Learning Environments" on page 2-2
- "Soft Actor-Critic (SAC) Agent" on page 3-78
- "Train Reinforcement Learning Agents" on page 5-3

Train DDPG Agent to Swing Up and Balance Pendulum with Bus Signal

This example shows how to convert a simple frictionless pendulum Simulink® model to a reinforcement learning environment object, and how to train a deep deterministic policy gradient (DDPG) agent in this environment.

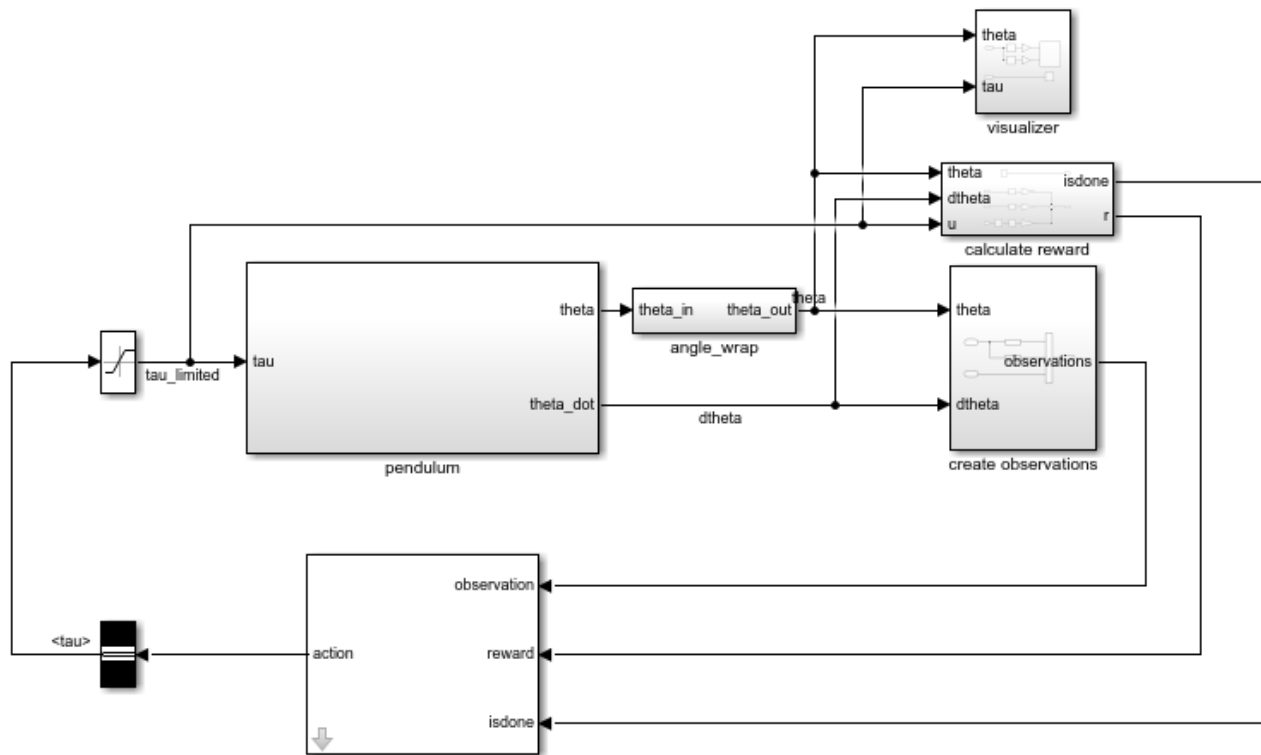
For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67. For an example showing how to train a DDPG agent in MATLAB®, see “Compare DDPG Agent to LQR Controller” on page 7-285.

Pendulum Swing-Up Model with Bus

The starting model for this example is a simple frictionless pendulum. The training goal is to make the pendulum stand upright using minimal control effort.

Open the model.

```
mdl = "rlSimplePendulumModelBus";
open_system(mdl)
```



In this model:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.

- The torque action signal from the agent to the environment is from -2 to 2 N·m.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- Both the observation and action signals are Simulink buses.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right).$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

The model used in this example is similar to the simple pendulum model described in “Use Predefined Control System Environments” on page 2-42. The difference is that the model in this example uses Simulink buses for the action and observation signals.

Create Environment Interface with Bus

The environment object from a Simulink model is created using `rlSimulinkEnv`, which requires the name of the Simulink model, the path to the agent block, and observation and action reinforcement learning data specifications. For models that use bus signals for actions or observations, you can create the corresponding specifications using the `bus2RLSpec` function.

Specify the path to the agent block.

```
agentBlk = "rlSimplePendulumModelBus/RL Agent";
```

Create the observation Bus object. The channel names must correspond to the signal names specified in the Simulink buses used to create the observation and action signals.

```
obsBus = Simulink.Bus();  
obs(1) = Simulink.BusElement;  
obs(1).Name = "sin_theta";  
obs(2) = Simulink.BusElement;  
obs(2).Name = "cos_theta";  
obs(3) = Simulink.BusElement;  
obs(3).Name = "dtheta";  
obsBus.Elements = obs;
```

Create the action Bus object.

```
actBus = Simulink.Bus();  
act(1) = Simulink.BusElement;  
act(1).Name = "tau";  
act(1).Min = -2;  
act(1).Max = 2;  
actBus.Elements = act;
```

Create the action and observation specification objects using the Simulink buses.

```
obsInfo = bus2RLSpec("obsBus", "Model", mdl)
```

```
obsInfo=1x3 rlNumericSpec array with properties:
    LowerLimit
    UpperLimit
    Name
    Description
    Dimension
    DataType
```

```
actInfo = bus2RLSpec("actBus","Model",mdl)
```

```
actInfo =
    rlNumericSpec with properties:
        LowerLimit: -2
        UpperLimit: 2
        Name: "tau"
        Description: ""
        Dimension: [1 1]
        DataType: "double"
```

Create the reinforcement learning environment for the pendulum model.

```
env = rlSimulinkEnv(mdl,agentBlk,obsInfo,actInfo);
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle.

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the reset function uses the `setVariable` (Simulink) function to set the model workspace variable `theta0` to `pi`. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in)setVariable(in,"theta0",pi,"Workspace",mdl);
```

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;
Tf = 20;
```

Fix the random generator seed for reproducibility.

```
rng(0)
```

Create DDPG Agent

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with three input layers (each receiving one scalar observation, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`).

Define the network as an array of layer objects, and combine the outputs from the input layers using a `concatenationLayer`. Because the output of `tanhLayer` is limited between -1 and 1, scale the network output to the range of the action using `scalingLayer`.

For more information on creating actors based on deep neural networks, see “Create Actors, Critics, and Policy Objects” on page 4-2.

```
% Define input layers.
sinThetaInputLayer = featureInputLayer(1,Name="sthLyr");
cosThetaInputLayer = featureInputLayer(1,Name="cthLyr");
dThetaInputLayer   = featureInputLayer(1,Name="dthLyr");

% Define common path.
commonPath = [
    concatenationLayer(1,3,Name="concat")
    fullyConnectedLayer(400)
    reluLayer
    fullyConnectedLayer(300)
    reluLayer
    fullyConnectedLayer(1)
    tanhLayer
    scalingLayer(Scale=max(actInfo.UpperLimit))
];
```

Create `dlnetwork` object and add layers.

```
actorNetwork = dlnetwork;
actorNetwork = addLayers(actorNetwork,sinThetaInputLayer);
actorNetwork = addLayers(actorNetwork,cosThetaInputLayer);
actorNetwork = addLayers(actorNetwork,dThetaInputLayer);
actorNetwork = addLayers(actorNetwork,commonPath);
```

Connect layers.

```
actorNetwork = connectLayers(actorNetwork,"sthLyr","concat/in1");
actorNetwork = connectLayers(actorNetwork,"cthLyr","concat/in2");
actorNetwork = connectLayers(actorNetwork,"dthLyr","concat/in3");
```

Initialize network and display the number of weights.

```
actorNetwork = initialize(actorNetwork);
summary(actorNetwork)
```

```
    Initialized: true
```

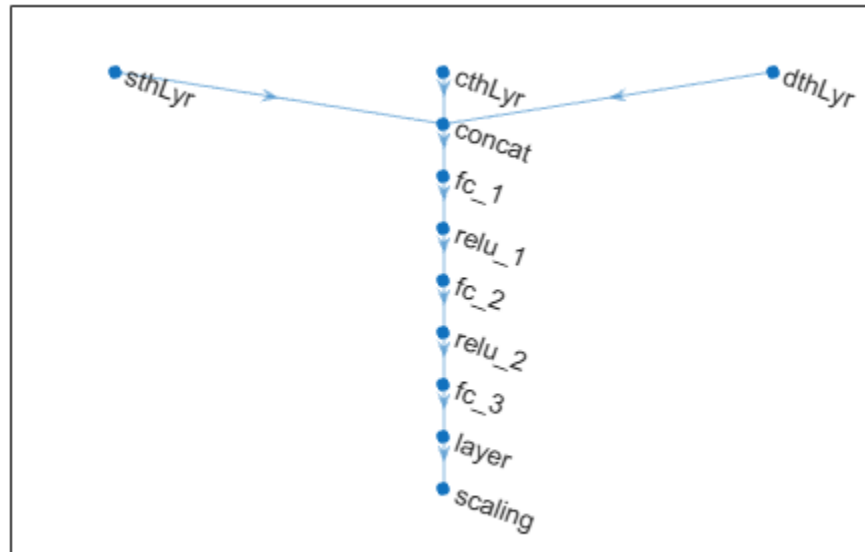
```
    Number of learnables: 122.2k
```

```
    Inputs:
```

```
      1  'sthLyr'    1 features
      2  'cthLyr'    1 features
      3  'dthLyr'    1 features
```

View the actor network configuration.

```
plot(actorNetwork)
```



Create the actor using the specified deep neural network. Specify the action and observation info for the actor, as well as the names of the network input layers to be connected with the observation channels. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNetwork,obsInfo,actInfo, ...
    "ObservationInputNames",["sthLyr","cthLyr","dthLyr"]);
```

DDPG agents use a parameterized Q-value function approximator to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated value of the policy).

To model the parameterized Q-value function within the critic, use a neural network with four input layers (three for the observation channels, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value).

Define each network path as an array of layer objects, assigning names to the input and output layers of each path. Use the input layers previously defined for the actor.

% Define observation path.

```
obsPath = [
    concatenationLayer(1,3,Name="concat")
    fullyConnectedLayer(400)
    reluLayer
    fullyConnectedLayer(300,Name="obsPathOutLyr")
];
```

% Define action path.

```
actPath = [
    featureInputLayer(1,Name="action")
    fullyConnectedLayer(300, ...
        Name="actPathOutLyr", ...
        BiasLearnRateFactor=0)
];
```

```
% Define common path.
commonPath = [
    additionLayer(2,Name="add")
    reluLayer
    fullyConnectedLayer(1,Name="CriticOutput")
];
```

Create dlnetwork object and add layers.

```
criticNetwork = dlnetwork;
criticNetwork = addLayers(criticNetwork,sinThetaInputLayer);
criticNetwork = addLayers(criticNetwork,cosThetaInputLayer);
criticNetwork = addLayers(criticNetwork,dThetaInputLayer);
criticNetwork = addLayers(criticNetwork,actPath);
criticNetwork = addLayers(criticNetwork,obsPath);
criticNetwork = addLayers(criticNetwork,commonPath);
```

Connect layers.

```
criticNetwork = connectLayers(criticNetwork,"sthLyr","concat/in1");
criticNetwork = connectLayers(criticNetwork,"cthLyr","concat/in2");
criticNetwork = connectLayers(criticNetwork,"dthLyr","concat/in3");
criticNetwork = connectLayers(criticNetwork,"obsPathOutLyr","add/in1");
criticNetwork = connectLayers(criticNetwork,"actPathOutLyr","add/in2");
```

Initialize network and display the number of weights.

```
criticNetwork = initialize(criticNetwork);
summary(criticNetwork)
```

```
    Initialized: true
```

```
    Number of learnables: 122.8k
```

```
Inputs:
```

```
 1 'sthLyr' 1 features
 2 'cthLyr' 1 features
 3 'dthLyr' 1 features
 4 'action' 1 features
```

Create the critic approximator object using `criticNet`, the environment observation and action specifications, and the names of the network input layers to be connected with the environment observation and action channels. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(criticNetwork, ...
    obsInfo,actInfo, ...
    ObservationInputNames=["sthLyr","cthLyr","dthLyr"], ...
    ActionInputNames="action");
```

Specify options for the critic and actor using `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions(LearnRate=1e-03,GradientThreshold=1);
actorOptions = rlOptimizerOptions(LearnRate=1e-03,GradientThreshold=1);
```

To create the DDPG agent, first specify the DDPG agent options using `rlDDPGAgentOptions`.

```
agentOpts = rlDDPGAgentOptions( ...
    SampleTime=Ts, ...
```

```

    CriticOptimizerOptions=criticOptions, ...
    ActorOptimizerOptions=actorOptions, ...
    ExperienceBufferLength=1e5, ...
    DiscountFactor=0.99, ...
    MiniBatchSize=128);
agentOpts.NoiseOptions.StandardDeviation = 0.6;

```

Then create the DDPG agent using the specified actor representation, critic representation, and agent options. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic,agentOpts);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run each training for a maximum of 50,000 episodes, with each episode lasting a maximum of `ceil(Tf/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the agent receives an average cumulative reward greater than -740 when evaluating the deterministic policy. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.
- Save a copy of the agent for each episode where the cumulative reward is greater than -740.

For more information on training options, see `rlTrainingOptions`.

```

maxepisodes = 5000;
maxsteps = ceil(Tf/Ts);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    ScoreAveragingWindowLength=5, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-740);

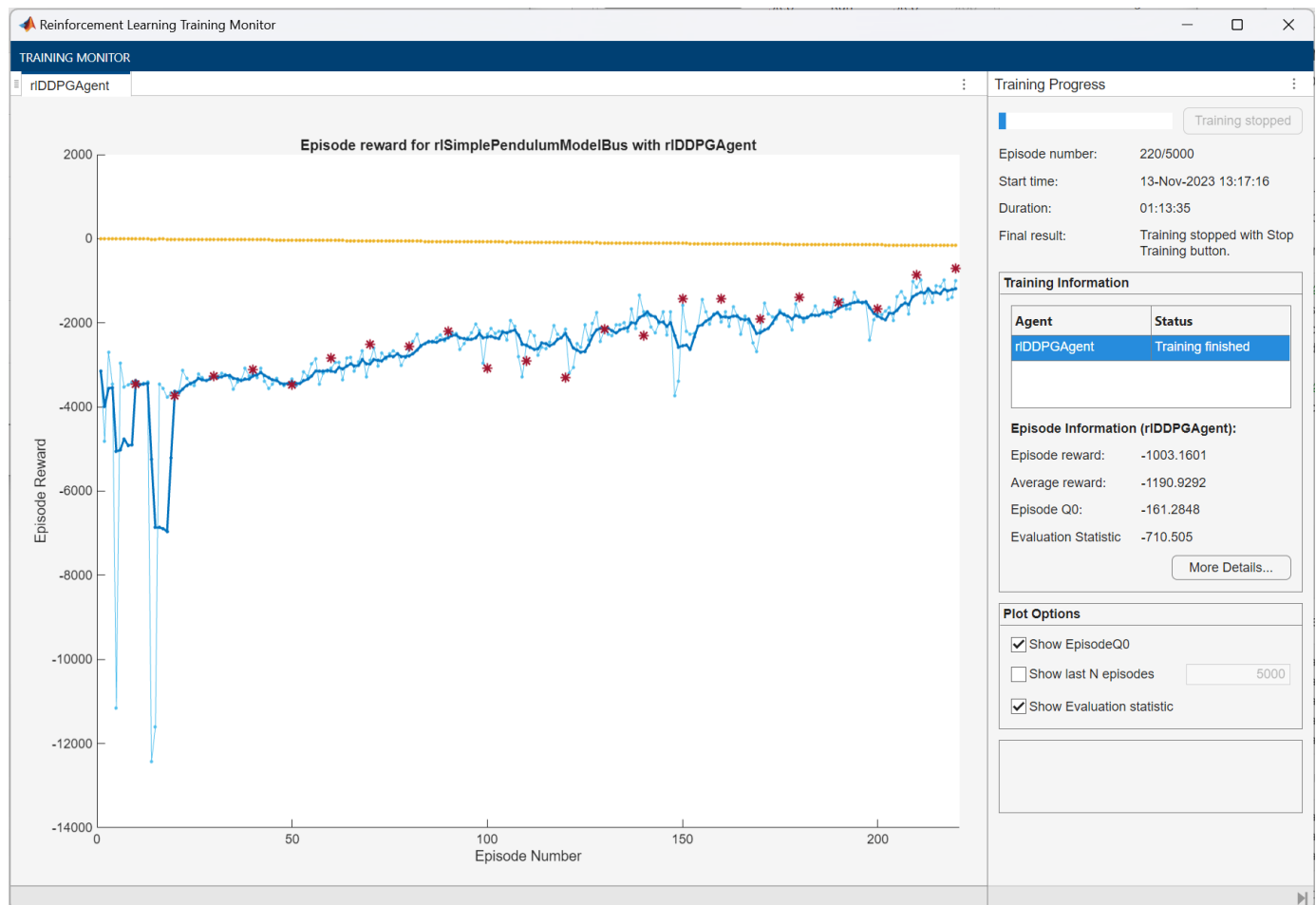
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = false;
if doTraining
    % Use the rlEvaluator object to measure policy performance every 10
    % episodes
    evaluator = rlEvaluator( ...
        NumEpisodes=1, ...
        EvaluationFrequency=10);
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=evaluator);
else
    % Load the pretrained agent for the example.
    load("SimulinkPendBusDDPG.mat","agent")
end

```



Simulate DDPG Agent

To validate the performance of the trained agent, simulate it within the pendulum environment.

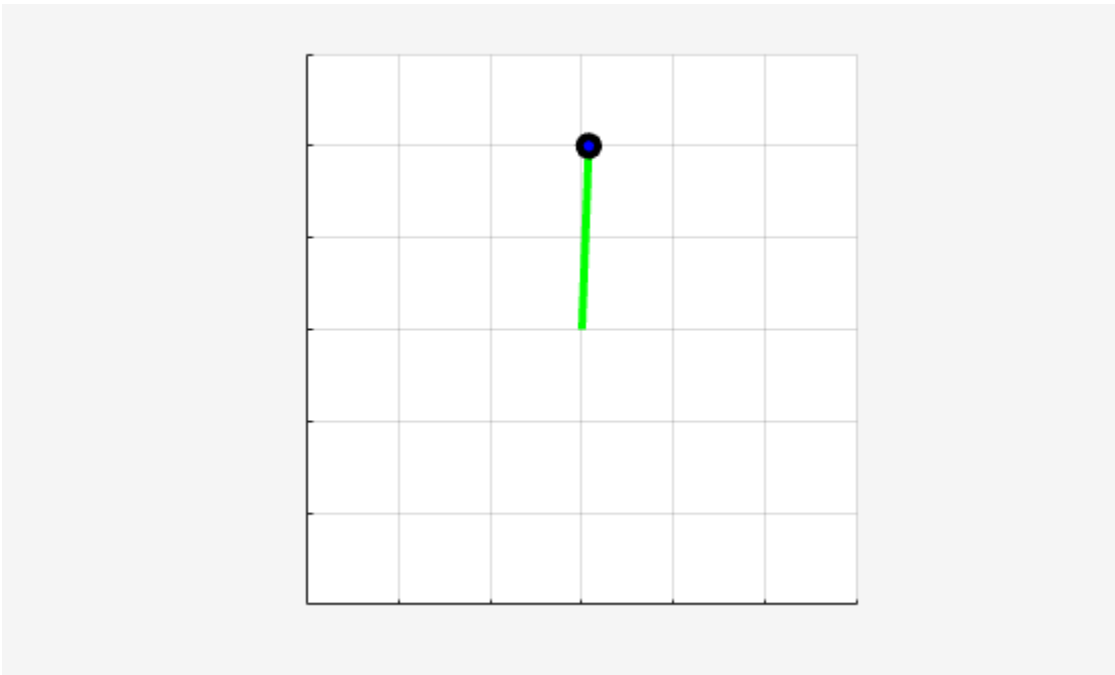
Fix the random number stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the agent. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```

Display the total reward.

```
totalRwd = sum(experience.Reward)
```

```
totalRwd =  
-710.9175
```

See Also

Functions

`rlSimulinkEnv` | `bus2RLSpec` | `train` | `sim`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122

More About

- “Create Custom Simulink Environments” on page 2-110

- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train Agents Using Parallel Computing and GPUs

If you have Parallel Computing Toolbox software, you can run parallel simulations on multicore processors or GPUs. If you additionally have MATLAB Parallel Server software, you can run parallel simulations on computer clusters or cloud resources.

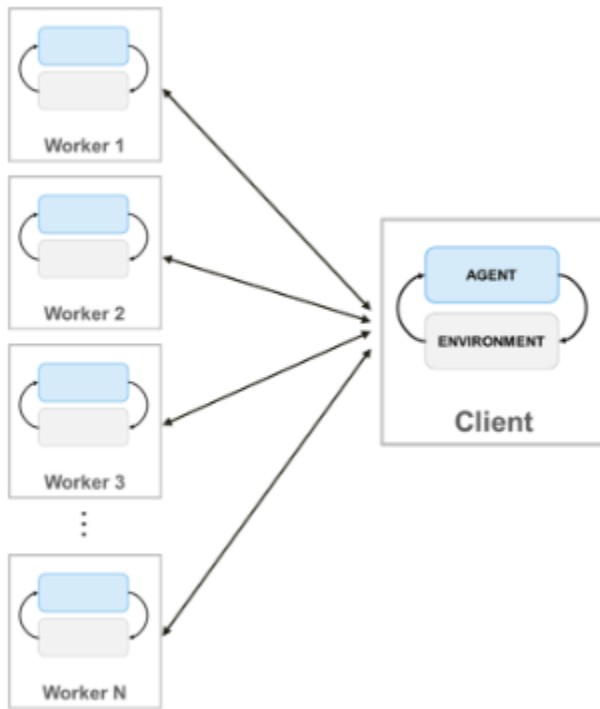
When you use parallel computing to train an agent (by setting the `UseParallel` training option), the training algorithm uses multiple processes to scale up the number of simulations with the environment. For example, multiple parallel workers can generate trajectories (synchronously or asynchronously). The client process then learns from those trajectories. Therefore, with this option, multiple workers can speed up the generation of data for learning. For more information on using parallel computing to scale up simulations, see “Using Multiple Processes” on page 5-151.

You can also use a GPU for learning. This is different from parallelizing simulations. To use a GPU for learning, set the `UseDevice` option to “gpu” in the actor and critic objects of the agent. Generally GPU usage is recommended when you are training from large batches of data, or the data is itself large, like an image. Using GPUs for images can speed up learning. Because there is a cost associated with copying data to and from a GPU, it might not be valuable to use GPU for learning if the data is smaller size. For more information on using GPU for learning, see “Using GPUs” on page 5-154.

Independently on which devices you use to simulate or train the agent, once the agent has been trained, you can generate code to deploy the optimal policy on a CPU or GPU. This is explained in more detail in “Generate Code from Trained Reinforcement Learning Policies” on page 6-2.

Using Multiple Processes

When you train agents using parallel computing, the parallel pool client (the MATLAB process that starts the training) sends copies of both its agent and environment to each parallel worker. Each worker simulates the agent within the environment and sends their simulation data back to the client. The client agent learns from the data sent by the workers and sends the updated policy parameters back to the workers.



Note Parallel training and simulation of agents within multiagent environments is not supported.

Note Due to the non-deterministic nature of parallel execution timing, the results of parallel training cannot in general be reproduced.

To train an agent using multiple processes you must pass to the `train` function an `rlTrainingOptions` object in which the `UseParallel` property is set to `true`.

To create a parallel pool of `N` workers, use the following syntax.

```
pool = parpool(N);
```

If you do not create a parallel pool using `parpool`, the `train` function automatically creates one using your default parallel pool settings. For more information on specifying these settings, see “Specify Your Parallel Settings” (Parallel Computing Toolbox). Note that using a parallel pool of thread workers, such as `pool = parpool("threads")`, is not supported.

For more information on configuring your training to use parallel computing, see the `UseParallel` and `ParallelizationOptions` options in `rlTrainingOptions`. For an example on how to configure options for asynchronous advantage actor-critic (A3C) agent training, see the last example in `rlTrainingOptions`.

For an example that trains an agent using parallel computing in MATLAB, see “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157. For an example that trains an agent using parallel computing in Simulink, see “Train DQN Agent for Lane Keeping Assist Using

Parallel Computing” on page 8-134 and “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144.

Agent-Specific Parallel Training Considerations

Reinforcement learning agents can be trained in parallel in two main ways, experience-based parallelization, in which the workers only calculate experiences, and gradient-based parallelization, in which the workers also calculate the gradients that allow the agent approximators to learn. You do not need to specify when using gradient vs. experience based parallelization, the software automatically selects the parallelization type depending on the agent type. However, you can specify whether to use asynchronous training instead of synchronous training in some cases.

Experience-Based Parallelization (DQN, DDPG, TD3, SAC, PPO, TRPO)

When training an DQN, DDPG, TD3, SAC, PPO or TRPO agent in parallel, the environment simulation is done by the workers and the gradient computation is done by the client. Specifically, the workers simulate (their copy of) the agent within (their copy of) the environment, and send experience data (observation, action, reward, next observation, and a termination signal) to the client. The client then computes the gradients from experiences, updates the agent parameters and sends the updated parameters back to the workers, which then continue to perform simulations using their copy of the updated agent.

This type of parallel training is also known as experience-based parallelization, and *can* run using *asynchronous* training (to specify asynchronous training, set the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function to “`async`”). Asynchronous training can improve performance for off-policy agents.

With asynchronous training, the client agent calculates gradients and updates agent parameters from the received experiences, without waiting to receive experiences from all the workers. The client then sends the updated parameters back to the worker that provided the experiences. Then, while other workers are still running, the worker updates its copy of the agent and continues to generate experiences using its copy of the environment.

Experience-based parallelization can also run using *synchronous* training, (to specify synchronous training, set the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function to “`sync`”). Synchronous training is the default behavior for all agents, and often yields better performance for on-policy agents.

With synchronous training, the client agent waits to receive experiences from all of the workers and then calculates the gradients from all these experiences. The client updates the agent parameters, and sends the updated parameters to all the workers at the same time. Then, all workers use a single updated agent copy, together with their copy of the environment, to generate experiences. Because each worker must pause execution until all the workers are finished, synchronous training only advances as fast as the slowest worker allows.

With either synchronous or asynchronous training, experience-based parallelization can reduce training time only when the computational cost of simulating the environment is high compared to the cost of optimizing network parameters. Otherwise, when the environment simulation is fast enough, some workers might lie idle waiting for the client to learn and send back the updated parameters.

In other words, experience-based parallelization can improve sample efficiency (intended as the number of samples an agent can process within a given time) only when the ratio R between the

environment step complexity and the learning complexity is large. If both environment simulation and gradient computation (that is, learning) are similarly computationally expensive, experience-based parallelization is unlikely to improve sample efficiency. In this case, for off-policy agents that are supported in parallel (DQN, DDPG, TD3, and SAC) you can reduce the mini-batch size to make R larger, thereby improving sample efficiency.

Note For experience-based parallelization, do not use all of your processor cores for parallel training. For example, if your CPU has six cores, train with four workers. Doing so provides more resources for the parallel pool client to compute gradients based on the experiences sent back from the workers.

For an example of experience-based parallel training, see “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134.

Gradient-Based Parallelization (AC and PG)

When training an AC or PG agent in parallel, both the environment simulation and gradient computations are done by the workers. Specifically, workers simulate (their copy of) the agent within (their copy of) the environment, obtain the experiences, compute the gradients from the experiences, and send the gradients to the client. The client averages the gradients, updates the agent parameters and sends the updated parameters back to the workers so they can continue to perform simulations using an updated copy of the agent.

For PG agents gradient-based parallelization *requires* synchronous training. For AC agent, you can still chose either synchronous or asynchronous training. The algorithm used to train an AC agent in asynchronous mode is also referred to as asynchronous advantage AC (A3C).

Synchronous gradient-based parallelization allows you to achieve, in principle, a speed improvement which is nearly linear in the number of workers. However, since each worker must pause execution until all workers are finished, synchronous training only advances as fast as the slowest worker allows.

In general, limiting the number of workers in order to leave some processor cores for the client is not necessary when using gradient-based parallelization, because the gradients are not computed on the client. Therefore, for gradient-based parallelization, it might be beneficial to use all your processor cores for parallel training.

For an example of gradient-based parallel training, see “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157.

Using GPUs

You can speed up training by performing actor and critic operations (such as gradient computation and prediction), on a local GPU rather than a CPU. To do so, when creating a critic or actor, set its `UseDevice` option to "gpu" instead of "cpu".

The "gpu" option requires both Parallel Computing Toolbox software and a CUDA enabled NVIDIA® GPU. For more information on supported GPUs see “GPU Computing Requirements” (Parallel Computing Toolbox).

You can use `gpuDevice` to query or select a local GPU device to be used with MATLAB.

Using GPUs is likely to be beneficial when you have a deep neural network in the actor or critic which has large batch sizes or needs to perform operations such as multiple convolutional layers on input images.

For an example on how to train an agent using the GPU, see “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122.

Using Both Multiple Processes and GPUs

You can also train agents using both multiple processes and a local GPU (previously selected using `gpuDevice`) at the same time. To do so, first create a critic or actor approximator object in which the `UseDevice` option is set to `"gpu"`. You can then use the critic and actor to create an agent, and then train the agent using multiple processes. This is done by creating an `rlTrainingOptions` object in which `UseParallel` is set to `true` and passing it to the `train` function.

For gradient-based parallelization, (which must run in synchronous mode) the environment simulation is done by the workers, which also use their local GPU to calculate the gradients and perform a prediction step. The gradients are then sent back to the parallel pool client process which calculates the averages, updates the network parameters and sends them back to the workers so they continue to simulate the agent, with the new parameters, against the environment.

For experience-based parallelization, (which can run in asynchronous mode), the workers simulate the agent against the environment, and send experiences data back to the parallel pool client. The client then uses its local GPU to compute the gradients from the experiences, then updates the network parameters and sends the updated parameters back to the workers, which continue to simulate the agent, with the new parameters, against the environment.

Note that when using both parallel processing and GPU to train PPO agents, the workers use their local GPU to compute the advantages, and then send processed experience trajectories (which include advantages, targets and action probabilities) back to the client.

See Also

Functions

`train`

Objects

`rlTrainingOptions` | `gpuDevice`

Related Examples

- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122
- “Train Reinforcement Learning Agents” on page 5-3

More About

- “GPU Computing Requirements” (Parallel Computing Toolbox)

- “Reinforcement Learning Agents” on page 3-2
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing

This example shows how to train an actor-critic (AC) agent to balance a discrete action space cart-pole system modeled in MATLAB® by using asynchronous parallel training.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Actor-Critic Agent Parallel Training (A3C vs Synchronous)

AC agents use gradient-based parallelization, in which both environment simulation and gradient computation is done by the workers and the gradient averaging is done by the client. Specifically, each worker generates new experiences from its copy of agent and environment, calculates the gradient, and sends it back to the client. The client updates its parameters as follows.

- For asynchronous training (for AC agents this option is also referred to as A3C), the client agent averages the gradients and updates its agent parameters, without waiting to receive experiences from all the workers. The client then sends the updated parameters back to the worker that provided the gradient. Then, the worker updates its copy of the agent and continues to generate experiences (and gradients) using its copy of the environment. To specify asynchronous training, the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function must be set to `async`.
- For synchronous training, the client agent waits until it has received gradients from all of the workers. It then averages the gradients, updates its agent parameters, and sends the updated parameters to all the workers at the same time. Then, all workers use a single updated agent copy, together with their copy of the environment, to generate experiences and calculate new gradients. Synchronous, gradient-based parallelization allows you to achieve, in principle, a speed improvement which is nearly linear in the number of workers. However, since each worker must pause execution until all workers are finished, synchronous training only advances as fast as the slowest worker allows. To specify synchronous training, the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function must be set to `sync`.

For more information on gradient-based parallelization, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

Create Environment Object

Create a predefined environment object for the cart-pole system. For more information on this environment, see “Use Predefined Control System Environments” on page 2-42.

```
env = rlPredefinedEnv("CartPole-Discrete");  
env.PenaltyForFalling = -10;
```

Obtain the observation and action information from the environment object.

```
obsInfo = getObservationInfo(env);  
numObservations = obsInfo.Dimension(1);  
actInfo = getActionInfo(env);
```

Create AC Agent with Custom Networks

Actor-critic agents use a parameterized value function approximator to estimate the value of the policy. A value-function critic takes the current observation as input and returns a single scalar as output (the estimated discounted cumulative long-term reward for following the policy from the state corresponding to the current observation).

To model the parameterized value function within the critic, use a neural network with one input layer (which receives the content of the observation channel, as specified by `obsInfo`) and one output layer (which returns the scalar value). Note that `prod(obsInfo.Dimension)` returns the total number of dimensions of the observation space regardless of whether the observation space is a column vector, row vector, or matrix.

Define the network as an array of layer objects.

```
criticNetwork = [  
    featureInputLayer(prod(obsInfo.Dimension))  
    fullyConnectedLayer(32)  
    reluLayer  
    fullyConnectedLayer(1)  
];
```

Create the value function approximator object using `criticNetwork` and the environment action and observation specifications.

```
critic = rlValueFunction(criticNetwork,obsInfo);
```

Actor-critic agents use a parameterized stochastic policy, which for discrete action spaces is implemented by a discrete categorical actor. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer. The output layer must return a vector of probabilities for each possible action, as specified by `actInfo`. Note that `numel(actInfo.Dimension)` returns the number of elements of the discrete action space.

Define the network as an array of layer objects.

```
actorNetwork = [  
    featureInputLayer(prod(obsInfo.Dimension))  
    fullyConnectedLayer(32)  
    reluLayer  
    fullyConnectedLayer(numel(actInfo.Dimension))  
];
```

Create the actor approximator object using `actorNetwork` and the environment action and observation specifications.

```
actor = rlDiscreteCategoricalActor(actorNetwork,obsInfo,actInfo);
```

For more information on creating approximator objects such as actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Specify options for the critic and actor using `rlOptimizerOptions`.

```
criticOpts = rlOptimizerOptions(LearnRate=1e-2,GradientThreshold=1);
actorOpts = rlOptimizerOptions(LearnRate=1e-2,GradientThreshold=1);
```

Specify the AC agent options using `rlACAgentOptions`, include the training options for the actor and critic.

```
agentOpts = rlACAgentOptions( ...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=criticOpts, ...
    EntropyLossWeight=0.01);
```

Create the agent using the specified actor representation and agent options. For more information, see `rlACAgent`.

```
agent = rlACAgent(actor,critic,agentOpts);
```

Training Options

To train the agent, first specify the training options. For this example, use the following options.

- Run each training for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the agent receives an average cumulative reward greater than 500 over 10 consecutive episodes. At this point, the agent can balance the pendulum in the upright position.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=500, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=500, ...
    ScoreAveragingWindowLength=10);
```

Parallel Training Options

To train the agent using parallel computing, specify the following training options.

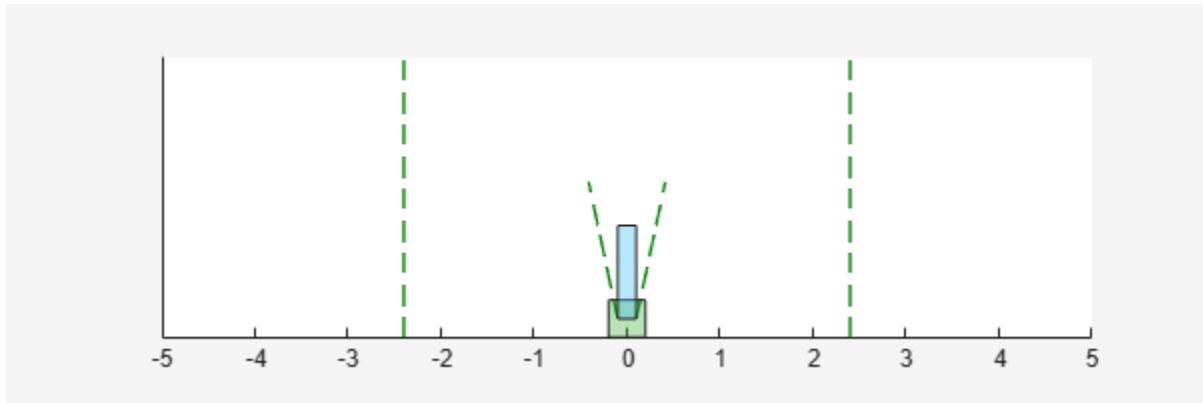
- Set the `UseParallel` option to `True`.
- Train the agent in parallel asynchronously by setting the `ParallelizationOptions.Mode` option to `"async"`. Training an AC agent in asynchronous mode is also referred to as asynchronous advantage AC (A3C) training.

```
trainOpts.UseParallel = true;
trainOpts.ParallelizationOptions.Mode = "async";
```

For more information on training options, see `rlTrainingOptions`.

You can visualize the cart-pole system can during training or simulation using the `plot` function.

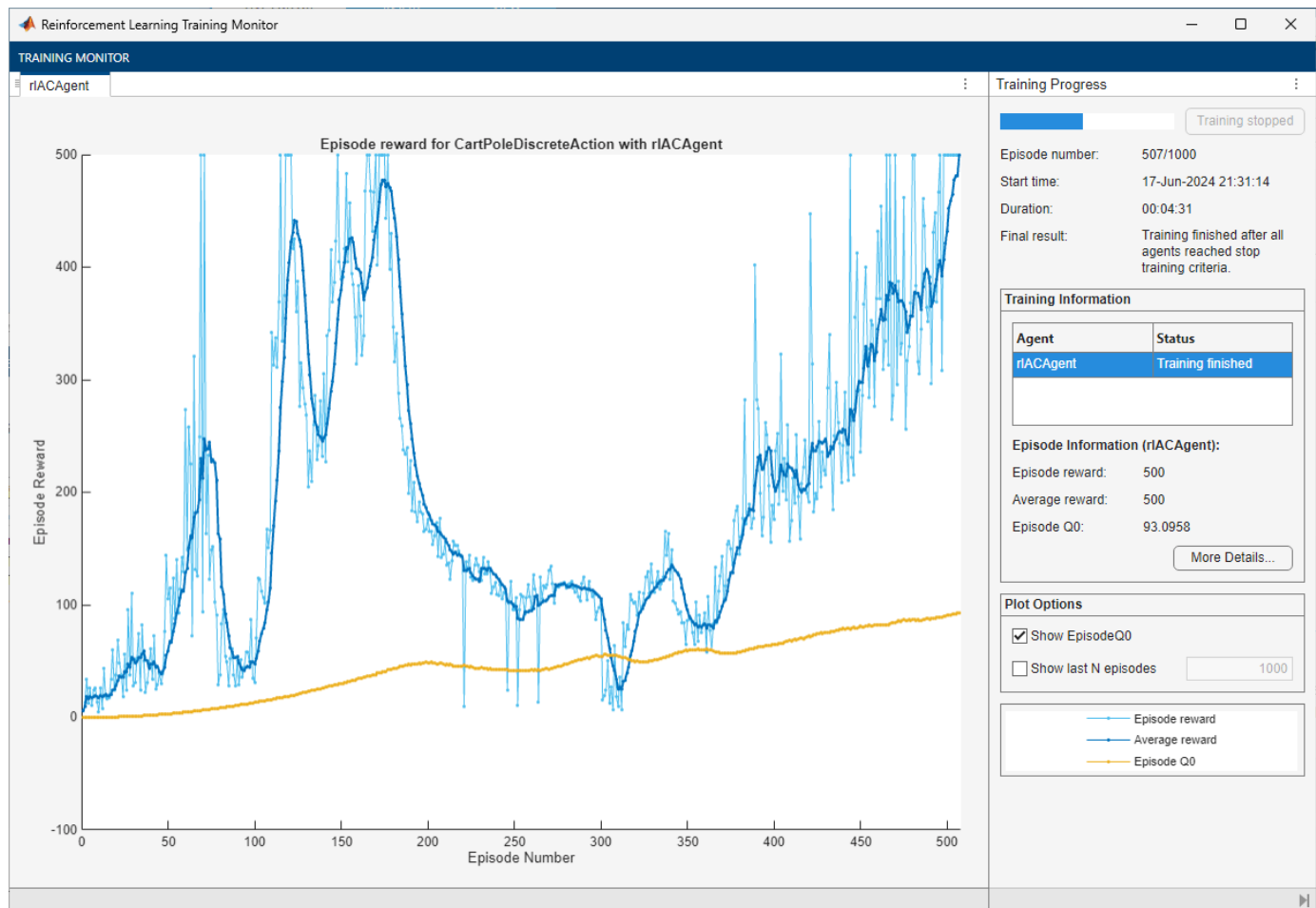
```
plot(env)
```



Train Agent

Train the agent using the `train` function. Training the agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`. Due to randomness in the asynchronous parallel training, you can expect different training results from the following training plot. The plot shows the result of training with six workers.

```
doTraining =  ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts);
else
    % Load the pretrained agent for the example.
    load("MATLABCartpoleParAC.mat","agent");
end
```



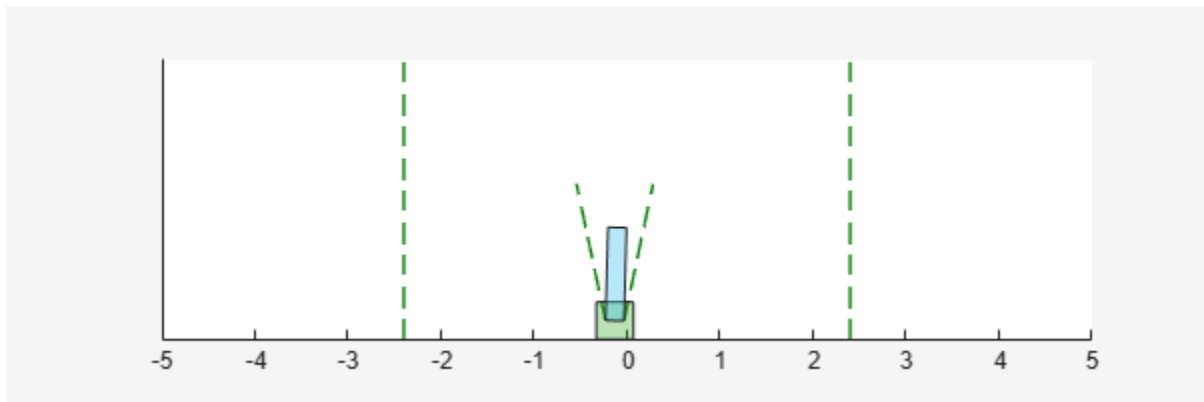
Simulate AC Agent

You can visualize the cart-pole system with the plot function during simulation.

```
plot(env)
```

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
```



```
totalReward = sum(experience.Reward)
```

```
totalReward =  
500
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

[1] Mnih, Volodymyr, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. ‘Asynchronous Methods for Deep Reinforcement Learning’. *ArXiv:1602.01783 [Cs]*, 16 June 2016. <https://arxiv.org/abs/1602.01783>.

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlACAgent` | `rlACAgentOptions` | `rlTrainingOptions`

Related Examples

- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115
- “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144

More About

- “Configure Options for A3C Training”
- “Actor-Critic (AC) Agent” on page 3-44
- “Create Actors, Critics, and Policy Objects” on page 4-2

- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Train PPO Agent with Curriculum Learning for a Lane Keeping Application

This example shows how to use a curriculum to train a Proximal Policy Optimization (PPO) agent for Lane Keeping Assist (LKA) in Simulink®. A curriculum is a training strategy that begins with an easy task and gradually increases the task complexity to enhance the training efficiency of the agent. Task difficulty is controlled by the initial states and the curvature of the lane.

For more information on PPO agents, see “Proximal Policy Optimization (PPO) Agent” on page 3-50.

Open Simulink Model

The training goal for the lane keeping application is to keep the ego vehicle traveling along the centerline of a lane by adjusting the front steering angle. This example uses the same ego vehicle dynamics and sensor dynamics as the “Train DQN Agent for Lane Keeping Assist” on page 8-76 example. Define the ego vehicle model parameters. The longitudinal distances of the front and rear tires are taken from the vehicle center of gravity (CG).

```
m = 1575;    % Total vehicle mass (kg)
Iz = 2875;   % Yaw moment of inertia (mNs^2)
lf = 1.2;    % Longitudinal distance from CG to front tires (m)
lr = 1.6;    % Longitudinal distance from CG to rear tires (m)
Cf = 19000;  % Cornering stiffness of front tires (N/rad)
Cr = 33000;  % Cornering stiffness of rear tires (N/rad)
Vx = 15;     % Longitudinal velocity (m/s)
```

Define the sample time T_s and simulation duration T in seconds.

```
Ts = 0.1;
T = 15;
```

The output of the LKA system is the front steering angle of the ego vehicle. Considering the physical limitations of the ego vehicle, constrain its steering angle to the range $[-60, 60]$ degrees. Specify the constraints in radians.

```
u_min = -1.04;
u_max = 1.04;
```

Define the curvature of the road ρ as a constant 0.001 m^{-1} .

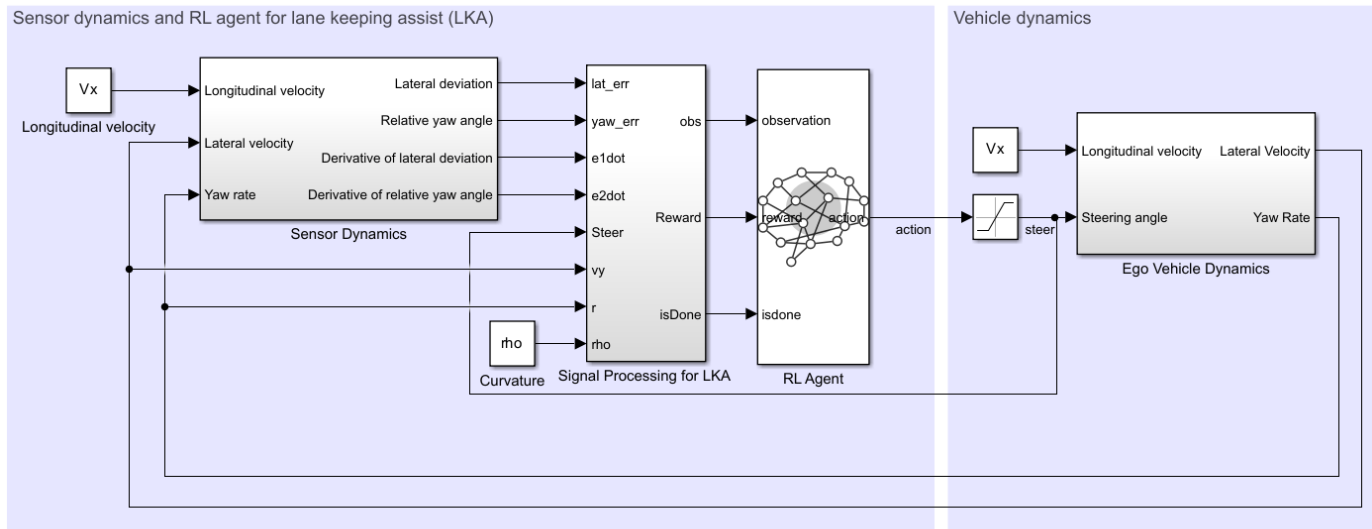
```
rho = 0.001;
```

Set initial values for the lateral deviation (`e1_initial`) and the relative yaw angle (`e2_initial`). Before each training episode, the environment reset function sets these initial conditions to random values.

```
e1_initial = 0.2;
e2_initial = -0.1;
```

Open the model.

```
mdl = "rlCurriculumLKAMdl";
open_system(mdl)
```

Copyright 2018 - 2023 The MathWorks, Inc.

Define the path to the RL Agent block within the model.

```
agentblk = mdl + "/RL Agent";
```

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment

Create a reinforcement learning environment object for the ego vehicle. First define the observation and action specifications. These observations and actions are the same as the features for supervised learning used in “Imitate MPC Controller for Lane Keeping Assist” on page 8-256.

The six observations for the environment are:

- the lateral velocity v_y ,
- the yaw rate $\dot{\psi}$,
- the lateral deviation e_1 ,
- the relative yaw angle e_2 ,
- the steering angle at previous step u_0 ,
- the curvature ρ .

```
obsInfo = rlNumericSpec([6 1], ...
    LowerLimit=-inf*ones(6,1), ...
    UpperLimit=inf*ones(6,1))

obsInfo =
    rlNumericSpec with properties:

        LowerLimit: [6x1 double]
        UpperLimit: [6x1 double]
        Name: [0x0 string]
        Description: [0x0 string]
        Dimension: [6 1]
        DataType: "double"
```

```
obsInfo.Name = "observations";
```

The action for the environment is the front steering angle. Specify the steering angle constraints when creating the action specification object.

```
actInfo = rlNumericSpec([1 1], ...
    LowerLimit=u_min, ...
    UpperLimit=u_max);
actInfo.Name = "steering";
```

In the model, the Signal Processing for LKA block creates the observation vector signal, computes the reward function, and calculates the stop signal.

In this example, the reward r_t , provided at every time step t , consists of a cost, a penalty for the terminal condition, and an incentive to keep the lateral error within a threshold.

$$r_t = \frac{\text{cost} + \text{penalty} + \text{incentive}}{\text{scale}}$$

$$\text{cost} = -(10e_1^2 + 5e_2^2 + 2u^2 + 5\dot{e}_1^2 + 5\dot{e}_2^2)$$

$$\text{penalty} = \begin{cases} 0 & \text{if } |e_1| \leq 1 \\ -300 & \text{else} \end{cases}$$

$$\text{incentive} = 2$$

$$\text{scale} = 100$$

Here, u is the control input from the previous time step $t - 1$.

The simulation stops when $|e_1| > 1$.

Create the reinforcement learning environment.

```
env = rlSimulinkEnv mdl, agentblk, obsInfo, actInfo);
```

To define the initial condition for lateral deviation and relative yaw angle, specify an environment reset function. The `localResetFcn` on page 5-174 function, which is defined at the end of the example, sets the initial lateral deviation and relative yaw angle to random values depending on the task number. Each subsequent task becomes more difficult.

Task 1: Straight road, starting with small errors

```
e1_initial ~ Uniform(-0.05, 0.05)
e2_initial ~ Uniform(-0.01, 0.01)
ρ = 0
```

Task 2: Road with small curvature, starting with relatively small errors

```
e1_initial ~ Uniform(-0.25, 0.25)
e2_initial ~ Uniform(-0.05, 0.05)
ρ = 0.0005
```

Task 3: Road with large curvature, starting with relatively large errors (final task)

```
e1_initial ~ Uniform(-0.5, 0.5)
e2_initial ~ Uniform(-0.1, 0.1)
ρ = 0.001
```

Create PPO Agent

Fix the random generator seed for reproducibility.

```
rng(0, "twister")
```

Create a PPO agent.

```
initOptions = rlAgentInitializationOptions("NumHiddenUnit", 256);
agentNoCrm = rlPPOAgent(obsInfo, actInfo, initOptions);
```

Specify the PPO agent options.

```
agentNoCrm.AgentOptions.MinibatchSize = 512;
agentNoCrm.AgentOptions.LearningFrequency = 4096;
agentNoCrm.AgentOptions.ActorOptimizerOptions.LearnRate = 5e-4;
agentNoCrm.AgentOptions.ActorOptimizerOptions.GradientThreshold = 5;
agentNoCrm.AgentOptions.CriticOptimizerOptions.LearnRate = 5e-4;
agentNoCrm.AgentOptions.CriticOptimizerOptions.GradientThreshold = 5;
agentNoCrm.AgentOptions.SampleTime = Ts;
```

Train Agent Without a Curriculum

As a baseline, train the agent for task 3 without a curriculum. Specify a reset function that calls `localResetFcn` on page 5-174 function, using an anonymous function handle. This syntax allows you to set `taskNumber` to 3 in the called function while using only one input argument for `env.ResetFcn`.

```
taskNumber = 3;
env.ResetFcn = @(in)localResetFcn(in, taskNumber);
```

To train the agent, first specify the training options.

- Run training for a maximum of 50,000 episodes, with each episode lasting a maximum of 150 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor.
- Stop the training when the evaluation statistic (mean of evaluation episode rewards) is 2.9. The training function evaluates the agent with 10 evaluation episodes at every 100 training episodes.

For more information on training options, see `rlTrainingOptions`.

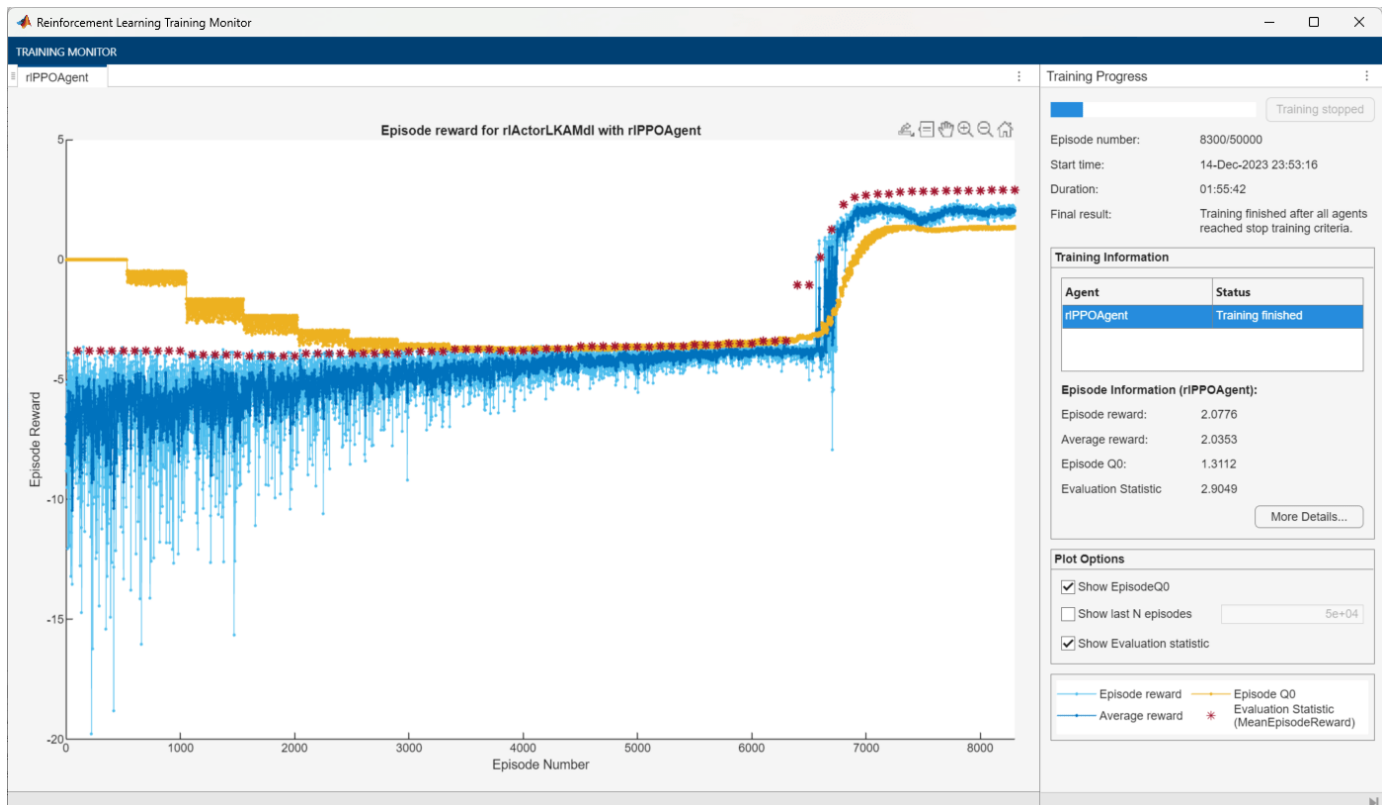
```
maxepisodes = 50000;
maxsteps = T/Ts;
trainingOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=2.9);
```

% Create an evaluator object.

```
evl = rlEvaluator( ...
    NumEpisodes=10, ...
    EvaluationFrequency=100, ...
    RandomSeeds=101:110);
```

Train the agent using the `train` function. The training process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTrainingWithoutCurriculum` to `false`. To train the agent yourself, set `doTrainingWithoutCurriculum` to `true`.

```
doTrainingWithoutCurriculum = ;
if doTrainingWithoutCurriculum
    % Train the agent without a curriculum.
    trainingStats = train(agentNoCrm,env,trainingOpts,Evaluator=evl);
end
```



Train Agent with a Curriculum

Curriculum learning starts with simpler tasks and gradually increases the complexity to train the agent more efficiently. The following example sections train the agent with two simple tasks and then completes the training with a more complex final task.

Create PPO Agent

Fix the random generator seed for reproducibility.

```
rng(0, "twister")
```

Create a PPO agent.

```
initOptions = rlAgentInitializationOptions("NumHiddenUnit", 256);
agent = rlPPOAgent(obsInfo, actInfo, initOptions);
agent.AgentOptions.SampleTime = Ts;
agent.AgentOptions.MinibatchSize = 512;
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 5;
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 5;
```

Train the agent using the `train` function. The training process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
```

Task 1: Straight road, starting with small errors.

Set the task to 1, the max number of episodes to 3000, and the stop the training value to 2.5. Redefine the reset function to use the new task number in the anonymous function workspace.

```
taskNumber = 1;
env.ResetFcn = @(in)localResetFcn(in, taskNumber);
trainingOpts.MaxEpisodes = 3000;
trainingOpts.StopTrainingValue = 2.5;
```

Specify the PPO agent options for task 1. To achieve a faster learning, set a greater `LearnRate` in the initial task. A larger `LearnRate` might cause issues with convergence. However, the goal is to train the agent in the final task, so it is not necessary to fine-tune the agent in the initial task. To update the agent more often in task 1, set `LearningFrequency` to 1024.

```
agent.AgentOptions.LearningFrequency = 1024;
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
```

Train the PPO agent.

```
if doTraining
    trainingStats1 = train(agent, env, trainingOpts, Evaluator=evl);
end
```



Task 2: Road with small curvature, starting with relatively small errors.

Set the task to 2, the max number of episodes to 3000, and the stop the training value to 2.5. Redefine the reset function to use the new task number in the anonymous function workspace.

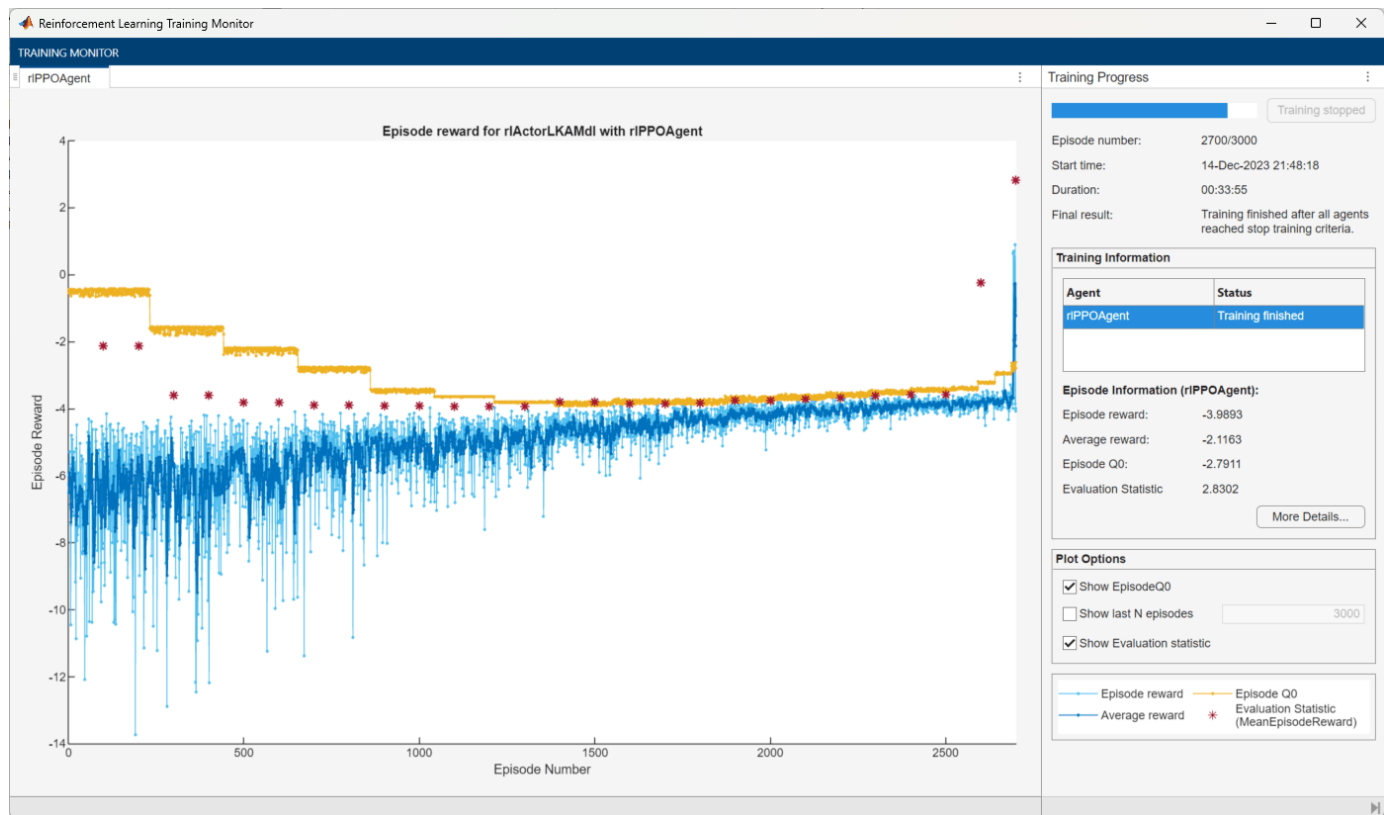
```
taskNumber = 2;
env.ResetFcn = @(in)localResetFcn(in,taskNumber);
trainingOpts.MaxEpisodes = 3000;
trainingOpts.StopTrainingValue = 2.5;
```

Specify the PPO agent options. To collect more trajectories before each update of the learnable parameters, set LearningFrequency to 2048.

```
agent.AgentOptions.LearningFrequency = 2048;
```

Train the PPO agent.

```
if doTraining
    trainingStats2 = train(agent,env,trainingOpts,Evaluator=evl);
end
```



Task 3: Road with large curvature, starting with relatively large errors (final task)

Set the task to 3, the max number of episode to 50,000, and the stop the training value to 2.9. Redefine the reset function to use the new task number in the anonymous function workspace.

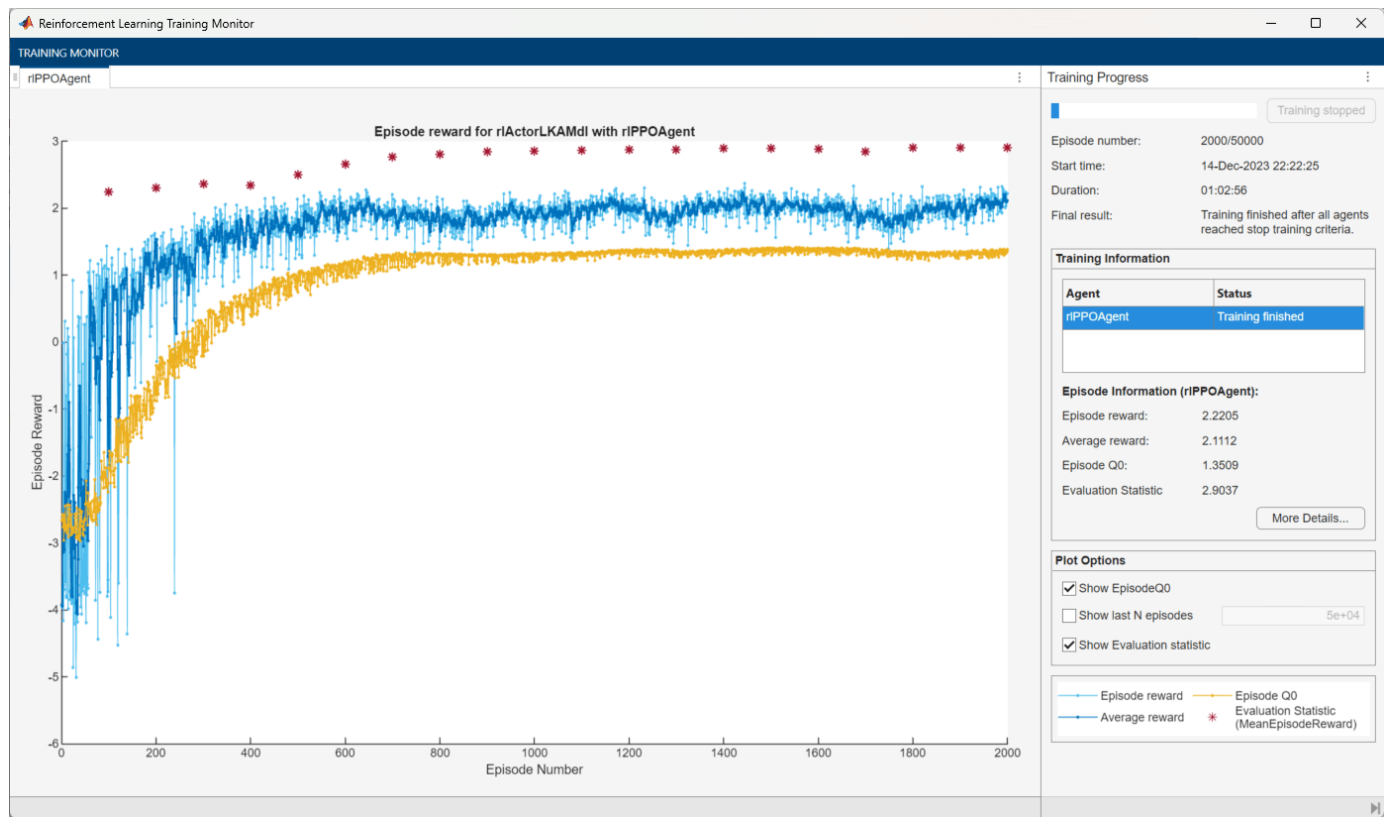
```
taskNumber = 3;
env.ResetFcn = @(in)localResetFcn(in,taskNumber);
trainingOpts.MaxEpisodes = 50000;
trainingOpts.StopTrainingValue = 2.9;
```

Specify the PPO agent options to match those used for the agent trained without a curriculum.

```
agent.AgentOptions.LearningFrequency = 4096;
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 5e-4;
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 5e-4;
```

Train the PPO agent.

```
if doTraining
    trainingStats3 = train(agent,env,trainingOpts,Evaluator=evl);
else
    load("LKAPPOAgent.mat")
end
```



The PPO agent trained with a curriculum meets the criteria after 5000 total episodes: 300 episodes in task 1, 2700 episodes in task 2, and 2000 episodes in task 3. In contrast, the agent trained without a curriculum requires 8300 episodes to meet the criteria. The training time is shorter with curriculum learning compared to the noncurriculum approach.

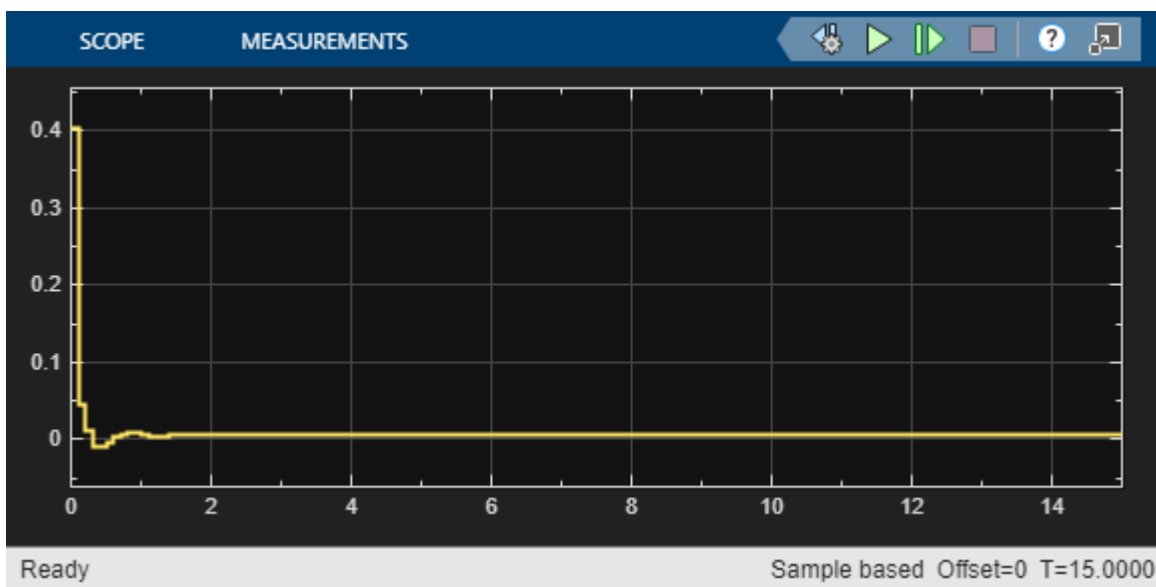
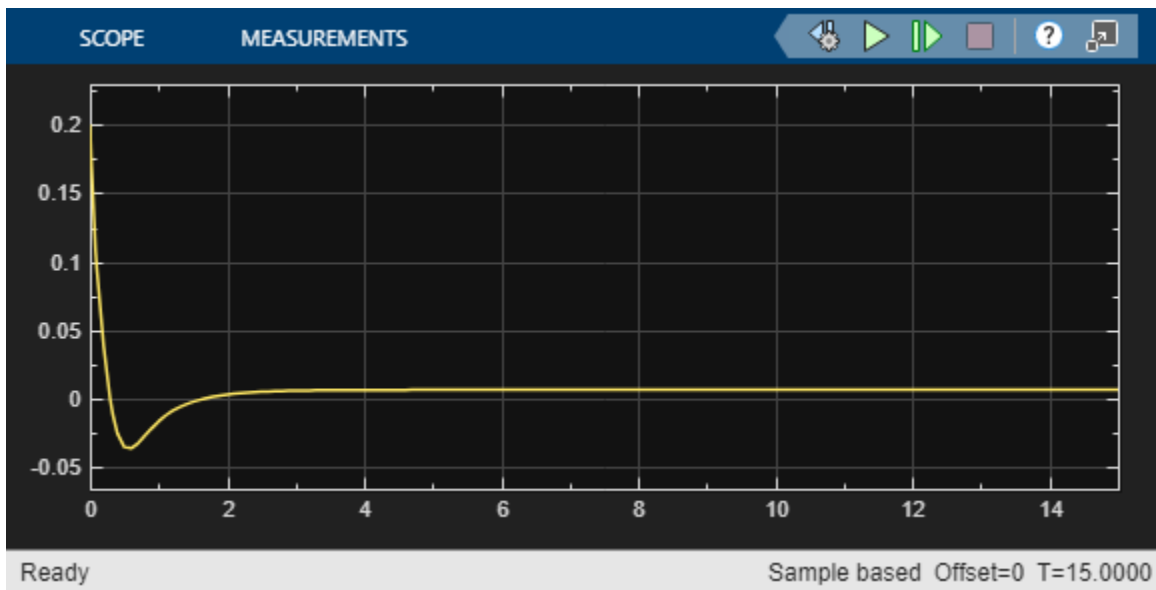
Simulate Trained PPO Agent

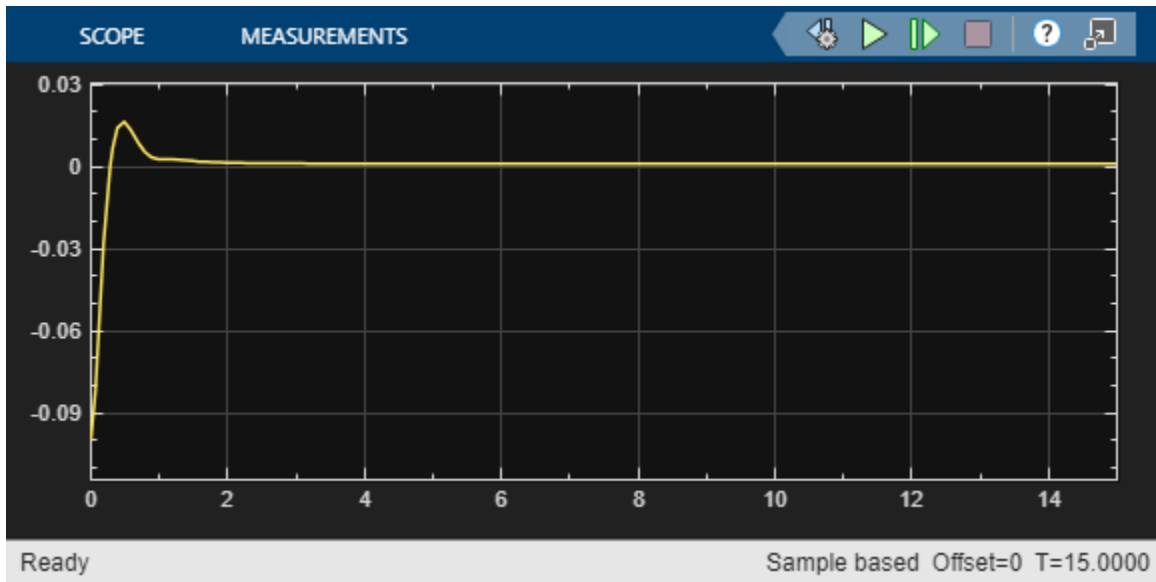
To validate the performance of the trained agent, simulate it within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
% Create simulation options object
simOptions = rlSimulationOptions(MaxSteps=maxsteps);

% Set the task number to 4 to evaluate the agent with a specific
% initial condition. The curvature is the same as in task 3.
taskNumber = 4;
env.ResetFcn = @(in)localResetFcn(in,taskNumber);

% Simulate the agent.
experience = sim(env,agent,simOptions);
```



The lateral error and the yaw error both approach zero, as shown in the respective plots. The vehicle starts with a lateral deviation from the centerline (0.2 m) and a nonzero yaw angle error (−0.1 rad). The lane keeping controller makes the ego vehicle travel along the centerline after approximately two seconds. The steering angle shows that the controller reaches a steady state within the same time frame. The steering angle also achieves a steady state after approximately two seconds.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Reset Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `reset` function uses the `setVariable` (Simulink) function to specify random values for the initial lateral deviation and relative yaw angle in the model workspace, depending on the task number. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in,taskNumber)
    switch taskNumber
        case 1
            % Task 1: Straight road,
            % starting with small errors.
            e1_initial = 0.05*(-1+2*rand);
            e2_initial = 0.01*(-1+2*rand);
            rho = 0;
        case 2
            % Task 2: Road with small curvature,
            % starting with relatively small errors.
            e1_initial = 0.25*(-1+2*rand);
            e2_initial = 0.05*(-1+2*rand);
            rho = 0.0005;
        case 3
            % Task 3 : Road with large curvature, starting
```

```

        % with relatively large errors (final task)
        e1_initial = 0.5*(-1+2*rand);
        e2_initial = 0.1*(-1+2*rand);
        rho = 0.001;
    case 4
        % Final simulation purpose.
        e1_initial = 0.2;
        e2_initial = -0.1;
        rho = 0.001;
    end
    % Set random value for lateral deviation.
    in = setVariable(in,"e1_initial",e1_initial);

    % Set random value for relative yaw angle.
    in = setVariable(in,"e2_initial",e2_initial);

    % Set curvature
    in = setVariable(in,"rho",rho);
end

```

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlPPOAgent | rlPPOAgentOptions | rlAgentInitializationOptions | rlTrainingOptions | rlOptimizerOptions | rlSimulationOptions

Blocks

RL Agent

Related Examples

- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox)
- “Transfer Learning: Fine-Tune DQN Agent for Pendulum Swing-Up from Earth to Mars” on page 5-207

More About

- “Train Reinforcement Learning Agents” on page 5-3

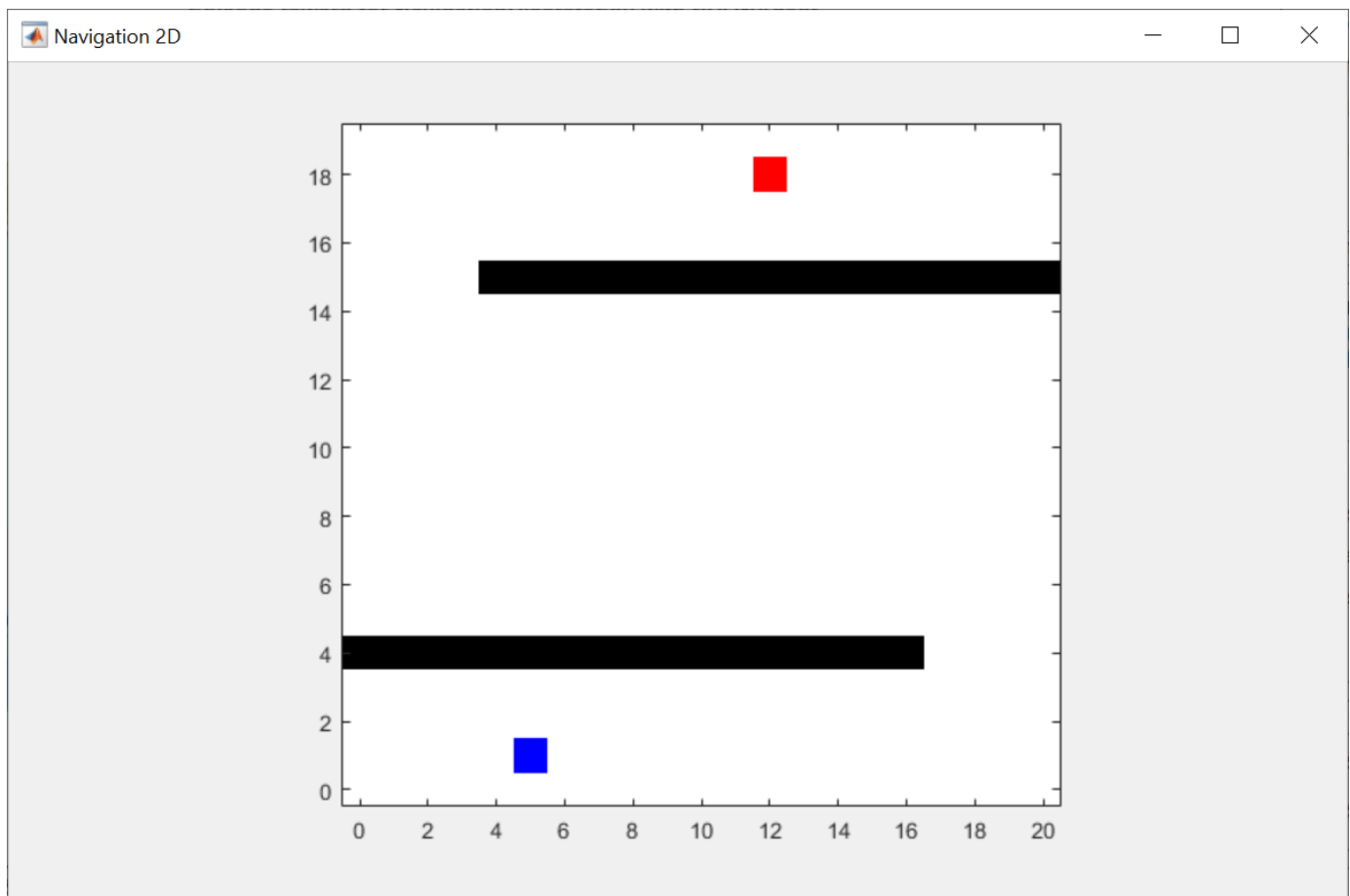
Train DQN Agent Using Hindsight Experience Replay

This example shows how to train a deep Q-learning network (DQN) agent using hindsight experience replay in a discrete navigation problem.

Discrete Navigation Environment

The reinforcement learning environment for this example is a simple discrete 2-D navigation problem. The training goal is to make a robot reach the goal state.

The environment is a 20-by-20 grid containing a robot (blue), a goal (red), and obstacles (black).



The navigation problem in this example is a goal-conditioned task for which the observation from the environment contains both the goal (navigation target) and the goal measurement (robot position). The observation column vector contains the robot position $(x_{\text{robot}}, y_{\text{robot}})$ and the navigation target $(x_{\text{goal}}, y_{\text{goal}})$.

$$O = [x_{\text{robot}}, y_{\text{robot}}, x_{\text{goal}}, y_{\text{goal}}]^T$$

The initial position of the robot and the goal are sampled as follows:

$$x_{\text{robot}} \sim U\{0, 19\}$$

$$y_{\text{robot}} \sim U\{0, 3\}$$

$$x_{\text{goal}} \sim U\{0, 19\}$$

$$y_{\text{goal}} \sim U\{16, 19\}$$

where $U\{a, b\}$ is a discrete uniform distribution between a and b .

The discrete actions for this environment are defined as follows:

$$A = \begin{cases} 1 & y_{\text{robot}} = y_{\text{robot}} + 1 \quad (\text{Go up}) \\ 2 & y_{\text{robot}} = y_{\text{robot}} - 1 \quad (\text{Go down}) \\ 3 & x_{\text{robot}} = x_{\text{robot}} - 1 \quad (\text{Go left}) \\ 4 & x_{\text{robot}} = x_{\text{robot}} + 1 \quad (\text{Go right}) \end{cases}$$

If the action leads to the obstacle location or the map's boundary, the robot doesn't move.

The reward signal is defined as follows:

$$R = \begin{cases} 2 & \text{if } x_{\text{robot}} = x_{\text{goal}} \text{ and } y_{\text{robot}} = y_{\text{goal}} \\ -0.01 - 0.01I(O, O') & \text{otherwise} \end{cases}$$

where O' is the next observation, and

$$I(O, O') = \begin{cases} 1 & \text{if the robot doesn't move due to obstacles or boundaries.} \\ 0 & \text{otherwise} \end{cases}$$

Terminal condition is defined as follows:

$$\text{IsDone} = \begin{cases} 1 & \text{if } x_{\text{robot}} = x_{\text{goal}} \text{ and } y_{\text{robot}} = y_{\text{goal}} \\ 0 & \text{otherwise} \end{cases}$$

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(1, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Interface

Create a discrete navigation environment. `NavigationDiscreteEnv.m` is located in this example folder.

```
env = NavigationDiscreteEnv;
```

Create DQN Agent

For a given observation and action, a DQN agent estimates the value of the policy (the cumulative discounted long-term reward) using a parameterized Q-value function critic.

Create a default DQN agent from the environment specifications. For more information on DQN agents, see `rlDQNAgent`.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
agent = rlDQNAgent(obsInfo,actInfo);
```

Specify the DQN agent options, including training options for the critic. Alternatively, you can use `rlDQNAgentOptions` object.

```
agent.AgentOptions.EpsilonGreedyExploration.Epsilon = 1.0;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonMin = 0.1;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-5;
agent.AgentOptions.MiniBatchSize = 128;
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 5e-4;
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Construct Hindsight Replay Memory

The DQN agent is unable to efficiently learn a good policy with a default replay memory because of the sparse rewards. You can improve the sample efficiency by using a hindsight replay memory for a goal-conditioned task with sparse rewards. The hindsight experience replay augments the acquired experiences by replacing the goal with the goal measurement so that agent can use the data that reaches the replaced goal. Thus, the agent can be trained with meaningful rewards even if the agent does not reach the goal.

To use a hindsight replay memory, set `ExperienceBuffer` of the agent to `rlHindsightReplayMemory`. You need to specify the following.

- A reward function: The reward function, `myNavigationGoalRewardFcn`, computes the true reward given observation, action, and next observation.
- An is-done function: The is-done function, `myNavigationGoalIsDoneFcn`, computes the true is-done signal given observation, action, and the next observation.
- Goal condition information: `goalConditionInfo` specifies the channel and indices of goal measurements and goals. In this example, there is only one observation channel; the first and second observation elements correspond to the goal measurements (the robot position), and the third and fourth observation elements correspond to the goal position.
- Experience buffer length (optional): This examples uses `1e5`.

`myNavigationGoalRewardFcn.m` and `myNavigationGoalIsDoneFcn.m` are located in this example folder. For more information, type `help rlHindsightReplayMemory`.

To use a standard replay memory, set `useHER` to `false`.

```
useHER = true;
if useHER
    rewardFcn = @myNavigationGoalRewardFcn;
    isDoneFcn = @myNavigationGoalIsDoneFcn;
    % observation is [robot_x, robot_y, goal_x, goal_y]'
    % goal measurement: channel --- 1, indices --- 1,2
```

```

% goal: channel --- 1, indices --- 3,4
goalConditionInfo = {{1,[1,2], 1, [3,4]}};
bufferLength = 1e5;
agent.ExperienceBuffer = rlHindsightReplayMemory( ...
    obsInfo,actInfo, ...
    rewardFcn,isDoneFcn,goalConditionInfo,bufferLength);
else
    agent.AgentOptions.ExperienceBufferLength = 1e5;
end

```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run one training session containing 3000 episodes, with each episode lasting a maximum of 120 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the Plots option) and disable the command line display (set the Verbose option to false).
- Set the Plots options to "training-progress".
- Stop the training when the agent receives an evaluation statistic (the mean value of evaluation episode rewards in this example) greater than 1.0.

For more information on training options, see `rlTrainingOptions`.

```

maxEpisodes = 3000;
maxStepsPerEpisode = 120;
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxEpisodes, ...
    MaxStepsPerEpisode=maxStepsPerEpisode, ...
    Verbose=false, ...
    ScoreAveragingWindowLength=100, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=1.0);

```

Use an `rlEvaluator` object to evaluate the agent periodically during training and to use the evaluation statistic for the stop the training criteria. For this example, use the following options:

- Run 10 evaluation episodes at every 50 training episodes.
- Use random seeds from 101 to 110 so that the same initial conditions can be used. For example, the first evaluation episode uses 101, and the 10th evaluation episode uses 110.
- The evaluator uses the mean of the evaluation episode rewards as the default for "EvaluationStatisticType".

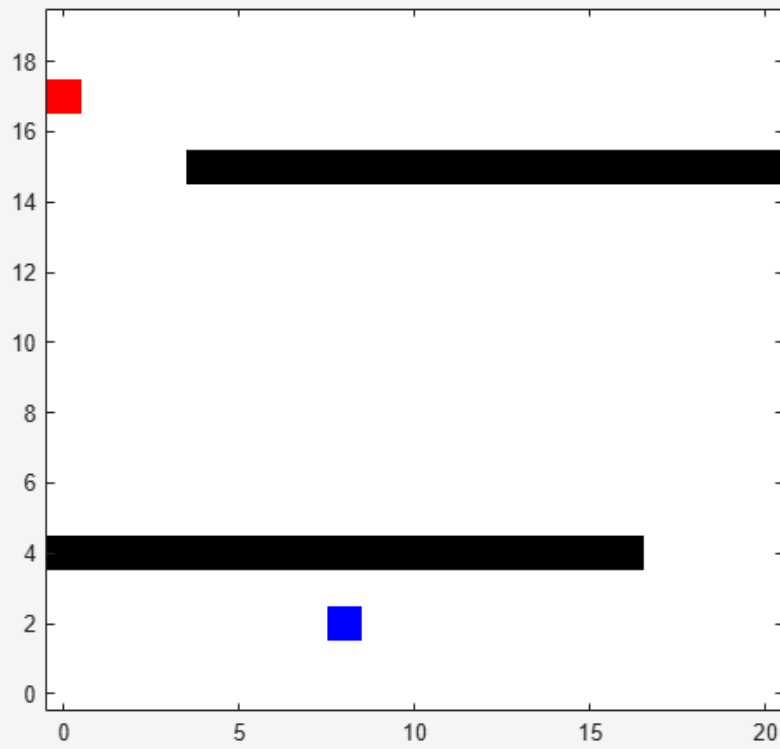
```

evaluator = rlEvaluator( ...
    EvaluationFrequency=50, ...
    NumEpisodes=10, ...
    RandomSeeds=101:110);

```

You can visualize the environment by using the `plot` function during training or simulation.

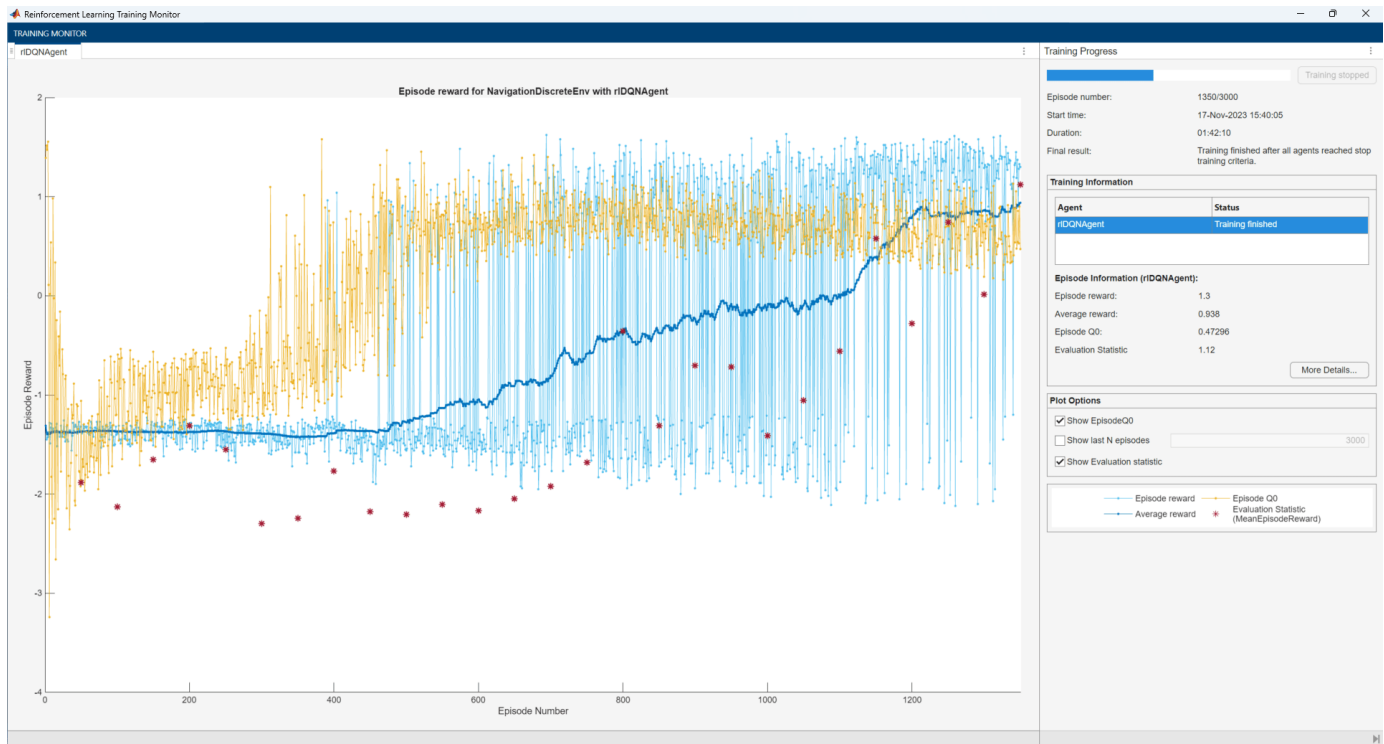
```
plot(env)
```



Train the agent using the `train` function. Training this agent is a computationally intensive process. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts, Evaluator=evaluator);
else
    % Load the pretrained agent for the example.
    load("DiscreteNavigationDQNAgent.mat","agent")
end
```

The following figure shows a snapshot of training progress.



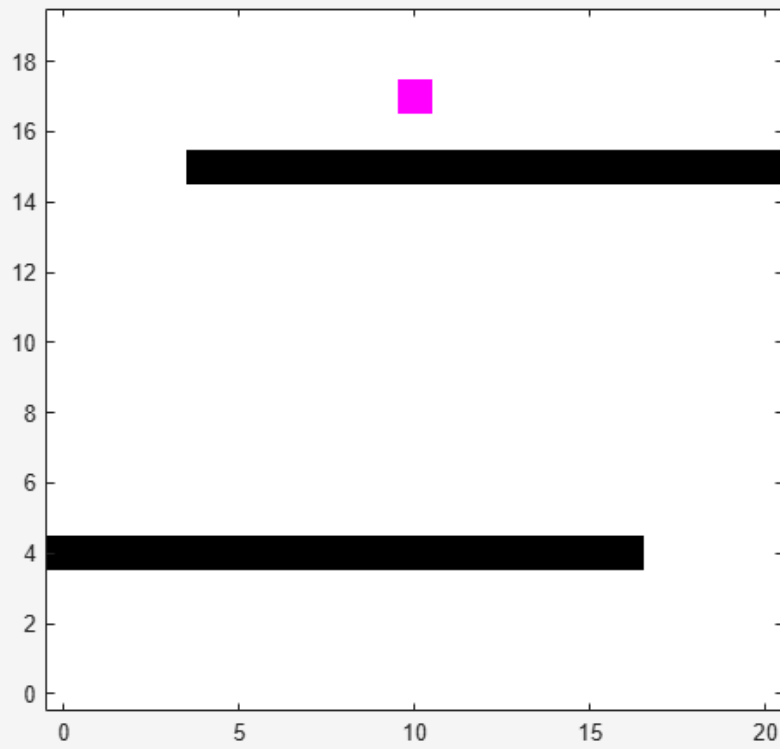
Simulate DQN Agent

Because the reset function randomizes the reference values, fix the random generator seed to ensure simulation reproducibility.

```
rng(2)
```

To validate the performance of the trained agent, simulate it within the navigation environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions( ...
    NumSimulations=1, ...
    MaxSteps=maxStepsPerEpisode);
simResults = sim(env,agent,simOptions);
```



The trained agent successfully reached the goal.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Reference

Andrychowicz, Marcin, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. "Hindsight experience replay." *Advances in neural information processing systems* 30 (2017).

See Also

Functions

`train` | `sim`

Objects

`rlHindsightReplayMemory` | `rlHindsightPrioritizedReplayMemory` | `rlTrainingOptions`
| `rlSimulationOptions` | `rlDQNAgent` | `rlDQNAgentOptions`

Related Examples

- "Train Default DQN Agent to Balance Discrete Cart-Pole" on page 5-64

- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217

More About

- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3

Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum

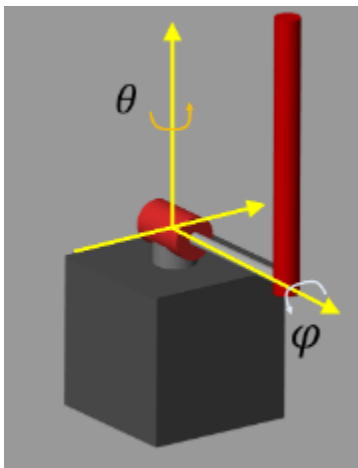
This example shows how you can train an agent from existing data instead of using an environment. Training an agent from data (offline reinforcement learning) can be useful to efficiently prototype different design choices by leveraging already collected data. In this example, two reinforcement learning (RL) agents are trained to swing up and balance a Quanser QUBE™-Servo 2 inverted pendulum system. First, you train a TD3 agent online using a Simulink® environment and collect training data using a data logger. Then, you train another TD3 agent offline using the collected data and a modified reward function to achieve a different behavior from the agent. You then compare the behavior of both agents.

Inverted Pendulum Model

The Quanser QUBE-Servo 2 pendulum system is a rotational inverted pendulum with two degrees of freedom. It is non-linear, underactuated, non-minimum phase, and it is modeled in Simulink using Simscape™ Electrical™ and Simscape Multibody™. For a detailed description of the dynamics, see [1]. For related examples that use this pendulum system model, see “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14 and “Run SIL and PIL Verification for Reinforcement Learning” on page 6-17.

The pendulum is attached to the motor arm through a free revolute joint. The arm is actuated by a DC motor. The environment has the following properties:

- The angles and angular velocities of the motor arm ($\theta, \dot{\theta}$) and pendulum ($\phi, \dot{\phi}$) are measurable.
- The motor is constrained to $|\theta| \leq 5\frac{\pi}{8}$ rad and $|\dot{\theta}| \leq 30 \frac{\text{rad}}{\text{sec}}$.
- The pendulum is upright when $\phi = 0$.
- The motor input is constrained to $\pm 12V$.
- The agent action ($|u| \leq 1$) is scaled to the motor voltage in the environment.



Open the Simulink model.

```
mdl = "rlQubeServoModel";
open_system(mdl)
```

Define the angular and voltage limits, as well as the model's sample time.

```
theta_limit = 5*pi/8;
dtheta_limit = 30;
volt_limit = 12;
Ts = 0.005;
```

Train TD3 Agent Online

A TD3 agent is trained online against the Simulink environment. For this agent:

- The environment is the pendulum system modeled in Simscape Multibody.
- The observation is the vector $s_k = [\sin\theta_k, \cos\theta_k, \dot{\theta}_k, \sin\varphi_k, \cos\varphi_k, \dot{\varphi}_k, u_{k-1}]$. Using the sine and cosine of the measured angles can facilitate training by representing the otherwise discontinuous angular measurements by a continuous two-dimensional parametrization.
- The action is the normalized input voltage command to the servo motor.
- The reward signal is defined as follows:

$$r(s_k, u_{k-1}) = F_k - 0.1 \left(\theta_k^2 + \varphi_k^2 + \dot{\theta}_k^2 + \dot{\varphi}_k^2 + u_{k-1}^2 + 0.3(u_{k-1} - u_{k-2})^2 \right)$$

$$F_k = \begin{cases} 1 & \theta_k \in \pm 5\frac{\pi}{8} \text{ rad and } \dot{\theta}_k \in \pm 30 \frac{\text{rad}}{\text{sec}} \\ 0 & \text{otherwise} \end{cases}.$$

The above reward function penalizes six different terms:

- Deviations from the forward position of the motor arm ($\theta_k = 0$)
- Deviations for the inverted position of the pendulum ($\varphi_k = 0$)
- The angular speed of the motor arm $\dot{\theta}_k$
- The angular speed of the pendulum $\dot{\varphi}_k$
- The control action u_k
- Changes to the control action ($u_{k-1} - u_{k-2}$)

The agent is rewarded while the system constraints are satisfied (that is, $F_k = 1$).

Set the random seed for reproducibility.

```
rng(0)
```

Create the input and output specifications for the agent. Set the action upper and lower limits to constrain the actions selected by the agent.

```
obsInfo = rlNumericSpec([7 1]);
actInfo = rlNumericSpec([1 1],UpperLimit=1,LowerLimit=-1);
```

Create the environment object. Specify a reset function, defined at the end of the example, that sets random initial conditions.

```
agentBlk = mdl + "/RL Agent";
simEnv = rlSimulinkEnv(mdl,agentBlk,obsInfo,actInfo);
simEnv.ResetFcn = @localResetFcn;
```

To create an agent option object, use `rlTD3AgentOptions`. Specify sample time, experience buffer length, and the mini batch size.

```
agentOpts = rlTD3AgentOptions( SampleTime=Ts, ...  
    ExperienceBufferLength=1e6, ...  
    MiniBatchSize=128);  
agentOpts.LearningFrequency = 1;  
agentOpts.MaxMiniBatchPerEpoch = 1;
```

Specify the actor and critic optimizer options.

```
agentOpts.ActorOptimizerOptions.LearnRate = 1e-4;  
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;  
agentOpts.CriticOptimizerOptions(1).LearnRate = 1e-3;  
agentOpts.CriticOptimizerOptions(1).GradientThreshold = 1;  
agentOpts.CriticOptimizerOptions(2).LearnRate = 1e-3;  
agentOpts.CriticOptimizerOptions(2).GradientThreshold = 1;
```

Set to 64 the number of neurons in each hidden layer of the actor and critic networks. To create a default TD3 agent, use `rlTD3Agent`.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=64);  
TD3agent = rlTD3Agent(obsInfo,actInfo,initOpts,agentOpts);
```

Define training options. The length of an episode is given by the simulation time T_f divided by the sample time T_s .

```
Tf = 5;  
maxSteps = ceil(Tf/Ts);
```

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=300, ...  
    MaxStepsPerEpisode=maxSteps, ...  
    ScoreAveragingWindowLength= 10, ...  
    Verbose= false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="AverageReward", ...  
    StopTrainingValue=inf);
```

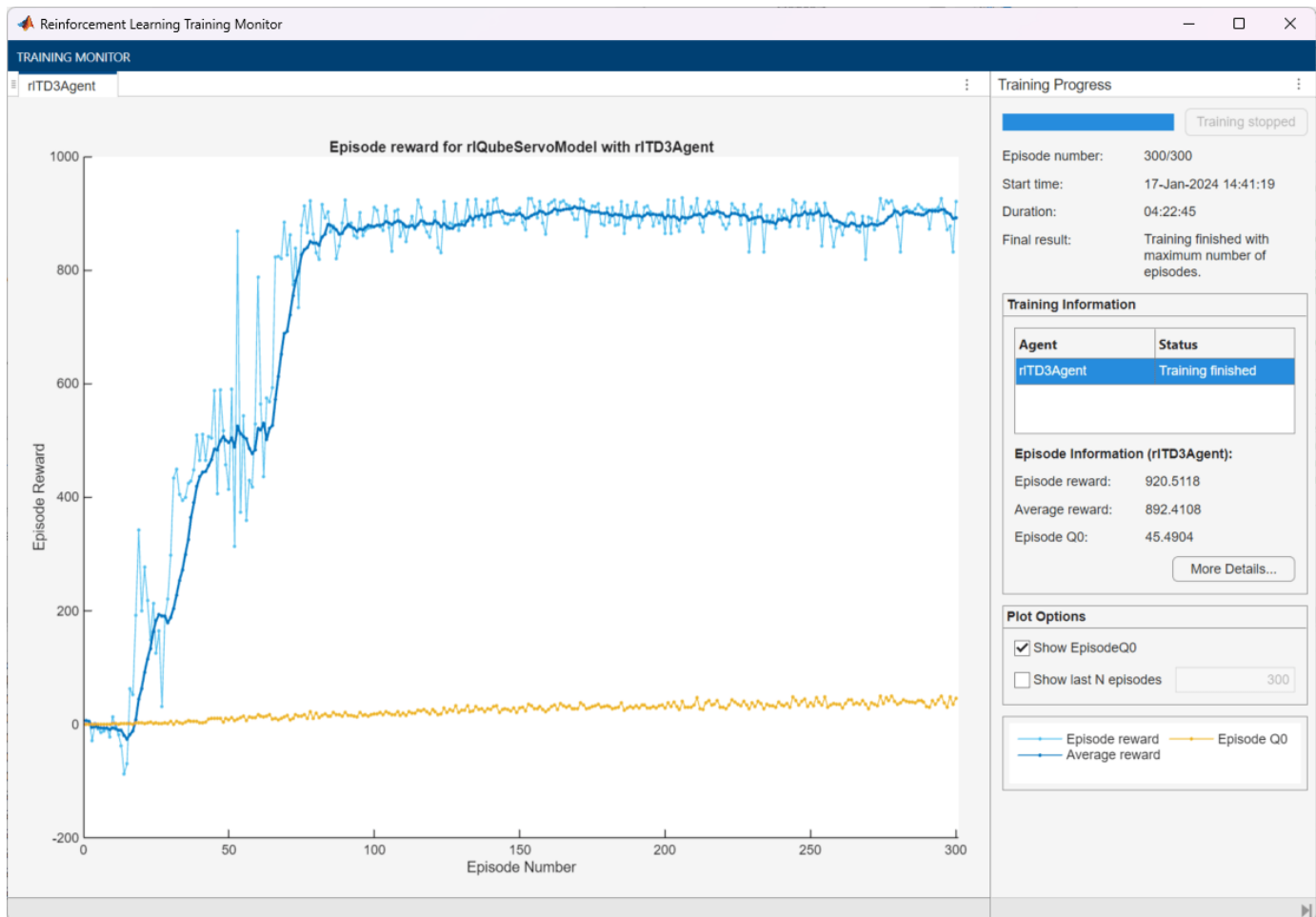
Create a data logger to save experiences that are later used for offline training of different agents. For more information, see `rlDataLogger`.

```
logger = rlDataLogger();  
logger.EpisodeFinishedFcn = @localEpisodeFinishedFcn;  
logger.LoggingOptions.LoggingDirectory = "SimulatedPendulumDataSet";
```

Train the TD3 agent using the Simulink environment using `train`. As shown in the following Reinforcement Learning Training Monitor screenshot, training can take over a hour, so you can save time while running this example by setting `doTD3Training` to `false` to load a pretrained agent. To train the agent yourself, set `doTD3Training` to `true`. During training, the experiences for each episode are saved in a file named `loggedDataN.mat` (where N is the episode number) in the `SimulatedPendulumDataSet` subfolder, under the current folder.

```
doTD3Training = false;  
if doTD3Training  
    trainStats = train(TD3agent, ...  
        simEnv,trainOpts,Logger=logger);  
else
```

```
load("rlQuanserQubeAgents.mat", "TD3agent");
end
```



Evaluate TD3 Agent

Test the trained agent and evaluate its performance. Because the environment reset function sets the initial state randomly, fix the seed of the random generator to ensure that the model uses the same initial conditions when running simulations.

```
testingSeed = 1;
rng(testingSeed)
```

For simulation, do not use an explorative policy.

Define simulation options.

```
numTestEpisodes = 1;
simOpts = rlSimulationOptions( ...
    MaxSteps=maxSteps, ...
    Numsimulations=numTestEpisodes);
```

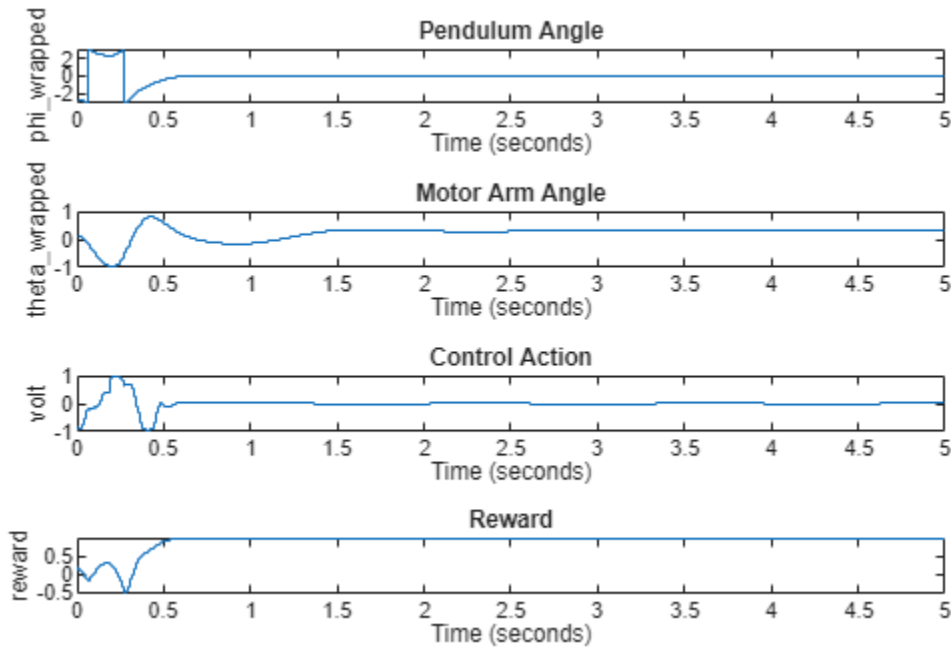
Simulate the trained agent.

```
simResult = sim(TD3agent,simEnv,simOpts);
```

You can view the behavior of the trained agent in the mechanics explorer animation of the Simscape model, or you can plot a sample trajectory of the angles, control actions, and reward.

```
% Extract signals.
simInfo = simResult(1).SimulationInfo(1);
phi = get(simInfo.logouts,"phi_wrapped");
theta = get(simInfo.logouts,"theta_wrapped");
action = get(simInfo.logouts,"volt");
reward = get(simInfo.logouts,"reward");

% Plot values.
figure
tiledlayout(4,1)
nexttile
plot(phi.Values); title("Pendulum Angle")
nexttile
plot(theta.Values); title("Motor Arm Angle")
nexttile
plot(action.Values); title("Control Action")
nexttile
plot(reward.Values); title("Reward")
```



Train TD3 Agent Offline with Modified Reward Function

Suppose you want to modify the reward function to encourage to balance the pendulum at a motor arm angle that matches some new reference angle β . Consider the new reward function:

$$r'(s_k, u_{k-1}) = F_k - 0.1 \left((\theta_k - \beta)^2 + \varphi_k^2 + \dot{\theta}_k^2 + \dot{\varphi}_k^2 + u_{k-1}^2 + 0.3(u_{k-1} - u_{k-2})^2 \right),$$

where β corresponds to the desired angle that the motor arm should target. By expanding the modified term, you can express the reward function in terms of the original reward.

$$r'(s_k, u_{k-1}) = r(s_k, u_{k-1}) - 0.1(-2\beta \theta_k + \beta^2)$$

Instead of modifying the environment and training in simulation, you can leverage the already collected data to train the new agent offline. This process can be faster and more computationally efficient.

To train an agent from data you can:

- 1 Define a `ReadFcn` function that reads the collected files and returns data in appropriate form. If necessary, this function can also modify data (to change the reward, for example).
- 2 Create a `fileDatastore` object pointing to the collected data.
- 3 Create the agent to be trained.
- 4 Define training options.
- 5 Train the agent using `trainFromData`.

If, in the previous section, you have loaded the pre-trained TD3 agent instead of training the new agent, then at this point you do not have the training data in your local disk. In this case, download the data from the MathWorks® server, and unzip it to recreate the same folder and files that would exist after training the agent. If you already have the training data in the previously defined logging directory, set `downloadData` to `false` to avoid downloading the data.

```
downloadData = true;
if downloadData
    zipfilename = "SimulatedPendulumDataSet.zip";
    filename = ...
        matlab.internal.examples.downloadSupportFile("rl",zipfilename);
    unzip(filename)
end
```

Set the random seed for reproducibility.

```
rng(0)
```

Define the `fileDatastore` that points to the collected training data. Use the `ReadFcn` function of the data store to add the additional term to the reward of each experience. You can find the read function used in this example at the end of the example.

```
dataFolder = fullfile( ...
    logger.LoggingOptions.LoggingDirectory, ...
    "loggedData*.mat");
fds = fileDatastore(dataFolder, ...
    ReadFcn=@localReadFcn, ...
    FileExtensions=".mat");
fds.shuffle();
```

To create an agent option object, use `rlTD3AgentOptions`. Specify sample time, experience buffer length, and the mini batch size.

```
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=128);
```

Specify the actor and critic optimizer options.

```
agentOpts.ActorOptimizerOptions.LearnRate = 1e-4;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
agentOpts.CriticOptimizerOptions(1).LearnRate = 1e-3;
```

```
agentOpts.CriticOptimizerOptions(1).GradientThreshold = 1;
agentOpts.CriticOptimizerOptions(2).LearnRate = 1e-3;
agentOpts.CriticOptimizerOptions(2).GradientThreshold = 1;
```

Set the random seed for reproducibility.

```
rng(0)
```

Set to 64 the number of neurons in each hidden layer of the actor and critic networks. To create a default TD3 agent, use `rlTD3Agent`.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=64);
offlineTD3agent = rlTD3Agent(obsInfo,actInfo,initOpts,agentOpts);
```

You can use the batch data regularizer options to alleviate the overestimation issue often caused by offline training. For more information, see `rlBehaviorCloningRegularizerOptions`.

```
offlineTD3agent.AgentOptions.BatchDataRegularizerOptions...
    = rlBehaviorCloningRegularizerOptions;
```

Define offline training options using the `rlTrainingFromDataOptions` object.

```
options = rlTrainingFromDataOptions;
options.MaxEpochs = 300;
options.NumStepsPerEpoch = 500;
```

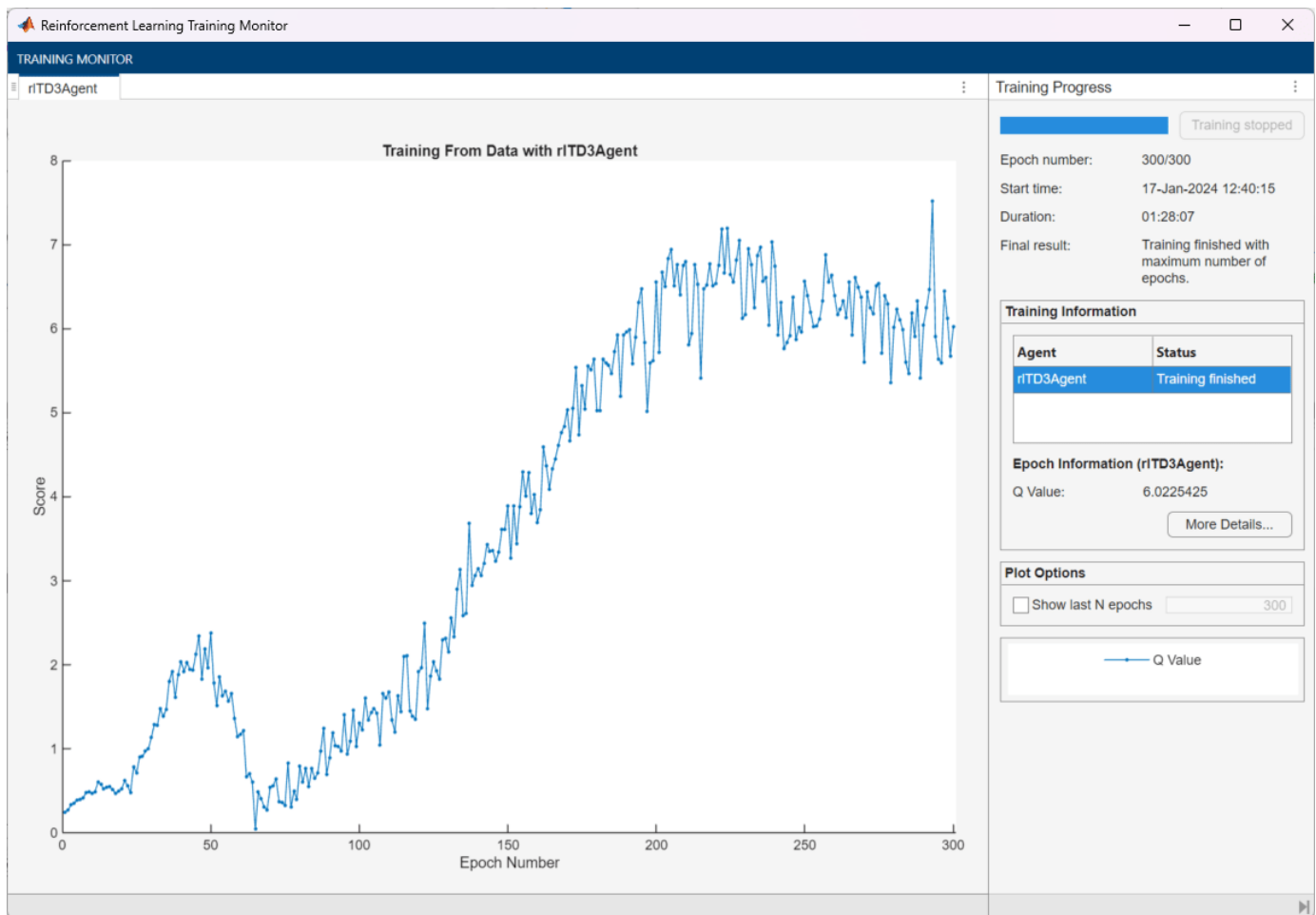
To calculate a training progress metric, use the observation vector for the computation of the current Q value estimate. For this example, you can use the stable equilibrium at $\theta = \dot{\theta} = \dot{\phi} = 0$ and $\phi = \pi$.

```
options.QValueObservations = ...
    {[sin(0); cos(0); 0; sin(pi); cos(pi); 0; 0]};
```

Train the agent offline from the collected data using `trainFromData`. The training duration can be seen in the following Episode Manager screenshot, and it highlights how training from data can be faster. For this example, the training took less than half the time of the online training.

```
doTD3offlineTraining = false;
if doTD3offlineTraining
    offlineTrainStats = ...
        trainFromData(offlineTD3agent, fds ,options);
else
    load("rlQuanserQubeAgentsFromData.mat","offlineTD3agent");
end
```

The Q values during offline training do not necessarily indicate the agent's actual performance. Overestimation is a common issue of offline reinforcement learning due to the state-action distribution shift between the dataset and the learned policy. You need to examine hyperparameters or training options if the Q values become too big. This example uses the batch data regularizer to alleviate the overestimation issue by penalizing the learned policy's actions that are different from the dataset.



Evaluate TD3 Agent in Simulation

Fix the random seed generator to the testing seed.

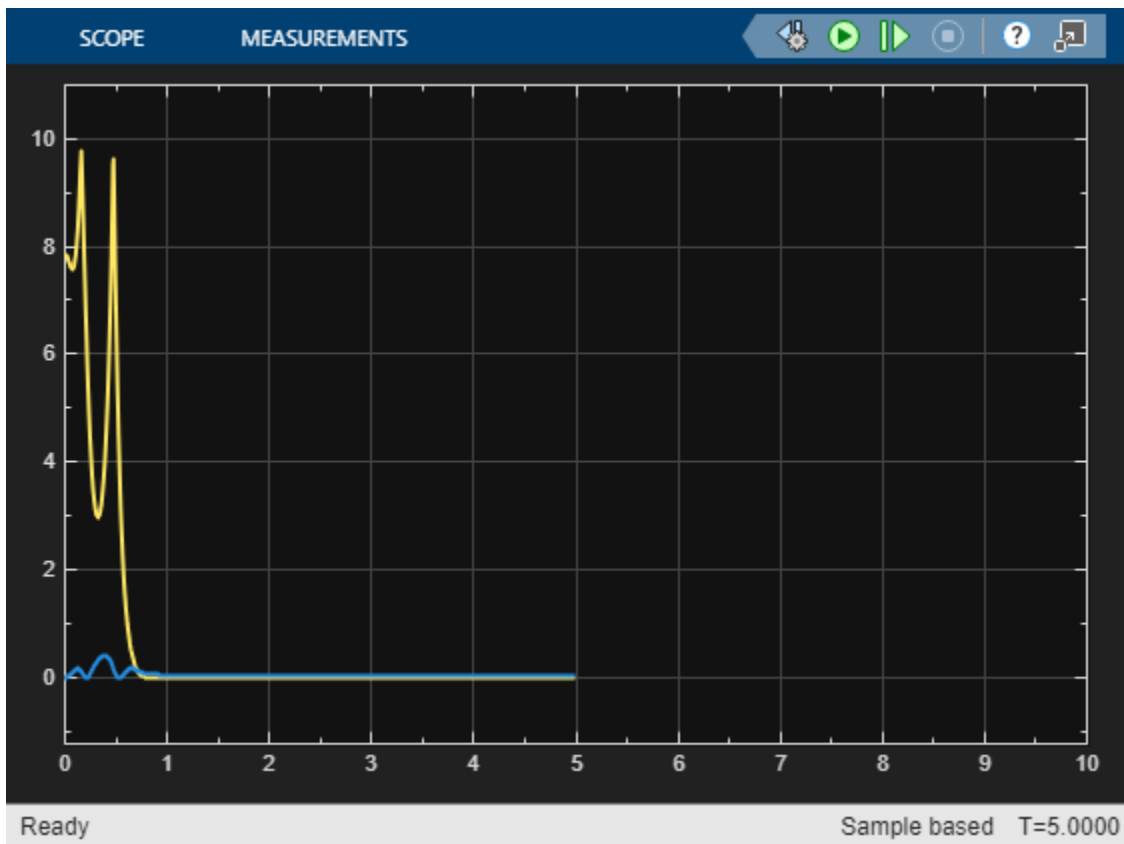
```
rng(testingSeed)
```

Define simulation options.

```
simOpts = rlSimulationOptions( ...
    MaxSteps=maxSteps, ...
    Numsimulations=numTestEpisodes);
```

Simulate the trained agent.

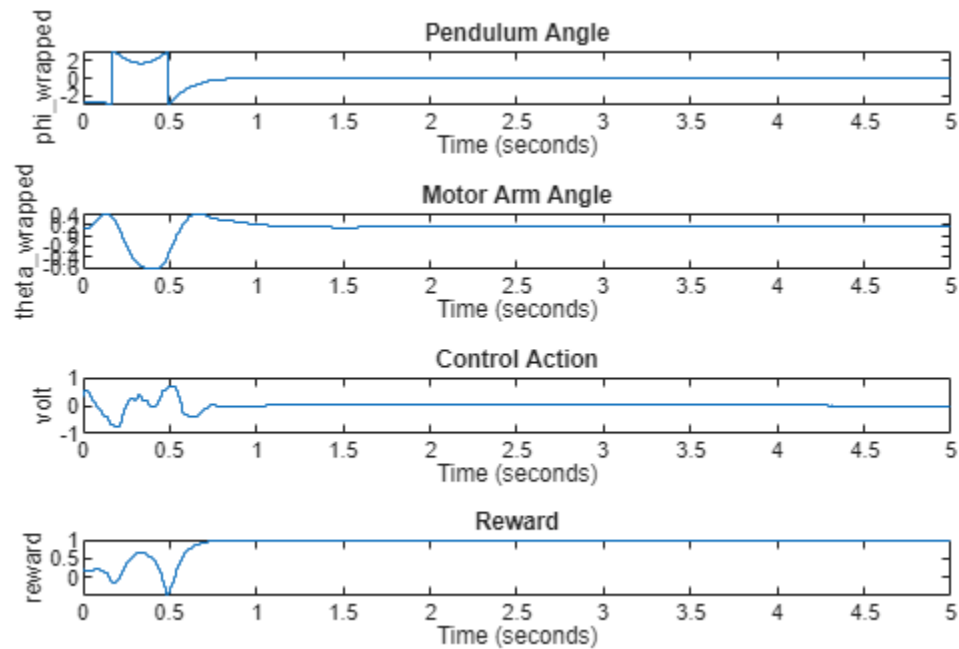
```
offlineSimResult = sim(offlineTD3agent,simEnv,simOpts);
```



Plot a sample trajectory of the angles, control action, and reward.

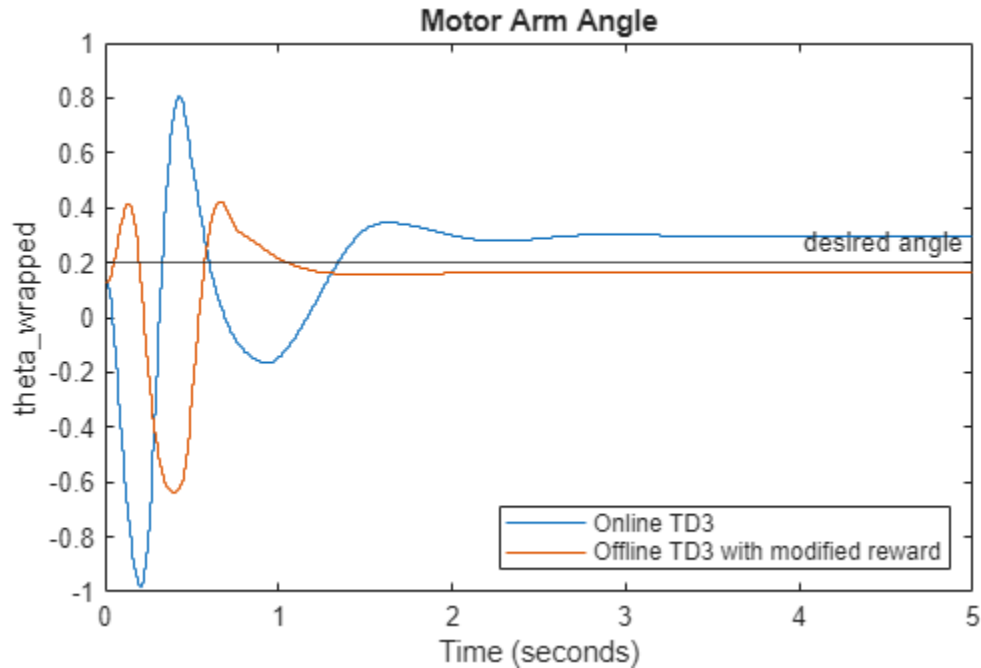
```
% Extract signals.
offlineSimInfo = offlineSimResult(1).SimulationInfo(1);
offlinePhi = ...
    get(offlineSimInfo.logout, "phi_wrapped");
offlineTheta = ...
    get(offlineSimInfo.logout, "theta_wrapped");
offlineAction = ...
    get(offlineSimInfo.logout, "volt");
offlineReward = ...
    get(offlineSimInfo.logout, "reward");

% Plot values.
figure
tiledlayout(4,1)
nexttile
plot(offlinePhi.Values); title("Pendulum Angle")
nexttile
plot(offlineTheta.Values); title("Motor Arm Angle")
nexttile
plot(offlineAction.Values); title("Control Action")
nexttile
plot(offlineReward.Values); title("Reward")
```



You can see that the agent learns to balance the pendulum, as the angle φ , shown in the first plot, is driven to 0. For comparison with the original agent, plot the sampled trajectories of the arm angle in both cases (offline and online) in the same figure, as well as the desired angle.

```
figure
plot(theta.Values);
hold on
plot(offlineTheta.Values);
yline(0.2, '--', 'desired angle');
legend("Online TD3", "Offline TD3 with modified reward", ...
      location="southeast")
title("Motor Arm Angle")
```



As expected, the TD3 agent trained offline does not drive the motor arm back to 0 because you change the desired angle from 0 to 0.2 rad in the new reward function. Note that online agent drives the motor arm to a position in which $\theta = 0.297$ rad (instead of the front position in which $\theta = 0$), despite θ being penalized with a negative reward. Similarly the offline agent drives the motor arm to a position in which $\theta = 0.167$ rad, (instead of the desired position $\theta = 0.2$). More training or other modifications of the reward function could achieve a more precise behavior. For example, you can experiment with the weights of each term in the reward function.

Helper Functions

The function `localEpisodeFinishedFcn` selects the data that should be logged in every training episode. The function is used by the `rlDataLogger` object.

```
function dataToLog = localEpisodeFinishedFcn(data)
dataToLog = struct("data",data.Experience);
end
```

The function `localResetFcn` resets the initial angles to a random number and the initial speeds to 0. It is used by the Simulink environment.

```
function in = localResetFcn(in)
theta0 = -pi/4+rand*pi/2;
phi0 = pi-pi/4+rand*pi/2;
in = setVariable(in,"theta0",theta0);
in = setVariable(in,"phi0",phi0);
in = setVariable(in,"dtheta0",0);
in = setVariable(in,"dphi0",0);
end
```

The function `localReadFcn` reads experience data from the logged data files and modifies the reward of each experience. It is used by the `fileDatastore` object.

```

function experiences = localReadFcn(fileName)
data = load(fileName);
experiences = data.episodeData.data{:};
desired = 0.2; % desired angle
for idx = 1:numel(experiences)
    nextObs = experiences(idx).NextObservation;
    theta = atan2(nextObs{1}(1),nextObs{1}(2));
    newReward = ...
        experiences(idx).Reward - 0.1*(-2*desired*theta+desired^2);
    experiences(idx).Reward = newReward;
end
end

```

References

[1] Cazzolato, Benjamin Seth, and Zebb Prime. "On the Dynamics of the Furuta Pendulum." *Journal of Control Science and Engineering* 2011 (2011): 1-8. <https://doi.org/10.1155/2011/528341>.

See Also

Functions

trainFromData | fileDatastore | rlDataLogger

Objects

rlTrainingFromDataOptions | rlBehaviorCloningRegularizerOptions |
rlConservativeQLearningOptions | rlReplayMemory | FileLogger

Related Examples

- "Train Reinforcement Learning Agents" on page 5-3

More About

- "Reinforcement Learning Agents" on page 3-2
- "Getting Started with Datastore"

Train Biped Robot to Walk Using Evolution Strategy-Reinforcement Learning Agents

This example shows how to train a biped robot to walk using an evolution strategy with twin-delayed deep deterministic policy gradient (TD3) reinforcement learning agent. The robot in this example is modeled in Simscape™ Multibody™.

For a related example, see “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144. For more information on these agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

The agent is trained using an evolution strategy reinforcement learning algorithm (ES-RL). The ES-RL method [3] combines cross entropy method (CEM) with off-policy RL algorithms like SAC, DDPG or TD3. CEM-RL is built on the framework of *evolutionary reinforcement learning (ERL)* [4] in which the standard evolution algorithm selects and evolves a population of actors and generate the experiences in the process. These experiences are then added into the reply buffer to train the actor and critic networks.

A general flow of the ES-RL algorithm proceeds as follows:

- 1 A population of actor networks is initialized with random weights. In addition to the population, one additional actor network is initialized alongside a critic network.
- 2 The population of actors is then evaluated in an episode of interaction with the environment.
- 3 The actor and critic are updated on the data buffer populated using population actor evaluation.
- 4 Fitness of all actors is computed through interacting with the environment. The average return over the episode is used as their respective fitness index.
- 5 A selection operator selects a surviving actors in the population based on their relative fitness scores.
- 6 The surviving elite set of actor is used to generate the next population of the actor.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

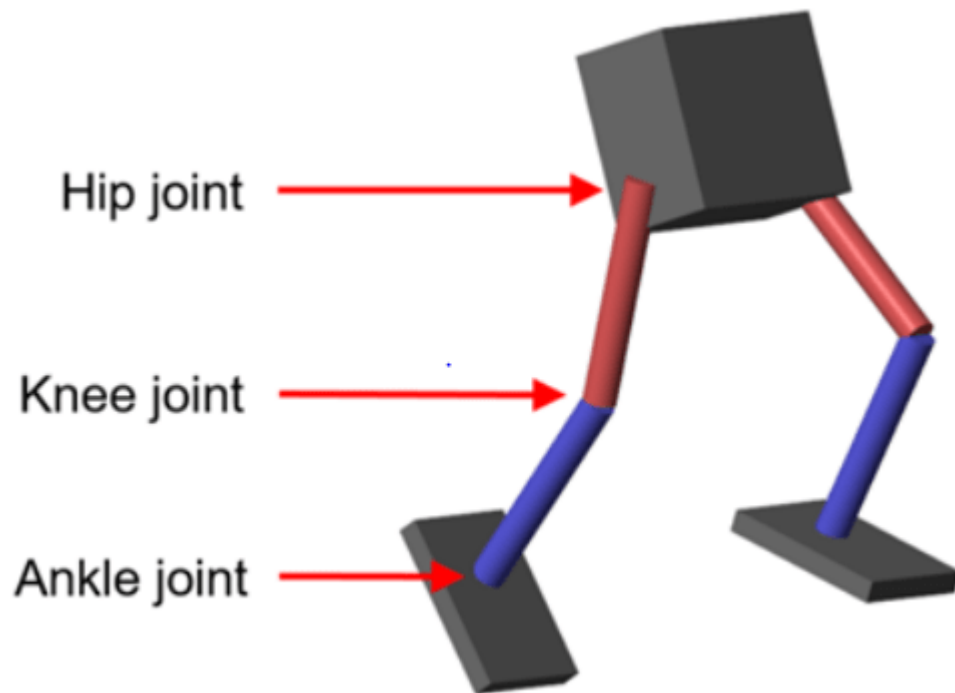
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Biped Robot Model

The reinforcement learning environment for this example is a biped robot. The training goal is to make the robot walk in a straight line using minimal control effort.



Load the parameters of the model into the MATLAB® workspace.

```
robotParametersRL
```

Open the Simulink® model.

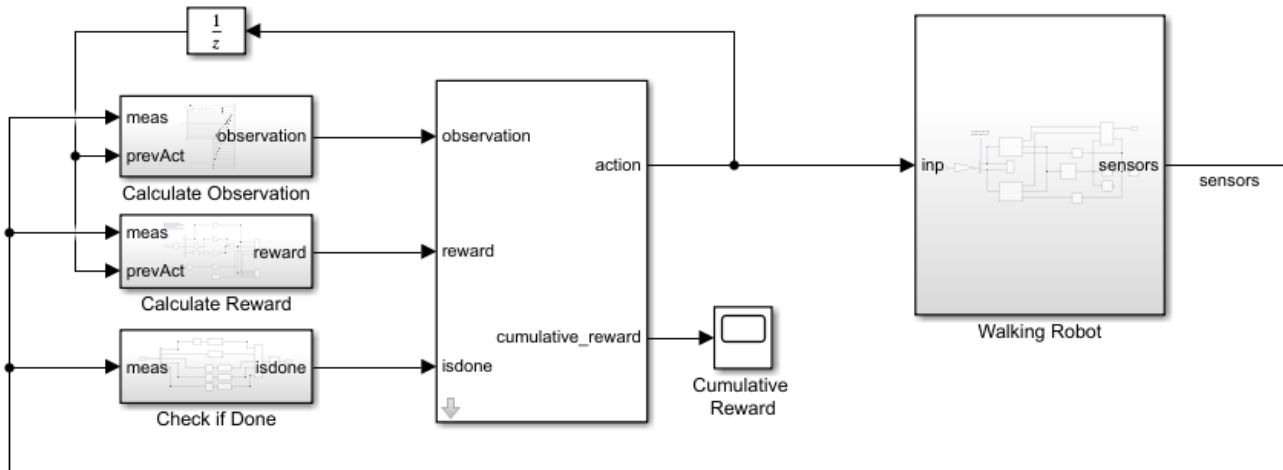
```
mdl = "rlWalkingBipedRobot";  
open_system(mdl)
```

Walking Robot: Reinforcement Learning (2-D)

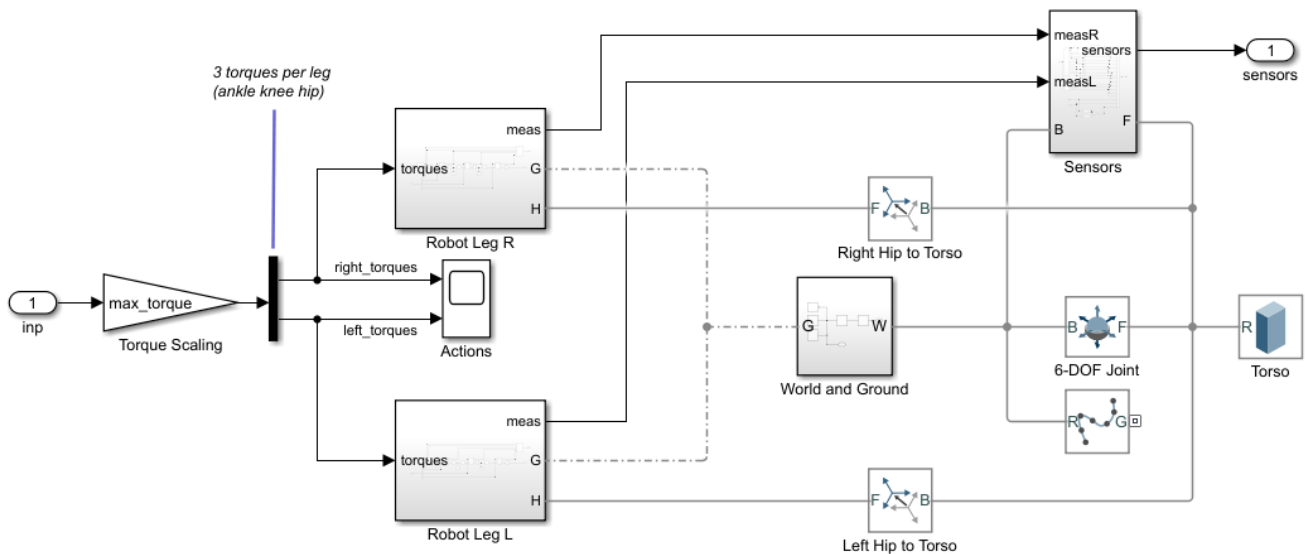
Copyright 2020 The MathWorks, Inc.

[Enable animation](#)

[Disable animation](#)



The robot is modeled using Simscape™ Multibody™.



In this model:

- In the neutral 0 rad position, both legs are straight and the ankles are flat.
- The foot contact is modeled using the Spatial Contact Force (Simscape Multibody) block.
- The agent can control 3 individual joints (ankle, knee, and hip) on both legs of the robot by applying joint torques bounded between $[-3, 3]$ N·m. The actual computed action signals are normalized between -1 and 1.

The environment provides the following 29 observations to the agent.

- Y (lateral) and Z (vertical) translations of the torso center of mass. The translation in the Z direction is normalized to a similar range as the other observations.
- X (forward), Y (lateral), and Z (vertical) translation velocities.
- Yaw, pitch, and roll angles of the torso.
- Yaw, pitch, and roll angular velocities of the torso.
- Angular positions and velocities of the three joints (ankle, knee, hip) on both legs.
- Action values from the previous time step.

The episode terminates if either of the following conditions occur.

- The robot torso center of mass is less than 0.1 m in the Z direction (the robot falls) or more than 1 m in the either Y direction (the robot moves too far to the side).
- The absolute value of either the roll, pitch, or yaw is greater than 0.7854 rad.

The following reward function r_t , which is provided at every time step is inspired by [2].

$$r_t = v_x - 3y^2 - 50\hat{z}^2 + 25\frac{T_s}{T_f} - 0.02\sum_i u_{t-1}^i{}^2$$

Here:

- v_x is the translation velocity in the X direction (forward toward goal) of the robot.
- y is the lateral translation displacement of the robot from the target straight line trajectory.
- $\hat{z}$ is the normalized vertical translation displacement of the robot center of mass.
- u_{t-1}^i is the torque from joint i from the previous time step.
- T_s is the sample time of the environment.
- T_f is the final simulation time of the environment.

This reward function encourages the agent to move forward by providing a positive reward for positive forward velocity. It also encourages the agent to avoid episode termination by providing a constant reward ($25\frac{T_s}{T_f}$) at every time step. The other terms in the reward function are penalties for substantial changes in lateral and vertical translations, and for the use of excess control effort.

Create Environment Object

Create the observation specification.

```
numObs = 29;
obsInfo = rlNumericSpec([numObs 1]);
obsInfo.Name = "observations";
```

Create the action specification.

```
numAct = 6;
actInfo = rlNumericSpec([numAct 1], LowerLimit=-1, UpperLimit=1);
actInfo.Name = "foot_torque";
```

Create the environment object for the walking robot model.

```
blk = mdl + "/RL Agent";  
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo);
```

Specify the helper function `walkerResetFcn` as the reset function for the environment. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in) walkerResetFcn(in, ...  
    upper_leg_length/100, ...  
    lower_leg_length/100, ...  
    h/100);
```

Create TD3 Agent

This example trains a TD3 RL agent using Evolution Strategy based gradient free optimization technique to learn biped locomotion. The TD3 algorithm is an extension of DDPG with improvements that make it more robust by preventing overestimation of Q values [3].

- Two critic networks — TD3 agents learn two critic networks independently and use the minimum value function estimate to update the actor (policy). Doing so avoids overestimation of Q values through the maximum operator in the critic update.
- Addition of target policy noise — Adding clipped noise to value functions smooths out Q function values over similar actions. Doing so prevents learning an incorrect sharp peak of noisy value estimate.
- Delayed policy and target updates — For a TD3 agent, delaying the actor network update allows more time for the Q function to reduce error (get closer to the required target) before updating the policy. Doing so prevents variance in value estimates and results in a higher quality policy update.

The `createTD3Agent` helper function perform the following actions.

- Create the actor and critic networks.
- Specify training options for actor and critic.
- Create actor and critic using the networks and options defined.
- Configure agent specific options.
- Create agent.

Create the TD3 agent.

```
agent = createTD3Agent(numObs,obsInfo,numAct,actInfo,Ts);
```

The structure of the actor and critic networks used for this agent are the same as the ones used for DDPG agent. For details on the creating the TD3 agent, see the `createTD3Agent` helper function. For information on configuring TD3 agent options, see `rlTD3AgentOptions`.

Specify Evolution Strategy Training Options and Train Agent

Fix the random generator seed for reproducibility.

```
rng(0,"twister");
```

The ES-RL training requires the following few important train options to be set.

- `PopulationSize`, number of actor that are evaluated in each generation. We choose 20 actors

- `PercentageEliteSize`, This is the size of the surviving elite population which form parent for next generation actors. For this example, set the size of the elite to 50% of the total population.
- `MaxGenerations` sets maximum number of generations for population to evolve. For this example `MaxGenerations` is set to 2000.
- `MaxStepsPerEpisode`, sets maximum simulation steps for each run of episode for each actor.
- `TrainEpochs` sets the number for epochs of gradient based update of the reinforcement learning agent.
- `WeightedMixing` set the update method for mean and standard deviation.
- Display the training progress in the Reinforcement Learning Training Monitor (set the `Plots` option) and for the command line display (set the `Verbose` option).
- Terminate the training when the agent reaches an average score of 250.

For more information and additional options, see `rlEvolutionStrategyTrainingOptions`.

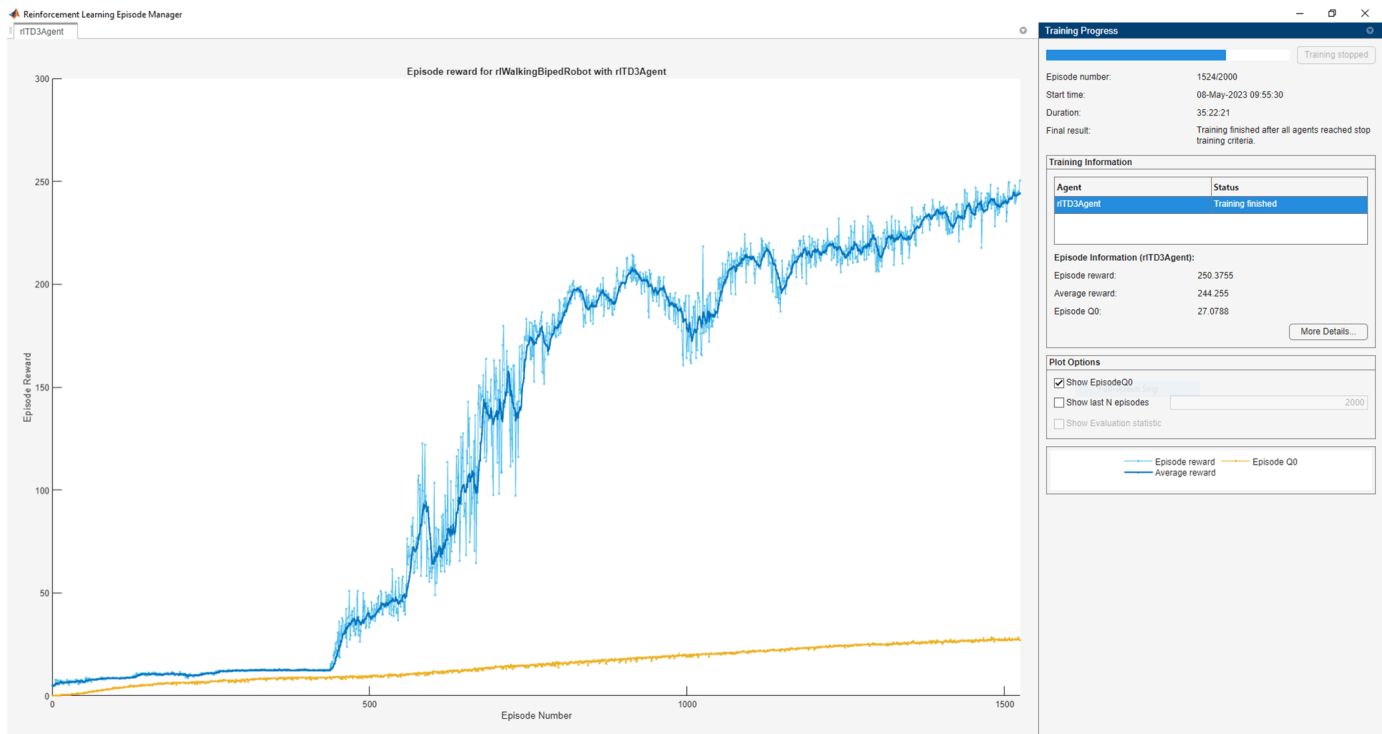
```
maxGeneration = 2000;
maxSteps = floor(Tf/Ts);
trainOpts = rlEvolutionStrategyTrainingOptions( ...
    "MaxGeneration", maxGeneration, ...
    "MaxStepsPerEpisode", maxSteps, ...
    "ScoreAveragingWindowLength", 10, ...
    "Plots", "none", ...
    "StopTrainingCriteria", "GenerationReward", ...
    "StopTrainingValue", 250, ...
    "PopulationSize", 25, ...
    "PercentageEliteSize", 50, ...
    "ReturnedPolicy", 'BestPolicy', ...
    "Verbose", 1, ...
    "SaveAgentCriteria", 'none');

trainOpts.TrainEpochs = 50;
trainOpts.EvaluationsPerIndividual = 1;

trainOpts.PopulationUpdateOptions.UpdateMethod = "WeightedMixing";
trainOpts.PopulationUpdateOptions.InitialStandardDeviation = 0.25;
trainOpts.PopulationUpdateOptions.InitialStandardDeviationBias = 0.25;
```

Train the agent using the `trainWithEvolutionStrategy` function. This process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true.

```
doTraining = ;
if doTraining
    % Set UseParallel to false
    trainOpts.UseParallel = 0;
    % Train the agent.
    trainingStats = trainWithEvolutionStrategy(agent, env, trainOpts);
else
    % Load a pretrained agent.
    load("rlWalkingBipedRobotESTD3.mat", "saved_agent")
end
```



By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` property to `true`.

To validate the performance of the trained agent, simulate it within the biped robot environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=maxSteps);
experience = sim(env,saved_agent,simOptions);
```

Parallel Training Options

Fix the random generator seed for reproducibility.

```
rng(0,"twister");
```

ES-RL can train the agents in parallel. The actor evaluation over the environment can be parallelized to accelerate the computation and achieve faster training time. To train the agent in parallel set `UseParallel` option to `true`. The training curve for ES-RL with parallel train for biped locomotion is shown below, the training time is reduced significantly compared to non-parallel train option.

```
maxGeneration = 2000;
maxSteps = floor(Tf/Ts);
trainOpts = rlEvolutionStrategyTrainingOptions( ...
    "MaxGeneration", maxGeneration, ...
    "MaxStepsPerEpisode", maxSteps, ...
    "ScoreAveragingWindowLength", 10, ...
    "Plots", "none", ...
    "StopTrainingCriteria", "AverageReward", ...
    "StopTrainingValue", 350, ...
    "PopulationSize", 25, ...
    "PercentageEliteSize", 50, ...
```

```

    "ReturnedPolicy", 'BestPolicy', ...
    "Verbose", 1, ...
    "SaveAgentCriteria", 'AverageReward');

trainOpts.TrainEpochs = 50;
trainOpts.EvaluationsPerIndividual = 1;

% Set UseParallel to True for parallel training
trainOpts.UseParallel = ;

trainOpts.PopulationUpdateOptions.UpdateMethod = "WeightedMixing";
trainOpts.PopulationUpdateOptions.InitialStandardDeviation = 0.25;
trainOpts.PopulationUpdateOptions.InitialStandardDeviationBias = 0.25;

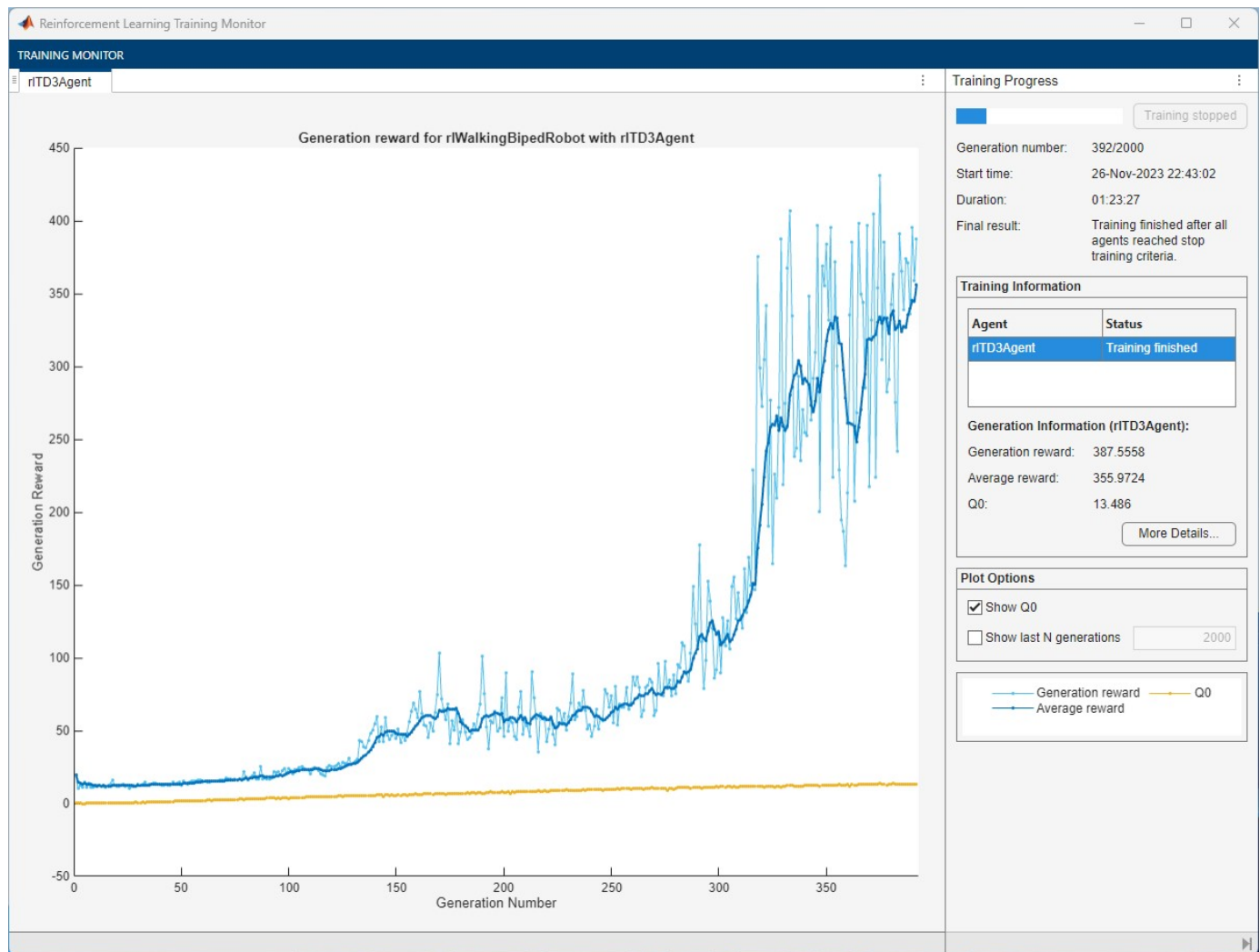
```

To simulate a parallel pretrained agent set doTraining to false. To train the parallel agent yourself, set doTraining to true.

```

doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = trainWithEvolutionStrategy(agent, env, trainOpts);
else
    % Load a pretrained agent.
    load("rlWalkingBipedRobotTD3_ParallelTraining.mat", "agent")
end

```



Simulate Parallel-Trained Agent

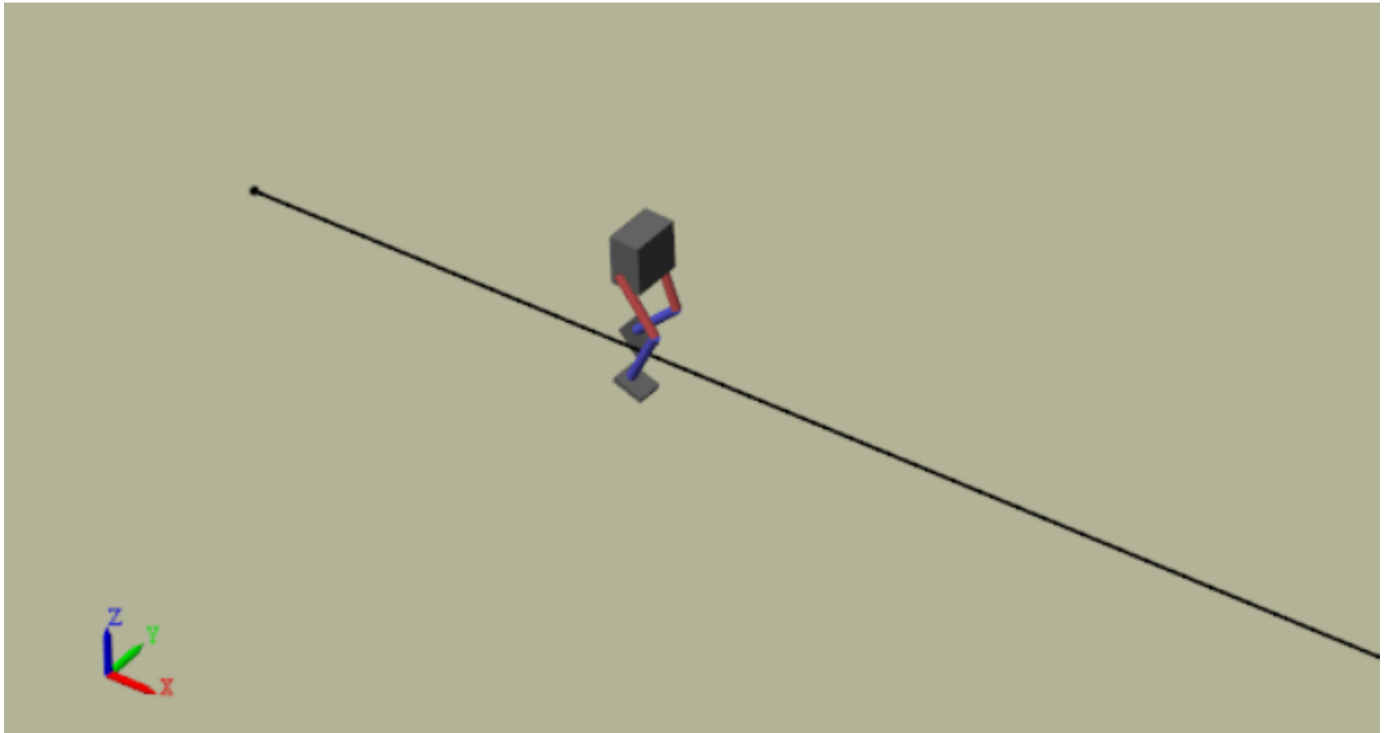
Fix the random generator seed for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` property to `true`.

To validate the performance of the trained agent, simulate it within the biped robot environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=maxSteps);
experience = sim(env, agent, simOptions);
```

Display the cumulative reward over the simulation episode.

```
sum(experience.Reward.Data)
```

Training the agent for longer allows the critics to steadily improve their estimates and potentially converge to the true Q-value function.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

- [1] Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous Control with Deep Reinforcement Learning." Preprint, submitted July 5, 2019. <https://arxiv.org/abs/1509.02971>.
- [2] Heess, Nicolas, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, et al. "Emergence of Locomotion Behaviours in Rich Environments." Preprint, submitted July 10, 2017. <https://arxiv.org/abs/1707.02286>.
- [3] Fujimoto, Scott, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods." Preprint, submitted October 22, 2018. <https://arxiv.org/abs/1802.09477>
- [4] Pourchot, Aloïs, and Olivier Sigaud. "CEM-RL: Combining evolutionary and gradient-based methods for policy search." <https://arxiv.org/abs/1810.01222>

[5] Khadka, Shauharda, and Kagan Tumer. "Evolution-guided policy gradient in reinforcement learning." *Advances in Neural Information Processing Systems* 31 (2018). <https://arxiv.org/abs/1805.07917>

See Also

Functions

`trainWithEvolutionStrategy` | `train` | `sim`

Objects

`rlEvolutionStrategyTrainingOptions` | `rlTrainingOptions` | `rlDDPGAgent` | `rlTD3Agent`

More About

- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Transfer Learning: Fine-Tune DQN Agent for Pendulum Swing-Up from Earth to Mars

This example shows how to use transfer learning to retrain a model trained for one task to perform a similar task. Using transfer learning can save training time and computing resources compared to training a model from scratch.

In this example, you take a deep Q-learning network (DQN) agent that was previously trained to swing up and balance a pendulum in Earth's gravity conditions and adapt it to swing the pendulum in Mars' gravity conditions. To do this, you freeze the beginning layers of the critic network and retrain only the final layer, which allows the agent to quickly recover performance under the new dynamics. The main steps for transfer learning are the following:

- 1 Load a pretrained agent.
- 2 Extract the critic and actor networks from the agent and freeze the learnable parameters of their initial layers.
- 3 Retrain the agent to adapt the unfrozen layers to the new environment.

For a general overview of transfer learning in the context of supervised learning, see Transfer Learning. For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32. For an example that shows how to train a DQN agent from scratch, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70.

Specify Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some section in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Specify the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

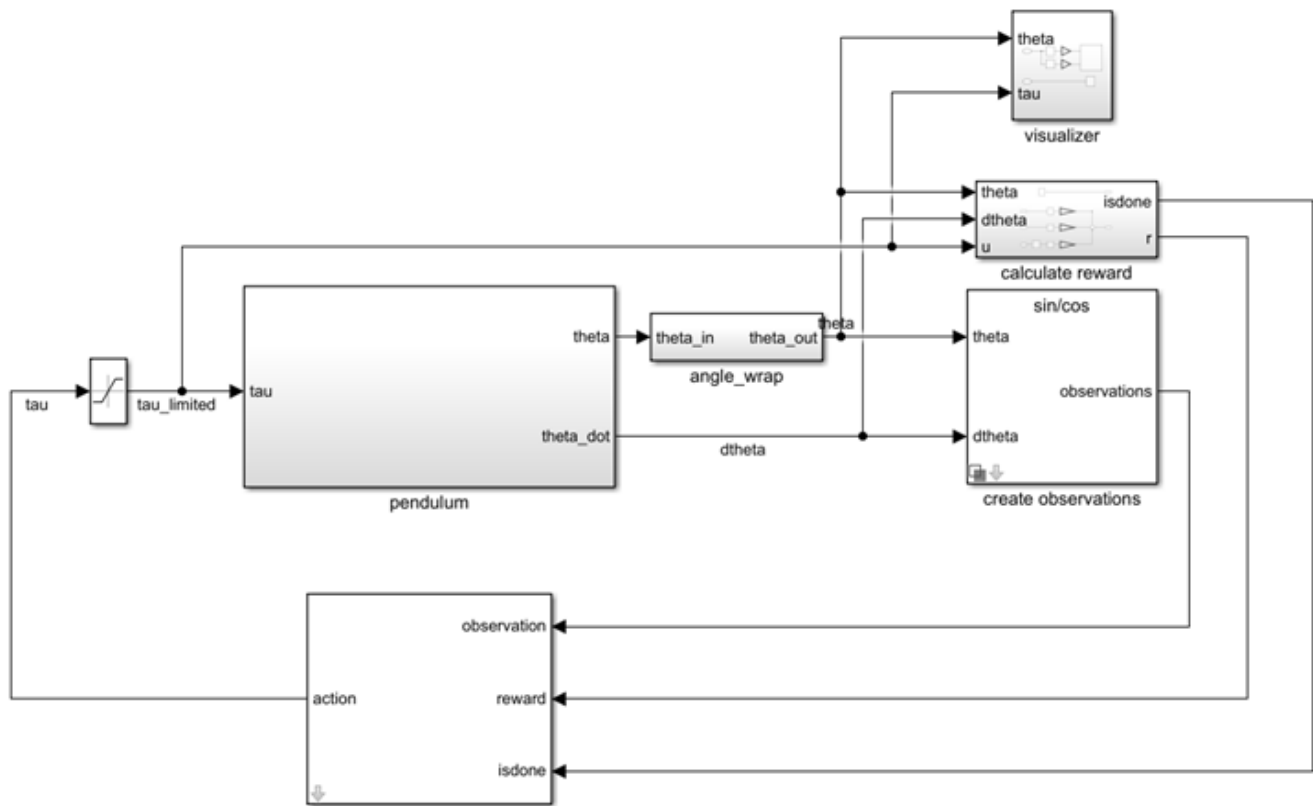
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Pendulum Swing-Up Environment Model

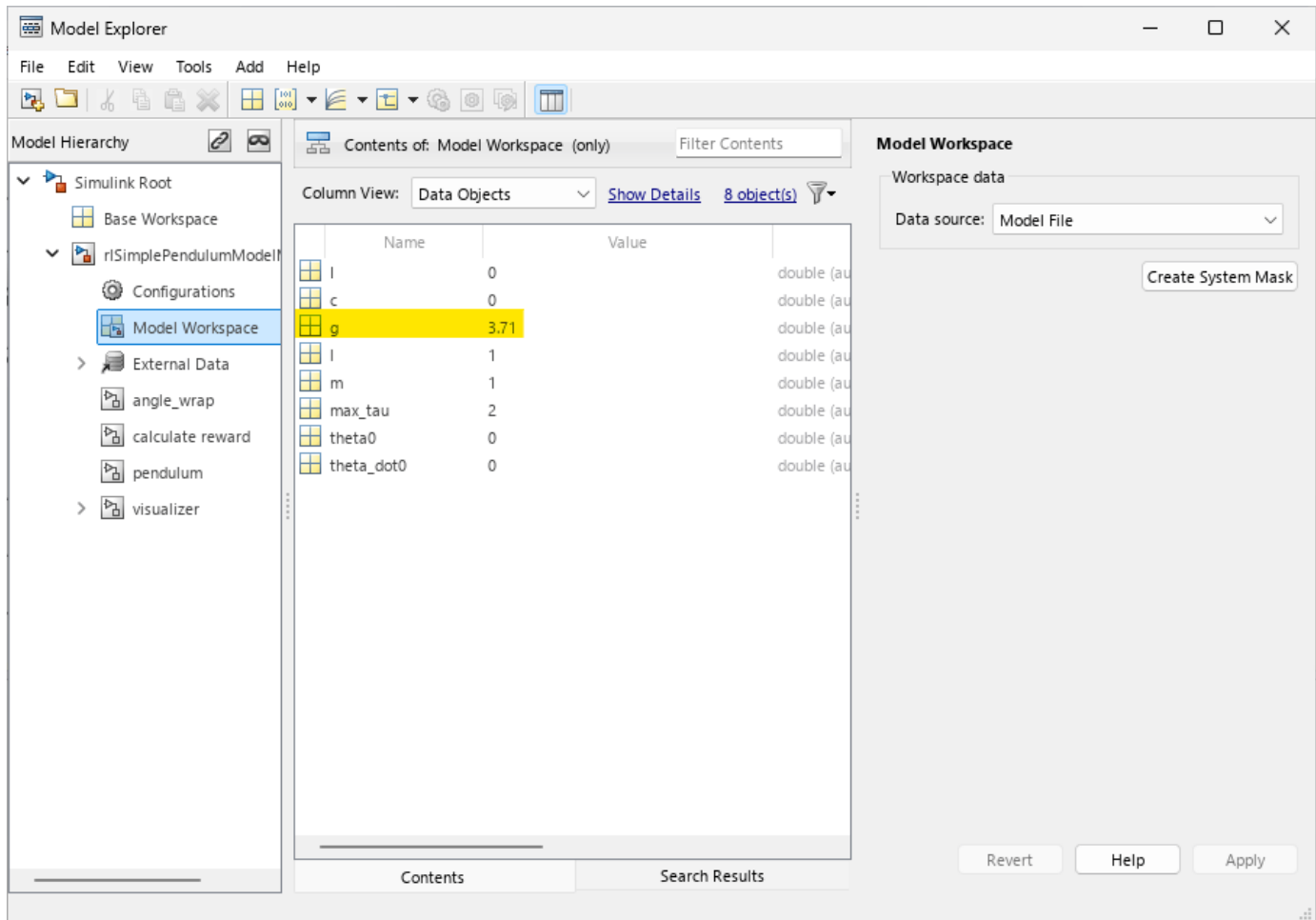
In this example, the reinforcement learning environment is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

Open the Simulink model configured for Mars, which uses a gravitational acceleration of 3.71 (instead of 9.81) meters per second squared.

```
mdl = "rlSimplePendulumModelMars";  
open_system(mdl)
```



To access the gravitational acceleration g , open the model workspace.



In this model:

- The balanced, upright pendulum position is zero radians, and the downward, hanging pendulum position is π radians.
- The torque action signal from the agent to the environment can be either -2, 0, or 2 N·m.
- The observations from the environment are the sine, cosine, and derivative of the pendulum angle.
- The reward r_t , which the environments provides to the agent at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

where:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Define the observation specification `obsInfo` and the action specification `actInfo`.

```
obsInfo = rlNumericSpec([3 1]);  
obsInfo.Name = "Observations";  
  
actInfo = rlFiniteSetSpec([-2 0 2]);  
actInfo.Name = "Torque";
```

Use `rlSimulinkEnv` to create an environment object from the Simulink model.

```
env = rlSimulinkEnv mdl, mdl + "/RL Agent", obsInfo, actInfo);
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function uses `setVariable` (Simulink) function to set the model workspace variable `theta0` to `pi`. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in)setVariable(in, "theta0", pi, "Workspace", mdl);
```

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;  
Tf = 20;
```

Load Pretrained Agent and Freeze Initial Layers

In this example, you reuse the critic network from the agent trained to swing-up and balance the pendulum with Earth gravity conditions. Specifically, you freeze the initial layers of this critic, and retrain only the last fully connected output layer. Freezing the first two fully connected layers preserves the mapping from the observations to the intermediate features produced by these layers. Retraining the output layer allows the mapping from the intermediate features to the value output to adapt to Mars gravity conditions.

First, load the pretrained DQN agent that was trained with Earth's gravity. For more details, see “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70.

```
load("SimulinkPendulumDQNMulti.mat", "agent")
```

Extract learnable parameters for later comparison.

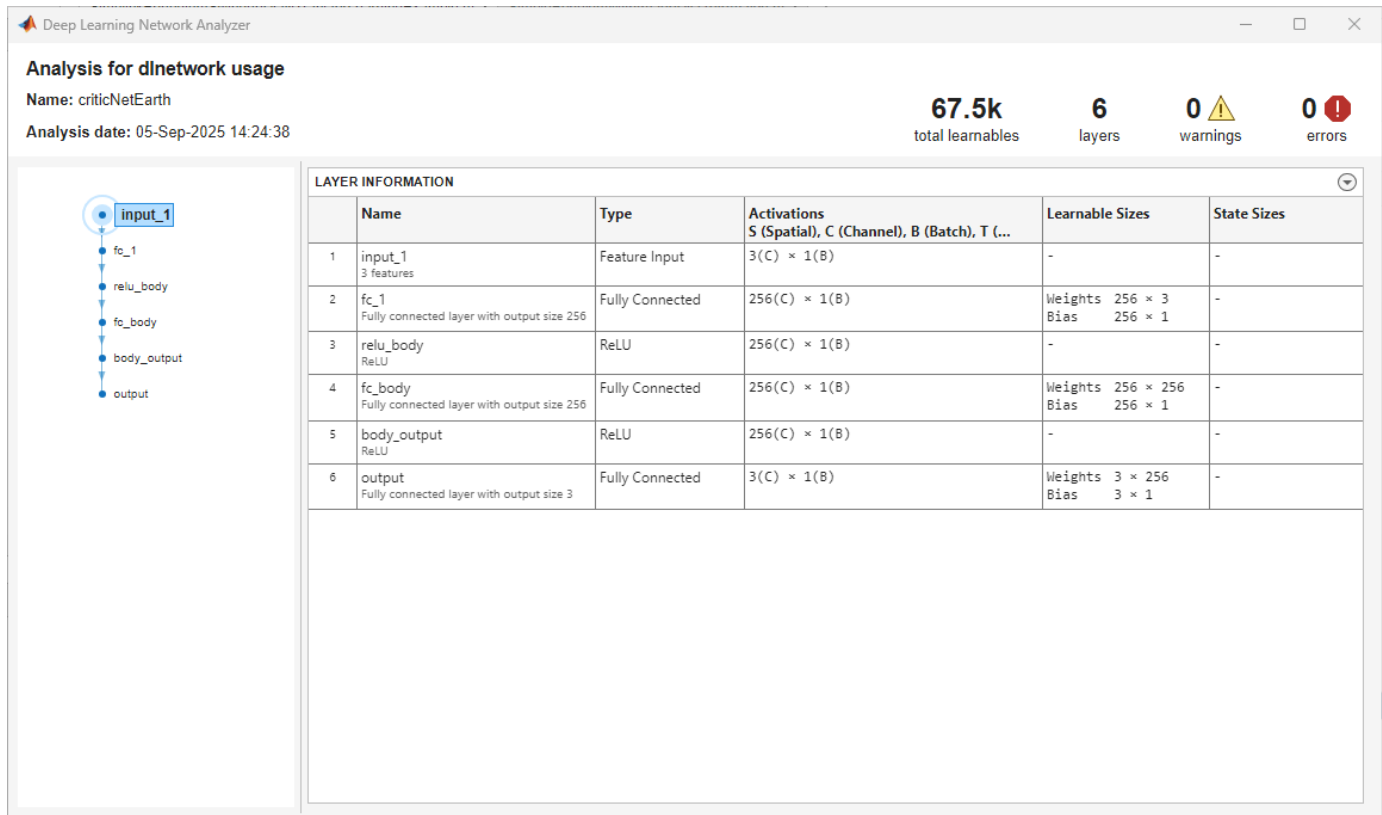
```
criticEarth = getCritic(agent);  
criticNetEarth = getModel(criticEarth);  
learnablesEarth = criticNetEarth.Learnables;
```

Extract the critic, the critic network, and the layers of the critic network from the agent. For more information, see `getModel` and `getCritic`.

```
critic = getCritic(agent);  
criticNet = getModel(critic);  
CriticNetLayersMars = criticNet.Layers;
```

Visualize the pretrained DQN agent critic network. For more information, see `analyzeNetwork`.

```
analyzeNetwork(criticNet);
```



To freeze the initial fully connected layers (indexed by the integers 2 and 4, respectively), set their `WeightLearnRateFactor` and `BiasLearnRateFactor` properties to 0. For the output fully connected layer (indexed by the integer 6), leave these properties to their default values.

```
CriticNetLayersMars(2,1).WeightLearnRateFactor = 0;
CriticNetLayersMars(2,1).BiasLearnRateFactor = 0;
CriticNetLayersMars(4,1).WeightLearnRateFactor = 0;
CriticNetLayersMars(4,1).BiasLearnRateFactor = 0;
criticNetMars = dlnetwork(CriticNetLayersMars);
```

Set the modified network as the new critic approximation model, and set the agent critic to the updated critic. For more information, see `setModel` and `setCritic`.

```
critic = setModel(critic,criticNetMars);
setCritic(agent,critic);
```

During training, DQN agents use the epsilon-greedy algorithm to explore the action space. Specifically, in each training episode, the agent either chooses a random action (exploration) or applies a deterministic greedy policy to try to maximize the reward (exploitation). The value of the exploration probability *epsilon* constantly decays during training, which encourages broad exploration early in training, when the greedy policy is unrefined, and increases exploitation later.

Since pretrained layers reduce the need for exploration, you could often use a faster epsilon decay schedule in transfer learning. However, for this example, use keep the same epsilon decay policy that was used to train `agentEarth`. For more information on DQN agent options, see `rlDQNAgentOptions`.

Display the values of the `EpsilonDecay` and `EpsilonMin` properties.

```
agent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay
```

```
ans =  
5.0000e-05
```

```
agent.AgentOptions.EpsilonGreedyExploration.EpsilonMin
```

```
ans =  
0.1000
```

Train DQN Agent

To train the agent, first, specify the training options. For this example:

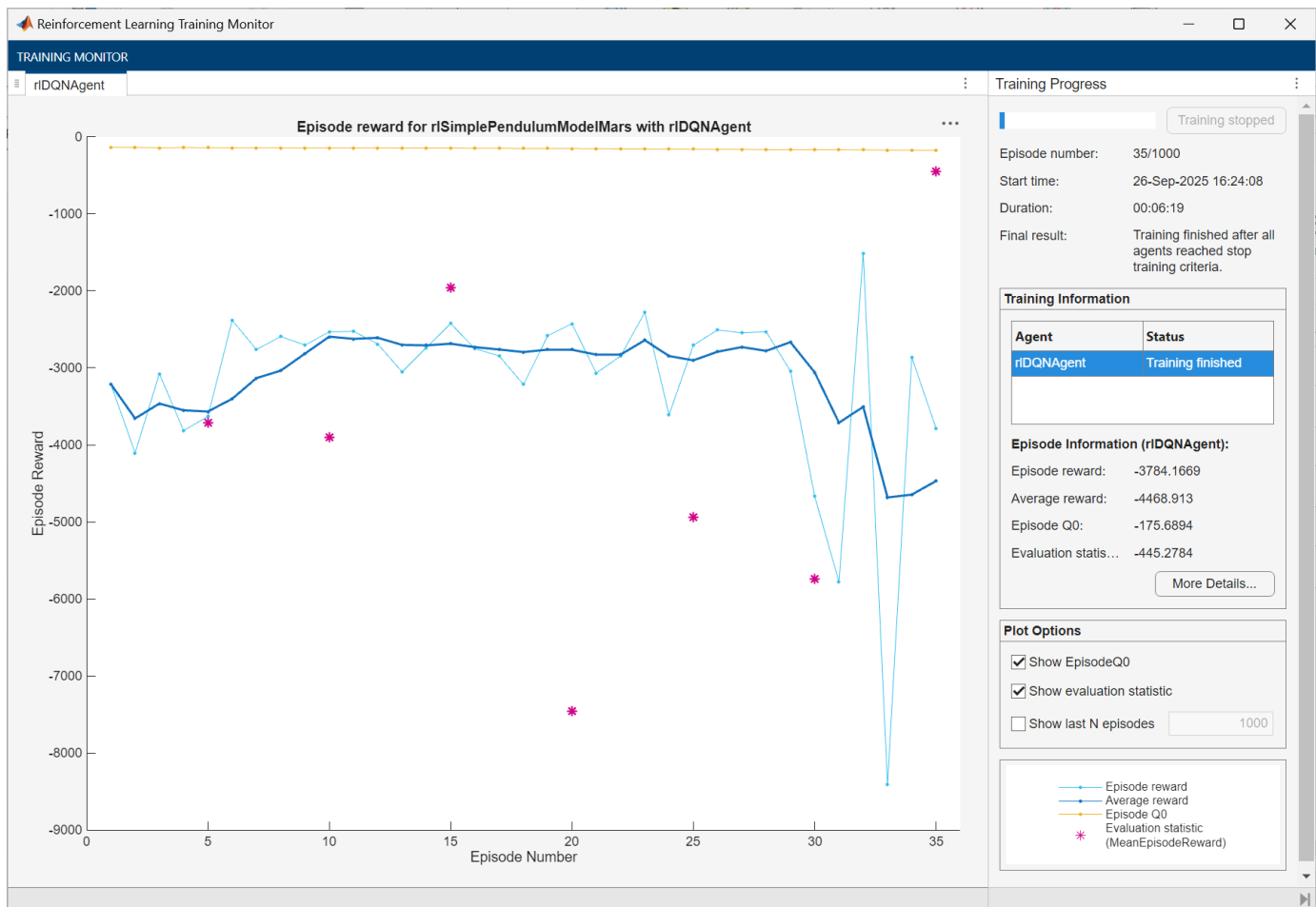
- Run each training for a maximum of 1000 episodes, with each episode lasting a maximum of 500 time steps.
- Evaluate the performance of the greedy policy every 5 training episodes.
- Disable the command line display.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box.
- Stop the training when the agent receives an average cumulative reward greater than -1100 when evaluating the greedy policy. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.

For more information on training options, see `rlTrainingOptions`.

```
trainingOptions = rlTrainingOptions( ...  
    MaxEpisodes=1000, ...  
    MaxStepsPerEpisode=500, ...  
    ScoreAveragingWindowLength=5, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=-1100);
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Agent Evaluator: evaluate the performance of the greedy policy  
    % every 5 training episodes.  
    evl = rlEvaluator(EvaluationFrequency=5, ...  
        NumEpisodes=1, RandomSeeds=0); %#ok<UNRCH>  
  
    % Train the agent.  
    trainingStats = train(agent,env,trainingOptions, ...  
        Evaluator=evl);  
else  
    % Load the pretrained agent for the example.  
    load("trainedMarsAgent.mat","agent");  
end
```

Simulate DQN Agent

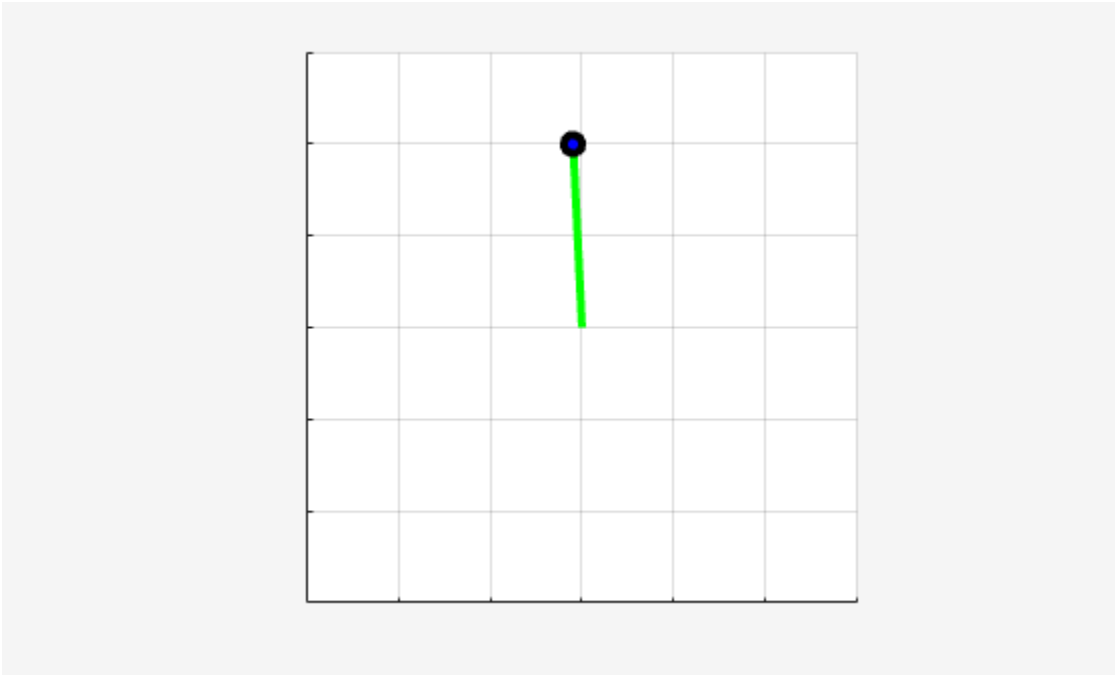
Specify the random stream for reproducibility.

```
rng(0, "twister");
```

In simulation, the agent uses the policy behavior specified its `UseExplorationPolicy` property, which is set to `false` by default. If you want to use an exploratory policy in simulation, set `UseExplorationPolicy` to `true`.

To validate the performance of the trained agent, simulate it within the pendulum environment for 500 steps. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, agent, simOptions);
```



Display the total reward.

```
totalRwd = sum(experience.Reward)
```

```
totalRwd =  
-445.2784
```

Compare Critic Learnable Parameters of Both Agents

To compare the critics' learnable parameters of the Earth and Mars agents' critic networks, compute the Euclidean (L2) norm of their differences.

```
% Extract critic networks.  
criticMars = getCritic(agent);  
criticNetMars = getModel(criticMars);  
  
% Access learnables.  
learnablesMars = criticNetMars.Learnables;  
  
% Initialize comparison table.  
n = height(learnablesEarth);  
layerName = strings(n,1);  
paramType = strings(n,1); % Weights/Bias/etc.  
sizeStr = strings(n,1);  
absDiff = zeros(n,1);  
  
% Fill comparison table.  
for ii = 1:n  
    layerName(ii) = learnablesEarth.Layer(ii);  
    paramType(ii) = learnablesEarth.Parameter(ii);  
  
    paramEarth = learnablesEarth.Value{ii};  
    paramMars = learnablesMars.Value{ii};
```

```

% Extract data from dldarray.
paramEarth = extractdata(paramEarth);
paramMars = extractdata(paramMars);

% Compute size and Euclidean norm of parameter difference.
sizeStr(ii) = strjoin(string(size(paramEarth)), 'x');
absDiff(ii) = norm((paramMars(:) - paramEarth(:)), 2);

end

results = table(layerName, paramType, sizeStr, absDiff, ...
    'VariableNames', ...
    {'LayerName', 'ParameterType', 'Size', 'L2NormOfDiff'})

```

```

results=6x4 table
    LayerName    ParameterType    Size    L2NormOfDiff
    _____    _____    _____    _____
    "fc_1"        "Weights"        "256x3"        0
    "fc_1"        "Bias"           "256x1"        0
    "fc_body"     "Weights"        "256x256"      0
    "fc_body"     "Bias"           "256x1"        0
    "output"      "Weights"        "3x256"        14.434
    "output"      "Bias"           "3x1"          2.8885

```

As expected, the norm is zero for frozen layers and nonzero for the weights and bias of the last fully connected layer.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim` | `analyzeNetwork`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlVectorQValueFunction` | `rlTrainingOptions` | `rlSimulationOptions` | `rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train PPO Agent with Curriculum Learning for a Lane Keeping Application” on page 5-164
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217

More About

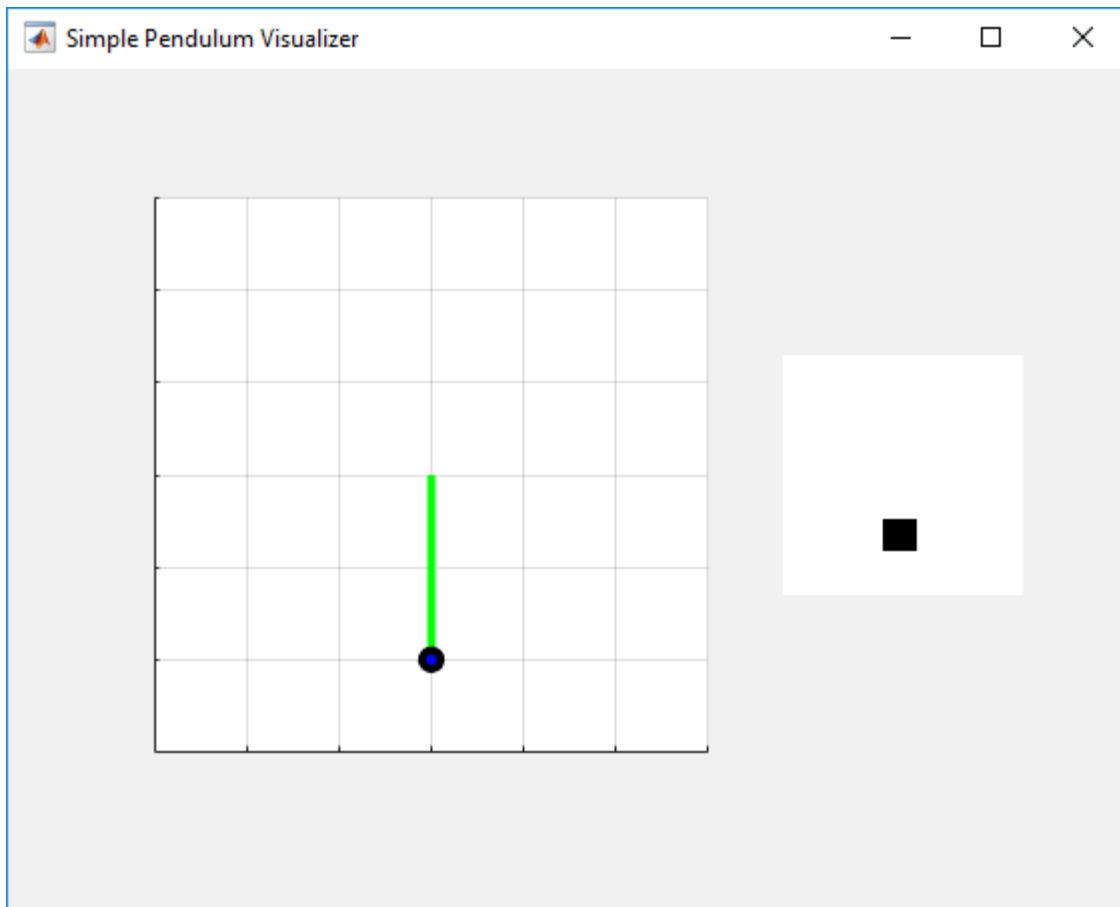
- Transfer Learning
- “Use Predefined Control System Environments” on page 2-42
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3

Create DQN Agent Using Deep Network Designer and Train Using Image Observations

This example shows how to create a deep Q-learning network (DQN) agent that can swing up and balance a pendulum modeled in MATLAB®. In this example, you create the DQN agent using Deep Network Designer. For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32.

Pendulum Swing-Up with Image MATLAB Environment

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.



For this environment:

- The upward balanced pendulum position is zero radians, and the downward hanging position is π radians.
- The torque action signal from the agent to the environment can take any of the five possible integer values from -2 to 2 N·m (counterclockwise positive).

- The observations from the environment are the simplified grayscale image of the pendulum and the pendulum angle derivative.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position (counterclockwise positive).
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on the continuous action space version of this model, see “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("SimplePendulumWithImage-Discrete");
```

The interface has two observations. The first observation, named "pendImage", is a 50-by-50 grayscale image.

```
obsInfo = getObservationInfo(env);  
obsInfo(1)
```

```
ans =  
    rlNumericSpec with properties:  
  
    LowerLimit: 0  
    UpperLimit: 1  
    Name: "pendImage"  
    Description: [0x0 string]  
    Dimension: [50 50]  
    DataType: "double"
```

The second observation, named "angularRate", is the angular velocity of the pendulum.

```
obsInfo(2)
```

```
ans =  
    rlNumericSpec with properties:  
  
    LowerLimit: -Inf  
    UpperLimit: Inf  
    Name: "angularRate"  
    Description: [0x0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

The interface has a discrete action space where the agent can apply one of five possible torque values to the pendulum: -2, -1, 0, 1, or 2 N·m.

```
actInfo = getActionInfo(env)

actInfo =
    rlFiniteSetSpec with properties:

        Elements: [-2 -1 0 1 2]
        Name: "torque"
        Description: [0x0 string]
        Dimension: [1 1]
        DataType: "double"
```

Fix the random generator seed for reproducibility.

```
rng(0)
```

Construct Critic Network Using Deep Network Designer

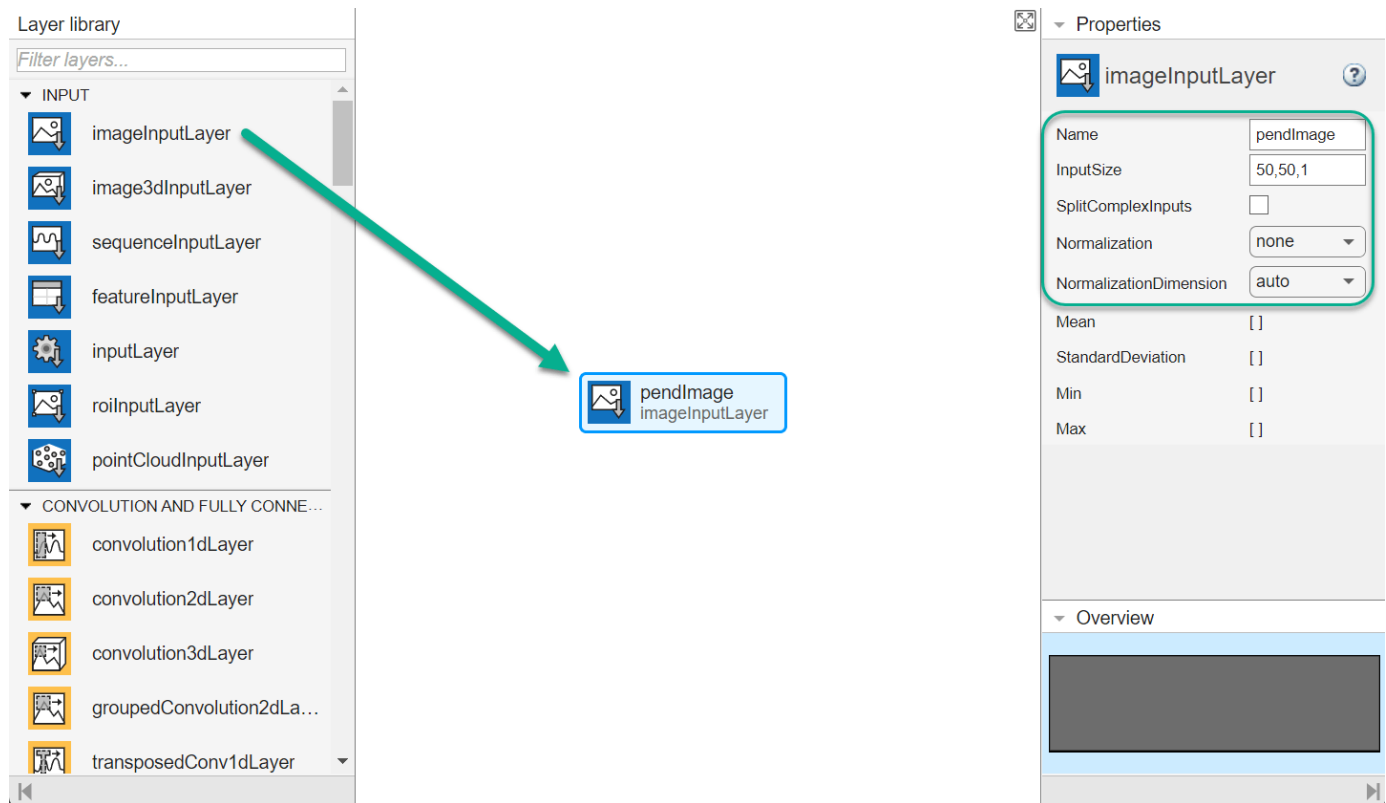
DQN agents use a parameterized Q-value function approximator to estimate the value of the policy. Because a DQN agent has a discrete action space, you can use a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic. However, for this example, use a standard single-output Q-value function critic.

To model the parameterized Q-value function within the critic, use a neural network with three input layers (two for the observation channels, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value). For more information on creating Q-value function approximators based on a deep neural network, see “Create Actors, Critics, and Policy Objects” on page 4-2.

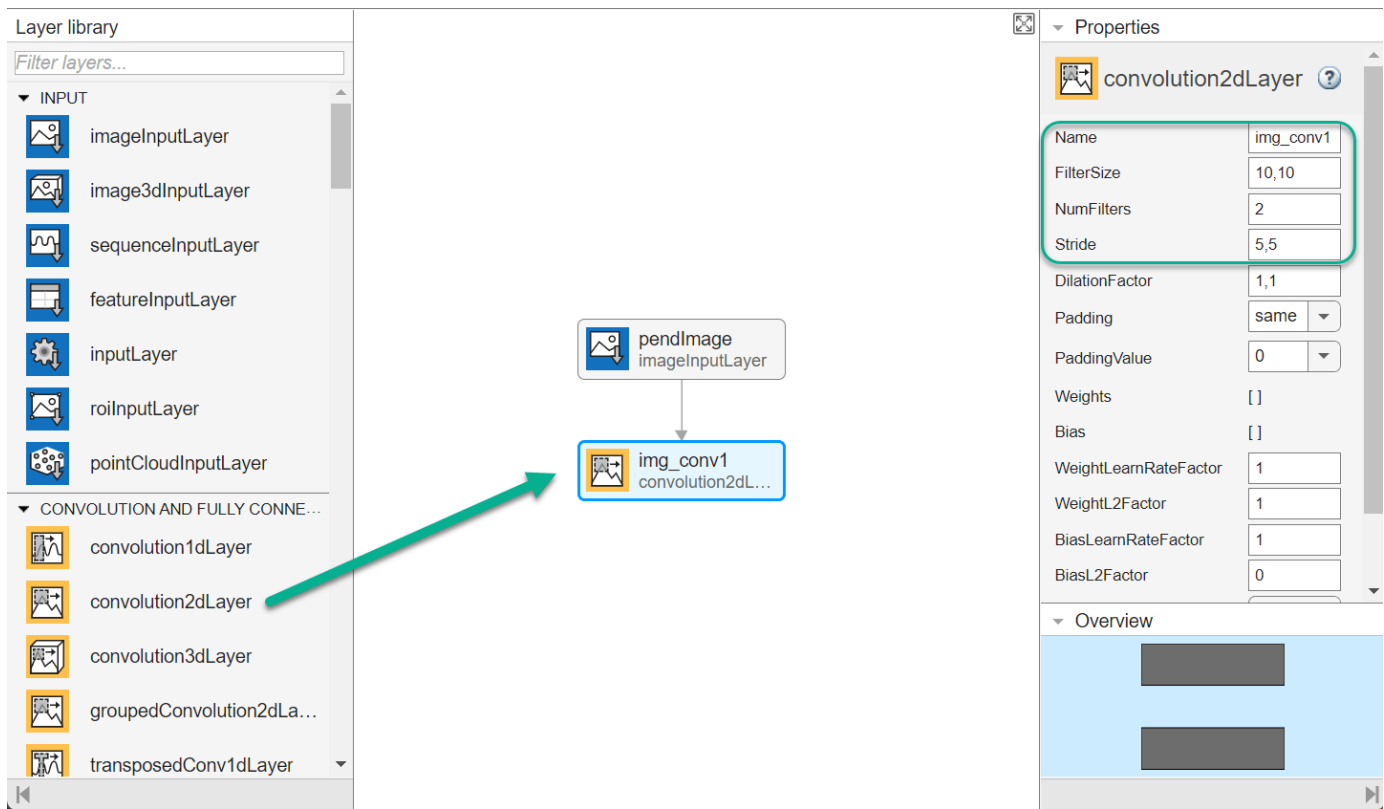
Construct the critic network interactively by using the Deep Network Designer app. To do so, you first create separate input paths for each observation and action. These paths learn lower-level features from their respective inputs. You then create a common output path that combines the outputs from the input paths.

Create Image Observation Path

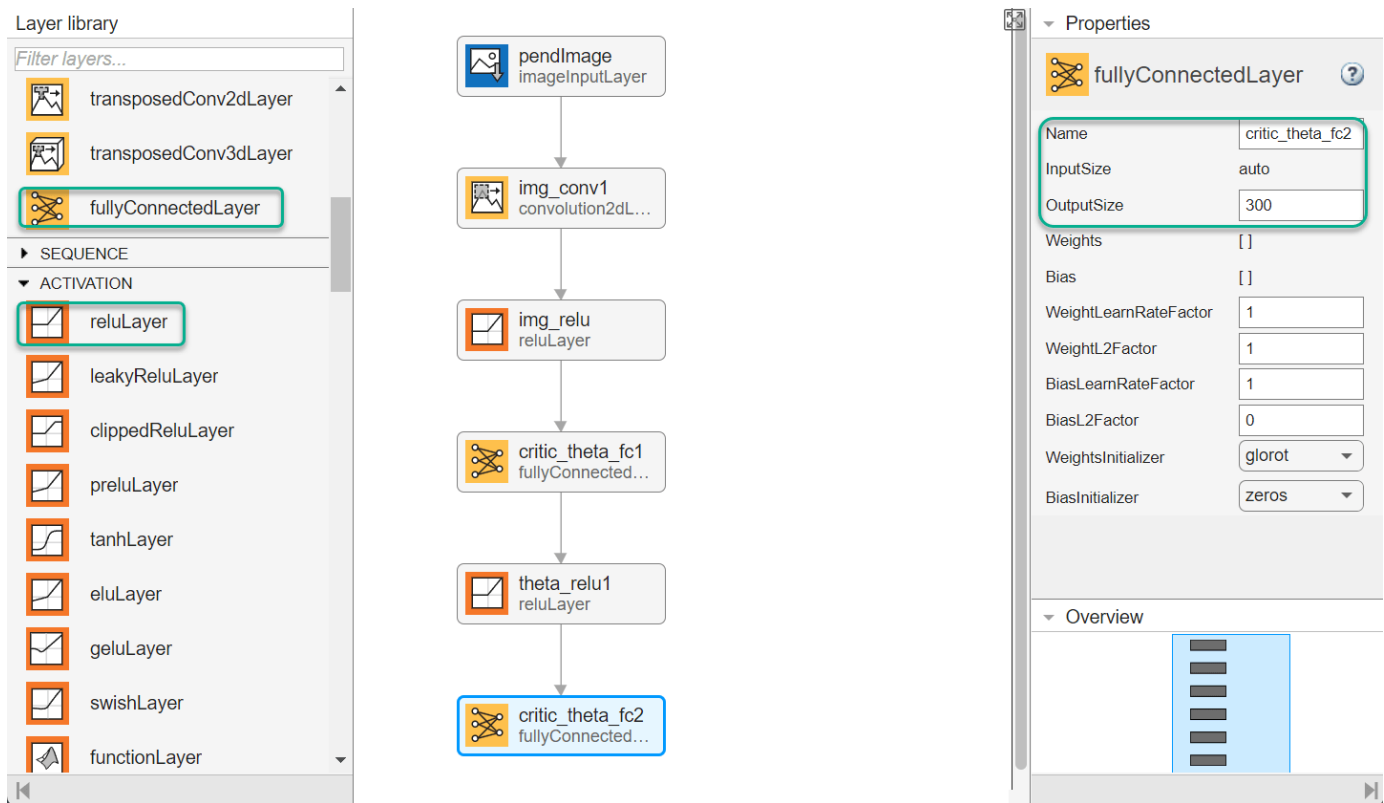
To create the image observation path, first drag an `imageInputLayer` from the **Layer Library** pane to the canvas. Set the layer **InputSize** to 50, 50, 1 for the image observation, and set **Normalization** to none.



Second, drag a `convolution2DLayer` to the canvas and connect the input of this layer to the output of the `imageInputLayer`. Create a convolution layer with 2 filters (**NumFilters** property) that have a height and width of 10 (**FilterSize** property), and use a stride of 5 in the horizontal and vertical directions (**Stride** property).



Finally, complete the image path network with two sets of `reLULayer` and `fullyConnectedLayer` layers. The output sizes of the first and second `fullyConnectedLayer` layers are 400 and 300, respectively.



Create All Input Paths and Output Path

Construct the other input paths and the output path in a similar manner. For this example, use the following options.

Angular velocity path (scalar input):

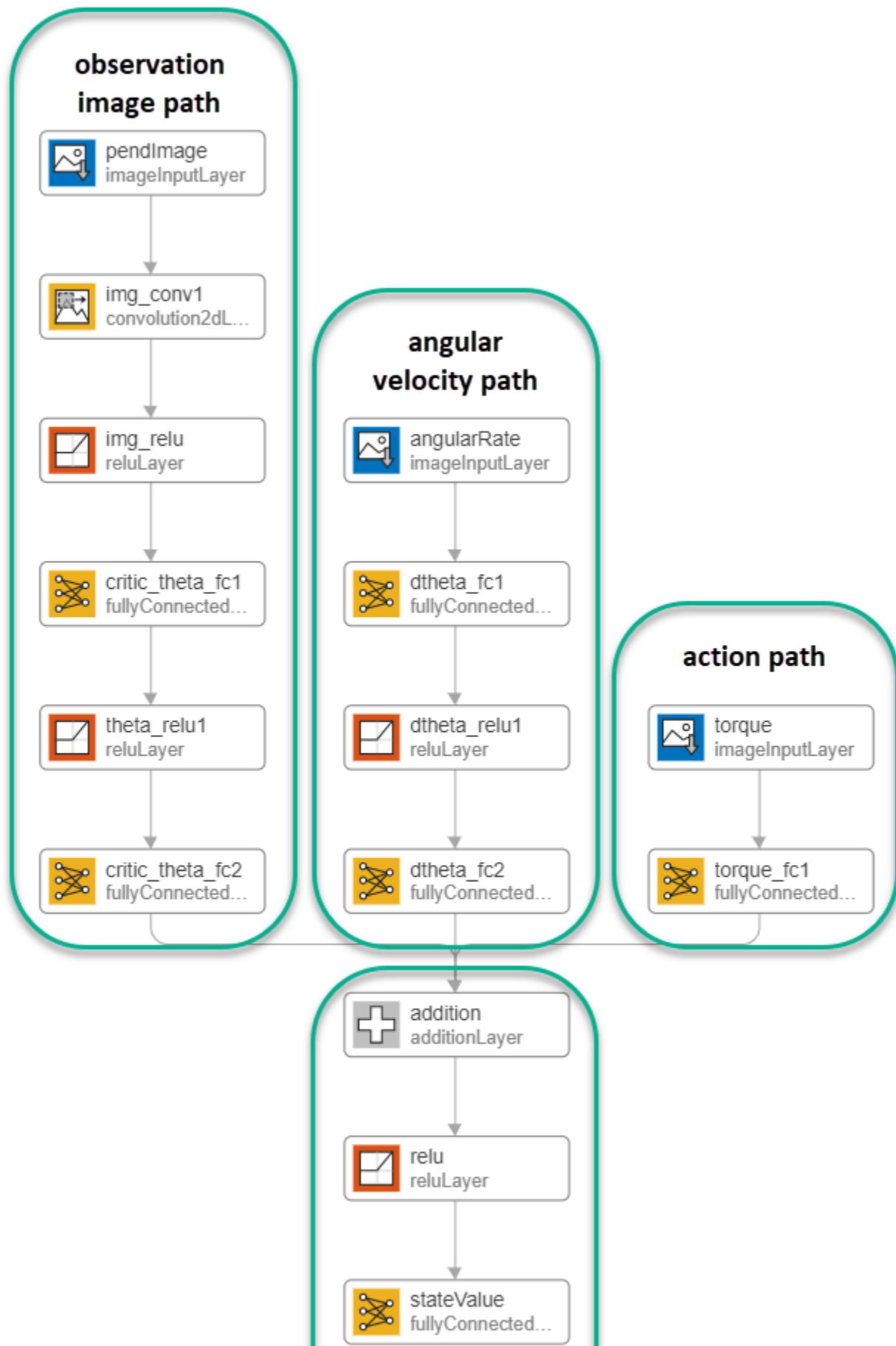
- `imageInputLayer` — Set **InputSize** to 1, 1 and **Normalization** to none.
- `fullyConnectedLayer` — Set **OutputSize** to 400.
- `reLULayer`
- `fullyConnectedLayer` — Set **OutputSize** to 300.

Action path (scalar input):

- `imageInputLayer` — Set **InputSize** to 1, 1 and **Normalization** to none.
- `fullyConnectedLayer` — Set **OutputSize** to 300.

Output path:

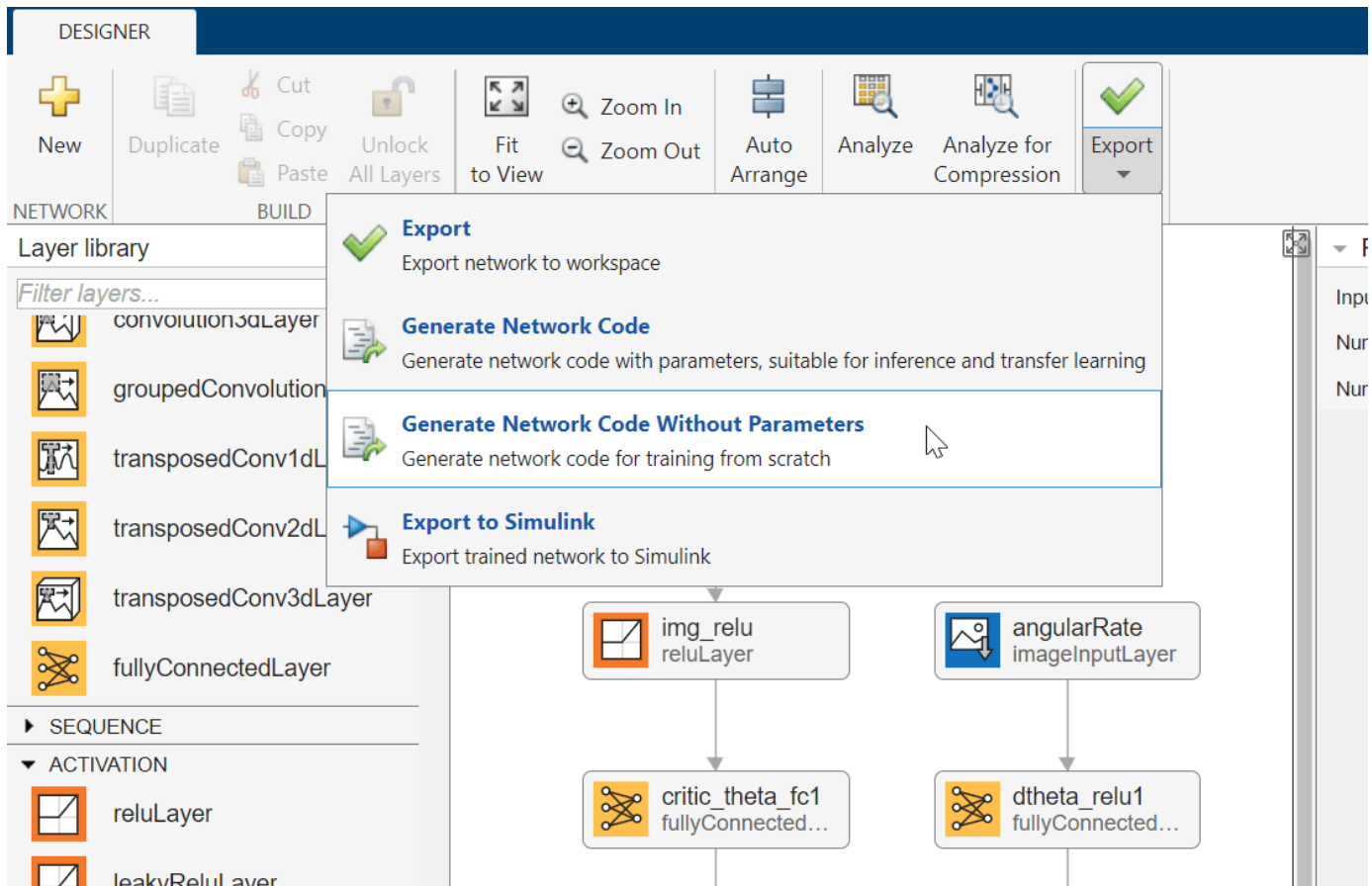
- `additionLayer` — Connect the output of all input paths to the input of this layer.
- `reLULayer`
- `fullyConnectedLayer` — Set **OutputSize** to 1 for the scalar value function.



Export Network from Deep Network Designer

To export the network to the MATLAB workspace, in **Deep Network Designer**, click **Export**. **Deep Network Designer** exports the network as a new variable containing the network layers. You can create the critic representation using this layer network variable.

Alternatively, to generate equivalent MATLAB code for the network, click **Export > Generate Network Code Without Parameters**.



The generated code is as follows.

```
net = dlnetwork();

tempLayers = [
    imageInputLayer([1 1 1],"Name","angularRate","Normalization","none")
    fullyConnectedLayer(400,"Name","dtheta_fc1")
    reluLayer("Name","dtheta_relu1")
    fullyConnectedLayer(300,"Name","dtheta_fc2")];
net = addLayers(net,tempLayers);

tempLayers = [
    imageInputLayer([1 1 1],"Name","torque","Normalization","none")
    fullyConnectedLayer(300,"Name","torque_fc1")];
net = addLayers(net,tempLayers);
```

```

templayers = [
    imageInputLayer([50 50 1], "Name", "pendImage", "Normalization", "none")
    convolution2dLayer([10 10], 2, "Name", "img_conv1", "Padding", "same", "Stride", [5 5])
    reluLayer("Name", "relu_1")
    fullyConnectedLayer(400, "Name", "critic_theta_fc1")
    reluLayer("Name", "theta_relu1")
    fullyConnectedLayer(300, "Name", "critic_theta_fc2")];
net = addLayers(net, templayers);

templayers = [
    additionLayer(3, "Name", "addition")
    reluLayer("Name", "relu_2")
    fullyConnectedLayer(1, "Name", "stateValue")];
net = addLayers(net, templayers);

net = connectLayers(net, "torque_fc1", "addition/in3");
net = connectLayers(net, "critic_theta_fc2", "addition/in1");
net = connectLayers(net, "dtheta_fc2", "addition/in2");

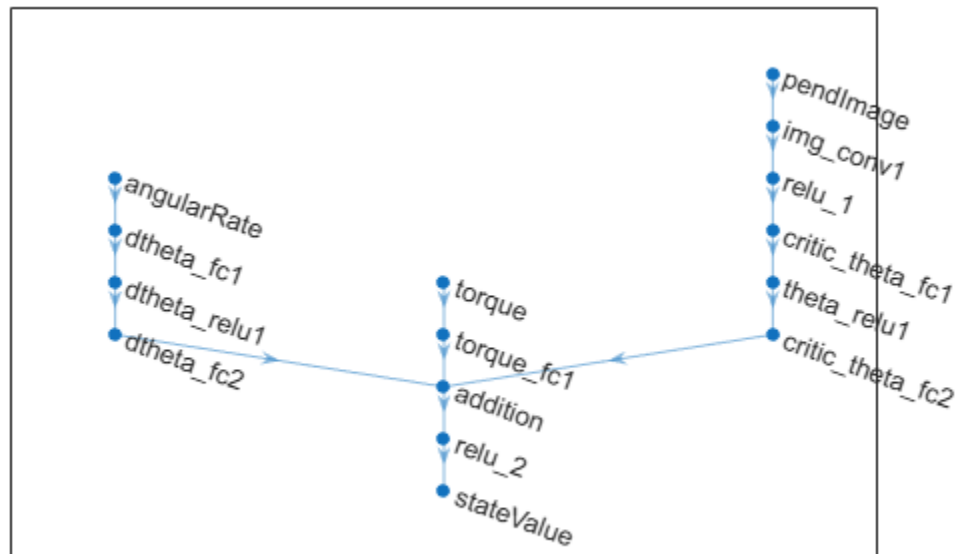
```

View the critic network configuration.

```

figure
plot(net)

```



Initialize the `dlnetwork` object and display the number of parameters.

```

net = initialize(net);
summary(net)

Initialized: true

Number of learnables: 322.9k

Inputs:

```

```
1 'angularRate' 1x1x1 images
2 'torque'      1x1x1 images
3 'pendImage'   50x50x1 images
```

Create the critic using the neural network, the action and observation specifications, and the names of the input layers to be connected to the observations and action channels. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(net,obsInfo,actInfo, ...
    "ObservationInputNames",["pendImage","angularRate"], ...
    "ActionInputNames","torque");
```

Specify options for the critic using `rlOptimizerOptions`.

```
criticOpts = rlOptimizerOptions(LearnRate=1e-03,GradientThreshold=1);
```

Specify the DQN agent options using `rlDQNAgentOptions`. Include the training options for the actor and critic.

```
agentOpts = rlDQNAgentOptions( ...
    UseDoubleDQN=false, ...
    CriticOptimizerOptions=criticOpts, ...
    ExperienceBufferLength=1e6, ...
    SampleTime=env.Ts);
```

You can also set or modify agent options using dot notation.

```
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-5;
```

Alternatively, you can create the agent first, and then modify its options using dot notation.

Create the DQN agent using the critic and the agent options object. For more information, see `rlDQNAgent`.

```
agent = rlDQNAgent(critic,agentOpts);
```

Train DQN Agent

To train the agent, first specify the training options. For this example, use the following options.

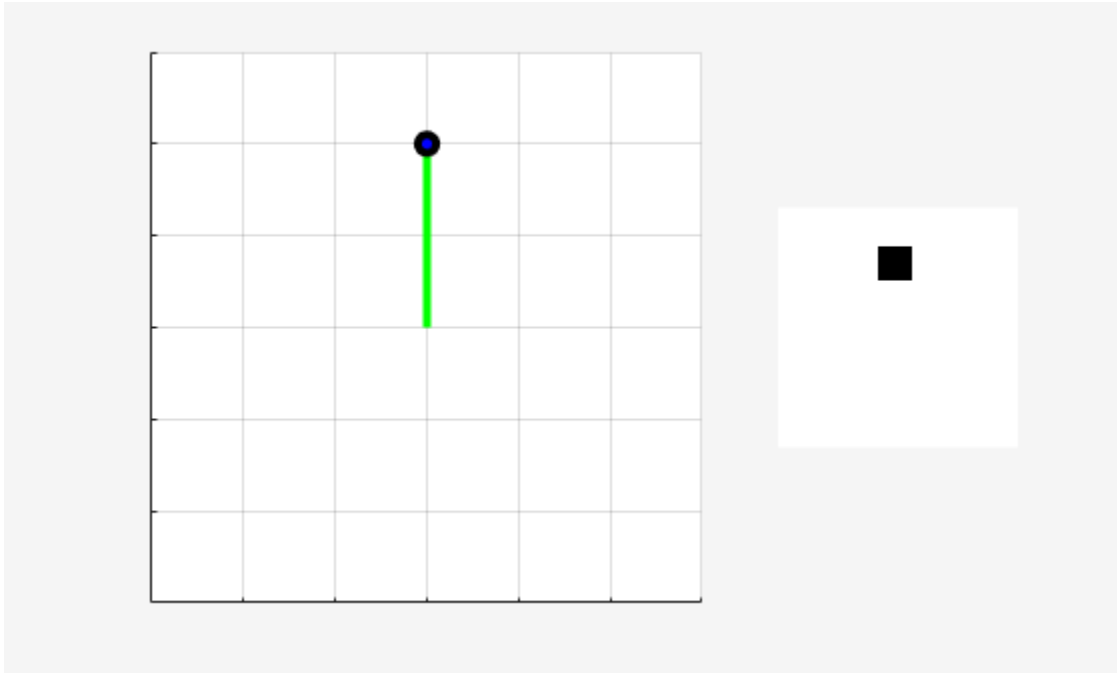
- Run each training for a maximum of 5000 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Episode Manager dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the agent receives an average cumulative reward greater than -1000 over the default window length of five consecutive episodes. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=500, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=-1000);
```

Visualize the pendulum system during training or simulation using the `plot` function.

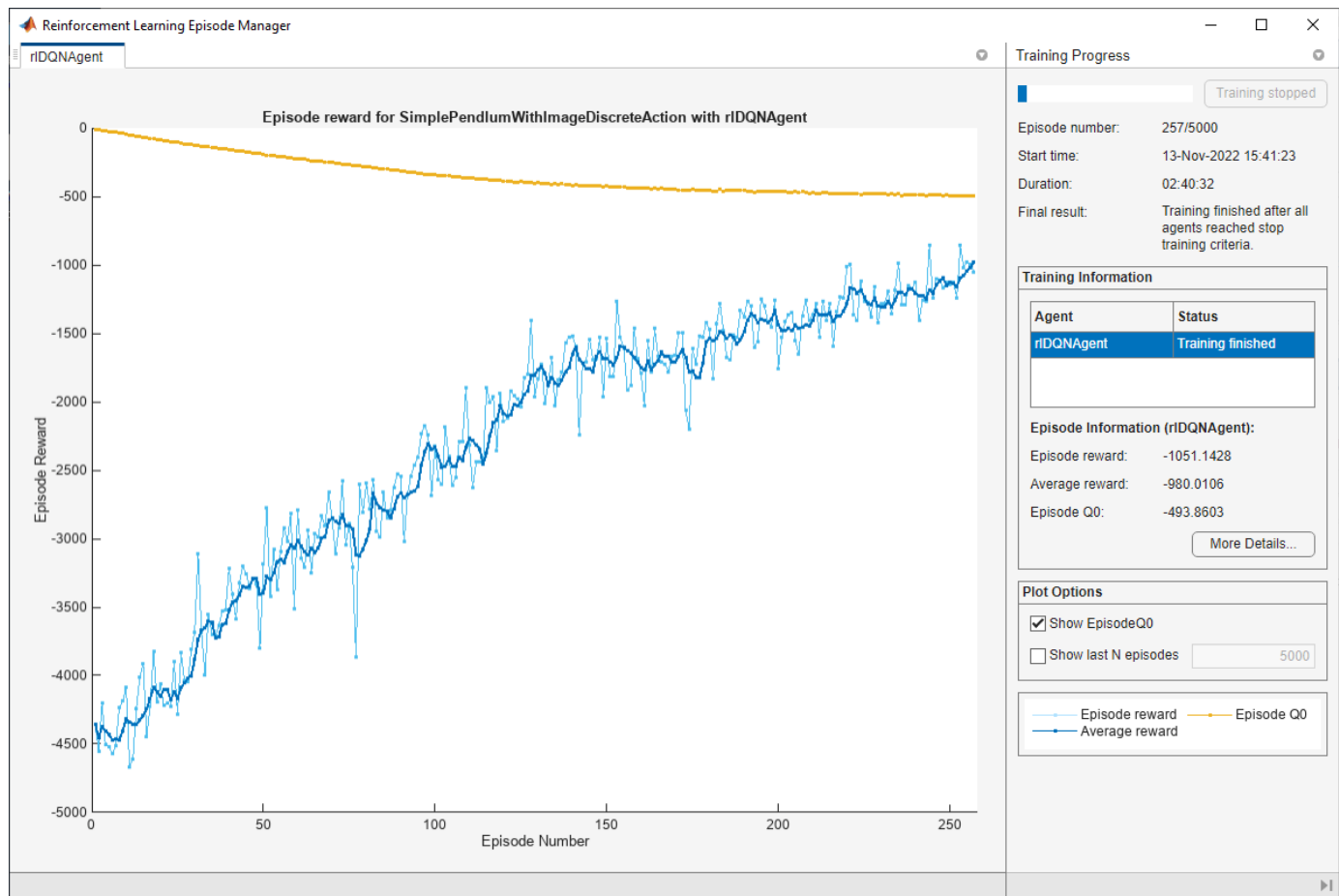
```
plot(env)
```



Train the agent using the `train` function. This is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;

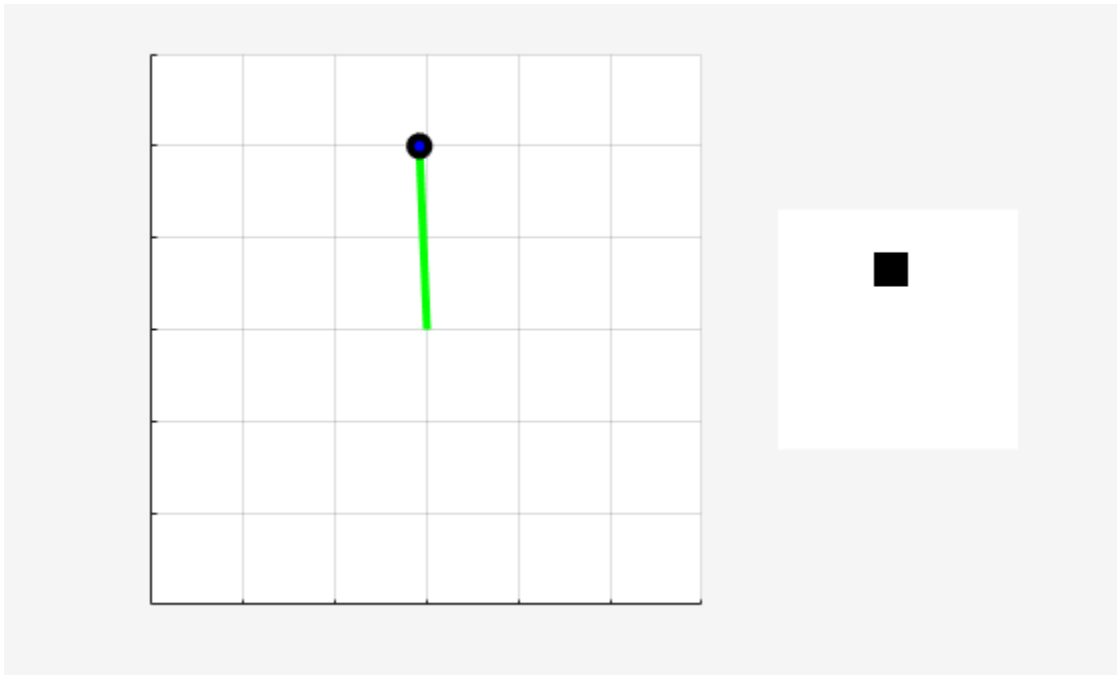
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts);
else
    % Load pretrained agent for the example.
    load("MATLABPendImageDQN.mat","agent");
end
```



Simulate Trained Agent

To validate the performance of the trained agent, simulate it within the pendulum environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
```

```
totalReward = sum(experience.Reward)
```

```
totalReward =  
-713.0336
```

See Also

Apps

Deep Network Designer | Reinforcement Learning Designer

Functions

train | sim

Objects

rldQNAgent | rldQNAgentOptions | rlTrainingOptions

Related Examples

- “Design and Train Agent Using Reinforcement Learning Designer” on page 5-13
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122
- “Transfer Learning: Fine-Tune DQN Agent for Pendulum Swing-Up from Earth to Mars” on page 5-207

More About

- “Use Predefined Control System Environments” on page 2-42
- “Deep Q-Network (DQN) Agent” on page 3-32

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Log Training Data to Disk

This example shows how to log custom data to disk when training agents using the Reinforcement Learning Toolbox™ `train` function.

Data Logging Overview

The general steps for data logging are:

- 1 Create a data logger object using the `rlDataLogger` function.
- 2 Configure the data logger object with callback functions to specify the data to log at different stages of the training process.
- 3 Specify the logger object as a name-value input argument in the `train` function.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Data Logger

Create a file data logger object using the `rlDataLogger` function.

```
fileLogger = rlDataLogger()

fileLogger =
    FileLogger with properties:

        LoggingOptions: [1x1 rl.logging.option.MATFileLoggingOptions]
        EpisodeFinishedFcn: []
        AgentStepFinishedFcn: []
        AgentLearnFinishedFcn: []
```

Optionally specify the data logging directory and a naming rule for the files to be saved.

```
% Specify a logging directory. You must have write
% access for this directory.
logDir = fullfile(pwd,"myDataLog");
fileLogger.LoggingOptions.LoggingDirectory = logDir;

% Specify a naming rule for files. The naming rule episode<id>
% saves files as episode001.mat, episode002.mat and so on.
fileLogger.LoggingOptions.FileNameRule = "episode<id>";
```

Configure Data Logging

Training data of interest is generated at different stages of the training loop; for example, experience data is available after the completion of an episode. You can configure the logger object with callback functions to log data at these stages. The functions must return either a structure containing the data to log, or an empty array if no data needs to be logged at that stage.

The callback functions are:

- **EpisodeFinishedFcn** - Callback function to log data such as experiences, logged Simulink® signals, or initial observations. The training loop executes this function after the completion of a training episode. The following is an example of the function.

```
function dataToLog = episodeLogFcn(data)
% episodeLogFcn logs data after every episode.
%
% data is a structure that contains the following fields:
%
% EpisodeCount: The current episode number.
% Environment: Environment object.
% Agent: Agent object.
% Experience: A structure containing the experiences
%             from the current episode.
% EpisodeInfo: A structure containing the fields
%             CumulativeReward, StepsTaken, and
%             InitialObservation.
% SimulationInfo: Contains simulation information for the
%                 current episode.
%                 For MATLAB environments this is a structure
%                 with the field "SimulationError".
%                 For Simulink environments this is a
%                 Simulink.SimulationOutput object.
%
% dataToLog is a structure containing the data to be logged
% to disk.

% Write your code to log data to disk. For example,
% dataToLog.Experience = data.Experience;

end
```

- **AgentStepFinishedFcn** - Callback function to log data such as the state of exploration. The training loop executes this function after the completion of an agent step within an episode. The following is an example of the function.

```
function dataToLog = agentStepLogFcn(data)
% agentStepLogFcn logs data after every agent step.
%
% data is a structure that contains the following fields:
%
% EpisodeCount: The current episode number.
% AgentStepCount: The cumulative number of steps taken by
%                 the agent.
% SimulationTime: The current simulation time in the
%                 environment.
% Agent: Agent object.
%
```

```
% dataToLog is a structure containing the data to be logged
% to disk.
```

```
% Write your code to log data to disk. For example,
% noiseState = getState(getExplorationPolicy(data.Agent));
% dataToLog.noiseState = noiseState;
```

```
end
```

- **AgentLearnFinishedFcn** - Callback function to log data such as the actor and critic training losses. The training loop executes this function after the updating the actor or critic networks. The following is an example of the function.

```
function dataToLog = agentLearnLogFcn(data)
% agentLearnLogFcn logs data after updating the agent's
% representations (actor or critic).
%
% data is a structure that contains the following fields:
%
% Agent          : Agent object.
% EpisodeCount   : The current episode number.
% AgentStepCount : The cumulative number of steps taken by
%                  the agent.
%
% For agents with an actor:
% ActorLoss       : Training loss of the actor.
% ActorGradientStepCount : Cumulative number of actor
%                  gradient computation steps.
%
% For agents with a critic:
% CriticLoss      : Training loss of the critic.
% CriticGradientStepCount : Cumulative number of critic
%                  gradient computation steps.
% TDTarget        : Future value of rewards as computed by
%                  the target critic network.
% TDError         : Error between the critic and target
%                  critic estimates of the reward.
%
% For PPO/TRPO agents:
% Advantage       : Advantage values.
%
% For PPO agents:
% AdvantageLoss   : Advantage loss value.
% EntropyLoss     : Entropy loss value.
% PolicyRatio     : Ratio between current and old policies.
%
% For model-based agents:
% EnvModelTrainingInfo: A structure containing the fields:
% a. TransitionFcnLoss
% b. RewardFcnLoss
% c. IsDoneFcnLoss.
%
% For off-policy agents with replay memory:
% SampleIndex     : Indices of experiences sampled from the
%                  replay memory for training.
%
% dataToLog is a structure containing the data to be logged
% to disk.
```

```
% Write your code to log data to disk. For example,  
% dataToLog.ActorLoss = data.ActorLoss;
```

```
end
```

For this example, you will log the actor and critic training losses and visualize the loss data in the Reinforcement Learning Data Viewer window.

- Specify the `logTrainingLoss` function for the `AgentLearnFinishedFcn` callback. The function is provided at the end of this example.

```
fileLogger.AgentLearnFinishedFcn = @logTrainingLoss;
```

For logging data with several agent algorithms, see [Logging Functions](#) on page 5-236.

Run Training

Create a predefined `CartPole-continuous` environment and a deep deterministic policy gradient (DDPG) agent for training.

```
% Set the random seed to facilitate reproducibility  
rng(0, "twister");  
  
% Create a CartPole-continuous environment  
env = rlPredefinedEnv("CartPole-Continuous");  
  
% Create a DDPG agent  
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);  
agent = rlDDPGAgent(obsInfo, actInfo);  
agent.AgentOptions.NoiseOptions.StandardDeviationDecayRate = 0.001;
```

Specify training options to train the agent for 100 episodes and visualize the training in the Reinforcement Learning Training Monitor (set the `Plots` option).

Note that you can still use the `SaveAgentCriteria`, `SaveAgentValue` and `SaveAgentDirectory` options of the `rlTrainingOptions` object to save the agent during training. Such options do not affect (and are not affected by) any usage of `FileLogger` or `MonitorLogger` objects.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=100, ...  
    Plots="training-progress");
```

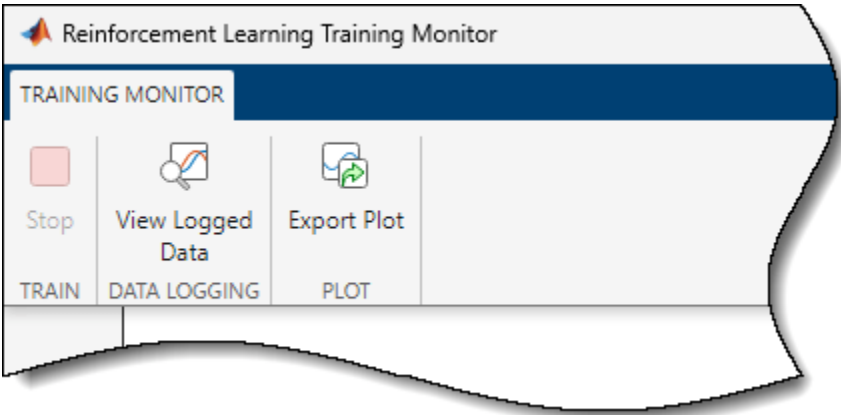
Train the agent using the `train` function. Specify the file logger object in the `Logger` name-value option.

```
results = train(agent, env, trainOpts, Logger=fileLogger);
```

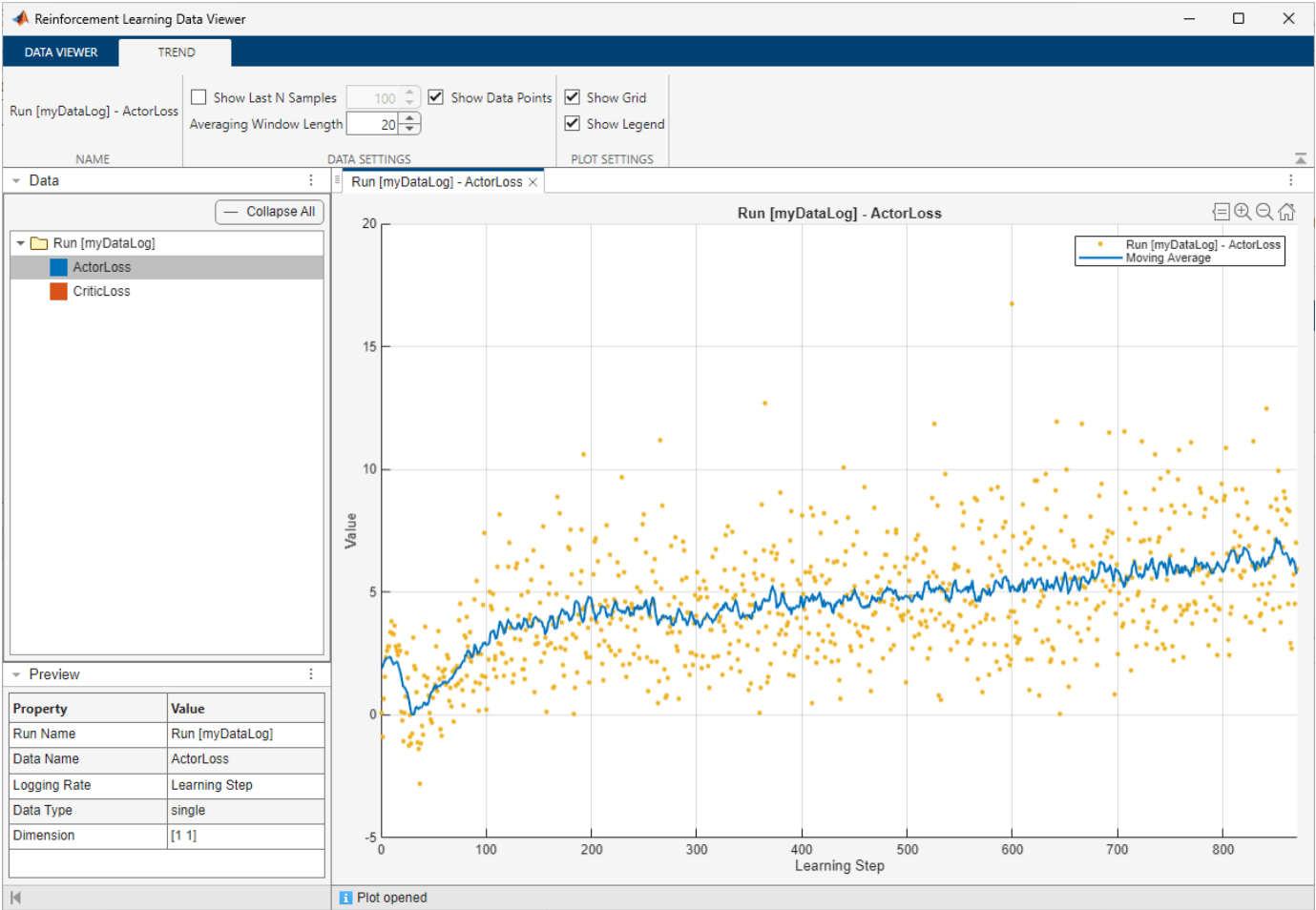
The logged data is saved within the directory specified by `logDir`.

Visualize Logged Data

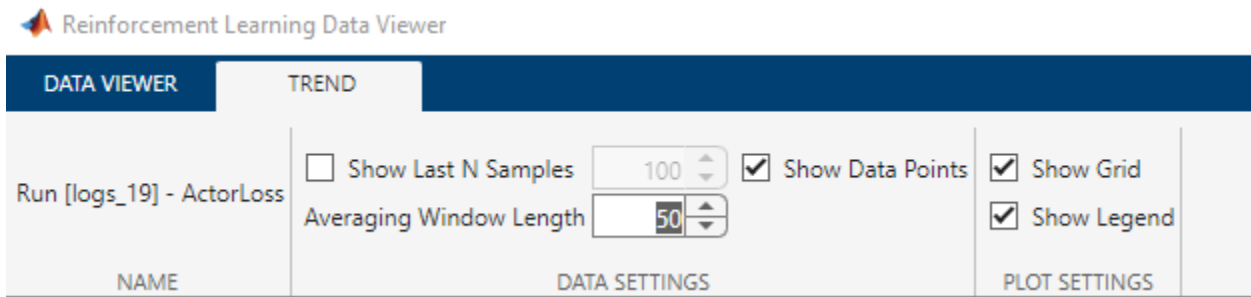
You can visualize data logged to disk using the interactive **Reinforcement Learning Data Viewer** graphical user interface. To open the visualization, click **View Logged Data** in the **Reinforcement Learning Training Monitor** window.



To create plots in the **Reinforcement Learning Data Viewer**, select a data from the **Data** panel and a choice of plot from the toolbar. The following image shows a plot of the ActorLoss data generated using the Trend plot type. The plot shows logged data points and a moving average line.



On the toolbar, navigate to the **Trend** tab to configure plot options. Set the window length for averaging data to 50. The plot updates with the new configuration.



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Logging Functions

Use the code below to create and configure a data logger object for several built-in agent algorithms.

Q, SARSA, DQN

```
% Create and configure a data logger
logger = rlDataLogger();
logger.AgentLearnFinishedFcn = @agentLearnFinishedFcn;
logger.AgentStepFinishedFcn = @agentStepFinishedFcn;
logger.EpisodeFinishedFcn = @episodeFinishedFcn;

function dataToLog = agentLearnFinishedFcn(data)
% AgentLearnFinished callback function for Q, SARSA, DQN agents
dataToLog.CriticLoss = data.CriticLoss;
end

function dataToLog = agentStepFinishedFcn(data)
% AgentStepFinished callback function for Q, SARSA, DQN
policy = getExplorationPolicy(data.Agent);
state = getState(policy);
dataToLog.Epsilon = state.Epsilon;
end

function dataToLog = episodeFinishedFcn(data)
% EpisodeFinished callback function for Q, SARSA, DQN
dataToLog.Experience = data.Experience;
dataToLog.EpisodeReward = data.EpisodeInfo.CumulativeReward;
if data.EpisodeInfo.StepsTaken > 0
    dataToLog.EpisodeQ0 = evaluateQ0(data.Agent, ...
        data.EpisodeInfo.InitialObservation);
else
    dataToLog.EpisodeQ0 = 0;
end
```

DDPG, TD3

```
% Create and configure a data logger
logger = rlDataLogger();
logger.AgentLearnFinishedFcn = @agentLearnFinishedFcn;
logger.AgentStepFinishedFcn = @agentStepFinishedFcn;
logger.EpisodeFinishedFcn = @episodeFinishedFcn;
```



```

function dataToLog = agentLearnFinishedFcn(data)
% AgentLearnFinished callback function for DDPG, TD3 agents
dataToLog.ActorLoss = data.ActorLoss;
dataToLog.CriticLoss = data.CriticLoss;
dataToLog.TDError = data.TDError;
end

function dataToLog = agentStepFinishedFcn(data)
% AgentStepFinished callback function for DDPG, TD3 agents
policy = getExplorationPolicy(data.Agent);
switch policy.NoiseType
    case "ou"
        dataToLog.OUNoise = state.Noise{1};
        dataToLog.StandardDeviation = state.StandardDeviation{1};
    case "gaussian"
        dataToLog.StandardDeviation = state.StandardDeviation{1};
end
end

function dataToLog = episodeFinishedFcn(data)
% EpisodeFinished callback function for DDPG, TD3 agents
dataToLog.Experience = data.Experience;
dataToLog.EpisodeReward = data.EpisodeInfo.CumulativeReward;
if data.EpisodeInfo.StepsTaken > 0
    dataToLog.EpisodeQ0 = evaluateQ0(data.Agent, ...
        data.EpisodeInfo.InitialObservation);
else
    dataToLog.EpisodeQ0 = 0;
end
end

```

AC, PG, SAC, TRPO

```

% Create and configure a data logger
logger = rlDataLogger();
logger.AgentLearnFinishedFcn = @agentLearnFinishedFcn;
logger.EpisodeFinishedFcn    = @episodeFinishedFcn;

function dataToLog = agentLearnFinishedFcn(data)
% AgentLearnFinished callback function for AC, PG, SAC, TRPO agents
dataToLog.ActorLoss = data.ActorLoss;
dataToLog.CriticLoss = data.CriticLoss;
end

function dataToLog = episodeFinishedFcn(data)
% EpisodeFinished callback function for AC, PG, SAC, TRPO agents
dataToLog.Experience = data.Experience;
dataToLog.EpisodeReward = data.EpisodeInfo.CumulativeReward;
if data.EpisodeInfo.StepsTaken > 0
    dataToLog.EpisodeQ0 = evaluateQ0(data.Agent, ...
        data.EpisodeInfo.InitialObservation);
else
    dataToLog.EpisodeQ0 = 0;
end
end

```

PPO

```

% Create and configure a data logger
logger = rlDataLogger();

```

```
logger.AgentLearnFinishedFcn = @agentLearnFinishedFcn;
logger.EpisodeFinishedFcn    = @episodeFinishedFcn;

function dataToLog = agentLearnFinishedFcn(data)
% AgentLearnFinished callback function for PPO agents
dataToLog.ActorLoss = data.ActorLoss;
dataToLog.CriticLoss = data.CriticLoss;
dataToLog.AdvantageLoss = data.AdvantageLoss;
dataToLog.EntropyLoss = data.EntropyLoss;
dataToLog.PolicyRatio = data.PolicyRatio;
end

function dataToLog = episodeFinishedFcn(data)
% EpisodeFinished callback function for PPO agents
dataToLog.Experience = data.Experience;
dataToLog.EpisodeReward = data.EpisodeInfo.CumulativeReward;
if data.EpisodeInfo.StepsTaken > 0
    dataToLog.EpisodeQ0 = evaluateQ0(data.Agent, ...
        data.EpisodeInfo.InitialObservation);
else
    dataToLog.EpisodeQ0 = 0;
end
```

MBPO

```
% Create and configure a data logger
logger = rlDataLogger();
logger.AgentLearnFinishedFcn = @agentLearnFinishedFcn;
logger.EpisodeFinishedFcn    = @episodeFinishedFcn;

function dataToLog = agentLearnFinishedFcn(data)
% AgentLearnFinished callback function for MBPO agents
dataToLog.ActorLoss = data.ActorLoss;
dataToLog.CriticLoss = data.CriticLoss;
mdlInfo = data.EnvModelTrainingInfo;
dataToLog.TransitionFcnLoss = mdlInfo.TransitionFcnLoss;
dataToLog.RewardFcnLoss = mdlInfo.RewardFcnLoss;
dataToLog.IsDoneFcnLoss = mdlInfo.IsDoneFcnLoss;
end

function dataToLog = episodeFinishedFcn(data)
% EpisodeFinished callback function for MBPO agents
dataToLog.Experience = data.Experience;
dataToLog.EpisodeReward = data.EpisodeInfo.CumulativeReward;
if data.EpisodeInfo.StepsTaken > 0
    dataToLog.EpisodeQ0 = evaluateQ0(data.Agent, ...
        data.EpisodeInfo.InitialObservation);
else
    dataToLog.EpisodeQ0 = 0;
end
```

Local Functions

```
function dataToLog = logTrainingLoss(data)
% Function to log the actor and critic training losses
dataToLog.ActorLoss = data.ActorLoss;
```

```
dataToLog.CriticLoss = data.CriticLoss;  
end
```

See Also

Functions

`rlDataLogger` | `train` | `sim`

Objects

`FileLogger` | `MonitorLogger` | `trainingProgressMonitor` | `rlTrainingOptions`

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85
- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240

More About

- “Use Predefined Control System Environments” on page 2-42
- “Monitor Custom Training Loop Progress”
- “Train Reinforcement Learning Agents” on page 5-3
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Train Agent or Tune Environment Parameters Using Parameter Sweeping

This example shows how to train a reinforcement learning agent with the water tank reinforcement learning Simulink® environment by sweeping parameters. You can use this example as a template for tuning parameters when training reinforcement learning agents. To tune hyperparameters in the Reinforcement Learning Designer app, see “Specify Hyperparameter Tuning Options” on page 5-35. For an general example on how to tune a custom set of hyperparameters programmatically, using Bayesian optimization, see “Tune Hyperparameters Using Bayesian Optimization” on page 5-250.

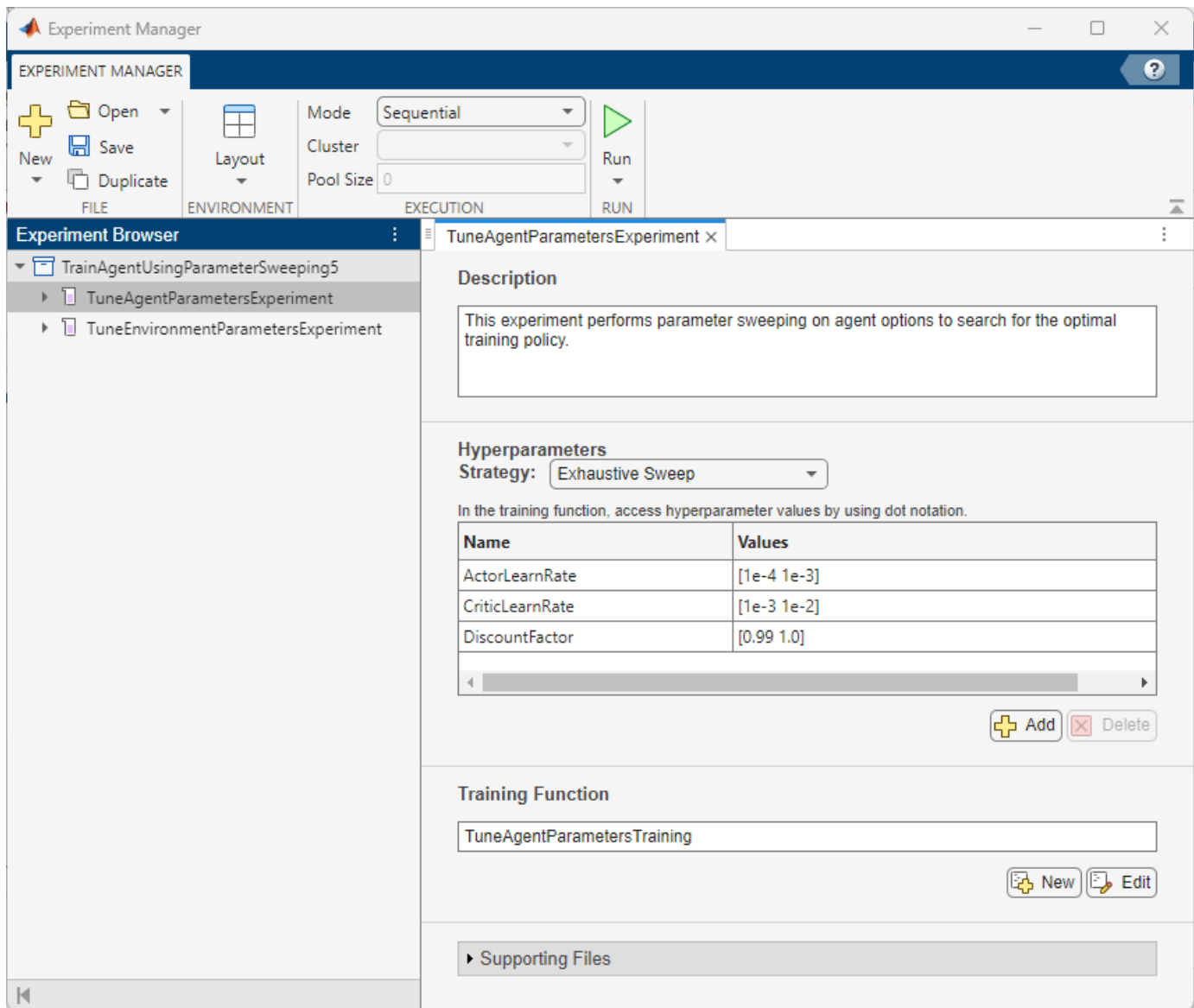
Open a preconfigured project, which has all required files added as project dependencies. Opening the project also launches the Experiment Manager app.

`TrainAgentUsingParameterSweepingStart`

Note that it is best practice to add any Simulink models and supporting files as dependencies to your project.

Tune Agent Parameters Using Parameter Sweeping

In this section you tune the agent parameters to search for an optimal training policy.



Open Experiment

- In the **Experiment Browser** pane double-click the name of the experiment (TuneAgentParametersExperiment). This opens a tab for the experiment.
- The **Hyperparameters** section contains the hyperparameters to tune for this experiment. A set of hyperparameters has been added for this experiment. To add a new parameter, click **Add** and specify a name and array of values for the hyperparameter. When you run the experiment, Experiment Manager runs the training using every combination of parameter values specified in the hyperparameter table.
- Verify that **Strategy** is set to Exhaustive Sweep.
- Under **Training Function**, click **Edit**. The MATLAB® Editor opens to show code for the training function TuneAgentParametersTraining. The training function creates the environment and agent objects and runs the training using one combination of the specified hyperparameters.

```
function output = TuneAgentParametersTraining(params,monitor)

% Set the random seed generator
rng(0);

% Load the Simulink(R) model
mdl = "rlwatertank";
load_system(mdl);

% Create variables in the base workspace. When running on a parallel worker
% this will also create variables in the base workspace of the worker.
evalin("base","loadWaterTankParams");
Ts = evalin("base","Ts");
Tf = evalin("base","Tf");

% Create a reinforcement learning environment
actionInfo = rlNumericSpec([1 1]);
observationInfo = rlNumericSpec([3 1], ...
    LowerLimit=[-inf -inf 0 ], ...
    UpperLimit=[ inf  inf inf]);
blk = mdl + "/RL Agent";
env = rlSimulinkEnv(mdl, blk, observationInfo, actionInfo);

% Specify a reset function for the environment
env.ResetFcn = @localResetFcn;

% Create options for the reinforcement learning agent. You can assign
% values from the params struct for sweeping parameters.
agentOpts = rlDDPGAgentOptions();
agentOpts.MinibatchSize = 64;
agentOpts.TargetSmoothFactor = 1e-3;
agentOpts.SampleTime = Ts;
agentOpts.DiscountFactor = params.DiscountFactor;
agentOpts.ActorOptimizerOptions.LearnRate = params.ActorLearnRate;
agentOpts.CriticOptimizerOptions.LearnRate = params.CriticLearnRate;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
agentOpts.CriticOptimizerOptions.GradientThreshold = 1;
agentOpts.NoiseOptions.StandardDeviation = 0.3;
agentOpts.NoiseOptions.StandardDeviationDecayRate = 1e-5;

% Create the reinforcement learning agent. You can modify the
% localCreateActorAndCritic function to edit the agent model.
[actor, critic] = localCreateActorAndCritic(observationInfo, actionInfo);
agent = rlDDPGAgent(actor, critic, agentOpts);

trainOpts = rlTrainingOptions( ...
    MaxEpisodes=200, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    ScoreAveragingWindowLength=20, ...
    Verbose=false, ...
    Plots="none", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=800);

% Create a data logger for logging data to the monitor object
logger = rlDataLogger(monitor);

% Run the training
```

```

result = train(agent, env, trainOpts, Logger=logger);

% Export experiment results
output.Agent = agent;
output.Environment = env;
output.TrainingResult = result;
output.Parameters = params;

end

```

Run Experiment

When you run the experiment, Experiment Manager executes the training function multiple times. Each trial uses one combination of hyperparameter values. By default, Experiment Manager runs one trial at a time. If you have the Parallel Computing Toolbox™, you can run multiple trials at the same time or offload your experiment as a batch job in a cluster.

Under **Mode**, select **Sequential**, and click **Run** to run the experiment one trial at a time.

- To run multiple trials simultaneously, under **Mode**, select **Simultaneous**, and click **Run**. This requires a Parallel Computing Toolbox license.
- To offload the experiment as a batch job under **Mode**, select **Batch Sequential** or **Batch Simultaneous**, specify your **Cluster** and **Pool Size**, and click **Run**. Note that you will need to configure the cluster with the files necessary for this example. This mode also requires a Parallel Computing Toolbox license.

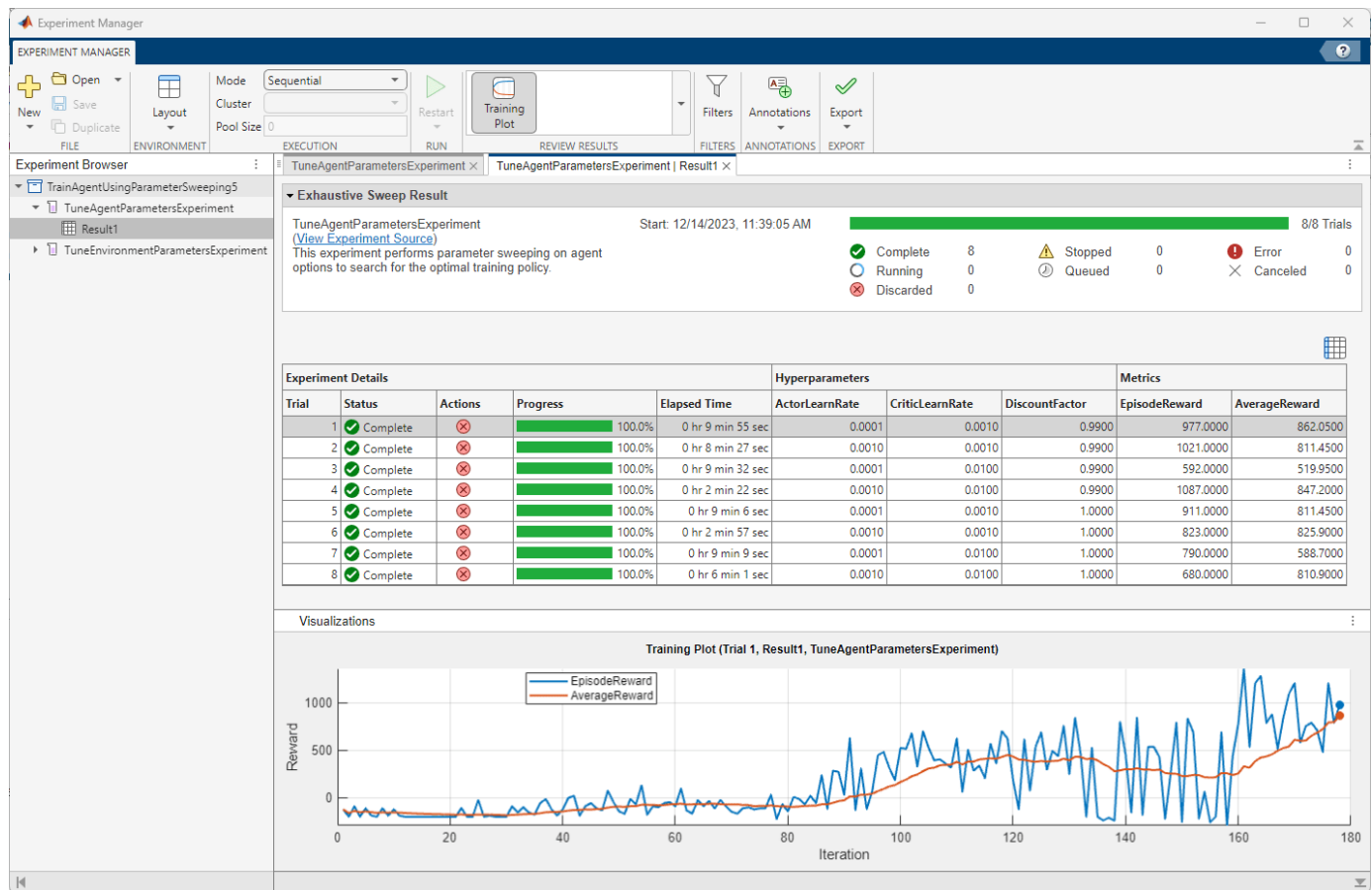
Note that your cluster needs to be configured with files necessary for this experiment when running in the **Batch Sequential** or **Batch Simultaneous** modes. For more information on the Cluster Profile Manager, see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox). To configure your cluster:

- Open the Cluster Profile Manager and under **Properties**, click **Edit**.
- Under the **AttachedFiles** option, click **Add** and specify the files `rlwatertank.slx` and `loadWaterTankParams.m`.
- Click **Done**.

When the experiment is running, select a trial row from the table of results, and under the toolstrip, click **Training Plot**. This shows the episode and average reward plots for that trial.

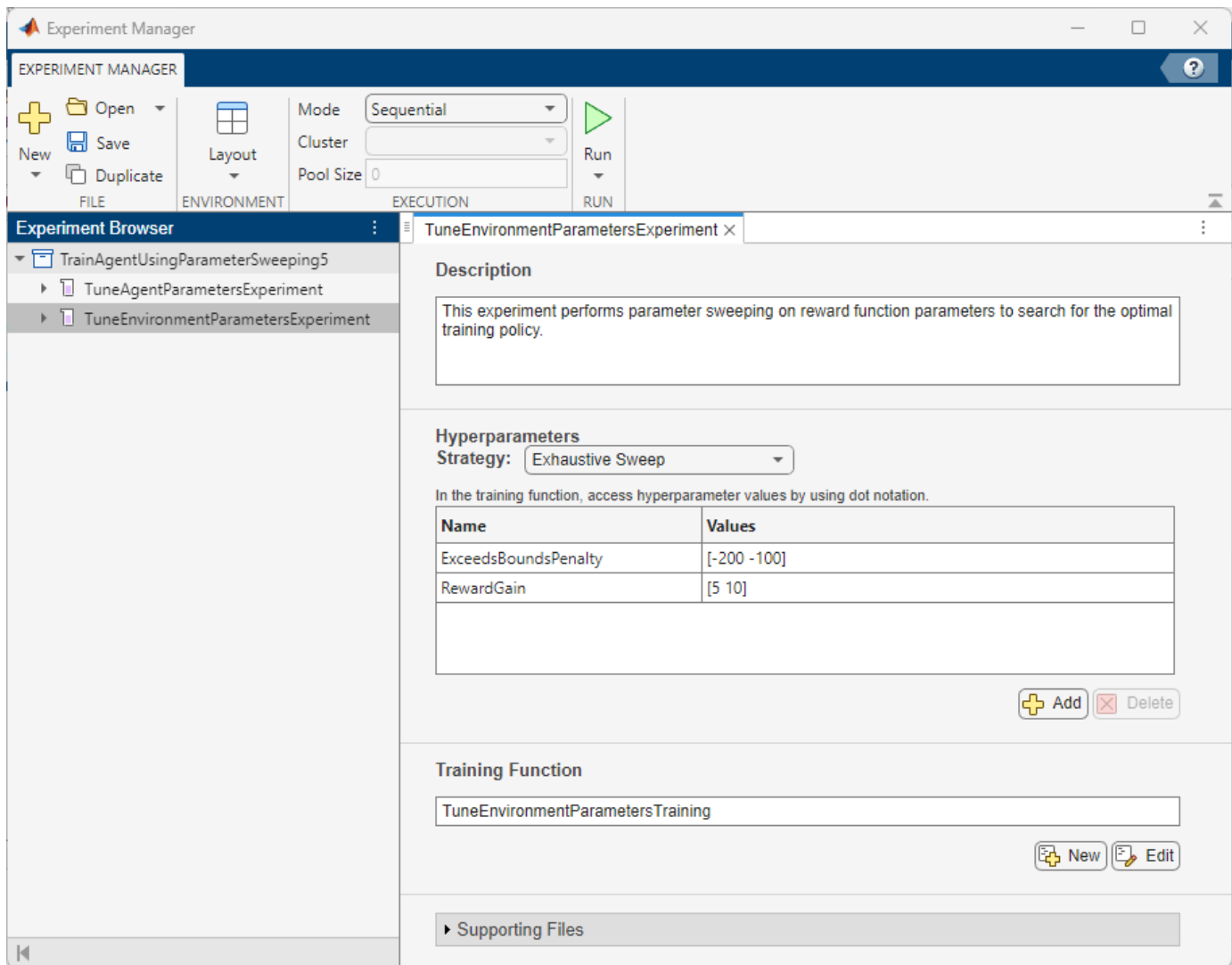
After the experiment is finished:

- Select the row corresponding to trial 7 which has the average reward 817.5, and under the toolstrip, click **Export**. This action exports the results of the trial to a base workspace variable.
- Name the variable `agentParamSweepTrainingOutput`.



Tune Environment Parameters Using Parameter Sweeping

In this section you tune the environment's reward function parameters to search for an optimal training policy.



Open Experiment

In the **Experiment Browser** pane, open `TuneEnvironmentParametersExperiment`. Verify, as with the agent tuning, that **Strategy** is set to `Exhaustive Sweep`. View code for the training function `TuneEnvironmentParametersTraining` as before.

```
function output = TuneEnvironmentParametersTraining(params,monitor)

% Set the random seed generator
rng(0);

% Load the Simulink model
mdl = "rlwatertank";
load_system(mdl);

% Create variables in the base workspace. When running on a parallel worker
% this will also create variables in the base workspace of the worker.
evalin("base","loadWaterTankParams");
Ts = evalin("base","Ts");
```

```
Tf = evalin("base","Tf");

% Create a reinforcement learning environment
actionInfo = rlNumericSpec([1 1]);
observationInfo = rlNumericSpec([3 1], ...
    LowerLimit=[-inf -inf 0  ], ...
    UpperLimit=[ inf  inf inf]');
blk = mdl + "/RL Agent";
env = rlSimulinkEnv(mdl, blk, observationInfo, actionInfo);

% Specify a reset function for the environment. You can tune environment
% parameters such as reward or initial condition within this function.
env.ResetFcn = @(in) localResetFcn(in, params);

% Create options for the reinforcement learning agent. You can assign
% values from the params struct for sweeping parameters.
agentOpts = rlDDPGAgentOptions();
agentOpts.MinibatchSize           = 64;
agentOpts.TargetSmoothFactor      = 1e-3;
agentOpts.SampleTime              = Ts;
agentOpts.DiscountFactor          = 0.99;
agentOpts.ActorOptimizerOptions.LearnRate = 1e-3;
agentOpts.CriticOptimizerOptions.LearnRate = 1e-3;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
agentOpts.CriticOptimizerOptions.GradientThreshold = 1;
agentOpts.NoiseOptions.Variance    = 0.3;
agentOpts.NoiseOptions.VarianceDecayRate = 1e-5;

% Create the reinforcement learning agent. You can modify the
% localCreateActorAndCritic function to edit the agent model.
[actor, critic] = localCreateActorAndCritic(observationInfo, actionInfo);
agent = rlDDPGAgent(actor, critic, agentOpts);

trainOpts = rlTrainingOptions( ...
    MaxEpisodes=200, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    ScoreAveragingWindowLength=20, ...
    Verbose=false, ...
    Plots="none", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=800);

% Create a data logger for logging data to the monitor object
logger = rlDataLogger(monitor);

% Run the training
result = train(agent, env, trainOpts, Logger=logger);

% Export experiment results
output.Agent = agent;
output.Environment = env;
output.TrainingResult = result;
output.Parameters = params;

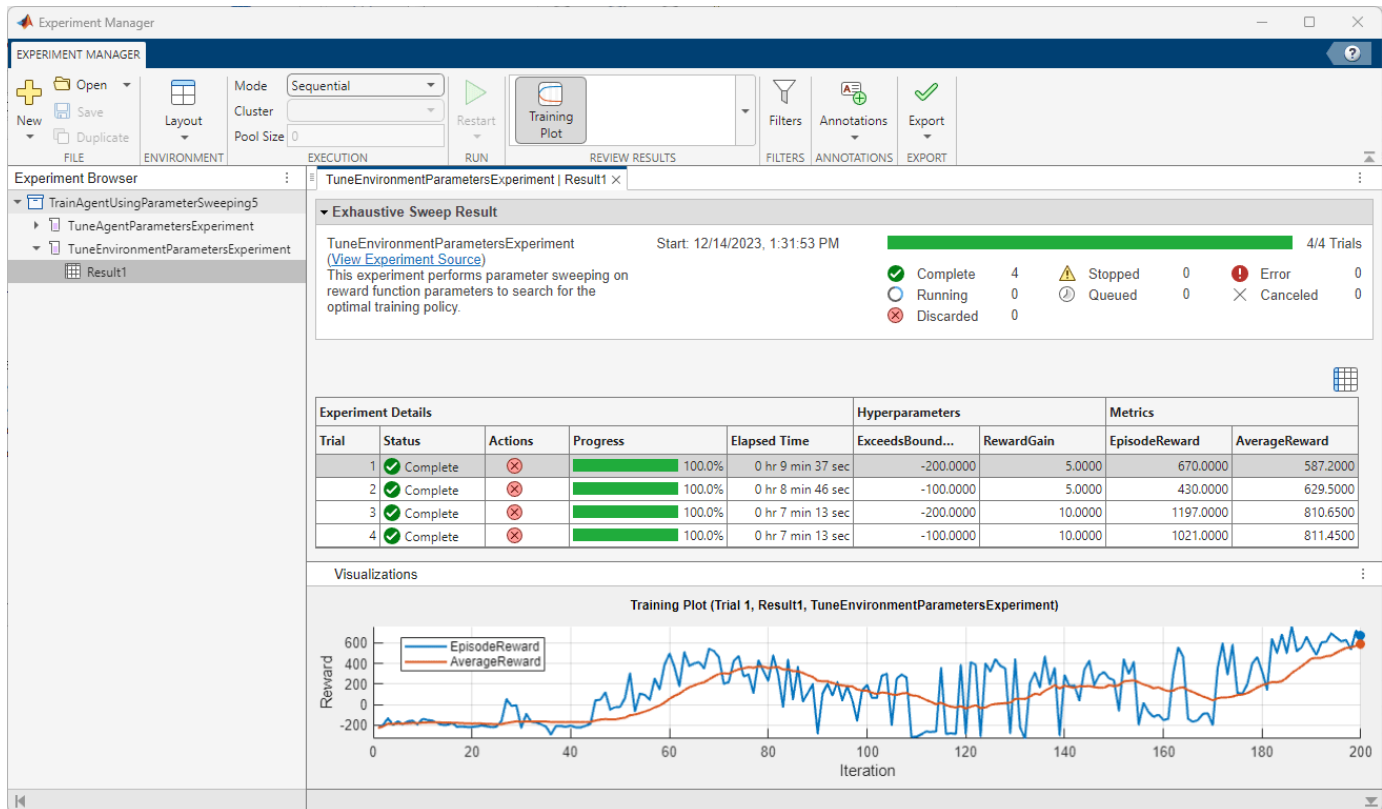
end
```

Run Experiment

Run the experiment using the same settings you use for agent tuning.

After the experiment is finished:

- Select the row corresponding to trial 4, which has the maximum average reward, and export the result to a base workspace variable.
- Name the variable as `envParamSweepTrainingOutput`.



Evaluate Agent Performance

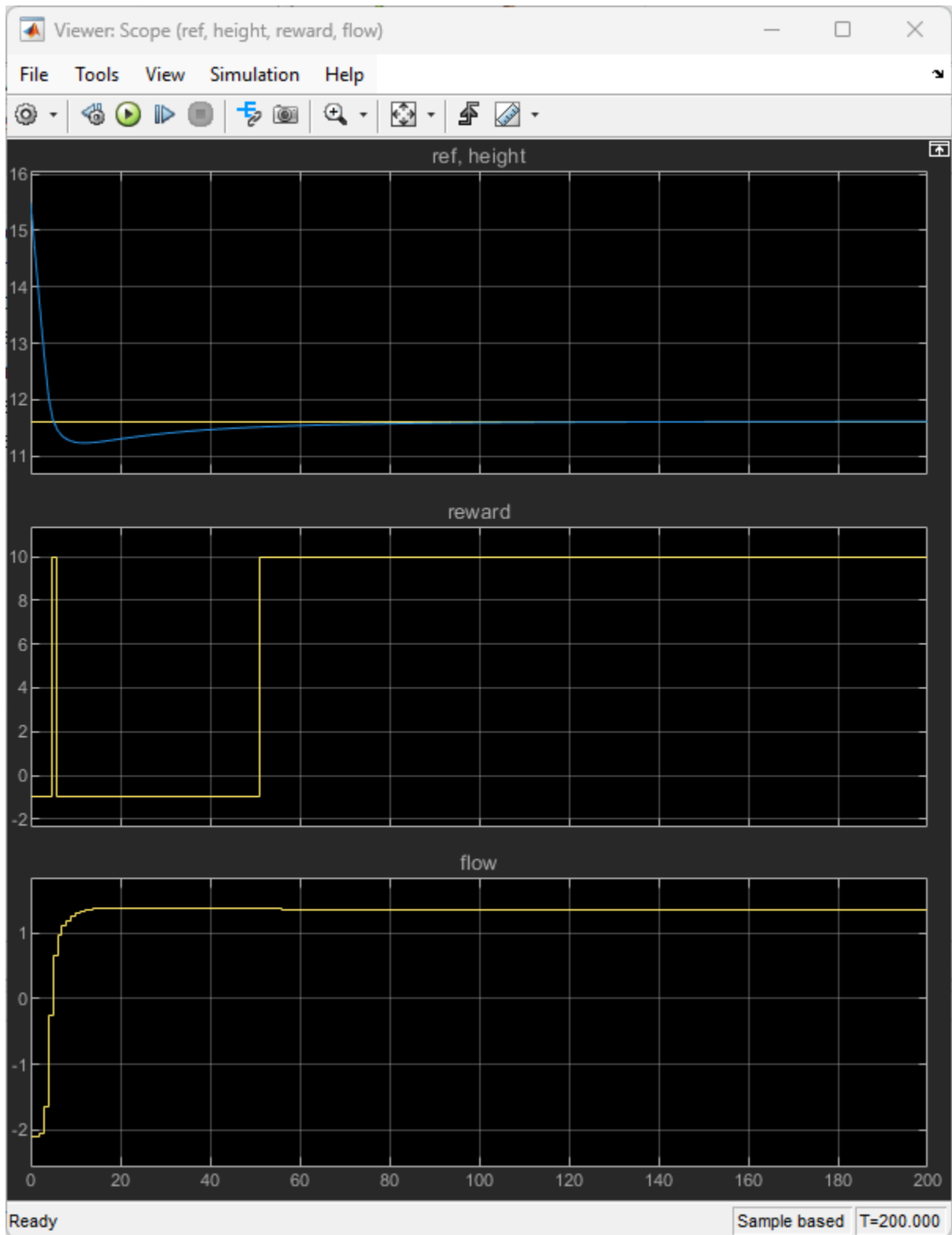
Execute the following code in MATLAB after exporting the agents from the above experiments. This code simulates the agent with the environment and displays the performance in the Scope blocks.

```
open_system("rlwatertank");
simOpts = rlSimulationOptions(MaxSteps=200);

% evaluate the agent exported from
% TuneAgentParametersExperiment
experience = sim(agentParamSweepTrainingOutput.Agent, ...
    agentParamSweepTrainingOutput.Environment, ...
    simOpts);

% evaluate the agent exported from
% TuneEnvironmentParametersExperiment
experience = sim(envParamSweepTrainingOutput.Agent, ...
    envParamSweepTrainingOutput.Environment, ...
    simOpts);
```

The agent is able to track the desired water level.



Close the project.

```
close(prj);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlTrainingOptions`

Related Examples

- “Log Training Data to Disk” on page 5-231
- “Tune Hyperparameters Using Bayesian Optimization” on page 5-250
- “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53
- “Configure Exploration for Reinforcement Learning Agents” on page 5-259

More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Simulink Environments” on page 2-110
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67

Tune Hyperparameters Using Bayesian Optimization

This example shows how to tune the hyperparameters of a reinforcement learning agent using Bayesian optimization. To train a reinforcement learning agent, you must specify the network architecture and a set of hyperparameters for the agent's learning algorithm. The hyperparameters vary depending on the application, so the default hyperparameters might need tuning for the agent to learn the desired policy. To tune hyperparameters in the Reinforcement Learning Designer app, see “Specify Hyperparameter Tuning Options” on page 5-35. For an example on how to perform parameter sweeping using the Experiment Manager app, see “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240.

Introduction to Bayesian Optimization

Bayesian optimization involves locating a point (a set of hyperparameters) that minimizes a real-valued function $f(x)$, which is also known as the objective function.

In machine learning, hyperparameters are parameters that directly affect the learning process, whereas model parameters are directly affected by the learning process. In reinforcement learning, the learning rates of the agent networks and the mini-batch sizes are examples of hyperparameters. Hyperparameters typically strongly affect the performance of the algorithm and it is often difficult and time-consuming to optimize them using techniques such as grid search or random search across the entire hyperparameter space. The Bayesian optimization algorithm reduces the number of evaluations needed to find the optimal set of hyperparameters. The algorithm maintains a Gaussian process model of the objective function, and uses the objective function evaluations to iteratively train the model. The algorithm also uses an acquisition function to determine the next set of hyperparameters for evaluation. For more information, see “Bayesian Optimization Algorithm” (Statistics and Machine Learning Toolbox).

In this example you use the `bayesopt` command (see “Optimize Cross-Validated Classifier Using `bayesopt`” (Statistics and Machine Learning Toolbox)) to optimize hyperparameters. This command requires Statistics and Machine Learning Toolbox™.

Create Environment Object

This example uses the lander vehicle environment from the “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93 example. The `LanderVehicle` class is provided in the example folder.

To view the environment object code, open the `LanderVehicle` class.

```
open("LanderVehicle.m");
```

Create the environment object.

```
env = LanderVehicle

env =
    LanderVehicle with properties:

        Mass: 1
         L1: 10
         L2: 5
    Gravity: 9.8060
 ThrustLimits: [0 8.5000]
           Ts: 0.1000
```

```

        State: [6×1 double]
        LastAction: [2×1 double]
        LastShaping: 0
        DistanceIntegral: 0
        VelocityIntegral: 0
        TimeCount: 0

```

Specify Variables to Optimize

In this example, you optimize the hyperparameters of a proximal policy optimization (PPO) agent. To run Bayesian optimization, specify these hyperparameters and their search ranges using the `optimizableVariable` (Statistics and Machine Learning Toolbox) function.

- Experience horizon for the PPO algorithm - A higher value can improve the stability of the training. Specify a range of 100 to 600.
- Mini-batch size for learning - Small mini-batches are computationally efficient but might introduce variance in training. Larger batch sizes can make the training stable but require more memory. Specify a range of 50 to 500.
- Actor and critic learning rates - A large learning rate causes drastic updates can lead to divergent behaviors. A small learning rate can cause the algorithm to perform many updates before reaching the optimal point. Specify a range of $1e-6$ to $1e-2$.
- Discount factor - This value controls the importance of long-term rewards. Specify a range of 0.95 to 1.0.
- Number of learning epochs - Specify a range of 1 to 10.
- Clip factor for the PPO algorithm - This value limits the change in each policy update step - Specify a range of 0.01 to 0.1.
- Entropy loss weight - A higher value promotes agent exploration. Specify a range of 0.01 to 0.1.

For more information on these parameters see `rlPPOAgentOptions`.

```

% Size of mini-batch
mbsz = optimizableVariable( ...
    MiniBatchSize=[50,500], ...
    Type="integer");

% Experience horizon
hrz = optimizableVariable( ...
    ExperienceHorizon=[100 600], ...
    Type="integer");

% Actor and critic learning rates
actorlr = optimizableVariable(ActorLearnRate=[1e-6,1e-2]);
criticlr = optimizableVariable(CriticLearnRate=[1e-6,1e-2]);

% Clip factor
clipf = optimizableVariable(ClipFactor=[0.01,0.1]);

% Entropy loss weight
entw = optimizableVariable(EntropyLossWeight=[0.01,0.1]);

% Number of epochs
nepoch = optimizableVariable(NumEpoch=[1,10],Type="integer");

```

```
% Discount factor
discf = optimizableVariable(DiscountFactor=[0.95,1.0]);
```

Create an array of hyperparameters to optimize. You use this array when running the Bayesian optimization algorithm.

```
optimVars = [mbsz,hrz,actorlr,criticlr,clipf,entw,nepoch,discf];
```

Define Objective Function

The objective function runs one training of the agent using one set of hyperparameters and returns a score that represents the optimality of the hyperparameters. You can specify the cumulative reward obtained by the agent over an evaluation episode as the objective function value. Minimizing the cumulative reward improves the performance of the agent. Alternatively, you can specify other metrics as the objective function value, such as the best or last average episodic reward.

The function performs the following steps:

- Creates the agent object using networks that have 400 units in their hidden layers.
- Configures the agent hyperparameters using the `params` input argument. Here, `params` is the structure containing the hyperparameters that you configured in the previous section.
- Trains the agent for 3000 episodes, with a maximum of 600 steps per episode.
- Performs agent evaluations every 20 training episodes, with each evaluation running five simulations.
- Returns the objective (last evaluation score), constraints, and user data (trained agent and training result) as the function outputs.

Define the objective function.

```
function [objective,constraints,UserData] = objectiveFun(params,env)
```

```
% Fix the random stream for reproducibility.
rng(0,"threefry");
```

```
% Get the observation and action input specifications.
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

```
% Create initialization options for the agent networks and
% specify the hidden unit size of 400.
initOpts = rlAgentInitializationOptions(NumHiddenUnit=400);
```

```
% Create agent options and configure hyperparameters
% using the params input argument.
```

```
actorOpts = rlOptimizerOptions( ...
    LearnRate=params.ActorLearnRate, ...
    GradientThreshold=1);
criticOpts = rlOptimizerOptions( ...
    LearnRate=params.CriticLearnRate, ...
    GradientThreshold=1);
agentOpts = rlPPOAgentOptions( ...
    ExperienceHorizon=params.ExperienceHorizon, ...
    ClipFactor=params.ClipFactor, ...
    EntropyLossWeight=params.EntropyLossWeight, ...
    ActorOptimizerOptions=actorOpts, ...
```



```

    CriticOptimizerOptions=criticOpts, ...
    MiniBatchSize=params.MiniBatchSize, ...
    NumEpoch=params.NumEpoch, ...
    SampleTime=0.1, ...
    DiscountFactor=params.DiscountFactor);

% Create the agent object.
agent = rlPPOAgent(obsInfo,actInfo,initOpts,agentOpts);

% Specify the training options.
% To improve performance, do not store the simulation data
% obtained from the episodes, and do not show the training plot.
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=3000, ...
    MaxStepsPerEpisode=600, ...
    StopTrainingCriteria="none", ...
    SimulationStorageType="none", ...
    Plots="none", ...
    Verbose=false);

% Agent evaluator
evl = rlEvaluator(EvaluationFrequency=20,NumEpisodes=5);

% Train the agent
result = train(agent,env,trainOpts,Evaluator=evl);

% Objective function score (last evaluation score)
score = result.EvaluationStatistic;
score(isnan(score)) = [];
objective = -score(end);

% Constraints (this example has no constraints)
constraints = [];

% Store the training result and trained agent in UserData.
UserData.TrainingResult = result;
UserData.Agent = agent;

end

```

Run Bayesian Optimization

Fix the random stream for reproducibility. The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

```

previousRngState = rng(0,"twister")

previousRngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]

```

Create an anonymous function handle for the objective function. This syntax allows you to pass the value that the variable `env` has when the handle is created to the objective function, even though the objective function used by `bayesopt` can have only one input argument.

```
objFun = @(params) objectiveFun(params,env);
```

Tune the hyperparameters using `bayesopt`.

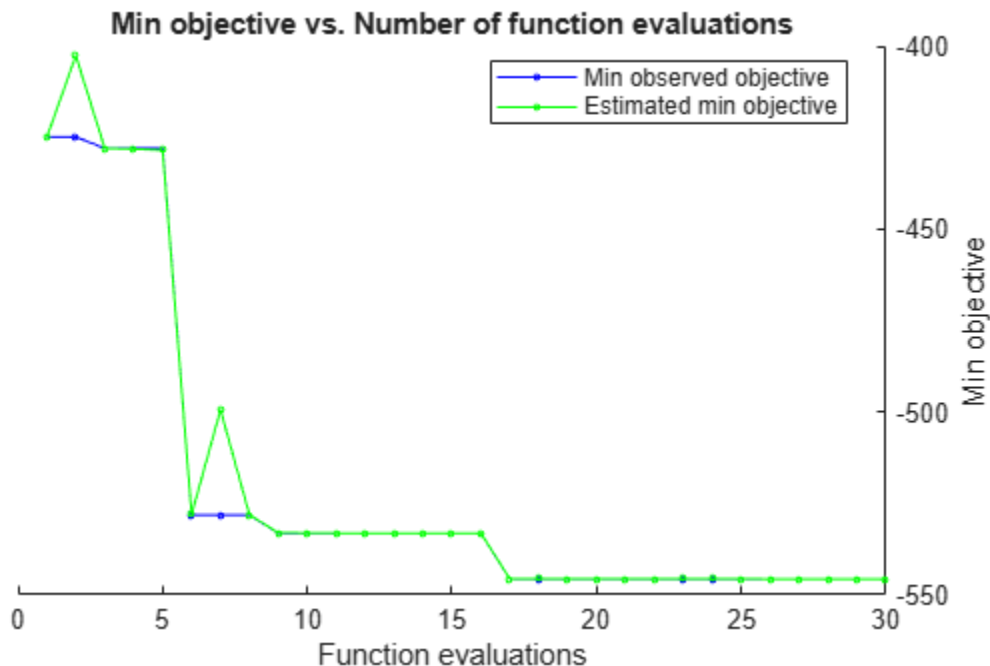
- Specify `objFun` and `optimVars` as input arguments.
- Run the optimization using parallel workers. This option requires Parallel Computing Toolbox™. If you do not have the toolbox installed, set `UseParallel` to `false`.
- Because the optimization process is computationally intensive and takes several hours to complete, load an existing result from a MAT file. If you want to run the optimization algorithm, set `runOptimization` to `true`.

```
runOptimization = ;
if runOptimization
    % Run Bayesian optimization.
    optimResults = bayesopt(objFun,optimVars,UseParallel=true);
else
    % Load existing result.
    load("LanderOptimResults.mat","optimResults");
end
```

Analyze Results

Plot the minimum observed and estimated function values against the number of function evaluations.

```
plot(optimResults,@plotMinObjective)
```



Get the best set of hyperparameters and best objective score using the `bestPoint` method. Store the iteration number for the best point in `iteration`.

```
[tunedParams,maxScore,iteration] = bestPoint(optimResults);
```

Display the maximum score and the iteration.

```
maxScore
```

```
maxScore =
-541.7592
```

```
iteration
```

```
iteration =
17
```

Display the related tuned parameters.

```
rows2vars(tunedParams)
```

```
ans=8×2 table
   OriginalVariableNames   Var1
   _____
   {'MiniBatchSize'      }    453
   {'ExperienceHorizon'}    853
   {'ActorLearnRate'     }  0.0021441
   {'CriticLearnRate'    }  0.0064594
   {'ClipFactor'         }  0.019635
   {'EntropyLossWeight'}  0.045334
   {'NumEpoch'          }     7
   {'DiscountFactor'     }  0.99282
```

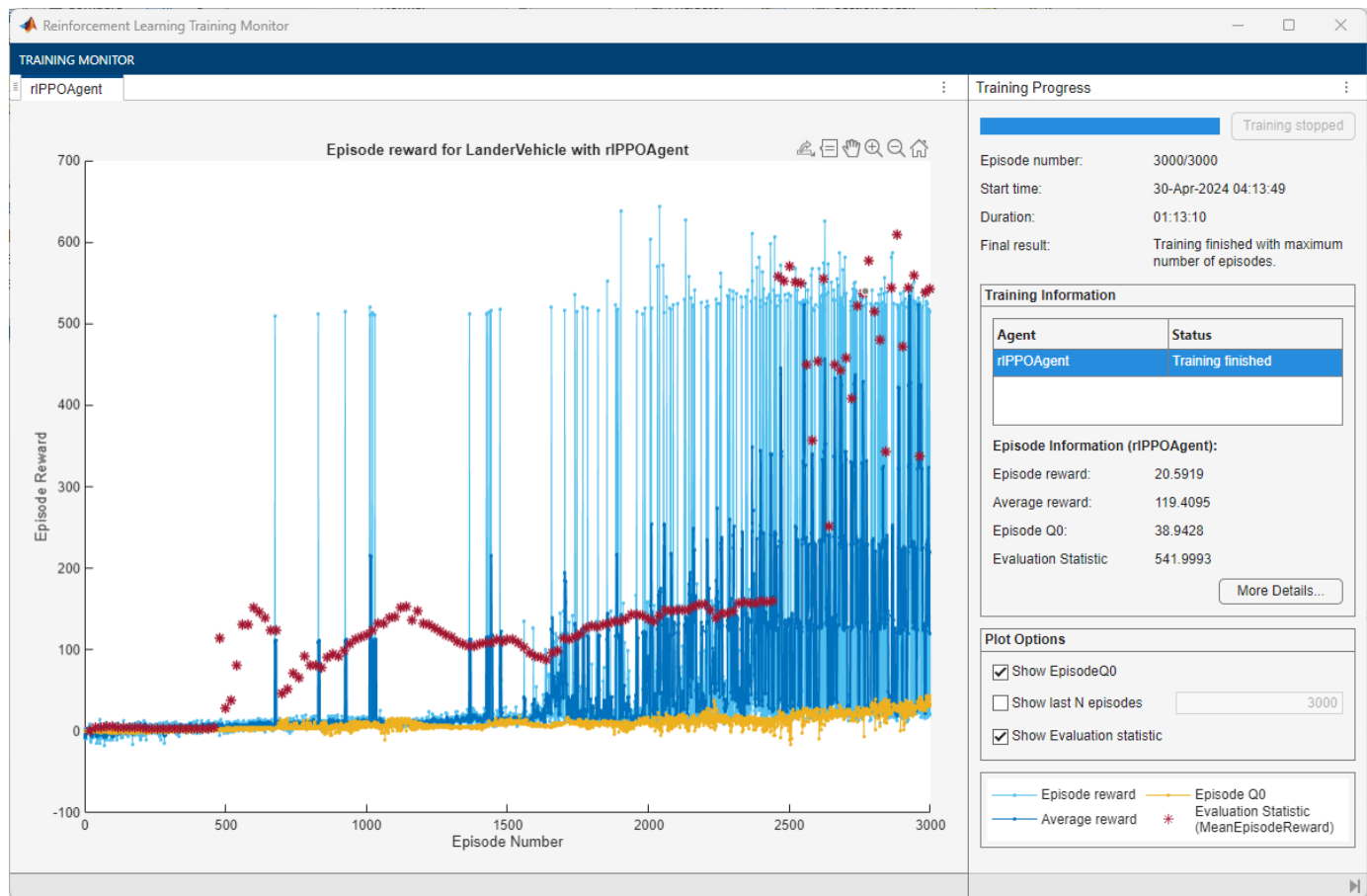
Get the tuned agent and training result stored in the `UserDataTrace` property. For convenience, load the tuned agent and training result from disk.

```
if runOptimization
    % Get the tuned agent and training result from the optimization.
    userData = optimResults.UserDataTrace{iteration};
    tunedAgent = userData.Agent;
    tunedResult = userData.TrainingResult;
else
    % Load previously tuned agent and training result.
    load("LanderOptimResults.mat","tunedAgent","tunedResult");
end
```

View the training plot for the tuned agent.

```
show(tunedResult)
```

The following figure shows the training progress. You can expect different results due to randomness in training.



Simulate Trained Agent

Fix the random stream for reproducibility.

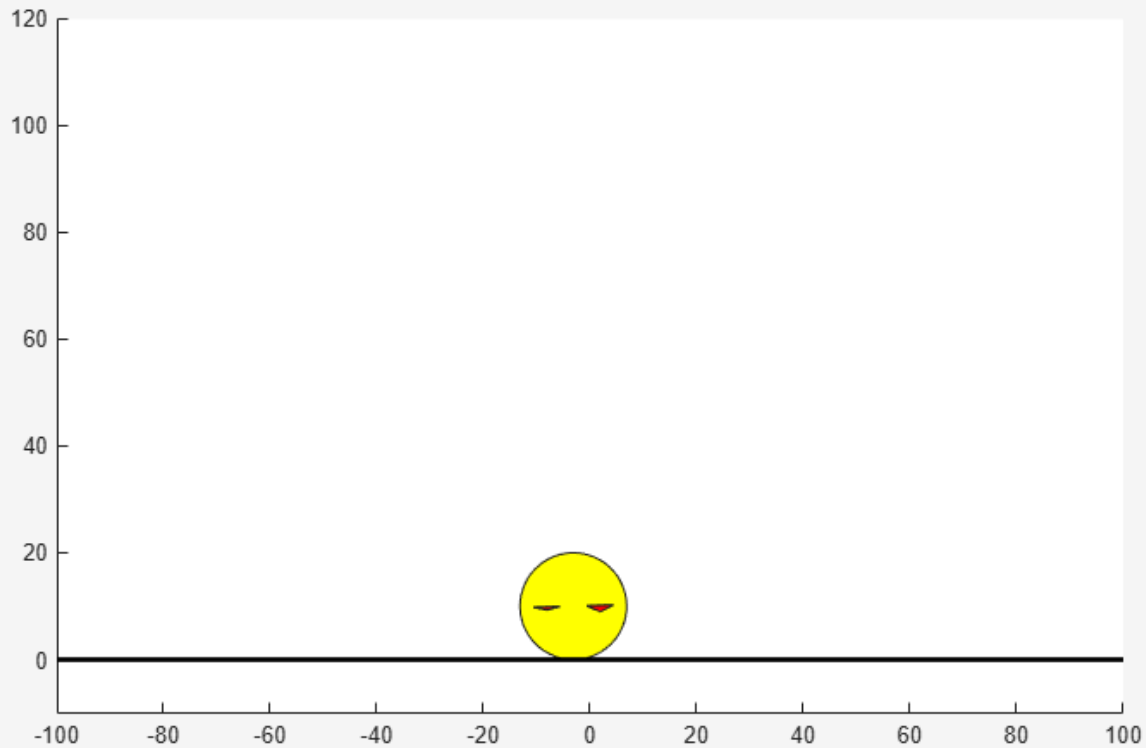
```
rng(0, "twister");
```

Plot the environment.

```
plot(env);
```

Simulate the tuned agent. The environment plot displays an animation of the lander vehicle.

```
simOpts = rlSimulationOptions(MaxSteps=600);
experience = sim(tunedAgent, env, simOpts);
```



The agent successfully lands the vehicle at the target location.

Display the total reward obtained from the simulation.

```
cumulativeReward = sum(experience.Reward.Data)
```

```
cumulativeReward =  
527.7665
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`bayesopt` | `train` | `sim`

Objects

`rlPP0Agent` | `rlPP0AgentOptions` | `rlTrainingOptions` | `optimizableVariable`

Related Examples

- “Log Training Data to Disk” on page 5-231
- “Optimize Cross-Validated Classifier Using bayesopt” (Statistics and Machine Learning Toolbox)
- “Tune Hyperparameters Using Reinforcement Learning Designer” on page 5-53
- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240
- “Configure Exploration for Reinforcement Learning Agents” on page 5-259
- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93

More About

- “Bayesian Optimization Algorithm” (Statistics and Machine Learning Toolbox)
- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Simulink Environments” on page 2-110

Configure Exploration for Reinforcement Learning Agents

This example shows how to use visualization for configuring exploration settings for reinforcement learning agents.

Overview

Exploration in reinforcement learning refers to the strategy that an agent uses to discover new knowledge about its environment. Configuring exploration involves adjusting the parameters that govern how the agent explores the environment, which typically requires numerous iterations to achieve a satisfactory training performance. Visualizing data reduces the number of required experiments by helping to configure exploration.

In this example, you visualize and configure exploration metrics for these reinforcement learning agents:

- Deep Q-network (DQN) agent (using epsilon-greedy exploration)
- Deep deterministic policy gradient (DDPG) agent (using an Ornstein-Uhlenbeck noise model)

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Reinforcement Learning Environments

Create two cart-pole reinforcement learning environments, one with a continuous, and one with a discrete action space. For more information about these environments, see `rlPredefinedEnv` and “Use Predefined Control System Environments” on page 2-42.

```
cEnv = rlPredefinedEnv("CartPole-Continuous");
dEnv = rlPredefinedEnv("CartPole-Discrete");
```

Create the variable `doTraining` to enable or disable training of agents in the example. Because training can be computationally intensive the default value for the variable is `false`. You can enable training by setting the value to `true`.

```
doTraining =  ;
```

Configure Epsilon-Greedy Exploration

A deep Q-network (DQN) agent performs exploration with an epsilon-greedy policy. For more information, see “Deep Q-Network (DQN) Agent” on page 3-32.

The parameters of interest are:

- Initial epsilon value (default 1.0)
- Minimum epsilon value (default 0.01)
- Epsilon decay rate (default 0.005)

When you create a DQN agent, the software assigns the above default values to the parameters. When you first train the agent, the `rlTrainingOptions` function uses these default values.

Create the agent object using the observation and action input specifications of the `dEnv` environment. The agent has these options:

- A learning rate of `1e-4` and 20 hidden units for the critic neural network
- A value of `false` for the option to use the double-DQN algorithm
- A mini-batch size of 256
- A smoothing factor of 1.0 for updating the target critic network every 4 learning iterations

% Observation and action input specifications

```
dEnvObsInfo = getObservationInfo(dEnv);  
dEnvActInfo = getActionInfo(dEnv);
```

% Agent initialization options

```
dqnInitOpts = rlAgentInitializationOptions(NumHiddenUnit=20);
```

% DQN agent options

```
criticOpts = rlOptimizerOptions( ...  
    LearnRate=1e-4, ...  
    GradientThreshold=1);  
dqnOpts = rlDQNAgentOptions( ...  
    CriticOptimizerOptions=criticOpts, ...  
    MiniBatchSize=256, ...  
    TargetSmoothFactor=1, ...  
    TargetUpdateFrequency=4, ...  
    UseDoubleDQN=false);
```

% Create the agent.

```
dqnAgent = rlDQNAgent(dEnvObsInfo, dEnvActInfo, ...  
    dqnInitOpts, dqnOpts);
```

For more information, see `rlDQNAgent`.

Create a data logger object to log data during training. The helper callback function `logEpsilon` on page 5-271 (provided at the end of the example) logs the epsilon values from the training. The logged data is saved in the current directory under the folder named `dqn`.

```
dqnLogger = rlDataLogger();  
dqnLogger.LoggingOptions.LoggingDirectory = "dqn";  
dqnLogger.AgentStepFinishedFcn = @logEpsilon;
```

Train the agent for 500 episodes.

```
dqnTrainOpts = rlTrainingOptions( ...  
    MaxEpisodes=500, ...  
    StopTrainingCriteria="AverageReward", ...  
    StopTrainingValue=480);
```

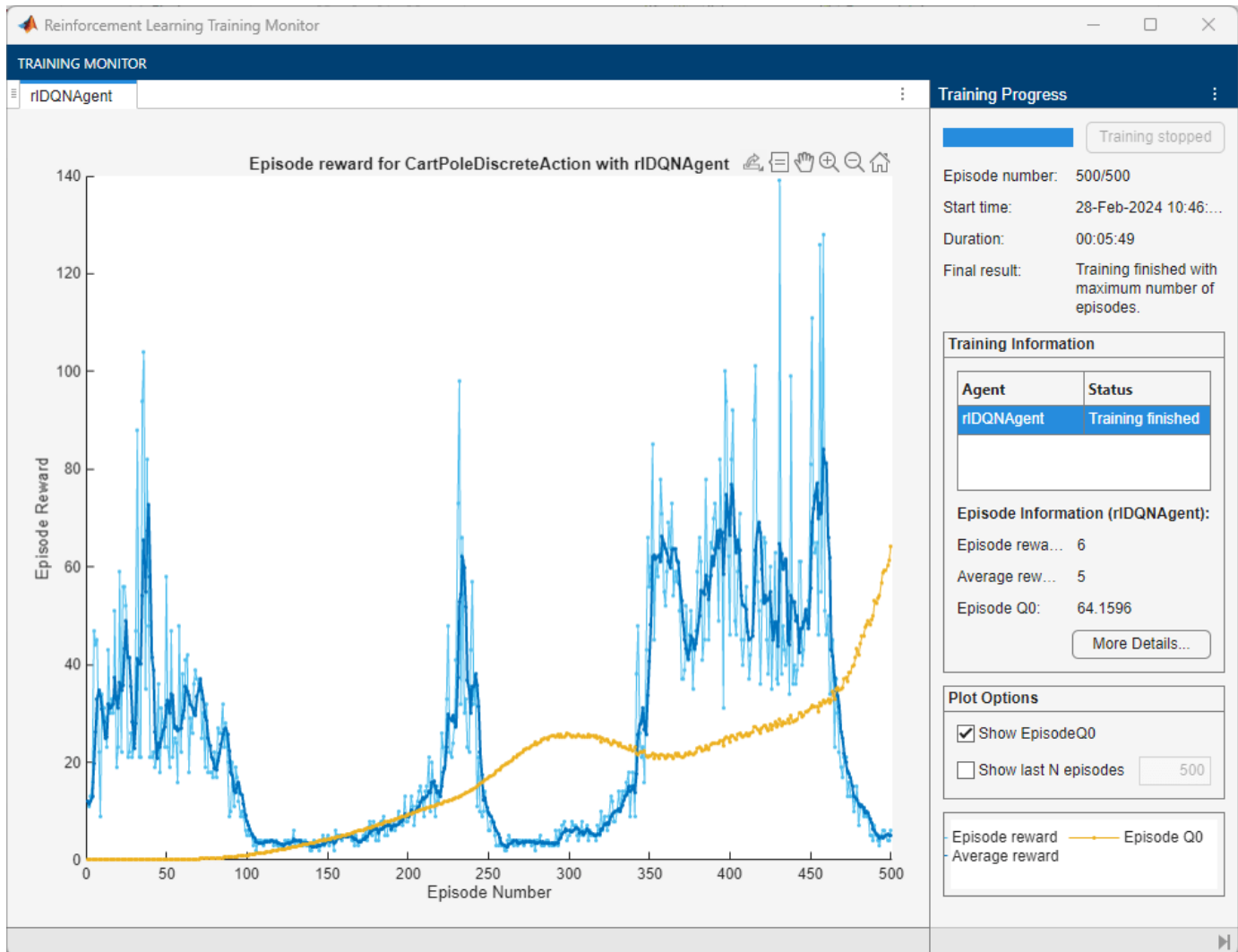


```

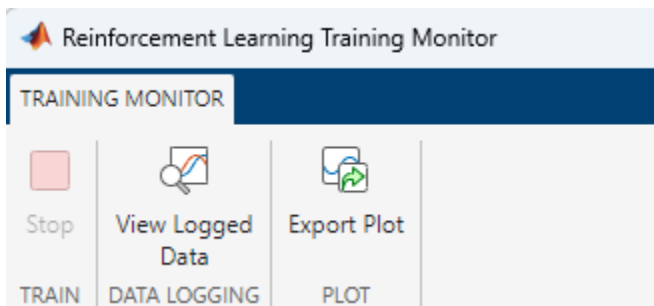
if doTraining
    dqnResult = train(dqnAgent, dEnv, dqnTrainOpts, ...
        Logger=dqnLogger);
end

```

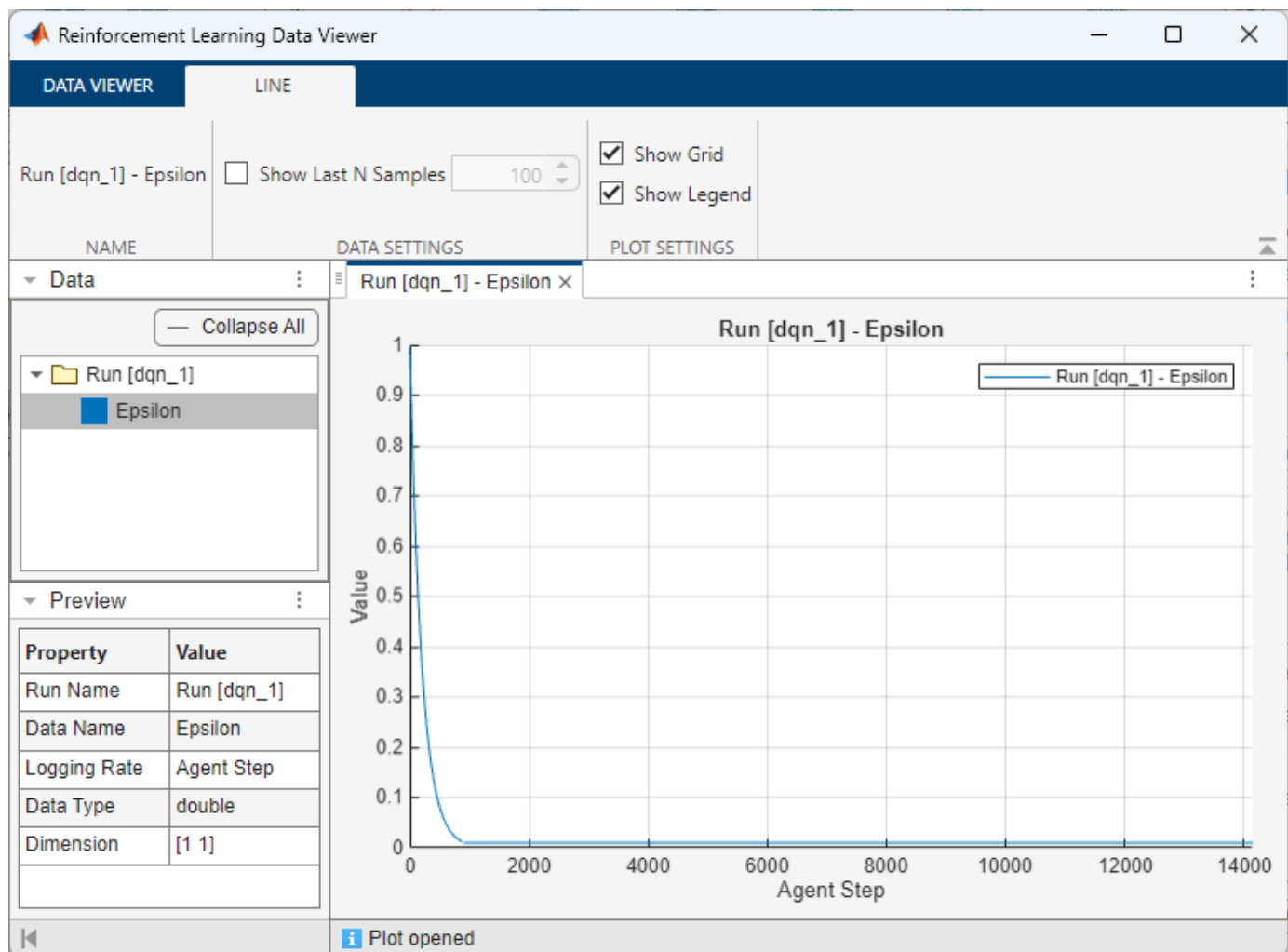
The Reinforcement Learning Training Monitor window shows an example of the training. You might get a different training result depending on your system.



To visualize exploration, click **View Logged Data** in the **Data Logging** section of the Training Monitor tab.



In the Reinforcement Learning Data Viewer window, in the **Data** section, select **Epsilon**, then, from the **Data Viewer** tab in the toolbar, select the **Line** plot type. From the **Line** tab in the toolbar, select **Show Grid** and **Show Legend**.



The plots show the following:

- The average reward that the agent receives over 500 episodes does not reach the value of 480 that is specified to stop training.

- The epsilon value decays to the minimum value after approximately 1000 iterations and the agent does not perform further exploration.

Close the Reinforcement Learning Data Viewer and Reinforcement Learning Training Monitor windows.

Inspect the exploration parameters.

```
dqn0pts.EpsilonGreedyExploration
```

```
ans =
  EpsilonGreedyExploration with properties:

    EpsilonDecay: 0.0050
      Epsilon: 1
    EpsilonMin: 0.0100
```

The default value of the epsilon decay rate is 0.005. Specify a smaller decay rate so the agent performs more exploration.

```
dqn0pts.EpsilonGreedyExploration.EpsilonDecay = 1e-3;
```

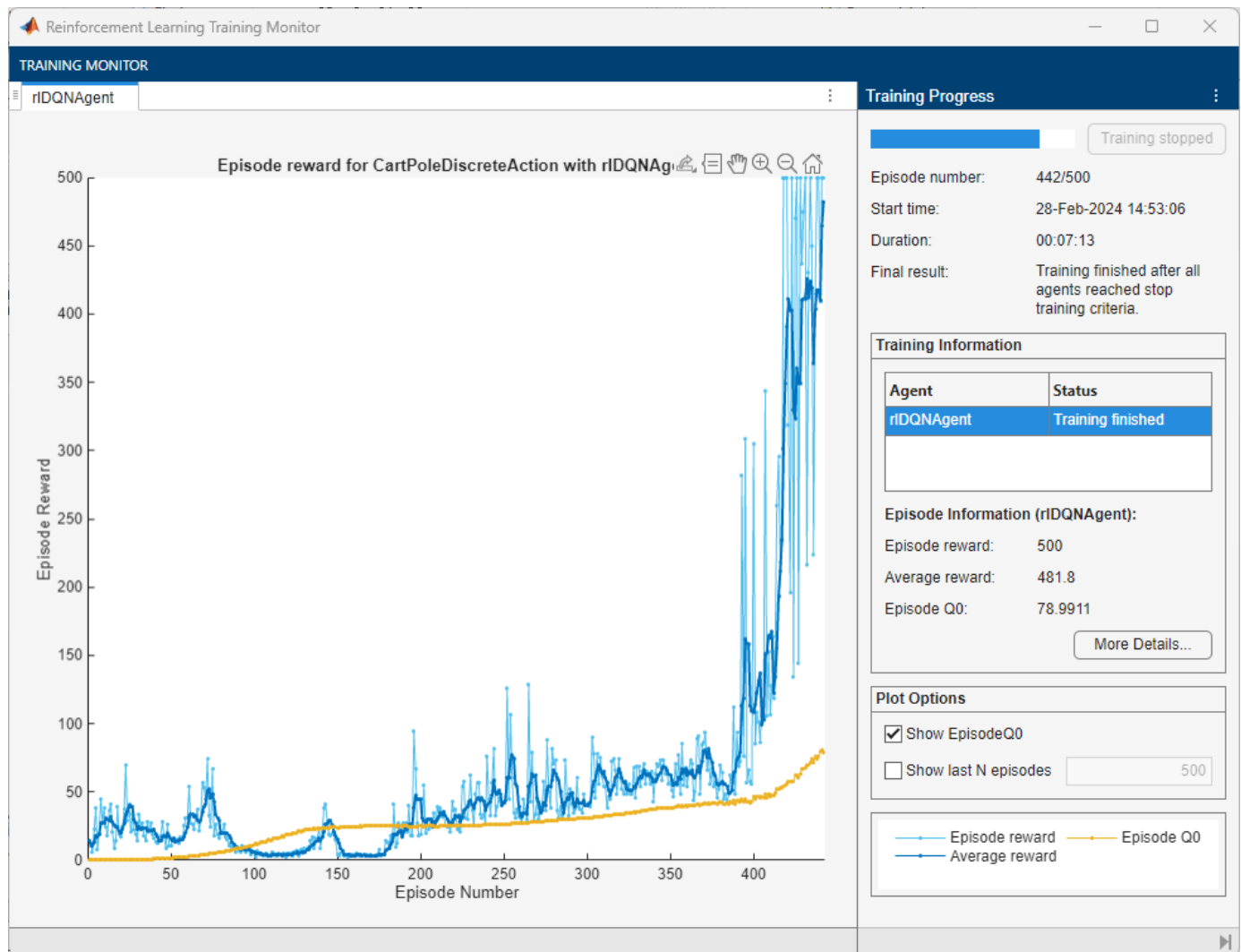
Configure and train the agent with new exploration parameters.

```
% Fix the random stream for reproducibility
rng(0, "twister");

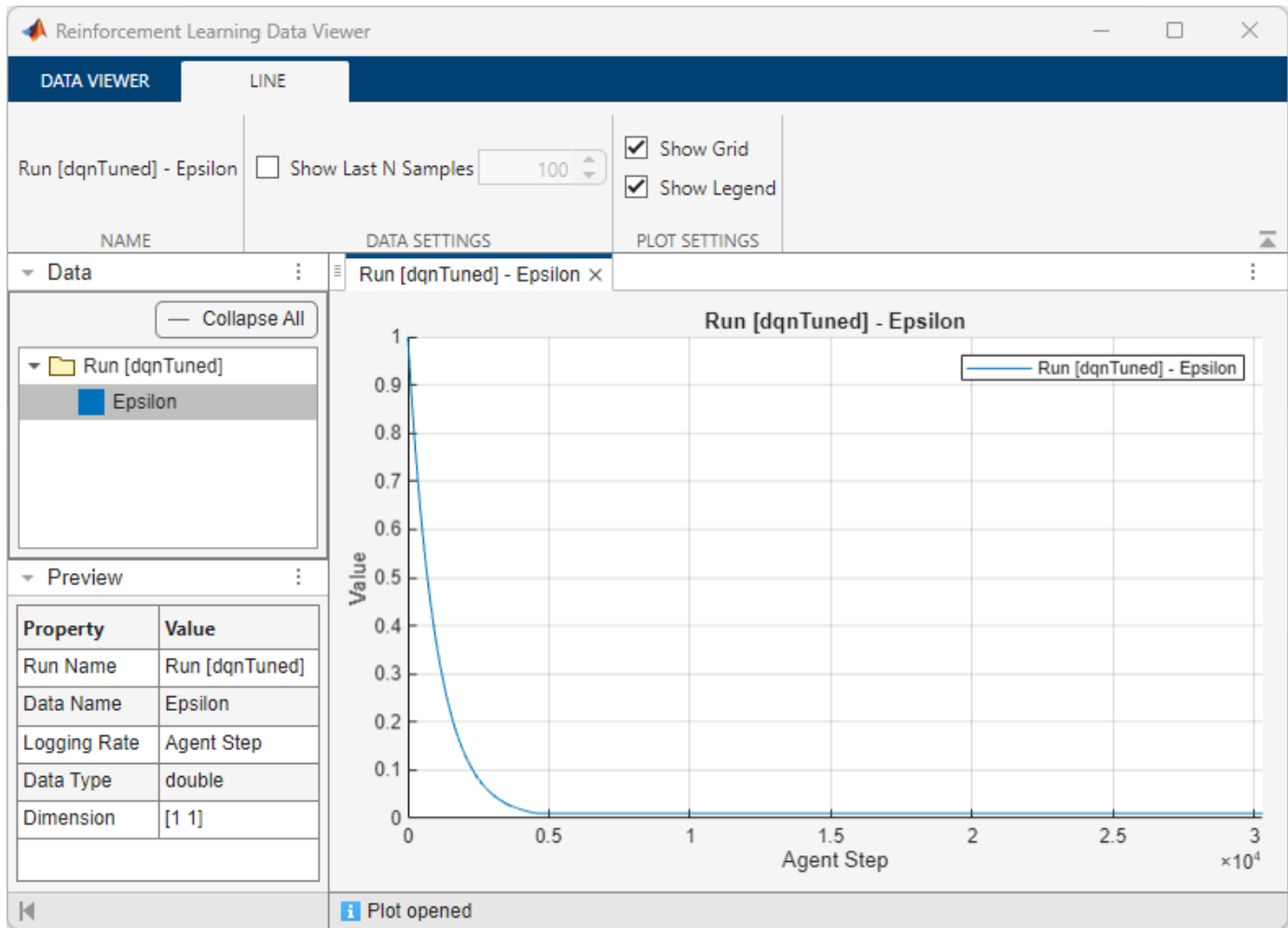
% Create the agent.
dqnAgent = rldQNAgent(dEnvObsInfo, dEnvActInfo, ...
    dqnInitOpts, dqn0pts);

% Train the agent.
dqnLogger.LoggingOptions.LoggingDirectory = "dqnTuned";
if doTraining
    dqnResult = train(dqnAgent, dEnv, dqnTrainOpts, ...
        Logger=dqnLogger);
end
```

This figure shows the training with new exploration parameters.



Open the Reinforcement Learning Data Viewer window and plot the **Epsilon** values again.



The plots show the following:

- The average reward reaches 480.
- The epsilon value decays more slowly than in the previous training. Increasing exploration improves the training performance.

Close the Reinforcement Learning Data Viewer and Reinforcement Learning Training Monitor windows.

Configure Ornstein-Uhlenbeck (OU) Noise Parameters for Exploration

A deep deterministic policy gradient (DDPG) agent uses the Ornstein-Uhlenbeck noise model for exploration. For more information, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

The parameters of interest for the noise model are:

- Mean of the noise (default 0)
- Mean attraction constant (default 0.15)
- Initial standard deviation (default 0.3)

- Standard deviation decay rate (default 0)
- Minimum standard deviation (default 0)

When you create a DQN agent, the software assigns the above default values to the parameters. When you first train the agent, the `rlTrainingOptions` function uses these default values.

Create the agent object using the observation and action input specifications of the `cEnv` environment. The agent uses the following options:

- A learning rate of `1e-4` and 200 hidden units for the actor neural network
- A learning rate of `1e-3` and 200 hidden units for the critic neural network
- A mini-batch size of 64
- A sample time of `0.02s`.

```
% Fix the random stream for reproducibility.
rng(0, "twister");

% Observation and action input specifications
cEnvObsInfo = getObservationInfo(cEnv);
cEnvActInfo = getActionInfo(cEnv);

% Agent initialization options
ddpgInitOpts = rlAgentInitializationOptions(NumHiddenUnit=200);

% Create DDPG agent options
actorOpts = rlOptimizerOptions( ...
    LearnRate=1e-4, ...
    GradientThreshold=1);
criticOpts = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1);
ddpgOpts = rlDDPGAgentOptions( ...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=criticOpts, ...
    MiniBatchSize=64, ...
    SampleTime=cEnv.Ts);

% Create the agent.
ddpgAgent = rlDDPGAgent(cEnvObsInfo, cEnvActInfo, ...
    ddpInitOpts, ddpOpts);
```

For more information, see `rlDDPGAgent`.

Create a data logger object to log data during training. The helper callback function `logOUNoise` on page 5-271 (provided at the end of the example) logs the noise and standard deviation values from the training. Save the logged data in the folder named `ddpg`.

```
ddpgLogger = rlDataLogger();
ddpgLogger.LoggingOptions.LoggingDirectory = "ddpg";
ddpgLogger.AgentStepFinishedFcn = @logOUNoise;
```

Train the agent for 500 episodes.

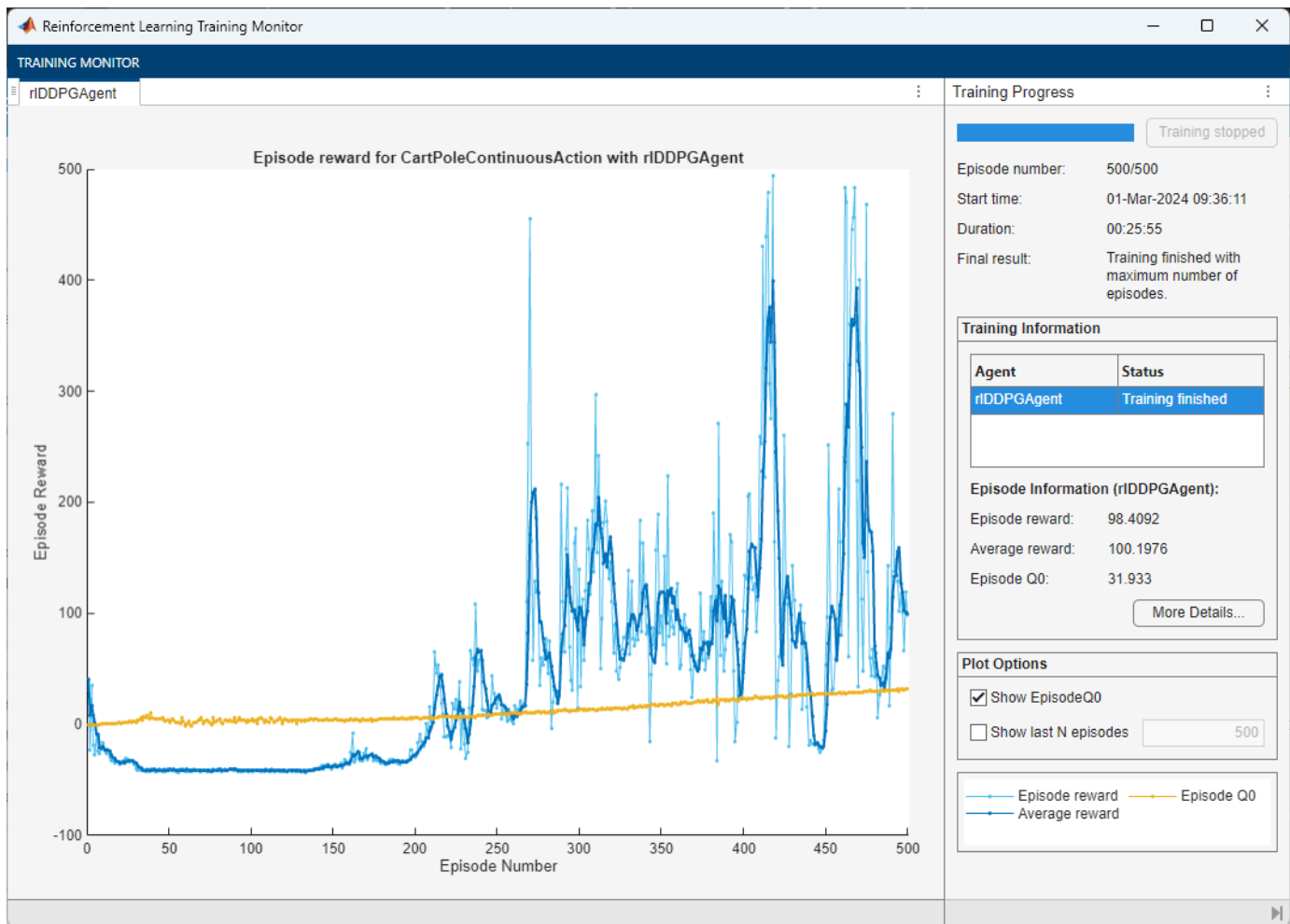
```
ddpgTrainOpts = rlTrainingOptions( ...
    MaxEpisodes=500, ...
    StopTrainingCriteria="AverageReward", ...
```

```

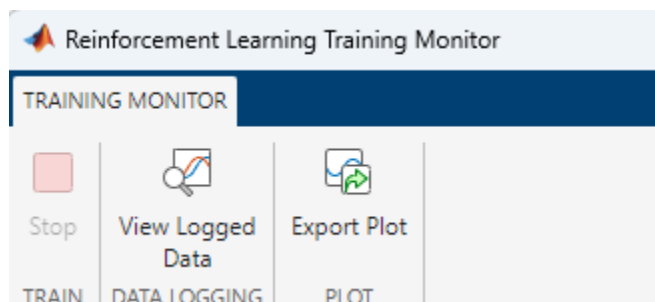
StopTrainingValue=480);
if doTraining
    ddpGResult = train(ddpgAgent, cEnv, ddpGTrainOpts, ...
        Logger=ddpgLogger);
end

```

The Reinforcement Learning Training Monitor window shows an example of the training. You might get a different training result depending on your system.

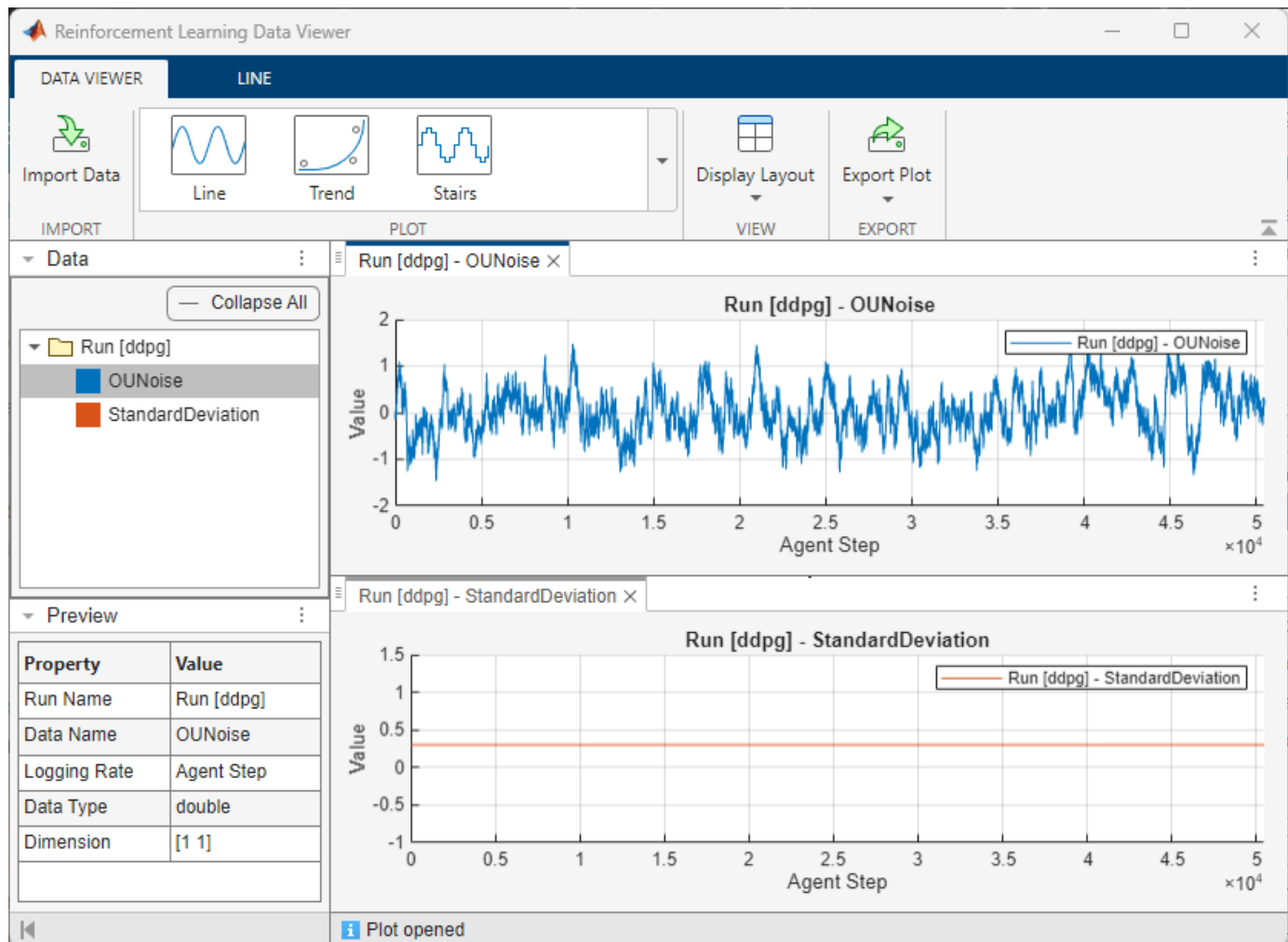


Click the **View Logged Data** button in the **Data Logging** section of the Training Monitor tab.



In the **Data** section of the Reinforcement Learning Data Viewer window:

- Select **OUNoise**. Then, in the toolbar **Data Viewer** tab, select the **Line** plot type.
- Select **StandardDeviation**. Then, in the toolbar **Data Viewer** tab, select the **Line** plot type.



The plots show the following:

- The average reward that the agent receives over 500 episodes does not reach the value of 480 that is specified to stop training.
- The noise value generally remains within ± 1 .
- The standard deviation value remains constant throughout the training.

Close the Reinforcement Learning Data Viewer and Reinforcement Learning Training Monitor windows.

Inspect the default exploration parameters.

```
ddpgOpts.NoiseOptions
```

```
ans =  
OrnsteinUhlenbeckActionNoise with properties:
```



```

        InitialAction: 0
            Mean: 0
    MeanAttractionConstant: 0.1500
    StandardDeviationDecayRate: 0
        StandardDeviation: 0.3000
    StandardDeviationMin: 0

```

You can decay the agent's exploration and gradually shift its behavior from exploration to exploitation as it learns more about the environment. In the early stages of training, more exploration ensures a diverse range of experiences, which is crucial for the agent to learn a robust policy. However, as learning progresses, too much exploration can introduce unnecessary variance and instability into the learning process. Decaying exploration helps to stabilize learning by gradually reducing variance.

- Specify a mean attraction constant value of 0.1. A smaller value reduces the attraction of the noise process toward the mean value.
- Specify an initial standard deviation value of 0.3.
- Specify a standard deviation decay rate of $1e-4$.

```

ddpgOpts.NoiseOptions.MeanAttractionConstant = 0.1;
ddpgOpts.NoiseOptions.StandardDeviation = 0.3;
ddpgOpts.NoiseOptions.StandardDeviationDecayRate = 1e-4;

```

Train the agent with the new exploration options.

```

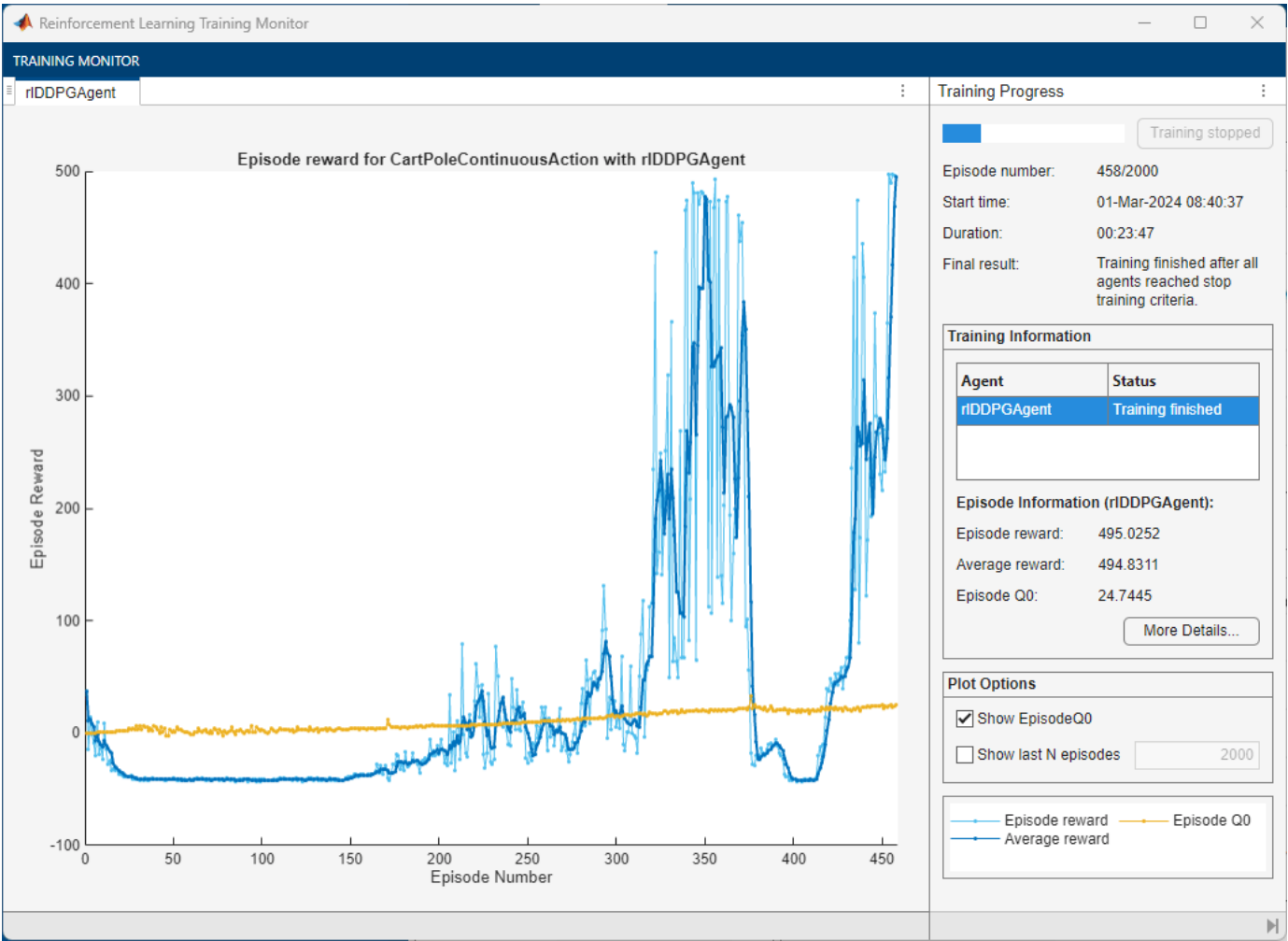
% Fix the random stream for reproducibility.
rng(0, "twister");

% Create the agent.
ddpgAgent = rlDDPGAgent(cEnvObsInfo, cEnvActInfo, ...
    ddpInitOpts, ddpOpts);

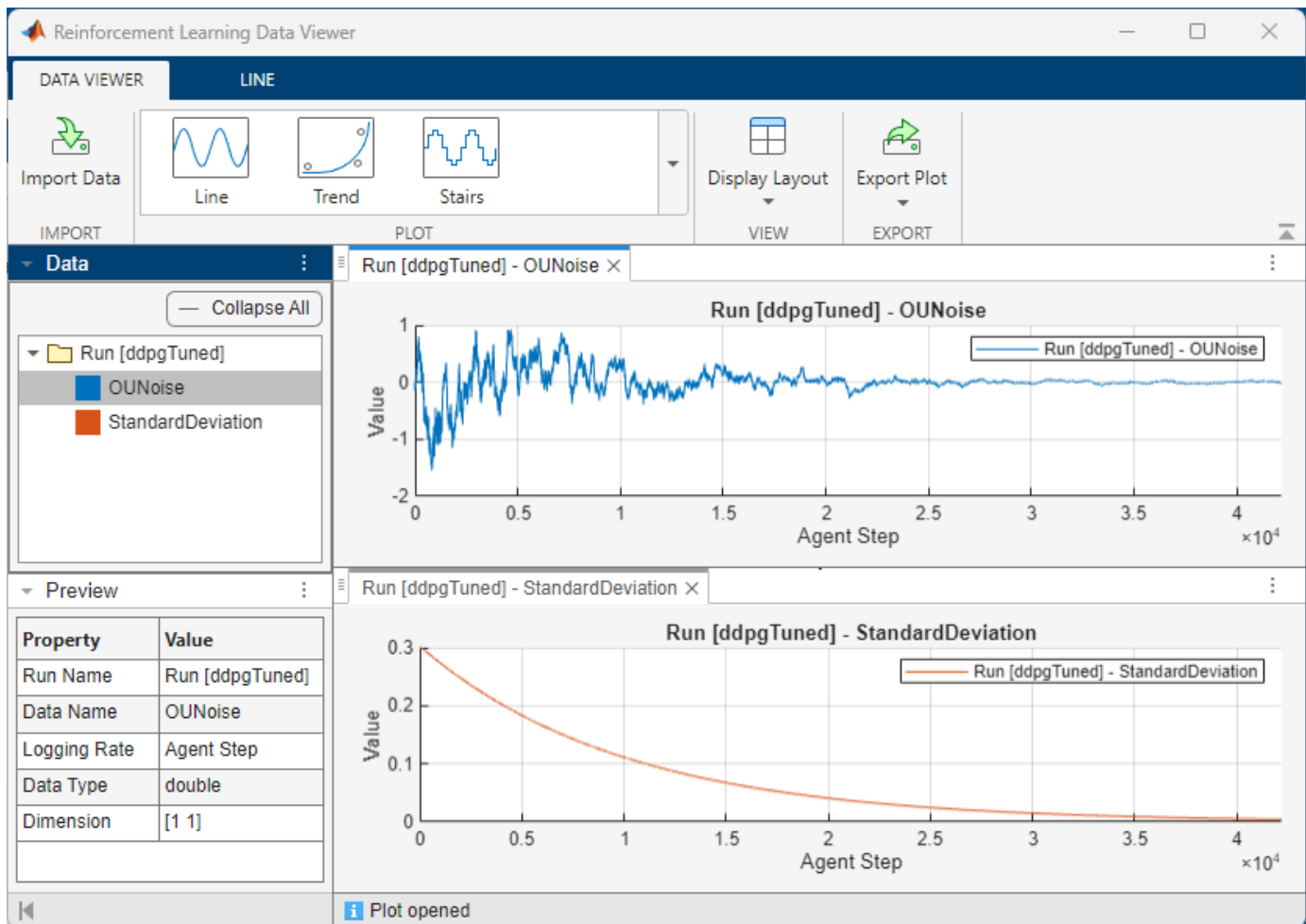
% Train the agent.
ddpgLogger.LoggingOptions.LoggingDirectory = "ddpgTuned";
if doTraining
    ddpResult = train(ddpgAgent, cEnv, ddpTrainOpts, ...
        Logger=ddpgLogger);
end

```

This figure shows the training with new exploration parameters.



Open the Reinforcement Learning Data Viewer window and plot the **OUnoise** and **StandardDeviation** values again.



The plots show the following:

- The average reward reaches the desired value of 480.
- The standard deviation value decays. Consequently, the noise values are larger toward the beginning and smaller toward the end of the training.

Close the Reinforcement Learning Data Viewer and Reinforcement Learning Training Monitor windows.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Helper Functions Used for Logging

```
function dataToLog = logEpsilon(data)
policy = getExplorationPolicy(data.Agent);
pstate = getState(policy);
dataToLog.Epsilon = pstate.Epsilon;
end
```

```
function dataToLog = logOUNoise(data)
policy = getExplorationPolicy(data.Agent);
pstate = getState(policy);
dataToLog.OUNoise = pstate.Noise{1};
dataToLog.StandardDeviation = pstate.StandardDeviation{1};
end
```

See Also

Functions

rlPredefinedEnv | train | sim

Objects

rlTrainingOptions | rlSimulationOptions | rlDQNAgent | rlDQNAgentOptions |
rlDDPGAgent | rlDDPGAgentOptions

Related Examples

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train Agent or Tune Environment Parameters Using Parameter Sweeping” on page 5-240
- “Tune Hyperparameters Using Bayesian Optimization” on page 5-250

More About

- “Use Predefined Control System Environments” on page 2-42
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train Reinforcement Learning Policy Using Custom Training Loop

This example shows how to define a custom training loop for a reinforcement learning policy. You can use this workflow to train reinforcement learning policies with your own custom training algorithms rather than using one of the built-in agents from the Reinforcement Learning Toolbox™ software.

Using this workflow, you can train policies that use any of the following policy and value function approximators.

- `rlValueFunction` - State value function approximator
- `rlQValueFunction` - State-action value function approximator with scalar output
- `rlVectorQValueFunction` - State-action function approximator with vector output
- `rlContinuousDeterministicActor` - Continuous deterministic actor
- `rlDiscreteCategoricalActor` - Discrete stochastic actor
- `rlContinuousGaussianActor` - Continuous Gaussian actor (stochastic)

In this example, a discrete actor policy with a discrete action space is trained using the REINFORCE algorithm (with no baseline). For more information on the REINFORCE algorithm, see “REINFORCE Policy Gradient (PG) Agent” on page 3-37.

For more information on the functions you can use for custom training, see Functions for Custom Training on page 5-279.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Environment

For this example, a reinforcement learning policy is trained in a discrete cart-pole environment. The objective in this environment is to balance the pole by applying forces (actions) on the cart. Create the environment using the `rlPredefinedEnv` function.

```
env = rlPredefinedEnv("CartPole-Discrete");
```

Extract the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Obtain the dimension of the observation space (`numObs`) and the number of possible actions (`numAct`).

```
numObs = obsInfo.Dimension(1);  
numAct = actInfo.Dimension(1);
```

For more information on this environment, see “Use Predefined Control System Environments” on page 2-42.

Policy

The reinforcement learning policy in this example is a discrete-action stochastic policy. It is modeled by a deep neural network that contains `fullyConnectedLayer`, `reluLayer`, and `softmaxLayer` layers. This network outputs probabilities for each discrete action given the current observations. The `softmaxLayer` ensures that the actor outputs probability values in the range [0 1] and that all probabilities sum to 1.

Create the deep neural network for the actor.

```
actorNetwork = [  
    featureInputLayer(numObs)  
    fullyConnectedLayer(24)  
    reluLayer  
    fullyConnectedLayer(24)  
    reluLayer  
    fullyConnectedLayer(2)  
    softmaxLayer  
];
```

Convert to `dlnetwork`.

```
actorNetwork = dlnetwork(actorNetwork);
```

Create the actor using an `rlDiscreteCategoricalActor` object.

```
actor = rlDiscreteCategoricalActor(actorNetwork,obsInfo,actInfo);
```

Accelerate the gradient computation of the actor. For more information, see `dlaccelerate`.

```
actorGradFcn = dlaccelerate(@actorLossFunction);
```

Evaluate the policy with a random observation as input.

```
policyEvalOutCell = evaluate(actor,{rand(obsInfo.Dimension)});  
policyEvalOut = policyEvalOutCell{1}
```

```
policyEvalOut = 2×1 single column vector
```

```
    0.4682  
    0.5318
```

Create a noisy policy from the actor.

```
policy = rlStochasticActorPolicy(actor);
```

Create the optimizer using `rlOptimizer` and `rlOptimizerOptions` function.

```
optimOpt = rlOptimizerOptions(LearnRate=5e-3,GradientThreshold=1);
actorOptimizer = rlOptimizer(optimOpt);
```

Training Setup

Configure the training to use the following options:

- Set up the training to last a maximum of 5000 episodes, with each episode lasting a maximum of 250 steps.
- To calculate the discounted reward, choose a discount factor of 0.995.
- Terminate the training after the maximum number of episodes is reached or when the average reward across 100 episodes reaches the value of 220.

```
numEpisodes = 5000;
maxStepsPerEpisode = 250;
discountFactor = 0.995;
avgWindowSize = 100;
trainingTerminationValue = 220;
```

Create a vector to store the cumulative reward for each training episode.

```
episodeCumulativeRewardVector = [];
```

Create a figure for training visualization using the `hBuildFigure` on page 5-281 helper function.

```
[trainingPlot,lineReward,lineAveReward] = hBuildFigure;
```

Custom Training Loop

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

The algorithm for the custom training loop is as follows. For each episode:

- 1 Create aggregated trajectory data buffers for storing experience information: observations, actions, and rewards.
- 2 Reset the environment.
- 3 Generate experiences until a terminal condition occurs. To do so, evaluate the policy to get actions, apply those actions to the environment, and obtain the resulting observations and rewards. Store the actions, observations, and rewards in buffers.
- 4 Compute the episode Monte Carlo return, which is the discounted future reward.
- 5 Compute the gradient of the loss function with respect to the policy parameters.
- 6 Update the actor using the computed gradients.
- 7 Update the training visualization.
- 8 Terminate training if the policy is sufficiently trained.

Enable the training visualization plot.

```
set(trainingPlot,Visible="on");
```

Create buffers for the training data, allocating enough data for 5 full episodes. Learning occurs after each 5th episode finishes. Aggregating multiple trajectories for on-policy learning can improve training convergence.

```
trajectoriesForLearning = 5;
```

Buffers are allocated as `dlarray` objects to support automatic differentiation with `dlfeval` and `dlgradient`.

```
buffLen = trajectoriesForLearning*maxStepsPerEpisode;  
observationBuffer = dlarray(zeros(numObs,1,buffLen));
```

Allocate valid actions for the action buffer.

```
actionBuffer = dlarray( ...  
    zeros(numAct,1,buffLen) + actInfo.Elements(1));  
rewardBuffer = dlarray(zeros(1,buffLen));  
returnBuffer = dlarray(zeros(1,buffLen));
```

Create a "mask" variable to ensure that we are using consistent batch sizes for every gradient computation. The mask is used to set to zero advantages in the loss function for experiences beyond the actual number of steps taken in any given simulation. This ensures that additional accelerated functions are not generated for the actor loss function.

```
% mask(i) == 1 for i <= batchSize  
% mask(i) == 0 for i > batchSize  
maskBuffer = dlarray(zeros(1,buffLen));  
actionSet = repmat(actInfo.Elements',1,buffLen);
```

Create a weighted discount matrix based on the maximum episode length. This will be used to compute `discountedReturn` before learning.

```
v = 0:(maxStepsPerEpisode-1);  
p = repmat(v',1,maxStepsPerEpisode) - v;  
discountWeights = tril(discountFactor.^p);
```

Train the policy for the maximum number of episodes or until the average reward indicates that the policy is sufficiently trained.

```
for episodeCt = 1:numEpisodes  
    episodeOffset = ...  
        mod(episodeCt-1,trajectoriesForLearning)*maxStepsPerEpisode;  
  
    % 1. Reset the environment at the start of the episode  
    obs = reset(env);  
    episodeReward = zeros(maxStepsPerEpisode,1);  
  
    % 3. Generate experiences  
    % for the maximum number of steps per episode  
    % or until a terminal condition is reached.  
    for stepCt = 1:maxStepsPerEpisode  
  
        % Compute an action using the policy  
        % based on the current observation.  
        action = getAction(policy,{obs});  
  
        % Apply the action to the environment  
        % and obtain the resulting observation and reward.  
        [nextObs,reward,isdone] = step(env,action{1});  
  
        % Store the action, observation,  
        % and reward experiences in their buffers.
```



```

j = episodeOffset + stepCt;
observationBuffer(:,j) = obs;
actionBuffer(:,j) = action{1};
rewardBuffer(:,j) = reward;
maskBuffer(:,j) = 1;

obs = nextObs;

% Stop if a terminal condition is reached.
if isdone
    break;
end

end

% Update the return buffer and the
% cumulative reward for this episode.
episodeElements = episodeOffset + (1:maxStepsPerEpisode);
episodeCumulativeReward = extractdata( ...
    sum(rewardBuffer(episodeElements)) );

% 4. Compute the discounted future reward.
returnBuffer(episodeElements) = ...
    rewardBuffer(episodeElements)*discountWeights;

% Learn the set of aggregated trajectories.
if mod(episodeCt,trajectoriesForLearning) == 0

    % Get the indices of each action taken in the action buffer.
    actionIndicationMatrix = ...
        dldarray(single(actionBuffer(:, :) == actionSet));

    % 5. Compute the gradient of the loss
    % with respect to the actor learnable parameters.
    actorGradient = dlfeval( ...
        actorGradFcn, ...
        actor, ...
        {observationBuffer}, ...
        actionIndicationMatrix, ...
        returnBuffer, ...
        maskBuffer);

    % 6. Update the actor using the computed gradients.
    [actor,actorOptimizer] = update( ...
        actorOptimizer, ...
        actor, ...
        actorGradient);

    % Update the policy from the actor
    policy = rlStochasticActorPolicy(actor);

    % flush the mask and reward buffer
    maskBuffer(:) = 0;
    rewardBuffer(:) = 0;
end

% 7. Update the training visualization.

```

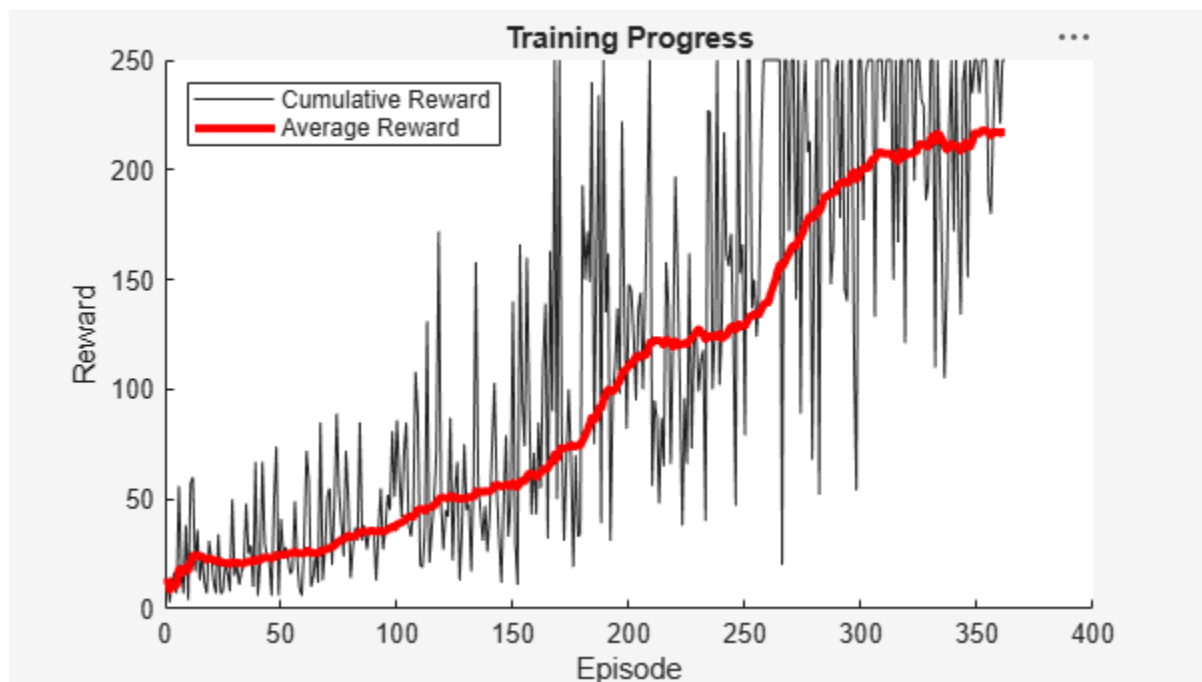
```

episodeCumulativeRewardVector = cat(2, ...
    episodeCumulativeRewardVector, episodeCumulativeReward);
movingAvgReward = movmean(episodeCumulativeRewardVector, ...
    avgWindowSize, 2);
addpoints(lineReward, episodeCt, episodeCumulativeReward);
addpoints(lineAveReward, episodeCt, movingAvgReward(end));
drawnow;

% 8. Terminate training if the network is sufficiently trained.
if max(movingAvgReward) > trainingTerminationValue
    break
end

end

```



Simulation

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

After training, simulate the trained policy.

Before simulation, reset the environment and set the policy to use the maximum likelihood (greedy) action.

```
obs = reset(env);
policy.UseMaxLikelihoodAction = true;
```

Enable the environment visualization, which is updated each time the environment step function is called.

```
plot(env)
```

For each simulation step, perform the following actions.

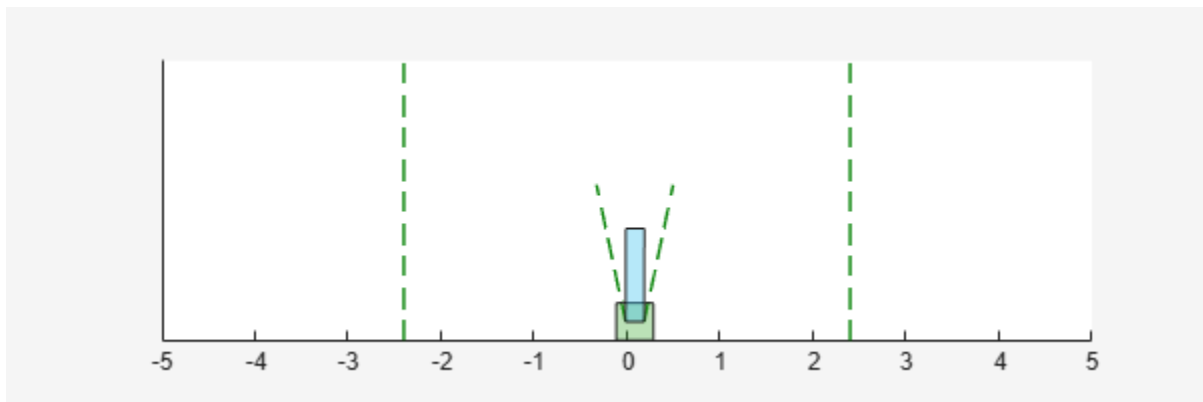
- 1 Get the action by sampling from the policy using the `getAction` function.
- 2 Step the environment using the obtained action value.
- 3 Terminate if a terminal condition is reached.

```
for stepCt = 1:maxStepsPerEpisode
    % Select action according to trained policy
    action = getAction(policy,{obs});

    % Step the environment
    [nextObs,reward,isdone] = step(env,action{1});

    % Check for terminal condition
    if isdone
        break
    end

    obs = nextObs;
end
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Functions for Custom Training

To obtain actions and value functions for given observations from Reinforcement Learning Toolbox policy and value function approximators, you can use the following functions. These functions can also be used in automatically differentiated functions with `dlfeval`, `dlgradient`, and `dlaccelerate`.

- `getValue` — Obtain the estimated state value or state-action value function.
- `getAction` — Obtain the action from an actor based on the current observation.
- `getMaxQValue` — Obtain the estimated maximum state-action value function for a discrete Q-value approximator.

If your policy or value function approximator is a recurrent neural network, that is, a neural network with at least one layer that has hidden state information, the preceding functions can return the

current network state. You can use the following function syntaxes to get and set the state of your approximator.

- `state = critic.State` — Obtain the state of approximator `critic`.
- `critic.State = state` — Set the state of approximator `critic`.

You can get and set the learnable parameters of your approximator using the `getLearnableParameters` and `setLearnableParameters` function, respectively.

In addition to these functions, you can use the `dlgradient`, `update`, and `syncParameters` functions to set parameters and compute gradients for your policy and value function approximators.

Loss Function

The loss function in the REINFORCE algorithm the product between the discounted reward and the logarithm of the probability distribution of the action (coming from the policy evaluation for a given observation), summed across all time steps. An additional entropy loss is added to improve exploration and reduce over-fitting during training.

```
function actorGradient = actorLossFunction( ...
    actor, observation, actionIndicationMatrix, discountedReturn, mask)

    % Compute the advantage and apply it
    % to the actions taken by the policy.
    advantage = actionIndicationMatrix.*discountedReturn.*mask;

    % Evaluate the action probabilities.
    % Set the UseForward name-value pair to true
    % to support cases where the actor has layers
    % that define a forward pass different than prediction
    % (e.g. batch normalization or dropout layers).
    out = evaluate(actor, observation, UseForward=true);
    actionProbabilities = out{1};

    % Clip any action probability values less than eps to eps.
    actionProbabilities(actionProbabilities < eps) = eps;

    % Compute the log of the action probabilities.
    actionLogProbabilities = log(actionProbabilities);

    % Compute the policy gradient loss.
    pgLoss = -sum(advantage.*actionLogProbabilities, "all");

    % Compute the entropy loss.
    entropyLoss = -sum( ...
        actionProbabilities.*actionLogProbabilities.*mask, ...
        "all");

    % Compute the gradient of loss
    % with respect to the actor learnable parameters.
    loss = (pgLoss + 1e-4*entropyLoss)/(sum(mask));
    actorGradient = dlgradient(loss, actor.Learnables);
end
```

Helper Function

The following helper function creates a figure for training visualization.

```
function [trainingPlt, lineRewd, lineAvgRwd] = hBuildFigure()
    plotRatio = 16/9;
    trainingPlt = figure( ...
        Visible="off", ...
        HandleVisibility="off", ...
        NumberTitle="off", ...
        Name="Cart Pole Custom Training");

    trainingPlt.Position(3) = ...
        plotRatio * trainingPlt.Position(4);

    ax = gca(trainingPlt);

    lineRewd = animatedline(ax);
    lineAvgRwd = animatedline(ax,Color="r",LineWidth=3);
    xlabel(ax,"Episode");
    ylabel(ax,"Reward");
    legend(ax,"Cumulative Reward","Average Reward", ...
        Location="northwest")
    title(ax,"Training Progress");
end
```

See Also

Functions

`dlaccelerate` | `dlgradient` | `dlfeval` | `evaluate` | `rlOptimizer` | `getLearnableParameters`
| `setLearnableParameters`

Objects

`rlDiscreteCategoricalActor` | `rlOptimizerOptions`

Related Examples

- “Custom Training Loop with Simulink Action Noise” on page 5-282
- “Create and Train Custom LQR Agent” on page 5-345
- “Create and Train Custom PG Agent” on page 5-334
- “Custom DQN Training Loop with LSTM Network” on page 5-317
- “Custom PPO Training Loop with Random Network Distillation” on page 5-298

More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Reinforcement Learning Agents” on page 3-92
- “Use Predefined Control System Environments” on page 2-42

Custom Training Loop with Simulink Action Noise

This example shows how to tune a controller for vehicle platooning applications using a custom reinforcement learning (RL) training loop. For this application, action noise is generated in the Simulink® model to promote exploration during training.

For an example on tuning a PID-based vehicle platooning system, see “Design Controller for Vehicle Platooning” (Simulink Control Design).



Platooning has the following control objectives [1].

- Individual vehicle stability — Spacing error for each following vehicle converges to zero if the preceding vehicle is traveling at constant speed.
- String stability — Spacing errors do not amplify as they propagate toward the tail of the vehicle string.

Platooning Environment Model

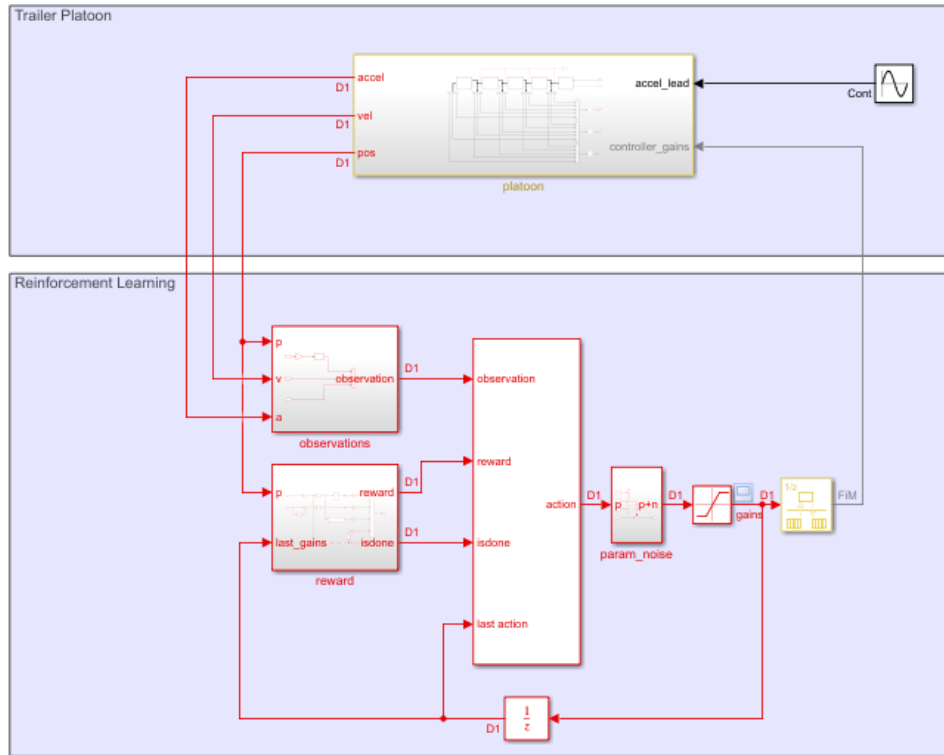
In this example, there are five vehicles in the platoon. Every vehicle is modeled as a truck-trailer system with the following parameters. All lengths are in meters.

```
L1 = 6;           % Truck length
L2 = 10;          % Trailer length
M1 = 1;           % Hitch length
L = L1 + L2 + M1 + 5; % Desired front-to-front vehicle spacing
```

The lead vehicle follows a given acceleration profile. Each trailing vehicle has a controller that controls its acceleration.

Open the Simulink® model.

```
mdl = "fiveVehiclePlatoonEnv";
open_system(mdl)
```



The model contains an RL Agent block with its **last action** input port enabled. This input port allows the specification of custom noise in the Simulink model for off-policy RL agents, such as deep deterministic policy gradient (DDPG) agents.

Specify the path to the RL Agent block.

```
agentBlk = mdl + "/RL Agent";
```

Controller Structure

In this example, each trailing vehicle (ego vehicle) has the same continuous-time controller structure and parameterization.

$$a_{\text{ego}} = K_1 a_{\text{front}} - K_2 (v_{\text{ego}} - v_{\text{front}}) - K_3 (x_{\text{ego}} - x_{\text{front}} + L)$$

Here:

- a_{ego} , v_{ego} , and x_{ego} are the respective acceleration, velocity, and position of the ego vehicle.
- a_{front} , v_{front} , and x_{front} are the respective acceleration, velocity, and position of the vehicle directly in front of the ego vehicle.

Each vehicle has full access to its own velocity and position states but can only access the acceleration, velocity, and position of the vehicle directly in front using wireless communication.

The controller minimizes the velocity error $v_{\text{front}} - v_{\text{ego}}$ using the velocity gain K_2 and minimizes the spacing error $x_{\text{ego}} - x_{\text{front}} + L$ using the spacing gain K_3 . The feedforward gain K_1 is used to improve tracking of the front vehicle.

The lead vehicle' acceleration is assumed to be a sine wave

$$a_{\text{lead}} = A \sin(wt)$$

Here:

- a_{lead} is the lead vehicle acceleration (m/s²).
- A is the amplitude of the sine wave (m/s).
- w is the frequency of the sine wave (rad/s).
- t is the current simulation time (s).

Reinforcement Learning Agent Design

The objective of the agent is to compute adaptive gains so that each vehicle can track the desired spacing with respect to the vehicle immediately in front. Therefore, the model is configured such that:

- The action signal consists of the gains $\mathbf{K} = [K_1 \ K_2 \ K_3]$ shared by all vehicles except the lead vehicle. Each gain has a lower bound of 0 and upper bounds of 1, 20, and 20, respectively. The agent calculates new gains once per second. To encourage exploration during training, the gains are perturbed by random noise with a zero-mean normal distribution: $K_{\text{noisy}} = K + N(0, \sigma^2)$ where the variance $\sigma^2 = [0.02, 0.1, 0.1]$.
- The observation signal consists of the vehicle spacing ($s_t = \text{diff}(\mathbf{x})$) minus the target spacing (L), the vehicle velocities ($\mathbf{v}$), and the vehicle accelerations ($\mathbf{a}$).
- The reward calculated at every time step t is

$$r_t = \sum \frac{1}{1 + (\mathbf{s}_t - L)^2} - 0.2 \sum (\mathbf{K}_{t-1} - \mathbf{K}_{t-2})^2 - \text{maxOvershoot}(\mathbf{s}_t, L) - 10c_t$$

where:

- $\mathbf{s}_t$ is the vehicle spacing ($\text{diff}(\mathbf{x}_t)$) at time step t .
- $\text{maxOvershoot}(\mathbf{s}_t, L)$ calculates the max overshoot of all vehicles given the actual vehicle spacing s_t and the desired spacing L . In this case, overshoot is defined as when the vehicle spacing is less than the desired spacing $\mathbf{s}_t < L$.
- $c_t = \text{any}((\mathbf{s}_t + L_1 + L_2 + M_1 - L) \leq 0)$ indicates if a vehicle collision occurs. The simulation will terminate if c_t is true.

The first term in the reward function encourages the vehicle spacing to match L . The second term penalizes large changes in gain between time steps. The third term penalizes overshooting the target spacing (getting too close to the front vehicle). Finally, the fourth term penalizes collisions.

For this example, to accommodate the custom noise specified in the model, you implement a custom DDPG training loop.

Define Model Parameters

Define the training and simulation parameters that remain fixed during training.

```
Ts = 1; % Sample time (seconds)
Tf = 100; % Simulation length (seconds)
```



```

AccelNoiseV = ones(1,5)*0.01; % Acceleration input noise variance
VelNoiseV = ones(1,5)*0.01;   % Velocity sensor noise variance
PosNoiseV = ones(1,5)*0.01;   % Position sensor noise variance
ParamLowerLimit = [0 0 0]';   % Lower limits for the controller gains
ParamUpperLimit = [1 20 20]'; % Upper limits for the controller gains
UseParamNoise = 1;            % Option to indicate noise injection

```

Define the parameters that change every training episode. The values for these parameters are updated in the environment reset function `resetFunction`.

```

LeadA = 2; % Lead vehicle acceleration amplitude
LeadF = 1; % Lead vehicle acceleration frequency
ParamNoiseV = [0.02 0.1 0.1]; % Variance for controller gains

% Random noise seeds
ParamNoiseSeed = 1:3; % Controller gain noise seed
AccelNoiseSeed = 1:5 + 100; % Acceleration input noise seed
VelNoiseSeed = 1:5 + 200; % Velocity sensor noise seed
PosNoiseSeed = 1:5 + 300; % Position sensor noise seed

% Initial position and velocity of each vehicle
InitialPositions = [200 150 100 50 0] + 50; % Positions
InitialVelocities = [10 10 10 10 10]; % Velocities

```

Create Environment

Create an environment using `rlSimulinkEnv`.

To do so, first define the observation and action specifications for the environment.

```

obsInfo = rlNumericSpec([14 1]);
actInfo = rlNumericSpec([3 1], ...
    LowerLimit=ParamLowerLimit, ...
    UpperLimit=ParamUpperLimit);

obsInfo.Name = "measurements";
actInfo.Name = "control_gains";

```

Next, create the environment object.

```
env = rlSimulinkEnv mdl, agentBlk, obsInfo, actInfo);
```

Set the environment reset function to the local function `resetFunction` included with this example. This function varies the training conditions for each episode.

```
env.ResetFcn = @resetFunction;
```

Noise in the model is specified using the Random Number (Simulink) block. Each block has its own random number generator and thus its own starting seed parameter. To ensure the noise stream varies across episodes, the seed variables are updated using `resetFunction`.

Create Actor, Critic, and Policy

Create actor and critic function approximators for the agent using the local function `createNetworks` included with this example.

```
[critic, actor] = createNetworks(obsInfo, actInfo);
```

Create optimizer objects for updating the actor and critic. Use the same options object for both optimizers. For more information, see `rlOptimizerOptions` and `rlOptimizer`.

```
optimizerOpt = rlOptimizerOptions( ...  
    LearnRate=1e-3, ...  
    GradientThreshold=1, ...  
    L2RegularizationFactor=1e-2);  
criticOptimizer = rlOptimizer(optimizerOpt);  
actorOptimizer = rlOptimizer(optimizerOpt);
```

Create a deterministic policy for the actor approximator. For more information, see `rlDeterministicActorPolicy`.

```
policy = rlDeterministicActorPolicy(actor);
```

Specify the policy sample time.

```
policy.SampleTime = Ts;
```

Create Experience Buffer

Create an experience buffer for the agent with a maximum length of `1e6`.

```
replayMemory = rlReplayMemory(obsInfo,actInfo,1e6);
```

Data Required for Learning

To update the actor and critic during training, the `runEpisode` function calls a processing function to process each experience as it is received from the environment. For this example, the processing function is the local function `processExperienceFcn`.

This function requires additional data to perform its processing. Create a structure to store this additional data.

```
processExpData.Critic = critic;  
processExpData.TargetCritic = critic;  
processExpData.Actor = actor;  
processExpData.TargetActor = actor;  
processExpData.ReplayMemory = replayMemory;  
processExpData.CriticOptimizer = criticOptimizer;  
processExpData.ActorOptimizer = actorOptimizer;
```

```
processExpData.MinibatchSize = 128;  
processExpData.DiscountFactor = 0.99;  
processExpData.TargetSmoothFactor = 1e-3;
```

```
% provide accelerated functions to processExpData  
% to accelerate gradient computations
```

```
processExpData.AccelCriticGradFcn = ...  
    dlaccelerate(@deepCriticGradient);
```

```
processExpData.AccelActorGradFcn = ...  
    dlaccelerate(@deepActorGradient);
```

During each episode, the `processExperienceFcn` function updates the critics, actors, replay memory, and optimizers. The updated data is used as the input for the next episode.

Training Loop

To train the agent, the custom training loop simulates the agent in the environment for a maximum of `maxEpisodes` episodes.

```
maxEpisodes = 1000;
```

Compute the maximum number of steps per episode using the simulation time and sample time.

```
maxSteps = ceil(Tf/Ts);
```

For this custom training loop:

- The `runEpisode` function simulates the agent in the environment for one episode.
- Experiences are processed as they are received from the environment using the `processExperienceFcn` function.
- Experiences are not logged by `runEpisode` since they are processed as they are received.
- To speed up training, when calling `runEpisode`, the `CleanupPostSim` option is set to `false`. Doing so keeps the model compiled between episodes.
- The `PlatooningTrainingCurvePlotter` object is a helper object to plot training data while the training is running.
- You can stop the training using a **Stop** button in the training plot.
- After all the episodes are complete, the `cleanup` function cleans up the environment and terminates the model compilation.

Training the policy is a computationally intensive process that can take several minutes to hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the policy yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Create plotting helper object.
    plotObj = PlatooningTrainingCurvePlotter();

    % Training loop
    for i = 1:maxEpisodes

        % Run the episode.
        out = runEpisode( ...
            env,policy, ...
            MaxSteps=maxSteps, ...
            ProcessExperienceFcn=@processExperienceFcn, ...
            ProcessExperienceData=processExpData, ...
            LogExperiences=false, ...
            CleanupPostSim=false);

        % Extract episode information
        % to update the training curves.
        episodeInfo = out.AgentData.EpisodeInfo;

        % Extract updated processExpData for the next episode.
        processExpData = out.AgentData.ProcessExperienceData;
        % Extract the updated policy for the next episode.
        policy = out.AgentData.Agent;
```

```
% Extract critic and actor approximators
% from processExpData.
critic = processExpData.Critic;
actor  = processExpData.Actor;

% Extract the cumulative reward and calculate
% average reward per step for this episode.
cumulativeRwd = episodeInfo.CumulativeReward;
avgRwdPerStep = cumulativeRwd/episodeInfo.StepsTaken;

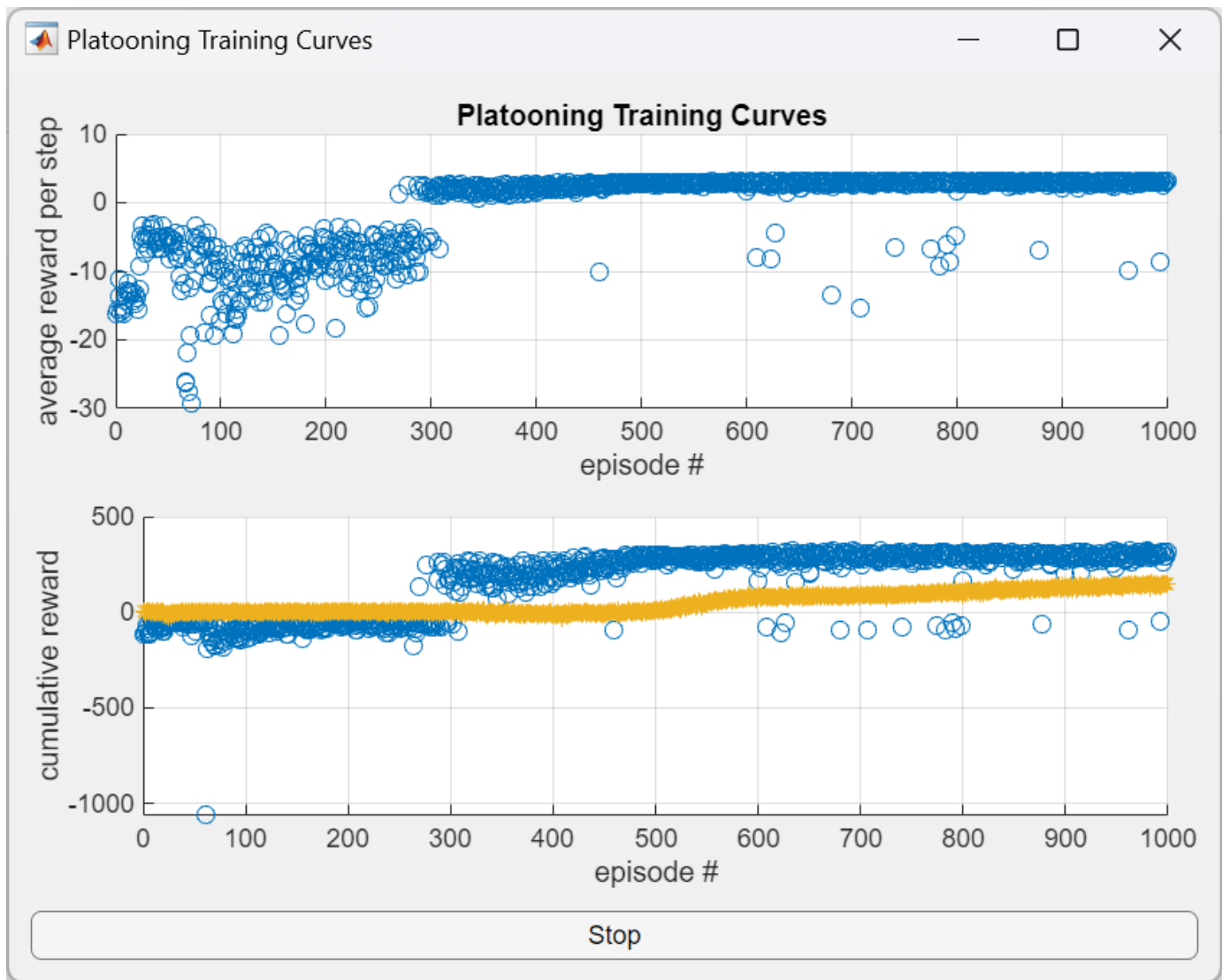
% Evaluate q0 from the initial episode observation.
obs0 = episodeInfo.InitialObservation;
q0 = evaluate(critic,[obs0,evaluate(actor,obs0)]);
q0 = double(q0{1});

% Update the plot.
update(plotObj,i,avgRwdPerStep,cumulativeRwd,q0);

% Exit training if button pushed.
if plotObj.StopTraining
    break;
end
end

% Clean up the environment.
cleanup(env);

% Save the policy.
save("PlatooningDDPGPolicy.mat","policy");
else
    % Load the pretrained policy.
    load("PlatooningDDPGPolicy.mat");
end
```



Validate Trained Policy

Validate the learned policy by running five simulations with random initial conditions specified by the reset function.

First, turn off parameter noise in the model.

```
UseParamNoise = 0;
```

Simulate the model against the trained policy five times.

```
N = 5;
simOpts = rlSimulationOptions( ...
    MaxSteps=maxSteps, ...
    NumSimulations=N);
experiences = sim(env,policy,simOpts);
```



Plot the vehicle spacing error, gains, and reward from the experiences output structure.

```
f = figure(Position=[100 100 1024 768]);
tiledlayout(f,N,3);
for i = 1:N

    % Get the spacing.
    tspacing = experiences(i).Observation.measurements.Time;
    spacing = ...
    squeeze(experiences(i).Observation.measurements.Data(1:4,:,:));

    % Get the gains.
    tgains = experiences(i).Action.control_gains.Time;
    gains = squeeze(experiences(i).Action.control_gains.Data);

    % Get the reward.
    trwd = experiences(i).Reward.Time;
    rwd = experiences(i).Reward.Data;

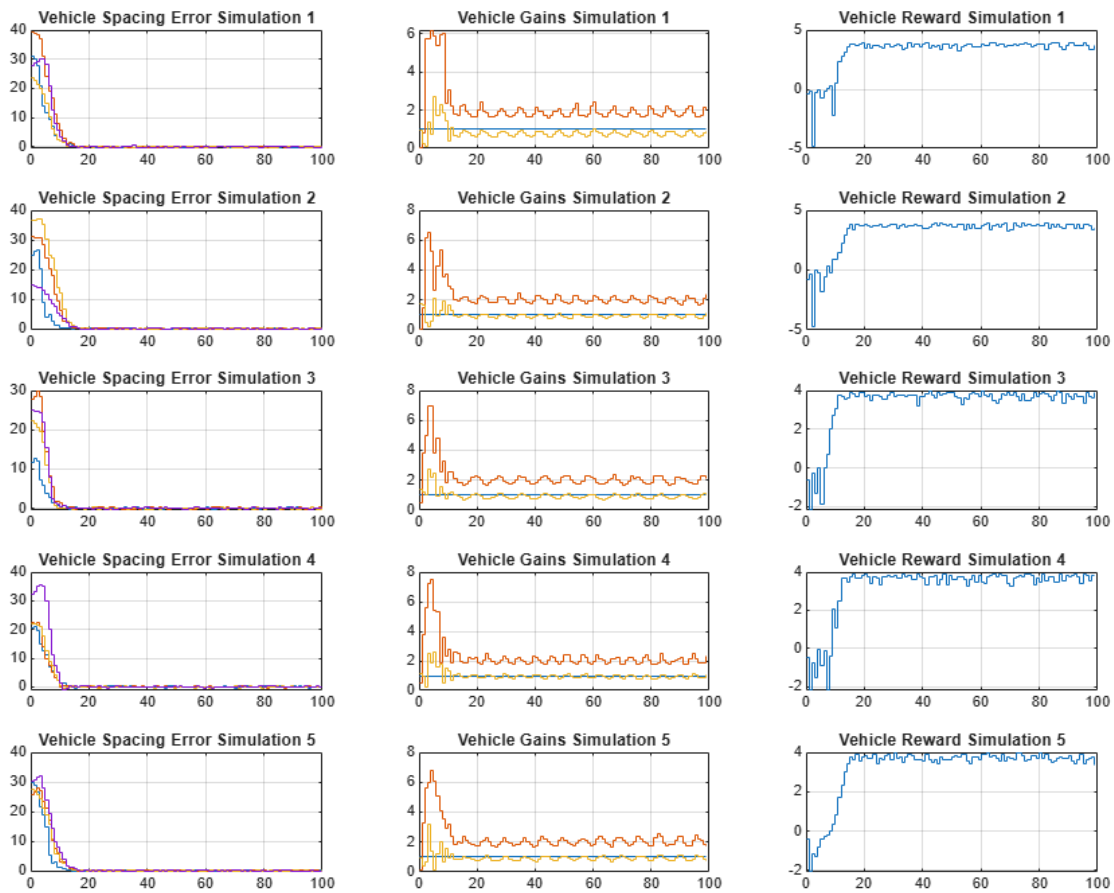
    % Plot the spacing.
    nexttile
    stairs(tspacing,spacing');
    title(sprintf("Vehicle Spacing Error Simulation %u",i))
    grid on
```

```

% plot the gains.
nexttile
stairs(tgains,gains');
title(sprintf("Vehicle Gains Simulation %u",i))
grid on

% Plot the reward.
nexttile
stairs(trwd,rwd);
title(sprintf("Vehicle Reward Simulation %u",i))
grid on
end

```



From the plots, you can see that the trained policy generates adaptive gains that adequately track the desired spacing for all vehicles.

Local Functions

The process experience function is called every time an experience is processed by the RL Agent block. Here, processExperienceFcn appends the experience to the replay memory, samples a mini-batch of experiences from the replay memory, and updates the critic, actor, and target networks.

```
function [policy,procExpData] = processExperienceFcn( ...
    exp,episodeInfo,policy,procExpData)

% Append experience to replay memory buffer.
append(procExpData.ReplayMemory,exp);

% Sample a mini-batch of experiences from replay memory.
% Return the mini-batch elements as darray objects so
% they can be treated as input to the automatically
% differentiated loss functions.
miniBatch = sample(procExpData.ReplayMemory, ...
    procExpData.MiniBatchSize, ...
    DiscountFactor=procExpData.DiscountFactor, ...
    ReturnDlarray=true);

if ~isempty(miniBatch)
    % Update network parameters using the mini-batch.
    procExpData = learnFcn(procExpData,miniBatch);

    % Update the policy parameters using the actor parameters.
    policy = setLearnableParameters( ...
        policy, ...
        getLearnableParameters(procExpData.Actor) ...
    );
end

end
```

The learnFcn function uses syncParameters to update the critic, actor, and target networks given a sampled mini-batch.

```
function processExpData = learnFcn(processExpData,miniBatch)

% Compute target next actions against the next observations.
nextAction = getAction( ...
    processExpData.TargetActor,miniBatch.NextObservation);

% compute qtarget = reward + gamma*Q(nextObservation,nextAction)
%               = reward + gamma*expectedFutureReturn
% Bootstrap the target at non-terminal experiences.
gamma = processExpData.DiscountFactor.*single(miniBatch.IsDone~=1);
expectedFutureReturn = getValue(processExpData.TargetCritic, ...
    miniBatch.NextObservation,nextAction);
targetq = miniBatch.Reward + gamma.*expectedFutureReturn;

% Compute critic gradient using deepCriticGradient function.
criticGradient = dlfeval( ...
    processExpData.AccelCriticGradFcn, ...
    processExpData.Critic, ...
    miniBatch.Observation, ...
    miniBatch.Action,targetq);
```



```

% Update the critic parameters.
[processExpData.Critic,processExpData.CriticOptimizer] = update( ...
    processExpData.CriticOptimizer,processExpData.Critic, ...
    criticGradient);

% Compute the actor gradient using the deepActorGradient function.
actorGradient = dlfeval( ...
    processExpData.AccelActorGradFcn, ...
    processExpData.Critic, ...
    processExpData.Actor, ...
    miniBatch.Observation);

% Update the actor parameters.
[processExpData.Actor,processExpData.ActorOptimizer] = update( ...
    processExpData.ActorOptimizer,processExpData.Actor, ...
    actorGradient);

% Update targets using the TargetSmoothFactor hyper-parameter.
processExpData.TargetCritic = syncParameters( ...
    processExpData.TargetCritic, ...
    processExpData.Critic, ...
    processExpData.TargetSmoothFactor);
processExpData.TargetActor = syncParameters( ...
    processExpData.TargetActor, ...
    processExpData.Actor, ...
    processExpData.TargetSmoothFactor);

end

```

The critic gradient is computed in the `deepCriticGradient` function. The loss is evaluated as the mean-squared-error (see `mse`) of the evaluated q-values and targets. The gradient is evaluated from the loss with respect to the critic learnable parameters.

```

function criticGradient = deepCriticGradient( ...
    critic,observation,action,targetq)

% Compute: q = Q(s,a). Set the UseForward name-value
% pair to true to support cases where the critic
% has layers that define a forward pass different than prediction
% (e.g. batch normalization or dropout layers).
q = getValue(critic,observation,action,UseForward=true);

% Loss is the half mean-square error of
% q = Q(observation,action) against target1
criticLoss = mse(q,reshape(targetq,size(q)),DataFormat="CB");

% compute the gradient of the loss
% with respect to the critic parameters
criticGradient = dlgradient(criticLoss,critic.Learnables);

end

```

The actor gradient is computed to maximize the expected value of an observation-action pair given the policy parameters. Here, the negative sign is used to maximize Q with respect to θ .

$$\frac{d}{d\theta} \left(-\frac{1}{N} \sum Q_{\phi}(\mathbf{s}, \mathbf{a}) \right) = \frac{d}{d\theta} \left(-\frac{1}{N} \sum Q_{\phi}(\mathbf{s}, \pi_{\theta}(\mathbf{s})) \right)$$

Here:

- s is the batch observations
- a is the batch actions
- Q_ϕ is the critic network parameterized by ϕ
- π_θ is the actor network parameterized by θ
- N is the mini batch size

```
function actorGradient = deepActorGradient(critic,actor,observation)

% Evaluate actions from current observations.
% Set the UseForward name-value pair to true
% to support cases where the actor has layers
% that define a forward pass different than prediction
% (e.g. batch normalization or dropout layers).
action = getAction(actor,observation,UseForward=true);

% Compute: q = Q(s,a)
q = getValue(critic,observation,action);

% Compute: actorLoss = -sum(q)/N to maximize q
actorLoss = -sum(q,"all")/numel(q);

% Compute: d(-sum(q)/N)/dActorParams
actorGradient = dlgradient(actorLoss,actor.Learnables);

end
```

The environment reset function varies the initial conditions, reference trajectory, and noise seeds for every episode. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = resetFunction(in)
% Perturb the nominal reference amplitude and frequency.
LeadA = max(2 + 0.1*randn,0.1);
LeadF = max(1 + 0.1*randn,0.1);

% Perturb the nominal spacing.
L = 22 + 3*randn;

% Perturb the initial states.
InitialPositions = [250 200 150 100 50] + 5*randn(1,5);
InitialVelocities = [10 10 10 10 10] + 1*randn(1,5);

% Update the noise seeds.
ParamNoiseSeed = randi(100,1,3);
AccelNoiseSeed = randi(100,1,5) + 100;
VelNoiseSeed = randi(100,1,5) + 200;
PosNoiseSeed = randi(100,1,5) + 300;

% Update the model variables.
in = setVariable(in,"L",L);
in = setVariable(in,"LeadA",LeadA);
in = setVariable(in,"LeadF",LeadF);
in = setVariable(in,"InitialPositions",InitialPositions);
in = setVariable(in,"InitialVelocities",InitialVelocities);
in = setVariable(in,"ParamNoiseSeed",ParamNoiseSeed);
```

```

in = setVariable(in,"AccelNoiseSeed",AccelNoiseSeed);
in = setVariable(in,"VelNoiseSeed",VelNoiseSeed);
in = setVariable(in,"PosNoiseSeed",PosNoiseSeed);
end

```

Create the critic and actor networks.

```

function [critic,actor] = createNetworks(obsInfo,actInfo)

% The actor and critic networks are initialized randomly.
% Ensure reproducibility of the section by fixing random generator seed.
rng(0);

% Number of neurons in hidden layers
hiddenLayerSize = 64;

% Extract dimensions of observation and action spaces.
numObs = prod(obsInfo.Dimension);
numAct = prod(actInfo.Dimension);

% Use a Q-value function critic. This critic takes the current
% observation and an action as inputs and returns a single
% scalar as output (the estimated discounted cumulative long-term
% reward given the action from the state corresponding to the
% current observation, and following the policy thereafter).

% To model the parameterized Q-value function within the critic,
% use a neural network with two input layers (one for the
% observation channel and the other for the action channel),
% and one output layer (which returns the scalar value).

% Create the critic network.
% Define each network path as an array of layer objects.
obsInput = featureInputLayer(numObs,Name=obsInfo.Name);
actInput = featureInputLayer(numAct,Name=actInfo.Name);
catPath = [
    concatenationLayer(1,2,Name="concat")
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(1,Name="q")
];

% Create dlnetwork object and add layers.
net = dlnetwork();
net = addLayers(net,obsInput);
net = addLayers(net,actInput);
net = addLayers(net,catPath);

% Connect layers.
net = connectLayers(net,obsInfo.Name,"concat/in1");
net = connectLayers(net,actInfo.Name,"concat/in2");

% Initialize network.
net = initialize(net);

% Create the critic object.

```

```
critic = rlQValueFunction(net,obsInfo,actInfo);

% Use a continuous deterministic actor.
% This actor learns a parameterized deterministic policy
% for a continuous action space. It takes the current
% observation as input and returns as output an action
% that is a deterministic function of the observation.
%
% To model the parameterized policy within the actor, use a
% neural network with one input layer (which receives the
% content of the environment observation channel)
% and one output layer (which returns the action to the
% environment action channel).

% Define scale and offset for the output layer
scale = (actInfo.UpperLimit - actInfo.LowerLimit)/2;
offset = actInfo.LowerLimit + scale;

% Create the actor network as an array of layer objects.
% Use tanhLayer to scale the signal to the (-1,1) range,
% and scalingLayer to scale the output to the action range.
obsPath = [
    featureInputLayer(numObs,Name=obsInfo.Name)
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer( ...
        numAct, ...
        WeightsInitializer=@(sz) 1e-4.*(rand(sz)-0.5))
    tanhLayer
    scalingLayer(Scale=scale, ...
        Offset=offset, ...
        Name=actInfo.Name)
];

% Create dlnetwork object and add layers.
net = dlnetwork();
net = addLayers(net,obsPath);

% Initialize network.
net = initialize(net);

% Create the actor object.
actor = rlContinuousDeterministicActor(net,obsInfo,actInfo);

end
```

References

[1] Rajamani, Rajesh. *Vehicle Dynamics and Control*. 2. ed. Mechanical Engineering Series. New York, NY Heidelberg: Springer, 2012.

See Also

Functions

runEpisode | setup | cleanup | gradient | rlOptimizer | rlSimulinkEnv

Objects

`rlReplayMemory` | `rlQValueFunction` | `rlDeterministicActorPolicy` |
`rlContinuousDeterministicActor` | `rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Design Controller for Vehicle Platooning” (Simulink Control Design)
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Custom PPO Training Loop with Random Network Distillation” on page 5-298
- “Create and Train Custom LQR Agent” on page 5-345
- “Create and Train Custom PG Agent” on page 5-334

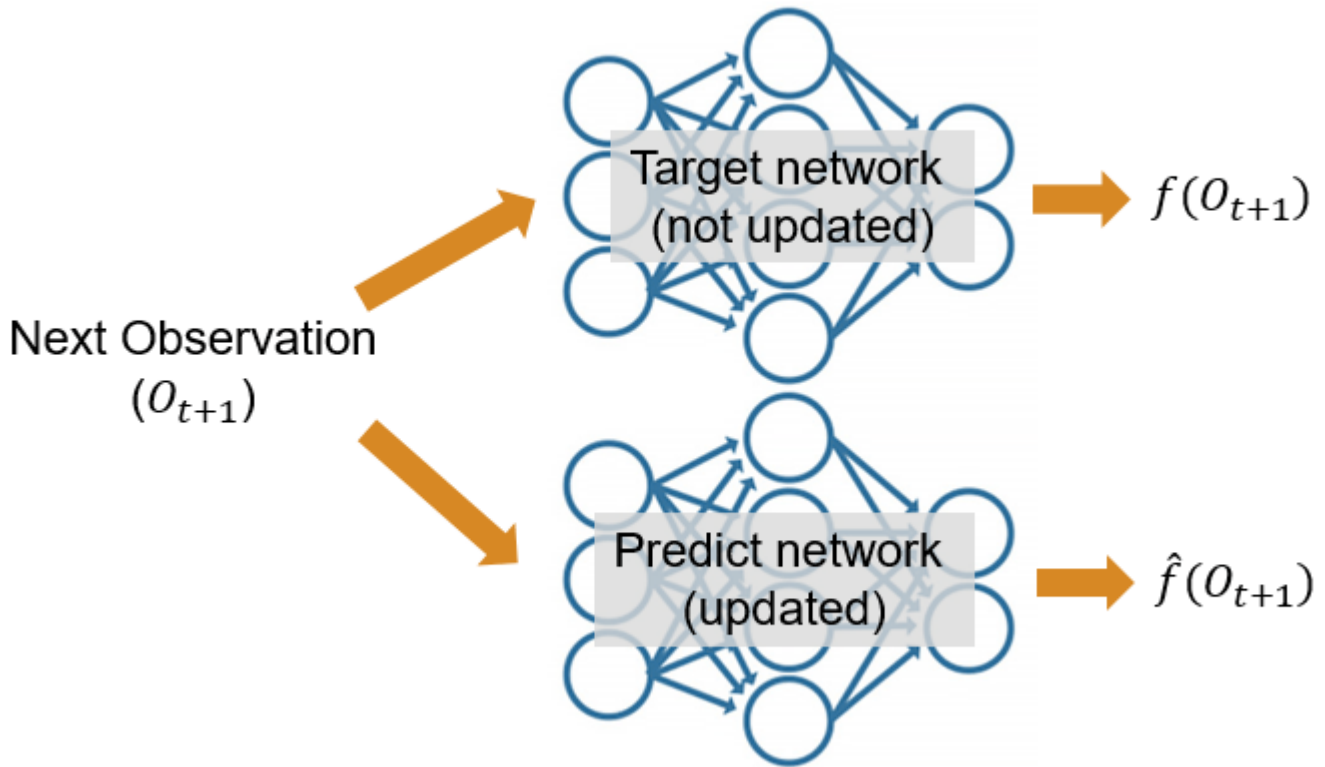
More About

- “Create Custom Reinforcement Learning Agents” on page 3-92
- “Train Reinforcement Learning Agents” on page 5-3

Custom PPO Training Loop with Random Network Distillation

This example shows how to train a custom proximal policy optimization (PPO) agent in a custom training loop using a random network distillation (RND) bonus [1]. In the example, you train the PPO agent to swing up and balance a pendulum environment with sparse rewards.

In environments with sparse rewards, it is helpful to use RND to create an exploration reward bonus that guides the agent toward new observations. The RND algorithm randomly initializes two neural networks: a target network that is not updated and a predict network that is updated. The parameters of the predict network (θ) are updated based on gradient descent by minimizing the expected mean squared error (L_{RND}) between $\hat{f}_\theta(O)$ and $f(O)$. Here $\hat{f}_\theta$ is the output of the predict network, f is the output of the target network, and O is the observation. This figure shows the networks with their input and outputs.



As a consequence of how its parameters are updated, over time, the predict network learns to reproduce the output of the target network.

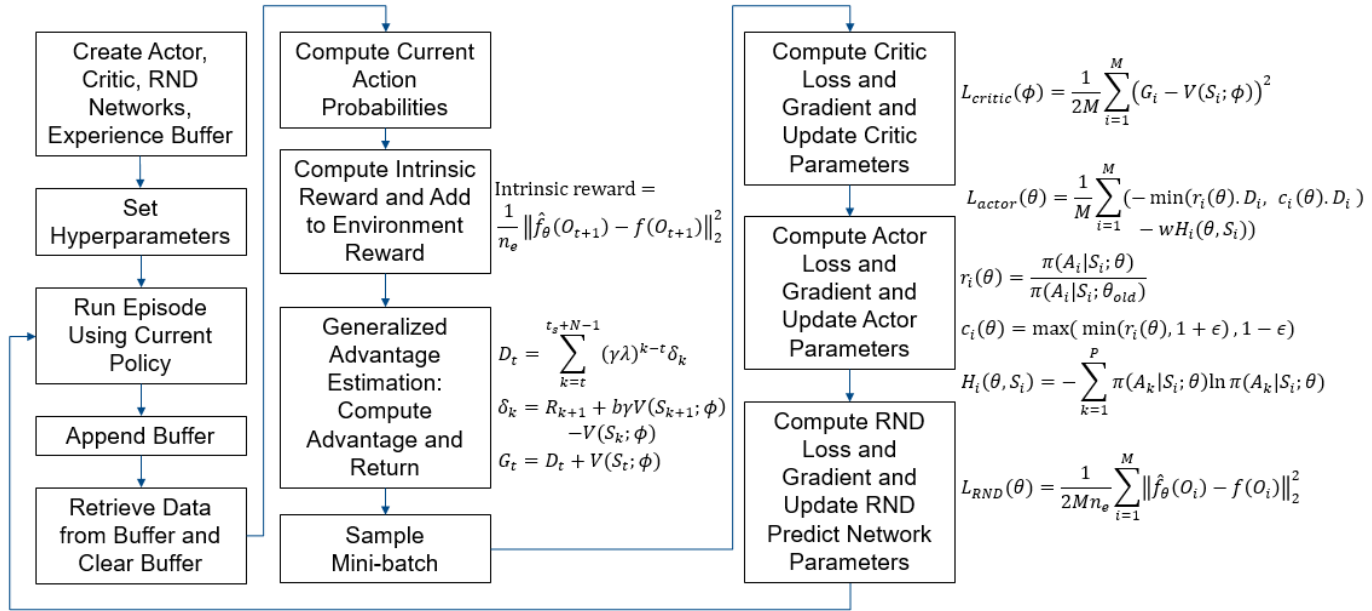
The agent uses the target and predict networks to compute the reward. Specifically, the total reward is equal to the reward from the environment plus the intrinsic reward, where the intrinsic reward is proportional to the squared norm of the difference between the two network outputs

$$\text{intrinsic reward} = \frac{1}{n_e} \|\hat{f}_\theta(O_{t+1}) - f(O_{t+1})\|_2^2$$

and where n_e is the embedding size.

For observations that the agent has seen before, the error and the resulting intrinsic reward are low, because the predict network was trained in that region of the observation space. Conversely, for new observations, the error is larger because the predict network was not trained with the observation, so the intrinsic reward is high. The higher intrinsic reward promotes exploration in new regions of the observation space, which helps in environments with sparse rewards.

This figure shows the overall process. For more information regarding the PPO training algorithm, refer to “Proximal Policy Optimization (PPO) Agent” on page 3-50.



Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

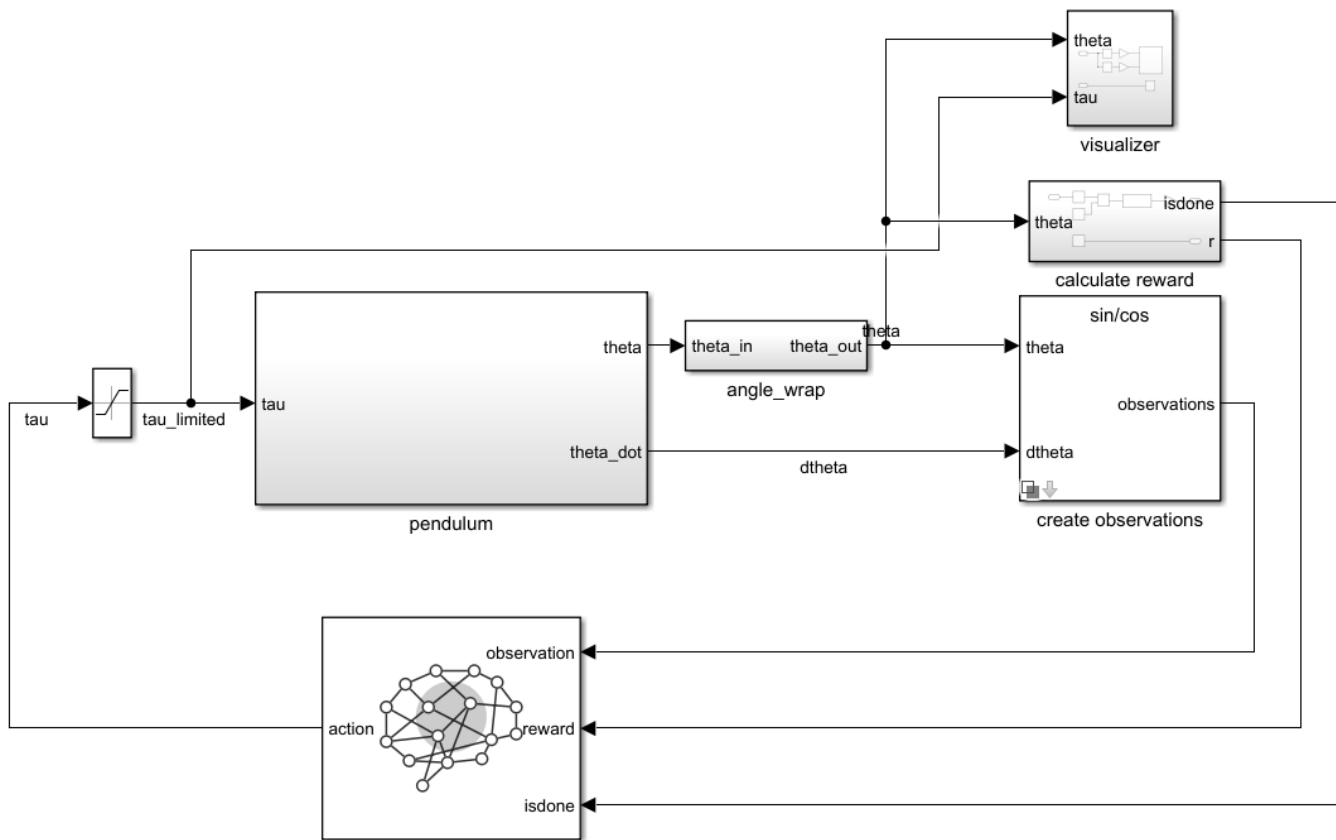
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Open Pendulum Swing-Up with Sparse Rewards Model

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright and balanced. The environment is modeled in Simulink®.

Open the model.

```
mdl = "rlSimplePendulumSparseRewardModel";
open_system(mdl);
```



For this model:

- The upward balanced pendulum position is 0 radians, and the downward hanging position is π radians.
- The torque action signal is discrete from the agent to the environment and can be -2, 0, or 2 N·m.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- The reward r_t , provided at every timestep, is

$$r_t = 1, \text{ if } |\theta_t| < \frac{\pi}{8} \text{ and } 0 \text{ otherwise.}$$

Here, θ_t is the angle (counterclockwise-positive) of displacement from the upright position. As you can see, the reward function used here is sparse to show the benefit of using RND in such environments.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Define the observation specification `obsInfo` and the action specification `actInfo`.

```
obsInfo = rlNumericSpec([3 1]);
obsInfo.Name = "Observations";
```



```
actInfo = rlFiniteSetSpec([-2 0 2]);
actInfo.Name = "Torque";
```

Use `rlSimulinkEnv` to create the environment object.

```
env = rlSimulinkEnv mdl, mdl + "/RL Agent", obsInfo, actInfo);
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function sets the model workspace variable `theta0` to `pi`. For more information on Simulink environments and their reset functions, see `SimulinkEnvWithAgent` and “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in)setVariable(in, "theta0", pi, "Workspace", mdl);
```

Specify the agent sample time T_s and the simulation time T_f in seconds. Define the maximum number of steps per episode.

```
Ts = 0.05;
Tf = 20;
```

Define the maximum number of steps per episode.

```
maxStepsPerEpisode = ceil(Tf/Ts);
```

Create Actor, Critic, and Experience Buffer

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister");
```

Create a critic network and an actor network. The `createCriticNet` on page 5-310 and `createActorNet` on page 5-311 functions are defined at the end of this example.

```
criticNet = createCriticNet(obsInfo);
actorNet = createActorNet(obsInfo, actInfo);
```

Create a critic and an actor. For more information, see `rlValueFunction` and `rlDiscreteCategoricalActor`. This example uses the structure `agentData` to store the critic, the actor, and the agent options and hyperparameters.

```
agentData.Critic = rlValueFunction(criticNet, obsInfo);
agentData.Actor = rlDiscreteCategoricalActor(actorNet, obsInfo, actInfo);
```

Use `rlOptimizer` to create optimizer objects for updating the actor and critic.

- Set the learning rate to $5e-3$.
- Set the gradient threshold to 5 to avoid large gradients.

```
optimizerOpt = rlOptimizerOptions( ...
    LearnRate = 5e-3, ...
    GradientThreshold = 5);
agentData.CriticOptimizer = rlOptimizer(optimizerOpt);
agentData.ActorOptimizer = rlOptimizer(optimizerOpt);
```

Set these agent learning options:

- Set the discount factor to 0.99.
- Use mini-batches of 256 experiences. Large batch sizes typically stabilize training.
- Set the experience horizon to the maximum steps per episode.
- Set MinMiniBatchesForLearning to 10 to collect at least 10*MiniBatchSize experiences before each agent update.

```
agentData.DiscountFactor = 0.99;  
agentData.MiniBatchSize = 256;  
agentData.NumEpoch = 3;  
agentData.ExperienceHorizon = maxStepsPerEpisode;  
agentData.MinMiniBatchesForLearning = 10;  
agentData.ActionInfo = actInfo;
```

Accelerate the loss functions to speed up the gradient computation.

```
agentData.AccelCriticGrad = dlaccelerate(@criticLoss);  
agentData.AccelActorGrad = dlaccelerate(@actorLoss);
```

Create a replay memory to store the experiences. For more information, see `rlReplayMemory`.

```
agentData.Replay = rlReplayMemory(obsInfo,actInfo, ...  
    (agentData.MinMiniBatchesForLearning+1)*agentData.MiniBatchSize);
```

Create RND Networks

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister");
```

Create two RND networks, a target network and a predict network. The `createRndNet` on page 5-311 function is defined at the end of this example.

```
% The algorithm does not update the target network.  
rndTargetNet = createRndNet(obsInfo);  
% The algorithm updates the predict network.  
rndPredictNet = createRndNet(obsInfo);
```

To store the options for the random networks, create the `rndOptions` structure. Set the RND network learning options:

- Set the learning rate to 0.001.
- Set the momentum of the optimizer for RND to 0.9.
- Set the weight for the intrinsic reward to 5.

```
rndOptions.LearnRate = 0.001;  
rndOptions.Momentum = 0.9;  
rndOptions.IntrinsicRewardWeight = 5;
```

```
agentData.RNDOptions = rndOptions;  
agentData.RNDTargetNet = rndTargetNet;  
agentData.RNDPredictNet = rndPredictNet;  
agentData.RNDVelocity = []; % Parameter for the SGDM optimizer  
agentData.RNDLoss = NaN;  
agentData.MeanIntrinsicReward = NaN;  
agentData.RNDNumOfLearnSteps = 0;
```

Set Up Training and Create Data Logger

Create a file data logger object using the `rlDataLogger` function. For more information, see `rlDataLogger`. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`. Change `agentData.UseRND` to `false` if you don't want to use RND.

```
doTraining = false ;
agentData.UseRND = true ;
useLogger = true ;
if useLogger && doTraining
    % Create a file data logger object.
    logger = rlDataLogger();
    % Open a Reinforcement Learning Data Viewer window
    % to visualize the logged data.
    rlDataViewer(logger);
    setup(logger);

    agentData.Logger = logger;
end

agentData.UseLogger = useLogger;

agentData.StopTrainingValue = 147;
agentData.AverageReward = 0;
agentData.ScoreAveragingWindowLength = 10;
```

Run Training Loop

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister");
```

To train the agent, the custom training loop simulates the agent in the environment for a maximum of `maxEpisodes` episodes.

```
maxEpisodes = 5000;
```

For this custom training loop:

- The `runEpisode` function simulates the agent in the environment for one episode.
- To speed up training, the `CleanupPostSim` option is set to `false` when calling `runEpisode`. Setting this option to `false` keeps the model compiled between episodes.
- After the training algorithm collects `MinMiniBatchesForLearning*MiniBatchSize` experiences over multiple episodes, the algorithm updates the agent and the predict RND network through the `stepAgent` on page 5-308 function defined at the end of this example.
- After all the episodes are complete, the `cleanup` function cleans up the environment. For more information, see `cleanup`.

```
% Store the cumulative rewards of all episodes.
```

```
rewardAllEpisodes = zeros(maxEpisodes,1);
```

```
% Reset the replay memory.
```

```
reset(agentData.Replay);

if doTraining
    % Create the policy object from the actor.
    policy = rlStochasticActorPolicy(agentData.Actor);

    % Set the sampleTime property to make sure that the RL Agent block
    % in the environment Simulink model runs at the desired sample time.
    policy.SampleTime = Ts;

    % Number of experiences to collect before updating the RL agent.
    numExperiencesToCollect = ...
        agentData.MinMiniBatchesForLearning*agentData.MinBatchSize;

    for episodeNumber = 1:maxEpisodes

        % Run an episode.
        out = runEpisode(env,policy, ...
            MaxSteps=maxStepsPerEpisode, ...
            CleanupPostSim=false);

        % Collect the experiences.
        experiences = out.AgentData.Experiences;

        % Append the experiences to the replay used for updating
        % the actor and critic.
        append(agentData.Replay,experiences);

        if agentData.Replay.Length >= numExperiencesToCollect
            % Learn when the agent has collected enough data.
            % The stepAgent function updates the actor, the critic,
            % and the RND predict network.
            agentData = stepAgent(agentData);

            % Create the policy from the actor.
            policy = rlStochasticActorPolicy(agentData.Actor);
            policy.SampleTime = Ts;

        end

        % Collect additional episode information.
        epsInfo = out.AgentData.EpisodeInfo;

        cumRew = epsInfo.CumulativeReward;
        obs0 = epsInfo.InitialObservation;
        v0 = getValue(agentData.Critic,obs0);

        % Store the episode cumulative reward.
        rewardAllEpisodes(episodeNumber) = cumRew;

        % Compute the average reward.
        if episodeNumber > agentData.ScoreAveragingWindowLength
            agentData.AverageReward = ...
                mean(rewardAllEpisodes ...
                    (episodeNumber- ...
                     agentData.ScoreAveragingWindowLength:episodeNumber));
        else
            agentData.AverageReward = ...
```

```

        mean(rewardAllEpisodes(1:episodeNumber));
    end

    % Print the results.
    str1 = "Episode Number" + num2str(episodeNumber) + ", ";
    str2 = "Cumulative Reward = " + num2str(cumRew) + ", ";
    str3 = "Mean intrinsic reward = " ...
        + num2str(agentData.MeanIntrinsicReward) + ", ";
    str4 = "V0 = " + num2str(v0) + "]\n";
    fprintf(str1 + str2 + str3 + str4);

    % Log data.
    if agentData.UseLogger
        store(agentData.Logger, ...
            "EpisodeReward", cumRew, episodeNumber);
        store(agentData.Logger, ...
            "AverageReward", agentData.AverageReward, episodeNumber);
    if agentData.UseRND
        store(agentData.Logger, ...
            "MeanIntrinsicReward", ...
            agentData.MeanIntrinsicReward, ...
            episodeNumber);
    end
    drawnow();
end

% Stop the training if the average reward is greater than
% or equal to the previously defined value.
if agentData.AverageReward >= agentData.StopTrainingValue
    break;
end
end

% Clean up the environment.
cleanup(env);

if agentData.UseRND
    save('trainedPolicyRND.mat', 'policy');
end
else

    % Load the trained policy (RND was used).
    load('trainedPolicyRND.mat');

end
end

```

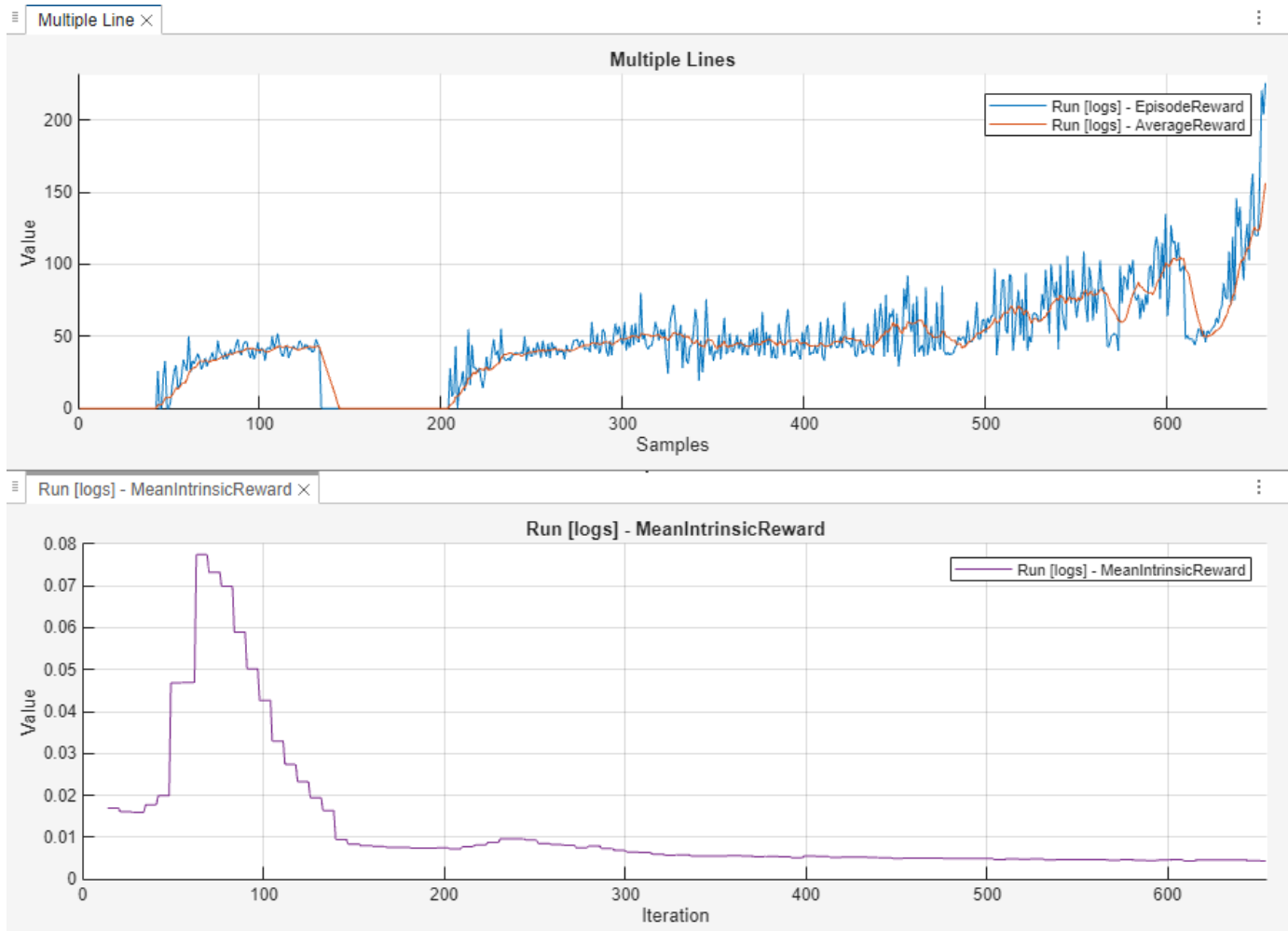
Analyze Training Results with and Without RND

To examine the usefulness of using an exploration reward bonus in environments with sparse rewards, run two training sessions. In the first session, train the custom PPO agent with RND. In the second session, do not use RND. The example creates the plots using the logged data and the Reinforcement Learning Data Viewer tool. See `rlDataViewer` for more information.

Training with RND

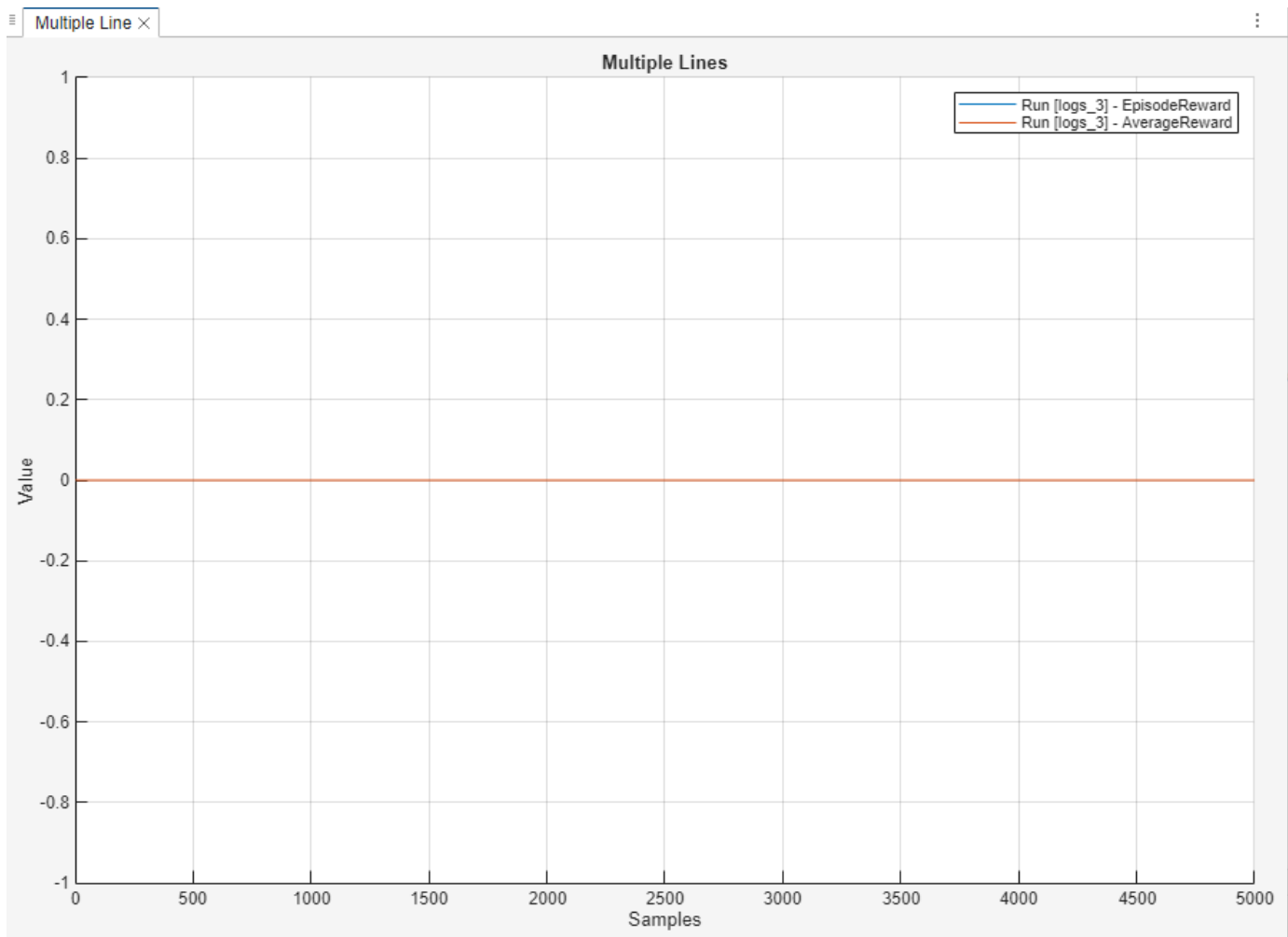
The following plots show the episode reward, average episode reward, and mean intrinsic reward. When the agent trains using RND, the reward is zero for the first 50 episodes but the intrinsic

exploration reward bonus is nonzero. This reward bonus encourages the agent to inject more energy in the pendulum and reach the upright sparse reward region of $|\theta_t| < \frac{\pi}{8}$. After 50 episodes the pendulum starts to reach the upright position. Then both the extrinsic and intrinsic rewards help the agent learn to swing-up and balance the pendulum.



Training Without RND

The following plot shows the episode reward and the average episode reward without using any exploration bonus. The agent does not learn to reach the upright position despite running the training for 5000 episodes. The rewards stay at the same value of zero, because the environment observation never reaches the sparse reward region of $(|\theta_t| < \frac{\pi}{8})$.



Simulate Trained Policy

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister");
```

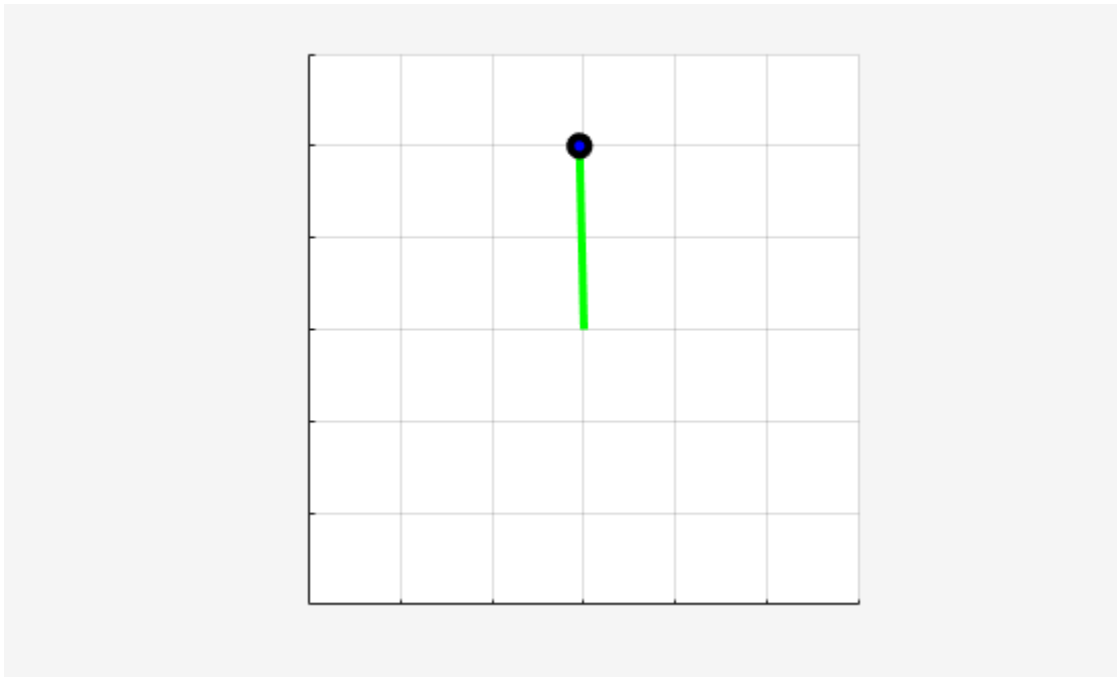
To simulate the trained agent, create a simulation options object and configure it to simulate the environment for 400 steps. For more information on agent simulation, see `rlSimulationOptions`.

```
simOptions = rlSimulationOptions("MaxSteps",maxStepsPerEpisode);
```

```
% Use the maximum likelihood action (no exploration).
policy.UseMaxLikelihoodAction = true;
```

Simulate the trained policy. For more information, see `sim`.

```
simOut = sim(env,policy,simOptions);
```



Displaying the cumulative reward obtained by the policy.

```
sum(simOut(1).Reward)
```

```
ans =  
281
```

The trained policy successfully learned to swing up and balance the pendulum.

Clean up the environment and the logger objects. This operation performs clean up tasks such as writing to file any data still in memory.

```
cleanup(env);  
if useLogger && doTraining  
    cleanup(logger);  
end
```

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

Agent Step Function

The stepAgent function updates the actor, the critic, and the RND predict network.

```
function agentData = stepAgent(agentData)  
  
    % Get the current critic object.  
    currentCritic = agentData.Critic;  
  
    % Generate the experience trajectory.  
    allExp = allExperiences(agentData.Replay, ...  
        ConcatenateMode="batch", ReturnDlarray=true);  
    cln1 = onCleanup(@() reset(agentData.Replay));
```



```

% Generate the action data for the current policy.
currentActPmf = evaluate(agentData.Actor, ...
    allExp.Observation,UseForward=false);

% Collect experience metrics.
allExpLen = numel(allExp.Reward);
eH = agentData.ExperienceHorizon;
mbLen = agentData.MiniBatchSize;

% Compute Intrinsic Reward.

% Ensure the agent was trained at least once.
if agentData.RNDNumOfLearnSteps>1
    % Obtain the intrinsic reward as defined
    % in the first section of the example.
    intrinsicReward = rndIntrinsicReward( ...
        agentData.RNDPredictNet, ...
        agentData.RNDTargetNet, allExp.NextObservation);

    % Calculate the average intrinsic reward.
    agentData.MeanIntrinsicReward = mean(intrinsicReward);

    % Weight the intrinsic reward and
    % add it to the environment reward.
    allExp.Reward = allExp.Reward + ...
        agentData.RNDOptions.IntrinsicRewardWeight* ...
        stripdims(intrinsicReward);
end

% Compute the advantage against the current critic.
[advantages,tdTargets] = gaeAdvantage(allExp,currentCritic, ...
    eH,agentData.DiscountFactor,0.95);

% Loop for the predefined number of epochs.
for epoch = 1:agentData.NumEpoch

    % Generate the mini-batch indices.
    subTrajIdx = randperm(allExpLen);
    startIdx2 = 1;

    for ii=1:ceil(allExpLen/mbLen)

        % Get the mini-batch indices.
        endIdx2 = min(allExpLen,startIdx2 + mbLen - 1);
        mbIdx = subTrajIdx(startIdx2:endIdx2);
        startIdx2 = endIdx2 + 1;

        % Get the mini-batch data.
        mbObs = {allExp.Observation{:}}(:, :, mbIdx);
        mbNobs = {allExp.NextObservation{:}}(:, :, mbIdx);
        mbAct = {allExp.Action{:}}(:, :, mbIdx);
        mbActPmf = {currentActPmf{:}}(:, mbIdx);
        mbTrg = tdTargets(:, mbIdx);
        mbAdv = advantages(:, mbIdx);

        % Calculate the critic gradient.
        [~,criticGradient] = ...

```

```
        dlfeval(agentData.AccelCriticGrad,agentData.Critic, ...
        mbObs,mbTrg);

    % Update the critic.
    [agentData.Critic,agentData.CriticOptimizer] = ...
        update(agentData.CriticOptimizer,agentData.Critic, ...
        criticGradient);

    % actIdx is a matrix containing a one-hot-vector in
    % each column that indicates which action is selected.
    actIdx = extractdata(squeeze(mbAct{:}))'== ...
        agentData.ActionInfo.Elements;

    % Calculate the actor gradient.
    [~,actorGradient] = ...
        dlfeval(agentData.AccelActorGrad,agentData.Actor, ...
        mbObs,mbActPmf,mbAdv,actIdx);

    % Update the actor.
    [agentData.Actor,agentData.ActorOptimizer] = ...
        update(agentData.ActorOptimizer,agentData.Actor, ...
        actorGradient);

    % If you use RND, update the RND networks.
    if agentData.UseRND
        % Calculate the RND gradient for the predict network.
        [agentData.RNDLoss,rndGradient] = dlfeval( ...
            dlaccelerate(@rndLoss), ...
            agentData.RNDPredictNet, ...
            mbNobs, ...
            agentData.RNDTargetNet);

        % Update the predict network parameters
        % using the SGDM optimizer.
        [agentData.RNDPredictNet,agentData.RNDVelocity] = ...
            sgdmupdate(agentData.RNDPredictNet, ...
            rndGradient, ...
            agentData.RNDVelocity, ...
            agentData.RNDOptions.LearnRate, ...
            agentData.RNDOptions.Momentum);

        agentData.RNDNumOfLearnSteps = ...
            agentData.RNDNumOfLearnSteps+1;
    end
end
end
end
```

Create Critic Network

The createCriticNet function creates a custom network for the critic.

```
function net = createCriticNet(obsInfo)

    numObs = obsInfo.Dimension(1);

    net = [
```

```

        featureInputLayer(numObs, Name="obs")
        fullyConnectedLayer(256)
        reluLayer()
        fullyConnectedLayer(256)
        reluLayer()
        fullyConnectedLayer(1, Name="fc_value")
    ];
    net = dlnetwork(net);

```

end

Create Actor Network

The createActorNet function creates a custom network for the actor.

```

function net = createActorNet(obsInfo, actInfo)

    numObs = obsInfo.Dimension(1);
    numAct = numel(actInfo.Elements);

    net = [
        featureInputLayer(numObs, Name="obs")
        fullyConnectedLayer(256)
        reluLayer()
        fullyConnectedLayer(256)
        reluLayer()
        fullyConnectedLayer(numAct, Name="fc_action")
        softmaxLayer(Name="output")
    ];

    net = dlnetwork(net);

```

end

Create RND Network

The createRndNet function creates a RND network.

```

function net = createRndNet(obsInfo)

    numObs = obsInfo.Dimension(1);
    netSize = [128 128];
    numOutputs = 64;

    net = [
        featureInputLayer(numObs, Name="nextobs")
        fullyConnectedLayer(netSize(1), Weights=2/sqrt(numObs)* ...
            (rand(netSize(1), numObs)-0.5), ...
            Bias=2/sqrt(numObs)*(rand(netSize(1), 1)-0.5))
        reluLayer()
        fullyConnectedLayer(netSize(2), Weights=2/sqrt(netSize(1))* ...
            (rand(netSize(2), netSize(1))-0.5), ...
            Bias=2/sqrt(netSize(1))*(rand(netSize(2), 1)-0.5))
        reluLayer()
        fullyConnectedLayer(numOutputs, Name="fc_act", Weights= ...
            2/sqrt(netSize(2))*(rand(numOutputs, netSize(2))-0.5), ...
            Bias=2/sqrt(netSize(2))*(rand(numOutputs, 1)-0.5))
    ];

```

```
net = dlnetwork(net);
```

```
end
```

Compute RND Loss

The `rndLoss` function computes loss and the gradient to update the RND predict network. This function uses the `rndSquaredError` on page 5-312 function defined below in this example.

```
function [loss,gradients] = rndLoss( ...  
    rndPredictNet,nextObs,rndTargetNet)  
  
    rndSquaredErrorValues = rndSquaredError( ...  
        rndPredictNet,rndTargetNet,nextObs,true);  
  
    loss = 0.5.*sum(rndSquaredErrorValues,"all") ...  
        /numel(rndSquaredErrorValues);  
    gradients = dlgradient(loss,rndPredictNet.Learnables);
```

```
end
```

Get Intrinsic Reward

The `rndIntrinsicReward` function computes the intrinsic reward by computing the error between the outputs of the target RND network and the predict RND network, given the observation. This function uses the `rndSquaredError` on page 5-312 function defined below in this example.

```
function intrinsicReward = rndIntrinsicReward( ...  
    rndPredictNet,rndTargetNet,nextObs)  
  
    rndSquaredErrorValues = rndSquaredError( ...  
        rndPredictNet,rndTargetNet,nextObs,false);  
  
    % Calculate the intrinsic reward as explained  
    % in the beginning of the example.  
    intrinsicReward = sum( ...  
        rndSquaredErrorValues,1)/size(rndSquaredErrorValues, ...  
        finddim(rndSquaredErrorValues,'C'));
```

```
end
```

Compute Squared RND Error

The `rndSquaredError` function computes the squared error by evaluating the RND predict and target networks using the next observation.

```
function rndSquaredErrorValues = rndSquaredError( ...  
    rndPredictNet,rndTargetNet,nextObs,useForward)  
  
    % Reshape and format the input data  
    % for featureInputLayer in dlnetwork object.  
    formattedObs = dlarray(squeeze(nextObs{:}),"CB");  
  
    % Limit the angle derivative when computing the intrinsic reward.  
    % Limiting the angle derivative prevents the agent from increasing  
    % the derivative to gain an exploration bonus that is not useful.  
    lowerLimit = -6.5;
```

```

upperLimit = 6.5;
formattedObs(3,:) = ...
    max(lowerLimit,min(formattedObs(3,:),upperLimit));

if useForward
    [prediction,state] = forward(rndPredictNet,formattedObs);
else
    [prediction,state] = predict(rndPredictNet,formattedObs);
end
[target,state] = predict(rndTargetNet,formattedObs);

% Compute the squared RND error.
rndSquaredErrorValues = (prediction - target).^2;

end

```

Critic Loss

The criticLoss function computes the loss and the gradient to update the critic.

```

function [loss,dLdP] = criticLoss(critic,obs,target)

    q = getValue(critic,obs,UseForward=true);
    loss = 0.5.*sum((q - target).^2)/numel(q);
    dLdP = dlgradient(loss,critic.Learnables);

end

```

Actor Loss

The actorLoss function computes the loss and the gradient to update the actor.

```

function [loss,dLdP] = actorLoss( ...
    actor,obs,currentActPmf,adv,actIdx)

    clipFactor = 0.2;
    entropyW = 0.01;

    % Calculate the new actor probabilities.
    actProbs = evaluate(actor,obs,UseForward=true);

    % Compute the policy ratio from the new policy
    % against the current policy.
    threshold_ = 1e-8; % using this as threshold
    % Round any values less than threshold_ to threshold_.
    policy_ = actProbs{1}(actIdx);
    policy_(policy_<threshold_) = threshold_;
    previousPolicy_ = currentActPmf{1}(actIdx);
    previousPolicy_(previousPolicy_<threshold_)=threshold_;
    % Calculate the policy ratio.
    policyRatio = exp( log(policy_) - log(previousPolicy_) );
    policyRatio = policyRatio';

    % Compute the (clipped) objective against the advantage.
    objective = adv.*policyRatio;
    clippedObjective = adv.*max(min(policyRatio,1 + clipFactor), ...
        1 - clipFactor);

```

```
% Compute the advantage loss.
advantageLoss = -sum(min(objective,clippedObjective),"all");

% Compute the entropy loss.
actProbs = actProbs{1};
actProbs(actProbs<1e-8) = 1e-8;
entropy_ = sum(-actProbs.*log(actProbs),1);
entropyLoss = -entropyW*sum(entropy_,"all");

% Sum the losses.
loss = (advantageLoss + entropyLoss)/numel(adv);

% Compute the gradients.
dLdP = dlgradient(loss,actor.Learnables);
```

end

Compute GAE Advantage

The `gaeAdvantage` function computes the GAE advantage. This function uses `gaeWeightMatrix` on page 5-314 and `onPolicyValueEstimate` on page 5-315, which are defined as local functions at the end of the example.

```
function [adv,tdTarget] = gaeAdvantage(trajs,critic,eH,gamma,gaeFactor)

% Find the end of the episode.
endOfEpisode = find(extractdata(trajs.IsDone > 0));

% Calculate the discount weights.
G = gaeWeightMatrix( ...
    numel(trajs.Reward),eH,gamma*gaeFactor,endOfEpisode);

sz = size(trajs.Reward);

% Estimate v and v'.
[v1,v2] = onPolicyValueEstimate(trajs,critic);

% Compute the TD error.
tdErr = trajs.Reward + gamma*v2 - v1;

% Compute the advantage.
adv = reshape(tdErr(:)'*G,sz);

% Compute the TD target.
tdTarget = adv + v1;
```

end

GAE Weight Matrix

The `gaeWeightMatrix` function calculates the discount weights.

```
function G = gaeWeightMatrix(n,eH,g,endOfEpisode)

% Treat the last element as endOfEpisode.
endOfEpisode = unique([endOfEpisode(:);n]);

% Create the full episode nstep weight matrices.
```

```

P = repmat((0:n-1)',1,n) - (0:n-1);
G = tril(g.^P);

% Decouple each experience horizon trajectory.
m = numel(endOfEpisode);
horizonStart = 1;
for i = 1:m
    eoe = endOfEpisode(i);
    while horizonStart <= eoe
        % Find the horizon end (it can be the episode end).
        horizonEnd = min(horizonStart + eH - 1,eoe);
        % Get the indices of all trajectories not part of
        % this trajectory.
        trajThis = horizonStart:horizonEnd;
        trajOthers = [(1:(horizonStart-1)),((horizonEnd+1):n)];
        % Decouple only the rows, because G is lower triangular.
        G(trajOthers,trajThis) = 0;
        horizonStart = horizonEnd + 1;
    end
end
end

```

On-Policy Value Estimate

The `onPolicyValueEstimate` function obtains the estimated value from a critic. This function is used by the `gaeAdvantage` on page 5-314 function to calculate the TD error.

```

function [v1,v2] = onPolicyValueEstimate(trajs,critic)

    isd = trajs.IsDone;

    sz = size(trajs.Reward);

    v1 = reshape(getValue(critic,trajs.Observation),sz);
    v2 = reshape(getValue(critic,trajs.NextObservation),sz);

    v2(isd == 1) = 0;

end

```

References

[1] Burda, Yuri, Harrison Edwards, Amos Storkey, and Oleg Klimov. "Exploration by Random Network Distillation." arXiv, October 30, 2018. <https://arxiv.org/abs/1810.12894>.

See Also

Functions

`dlaccelerate` | `dlgradient` | `dlfeval` | `evaluate` | `rlReplayMemory` | `rlOptimizer` | `runEpisode` | `getLearnableParameters` | `setLearnableParameters` | `rlDataLogger`

Objects

`rlDiscreteCategoricalActor` | `rlOptimizerOptions`

Related Examples

- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Custom Training Loop with Simulink Action Noise” on page 5-282
- “Create and Train Custom PG Agent” on page 5-334

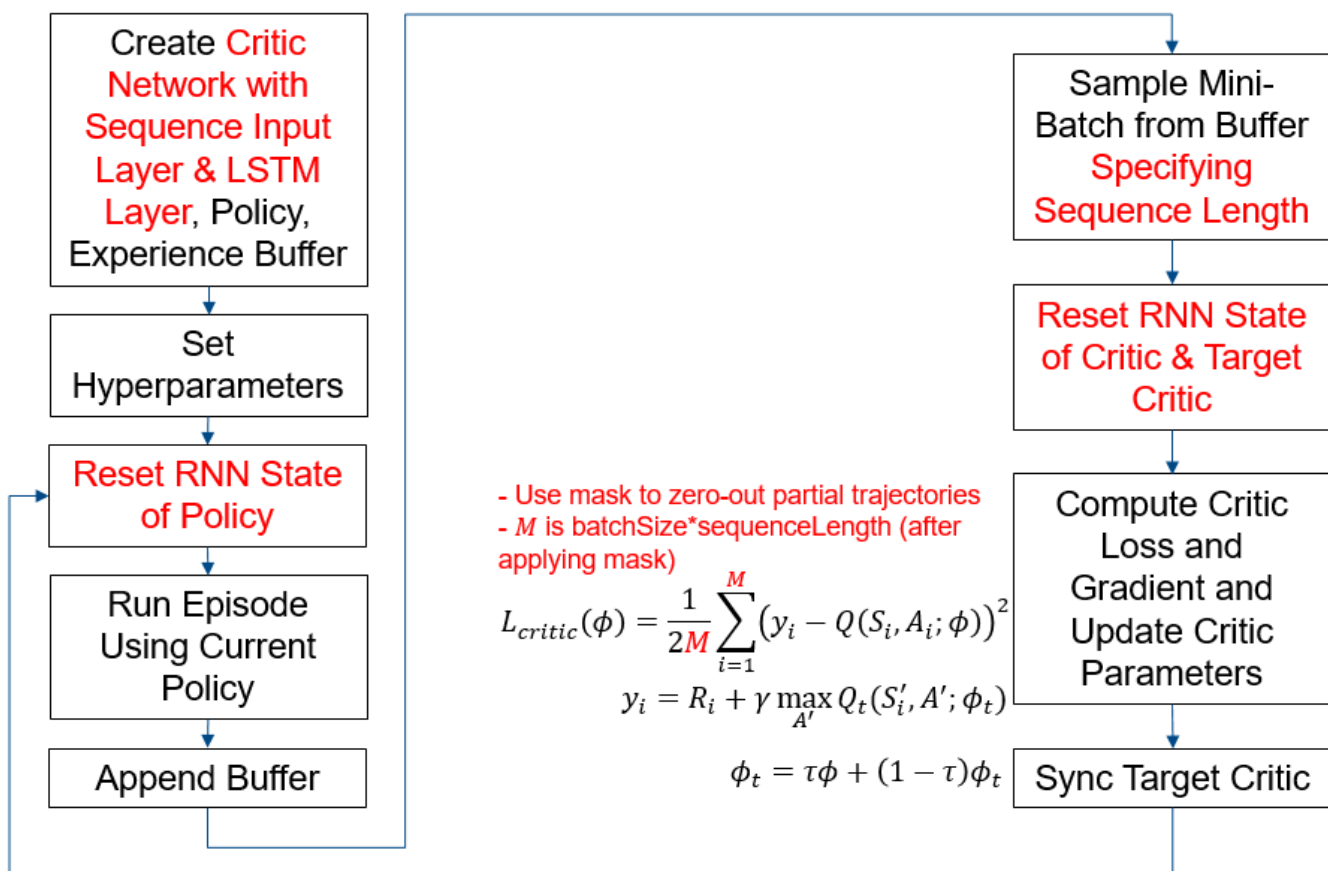
More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Reinforcement Learning Agents” on page 3-92
- “Use Predefined Control System Environments” on page 2-42

Custom DQN Training Loop with LSTM Network

This example shows how to use a custom training loop to train a deep Q-learning network (DQN) agent with a Long Short-Term Memory (LSTM) network. Specifically, in this example, you train the agent to control a house heating system modeled in Simscape™. For an example in which you train a default DQN agent with an LSTM network to control a house heating system, “Train DQN Agent with LSTM Network to Control House Heating System” on page 8-299. For more information on the DQN training algorithm, see “Deep Q-Network (DQN) Agent” on page 3-32.

The figure below summarizes the workflow for training the agent. The red text highlights steps that are specific to LSTM networks.



Preserve Random Number Sequence for Reproducibility

Some sections in this example require random number computations. Specify the seed and random number generator algorithm at the beginning of a section to use the same random number sequence each time you run it. Preserving the random number sequence allows you to reproduce the results of the section. For more information, see “Results Reproducibility” on page 5-10.

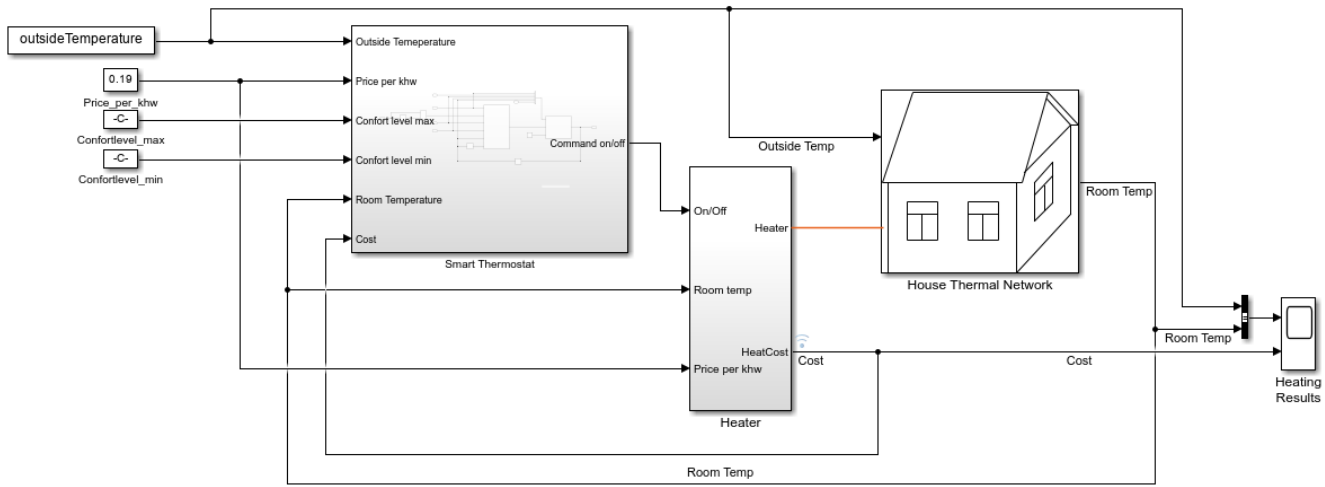
Set the random number seed to 0 and the algorithm to Mersenne Twister. For more information, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the sequence. You will restore the state at the end of the example.

House Heating Simulink Model

The reinforcement learning (RL) environment for this example uses the Simulink® model described in the “House Heating System” (Simscape) example. The model contains a heater block, a thermostat block (which contains an RL agent block and a reward block) and the House Thermal Network block (modeled in Simscape). Heat moves between the interior of the home and the outside environment through the walls, windows, and roof. You use weather station data between March 21 and April 15, 2022, to simulate the outside temperature. For more information about the data acquisition, see “Compare Temperature Data from Three Different Days” (ThingSpeak).



The training goal for the custom training loop is to minimize the energy cost and maximize the amount of time the house is in the comfortable temperature range by turning on/off the heater. The house is comfortable when its interior temperature T_{room} is between $T_{\text{comfortMin}}$ and $T_{\text{comfortMax}}$.

The reinforcement learning problem is defined as follows:

- The observation is a 6-dimensional column vector that consists of the room temperature ($^{\circ}\text{C}$), outside temperature ($^{\circ}\text{C}$), maximum comfort temperature ($^{\circ}\text{C}$), minimum comfort temperature ($^{\circ}\text{C}$), last action, and price per kWh (USD).
- The action is discrete. Specifically, the action 0 represents turning the heater off, and the action 1 represents turning it on.
- The reward block calculates the total reward R as the sum of the comfort reward R_c and a penalty for switching P_s , from which an energy cost C_e is subtracted: $R = R_c + P_s - C_e$. These three components are defined as follows.

$$R_c = \begin{cases} 0.1 & \text{if } T_{\text{comfortMin}} \leq T_{\text{room}} \leq T_{\text{comfortMax}} \\ -w|T_{\text{room}} - T_{\text{comfortMin}}| & \text{if } T_{\text{room}} < T_{\text{comfortMin}} \\ -w|T_{\text{room}} - T_{\text{comfortMax}}| & \text{if } T_{\text{room}} > T_{\text{comfortMax}} \end{cases}$$

where $w = 0.1$, $T_{\text{comfortMin}} = 18$, and $T_{\text{comfortMax}} = 23$.

$$P_s = \begin{cases} -0.01 & \text{if } a_t \neq a_{t-1} \text{ where } a_t \text{ is the current action and } a_{t-1} \text{ is the previous action} \\ 0 & \text{otherwise} \end{cases}$$

$$C_e = \text{CostPerStep} = \text{PricePerKwh} * \text{ElectricityUsed}$$

The reward function is inspired by [1].

- The Is-Done signal supplied to the RL Agent block is always 0, indicating that there is no early termination condition.

Define Simulink Environment

Open the model.

```
mdl = "rlHouseHeatingSystemDQNLSTMCustomLoop";
open_system(mdl)
```

Define the sample time and the maximum number of steps per episode. You need these variables for the script and reset functions.

```
sampleTime = 120; % seconds
maxStepsPerEpisode = 1000;
```

Assign the agent block path information to the `agentBlk` variable, for later use.

```
agentBlk = mdl + "/Smart Thermostat/RL Agent";
```

Load the outside temperature data to simulate the environment temperature.

```
data = load('temperatureMar21toApr15_2022CustomLoop.mat');
temperatureData = data.temperatureData;
```

Extract validation data.

```
temperatureMarch21 = temperatureData(1:60*24,:);
temperatureApril15 = temperatureData(end-60*24+1:end,:);
```

Extract training data.

```
temperatureData = temperatureData(60*24+1:end-60*24,:);
```

The Simulink model loads the outside temperature history, the maximum comfort temperature, and the minimum comfort temperature. These variables are part of the observation vector.

```
outsideTemperature = temperatureData;
comfortMax = 23;
comfortMin = 18;
```

Define the observation and action specifications.

```
obsInfo = rlNumericSpec([6,1]);
actInfo = rlFiniteSetSpec([0,1]); % (0=off,1=on)
```

Create an environment object for the house heating environment.

```
env = rlSimulinkEnv(mdl,agentBlk,obsInfo,actInfo);
```

Specify a custom reset function. Use `hRLHeatingSystemCustomLoopResetFcn` to reset the environment at the beginning of each episode. This function randomly selects a time between March

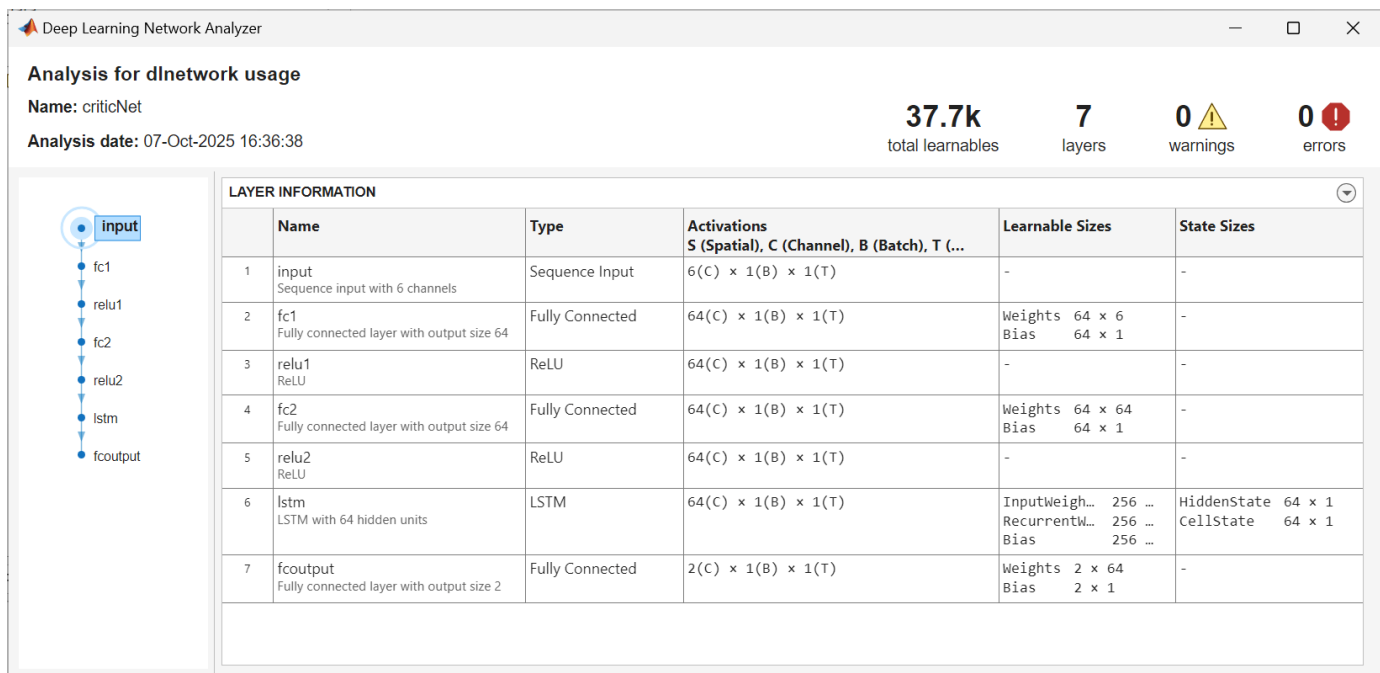
22 and April 14 such that at least `maxStepsPerEpisode` samples are available. The environment uses this time as the initial time for the outside temperatures. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in) hRLHeatingSystemCustomLoopResetFcn(in,...
    temperatureData,maxStepsPerEpisode,sampleTime);
```

Create Critic, Policy, and Experience Buffer

Create a critic network. The `createCriticNet` on page 5-328 function is defined at the end of this example. Note that a `sequenceInputLayer` is used in conjunction with LSTM layers to represent a RNN (recurrent neural network). For more information, see `sequenceInputLayer`.

```
criticNet = createCriticNet(obsInfo,actInfo);
analyzeNetwork(criticNet);
```



Create a critic and target critic. For more information, see `rlVectorQValueFunction`.

```
agentData.Critic = rlVectorQValueFunction(criticNet,obsInfo,actInfo);
agentData.TargetCritic = agentData.Critic;
```

To speed up the gradient computation, use `dlaccelerate` to accelerate the loss function. The `criticLossFcn` on page 5-329 function is defined at the end of this example.

```
agentData.AccelCriticGrad = dlaccelerate(@criticLossFcn);
```

Create an epsilon-greedy policy object. For more information, see `rlEpsilonGreedyPolicy`.

```
policy = rlEpsilonGreedyPolicy(agentData.Critic);
policy.UseEpsilonGreedyAction = true;
policy.SampleTime = sampleTime;
policy.ExplorationOptions.Epsilon = 1;
policy.ExplorationOptions.EpsilonDecay = 1e-4;
policy.ExplorationOptions.EpsilonMin = 0.01;
```

Use `rlOptimizer` to create optimizer objects for updating the critic.

- Set the learning rate to `1e-3`.
- Set the gradient threshold to 1 to avoid large gradients.

```
optimizerOpt = rlOptimizerOptions(LearnRate=1e-3, GradientThreshold=1);
agentData.CriticOptimizer = rlOptimizer(optimizerOpt);
```

Create Experience Buffer

Create an experience buffer for the agent with a maximum length of `1e6`. For more information, see `rlReplayMemory`.

```
replayMemorySize = 1e6;
agentData.Replay = rlReplayMemory(obsInfo, actInfo, replayMemorySize);
```

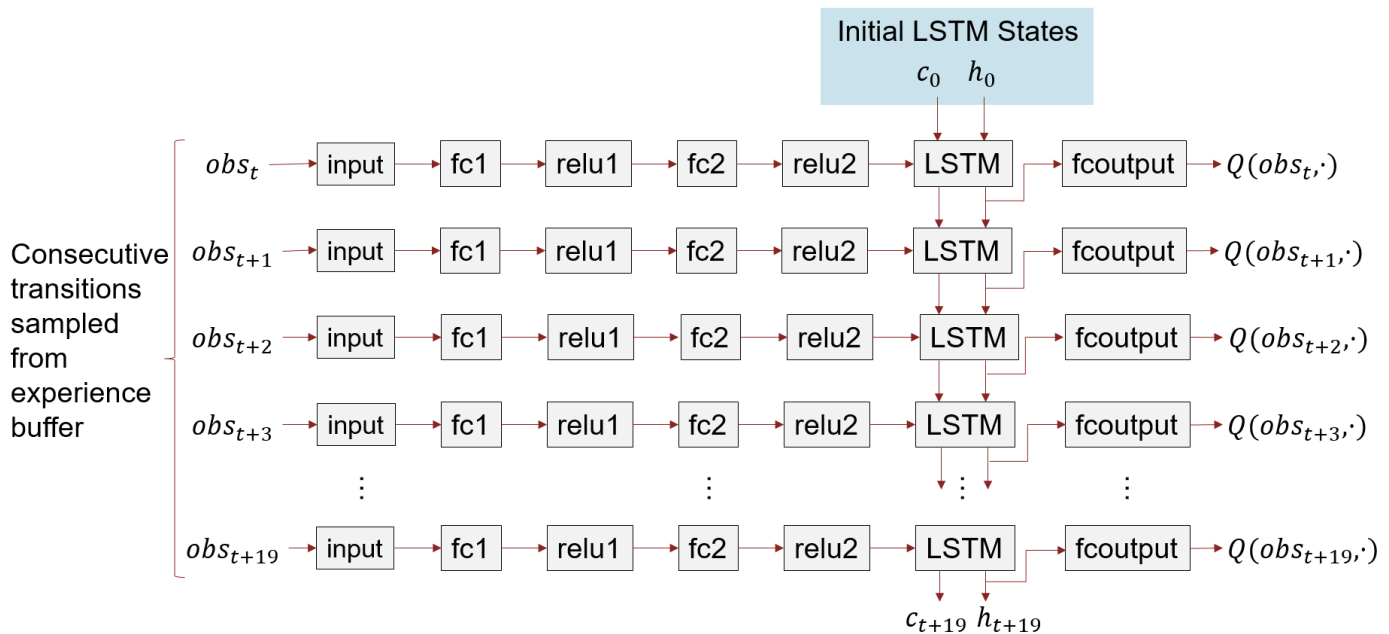
When using a recurrent neural network, you must set `SequenceLength` to a value greater than 1. This value is used in training to determine the length of the minibatch used to calculate the gradient. For more information about LSTM layers, see “Long Short-Term Memory Neural Networks”.

For this example, set `SequenceLength` to 20.

```
agentData.SequenceLength = 20;
```

The figure below shows an LSTM network unrolled for batch size of 1 and sequence length of 20.

LSTM (unrolled): batch size 1, sequence length 20



Create a structure to store additional data required for training the critic.

```
agentData.MaxEpisodes = 500;
agentData.DiscountFactor = 0.99;
agentData.MinibatchSize = 64;
agentData.MaxMinibatchPerEpoch = 100;
```

```
agentData.NumEpochs = 1;  
agentData.TargetSmoothFactor = 1e-3;
```

Set Up Training and Create Data Logger

Create a file data logger object using the `rlDataLogger` function. Open the Reinforcement Learning Data Viewer tool to visualize the logged data during training. For more information, see `rlDataLogger` and `rlDataViewer`.

To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;  
useLogger =  ;  
if useLogger && doTraining  
    logger = rlDataLogger();  
    rlDataViewer(logger);  
    setup(logger);  
end  
  
agentData.StopTrainingValue = 85;  
agentData.AverageReward = 0;  
agentData.ScoreAveragingWindowLength = 5;
```

Run Training Loop

To reproduce the results of this section, set random seed and algorithm.

```
rng(0, "twister");
```

Execute the custom training loop. In this example:

- The `runEpisode` function simulates the agent in the environment for one episode.
- To speed up training, set the `CleanupPostSim` option to `false` when calling `runEpisode`. This setting keeps the model compiled between episodes.
- Once the experience buffer contains at least `agentData.MinibatchSize` samples, the algorithm updates the agent at the end of each episode using the `updateAgent` on page 5-330 function defined at the end of this example.
- After all of the episodes are complete, the `cleanup` function cleans up the environment.

```
% Create variable to store the cumulative rewards of all episodes.  
rewardAllEpisodes = zeros(agentData.MaxEpisodes,1);
```

```
% Evaluate the agent every 25 training episodes.  
evaluatorFrequency = 25;
```

```
if doTraining  
    for episodeIdx = 1:agentData.MaxEpisodes  
  
        agentData.EpisodeIdx = episodeIdx;  
  
        % Reset RNN state of policy to zeros.  
        policy.QValueFunction.State = ...  
            cellfun(@(x) zeros(size(x), 'like', x), ...
```

```

        policy.QValueFunction.State, 'UniformOutput', false);

% Run an episode to simulate the environment with the policy.
out = runEpisode(env,policy,...
    MaxSteps=maxStepsPerEpisode,CleanupPostSim=false);

% Append experience to replay memory buffer.
append(agentData.Replay,out.AgentData.Experiences);

% Extract episode information to update the training curves.
episodeInfo = out.AgentData.EpisodeInfo;

% Extract the policy for the next episode.
policy = out.AgentData.Agent;

% Extract the cumulative reward.
cumulativeRwd = episodeInfo.CumulativeReward;
rewardAllEpisodes(episodeIdx) = cumulativeRwd;

if agentData.Replay.Length>=agentData.MinibatchSize
    % Learn when the agent has collected enough data.
    % The updateAgent function updates the critic
    % and the target critic networks.
    agentData = updateAgent(agentData);

    % Update the policy parameters using the critic parameters.
    policy = setLearnableParameters( ...
        policy,getLearnableParameters(agentData.Critic));
end

% Compute average reward.
if episodeIdx>agentData.ScoreAveragingWindowLength
    averageReward = mean(rewardAllEpisodes(episodeIdx-...
        agentData.ScoreAveragingWindowLength+1:episodeIdx));
else
    averageReward = mean(rewardAllEpisodes(1:episodeIdx));
end

% Log data.
if useLogger
    store(logger, "EpisodeReward", cumulativeRwd, episodeIdx);
    store(logger, "AverageReward", averageReward, episodeIdx);
    drawnow();
    write(logger);
end

% Evaluate the policy every evaluatorFrequency episodes.
if mod(episodeIdx,evaluatorFrequency)==0
    % Reset RNN state of policy to zeros.
    policy.QValueFunction.State = cellfun( ...
        @(x) zeros(size(x), 'like', x), ...
        policy.QValueFunction.State, 'UniformOutput', false);

    % Evaluate the trained policy by turning off exploration.
    policy.UseEpsilonGreedyAction = 0;
    numSimulations = 3;
    simOptions = rlSimulationOptions("MaxSteps",...
        maxStepsPerEpisode,"NumSimulations",numSimulations);

```

```
simOut = sim(env,policy,simOptions);
simCumReward = zeros(numSimulations,1);
for ii=1:numSimulations
    simCumReward(ii) = sum(simOut(ii).Reward);
end

% Exit the training loop if the cumulative reward exceeds
% StopTrainingValue.
if mean(simCumReward)>=agentData.StopTrainingValue
    break;
end

% Turn on exploration for the next episode.
policy.UseEpsilonGreedyAction = 1;
end

% Log the evaluation statistics.
if useLogger
    if mod(episodeIdx,evaluatorFrequency)==0
        store(logger,"EvaluationStatistic", ...
            mean(simCumReward),episodeIdx);
    else
        store(logger,"EvaluationStatistic",NaN,episodeIdx);
    end
end

end

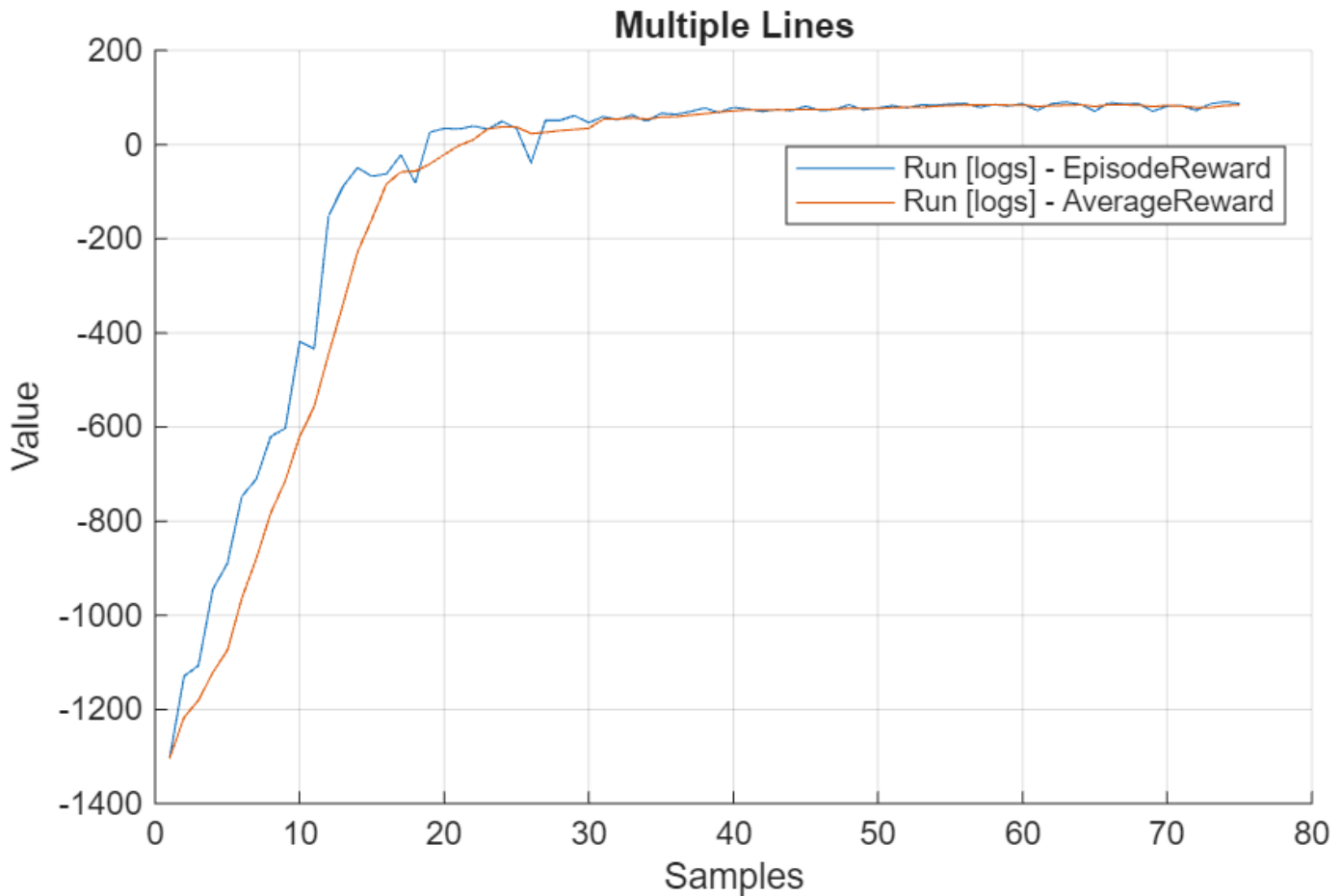
% Clean up the environment.
cleanup(env);

% Save the trained critic.
trainedCritic = agentData.Critic;
save("trainedAgentLSTMCustomLoop.mat","trainedCritic");
else
    % If you choose to skip the training loop,
    % load the trained critic.
    load('trainedAgentLSTMCustomLoop.mat');

    % Create epsilon-greedy policy from the trained critic
    % for simulation.
    policy = rlEpsilonGreedyPolicy(trainedCritic);
    policy.UseEpsilonGreedyAction = 0;
    policy.SampleTime = sampleTime;
end

% cleanup
if useLogger && doTraining
    cleanup(logger);
end
```

The plot shows the reward obtained by the agent for each episode and the average reward over `agentData.ScoreAveragingWindowLength`.



Simulate the Trained Policy

To validate the performance of the trained policy, simulate it within the house heating environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

To reproduce the results of this section, set random seed and algorithm.

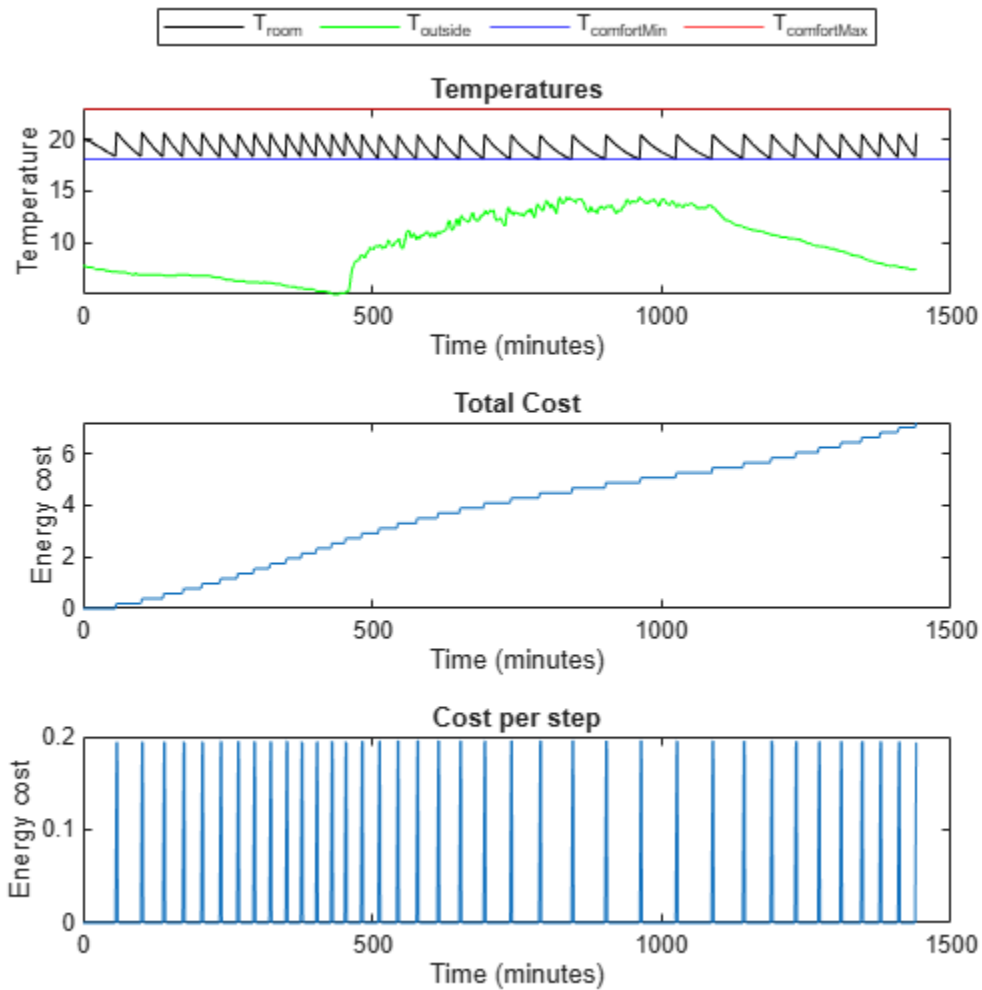
```
rng(0, "twister");
```

Evaluate the performance of the trained policy using the validation temperature data from March 21, 2022, which was not used for training. To supply this data set, use the environment reset function `hRLHeatingSystemValidateCustomLoopResetFcn`.

```
maxSteps= 720;
validationTemperature = temperatureMarch21;
env.ResetFcn = @(in) hRLHeatingSystemValidateCustomLoopResetFcn( ...
    in,validationTemperature);
simOptions = rlSimulationOptions(MaxSteps = maxSteps);
experience1 = sim(env,policy,simOptions);
```

Use the `plotResults` on page 5-332 function, provided at the end of the example, to visualize the performance of the trained policy.

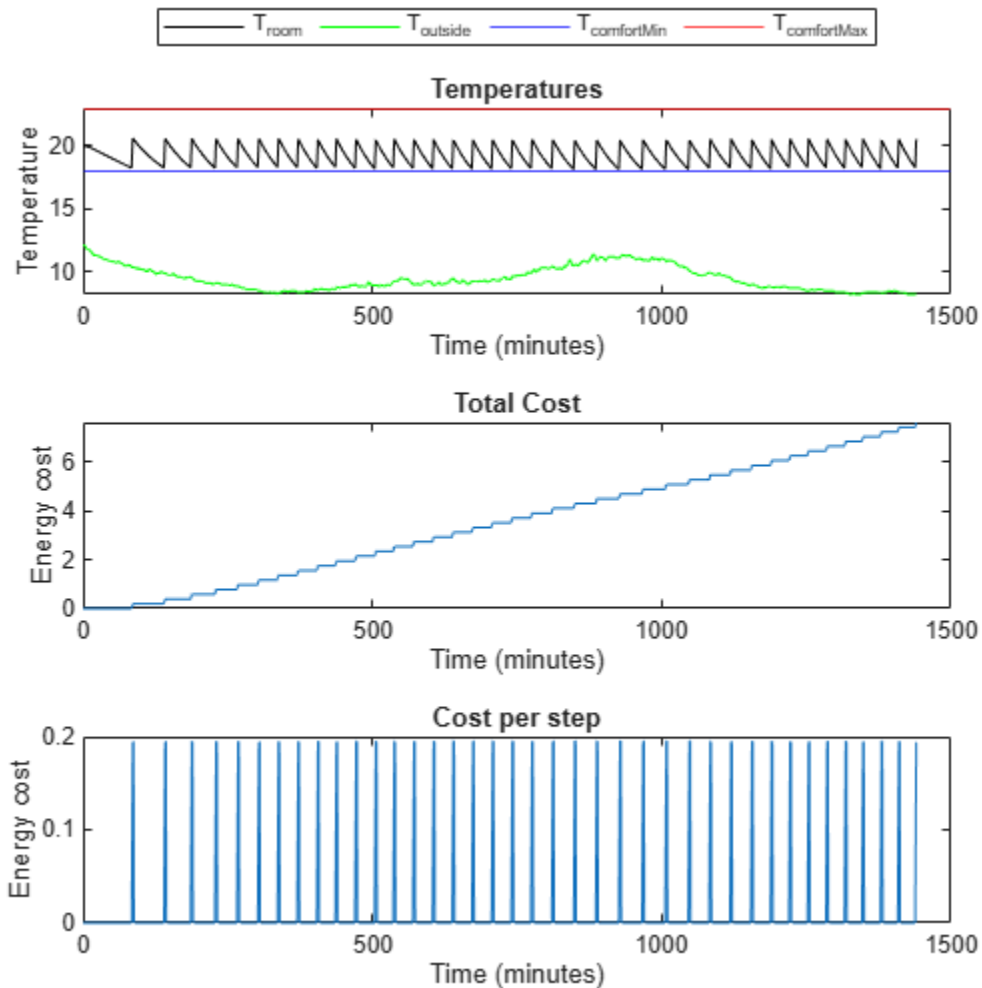
```
plotResults(experience1, maxSteps, ...
    comfortMax, comfortMin, sampleTime,1);
```



Comfort Temperature violation: 0/1440 minutes, cost: 7.214489 dollars

Next, evaluate the trained policy's performance using the validation temperature data from April 15th, 2022.

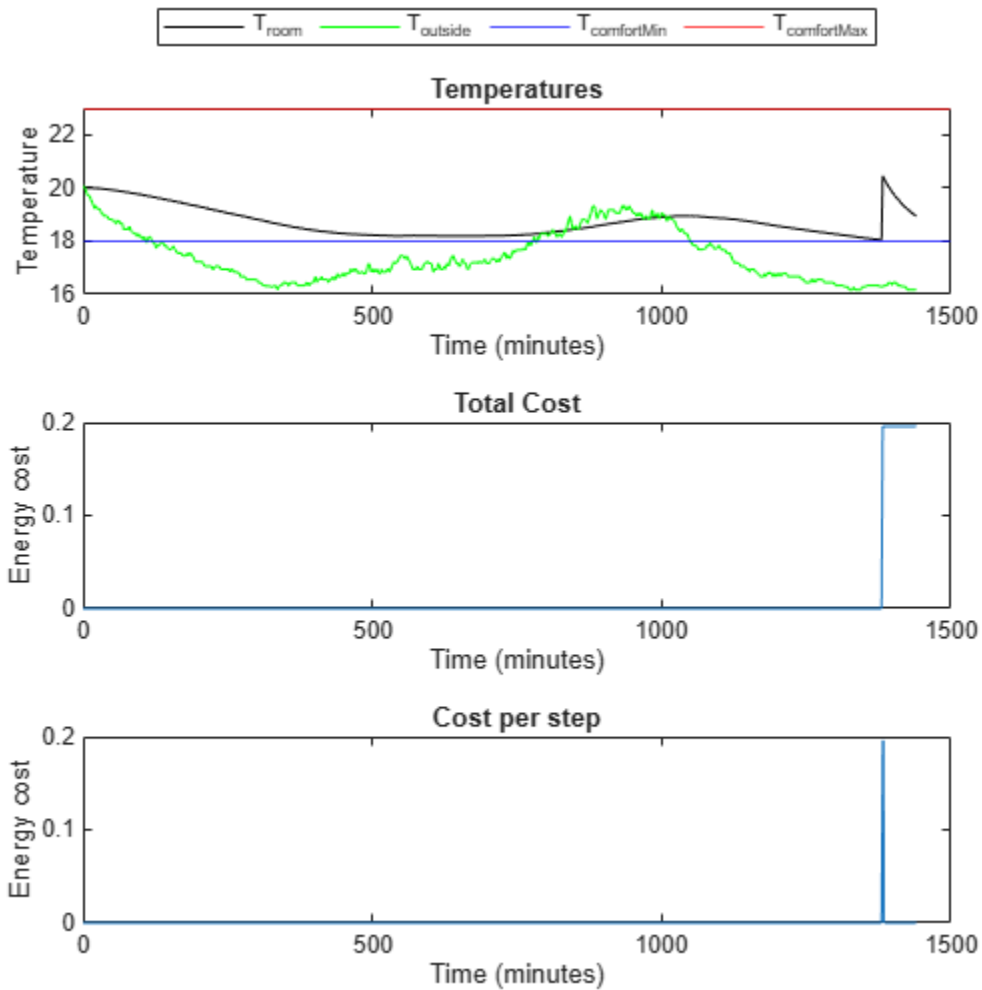
```
% Validate trained policy using the data from April 15.
validationTemperature = temperatureApril15;
env.ResetFcn = @(in) hRLHeatingSystemValidateCustomLoopResetFcn( ...
    in,validationTemperature);
experience2 = sim(env,policy,simOptions);
plotResults( ...
    experience2, ...
    maxSteps, ...
    comfortMax, ...
    comfortMin, ...
    sampleTime,2)
```



Comfort Temperature violation: 0/1440 minutes, cost: 7.604639 dollars

Evaluate the trained policy's performance when the temperature is mild. Add eight degrees to the temperatures from April 15th to create data for mild temperatures.

```
% Validate trained policy using the data from April 15 + 8 degrees.
validationTemperature = temperatureApril15;
validationTemperature(:,2) = validationTemperature(:,2) + 8;
env.ResetFcn = @(in) hRLHeatingSystemValidateCustomLoopResetFcn( ...
    in,validationTemperature);
experience3 = sim(env,policy,simOptions);
plotResults(experience3, ...
    maxSteps, ...
    comfortMax, ...
    comfortMin, ...
    sampleTime, ...
    3)
```



Comfort Temperature violation: 0/1440 minutes, cost: 0.195710 dollars

The trained policy performs well in all three scenarios.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

Create Critic Network

The `createCriticNet` function creates a custom network for the critic. Note that you must use a `sequenceInputLayer` in conjunction with LSTM layers to represent a RNN (recurrent neural network). For more information, see `sequenceInputLayer`.

```
function net = createCriticNet(obsInfo,actInfo)
```

```

numObs = obsInfo.Dimension(1);
numAct = getNumberOfElements(actInfo);

net = [
    sequenceInputLayer(numObs)
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    lstmLayer(64)
    fullyConnectedLayer(numAct)];

net = dlnetwork(net);

end

```

Critic Loss

The `criticLossFcn` function computes the loss and the gradient to update the critic. Here, to compute loss only on the actual experiences, not the padding data, multiply the loss by the sequence padding mask that the `updateAgent` function calculates when it samples from the experience buffer. For more information, see `rlReplayMemory` and `sample`.

```

function [criticGradient, loss_data] = criticLossFcn( ...
    critic, observation, action_index, targetq, mask)

    % Obtain the number of non-padded experiences.
    N = sum(mask, "all");

    % Extract the action values, Q(s,a), corresponding to the current
    % batch of observations.
    [q, ~] = getValue(critic, observation, UseForward=true);
    % Choosing the q values corresponding to the actions taken.
    q = q(action_index);
    q = q';

    % Calculate the TD error.
    targetq = reshape(targetq, size(q));
    tdError = q - targetq;

    % Multiply loss by the sequence padding mask returned
    % when sampling from the experience buffer to ignore
    % padding. Compute the loss only over actual experiences.
    tdError = tdError.*reshape(mask, size(tdError));
    % Loss is the half mean-square error of
    % q = Q(observation, action) against targetq.
    criticLossValue = 0.5.*sum(tdError.^2, "all")/N;

    % Compute the gradient of the loss with respect to the critic
    % parameters.
    criticGradient = dlgradient(criticLossValue, critic.Learnables);

    loss_data.CriticLoss = criticLossValue;

end

```

Agent Update Function

The `updateAgent` function updates the critic and the target critic. To make sure that for each batch element the sampler draws up to `SequenceLength` consecutive transitions from the experience buffer, set the `SequenceLength` name-value argument of the `sample` function. To keep all batch elements at the same `SequenceLength`, shorter sampled sequences are padded and a logical mask marks real steps (true) vs padded steps (false) is returned. The `criticLossFcn` on page 5-329 computes the loss only over real experiences by using this mask. For more information, see `rlReplayMemory` and `sample`.

```
function agentData = updateAgent(agentData)

% Obtain the number of batches needed.
numBatches = min( ...
    floor(agentData.Replay.Length/agentData.MiniBatchSize), ...
    agentData.MaxMiniBatchPerEpoch);

for epoch = 1:agentData.NumEpochs
    % Get the batch data.
    % To keep all batch elements at the same SequenceLength,
    % shorter sampled sequences are padded and a logical mask
    % marks real steps (true) vs padded steps (false)
    % is returned. The criticLossFcn use this mask to compute
    % the loss only over real experiences.
    [superMiniBatch, superMiniBatchMask] = sample(agentData.Replay, ...
        numBatches*agentData.MiniBatchSize,...
        SequenceLength=agentData.SequenceLength,ReturnDlarray=true);

    % Shuffle batch indices every epoch.
    idxshuffled = randperm(numBatches*agentData.MiniBatchSize);
    idxbeg = 1;
    idxend = idxbeg+agentData.MiniBatchSize-1;
    for iteration = 1:numBatches
        mbIdx = idxshuffled(idxbeg:idxend);
        idxbeg = idxend+1;
        idxend = idxbeg+agentData.MiniBatchSize-1;

        % Get the mini-batch data.
        mbObs = {superMiniBatch.Observation{}}(:, :, mbIdx, :);
        mbNobs = {superMiniBatch.NextObservation{}}(:, :, mbIdx, :);
        mbAct = {superMiniBatch.Action{}}(:, :, mbIdx, :);
        mbReward = superMiniBatch.Reward(:, mbIdx, :);
        mbIsDone = superMiniBatch.IsDone(:, mbIdx, :);
        mbMask = superMiniBatchMask(:, mbIdx, :);

        % Reset RNN state of critic and target critic to zeros.
        agentData.Critic.State = cellfun( ...
            @(x) zeros(size(x), 'like', x), ...
            agentData.Critic.State, 'UniformOutput', false);
        agentData.TargetCritic.State = cellfun( ...
            @(x) zeros(size(x), 'like', x), ...
            agentData.TargetCritic.State, 'UniformOutput', false);

        % Compute target.
        mbY = mbReward + (mbIsDone~=1).*getMaxQValue( ...
            agentData.TargetCritic, mbNobs).*agentData.DiscountFactor;
```

```

% Build a one-hot logical matrix of size
% (numElements, miniBatchSize*sequenceLength)
% indicating, for each action element in ActionInfo,
% which batch entries select that action.
% Convert to darray.
actionIndex = darray(localGetElementIndicationMatrix( ...
    agentData.Critic.ActionInfo, mbAct, numel(mbY)));

% Compute critic gradient using the accelerated
% criticLossFcn function.
[criticGradient, lossData] = dlfeval(agentData.AccelCriticGrad, ...
    agentData.Critic, mbObs, actionIndex, mbY, mbMask);

% Update the critic parameters.
% For more information, see <docid:rl_ref#mw_f48d6602-fa23-4aa6-b26b-4ef23f8a405c up
[agentData.Critic, agentData.CriticOptimizer] = update(...
    agentData.CriticOptimizer, agentData.Critic, ...
    criticGradient);

% Update targets using the TargetSmoothFactor hyper-parameter.
agentData.TargetCritic = syncParameters( ...
    agentData.TargetCritic, ...
    agentData.Critic, ...
    agentData.TargetSmoothFactor);

end
end
end

```

Get Element Indication Matrix

The `localGetElementIndicationMatrix` function returns a logical one-hot matrix with size `(numElement, batchSize)`. For each action element defined in `actInfo`, the matrix indicates which batch entries select that action.

```

function elementMat = localGetElementIndicationMatrix( ...
    actInfo, batchElement, batchSize)
% This function returns a True/False matrix
% with size (numElement, batchSize)

num = getNumberOfElements(actInfo); % number of elements

% Initialize element indication matrix.
elementMat = false(num, batchSize);

elementValuesFromSpec = actInfo.Elements;
elementValuesFromSpec = reshape(elementValuesFromSpec, 1, []);
elementValuesFromSpec = cast(elementValuesFromSpec, "single");

elementBatchFromInput = reshape(batchElement{1}, 1, []);

% Each row in elementValuesFromSpec contains a possible action.
% elementMat contains one-hot-vector in each row indicating which
% action is selected. The following part loops over actions
% (num) and finds the index of selected actions.

```

```
for ct = 1:num
    elementMat(ct,:) = reshape(all(elementBatchFromInput== ...
        elementValuesFromSpec(ct,:),2),size(elementMat(ct,:)));
end
end
```

Plot Results

The plotResults function plots the validation results.

```
function plotResults( ...
    experience, maxSteps, comfortMax, comfortMin, sampleTime, figNum)
% localPlotResults plots results of validation.

% Compute comfort temperature violation.
obsData = experience.Observation.obs1.Data(1,:,1:maxSteps);
minutesViolateComfort = ...
    sum(obsData < comfortMin) ...
    + sum(obsData > comfortMax);

% Compute the cost of energy.
totalCosts = ...
    experience.SimulationInfo(1).househeat_output{1}.Values;
totalCosts.Time = totalCosts.Time/60;
totalCosts.TimeInfo.Units="minutes";
totalCosts.Name = "Total Energy Cost";
finalCost = totalCosts.Data(end);

% Compute cost of energy per step.
costPerStep = ...
    experience.SimulationInfo(1).househeat_output{2}.Values;
costPerStep.Time = costPerStep.Time/60;
costPerStep.TimeInfo.Units="minutes";
costPerStep.Name = "Energy Cost per Step";
minutes = (0:maxSteps)*sampleTime/60;

% Plot results.
fig = figure(figNum);
% Change the size of the figure.
fig.Position = fig.Position + [0, 0, 0, 200];

% Temperatures
layoutResult = tiledlayout(3,1);
nexttile
plot(minutes, ...
    reshape(experience.Observation.obs1.Data(1,:,:), ...
        [1,length(experience.Observation.obs1.Data)]),"k")
hold on
plot(minutes, ...
    reshape(experience.Observation.obs1.Data(2,:,:), ...
        [1,length(experience.Observation.obs1.Data)]),"g")
yline(comfortMin,'b')
yline(comfortMax,'r')
lgd = legend("T_{room}", "T_{outside}","T_{comfortMin}", ...
    "T_{comfortMax}","location","northoutside");
lgd.NumColumns = 4;
title("Temperatures")
```



```

ylabel("Temperature")
xlabel("Time (minutes)")
hold off

% Total cost
nexttile
plot(totalCosts)
title("Total Cost")
ylabel("Energy cost")

% Cost per step
nexttile
plot(costPerStep)
title("Cost per step")
ylabel("Energy cost")
fprintf("Comfort Temperature violation:" + ...
        " %d/1440 minutes, cost: %f dollars\n", ...
        minutesViolateComfort, finalCost);
end

```

Reference

[1] Du, Yan, Fangxing Li, Kuldeep Kurte, Jeffrey Munk, and Helia Zandi. “Demonstration of Intelligent HVAC Load Management With Deep Reinforcement Learning: Real-World Experience of Machine Learning in Demand Control.” *IEEE Power and Energy Magazine* 20, no. 3 (May 2022): 42–53. <https://doi.org/10.1109/MPE.2022.3150825>.

See Also

Functions

`dlaccelerate` | `dlgradient` | `dlfeval` | `evaluate` | `rlReplayMemory` | `sample` | `rlOptimizer` | `rlDataLogger`

Objects

`rlOptimizerOptions`

Related Examples

- “Train DQN Agent with LSTM Network to Control House Heating System” on page 8-299
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Custom PPO Training Loop with Random Network Distillation” on page 5-298

More About

- “Deep Q-Network (DQN) Agent” on page 3-32
- “Create Custom Simulink Environments” on page 2-110
- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Reinforcement Learning Agents” on page 3-92

Create and Train Custom PG Agent

This example shows how to create and train a custom PG agent. A custom agent allows you to leverage the following built-in functionality from the Reinforcement Learning Toolbox™ software:

- Access to all agent functions, including `train` and `sim`.
- Visualize training progress using the Reinforcement Learning Training Monitor.
- Train agents within a Simulink® environment.

In this example, you convert a custom REINFORCE training loop into a custom agent class, and then train an object of this class (your custom agent) using `train`. For more information on writing custom agent classes, see “Create Custom Reinforcement Learning Agents” on page 3-92. For an example that shows how to create and train a custom agent that learns to solve an LQR problem, see “Create and Train Custom LQR Agent” on page 5-345.

For more information on custom training loops (that is, loops that do not rely on `train` or `sim`), see instead “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

Create the same training environment used in the “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273 example. The environment is a cart-pole balancing environment with a discrete action space. Create the environment using the `rlPredefinedEnv` function.

```
env = rlPredefinedEnv("CartPole-Discrete");
```

Extract the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

Obtain the dimension of the observation space (`numObs`) and the number of possible actions (`numAct`).

```
numObs = obsInfo.Dimension(1);  
numAct = numel(actInfo.Elements);
```

For more information on this environment, see “Use Predefined Control System Environments” on page 2-42.

Define Policy

The reinforcement learning policy in this example is a parameterized discrete action space stochastic policy, which is learned by a discrete categorical actor. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer. The output layer must return a vector of probabilities for each possible action, as specified by `actInfo`.

Define the network as an array of layer objects, using `fullyConnectedLayer`, `reluLayer`, and `softmaxLayer` layers. The `softmaxLayer` ensures that the policy outputs probability values in the range [0 1] and that all probabilities sum to 1.

```
actorNetwork = [
    featureInputLayer(numObs)
    fullyConnectedLayer(24)
    reluLayer
    fullyConnectedLayer(24)
    reluLayer
    fullyConnectedLayer(2)
    softmaxLayer
];
```

When you initialize a `dlnetwork` object, the network weights are initialized with random values. Fix the random number stream so that the network is always initialized with the same weight values.

```
rng(0, "twister");
```

Convert to a `dlnetwork` object and summarize its properties.

```
actorNetwork = dlnetwork(actorNetwork)

actorNetwork =
  dlnetwork with properties:

    Layers: [7×1 nnet.cnn.layer.Layer]
Connections: [6×2 table]
Learnables: [6×3 table]
    State: [0×3 table]
InputNames: {'input'}
OutputNames: {'softmax'}
  Initialized: 1
```

View summary with `summary`.

```
summary(actorNetwork)

  Initialized: true

  Number of learnables: 770
```

```
Inputs:
    1   'input'   4 features
```

Create the actor using an `rlDiscreteCategoricalActor` object.

```
actor = rlDiscreteCategoricalActor(actorNetwork,obsInfo,actInfo);
```

Create the optimizer options object using `rlOptimizerOptions`.

```
actorOpts = rlOptimizerOptions(LearnRate=1e-3);
```

Custom Agent Class

To define your custom agent, first create a class that is a subclass of the `rl.agent.CustomAgent` class. The custom agent class for this example is defined in `CustomReinforceAgent.m`.

The `CustomReinforceAgent` class has the following class definition, which indicates the agent class name and the associated abstract agent.

```
classdef CustomReinforceAgent < rl.agent.CustomAgent
```

To define your agent you must specify the following:

- Agent properties
- Constructor function
- Critic approximator, to estimate the value of the policy (if needed)
- Actor, to learn the policy (if needed)
- Required agent methods
- Optional agent methods

Agent Properties

In the `properties` section of the class file, specify any parameters necessary for creating and training the agent.

The `rl.Agent.CustomAgent` class already includes properties for the agent sample time (`SampleTime`) and the action and observation specifications (`ActionInfo` and `ObservationInfo`, respectively).

The custom REINFORCE agent defines the following additional agent properties.

```
properties
    % Policy
    Policy
    ActorOptimizer

    % Agent options
    Options

    % Experience buffer
    ObservationBuffer
    ActionBuffer
    RewardBuffer
    MaskBuffer
end
```

```

properties (Access = private)
    % Training utilities
    Counter
    NumObservation
    NumAction
end
properties (Access = private,Transient)
    % Accelerated gradient function, not saved with the agent
    AccelGradFcn = []
end

```

Constructor Function

To create your custom agent, you must define a constructor function. The constructor function performs the following actions.

- Defines the action and observation specifications. For more information about creating these specifications, see `rlNumericSpec` and `rlFiniteSetSpec`.
- Sets the agent properties.
- Calls the constructor of the base abstract class.
- Defines the sample time (required for training in Simulink environments).

For example, the `CustomReinforceAgent` constructor defines action and observation spaces based on the input actor.

```

function obj = CustomReinforceAgent(Actor,Options)
    %CUSTOMREINFORCEAGENT Construct custom agent
    % AGENT = CUSTOMREINFORCEAGENT(ACTOR,OPTIONS) creates custom
    % REINFORCE AGENT from rlDiscreteCategoricalActor ACTOR
    % and structure OPTIONS. OPTIONS has fields:
    %     - DiscountFactor
    %     - MaxStepsPerEpisode

    % (required) Call the abstract class constructor.
    obj = obj@rl.agent.CustomAgent();
    obj.ObservationInfo = Actor.ObservationInfo;
    obj.ActionInfo = Actor.ActionInfo;

    % (required for Simulink environment) Register sample time.
    % For MATLAB environment, use -1.
    obj.SampleTime = -1;

    % (optional) store the policy and agent options.
    obj.Policy = rlStochasticActorPolicy(Actor);
    obj.Options = Options;
    obj.ActorOptimizer = rlOptimizer(Options.OptimizerOptions);

    % (optional) Cache the number of observations and actions.
    obj.NumObservation = prod(obj.ObservationInfo.Dimension);
    obj.NumAction = prod(obj.ActionInfo.Dimension);

    % (optional) Initialize buffer and counter.
    resetImpl(obj);
end

```

Required Functions

To create a custom reinforcement learning agent you must define the following implementation functions.

- `getActionImpl` — Evaluates agent policy and selects an action during simulation.
- `getActionWithExplorationImpl` — Evaluates policy and selects an action with exploration during training.
- `learnImpl` — Updates learnable parameters, therefore allowing the agent to learn from the current experience.

To call these functions in your own code, use the wrapper methods from the abstract base class. For example, to call `getActionImpl`, use `getAction`. The wrapper methods have the same input and output arguments as the implementation methods.

`getActionImpl` Function

The `getActionImpl` function is used to evaluate the policy of your agent and select an action when simulating the agent using the `sim` function. This function must have the following signature, where `obj` is your custom agent object, `Observation` is the current observation, and `Action` is the selected action.

```
function Action = getActionImpl(obj,Observation)
```

For the custom REINFORCE agent, you select an action by calling the `getAction` function for the policy. The `rlStochasticActorPolicy` object created from the `rlDiscreteCategoricalActor` generates a discrete distribution from an observation. The policy then samples (and returns) the maximum likelihood action from that distribution.

```
function Action = getActionImpl(obj,Observation)
    % Compute the maximum likelihood action given an observation.
    obj.Policy.UseMaxLikelihoodAction = true;
    Action = getAction(obj.Policy,Observation);
end
```

`getActionWithExplorationImpl` Function

The `getActionWithExplorationImpl` function selects an action using the exploration model of your agent when training the agent using the `train` function. Using this function you can implement exploration techniques such as epsilon-greedy exploration or the addition of Gaussian noise. This function must have the following signature, where `obj` is your custom agent object, `Observation` is the current observation, and `Action` is the selected action.

```
function Action = getActionWithExplorationImpl(obj,Observation)
```

For the custom REINFORCE agent, the `getActionWithExplorationImpl` function randomly samples actions from the discrete action probability distribution corresponding to the observation.

```
function Action = getActionWithExplorationImpl(obj,Observation)
    % Compute an action using the exploration policy given an
    % observation.

    % REINFORCE: Stochastic actors always explore by default
    % (sample from a probability distribution)
    obj.Policy.UseMaxLikelihoodAction = false;
```

```

        Action = getAction(obj.Policy, Observation);
end

```

LearnImpl Function

The `learnImpl` function defines how the agent learns from the current experience. This function implements the custom learning algorithm of your agent by updating the policy parameters and selecting an action with exploration for the next state. This function must have the following signature, where `obj` is the agent object, `Experience` is the current agent experience, and `Action` is the selected action.

```
function Action = learnImpl(obj, Experience)
```

The agent experience is the cell array `Experience = {observation, action, reward, nextObs, isdone}`. Here:

- `observation` is the current observation.
- `action` is the current action. This is different from the output argument `Action`, which is an action for the next state.
- `reward` is the current reward.
- `nextObs` is the next observation.
- `isDone` is a logical flag indicating that the training episode is complete.

```

function Action = learnImpl(obj, Experience)
    % Define how the agent learns from an Experience, which is a
    % cell array with the following format.
    % Experience = ...
    % {observation, action, reward, nextObservation, isDone}

    % Extract data from experience.
    Obs = Experience{1};
    Action = Experience{2};
    Reward = Experience{3};
    NextObs = Experience{4};
    IsDone = Experience{5};

    % Save data to buffer.
    obj.ObservationBuffer(:, :, obj.Counter) = Obs{1};
    obj.ActionBuffer(:, :, obj.Counter) = Action{1};
    obj.RewardBuffer(:, obj.Counter) = Reward;
    obj.MaskBuffer(:, obj.Counter) = 1;

    if ~IsDone
        % Choose an action for the next state.
        Action = getActionWithExplorationImpl(obj, NextObs);
        obj.Counter = obj.Counter + 1;
    else
        % Learn from episodic data.
        BatchSize = obj.Options.MaxStepsPerEpisode;

        % Compute the discounted future reward.
        DiscountedReturn = dlarray(zeros(1, BatchSize));
        gamma = obj.Options.DiscountFactor;
        for t = 1:BatchSize
            k = t:BatchSize;
            G = sum(gamma.^(k-t) .* obj.RewardBuffer(k));

```

```

        DiscountedReturn(t) = G;
    end

    % Compute the indices of actions sampled during the
    % trajectory.
    Z = repmat(obj.ActionInfo.Elements',1,BatchSize);
    actionIndicationMatrix = obj.ActionBuffer(:, :) == Z;

    % Compute the gradient of the loss with respect to the
    % actor parameters. Use dlaccelerate to improve gradient
    % computation performance. Note, the mask buffer is used
    % here to make sure accelerated functions are not
    % re-generated due to varying episode lengths.
    if isempty(obj.AccelGradFcn)
        obj.AccelGradFcn = dlaccelerate(@lossFunction);
    end
    ActorGradient = dlfeval(obj.AccelGradFcn, ...
        obj.Policy.Actor,{obj.ObservationBuffer}, ...
        actionIndicationMatrix,DiscountedReturn,obj.MaskBuffer);

    % Update the actor parameters using the computed gradients.
    [obj.Policy.Actor,obj.ActorOptimizer] = update( ...
        obj.ActorOptimizer,obj.Policy.Actor,ActorGradient);

    % Reset the counter and flush the reward and mask buffers
    % for the next trajectory.
    obj.Counter = 1;
    obj.MaskBuffer(:) = 0;
    obj.RewardBuffer(:) = 0;
end
end

```

The custom REINFORCE agent is a similar implementation to the custom training loop found in “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273 with the exception that, for the custom agent, the built-in `train` function manages the training loop of the agent. For the custom training loop, instead, the training loop is not managed by `train`.

The `lossFunction` in `CustomREINFORCEAgent.m` computes the gradient of the loss function with respect to the parameters. The loss function in the REINFORCE algorithm is the product between the discounted reward and the logarithm of the probability distribution of the action (coming from the policy evaluation for a given observation), summed across all time steps.

The loss function accepts the actor function approximator as an input argument and calls `evaluate` to compute the batch discrete action probabilities given batch observations. `UseForward` is set to true to handle cases where the actor has layers that modify behavior during the forward pass of the network. The remaining input objects are required data to compute the REINFORCE loss function. The actor gradients are then evaluated from the loss with respect to the actor `Learnable` parameters.

To reduce the time to compute actor gradients, `dlaccelerate` is used in conjunction with `dlfeval` and `dlgradient`. To prevent multiple accelerated functions from being generated, all varying input arguments (aside from the actor) must be `dlarray` objects, cell arrays of `dlarray` objects, or structures of `dlarray` objects with fixed size. The mask argument is used to prevent certain batch elements from contributing to the loss function (for example, when the episode terminates early).

```

function actorGradient = lossFunction( ...
    actor,observations,actionIndicationMatrix,discountedReturn,mask)

```



```

% Evaluate the action probabilities given batch observations.
% Set UseForward=true to handle layers such as batch normalization
% which modify their behavior during the forward pass.
actionProbs = evaluate(actor,observations,UseForward=true);
actionProbs = actionProbs{1};

% Resize the discounted return to the size of actionProbs.
% Elements in the batch data corresponding to mask == 0
% do not contribute to the loss function.
G = actionIndicationMatrix .* discountedReturn .* mask;
G = reshape(G,size(actionProbs));

% Clip action probability values less than eps to eps.
actionProbs(actionProbs < eps) = eps;

% Compute the loss.
loss = -sum(G.*log(actionProbs),"all")/sum(mask);
actorGradient = dlgradient(loss,actor.Learnables);
end

```

Optional Functions

Optionally, you can define how your agent is reset at the start of training by specifying a `resetImpl` function with the following function signature, where `obj` is the agent object.

```
function resetImpl(obj)
```

Using this function, you can set the agent into a know or random condition before training.

```
function resetImpl(obj)
    % (Optional) Define how the agent is reset before training/

    resetBuffer(obj);
    obj.Counter = 1;
end

```

Also, you can define any other helper functions in your custom agent class as required. For example, the custom REINFORCE agent defines a `resetBuffer` function for re-initializing the experience buffer at the beginning of each training episode.

```
function resetBuffer(obj)
    % Reinitialize observation buffer. Allocate as darray to
    % support automatic differentiation with dlfeval and
    % dlgradient.
    obj.ObservationBuffer = darray( ...
        zeros(obj.NumObservation,1,obj.Options.MaxStepsPerEpisode));

    % Reinitialize action buffer with valid actions.
    obj.ActionBuffer = darray( ...
        repmat( ...
            obj.ActionInfo.Elements(1), ...
            1, ...
            1, ...
            obj.Options.MaxStepsPerEpisode) ...
        );

    % Reinitialize reward buffer.

```

```
obj.RewardBuffer = dlmatrix(zeros(1,obj.Options.MaxStepsPerEpisode));  
  
% Reinitialize mask buffer.  
obj.MaskBuffer = dlmatrix(zeros(1,obj.Options.MaxStepsPerEpisode));  
end
```

Create Custom Agent

Once you have defined your custom agent class, create an instance of it in the MATLAB® workspace. To create the custom REINFORCE agent, first specify the agent options.

```
options.MaxStepsPerEpisode = 250;  
options.DiscountFactor = 0.995;  
options.OptimizerOptions = actorOpts;
```

Then, using the options and the previously defined actor, call the constructor function of the custom agent.

```
agent = CustomReinforceAgent(actor,options);
```

Train Custom Agent

Configure the training to use the following options.

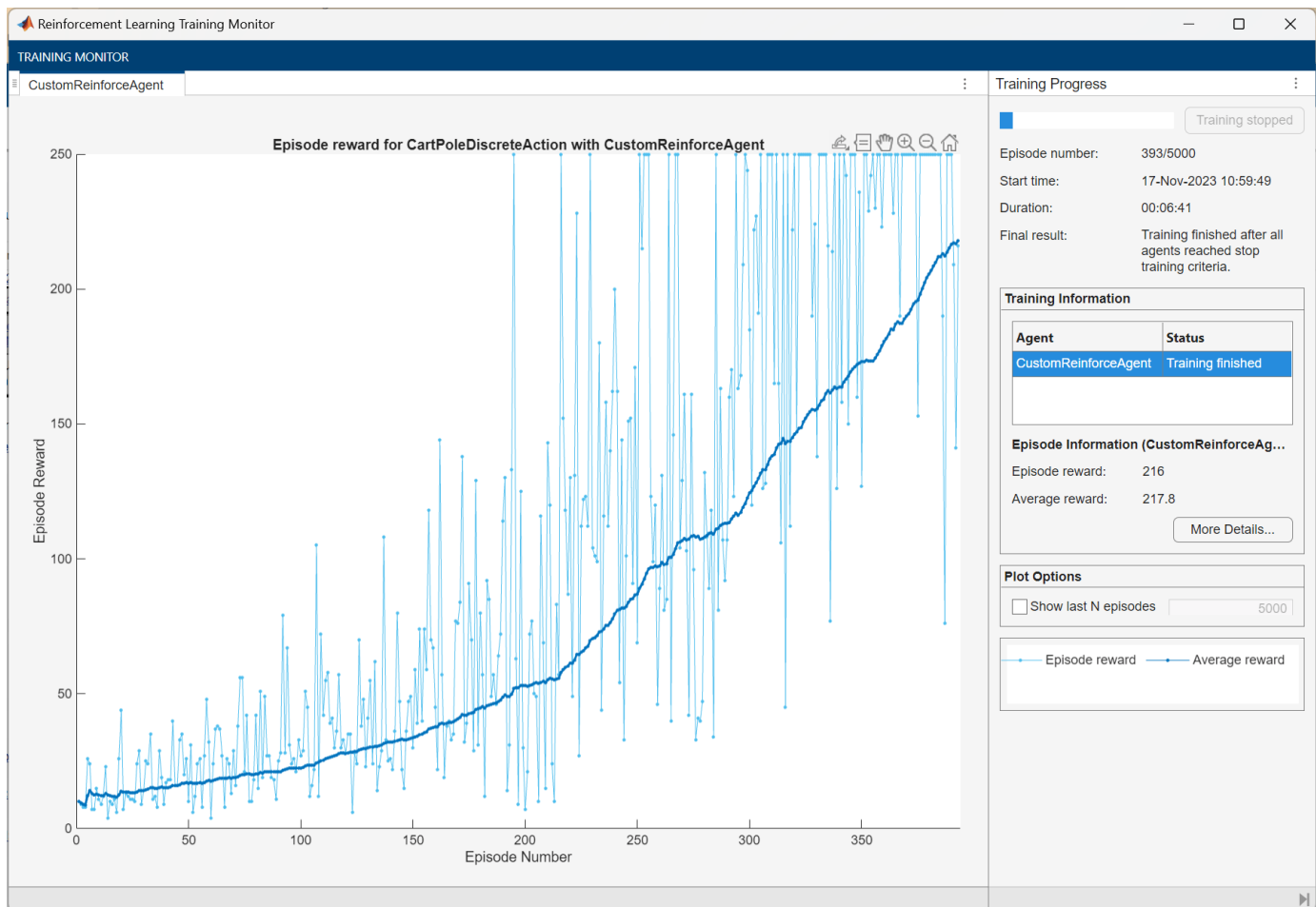
- Set up the training to last a maximum of 5000 episodes, with each episode lasting a maximum of 250 steps.
- Terminate the training after the maximum number of episodes is reached or when the average reward across 100 episodes reaches a value of 220.

For more information, see `rlTrainingOptions`.

```
numEpisodes = 5000;  
aveWindowSize = 100;  
trainingTerminationValue = 220;  
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=numEpisodes, ...  
    MaxStepsPerEpisode=options.MaxStepsPerEpisode, ...  
    ScoreAveragingWindowLength=aveWindowSize, ...  
    StopTrainingValue=trainingTerminationValue);
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainStats = train(agent,env,trainOpts);  
else  
    % Load pretrained agent for the example.  
    load("CustomReinforce.mat","agent");  
end
```



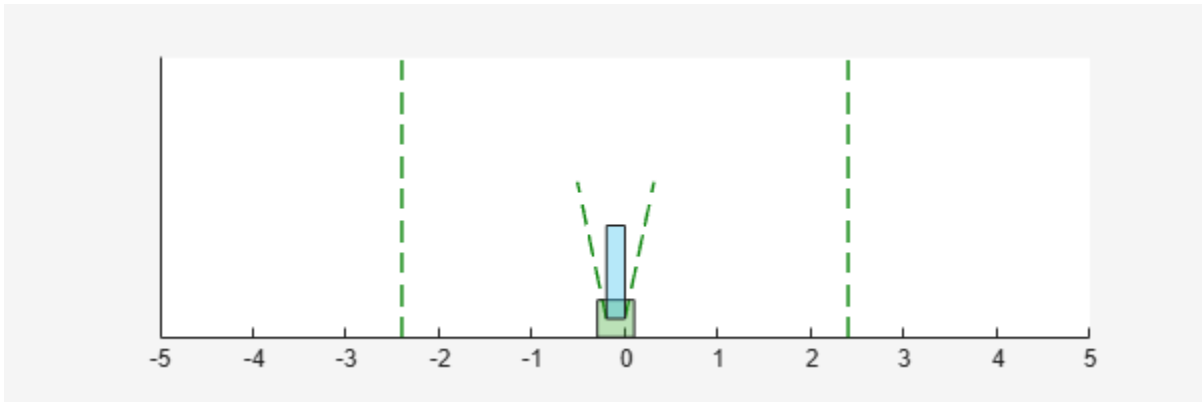
Simulate Custom Agent

Enable the environment visualization, which is updated each time the environment step function is called.

```
plot(env)
```

To validate the performance of the trained agent, simulate it within the cart-pole environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOpts = rlSimulationOptions(MaxSteps=options.MaxStepsPerEpisode);
experience = sim(env,agent,simOpts);
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `train` | `evaluate` | `gradient` | `accelerate` | `getAction` | `sim`

Objects

`rlDiscreteCategoricalActor` | `rlNumericSpec` | `rlFiniteSetSpec` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Create and Train Custom LQR Agent” on page 5-345
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273

More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Create Custom Reinforcement Learning Agents” on page 3-92

Create and Train Custom LQR Agent

This example shows how to create and train a custom linear quadratic regulation (LQR) agent to control a discrete-time linear system modeled in MATLAB®. For an introduction to custom agents, see “Create Custom Reinforcement Learning Agents” on page 3-92. For a step by step example on how to create a custom PG agent (using the REINFORCE algorithm) see “Create and Train Custom PG Agent” on page 5-334. For an example of how a DDPG agent can be used as an optimal controller for a discrete-time system, see “Compare DDPG Agent to LQR Controller” on page 7-285.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Linear System Environment

The reinforcement learning environment for this example is a discrete-time linear system. The dynamics for the system are given by:

$$x_{t+1} = Ax_t + Bu_t.$$

The feedback control law is:

$$u_t = -Kx_t.$$

The control objective is to minimize the quadratic cost: $J = \sum_{t=0}^{\infty} (x_t^T Q x_t + u_t^T R u_t)$.

In this example, the system matrices are:

$$A = \begin{bmatrix} 1.05 & 0.05 & 0.05 \\ 0.05 & 1.05 & 0.05 \\ 0 & 0.05 & 1.05 \end{bmatrix}$$

$$B = \begin{bmatrix} 0.1 & 0 & 0.2 \\ 0.1 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}$$

```
A = [1.05,0.05,0.05;0.05,1.05,0.05;0,0.05,1.05];
B = [0.1,0,0.2;0.1,0.5,0;0,0,0.5];
```

The quadratic cost matrices are:

$$Q = \begin{bmatrix} 10 & 3 & 1 \\ 3 & 5 & 4 \\ 1 & 4 & 9 \end{bmatrix}$$

$$R = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}$$

```
Q = [10,3,1;3,5,4;1,4,9];
R = 0.5*eye(3);
```

For this environment, the reward at time $t + 1$ is given by $r_{t+1} = -x_t^T Q x_t - u_t^T R u_t$ which is the negative of the quadratic cost. Therefore, maximizing the reward minimizes the cost. The initial conditions are set randomly by the reset function.

The `myDiscreteEnv` function, defined in the file `myDiscreteEnv.m`, creates an environment by defining custom `step` and `reset` functions. For more information on creating such a custom environment, see “Create Custom Environment Using Step and Reset Functions” on page 2-94.

```
type("myDiscreteEnv");

function env = myDiscreteEnv(A,B,Q,R)

% This function creates a discrete-time linear system environment.
%
% (A,B) are the system matrices, where dx = Ax + Bu.
% (Q,R) defines the quadratic cost, where cost = x'Qx + u'Ru.

% Copyright 2018-2026 The MathWorks Inc.

% Create the observation and action specification objects.
% The observation is the state, x, the action is the control input, u.
obsInfo = rlNumericSpec([size(A,1),1]);
actInfo = rlNumericSpec([size(B,2),1]);

% Create the environment. Define anonymous step and reset functions that
% take the required number of arguments and in turn call functions
% that use the parameters A,B,Q, and R.
env = rlFunctionEnv(obsInfo,actInfo,...
    @(action,state) myStepFunction(action,state,A,B,Q,R), ...
    @() myResetFunction(Q));

end

function [NextObservation,Reward,IsDone,xp] = myStepFunction(u,x,A,B,Q,R)
% This is the step function for the environment, which returns the
% next observation, reward, is-done signal, and next state for a given action.

% Calculate the reward.
% To be consistent with the LQR cost, the reward is computed based on the current state x
% and the action u.
Reward = -x'*Q*x - u'*R*u;

% Advance system according to its dynamics.
xp = A*x + B*u;
```

```
% Next observation and next state are the same in this environment.
NextObservation = xp;
```

```
% Set IsDone to false.
IsDone = false;
```

```
end
```

```
function [InitialObservation, State] = myResetFunction(Q)
% This is the reset function for the environment, which sets
% random initial conditions for observation and state.
```

```
n = size(Q,1);
x0 = rand(n,1);
InitialObservation = x0;
State=InitialObservation;
```

```
end
```

Create the environment object.

```
env = myDiscreteEnv(A,B,Q,R);
```

Create Custom LQR Agent

For an LQR problem, the optimal Q-value function can be represented by the quadratic form

$Q_V(x, u) = [x^T \ u^T] S \begin{bmatrix} x \\ u \end{bmatrix}$, where $S = \begin{bmatrix} S_{xx} & S_{xu} \\ S_{ux} & S_{uu} \end{bmatrix}$ is a symmetric, positive definite matrix defined in [1 on page 5-355]. Taking the partial derivative of $Q_V(x, u)$ with respect to u , setting that to zero and solving for u (as in [2 on page 5-355] and [3 on page 5-355]) yields $u = -S_{uu}^{-1} S_{ux} x$, which is the control law that maximizes the value of $Q_V(x, u)$.

Because it is symmetric, the matrix S contains $m = \frac{1}{2}n(n + 1)$ distinct element values, where n is the sum of the number of states and number of inputs. Define as θ^* the vector containing these m elements, in which the off-diagonal elements in S are multiplied by two. You can then express $Q_V(x, u)$ as the inner product of the vectors θ^* and $h(x, u)$, where $h(x, u)$ is a vector of quadratic monomials built from the combination of all the elements in x and u . For an example, see the W_Q matrix in “Compare DDPG Agent to LQR Controller” on page 7-285.

In this example, the custom agent needs to learn the parameter vector θ^* , (starting from an initial random vector θ) so that $\theta^{*T} h(x, u) = Q_V(x, u)$. To solve this least squares problem, `mldivide` (“\” operator) is used in this example.

Fix the random generator seed for reproducibility.

```
rng(0, "twister")
```

The LQR agent starts with a stabilizing controller K_0 . To get an initial stabilizing controller, place the poles of the closed-loop system $A - BK_0$ inside the unit circle.

```
K0 = place(A,B,[0.4,0.8,0.5]);
```

To create a custom agent, you must create a subclass of the `rl.agent.CustomAgent` abstract class. For the custom LQR agent, the defined custom subclass is `LQRCustomAgent`. For more information, see “Create Custom Reinforcement Learning Agents” on page 3-92.

```
type("LQRCustomAgent");
```

```
classdef LQRCustomAgent < rl.agent.CustomAgent
    % LQRCustomAgent: Creates an LQR Agent for a linear system.
    %
    % agent = LQRCustomAgent(Q,R,K0) creates a LQR agent
    % Q and R are penalty matrices for the state and action,
    % K0 is an initial stabilizing feedback gain.
    %
    % Copyright 2018-2026 The MathWorks Inc.
    %
    % Reference:
    % Lewis, Frank L., Draguna Vrabie, and Kyriakos G. Vamvoudakis.
    % "Reinforcement learning and feedback control: Using natural decision methods to
    % design optimal adaptive controllers." IEEE Control Systems Magazine 32, no. 6
    % (2012): 76-105.
    %% Public Properties
    properties
        % Q - Weights state deviation from zero
        Q

        % R - Weights control action
        R

        % K - Feedback gain: u = K*x
        K

        % Discount Factor
        Gamma = 0.95

        % Critic Weights
        Theta

        % Buffer for K
        KBuffer

        % Number of updates for K
        KUpdate = 1

        % Number of steps before updating the critic
        EstimateNum = 10
    end

    properties (Access = private)
        Counter = 1
        CostBuffer
        HBuffer
    end

    %% MAIN METHODS
    methods
        % Constructor
```



```

function obj = LQRCustomAgent(Q,R,K0)

    % Check the number of input arguments.
    narginchk(3,3);

    % Call the abstract class constructor.
    obj = obj@rl.agent.CustomAgent();

    % Set the Q and R properties.
    obj.Q = Q;
    obj.R = R;

    % Define the observation and action spaces.
    obj.ObservationInfo_ = rlNumericSpec([size(Q,1),1]);
    obj.ActionInfo_ = rlNumericSpec([size(R,1),1]);

    % Create the critic.
    obj.Theta = createCriticWeights(obj);

    % Initialize the gain matrix.
    obj.K = K0;

    % Initialize the experience buffers.
    obj.CostBuffer = zeros(obj.EstimateNum,1);
    num = size(Q,1) + size(R,1);
    obj.HBuffer = zeros(obj.EstimateNum,0.5*num*(num+1));
    obj.KBuffer = cell(1,1000);
    obj.KBuffer{1} = obj.K;
end

end

%% Implementation of abstract parent protected methods
methods (Access = protected)

    % Action methods

    function action = getActionImpl(obj,Observation)
        % Given the current state of the system, return an action.
        action = -obj.K*Observation{:};
    end

    function action = getActionWithExplorationImpl(obj,Observation)

        % Given the current observation, select an action
        action = getActionImpl(obj,Observation);

        % Add random noise to action
        num = size(obj.R,1);
        action = action + 0.1*randn(num,1);
    end

    % learn from current experiences, return action with exploration
    % exp = {state,action,reward,nextstate,isdone}
    function action = learnImpl(obj,exp)
        % Parse the experience input
        x = exp{1}{1};
        u = exp{2}{1};

```

```
dx = exp{4}{1};
% The cost is computed using Q and R defined in this custom LQR agent,
% instead of using the reward from the environment, exp{3}.
cost = (x'*obj.Q*x + u'*obj.R*u);
num = size(obj.Q,1) + size(obj.R,1);

% Wait N steps before updating critic parameters
N = obj.EstimateNum;

% Evaluating the critic at the points (x,u) and (dx,-K*dx) yields Q1 =
% theta'*h1 and Q2 = theta'*h2, respectively, where h1 = h(x,u) and h2 =
% h(dx,-K*dx).
% This LQR custom agent follows the formulation in the referenced paper,
% which uses a cost-minimization Bellman equation (i.e., it minimizes Q
% rather than maximizes it). The agent tries to learn theta so that Q1 =
% cost + gamma*Q2, that is, theta'*(h1-gamma*h2) = theta'*H = cost, where cost =
% (x'*obj.Q*x + u'*obj.R*u) is the LQR cost.

% Following is the least square solution.
h1 = computeQuadraticBasis(x,u,num);
h2 = computeQuadraticBasis(dx,-obj.K*dx,num);
H = h1 - obj.Gamma*h2;

if obj.Counter<=N
    obj.CostBuffer(obj.Counter) = cost;
    obj.HBuffer(obj.Counter,:) = H;
    obj.Counter = obj.Counter + 1;
else
    % Update the critic parameters
    % based on the batch of experiences
    H_buf = obj.HBuffer;
    cost_buf = obj.CostBuffer;

    % Solve for the parameter vector theta using least-squares
    % regression.
    theta = H_buf\cost_buf;
    obj.Theta = theta;

    % Derive a new gain matrix
    % based on the new critic parameters
    obj.K = getNewK(obj);

    % Reset the experience buffers
    obj.Counter = 1;
    obj.CostBuffer = zeros(N,1);
    obj.HBuffer = zeros(N,0.5*num*(num+1));
    obj.KUpdate = obj.KUpdate + 1;
    obj.KBuffer{obj.KUpdate} = obj.K;

end

% Find and return an action with exploration
action = getActionWithExplorationImpl(obj,exp{4});
end

% Create critic
function w0 = createCriticWeights(obj)
    nQ = size(obj.Q,1);
```

```

        nR = size(obj.R,1);
        n = nQ+nR;
        w0 = 0.1*ones(0.5*(n+1)*n,1);
    end

    % Update K from critic
    function k = getNewK(obj)
        w = obj.Theta;
        nQ = size(obj.Q,1);
        nR = size(obj.R,1);
        n = nQ+nR;
        idx = 1;
        for r = 1:n
            for c = r:n
                Phat(r,c) = w(idx);
                idx = idx + 1;
            end
        end
        S = 1/2*(Phat+Phat');
        Suu = S(nQ+1:end,nQ+1:end);
        Sux = S(nQ+1:end,1:nQ);
        if rank(Suu) == nR
            k = Suu\Sux;
        else
            k = obj.K;
        end
    end
end
end

%% local function
function h = computeQuadraticBasis(x,u,n)
    z = cat(1,x,u);
    idx = 1;
    for r = 1:n
        for c = r:n
            if idx == 1
                h = z(r)*z(c);
            else
                h = cat(1,h,z(r)*z(c));
            end
            idx = idx + 1;
        end
    end
end
end

```

Create the custom LQR agent using Q , R , and K_0 . The agent does not require information on the system matrices A and B .

```
agent = LQRCustomAgent(Q,R,K0);
```

For this example, set the agent discount factor to one. To use a discounted future reward, set the discount factor to a value less than one.

```
agent.Gamma = 1;
```

Because the linear system has three states and three inputs, the total number of learnable parameters is $m = 21$. To ensure satisfactory performance of the agent, set the number of data point

to be collected before updating the critic, N_p , to be greater than twice the number of learnable parameters. For this example, set N_p to 45.

```
m = numel(agent.Theta);      % m = 21
agent.EstimateNum = m*2 + 3; % Np = 45
```

To get good estimation results for θ , you must apply a persistently excited exploration model to the system. In this example, encourage model exploration by adding white noise to the controller output: $u_t = -Kx_t + e_t$. In general, the exploration model depends on the system models.

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

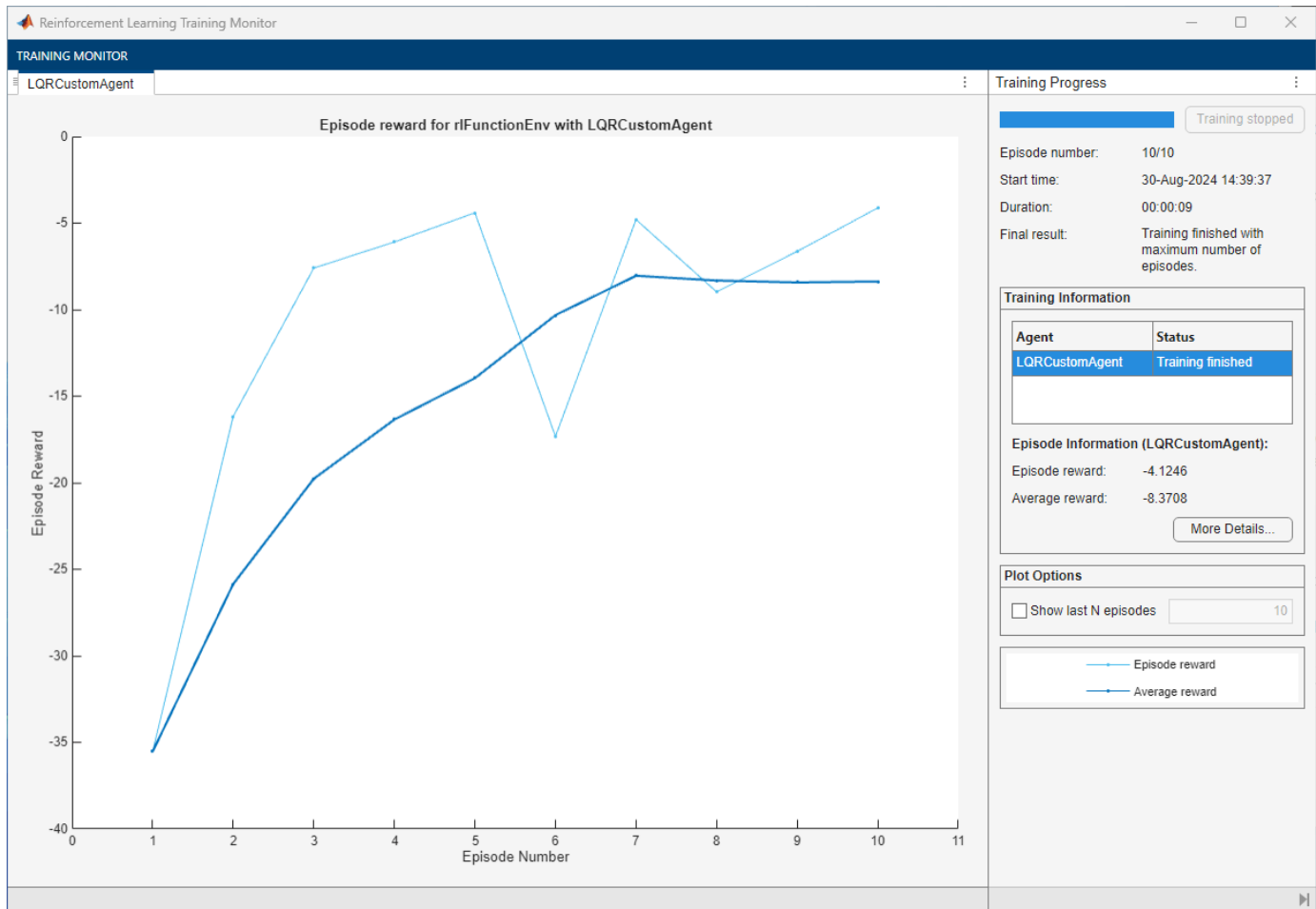
- Run each training episode for a maximum of 10 episodes, with each episode lasting a maximum of 50 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable command line display (set the `Verbose` option).

For more information on training options, see `rlTrainingOptions`.

```
trainingOpts = rlTrainingOptions( ...
    MaxEpisodes=10, ...
    MaxStepsPerEpisode=50, ...
    Verbose=false, ...
    Plots="training-progress");
```

Train the agent using the `train` function.

```
trainingStats = train(agent,env,trainingOpts);
```



Simulate Agent and Compare with Optimal Solution

To validate the performance of the trained agent, simulate it within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=20);
experience = sim(env,agent,simOptions);
totalReward = sum(experience.Reward)
```

```
totalReward =
-30.6482
```

You can compute the optimal solution for the LQR problem using the `dlqr` function.

```
[Koptimal,P] = dlqr(A,B,Q,R);
```

The reward that the optimal control gets is $J_{\text{optimal}} = -x_0'Px_0$.

```
x0 = experience.Observation.obs1.getdatasamples(1);
Joptimal = -x0'*P*x0;
```

Compute the error in the reward between the trained LQR agent and the optimal LQR solution.

```
rewardError = totalReward - Joptimal
```

```
rewardError =
5.0439e-07
```

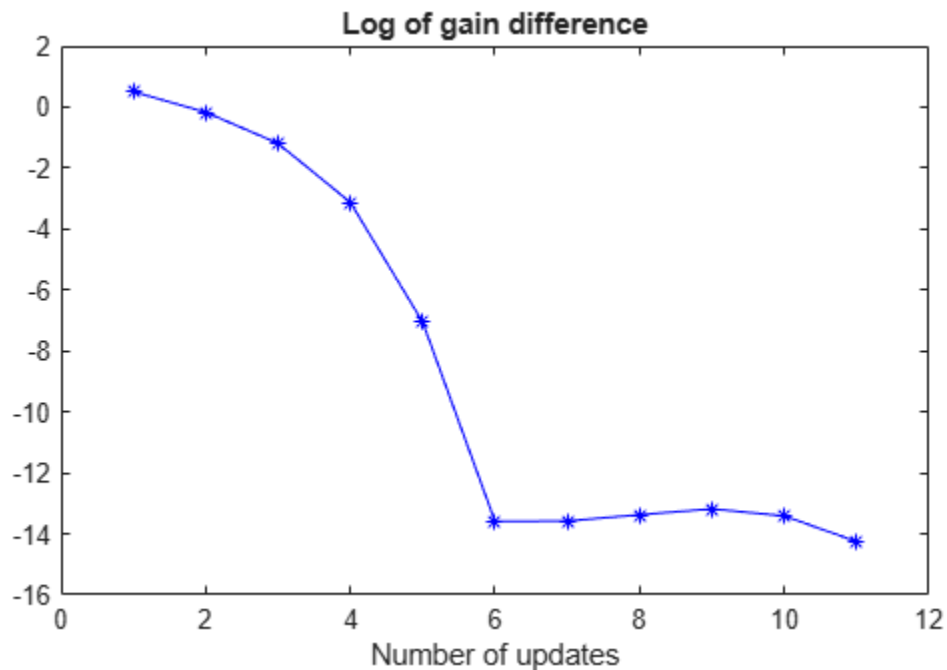
View the history of the norm of the difference between the gains between the trained LQR agent and the optimal LQR solution.

```
% Number of gain updates
len = agent.KUpdate;

% Initialize error vector
err = zeros(len,1);

% Fill elements
for i = 1:len
    err(i) = norm(agent.KBuffer{i}-Koptimal);
end

% Plot logarithm of the error vector
plot(log10(err),'b*-.')
title("Log of gain difference")
xlabel("Number of updates")
```



Compute the norm of final error for the feedback gain.

```
gainError = norm(agent.K - Koptimal)

gainError =
5.5807e-15
```

Overall, the trained agent finds a solution that is very close to the true optimal LQR solution.

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

References

[1] Bradtke, S., "Reinforcement Learning Applied to Linear Quadratic Regulation," *Advances in Neural Information Processing Systems* (1992), Vol. 5.

[2] Asri, S., and Luis Rodrigues. "Data-Driven LQR using Reinforcement Learning and Quadratic Neural Networks." *arXiv preprint arXiv:2311.10235* (2023).

[3] Lewis, Frank L., Draguna Vrabie, and Kyriakos G. Vamvoudakis. "Reinforcement learning and feedback control: Using natural decision methods to design optimal adaptive controllers." *IEEE Control Systems Magazine* 32, no. 6 (2012): 76-105.

See Also

Functions

`train` | `sim` | `lqr`

Objects

`rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- "Create and Train Custom PG Agent" on page 5-334
- "Compare DDPG Agent to LQR Controller" on page 7-285
- "Create Custom Environment Using Step and Reset Functions" on page 2-94
- "Train Reinforcement Learning Policy Using Custom Training Loop" on page 5-273

More About

- "Train Reinforcement Learning Agents" on page 5-3
- "Create Custom Reinforcement Learning Agents" on page 3-92

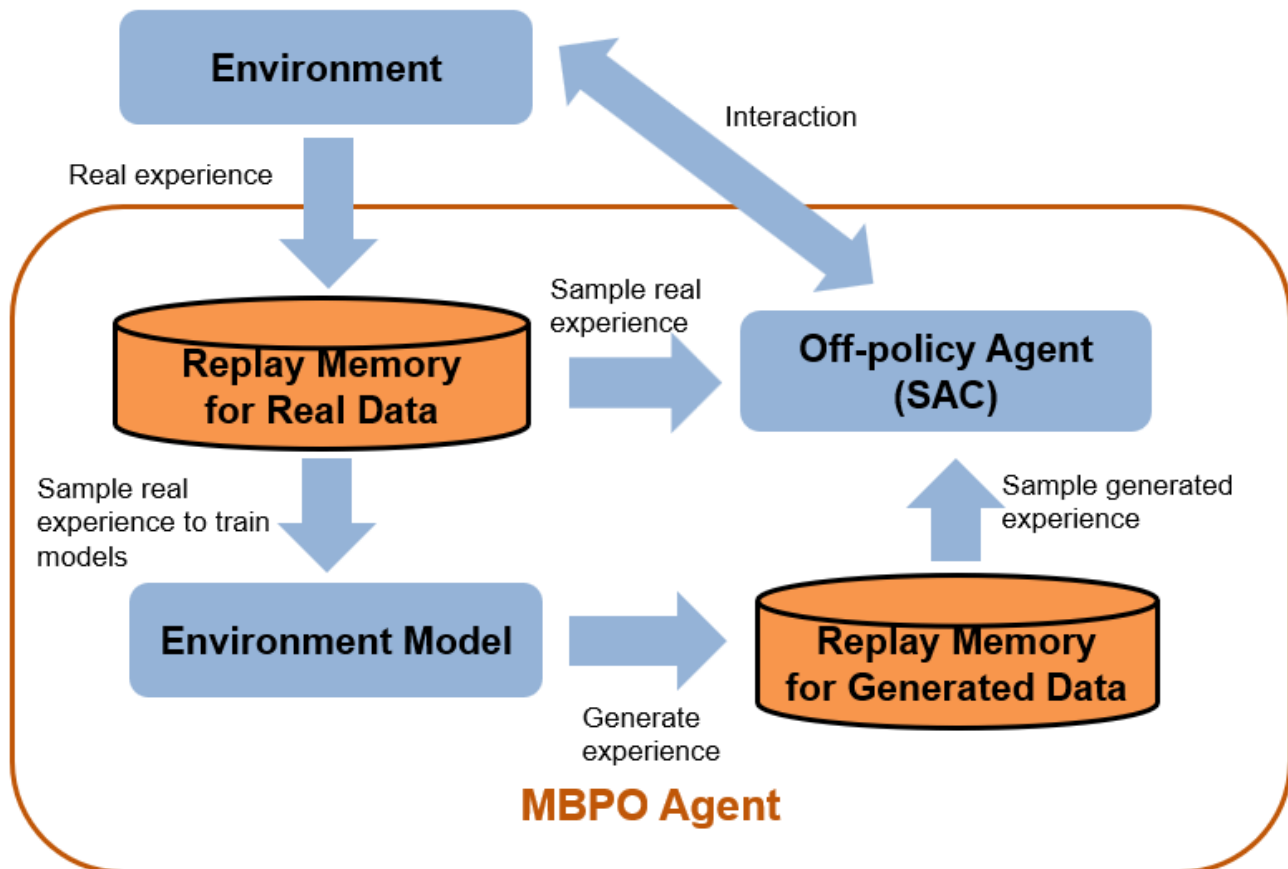
Train MBPO Agent to Balance Continuous Cart-Pole System

This example shows how to train a model-based policy optimization (MBPO) agent to balance a continuous action space cart-pole system modeled in MATLAB®. For more information on MBPO agents, see “Model-Based Policy Optimization (MBPO) Agent” on page 3-86.

MBPO agents use an environment model to generate more experiences while training a base agent. In this example, the base agent is a soft actor-critic (SAC) agent.

The built-in MBPO agent is based on a model-based policy optimization algorithm in [1]. The original MBPO algorithm trains an ensemble of stochastic models. In contrast, this example trains an ensemble of deterministic models. For an example in which an MBPO agent is implemented using a custom training loop, see “Model-Based Reinforcement Learning Using Custom Training Loop” on page 5-368.

The following figure summarizes the algorithm used in this example. During training the MBPO agent collects real experiences resulting from interactions with the environment. The MBPO agent uses these experiences to train its internal environment model. Then, it uses this model to generate experiences without interacting with the actual environment. Finally, the MBPO agent uses the real experiences and generated experiences to train the SAC base agent.



Fix Random Number Stream for Reproducibility

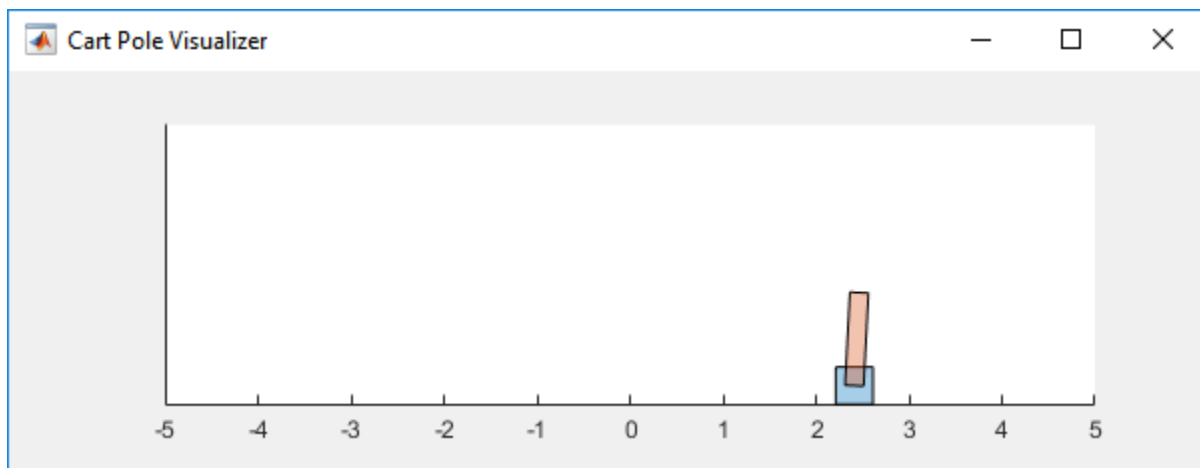
The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Cart-Pole MATLAB Environment

For this example, the reinforcement learning environment is a pole attached to an unactuated revolutionary joint on a cart. The cart has an actuated prismatic joint connected to a one-dimensional frictionless track. The training goal in this environment is to balance the pole by applying forces (actions) to the prismatic joint.



For this environment:

- The balanced, upright pendulum position is 0 radians and the downward hanging pendulum position is π radians.
- The pendulum starts upright with an initial angle between -0.05 radians and 0.05 radians.
- The force action signal from the agent to the environment is from -10 N to 10 N.
- The observations from the environment are the position and velocity of the cart, the pendulum angle (clockwise-positive), and the pendulum angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of +0.5 is provided for every time-step that the pole remains upright. An additional reward is provided based on the distance between the cart and the origin. A penalty of -50 is applied when the pendulum falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create a predefined environment object for the cart-pole system.

```
env = rlPredefinedEnv("CartPole-Continuous");
```

The object has a continuous action space where the agent can apply one force value ranging from -10 N to 10 N.

Obtain the observation and action specifications from the environment object.

```
obsInfo = getObservationInfo(env);  
numObservations = obsInfo.Dimension(1);  
actInfo = getActionInfo(env);
```

Create MBPO Agent

An MBPO agent decides which action to take given observations using a base off-policy agent. The MBPO agent trains both the base agent and an environmental model. The environmental model consists of transition functions, a reward function, and an is-done function. This model is used to create more samples without interacting with an environment. This example uses the following steps to construct an MBPO agent.

- 1 Define model-free off-policy agent.
- 2 Define transition models.
- 3 Define reward model.
- 4 Define is-done model.
- 5 Create neural network environment.
- 6 Create MBPO agent.

1. Define Model-Free Off-Policy Agent

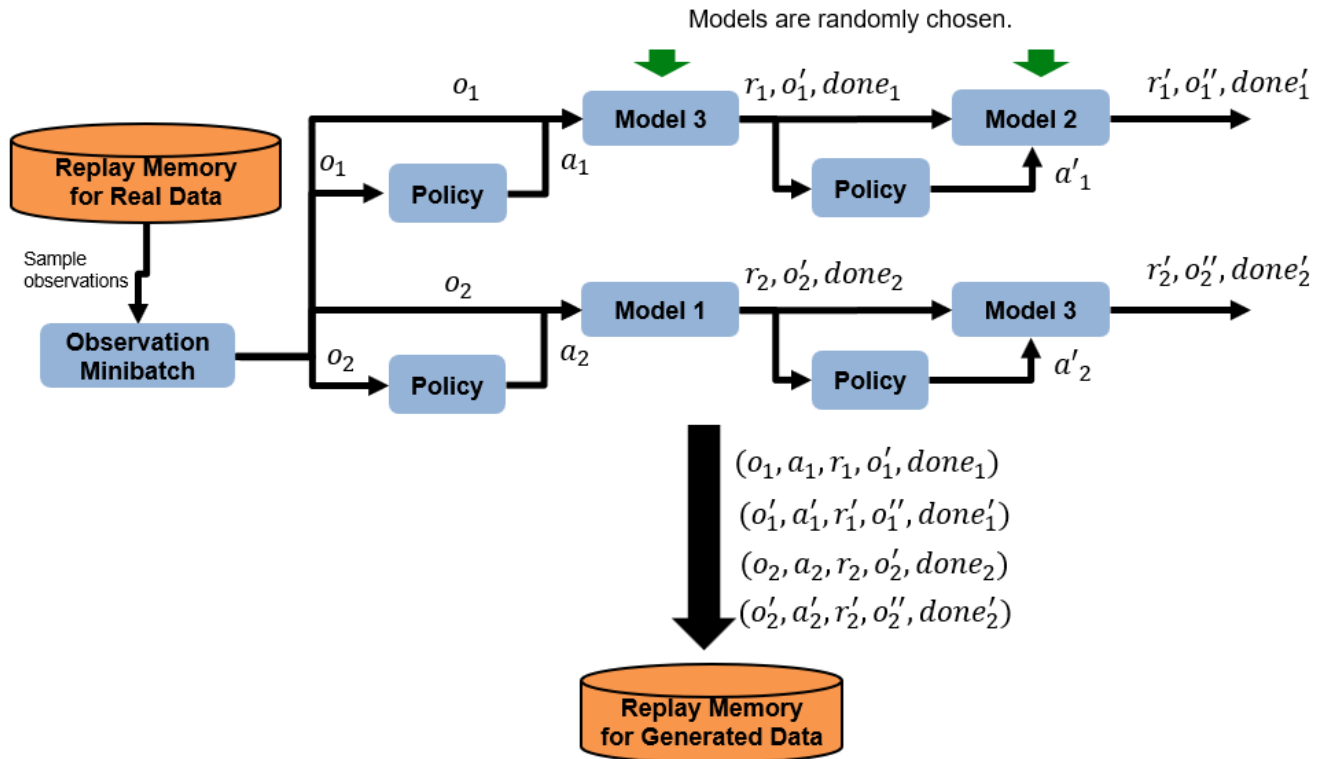
Create a SAC base agent with a default network structure. For more information on SAC agents, see “Soft Actor-Critic (SAC) Agent” on page 3-78. For an environment with a continuous action space, you can also use a DDPG or TD3 base agent. For discrete environments, you can use a DQN base agent.

```
agentOpts = rlSACAgentOptions;  
agentOpts.MinibatchSize = 256;  
agentOpts.ExperienceBufferLength = 1e6;  
agentOpts.NumEpoch = 1;  
initOpts = rlAgentInitializationOptions(NumHiddenUnit=128);  
baseagent = rlSACAgent(obsInfo,actInfo,initOpts,agentOpts);  
  
baseagent.AgentOptions.ActorOptimizerOptions.LearnRate = 5e-4;  
baseagent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
baseagent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 5e-4;  
baseagent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
baseagent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 5e-4;  
baseagent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

2. Define Transition Models

To model the environment, an MBPO agent trains one or more transition models. To model an environment effectively, you must consider two kinds of uncertainty: statistical uncertainty and modeling uncertainty. A stochastic transition function can model the statistical uncertainty better than a deterministic transition function. In this example, since the cart-pole environment is deterministic, you use deterministic transition functions.

It is challenging to have a perfect model, and a trained model usually has modeling uncertainty. One common approach to overcoming modeling uncertainty is to use multiple transition models. The original MBPO paper uses seven models [1]. For this example, to reduce computational cost, you use three models. The MBPO agent generates experiences using all three transition models. The following figure shows how an ensemble of transition models generates samples without interacting with the environment. In this figure, the models generate two trajectories with horizon = 2.



Create three deterministic transition functions. To do so, create a deep neural network using the `createDeterministicTransitionNetwork` helper function. Then, use the neural network to create an `rlContinuousDeterministicTransitionFunction` object. When creating a transition function object, you must specify the action and observation input/output names for the neural network.

```
net1 = createDeterministicTransitionNetwork(4,1);
transitionFcn1 = rlContinuousDeterministicTransitionFunction(net1, ...
    obsInfo, ...
    actInfo, ...
    ObservationInputNames="state", ...
    ActionInputNames="action", ...
    NextObservationOutputNames="nextObservation");

net2 = createDeterministicTransitionNetwork(4,1);
transitionFcn2 = rlContinuousDeterministicTransitionFunction(net2, ...
    obsInfo, ...
    actInfo, ...
    ObservationInputNames="state", ...
    ActionInputNames="action", ...
    NextObservationOutputNames="nextObservation");
```

```
net3 = createDeterministicTransitionNetwork(4,1);
transitionFcn3 = rlContinuousDeterministicTransitionFunction(net3, ...
    obsInfo, ...
    actInfo, ...
    ObservationInputNames="state", ...
    ActionInputNames="action", ...
    NextObservationOutputNames="nextObservation");
```

3. Define Reward Model

An MBPO agent also contains a reward model for the environment. If you know a ground-truth reward function, you can specify it using a custom function. In this example, the ground-truth reward function is defined in the `cartPoleRewardFunction` helper function. To use this reward function set `useGroundTruthReward` to `true`.

You can also specify a neural-network-based reward function that the MBPO agent can train. In this example, you can use such a reward function by setting `useGroundTruthReward` to `false`. The deep neural network for the reward function is defined in the `createRewardNetworkActionNextObs` helper function. To define an is-done function using the neural network, create an `rlContinuousDeterministicRewardFunction` object.

```
useGroundTruthReward = true;
if useGroundTruthReward
    rewardFcn = @cartPoleRewardFunction;
else
    % This neural network uses action and next observation as inputs.
    rewardnet = createRewardNetworkActionNextObs(4,1);
    rewardFcn = rlContinuousDeterministicRewardFunction(rewardnet, ...
        obsInfo, ...
        actInfo, ...
        ActionInputNames="action", ...
        NextObservationInputNames="nextState");
end
```

4. Define Is-Done Model

An MBPO agent also contains an is-done model for computing the termination signal for the environment. If you know a ground-truth termination signal, you can specify it using a custom function. In this example, the ground-truth termination signal is defined in the `cartPoleIsDoneFunction` helper function. To use this reward function set `useGroundTruthIsDone` to `true`.

You can also specify a neural-network-based is-done function that the MBPO agent can train. In this example, you can use such an is-done function by setting `useGroundTruthIsDone` to `false`. The deep neural network for the is-done function is defined in the `createIsDoneNetwork` helper function. To define an is-done function using the neural network, create an `rlIsDoneFunction` object.

```
useGroundTruthIsDone = true;
if useGroundTruthIsDone
    isdoneFcn = @cartPoleIsDoneFunction;
else
    % This neural network uses only next observation as inputs.
    isdoneNet = createIsDoneNetwork(4);
    isdoneFcn = rlIsDoneFunction(isdoneNet, ...
        obsInfo, ...
        actInfo, ...
```

```

        NextObservationInputNames="nextState");
end

```

5. Create Neural Network Environment

Define a neural network environment using the transition, reward, and is-done functions. To do so, create an `rlNeuralNetworkEnvironment` object.

```

generativeEnv = rlNeuralNetworkEnvironment( ...
    obsInfo, ...
    actInfo, ...
    [transitionFcn1,transitionFcn2,transitionFcn3], ...
    rewardFcn, ...
    isdoneFcn);

% Reset model environment.
reset(generativeEnv);

```

6. Create MBPO Agent

Define an MBPO agent using the base off-policy agent and environment model. To do so, first create an MBPO agent options object.

```
MBPOAgentOpts = rlMBPOAgentOptions;
```

Specify options for training the environment model. Train the model for 1 epoch at the beginning of each episode and use 15 mini-batches of size 512.

```

MBPOAgentOpts.NumEpochForTrainingModel = 1;
MBPOAgentOpts.NumMiniBatches = 15;
MBPOAgentOpts.MinibatchSize = 512;

```

Specify the ratio of real and generated experience used to train the base SAC agent. For this example, 20% of samples are from the real experience buffer and 80% of samples are from model experience buffer.

```
MBPOAgentOpts.RealSampleRatio = 0.2;
```

Specify options for generating samples using the environment model.

- Generate 20000 trajectories at the beginning of each epoch.
- Use a piecewise roll-out horizon schedule, which increases the horizon length gradually.
- Increase the horizon length every 100 epochs.
- Use an initial horizon length of 1.
- Use a maximum horizon length of 2.

```

MBPOAgentOpts.ModelRolloutOptions.NumRollout = 20000;
MBPOAgentOpts.ModelRolloutOptions.HorizonUpdateSchedule = "piecewise";
MBPOAgentOpts.ModelRolloutOptions.HorizonUpdateFrequency = 100;
MBPOAgentOpts.ModelRolloutOptions.Horizon = 1;
MBPOAgentOpts.ModelRolloutOptions.HorizonMax = 2;

```

Specify the size of the model's experience buffer.

```
MBPOAgentOpts.ModelExperienceBufferLength = 120000;
```

Specify optimizer options for training the transition models. Use the same optimizer options for all three transition models.

```
transitionOptimizerOptions1 = rlOptimizerOptions( ...  
    LearnRate=5e-4, ...  
    GradientThreshold=1.0);  
transitionOptimizerOptions2 = rlOptimizerOptions( ...  
    LearnRate=5e-4, ...  
    GradientThreshold=1.0);  
transitionOptimizerOptions3 = rlOptimizerOptions( ...  
    LearnRate=5e-4, ...  
    GradientThreshold=1.0);  
MBPOAgentOpts.TransitionOptimizerOptions = ...  
    [transitionOptimizerOptions1, ...  
     transitionOptimizerOptions2, ...  
     transitionOptimizerOptions3];
```

Specify optimizer options for training the reward model. If you use a custom ground-truth reward function, the agent ignores these options.

```
rewardOptimizerOptions = rlOptimizerOptions( ...  
    LearnRate=5e-4, ...  
    GradientThreshold=1.0);  
MBPOAgentOpts.RewardOptimizerOptions = rewardOptimizerOptions;
```

Specify optimizer options for training the is-done model. If you use a custom ground-truth reward function, the agent ignores these options.

```
isdoneOptimizerOptions = rlOptimizerOptions( ...  
    LearnRate=5e-4, ...  
    GradientThreshold=1.0);  
MBPOAgentOpts.IsDoneOptimizerOptions = isdoneOptimizerOptions;
```

Create the MBPO agent, specifying the base agent, environment model, and options.

```
agent = rlMBPOAgent(baseagent,generativeEnv,MBPOAgentOpts);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 500 episodes, with each episode lasting a maximum of 500 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Save the agent when the average episode reward is greater than or equal to 470.
- Stop the training when an evaluation statistic reaches 470. At this point, the agent can balance the pendulum in the upright position.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=500, ...  
    MaxStepsPerEpisode=500, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EvaluationStatistic", ...
```

```

StopTrainingValue=470, ...
ScoreAveragingWindowLength=5, ...
SaveAgentCriteria="EpisodeReward", ...
SaveAgentValue=470);

```

Create an evaluator object to evaluate the agent using 5 evaluation episodes at every 25 training episodes.

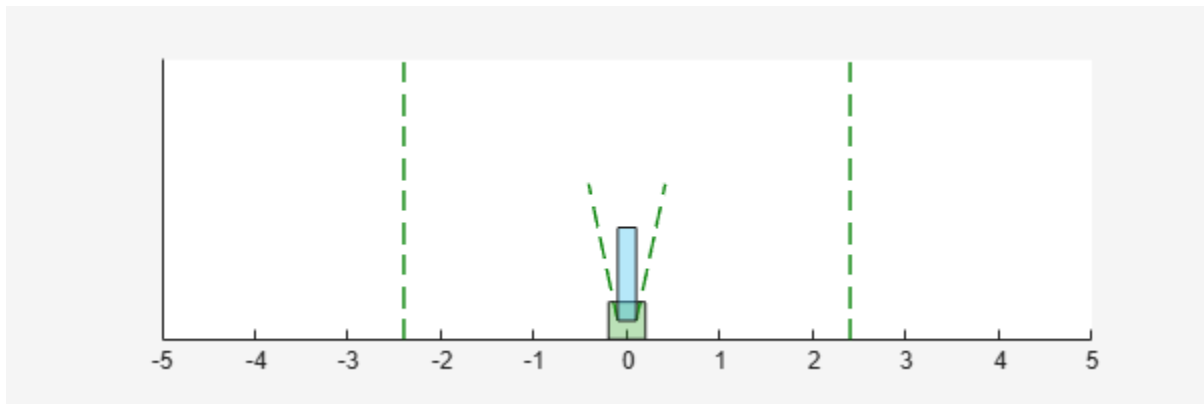
```

evl = rlEvaluator( ...
    NumEpisodes=5, ...
    EvaluationFrequency=25, ...
    RandomSeeds=101:105);

```

You can visualize the cart-pole system by using the `plot` function during training or simulation.

```
plot(env)
```



Fix the random generator seed for reproducibility.

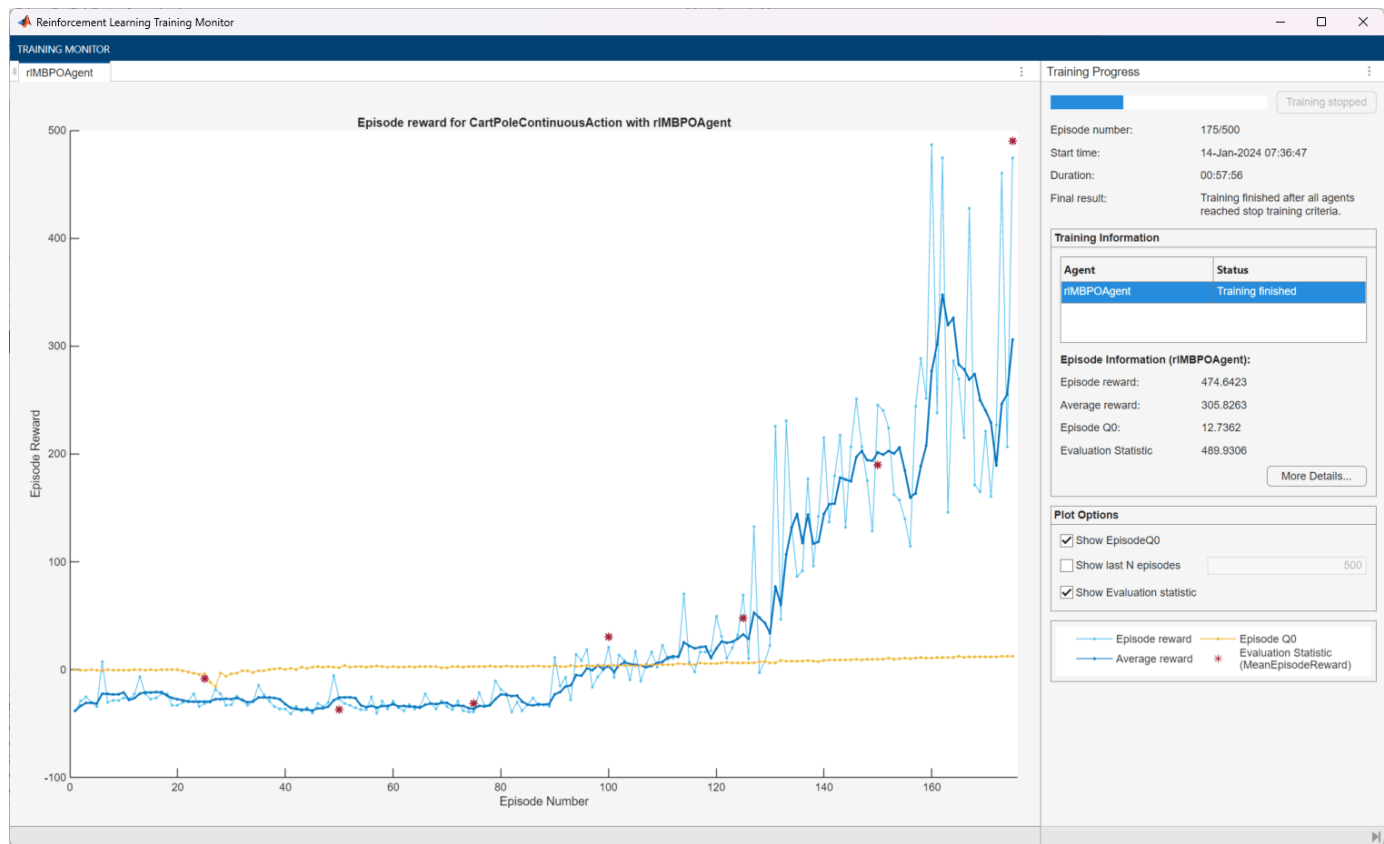
```
rng(0);
```

Train the agent using the `train` function. Training this agent is a computationally-intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("MATLABCartpoleMBPO.mat","agent");
end

```



Simulate MBPO Agent

To validate the performance of the trained agent, simulate it within the cart-pole environment. To make sure you use deterministic actions during the simulation, set the `UseExplorationPolicy` agent property to `false`. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
rng(1)

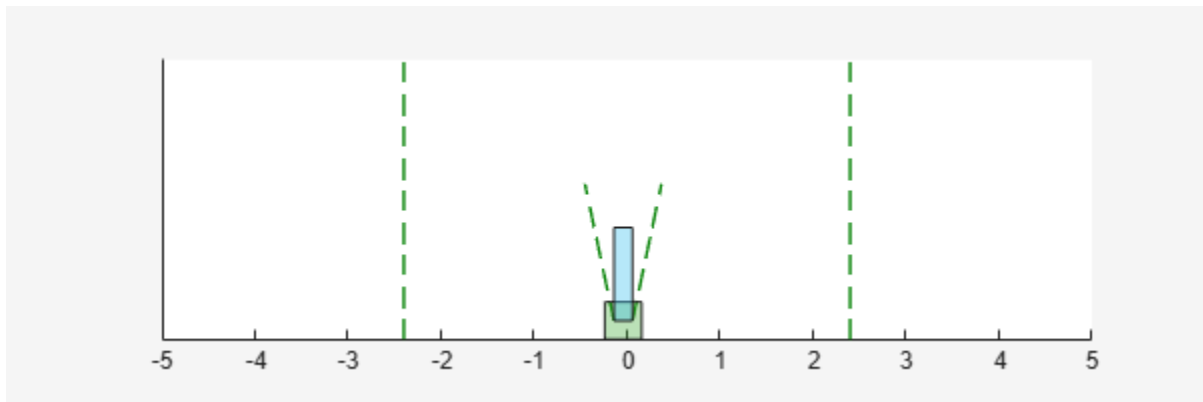
% Make sure exploration is disabled during simulation.
agent.UseExplorationPolicy = false;

simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
totalReward_MBPO = sum(experience.Reward)

totalReward_MBPO =
497.5506
```

Instead of simulating the MBPO agent, you can simulate the base agent. If you use the same random seed, you get the same result as simulating the MBPO agent.

```
rng(1)
experience = sim(env,agent.BaseAgent,simOptions);
```

```
totalReward_SAC = sum(experience.Reward)
```

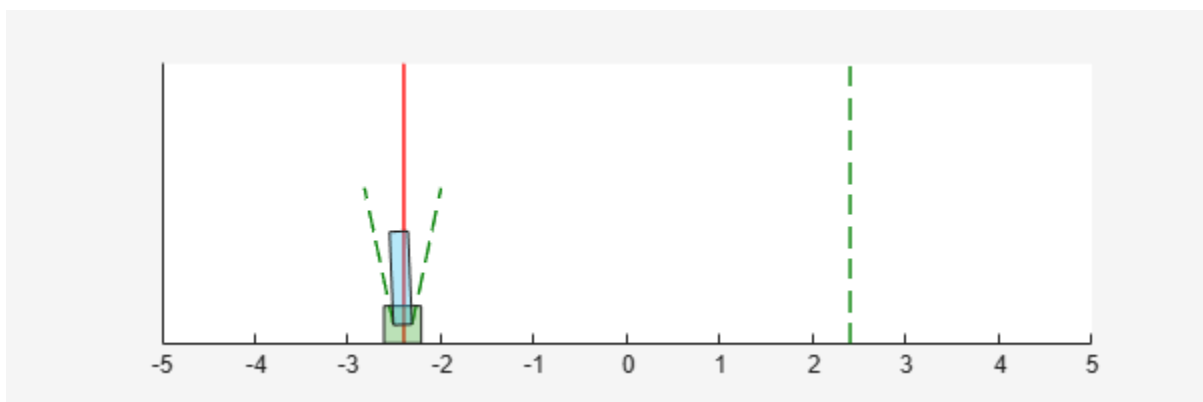
```
totalReward_SAC =  
497.5506
```

Evaluate Learned Environment Model

To validate the trained environment transition models, you can check whether they are able to correctly predict the next observations. Similarly, you can validate the performance of the reward and is-done functions. To make a prediction based on the environment model, use `step`.

Collect data for learned model evaluation.

```
rng(1)  
  
% Enable exploration during sim to create  
% diverse data for model evaluation  
agent.UseExplorationPolicy = true;  
  
simOptions = rlSimulationOptions(MaxSteps=500);  
experience = sim(env,agent,simOptions);
```



For this example, evaluate the performance of the first transition model.

```
agent.EnvModel.TransitionModelNum = 1;
```

For each simulation step, extract the actual next observation.

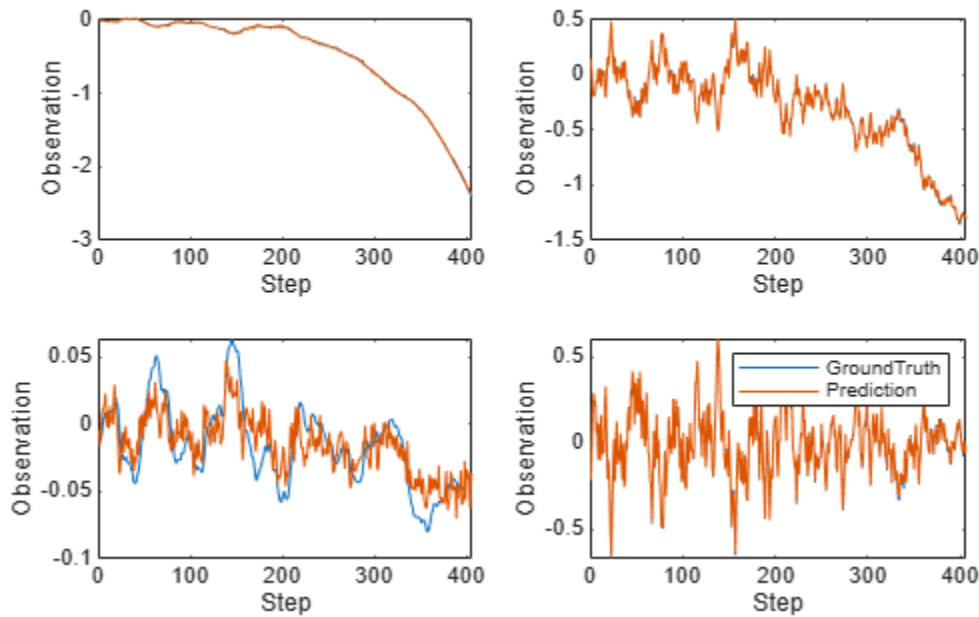
```
numSteps = length(experience.Reward.Data);
nextObsPrediction = zeros(4,1,numSteps);
rewardPrediction = zeros(1,numSteps);
isdonePrediction = zeros(1,numSteps);
nextObsGroundTruth = zeros(4,1,numSteps);
rewardGroundTruth = zeros(1,numSteps);
isdoneGroundTruth = zeros(1,numSteps);
for stepCt = 1:numSteps
    % Extract the actual next observation, reward, and is-done value.
    nextObsGroundTruth(:, :, stepCt) = ...
        experience.Observation.CartPoleStates.Data(:, :, stepCt+1);
    rewardGroundTruth(:, stepCt) = experience.Reward.Data(stepCt);
    isdoneGroundTruth(:, stepCt) = experience.IsDone.Data(stepCt);

    % Predict the next observation, reward, and is-done value
    % using the environment model.
    obs = experience.Observation.CartPoleStates.Data(:, :, stepCt);
    agent.EnvModel.Observation = {obs};
    action = experience.Action.CartPoleAction.Data(:, :, stepCt);
    [nextObs, reward, isdone] = step(agent.EnvModel, {action});

    nextObsPrediction(:, :, stepCt) = nextObs{1};
    rewardPrediction(:, stepCt) = reward;
    isdonePrediction(:, stepCt) = isdone;
end
```

Plot the ground truth and prediction of each dimension of the observations.

```
figure
for obsDimensionIndex = 1:4
    subplot(2,2,obsDimensionIndex)
    plot(reshape(nextObsGroundTruth(obsDimensionIndex, :, :), 1, numSteps))
    hold on
    plot(reshape(nextObsPrediction(obsDimensionIndex, :, :), 1, numSteps))
    hold off
    xlabel("Step")
    ylabel("Observation")
    if obsDimensionIndex == 4
        legend("GroundTruth", "Prediction", "Location", "northeast")
    end
end
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

[1] Janner, Michael, Justin Fu, Marvin Zhang, and Sergey Levine. “When to Trust Your Model: Model-Based Policy Optimization.” In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 12519–30. 1122. Red Hook, NY, USA: Curran Associates Inc., 2019.

See Also

Functions

`train` | `sim`

Objects

`rlMBPOAgent` | `rlNeuralNetworkEnvironment` | `rlMBPOAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions` | `rlSACAgent`

Related Examples

- “Model-Based Reinforcement Learning Using Custom Training Loop” on page 5-368

More About

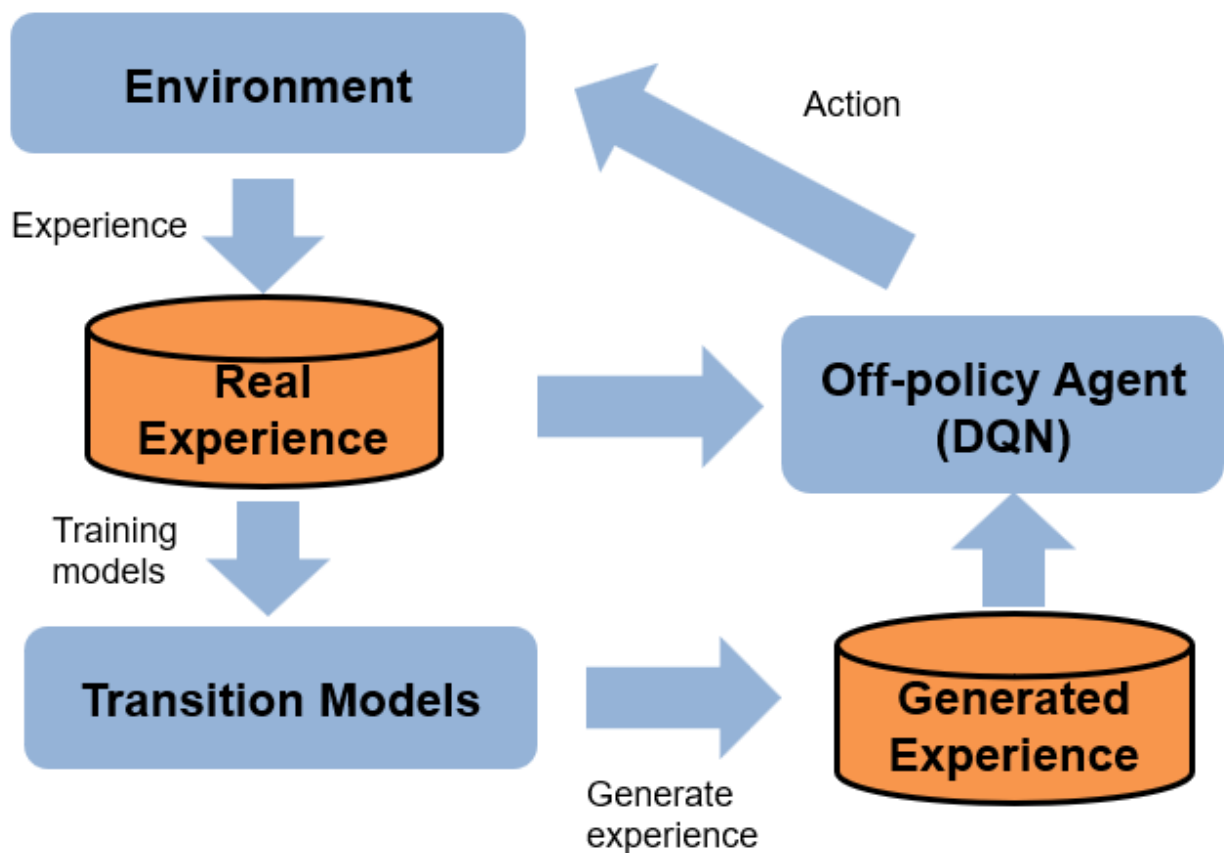
- “Use Predefined Control System Environments” on page 2-42
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Model-Based Policy Optimization (MBPO) Agent” on page 3-86

Model-Based Reinforcement Learning Using Custom Training Loop

This example shows how to define a custom training loop for a model-based reinforcement learning (MBRL) algorithm. You can use this workflow to train an MBRL policy with your custom training algorithm using policy and value function approximators from Reinforcement Learning Toolbox™ software.

For an example on how to use the built in model-based policy optimization (MBPO) agent, see “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356. For an overview of built-in MBPO agents, see “Model-Based Policy Optimization (MBPO) Agent” on page 3-86.

In this example, you use transition models to generate more experiences while training a custom DQN [2] agent in a cart-pole environment. The algorithm used in this example is based on an MBPO algorithm [1]. The original MBPO algorithm trains an ensemble of stochastic models and a soft actor-critic (SAC) agent in tasks with continuous actions. In contrast, this example trains three deterministic models and a DQN agent in a task with discrete actions. The following figure summarizes the algorithm used in this example.



The agent generates real experiences by interacting with the environment. These experiences are used to train a set of transition models, which are used to generate additional experiences. The training algorithm then uses both the real and generated experiences to update the agent policy.

Create Environment

For this example, a reinforcement learning policy is trained in a discrete cart-pole environment. The objective in this environment is to balance the pole by applying forces (actions) on the cart. Create the environment using the `rlPredefinedEnv` function. Fix the random generator seed for reproducibility. For more information on this environment, see “Use Predefined Control System Environments” on page 2-42.

```
clear
clc
rngSeed = 1;
rng(rngSeed);
env = rlPredefinedEnv("CartPole-Discrete");
```

Extract the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Obtain the number of observations (`numObservations`) and actions (`numActions`).

```
numObservations = obsInfo.Dimension(1);
% Number of discrete actions, -10 or 10
numActions = numel(actInfo.Elements);
numContinuousActions = 1; % force
```

Critic Construction

DQN agents use a parameterized Q-value function approximator to estimate the value of the policy. Because DQN agents have a discrete action space, you have the option to create a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic.

A vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible actions. The value of each output element represents the expected discounted cumulative long-term reward when an agent starts from the state corresponding to the given observation and executes the action corresponding to the element number (and follows the given policy afterwards).

To model the parameterized Q-value function within the critic, use a neural network. For more information about the layers, see `fullyConnectedLayer`, and `reluLayer`.

```
qNetwork = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(24)
    reluLayer
    fullyConnectedLayer(24)
    reluLayer
    fullyConnectedLayer(length(actInfo.Elements))
];
```

Convert to `dlnetwork` object.

```
qNetwork = dlnetwork(qNetwork);
```

Display the number of learnable parameters.

```
summary(qNetwork)
```

```
  Initialized: true
```

```
  Number of learnables: 770
```

```
  Inputs:
```

```
    1  'input'    4 features
```

Create the critic using the specified neural network and options. For more information, `rlVectorQValueFunction`.

```
critic = rlVectorQValueFunction(qNetwork,obsInfo,actInfo);
```

Create optimizer objects for updating the critic. For more information, see `rlOptimizerOptions`.

```
optimizerOpt = rlOptimizerOptions( ...
```

```
    LearnRate=1e-3, ...
```

```
    GradientThreshold=1);
```

```
criticOptimizer = rlOptimizer(optimizerOpt);
```

Create a max Q value policy.

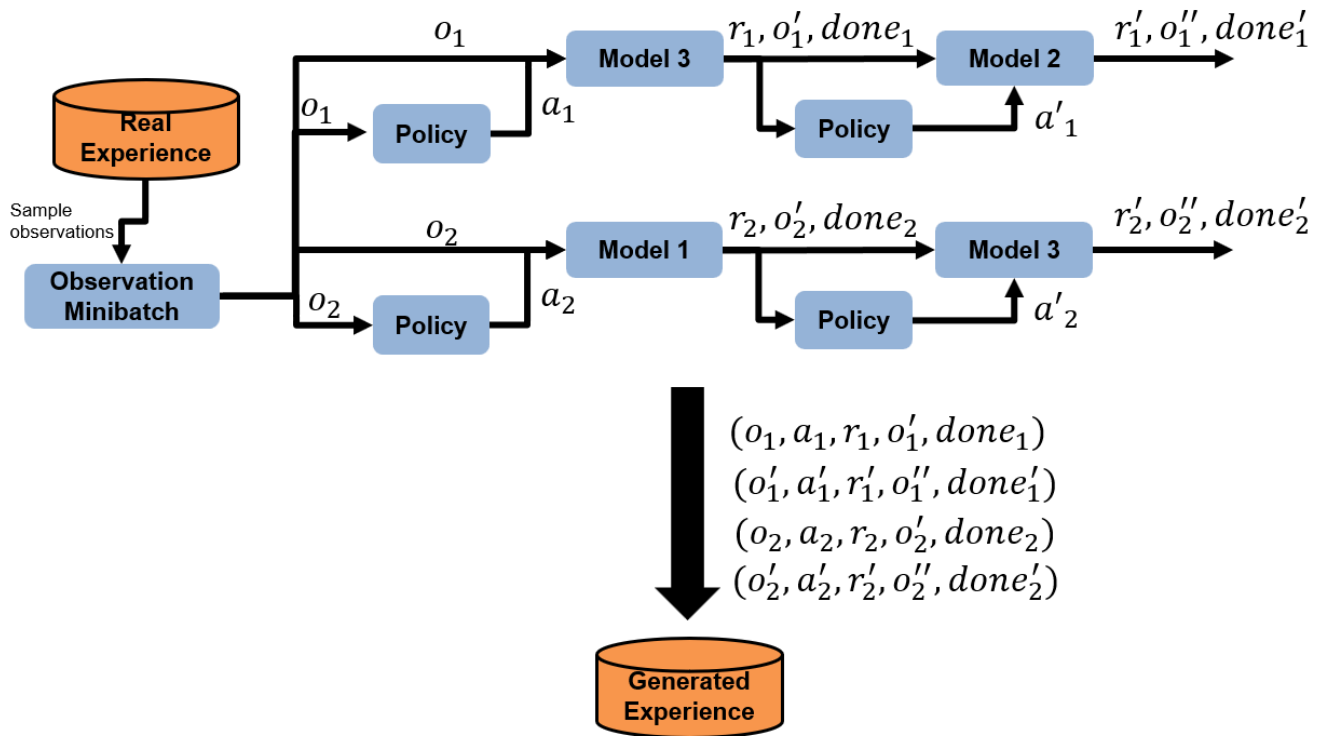
```
policy = rlMaxQPolicy(critic);
```

Create Transition Models

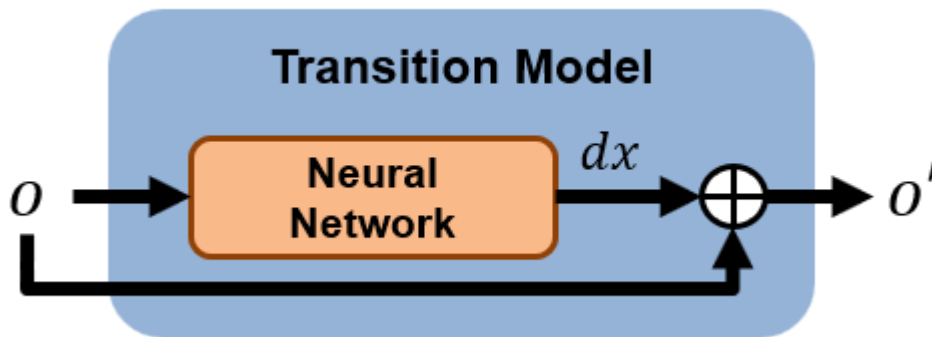
Model-based reinforcement learning uses transition models of the environment. The model usually consists of a transition function, a reward function, and a terminal state function.

- The transition function predicts the next observation given the current observation and the action.
- The reward function predicts the reward given the current observation, the action, and the next observation.
- The terminal state function predicts the terminal state given the observation.

As shown in the following figure, this example uses three transition functions as an ensemble of transition models to generate samples without interacting with the environment. The true reward function and the true terminal state function are given in this example.



Define three neural networks for transition models. The neural network predicts the difference between the next observation and the current observation.



```
numModels = 3;
transitionNetwork1 = ...
    createTransitionNetwork(numObservations, numContinuousActions);

transitionNetwork2 = ...
    createTransitionNetwork(numObservations, numContinuousActions);

transitionNetwork3 = ...
    createTransitionNetwork(numObservations, numContinuousActions);

transitionNetworkVector = [transitionNetwork1, ...
```

```
transitionNetwork2, ...  
transitionNetwork3];
```

Create Experience Buffers

Create an experience buffer for storing agent experiences (observation, action, next observation, reward, and isDone).

```
myBuffer.bufferSize = 1e5;  
myBuffer.bufferIndex = 0;  
myBuffer.currentBufferLength = 0;  
myBuffer.observation = zeros(numObservations,myBuffer.bufferSize);  
myBuffer.nextObservation = ...  
    zeros(numObservations,myBuffer.bufferSize);  
myBuffer.action = ...  
    zeros(numContinuousActions,1,myBuffer.bufferSize);  
myBuffer.reward = zeros(1,myBuffer.bufferSize);  
myBuffer.isDone = zeros(1,myBuffer.bufferSize);
```

Create a model experience buffer for storing the experiences generated by the models.

```
myModelBuffer.bufferSize = 1e5;  
myModelBuffer.bufferIndex = 0;  
myModelBuffer.currentBufferLength = 0;  
myModelBuffer.observation = ...  
    zeros(numObservations,myModelBuffer.bufferSize);  
myModelBuffer.nextObservation = ...  
    zeros(numObservations,myModelBuffer.bufferSize);  
myModelBuffer.action = ...  
    zeros(numContinuousActions,myModelBuffer.bufferSize);  
myModelBuffer.reward = zeros(1,myModelBuffer.bufferSize);  
myModelBuffer.isDone = zeros(1,myModelBuffer.bufferSize);
```

Configure Training

Configure the training to use the following options.

- Maximum number of training episodes — 250
- Maximum steps per training episode — 500
- Discount factor — 0.99
- Training termination condition — Average reward across 10 episodes reaches the value of 480

```
numEpisodes = 250;  
maxStepsPerEpisode = 500;  
discountFactor = 0.99;  
aveWindowSize = 10;  
trainingTerminationValue = 480;
```

Configure the model options.

- Train transition models only after 2000 samples are collected.
- Train the models using all experiences in the real experience buffer in each episode. Use a mini-batch size of 256.
- The models generate trajectories with a length of 2 at the beginning of each episode.
- The number of generated trajectories is numGenerateSampleIteration x numModels x miniBatchSize = 20 x 3 x 256 = 15360.

- Use the same epsilon-greedy parameters as the DQN agent, except for the minimum epsilon value.
- Use a minimum epsilon value of 0.1, which is higher than the value used for interacting with the environment. Doing so allows the model to generate more diverse data.

```
warmStartSamples = 2000;
numEpochs = 1;
miniBatchSize = 256;
horizonLength = 2;
epsilonMinModel = 0.1;
numGenerateSampleIteration = 20;
sampleGenerationOptions.horizonLength = horizonLength;
sampleGenerationOptions.numGenerateSampleIteration = ...
    numGenerateSampleIteration;
sampleGenerationOptions.miniBatchSize = miniBatchSize;
sampleGenerationOptions.numObservations = numObservations;
sampleGenerationOptions.epsilonMinModel = epsilonMinModel;

% optimizer options
velocity1 = [];
velocity2 = [];
velocity3 = [];
decay = 0.01;
momentum = 0.9;
learnRate = 0.0005;
```

Configure the DQN training options.

- Use the epsilon greedy algorithm with an initial epsilon value is 1, a minimum value of 0.01, and a decay rate of 0.005.
- Update the target network every 4 steps.
- Set the ratio of the real experiences to generated experiences to 0.2:0.8 by setting `RealRatio` to 0.2. Setting `RealRatio` to 1.0 is the same as the model-free DQN.
- Take 5 gradient steps at each environment step.

```
epsilon = 1;
epsilonMin = 0.01;
epsilonDecay = 0.005;
targetUpdateFrequency = 4;
realRatio = 0.2; % Set to 1 to run a standard DQN
numGradientSteps = 5;
```

Create a vector for storing the cumulative reward for each training episode.

```
episodeCumulativeRewardVector = [];
```

Create a figure for model training visualization using the `hBuildFigureModel` helper function.

```
[trainingPlotModel, ...
    lineLossTrain1, ...
    lineLossTrain2, ...
    lineLossTrain3, ...
    axModel] = hBuildFigureModel();
```

Create a figure for model validation visualization using the `hBuildFigureModelTest` helper function.

```
[testPlotModel, lineLossTest1, axModelTest] ...  
    = hBuildFigureModelTest();
```

Create a figure for DQN agent training visualization using the `hBuildFigure` helper function.

```
[trainingPlot,lineReward,lineAveReward, ax] = hBuildFigure;
```

Train Agent

Train the agent using a custom training loop. The training loop uses the following algorithm. For each episode:

- 1 Train the transition models.
- 2 Generate experiences using the transition models and store the samples in the model experience buffer.
- 3 Generate a real experience. To do so, generate an action using the policy, apply the action to the environment, and obtain the resulting observation, reward, and is-done values.
- 4 Create a mini-batch by sampling experiences from both the experience buffer and the model experience buffer.
- 5 Compute the target Q value.
- 6 Compute the gradient of the loss function with respect to the critic parameters.
- 7 Update the critic using the computed gradients.
- 8 Update the training visualization.
- 9 Terminate training if the critic is sufficiently trained.

Training the policy is a computationally intensive process. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the policy yourself, set `doTraining` to `true`.

```
doTraning = false;  
if doTraning  
    targetCritic = critic;  
    modelTrainedAtleastOnce = false;  
    totalStepCt = 0;  
    start = tic;  
  
    set(trainingPlotModel,Visible = "on");  
    set(testPlotModel,Visible = "on");  
    set(trainingPlot,Visible = "on");  
  
    for episodeCt = 1:numEpisodes  
        if myBuffer.currentBufferLength > miniBatchSize && ...  
            totalStepCt > warmStartSamples  
            if realRatio < 1.0  
                %-----  
                % 1. Train transition models.  
                %-----  
                % Training three transition models  
                [transitionNetworkVector(1),loss1,velocity1] = ...  
                    trainTransitionModel( ...  
                        transitionNetworkVector(1), ...  
                        myBuffer,velocity1,miniBatchSize, ...  
                        numEpochs,momentum,learnRate);  
                [transitionNetworkVector(2),loss2,velocity2] = ...
```

```

        trainTransitionModel( ...
        transitionNetworkVector(2), ...
        myBuffer,velocity2,miniBatchSize, ...
        numEpochs,momentum,learnRate);
[transitionNetworkVector(3),loss3,velocity3] = ...
    trainTransitionModel( ...
    transitionNetworkVector(3), ...
    myBuffer,velocity3,miniBatchSize, ...
    numEpochs,momentum,learnRate);
modelTrainedAtleastOnce = true;

% Display the training progress
d = duration(0,0,toc(start),"Format",'hh:mm:ss');
addpoints(lineLossTrain1,episodeCt,loss1)
addpoints(lineLossTrain2,episodeCt,loss2)
addpoints(lineLossTrain3,episodeCt,loss3)
legend(axModel,"Model1","Model2","Model3");
title(axModel, ...
    "Model Training Progress - Episode: "...
    + episodeCt + ", Elapsed: " + string(d))
drawnow

%-----
% 2. Generate experience using models.
%-----
% Create numGenerateSampleIteration x
% horizonLength x numModels x miniBatchSize
% ex) 20 x 2 x 3 x 256 = 30720 samples
myModelBuffer = generateSamples(myBuffer, ...
    myModelBuffer, ...
    transitionNetworkVector,policy,actInfo, ...
    epsilon,sampleGenerationOptions);
end
end

%-----
% Interact with environment and train agent.
%-----
% Reset the environment at the start of the episode
observation = reset(env);
episodeReward = zeros(maxStepsPerEpisode,1);
errorPredidction = zeros(maxStepsPerEpisode,1);

for stepCt = 1:maxStepsPerEpisode
    %-----
    % 3. Generate an experience.
    %-----
    totalStepCt = totalStepCt + 1;

    % Compute an action using the policy based on
    % the current observation.
    if rand() < epsilon
        action = actInfo.usample;
    else
        action = getAction(policy,{observation});
    end
    action = action{1};
    % Udpate epsilon

```

```
if totalStepCt > warmStartSamples
    epsilon = max(epsilon*(1-epsilonDecay), ...
                  epsilonMin);
end

% Apply the action to the environment and obtain the
% resulting observation and reward.
[nextObservation,reward,isDone] = step(env,action);

% Check prediction
dx = predict(transitionNetworkVector(1), ...
             dldarray(observation,"CB"),dldarray(action,"CB"));
predictedNextObservation = observation + dx;
errorPrediction(stepCt) = ...
    sqrt(sum((nextObservation - ...
              predictedNextObservation).^2));

% Store the action, observation, reward and is-done
% experience
myBuffer = storeExperience(myBuffer, ...
                           observation, ...
                           action, ...
                           nextObservation,reward,isDone);

episodeReward(stepCt) = reward;
observation = nextObservation;

% Train DQN agent
for gradientCt = 1:numGradientSteps
    if myBuffer.currentBufferLength >= miniBatchSize ...
        && totalStepCt>warmStartSamples
        %-----
        % 4. Sample mini-batch from experience buffers.
        %-----
        [sampledObservation, ...
         sampledAction, ...
         sampledNextObservation, ...
         sampledReward, ...
         sampledIsdone] ...
            = sampleMinibatch( ...
                modelTrainedAtleastOnce, ...
                realRatio, ...
                miniBatchSize, ...
                myBuffer,myModelBuffer);

        %-----
        % 5. Compute target Q value.
        %-----
        % Compute target Q value
        [targetQValues, MaxActionIndices] = ...
            getMaxQValue(targetCritic, ...
                {reshape(sampledNextObservation, ...
                    [numObservations,1,miniBatchSize])});

        % Compute target for nonterminal states
        targetQValues(~logical(sampledIsdone)) = ...
            sampledReward(~logical(sampledIsdone)) + ...
            discountFactor.* ...
    end
end
```

```

        targetQValues(~logical(sampledIsdone));
% Compute target for terminal states
targetQValues(logical(sampledIsdone)) = ...
    sampledReward(logical(sampledIsdone));

lossData.batchSize = miniBatchSize;
lossData.actInfo = actInfo;
lossData.actionBatch = sampledAction;
lossData.targetQValues = targetQValues;

%-----
% 6. Compute gradients.
%-----
criticGradient = ...
    gradient(critic, ...
        @criticLossFunction, ...
        {reshape(sampledObservation, ...
            [numObservations,1,miniBatchSize])}, ...
        lossData);

%-----
% 7. Update the critic network using gradients.
%-----
[critic, criticOptimizer] = update( ...
    criticOptimizer, critic, ...
    criticGradient);

% Update the policy parameters using the critic
% parameters.
policy = setLearnableParameters( ...
    policy, ...
    getLearnableParameters(critic));
    end
end
% Update target critic periodically
if mod(totalStepCt, targetUpdateFrequency)==0
    targetCritic = critic;
end

% Stop if a terminal condition is reached.
if isDone
    break;
end
end % End of episode

%-----
% 8. Update the training visualization.
%-----
episodeCumulativeReward = sum(episodeReward);
episodeCumulativeRewardVector = cat(2, ...
    episodeCumulativeRewardVector, episodeCumulativeReward);
movingAveReward = movmean(episodeCumulativeRewardVector, ...
    aveWindowSize, 2);
addpoints(lineReward, episodeCt, episodeCumulativeReward);
addpoints(lineAveReward, episodeCt, movingAveReward(end));
title(ax, "Training Progress - Episode: " + episodeCt + ...
    ", Total Step: " + string(totalStepCt) + ...
    ", epsilon:" + string(epsilon))

```

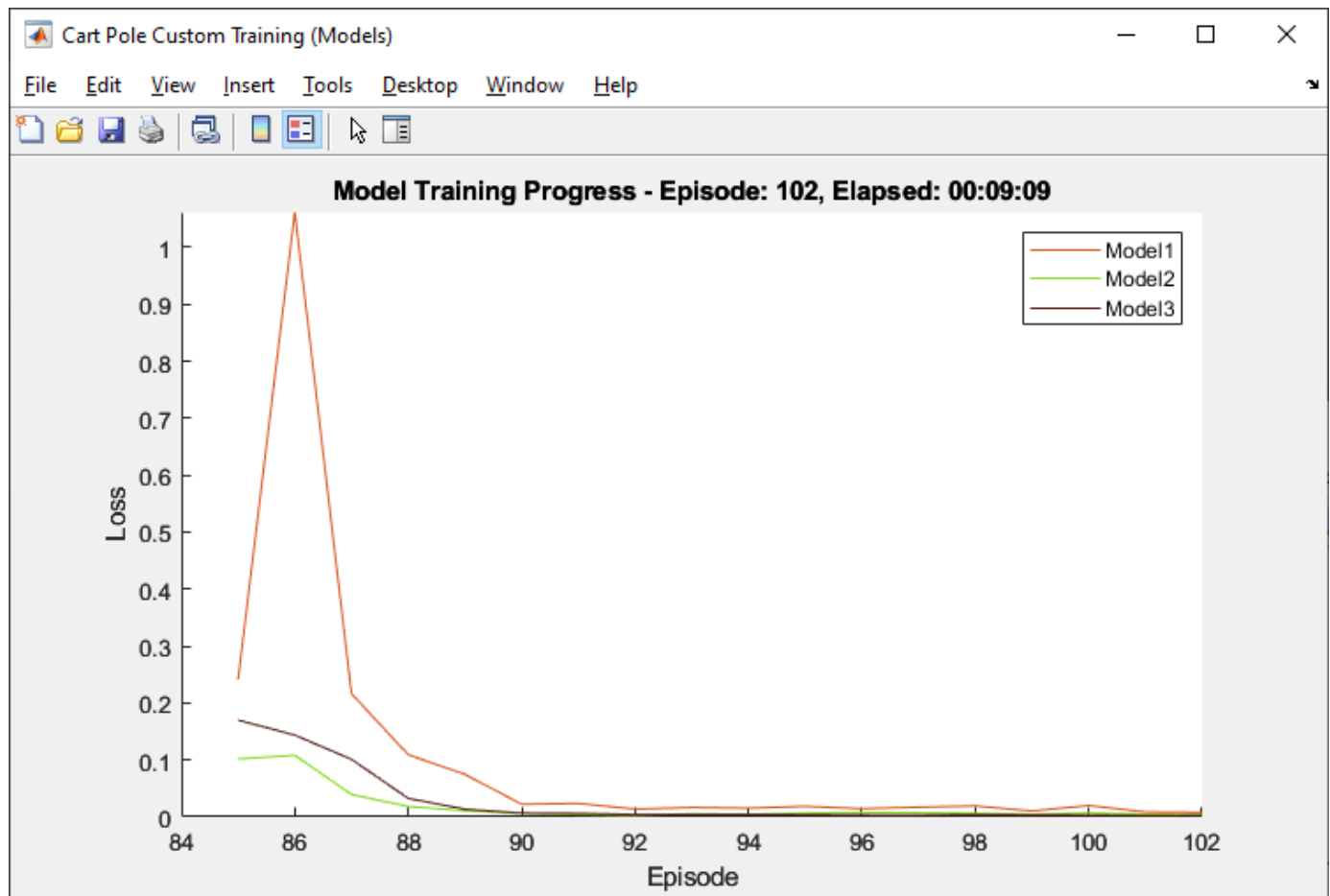
```
drawnow;

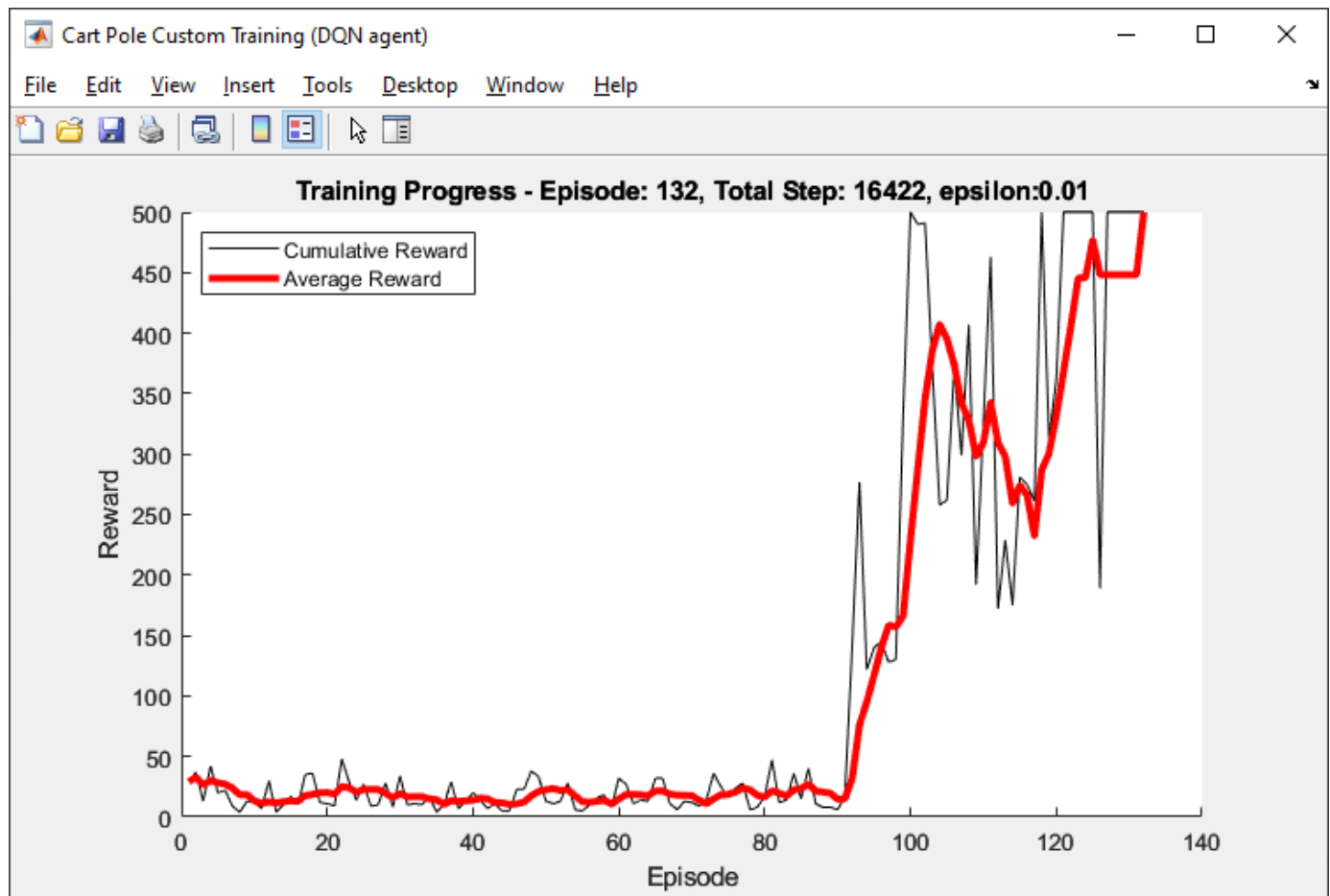
errorPreddiction = errorPreddiction(1:stepCt);

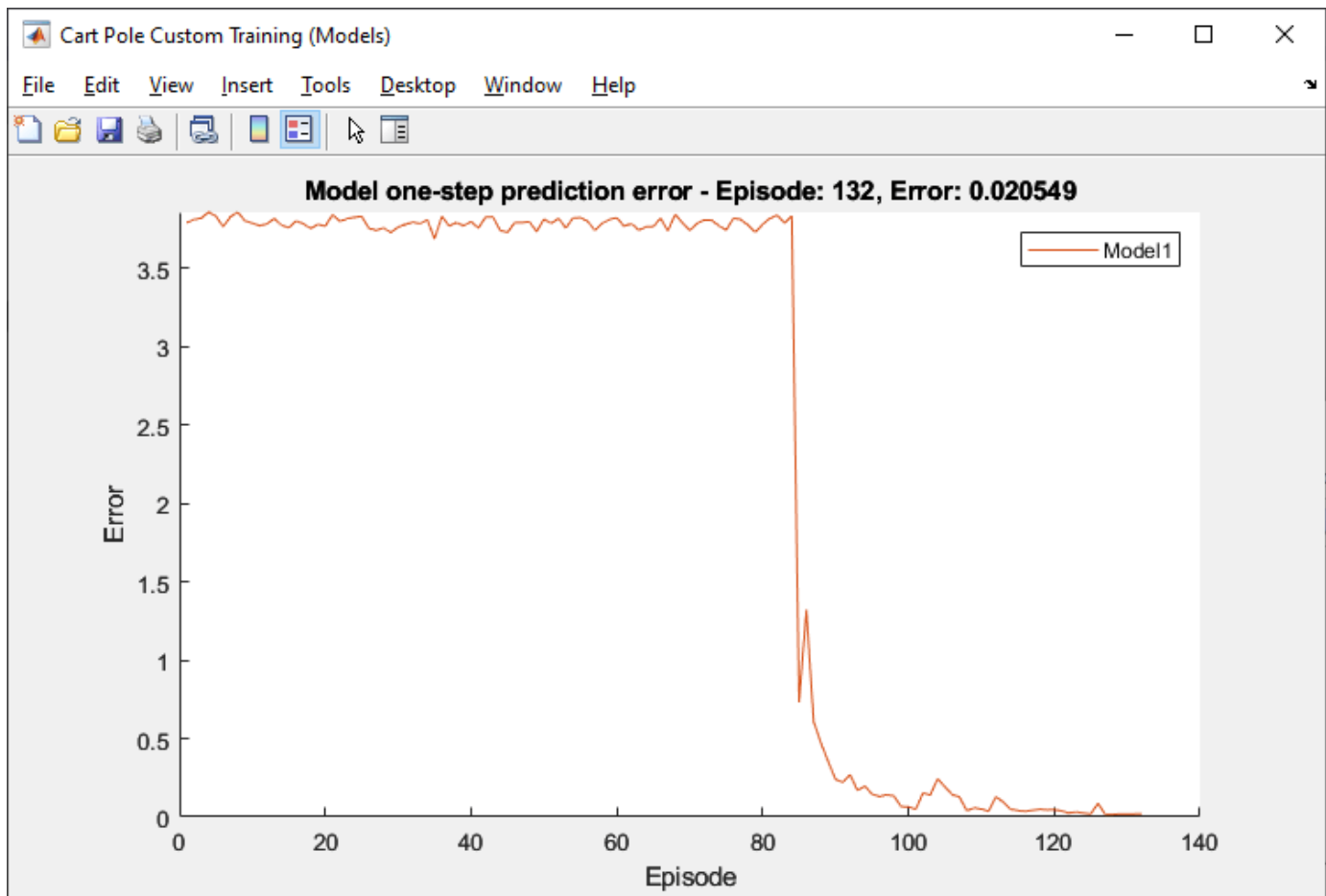
% Display one step prediction error.
addpoints(lineLossTest1,episodeCt,mean(errorPreddiction))
legend(axModelTest,"Model1");
title(axModelTest, ...
      "Model one-step prediction error - Episode: " + ...
      episodeCt + ", Error: " + ...
      string(mean(errorPreddiction)))
drawnow

% Display training progress every 10th episode
if (mod(episodeCt,10) == 0)
    fprintf("EP:%d, Reward:%.4f, AveReward:%.4f, " + ...
          "Steps:%d, TotalSteps:%d, epsilon:%f," + ...
          "error model:%f\n", ...
          episodeCt, ...
          episodeCumulativeReward, ...
          movingAveReward(end), ...
          stepCt,totalStepCt, ...
          epsilon, ...
          mean(errorPreddiction))
end

%-----
% 9. Terminate training
%   if the network is sufficiently trained.
%-----
if max(movingAveReward) > trainingTerminationValue
    break
end
end
else
    load("cartPoleModelBasedCustomLoopPolicy.mat");
end
end
```







Simulate Agent

To simulate the trained agent, first reset the environment.

```
obs0 = reset(env);
obs = obs0;
```

Enable the environment visualization, which is updated each time the environment step function is called.

```
plot(env)
```

For each simulation step, perform the following actions.

- 1 Get the action by sampling from the policy using the `getAction` function.
- 2 Step the environment using the obtained action value.
- 3 Terminate if a terminal condition is reached.

```
actionVector = zeros(1,maxStepsPerEpisode);
obsVector = zeros(numObservations,maxStepsPerEpisode+1);
obsVector(:,1) = obs0;
for stepCt = 1:maxStepsPerEpisode
    % Select action according to trained policy.
```

```

    action = getAction(policy,{obs});
    action= action{1};

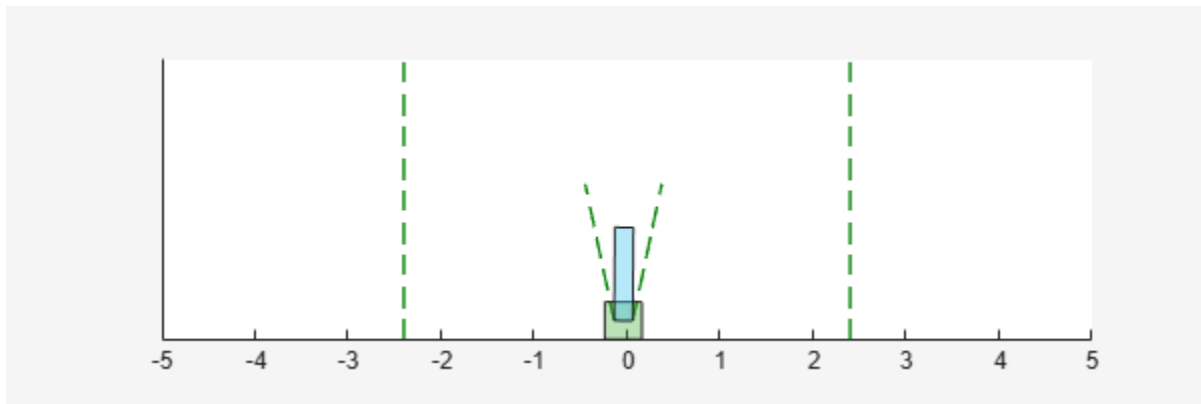
    % Step the environment.
    [nextObs,reward,isDone] = step(env,action);

    obsVector(:,stepCt+1) = nextObs;
    actionVector(1,stepCt) = action;

    % Check for terminal condition.
    if isDone
        break
    end

    obs = nextObs;
end

```



```
lastStepCt = stepCt;
```

Test Model

Test one of the models by predicting a next observation given a current observation and an action.

```

modelID = 3;
predictedObsVector = zeros(numObservations,lastStepCt);
obs = darray(obsVector(:,1),"CB");
predictedObsVector(:,1) = obs;
for stepCt = 1:lastStepCt
    obs = darray(obsVector(:,stepCt),"CB");
    action = darray(actionVector(1,stepCt),"CB");

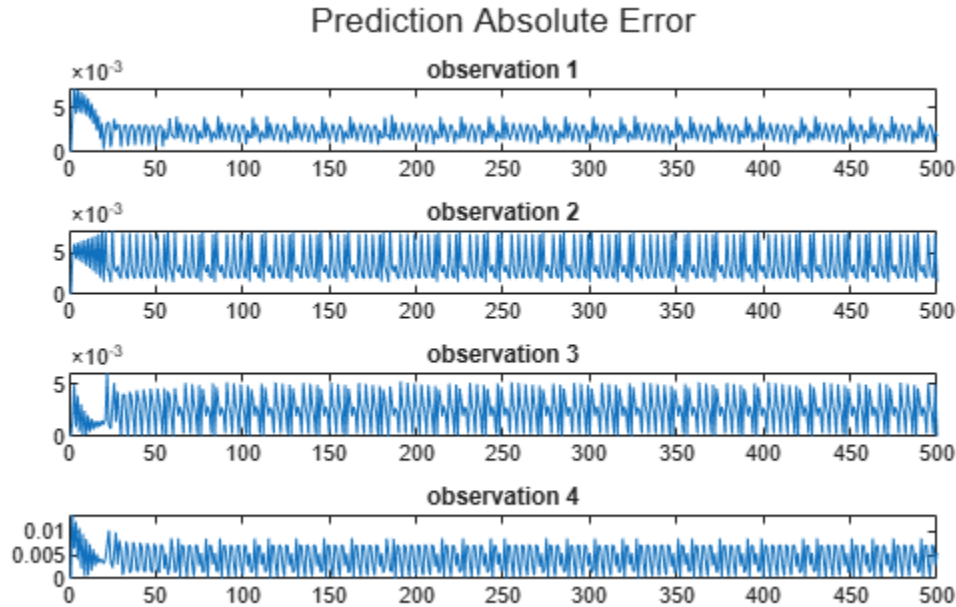
    dx = predict(transitionNetworkVector(modelID),obs, action);
    predictedObs = obs + dx;
    predictedObsVector(:,stepCt+1) = predictedObs;
end
predictedObsVector = predictedObsVector(:, 1:lastStepCt);
figure(5)
layOut = tiledlayout(4,1, "TileSpacing", "compact");
for i = 1:4
    nexttile;
    errorPrediction = abs(predictedObsVector(i,1:lastStepCt) - ...
                        obsVector(i,1:lastStepCt));
    line1 = plot(errorPrediction,"DisplayName", "Absolute Error");
end

```

```

    title("observation "+num2str(i));
end
title(layOut,"Prediction Absolute Error")

```



The small absolute prediction error shows that the model is successfully trained to predict the next observation.

References

[1] Minh, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. "Playing Atari with Deep Reinforcement Learning." Preprint, submitted December 19, 2013. <https://arxiv.org/abs/1312.5602>.

[2] Janner, Michael, Justin Fu, Marvin Zhang, and Sergey Levine. "When to trust your model: Model-based policy optimization." Preprint, submitted November 5, 2019. <https://arxiv.org/abs/1906.08253>.

See Also

Functions

rlPredefinedEnv | rlOptimizer | gradient

Objects

rlVectorQValueFunction | rlOptimizerOptions

Related Examples

- "Train Reinforcement Learning Agents" on page 5-3
- "Create and Train Custom PG Agent" on page 5-334
- "Train MBPO Agent to Balance Continuous Cart-Pole System" on page 5-356

More About

- “Use Predefined Control System Environments” on page 2-42
- “Train Reinforcement Learning Agents” on page 5-3
- “Model-Based Policy Optimization (MBPO) Agent” on page 3-86
- “Create Custom Reinforcement Learning Agents” on page 3-92

Deploy Trained Policies

- “Generate Code from Trained Reinforcement Learning Policies” on page 6-2
- “Generate Policy Block for Deployment” on page 6-7
- “Deploy Trained Reinforcement Learning Policy as Microservice Docker Image” on page 6-16
- “Run SIL and PIL Verification for Reinforcement Learning” on page 6-17
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Train Policy Deployed on Raspberry Pi” on page 6-31
- “Control a Quanser QUBE Pendulum with a Raspberry Pi using Reinforcement Learning” on page 6-50

Generate Code from Trained Reinforcement Learning Policies

Once you train a reinforcement learning agent, you can generate code to deploy the optimal policy. You can generate:

- CUDA code for policies relying on deep neural networks, using GPU Coder
- C/C++ code for policies relying on tables, deep neural networks, or linear basis functions, using MATLAB Coder

Code generation is supported for policies using feedforward neural networks in any of the input paths, provided that all the used layers are supported. Code generation is not supported for policies containing a recurrent neural network (RNN).

To generate a policy evaluation function that selects an action based on a given observation, use the `generatePolicyFunction` function. You can generate code to deploy this policy function using GPU Coder or MATLAB Coder. This function also creates a data file which stores policy information (such as for example, the trained neural network for the actor). The evaluation function loads this data file to properly initialize itself the first time it is called.

To generate a Simulink policy evaluation block that selects an action based on a given observation, use the `generatePolicyBlock` function. You can generate code to deploy this policy block using Simulink Coder. This function also creates a data file which stores policy information (such as for example, the trained neural network for the actor). The generated policy block loads this data file to properly initialize itself prior to simulation. You can use the block to simulate the policy and generate code for deployment purposes.

Note The deployed policy might have an internal state. In this case, the internal state must be consistent when comparing the simulated and deployed policy.

Generate Code Using GPU Coder

If your trained optimal policy uses a deep neural network, you can generate CUDA code for the policy using GPU Coder. For more information on supported GPUs see “GPU Computing Requirements” (Parallel Computing Toolbox). There are several required and recommended prerequisite products for generating CUDA code for deep neural networks. For more information, see “Installing Prerequisite Products” (GPU Coder) and “Setting Up the Prerequisite Products” (GPU Coder).

Not all deep neural network layers support GPU code generation. For a list of supported layers, see “Supported Networks, Layers, and Classes” (GPU Coder). For more information and examples on GPU code generation, see “Deep Learning with GPU Coder” (GPU Coder).

Generate CUDA Code for Deep Neural Network Policy

As an example, generate GPU code for the policy gradient agent trained in “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109.

Load the trained agent.

```
load('MATLABCartpolePG.mat','agent')
```

Create a policy evaluation function for this agent.

```
generatePolicyFunction(agent)
```

This command creates the `evaluatePolicy.m` file, which contains the policy function, and the `agentData.mat` file, which contains the trained deep neural network actor. In general, for a given observation, the policy function evaluates a probability for each potential action using the actor network. Then, the policy function randomly selects an action based on these probabilities (for SAC agents the policy function samples an unbounded action using the mean and the standard deviation obtained from the network, and then applies bounds, scale and bias to get the correct action).

You can generate code for this network using GPU Coder. For example, you can generate a CUDA compatible MEX function.

Configure the `codegen` function to create a CUDA compatible C++ MEX function.

```
cfg = coder.gpuConfig('mex');
cfg.TargetLang = 'C++';
cfg.DeepLearningConfig = coder.DeepLearningConfig('cudnn');
```

Set an example input value for the policy evaluation function. To find the observation dimension, use the `getObservationInfo` function. In this case, the observations are in a four-element vector.

```
argstr = '{ones(4,1)}';
```

Generate code using the `codegen` function.

```
codegen('-config','cfg','evaluatePolicy','-args',argstr,'-report');
```

This command generates the MEX function `evaluatePolicy_mex`.

Generate Code Using MATLAB Coder

You can generate C/C++ code for table, deep neural network, or linear basis function policies using MATLAB Coder.

Using MATLAB Coder, you can generate:

- C/C++ code for policies that use Q tables, value tables, or linear basis functions. For more information on general C/C++ code generation, see “Generating Code” (MATLAB Coder).
- C++ code for policies that use deep neural networks. Note that code generation is not supported for continuous actions PG, AC, PPO, and SAC agents using a recurrent neural network (RNN). For a list of supported layers, see “Networks and Layers Supported for Code Generation” (MATLAB Coder). For more information, see “Prerequisites for Deep Learning with MATLAB Coder” (MATLAB Coder) and “Deep Learning with MATLAB Coder” (MATLAB Coder).

Generate C Code for Deep Neural Network Policy Without Using Any Third-Party Library

As an example, generate C code without dependencies on third-party libraries for the policy gradient agent trained in “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109.

Load the trained agent.

```
load('MATLABCartpolePG.mat','agent')
```

Create a policy evaluation function for this agent.

```
generatePolicyFunction(agent)
```

This command creates the `evaluatePolicy.m` file, which contains the policy function, and the `agentData.mat` file, which contains the trained deep neural network actor. For a given observation, the policy function evaluates a probability for each potential action using the actor network. Then, the policy function randomly selects an action based on these probabilities (for SAC agents the policy function samples an unbounded action using the mean and the standard deviation obtained from the network, and then applies bounds, scale and bias to get the correct action).

Configure the `codegen` function to generate code suitable for building a MEX file.

```
cfg = coder.config('mex');
```

On the configuration object, set the target language to C, and set `DeepLearningConfig` to 'none'. This option generates code without using any third-party library.

```
cfg.TargetLang = 'C';  
cfg.DeepLearningConfig = coder.DeepLearningConfig('none');
```

Set an example input value for the policy evaluation function. To find the observation dimension, use the `getObservationInfo` function. In this case, the observations are in a four-element vector.

```
argstr = '{ones(4,1)}';
```

Generate code using the `codegen` function.

```
codegen('-config','cfg','evaluatePolicy','-args',argstr,'-report');
```

This command generates the C code for the policy gradient agent containing the deep neural network actor.

Generate C++ Code for Deep Neural Network Policy Using Third-Party Libraries

As an example, generate C++ code for the policy gradient agent trained in “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109 using the Intel Math Kernel Library for Deep Neural Networks (MKL-DNN).

Load the trained agent.

```
load('MATLABCartpolePG.mat','agent')
```

Create a policy evaluation function for this agent.

```
generatePolicyFunction(agent)
```

This command creates the `evaluatePolicy.m` file, which contains the policy function, and the `agentData.mat` file, which contains the trained deep neural network actor. For a given observation, the policy function evaluates a probability for each potential action using the actor network. Then, the policy function randomly selects an action based on these probabilities.

Configure the `codegen` function to generate code suitable for building a MEX file.

```
cfg = coder.config('mex');
```

On the configuration object, set the target language to C++, and set `DeepLearningConfig` to the target library 'mkl_dnn'. This option generates code using the Intel Math Kernel Library for Deep Neural Networks (Intel MKL-DNN).

```
cfg.TargetLang = 'C++';  
cfg.DeepLearningConfig = coder.DeepLearningConfig('mkl_dnn');
```


Set an example input value for the policy evaluation function. To find the observation dimension, use the `getObservationInfo` function. In this case, the observations are in a four-element vector.

```
argstr = '{ones(4,1)}';
```

Generate code using the `codegen` function.

```
codegen('-config','cfg','evaluatePolicy','-args',argstr,'-report');
```

This command generates the C++ code for the policy gradient agent containing the deep neural network actor.

Generate C Code for Q Table Policy

As an example, generate C code for the Q-learning agent trained in “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16.

Load the trained agent.

```
load('basicGWQAgent.mat','qAgent')
```

Create a policy evaluation function for this agent.

```
generatePolicyFunction(qAgent)
```

This command creates the `evaluatePolicy.m` file, which contains the policy function, and the `agentData.mat` file, which contains the trained Q table value function. For a given observation, the policy function looks up the value function for each potential action using the Q table. Then, the policy function selects the action for which the value function is greatest.

Set an example input value for the policy evaluation function. To find the observation dimension, use the `getObservationInfo` function. In this case, there is a single one dimensional observation (belonging to a discrete set of possible values).

```
argstr = '{[1]}';
```

Configure the `codegen` function to generate embeddable C code suitable for targeting a static library, and set the output folder to `buildFolder`.

```
cfg = coder.config('lib');  
outFolder = 'buildFolder';
```

Generate C code using the `codegen` function.

```
codegen('-c','-d',outFolder,'-config','cfg',...  
    'evaluatePolicy','-args',argstr,'-report');
```

See Also

Functions

`generatePolicyFunction` | `generatePolicyBlock` | `policyParameters` | `updatePolicyParameters` | `trainFromData`

Related Examples

- “Run SIL and PIL Verification for Reinforcement Learning” on page 6-17

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Train Reinforcement Learning Agents” on page 5-3
- “Generate Policy Block for Deployment” on page 6-7
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28

Generate Policy Block for Deployment

This example shows how to generate a Policy block ready for deployment from an agent object. You generate the policy from the agent in “Train TD3 Agent for PMSM Control” on page 8-242, then simulate it to validate its performance. The policy is simulated to validate performance. If Embedded Coder® is installed, a software-in-the-loop (SIL) simulation is run to validate the generated code of the policy.

In general, the workflow for the deployment of a reinforcement learning policy via a Simulink® model is:

- 1 Train the agent (see “Train TD3 Agent for PMSM Control” on page 8-242).
- 2 Generate a Policy block from the trained agent.
- 3 Replace the RL Agent block with the Policy block.
- 4 Configure the model for code generation.
- 5 Simulate the policy and verify policy performance.
- 6 Generate code for the policy, simulate the generated code, and verify policy performance.
- 7 Deploy to hardware for testing.

In this example, you do steps 2 through 6.

Load the motor parameters along with the trained TD3 agent.

`sim_data;`

```
### The Lq is observed to be lower than Ld. ###
### Using the lower of these two for the Ld (internal variable) ###
### and higher of these two for the Lq (internal variable) for computations. ###
### The Lq is observed to be lower than Ld. ###
### Using the lower of these two for the Ld (internal variable) ###
### and higher of these two for the Lq (internal variable) for computations. ###
    model: 'Maxon-645106'
        sn: '2295588'
        p: 7
        Rs: 0.2930
        Ld: 8.7678e-05
        Lq: 7.7724e-05
        Ke: 5.7835
        J: 8.3500e-05
        B: 7.0095e-05
    I_rated: 7.2600
    QEPSlits: 4096
    N_base: 3476
    N_max: 4300
    FluxPM: 0.0046
    T_rated: 0.3471
    PositionOffset: 0.1650

    model: 'BoostXL-DRV8305'
        sn: 'INV_XXXX'
        V_dc: 24
        I_trip: 10
        Rds_on: 0.0020
```

```
        Rshunt: 0.0070
        CtSensAOffset: 2295
        CtSensBOffset: 2286
        CtSensCOffset: 2295
        ADCGain: 1
        EnableLogic: 1
        invertingAmp: 1
        ISenseVref: 3.3000
        ISenseVoltPerAmp: 0.0700
        ISenseMax: 21.4286
        R_board: 0.0043
        CtSensOffsetMax: 2500
        CtSensOffsetMin: 1500

        model: 'LAUNCHXL-F28379D'
        sn: '123456'
        CPU_frequency: 200000000
        PWM_frequency: 5000
        PWM_Counter_Period: 20000
        ADC_Vref: 3
        ADC_MaxCount: 4095
        SCI_baud_rate: 12000000

        V_base: 13.8564
        I_base: 21.4286
        N_base: 3476
        T_base: 1.0249
        P_base: 445.3845
```

```
load("rlPMSMAgent.mat","agent");
```

Generate Policy Block

Open the Simulink model used for training the TD3 agent.

```
mdl_rl = "mcb_pmsm_foc_sim_RL";
open_system(mdl_rl);
```

Open the subsystem containing the **RL Agent** block.

```
agentblk = mdl_rl + ...
    "/Current Control/Control_System" + ...
    "/Closed Loop Control/Reinforcement Learning/RL Agent";
open_system(get_param(agentblk,"Parent"));
```

To create a deployable model, you replace the **RL Agent** block with a **Policy** block.

Set the agent's `UseExplorationPolicy` property to false to make sure that the generated policy takes the greedy action at each time step.

```
agent.UseExplorationPolicy = false;
```

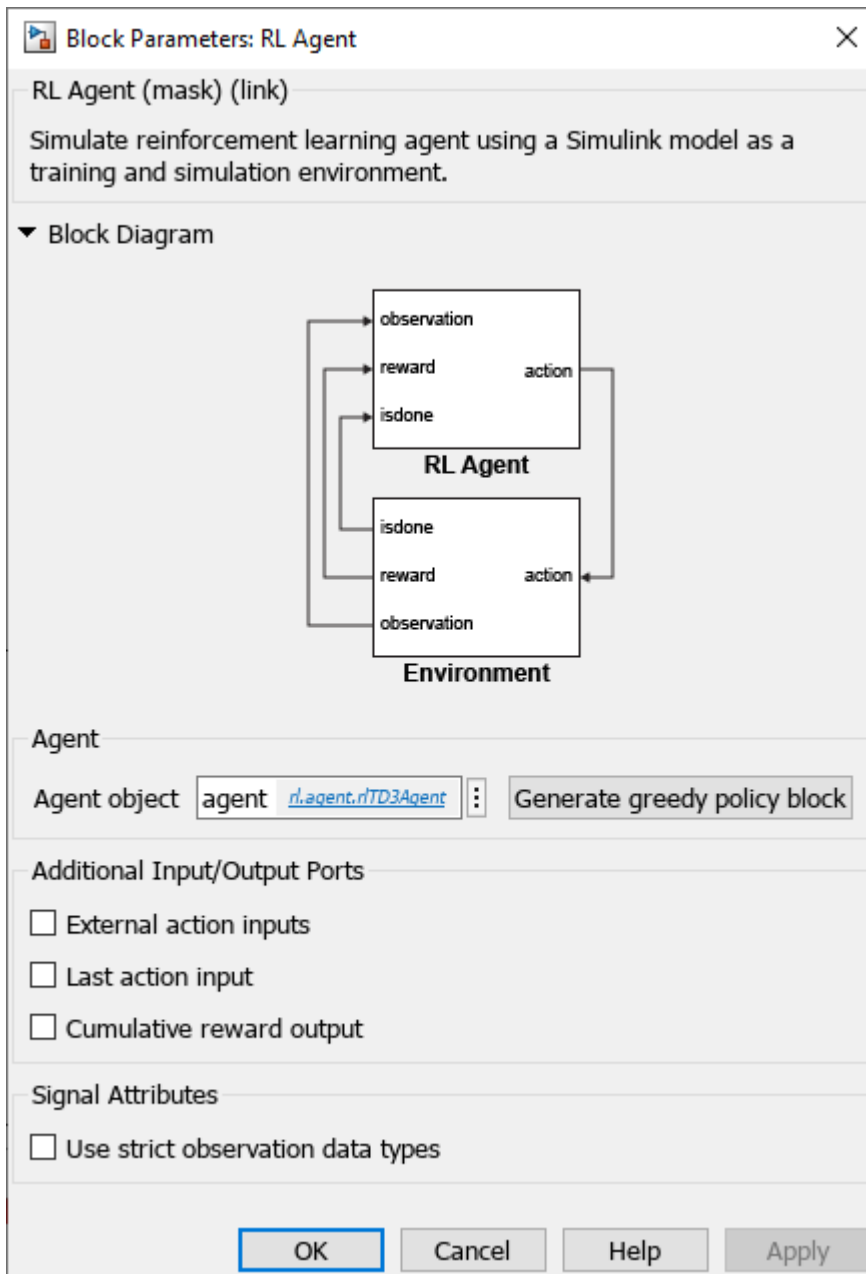
Generate the policy block using `generatePolicyBlock` and specify the name of the MAT file containing the policy data for the block.

```
% Specify the MAT file name for the policy data
fname = "PMSMPolicyBlockData.mat";
```

```
% Delete the file if it already exists
if isfile(fname)
    delete(fname);
end

% Generate the block and the policy data
generatePolicyBlock(agent,MATFileName=fname)
```

Alternatively, you can generate the policy block by clicking **Generate greedy policy block** from the block mask. Use `open_system(agentblk)` to open the **RL Agent** block mask, or simply double-click the block.



Simulate the Policy

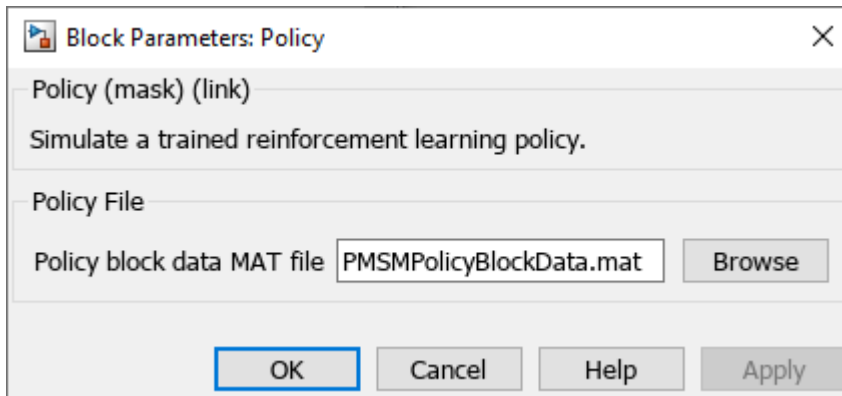
For this example, the **Policy** block has already replaced the **RL Agent** block inside of the `pmsm_current_control` model. This model has been configured for code generation, and the **Policy** block loads the trained policy from `PMSMPolicyBlockData.mat`.

```
mdl_current_ctrl = "pmsm_current_control";
open_system(mdl_current_ctrl);
```

Obtain the path of the policy block in the Simulink model.

```
policyblk = mdl_current_ctrl + ...
    "/Current Control/Control_System" + ...
    "/Closed Loop Control/Reinforcement Learning/Policy";
```

Use `open_system(policyblk)` to open the **Policy** block mask, or simply double-click the block.



The model `mcb_pmsm_foc_sim_policy` references `pmsm_current_control` using a model reference block. Simulate the top-level model and plot the responses for the inner and outer control loops.

Setup the **Simulation Data Inspector** (SDI).

```
Simulink.sdi.clear;
Simulink.sdi.setSubPlotLayout(3,1);
```

Open the model and get the path to the `Current Control` block.

```
mdl_policy = "mcb_pmsm_foc_sim_policy";
open_system(mdl_policy);
current_ctrl_blk = mdl_policy + "/Current Control";
```

To temporarily change the model and easily run multiple simulations with such changes, `Simulink.SimulationInput` (Simulink) object.

```
in = Simulink.SimulationInput(mdl_policy);
```

Simulate the model with the current controller, run the simulation in normal mode

```
in = setBlockParameter(in,current_ctrl_blk, ...
    "SimulationMode","Normal");
out_sim = sim(in);
```

Get results from the latest SDI run.

```
runSim = Simulink.sdi.Run.getLatest;
```

Extract the outer control loop signals.

```
speedSim    = getSignalsByName(runSim, "Speed_fb" );  
speedRefSim = getSignalsByName(runSim, "Speed_Ref");
```

Plot both signals.

```
plotOnSubPlot(speedSim    ,1,1,true);  
plotOnSubPlot(speedRefSim,1,1,true);
```

Extract the inner control loop signals.

```
idSim      = getSignalsByName(runSim, "id" );  
iqSim      = getSignalsByName(runSim, "iq" );  
idRefSim   = getSignalsByName(runSim, "id_ref");  
iqRefSim   = getSignalsByName(runSim, "iq_ref");
```

Plot the extracted signals.

```
plotOnSubPlot(idSim      ,2,1,true);  
plotOnSubPlot(idRefSim,2,1,true);  
plotOnSubPlot(iqSim      ,3,1,true);  
plotOnSubPlot(iqRefSim,3,1,true);
```

Open the **Simulation Data Inspector**.

```
Simulink.sdi.view;
```



Validate Generated Code for Policy

If Embedded Coder is installed, the current controller model reference can be run in SIL mode. Running the current controller in SIL mode generates code for the current controller model, including the policy block.

The Simulink model is configured only on Windows®, so display a message and stop execution when attempting to run on a different operating system.

```
if ~ispc
    disp("The model ""pmsm_current_control"" is configured " + ...
        "for SIL simulations only on Windows system.")
```



```

    return
end

```

Simulate the model with the current controller.

Enable SIL mode.

```

in = setBlockParameter(in,current_ctrl_blk, ...
    "SimulationMode","Software-in-the-loop");

```

Simulate the model. Use `evalc` to capture the text output from code generation, for possible later inspection.

```

txt_out = evalc("out_sil = sim(in)");

```

Get results from the latest SDI run.

```

runSIL = Simulink.sdi.Run.getLatest;

```

Extract the speed response when run in SIL mode.

```

speedSIL = getSignalsByName(runSIL,"Speed_fb");

```

Compare the SIL response to the simulated response. The SIL response should be close to the response simulated in normal mode.

```

speedSim.AbsTol = 1e-3;
cr = Simulink.sdi.compareSignals(speedSim.ID,speedSIL.ID);

```

Display the largest signal difference.

```

cr.MaxDifference

```

```

ans =
5.0485e-05

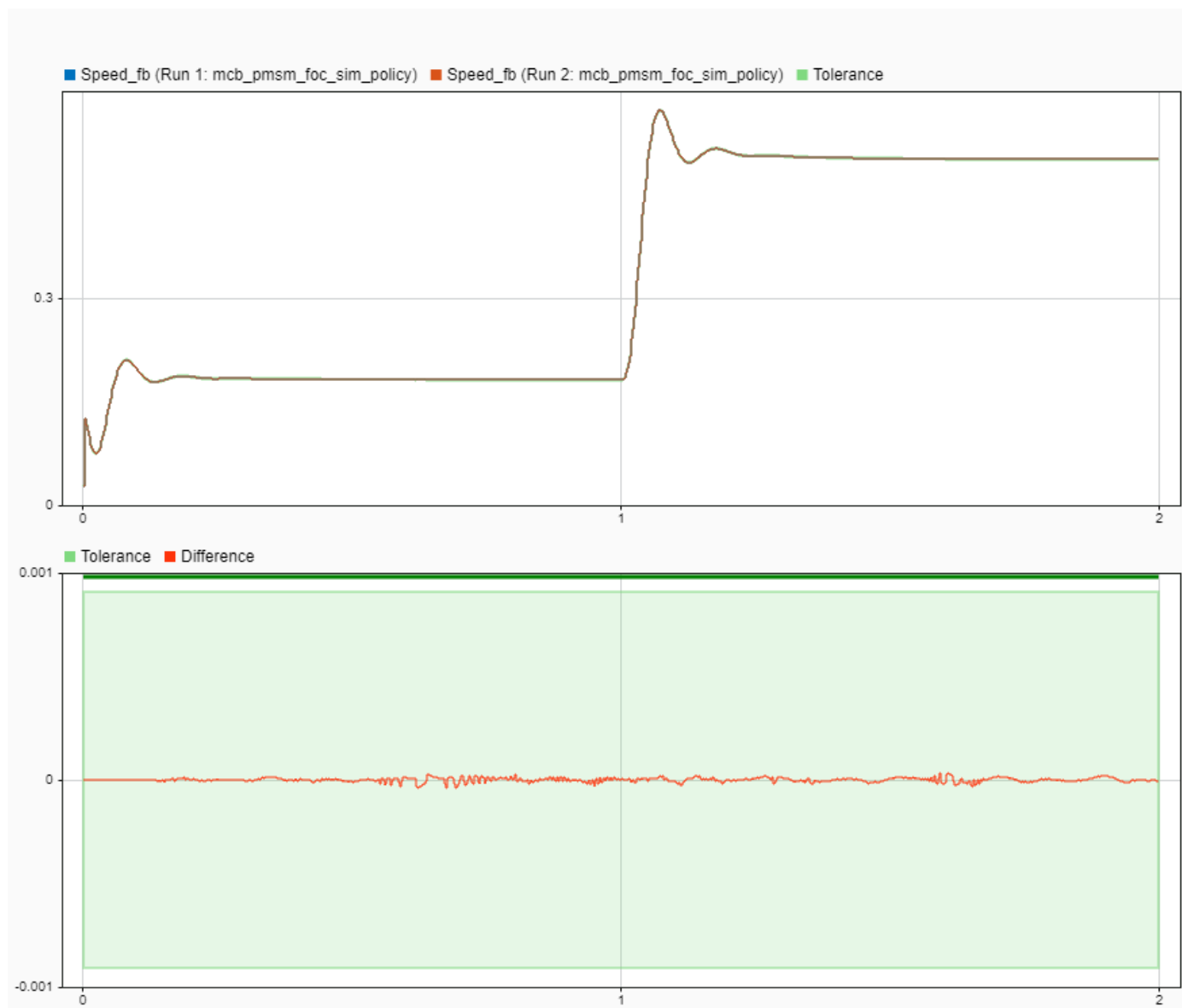
```

Open the **Simulation Data Inspector**.

```

Simulink.sdi.view;

```



Once you are satisfied with the performance of the policy in simulation, you can use an appropriate target/hardware support package to deploy the policy to hardware.

See Also

Functions

`generatePolicyBlock` | `generatePolicyFunction` | `train`

Objects

`dlnetwork`

Blocks

`Policy`

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Train Reinforcement Learning Agents” on page 5-3
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Generate Code from Trained Reinforcement Learning Policies” on page 6-2

Deploy Trained Reinforcement Learning Policy as Microservice Docker Image

To deploy a trained RL policy as a microservice Docker® image, follow three steps.

- 1 Package a MATLAB function that evaluates a reinforcement learning policy into a deployable archive.
- 2 Create a Docker image that contains the archive and a minimal MATLAB Runtime package.
- 3 Run the image in Docker and make calls to the service using any of the MATLAB Production Server™ client APIs.

For an example on how to do this, see “Deploy Trained Reinforcement Learning Policy as Microservice Docker Image” (MATLAB Compiler SDK).

See Also

Functions

`generatePolicyFunction` | `generatePolicyBlock` | `policyParameters` | `updatePolicyParameters` | `train` | `trainFromData`

Objects

`dlnetwork`

Blocks

`Policy`

Related Examples

- “Generate Policy Block for Deployment” on page 6-7

More About

- “Reinforcement Learning Workflow” on page 1-5
- “Train Reinforcement Learning Agents” on page 5-3
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Generate Code from Trained Reinforcement Learning Policies” on page 6-2

Run SIL and PIL Verification for Reinforcement Learning

This example shows how to perform software-in-the-loop (SIL) and processor-in-the-loop (PIL) verification workflows for reinforcement learning agents in Simulink®.

Prerequisites

This example requires the following hardware.

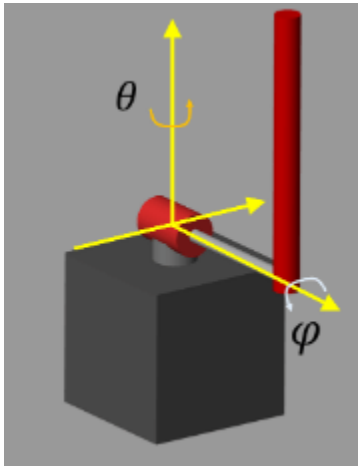
- Raspberry Pi® hardware.
- Wi-Fi® dongle or an Ethernet cable.
- Power source and cable for the Raspberry Pi hardware.

You must also download and install the following support packages using the **Add-Ons Explorer**.

- Raspberry Pi® Blockset™. See “Get Started with Raspberry Pi Blockset” (Raspberry Pi Blockset) for more information about how to install the support package and set up your Raspberry Pi hardware.
- MATLAB® Coder™ Interface for Deep Learning. This will install the Intel® MKL-DNN and ARM® Compute libraries.

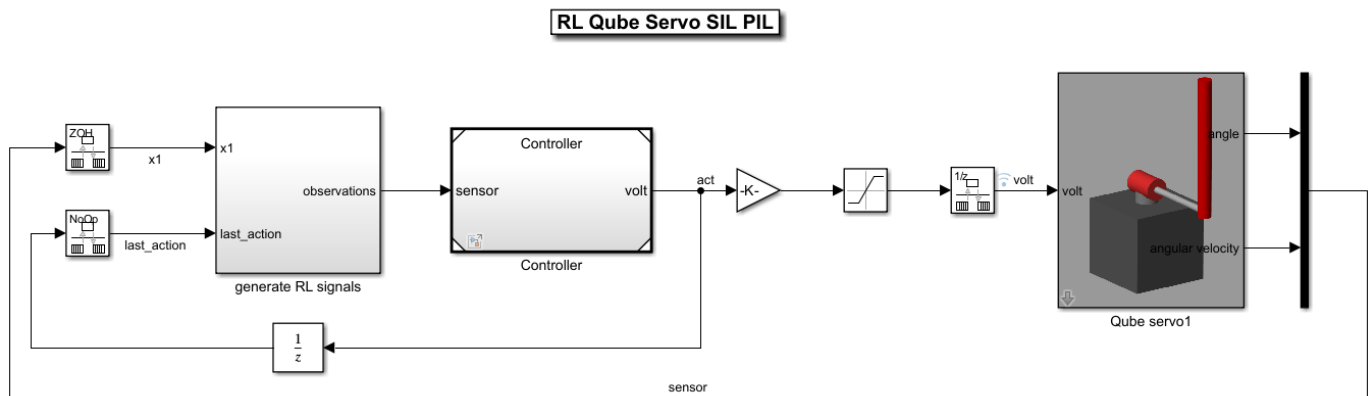
Simulink Environment

The environment for this example is a Quanser QUBE™-Servo 2 pendulum swing-up model. The controller for this environment is a reinforcement learning policy that computes the actions to swing up and balance the pendulum. For details about the environment and controller design, see the “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14 example.



Open the Simulink model.

```
mdl = "rlQubeServo_SIL_PIL";  
open_system(mdl)
```



Copyright 2021-2024, The MathWorks Inc.

Load the model configuration settings into the MATLAB workspace. These settings will be used for SIL and PIL simulations.

```
load("configs.mat")
```

Define the limit for θ ($5\pi/8$ radians), $\dot{\theta}$ (3zero radians/second), voltage limit (12 volts), as well as the agent's sample time (0.005 seconds).

```
theta_limit = 5*pi/8;
dtheta_limit = 30;
volt_limit = 12;
Ts = 0.005;
```

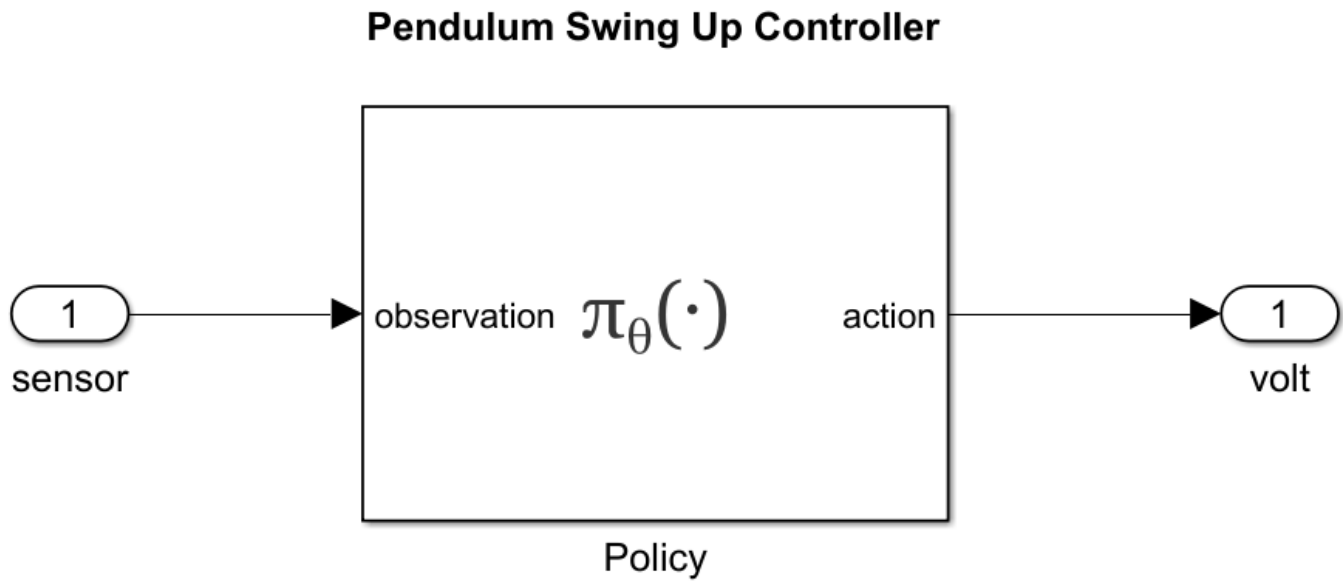
Define the initial conditions for $\theta(\pi/6$ radians), $\dot{\theta}$ (zero radians), $\phi(\pi$ radians), $\dot{\phi}$ (zero radians).

```
theta0 = -pi/6;
dtheta0 = 0;
phi0 = pi;
dphi0 = 0;
```

The **Controller** model reference consists of a **Policy** block that evaluates a trained reinforcement learning policy. The block is generated using the RL Agent option.

Open and view the Controller system.

```
open_system("Controller");
```

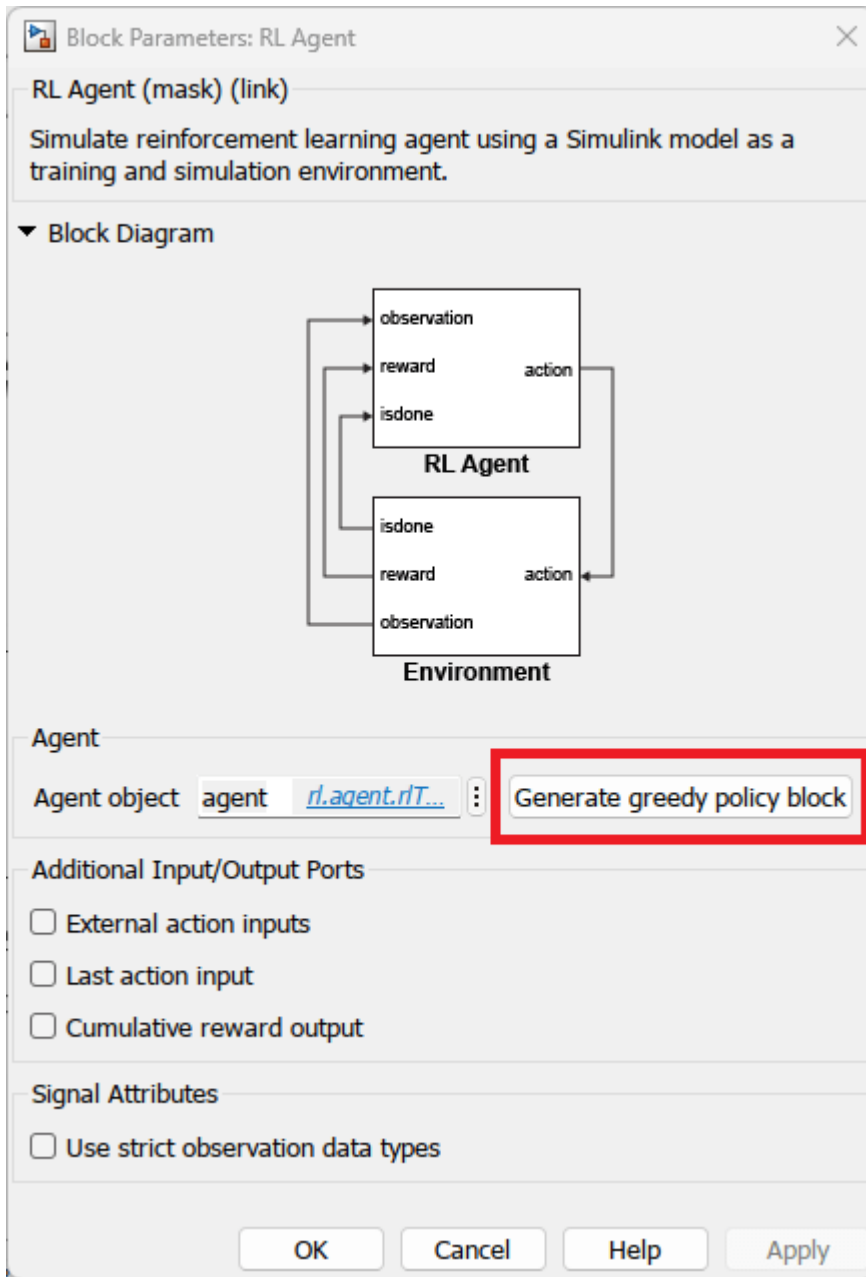


Copyright 2021-2024 The MathWorks, Inc.

Greedy Policy Block Generation

The example is already configured with the greedy policy block, but you can regenerate the block using the following steps:

- Open the “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14 example and run the example Live Script.
- From the Simulink model, open the RL Agent block dialog.
- Click **Generate greedy policy block**.
- Replace the Policy block in `Controller.slx` with the generated block. Save the model.
- Copy over the generated MAT file (`blockAgentData.mat`) to the working directory of this example.



Software-in-the-Loop Simulation

You can analyze code generation performance using software-in-the-loop (SIL) simulation. A SIL simulation generates and builds code on your development computer and then simulates the system using the generated code. You can then compare the results with a Normal mode simulation.

Configure Controller Model for SIL Simulation

Configure the Controller model reference for code generation using pre-defined settings.

```
setActiveConfigSet("Controller", "configSILReferenceRL");
```

To configure the model yourself:

- Open the `Controller.slx` model.
- In the Simulink model window, on the **Modeling** tab, click **Model Settings**.
- In the Configuration Parameters window, in the **Solver** section, set the **Type** parameter to **Fixed-step** and **Solver** parameter to **discrete**. Set the **Fixed-step size** parameter to `Ts`, which is 0.005 s.
- In the **Simulation Target** section, set the **Language** parameter to C++ and the **Target Library** parameter to **MKL-DNN**. If using the Deep Learning Toolbox Predict block for evaluation, set **Language** to C.
- In the **Code Generation** section, set the **System target file** parameter to `ert.tlc` and the **Language** parameter to C. Also, select an appropriate **Toolchain** parameter.
- Optionally, you can set the **Language** to C++ and set the **Target library** in the **Code Generation > Interface** section to **MKL-DNN**. Doing so generates C++ code for the controller.
- Save the model.

Optionally, you can view the generated code for the controller from the C-code perspective.

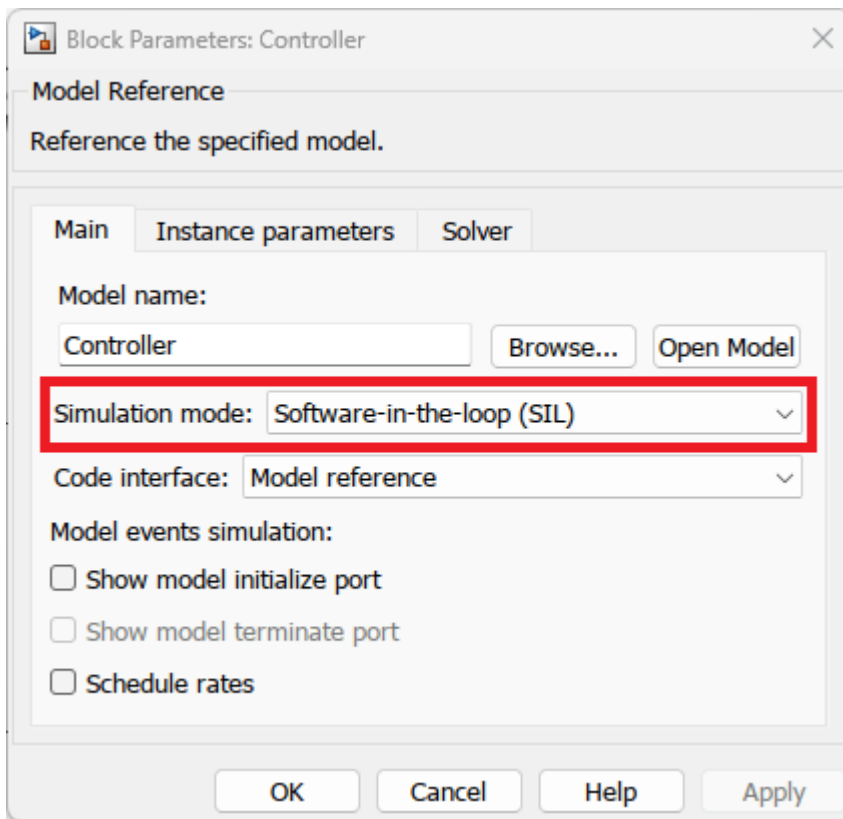
- In the Simulink model window, on the **Apps** tab, in the gallery, click **Embedded Coder**.
- To generate code and display it in the Code panel, on the **C Code** tab, click **Build**.
- Ensure that there are no errors in this process. You can reconfigure the code generation settings to optimize the generated code.

Configure Top-Level Model for SIL Simulation

Specify the simulation mode for the Controller model reference in the top-level model `rlQubeServo_SIL_PIL.slx`.

```
set_param("rlQubeServo_SIL_PIL/Controller", ...
    SimulationMode="Software-in-the-loop (SIL)")
```

Alternately, right-click the Controller subsystem and select **Block Parameters**. Then, in the Block Parameters dialog box, set the **Simulation mode** parameter to **Software-in-the-loop (SIL)**.



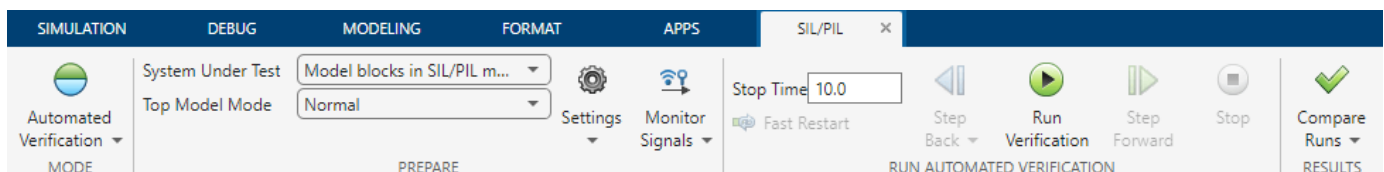
Run SIL Simulation

Fix the random seed for reproducibility.

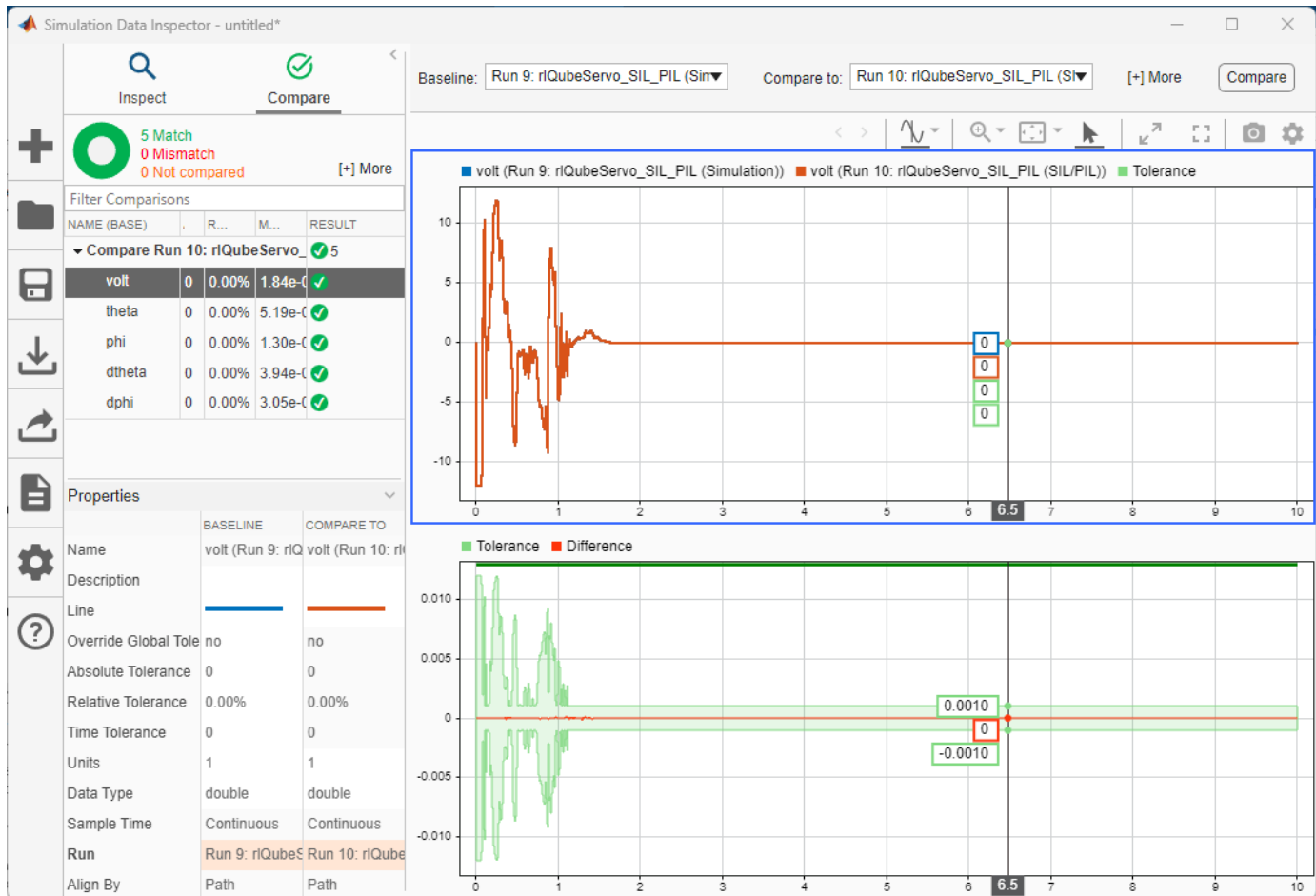
```
rng(0, "twister");
```

To run the SIL simulation:

- In the top-level model, on the **Apps** tab, click **SIL/PIL Manager**.
- On the **SIL/PIL** tab, in the **System Under Test** drop-down menu, select **Model blocks in SIL/PIL mode**.
- Then, in the **Top Model Mode**, select **Normal**.
- To simulate the model, generate and run the code in a SIL simulation, and compare the results, click **Run Verification**. The results are shown in the Simulink Data Inspector.



A comparison of the logged `volt` signal is shown between Normal and SIL simulations. The error tolerance of $1e-3$ is acceptable for this example.



Processor-in-the-Loop Simulation

In a processor-in-the-loop (PIL) simulation, you can generate code for the target hardware (in this case the Raspberry Pi), and deploy and run the code from the hardware. The results of the PIL simulation are transferred to Simulink to verify the numerical equivalence of the simulation and the code generation results. The PIL verification process is an important part of the design cycle to ensure that the behavior of the deployment code matches the design.

Setup Raspberry Pi Hardware

- Install the required add-ons and set up the Raspberry Pi hardware as per instructions in the Prerequisites on page 6-17 section.
- Connect the Raspberry Pi to the host computer and power it on.
- Replace the information in the code below with the logon credentials of your hardware.
- Execute the `raspi` (Raspberry Pi Blockset) command to ensure that the hardware is connected.

```
ipaddress = 172.31.28.34 ;
username = pi ;
password = raspberry ;
mypi = raspi(ipaddress, username, password)
```

```
mypi =
    raspi with properties:

        DeviceAddress: '172.31.28.34'
        Port: 18734
        BoardName: 'Raspberry Pi 2 Model B'
        AvailableLEDs: {'led0'}
        AvailableDigitalPins: [4,5,6,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27]
        AvailableSPIChannels: {'CE0','CE1'}
        AvailableI2CBuses: {'i2c-1'}
        AvailableWebcams: {}
        I2CBusSpeed: 100000
        AvailableCANInterfaces: {}

    Supported peripherals
```

Configure Controller Model for PIL Simulation

Configure the Controller model reference for code generation using pre-defined settings.

```
setActiveConfigSet("Controller","configPILReference");
```

To configure the model yourself:

- Open the `Controller.slx` model.
- In the Simulink model window, on the **Modeling** tab, click **Model Settings**.
- In the Configuration Parameters window, in the **Solver** section, set the **Type** parameter to **Fixed-step** and **Solver** parameter to **discrete**. Set the **Fixed-step size** parameter to `Ts`, which is 0.005 s.
- In the **Simulation Target** section, set the **Language** parameter to **C++** and the **Target Library** parameter to **MKL-DNN**. If using the Deep Learning Toolbox Predict block for evaluation, set **Language** to **C**.
- In the **Code Generation** section, set the **System target file** parameter to `ert.tlc` and the **Language** parameter to **C**.
- Select an appropriate **Toolchain** parameter. The **GNU GCC Embedded Linux** toolchain was selected for this example.
- Optionally, you can set the **Language** to **C++** and set the **Target library** in the **Code Generation > Interface** section to **MKL-DNN**. Doing so generates C++ code for the controller.
- Save the model.

In the Configuration Parameters window, under the **Hardware Implementation** section:

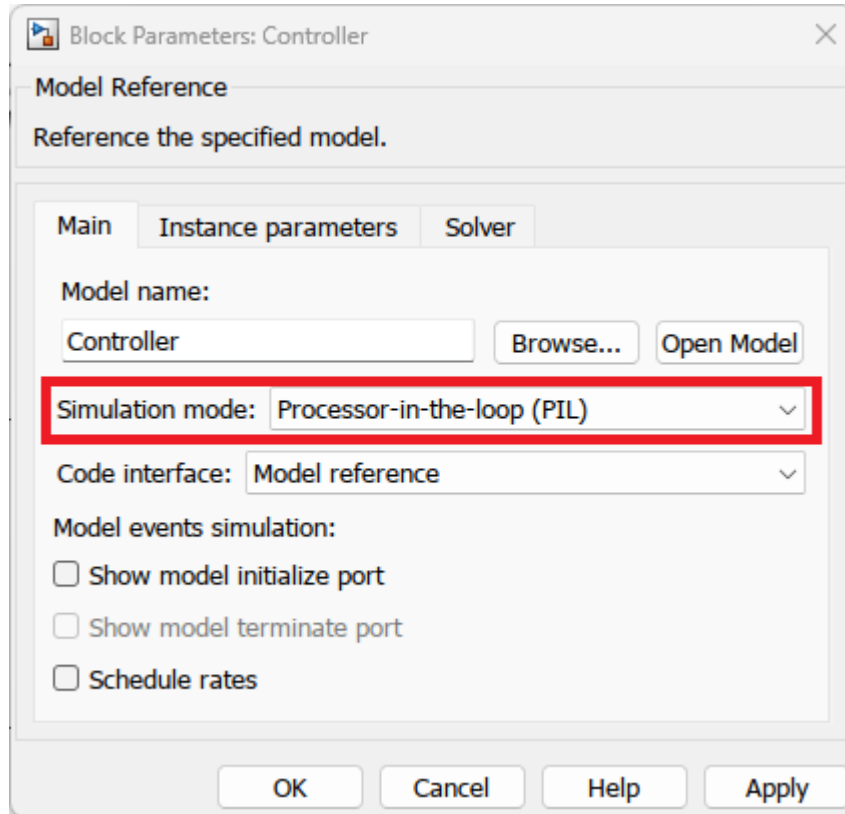
- Set the **Hardware board** parameter to **Raspberry Pi**.
- Under **Target hardware resources** enter the board parameters. Specify the **Device Address**, **Username**, and **Password** as parameters to appropriate values.
- Save the model.

Configure Top-Level Model for PIL Simulation

Specify the simulation mode for the Controller model reference in the top-level model `rlQubeServo_SIL_PIL.slx`.

```
set_param("rlQubeServo_SIL_PIL/Controller", ...
    SimulationMode="Processor-in-the-loop (PIL)")
```

Alternatively, open the top-level model `rlQubeServo_SIL_PIL.slx`. Then, right-click the Controller subsystem and select **Block Parameters**. In the Block Parameters dialog box, set the **Simulation mode** parameter to `Processor-in-the-loop (PIL)`.



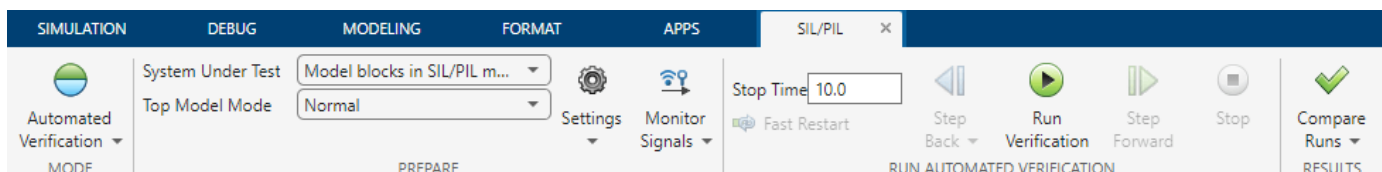
Run PIL Simulation

Fix the random seed for reproducibility.

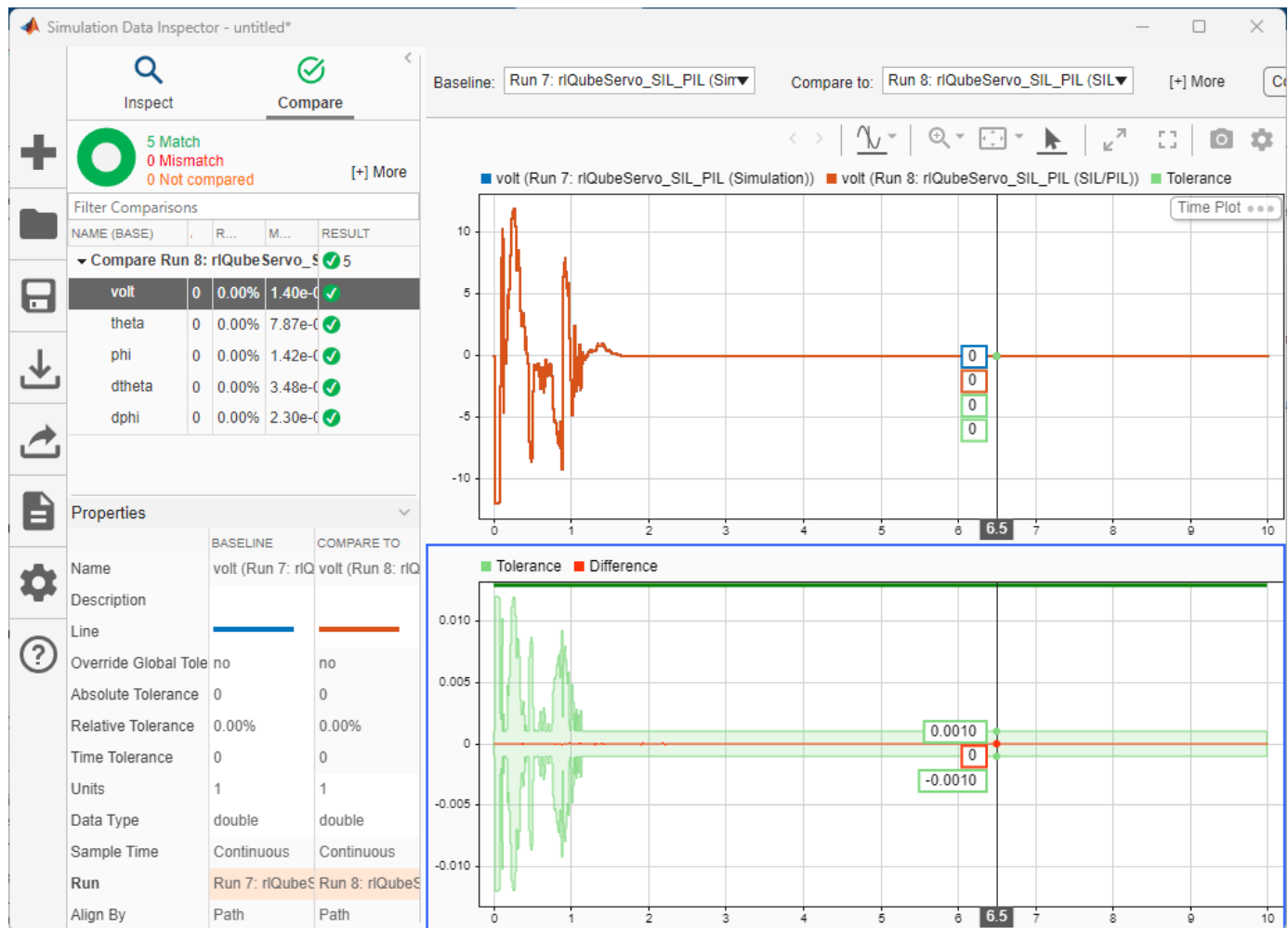
```
rng(0, "twister");
```

To run the PIL simulation:

- In the top-level model, on the **Apps** tab, click **SIL/PIL Manager**.
- On the **SIL/PIL** tab, in the **System Under Test** drop-down menu, select **Model blocks in SIL/PIL mode**.
- Then, in the **Top Model Mode**, select **Normal**.
- To simulate the model, generate and run the code on the hardware, and compare the results, click **Run Verification**. The results are shown in the Simulink Data Inspector.



A comparison of controller output values is shown between Normal and PIL simulations. The error tolerance of $1e-3$ is acceptable for this example.



See Also

Functions

raspi | generatePolicyBlock

Objects

rlSACAgent | rlPPOAgent | rlTrainingOptions

Blocks

Predict | RL Agent

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14

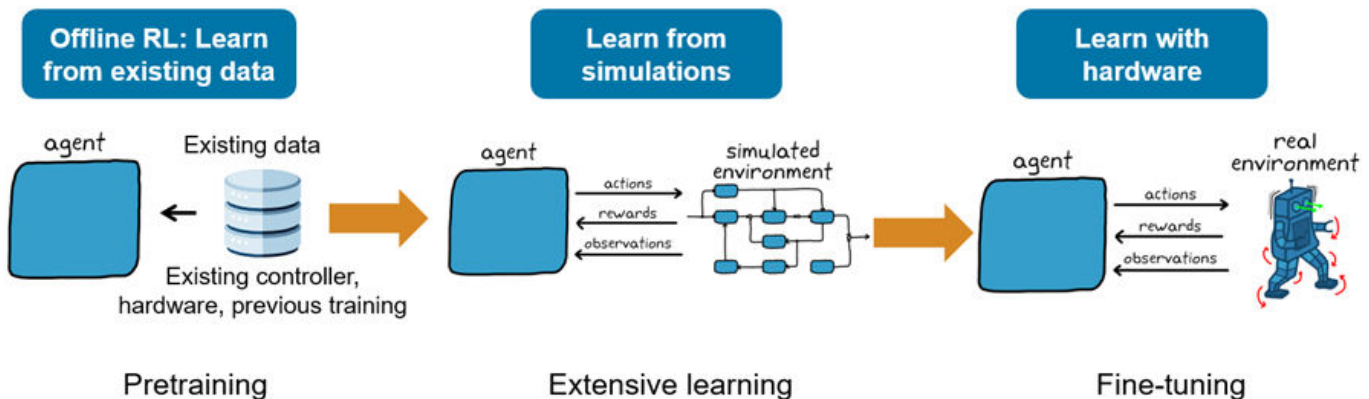
More About

- “SIL and PIL Simulations” (Embedded Coder)
- “Reinforcement Learning Workflow” on page 1-5
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Generate Code from Trained Reinforcement Learning Policies” on page 6-2
- “Build and Deploy Your First Simulink Model to Raspberry Pi” (Raspberry Pi Blockset)

Examine Approaches to Fine Tune a Deployed Policy

While learning a policy by interacting with a simulated environment has many practical advantages, there can be a substantial "reality gap" between the simulated environment and real world conditions. This gap often results from factors such as inaccurate or unknown model dynamics and environmental conditions. For these reasons, it might be necessary to refine a deployed policy using data collected from in real world conditions, to ensure it can effectively and safely solve a real-world task.

If data collected from previous simulation, (or real world data), is available, offline reinforcement learning can be used to pre-train the agent. The pretrained agent can then be further improved by interacting with a simulated environment. Finally, to refine the policy so it can also learn in real world conditions, you can deploy it on control hardware. This picture illustrates the workflow.



When deploying an agent to the real world, in theory, you could use a few different architectures to allocate the required computations.

Agent Runs on Desktop and Interacts with Real World

The first (simplest) architecture would rely on a custom MATLAB environment that sends actions and receives observations from the physical system. The agent then interacts with the custom environment within a reinforcement learning loop running in a MATLAB desktop process. In this architecture, the hardware applies the received actions to the physical system, then measures new observations and sends them back to the MATLAB desktop process.

The problem with this first approach is that the time that the agent needs to learn and execute its policy is both unpredictable, because the MATLAB process typically runs within a non-realtime operating system, and often longer than the sampling time needed to safely and successfully control the physical system. For similar reasons, the time needed to send actions and receive observations over a communication channel is also unpredictable and has a latency that is hard to constrain. So, this first architecture typically cannot accommodate for the real time and sampling rate constraints needed to control the physical system unless the physical system dynamics are sufficiently slow.

Agent Runs on Dedicated Hardware Board

A second possible way to allocate the required computations is to generate lower level code (typically C/C++) for the agent and deploy the generated code on a dedicated hardware board that interacts

with the physical system. This board typically runs a real time operating system, or at least an operating system with predictable process latency.

The problem with this second approach is that a typical dedicated controller board lacks the memory and computational power necessary to run the agent learning algorithm, especially for off-policy agents. In other words, this second approach might be feasible only by using a very powerful on-board computer with an appropriately capable data acquisition board.

Desktop Algorithm Trains Deployed Policy

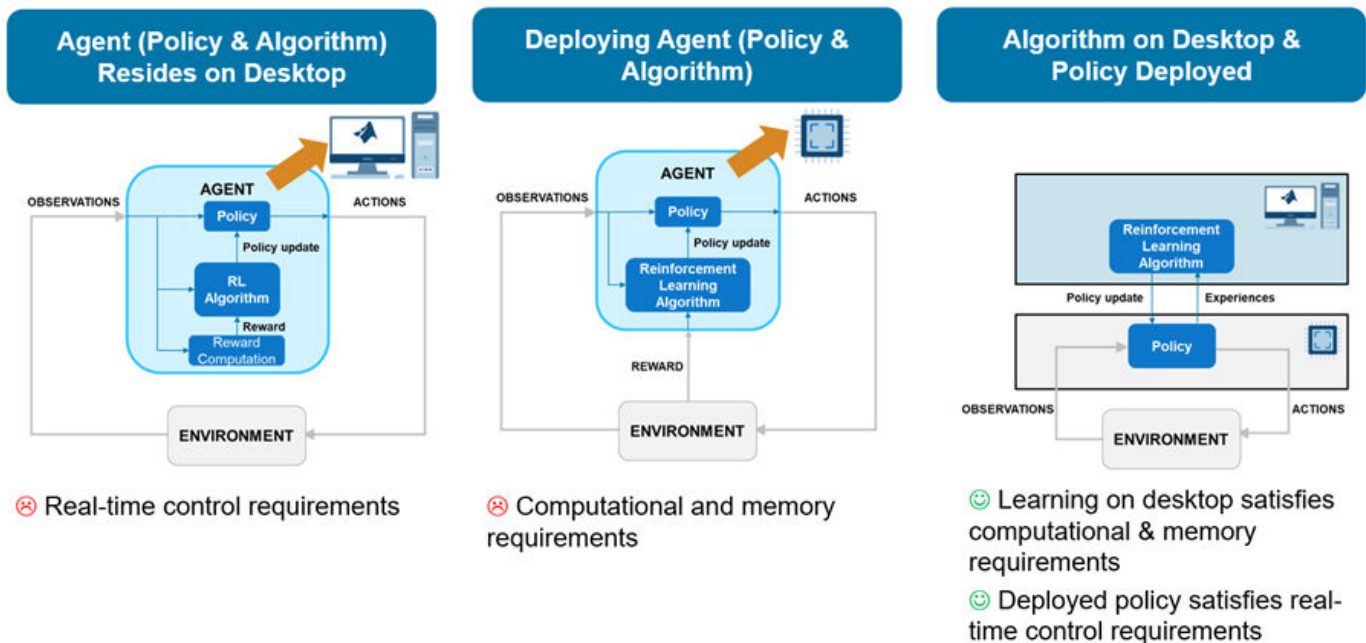
A third possible architecture separates the control policy from the learning algorithm. Specifically, the control policy runs on a dedicated experiment process (typically on a controller board) while the learning algorithm runs in a MATLAB process on a desktop computer. Here, differently from the previous architectures, learning happens asynchronously with respect to the hardware process, that is, the policy process computes actions independently of any learning that might take place.

This architecture makes it easier to satisfy real-time constraints, especially on systems with fast sample times, since the experiment process is not directly concerned with learning. Furthermore, the learning process can be executed on a powerful machine with a powerful GPU to accelerate learning.

In this architecture, the experiment process collects experiences by interacting with the physical system and periodically sends these experiences to the learning process. The learning process then uses the experiences received from the policy process to update the learnable parameters of the agent, and then periodically sends the updated parameters to the experiment process. The experiment process then updates its policy parameters with the new parameters received by the learning process. For an example implementing this architecture, see “Train Policy Deployed on Raspberry Pi” on page 6-31.

Summary: Advantages and Disadvantages

This picture illustrates the advantages and disadvantages of the three architectures.



Transferring a policy that has been trained in a simulated environment to a real-world setting is also referred to as a "Sim to Real" workflow. There are several intermediate verification steps that can facilitate such workflow, such as Software In the Loop (SIL, in which the code generated from the agent or policy runs in within a simulation), Processor In the Loop (PIL, in which the generated code from the agent or policy runs on a dedicated board and interacts with the simulation) and Hardware In the Loop (HIL, in which a model of the environment runs in real time on dedicated hardware). For an example showing how you can apply the SIL and PIL steps to a policy (that does not need to learn after deployment), see "Run SIL and PIL Verification for Reinforcement Learning" on page 6-17. For more information, see "SIL and PIL Simulations" (Embedded Coder).

See Also

Functions

`generatePolicyFunction` | `generatePolicyBlock` | `policyParameters` | `updatePolicyParameters` | `trainFromData`

Related Examples

- "Generate Policy Block for Deployment" on page 6-7
- "Run SIL and PIL Verification for Reinforcement Learning" on page 6-17
- "Train Policy Deployed on Raspberry Pi" on page 6-31
- "Deploy Trained Reinforcement Learning Policy as Microservice Docker Image" (MATLAB Compiler SDK)

More About

- "Reinforcement Learning Workflow" on page 1-5
- "Generate Code from Trained Reinforcement Learning Policies" on page 6-2
- "Train Reinforcement Learning Agents" on page 5-3

Train Policy Deployed on Raspberry Pi

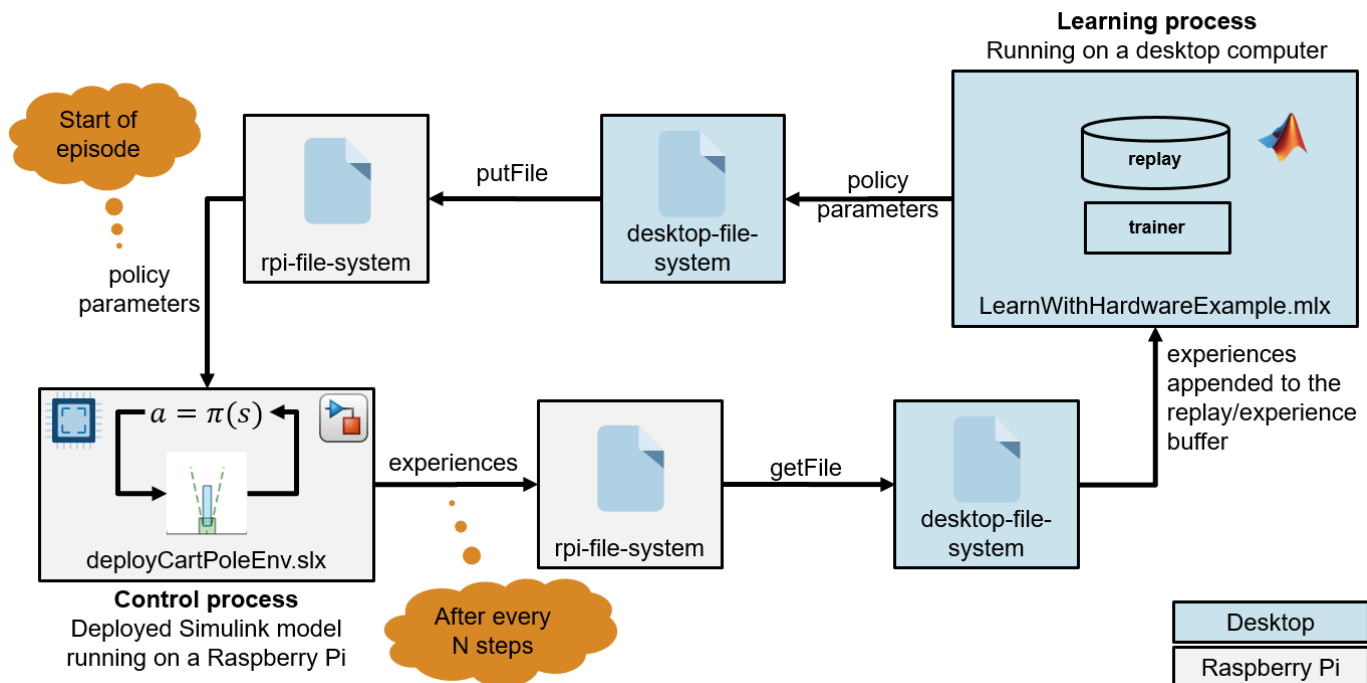
This example shows how to train an agent in MATLAB® on a desktop computer using data generated by a policy deployed on a Raspberry Pi® board. The policy controls a simulated environment, also running on the board.

Introduction

When an agent needs to interact with a physical system, in theory, you could use a few different architectures to allocate the required computations. Three different architectures, with their advantages and disadvantages are discussed in “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28. This example implements the third discussed architecture.

Here, the learning algorithm runs in a MATLAB process running on a desktop computer, while the policy (that is, the control process) runs on a Raspberry Pi board. The policy collects experiences by interacting with the physical system, and periodically sends these experiences to the learning process. The learning process then uses the experiences received from the policy process to update the actor and critic parameters of the agent, and then periodically sends the updated actor parameters to the control process. The control process then updates the policy parameters with the new parameters received by the learning process.

The figure illustrates this architecture.

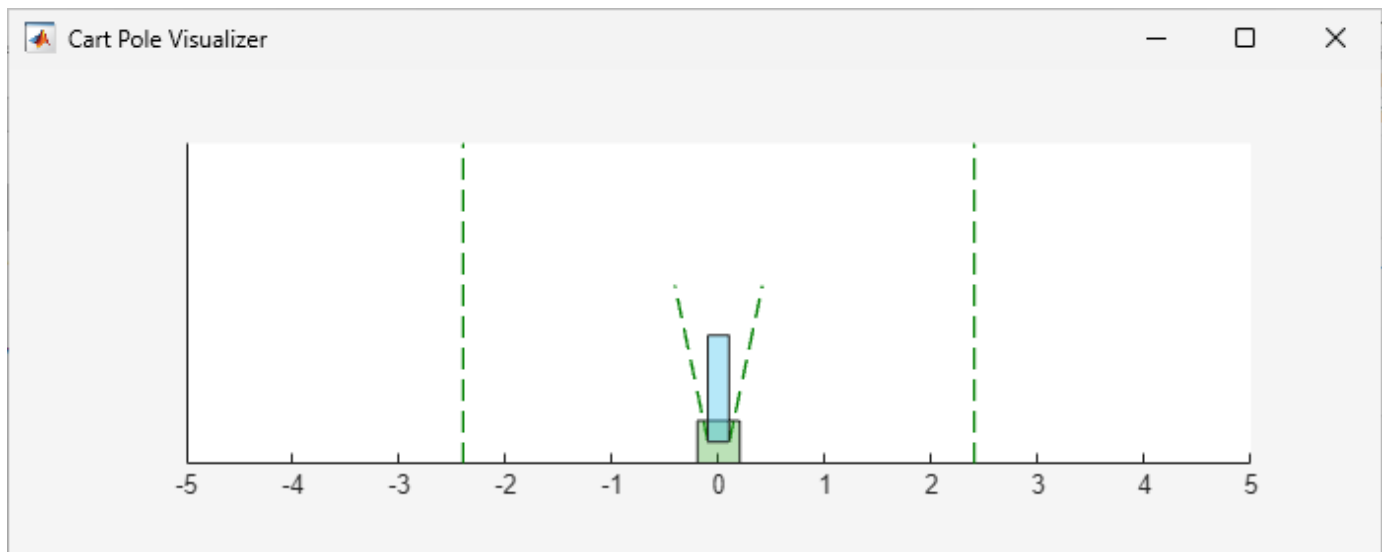


In this figure, the *control process* is deployed (from a Simulink® model) on an Raspberry Pi board. The control process runs the agent policy, which collects experiences by controlling a simulated cart-pole environment. The process regularly updates its policy parameters with new policy parameters read from a parameter file that the learning process regularly writes on the RPI board. The control process also periodically saves the collected experiences to experience files in the Raspberry Pi board file system.

The *learning process* runs (in MATLAB) on a desktop computer. This process reads the files on the Raspberry Pi board that contain the new collected experiences, uses the experiences to train the agent, and periodically writes on the Raspberry Pi board a file that contains new policy parameters for the control board to read.

Create Cart-Pole Environment Specifications

Within the control process, the policy interacts with a simulated continuous action space cart-pole environment. In this environment, the goal of the agent is to balance a pole in an upright position on a moving cart by applying a horizontal force to the cart.



For this environment:

- The upward balanced pendulum position is zero radians and the downward hanging position is π radians.
- The pendulum starts upright with an initial angle of zero radians.
- The force action signal from the agent to the environment is from -10 N to 10 N. A positive force value pushes the cart to the right, while a negative force value pushes it to the left.
- The observations from the environment are the position and velocity of the cart, the pendulum angle (clockwise-positive), and the pendulum angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of +0.5 is provided for every time-step that the pole remains upright. An additional reward is provided based on the distance between the cart and the origin. A penalty of -50 is applied when the pendulum falls.
- The sample time for the environment and the agent is 0.02 seconds.

Define the observation specification `obsInfo` and the action specification `actInfo`.

```
obsInfo = rlNumericSpec([4 1]);
obsInfo.Name = "CartPole States";
obsInfo.Description = "x, dx, theta, dtheta";

actInfo = rlNumericSpec([1 1]);
```

```
actInfo.LowerLimit = -10;
actInfo.UpperLimit = 10;
actInfo.Name = "CartPole Action";
```

Define the sample time.

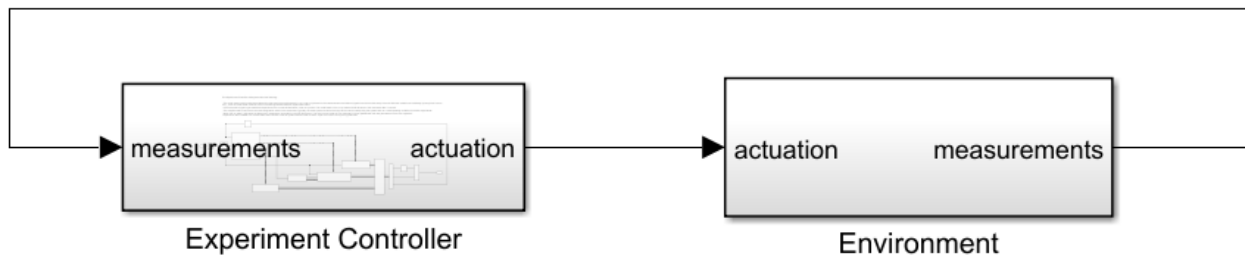
```
sampleTime = 0.02;
```

Review Control Process Simulink Model and Define Environment Parameters

The control process code is generated from a Simulink model and deployed on the Raspberry Pi board. This process runs indefinitely, because the stop time of the Simulink models used to generate the process executable is set to Inf.

Open the model.

```
mdl = 'deployCartPoleEnv';
open_system(mdl);
```

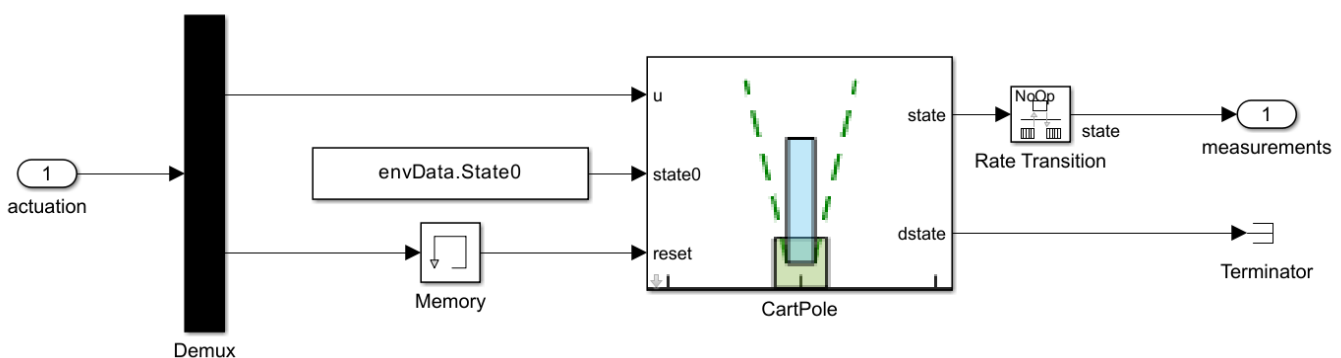


Copyright 2025 The MathWorks, Inc.

This model consists of two main subsystems:

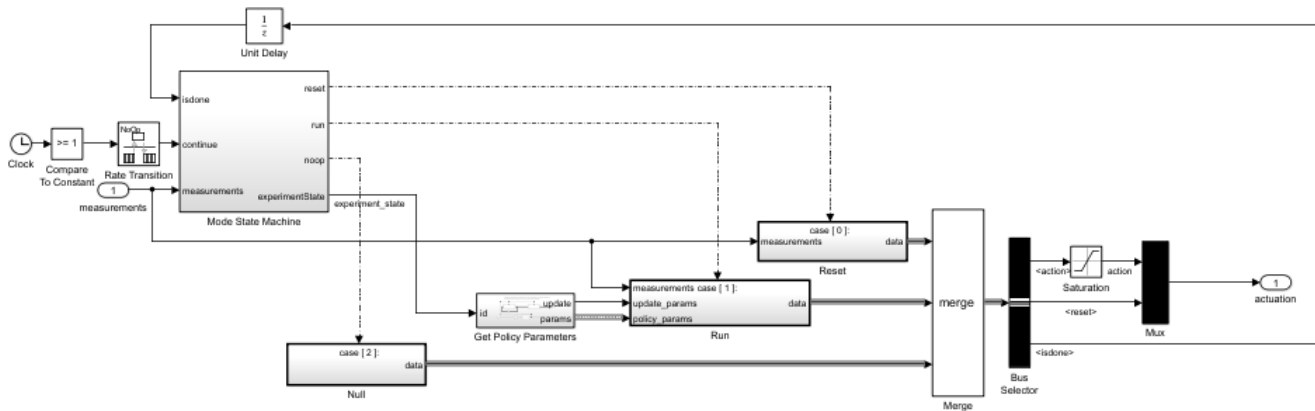
- The Environment subsystem models the continuous action space cart-pole environment.
- The Experiment Controller subsystem contains the reinforcement learning policy, the experiment mode state switching control module, the policy parameter update module, and the experience saving module.

The Environment subsystem implements the cart-pole system. Specifically, the CartPole block takes actuation u (the horizontal force applied to the cart), initial state $state0$, and reset signal as inputs and returns the next state $state$ and the derivative of the state $dstate$ as outputs.

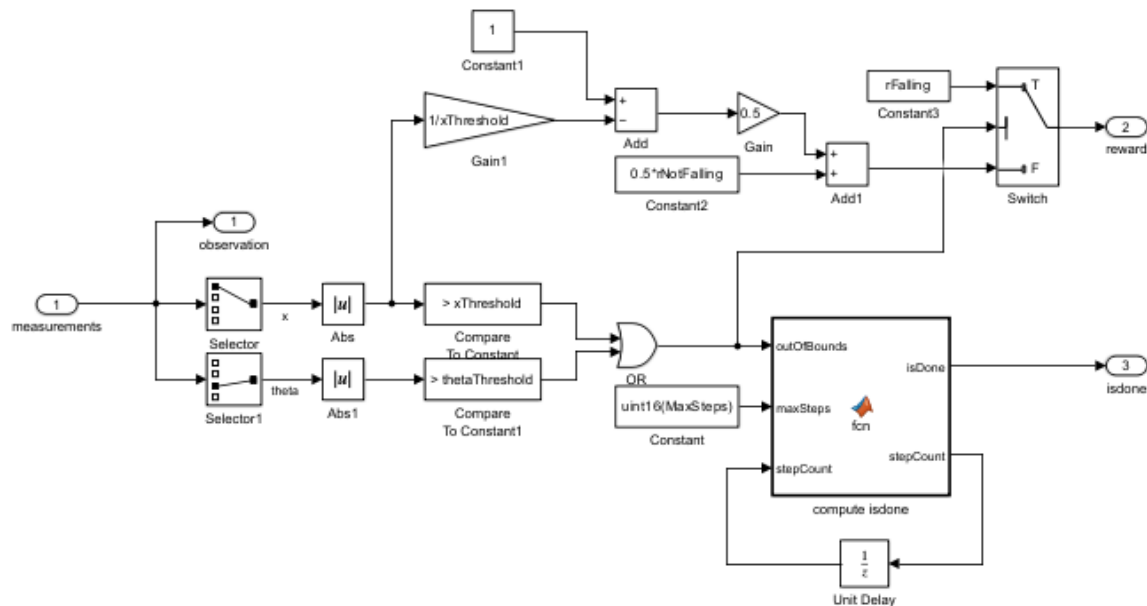


Within the Experiment Controller subsystem:

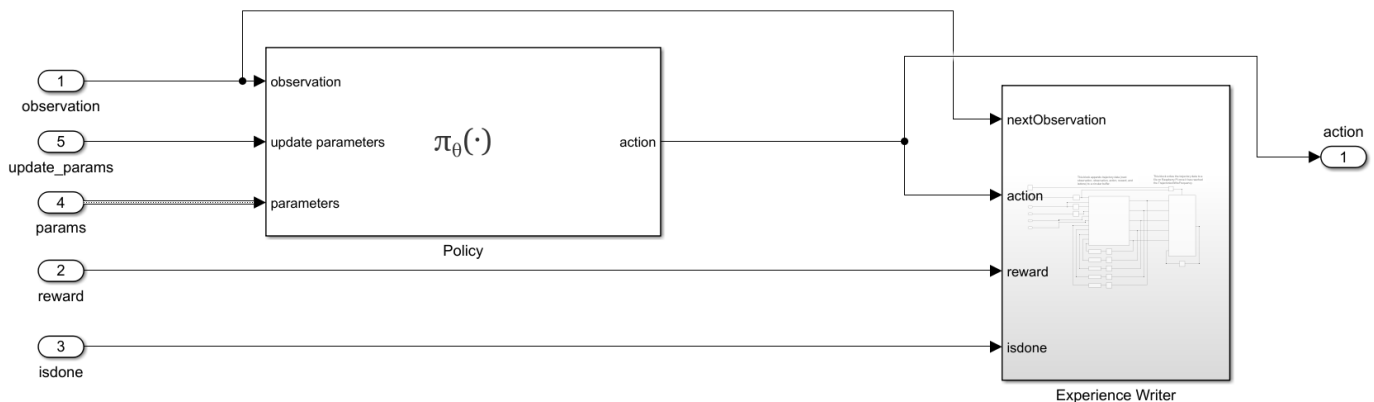
- The **Mode State Machine** subsystem determines the experiment mode states (no-op, reset, run) based on the measurements and `isdone` signals from the last time step. Because the Simulink model runs indefinitely (its stop time is set to `Inf`), the mode state machine is the one switching between different experiment states.
- The subsystem generates a zero actuation signal and does not transmit experiences when in the no-op state. The model starts in the no-op state and will advance to the reset state after one second.
- The cart-pole state is set back to its initial configuration when in the reset state. Typically, the reset system involves logic and control to safely bring the system back to a valid operating condition for further experiments. This reset system is a critical part of the experiment process.
- When the run state is activated, a MATLAB function called by the **Get Policy Parameters** subsystem reads the latest policy parameters from a parameter file in the Raspberry Pi file system. The policy block inside the Run subsystem is then updated with the new parameters for the next "episode".



- The **Process Data** subsystem (under the Run subsystem, and shown in the following figure) computes the observation, reward, and `isdone` signals given the measurements from the **Environment** subsystem. For this example, the observation is identical to the measurements.



- The Remote Agent subsystem (under the Run subsystem, and shown in the following figure) maps observations to actions using the updated policy parameters. Additionally, the Experience Writer subsystem collects experiences in a circular buffer and writes the buffer to experience files in the Raspberry Pi file system, with a frequency given by the `envData.ExperiencesWriteFrequency` parameter.



You can use the command `generatePolicyBlock(agent)` to generate a Simulink policy block. For more information, see `generatePolicyBlock`.

Use the helper function `environmentParameters` on page 6-48 to define the parameters needed by the environment. This function is defined at the end of this example.

```
envData = environmentParameters(sampleTime);
```

Create a structure representing a circular buffer to store experiences. The buffer is saved to a file based on the writing frequency specified by `envData.ExperiencesWriteFrequency`.

```
tempExperienceBuffer.NextObservation = ...  
    zeros(obsInfo.Dimension(1),envData.ExperiencesWriteFrequency);
```

```
tempExperienceBuffer.Observation = ...
    zeros(obsInfo.Dimension(1),envData.ExperiencesWriteFrequency);
tempExperienceBuffer.Action = ...
    zeros(actInfo.Dimension(1),envData.ExperiencesWriteFrequency);
tempExperienceBuffer.Reward = ...
    zeros(1,envData.ExperiencesWriteFrequency);
tempExperienceBuffer.IsDone = ...
    zeros(1,envData.ExperiencesWriteFrequency,"uint8");
```

You use this structure as a reference when the policy parameters are converted from bytes of data to a structure (deserialization).

```
agentData.TempExperienceBuffer = tempExperienceBuffer;
```

Create SAC Agent

Fix the random number stream with the seed 0 and random number algorithm Mersenne Twister to reproduce the same initial learnables used in the agent. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You restore the state at the end of the example.

Create an agent initialization object to initialize the actor and critic networks with the hidden layer size 128. For more information on agent initialization options, see `rlAgentInitializationOptions`.

```
initOptions = rlAgentInitializationOptions("NumHiddenUnit",128);
```

Create a SAC agent using the observation and action input specifications, and initialization options. For more information, see `rlSACAgent`.

```
agent = rlSACAgent(obsInfo,actInfo,initOptions);
```

Specify the agent options for training. For more information, see `rlSACAgentOptions`. For this training:

- Specify the sample time, experience buffer length, and mini-batch size.
- Specify the actor and critic learning rates to be $1e-3$, and entropy weight learning rate to be $3e-4$. A large learning rate causes drastic updates that can lead to divergent behaviors, while a low value can require many updates before reaching the optimal point.
- Use a gradient threshold of 1 to clip the gradients. Clipping the gradients can improve training stability.
- Specify the initial entropy component weight and the target entropy value.

```
agent.SampleTime = sampleTime;
agent.AgentOptions.ExperienceBufferLength = 1e6;
agent.AgentOptions.MinibatchSize = 128;
agent.AgentOptions.LearningFrequency = 1;
agent.AgentOptions.MaxMinibatchPerEpoch = 1;
```

```
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
for c = 1:2
```



```

agent.AgentOptions.CriticOptimizerOptions(c).LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions(c).GradientThreshold = 1;
end

```

```

agent.AgentOptions.EntropyWeightOptions.LearnRate = 3e-4;
agent.AgentOptions.EntropyWeightOptions.EntropyWeight = 0.1;
agent.AgentOptions.EntropyWeightOptions.TargetEntropy = -1;

```

Define offline (train from data) training options using the `rlTrainingFromDataOptions` object. You use `trainFromData` to train the agent from the collected experience data appended to the experience buffer. Set the number of steps to 100 and the max epochs to 1 to take 100 learning steps to train the agent when new experiences are collected. The actual training progress (episode rewards) can be obtained from the control process and it is not necessary to plot the progress of offline training.

```

trainFromDataOptions = rlTrainingFromDataOptions;
trainFromDataOptions.MaxEpochs = 1;
trainFromDataOptions.NumStepsPerEpoch = 100;
trainFromDataOptions.Plots = "none";

agentData.Agent = agent;
agentData.TrainFromDataOptions = trainFromDataOptions;

```

Build, Deploy, and Start the Control Process on Raspberry Pi Board

Connect to the Raspberry Pi board using the IP address, user name, and password. For this example, you need to install the Raspberry Pi® Blockset. See “Get Started with Raspberry Pi Blockset” (Raspberry Pi Blockset) for more information about how to install the support package and set up your Raspberry Pi hardware.

Replace the information in the code below with the login credentials of your Raspberry Pi board.

```

ipaddress = 172.31.172.54 ;
username   = guestuser ;
password   = guestpasswd ;

```

Create a `raspberrypi` object and store it in the `agentData` structure.

```

agentData.RPi = raspberrypi(ipaddress,username,password)

agentData = struct with fields:
    TempExperienceBuffer: [1x1 struct]
    Agent: [1x1 rl.agent.rlSACAgent]
    TrainFromDataOptions: [1x1 rl.option.rlTrainingFromDataOptions]
    RPi: [1x1 raspberrypi]

```

Create variables that store the folder and file information for parameters and experiences.

```

% Folder for storing experiences on Raspberry
experiencesFolderRPi = "~/cartPoleExperiments/experiences/";
% Folder for storing parameters on Raspberry
parametersFolderRPi = "~/cartPoleExperiments/parameters/";

% Folder for storing experiences on desktop computer

```

```

experiencesFolderDesktop = ...
    fullfile(".", "cartPoleExperiments", "experiences/");
% Folder for storing parameters on desktop computer
parametersFolderDesktop = ...
    fullfile(".", "cartPoleExperiments", "parameters/");

parametersFile = "parametersFile.dat";

```

Use the structure `agentData` to store information related to the agent.

```

agentData.ExperiencesFolderRPI = experiencesFolderRPI;
agentData.ExperiencesFolderDesktop = experiencesFolderDesktop;
agentData.ParametersFolderRPI = parametersFolderRPI;
agentData.ParametersFolderDesktop = parametersFolderDesktop;
agentData.ParametersFile = parametersFile;

```

Create a local folder to save the parameters and experiences if it doesn't already exist.

```

if ~exist(fullfile(".", "cartPoleExperiments/"), 'dir')
    % Create the experiences folder.
    mkdir(fullfile(".", "cartPoleExperiments/"));
end

```

Remove files containing variables that store old experience information if the experiences folder exists.

```

if exist(experiencesFolderDesktop, 'dir')
    % If the experiences folder exists,
    % remove the folder to delete any old files
    rmdir(experiencesFolderDesktop, 's');
end

```

Create the folder to collect the experiences.

```
mkdir(experiencesFolderDesktop);
```

Remove files containing variables that store old parameter information if the parameters folder exists.

```

if exist(parametersFolderDesktop, 'dir')
    % If the parameter folder exists,
    % remove the folder to delete any old files.
    rmdir(parametersFolderDesktop, 's');
end

```

Make the parameters folder.

```
mkdir(parametersFolderDesktop);
```

Check if the folder where experience files are stored exists on the RPi. If the folder does not exist, create one. To create the folder, use the supporting function `createFolderOnRPI` on page 6-48, which is defined at the end of this example. This function uses the `system` command, with the Raspberry Pi object as a first input argument, to send a command to the Raspberry Pi Linux® operating system. For more information, see “Run Linux Shell Commands on Raspberry Pi Hardware” (Raspberry Pi Blockset).

```
createFolderOnRPI(agentData.RPI, agentData.ExperiencesFolderRPI);
```

Check if the parameters folder exists on the RPi. If not, create one.

```
createFolderOnRPi(agentData.RPi,agentData.ParametersFolderRPi);
```

Read the current state of the experiences folder using the `listFolderContentsOnRPi` on page 6-48 function. This function is defined at the end of this example.

```
dirContent = listFolderContentsOnRPi( ...
    agentData.RPi,agentData.ExperiencesFolderRPi);
```

Check for and remove all the old experience files from the RPi.

```
if ~isempty(dirContent)
    system(agentData.RPi,convertStringsToChars("rm -f " + ...
        agentData.ExperiencesFolderRPi + ".*.*"));
end
```

The deployed policy uses an exploratory policy to collect data.

```
agentData.UseExplorationPolicy = true;
```

Use the `writePolicyParameters` on page 6-44 function to save the initial policy parameters to a file on the desktop computer and to send the parameters in the `agentData` structure to the Raspberry Pi. This function is defined at the end of this example.

```
writePolicyParameters(agentData);
```

Store the policy parameter structure to `envData`. This structure is used as a reference to convert the serialized data to a structure in the `Get Policy Parameters` subsystem mentioned earlier.

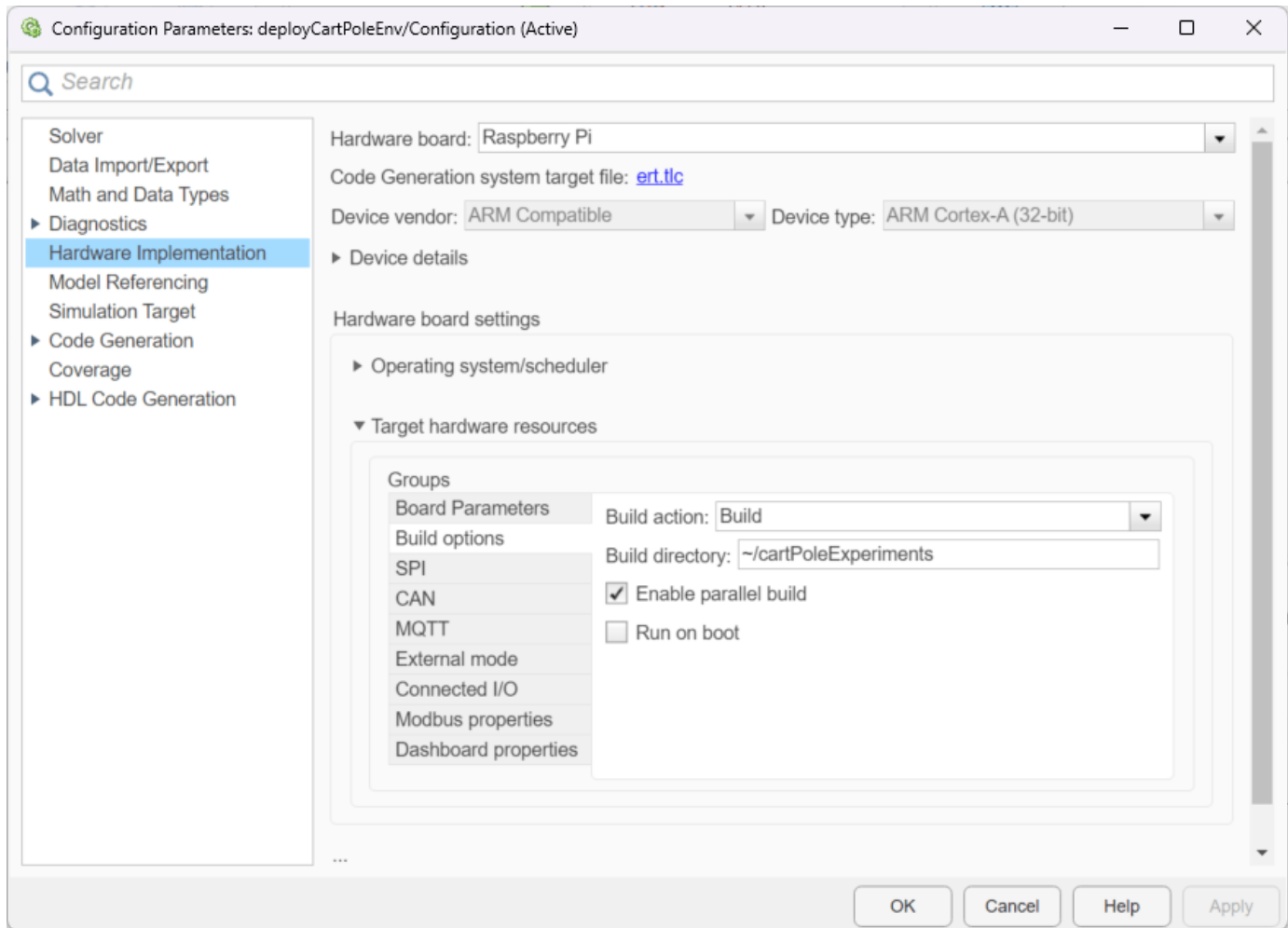
```
envData.ParametersStruct = policyParameters( ...
    getExplorationPolicy(agentData.Agent));
```

Use the `slbuild` (Simulink) command to build and deploy the model on the Raspberry Pi board.

Use the `evalc` function to capture the text output from code generation, for possible later inspection.

```
buildLog = evalc("slbuild mdl");
```

The model is configured only to build and deploy. You can also specify the build directory.



Start the model deployed to the Raspberry Pi Board.

```
runModel(agentData.RPi,mdl);
```

You can use `isModelRunning` to determine whether the model is running on the board. The `isModelRunning` method returns `true` if the model is running on the Raspberry Pi board.

```
isModelRunning(agentData.RPi,mdl)
```

```
ans = logical
     1
```

Run Training Loop

The example executes the training loop, as a part of the learning process, on the desktop computer.

Create variables to monitor training.

```
trainingResultData.CumulativeRewardTemp = 0;
trainingResultData.CumulativeReward = [];
trainingResultData.AverageCumulativeReward = [];
```

```
trainingResultData.AveragingWindowLength = 10;
trainingResultData.NumOfEpisodes = 0;
```

Stop the training when the agent receives an average cumulative reward greater than 470 or a maximum of 5000 episodes.

```
rewardStopTrainingValue = 470;
maxEpisodes = 5000;
```

To train the agent, set doTraining to true.

```
doTraining =  ;
```

Create a figure for training visualization using the buildRemoteTrainingMonitor on page 6-47 helper function. This function is defined at the end of this example.

```
if doTraining
    [trainingPlot, ...
     trainingResultData.LineReward, ...
     trainingResultData.LineAverageReward] = ...
        buildRemoteTrainingMonitor();
    % Enable the training visualization plot.
    set(trainingPlot,Visible="on");
end
```

The training repeats these steps:

- 1 Check for any new experiences from the deployed control process Simulink model. If new experiences exist, get them from the Raspberry Pi board to the desktop computer.
- 2 Read the experiences and append them to the replay buffer using the readNewExperiences on page 6-45 function. This function is defined at the end of this example. After the experiences are read, they are deleted from the Raspberry Pi board.
- 3 Train the agent from the collected data using the trainFromData function.
- 4 Save the updated policy parameters to the Raspberry Pi board using the writePolicyParameters on page 6-44 function. This function is defined at the end of this example.

```
if doTraining
    while true
        % Look for any experience files.
        dirContent = listFolderContentsOnRPi( ...
            agentData.RPi,agentData.ExperiencesFolderRPi);

        if ~isempty(dirContent)

            % Display the newfound experience files.
            fprintf('%s %4d new experience files found\n', ...
                datetime('now'),numel(dirContent));

            % Use the readNewExperiences function to read
            % new experience files and append experiences
            % to the replay buffer.
            [agentData, trainingResultData] = readNewExperiences( ...
                dirContent,agentData,trainingResultData);
```

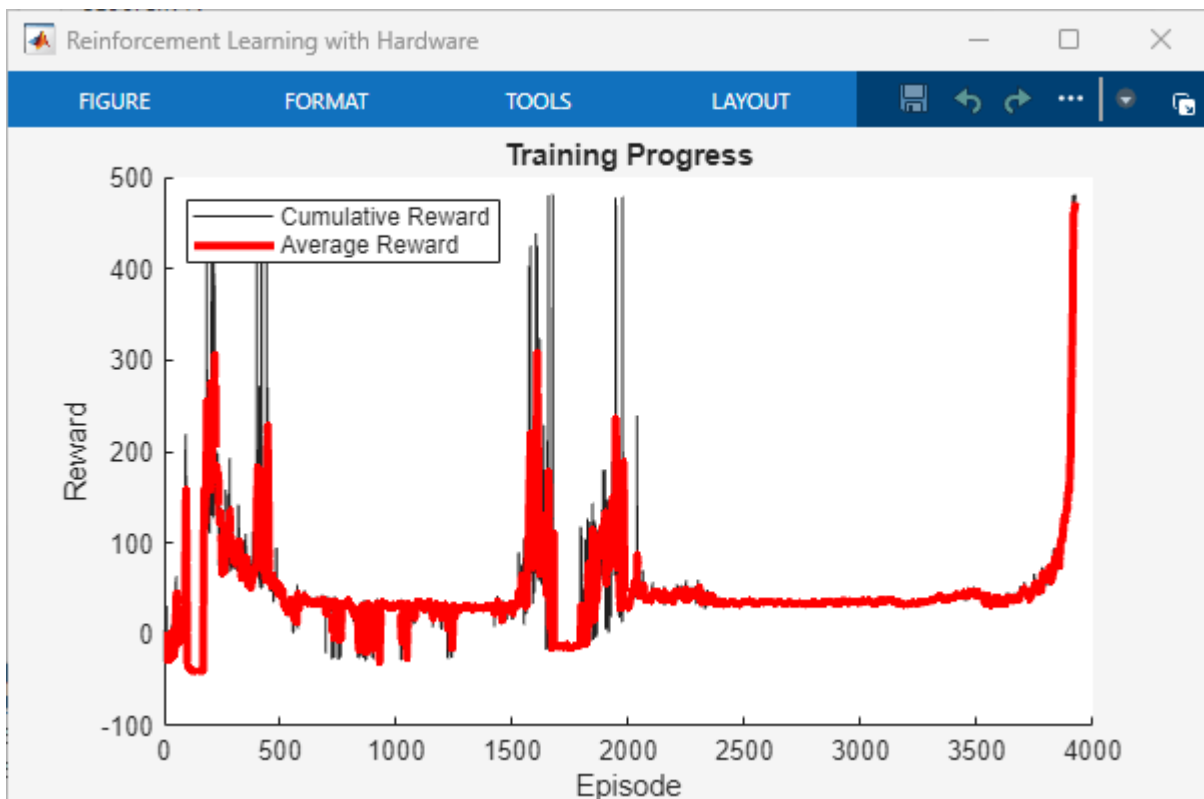
```

% Perform a learning step for the agent
% using data in the experience buffer.
offlineTrainStats = trainFromData( ...
    agentData.Agent,agentData.TrainFromDataOptions);

% Save the updated actor parameters to a file on the
% desktop computer and send the file to the Raspberry Pi.
writePolicyParameters(agentData);
end

if ~isempty(trainingResultData.AverageCumulativeReward)
    % Check if at least one experience was read to prevent
    % AverageCumulativeReward reward from being empty.
    if trainingResultData.AverageCumulativeReward(end) > ...
        rewardStopTrainingValue ...
        || trainingResultData.NumOfEpisodes>=maxEpisodes
        % Use break to exit the training loop when
        % the average cumulative reward is greater than
        % rewardStopTrainingValue or the number of
        % episodes is more than maxEpisodes.
        break
    end
end
end
else
    % If you did not train the agent, load the agent from a file.
    load("trainedAgentHardware.mat","agent");
    agentData.Agent = agent;
end
end

```



Stop the model deployed to the Raspberry Pi board.

```
if isModelRunning(agentData.RPi,mdl)
    stopModel(agentData.RPi,mdl);
end
```

Simulate Trained Policy

Simulate the trained policy on the Raspberry Pi board using greedy actions. Change the policy to act greedily by setting the policy parameter `Policy_UseMaxLikelihoodAction = true`.

Send the policy parameters to the Raspberry Pi board.

```
writePolicyParameters(agentData);
```

Remove all the old experience files on the board.

```
system(agentData.RPi,convertStringsToChars("rm -f " + ...
    agentData.ExperiencesFolderRPi + "*,*"));
```

Create variables to monitor the simulation.

```
simulationResultData.CumulativeRewardTemp = 0;
simulationResultData.CumulativeReward = [];
simulationResultData.AverageCumulativeReward = [];
simulationResultData.AveragingWindowLength = 10;
simulationResultData.NumOfEpisodes = 0;
simulationMaxEpisodes = 5;
```

Start the model deployed to the Raspberry Pi board.

```
if ~isModelRunning(agentData.RPi,mdl)
    runModel(agentData.RPi,mdl);
end
```

The simulation repeats these steps:

- 1 Check for new experiences from the deployed control process. If new experiences exist, send them from the Raspberry Pi board to the desktop computer.
- 2 Read the experiences and append them to the replay buffer using the `readNewExperiences` function.

```
while true
    % Check for new experiences.
    dirContent = listFolderContentsOnRPi( ...
        agentData.RPi, agentData.ExperiencesFolderRPi);

    if ~isempty(dirContent)

        fprintf('%s %4d new experiences found\n', ...
            datetime('now'), numel(dirContent));
```

```
29-Jan-2025 16:14:40      1 new experiences found
29-Jan-2025 16:14:44      1 new experiences found
29-Jan-2025 16:14:49      1 new experiences found
29-Jan-2025 16:14:53      1 new experiences found
29-Jan-2025 16:14:57      1 new experiences found
```

```
29-Jan-2025 16:15:02      1 new experiences found
29-Jan-2025 16:15:06      1 new experiences found
29-Jan-2025 16:15:11      1 new experiences found
29-Jan-2025 16:15:15      1 new experiences found
29-Jan-2025 16:15:19      1 new experiences found
29-Jan-2025 16:15:24      1 new experiences found
29-Jan-2025 16:15:28      1 new experiences found
29-Jan-2025 16:15:33      1 new experiences found

    % Read new experience files
    % and append experiences to the replay buffer.
    [agentData, simulationResultData] = readNewExperiences( ...
        dirContent,agentData,simulationResultData);
end

if simulationResultData.NumOfEpisodes>=simulationMaxEpisodes
    % Use break to exit the simulation loop when the number of
    % episodes is more than simulationMaxEpisodes.
    break
end
end
```

Compute the average of episode rewards. This value shows that the policy stabilizes the pole on the cart.

```
mean(simulationResultData.CumulativeReward)
```

Stop the model deployed to the Raspberry Pi board.

```
if isModelRunning(agentData.RPi,mdl)

ans =
477.9915

    stopModel(agentData.RPi,mdl);
end
```

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

Local Functions

Write Policy Parameters

The `writePolicyParameters` function extracts the policy parameters from the agent, writes them to a file, and sends the file to the Raspberry Pi board.

```
function writePolicyParameters(agentData)

    % Open file to save the policy parameters on the desktop computer.
    filepath = fullfile( ...
        agentData.ParametersFolderDesktop, agentData.ParametersFile);
    fid = fopen(filepath,'w+');
    cln = onCleanup(@( )fclose(fid));

    if agentData.UseExplorationPolicy
        % Get exploration policy parameters.
```



```

        parameters = policyParameters( ...
            getExplorationPolicy(agentData.Agent));
    else
        % Get greedy policy parameters.
        parameters = ...
            policyParameters(getGreedyPolicy(agentData.Agent));
    end

    % Convert the policy parameters into a vector of uint8s
    % (serialization) and save it to a file.
    fwrite(fid,structToBytes(parameters),'uint8');

    % Send the file from desktop to the Raspberry Pi board.
    putFile(agentData.RPi, ...
        convertStringsToChars(fullfile( ...
            agentData.ParametersFolderDesktop, ...
            agentData.ParametersFile)), ...
        convertStringsToChars( ...
            agentData.ParametersFolderRPi + ...
            agentData.ParametersFile));
end

```

Read New Experiences

The `readNewExperiences` function moves new experience files from the Raspberry Pi board to the desktop computer, reads the experiences, and appends the experiences to the replay buffer.

```

function [agentData, trainingResultData] = ...
    readNewExperiences(dirContent,agentData, trainingResultData)

for ii=1:length(dirContent)

    % Move file from the Raspberry Pi board to the desktop computer.
    getFile(agentData.RPi, ...
        convertStringsToChars( ...
            agentData.ExperiencesFolderRPi+dirContent(ii)), ...
        convertStringsToChars(agentData.ExperiencesFolderDesktop));

    % Remove the moved file from the Raspberry Pi board.
    system(agentData.RPi,convertStringsToChars("rm -f " + ...
        agentData.ExperiencesFolderRPi+dirContent(ii)));

    % Open the moved file on the desktop to read the data.
    fid = fopen(fullfile( ...
        agentData.ExperiencesFolderDesktop,dirContent(ii)),'r');
    cln = onCleanup(@( )fclose(fid));

    % Read data from the file.
    experiencesBytes = fread(fid,'*uint8');

    % Convert the uint8s to the experience structure.
    tempExperienceBuffer = bytesToStruct( ...
        experiencesBytes,agentData.TempExperienceBuffer);

    % Compute the cumulative reward and update the plots.
    trainingResultData = computeCumulativeReward( ...
        trainingResultData,tempExperienceBuffer);
end

```

```

% Get the experiences ready to append to the replay buffer.
for kk=size(tempExperienceBuffer.Observation,2):-1:1
    experiences(kk).NextObservation = ...
        {tempExperienceBuffer.NextObservation(:,kk)};
    experiences(kk).Observation = ...
        {tempExperienceBuffer.Observation(:,kk)};
    experiences(kk).Action = ...
        {tempExperienceBuffer.Action(:,kk)};
    experiences(kk).Reward = ...
        tempExperienceBuffer.Reward(kk);
    experiences(kk).IsDone = ...
        tempExperienceBuffer.IsDone(kk);
end

% Append the new experiences to the replay buffer.
append(agentData.Agent.ExperienceBuffer,experiences);
end
end

```

Compute Cumulative Reward from Experiences

The `computeCumulativeReward` function computes the cumulative reward and updates the learning plot.

```

function trainingResultData = computeCumulativeReward( ...
    trainingResultData,tempExperienceBuffer)

    isDone = tempExperienceBuffer.IsDone;
    reward = tempExperienceBuffer.Reward;
    N = length(isDone);
    while N > 0
        if any(isDone) % Check for isDones
            endOfEpIdx = find(isDone,1);
            trainingResultData.CumulativeRewardTemp = ...
                trainingResultData.CumulativeRewardTemp ...
                + sum(reward(1:endOfEpIdx));

            % Append the new episode cumulative reward.
            trainingResultData.CumulativeReward = ...
                [trainingResultData.CumulativeReward; ...
                 trainingResultData.CumulativeRewardTemp];

            % Initialize temporary variable used to
            % compute the cumulative reward
            trainingResultData.CumulativeRewardTemp = 0;

            % Update the plots.
            trainingResultData.NumOfEpisodes = ...
                trainingResultData.NumOfEpisodes+1;
            trainingResultData.AverageCumulativeReward = ...
                movmean(trainingResultData.CumulativeReward, ...
                    trainingResultData.AveragingWindowLength,1);

            % Update the monitor.
            if isfield(trainingResultData, "LineReward") && ...
                ~isempty(trainingResultData.LineReward)

```

```

        addpoints( ...
            trainingResultData.LineReward, ...
            trainingResultData.NumOfEpisodes, ...
            trainingResultData.CumulativeReward(end));

        addpoints( ...
            trainingResultData.LineAverageReward, ...
            trainingResultData.NumOfEpisodes, ...
            trainingResultData.AverageCumulativeReward(end));
    end
    drawnow;

    % Truncate the trajectory.
    isDone(1:endOfEpIdx) = [];
    reward(1:endOfEpIdx) = [];
    N = length(isDone);

else
    % If no more isDone exist, compute the cumulative reward
    % and store it using the temporary variable.
    trainingResultData.CumulativeRewardTemp = ...
        trainingResultData.CumulativeRewardTemp ...
        + sum(reward(1:N));
    N = 0;
end
end
end
end

```

Create Figure for Training Visualization

The buildRemoteTrainingMonitor function creates a figure for training visualization.

```

function [trainingPlot, lineReward, lineAverageReward] = ...
    buildRemoteTrainingMonitor()

    plotRatio = 16/9;
    trainingPlot = figure( ...
        Visible="off", ...
        HandleVisibility="off", ...
        NumberTitle="off", ...
        Name="Reinforcement Learning with Hardware");

    trainingPlot.Position(3) = ...
        plotRatio * trainingPlot.Position(4);

    ax = gca(trainingPlot);

    lineReward = animatedline(ax);
    lineAverageReward = animatedline(ax,Color="r",LineWidth=3);
    xlabel(ax,"Episode");
    ylabel(ax,"Reward");
    legend(ax,"Cumulative Reward","Average Reward", ...
        Location="northwest")
    title(ax,"Training Progress");
end

```

Assign Parameter Values Related to Environment

The `environmentParameters` function assigns several parameter values needed for the environment.

```
function envData = environmentParameters(sampleTime)

    % Initial condition of the pendulum
    envData.State0      = zeros(4,1);

    envData.G           = 9.8;           % m/s^2
    envData.MCart       = 1.0;           % kg
    envData.HCart       = 0.4;           % m
    envData.WCart       = 0.4;           % m
    envData.MPole       = 0.1;           % kg
    envData.LPole       = 0.5;           % m
    envData.MaxForce    = 10.0;          % N
    envData.ThetaUpper   = 0.05;         % rad
    envData.ThetaLower  = -0.05;        % rad
    envData.XThreshold   = 2.4;          % m
    envData.ThetaThreshold = 12*pi/180; % rad
    envData.ActGain     = 1.0;
    envData.RewardFall   = -50;
    envData.RewardNoFall = 1;
    envData.MaxStepsPerSim = uint32(500);
    envData.Ts           = sampleTime; % sec
    envData.ExperiencesWriteFrequency = 200;

end
```

`end`

Create Folder on Raspberry Pi Board

The `createFolderOnRPi` function creates a folder on the Raspberry Pi board if not exists.

```
function createFolderOnRPi(RPi, FolderRPi)

    system(RPi,convertStringsToChars("if [ ! -d " + ...
        FolderRPi + " ]; then mkdir -p " + ...
        FolderRPi + "; fi"));

end
```

`end`

List Contents of a Folder on a Raspberry Pi Board

The `listFolderContentsOnRPi` function lists contents of a folder on the Raspberry Pi board.

```
function dirContent = listFolderContentsOnRPi(RPi,experiencesFolderRPi)

    dirTemp = system(RPi,convertStringsToChars("ls " + ...
        experiencesFolderRPi));
    dirContent = strsplit(dirTemp);

end
```

```
        dirContent(end) = [];  
    end
```

See Also

Functions

`policyParameters` | `updatePolicyParameters` | `generatePolicyBlock` | `trainFromData`

Blocks

`Policy`

Objects

`rlReplayMemory` | `rlStochasticActorPolicy` | `rlSACAgent` | `rlSACAgentOptions` | `rlTrainingFromDataOptions`

Related Examples

- “Generate Policy Block for Deployment” on page 6-7
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum” on page 5-184
- “Control a Quanser QUBE Pendulum with a Raspberry Pi using Reinforcement Learning” on page 6-50

More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Create Custom Reinforcement Learning Agents” on page 3-92
- “Use Predefined Control System Environments” on page 2-42

Control a Quanser QUBE Pendulum with a Raspberry Pi using Reinforcement Learning

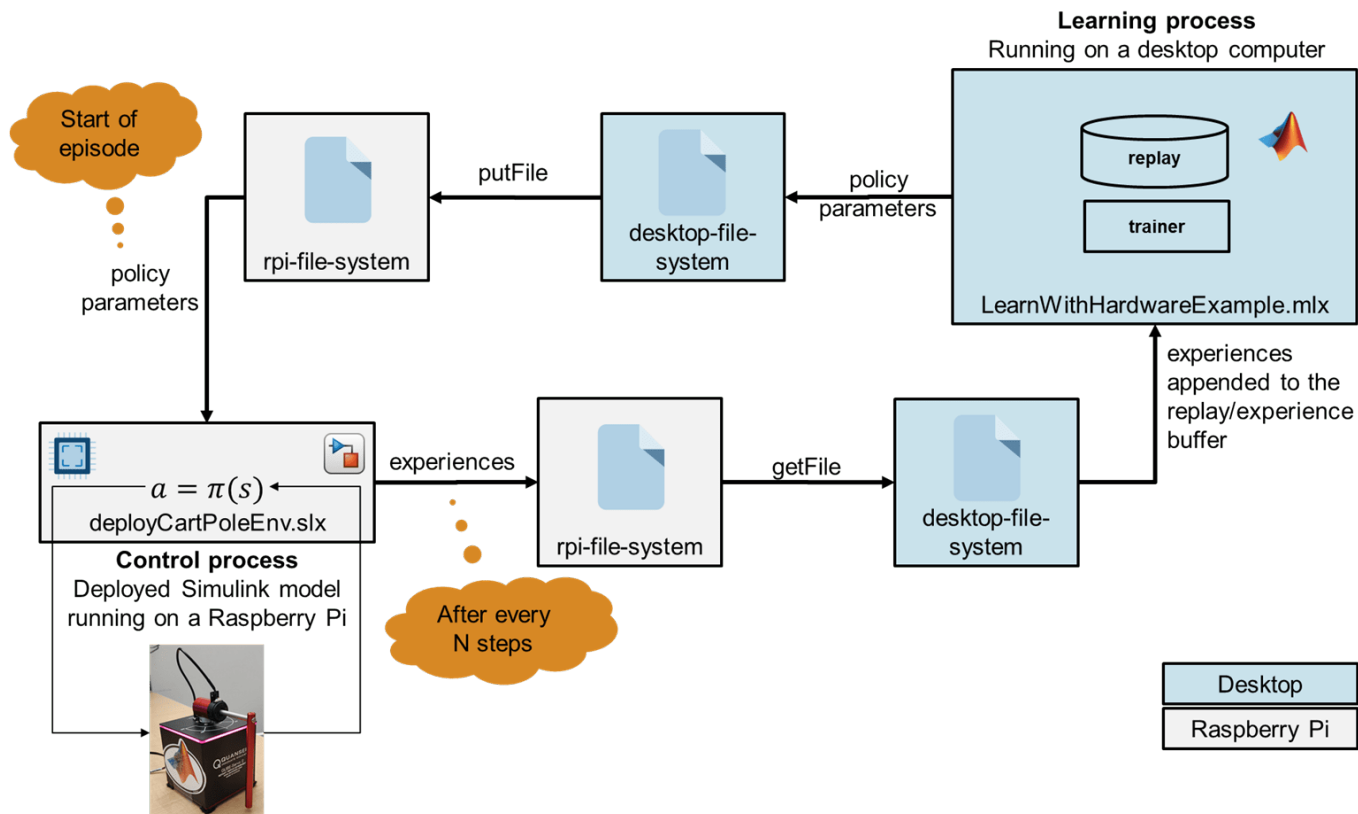
This example shows how to train a reinforcement learning policy deployed on a Raspberry Pi® board to control a Quanser QUBE™-Servo 2 inverted pendulum system. The goal of the policy is to swing up and balance the pendulum in the upright position. For more information regarding the Quanser pendulum system, see Quanser QUBE™-Servo 2.

Introduction

When an agent needs to interact with a physical system, you can use different architectures to allocate the required computations. Three different architectures, with their advantages and disadvantages, are discussed in “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28. This example implements the third architecture.

Here, the learning algorithm runs in a desktop MATLAB process, while the policy (that is, the control process) is executed on a Raspberry Pi board. The control process collects experiences by interacting with the pendulum system, and periodically sends these experiences to the learning process. The learning process then uses the experiences received from the control process to update the actor and critic parameters of the agent, and then periodically sends the updated actor parameters to the control process. The control process then updates the policy parameters with the new parameters received by the learning process.

The figure illustrates this architecture.



In this figure, the *control process* is deployed (from a Simulink® model) on a Raspberry Pi board. The Raspberry Pi board interacts with the Quanser QUBE pendulum system using a serial peripheral interface (SPI) connection. The control process runs the agent policy and collects experiences by controlling the Quanser QUBE pendulum system. The process regularly updates its policy parameters with new policy parameters read from a parameter file that the learning process regularly writes on the Raspberry Pi board. The control process also periodically saves the collected experiences to experience files in the Raspberry Pi board file system.

The *learning process* runs (in MATLAB) on a desktop computer. This process reads the files on the Raspberry Pi board that contain the new collected experiences, uses the experiences to train the agent, and periodically writes to the Raspberry Pi board a file that contains new policy parameters for the control board to read.

Create Quanser QUBE Pendulum Environment Specifications

The Quanser QUBE-Servo 2 pendulum system is a rotational inverted pendulum with two degrees of freedom. The pendulum is attached to the motor arm by a free revolute joint. The arm is actuated by a DC motor. The control process containing the agent policy is designed in Simulink and deployed (using the Raspberry Pi® Blockset) on a Raspberry Pi board, which interacts with the pendulum system using a Serial Peripheral Interface (SPI) connection. This example uses the Quanser QFLEX 2 Embedded module to enable SPI communication with the QUBE-Servo 2. The goal of the agent is to swing up and balance the pendulum in the upright position. The pendulum system is an underactuated system, with the agent controlling the motor that rotates along the vertical axis. For more information on the Quanser pendulum system, wiring, and command packet structure, see Quanser QUBE™-Servo 2.



For this environment:

- The observation is the vector $s_k = [\sin\theta_k, \cos\theta_k, \dot{\theta}_k, \sin\phi_k, \cos\phi_k, \dot{\phi}_k, u_{k-1}]$. Using the sine and cosine of the measured angles can facilitate training by representing the otherwise discontinuous angular measurements by a continuous two-dimensional parameterization.
- The action is the normalized input voltage command to the servo motor.
- The reward signal is defined as follows:

$$r(s_k, u_{k-1}, u_{k-2}) = F_k - 0.1 \left(\theta_k^2 + \phi_k^2 + 0.01 \dot{\theta}_k^2 + 0.01 \dot{\phi}_k^2 + u_{k-1}^2 + 0.3(u_{k-1} - u_{k-2})^2 \right)$$

$$F_k = \begin{cases} 1 & \theta_k \in \pm 5 \frac{\pi}{8} \text{ rad and } \dot{\phi}_k \in \pm 30 \frac{\text{rad}}{\text{sec}} \\ 0 & \text{otherwise} \end{cases}$$

The above reward function penalizes six terms:

- Deviations from the forward position of the motor arm ($\theta_k = 0$).

- Deviations for the inverted position of the pendulum ($\varphi_k = 0$).
- The angular speed of the motor arm $\dot{\theta}_k$.
- The angular speed of the pendulum $\dot{\phi}_k$.
- The control action u_k .
- Changes to the control action ($u_{k-1} - u_{k-2}$).

The system constraints F_k are needed to prevent the motor from deviating too far from the center position and to avoid the pendulum potentially hitting the power cord. They also ensure the pendulum does not swing too quickly, as swinging too fast can cause the magnetically coupled pendulum to decouple from the base. The agent is rewarded while the system constraints are satisfied (that is $F_k = 1$). Additionally, the episode is terminated early if any of the constraints are violated.

Define the observation specification `obsInfo` and action specification `actInfo`. These specifications are needed to create the agent. The sample time for the environment and the agent is 0.005 seconds.

```
obsInfo = rlNumericSpec([7 1]);
obsInfo.Description = "sinTheta, cosTheta, thetaDot, sinPhi, cosPhi, phiDot, uPrevious";

actInfo = rlNumericSpec([1 1]);
actInfo.LowerLimit = -1;
actInfo.UpperLimit = 1;
actInfo.Description = "motorVoltage";

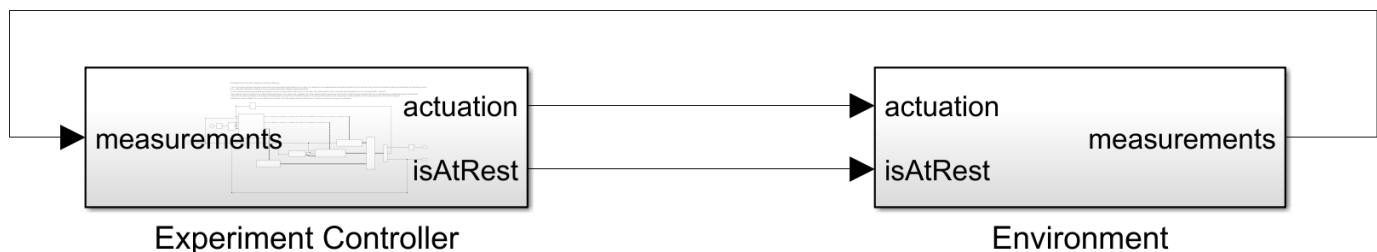
sampleTime = 0.005;
```

Review Control Process Simulink Model and Define Environment Parameters

The control process code is generated from a Simulink model and deployed on the Raspberry Pi board. This process runs indefinitely, because the stop time of the Simulink model used to generate the process executable is set to `Inf`.

Open the model.

```
mdl = 'deployQuanserQubeEnvironment';
open_system(mdl);
```



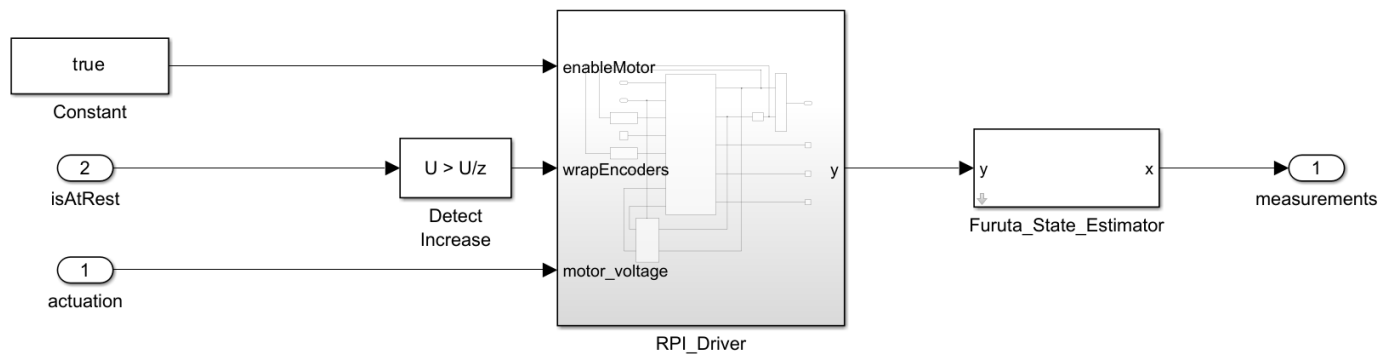
Copyright 2025 The MathWorks, Inc.

This model consists of two main subsystems:

- The `Environment` subsystem interacts with the Quanser QUBE Servo-2 hardware.

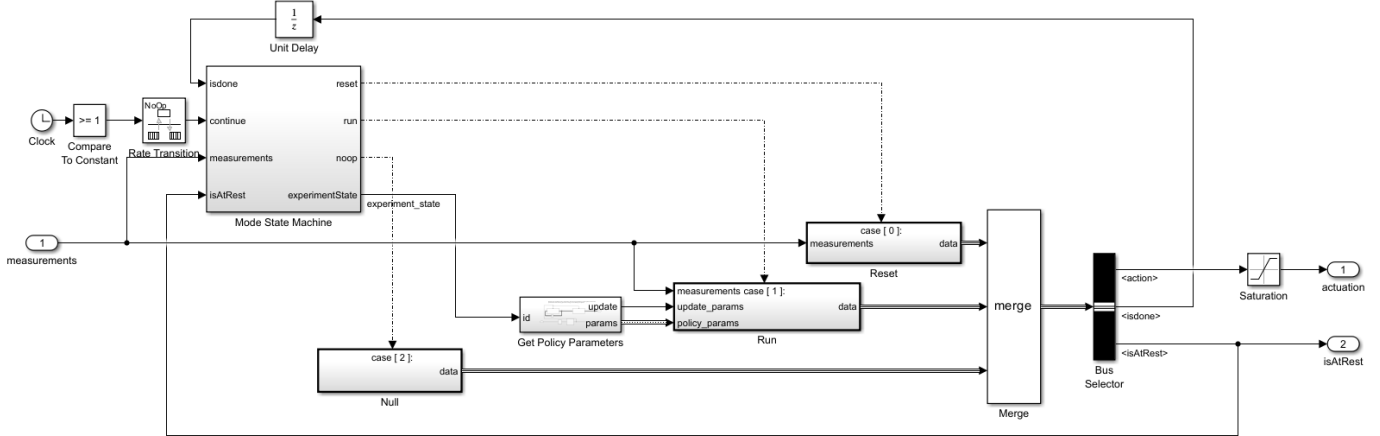
- The Experiment Controller subsystem contains the reinforcement learning policy, the experiment mode state switching control module, the policy parameter update module, and the experience saving module.

The Environment subsystem interacts with the Quanser QUBE Servo-2 hardware. Specifically, the RPI_Driver block takes the actuation (the motor voltage), the enable signal for the motor `enableMotor`, and the signal to reset the encoders as inputs and returns the next measurement `y` consisting of the motor angle (θ) and pendulum angle (ϕ) as output. It uses the SPI Controller Transfer block to write data to and read data from SPI peripheral device; see SPI Controller Transfer (Raspberry Pi Blockset) for more information. The Furuta_State_Estimator block uses the next measurement coming from the RPI_Drive block to estimate the states θ , $\dot{\theta}$, ϕ , and $\dot{\phi}$.

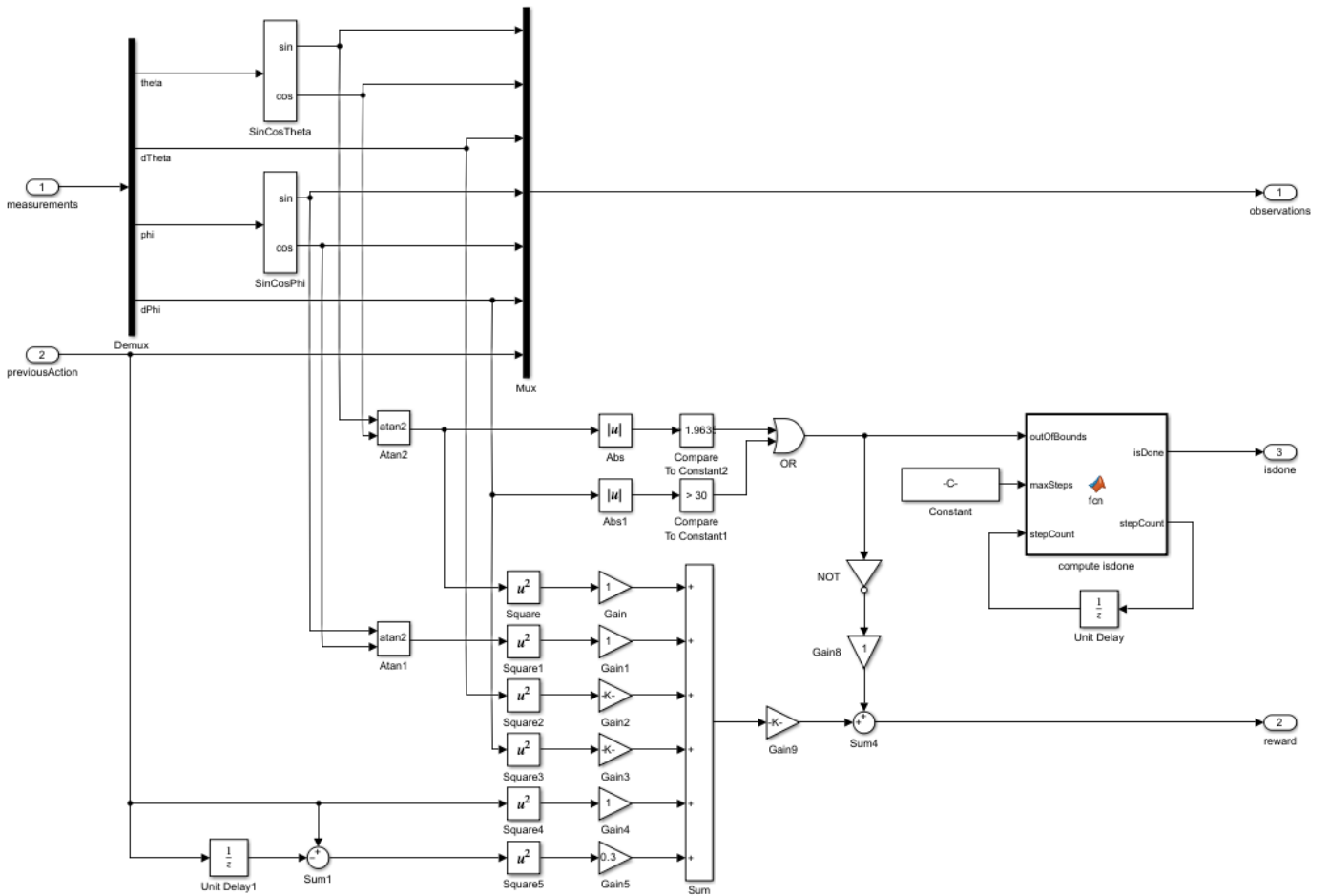


Within the Experiment Controller subsystem:

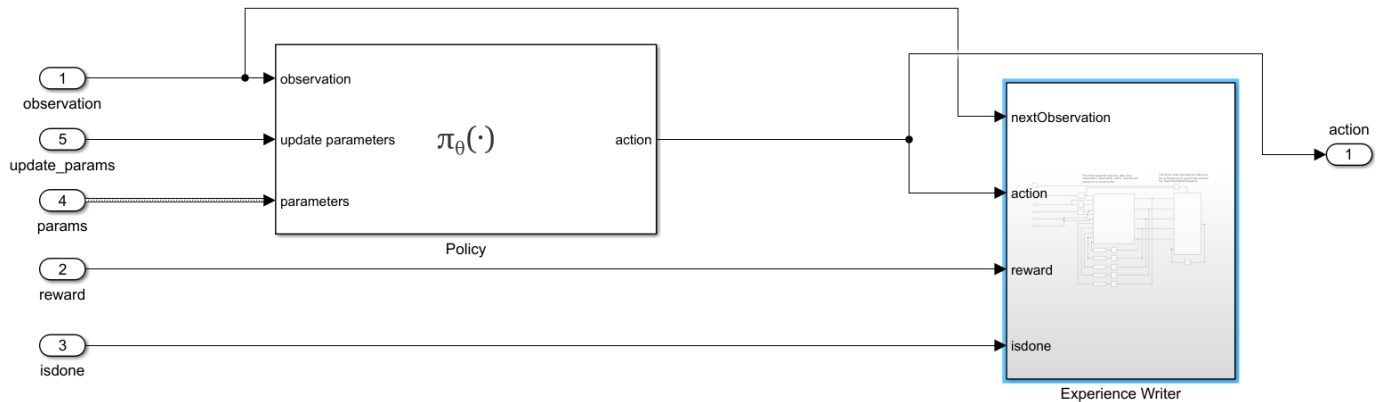
- The Mode State Machine subsystem determines the experiment mode states (no-op, reset, and run) based on the measurements and the `isdone` signals from the last time step. Because the Simulink model runs indefinitely (its stop time is set to `Inf`), the Mode State Machine is responsible for switching between different experiment states. It does so by enabling the Reset, Run and Null subsystems according to the needed mode.
- When in the no-op state, the Experiment Controller subsystem generates a zero actuation signal and does not transmit experiences to the learning process running on the MATLAB desktop. The model starts in the no-op state and advances to the reset state after one second.
- A reset system with appropriate logic and control is an essential part of the experiment process, as it is needed to safely return the system to a valid operating condition for further experiments or runs. In this example, the system is reset to its initial configuration ($\theta = 0$ for the motor arm and $\phi = \pi$ or $-\pi$ for the pendulum) using a linear-quadratic regulator (LQR), which takes the states $(\theta, \dot{\theta}, \phi, \dot{\phi})$ as inputs and outputs the motor voltage needed to quickly achieve the desired configuration.
- When the run state is activated, a MATLAB function called by the Get Policy Parameters subsystem reads the latest policy parameters from a parameter file in the Raspberry Pi file system. The policy block inside the Run subsystem is then updated with the new parameters for the next "episode."



- The Process Data subsystem (under the Run subsystem, and shown in the following figure) computes the observations, reward, and isdone signals given the measurements from the Environment subsystem. For this example, the measurements are θ , $\dot{\theta}$, ϕ , and $\dot{\phi}$. The seven observations $s_k = [\sin\theta_k, \cos\theta_k, \dot{\theta}_k, \sin\phi_k, \cos\phi_k, \dot{\phi}_k, u_{k-1}]$ are computed using the measurements.



- The Remote Agent subsystem (under the Run subsystem, and shown in the following figure) maps observations to actions using the updated policy parameters. Additionally, the Experience Writer subsystem collects experiences in a circular buffer and writes the buffer to experience files in the Raspberry Pi file system, with a frequency given by the `envData.ExperiencesWriteFrequency` parameter.



You can use the command `generatePolicyBlock(agent)` to generate a Simulink policy block. For more information, see `generatePolicyBlock`.

Use the local function `environmentParameters` on page 6-69 to define the parameters needed by the environment and the reset controller. This function is defined at the end of this example.

```
envData = environmentParameters(sampleTime);
```

Create a structure representing a circular buffer to store experiences. The buffer is saved to a file based on the writing frequency specified by `envData.ExperiencesWriteFrequency`.

```
tempExperienceBuffer.NextObservation = ...
    zeros(obsInfo.Dimension(1),envData.ExperiencesWriteFrequency);
tempExperienceBuffer.Observation = ...
    zeros(obsInfo.Dimension(1),envData.ExperiencesWriteFrequency);
tempExperienceBuffer.Action = ...
    zeros(actInfo.Dimension(1),envData.ExperiencesWriteFrequency);
tempExperienceBuffer.Reward = ...
    zeros(1,envData.ExperiencesWriteFrequency);
tempExperienceBuffer.IsDone = ...
    zeros(1,envData.ExperiencesWriteFrequency,"uint8");
```

You use this structure as a reference when the policy parameters are converted from bytes of data to a structure (deserialization).

```
agentData.TempExperienceBuffer = tempExperienceBuffer;
```

Create SAC Agent

Fix the random number stream with the seed 0 and random number algorithm Mersenne Twister to reproduce the same initial learnable parameters used in the agent. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You restore the state at the end of the example.

Create an agent initialization object to initialize the actor and critic networks with the hidden layer size 64. For more information on agent initialization options, see `rlAgentInitializationOptions`.

```
initOptions = rlAgentInitializationOptions("NumHiddenUnit",64);
```

Create a default SAC agent using the observation specifications, action specifications, and initialization options. For more information, see `rlSACAgent`.

```
agent = rlSACAgent(obsInfo,actInfo,initOptions);
```

Specify the agent options for training. For more information, see `rlSACAgentOptions`. For this training:

- Specify the sample time, experience buffer length, and mini-batch size.
- Set the actor and critic learning rates to $1e-3$, and entropy weight learning rate to $3e-4$. A fast learning rate causes drastic updates that can lead to divergent behaviors, while a slow learning rate value can require many updates before reaching the optimal point.
- Use a gradient threshold of 1 to clip the gradients. Clipping the gradients can improve training stability.
- Specify the initial entropy component weight and the target entropy value.

```
agent.SampleTime = sampleTime;
agent.AgentOptions.ExperienceBufferLength = 1e6;
agent.AgentOptions.MiniBatchSize = 256;

agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
for criticIndex = 1:2
    agent.AgentOptions.CriticOptimizerOptions(criticIndex).LearnRate = 1e-3;
    agent.AgentOptions.CriticOptimizerOptions(criticIndex).GradientThreshold = 1;
end

agent.AgentOptions.EntropyWeightOptions.LearnRate = 3e-4;
agent.AgentOptions.EntropyWeightOptions.EntropyWeight = 0.1;
agent.AgentOptions.EntropyWeightOptions.TargetEntropy = -1;
```

Define offline training options using the `rlTrainingFromDataOptions` object. You use `trainFromData` to train the agent from the collected experience data appended to the experience buffer. Set the number of steps to 200 and the max epochs to 1 to take 200 learning steps to train the agent when new experiences are collected. The actual training progress (episode rewards) can be obtained from the control process, so you do not need to plot the progress of offline training.

```
trainFromDataOptions = rlTrainingFromDataOptions;
trainFromDataOptions.MaxEpochs = 1;
trainFromDataOptions.NumStepsPerEpoch = 200;
trainFromDataOptions.Plots = "none";

agentData.Agent = agent;
agentData.TrainFromDataOptions = trainFromDataOptions;
agentData.Ts = sampleTime;
```

Build, Deploy, and Start the Control Process on Raspberry Pi Board

Connect to the Raspberry Pi board using the IP address, username, and password. For this example, you need Raspberry Pi® Blockset. See “Get Started with Raspberry Pi Blockset” (Raspberry Pi Blockset) for details on setting up your Raspberry Pi hardware.

Replace the information in the code below with the login credentials of your Raspberry Pi board.

```
ipaddress = 172.31.172.170 ;
username = guestuser ;
password = guestpasswd ;
```

Create a `raspberrypi` object and store it in the `agentData` structure.

```
agentData.RPi = raspberrypi(ipaddress,username,password)

agentData = struct with fields:
    TempExperienceBuffer: [1x1 struct]
                      Agent: [1x1 rl.agent.rlSACAgent]
    TrainFromDataOptions: [1x1 rl.option.rlTrainingFromDataOptions]
                      Ts: 0.0050
                      RPi: [1x1 raspberrypi]
```

Create variables that store folder and file information for parameters and experiences.

```
% Folder for storing experiences on Raspberry Pi
experiencesFolderRPi = "~/pendulumExperiments/experiences/";
% Folder for storing parameters on Raspberry Pi
parametersFolderRPi = "~/pendulumExperiments/parameters/";

% Folder for storing experiences on desktop computer
experiencesFolderDesktop = ...
    fullfile(".", "pendulumExperiments", "experiences/");
% Folder for storing parameters on desktop computer
parametersFolderDesktop = ...
    fullfile(".", "pendulumExperiments", "parameters/");

parametersFile = "parametersFile.dat";
```

Use the structure `agentData` to store information related to the agent.

```
agentData.ExperiencesFolderRPi = experiencesFolderRPi;
agentData.ExperiencesFolderDesktop = experiencesFolderDesktop;
agentData.ParametersFolderRPi = parametersFolderRPi;
agentData.ParametersFolderDesktop = parametersFolderDesktop;
agentData.ParametersFile = parametersFile;
```

Create a local folder to save the parameters and experiences if you do not already have one.

```
if ~exist(fullfile(".", "pendulumExperiments/"), 'dir')
    % Making the experiences folder
    mkdir(fullfile(".", "pendulumExperiments/"));
end
```

Remove files containing variables that store old experience information if the experiences folder exists.

```
if exist(experiencesFolderDesktop, 'dir')
    % If the experiences folder exists,
    % remove the folder to delete any old files.
    rmdir(experiencesFolderDesktop, 's');
end
```

Create the folder to collect the experiences.

```
mkdir(experiencesFolderDesktop);
```

Remove files containing variables that store old parameter information if the parameters folder exists.

```
if exist(parametersFolderDesktop, 'dir')
    % If the parameter folder exists,
    % remove the folder to delete any old files.
    rmdir(parametersFolderDesktop, 's');
end
```

Create the parameters folder.

```
mkdir(parametersFolderDesktop);
```

Check if the folder where experience files are stored exists on the Raspberry Pi. If the folder does not exist, create one. To create the folder, use the supporting function `createFolderOnRPi` on page 6-69, which is defined at the end of this example. This function uses the system command, with the Raspberry Pi object as first input argument, to send a command to the Raspberry Pi Linux operating system. For more information, see “Run Linux Shell Commands on Raspberry Pi Hardware” (Raspberry Pi Blockset).

```
createFolderOnRPi(agentData.RPi, agentData.ExperiencesFolderRPi);
```

Check if the parameters folder exists on the Raspberry Pi board. If not, create one.

```
createFolderOnRPi(agentData.RPi, agentData.ParametersFolderRPi);
```

Read the current state of the experiences folder using `listFolderContentsOnRPi` on page 6-69. This function is defined at the end of this example.

```
dirContent = listFolderContentsOnRPi( ...
    agentData.RPi, agentData.ExperiencesFolderRPi);
```

Check for and remove all the old experience files from the Raspberry Pi board.

```
if ~isempty(dirContent)
    system(agentData.RPi, convertStringsToChars("rm -f " + ...
        agentData.ExperiencesFolderRPi + ".*.*"));
end
```

Use the `writePolicyParameters` on page 6-65 function to save the initial policy parameters to a file on the desktop computer and to send the parameters in the `agentData` structure to the Raspberry Pi. This function is defined at the end of this example.

```
agentData.UseExplorationPolicy = true;
writePolicyParameters(agentData);
```

Store the policy parameter structure in `envData`. This structure is used as a reference to convert the serialized data to a structure in the Get Policy Parameters subsystem mentioned earlier.

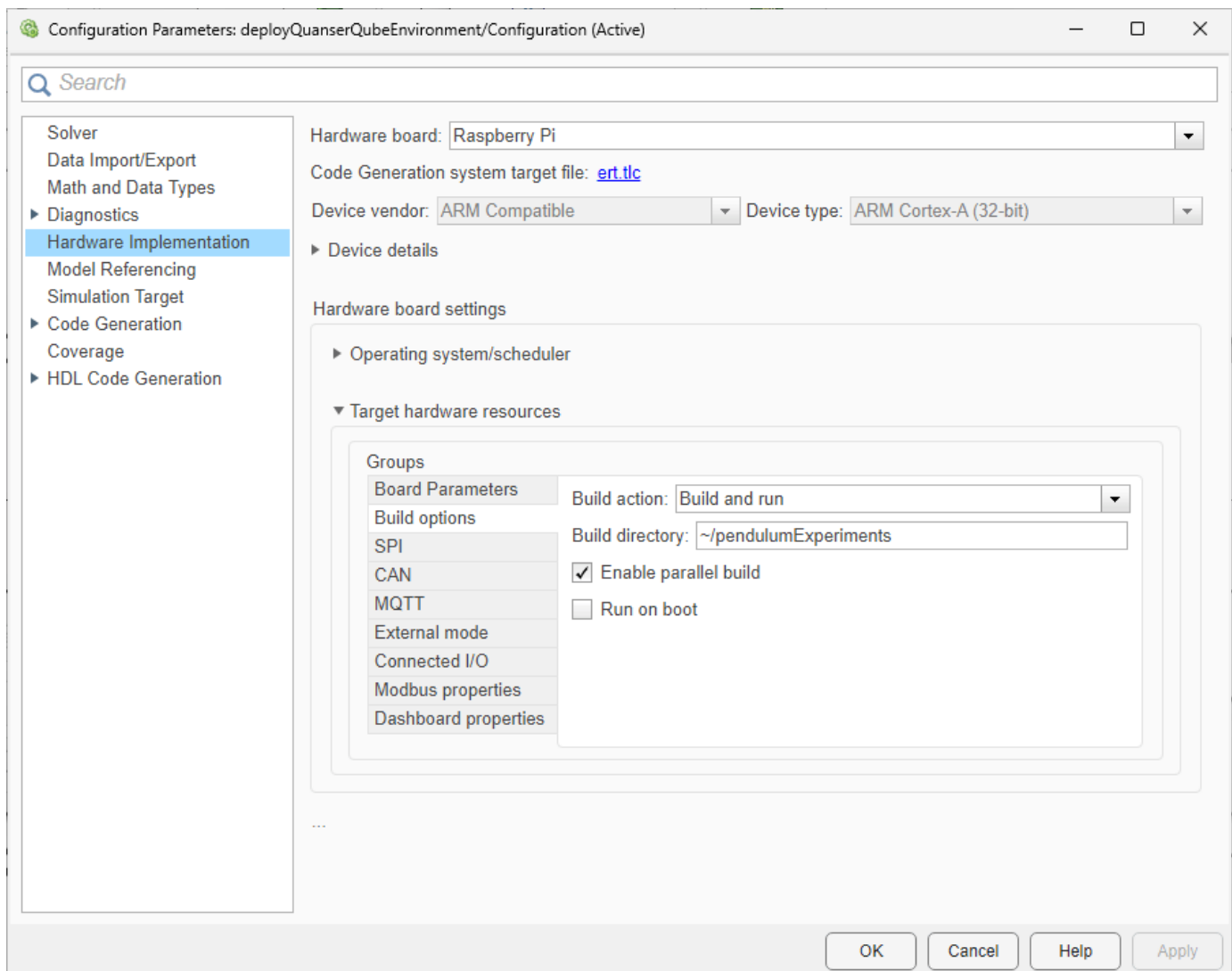
```
envData.ParametersStruct = policyParameters( ...  
    getExplorationPolicy(agentData.Agent));
```

Use the `slbuild` (Simulink) (Simulink) command to build and deploy the model on the Raspberry Pi board.

Use `evalc` to capture the text output from code generation, for possible later inspection.

```
buildLog = evalc("slbuild mdl");
```

The model is configured only to build and deploy. You can also specify the build directory.



Start the model deployed on the Raspberry Pi Board.

```
runModel(agentData.RPi, mdl);
```


You can use the `isModelRunning` function to determine whether the model is running on the board. The `isModelRunning` returns `true` if the model is running on the Raspberry Pi board.

```
isModelRunning(agentData.RPi,mdl)
```

```
ans = logical  
     1
```

Run Training Loop

The example executes the training loop, as a part of the learning process, on the desktop computer.

Create variables to monitor training.

```
trainingResultData.CumulativeRewardTemp = 0;  
trainingResultData.CumulativeReward = [];  
trainingResultData.AverageCumulativeReward = [];  
trainingResultData.AveragingWindowLength = 15;  
trainingResultData.NumOfEpisodes = 0;
```

Stop training when the agent receives an average cumulative reward greater than 800 or a maximum of 5000 episodes.

```
rewardStopTrainingValue = 800;  
maxEpisodes = 5000;
```

To train the agent, set `doTraining` to `true`.

```
doTraining =  ;
```

Create a figure for training visualization using the `buildRemoteTrainingMonitor` on page 6-68 local function. This function is defined at the end of this example.

```
if doTraining  
    [trainingPlot,...  
     trainingResultData.LineReward,...  
     trainingResultData.LineAverageReward] = ...  
        buildRemoteTrainingMonitor();  
    % Enable the training visualization plot.  
    set(trainingPlot,Visible="on");  
end
```

The training repeats the following steps:

- 1 Check if the deployed control process Simulink model has generated new experiences. If it did, get the experiences from the Raspberry Pi board to the desktop computer.
- 2 Read the experiences and append them to the replay buffer using `readNewExperiences`. The `readNewExperiences` on page 6-66 function is defined at the end of this example. Once the function reads the experiences, they are deleted from the Raspberry Pi board.
- 3 Train the agent from the collected data using `trainFromData`.
- 4 Save the updated policy parameters to the Raspberry Pi board using `writePolicyParameters` on page 6-65. This function is defined at the end of this example.

```
if doTraining  
    while true
```

```
% Look for any experience files.
dirContent = listFolderContentsOnRPi( ...
    agentData.RPi,agentData.ExperiencesFolderRPi);

if ~isempty(dirContent)

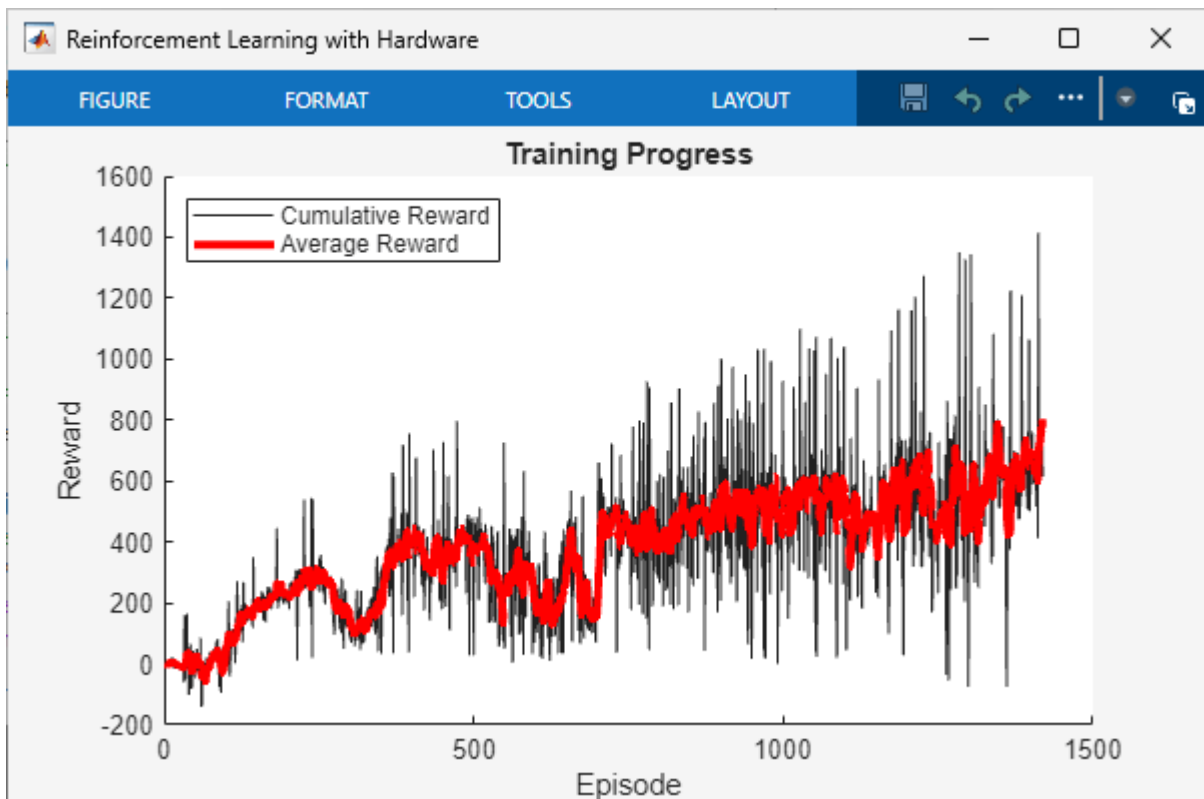
    % Display the new found experience files.
    fprintf('%s %4d new experience file found\n', ...
        datetime('now'), numel(dirContent));

    % Use the readNewExperiences function to read
    % new experience files and append experiences
    % to the replay buffer.
    [agentData, trainingResultData] = readNewExperiences( ...
        dirContent,agentData,trainingResultData);

    if agentData.Agent.ExperienceBuffer.Length>=...
        agent.AgentOptions.MinibatchSize
        % Perform a learning step for the agent
        % using data in the experience buffer.
        offlineTrainStats = trainFromData( ...
            agentData.Agent,agentData.TrainFromDataOptions);
    end

    % Save the updated actor parameters to a file on the
    % desktop computer and send the file to the Raspberry Pi.
    writePolicyParameters(agentData);
end

if ~isempty(trainingResultData.AverageCumulativeReward)
    % Check if at least one experience was read to prevent
    % AverageCumulativeReward reward from being empty.
    if trainingResultData.AverageCumulativeReward(end) > ...
        rewardStopTrainingValue ...
        || trainingResultData.NumOfEpisodes>=maxEpisodes
        % Use break to exit the training loop when
        % the average cumulative reward is greater than
        % rewardStopTrainingValue or the number of
        % episodes is more than maxEpisodes.
        break
    end
end
end
else
    % If you did not train the agent, load the agent from a file.
    load("trainedAgentQuanserQube.mat");
    agent.UseExplorationPolicy = false;
    agentData.Agent = agent;
end
```



Stop the model deployed to the Raspberry Pi Board.

```
if isModelRunning(agentData.RPi,mdl)
    stopModel(agentData.RPi,mdl);
end
```

Evaluate Trained Policy

Evaluate the trained policy on the Raspberry Pi board, using greedy actions. Change the policy to act greedily by setting the policy parameter `Policy_UseMaxLikelihoodAction` to true.

```
agentData.UseExplorationPolicy = false;
```

Send the policy parameters to the Raspberry Pi board.

```
writePolicyParameters(agentData);
```

Remove all the old experience files on the board.

```
system(agentData.RPi,convertStringsToChars("rm -f " + ...
    agentData.ExperiencesFolderRPi + ".*.*"));
```

Create variables to monitor evaluation.

```
evaluationResultData.CumulativeRewardTemp = 0;
evaluationResultData.CumulativeReward = [];
evaluationResultData.AverageCumulativeReward = [];
evaluationResultData.AveragingWindowLength = 10;
evaluationResultData.NumOfEpisodes = 0;
evaluationMaxEpisodes = 5;
```

Start the model deployed on the Raspberry Pi Board.

```
if ~isModelRunning(agentData.RPi,mdl)
    runModel(agentData.RPi,mdl);
end
```

The evaluation repeats the following steps:

- 1 Check if the deployed Control Process model has generated new experiences. If it did, send the experiences from the Raspberry Pi board to the desktop computer.
- 2 Read the experiences and append them to the replay buffer using `readNewExperiences`.

```
while true
    % See if there are any new experiences.
    dirContent = listFolderContentsOnRPi( ...
        agentData.RPi, agentData.ExperiencesFolderRPi);

    if ~isempty(dirContent)

        fprintf('%s %4d new experience file found\n', ...
            datetime('now'), numel(dirContent));

        % Read new experience files,
        % and append experiences to the replay buffer.
        [agentData, evaluationResultData] = readNewExperiences( ...
            dirContent, agentData, evaluationResultData);
    end

    if evaluationResultData.NumOfEpisodes >= evaluationMaxEpisodes
        % Use break to exit the evaluation loop when the number of
        % episodes is more than evaluationMaxEpisodes.
        break
    end
end
```

```
15-Dec-2025 08:30:59      1 new experience file found
15-Dec-2025 08:31:00      1 new experience file found
15-Dec-2025 08:31:01      1 new experience file found
15-Dec-2025 08:31:02      1 new experience file found
15-Dec-2025 08:31:03      1 new experience file found
15-Dec-2025 08:31:10      1 new experience file found
15-Dec-2025 08:31:11      1 new experience file found
15-Dec-2025 08:31:12      1 new experience file found
15-Dec-2025 08:31:13      1 new experience file found
15-Dec-2025 08:31:14      1 new experience file found
15-Dec-2025 08:31:22      1 new experience file found
15-Dec-2025 08:31:23      1 new experience file found
15-Dec-2025 08:31:24      1 new experience file found
15-Dec-2025 08:31:25      1 new experience file found
15-Dec-2025 08:31:26      1 new experience file found
15-Dec-2025 08:31:34      1 new experience file found
15-Dec-2025 08:31:35      1 new experience file found
15-Dec-2025 08:31:36      1 new experience file found
15-Dec-2025 08:31:37      1 new experience file found
15-Dec-2025 08:31:38      1 new experience file found
15-Dec-2025 08:31:46      1 new experience file found
15-Dec-2025 08:31:47      1 new experience file found
15-Dec-2025 08:31:48      1 new experience file found
```

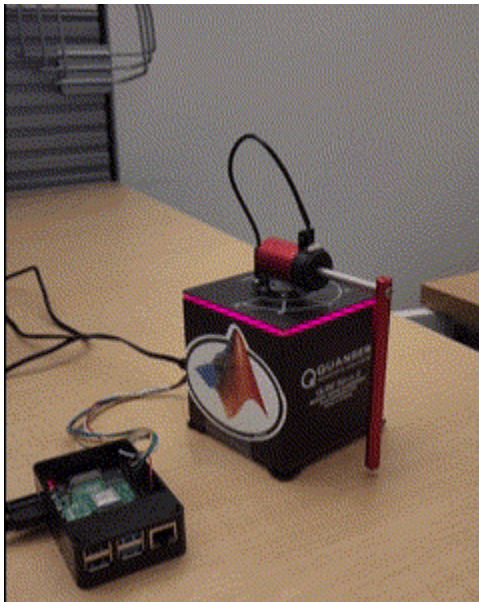
```
15-Dec-2025 08:31:49      1 new experience file found
15-Dec-2025 08:31:50      1 new experience file found
```

Compute the average of episode rewards.

```
mean(evaluationResultData.CumulativeReward)
```

```
ans =
664.4980
```

This value shows that that policy is able to swing up and balance the pendulum in the upright position as shown in the video below.



Stop the model deployed on the Raspberry Pi Board.

```
if isModelRunning(agentData.RPi,mdl)
    stopModel(agentData.RPi,mdl);
end
```

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

Local Functions

Write Policy Parameters

The writePolicyParameters function extracts the policy parameters from the agent, writes them to a file, and sends the file to the Raspberry Pi board.

```
function writePolicyParameters(agentData)

    % Open a file for saving the policy parameters on the desktop computer.
    filepath = fullfile( ...
        agentData.ParametersFolderDesktop, agentData.ParametersFile);
```

```

fid = fopen(filepath, 'w+');
cln = onCleanup(@()fclose(fid));

if agentData.UseExplorationPolicy
    % Get exploration policy parameters.
    parameters = policyParameters(getExplorationPolicy(agentData.Agent));
else
    % Get greedy policy parameters.
    parameters = policyParameters(getGreedyPolicy(agentData.Agent));
end

% Convert the policy parameters into a vector of uint8 values
% (serialization) and save it to a file.
fwrite(fid, structToBytes(parameters), 'uint8');

% Send the file from the desktop to the Raspberry Pi board.
putFile(agentData.RPi, ...
        convertStringsToChars(fullfile( ...
            agentData.ParametersFolderDesktop, ...
            agentData.ParametersFile)), ...
        convertStringsToChars( ...
            agentData.ParametersFolderRPi+agentData.ParametersFile));
end

```

Read New Experiences

The `readNewExperiences` function moves new experience files from the Raspberry Pi board to the desktop computer, reads the experiences, and appends them to the replay buffer.

```

function [agentData, trainingResultData] = ...
    readNewExperiences(dirContent, agentData, trainingResultData)

for ii=1:length(dirContent)

    % Move file from Raspberry Pi board to the desktop computer.
    getFile(agentData.RPi, ...
            convertStringsToChars( ...
                agentData.ExperiencesFolderRPi+dirContent(ii)), ...
            convertStringsToChars(agentData.ExperiencesFolderDesktop));

    % Remove the moved file from Raspberry Pi board.
    system(agentData.RPi, convertStringsToChars("rm -f " + ...
        agentData.ExperiencesFolderRPi+dirContent(ii)));

    % Open the moved file on desktop for reading the data.
    fid = fopen(fullfile( ...
        agentData.ExperiencesFolderDesktop, dirContent(ii)), 'r');
    cln = onCleanup(@()fclose(fid));

    % Read data from the file.
    experiencesBytes = fread(fid, '*uint8');

    % Convert the uint8s to experience structure.
    tempExperienceBuffer = bytesToStruct( ...
        experiencesBytes, agentData.TempExperienceBuffer);

    % Compute the cumulative reward and updating the plots.

```

```

trainingResultData = computeCumulativeReward( ...
    trainingResultData,tempExperienceBuffer);

% Get the experiences ready to append to the replay buffer.
for kk=size(tempExperienceBuffer.Observation,2):-1:1
    experiences(kk).NextObservation = ...
        {tempExperienceBuffer.NextObservation(:,kk)};
    experiences(kk).Observation = ...
        {tempExperienceBuffer.Observation(:,kk)};
    experiences(kk).Action = {tempExperienceBuffer.Action(:,kk)};
    experiences(kk).Reward = tempExperienceBuffer.Reward(kk);
    experiences(kk).IsDone = tempExperienceBuffer.IsDone(kk);
end

% Append the new experiences to the replay buffer.
append(agentData.Agent.ExperienceBuffer,experiences);
end
end

```

Compute Cumulative Reward from Experiences

The computeCumulativeReward function computes the cumulative reward and updates the learning plot.

```

function trainingResultData = computeCumulativeReward( ...
    trainingResultData,tempExperienceBuffer)

isDone = tempExperienceBuffer.IsDone;
reward = tempExperienceBuffer.Reward;
N = length(isDone);
while N > 0
    if any(isDone) % checking if there are any isDones
        endOfEpIdx = find(isDone,1);
        trainingResultData.CumulativeRewardTemp = ...
            trainingResultData.CumulativeRewardTemp ...
            + sum(reward(1:endOfEpIdx));

        % Append the new episode cumulative reward.
        trainingResultData.CumulativeReward = ...
            [trainingResultData.CumulativeReward; ...
            trainingResultData.CumulativeRewardTemp];

        % Temporary variable used to compute the cumulative reward
        trainingResultData.CumulativeRewardTemp = 0;

        % Update the plots.
        trainingResultData.NumOfEpisodes = ...
            trainingResultData.NumOfEpisodes+1;
        trainingResultData.AverageCumulativeReward = ...
            movmean(trainingResultData.CumulativeReward,...
            trainingResultData.AveragingWindowLength,1);

        % Update the monitor.
        if isfield(trainingResultData, "LineReward") && ...
            ~isempty(trainingResultData.LineReward)

            addpoints( ...
                trainingResultData.LineReward, ...

```

```

        trainingResultData.NumOfEpisodes,...
        trainingResultData.CumulativeReward(end));

        addpoints( ...
            trainingResultData.LineAverageReward, ...
            trainingResultData.NumOfEpisodes,...
            trainingResultData.AverageCumulativeReward(end));
    end
    drawnow;

    % Truncate the trajectory.
    isDone(1:endOfEpIdx) = [];
    reward(1:endOfEpIdx) = [];
    N = length(isDone);

else
    % If there are no more isDone (plural noun),
    % compute the cumulative reward and store it
    % in the temporary variable.
    trainingResultData.CumulativeRewardTemp = ...
        trainingResultData.CumulativeRewardTemp ...
        + sum(reward(1:N));
    N = 0;
end
end
end
end

```

Create Figure for Training Visualization

The buildRemoteTrainingMonitor function creates a figure for training visualization.

```

function [trainingPlot, lineReward, lineAverageReward] = ...
    buildRemoteTrainingMonitor()

    plotRatio = 16/9;
    trainingPlot = figure( ...
        Visible="off", ...
        HandleVisibility="off", ...
        NumberTitle="off", ...
        Name="Reinforcement Learning with Hardware");

    trainingPlot.Position(3) = ...
        plotRatio * trainingPlot.Position(4);

    ax = gca(trainingPlot);

    lineReward = animatedline(ax);
    lineAverageReward = animatedline(ax,Color="r",LineWidth=3);
    xlabel(ax,"Episode");
    ylabel(ax,"Reward");
    legend(ax,"Cumulative Reward","Average Reward", ...
        Location="northwest")
    title(ax,"Training Progress");
end

```

Assign Parameter Values Related to Environment

The `environmentParameters` function assigns various parameter values needed for the environment and the reset controller.

```
function envData = environmentParameters(sampleTime)

    envData.MaxStepsPerSim = uint32(1000);
    envData.Ts = sampleTime; % sec
    envData.ExperiencesWriteFrequency = 200;

    % For the reset controller: using LQR
    envData.LQRK = [1.3831 0.39192 -1.085 -0.082812];

end
```

Create Folder on Raspberry Pi Board

The `createFolderOnRPi` function creates a folder on the Raspberry Pi board if not exists.

```
function createFolderOnRPi(RPi, FolderRPi)

    system(RPi,convertStringsToChars("if [ ! -d " +...
        FolderRPi + " ]; then mkdir -p " +...
        FolderRPi + "; fi"));

end
```

List Contents of Folder on Raspberry Pi Board

The `listFolderContentsOnRPi` function lists contents of a folder on the Raspberry Pi board.

```
function dirContent = listFolderContentsOnRPi(RPi, experiencesFolderRPi)

    dirTemp = system(RPi,convertStringsToChars("ls " +...
        experiencesFolderRPi));
    dirContent = strsplit(dirTemp);
    dirContent(end) = [];

end
```

See Also

Functions

`policyParameters` | `updatePolicyParameters` | `generatePolicyBlock` | `trainFromData`

Blocks

Policy

Objects

`rlReplayMemory` | `rlStochasticActorPolicy` | `rlSACAgent` | `rlSACAgentOptions` | `rlTrainingFromDataOptions`

Related Examples

- “Generate Policy Block for Deployment” on page 6-7

- “Train Policy Deployed on Raspberry Pi” on page 6-31
- “Train Reinforcement Learning Policy Using Custom Training Loop” on page 5-273
- “Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum” on page 5-184

More About

- “Train Reinforcement Learning Agents” on page 5-3
- “Examine Approaches to Fine Tune a Deployed Policy” on page 6-28
- “Create Custom Reinforcement Learning Agents” on page 3-92
- “Use Predefined Control System Environments” on page 2-42

Benchmark Examples

- “Compare Agents on Basic Grid World” on page 7-2
- “Compare Agents on Deterministic Waterfall Grid World” on page 7-36
- “Compare Agents on Stochastic Waterfall Grid World” on page 7-70
- “Compare Agents on Discrete Double-Integrator” on page 7-104
- “Compare Agents on Discrete Cart-Pole” on page 7-121
- “Compare Agents on the Discrete Pendulum Swing-Up” on page 7-138
- “Compare Agents on Discrete Simscape Cart-Pole Swing-Up” on page 7-157
- “Compare Agents on Discrete Pendulum Swing-Up with Image” on page 7-174
- “Compare Agents on Continuous Double-Integrator” on page 7-189
- “Compare Agents on Continuous Cart Pole” on page 7-208
- “Compare Agents on Continuous Pendulum Swing-Up” on page 7-228
- “Compare Agents on Continuous Cart Pole Swing-Up” on page 7-251
- “Compare Agents on Continuous Pendulum Swing-Up with Image” on page 7-268
- “Compare DDPG Agent to LQR Controller” on page 7-285

Compare Agents on Basic Grid World

This example shows how to create and train frequently used discrete action space default agents on a basic 5-by-5 gridworld environment. The environment has a starting location, a terminal location, obstacles, and a special jump from state [2,4] to state [4,4]. The goal of the agent is to move from the starting location to the terminal location while avoiding obstacles and maximizing the total reward.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

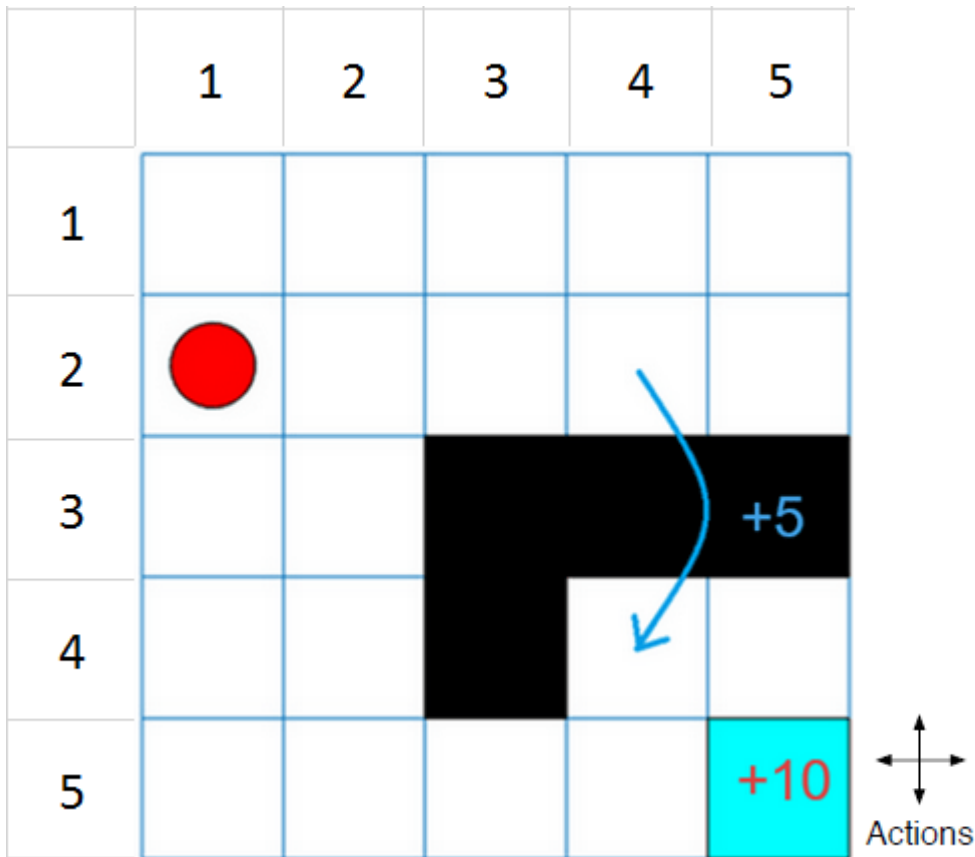
```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Basic Grid World Environment

This grid world environment has the following configuration and rules:

- 1 The grid world is 5-by-5 and bounded by borders, with four possible actions (North = 1, South = 2, East = 3, West = 4).
- 2 The agent begins from cell [2,1] (second row, first column).
- 3 The agent receives a reward of +10 if it reaches the terminal state at cell [5,5] (light blue).
- 4 The environment contains a special jump from cell [2,4] to cell [4,4] with a reward of +5.
- 5 The agent is blocked by obstacles (black cells).
- 6 All other actions result in -1 reward.



For more information on this environment, see “Use Predefined Grid World Environments” on page 2-9.

Create Environment Object

Create the basic grid world environment object.

```
env = rlPredefinedEnv("BasicGridWorld");
```

To specify that the initial state of the agent is always [2,1], create a reset function that returns the state number for the initial agent state. This function is called at the start of each training episode and simulation. States are numbered starting at position [1,1]. The state number increases as you move down the first column and then down each subsequent column.

Create an anonymous function handle that sets the initial state to 2.

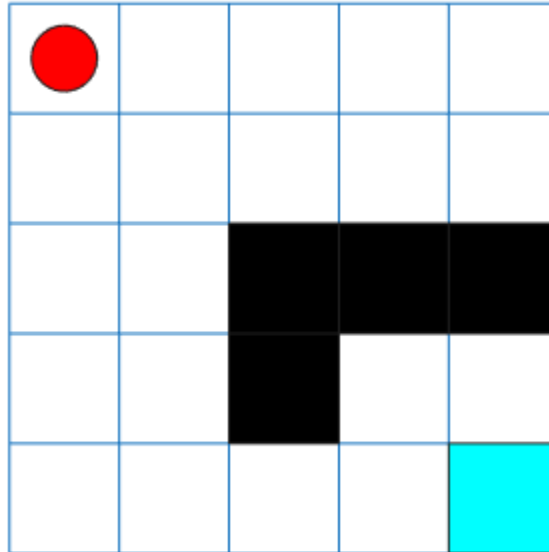
```
env.ResetFcn = @() 2;
```

Extract the environment observation and action specification objects for later use when creating agents.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

You can visualize the environment during training or simulation by using the plot function.

```
plot(env)
```



The plot shows the agent location with a red circle and the terminal location in light blue. Note that the initial agent location is [1,1] because the environment reset function has not yet been called.

Create a Table-Based Q-Value Function Critic

In this section, you create a table-based Q-value function critic for use within Q, SARSA, and DQN agents.

Create a table object.

```
table = rlTable(obsInfo,actInfo);
```

Create a critic approximator object using `rlQValueFunction`.

```
qcritic = rlQValueFunction(table,obsInfo,actInfo);
```

Check the critic with random observation and action inputs.

```
getValue(qcritic, ...  
    {randi(numel(obsInfo.Elements))}, ...  
    {randi(numel(actInfo.Elements))})
```

```
ans =  
0
```

Create a Table-Based Value Function Critic

In this section, you create a table-based value function critic for use within AC and PPO agents. You can also use this critic as a baseline for a PG agent.

Create a table object.

```
table = rlTable(obsInfo);
```

Create a critic approximator object using `rlValueFunction`.

```
vcritic = rlValueFunction(table,obsInfo);
```

Check the critic with random observation input.

```
getValue(vcritic,{randi(numel(obsInfo.Elements))})
```

```
ans =  
0
```

Create a Custom Basis Function-Based Vector Q-Value Function Critic

In this section, you create a vector value function critic for use within the SAC agent. The critic uses a custom basis function that implements a table.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Because `rlVectorQValueFunction` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```
type("cbftbl.m")
```

```
function out = cbftbl(obs,nrows)
```

```
% Table with nrows rows and one column implemented using a custom basis  
% function. The third dimension of obs is the batch dimension.
```

```
    % Get batch dimension and allocate output matrix.  
    nbtc = size(obs,3);  
    out = zeros(nrows,nbtc,like=obs);
```

```
    % Cycle through batch dimension and set to one the output  
    % corresponding to the index received as observation.
```

```
    for k = 1:nbtc  
        if obs(1,1,k)  
            out(obs(1,1,k),k) = 1;  
        end  
    end
```

```
end
```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbftbl` function (the value of the second argument is stored in the `basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $8 \times 7 = 56$ elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of `W0` is multiplied by the output of `basisFcn` the result is a vector in which each of the four elements is the value of the corresponding action given the observation.

```
W0 = rand(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlVectorQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```
vqcritic = rlVectorQValueFunction({basisFcn,W0},obsInfo,actInfo);
```

Check the critic with a batch of 10 random observation inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);  
v = getValue(vqcritic,{robs});
```

Display the seventh element of the batch.

```
v(:,7)  
  
ans = 4×1  
  
    0.4218  
    0.3816  
    0.5472  
    0.0540
```

Create a Custom Basis Function-Based Q-Value Function Critic

In this section, you create a value function critic for use within the LSPI agent. The critic uses a custom basis function that implements a table.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Because the LSPI agent does not support tables as approximator models, use the helper function defined in the supporting file `cbfqtbl.m`, which implements a q-value function table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```
type("cbfqtbl.m")  
  
function out = cbfqtbl(obs,act,nr,nc)  
  
% Table with nr rows and nc columns implemented using a custom basis
```



```

% function. The third dimension of obs and act is the batch dimension.

% Get batch dimension.
nbtc = size(obs,3);

% Allocate output matrix in columnwise format, that is the first two
% dimensions are vectorized as in out=table(:)
out = zeros(nr*nc,nbtc,like=obs);

% Cycle through batch dimension and set to one the output
% corresponding to the indexes of observation and action.
for k = 1:nbtc
    if obs(1,1,k)
        out(nr*(act(1,1,k)-1)+obs(1,1,k),k) = 1;
    end
end
end
end

```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbftbl` function (the value of the last two arguments is stored in the `basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $5*5*4=100$ elements, where the element equal to 1 indicates the columnwise location of the current observation and action.

```

basisFcnQ = @(obs,act) cbftbl(obs,act, ...
    numel(obsInfo.Elements), ...
    numel(actInfo.Elements));

```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result is a scalar indicating the value of the corresponding action given the observation.

```

W0 = rand(numel(obsInfo.Elements)*numel(actInfo.Elements),1);

```

For example, the weight corresponding to observation number 13 and action number 3 is:

```

W0(numel(obsInfo.Elements)*(3-1)+13)

```

```

ans =
0.5060

```

Use `rlQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```

qcbfcritic = rlQValueFunction({basisFcnQ,W0},obsInfo,actInfo);

```

Ignoring the batch dimension, check the critic by returning the value corresponding to the observation number 13 and the action number 3.

```

getValue(qcbfcritic,{13},{3})

```

```

ans =
0.5060

```

Check the critic with a batch of 10 random observation and action inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);
ract = randi(numel(actInfo.Elements),[1 1 10]);
v = getValue(qcbfcritic,{robs},{ract});
```

Display the seventh element of the batch.

```
v(:,7)
```

```
ans =
0.9157
```

Create a Custom Basis Function-Based Actor

In this section, you create a discrete categorical actor for use within PG, AC, and PPO agents. The actor uses a custom basis function that implements a table.

Because `rlDiscreteCategoricalActor` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Define the custom basis function. Here, the output of the basis function is a one-hot vector of 25 elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result, after passing through a `softmax` function, is a vector in which each of the four elements is the probability of executing the corresponding action.

```
W0 = zeros(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlDiscreteCategoricalActor` to create the actor object, passing the basis function handle and the initial condition as a cell.

```
actor = rlDiscreteCategoricalActor({basisFcn,W0},obsInfo,actInfo);
```

Check the actor with a batch of 10 random observation inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);
v = getAction(actor,{robs});
```

Display the seventh element of the batch.

```
v{1}(:, :, 7)
```

```
ans =
4
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 50 training episodes. For more information, see `rlEvaluator`.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=50);
```

Create a training options object.

```
trainOpts = rlTrainingOptions;
```

For this example, use the following options:

- Train for a maximum of 200 episodes. Specify that each episode lasts for most 50 time steps.
- Stop the training when the average reward in the evaluation episodes is greater than 10.

```
trainOpts.MaxEpisodes= 200;
trainOpts.MaxStepsPerEpisode = 50;
trainOpts.StopTrainingCriteria = "EvaluationStatistic";
trainOpts.StopTrainingValue = 10;
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 50 steps.

```
simOpts = rlSimulationOptions(MaxSteps=50);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a Q-Agent

In this section, you create a Q-learning agent and then configure its parameters.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rlQAgent` object using the critic you have created before.

```
qAgent = rlQAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
qAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
qAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

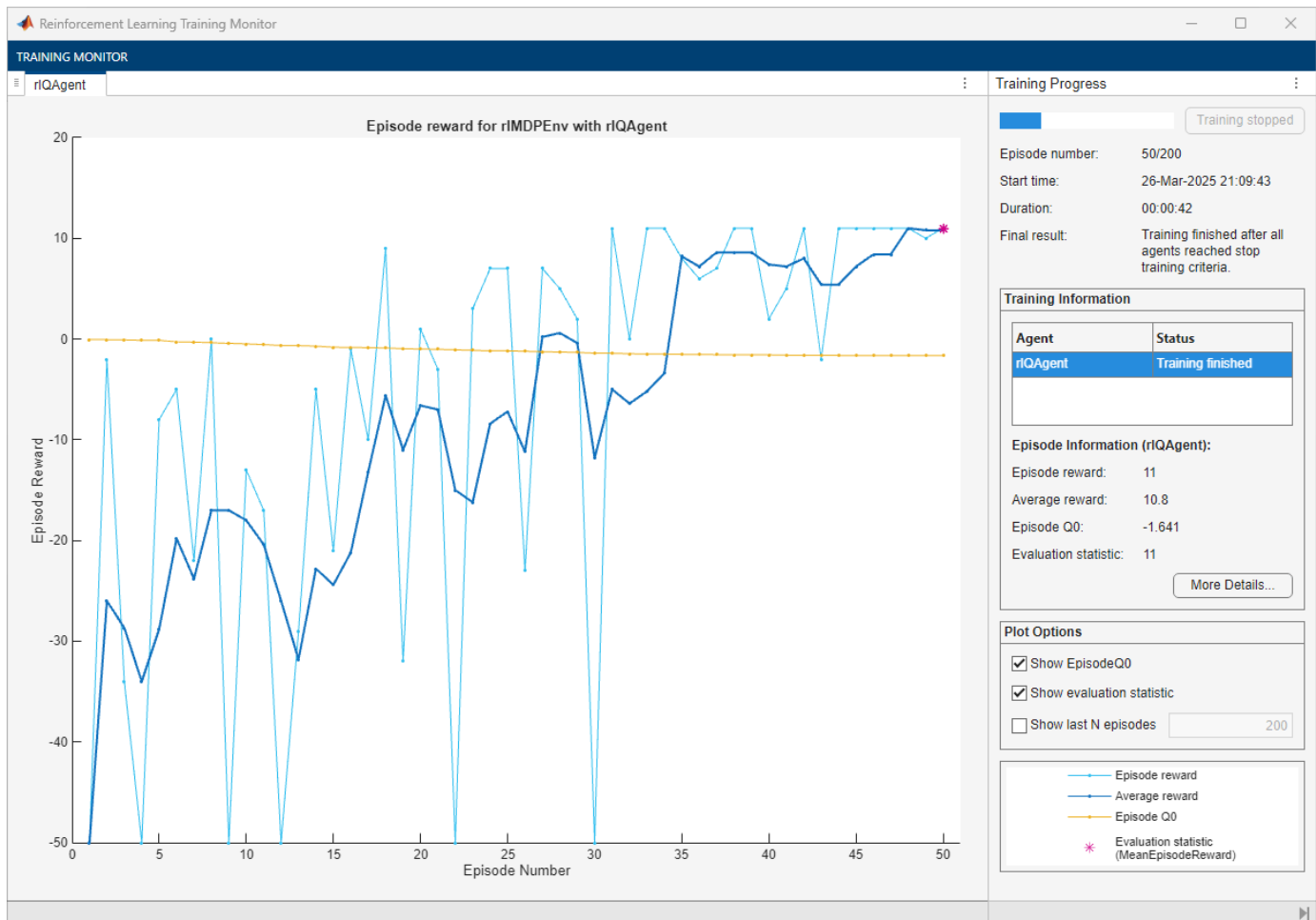
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
```

```

env.ResetFcn = @() 2;
% Train the agent. Record the training time.
tic
qTngRes = train(qAgent,env,trainOpts,Evaluator=evl);
qTngTime = toc;
% Extract the number of training episodes and total steps.
qTngEps = qTngRes.EpisodeIndex(end);
qTngSteps = sum(qTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("bgwBchQAgent.mat", ...
%      "qAgent","qTngEps","qTngSteps","qTngTime")
else
% Load the pretrained agent and results for the example.
load("bgwBchQAgent.mat", ...
     "qAgent","qTngEps","qTngSteps","qTngTime")
end

```



The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

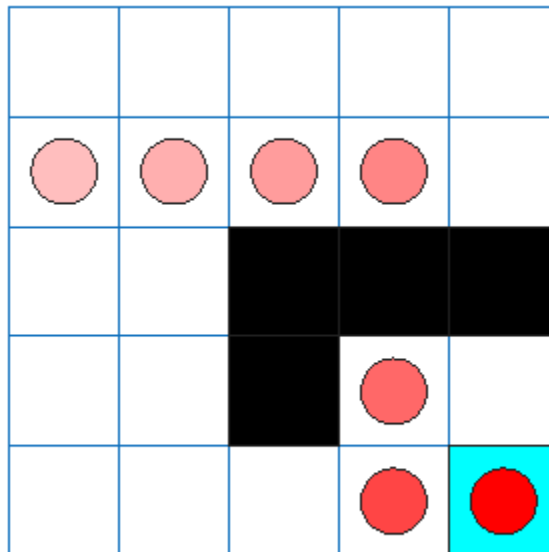
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(qAgent, env, simOpts);
```



```
qTotalRwd = sum(experience.Reward)
```

```
qTotalRwd =
11
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Extract the table with the final Q-value function.

```
qAgtFinalQ = getLearnableParameters(getCritic(qAgent));
```

Display the maximum values for each state (that is, the approximate value function) in a 5-by-5 format.

```
reshape(max(qAgtFinalQ{1}'), 5, 5)
```

```
ans = 5×5 single matrix
```

-1.3469	-1.0099	-0.6227	0.0046	-0.1782
-1.6410	-0.5100	0.9153	2.2912	-0.1781
-1.3224	-1.1052	0	0	0
-1.0821	-1.0359	0	1.0260	0.4685
-0.9660	-0.8167	0.1255	2.4065	0

Create, Train, and Simulate a SARSA Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rlSARSAAgent` object using the critic you have created before.

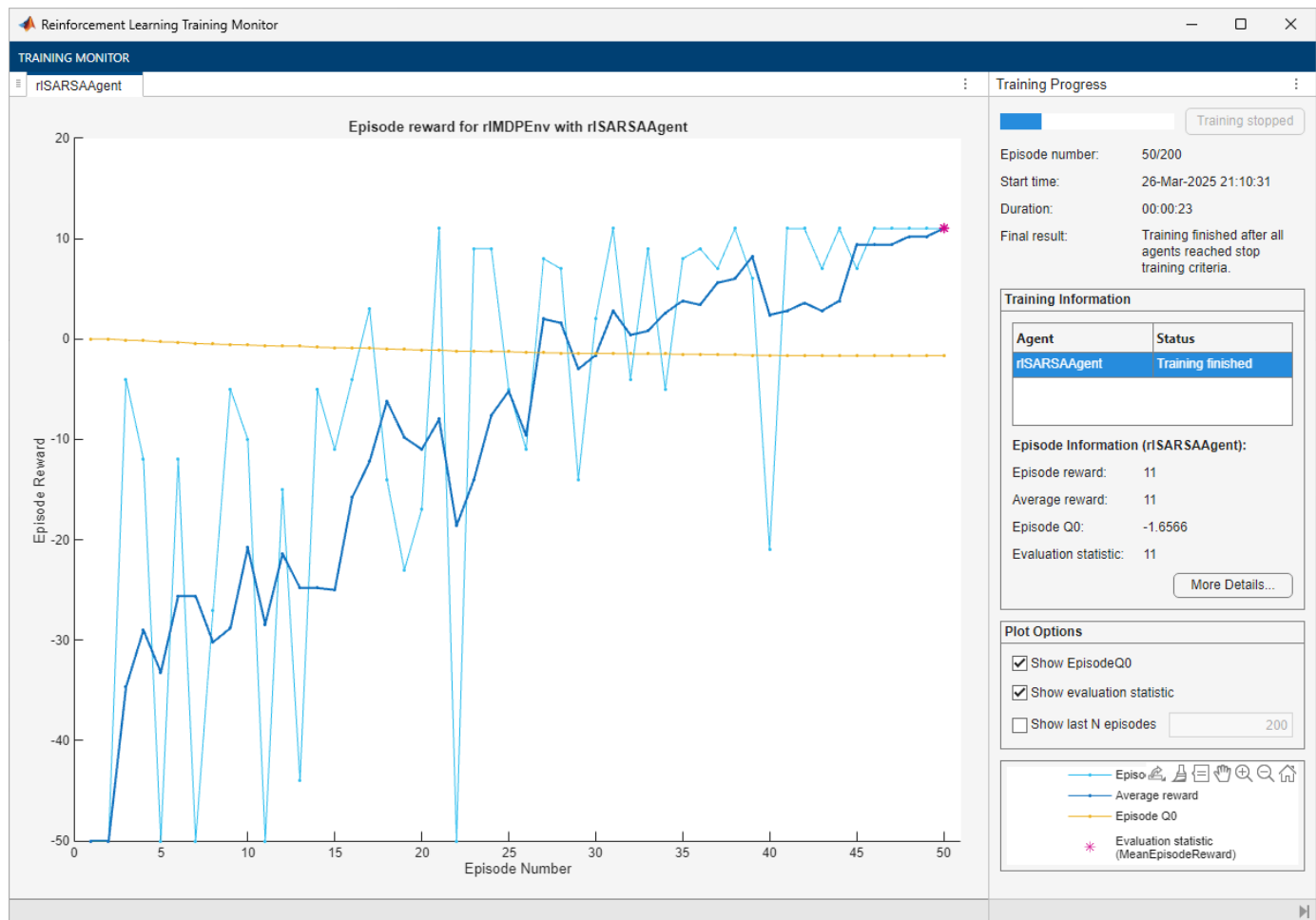
```
sarsaAgent = rlSARSAAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
sarsaAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;  
sarsaAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("BasicGridWorld");  
    env.ResetFcn = @() 2;  
    % Train the agent. Record the training time.  
    tic  
    sarsaTngRes = train(sarsaAgent,env,trainOpts,Evaluator=evl);  
    sarsaTngTime = toc;  
    % Extract the number of training episodes and total steps.  
    sarsaTngEps = sarsaTngRes.EpisodeIndex(end);  
    sarsaTngSteps = sum(sarsaTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("bgwBchSARSAAgent.mat", ...  
    %     "sarsaAgent","sarsaTngEps","sarsaTngSteps","sarsaTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("bgwBchSARSAAgent.mat", ...  
        "sarsaAgent","sarsaTngEps","sarsaTngSteps","sarsaTngTime")  
end
```



The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

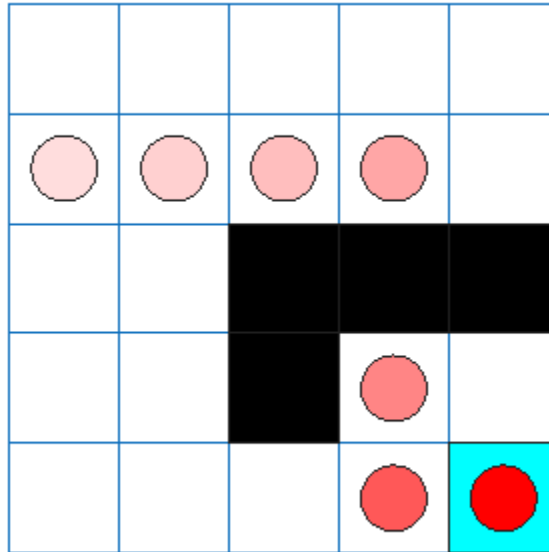
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sarsaAgent, env, simOpts);
```



```
sarsaTotalRwd = sum(experience.Reward)
```

```
sarsaTotalRwd =  
11
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Extract the table with the final Q-value function.

```
sarsaAgtFinalQ = getLearnableParameters(getCritic(sarsaAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(sarsaAgtFinalQ{1}'),5,5)
```

```
ans = 5x5 single matrix
```

```
-1.4562   -1.0796   -0.5463   -0.0675   -0.1163  
-1.6566   -0.4843    0.9977    2.3779   -0.1159  
-1.4030   -1.1944         0         0         0  
-1.1575   -1.0314         0    1.0373    0.5081  
-1.0411   -0.8055    0.0217    2.4975         0
```

Create, Train, and Simulate an LSPI Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

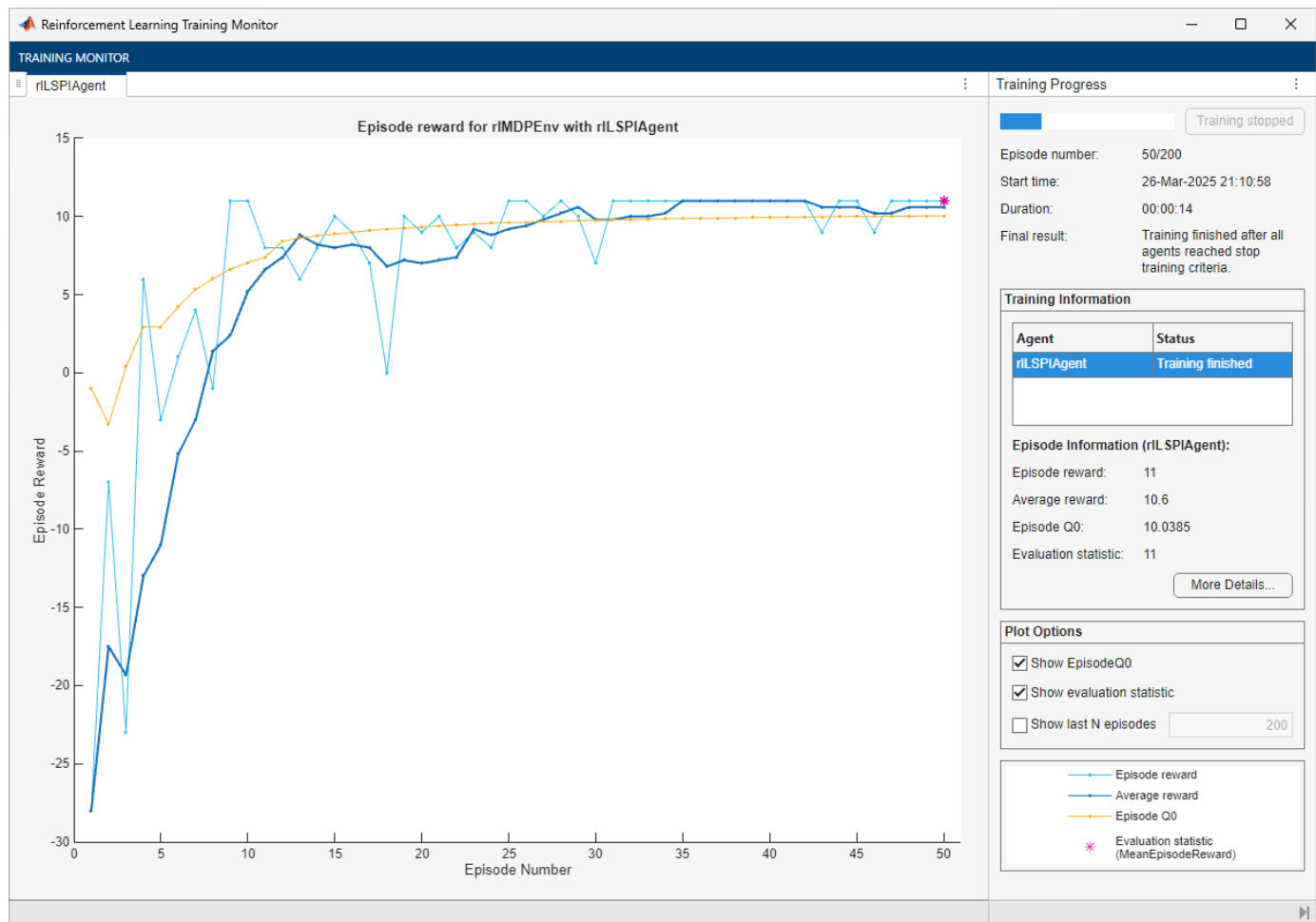
```
rng(0,"twister")
```

Create an `rLLSPIAgent` object using the critic you have created before.

```
lspiAgent = rLLSPIAgent(qcbfcritic);
```


Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
    env.ResetFcn = @() 2;
    % Train the agent. Record the training time.
    tic
    lspiTngRes = train(lspiAgent,env,trainOpts,Evaluator=evl);
    lspiTngTime = toc;
    % Extract the number of training episodes and total steps.
    lspiTngEps = lspiTngRes.EpisodeIndex(end);
    lspiTngSteps = sum(lspiTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("bgwBchLSPIAgent.mat", ...
    %     "lspiAgent","lspiTngEps","lspiTngSteps","lspiTngTime")
else
    % Load the pretrained agent and results for the example.
    load("bgwBchLSPIAgent.mat", ...
        "lspiAgent","lspiTngEps","lspiTngSteps","lspiTngTime")
end
```



The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

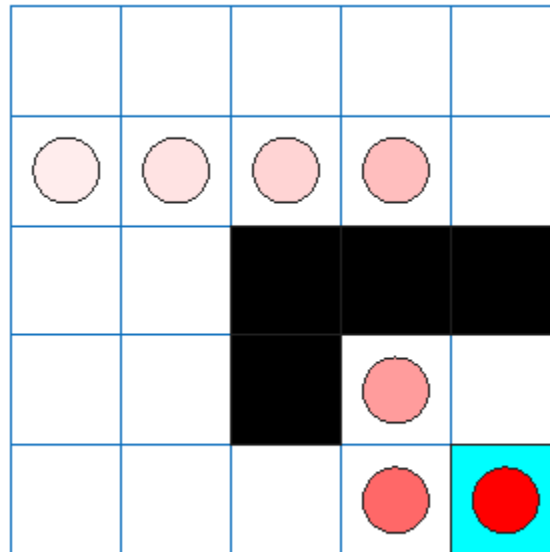
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(lspiAgent, env, simOpts);
```



```
lspiTotalRwd = sum(experience.Reward)
```

```
lspiTotalRwd =  
11
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Extract the table with the final Q-value function.

```
lspiAgtFinalQ = getLearnableParameters(getCritic(lspiAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(reshape(lspiAgtFinalQ{1},25,4)'),5,5)
```

```
ans = 5x5 single matrix
```

```
    8.7220    10.2598    11.3744    12.6264         0  
   10.0385    11.4187    12.6174    13.7655         0  
    8.6381     7.3589         0         0         0  
    6.8769   -42.5747         0    8.8544    9.9950  
   -1.9889   -0.9995    8.8980    9.9998         0
```

Create, Train, and Simulate a DQN Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rLDQNAgent` object using the critic you have created before.

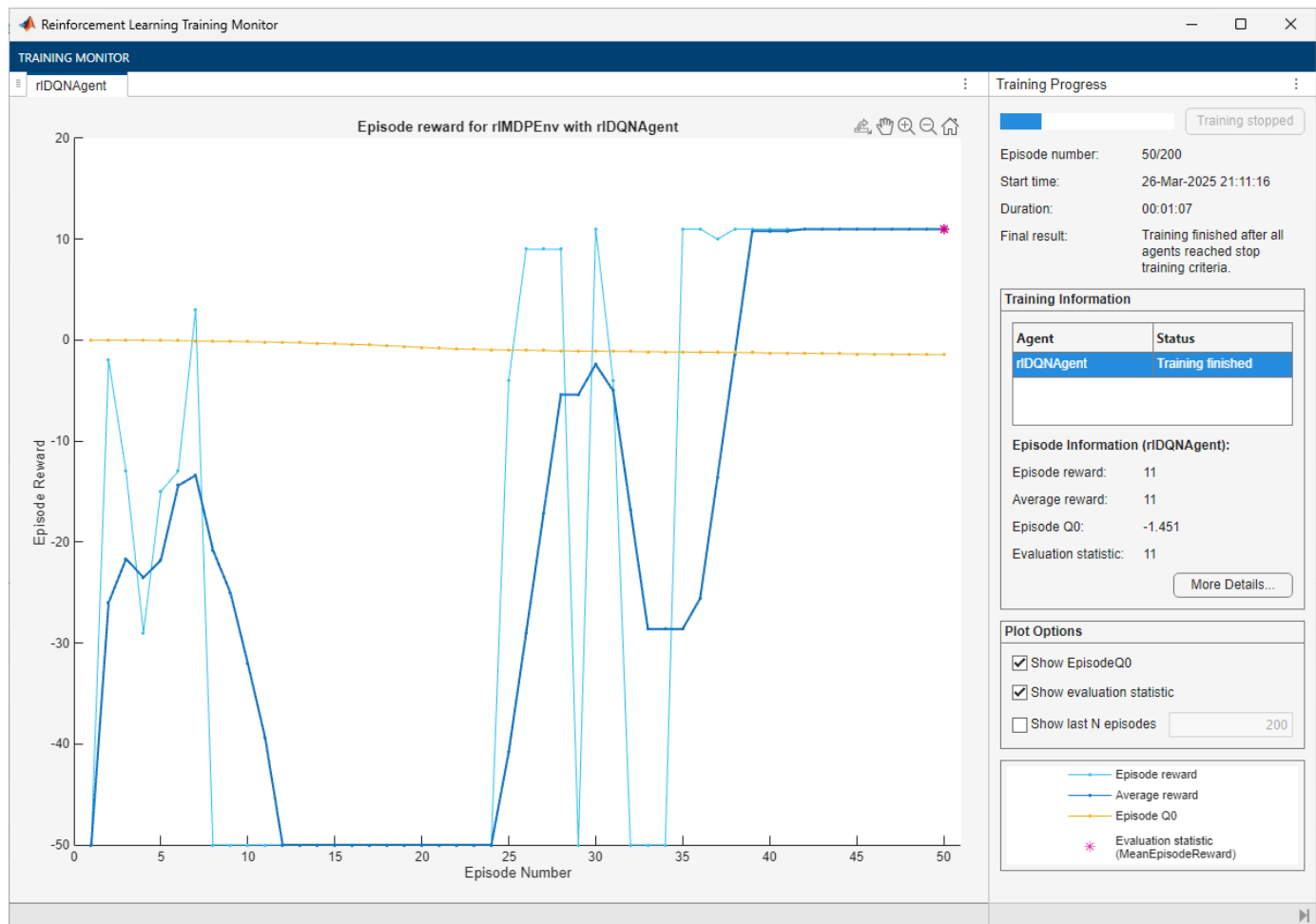
```
dqnAgent = rLDQNAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;  
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("BasicGridWorld");  
    env.ResetFcn = @() 2;  
    % Train the agent. Record the training time.  
    tic  
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);  
    dqnTngTime = toc;  
    % Extract the number of training episodes and total steps.  
    dqnTngEps = dqnTngRes.EpisodeIndex(end);  
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("bgwBchDQNAgent.mat", ...  
    %     "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("bgwBchDQNAgent.mat", ...  
        "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")  
end
```



The training converges after 40 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

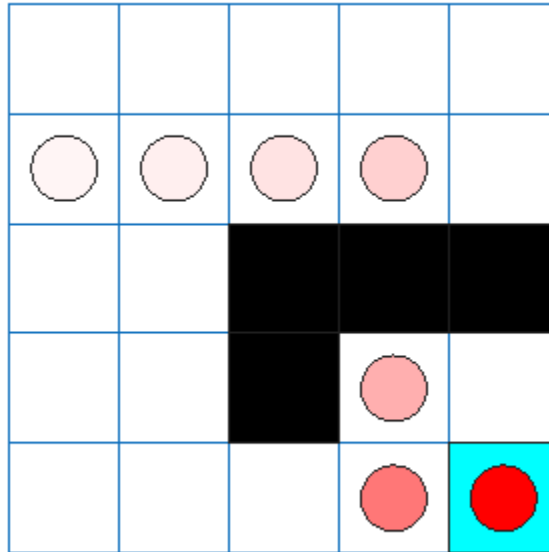
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(dqnAgent, env, simOpts);
```



```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
11
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Extract the table with the final Q-value function.

```
dqnAgtFinalQ = getLearnableParameters(getCritic(dqnAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(dqnAgtFinalQ{1}'),5,5)
```

```
ans = 5x5 single matrix
```

```
-1.4349  -1.3778  -1.1255  -0.4629  -0.9042  
-1.4510  -1.2435  -0.2133   3.4257   0  
-1.3386  -1.2477   0         0         0  
-1.0974  -1.2294   0        -0.1766   2.7995  
-1.2124  -1.1393  -0.3778   3.7252   0
```

The value function is still converging toward its optimal value.

Create, Train, and Simulate a PG Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPGAgent` object using the actor you have created before. Alternatively, to use a baseline, supply the critic you have created before `vcritic` as a second argument. However, for this example, do not use any baseline.

```
pgAgent = rlPGAgent(actor);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

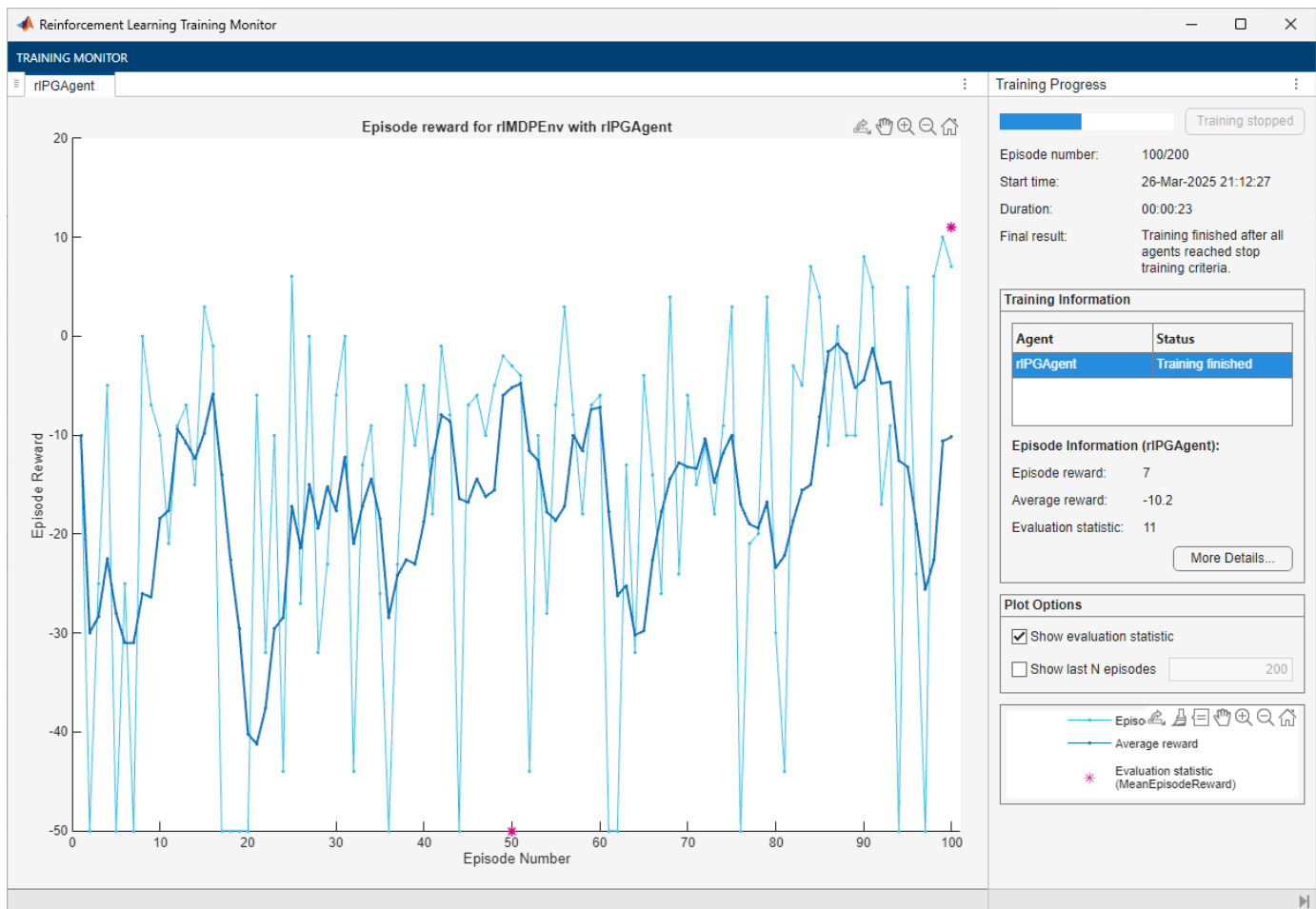
```
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
    env.ResetFcn = @() 2;
    % Train the agent. Record the training time.
    tic
    pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);
    pgTngTime = toc;
    % Extract the number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("bgwBchPGAgent.mat", ...
    %     "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
else
    % Load the pretrained agent and results for the example.
    load("bgwBchPGAgent.mat", ...
        "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
end
```



The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

Note that the average reward over the episode, in which the agent still explores, is still much lower than the reward obtained during evaluation, in which the agent uses a greedy policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

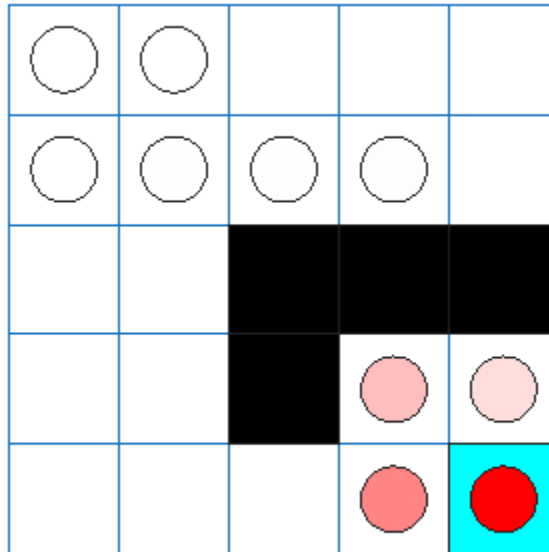
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.


```
experience = sim(pgAgent,env,simOpts);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-5
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Create, Train, and Simulate an AC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rlACAgent` object using the actor and critic you have created before.

```
acAgent = rlACAgent(actor,vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

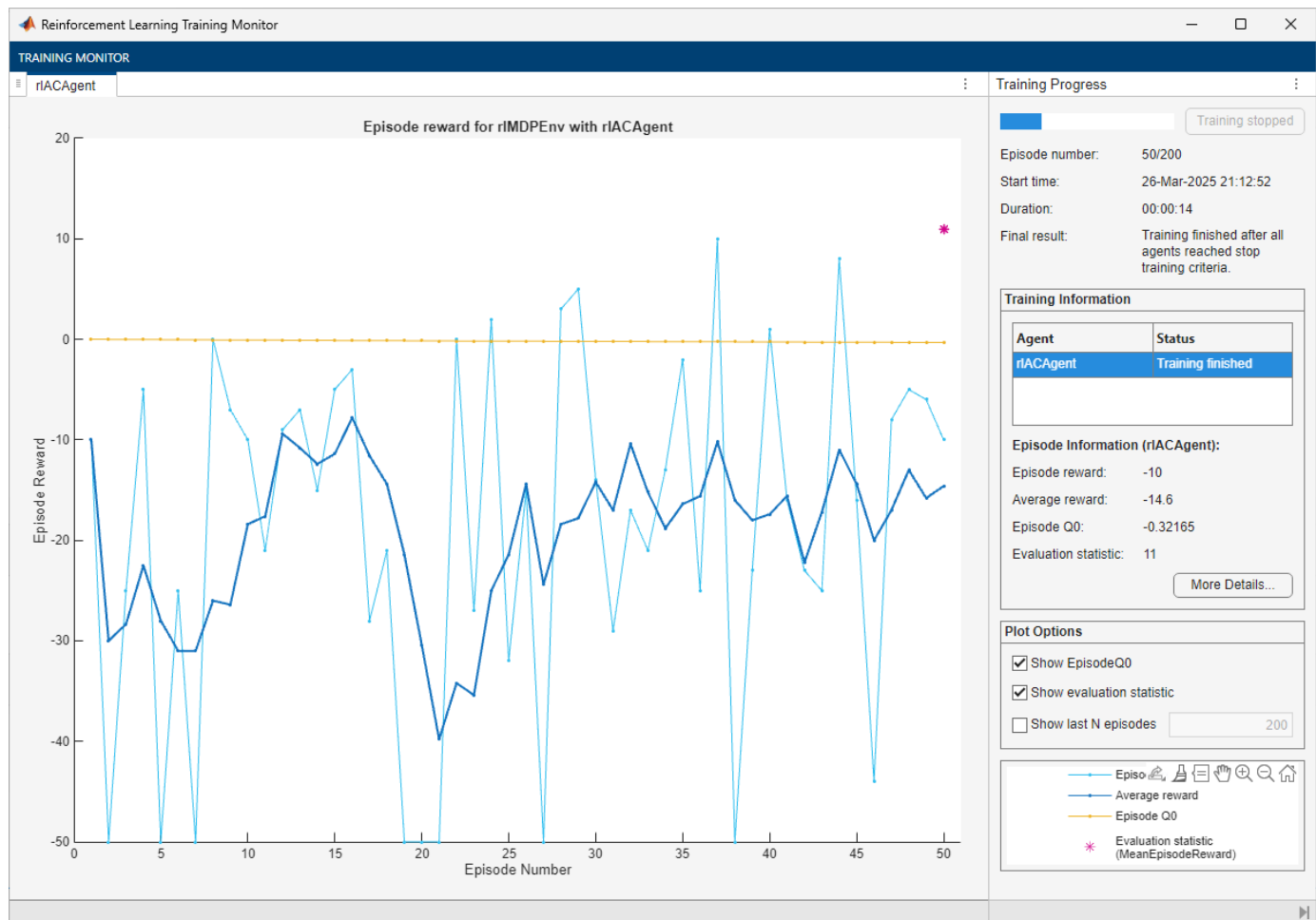
Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several

minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
    env.ResetFcn = @() 2;
    % Train the agent. Record the training time.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract the number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("bgwBchACAgent.mat", ...
    %     "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("bgwBchACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end
```



The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

Note that the average reward over the episode, in which the agent still explores, is still much lower than the reward obtained during evaluation, in which the agent uses a greedy policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

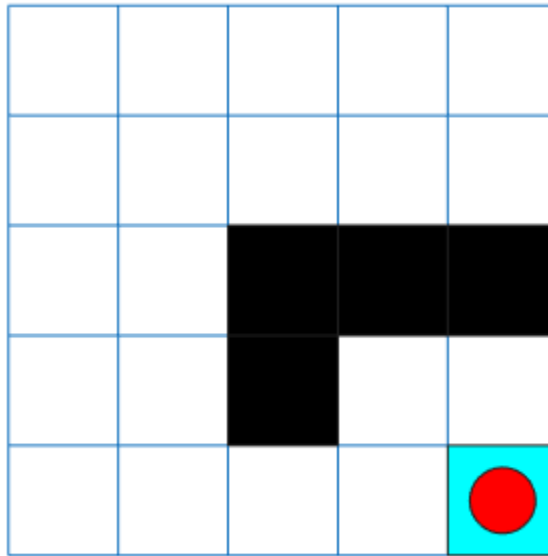
Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(acAgent,env,simOpts);
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =
-5
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
```



```
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Because the AC agent has a critic, you can display the final value function represented by the critic.

```
acAgtFinalV = getLearnableParameters(getCritic(acAgent));
reshape(acAgtFinalV{1},5,5)
```

```
ans = 5×5 single matrix
```

```
-0.1886    -0.2014    0.0052    0.0869   -0.0988
-0.3278    -0.2416    0.0477    0.2657    0.0360
-0.2853    -0.2150         0         0         0
-0.2533    -0.1802         0    0.1362    0.2486
-0.1948    -0.0836    0.0548    0.2329         0
```

The value function generally converges very slowly toward its optimal value.

Create, Train, and Simulate a PPO Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPPOAgent` object using the actor and critic you have created before.

```
ppoAgent = rlPPOAgent(actor, vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
    env.ResetFcn = @() 2;
    % Train the agent. Record the training time.
    tic
    ppoTngRes = train(ppoAgent, env, trainOpts, Evaluator=evl);
    ppoTngTime = toc;
    % Extract the number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("bgwBchPPOAgent.mat", ...
    %     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
else
    % Load the pretrained agent and results for the example.
    load("bgwBchPPOAgent.mat", ...
        "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
end
```



The training converges after 150 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

Note that the average reward over the episode, in which the agent still explores, is still much lower than the reward obtained during evaluation, in which the agent uses a greedy policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

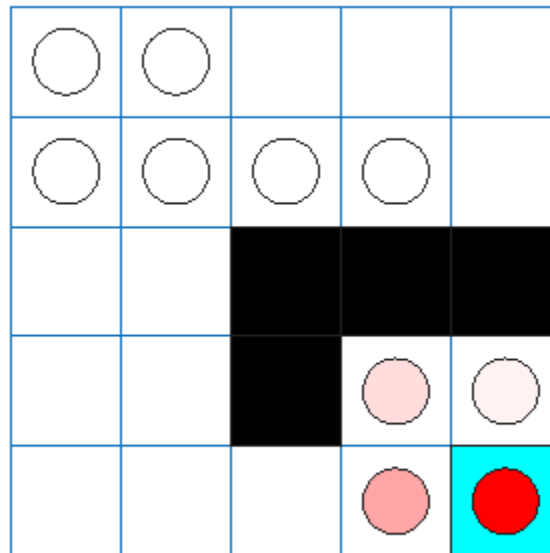
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(ppoAgent,env,simOpts);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-5
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Because the PPO agent has a critic, you can display the final value function represented by the critic.

```
ppoAgtFinalV = getLearnableParameters(getCritic(ppoAgent));  
reshape(ppoAgtFinalV{1},5,5)
```

```
ans = 5x5 single matrix
```

```
-0.7963    -0.6897    -0.4810    -0.0518     0.4197  
-0.8164    -0.7468    -0.2576     0.6862     0.4948  
-0.7987    -0.8040         0         0         0  
-0.7405    -0.6856         0     0.3711     0.6496  
-0.6497    -0.6676    -0.5004     0.3954         0
```

Similarly to the other agents with actors, the value function converges slowly toward its optimal value.

Create, Train, and Simulate a SAC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rlSACAgent` object using the actor you have created before and critic.

```
sacAgent = rlSACAgent(actor,vqcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

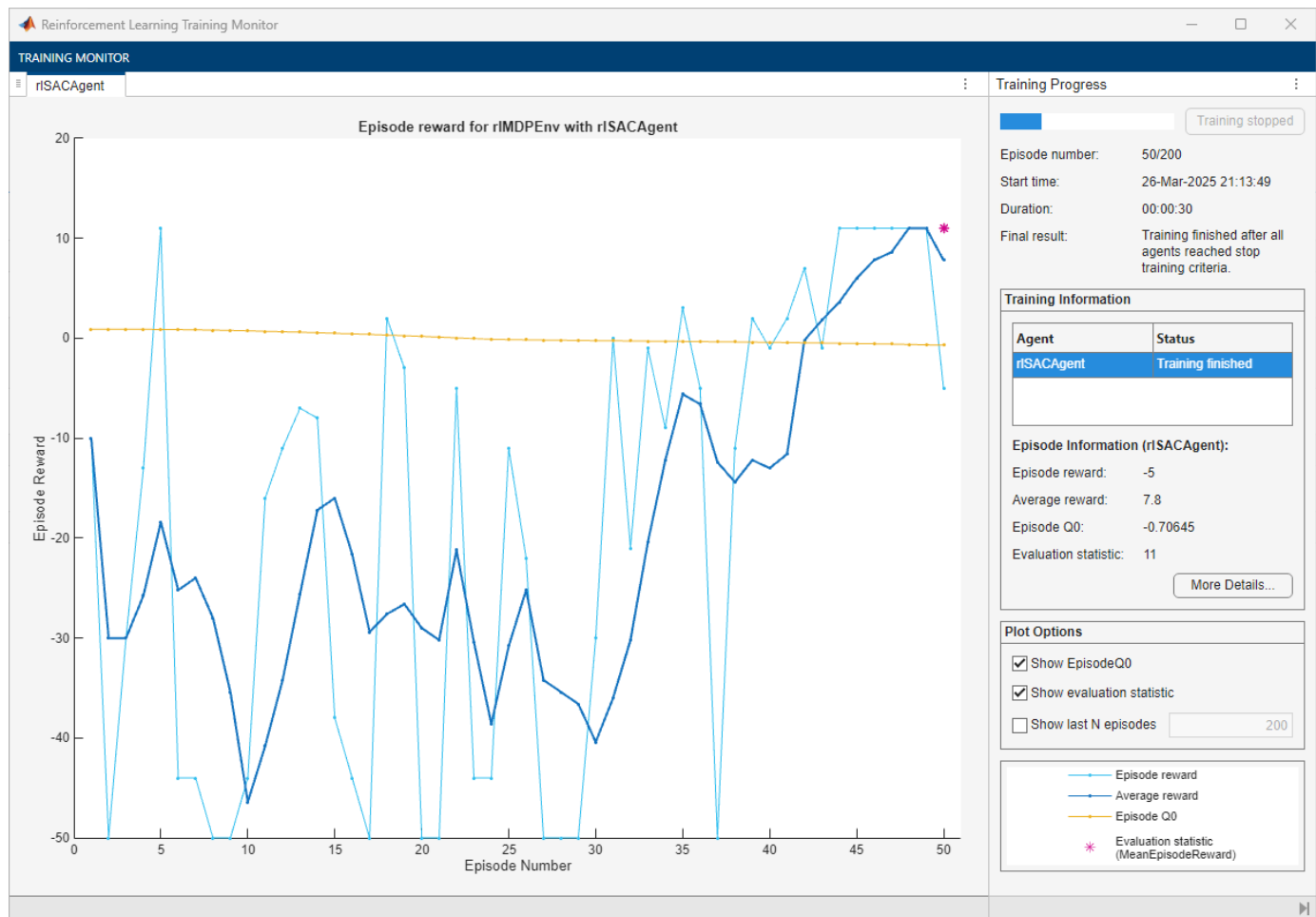
```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Train the agent, passing the previously created evaluator object to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("BasicGridWorld");
    env.ResetFcn = @() 2;
    % Train the agent. Record the training time.
    tic
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);
    sacTngTime = toc;
    % Extract the number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("bgwBchSACAgent.mat", ...
    %     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("bgwBchSACAgent.mat", ...
        "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
end
```

The training converges after 50 episodes. The evaluation statistic indicates a reward of 11, suggesting that the agent implements the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

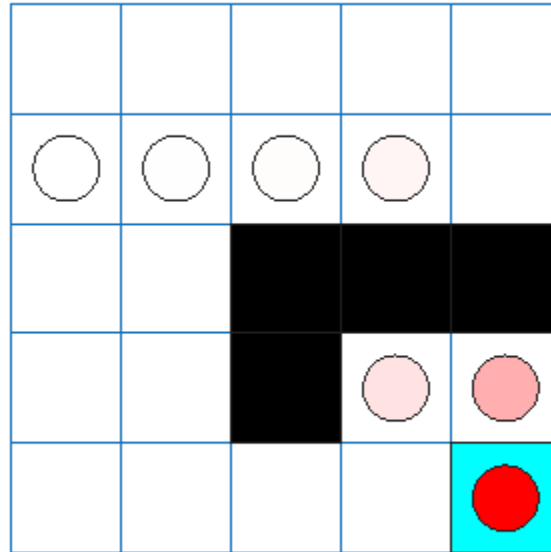
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sacAgent, env, simOpts);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
11
```

The agent trace shows that the trained agent successfully finds the jump from cell [2,4] to cell [4,4].

Because the SAC agent has a critic, you can display the final value function represented by critic.

```
sacAgtFinalV = getLearnableParameters(getCritic(sacAgent));  
reshape(max(sacAgtFinalV{1}),5,5)
```

```
ans = 5×5 single matrix
```

```
-0.7541    -0.6509    -0.7527     0.4999    -0.7254  
-0.7064    -0.7534     0.4724     4.0659     0.4067  
-0.7839    -0.6588     0.0000     0.0000     0.0000  
-0.6855    -0.6448     0.0000     0.6516     4.6636  
-0.6580    -0.5960    -0.1099     3.7517     0.0000
```

The value function generally converges slowly toward its optimal value.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the training time.

```
simReward = [  
    qTotalRwd  
    sarsaTotalRwd  
    lspiTtotalRwd  
    dqnTotalRwd  
    pgTotalRwd
```

```

        acTotalRwd
        ppoTotalRwd
        sacTotalRwd
    ];

    tngEpisodes = [
        qTngEps
        sarsaTngEps
        lspiTngEps
        dqnTngEps
        pgTngEps
        acTngEps
        ppoTngEps
        sacTngEps
    ];

    tngSteps = [
        qTngSteps
        sarsaTngSteps
        lspiTngSteps
        dqnTngSteps
        pgTngSteps
        acTngSteps
        ppoTngSteps
        sacTngSteps
    ];

    tngTime = [
        qTngTime
        sarsaTngTime
        lspiTngTime
        dqnTngTime
        pgTngTime
        acTngTime
        ppoTngTime
        sacTngTime
    ];

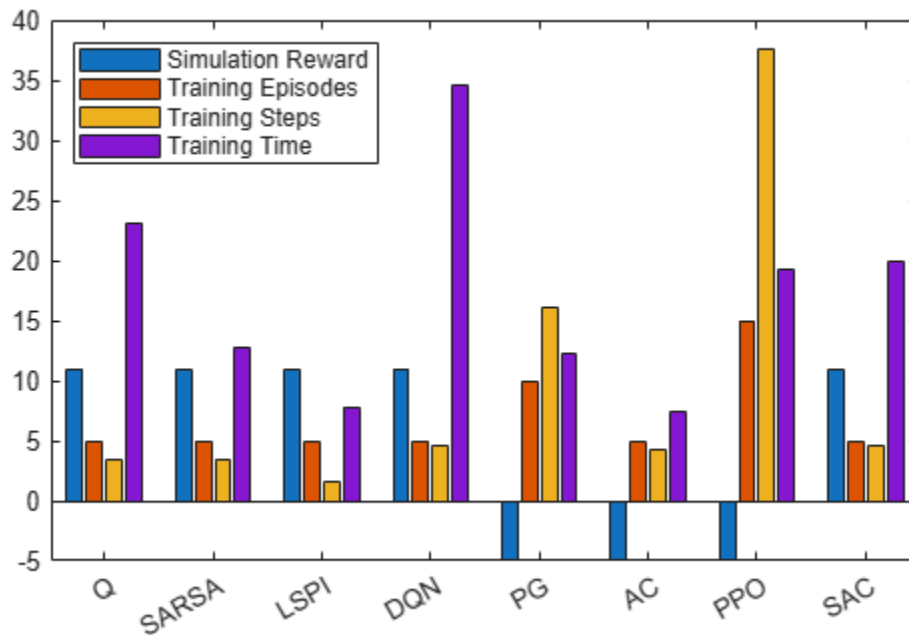
```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor `[1 10 5e5 5]` for better visualization.

```

figure;
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 10 1e4 2])
xticklabels(["Q" "SARSA" "LSPI" "DQN" "PG" "AC" "PP0" "SAC"])
legend( ...
    ["Simulation Reward", ...
    "Training Episodes", ...
    "Training Steps", ...
    "Training Time"], ...
    "Location","northwest")

```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, all agents successfully complete the task, achieving a total reward of 11 in simulation. The AC and LSPI agents require less training time. The training for the DQN agent takes slightly longer (although if you stop the DQN agent training after 40 episodes, as the training plot suggests it is possible, the training time would be shorter). The PPO agent needs more episodes, and steps, to complete training. In general, for tabular problems with a small observation and action spaces, the differences among the agents are minimal and do not justify using agents more complex than Q and SARSA.

Finally, note that with a different random seed, the initial parameters of some agents would be different, and therefore, results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlTrainingOptions` | `rlSimulationOptions` | `rlQValueFunction` | `rlValueFunction` | `rlVectorQValueFunction` | `rlDiscreteCategoricalActor`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

- “Compare Agents on Deterministic Waterfall Grid World” on page 7-36
- “Compare Agents on Stochastic Waterfall Grid World” on page 7-70

More About

- “Reinforcement Learning Environments” on page 2-2
- “Use Predefined Grid World Environments” on page 2-9
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Deterministic Waterfall Grid World

This example shows how to create and train frequently used discrete action space default agents on an 8-by-7 grid world environment. The environment has a starting location, a terminal location, and a deterministic waterfall that pushes the agent toward the bottom of the grid. The goal of the agent is to move from the starting location to the terminal location while maximizing the total reward.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

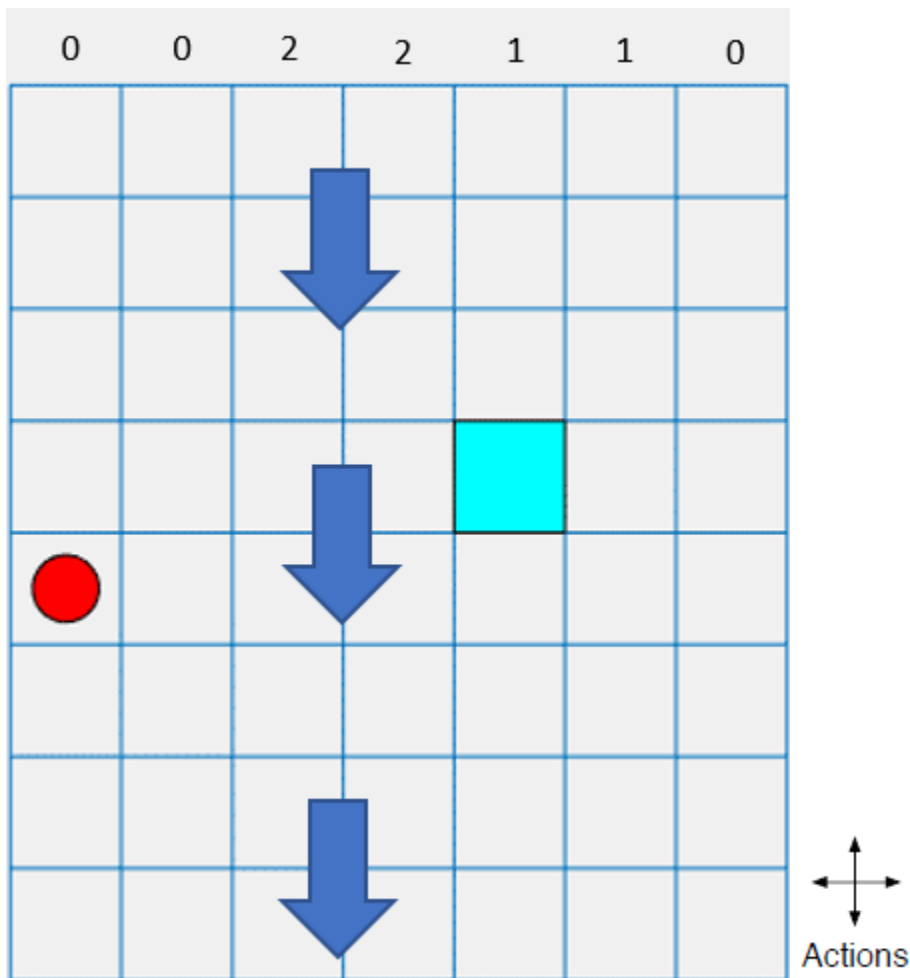
```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Deterministic Waterfall Grid World Environment

This grid world environment used in this example has the following configuration and rules:

- 1 The grid world is 8-by-7 and bounded by borders, with four possible actions (North = 1, South = 2, East = 3, West = 4).
- 2 The agent begins from cell [5,1] (fifth row, first column).
- 3 The agent receives a reward of +10 if it reaches the terminal state at cell [4,5] (light blue).
- 4 All other actions result in -1 reward.
- 5 The waterfall pushes the agent toward the bottom of the grid. Its intensity varies between the columns, as shown in the figure. That is, when the agent moves into a column with a nonzero intensity, the waterfall pushes it downward by the indicated number of squares. For example, if the agent attempts to move east from state [5,2], it reaches state [7,3].



For more information on this environment, see “Use Predefined Grid World Environments” on page 2-9.

Create Environment Object

Create a deterministic waterfall grid world.

```
env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
```

To specify that the initial state of the agent is always [5,1], create a reset function that returns the state number for the initial agent state. This function is called at the start of each training episode and simulation. States are numbered starting at position [1,1]. The state number increases as you move down the first column and then down each subsequent column. Therefore, create an anonymous function handle that sets the initial state to 5.

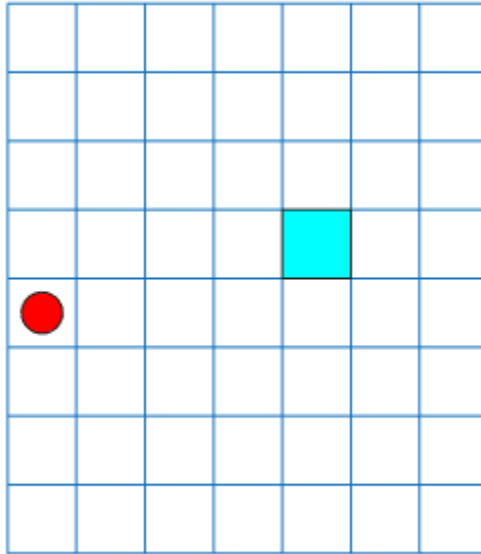
```
env.ResetFcn = @() 5;
```

Extract the environment observation and action specification objects for later use when creating agents.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
```



The plot shows the agent location with a red circle and the terminal location in light blue.

Create a Table-Based Q-Value Function Critic

In this section, you create a table-based Q-value function critic for use within Q, SARSA, and DQN agents.

Create a table object.

```
table = rlTable(obsInfo,actInfo);
```

Create a critic approximator object using `rlQValueFunction`.

```
qcritic = rlQValueFunction(table,obsInfo,actInfo);
```

Check the critic with random observation and action inputs.

```
getValue(qcritic, ...  
    {randi(numel(obsInfo.Elements))}, ...  
    {randi(numel(actInfo.Elements))})
```

```
ans =  
0
```

Create a Table-Based Value Function Critic

In this section, you create a table-based value function critic for use within AC and PPO agents. You can also use this critic as a baseline for a PG agent.

Create a table object.


```
table = rlTable(obsInfo);
```

Create a critic approximator object using `rlValueFunction`.

```
vcritic = rlValueFunction(table,obsInfo);
```

Check the critic with random observation input.

```
getValue(vcritic,{randi(numel(obsInfo.Elements))})
```

```
ans =  
0
```

Create a Custom Basis Function-Based Vector Q-Value Function Critic

In this section, you create a value function critic for use within the LSPI agent. The critic uses a custom basis function that implements a table.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Because `rlVectorQValueFunction` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```
type("cbftbl.m")
```

```
function out = cbftbl(obs,nrows)
```

```
% Table with nrows rows and one column implemented using a custom basis  
% function. The third dimension of obs is the batch dimension.
```

```
    % Get batch dimension and allocate output matrix.  
    nbtc = size(obs,3);  
    out = zeros(nrows,nbtc,like=obs);
```

```
    % Cycle through batch dimension and set to one the output  
    % corresponding to the index received as observation.  
    for k = 1:nbtc  
        if obs(1,1,k)  
            out(obs(1,1,k),k) = 1;  
        end  
    end
```

```
end
```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbftbl` function (the value of the second argument is stored in the

`basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $8 \times 7 = 56$ elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of `W0` is multiplied by the output of `basisFcn` the result is a vector in which each of the four elements is the value of the corresponding action given the observation.

```
W0 = rand(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlVectorQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```
vqcritic = rlVectorQValueFunction({basisFcn,W0},obsInfo,actInfo);
```

Check the critic with a batch of 10 random observation inputs.

```
r = randi(numel(obsInfo.Elements),[1 1 10]);  
v = getValue(vqcritic,{r});
```

Display the seventh element of the batch.

```
v(:,7)  
  
ans = 4×1  
  
    0.8003  
    0.8407  
    0.0844  
    0.7447
```

Create a Custom Basis Function-Based Q-Value Function Critic

In this section, you create a custom basis function-based value function critic for use within the LSPI agent.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Because the LSPI agent does not support tables as approximator models, use the helper function defined in the supporting file `cbfqtbl.m`, which implements a q-value function table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```
type("cbfqtbl.m")
```

```
function out = cbfqtbl(obs,act,nr,nc)

% Table with nr rows and nc columns implemented using a custom basis
% function. The third dimension of obs and act is the batch dimension.

% Get batch dimension.
nbtc = size(obs,3);

% Allocate output matrix in columnwise format, that is the first two
% dimensions are vectorized as in out=table(:)
out = zeros(nr*nc,nbtc,like=obs);

% Cycle through batch dimension and set to one the output
% corresponding to the indexes of observation and action.
for k = 1:nbtc
    if obs(1,1,k)
        out(nr*(act(1,1,k)-1)+obs(1,1,k),k) = 1;
    end
end

end
```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbfqtbl` function (the value of the last two arguments is stored in the `basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $8*7*4=224$ elements, where the element equal to 1 indicates the columnwise location of the current observation and action.

```
basisFcnQ = @(obs,act) cbfqtbl(obs,act, ...
    numel(obsInfo.Elements), ...
    numel(actInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result is a scalar indicating the value of the corresponding action given the observation.

```
W0 = rand(numel(obsInfo.Elements)*numel(actInfo.Elements),1);
```

For example, the weight corresponding to observation number 13 and action number 3 is:

```
W0(numel(obsInfo.Elements)*(3-1)+13)
```

```
ans =
0.8173
```

Use `rlQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```
qcbfcritic = rlQValueFunction({basisFcnQ,W0},obsInfo,actInfo);
```

Ignoring the batch dimension, check the critic by returning the value corresponding to the observation number 13 and the action number 3.

```
getValue(qcbfcritic,{13},{3})
```

```
ans =
0.8173
```

Check the critic with a batch of 10 random observation and action inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);
ract = randi(numel(actInfo.Elements),[1 1 10]);
v = getValue(qcbfcritic,{robs},{ract});
```

Display the seventh element of the batch.

```
v(:,7)
```

```
ans =
0.3786
```

Create a Custom Basis Function-Based Actor

In this section, you create a discrete categorical actor for use within PG, AC, and PPO agents. The actor uses a custom basis function that implements a table.

Because `rlDiscreteCategoricalActor` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Define the custom basis function. Here, the output of the basis function is a one-hot vector of 8×7 elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result, after passing through a `softmax` function, is a vector in which each of the four elements is the probability of executing the corresponding action.

```
W0 = zeros(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlDiscreteCategoricalActor` to create the actor object, passing the basis function handle and the initial condition as a cell.

```
actor = rlDiscreteCategoricalActor({basisFcn,W0},obsInfo,actInfo);
```

Check the actor with a batch of 10 random observation inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);
v = getAction(actor,{robs});
```

Display the seventh element of the batch.

```
v{1}(:, :, 7)
```

```
ans =
1
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 50 training episodes. For more information, see `rlEvaluator`.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=50);
```

Create a training options object.

```
trainOpts = rlTrainingOptions;
```

For this example, use the following options:

- Train for a maximum of 5000 episodes. Specify that each episode lasts for most 50 time steps.
- Stop the training when the average reward in the evaluation episodes is greater than -4.

```
trainOpts.MaxEpisodes= 5000;
trainOpts.MaxStepsPerEpisode = 50;
trainOpts.StopTrainingCriteria = "EvaluationStatistic";
trainOpts.StopTrainingValue = -4;
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 50 steps.

```
simOpts = rlSimulationOptions(MaxSteps=50);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a Q-Agent

In this section, you create a Q-learning agent and then configure its parameters.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create an `rlQAgent` object using the critic you have created before.

```
qAgent = rlQAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
qAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
qAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

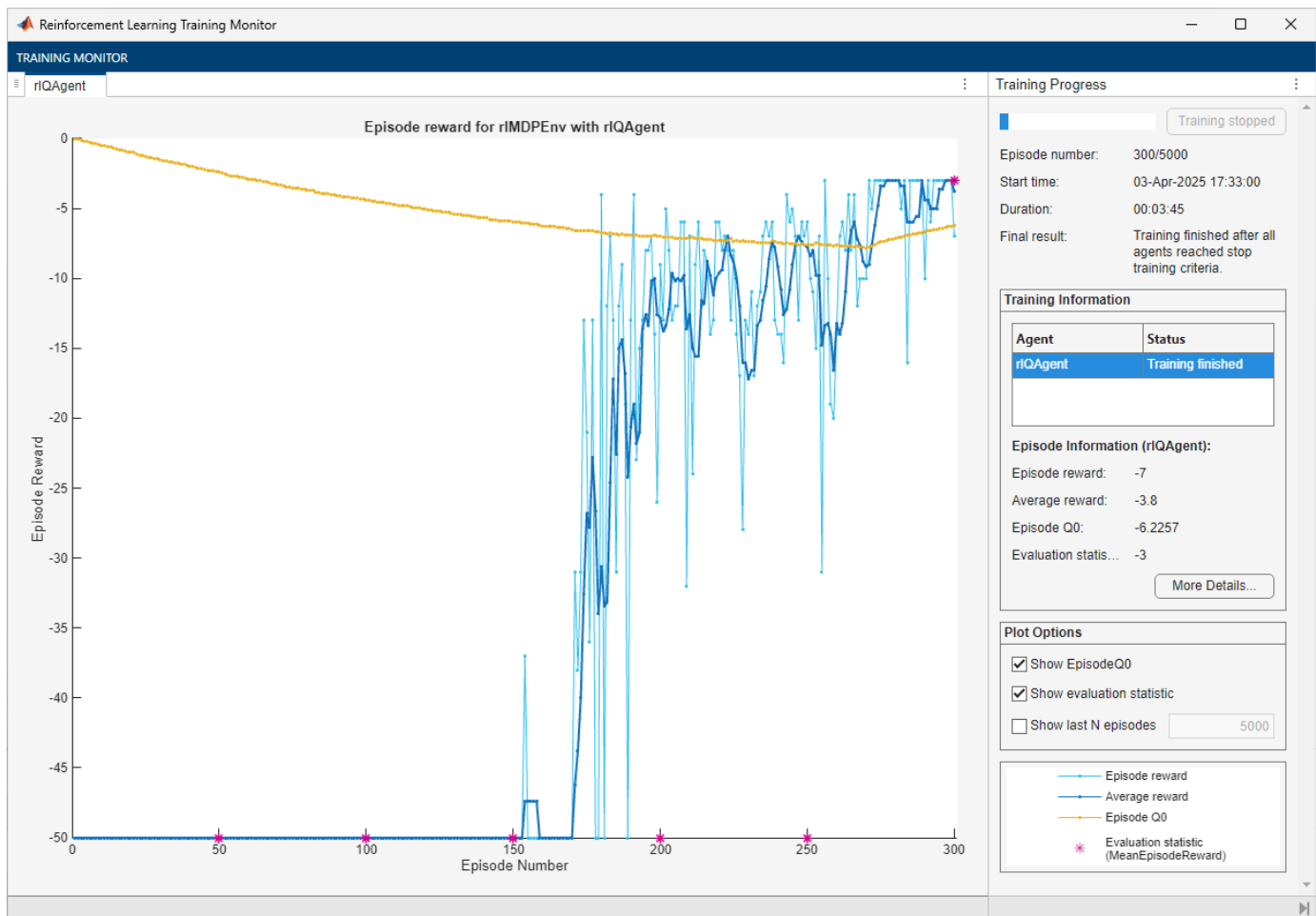
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
```

```

env.ResetFcn = @() 5;
% Train the agent. Record the training time.
tic
qTngRes = train(qAgent,env,trainOpts,Evaluator=evl);
qTngTime = toc;
% Extract the number of training episodes and total steps.
qTngEps = qTngRes.EpisodeIndex(end);
qTngSteps = sum(qTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("dwgwbchQAgent.mat", ...
%      "qAgent","qTngEps","qTngSteps","qTngTime")
else
% Load the pretrained agent and results for the example.
load("dwgwbchQAgent.mat", ...
     "qAgent","qTngEps","qTngSteps","qTngTime")
end

```



The training converges after 300 episodes. The evaluation statistic indicates a reward of -3, suggesting that the agent learns the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

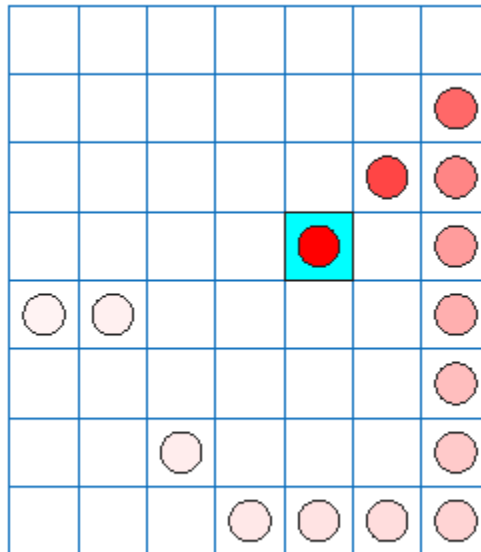
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(qAgent, env, simOpts);
```



```
qTotalRwd = sum(experience.Reward)
```

```
qTotalRwd =
-3
```

The agent trace shows that the trained agent successfully finds the target cell using 13 moves before the final one. As a consequence, the total reward is $-13+10=-3$.

Extract the table with the final Q-value function.

```
qAgtFinalQ = getLearnableParameters(getCritic(qAgent));
```

Display the maximum values for each state (that is, the approximate value function) in a 5-by-5 format.

```
reshape(max(qAgtFinalQ{1}'), 8, 7)
```

```
ans = 8×7 single matrix
```

-4.2257	-3.1157	0	0	0	0	-0.3175
-4.9957	-4.0483	0	0	0	0	7.1173
-5.8562	-4.9840	-2.1289	0	0	8.5674	5.8588
-6.8597	-5.8324	-3.1404	0	0	-0.8528	4.6856
-6.2257	-5.2362	-4.0187	-1.3179	-0.3176	-1.6152	3.5586
-7.8896	-7.5039	-5.0840	-2.1760	-1.3758	-2.7585	2.4564
-8.2139	-8.2086	-4.2510	-3.0622	-2.2710	-3.8699	1.3602
-8.6390	-8.7819	-6.2467	-3.0087	-1.9119	-0.8223	0.2664

Create, Train, and Simulate a SARSA Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlSARSAAgent` object using the critic you have created before.

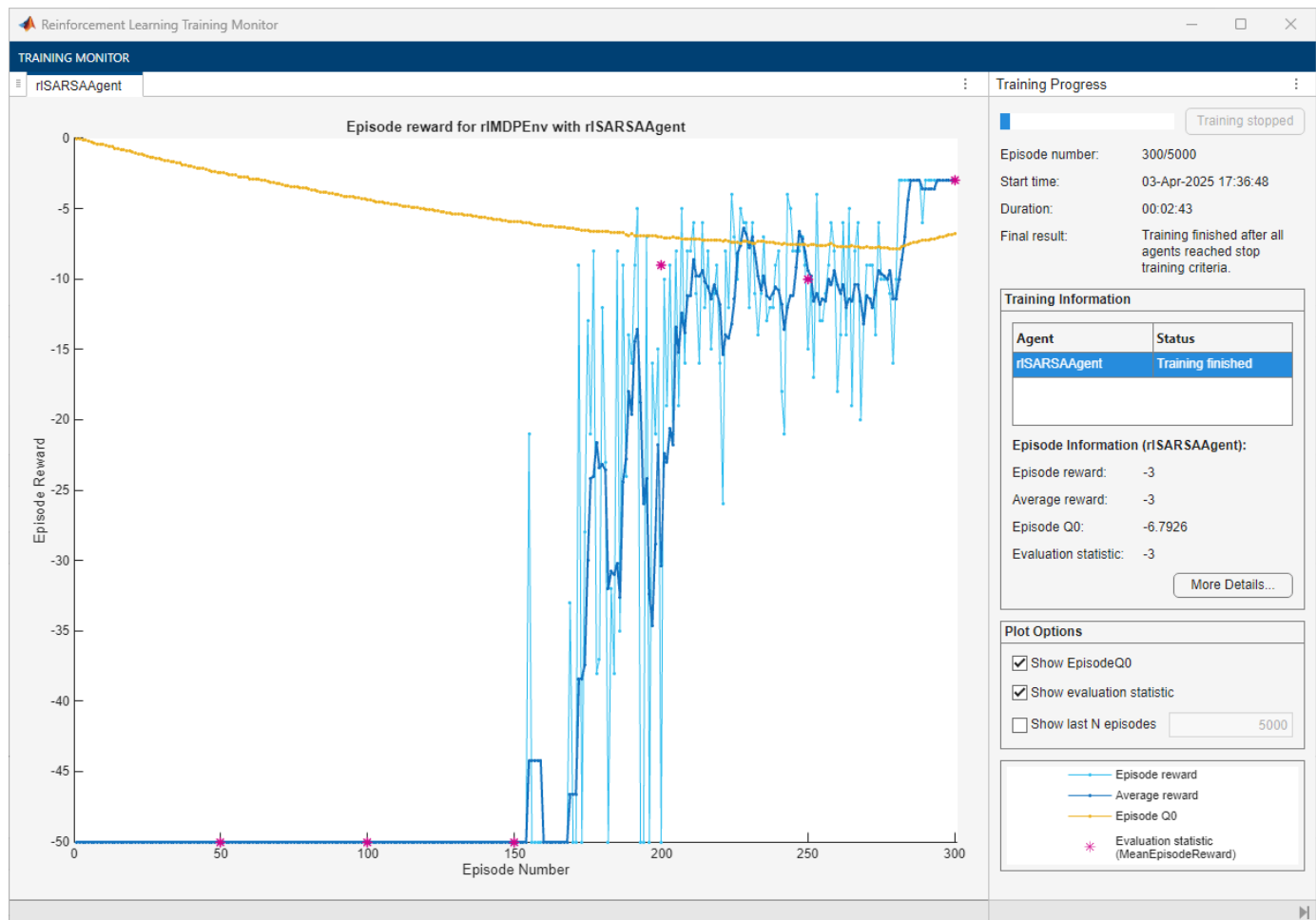
```
sarsaAgent = rlSARSAAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
sarsaAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
sarsaAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    sarsaTngRes = train(sarsaAgent, env, trainOpts, Evaluator=evl);
    sarsaTngTime = toc;
    % Extract the number of training episodes and total steps.
    sarsaTngEps = sarsaTngRes.EpisodeIndex(end);
    sarsaTngSteps = sum(sarsaTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dwgwBchSARSAAgent.mat", ...
    %     "sarsaAgent", "sarsaTngEps", "sarsaTngSteps", "sarsaTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dwgwBchSARSAAgent.mat", ...
        "sarsaAgent", "sarsaTngEps", "sarsaTngSteps", "sarsaTngTime")
end
```

The training converges after 300 episodes. The evaluation statistic indicates a reward of -3, suggesting that the agent learns the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

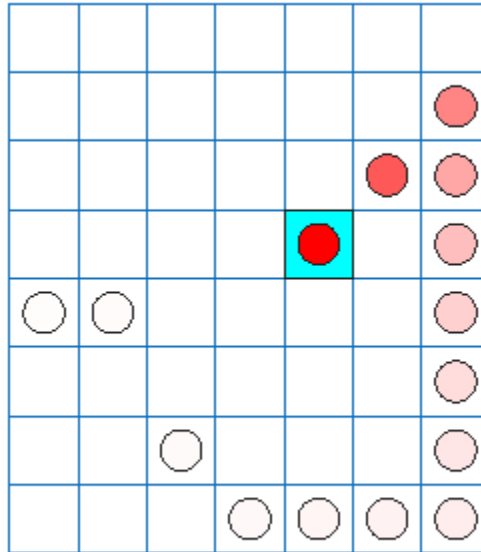
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sarsaAgent, env, simOpts);
```



```
sarsaTotalRwd = sum(experience.Reward)
```

```
sarsaTotalRwd =  
-3
```

The agent trace shows that the trained agent successfully finds the target cell using 13 moves before the final one. As a consequence, the total reward is $-13+10=-3$.

Extract the table with the final Q-value function.

```
sarsaAgtFinalQ = getLearnableParameters(getCritic(sarsaAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(sarsaAgtFinalQ{1}'),8,7)
```

```
ans = 8×7 single matrix
```

-4.1940	-3.2600	0	0	0	0	-0.3174
-5.0751	-4.2347	0	0	0	0	7.0528
-6.0143	-5.1694	-2.3277	0	0	8.6155	5.8630
-6.9340	-6.0262	-3.2752	0	0	-0.9304	4.7042
-6.7926	-5.8077	-4.2307	-1.3857	-0.3174	-1.9503	3.3942
-8.0133	-7.5613	-5.1869	-2.3323	-1.5106	-2.8342	2.2368
-8.4134	-8.5842	-4.8273	-3.2116	-2.4773	-4.0990	0.9018
-8.8060	-8.7745	-6.3283	-3.6302	-2.5059	-1.3913	-0.2761

Create, Train, and Simulate an LSPI Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

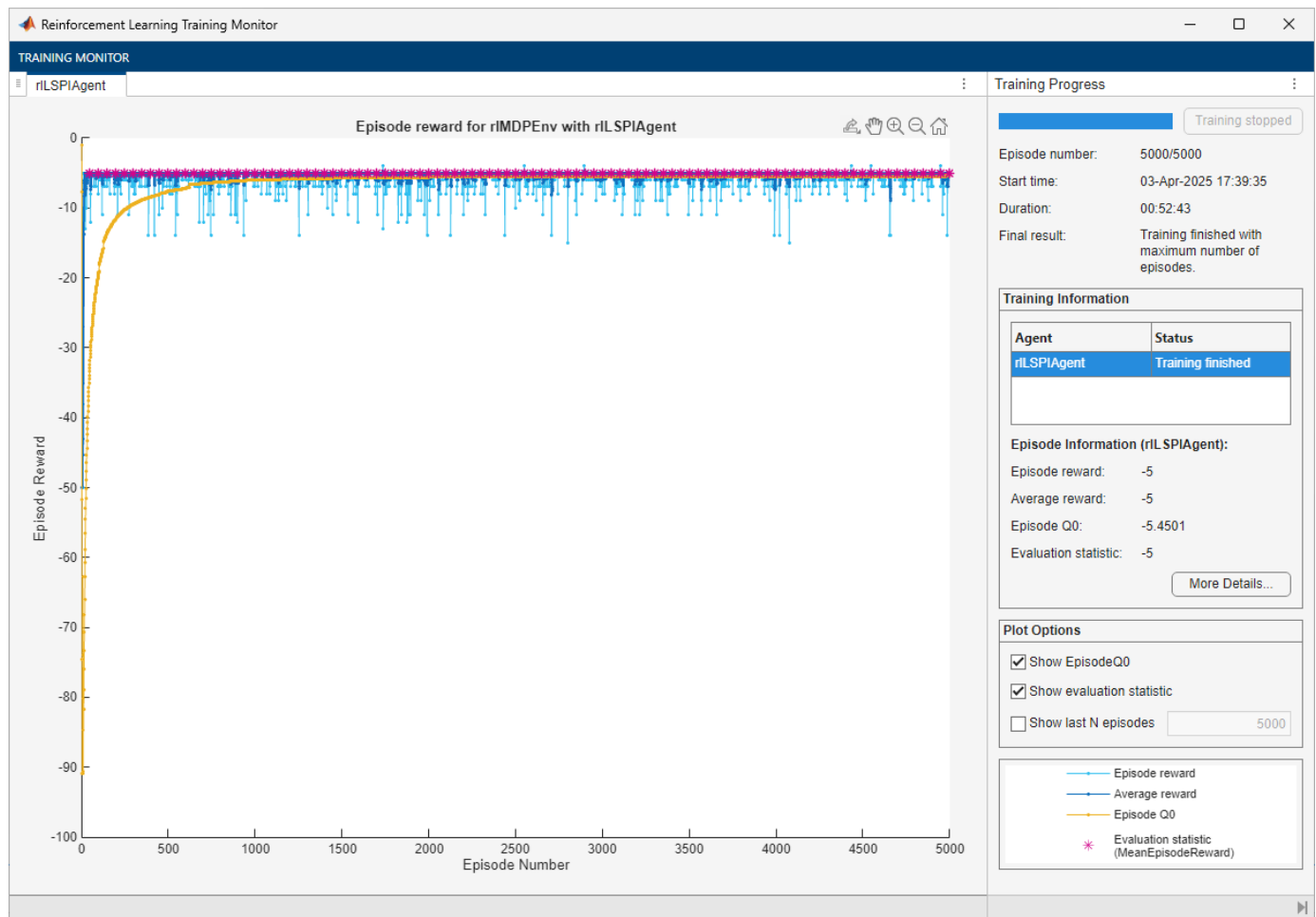
```
rng(0, "twister")
```

Create an `rLLSPIAgent` object using the critic you have created before.

```
lsplAgent = rlLSPIAgent(qcbfcritic);
```

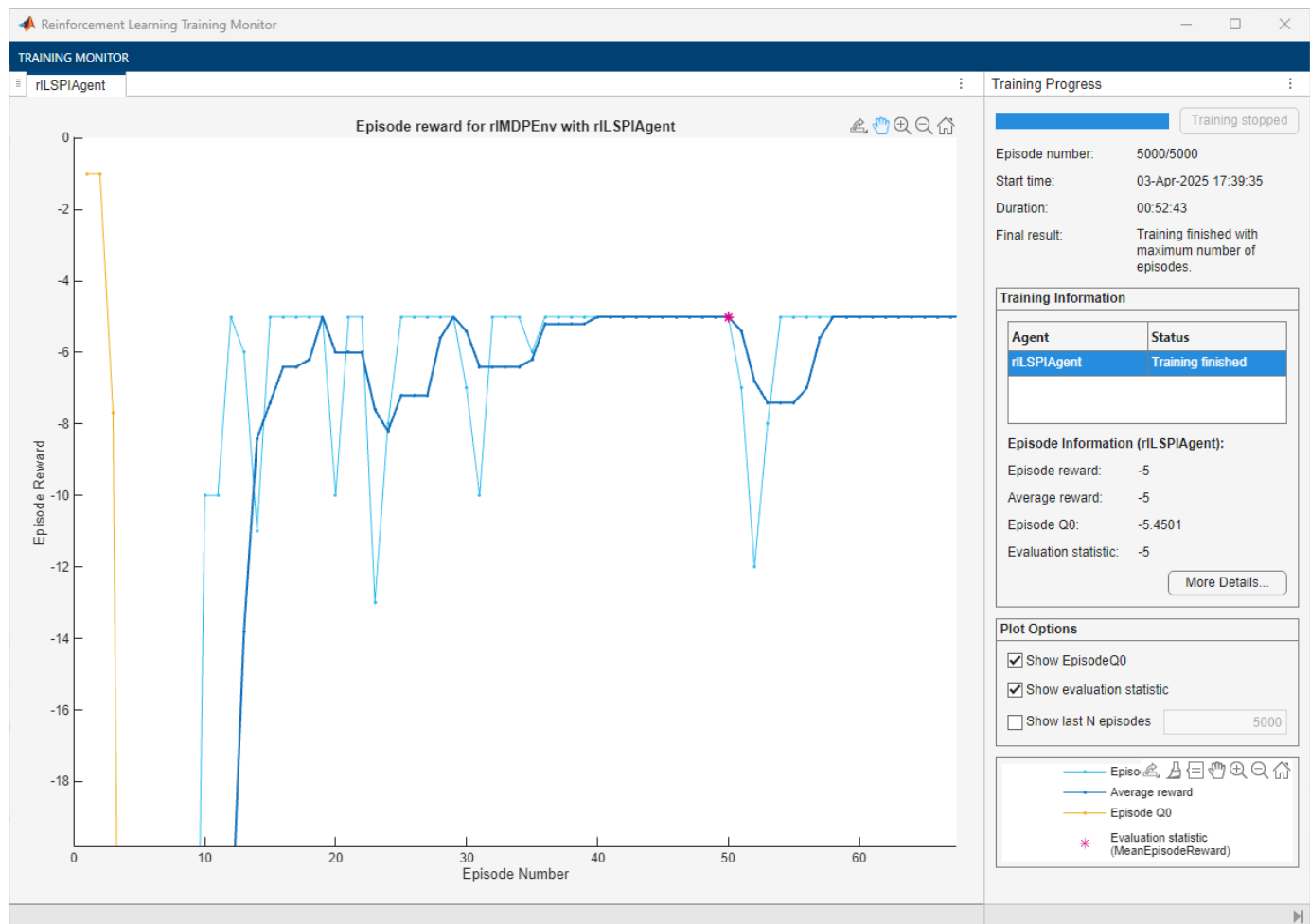
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    lsplTngRes = train(lsplAgent,env,trainOpts,Evaluator=evl);
    lsplTngTime = toc;
    % Extract the number of training episodes and total steps.
    lsplTngEps = lsplTngRes.EpisodeIndex(end);
    lsplTngSteps = sum(lsplTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dwgwBchLSPIAgent.mat", ...
    %     "lsplAgent","lsplTngEps","lsplTngSteps","lsplTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dwgwBchLSPIAgent.mat", ...
        "lsplAgent","lsplTngEps","lsplTngSteps","lsplTngTime")
end
```



The training does not converge after 5000 episodes. The evaluation statistic indicates a reward of -5, suggesting that the agent learns a good policy, though not the optimal one.

Zooming in, you can see that the agent learns a relatively good policy after just 50 steps. However, it is unable to improve upon it afterwards.



You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

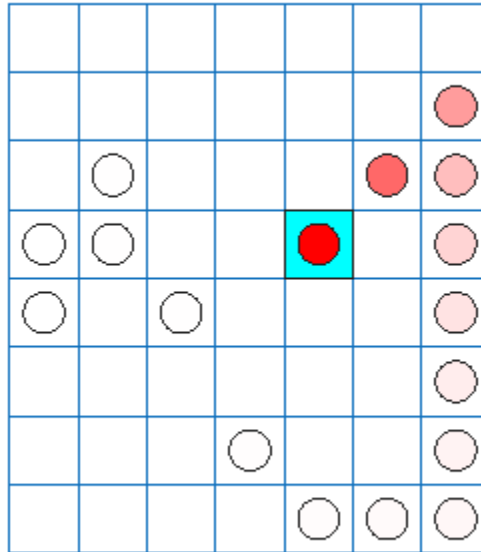
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(lspiAgent, env, simOpts);
```



```
lspiTotalRwd = sum(experience.Reward)
```

```
lspiTotalRwd =  
-5
```

The agent trace shows that the trained agent successfully finds the target cell using 15 moves before the final one. As a consequence, the total reward is $-15+10=-5$.

Extract the table with the final Q-value function.

```
lspiAgtFinalQ = getLearnableParameters(getCritic(lspiAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(reshape(lspiAgtFinalQ{1},8*7,4)'),8,7)
```

```
ans = 8×7 single matrix
```

-5.4907	-0.9990	0	0	0	0	7.5611
-4.5389	-5.4930	0	0	0	4.5907	8.8997
-3.5750	-2.5442	0	0	1.4559	9.9999	7.7988
-4.4855	-3.5207	-2.5366	0	0	5.6518	6.7193
-5.4501	-6.7605	-1.5566	0	2.4822	1.4586	5.6263
-7.0494	-32.5298	-1.5844	0	-0.5613	3.5186	4.5646
-17.0282	-33.4044	-34.0533	-0.5618	0.4424	2.4835	3.5187
-44.5640	-45.1093	-1.6356	-0.5904	0.4426	1.4574	2.4827

Create, Train, and Simulate a DQN Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlDQNAgent` object using the critic you have created before.

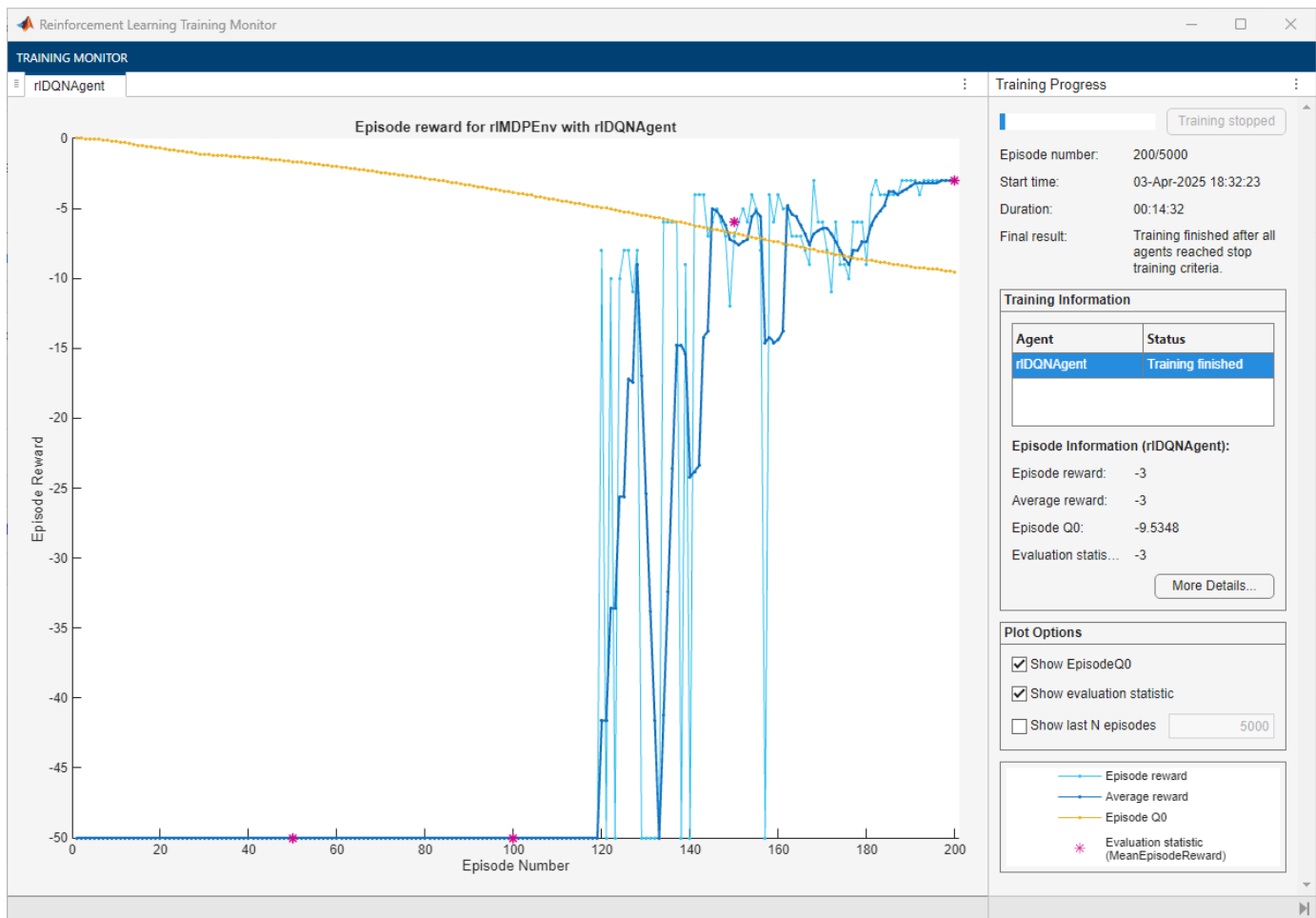
```
dqnAgent = rLDQNAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);
    dqnTngTime = toc;
    % Extract the number of training episodes and total steps.
    dqnTngEps = dqnTngRes.EpisodeIndex(end);
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dwgwBchDQNAgent.mat", ...
    %      "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dwgwBchDQNAgent.mat", ...
         "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
end
```



The training converges after 200 episodes. The evaluation statistic indicates a reward of -3, suggesting that the agent learns the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

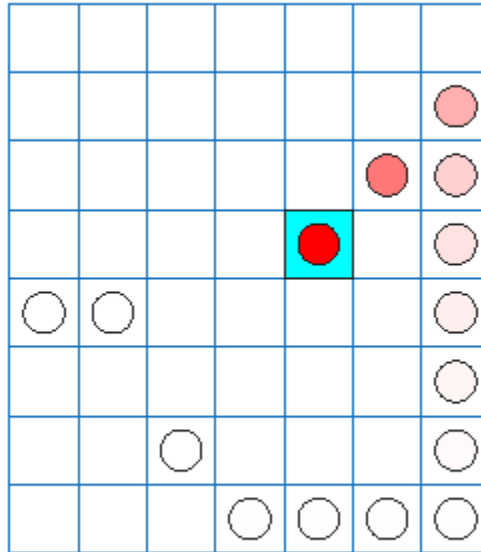
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(dqnAgent, env, simOpts);
```

```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
-3
```

The agent trace shows that the trained agent successfully finds the target cell using 13 moves before the final one. As a consequence, the total reward is $-13+10=-3$.

Extract the table with the final Q-value function.

```
dqnAgtFinalQ = getLearnableParameters(getCritic(dqnAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(dqnAgtFinalQ{1}'),8,7)
```

```
ans = 8×7 single matrix
```

```
-8.6657    -7.9881         0         0         0         0    3.5942  
-8.8529    -8.4621         0         0         0    0.6737    8.7004  
-8.8933    -8.4340    -7.0275         0         0    9.8776    7.3451  
-9.2301    -8.7012    -7.9135         0         0    3.7277    5.8220  
-9.5348    -9.0974    -7.8437    -5.8312    -2.3116    1.7081    3.9497  
-9.3617    -8.9902    -8.2519    -7.1921    -4.3606   -0.4490    1.8238  
-9.3011    -8.9294    -8.5626    -7.1970    -6.2022   -2.5998   -0.4476  
-9.5020    -9.4107    -8.6882    -7.6938    -6.3493   -4.6553   -2.6607
```

The value function is still converging toward its optimal value.

Create, Train, and Simulate a PG Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPGAgent` object using the actor you have created before. Alternatively, to use a baseline, supply the critic you have created before `vcritic` as a second argument. However, for this example, do not use any baseline.

```
pgAgent = rlPGAgent(actor);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

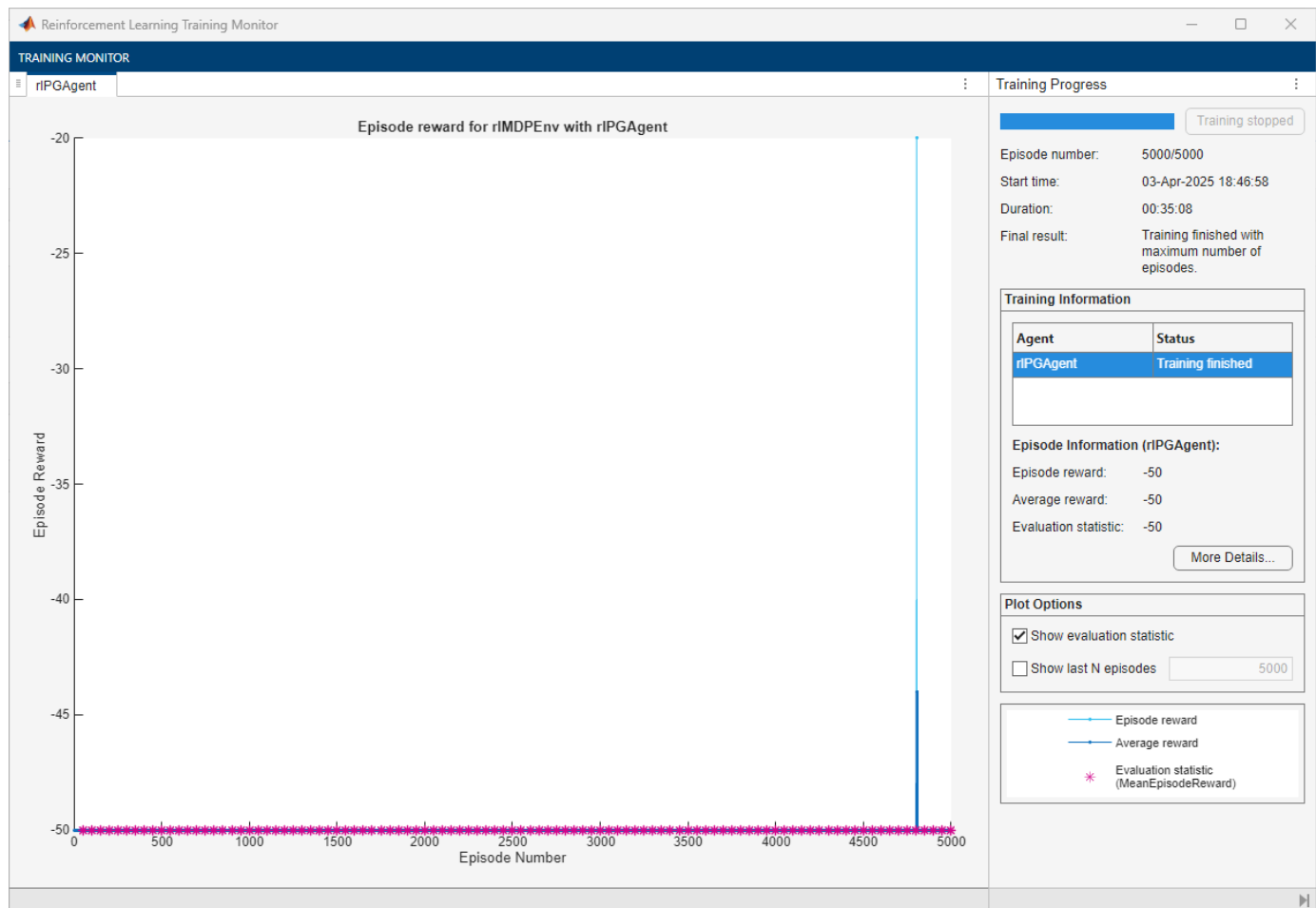
```
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");  
    env.ResetFcn = @() 5;  
    % Train the agent. Record the training time.  
    tic  
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);  
    pgTngTime = toc;  
    % Extract the number of training episodes and total steps.  
    pgTngEps = pgTngRes.EpisodeIndex(end);  
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("dwgwBchPGAgent.mat", ...  
    %     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("dwgwBchPGAgent.mat", ...  
        "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")  
end
```



The training does not converge within 5000 episodes.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

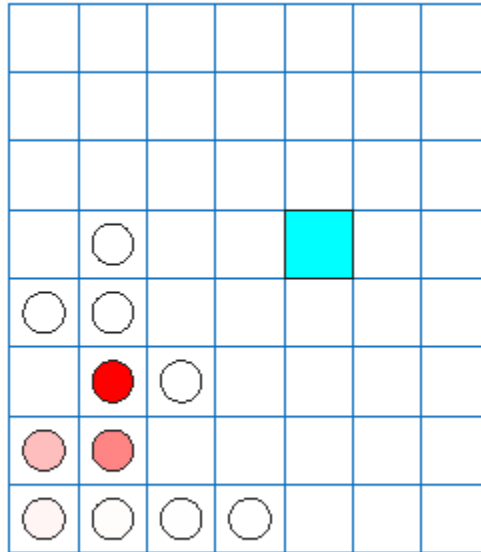
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(pgAgent, env, simOpts);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-50
```

The agent trace shows that the trained agent does not move toward the goal.

Create, Train, and Simulate an AC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlACAgent` object using the actor and critic you have created before.

```
acAgent = rlACAgent(actor,vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

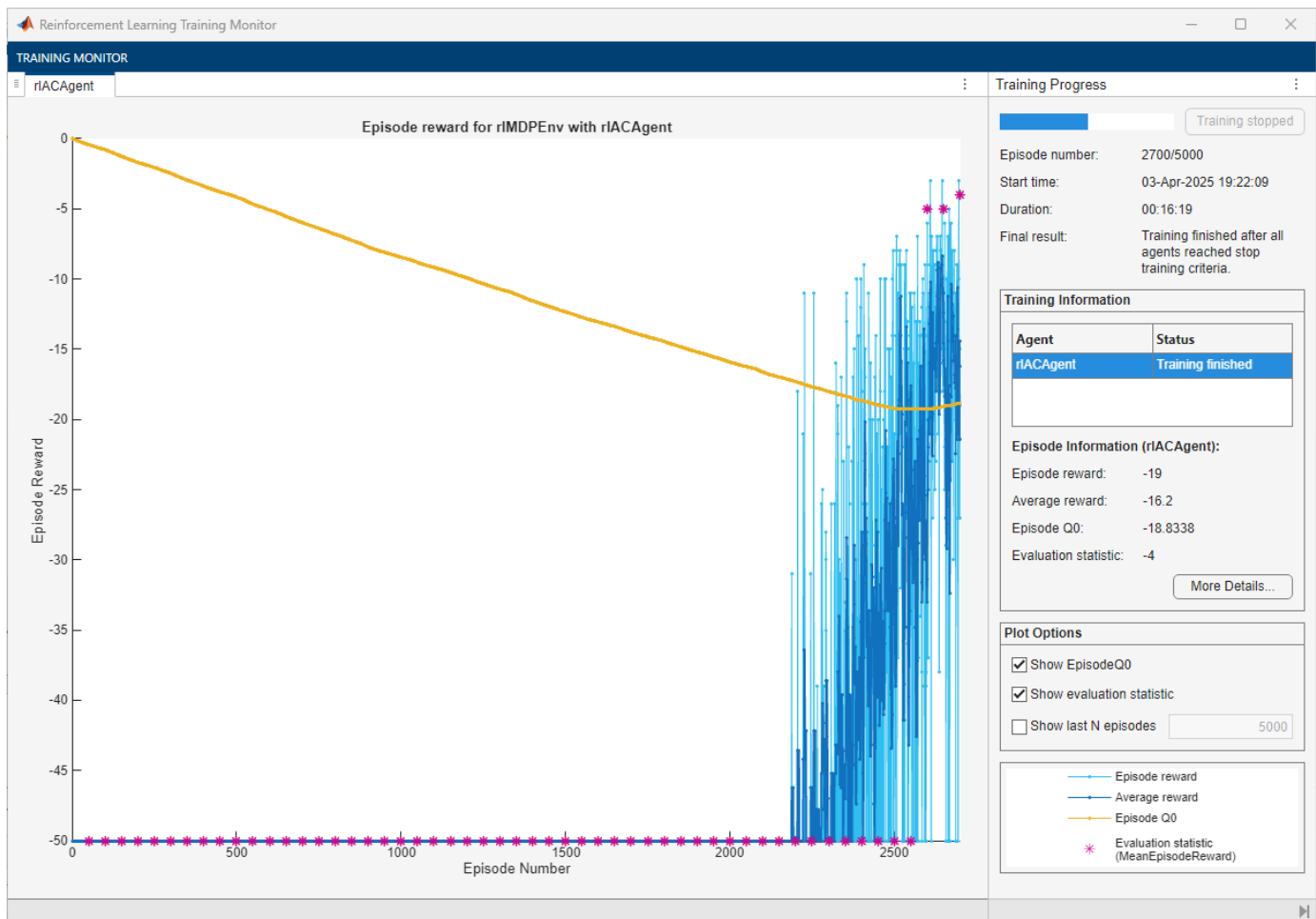
```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract the number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dwgwbChACAgent.mat", ...
    %     "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dwgwbChACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end

```



The training converges after 2700 episodes. The evaluation statistic indicates a reward of -4, suggesting that the agent learns a policy that is close to optimal.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

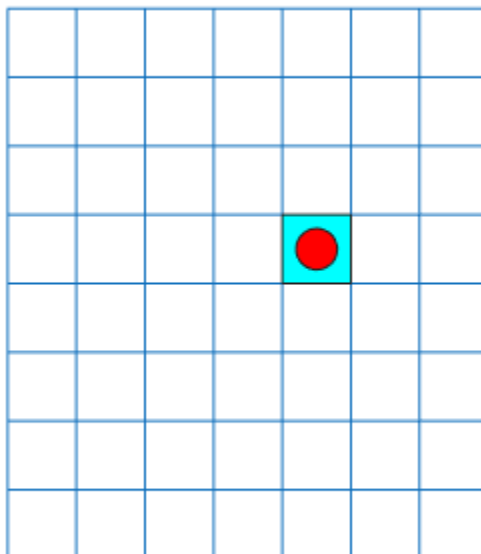
Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(acAgent, env, simOpts);
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =
-32
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
```



```
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

The agent trace shows that the trained agent successfully finds the target cell using 14 moves before the final one. As a consequence, the total reward is $-14+10=-4$.

Because the AC agent has a critic, you can display the final value function represented by the critic.

```
acAgtFinalV = getLearnableParameters(getCritic(acAgent));
reshape(acAgtFinalV{1},8,7)
```

```
ans = 8×7 single matrix
```

```
-12.1903 -10.5137      0      0      0      0 -2.0298
-13.8635 -12.5657      0      0      0 -1.3498  2.6374
-14.8977 -13.8307 -8.3393      0 -1.3034  5.5643  1.7149
-16.8806 -15.0335 -9.0708      0      0 -2.8500 -0.9160
-18.8299 -15.6998 -9.9807 -5.8399 -1.7689 -3.5746 -4.7313
-15.7156 -14.0735 -10.8539 -4.8133 -4.3911 -7.1281 -9.1139
-14.1992 -14.1154 -13.2906 -6.7896 -5.2680 -7.3886 -8.3594
-15.1064 -14.9141 -14.7722 -13.0220 -12.3303 -11.4281 -8.9485
```

The value function generally converges very slowly toward its optimal value.

Create, Train, and Simulate a PPO Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPPOAgent` object using the actor and critic you have created before.

```
ppoAgent = rlPPOAgent(actor,vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

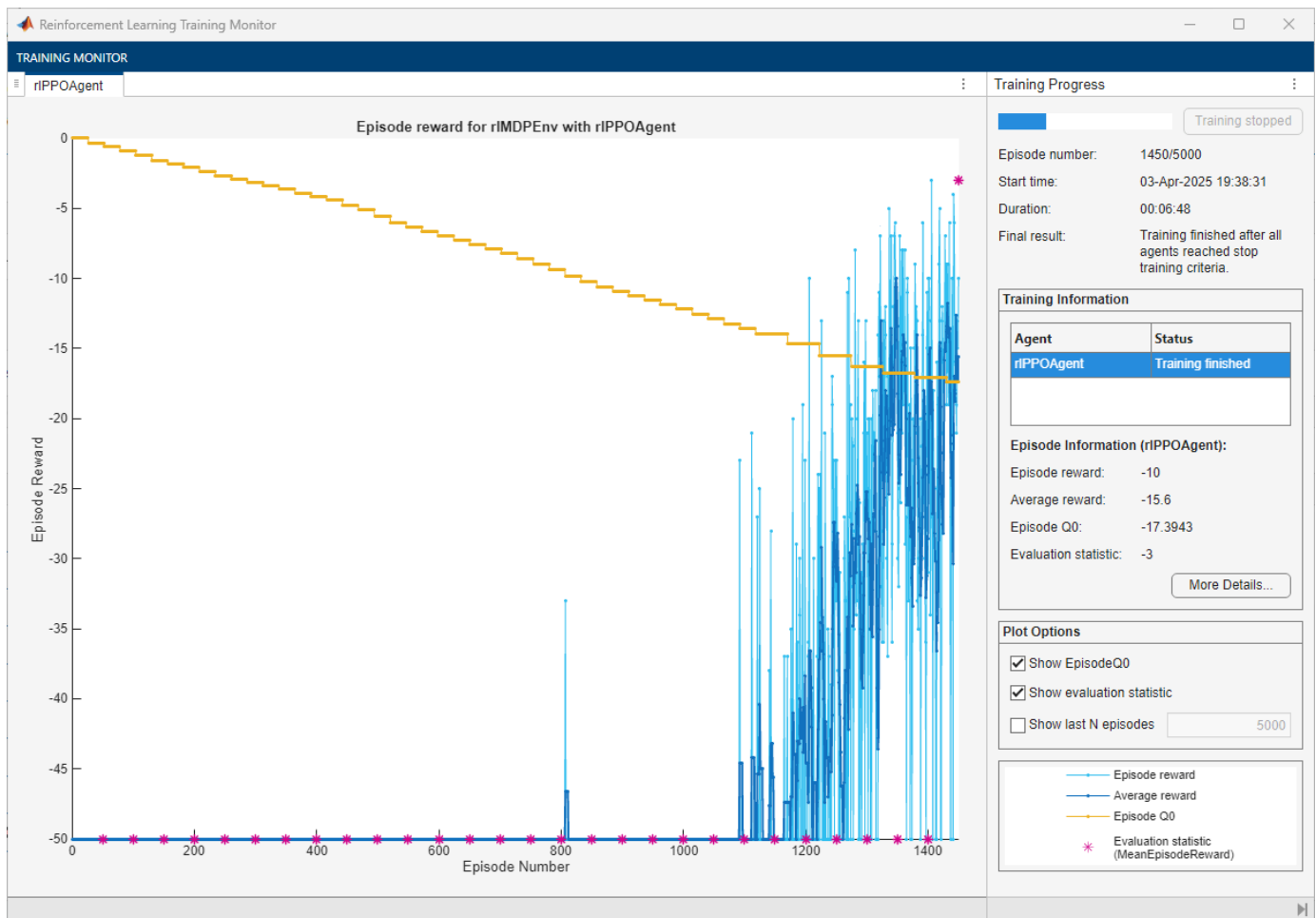
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
    ppoTngTime = toc;
    % Extract the number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
```

```

% Uncomment to save the trained agent and the training metrics.
% save("dwgwBchPPOAgent.mat", ...
%      "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
else
% Load the pretrained agent and results for the example.
load("dwgwBchPPOAgent.mat", ...
     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
end

```



The training does not converge after 1450 episodes.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```

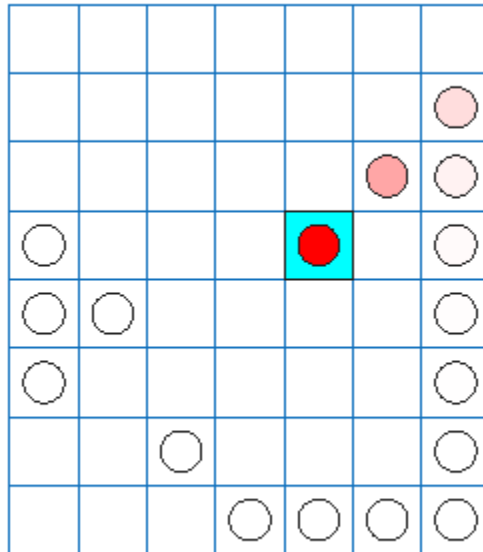
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;

```


By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(ppoAgent,env,simOpts);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-11
```

The agent trace shows that the trained agent successfully finds the target cell using 13 moves before the final one. As a consequence, the total reward is $-13+10=-3$.

Because the PPO agent has a critic, you can display the final value function represented by the critic.

```
ppoAgtFinalV = getLearnableParameters(getCritic(ppoAgent));  
reshape(ppoAgtFinalV{1},8,7)
```

```
ans = 8×7 single matrix
```

```
-8.4321  -8.1510      0      0      0      0      0.3839  
-10.8324 -9.3702      0      0      0  -1.0838  4.5645  
-12.5307 -12.5771 -5.3474      0      0   6.0063  1.8015  
-15.4225 -13.2510 -5.3307      0      0  -4.5348 -1.4900  
-17.3943 -15.5440 -10.8317 -2.8600 -2.2014 -4.9594 -6.5872  
-16.6648 -14.2710 -9.0944 -2.2028 -2.3622 -7.0213 -9.0436  
-15.3039 -13.8544 -13.3818 -9.3098 -4.5140 -10.3826 -10.8493  
-13.2041 -13.8543 -16.0321 -13.0978 -13.0685 -12.8697 -11.9754
```

Similarly to the other agents with actors, the value function converges slowly toward its optimal value.

Create, Train, and Simulate a SAC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlSACAgent` object using the actor you have created before and critic.

```
sacAgent = rlSACAgent(actor, vqcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

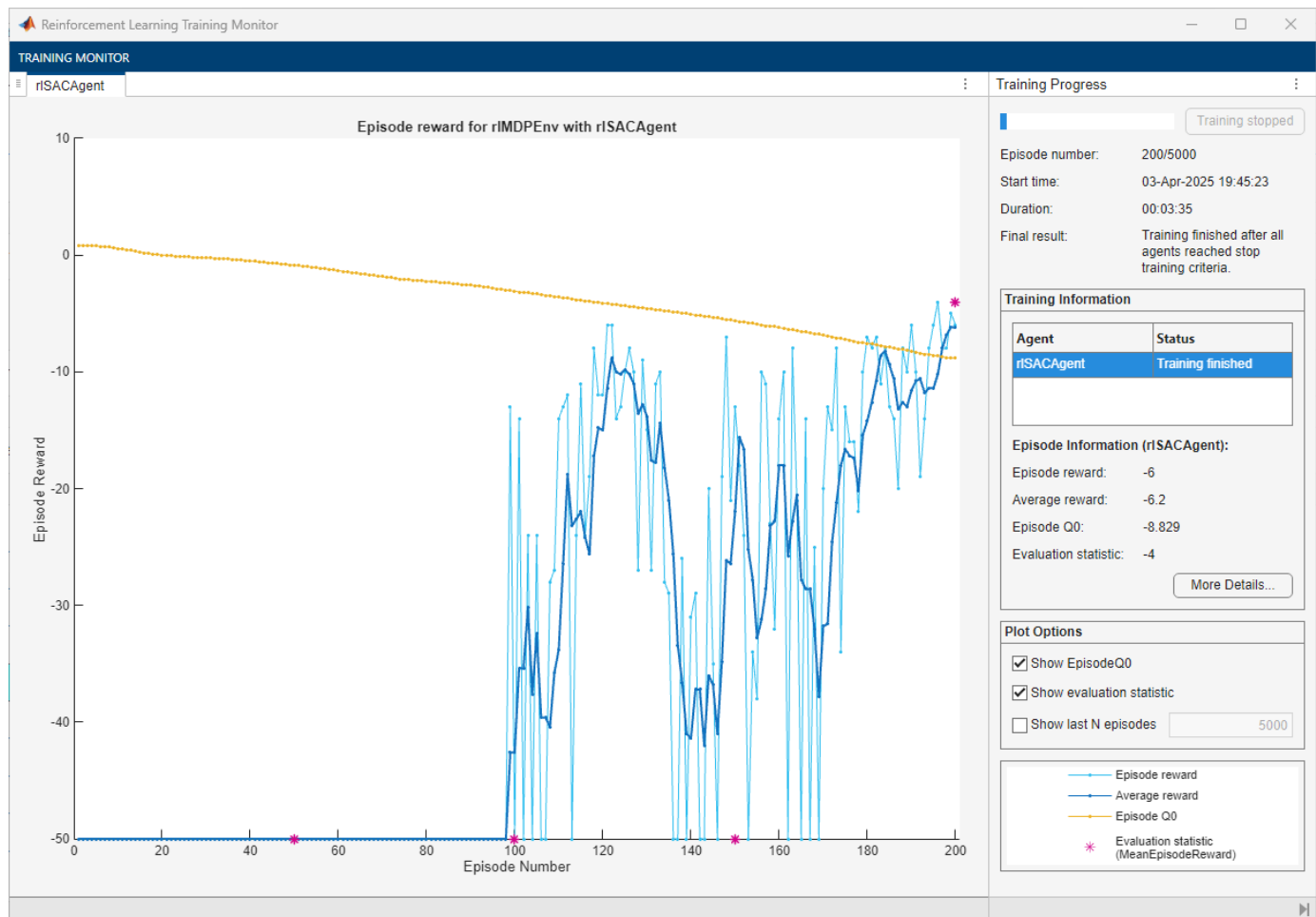
```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Train the agent, passing the previously created evaluator object to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Deterministic");
    env.ResetFcn = @() 5;
    % Train the agent. Record the training time.
    tic
    sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
    sacTngTime = toc;
    % Extract the number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dwgWBchSACAgent.mat", ...
    %     "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dwgWBchSACAgent.mat", ...
        "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
end
```



The training converges after 200 episodes. The evaluation statistic indicates a reward of -4, suggesting that the agent learns a policy that is close to optimal.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

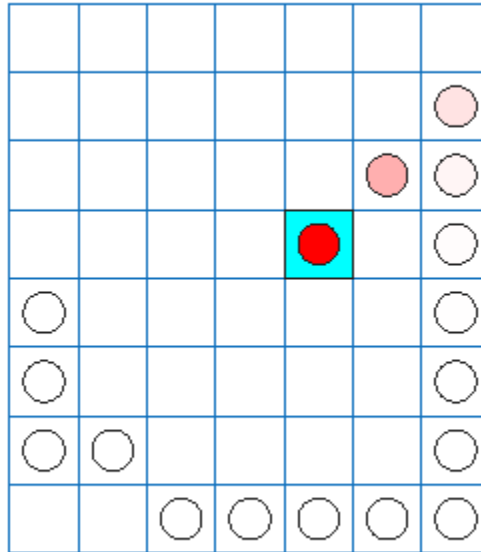
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sacAgent, env, simOpts);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-7
```

The agent trace shows that the trained agent successfully finds the target cell using 14 moves before the final one. As a consequence, the total reward is $-14+10=-4$.

Because the SAC agent has a critic, you can display the final value function represented by critic.

```
sacAgtFinalV = getLearnableParameters(getCritic(sacAgent));  
reshape(max(sacAgtFinalV{1}),8,7)
```

```
ans = 8x7 single matrix
```

-7.8392	-7.1230	0.0000	0.0000	0.0000	0.0000	6.3223
-8.2452	-7.6912	0.0000	0.0000	0.0000	4.0144	8.6009
-8.4075	-7.8882	-5.9942	0.0000	1.3674	9.8960	7.4325
-8.5092	-8.0298	-7.0145	0.0000	0.0000	4.3572	6.1081
-8.8290	-8.3151	-7.0877	-4.4688	-0.0000	2.9514	4.7192
-8.8351	-8.6081	-7.3739	-5.9841	-2.6657	0.9844	3.1032
-8.6044	-8.1301	-7.4058	-6.0216	-4.5380	-0.7355	1.2507
-9.0038	-8.7088	-7.6644	-6.2942	-4.6663	-2.7624	-0.7422

The value function generally converges slowly toward its optimal value.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the training time.

```
simReward = [  
    qTotalRwd
```

```

    sarsaTotalRwd
    lspiTtotalRwd
    dqnTotalRwd
    pgTotalRwd
    acTotalRwd
    ppoTotalRwd
    sacTotalRwd
];

tngEpisodes = [
    qTngEps
    sarsaTngEps
    lspiTngEps
    dqnTngEps
    pgTngEps
    acTngEps
    ppoTngEps
    sacTngEps
];

tngSteps = [
    qTngSteps
    sarsaTngSteps
    lspiTngSteps
    dqnTngSteps
    pgTngSteps
    acTngSteps
    ppoTngSteps
    sacTngSteps
];

tngTime = [
    qTngTime
    sarsaTngTime
    lspiTngTime
    dqnTngTime
    pgTngTime
    acTngTime
    ppoTngTime
    sacTngTime
];

```

Because the training for the PG and LSPI agents does not converge, to avoid visualizing their metrics, set them to NaN.

```

simReward([3,5]) = NaN;
tngEpisodes([3,5]) = NaN;
tngSteps([3,5]) = NaN;
tngTime([3,5]) = NaN;

```

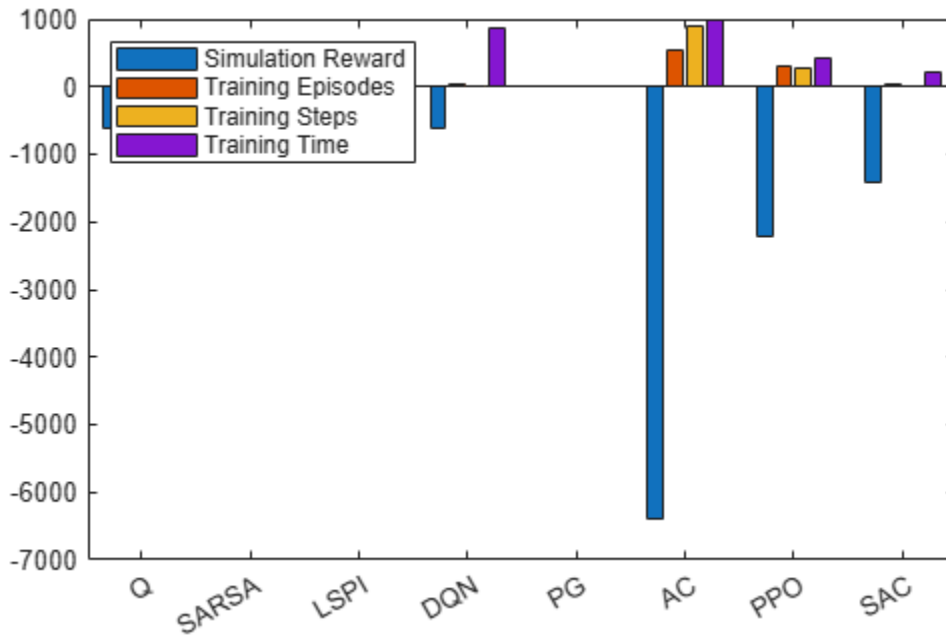
Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [1 10 5e5 5] for better visualization.

```

figure;
bar([simReward,tngEpisodes,tngSteps,tngTime]./[5e-3 5 2e5 1])
xticklabels(["Q" "SARSA" "LSPI" "DQN" "PG" "AC" "PPO" "SAC"])
legend( ...

```

```
["Simulation Reward", ...
"Training Episodes", ...
"Training Steps", ...
"Training Time"], ...
"Location", "northwest")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, all agents except PG, and LSPI successfully complete the task, achieving a total reward of at least -4 in simulation. The LSPI agent finds a policy that achieves a reward of -5 in very few steps, but it is unable to improve upon it afterward. The SARSA agent finds the optimal policy using slightly less training time. The training for the DQN agent takes longer, despite needing comparatively less episodes and training steps. The AC algorithm is faster but needs a lot of training episodes to converge, and it does not find the optimal policy. The SAC agent converges using less training episodes and steps, though it too does not find the optimal policy. Overall, for this problem, the SARSA agent performs better.

Finally, note that with a different random seed, the initial parameters of some agents would be different, and therefore, results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlTrainingOptions` | `rlSimulationOptions` | `rlQValueFunction` | `rlValueFunction` | `rlVectorQValueFunction` | `rlDiscreteCategoricalActor`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16
- “Compare Agents on Basic Grid World” on page 7-2
- “Compare Agents on Stochastic Waterfall Grid World” on page 7-70

More About

- “Reinforcement Learning Environments” on page 2-2
- “Use Predefined Grid World Environments” on page 2-9
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Stochastic Waterfall Grid World

This example shows how to create and train frequently used discrete action space default agents on an 8-by-7 gridworld environment. The environment has a starting location, eight terminal locations, and a stochastic waterfall that pushes the agent toward the bottom of the grid. The goal of the agent is to move from the starting location to the target terminal location while maximizing the total reward.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

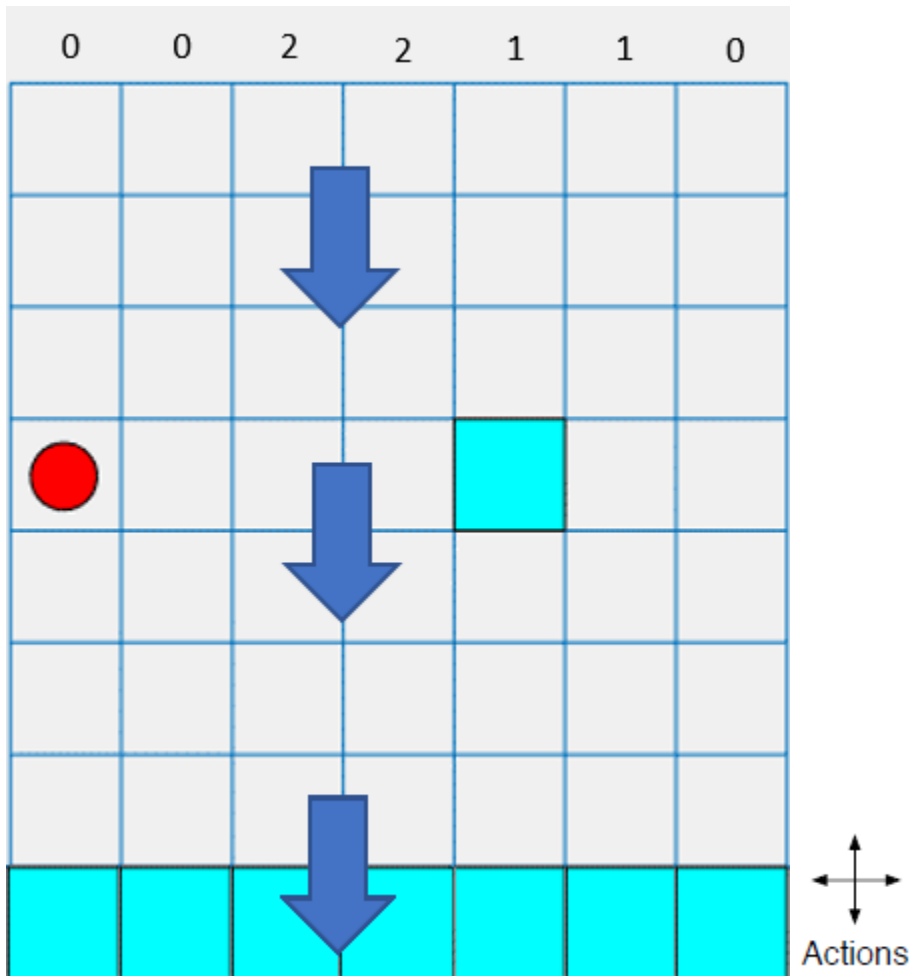
```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Stochastic Waterfall Grid World Environment

This grid world environment used in this example has the following configuration and rules:

- 1 The grid world is 8-by-7 and bounded by borders, with four possible actions (North = 1, South = 2, East = 3, West = 4).
- 2 The agent begins from cell [4,1] (fifth row, first column).
- 3 The agent receives a reward of +50 if it reaches the terminal state at cell [4,5] (light blue).
- 4 All the locations in the bottom row of the grid are terminal, and agent receives a -10 penalty for reaching them.
- 5 All other actions result in -1 reward.
- 6 The waterfall pushes the agent toward the bottom of the grid with a stochastic intensity. Specifically, the baseline intensity varies between the columns, as shown in the figure, and the agent has an equal chance of experiencing the indicated intensity, one level above that intensity, or one level below that intensity. As a result, when the agent moves into a column with a nonzero intensity, the waterfall pushes it downward a number of squares according to the intensity experienced by the agent. For example, if the agent moves east from state [5,2], it has an equal chance of reaching state [6,3], [7,3], or [8,3].



With respect to the deterministic waterfall comparison example, the main differences are the trap terminal states on the bottom, which prevent the agent from using the bottom pathway to advance toward the goal, and the stochastic nature of the waterfall, which on the other hand opens up the possibility of going up first and then, with luck, use the waterfall to better advance toward the target. Note that even if the agent executes an optimal policy, it sometimes cannot arrive at the target cell because the waterfall might be too strong to overcome.

For more information on this environment, see “Use Predefined Grid World Environments” on page 2-9.

Create Environment Object

Create a stochastic waterfall grid world.

```
env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
```

To specify that the initial state of the agent is always [4,1], create a reset function that returns the state number for the initial agent state. This function is called at the start of each training episode and simulation. States are numbered starting at position [1,1]. The state number increases as you move down the first column and then down each subsequent column. Therefore, create an anonymous function handle that sets the initial state to 4.

```
env.ResetFcn = @() 4;
```

For this example, enhance the reward for getting to the [4,5] terminal state.

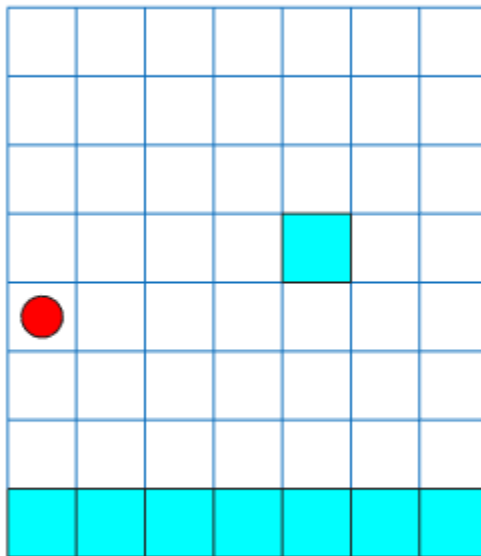
```
env.Model.R(env.Model.R>0)=50;
```

Extract the environment observation and action specification objects for later use when creating agents.

```
obsInfo = getObservationInfo(env);  
actInfo = getActionInfo(env);
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
```



The plot shows the agent location with a red circle and the terminal locations in light blue. Note that the initial agent location is [5,1] because the environment reset function has not yet been called.

Create a Table-Based Q-Value Function Critic

In this section, you create a table-based Q-value function critic for use within Q, SARSA, and DQN agents.

Create a table object.

```
table = rlTable(obsInfo,actInfo);
```

Create a critic approximator object using `rlQValueFunction`.

```
qcritic = rlQValueFunction(table,obsInfo,actInfo);
```

Check the critic with random observation and action inputs.

```

getValue(qcritic, ...
    {randi(numel(obsInfo.Elements))}, ...
    {randi(numel(actInfo.Elements))})

ans =
0

```

Create a Table-Based Value Function Critic

In this section, you create a table-based value function critic for use within AC and PPO agents. You can also use this critic as a baseline for a PG agent.

Create a table object.

```
table = rlTable(obsInfo);
```

Create a critic approximator object using `rlValueFunction`.

```
vcritic = rlValueFunction(table,obsInfo);
```

Check the critic with random observation input.

```

getValue(vcritic,{randi(numel(obsInfo.Elements))})

ans =
0

```

Create a Custom Basis Function-Based Vector Q-Value Function Critic

In this section, you create a vector value function critic for use within the SAC agent. The critic uses a custom basis function that implements a table.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Because `rlVectorQValueFunction` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```

type("cbftbl.m")

function out = cbftbl(obs,nrows)

% Table with nrows rows and one column implemented using a custom basis
% function. The third dimension of obs is the batch dimension.

% Get batch dimension and allocate output matrix.
nbtch = size(obs,3);

```

```
out = zeros(nrows,nbtc,like=obs);

% Cycle through batch dimension and set to one the output
% corresponding to the index received as observation.
for k = 1:nbtc
    if obs(1,1,k)
        out(obs(1,1,k),k) = 1;
    end
end

end
```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbftbl` function (the value of the second argument is stored in the `basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $8 \times 7 = 56$ elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result is a vector in which each of the four elements is the value of the corresponding action given the observation.

```
W0 = rand(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlVectorQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```
vqcritic = rlVectorQValueFunction({basisFcn,W0},obsInfo,actInfo);
```

Check the critic with a batch of 10 random observation inputs.

```
r = randi(numel(obsInfo.Elements),[1 1 10]);
v = getValue(vqcritic,{r});
```

Display the seventh element of the batch.

```
v(:,7)

ans = 4×1

    0.8003
    0.8407
    0.0844
    0.7447
```

Create a Custom Basis Function-Based Q-Value Function Critic

In this section, you create a value function critic for use within the LSPI agent. The critic uses a custom basis function that implements a table.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Because the LSPI agent does not support tables as approximator models, use the helper function defined in the supporting file `cbfqtbl.m`, which implements a q-value function table using custom

basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Display the helper function.

```
type("cbfqtbl.m")

function out = cbfqtbl(obs,act,nr,nc)

% Table with nr rows and nc columns implemented using a custom basis
% function. The third dimension of obs and act is the batch dimension.

% Get batch dimension.
nbtc = size(obs,3);

% Allocate output matrix in columnwise format, that is the first two
% dimensions are vectorized as in out=table(:)
out = zeros(nr*nc,nbtc,like=obs);

% Cycle through batch dimension and set to one the output
% corresponding to the indexes of observation and action.
for k = 1:nbtc
    if obs(1,1,k)
        out(nr*(act(1,1,k)-1)+obs(1,1,k),k) = 1;
    end
end

end
```

Define the custom basis function. Specifically, the anonymous function `basisFcn` accepts as input the current observation, calls the `cbfqtbl` function (the value of the last two arguments is stored in the `basisFcn` workspace at definition time) and returns, for each batch element, a one-hot vector of $8 \times 7 \times 4 = 224$ elements, where the element equal to 1 indicates the columnwise location of the current observation and action.

```
basisFcnQ = @(obs,act) cbfqtbl(obs,act, ...
    numel(obsInfo.Elements), ...
    numel(actInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of W_0 is multiplied by the output of `basisFcn` the result is a scalar indicating the value of the corresponding action given the observation.

```
W0 = rand(numel(obsInfo.Elements)*numel(actInfo.Elements),1);
```

For example, the weight corresponding to observation number 13 and action number 3 is:

```
W0(numel(obsInfo.Elements)*(3-1)+13)
```

```
ans =
0.8173
```

Use `rlQValueFunction` to create the critic object, passing the basis function handle and the initial condition as a cell.

```
qcbfcritic = rlQValueFunction({basisFcnQ,W0},obsInfo,actInfo);
```

Ignoring the batch dimension, check the critic by returning the value corresponding to the observation number 13 and the action number 3.

```
getValue(qcbfcritic,{13},{3})
```

```
ans =  
0.8173
```

Check the critic with a batch of 10 random observation and action inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);  
ract = randi(numel(actInfo.Elements),[1 1 10]);  
v = getValue(qcbfcritic,{robs},{ract});
```

Display the seventh element of the batch.

```
v(:,7)  
  
ans =  
0.3786
```

Create a Custom Basis Function-Based Actor

In this section, you create a discrete categorical actor for use within PG, AC, and PPO agents. The actor uses a custom basis function that implements a table.

Because `rlDiscreteCategoricalActor` does not support tables, use the helper function defined in the supporting file `cbftbl.m`, which implements a table using custom basis functions. Note that using local functions to implement a custom basis function is not recommended if you want to save an agent and load it later. This is because local functions are available only in the file in which they are defined, and when you load an agent in the workspace the function is no longer available to the agent. Additionally, local functions are not supported for code generation.

Alternatively, you could also define a custom layer and then use it in a custom network that effectively implements a table. For an example on how to do that, see “Use Custom Layer in TRPO Agent to Solve Tabular Approximation Problem”.

Define the custom basis function. Here, the output of the basis function is a one-hot vector of 8*7 elements, where the element equal to 1 indicates the current observation.

```
basisFcn = @(obs) cbftbl(obs,numel(obsInfo.Elements));
```

Specify initial conditions for the table values. When the transpose of `W0` is multiplied by the output of `basisFcn` the result, after passing through a `softmax` function, is a vector in which each of the four elements is the probability of executing the corresponding action.

```
W0 = zeros(numel(obsInfo.Elements),numel(actInfo.Elements));
```

Use `rlDiscreteCategoricalActor` to create the actor object, passing the basis function handle and the initial condition as a cell.

```
actor = rlDiscreteCategoricalActor({basisFcn,W0},obsInfo,actInfo);
```

Check the actor with a batch of 10 random observation inputs.

```
robs = randi(numel(obsInfo.Elements),[1 1 10]);
v = getAction(actor,{robs});
```

Display the seventh element of the batch.

```
v{1}(:, :, 7)
```

```
ans =
1
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 100 training episodes. For more information, see `rlEvaluator`.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object.

```
trainOpts = rlTrainingOptions;
```

For this example, use the following options:

- Train for a maximum of 5000 episodes. Specify that each episode lasts for most 50 time steps.
- Stop the training when the average reward in the evaluation episodes is greater than 1.

```
trainOpts.MaxEpisodes= 5000;
trainOpts.MaxStepsPerEpisode = 50;
trainOpts.StopTrainingCriteria = "EvaluationStatistic";
trainOpts.StopTrainingValue = 1;
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 50 steps.

```
simOpts = rlSimulationOptions(MaxSteps=50);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a Q-Agent

In this section, you create a Q-learning agent and then configure its parameters.

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlQAgent` object using the critic you have created before.

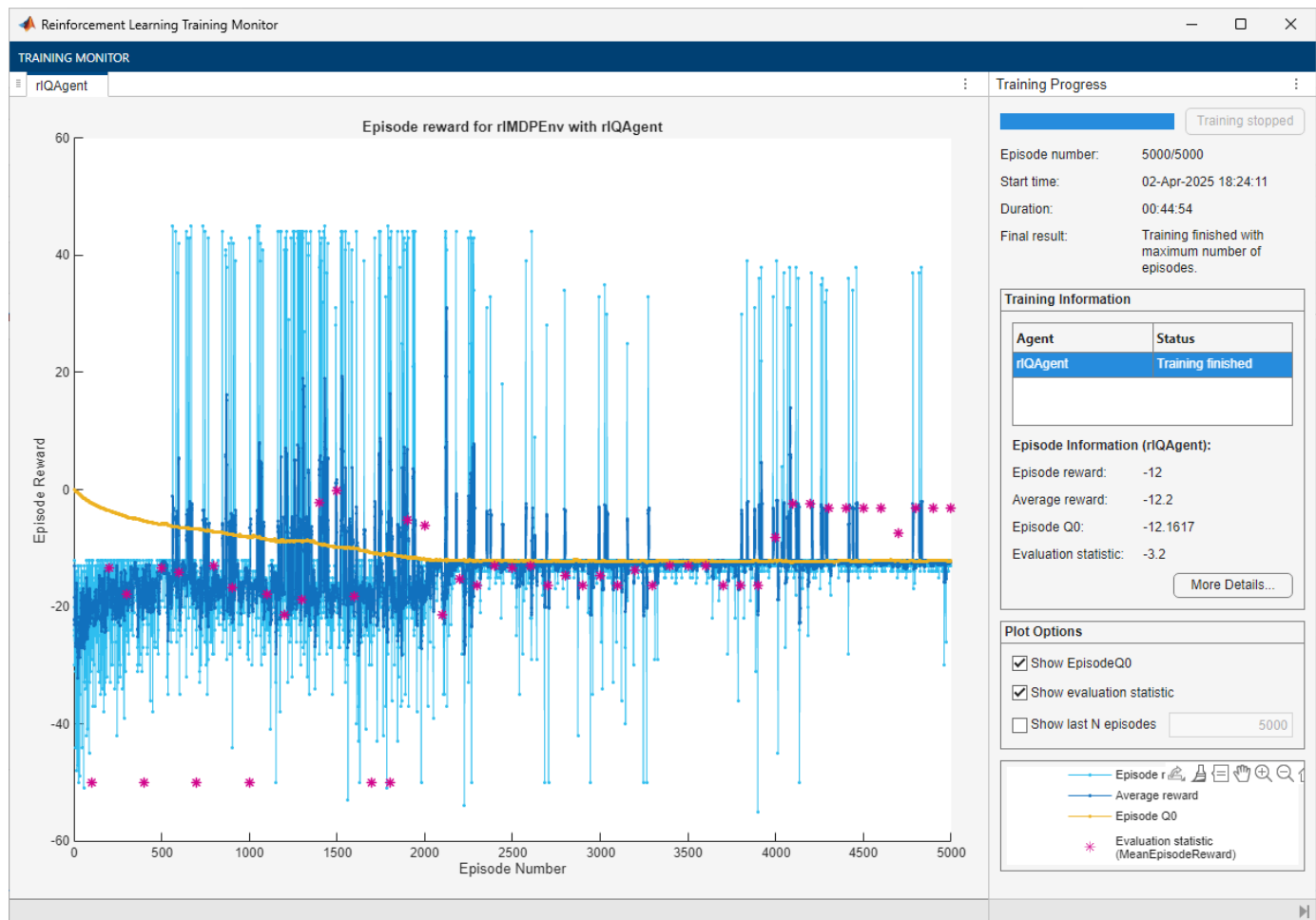
```
qAgent = rlQAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
qAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
qAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    qTngRes = train(qAgent,env,trainOpts,Evaluator=evl);
    qTngTime = toc;
    % Extract the number of training episodes and total steps.
    qTngEps = qTngRes.EpisodeIndex(end);
    qTngSteps = sum(qTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchQAgent.mat", ...
    %     "qAgent","qTngEps","qTngSteps","qTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchQAgent.mat", ...
        "qAgent","qTngEps","qTngSteps","qTngTime")
end
```

The training does not stop within 5000 episodes. The evaluation statistic indicates a reward of -3.2, suggesting that the agent does not enact the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

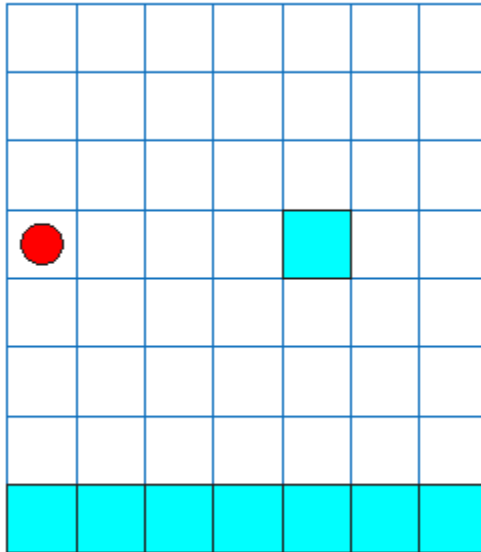
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(qAgent, env, simOpts);
```



```
qTotalRwd = sum(experience.Reward)
```

```
qTotalRwd =  
-50
```

The agent trace shows that the trained agent does not find the target cell.

Extract the table with the final Q-value function.

```
qAgtFinalQ = getLearnableParameters(getCritic(qAgent));
```

Display the maximum values for each state (that is, the approximate value function) in a 5-by-5 format.

```
reshape(max(qAgtFinalQ{1}'),8,7)
```

```
ans = 8x7 single matrix
```

-8.4910	-7.6386	0	0	0	0	-0.3060
-9.7053	-8.2515	-3.7453	0	0	0	-0.3177
-10.3059	-9.7609	-6.6872	-1.4023	-0.6185	-0.3177	-0.9506
-11.3702	-10.3471	-8.4235	-3.6939	0	-0.6037	-2.0492
-11.7910	-11.0499	-8.9738	-5.4382	-4.3062	-1.6570	-2.8689
-10.9000	-10.5582	-10.2957	-7.4134	-5.6825	-4.2986	-3.6178
-10.0000	-10.0393	-10.2703	-9.2107	-8.4403	-5.5935	-4.6992
0	0	0	0	0	0	0

Create, Train, and Simulate a SARSA Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlSARSAAgent` object using the critic you have created before.

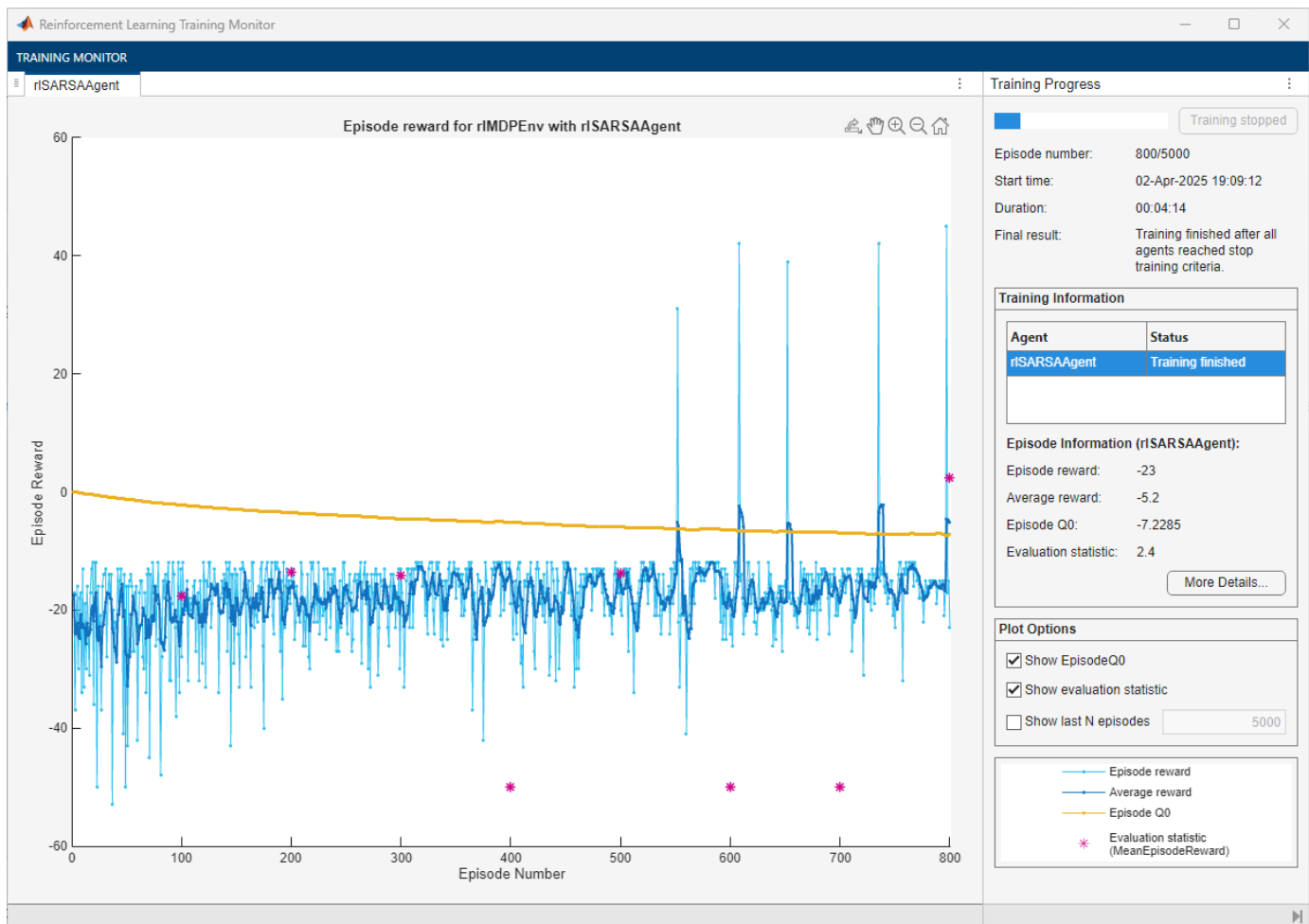
```
sarsaAgent = rlSARSAAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
sarsaAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
sarsaAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    sarsaTngRes = train(sarsaAgent,env,trainOpts,Evaluator=evl);
    sarsaTngTime = toc;
    % Extract the number of training episodes and total steps.
    sarsaTngEps = sarsaTngRes.EpisodeIndex(end);
    sarsaTngSteps = sum(sarsaTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchSARSAAgent.mat", ...
    %     "sarsaAgent","sarsaTngEps","sarsaTngSteps","sarsaTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchSARSAAgent.mat", ...
        "sarsaAgent","sarsaTngEps","sarsaTngSteps","sarsaTngTime")
end
```



The training stops after 800 episodes. The evaluation statistic indicates a reward of 2.4, suggesting that the agent learns a policy that can be close to the optimal one.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

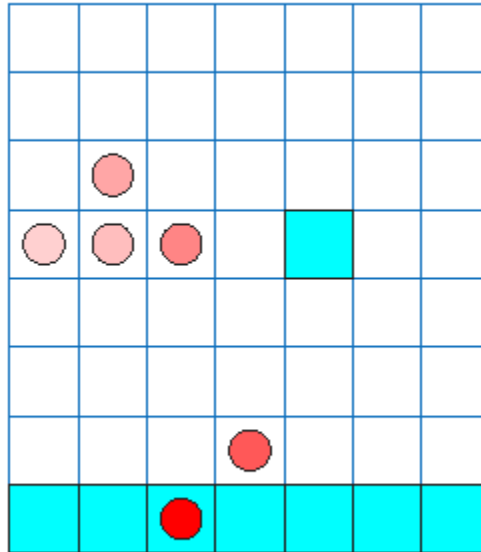
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sarsaAgent, env, simOpts);
```



```
sarsaTotalRwd = sum(experience.Reward)
```

```
sarsaTotalRwd =  
-14
```

The agent trace shows that, in this simulation, the trained agent does not find the target cell. Note that even if the agent executes an optimal policy, arriving at the target cell cannot be guaranteed, as sometimes the waterfall is too strong to overcome, as it seems to be the case in this simulation.

Extract the table with the final Q-value function.

```
sarsaAgtFinalQ = getLearnableParameters(getCritic(sarsaAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(sarsaAgtFinalQ{1}'),8,7)
```

```
ans = 8×7 single matrix
```

```
-8.4358  -7.6420      0      0      0      0  -0.3177  
-9.2846  -8.3873  -3.5061      0  0.3176      0  -0.9372  
-10.2814 -9.3075  -6.3571  -1.2661  0.0602  -0.6334  -1.7699  
-10.8428 -10.3274 -8.2447  -2.5470      0  -0.9809  -2.6761  
-11.7603 -10.9135 -8.4797  -5.9128 -3.0310 -3.0969  -3.5276  
-11.0152 -10.8841 -10.4255  -7.1040 -5.5859 -4.3203  -4.7720  
-9.9046  -9.8601 -10.3371 -10.2489 -9.6857 -6.7507  -5.9167  
      0      0      0      0      0      0      0
```

Create, Train, and Simulate an LSPI Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

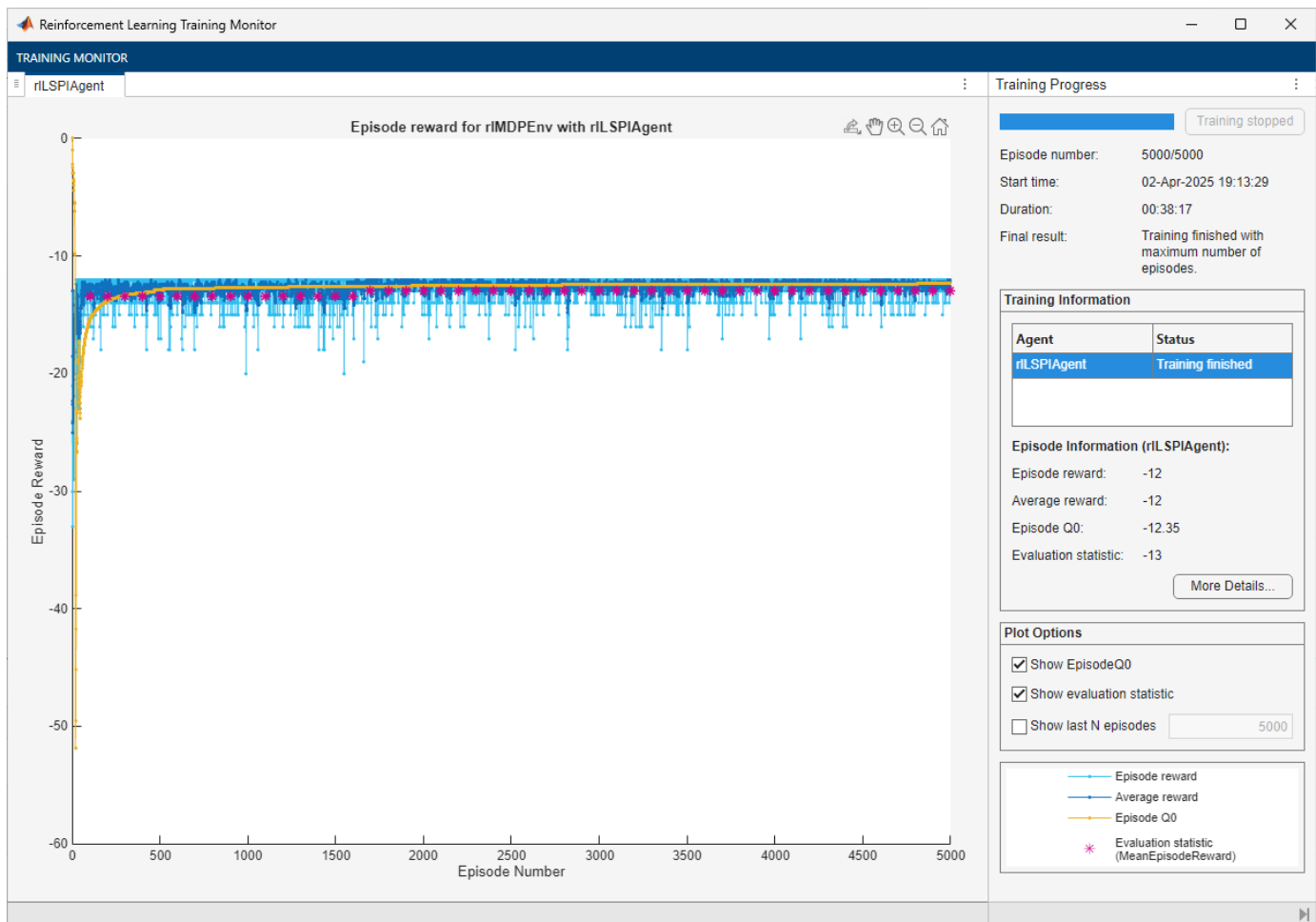
```
rng(0, "twister")
```

Create an `rLLSPIAgent` object using the critic you have created before.

```
lspiAgent = rLLSPIAgent(qcbfcritic);
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    lspiTngRes = train(lspiAgent,env,trainOpts,Evaluator=evl);
    lspiTngTime = toc;
    % Extract the number of training episodes and total steps.
    lspiTngEps = lspiTngRes.EpisodeIndex(end);
    lspiTngSteps = sum(lspiTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchLSPIAgent.mat", ...
    %     "lspiAgent","lspiTngEps","lspiTngSteps","lspiTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchLSPIAgent.mat", ...
        "lspiAgent","lspiTngEps","lspiTngSteps","lspiTngTime")
end
```



The training does not converge after 5000 episodes. The evaluation statistic indicates a reward of -13, suggesting that the agent does not learn the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

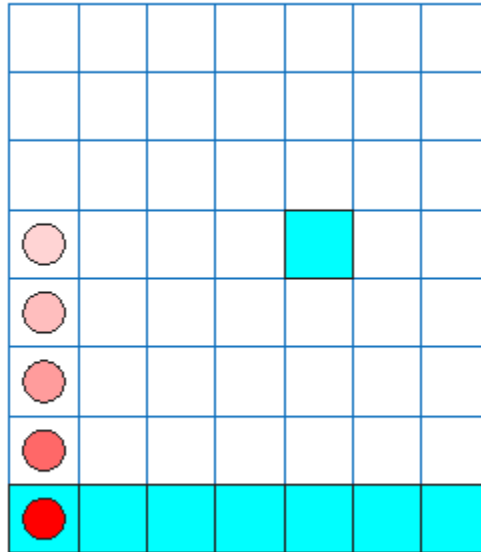
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(lspiAgent, env, simOpts);
```



```
lspiTotalRwd = sum(experience.Reward)
```

```
lspiTotalRwd =  
-13
```

The agent trace shows that the trained agent does not find the target cell.

Extract the table with the final Q-value function.

```
lspiAgtFinalQ = getLearnableParameters(getCritic(lspiAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(reshape(lspiAgtFinalQ{1},8*7,4)'),8,7)
```

```
ans = 8×7 single matrix
```

0	0	0	0	0	0	0
0.0000	0	0	0	0	0	0
0	-10.8792	0	0	0	0	0
-12.9099	-14.2565	0	0	0	0	0
-11.7942	-12.7368	-11.1774	0	0	0	0
-10.9021	-10.3578	0	0	0	0	0
-10.0000	-9.9995	-10.3403	-9.9900	0	0	0
0	0	0	0	0	0	0

Create, Train, and Simulate a DQN Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlDQNAgent` object using the critic you have created before.

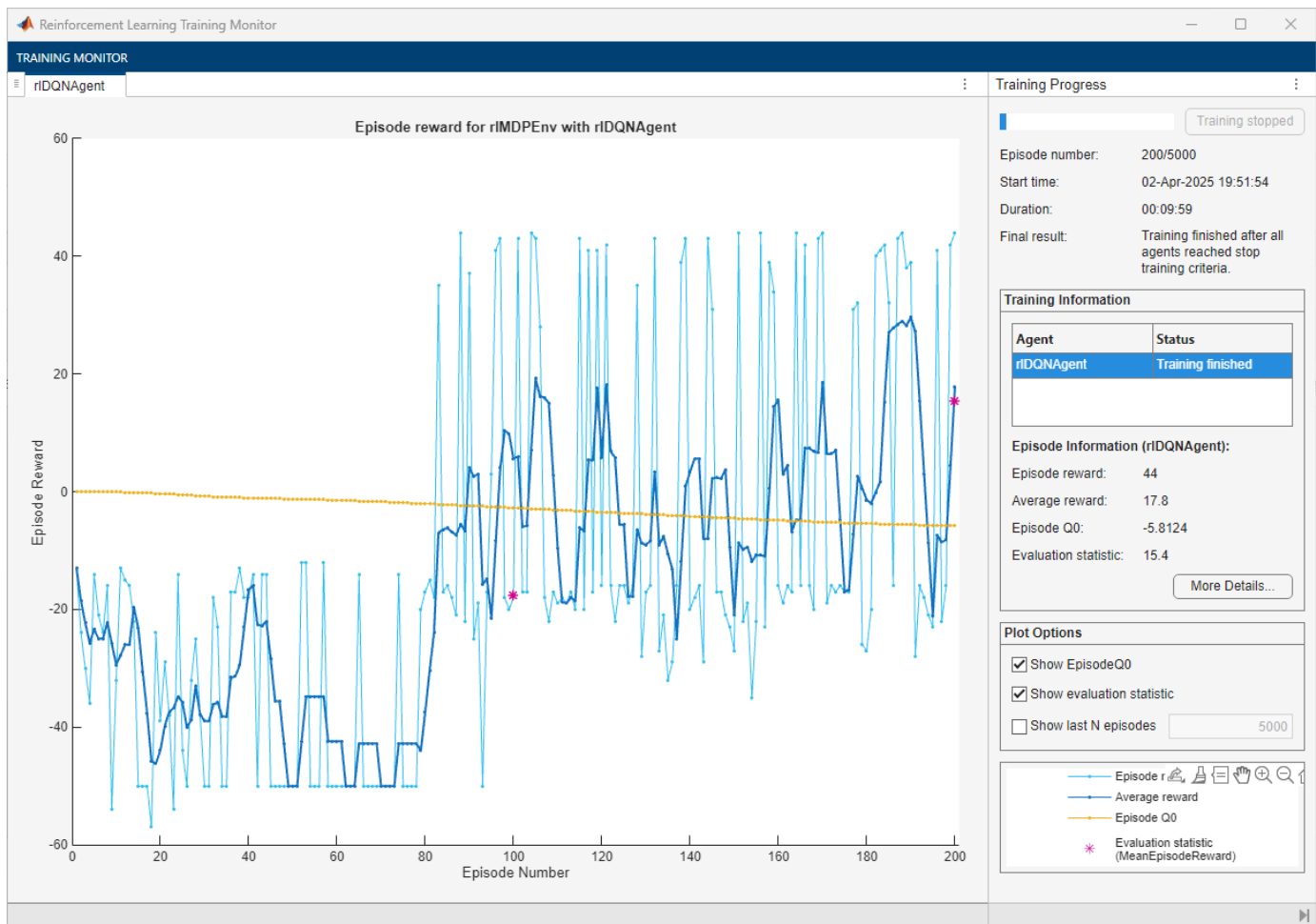

```
dqnAgent = rLDQNAgent(qcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);
    dqnTngTime = toc;
    % Extract the number of training episodes and total steps.
    dqnTngEps = dqnTngRes.EpisodeIndex(end);
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchDQNAgent.mat", ...
    %     "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchDQNAgent.mat", ...
        "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
end
```



The training stops after 200 episodes. The evaluation statistic indicates a reward of 15, suggesting that the agent learns the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

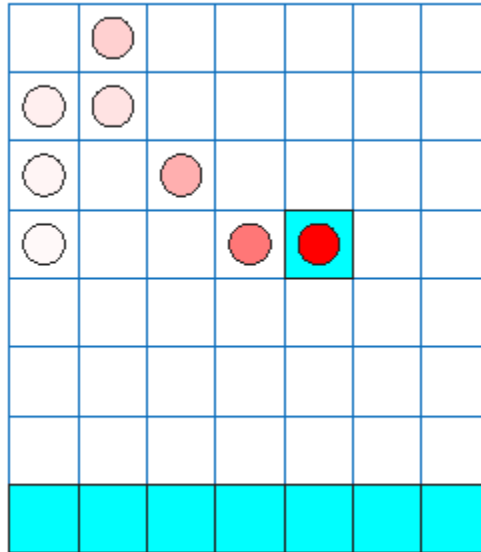
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(dqnAgent, env, simOpts);
```



```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
44
```

The agent trace shows that the trained agent successfully finds the target cell using just 6 moves before the final one. As a consequence, the total reward is $50-6=44$.

Extract the table with the final Q-value function.

```
dqnAgtFinalQ = getLearnableParameters(getCritic(dqnAgent));
```

Display the maximum values for each state (that is, the value function) in a 5-by-5 format.

```
reshape(max(dqnAgtFinalQ{1}'),8,7)
```

```
ans = 8x7 single matrix
```

```
-7.0829    -6.1218         0         0    0.0000    0.0000    0.0000  
-7.9759    -7.1075    -1.3664         0    3.9302    6.4918    3.9297  
-8.8278    -8.0157    -5.2283    4.9758    6.1051    5.1754    2.9144  
-9.6662    -8.9143    -8.6060    0.4956         0    2.4971    1.8557  
-10.5174   -9.7126    -7.9808   -5.3856    0.3213   -0.2988    0.7551  
-10.1551   -9.3803    -8.9581   -8.1845   -5.4431   -1.0009   -0.3174  
-9.3846    -8.6961   -9.2285   -9.1227   -7.5849   -2.4418   -1.4120  
         0         0         0         0         0         0         0
```

Create, Train, and Simulate a PG Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPGAgent` object using the actor you have created before. Alternatively, to use a baseline, supply the critic you have created before `vcritic` as a second argument. However, for this example, do not use any baseline.

```
pgAgent = rlPGAgent(actor);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

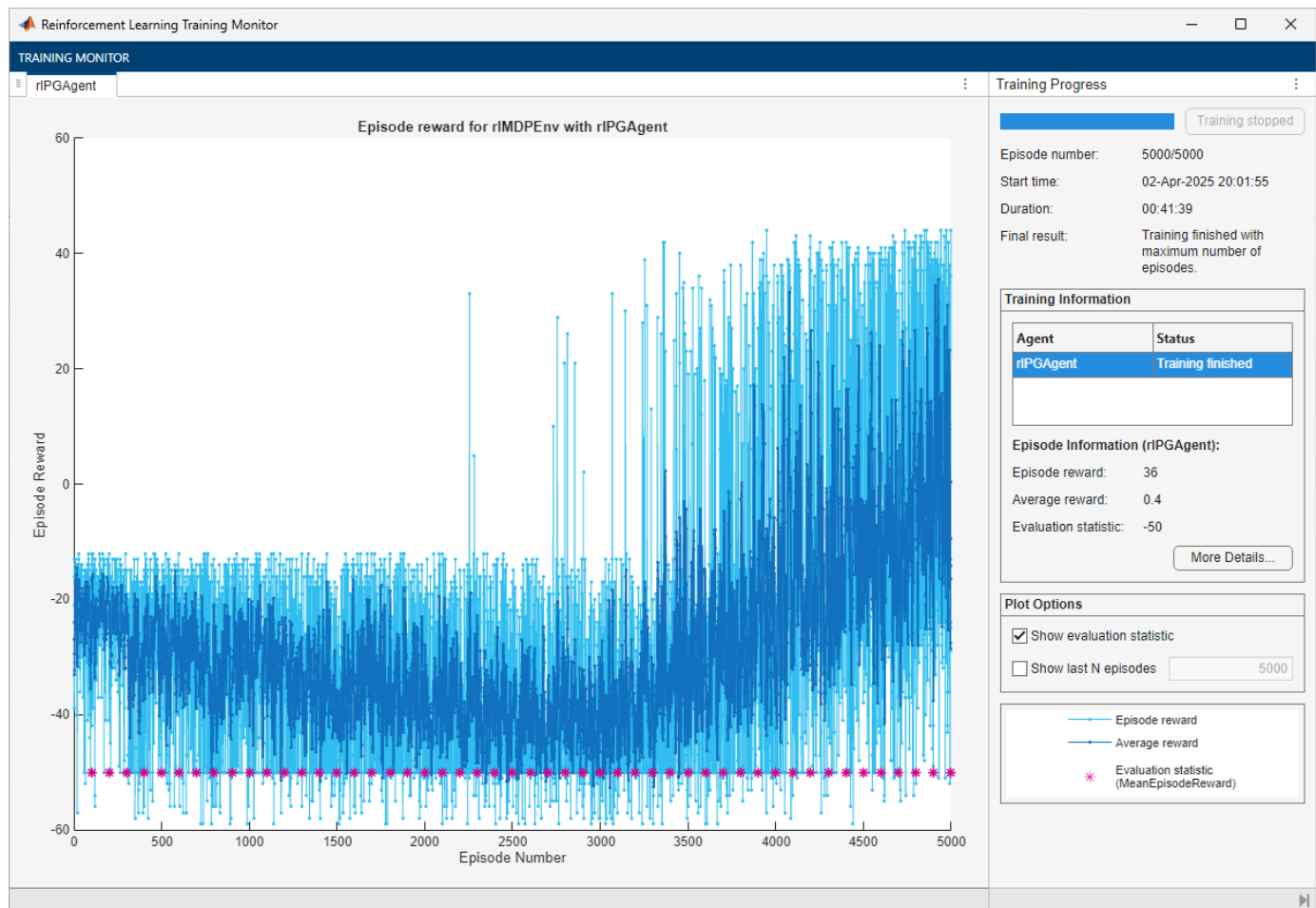
```
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");  
    env.ResetFcn = @() 4;  
    env.Model.R(env.Model.R>0)=50;  
    % Train the agent. Record the training time.  
    tic  
    pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);  
    pgTngTime = toc;  
    % Extract the number of training episodes and total steps.  
    pgTngEps = pgTngRes.EpisodeIndex(end);  
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("swgwBchPGAgent.mat", ...  
    %     "pgAgent","pgTngEps","pgTngSteps","pgTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("swgwBchPGAgent.mat", ...  
        "pgAgent","pgTngEps","pgTngSteps","pgTngTime")  
end
```



The training does not stop within 5000 episodes. The evaluation statistic indicates a reward of -50, suggesting that the agent does not learn any acceptable policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

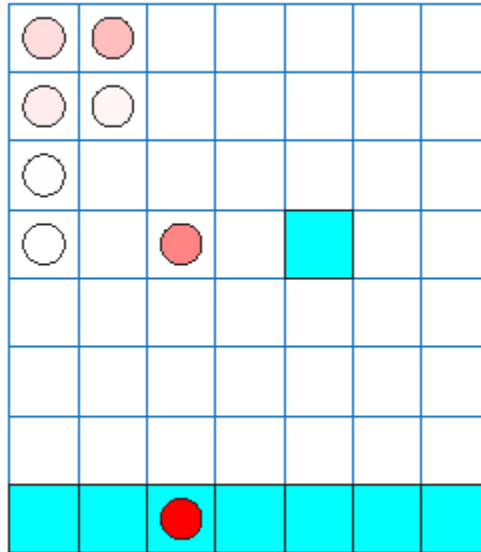
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(pgAgent, env, simOpts);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-35
```

The agent trace shows that the trained agent does not even move toward the target cell.

Create, Train, and Simulate an AC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlACAgent` object using the actor and critic you have created before.

```
acAgent = rlACAgent(actor,vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

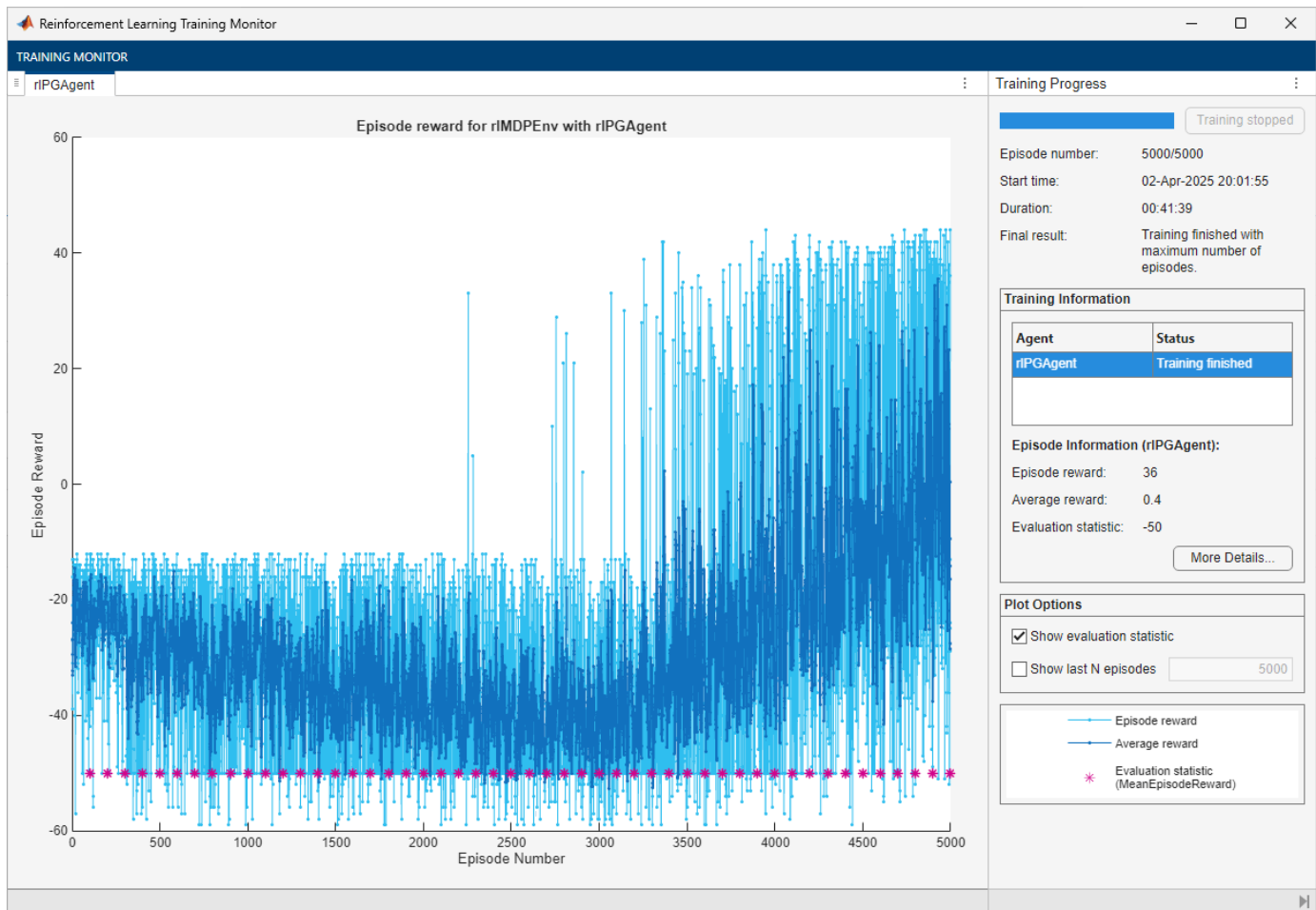
```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract the number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchACAgent.mat", ...
    %      "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end

```



The training does not stop within 5000 episodes. The evaluation statistic indicates a reward of -50, suggesting that the agent does not learn any acceptable policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

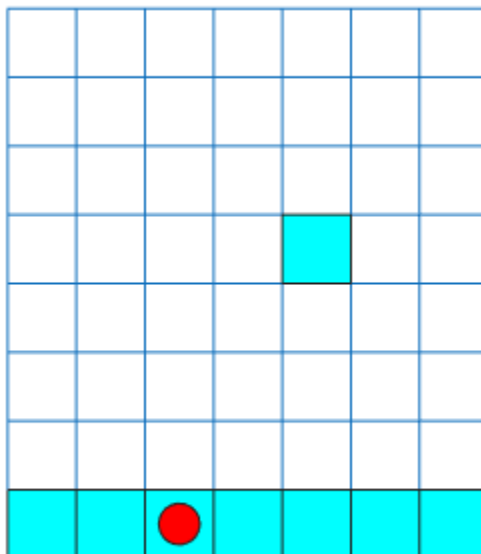
Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(acAgent, env, simOpts);
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =
-15
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
```



```
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```


The agent trace shows that the trained agent does not learn to avoid the terminal locations in the bottom row.

Because the AC agent has a critic, you can display the final value function represented by the critic.

```
acAgtFinalV = getLearnableParameters(getCritic(acAgent));
reshape(acAgtFinalV{1},8,7)
```

ans = 8×7 single matrix

-8.0807	-8.0911	0	0	0	0	0
-8.9049	-9.2618	-4.5136	0	0	0	0
-11.2843	-10.7371	-5.1383	-1.6528	-0.2732	-0.1013	0
-12.9438	-11.6077	-7.0862	-1.3626	0	-0.1750	0
-11.5213	-10.6213	-7.5784	-3.1510	-1.5609	-0.0005	-0.0006
-11.3652	-10.6607	-10.3237	-3.7910	-2.7339	-0.1979	-0.2384
-8.6697	-10.1707	-10.2305	-10.2505	-7.4023	-1.8267	-1.0707
0	0	0	0	0	0	0

Create, Train, and Simulate a PPO Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlPPOAgent` object using the actor and critic you have created before.

```
ppoAgent = rlPPOAgent(actor,vcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

```
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-2;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

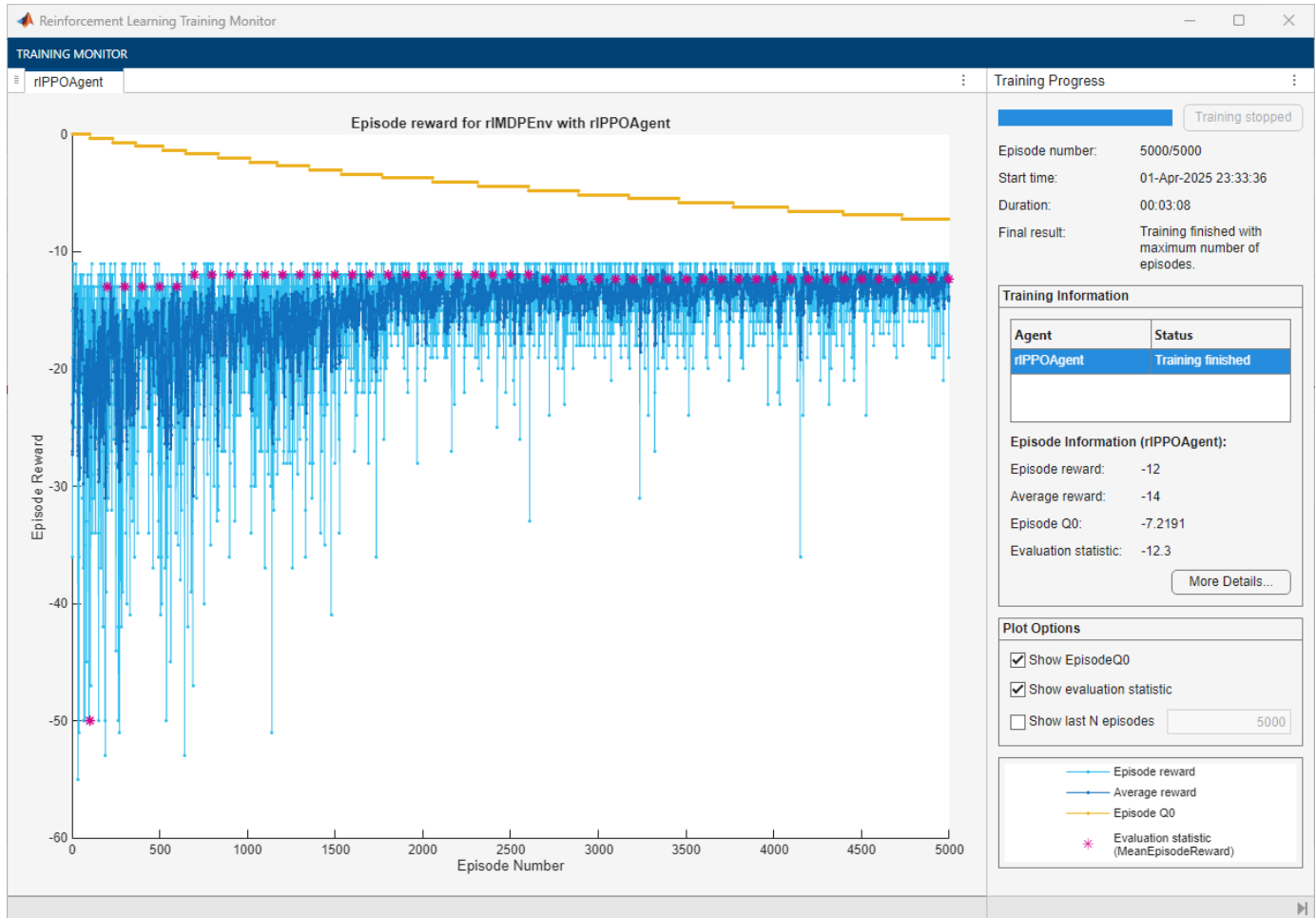
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
    ppoTngTime = toc;
    % Extract the number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
```

```

% save("swgwBchPPOAgent.mat", ...
%     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
else
% Load the pretrained agent and results for the example.
load("swgwBchPPOAgent.mat", ...
     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
end

```



The training does not stop within 5000 episodes. The evaluation statistic indicates a reward of -12.3, suggesting that the agent does not learn the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment and configure the visualization to maintain a trace of the agent states.

```

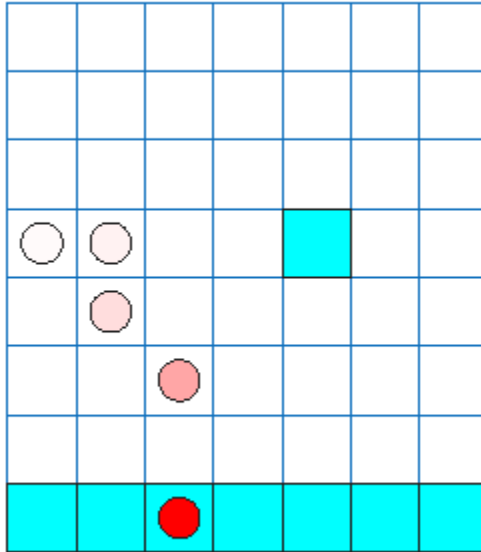
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;

```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(ppoAgent,env,simOpts);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-13
```

The agent trace shows that the trained agent does not learn to avoid the terminal states in the bottom row,

Because the PPO agent has a critic, you can display the final value function represented by the critic.

```
ppoAgtFinalV = getLearnableParameters(getCritic(ppoAgent));  
reshape(ppoAgtFinalV{1},8,7)
```

```
ans = 8×7 single matrix
```

```
-2.5166  -2.3795      0      0      0      0      0  
-2.8635  -2.8792  -1.0599      0      0      0      0  
-3.8991  -3.8537  -2.3695  -0.3886      0      0      0  
-4.9219  -5.7343  -3.0409  -0.3288      0      0      0  
-7.2191  -7.2691  -5.6524  -1.2116  -0.4680      0      0  
-6.8135  -6.6869  -6.7841  -2.9645  -1.6540  -0.4749  -0.1796  
-6.7691  -6.0512  -7.5760  -6.1112  -3.7429  -1.2074  -0.7553  
      0      0      0      0      0      0      0
```

Create, Train, and Simulate a SAC Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create an `rlSACAgent` object using the actor you have created before and critic.

```
sacAgent = rlSACAgent(actor, vqcritic);
```

For tabular problems, you can typically safely set a slightly higher learning rate. However, set a gradient threshold to minimize the risk of learning instabilities.

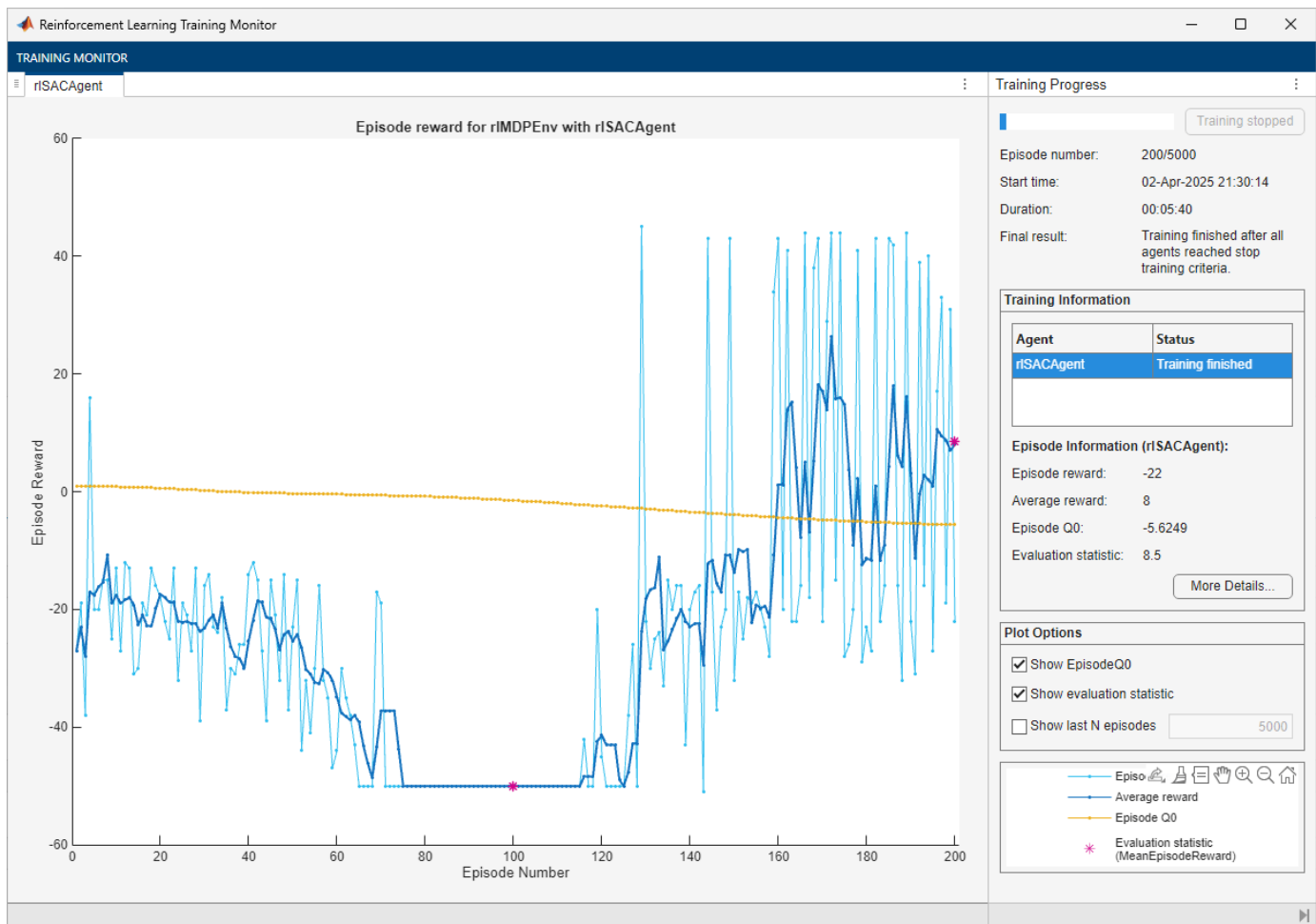
```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-2;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-2;
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Train the agent, passing the previously created evaluator object to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("WaterFallGridWorld-Stochastic");
    env.ResetFcn = @() 4;
    env.Model.R(env.Model.R>0)=50;
    % Train the agent. Record the training time.
    tic
    sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
    sacTngTime = toc;
    % Extract the number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("swgwBchSACAgent.mat", ...
    %      "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("swgwBchSACAgent.mat", ...
        "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
end
```



The training stops after 200 episodes. The evaluation statistic indicates a reward of 8.5, suggesting that the agent learns the optimal policy.

You can check the trained agent within the environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

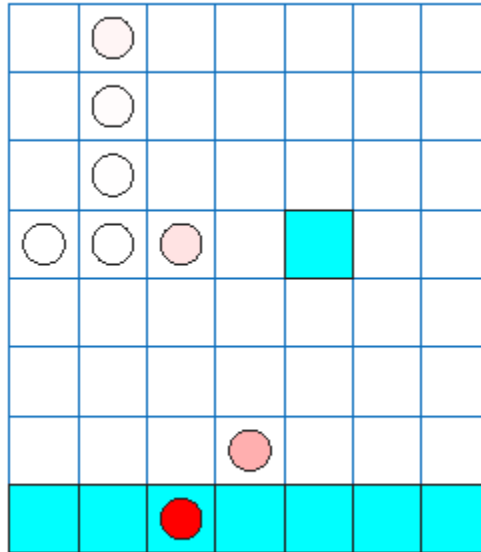
Visualize the environment and configure the visualization to maintain a trace of the agent states.

```
plot(env)
env.Model.Viewer.ShowTrace = true;
env.Model.Viewer.clearTrace;
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 50 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(sacAgent, env, simOpts);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-16
```

The agent trace shows that the trained agent successfully finds the target cell using just 6 moves before the final one. As a consequence, the total reward is $50 - 6 = 44$.

Because the SAC agent has a critic, you can display the final value function represented by critic.

```
sacAgtFinalV = getLearnableParameters(getCritic(sacAgent));  
reshape(max(sacAgtFinalV{1}),8,7)
```

```
ans = 8x7 single matrix
```

-5.9484	-4.9739	0.0000	0.0000	0.0000	2.3252	1.3694
-6.8278	-5.9814	0.1031	0.0000	5.1474	6.4981	3.7063
-7.5995	-6.9054	-4.4824	5.6607	6.6548	6.2080	2.5382
-8.4234	-7.7959	-8.1279	1.8500	0.0000	2.2902	1.3569
-9.2436	-8.6889	-7.8357	-4.4469	1.3079	-0.1922	0.2753
-9.4661	-8.8603	-8.7405	-7.9632	-4.6766	-1.6075	-0.7324
-8.8719	-8.3455	-9.3886	-9.4166	-7.9859	-2.7729	-1.7554
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the training time.

```
simReward = [  
    qTotalRwd  
    sarsaTotalRwd  
    lspiTTotalRwd
```

```

    dqnTotalRwd
    pgTotalRwd
    acTotalRwd
    ppoTotalRwd
    sacTotalRwd
];

tngEpisodes = [
    qTngEps
    sarsaTngEps
    lspiTngEps
    dqnTngEps
    pgTngEps
    acTngEps
    ppoTngEps
    sacTngEps
];

tngSteps = [
    qTngSteps
    sarsaTngSteps
    lspiTngSteps
    dqnTngSteps
    pgTngSteps
    acTngSteps
    ppoTngSteps
    sacTngSteps
];

tngTime = [
    qTngTime
    sarsaTngTime
    lspiTngTime
    dqnTngTime
    pgTngTime
    acTngTime
    ppoTngTime
    sacTngTime
];

```

Because the training for the PG and LSPI agents does not converge, to avoid visualizing their metrics, set them to NaN.

```

simReward([1 3 5 6 7]) = NaN;
tngEpisodes([1 3 5 6 7]) = NaN;
tngSteps([1 3 5 6 7]) = NaN;
tngTime([1 3 5 6 7]) = NaN;

```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor `[5e-3 2 1e4 1]` for better visualization.

```

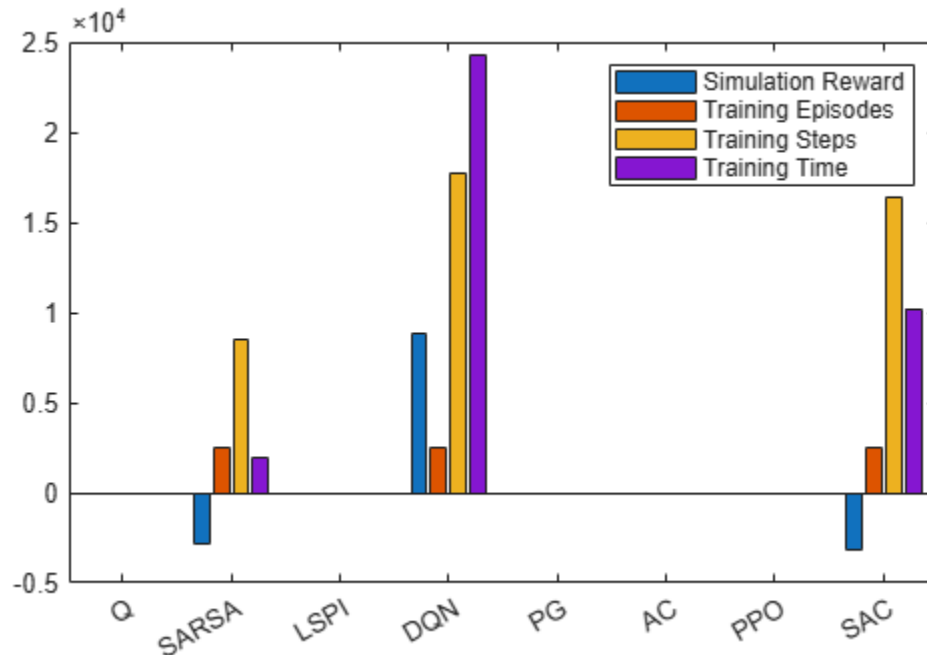
figure;
bar([simReward,tngEpisodes,tngSteps,tngTime]./[5e-3 2 1e4 1])
xticklabels(["Q" "SARSA" "LSPI" "DQN" "PG" "AC" "PP0" "SAC"])
legend( ...
    ["Simulation Reward", ...
    "Training Episodes", ...

```

```

"Training Steps", ...
"Training Time"], ...
"Location", "northeast")

```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, only DQN and SAC successfully complete the task, with SAC requiring slightly less training time. SARSA probably also learns the optimal policy though it is not able to reach the target cell in the final simulation.

Finally, note that with a different random seed, the initial parameters of some agents would be different, and therefore, results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlTrainingOptions` | `rlSimulationOptions` | `rlQValueFunction` | `rlValueFunction` | `rlVectorQValueFunction` | `rlDiscreteCategoricalActor`

Related Examples

- “Train Reinforcement Learning Agent in MDP Environment” on page 1-10
- “Train Reinforcement Learning Agent in Basic Grid World” on page 1-16

- “Compare Agents on Basic Grid World” on page 7-2
- “Compare Agents on Deterministic Waterfall Grid World” on page 7-36

More About

- “Reinforcement Learning Environments” on page 2-2
- “Use Predefined Grid World Environments” on page 2-9
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Discrete Double-Integrator

This example shows how to create and train frequently used agents on a discrete double-integrator environment. The training goal is to control the position of a mass in the second-order system by applying a force input. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

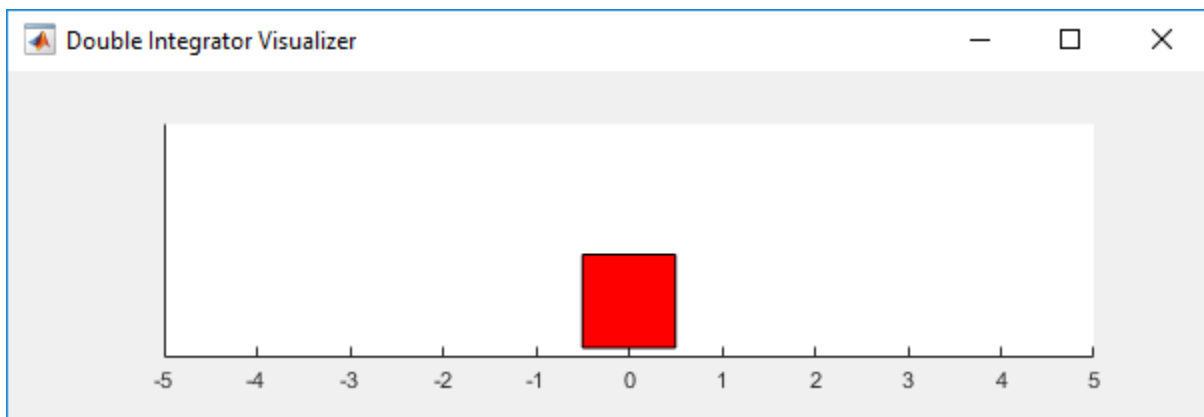
Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Double-Integrator MATLAB Environment

The reinforcement learning environment for this example is a second-order double-integrator system with a gain and a discrete action space. The training goal is to control the position of a mass in the second-order system by applying a force input.



For this environment:

- The mass starts at an initial position of 2 m and zero velocity.
- The agent can apply one of three possible force values to the mass: -2, 0, or 2 N.

- The observations from the environment are the position and velocity of the mass.
- The episode terminates if the mass moves more than 5 m from the original position or if $|x| < 0.01$.
- The reward r_t , provided at every time step, is a discretization of $r(t)$:

$$r(t) = -(x(t)' Q x(t) + u(t)' R u(t))$$

Here:

- x is the state vector of the mass.
- u is the force applied to the mass.
- Q is the weight on the state deviation from zero; $Q = [10 \ 0; 0 \ 1]$.
- R is the weight on the control effort; $R = 0.01$.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the double-integrator.

```
env = rlPredefinedEnv("DoubleIntegrator-Discrete")
```

```
env =  
    DoubleIntegratorDiscreteAction with properties:
```

```
        Gain: 1  
        Ts: 0.1000  
    MaxDistance: 5  
    GoalThreshold: 0.0100  
           Q: [2x2 double]  
           R: 0.0100  
    MaxForce: 2  
    State: [2x1 double]
```

The environment reset function initializes and returns the environment state (position and velocity).

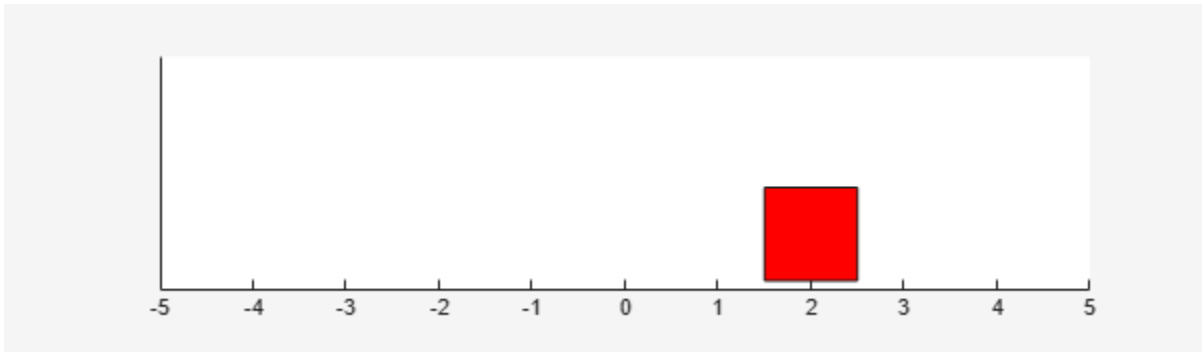
```
reset(env)
```

```
ans = 2x1
```

```
    2  
    0
```

You can visualize the double-integrator system during training or simulation using the `plot` function.

```
plot(env)
```



Obtain the observation and action information for later use when creating agents.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
  rlNumericSpec with properties:  
    LowerLimit: -Inf  
    UpperLimit: Inf  
    Name: "states"  
    Description: "x, dx"  
    Dimension: [2 1]  
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =  
  rlFiniteSetSpec with properties:  
    Elements: [-2 0 2]  
    Name: "force"  
    Description: [0x0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run the training for a maximum of 5000 episodes, with each episode lasting a maximum of 200 time steps.
- Stop the training when the average reward in the evaluation episodes is greater than -40. At this point, the agent can control the position of the mass using minimal control effort.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=5000, ...
```

```
MaxStepsPerEpisode=200, ...
StopTrainingCriteria="EvaluationStatistic", ...
StopTrainingValue=-40);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a DQN Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlDQNAgent` object using the environment specification objects.

```
dqnAgent = rlDQNAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
dqnAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

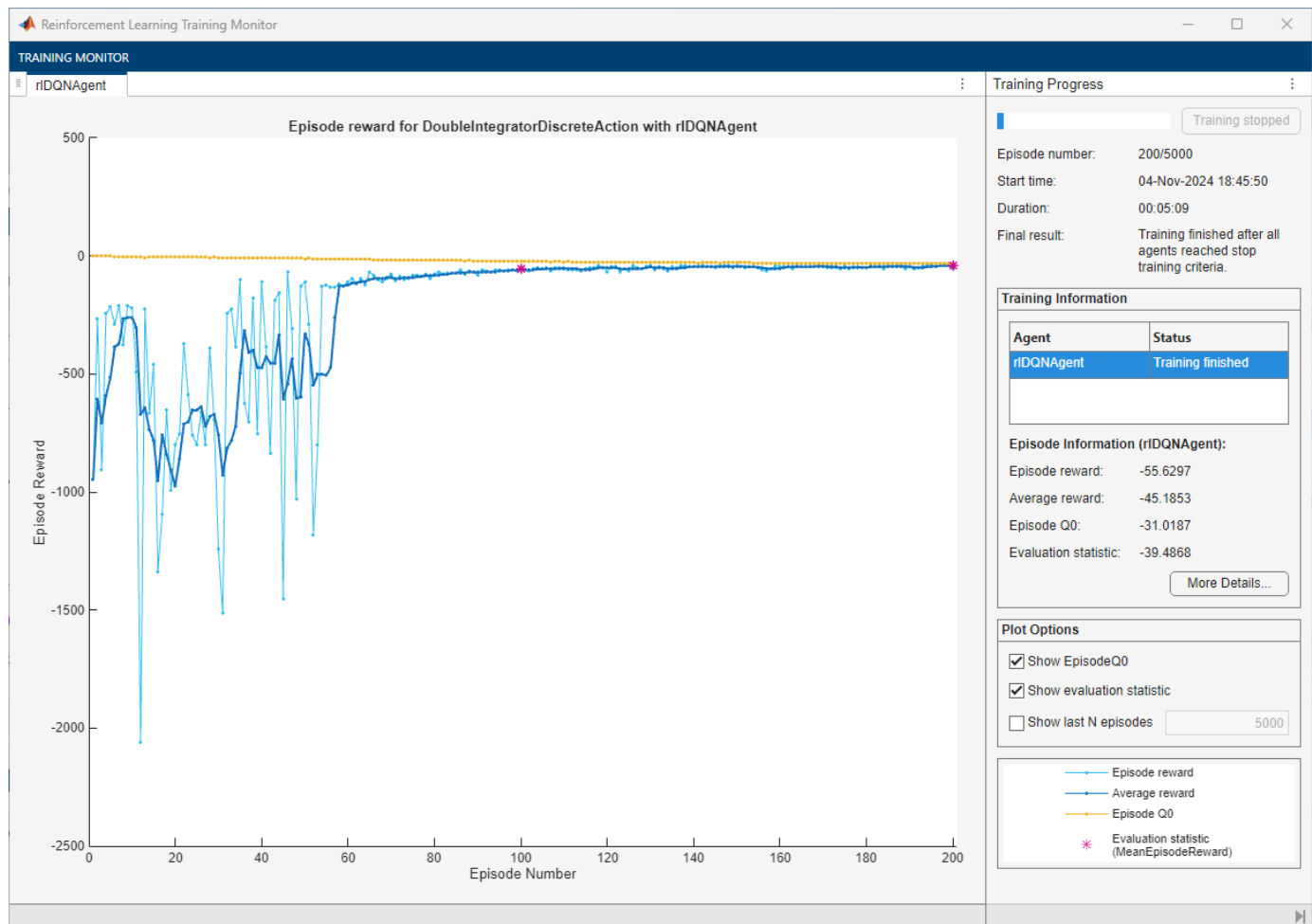
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("DoubleIntegrator-Discrete");
    % Train the agent. Record the training time.
    tic
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);
    dqnTngTime = toc;
    % Extract number of training episodes and total steps.
    dqnTngEps = dqnTngRes.EpisodeIndex(end);
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ddiBchDQNAgent.mat", ...
    %     "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
else
    % Load the pretrained agent and results for the example.
    load("ddiBchDQNAgent.mat", ...
```

```

end
    "dqnAgent", "dqnTngEps", "dqnTngSteps", "dqnTngTime")
end

```



For the DQN Agent, the training converges to a solution after 200 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

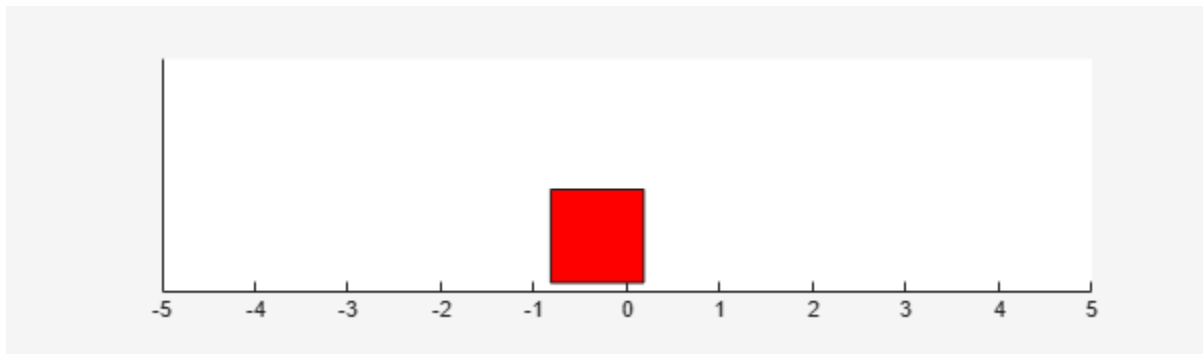
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, dqnAgent, simOptions);
```



```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
-82.8390
```

The trained DQN agent stabilizes the mass at the origin.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

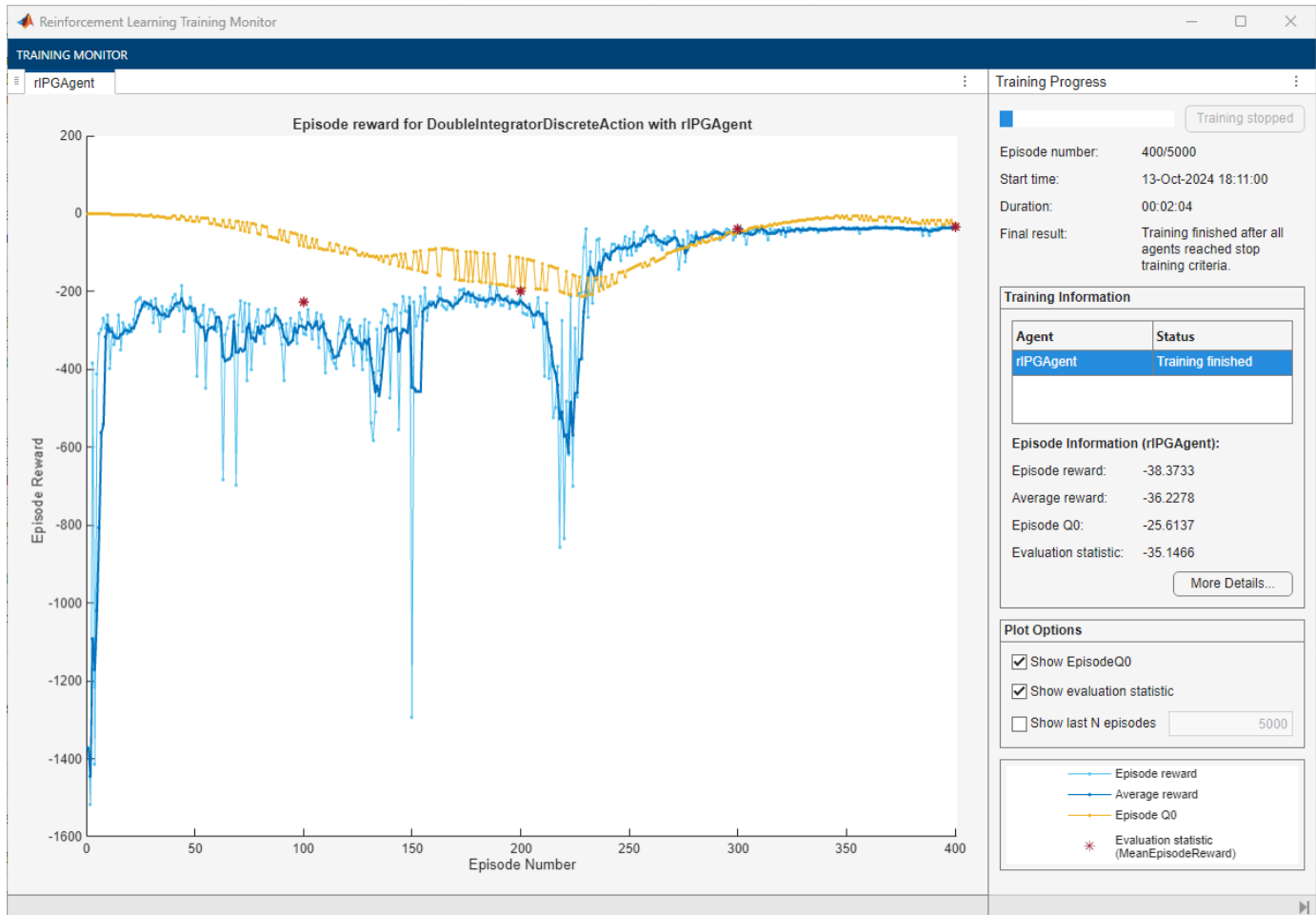
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("DoubleIntegrator-Discrete");  
    % Train the agent. Record the training time.  
    tic  
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);  
    pgTngTime = toc;  
    % Extract number of training episodes and total steps.
```

```

pgTngEps = pgTngRes.EpisodeIndex(end);
pgTngSteps = sum(pgTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ddiBchPGAgent.mat", ...
%      "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
else
% Load the pretrained agent and results for the example.
load("ddiBchPGAgent.mat", ...
     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
end

```



For the PG agent, the training converges to a solution after 400 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

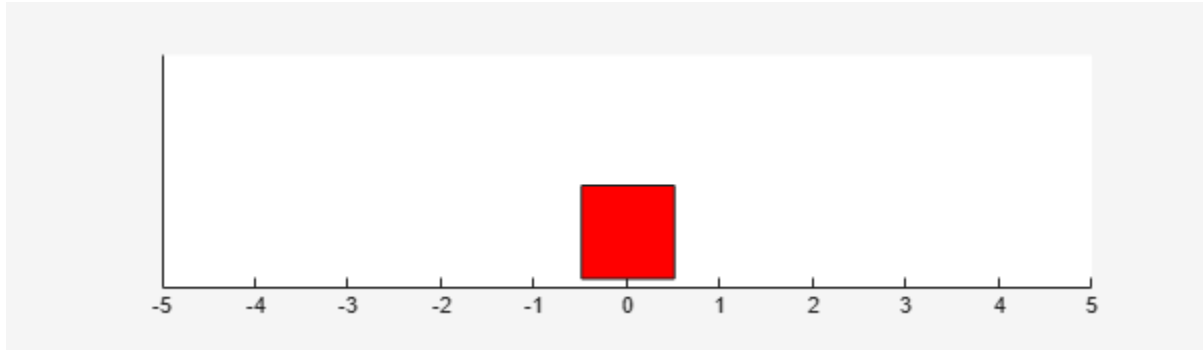
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,pgAgent,simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-38.9641
```

The trained PG agent does not stabilize the mass at the origin.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

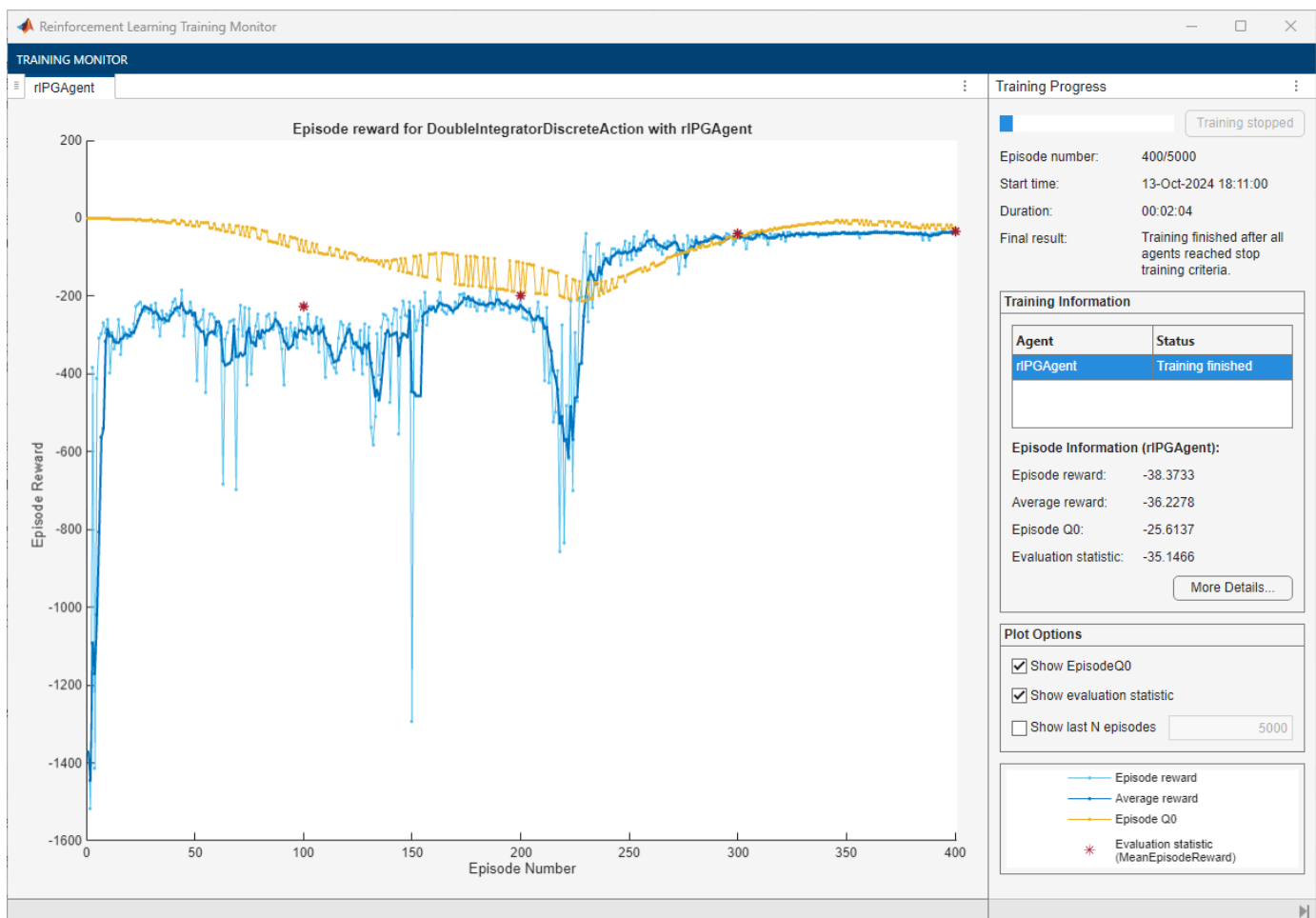
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("DoubleIntegrator-Discrete");
```

```

% Train the agent.
tic
acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
acTngTime = toc;
% Extract number of training episodes and total steps.
acTngEps = acTngRes.EpisodeIndex(end);
acTngSteps = sum(acTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ddiBchACAgent.mat", ...
%      "acAgent","acTngEps","acTngSteps","acTngTime")
else
% Load the pretrained agent and results for the example.
load("ddiBchACAgent.mat", ...
     "acAgent","acTngEps","acTngSteps","acTngTime")
end

```



For the AC agent, the training converges to a solution after 400 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

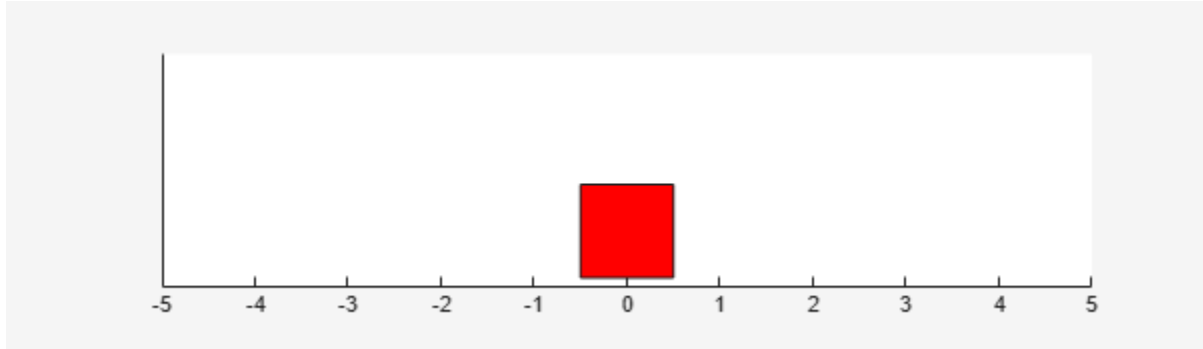
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 episodes, and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,acAgent,simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
-35.4029
```

The trained AC agent stabilizes the mass at the origin.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

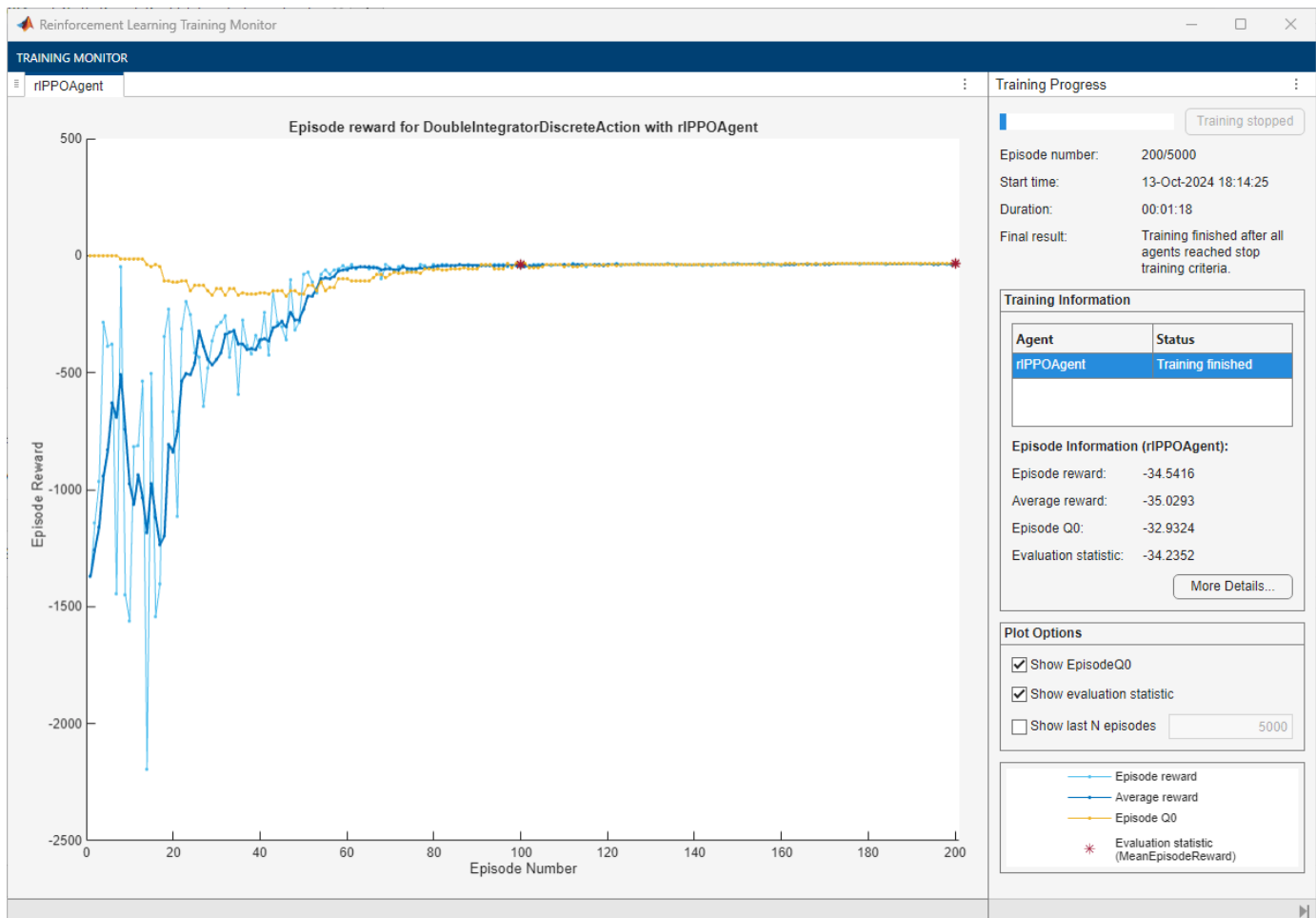
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.
```

```

env = rlPredefinedEnv("DoubleIntegrator-Discrete");
% Train the agent. Record the training time.
tic
ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
ppoTngTime = toc;
% Extract number of training episodes and total steps.
ppoTngEps = ppoTngRes.EpisodeIndex(end);
ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ddiBchPPOAgent.mat", ...
%      "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
else
% Load the pretrained agent and results for the example.
load("ddiBchPPOAgent.mat", ...
     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
end

```



For the PPO Agent, the training converges to a solution after 200 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

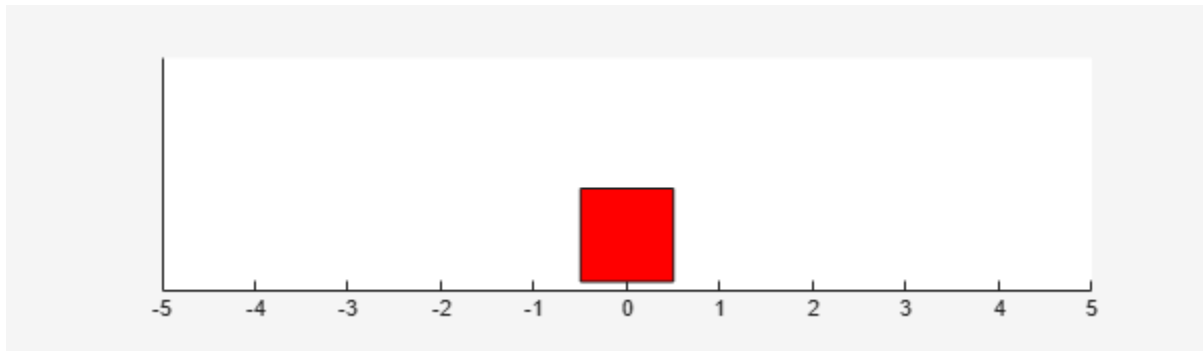
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,ppoAgent,simOptions);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-36.8182
```

The trained PPO agent stabilizes the mass at the origin.

Create, Train, and Simulate a SAC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

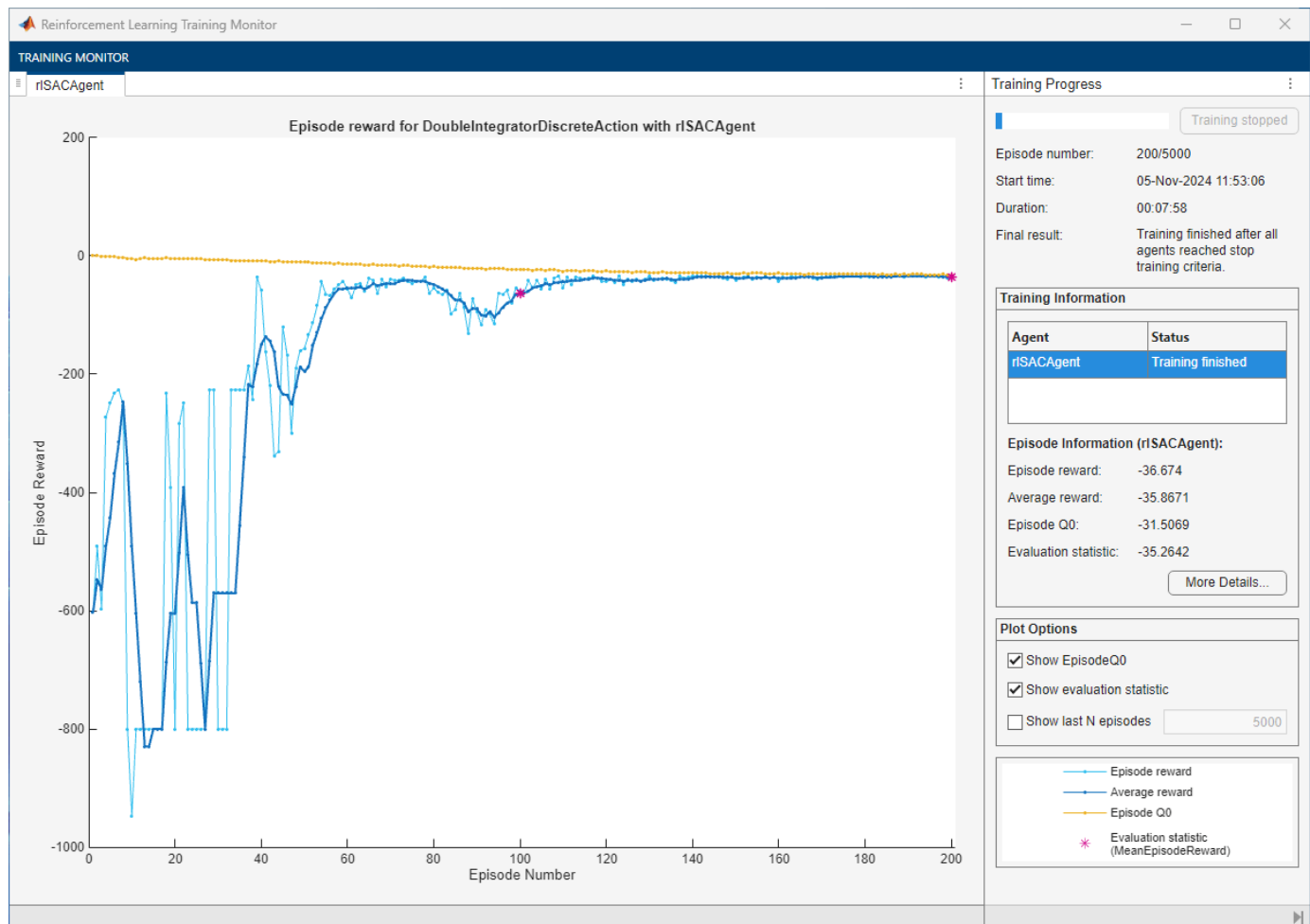
```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 0.005;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 0.5;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("DoubleIntegrator-Discrete");  
    % Train the agent. Record the training time.  
    tic  
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);  
    sacTngTime = toc;  
    % Extract number of training episodes and total steps.  
    sacTngEps = sacTngRes.EpisodeIndex(end);  
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    save("ddiBchSACAgent.mat", ...  
        "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("ddiBchSACAgent.mat", ...  
        "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")  
end
```



For the SAC agent, the training converges to a solution after 200 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

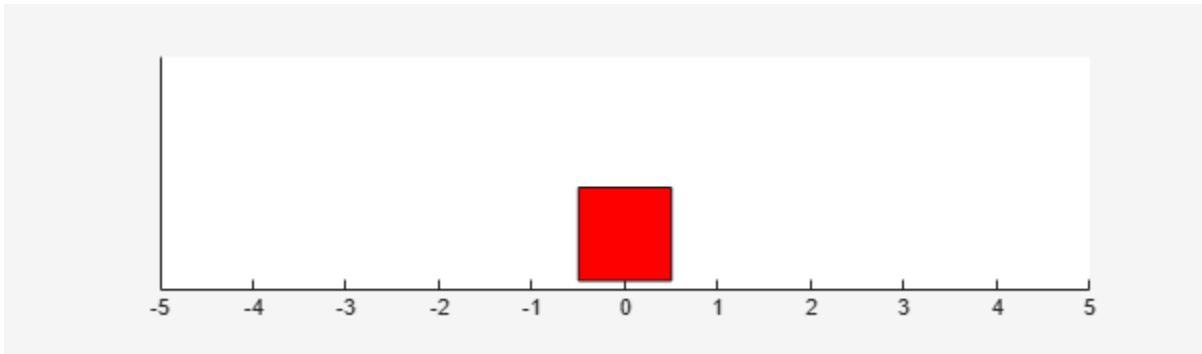
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env, sacAgent, simOptions);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-36.0284
```

The trained SAC agent stabilizes the mass at the origin.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    dqnTotalRwd  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    sacTotalRwd  
];
```

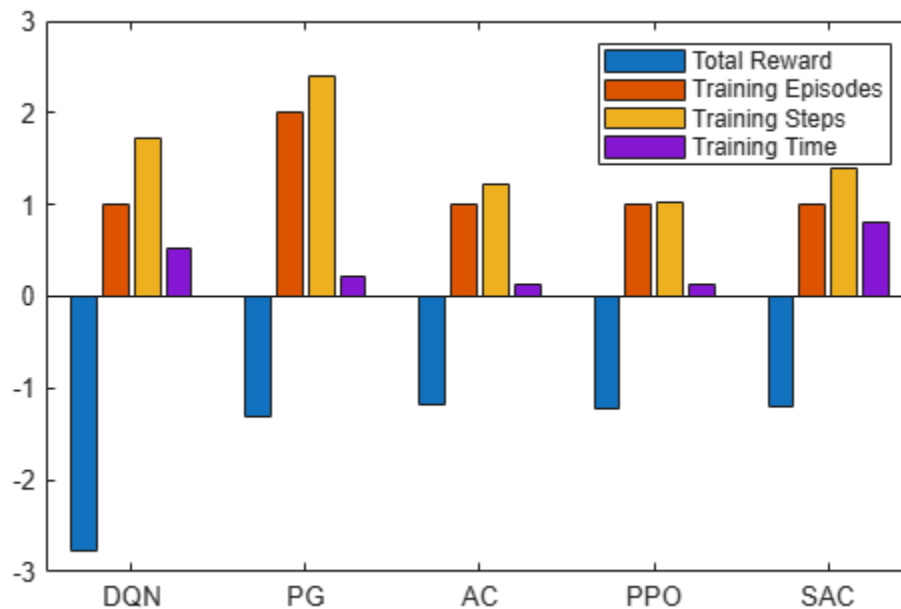
```
tngEpisodes = [  
    dqnTngEps  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    dqnTngSteps  
    pgTngSteps  
    acTngSteps  
    ppoTngSteps  
    sacTngSteps  
];
```

```
tngTime = [  
    dqnTngTime  
    pgTngTime  
    acTngTime  
    ppoTngTime  
    sacTngTime  
];
```


Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [30 200 2e6 600] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[30 200 2e6 600])
xticklabels(["DQN" "PG" "AC" "PPO" "SAC"])
legend( ...
    "Total Reward","Training Episodes", ...
    "Training Steps","Training Time", ...
    "Location","northeast")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, DQN performs worse in terms of total reward, with PPO and AC using less training time, and SAC (due to its more complex algorithm that needs to calculate more gradients), using more training time. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment the following line to save all the workspace variables
% save ddiAllVariables.mat
```

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

See Also

Functions

rlPredefinedEnv | getActionInfo | getObservationInfo | train | sim

Objects

rlEvaluator | rlAgentInitializationOptions | rlTrainingOptions |
rlSimulationOptions

Related Examples

- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Discrete Cart-Pole

This example shows how to create and train frequently used default agents on a discrete action space cart-pole environment. This environment represents a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The agent can apply a force on the cart and its training goal is to balance the pole upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

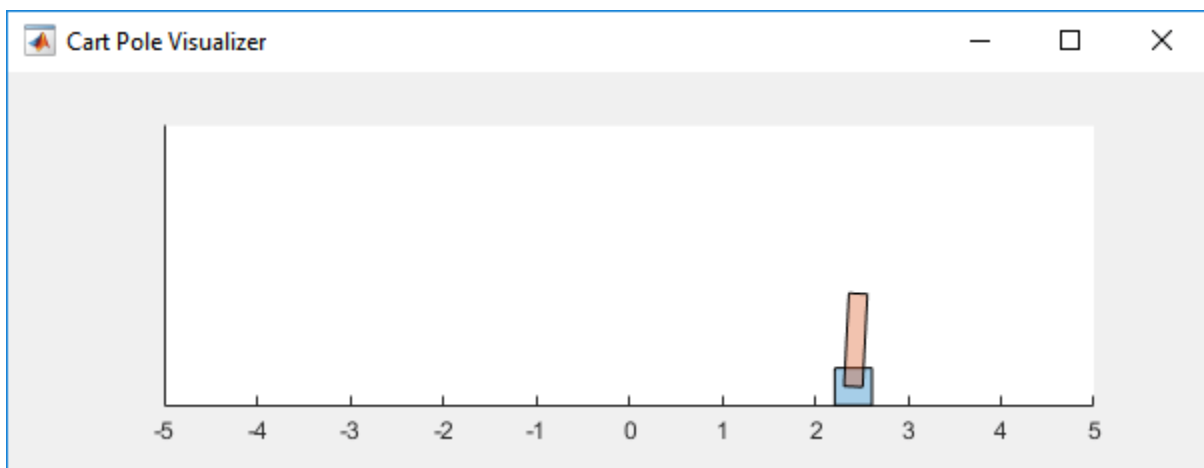
Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Cart-Pole MATLAB Environment

The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The agent can apply a force on the cart and its training goal is to balance the pole upright using minimal control effort.



For this environment:

- The upright pole angle is zero radians. Initially, the pole hangs downward (with an angle of π radians) without moving.
- The pole starts balanced, upright with an initial angle between -0.05 and 0.05 radians.
- The force action signal from the agent to the environment is either -10 or 10 N.
- The observations from the environment are the position and velocity of the cart, the pole angle, and the pole angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of $+1$ is provided for every time step that the pole remains balanced, upright. A penalty of -5 is applied when the pole falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the system.

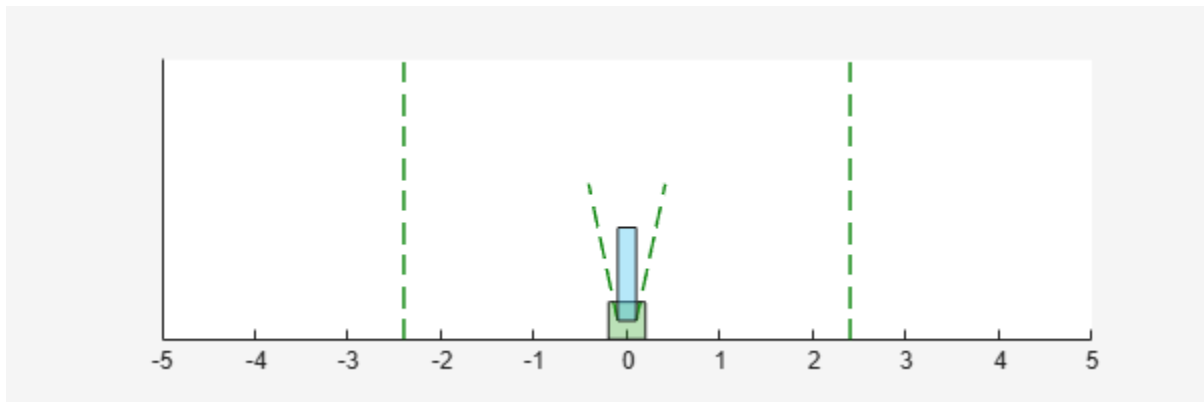
```
env = rlPredefinedEnv("CartPole-Discrete")
```

```
env =  
    CartPoleDiscreteAction with properties:  
        Gravity: 9.8000  
        MassCart: 1  
        MassPole: 0.1000  
        Length: 0.5000  
        MaxForce: 10  
        Ts: 0.0200  
        ThetaThresholdRadians: 0.2094  
        XThreshold: 2.4000  
        RewardForNotFalling: 1  
        PenaltyForFalling: -5  
        State: [4×1 double]
```

The object has a discrete action space where the agent can apply one of two possible force values to the cart, -10 or 10 N.

You can visualize the cart-pole system during training or simulation by using the `plot` function.

```
plot(env)
```



Get the observation and action specification objects.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "CartPole States"
    Description: "x, dx, theta, dtheta"
    Dimension: [4 1]
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlFiniteSetSpec with properties:
    Elements: [-10 10]
    Name: "CartPole Action"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options:

- Run the training for a maximum of 5000 episodes, with each episode lasting a maximum of 500 time steps.
- Stop the training when the average reward in the evaluation episodes is greater than 500. At this point, the agent can control the position of the pole using minimal control effort.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=5000, ...  
    MaxStepsPerEpisode=500, ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=500);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a DQN Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlDQNAgent` object using the environment specification objects.

```
dqnAgent = rlDQNAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
dqnAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

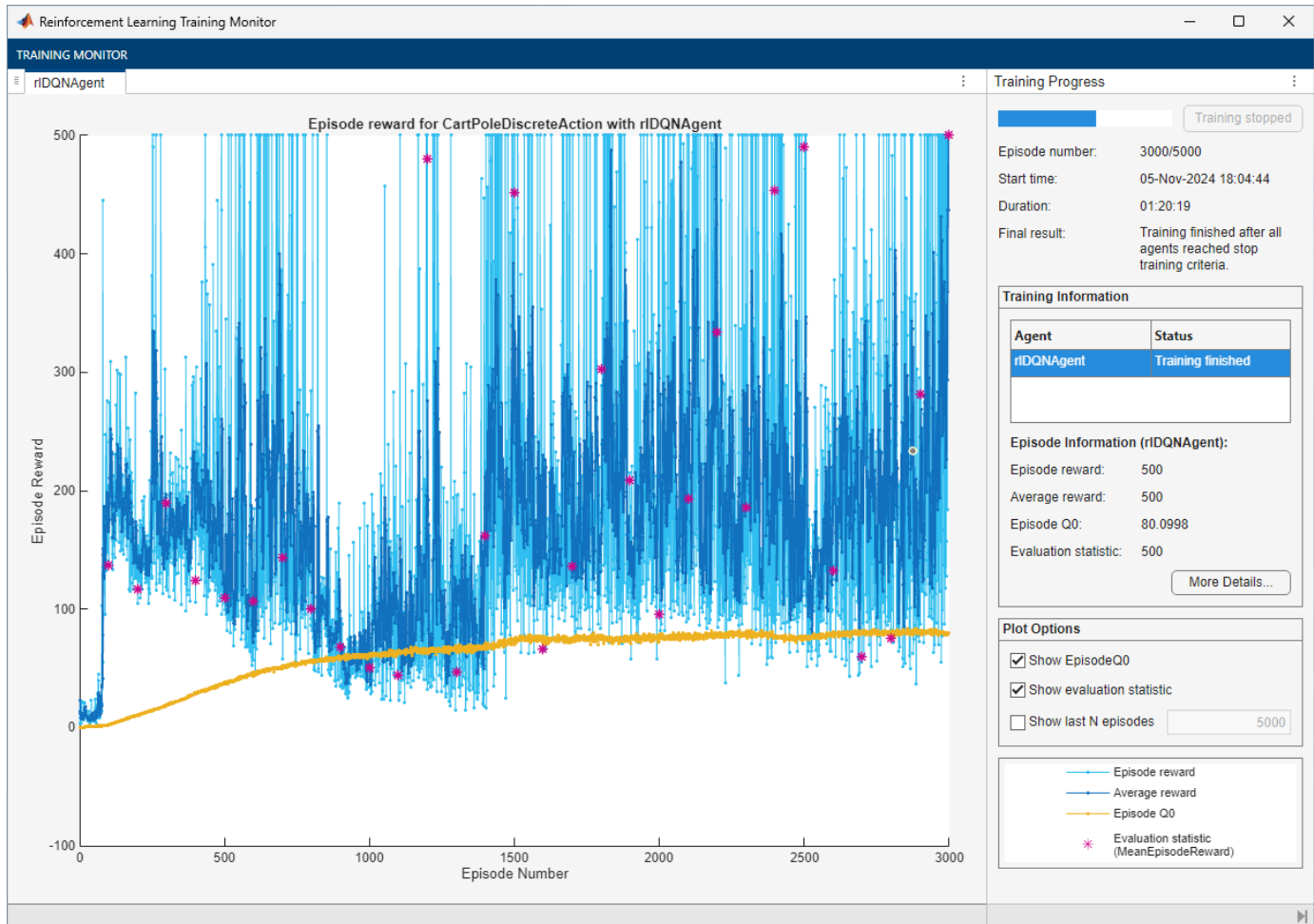
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("CartPole-Discrete");  
    % Train the agent. Record the training time.  
    tic  
    dqnTngRes = train(dqnAgent, env, trainOpts, Evaluator=evl);  
    dqnTngTime = toc;  
    % Extract number of training episodes and total steps.  
    dqnTngEps = dqnTngRes.EpisodeIndex(end);  
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("dcpBchDQNAgent.mat", ...  
    %     "dqnAgent", "dqnTngEps", "dqnTngSteps", "dqnTngTime")  
else
```

```

% Load the pretrained agent and results for the example.
load("dcpBchDQNAgent.mat", ...
     "dqnAgent", "dqnTngEps", "dqnTngSteps", "dqnTngTime")
end

```



For the DQN Agent, the training converges to a solution after 3000 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

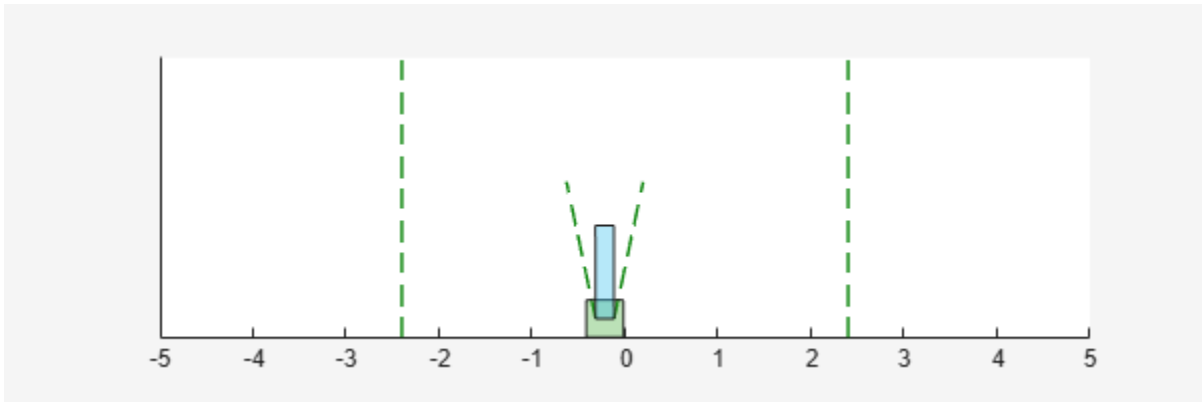
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, dqnAgent, simOptions);
```



```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
500
```

The trained DQN agent stabilizes the pole on the cart.

Create, Train, and Simulate a PG Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

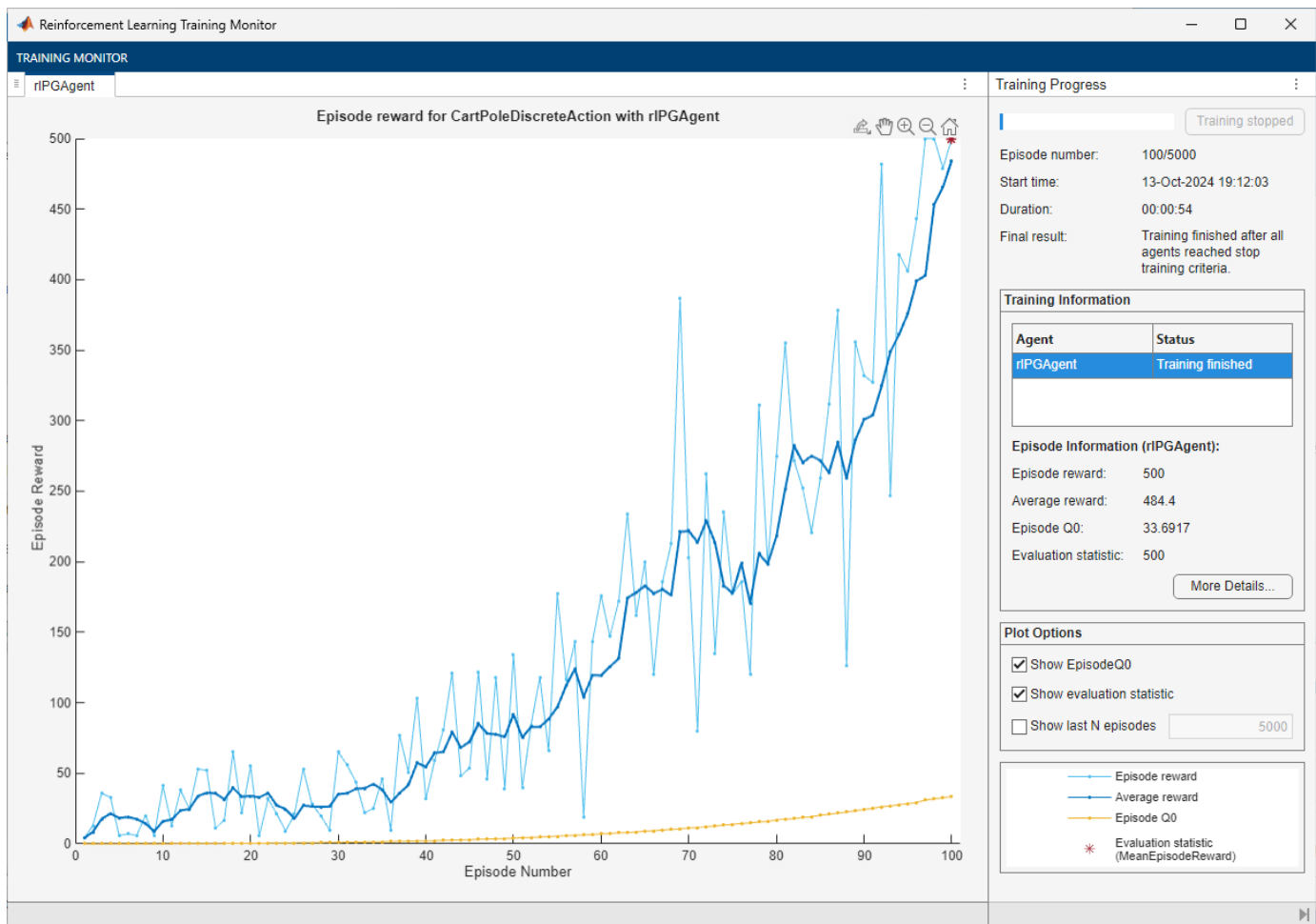
```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("CartPole-Discrete");  
    % Train the agent. Record the training time.  
    tic  
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);
```



```

pgTngTime = toc;
% Extract number of training episodes and total steps.
pgTngEps = pgTngRes.EpisodeIndex(end);
pgTngSteps = sum(pgTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("dcpBchPGAgent.mat", ...
%      "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
else
% Load the pretrained agent and results for the example.
load("dcpBchPGAgent.mat", ...
     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
end

```



For the PG Agent, the training converges to a solution after 100 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

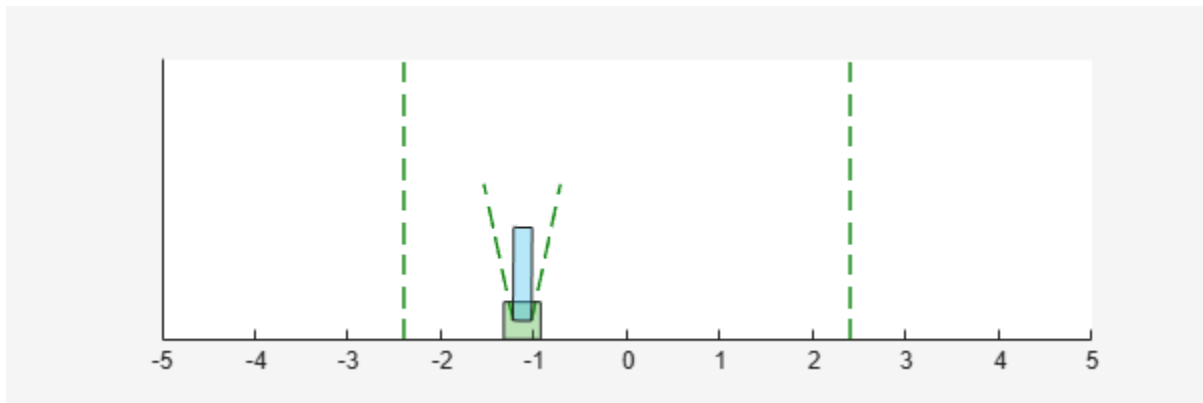
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,pgAgent,simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
500
```

The trained PG agent stabilizes the pole on the cart.

Create, Train, and Simulate an AC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

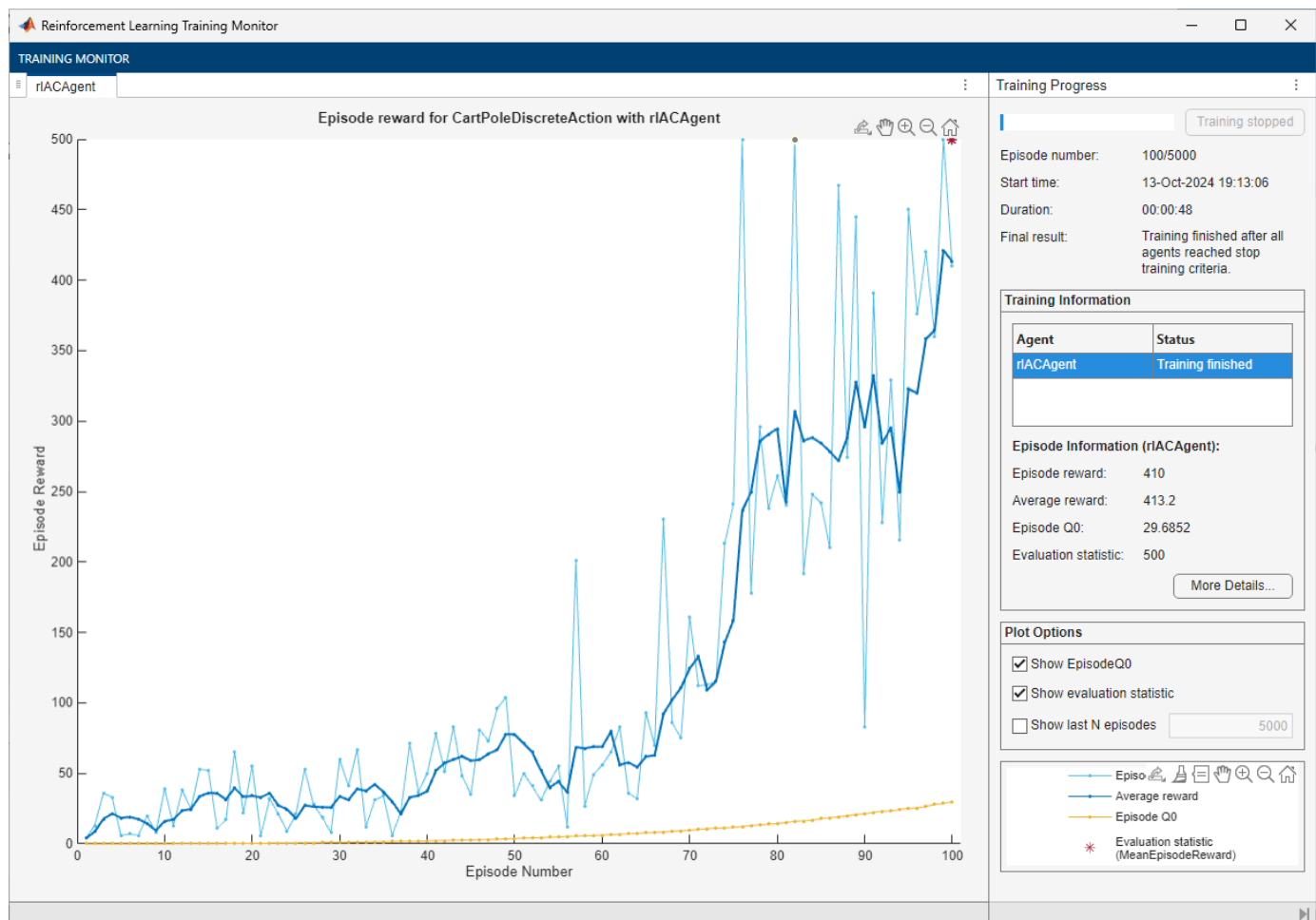
```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Discrete");
    % Train the agent. Save the final agent and training results.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dcpBchACAgent.mat", ...
    %      "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dcpBchACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end

```



For the AC agent, the training converges to a solution after 100 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

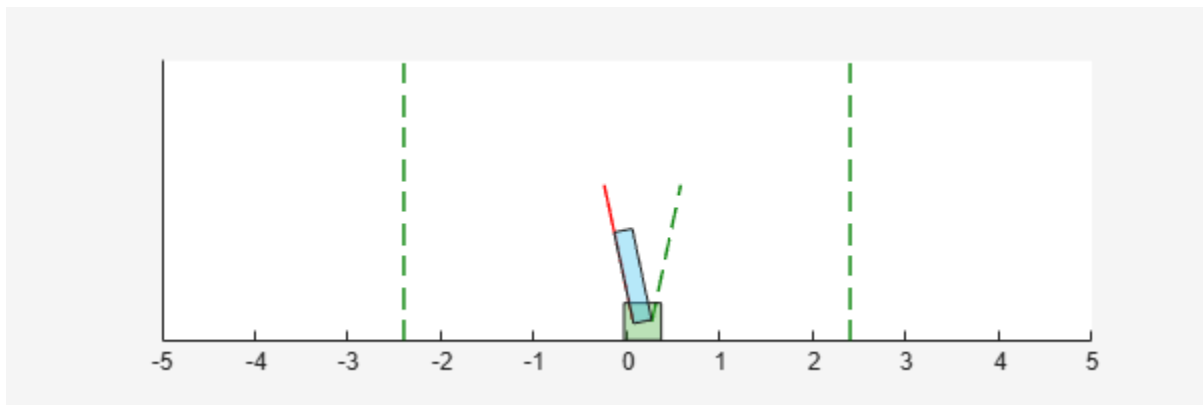
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
225
```

The trained AC agent does not stabilize the pole on the cart.

Create, Train, and Simulate a PPO Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo, actInfo);
```

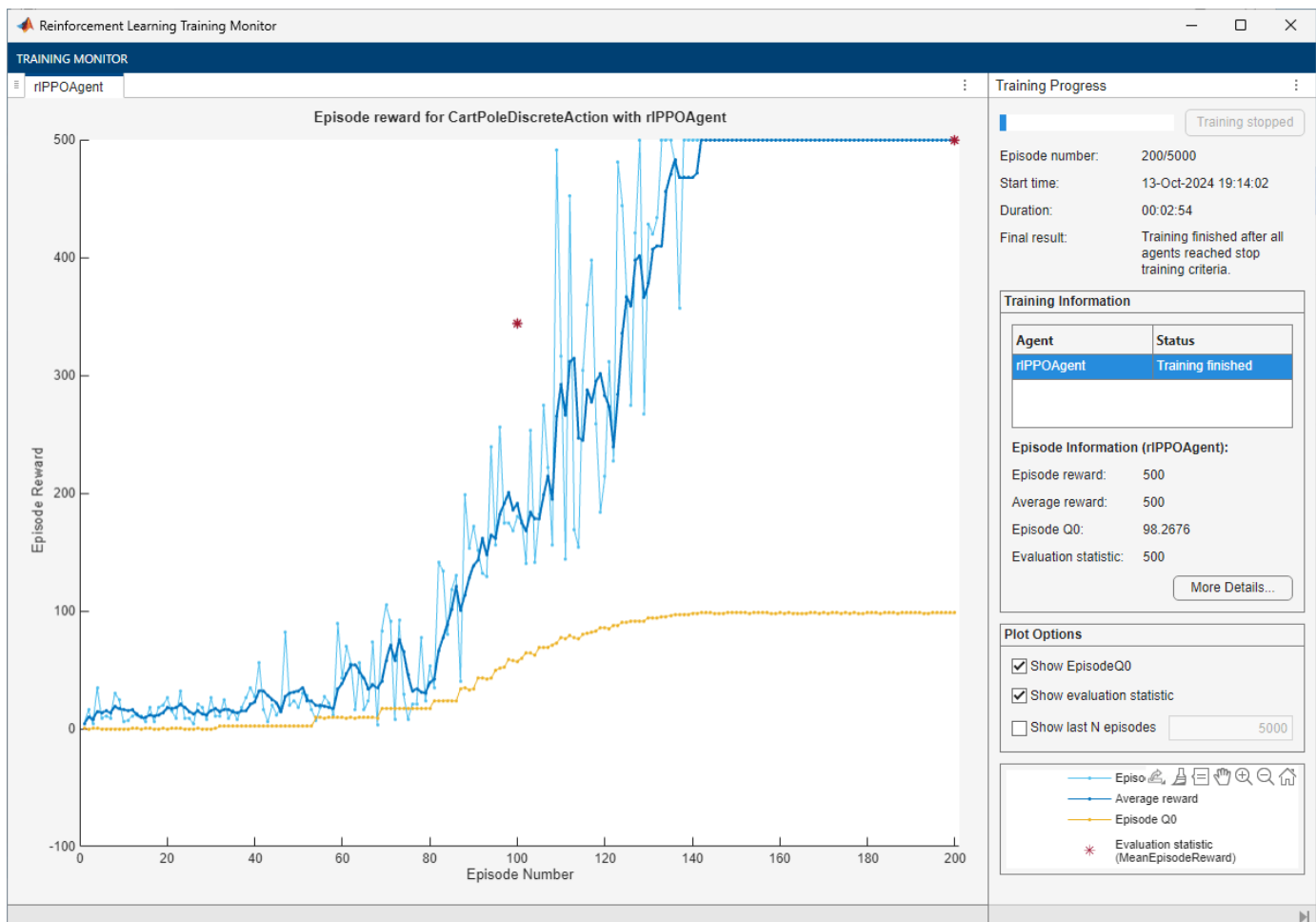
Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several

minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Discrete");
    % Train the agent. Record the training time.
    tic
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
    ppoTngTime = toc;
    % Extract number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dcpBchPPOAgent.mat", ...
    %     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dcpBchPPOAgent.mat", ...
        "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
end
```



For the PPO Agent, the training converges to a solution after 200 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

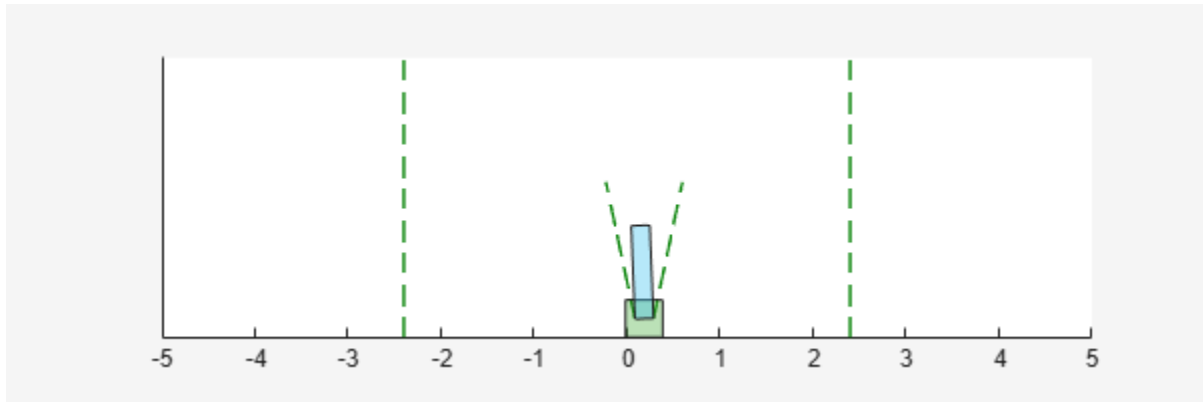
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ppoAgent, simOptions);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
500
```

The trained PPO agent stabilizes the pole on the cart.

Create, Train, and Simulate a SAC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

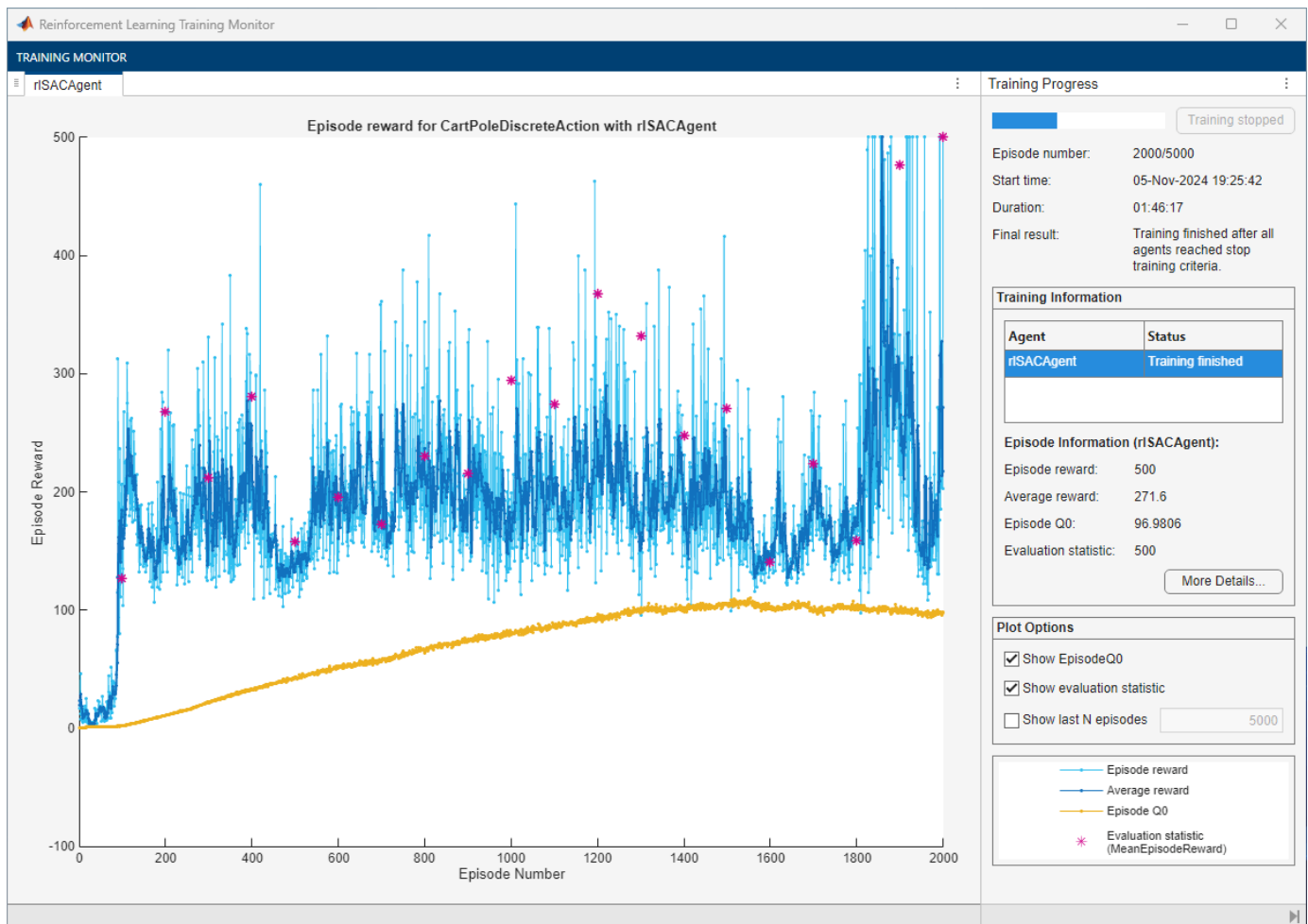
```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Discrete");
    % Train the agent. Record the training time.
    tic
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);
    sacTngTime = toc;
    % Extract number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dcpBchSACAgent.mat", ...
    %     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dcpBchSACAgent.mat", ...
        "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
end
```



For the AC agent, the training converges to a solution after 2000 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

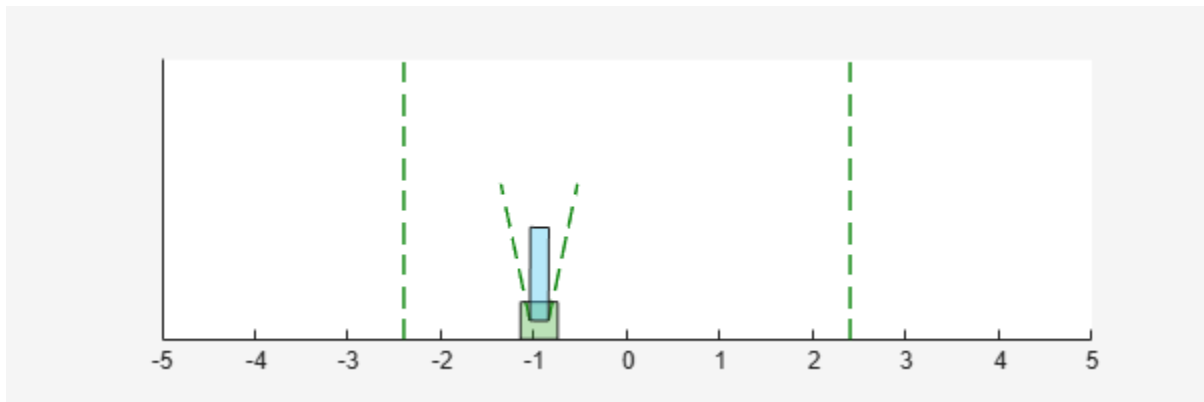
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, sacAgent, simOptions);
```

```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
500
```

The trained SAC agent stabilizes the pole on the cart.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    dqnTotalRwd  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    dqnTngEps  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    sacTngEps  
];
```

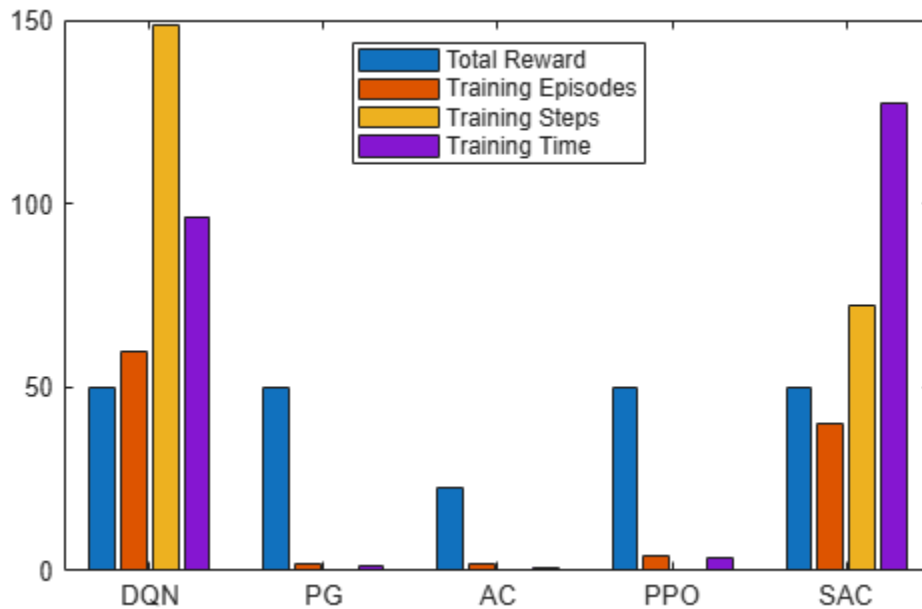
```
tngSteps = [  
    dqnTngSteps  
    pgTngSteps  
    acTngSteps  
    ppoTngSteps  
    sacTngSteps  
];
```

```
tngTime = [  
    dqnTngTime  
    pgTngTime  
    acTngTime  
    ppoTngTime  
];
```

```
sacTngTime
];
```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [10 50 5e6 50] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[10 50 5e6 50])
xticklabels(["DQN" "PG" "AC" "PPO" "SAC"])
legend("Total Reward","Training Episodes", ...
    "Training Steps","Training Time", ...
    "Location","north")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, all agents except AC are able to stabilize the pole on the cart in simulation. AC, PG and PPO use considerably less training time, while SAC (due to its more complex algorithm that needs to perform more gradient calculations), uses considerably more training time. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables
% save dcpAllVariables.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train PG Agent with Custom Actor Network to Balance Discrete Cart-Pole” on page 5-109

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on the Discrete Pendulum Swing-Up

This example shows how to create and train frequently used default agents on a discrete action space pendulum swing-up environment. This environment represents a simple frictionless pendulum that initially hangs in a downward position. The agent can apply a control torque on the pendulum, and its goal is to make the pendulum stand upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are in general better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

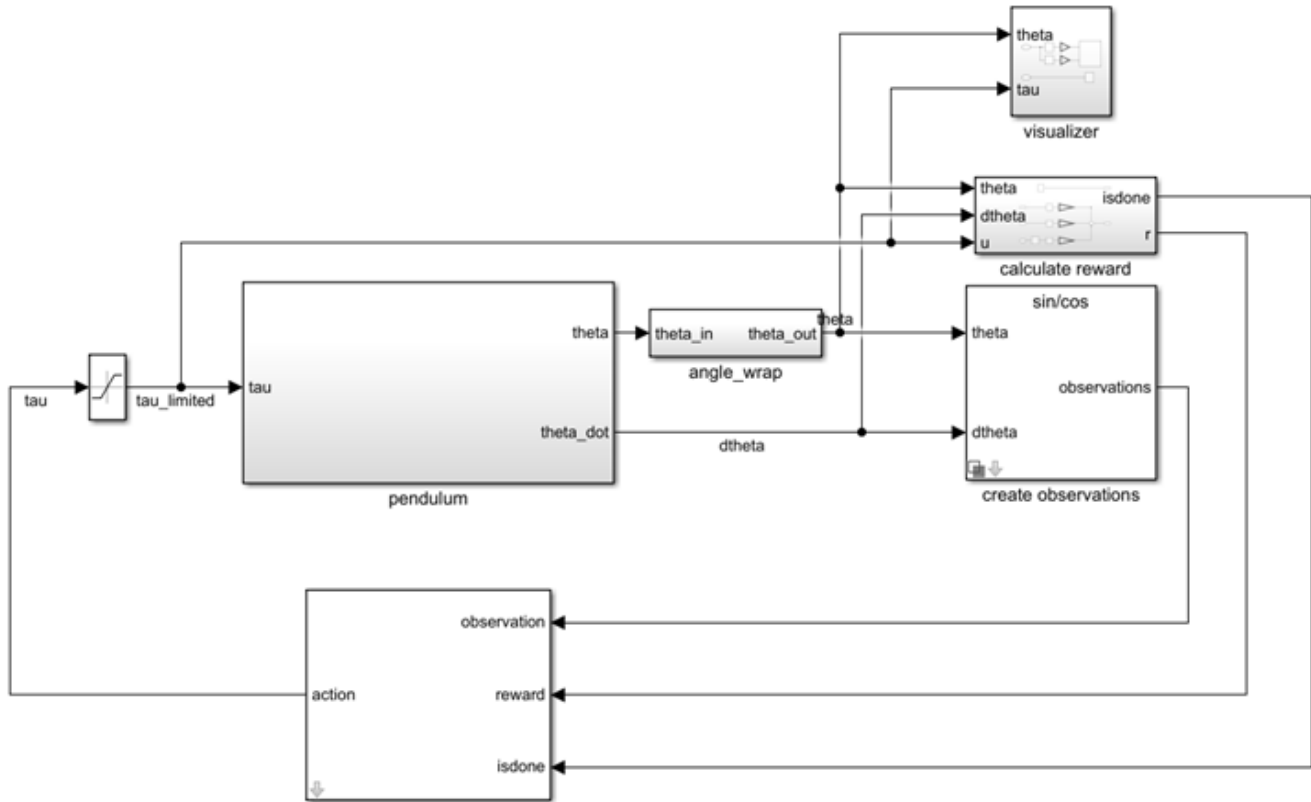
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Pendulum Swing-Up Model

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The agent can apply a control torque on the pendulum, and its goal is to make the pendulum stand upright using minimal control effort.

Open the model.

```
mdl = "rlSimplePendulumModel";  
open_system(mdl)
```



In this model:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is pi radians (the pendulum angle is counterclockwise-positive).
- The torque (counterclockwise-positive) action can be one of three possible values: -2, 0, or 2 Newtons per meter.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- The reward r_t , provided at every timestep, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the pendulum.

```
env = rlPredefinedEnv("SimplePendulumModel-Discrete")
```

```
env =  
SimulinkEnvWithAgent with properties:  
    Model : rlSimplePendulumModel  
    AgentBlock : rlSimplePendulumModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function sets the model workspace variable `theta0` to `pi`.

```
env.ResetFcn = @(in)setVariable(in,"theta0",pi,"Workspace",mdl);
```

Reset the environment.

```
reset(env);
```

For more information on Simulink® environments and their reset functions, see `SimulinkEnvWithAgent`.

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;  
Tf = 20;
```

Get the observation and action specification information from the environment.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
rlNumericSpec with properties:  
    LowerLimit: -Inf  
    UpperLimit: Inf  
    Name: "observations"  
    Description: [0×0 string]  
    Dimension: [3 1]  
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =  
rlFiniteSetSpec with properties:  
    Elements: [3×1 double]  
    Name: "torque"  
    Description: [0×0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run the training for a maximum of 1000 episodes, with each episode lasting 500 time steps.
- Stop the training when the agent receives an average cumulative reward greater than -1100 over five consecutive evaluation episodes. At this point, the agent can quickly balance the pendulum in the upright position using minimal control effort.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1000, ...
    MaxStepsPerEpisode=500, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-1100, ...
    Plots="training-progress");
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a DQN Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlDQNAgent` object using the environment specification objects.

```
dqnAgent = rlDQNAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `dqnAgent`.

```
dqnAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

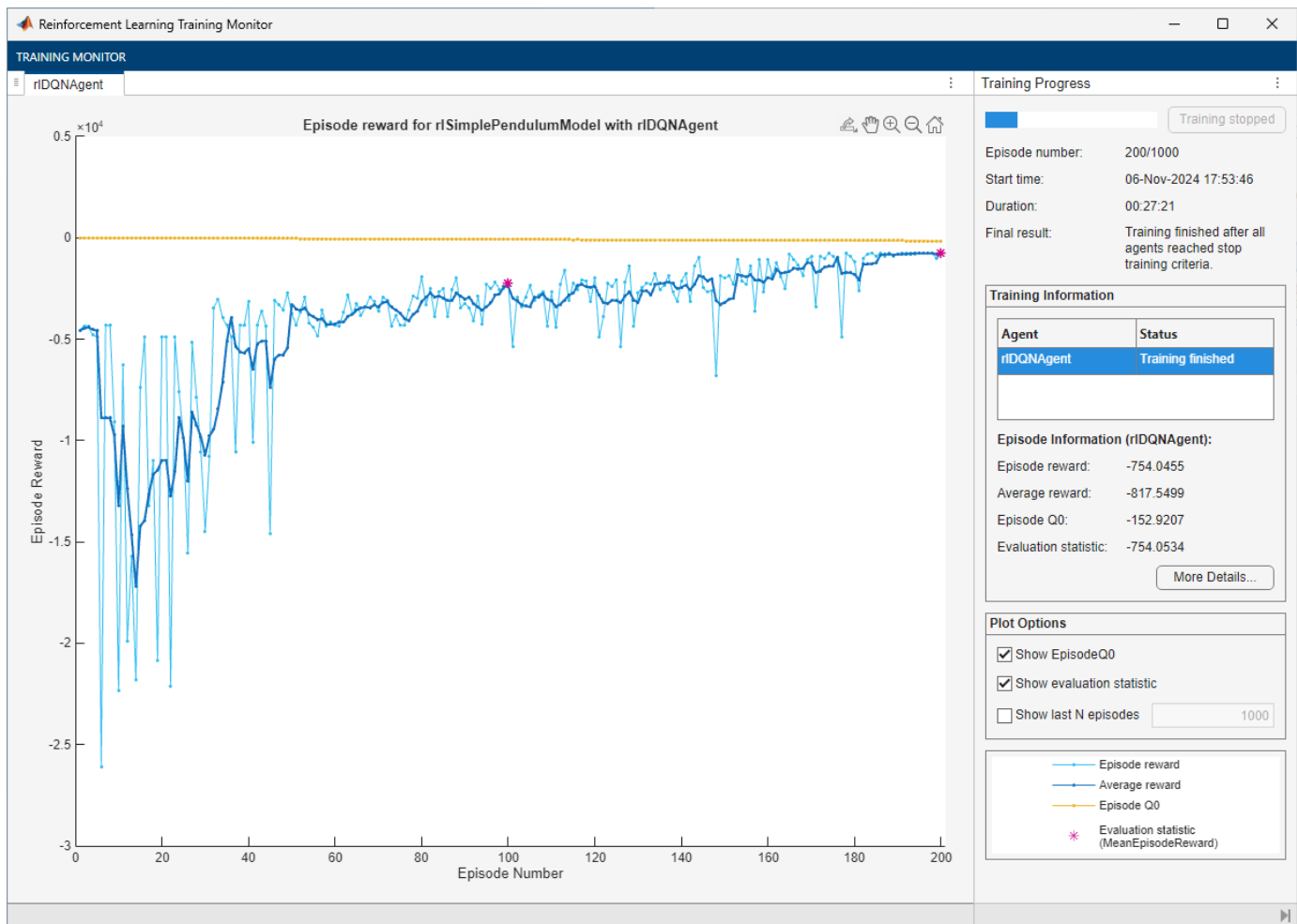
```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
dqnAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Record the training time.
    tic
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);
    dqnTngTime = toc;
    % Extract number of training episodes and total steps.
    dqnTngEps = dqnTngRes.EpisodeIndex(end);
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dpsuBchDQNData.mat", ...
    %      "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
else
    % Load the pretrained agent and the metrics for the example.
    load("dpsuBchDQNData.mat", ...
         "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
end
```

For the DQN Agent, the training converges to a solution after 200 episodes. You can check the trained agent within the pendulum swing-up environment.

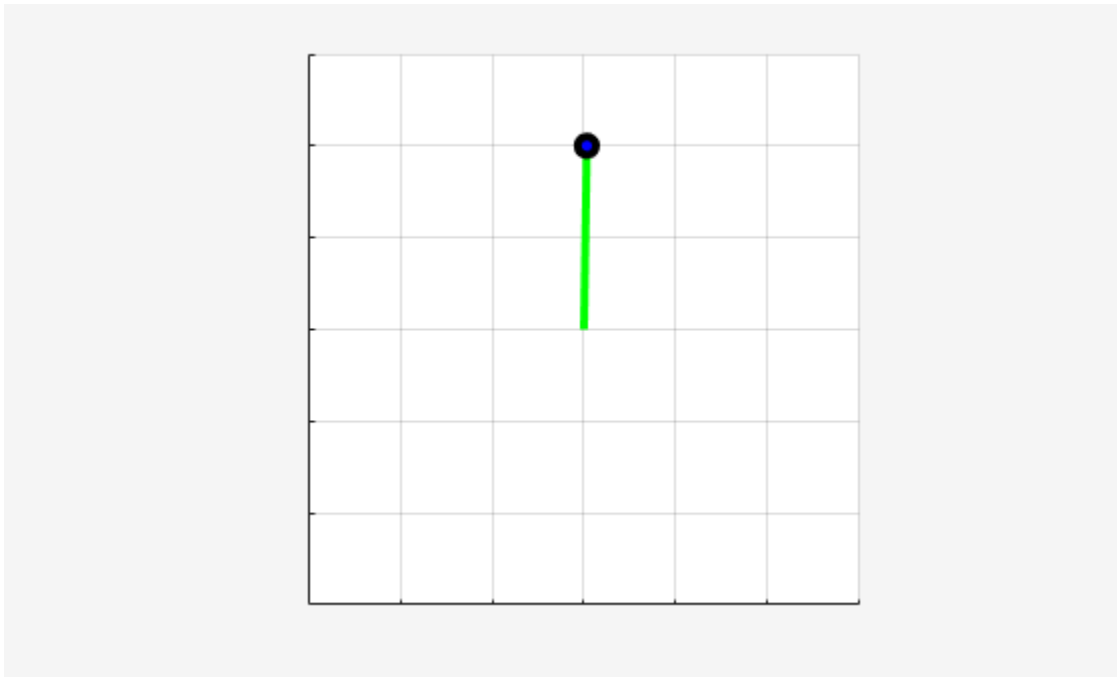
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,dqnAgent,simOptions);
```



```
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =  
-754.0534
```

The trained DQN agent is able to swing up and stabilize the pendulum upright.

Create, Train, and Simulate a PG Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `pgAgent`.

```
pgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent. Record the training time.
    tic
    pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dpsuBchPGData.mat", ...
    %     "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
else
    % Load the pretrained agent and the metrics for the example.
    load("dpsuBchPGData.mat", ...
        "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
end
```



For the PG Agent, the training does not converge to any solution after 1000 episodes. You can check the trained agent within the pendulum swing-up environment.

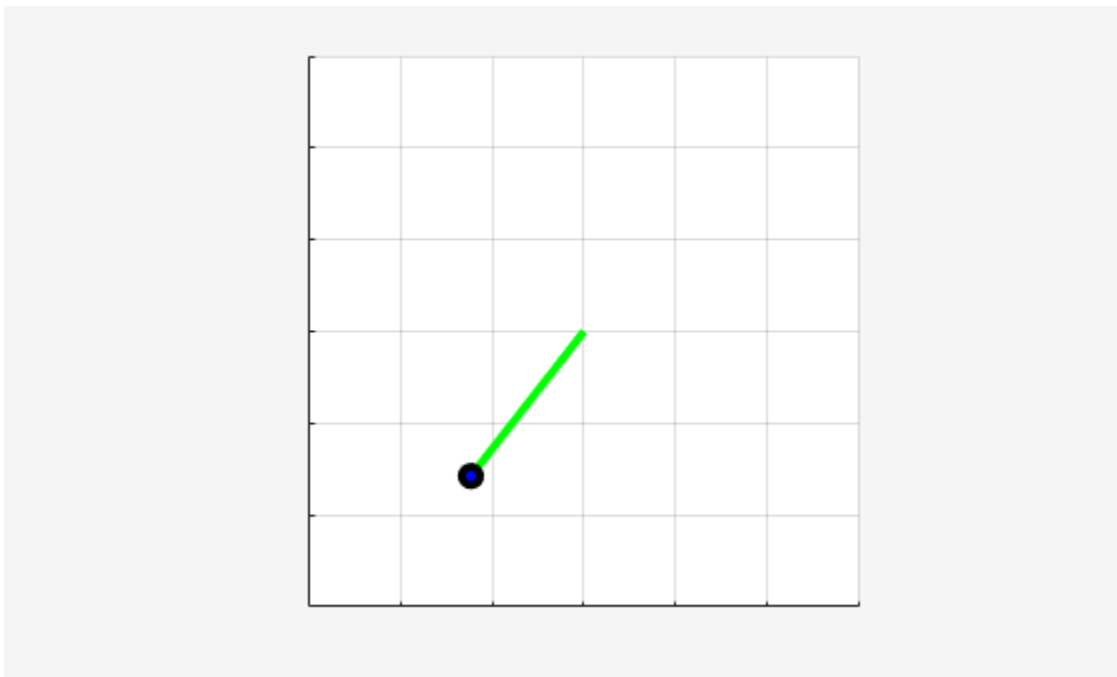
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-3.0525e+03
```

The trained PG agent is not able to swing up the pendulum.

Create, Train, and Simulate an AC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `acAgent`.

```
acAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

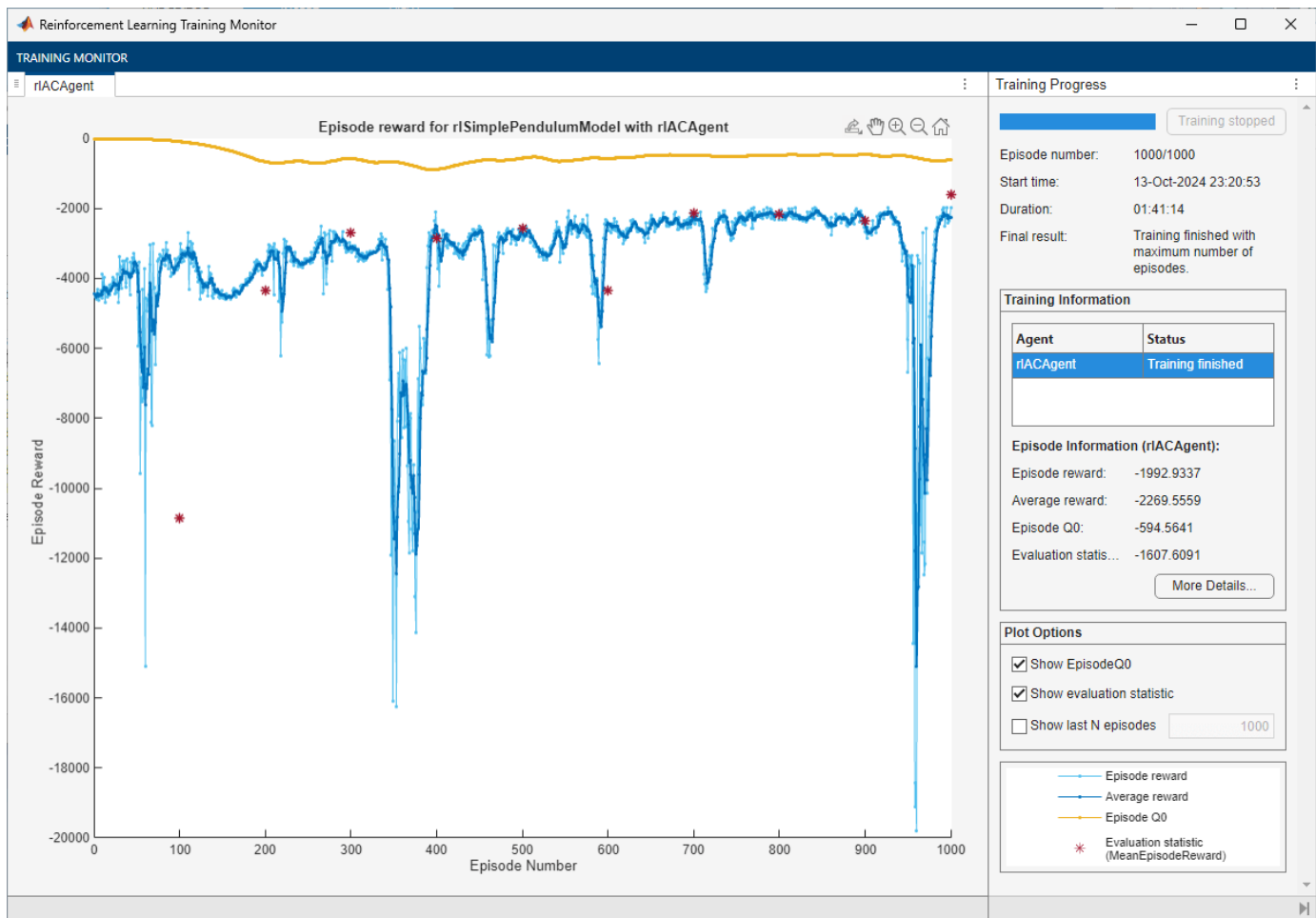
```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Record the training time.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dpsuBchACData.mat", ...
    %     "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and the metrics for the example.
    load("dpsuBchACData.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end
```



For the AC agent, as for the PG agent, the training does not converge to a solution after 1000 episodes. However, the evaluation score of the agent seemed to be steadily improving, which suggests that training the AC agent for longer might yield a trained agent able to stabilize the pendulum upright. You can check the trained agent within the pendulum swing-up environment.

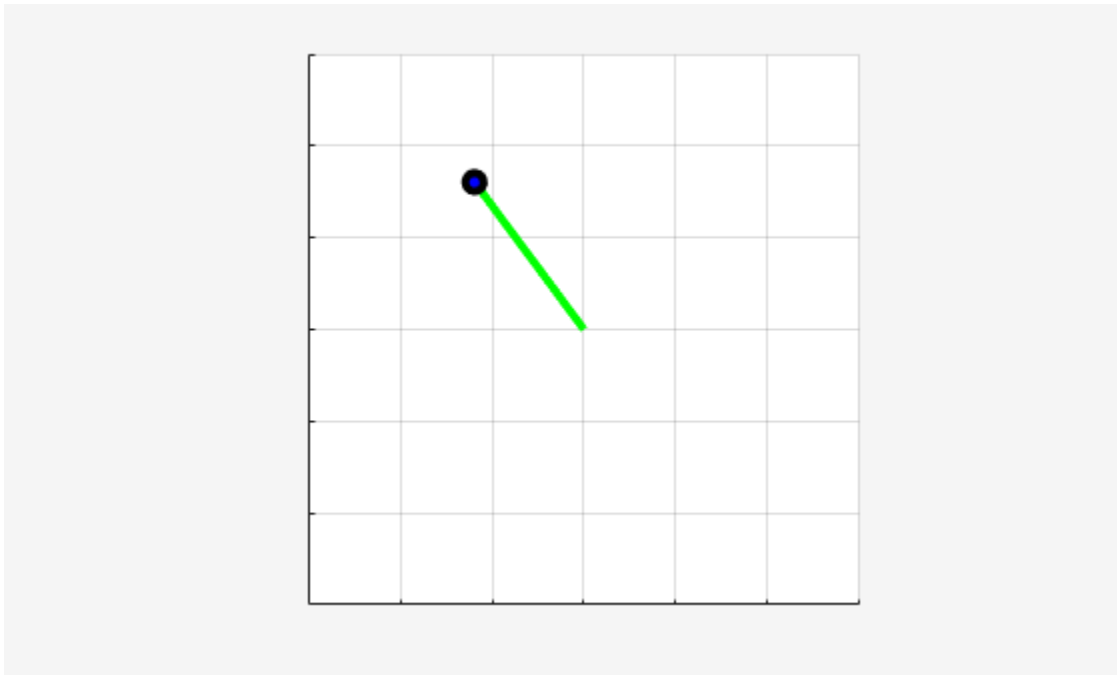
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
-2.0415e+03
```

The trained AC agent is able to swing up the pendulum to some extent, however, it does not stabilize it in the upright position.

Create, Train, and Simulate a PPO Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `ppoAgent`.

```
ppoAgent.AgentOptions.SampleTime = Ts;
```

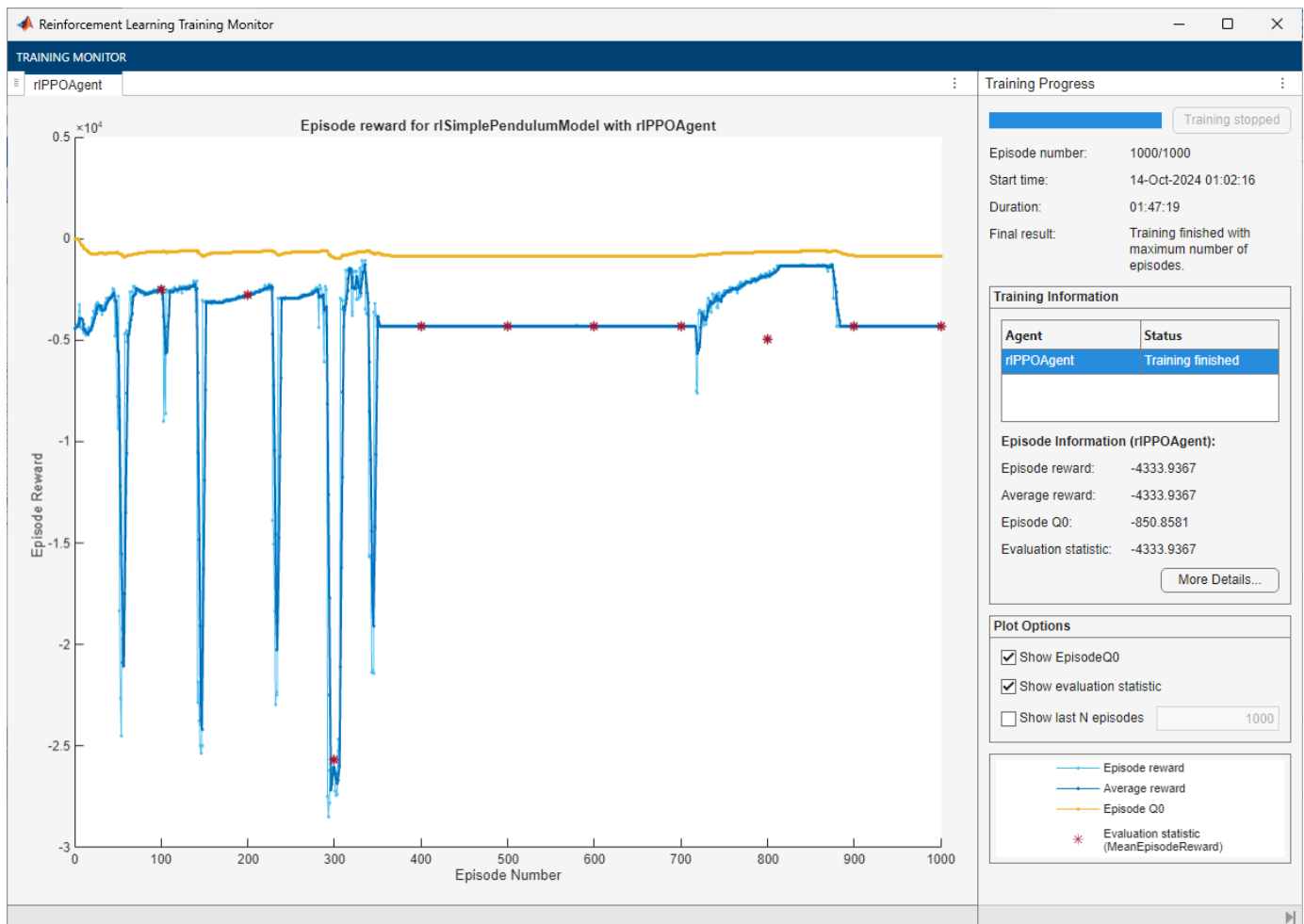
Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several

minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent. Record the training time.
    tic
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
    ppoTngTime = toc;
    % Extract number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dpsuBchPP0Data.mat", ...
    %     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dpsuBchPP0Data.mat", ...
        "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
end
```



For the PPO agent, as for the PG and AC agents, the training does not converge to a solution after 1000 episodes. You can check the trained agent within the pendulum swing-up environment.

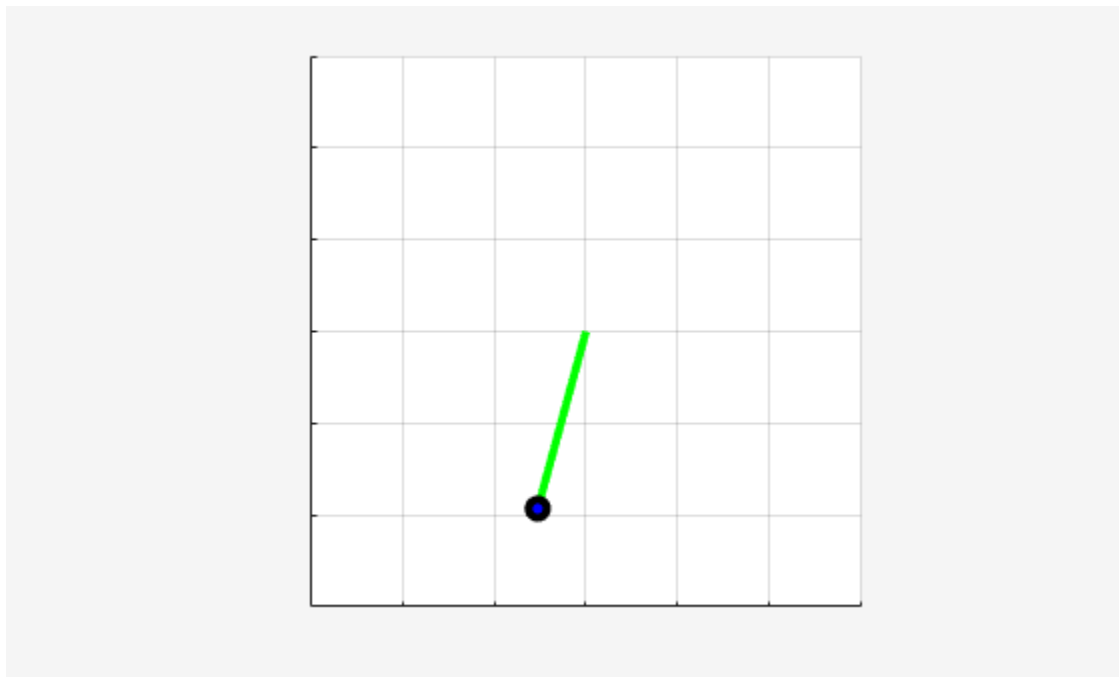
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ppoAgent, simOptions);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-4.3339e+03
```

The trained PPO agent is not able to swing up the pendulum.

Create, Train, and Simulate a SAC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `ppoAgent`.

```
sacAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent. Record the training time.  
    tic  
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);  
    sacTngTime = toc;  
    % Extract number of training episodes and total steps.  
    sacTngEps = sacTngRes.EpisodeIndex(end);  
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("dpsuBchSACData.mat", ...  
    %     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("dpsuBchSACData.mat", ...  
        "sacAgent","sacTngEps","sacTngSteps","sacTngTime")  
end
```



For the SAC agent, the training converges to a solution after 200 episodes. You can check the trained agent within the pendulum swing-up environment.

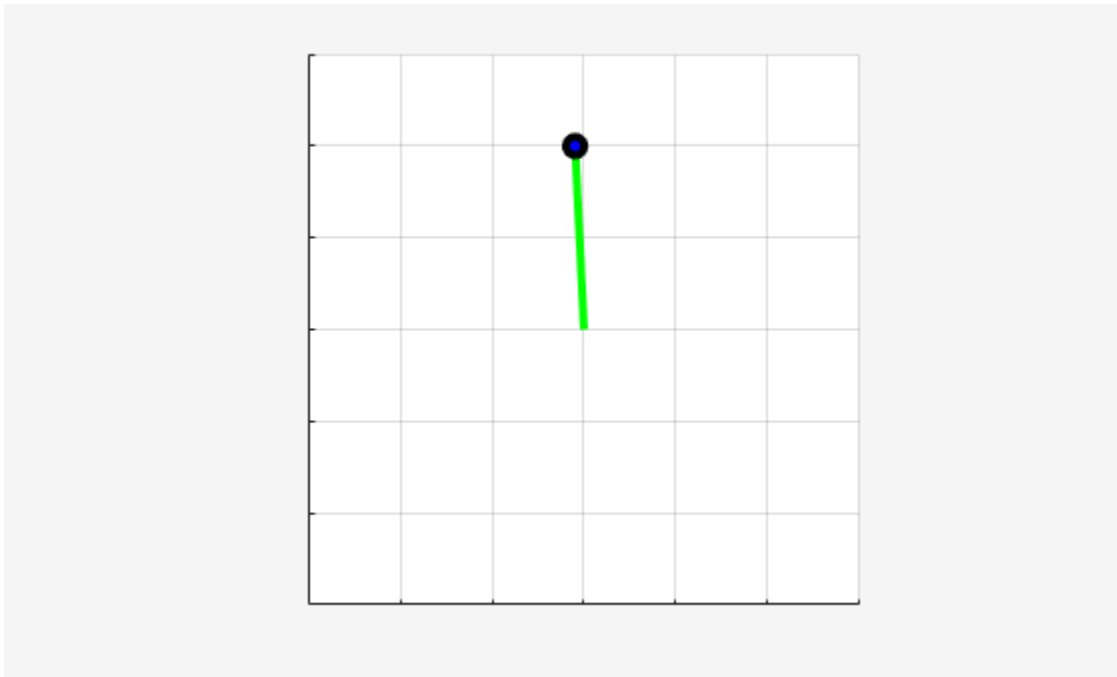
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
experience = sim(env, sacAgent, simOptions);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-863.4626
```

The trained SAC agent is able to swing up and stabilize the pendulum upright.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    dqnTotalRwd  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    dqnTngEps  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    dqnTngSteps  
    pgTngSteps  
    acTngSteps  
];
```

```

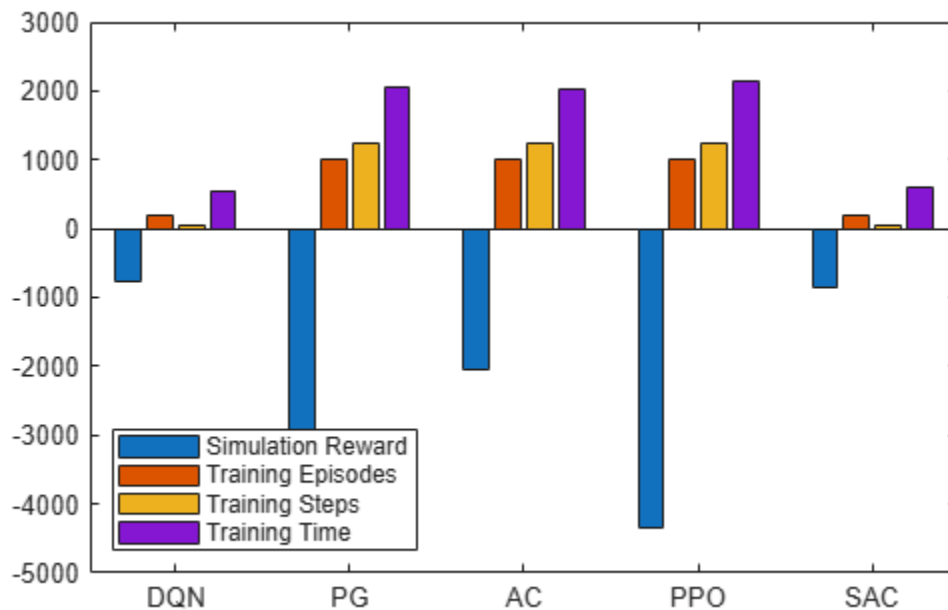
ppoTngSteps
sacTngSteps
];

tngTime = [
    dqnTngTime
    pgTngTime
    acTngTime
    ppoTngTime
    sacTngTime
];

Plot the simulation reward, number of training episodes, number of training steps (that is, the
number of interactions between the agent and the environment) and the training time. Scale the data
by the factor [1 1 2e5 3] for better visualization.

bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 1 2e5 3])
xticklabels(["DQN" "PG" "AC" "PPO" "SAC"])
legend( ...
    ["Simulation Reward", ...
    "Training Episodes", ...
    "Training Steps", ...
    "Training Time"], ...
    "Location", "southwest")

```



Only the trained DQN and SAC agents are able to stabilize the pendulum. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```

% Uncomment to save all the workspace variables
% save dpsuAllVariables.mat

```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Discrete Simscape Cart-Pole Swing-Up

This example shows how to create and train frequently used default agents on a discrete action space cart-pole environment. This environment is modeled using Simscape™ Multibody™, and represents a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The agent can apply a force to the cart and its training goal is to swing-up and balance the pole upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

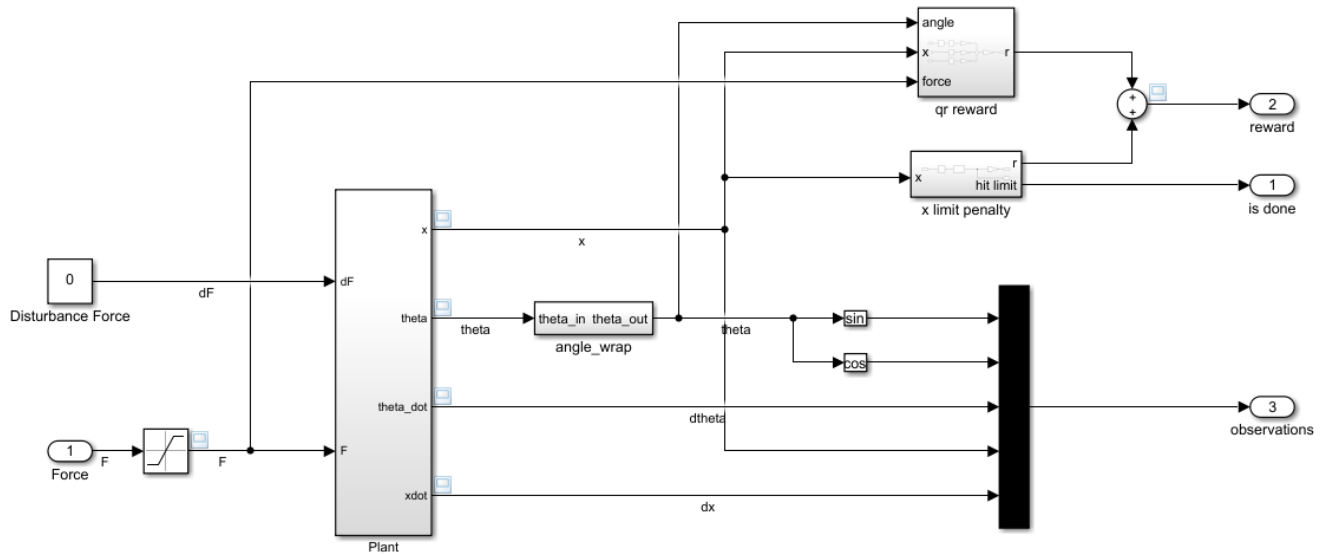
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Simscape Cart-Pole Environment

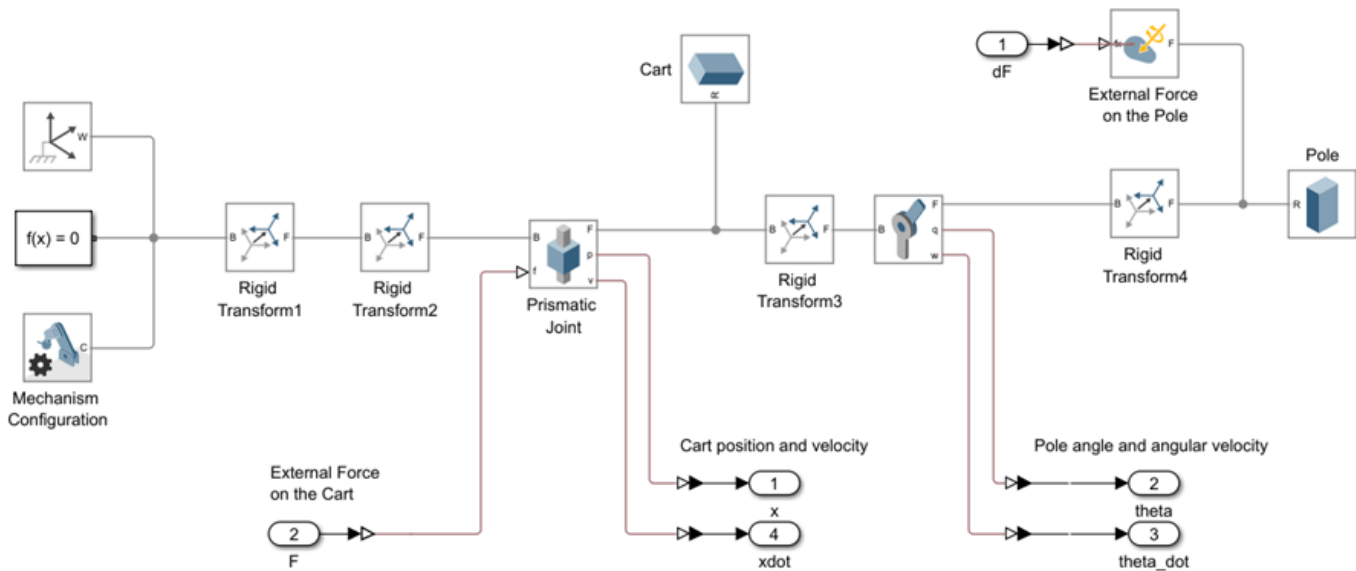
The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to swing up and balance the pole using minimal control effort.

Open the model.

```
mdl = "rlCartPoleSimscapeModel";
open_system(mdl)
```



The cart-pole system is modeled using Simscape Multibody.



In this model:

- The upright pole angle is zero radians. Initially the pole hangs downwards (the angle is π radians) without moving.
- The force (rightwise-positive) action signal from the agent to the environment is either -15, 0 or 15 N.
- The observations from the environment are the position and velocity of the cart, and the sine, cosine, and derivative of the pole angle.
- The episode terminates if the cart moves more than 3.5 m from the original position.

- The reward r_t , provided at every time step, is

$$r_t = -0.1(5\theta_t^2 + x_t^2 + 0.05u_{t-1}^2) - 100B$$

Here:

- θ_t is the (counterclockwise-positive) angle of displacement from the upright position of the pole.
- x_t is the position displacement from the center position of the cart.
- u_{t-1} is the control effort from the previous time step.
- B is a flag (1 or 0) that indicates whether the cart is out of bounds.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the system.

```
env = rlPredefinedEnv("CartPoleSimscapeModel-Discrete")
```

```
env =
```

```
SimulinkEnvWithAgent with properties:
```

```
    Model : rlCartPoleSimscapeModel
  AgentBlock : rlCartPoleSimscapeModel/RL Agent
    ResetFcn : []
  UseFastRestart : on
```

The object has a discrete action space where the agent can apply one of two possible force values to the cart, -10 or 10 N.

Get the observation and action specification objects.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
```

```
rlNumericSpec with properties:
```

```
    LowerLimit: -Inf
    UpperLimit: Inf
        Name: "observations"
  Description: [0x0 string]
    Dimension: [5 1]
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
```

```
rlFiniteSetSpec with properties:
```

```
    Elements: [3x1 double]
        Name: "force"
  Description: [0x0 string]
    Dimension: [1 1]
```

```
DataType: "double"
```

Specify the agent sample time T_s and the simulation time T_f in seconds.

```
Ts = 0.02;  
Tf = 25;
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run each training episode for a maximum of 2000 episodes, with each episode lasting a maximum of `ceil(Tf/Ts)` time steps.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.
- Stop the training when the average cumulative reward over the evaluation episodes is greater than -400. At this point, the agent can quickly swing up and balance the pole in the upright position using minimal control effort.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=2000, ...  
    MaxStepsPerEpisode=ceil(Tf/Ts), ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=-400);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a DQN Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlDQNAgent` object using the environment specification objects.

```
dqnAgent = rlDQNAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `dqnAgent`.

```
dqnAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

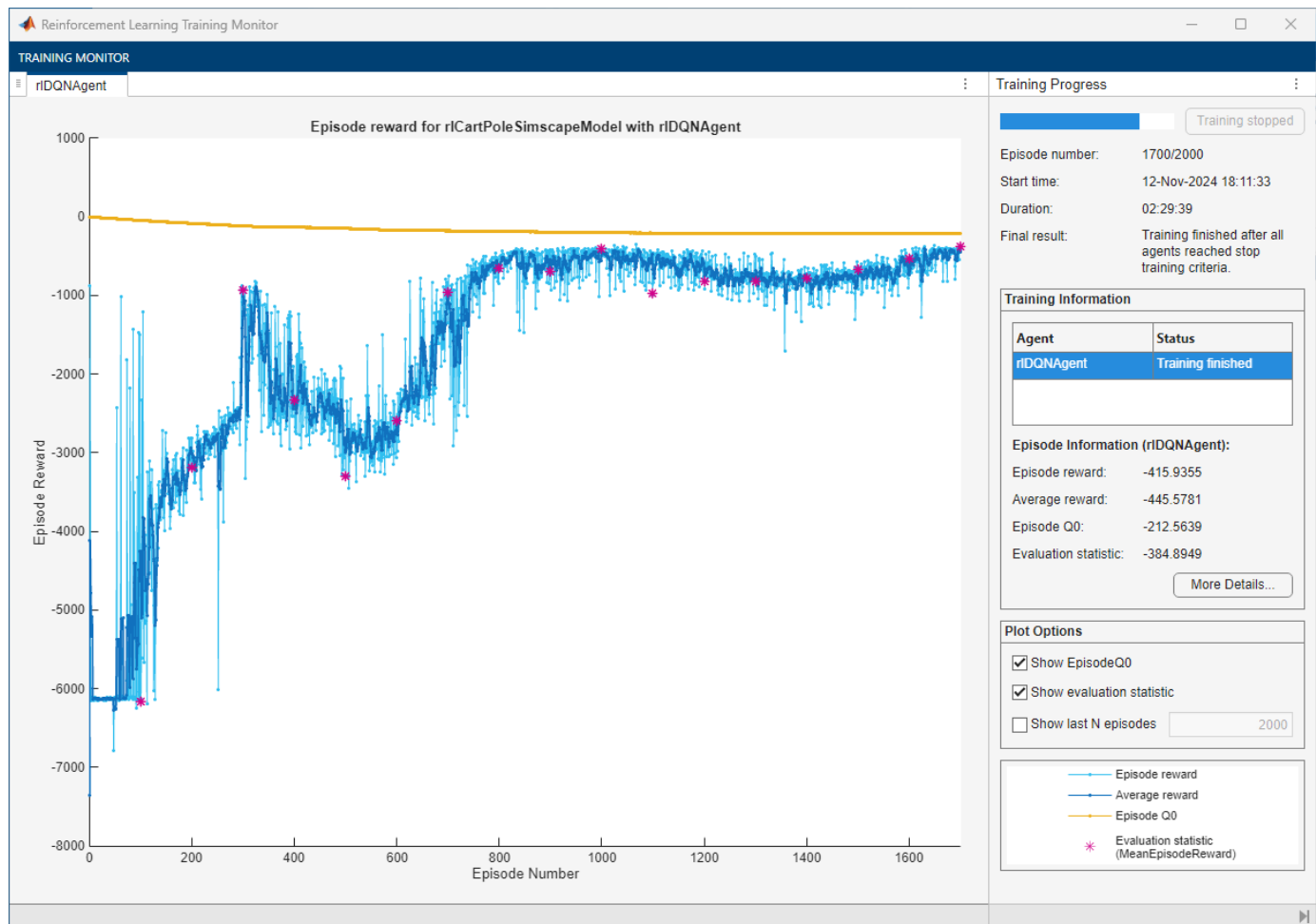
```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
dqnAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);
    dqnTngTime = toc;
    % Extract number of training episodes and total steps.
    dqnTngEps = dqnTngRes.EpisodeIndex(end);
    dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dscpsuBchDQNAgent.mat", ...
    %     "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
else
    load("dscpsuBchDQNAgent.mat", ...
        "dqnAgent","dqnTngEps","dqnTngSteps","dqnTngTime")
end
```



For the DQN Agent, the training converges to a solution after 1700 episodes. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,dqnAgent,simOptions);
dqnTotalRwd = sum(experience.Reward)
```

```
dqnTotalRwd =
-333.1581
```

The trained DQN agent swings up the pole and maintains it upright.

Create, Train, and Simulate a PG Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `pgAgent`.

```
pgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

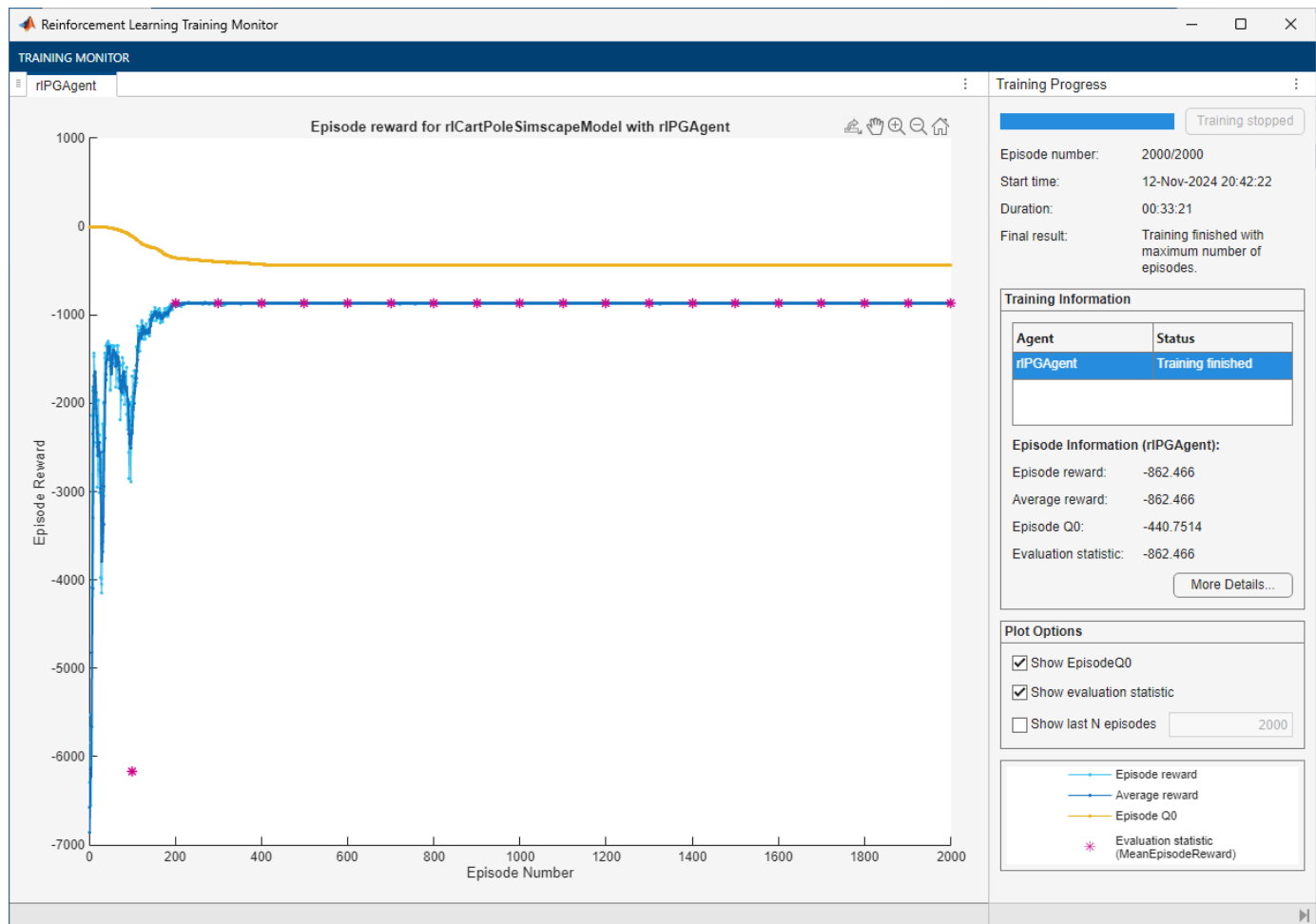
```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dscpsuBchPGAgent.mat", ...
    %     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dscpsuBchPGAgent.mat", ...
        "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
end
```



For the PG Agent, the training does not converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =
-862.4660
```

The trained PG agent is not able to swing up the pole.

Create, Train, and Simulate an AC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `acAgent`.

```
acAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    acTngRes = train(acAgent, env, trainOpts, Evaluator=evl);
    acTngTime = toc;
    % Extract number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dscpsuBchACAgent.mat", ...
    %     "acAgent", "acTngEps", "acTngSteps", "acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dscpsuBchACAgent.mat", ...
        "acAgent", "acTngEps", "acTngSteps", "acTngTime")
end
```



For the PG Agent, the training does not converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =
-1.0467e+03
```

The trained AC agent is not able to swing up the pole.

Create, Train, and Simulate a PPO Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `ppoAgent`.

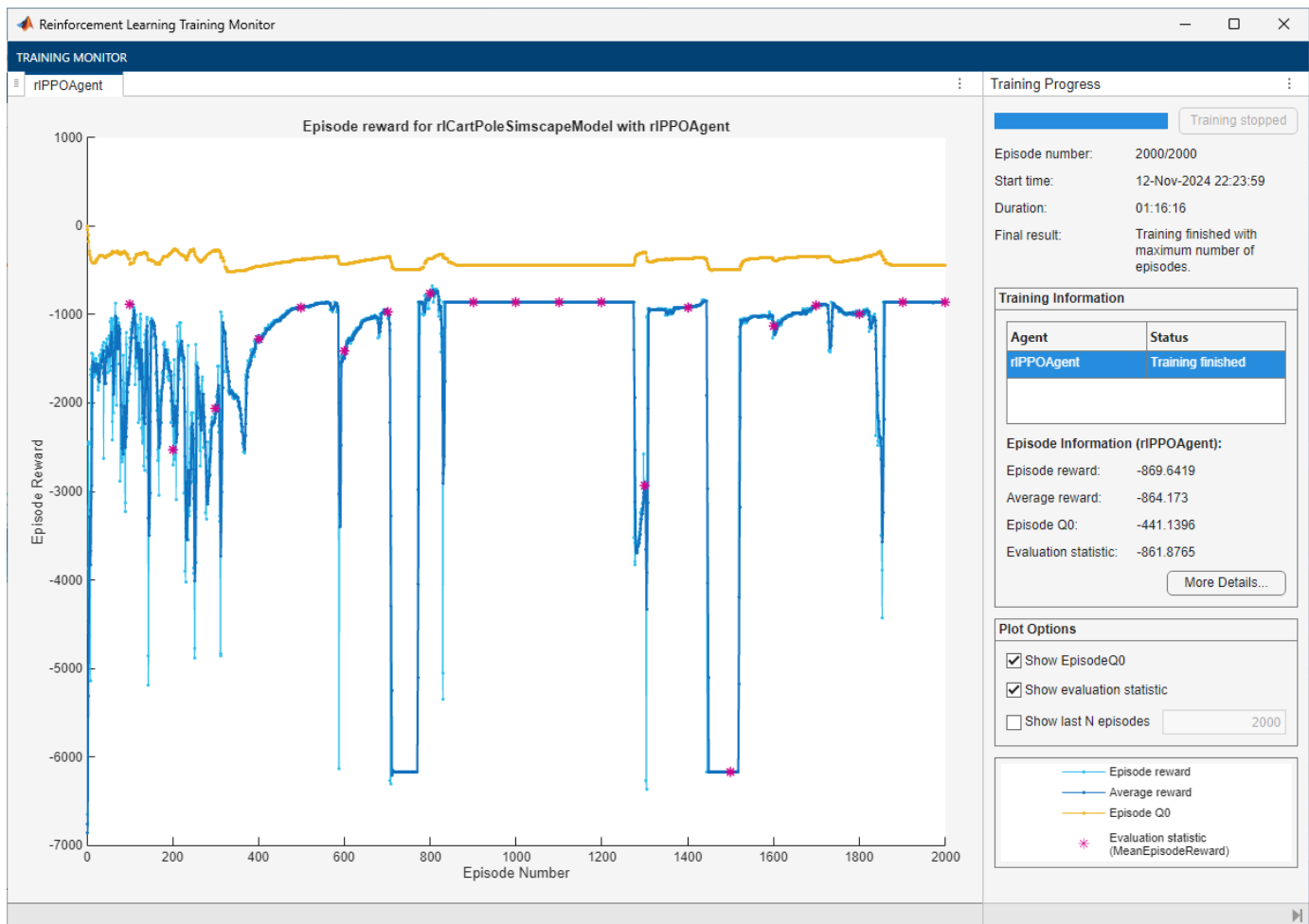
```
ppoAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    ppoTngRes = train(ppoAgent, env, trainOpts, Evaluator=evl);
    ppoTngTime = toc;
    % Extract number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dscpsuBchPPOAgent.mat", ...
    %     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dscpsuBchPPOAgent.mat", ...
        "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
end
```



For the PPO Agent, the training does not converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ppoAgent, simOptions);
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =
-864.8414
```

The trained PPO agent is not able to swing up the pole.

Create, Train, and Simulate a SAC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `sacAgent`.

```
sacAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

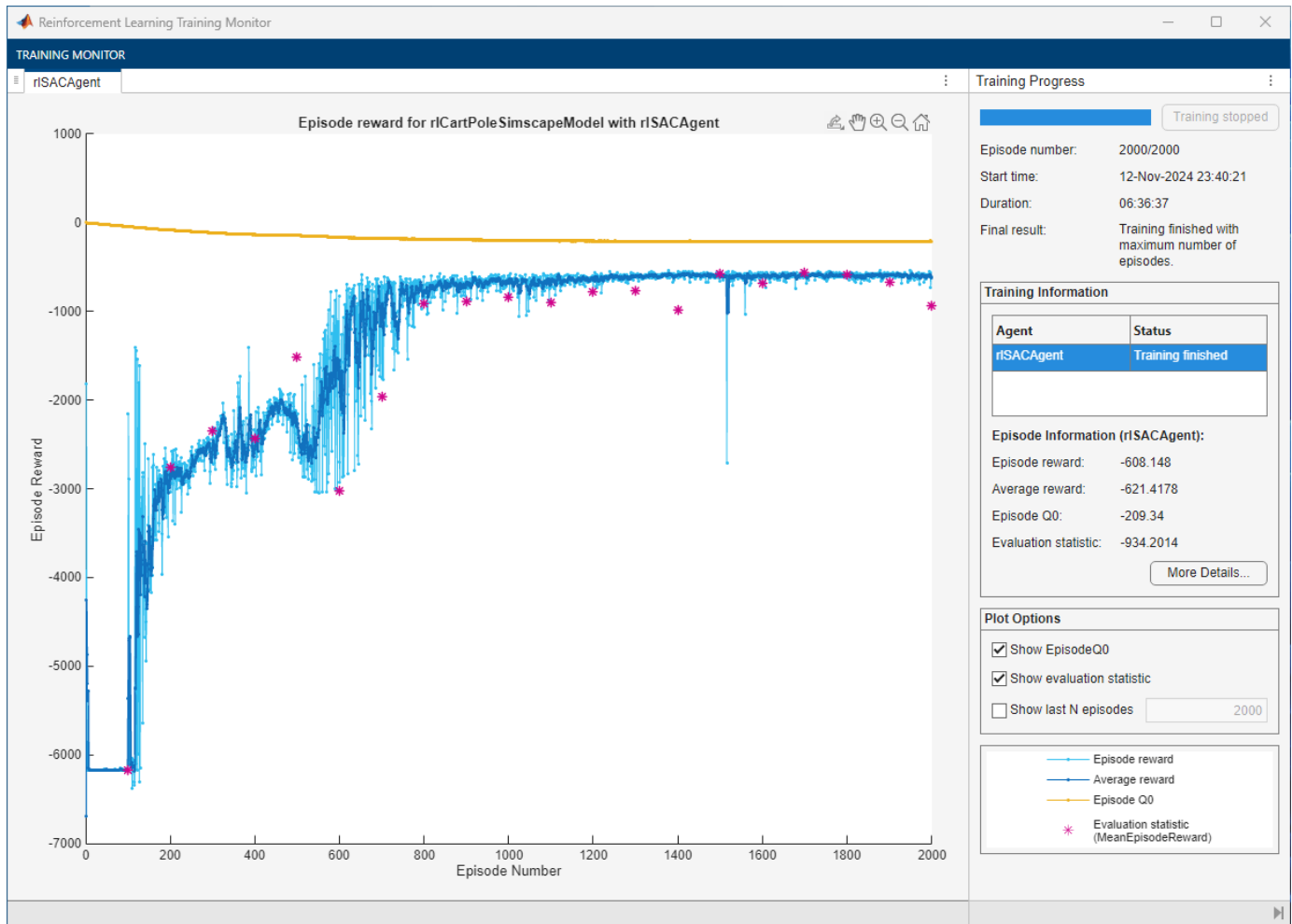
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
    sacTngTime = toc;
    % Extract number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("dscpsuBchSACAgent.mat", ...
    %     "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("dscpsuBchSACAgent.mat", ...
```

```

    "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")
end

```



For the SAC agent, the training does not seem to converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on simulation options, see `rlSimulationOptions` and `sim`.

```
experience = sim(env, sacAgent, simOptions);
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =
-390.8948
```

The trained SAC agent is able to swing-up and maintain the pole somewhat upright, but it is not able to stabilize it as the DQN agent does.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [
    dqnTotalRwd
    pgTotalRwd
    acTotalRwd
    ppoTotalRwd
    sacTotalRwd
];

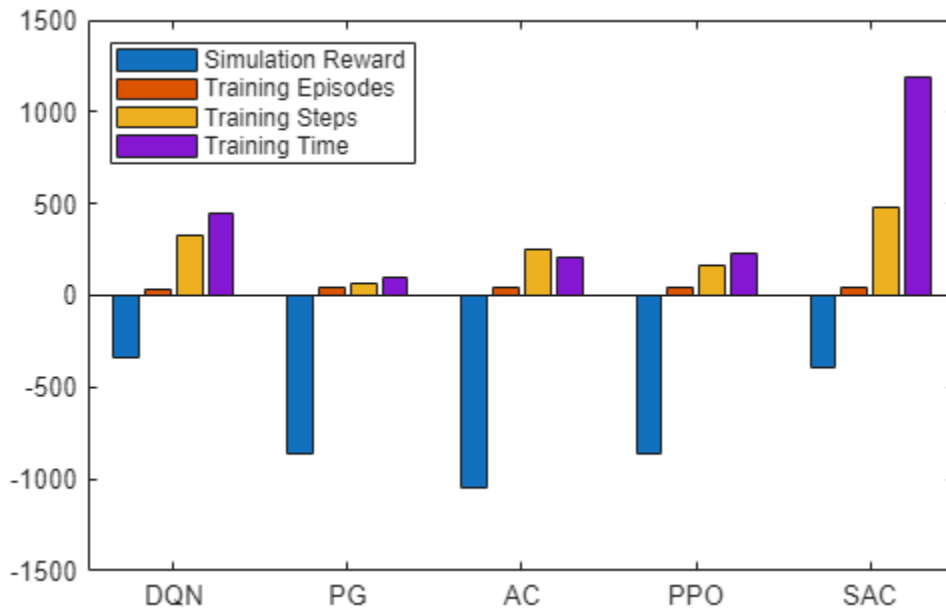
tngEpisodes = [
    dqnTngEps
    pgTngEps
    acTngEps
    ppoTngEps
    sacTngEps
];

tngSteps = [
    dqnTngSteps
    pgTngSteps
    acTngSteps
    ppoTngSteps
    sacTngSteps
];

tngTime = [
    dqnTngTime
    pgTngTime
    acTngTime
    ppoTngTime
    sacTngTime
];
```

Plot the simulation reward, number of training episodes, number of training steps and training time. Scale the data by the factor [1 50 5e6 20] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 50 5e6 20])
xticklabels(["DQN" "PG" "AC" "PPO" "SAC"])
legend(["Simulation Reward","Training Episodes","Training Steps","Training Time"], ...
    "Location", "northwest")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions only the DQN and SAC agents are able to swing up the pole, with DQN performing better in terms of stabilization and requiring less training time (due to its simpler structure). With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables
% save dscpsuAll.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Discrete Pendulum Swing-Up with Image

This example shows how to create and train frequently used default agents on a discrete action space pendulum swing-up environment. This environment represents a simple frictionless pendulum that initially hangs in a downward position. The agent, which senses the pendulum position from an image, can apply a control torque on the pendulum, and its goal is to make the pendulum stand upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are in general better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Due to the large observation space, the consequently large size of the default neural networks used in the agents, and the fact that the maximum number of training episodes is reached for many agents (because training often does not converge), the example might take days to run, even when using a GPU. To run the complete example, set `doTraining` to true. Otherwise, to execute only the result section, keep `doTraining` to false.

```
doTraining =  ;  
if doTraining
```

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Discrete Action Space Pendulum with Image MATLAB Environment

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

For this environment:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment can take any of the five possible integer values from -2 to 2 N·m.

- The observations from the environment are the simplified grayscale image of the pendulum and the pendulum angle derivative.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42. For an example on the continuous action space version of this model, see “Train DDPG Agent with Custom Networks Using Image Observation” on page 5-122.

Create Environment Object

Create a predefined environment object for the discrete pendulum environment.

```
env = rlPredefinedEnv("SimplePendulumWithImage-discrete")
```

The environment reset function initializes and returns the environment state (linear and angular positions and velocities).

```
reset(env)
```

You can visualize the pendulum system using the `plot` function during training or simulation.

```
plot(env)
```

Get the observation and action specification objects.

```
obsInfo = getObservationInfo(env)
actInfo = getActionInfo(env)
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run the training for a maximum of 5000 episodes, with each episode lasting 500 time steps.
- Stop the training when the agent receives an average cumulative reward greater than -1000 over the consecutive evaluation episodes. At this point, the agent can quickly balance the pendulum upright using minimal control effort.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
```

```
MaxStepsPerEpisode=500, ...  
StopTrainingCriteria="EvaluationStatistic", ...  
StopTrainingValue=-1000);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a DQN Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Create a default `rlDQNAgent` object using the environment specification objects.

```
dqnAgent = rlDQNAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
dqnAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
dqnAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
dqnAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0,"threefry")  
    % Use GPU for critic gradient calculations  
    critic = getCritic(dqnAgent);  
    critic.UseDevice = "gpu";  
    setCritic(dqnAgent,critic);  
end
```

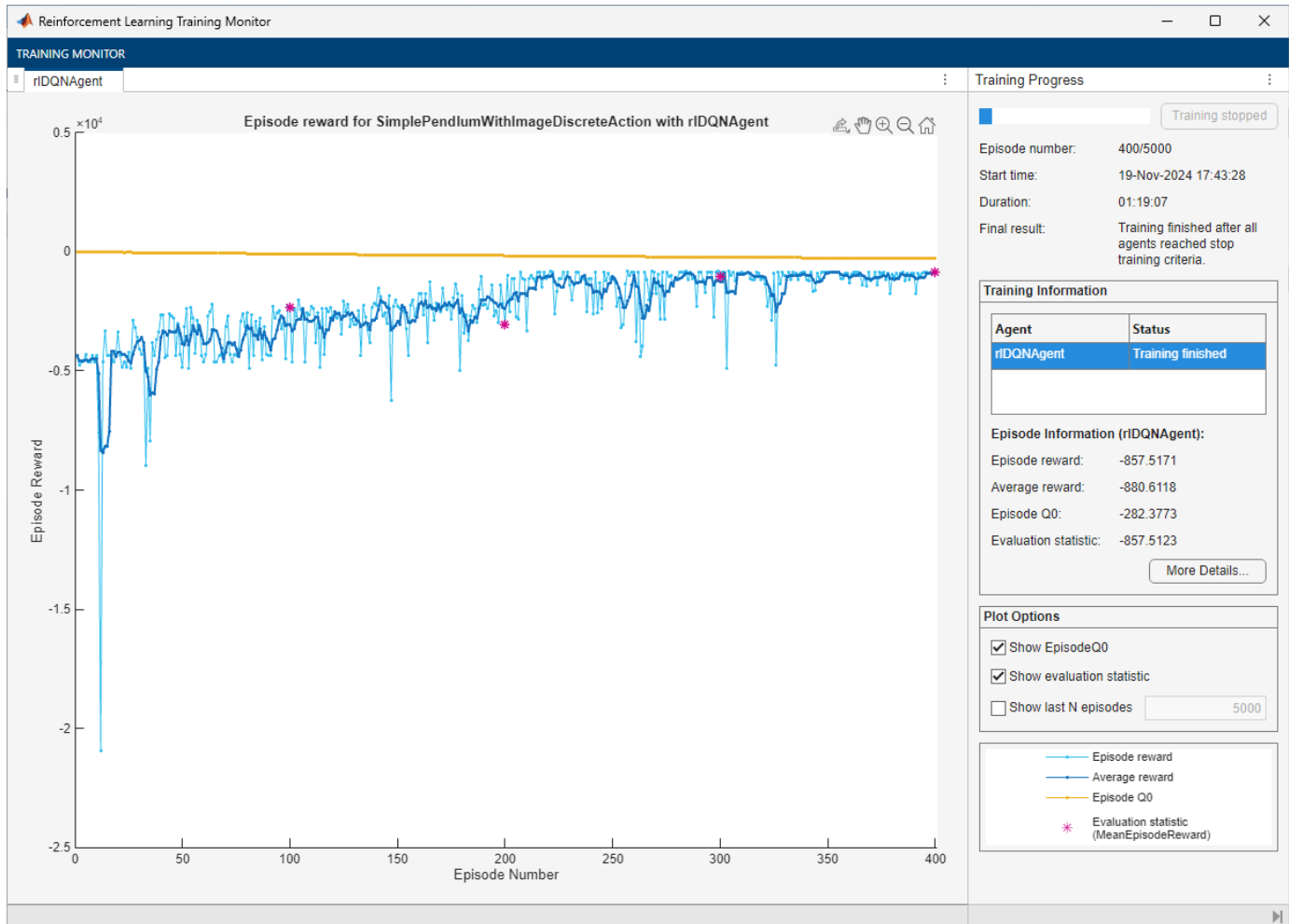
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-discrete");  
% Train the agent. Record the training time.  
tic  
dqnTngRes = train(dqnAgent,env,trainOpts,Evaluator=evl);  
dqnTngTime = toc;  
% Extract number of training episodes and total steps.
```

```

dqnTngEps = dqnTngRes.EpisodeIndex(end);
dqnTngSteps = sum(dqnTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("dpsuBchDQNData.mat", ...
%      "dqnAgent", "dqnTngEps", "dqnTngSteps", "dqnTngTime")

```



For the DQN agent, the training converges to a solution after 400 episodes. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,dqnAgent,simOptions);  
dqnTotalRwd = sum(experience.Reward)
```

The trained DQN agent swings up the pendulum and stabilizes it upright.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

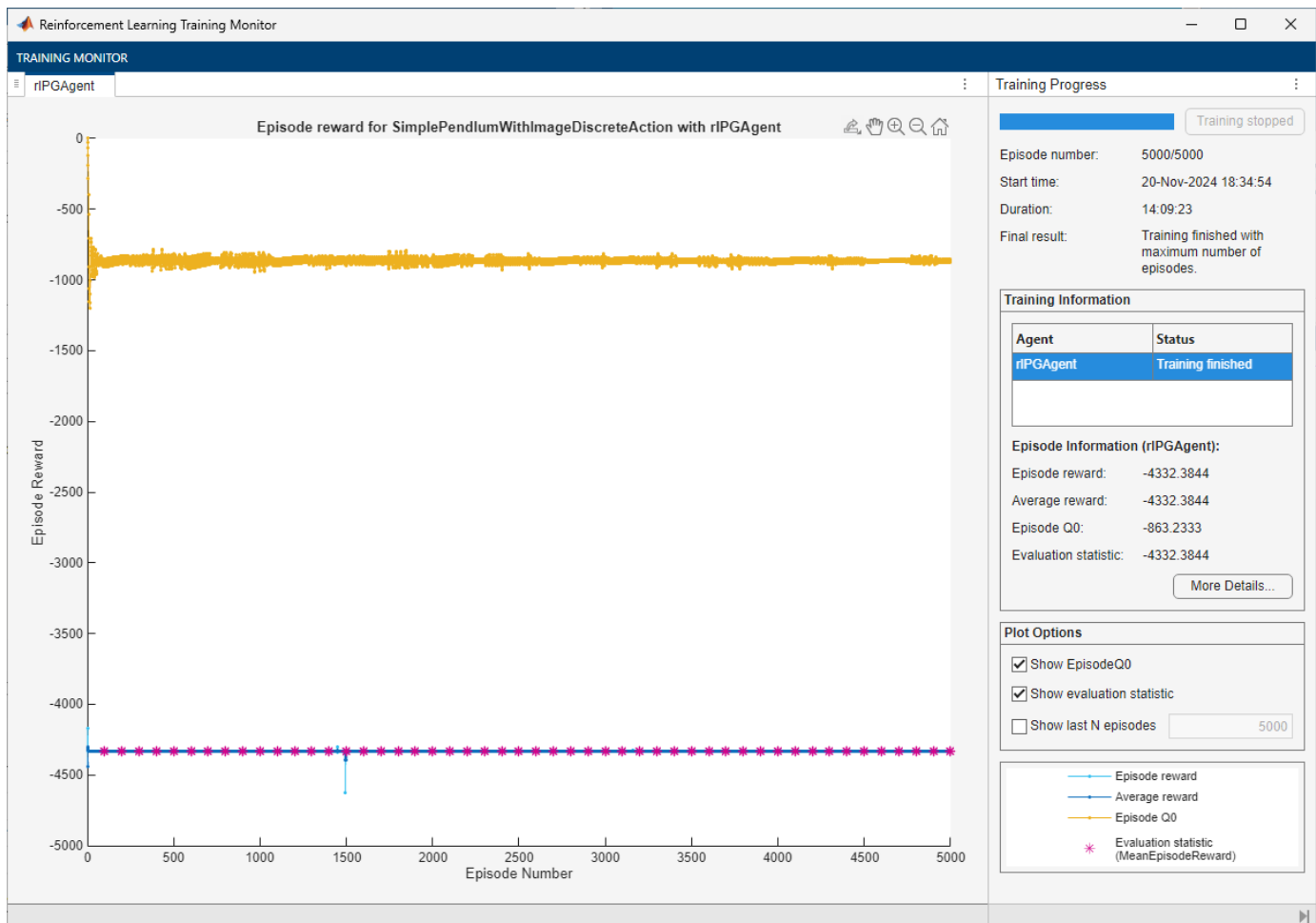
If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0,"threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(pgAgent);  
    actor.UseDevice = "gpu";  
    setActor(pgAgent,actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(pgAgent);  
    critic.UseDevice = "gpu";  
    setCritic(pgAgent,critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-discrete");  
% Train the agent. Record the training time.  
tic  
pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);  
pgTngTime = toc;  
% Extract number of training episodes and total steps.  
pgTngEps = pgTngRes.EpisodeIndex(end);  
pgTngSteps = sum(pgTngRes.TotalAgentSteps);  
% Uncomment to save the trained agent and the training metrics.
```

```
% save("dpsuBchPGData.mat", ...
%      "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
```



For the PG agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
pgTotalRwd = sum(experience.Reward)
```

The trained PG agent is not able to swing up the pendulum.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

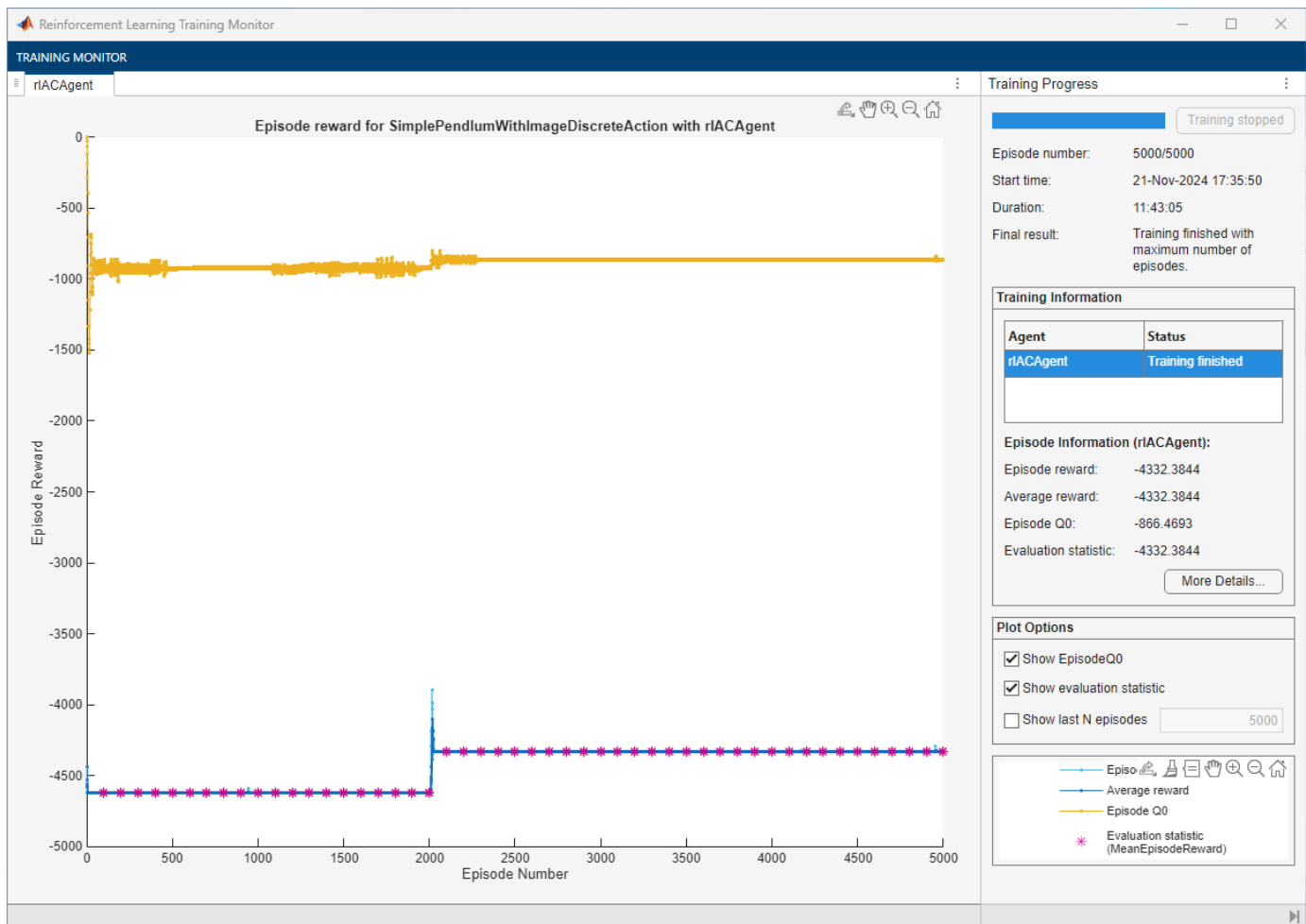
```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0,"threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(acAgent);  
    actor.UseDevice = "gpu";  
    setActor(acAgent,actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(acAgent);  
    critic.UseDevice = "gpu";  
    setCritic(acAgent,critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-discrete");  
% Train the agent. Record the training time.  
tic  
acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);  
acTngTime = toc;  
% Extract number of training episodes and total steps.  
acTngEps = acTngRes.EpisodeIndex(end);  
acTngSteps = sum(acTngRes.TotalAgentSteps);  
% Uncomment to save the trained agent and the training metrics.  
% save("dpsuBchACData.mat", ...  
%     "acAgent","acTngEps","acTngSteps","acTngTime")
```



For the AC agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
acTotalRwd = sum(experience.Reward)
```

The trained AC agent is not able to swing up the pendulum.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

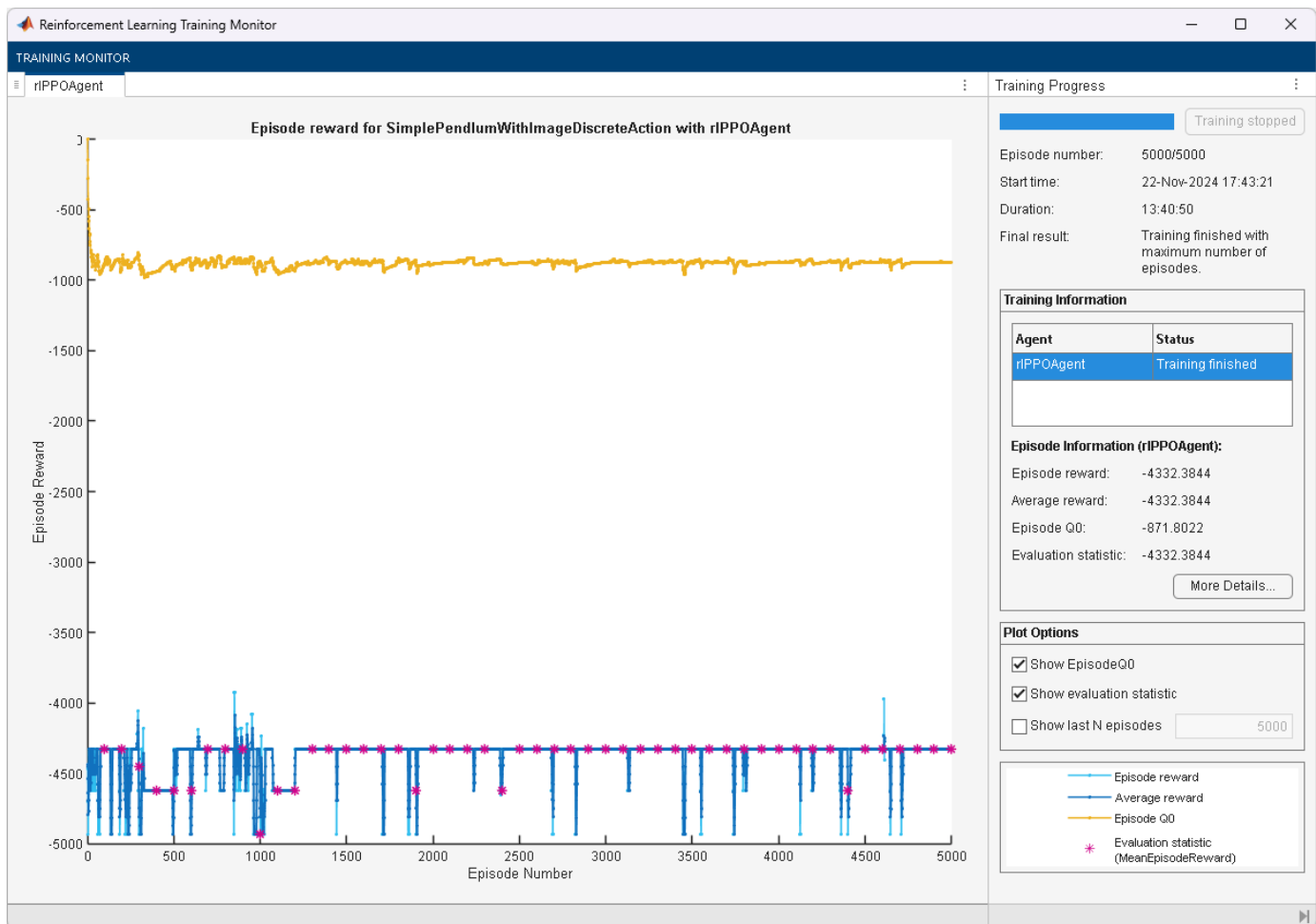
```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0, "threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(ppoAgent);  
    actor.UseDevice = "gpu";  
    setActor(ppoAgent, actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(ppoAgent);  
    critic.UseDevice = "gpu";  
    setCritic(ppoAgent, critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-discrete");  
% Train the agent. Record the training time.  
tic  
ppoTngRes = train(ppoAgent, env, trainOpts, Evaluator=evl);  
ppoTngTime = toc;  
% Extract number of training episodes and total steps.  
ppoTngEps = ppoTngRes.EpisodeIndex(end);  
ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);  
% Uncomment to save the trained agent and the training metrics.  
% save("dpsuBchPPOData.mat", ...  
%     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
```

For the PPO Agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ppoAgent, simOptions);
ppoTotalRwd = sum(experience.Reward)
```

The trained PPO agent does not swing up the pendulum.

Create, Train, and Simulate a SAC Agent

The constructor functions initialize the agent networks randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0, "threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(sacAgent);  
    actor.UseDevice = "gpu";  
    setActor(sacAgent, actor);  
    % Use GPU for critic gradient calculations  
    critics = getCritic(sacAgent);  
    critics(1).UseDevice = "gpu";  
    critics(2).UseDevice = "gpu";  
    setCritic(sacAgent, critics);  
end
```

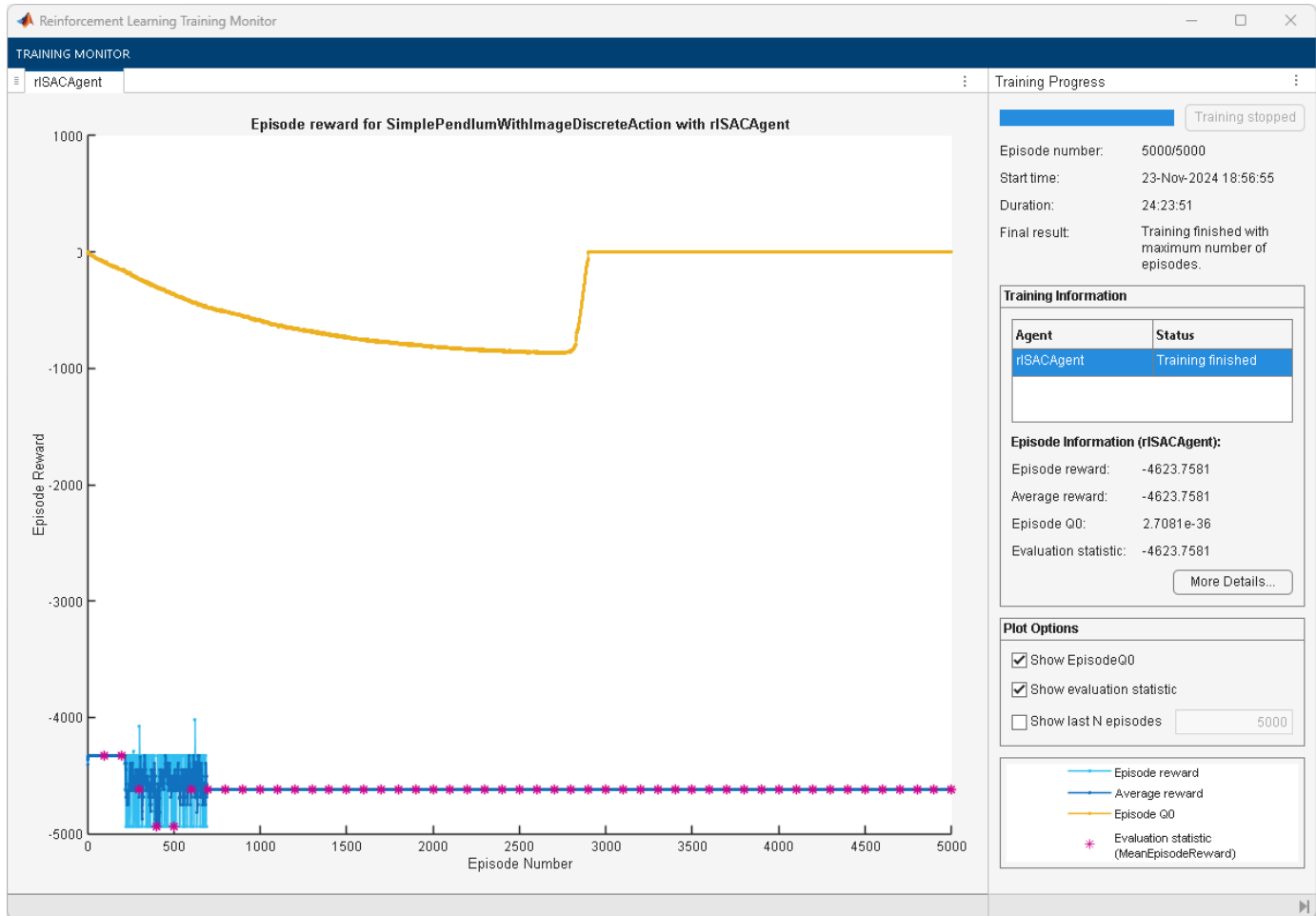
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-discrete");  
% Train the agent. Record the training time.  
tic  
sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
```

```

sacTngTime = toc;
% Extract number of training episodes and total steps.
sacTngEps = sacTngRes.EpisodeIndex(end);
sacTngSteps = sum(sacTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("dpsuBchSACData.mat", ...
%      "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")

```



For the SAC agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
experience = sim(env,sacAgent,simOptions);  
sacTotalRwd = sum(experience.Reward)
```

The trained SAC agent is not able to swing up the pendulum.

Terminate All Agents Training and Simulation Part

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables  
% save dpsuAllVariables.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

End the training and simulation part of the example.

```
end
```

Plot Training and Simulation Metrics

If the training and simulation part of the example executes, collect all the agent metrics from the existing workspace variables.

```
if doTraining
```

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the training time.

```
simReward = [  
    dqnTotalRwd  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    dqnTngEps  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    dqnTngSteps  
    pgTngSteps  
    acTngSteps  
    ppoTngSteps  
    sacTngSteps  
];
```

```
tngTime = [  
    dqnTngTime  
    pgTngTime  
    acTngTime  
    ppoTngTime
```

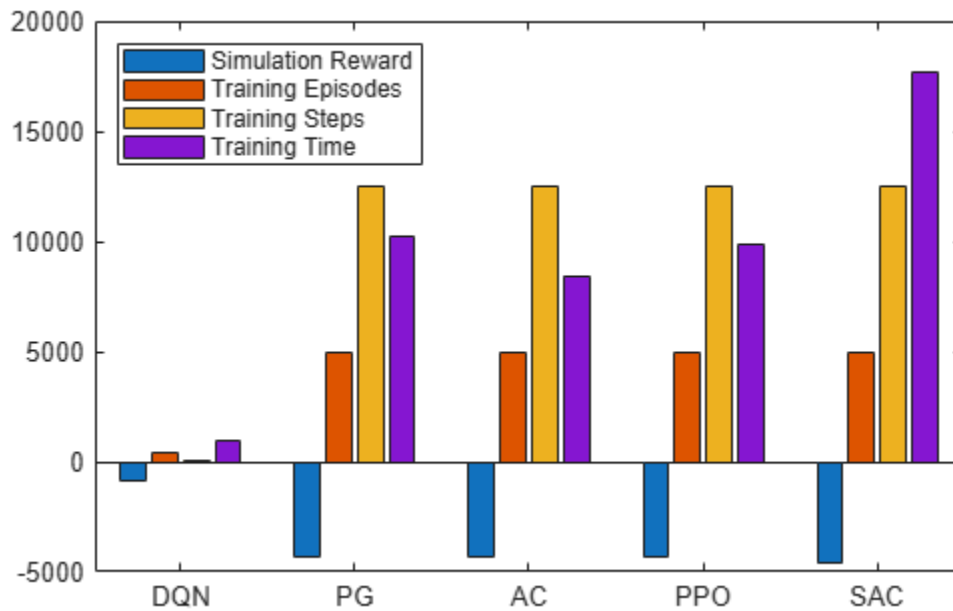
```
sacTngTime
];
```

If the training and simulation part does not execute, load the metrics from the supporting file.

```
else
load("dpsuImgBchResults.mat")
end
```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [1 10 5e5 5] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 1 5e5 5])
xticklabels(["DQN" "PG" "AC" "PPO" "SAC"])
legend( ...
["Simulation Reward", ...
"Training Episodes", ...
"Training Steps", ...
"Training Time"], ...
"Location", "northwest")
```



Only the trained DQN agent is able to swing up and stabilize the pendulum. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

See Also

Functions

rlPredefinedEnv | getActionInfo | getObservationInfo | train | sim

Objects

rlEvaluator | rlAgentInitializationOptions | rlTrainingOptions |
rlSimulationOptions

Related Examples

- “Create DQN Agent Using Deep Network Designer and Train Using Image Observations” on page 5-217

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Continuous Double-Integrator

This example shows how to create and train frequently used default agents on a continuous action space double-integrator environment. This environment is modeled in MATLAB®, and represents a mass that can move along a frictionless track. The agent can apply a force to the mass and its training goal is to stabilize the mass at the origin. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

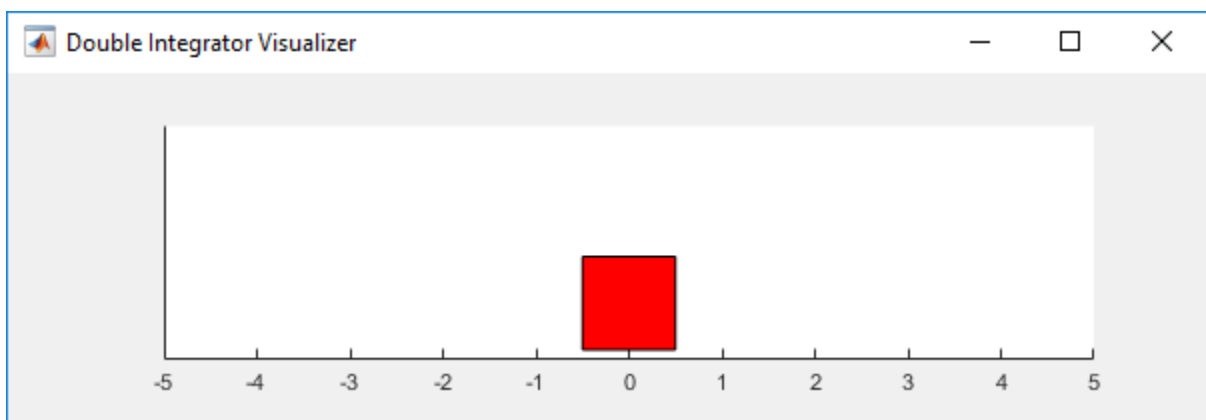
Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Double Integrator MATLAB Environment

The reinforcement learning environment for this example is a second-order double-integrator system with a gain. The training goal is to control the position of a mass in the second-order system by applying a force input.



For this environment:

- The mass starts at an initial position of either -4 or 4 units.

- The observations from the environment are the position and velocity of the mass.
- The episode terminates if the mass moves more than 5 m from the original position or if $|x| < 0.01$.
- The reward r_t , provided at every time step, is a discretization of $r(t)$:
- $r(t) = - (x(t)' Q x(t) + u(t)' R u(t))$

Here:

- x is the state vector of the mass.
- u is the force applied to the mass.
- Q is the matrix of weight on the state deviation from zero; $Q = [10 \ 0; 0 \ 1]$.
- R is the weight on the control effort; $R = 0.01$.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the continuous double-integrator environment.

```
env = rlPredefinedEnv("DoubleIntegrator-Continuous")
```

```
env =  
    DoubleIntegratorContinuousAction with properties:
```

```
        Gain: 1  
        Ts: 0.1000  
    MaxDistance: 5  
    GoalThreshold: 0.0100  
           Q: [2×2 double]  
           R: 0.0100  
    MaxForce: Inf  
       State: [2×1 double]
```

The object has a continuous action space where the agent can apply any force to the mass.

The environment reset function initializes (randomly) and returns the environment state (position and velocity).

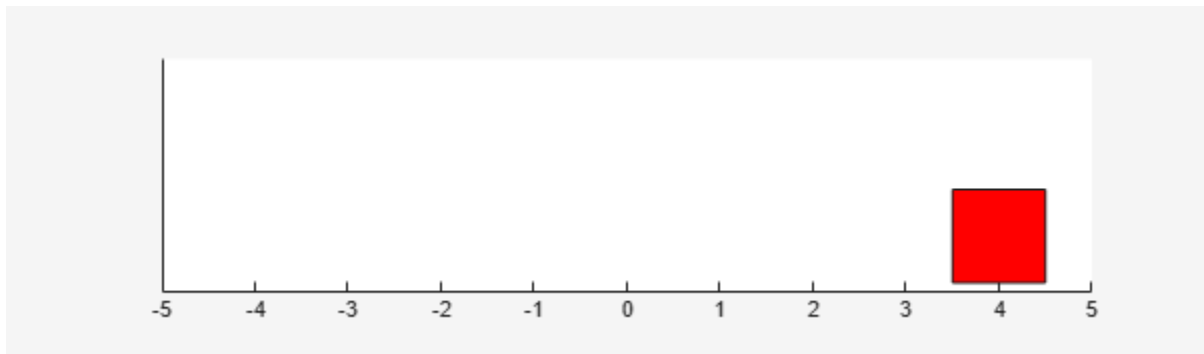
```
reset(env)
```

```
ans = 2×1
```

```
    4  
    0
```

You can visualize the double integrator system during training or simulation using the `plot` function.

```
plot(env)
```

Obtain the observation and action information for later use when creating agents.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "states"
    Description: "x, dx"
    Dimension: [2 1]
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "force"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent ten times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of 200 time steps.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.
- Stop the training when the average cumulative reward over the evaluation episodes is greater than -80. At this point, the agent can control the position of the mass.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=200, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-80);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 200 steps.

```
simOptions = rlSimulationOptions(MaxSteps=200);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

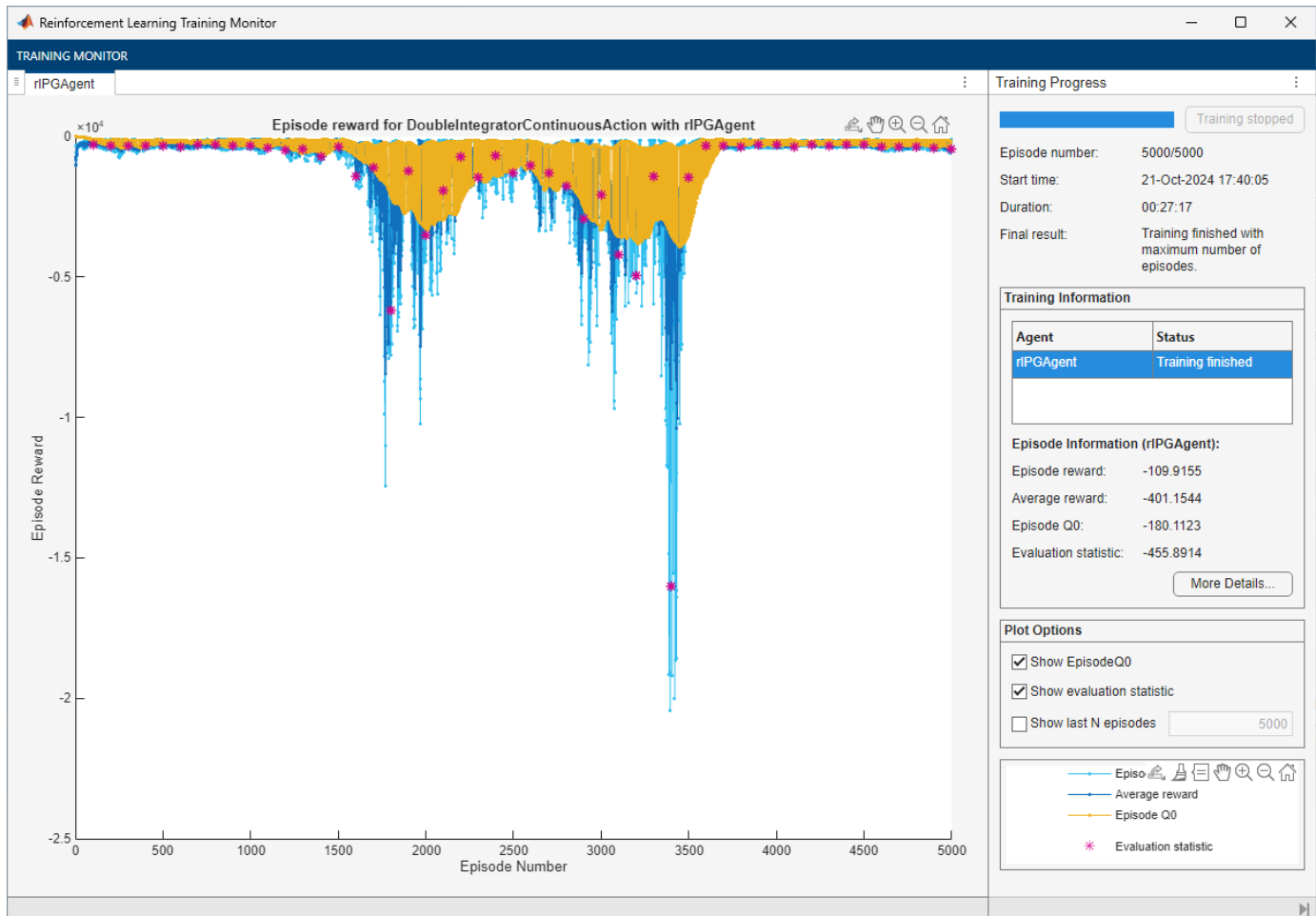
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Recreate the environment so it does not plot during training.
    env = rlPredefinedEnv("DoubleIntegrator-continuous");
    % Train the agent. Save the final agent and training results.
    tic
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cdiBchPGAgent.mat", ...
    %     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
```

```

else
    % Load the pretrained agent and results for the example.
    load("cdiBchPGAgent.mat", ...
        "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
end

```



For the PG agent, the training does not converge to a solution. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

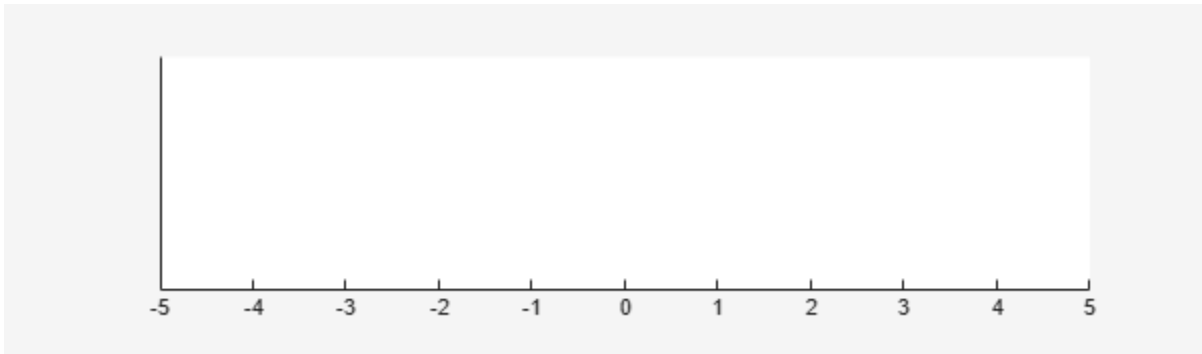
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-110.9510
```

The trained PG agent does not stabilize the mass at the origin.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

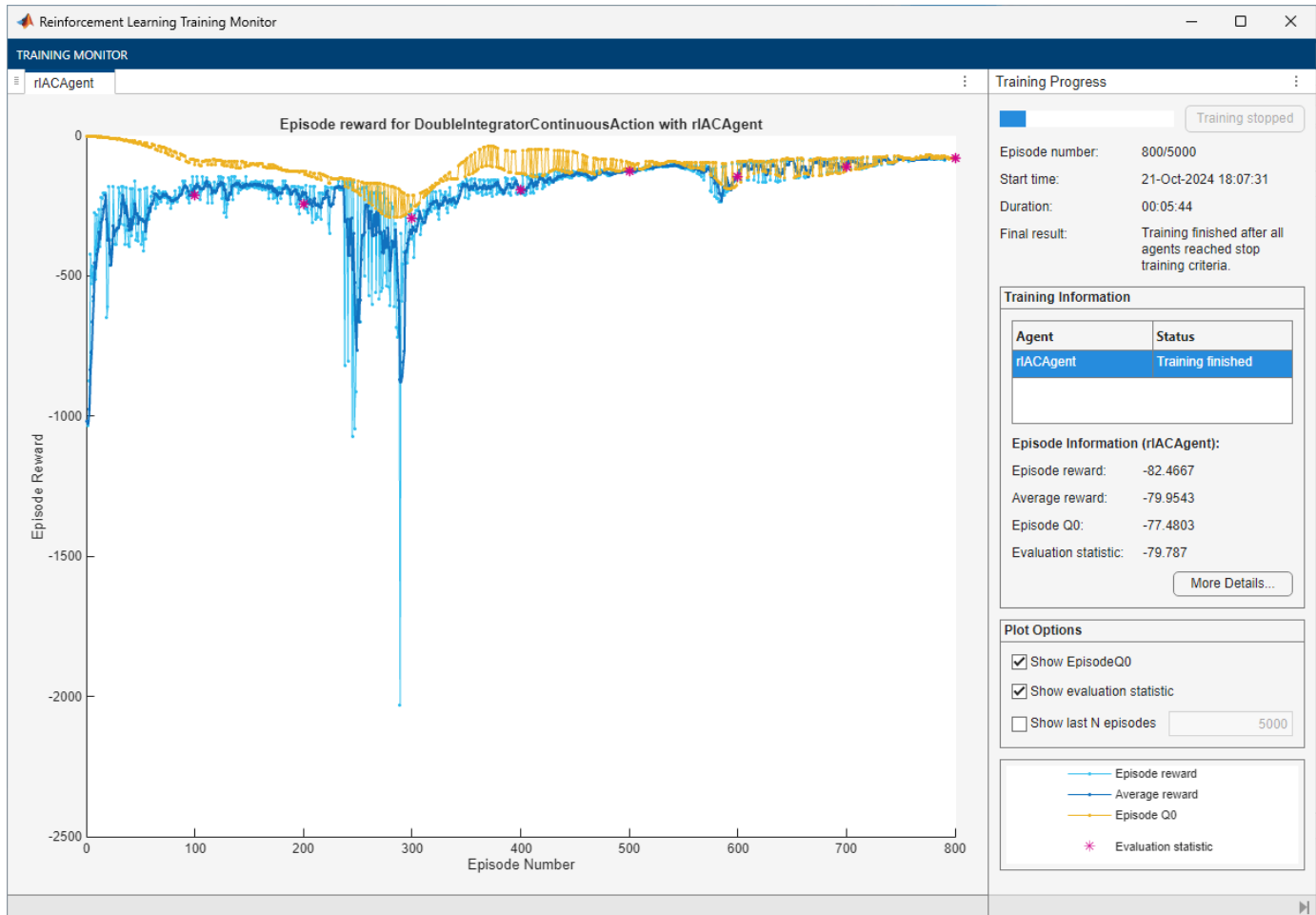
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Recreate the environment so it does not plot during training.  
    env = rlPredefinedEnv("DoubleIntegrator-continuous");  
    % Train the agent. Save the final agent and training results.  
    tic  
    acTngRes = train(acAgent, env, trainOpts, Evaluator=evl);  
    acTngTime = toc;  
    % Extract number of training episodes and total steps.
```

```

acTngEps = acTngRes.EpisodeIndex(end);
acTngSteps = sum(acTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cdiBchACAgent.mat", ...
%      "acAgent", "acTngEps", "acTngSteps", "acTngTime")
else
% Load the pretrained agent and results for the example.
load("cdiBchACAgent.mat", ...
     "acAgent", "acTngEps", "acTngSteps", "acTngTime")
end

```



For the AC agent, the training converges to a solution after 800 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

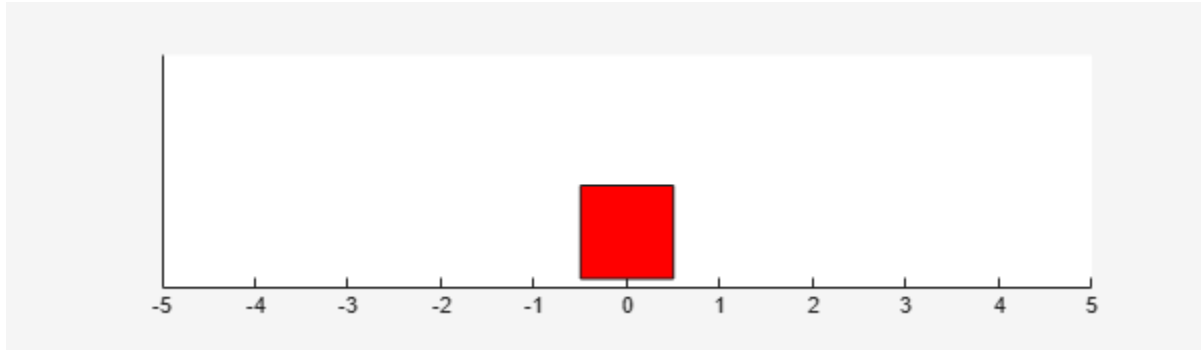
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,acAgent,simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
-80.6300
```

The trained AC agent stabilizes the mass at the origin.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

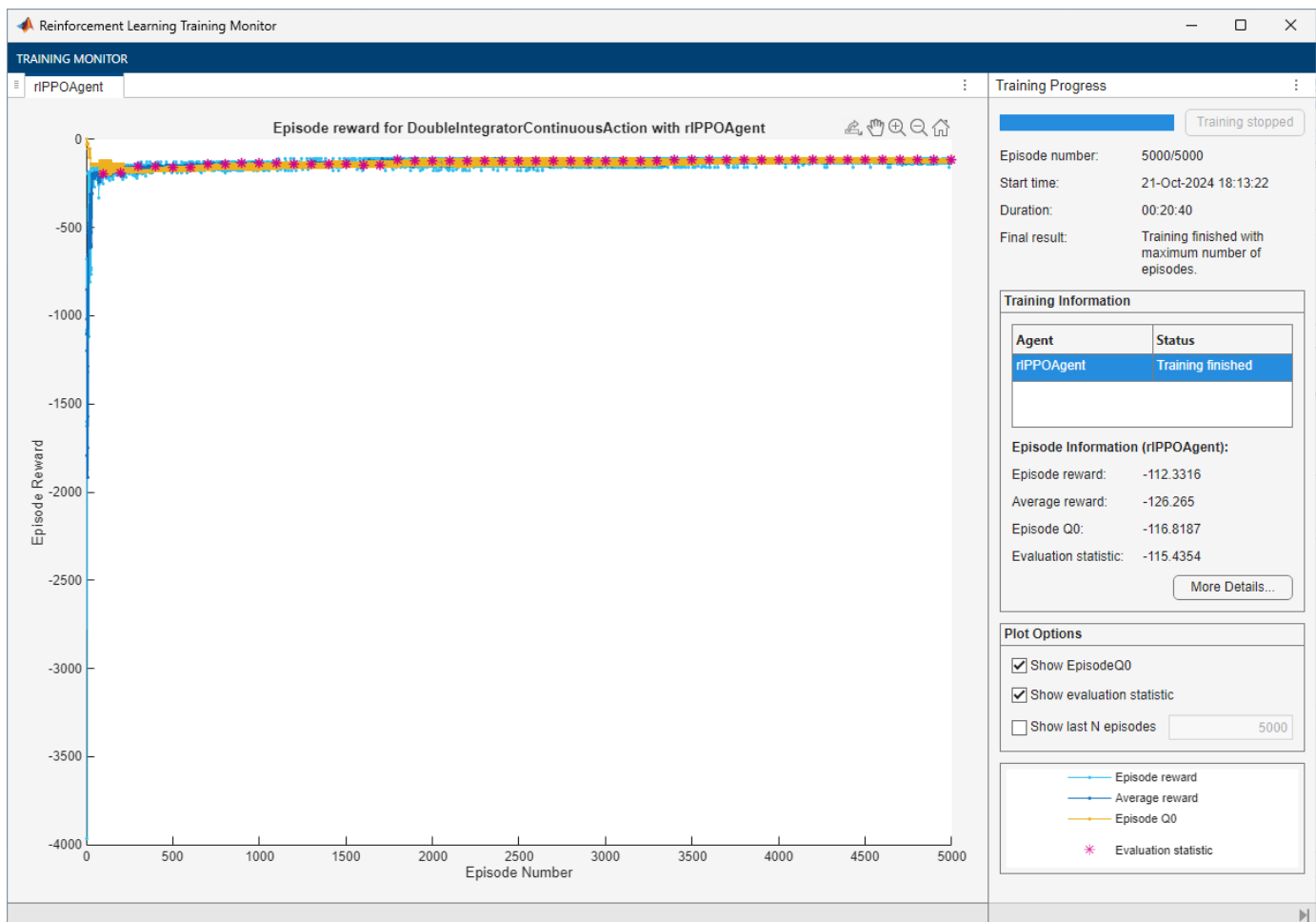
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Recreate the environment so it does not plot during training.  
    env = rlPredefinedEnv("DoubleIntegrator-continuous");  
    % Train the agent. Save the final agent and training results.  
    tic  
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);  
    ppoTngTime = toc;
```

```

% Extract number of training episodes and total steps.
ppoTngEps = ppoTngRes.EpisodeIndex(end);
ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cdiBchPPOAgent.mat", ...
%      "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
else
% Load the pretrained agent and results for the example.
load("cdiBchPPOAgent.mat", ...
     "ppoAgent", "ppoTngEps", "ppoTngSteps", "ppoTngTime")
end

```



For the PPO Agent, the training does not converge to a solution. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

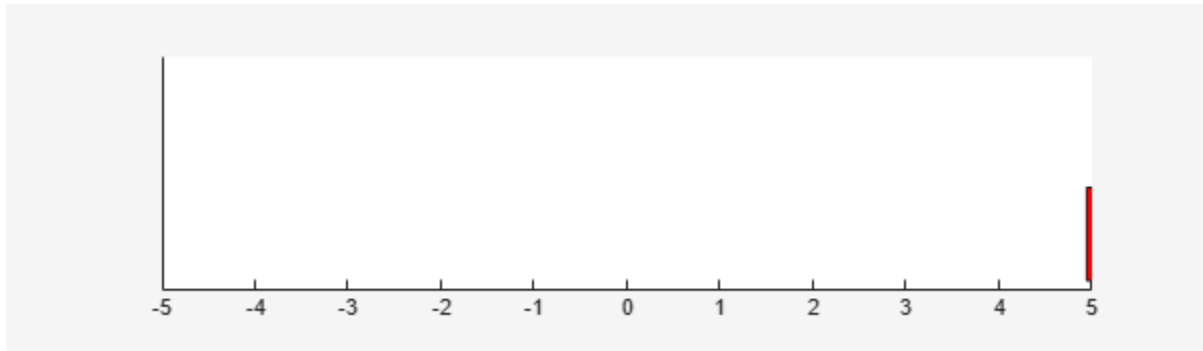
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,ppoAgent,simOptions);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-131.0296
```

The trained PPO agent does not stabilize the mass at the origin.

Create, Train, and Simulate a DDPG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rLDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rLDDPGAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

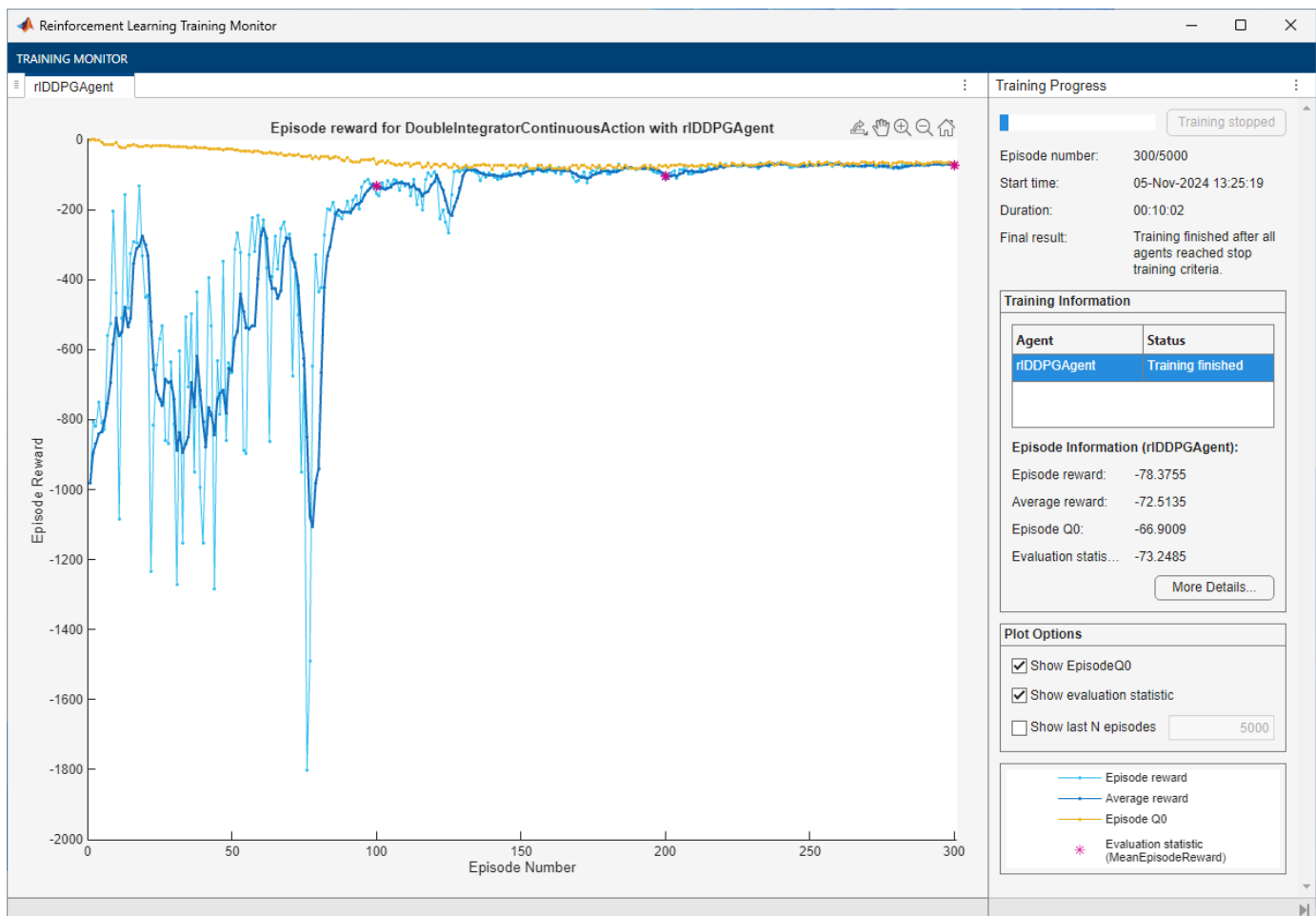
```
ddpgAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.


```

doTraining = ;
if doTraining
    % Recreate the environment so it does not plot during training.
    env = rlPredefinedEnv("DoubleIntegrator-Continuous");
    % Train the agent. Save the final agent and training results.
    tic
    ddpdTngRes = train(ddpgAgent,env,trainOpts,Evaluator=evl);
    ddpdTngTime = toc;
    % Extract number of training episodes and total steps.
    ddpdTngEps = ddpdTngRes.EpisodeIndex(end);
    ddpdTngSteps = sum(ddpgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cdiBchDDPGAgent.mat", ...
    %      "ddpgAgent","ddpgTngEps","ddpgTngSteps","ddpgTngTime")
else
    % Load the pretrained agent and results for the example.
    load("cdiBchDDPGAgent.mat", ...
        "ddpgAgent","ddpgTngEps","ddpgTngSteps","ddpgTngTime")
end

```



For the DDPG Agent, the training converges to a solution after 300 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

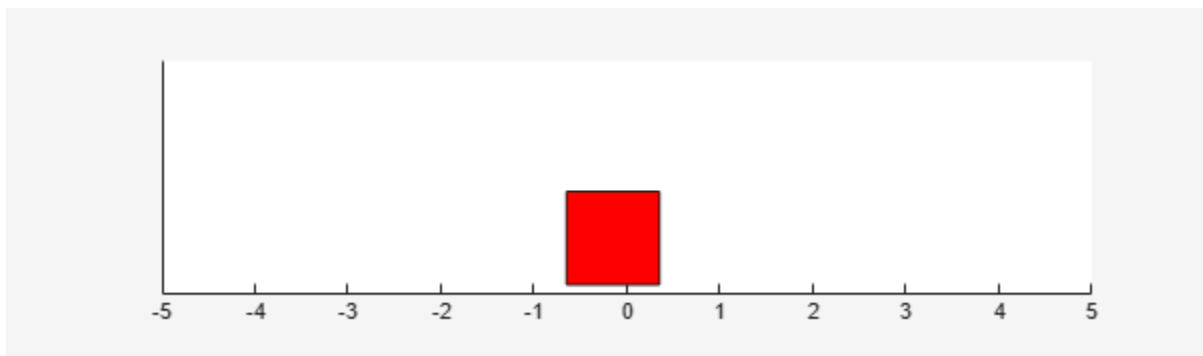
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ddpGAgent, simOptions);
```



```
ddpGTotalRwd = sum(experience.Reward)
```

```
ddpGTotalRwd =  
-78.1618
```

The trained DDPG agent stabilizes the mass at the origin.

Create, Train, and Simulate a TD3 Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
td3Agent = rlTD3Agent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

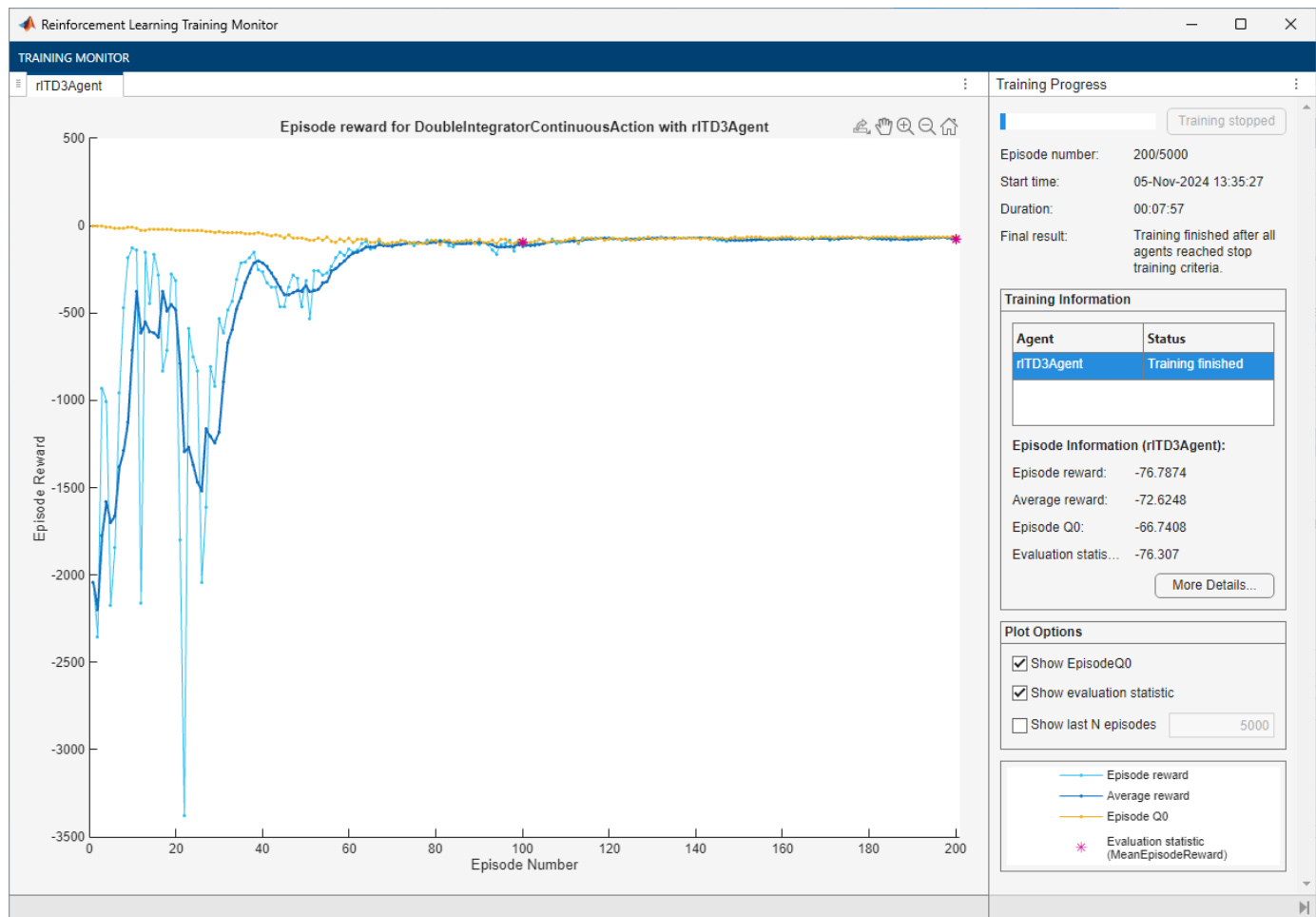
```
td3Agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
td3Agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
td3Agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
td3Agent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Recreate the environment so it does not plot during training.
    env = rlPredefinedEnv("DoubleIntegrator-Continuous");
    % Train the agent. Save the final agent and training results.
    tic
    td3TngRes = train(td3Agent,env,trainOpts,Evaluator=evl);
    td3TngTime = toc;
    % Extract number of training episodes and total steps.
    td3TngEps = td3TngRes.EpisodeIndex(end);
    td3TngSteps = sum(td3TngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cdiBchTD3Agent.mat", ...
    %     "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
else
    % Load the pretrained agent and results for the example.
    load("cdiBchTD3Agent.mat", ...
        "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
end
```



For the TD3 Agent, the training converges to a solution after 200 episodes. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

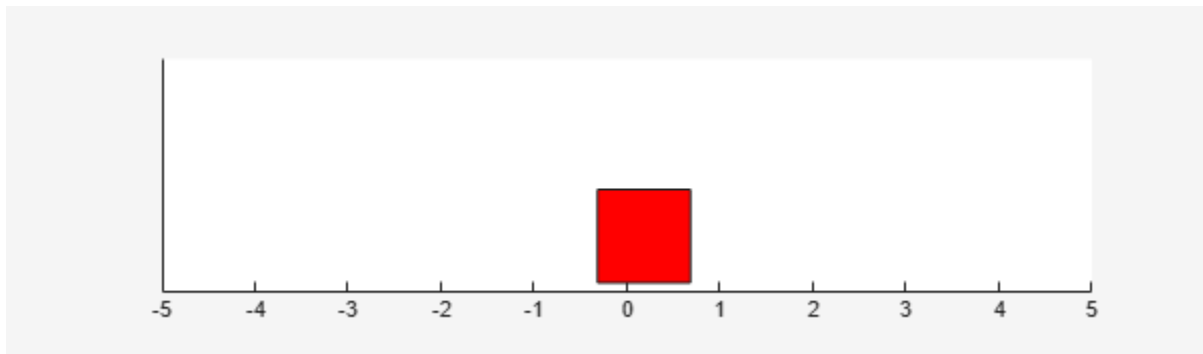
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, td3Agent, simOptions);
```



```
td3TotalRwd = sum(experience.Reward)
```

```
td3TotalRwd =  
-74.1233
```

The trained TD3 agent stabilizes the mass at the origin.

Create, Train, and Simulate a SAC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

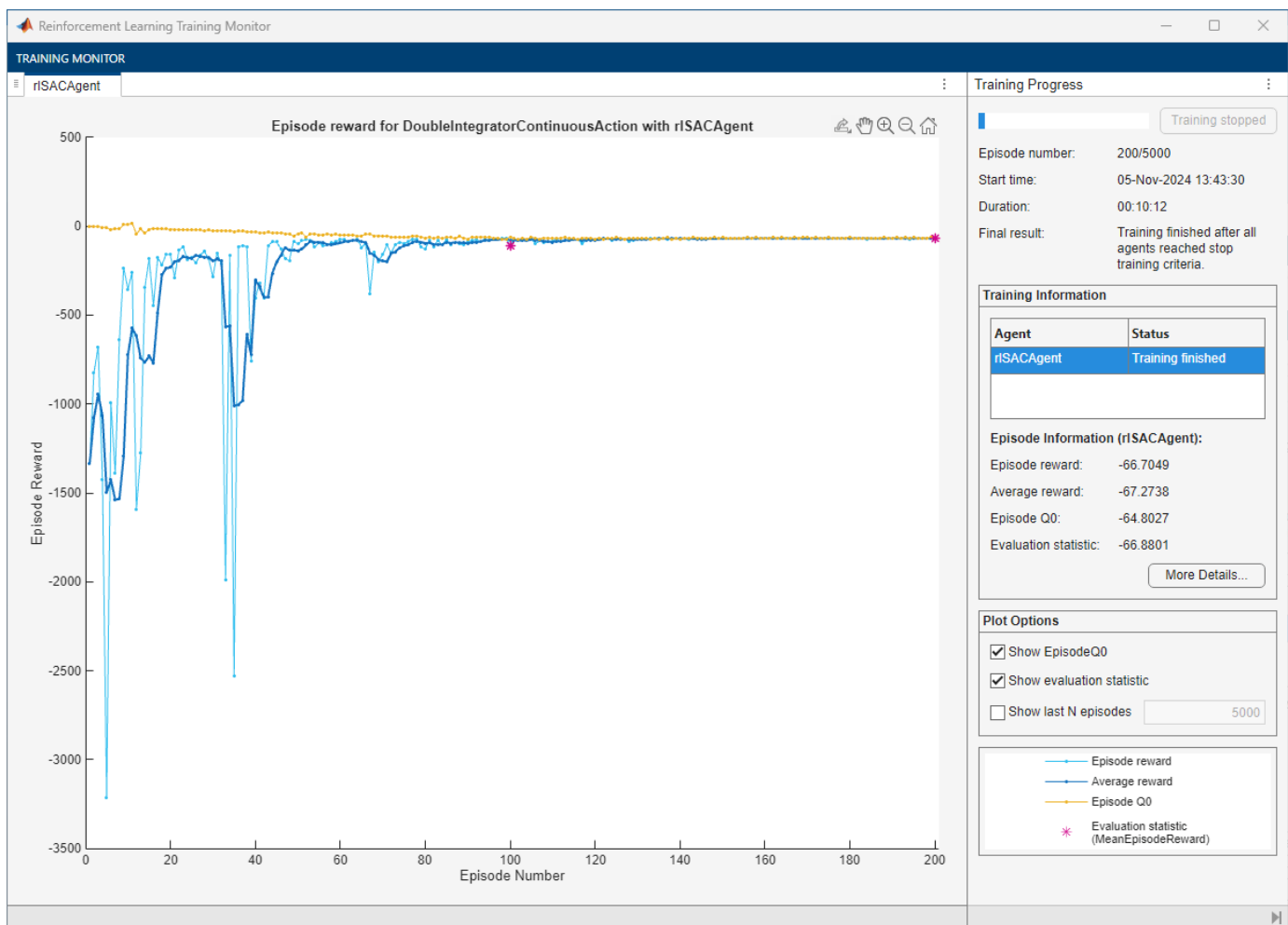
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining
```

```

% Recreate the environment so it does not plot during training.
env = rlPredefinedEnv("DoubleIntegrator-Continuous");
% Train the agent. Save the final agent and training results.
tic
sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);
sacTngTime = toc;
% Extract number of training episodes and total steps.
sacTngEps = sacTngRes.EpisodeIndex(end);
sacTngSteps = sum(sacTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cdiBchSACAgent.mat", ...
%      "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
else
% Load the pretrained agent and results for the example.
load("cdiBchSACAgent.mat", ...
     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
end

```



For the SAC Agent, the training converges to a solution after 200 steps. You can check the trained agent within the double-integrator environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

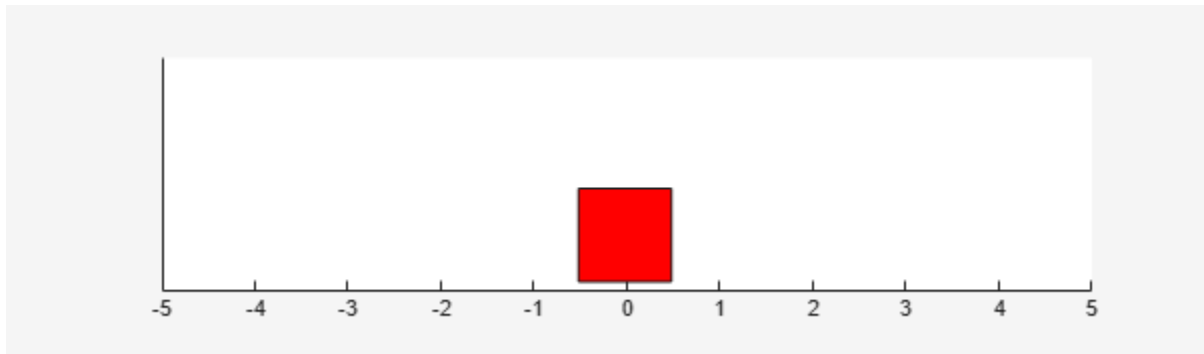
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 200 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, sacAgent, simOptions);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-66.5211
```

The trained SAC agent stabilizes the mass near the origin.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    ddpgTotalRwd  
    td3TotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    ddpgTngEps  
    td3TngEps  
    sacTngEps  
];
```

```
tngSteps = [
    pgTngSteps
    acTngSteps
    ppoTngSteps
    ddpTngSteps
    td3TngSteps
    sacTngSteps
];
```

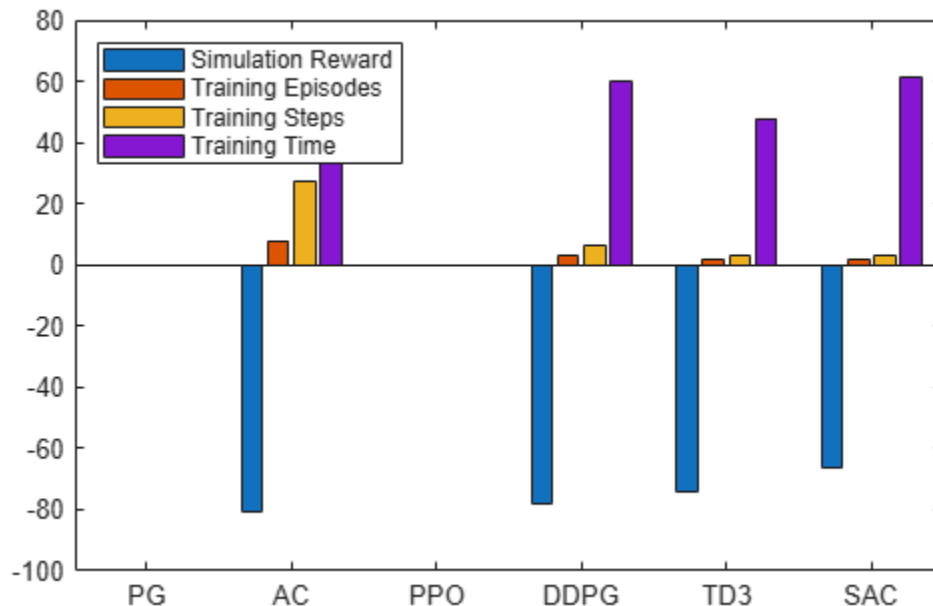
```
tngTime = [
    pgTngTime
    acTngTime
    ppoTngTime
    ddpTngTime
    td3TngTime
    sacTngTime
];
```

Because the training for the PG, and PPO agents does not converge, to avoid visualizing their metrics, set them to NaN.

```
simReward([1 3]) = NaN;
tngEpisodes([1 3]) = NaN;
tngSteps([1 3]) = NaN;
tngTime([1 3]) = NaN;
```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [1 1e2 1e6 10] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 1e2 1e6 10])
xticklabels(["PG" "AC" "PPO" "DDPG" "TD3" "SAC"])
legend(["Simulation Reward","Training Episodes","Training Steps","Training Time"], ...
    "Location","northwest")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, SAC performs slightly better in terms of total reward, with TD3 using slightly less training time than DDPG and SAC. The AC training algorithm takes more episodes but still less time than other algorithms to converge. This largely happens because it is a simpler algorithm that does not need to calculate many gradients. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables
% save cdiAllVars.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Compare DDPG Agent to LQR Controller” on page 7-285

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Continuous Cart Pole

This example shows how to create and train frequently used default agents on a continuous action space cart-pole environment. This environment is modeled in MATLAB®, and represents a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The agent can apply a force to the cart and its training goal is to balance the pole upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

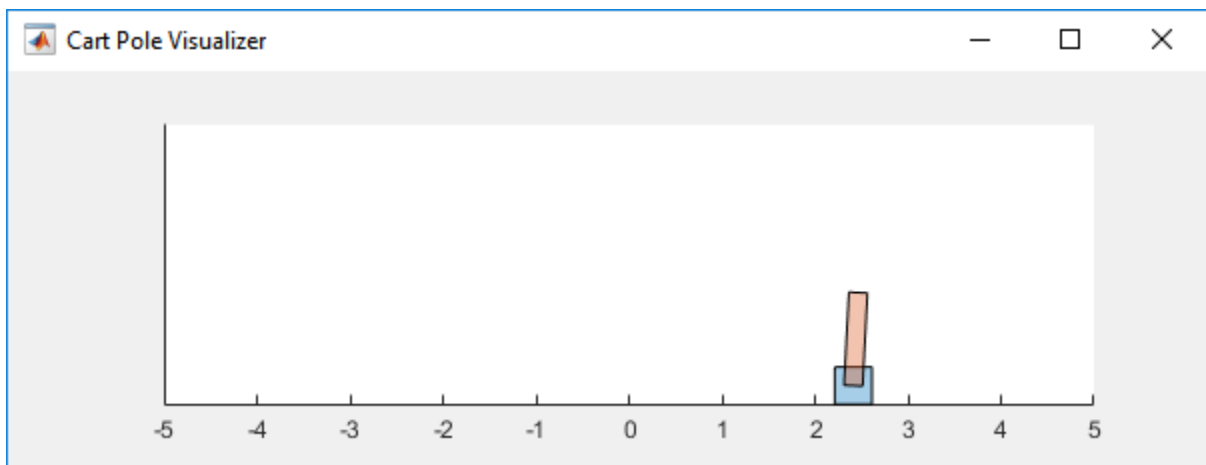
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Cart Pole MATLAB Environment

For this example, the reinforcement learning environment is a pole attached to an unactuated revolutionary joint on a cart. The cart has an actuated prismatic joint connected to a one-dimensional frictionless track. The training goal in this environment is to balance the pole by applying forces (actions) to the prismatic joint.



For this environment:

- The balanced, upright pole position is 0 radians and the downward hanging pole position is π radians.
- The pole starts upright with an initial angle between -0.05 radians and 0.05 radians.
- The force action signal from the agent to the environment is from -10 N to 10 N.
- The observations from the environment are the position and velocity of the cart, the pole angle (clockwise-positive), and the pole angle derivative.
- The episode terminates if the pole is more than 12 degrees from vertical or if the cart moves more than 2.4 m from the original position.
- A reward of +0.5 is provided for every time-step that the pole remains upright. An additional reward is provided based on the distance between the cart and the origin. A penalty of -50 is applied when the pole falls.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the continuous cart-pole environment.

```
env = rlPredefinedEnv("CartPole-Continuous")
```

```
env =
  CartPoleContinuousAction with properties:

        Gravity: 9.8000
        MassCart: 1
        MassPole: 0.1000
        Length: 0.5000
        MaxForce: 10
        Ts: 0.0200
    ThetaThresholdRadians: 0.2094
        XThreshold: 2.4000
    RewardForNotFalling: 1
    PenaltyForFalling: -50
        State: [4x1 double]
```

The environment reset function initializes (randomly) and returns the environment state (linear and angular positions and velocities).

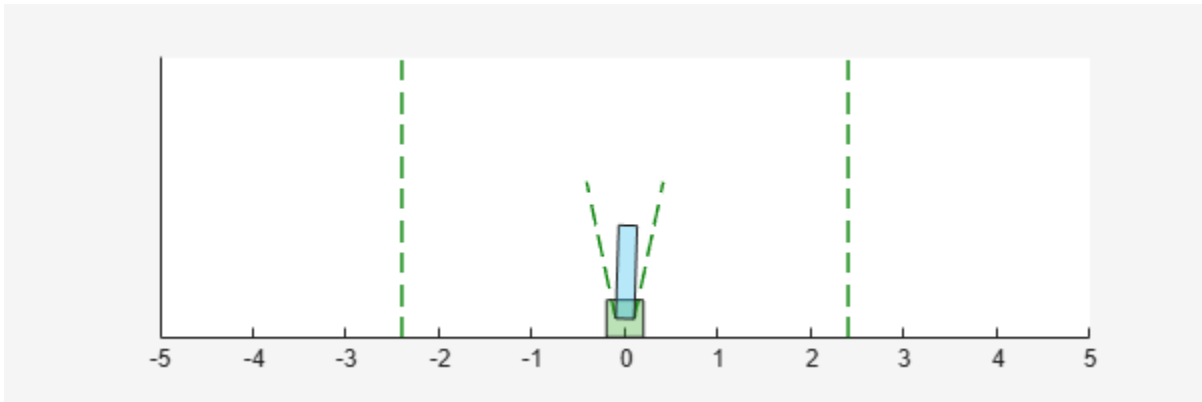
```
reset(env)
```

```
ans = 4x1

    0
    0
0.0315
    0
```

You can visualize the cart-pole system during training or simulation using the `plot` function.

```
plot(env)
```



Obtain the observation and action information for later use when creating agents.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =
  rlNumericSpec with properties:
    LowerLimit: -Inf
    UpperLimit: Inf
    Name: "CartPole States"
    Description: "x, dx, theta, dtheta"
    Dimension: [4 1]
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =
  rlNumericSpec with properties:
    LowerLimit: -10
    UpperLimit: 10
    Name: "CartPole Action"
    Description: [0x0 string]
    Dimension: [1 1]
    DataType: "double"
```

Configure Training Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of 500 time steps.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.
- Stop the training when the average cumulative reward over the evaluation episodes is greater than 480. At this point, the agent can mostly control the position of the pole.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=500, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=480);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

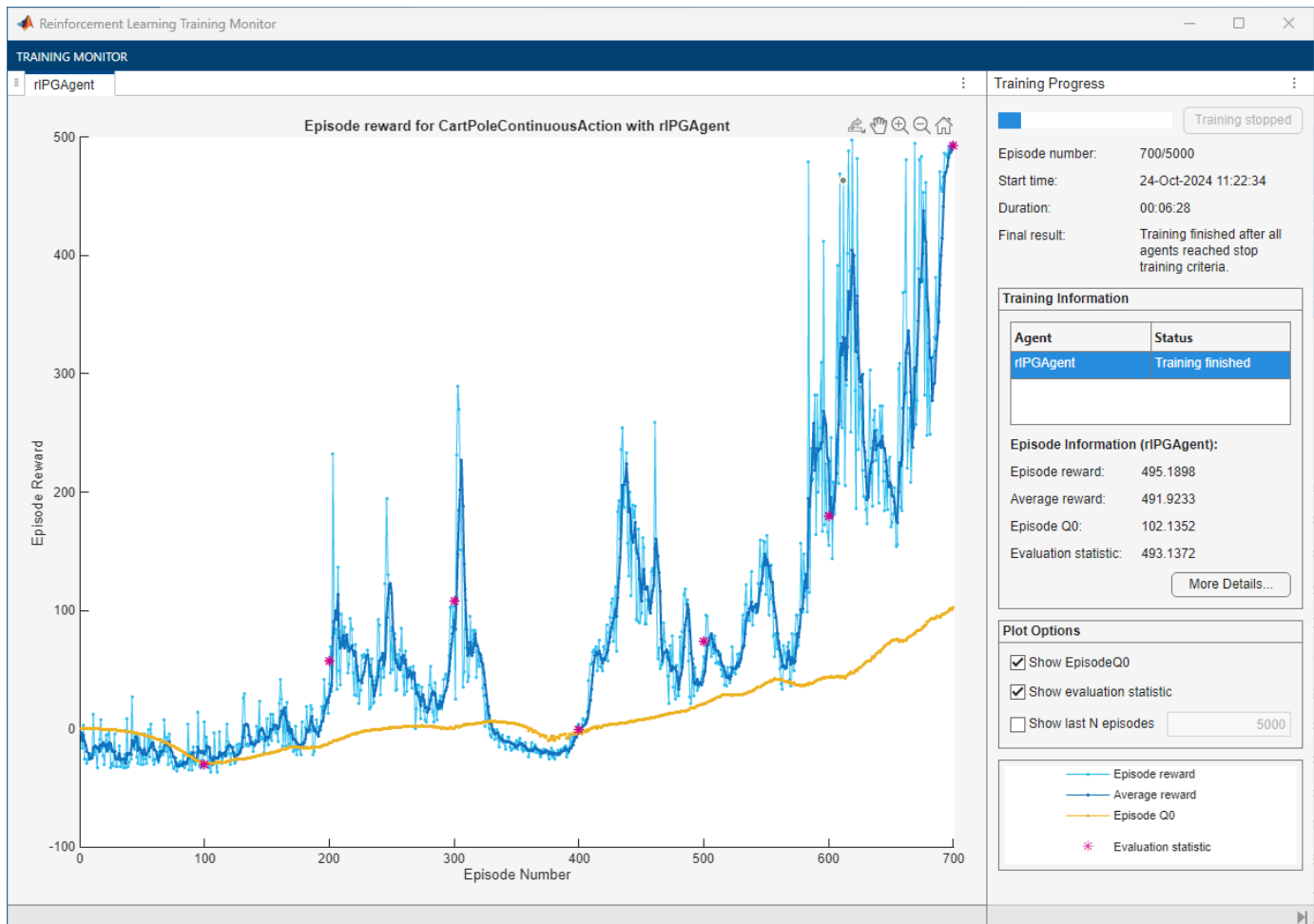
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Continuous");
    % Train the agent. Save the final agent and training results.
    tic
    pgTngRes = train(pgAgent, env, trainOpts, Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpBchPGAgent.mat", ...
    %     "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
```

```

else
    % Load the pretrained agent and results for the example.
    load("ccpBchPGAgent.mat", ...
        "pgAgent", "pgTngEps", "pgTngSteps", "pgTngTime")
end

```



For the PG agent, the training converges to a solution after 700 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

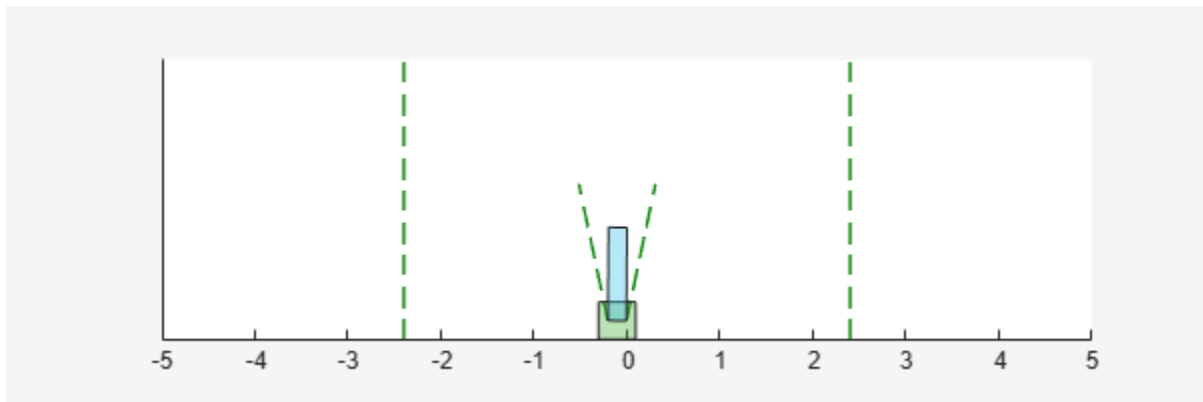
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
495.6761
```

The trained PG agent stabilizes the pole in the upright position.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

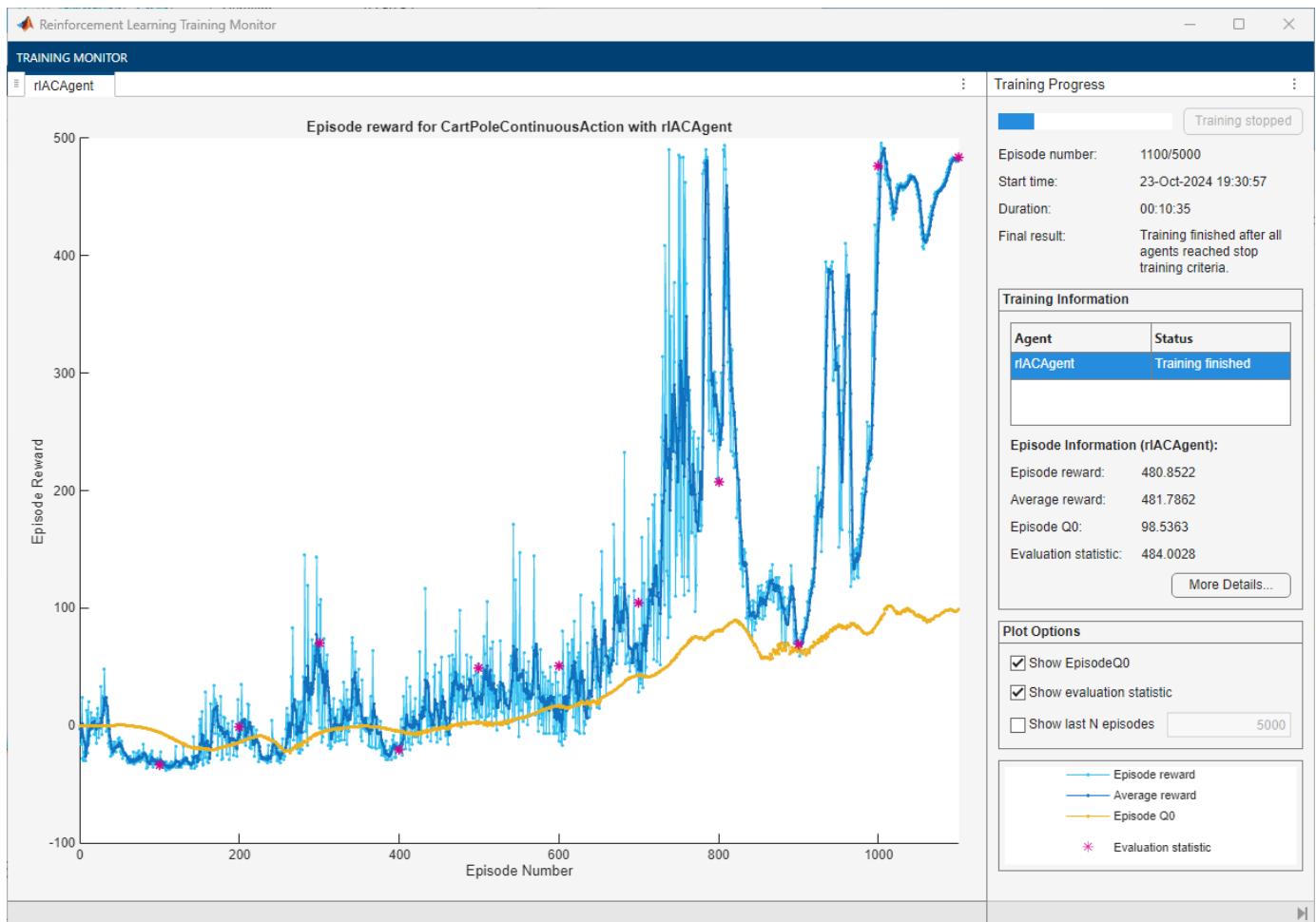
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("CartPole-Continuous");  
    % Train the agent. Save the final agent and training results.  
    tic  
    acTngRes = train(acAgent, env, trainOpts, Evaluator=evl);
```

```

acTngTime = toc;
% Extract number of training episodes and total steps.
acTngEps = acTngRes.EpisodeIndex(end);
acTngSteps = sum(acTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ccpBchACAgent.mat", ...
%      "acAgent","acTngEps","acTngSteps","acTngTime")
else
% Load the pretrained agent and results for the example.
load("ccpBchACAgent.mat", ...
     "acAgent","acTngEps","acTngSteps","acTngTime")
end

```



For the AC agent, the training converges to a solution after 1100 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

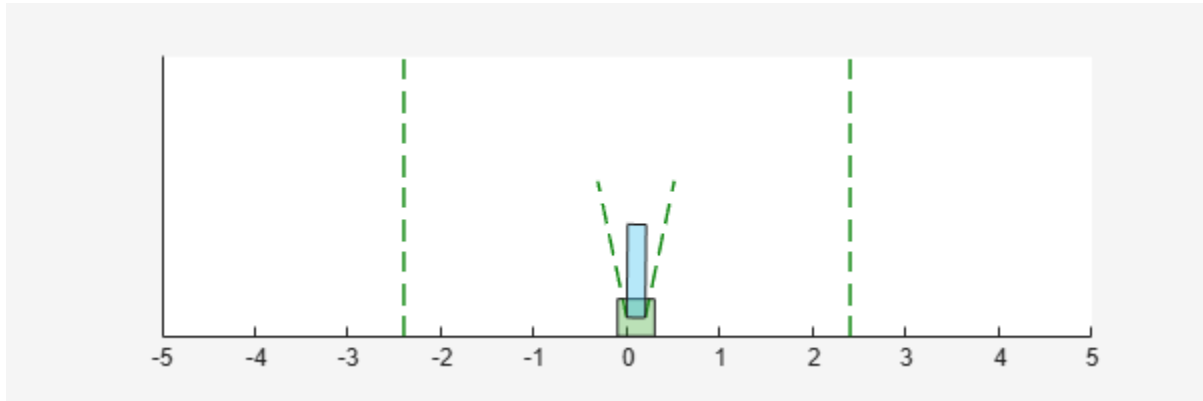
Visualize the environment.

```
plot(env)
```


By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,acAgent,simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
483.2565
```

The trained AC agent stabilizes the pole in the upright position.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

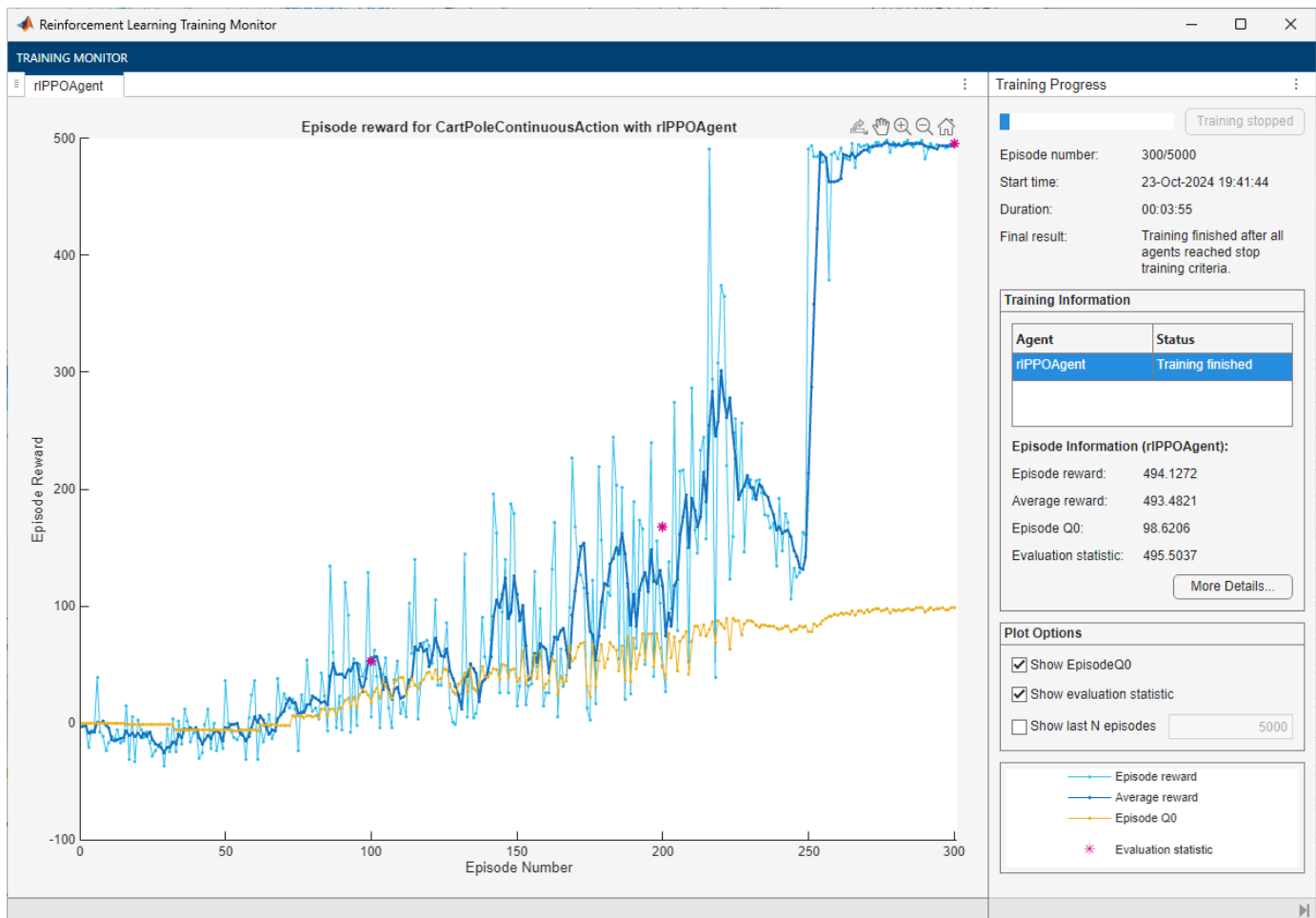
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.
```

```

env = rlPredefinedEnv("CartPole-Continuous");
% Train the agent. Save the final agent and training results.
tic
ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
ppoTngTime = toc;
% Extract number of training episodes and total steps.
ppoTngEps = ppoTngRes.EpisodeIndex(end);
ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ccpBchPPOAgent.mat", ...
%      "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
else
% Load the pretrained agent and results for the example.
load("ccpBchPPOAgent.mat", ...
     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
end

```



For the PPO Agent, the training converges to a solution after 300 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

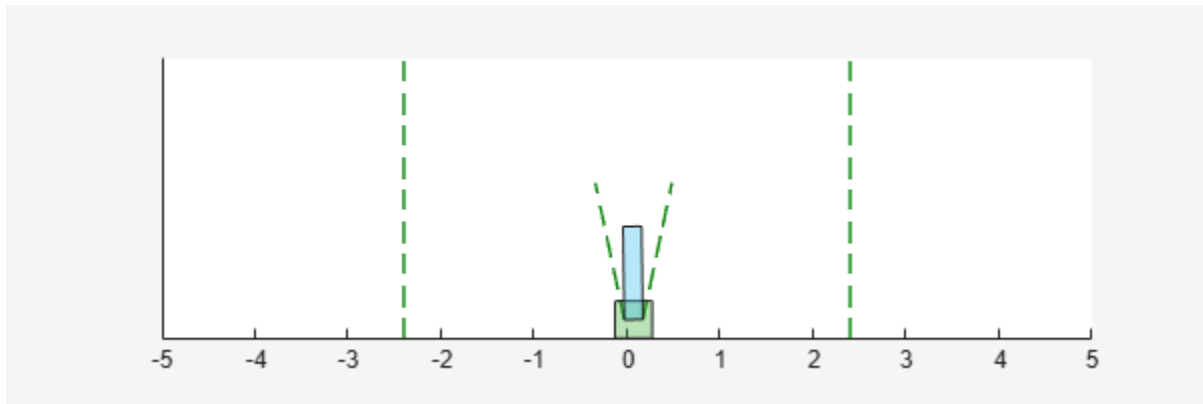
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,ppoAgent,simOptions);
```



```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
494.6438
```

The trained PPO agent stabilizes the pole in the upright position.

Create, Train, and Simulate a DDPG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rLDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rLDDPGAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

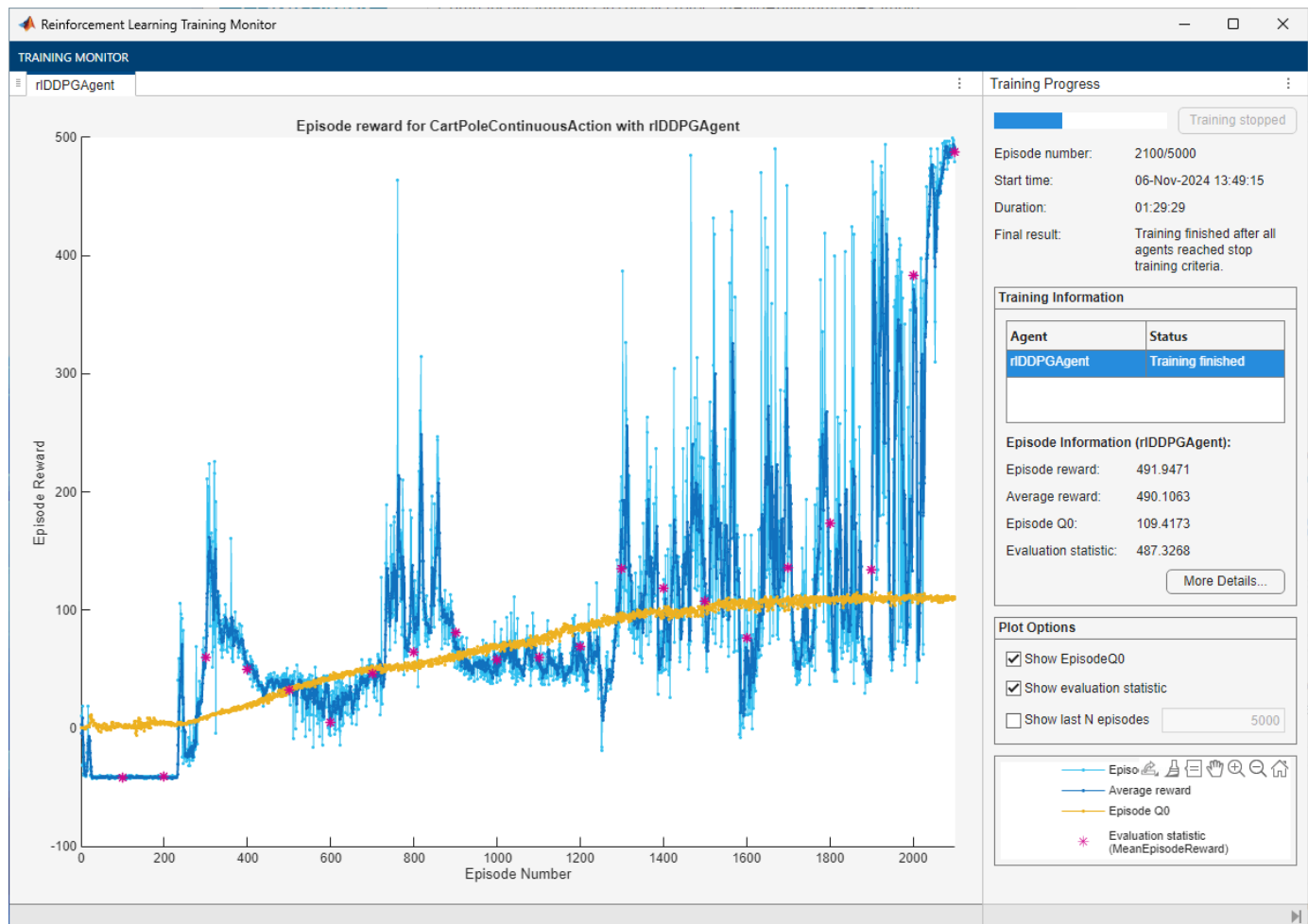
```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
ddpgAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Continuous");
    % Train the agent. Save the final agent and training results.
    tic
    ddpdTngRes = train(ddpTngAgent,env,trainOpts,Evaluator=evl);
    ddpdTngTime = toc;
    % Extract number of training episodes and total steps.
    ddpdTngEps = ddpdTngRes.EpisodeIndex(end);
    ddpdTngSteps = sum(ddpdTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpBchDDPDAgent.mat", ...
    %      "ddpTngAgent","ddpTngEps","ddpTngSteps","ddpTngTime")
else
    % Load the pretrained agent and results for the example.
    load("ccpBchDDPDAgent.mat", ...
         "ddpTngAgent","ddpTngEps","ddpTngSteps","ddpTngTime")
end
```



For the DDPG Agent, the training converges to a solution after 2100 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

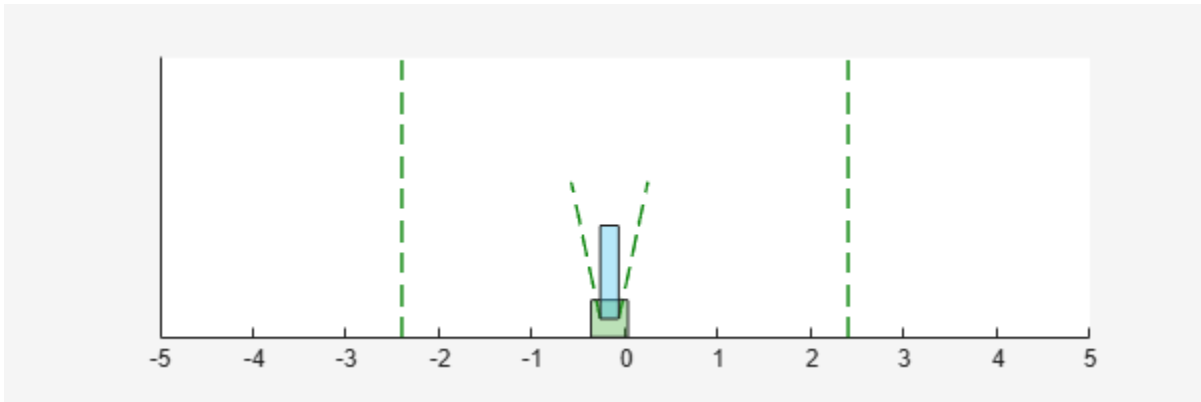
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ddpAgent, simOptions);
```



```
ddpgTotalRwd = sum(experience.Reward)
```

```
ddpgTotalRwd =  
489.5395
```

The trained DDPG agent stabilizes the pole in the upright position.

Create, Train, and Simulate a TD3 Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
td3Agent = rlTD3Agent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
td3Agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
td3Agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
td3Agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
td3Agent.AgentOptions.ExperienceBufferLength = 1e6;
```

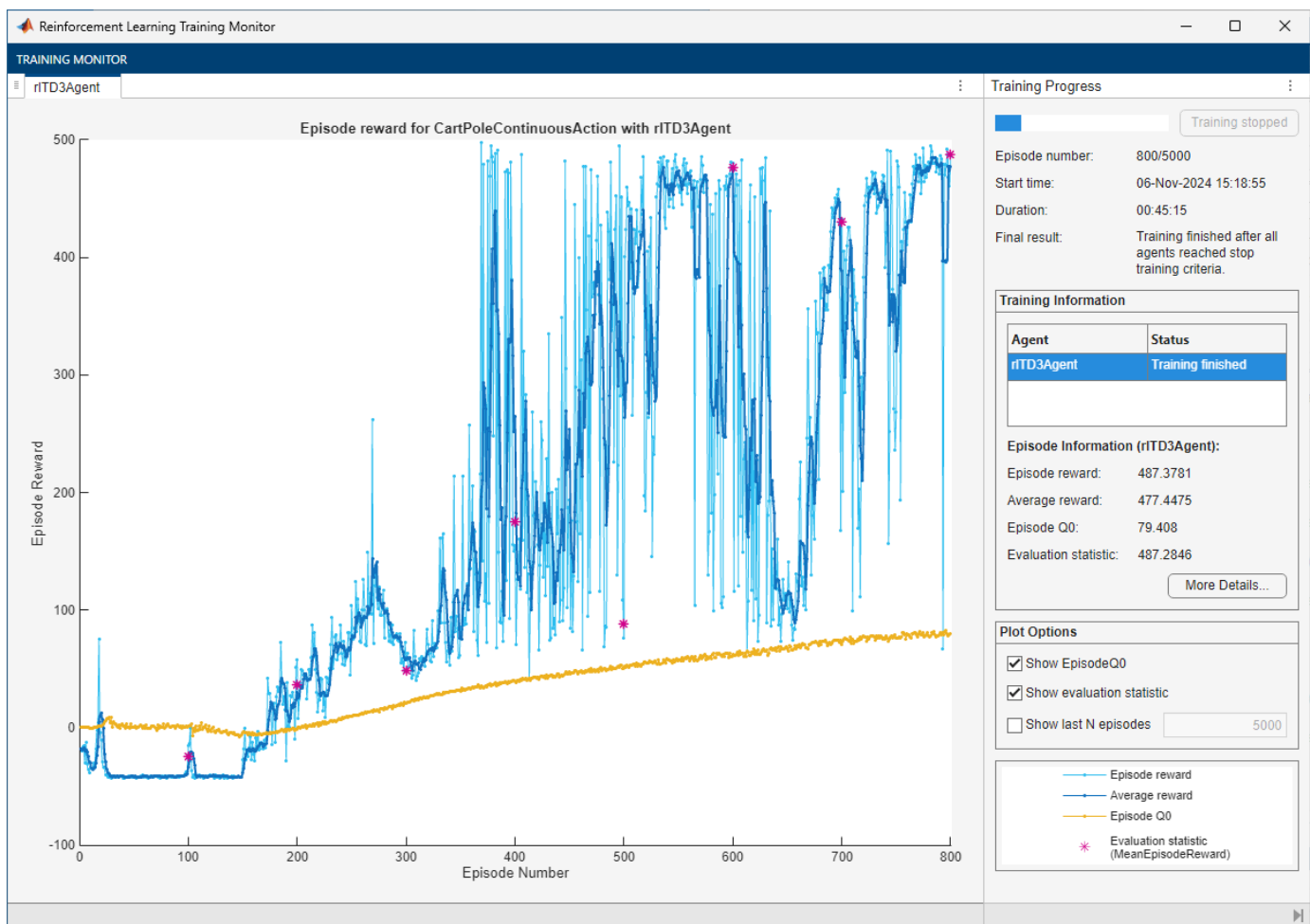
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % To avoid plotting in training, recreate the environment.  
    env = rlPredefinedEnv("CartPole-Continuous");
```

```

% Train the agent. Save the final agent and training results.
tic
td3TngRes = train(td3Agent,env,trainOpts,Evaluator=evl);
td3TngTime = toc;
% Extract number of training episodes and total steps.
td3TngEps = td3TngRes.EpisodeIndex(end);
td3TngSteps = sum(td3TngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ccpBchTD3Agent.mat", ...
%      "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
else
% Load the pretrained agent and results for the example.
load("ccpBchTD3Agent.mat", ...
      "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
end

```



For the TD3 Agent, the training converges to a solution after 800 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

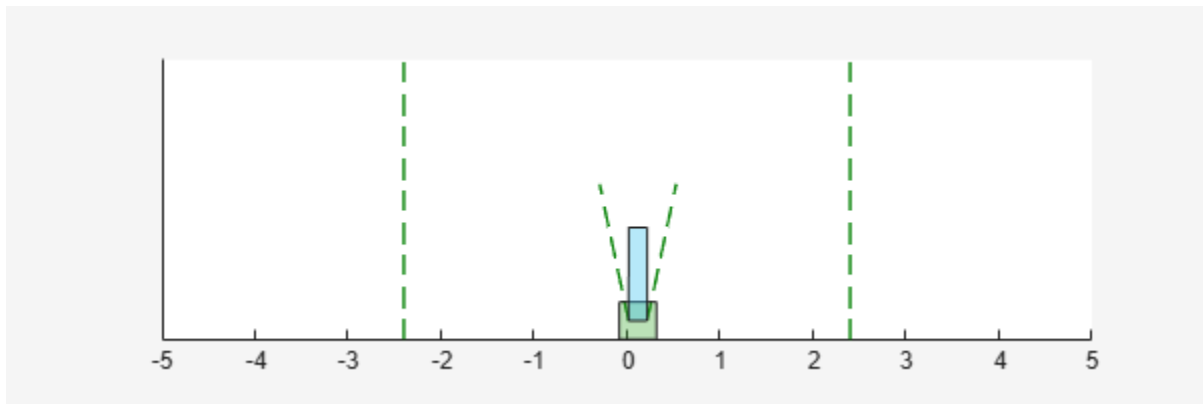
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,td3Agent,simOptions);
```



```
td3TotalRwd = sum(experience.Reward)
```

```
td3TotalRwd =  
486.8400
```

The trained TD3 agent stabilizes the pole in the upright position.

Create, Train, and Simulate a SAC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

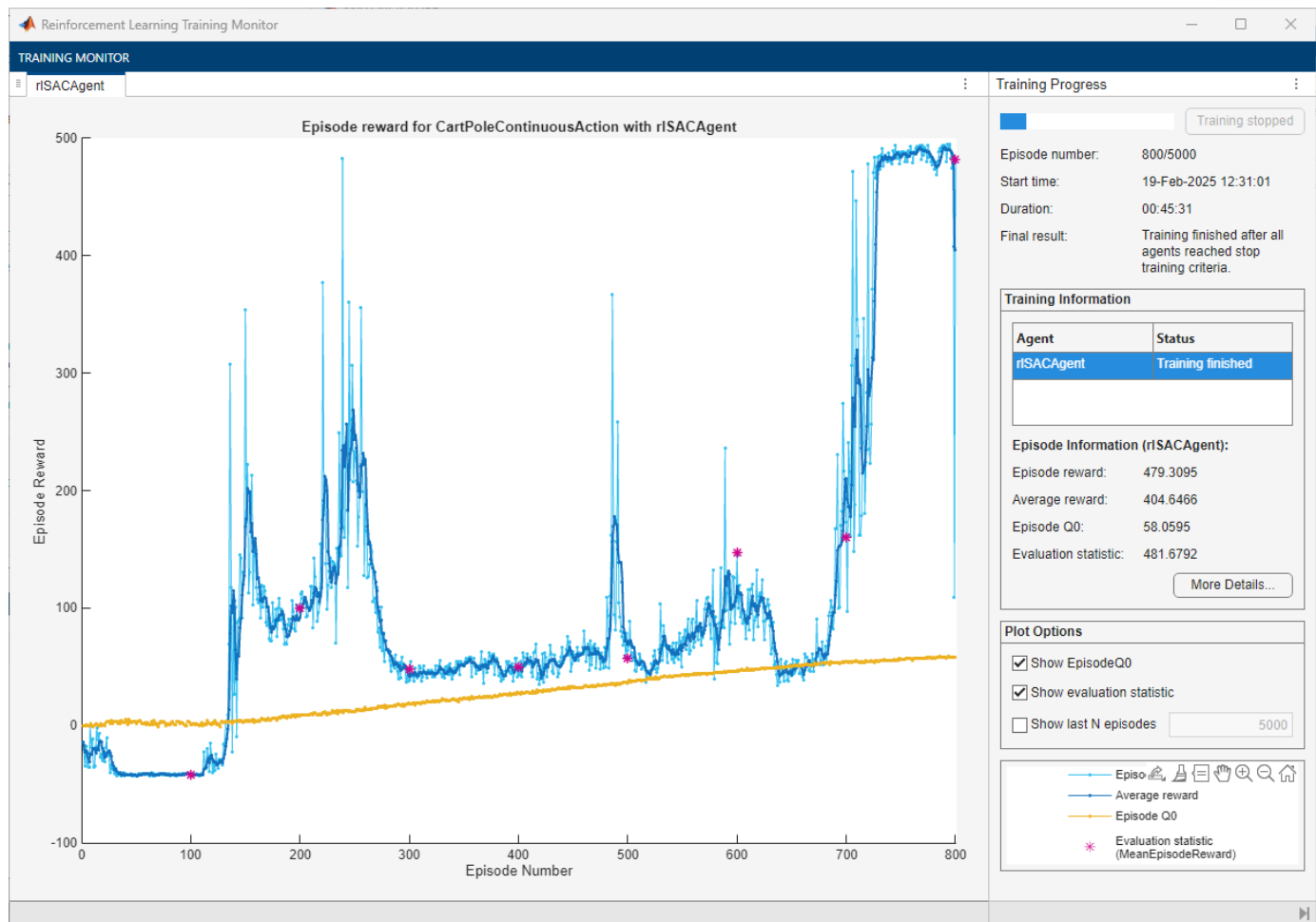
```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```


Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % To avoid plotting in training, recreate the environment.
    env = rlPredefinedEnv("CartPole-Continuous");
    % Train the agent. Save the final agent and training results.
    tic
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);
    sacTngTime = toc;
    % Extract number of training episodes and total steps.
    sacTngEps = sacTngRes.EpisodeIndex(end);
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpBchSACAgent.mat", ...
    %     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
else
    % Load the pretrained agent and results for the example.
    load("ccpBchSACAgent.mat", ...
        "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
end
```



For the SAC Agent, the training converges to a solution after 800 episodes. You can check the trained agent within the cart-pole environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

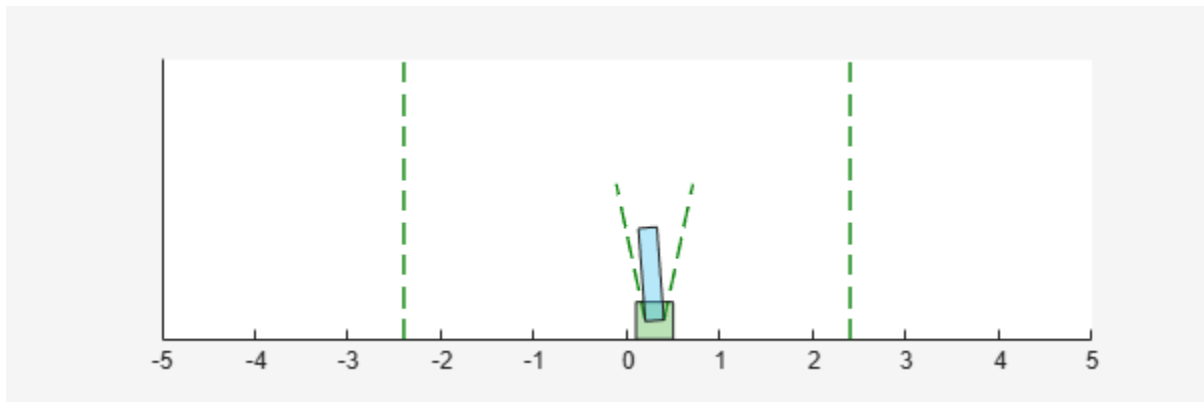
Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, sacAgent, simOptions);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
481.0800
```

The trained SAC agent stabilizes the pole in the upright position.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    ddpGTotalRwd  
    td3TotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    ddpGtngEps  
    td3TngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    pgTngSteps  
    acTngSteps  
    ppoTngSteps  
    ddpGtngSteps  
    td3TngSteps  
    sacTngSteps  
];
```

```
tngTime = [  
    pgTngTime  
    acTngTime
```

```

ppoTngTime
ddpgTngTime
td3TngTime
sacTngTime
];

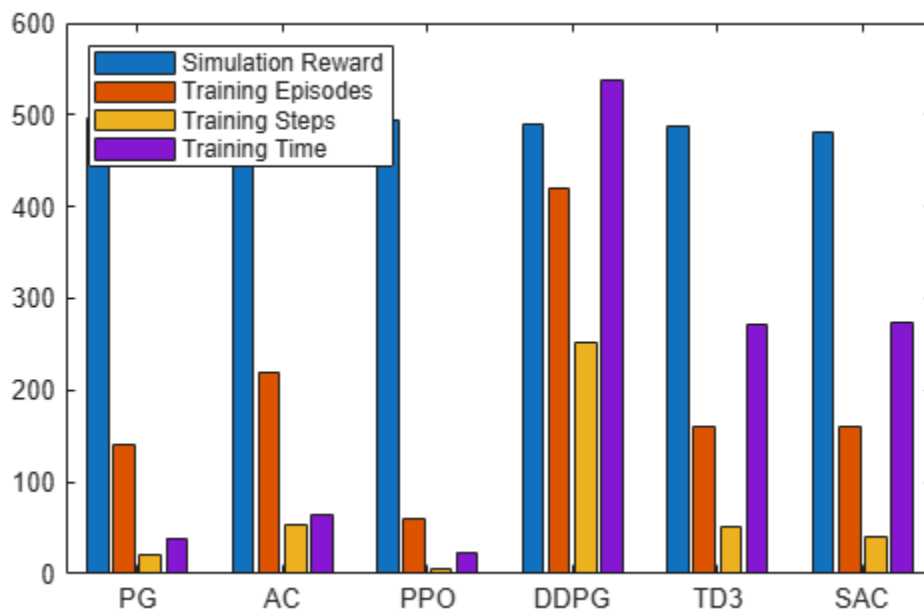
```

Plot the simulation reward, number of training episodes, number of training steps (that is, the number of interactions between the agent and the environment) and the training time. Scale the data by the factor [1 5 1e6 10] for better visualization.

```

bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 5 1e6 10])
xticklabels(["PG" "AC" "PPO" "DDPG" "TD3" "SAC"])
legend(["Simulation Reward","Training Episodes","Training Steps","Training Time"], ...
       "Location","northwest")

```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, all agents perform satisfactorily in terms of total simulation reward, with PPO using much less training time (because it is fast as a training algorithm and because it converges in just 300 episodes). DDPG takes considerably more time than PPO and AC, mostly because it needs many more training steps to converge. TD3 needs more time than AC despite taking less training steps. This happens because TD3 needs to calculate more gradients than AC. Similarly, SAC also takes almost as much time as DDPG to converge despite needing less training steps, and this also happens largely because SAC needs to calculate more gradients. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```

% Uncomment to save all the workspace variables
% save ccpAllVars.mat

```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Continuous Pendulum Swing-Up

This example shows how to create and train frequently used default agents on a continuous action space pendulum swing-up environment. This environment is modeled in Simulink®, and represents a simple frictionless pendulum that initially hangs in a downward position. The agent can apply a control torque on the pendulum, and its goal is to make the pendulum stand upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

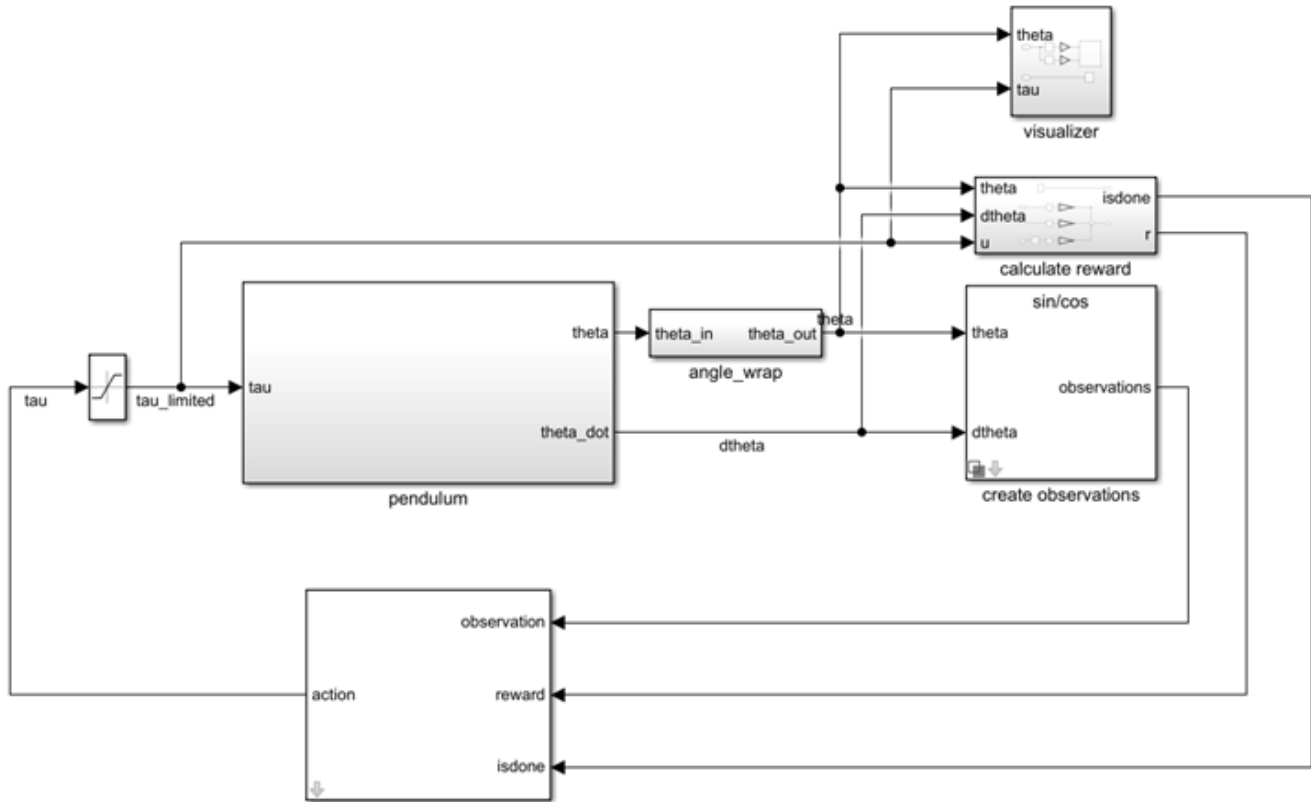
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Pendulum Swing-Up Simulink Environment

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

Open the model.

```
mdl = "rlSimplePendulumModel";  
open_system(mdl)
```



In this model:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment is from -2 to 2 N·m.
- The observations from the environment are the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the continuous pendulum environment.

```
env = rlPredefinedEnv("SimplePendulumModel-Continuous")
```

```
env =  
SimulinkEnvWithAgent with properties:  
  
    Model : rlSimplePendulumModel  
    AgentBlock : rlSimplePendulumModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

Obtain the observation and action information for later use when creating agent.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
    rlNumericSpec with properties:  
  
        LowerLimit: -Inf  
        UpperLimit: Inf  
        Name: "observations"  
        Description: [0×0 string]  
        Dimension: [3 1]  
        DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =  
    rlNumericSpec with properties:  
  
        LowerLimit: -2  
        UpperLimit: 2  
        Name: "torque"  
        Description: [0×0 string]  
        Dimension: [1 1]  
        DataType: "double"
```

The object has a continuous action space where the agent can apply torque values between -2 to 2 N·m to the pendulum.

Set the observations of the environment to be the sine of the pendulum angle, the cosine of the pendulum angle, and the pendulum angle derivative.

```
set_param( ...  
    "rlSimplePendulumModel/create_observations", ...  
    "ThetaObservationHandling","sincos");
```

To define the initial condition of the pendulum as hanging downward, specify an environment reset function using an anonymous function handle. This reset function sets the model workspace variable `theta0` to `pi`.

```
env.ResetFcn = @(in)setVariable(in,"theta0",pi,"Workspace",mdl);
```

Specify the agent sample time `Ts` and the simulation time `Tf` in seconds.

```
Ts = 0.05;  
Tf = 20;
```


Reset the environment and return the environment state.

```
reset(env)

ans =
  SimulationInput with properties:

      ModelName: "rlSimplePendulumModel"
    InitialState: [0x0 Simulink.op.ModelOperatingPoint]
    ExternalInput: []
    ModelParameters: [0x0 Simulink.Simulation.ModelParameter]
    BlockParameters: [0x0 Simulink.Simulation.BlockParameter]
      Variables: [1x1 Simulink.Simulation.Variable]
    PreSimFcn: []
    PostSimFcn: []
    UserString: ''
  VariantConfiguration: ''
```

Configure Training Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of T_f/T_s (by default 400) time steps.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.
- Stop the training when the average reward in the evaluation episodes is greater than -740. At this point, the agent can balance the position of the pendulum.

```
trainOpts = rlTrainingOptions(...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    StopTrainingCriteria="EvaluationStatistic",...
    StopTrainingValue=-740);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for $\text{ceil}(T_f/T_s)$ steps.

```
simOptions = rlSimulationOptions(MaxSteps=ceil(Tf/Ts));
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `pgAgent`.

```
pgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

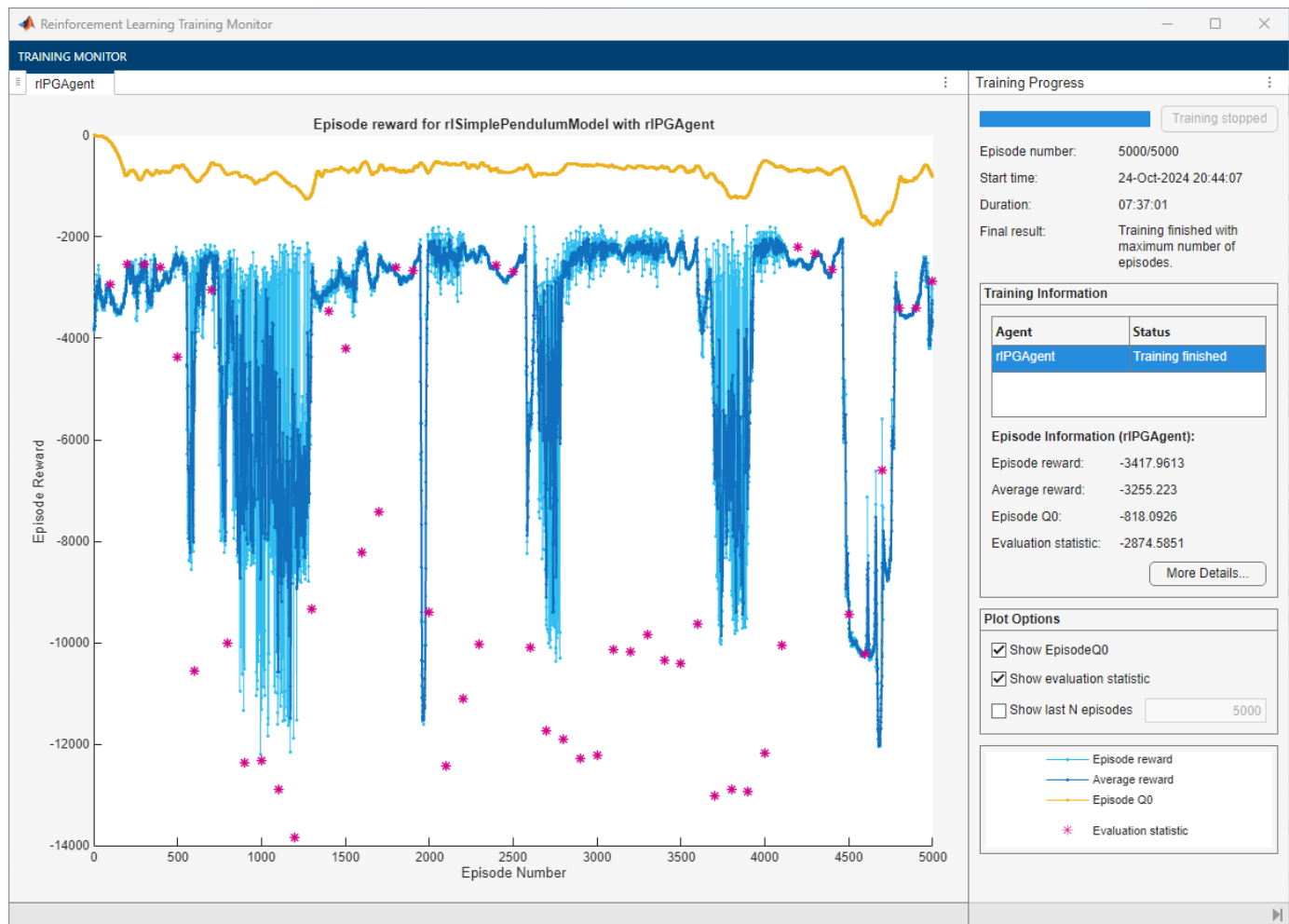
```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cpsuBchPGAgent.mat", ...
    %     "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
else
    % Load the pretrained agent and results for the example.
    load("cpsuBchPGAgent.mat", ...
        "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
end
```



For the PG agent, the training does not converge to a solution. You can check the trained agent within the pendulum swing-up environment.

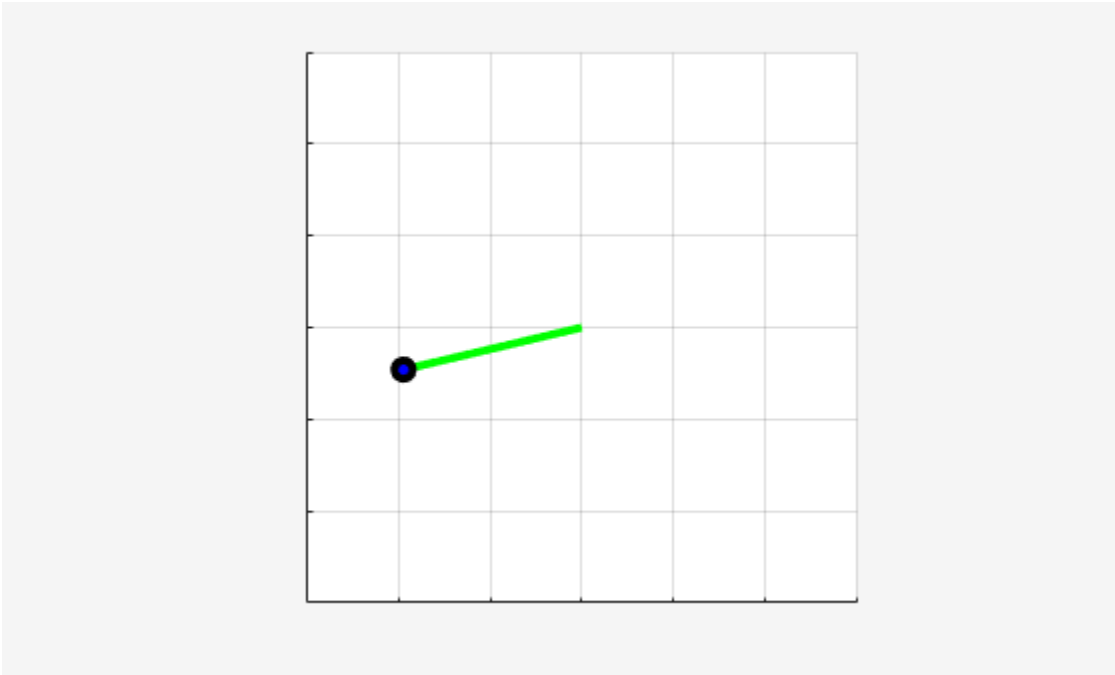
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
```



```
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =  
-3.7799e+03
```

The trained PG agent is able to swing up the pendulum but it cannot stabilize it upright.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `acAgent`.

```
acAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

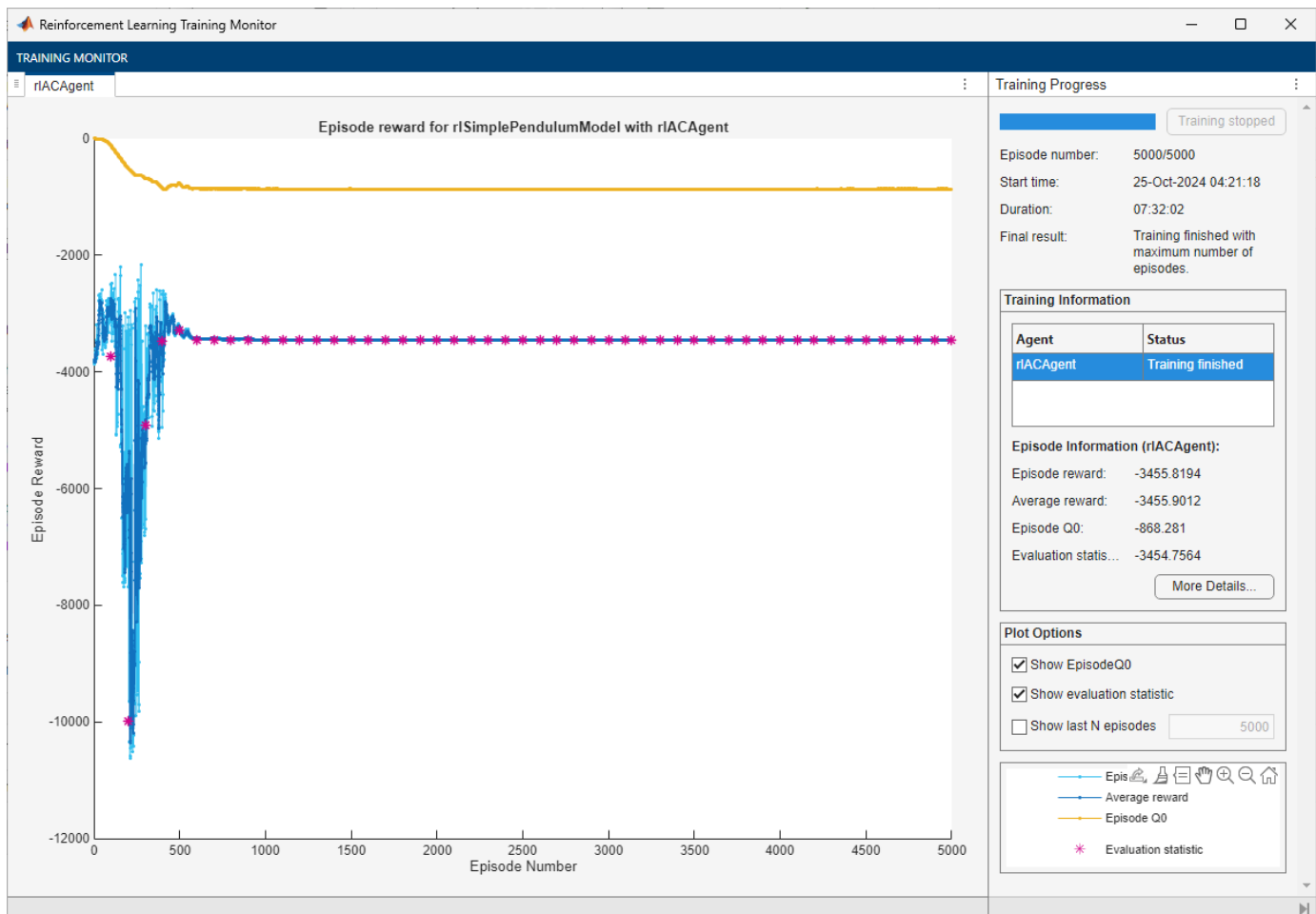
```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cpsuBchACAgent.mat", ...
    %     "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("cpsuBchACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end
```



For the AC agent, the training does not converge to a solution. You can check the trained agent within the pendulum swing-up environment.

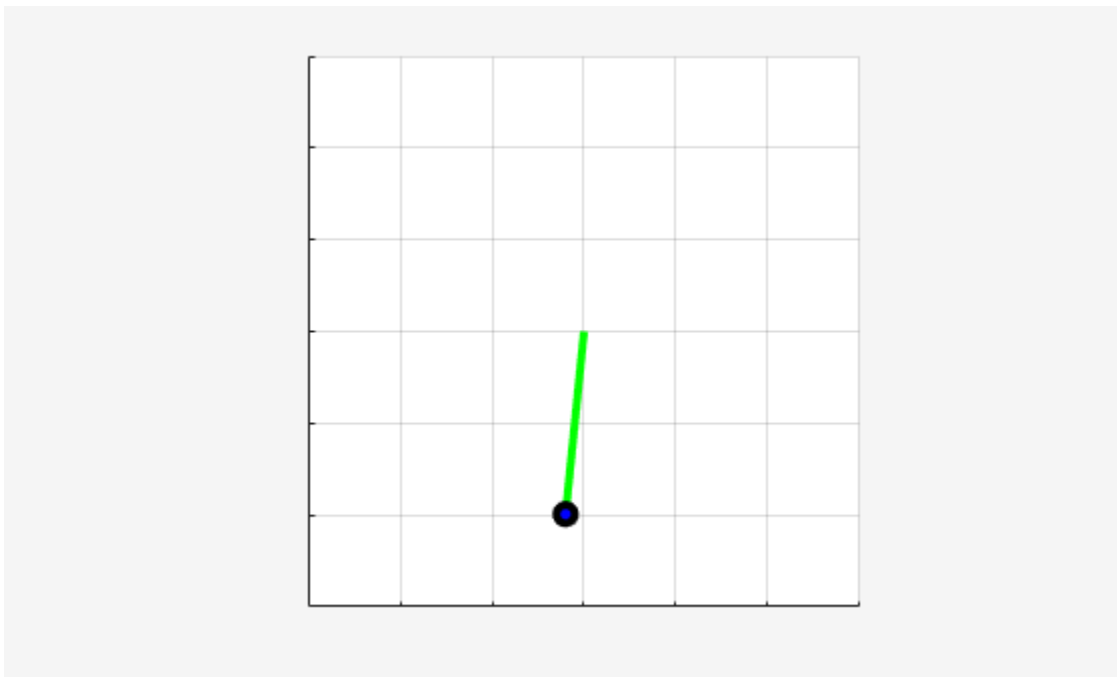
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
```



```
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =  
-3.4558e+03
```

The trained AC agent does not swing up the pendulum.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `ppoAgent`.

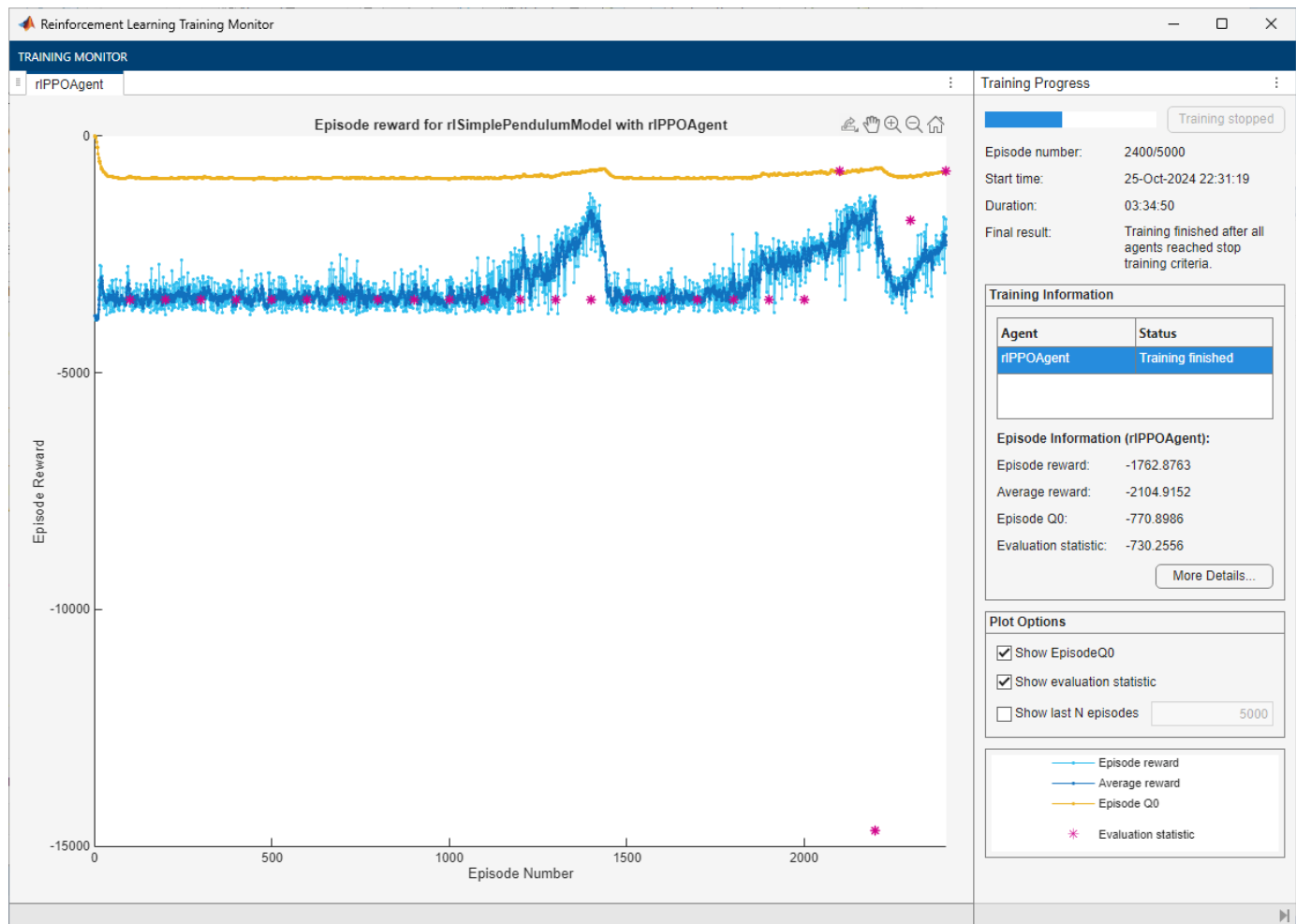
```
ppoAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
    ppoTngTime = toc;
    % Extract number of training episodes and total steps.
    ppoTngEps = ppoTngRes.EpisodeIndex(end);
    ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cpsuBchPPOAgent.mat", ...
    %     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
else
    % Load the pretrained agent and results for the example.
    load("cpsuBchPPOAgent.mat", ...
        "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
end
```



For the PPO Agent, the training converges to a solution after 2400 episodes. You can check the trained agent within the pendulum swing-up environment.

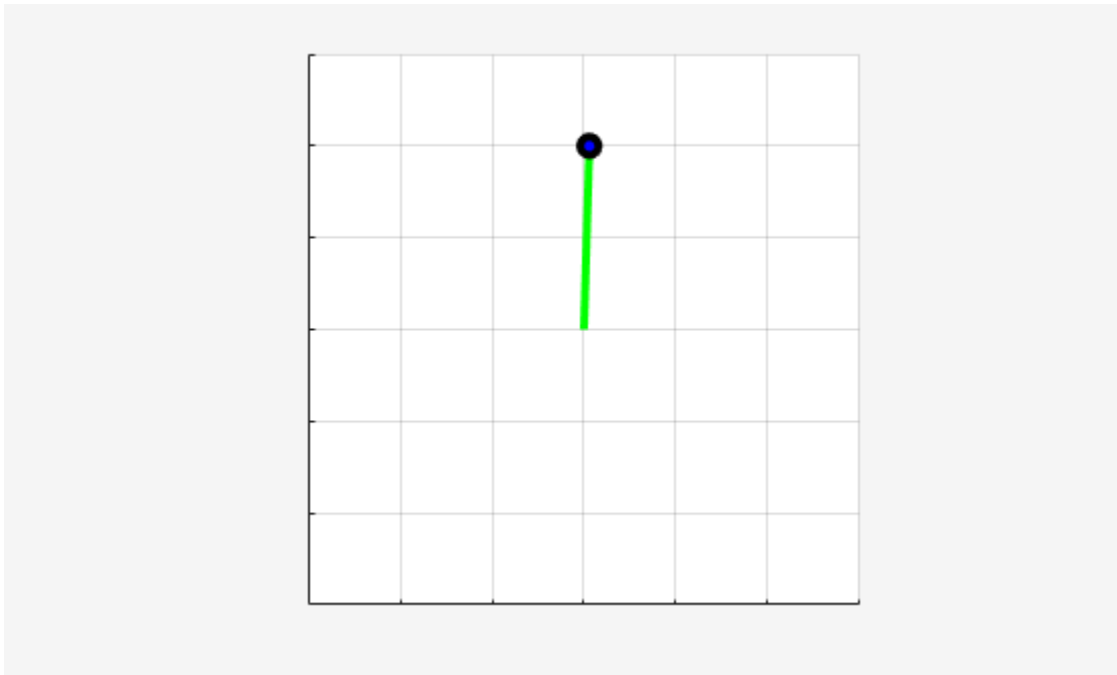
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env,ppoAgent,simOptions);
```

```
ppoTotalRwd = sum(experience.Reward)
```

```
ppoTotalRwd =  
-1.9853e+03
```

The trained PPO agent does not swing up the pendulum.

Create, Train, and Simulate a DDPG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rlDDPGAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `ddpgAgent`.

```
ddpgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

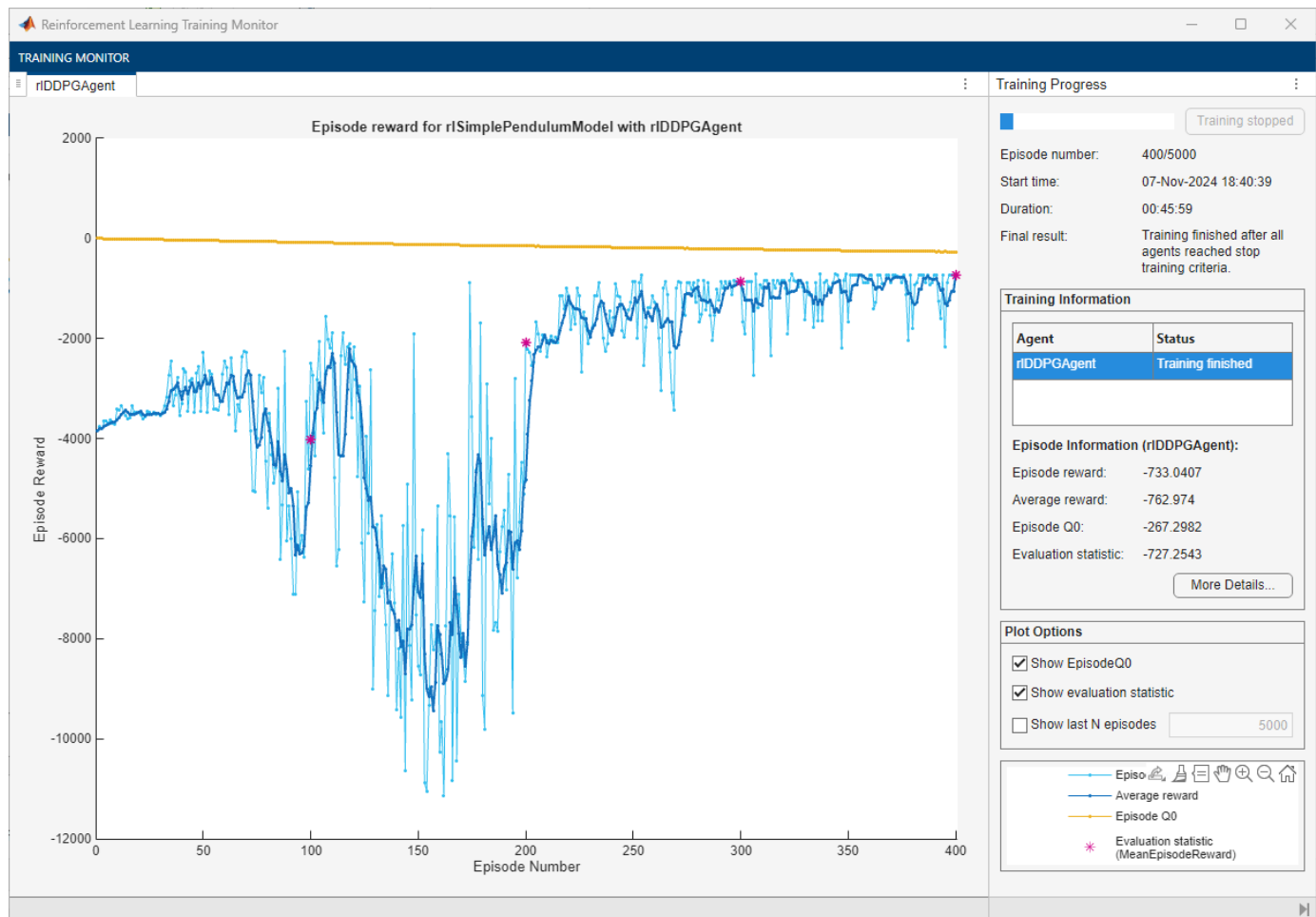
```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
ddpgAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    ddpdTngRes = train(ddpgAgent,env,trainOpts,Evaluator=evl);
    ddpdTngTime = toc;
    % Extract number of training episodes and total steps.
    ddpdTngEps = ddpdTngRes.EpisodeIndex(end);
    ddpdTngSteps = sum(ddpdTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cpsuBchDDPGAgent.mat", ...
    %     "ddpgAgent","ddpdTngEps","ddpdTngSteps","ddpdTngTime")
else
    % Load the pretrained agent and results for the example.
    load("cpsuBchDDPGAgent.mat", ...
        "ddpgAgent","ddpdTngEps","ddpdTngSteps","ddpdTngTime")
end
```



For the DDPG Agent, the training converges to a solution after 400 episodes. You can check the trained agent within the pendulum swing-up environment.

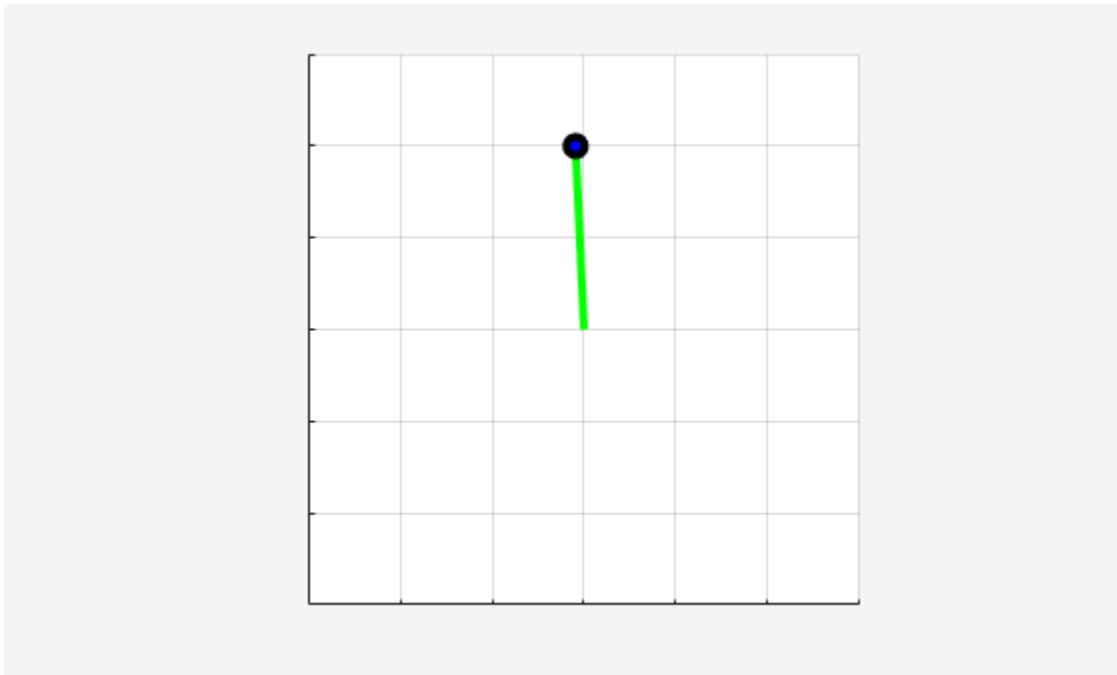
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env,ddpgAgent,simOptions);
```



```
ddpgTotalRwd = sum(experience.Reward)
```

```
ddpgTotalRwd =  
-727.2543
```

The trained DDPG agent is able to swing up and stabilize the pendulum upright.

Create, Train, and Simulate a TD3 Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
td3Agent = rlTD3Agent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `td3Agent`.

```
td3Agent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

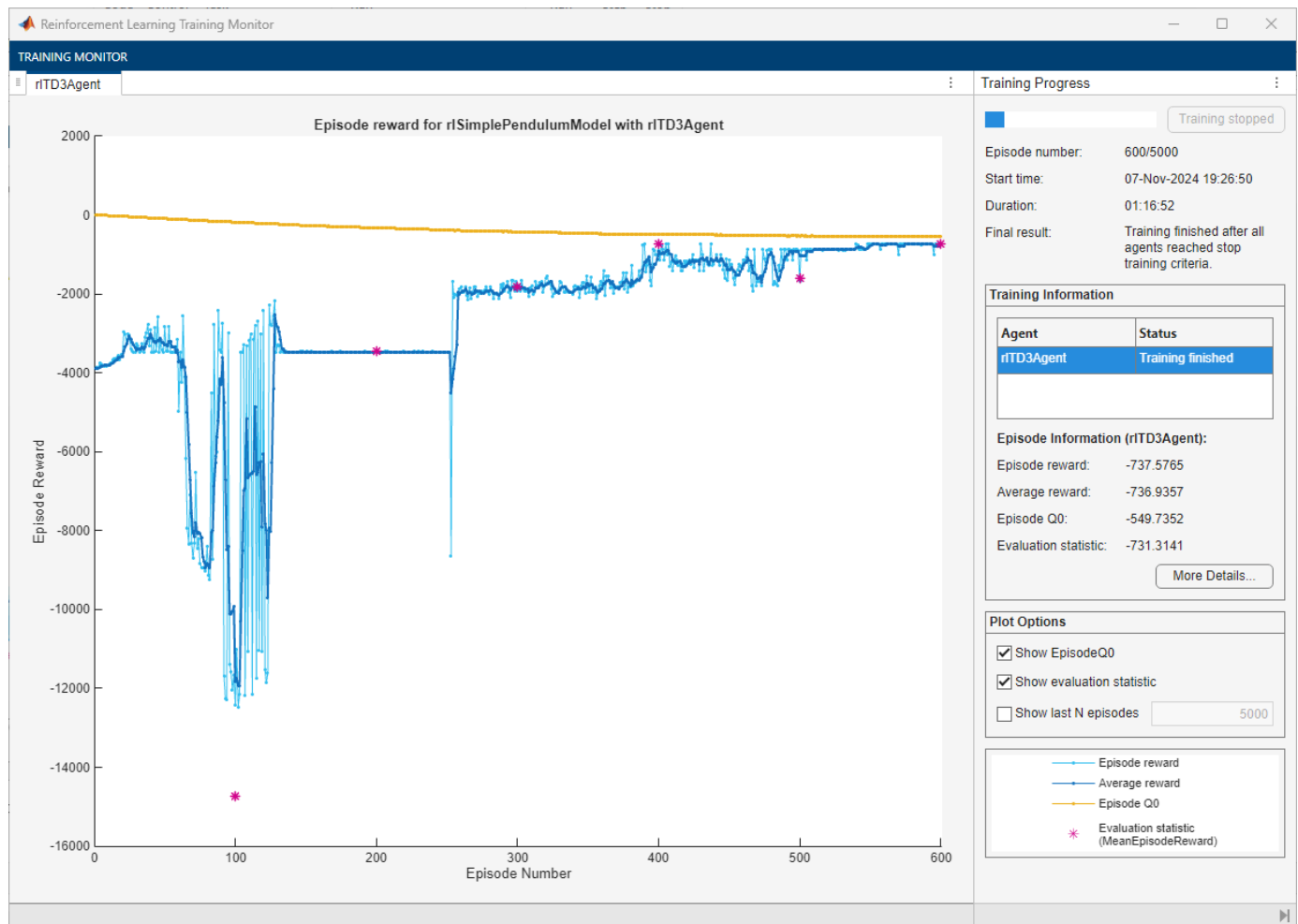
```
td3Agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
td3Agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
td3Agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
td3Agent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    td3TngRes = train(td3Agent,env,trainOpts,Evaluator=evl);
    td3TngTime = toc;
    % Extract number of training episodes and total steps.
    td3TngEps = td3TngRes.EpisodeIndex(end);
    td3TngSteps = sum(td3TngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("cpsuBchTD3Agent.mat", ...
    %     "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
else
    % Load the pretrained agent and results for the example.
    load("cpsuBchTD3Agent.mat", ...
        "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
end
```



For the TD3 Agent, the training converges to a solution after 600 episodes. You can check the trained agent within the pendulum swing-up environment.

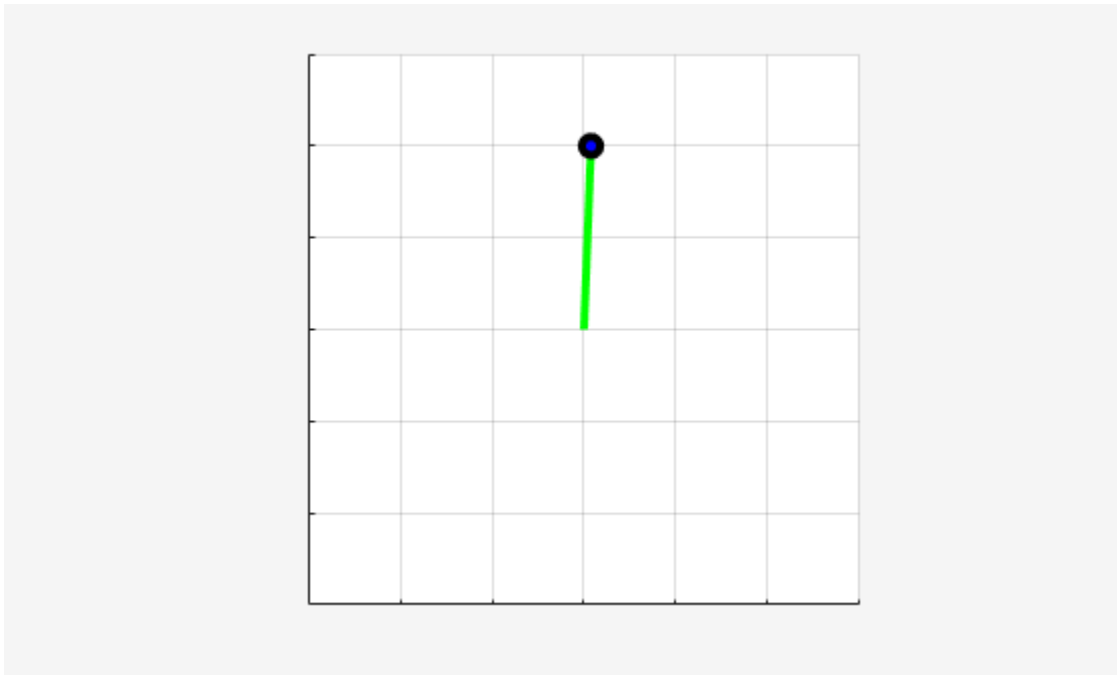
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, td3Agent, simOptions);
```



```
td3TotalRwd = sum(experience.Reward)
```

```
td3TotalRwd =  
-731.3141
```

The trained TD3 agent is able to swing up and stabilize the pendulum upright.

Create, Train, and Simulate a SAC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `sacAgent`.

```
sacAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

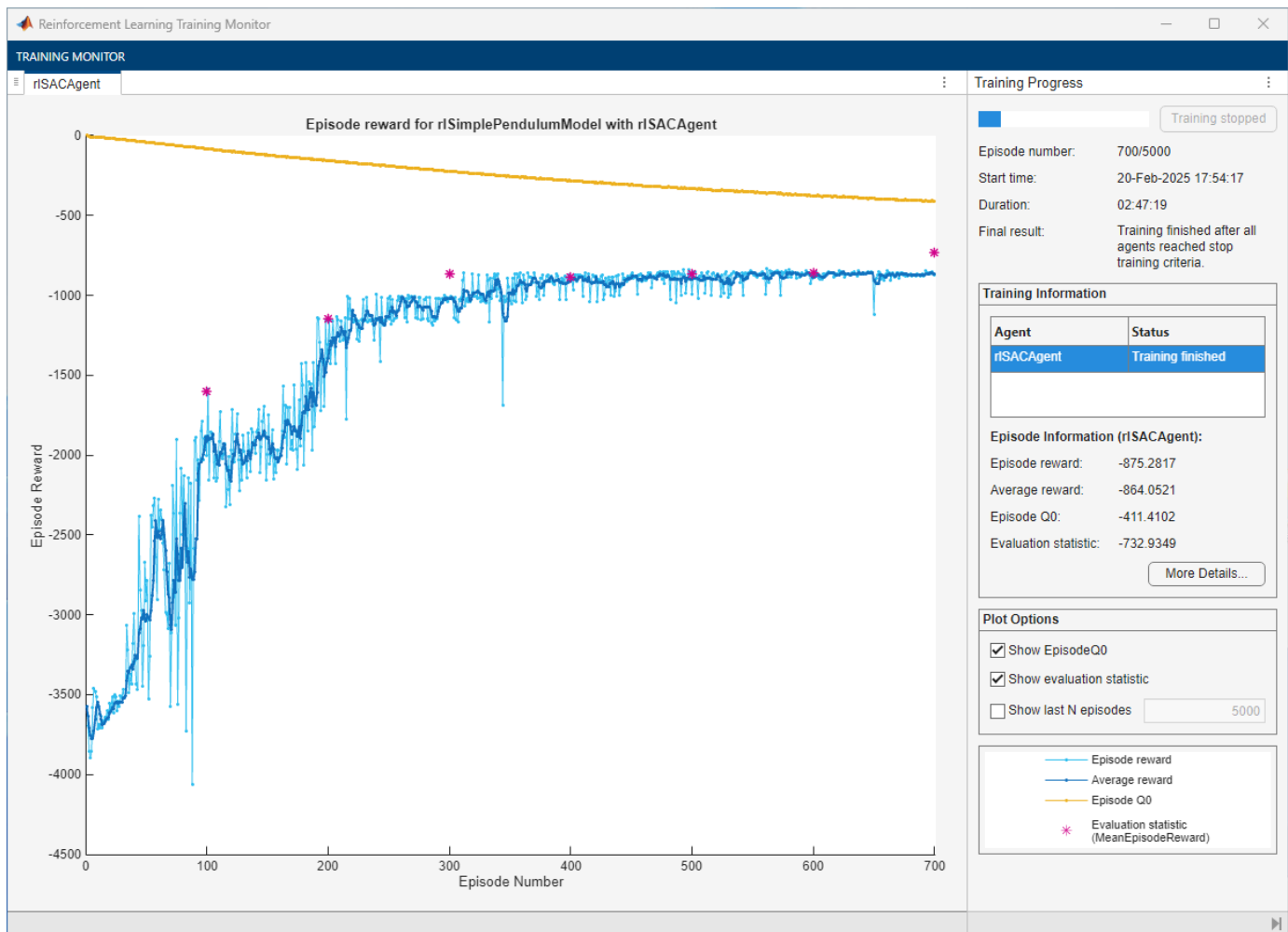
```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent. Save the final agent and training results.  
    tic  
    sacTngRes = train(sacAgent,env,trainOpts,Evaluator=evl);  
    sacTngTime = toc;  
    % Extract number of training episodes and total steps.  
    sacTngEps = sacTngRes.EpisodeIndex(end);  
    sacTngSteps = sum(sacTngRes.TotalAgentSteps);  
    % Uncomment to save the trained agent and the training metrics.  
    % save("cpsuBchSACAgent.mat", ...  
    %     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")  
else  
    % Load the pretrained agent and results for the example.  
    load("cpsuBchSACAgent.mat", ...  
        "sacAgent","sacTngEps","sacTngSteps","sacTngTime")  
end
```

For the SAC Agent, the training converges to a solution after 700 episodes. You can check the trained agent within the pendulum swing-up environment.

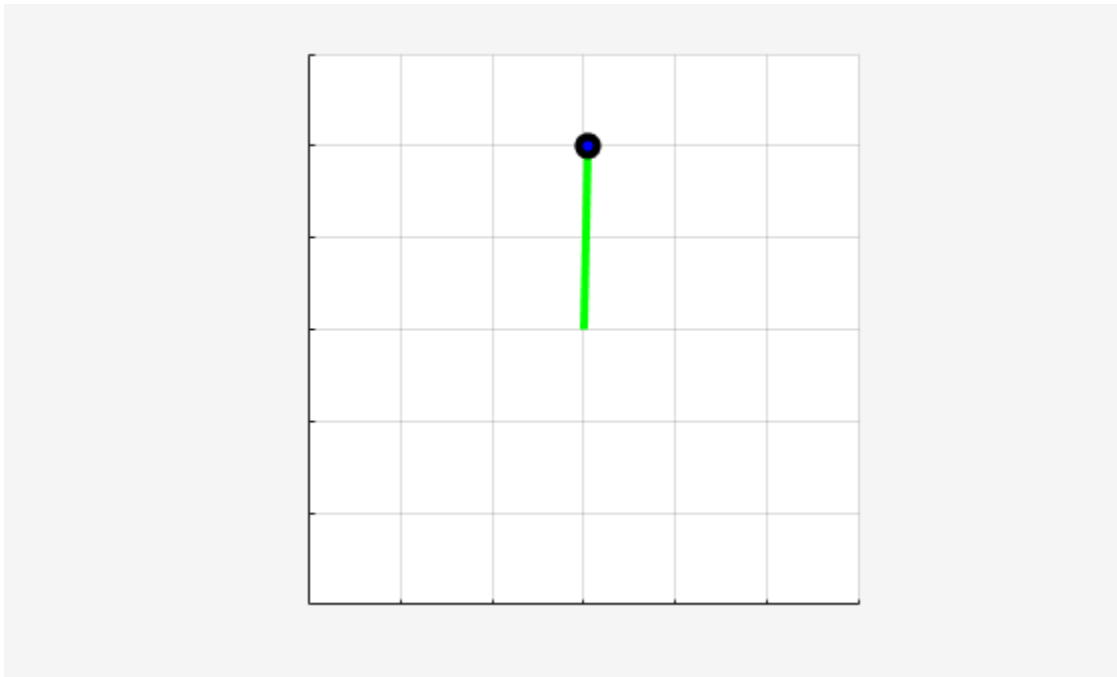
Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, sacAgent, simOptions);
```



```
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =  
-886.4478
```

The trained SAC agent is able to swing up and stabilize the pendulum upright.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    ddpgTotalRwd  
    td3TotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    ddpgTngEps  
    td3TngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    pgTngSteps
```

```

acTngSteps
ppoTngSteps
ddpgTngSteps
td3TngSteps
sacTngSteps
];

tngTime = [
    pgTngTime
    acTngTime
    ppoTngTime
    ddpTngTime
    td3TngTime
    sacTngTime
];

```

Because the training for the PG, AC, and PPO agents does not converge, to avoid visualizing their metrics, set them to NaN.

```

simReward(1:3) = NaN;
tngEpisodes(1:3) = NaN;
tngSteps(1:3) = NaN;
tngTime(1:3) = NaN;

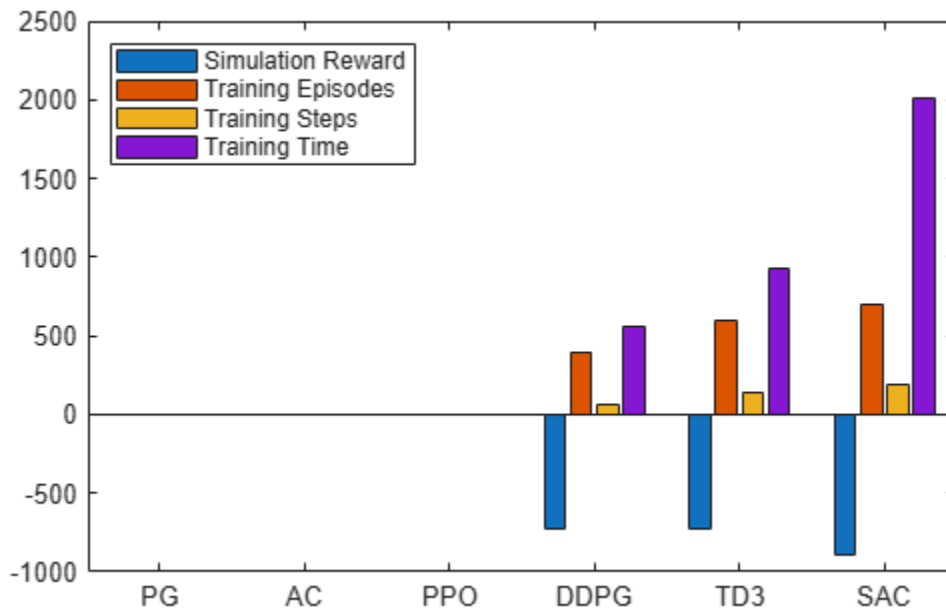
```

Plot the simulation reward, number of training episodes, number of training steps and training time. Scale the data by the factor [1 1 5e5 5] for better visualization.

```

bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 1 5e5 5])
xticklabels(["PG" "AC" "PPO" "DDPG" "TD3" "SAC"])
legend(["Simulation Reward","Training Episodes","Training Steps","Training Time"], ...
    "Location","northwest")

```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, DDPG and SAC use about the same number of steps to converge, with DDPG using

less training time because of its simpler algorithm. With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables  
% save cpsuAllVars.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Continuous Cart Pole Swing-Up

This example shows how to create and train frequently used default agents on a continuous action space cart-pole environment. This environment is modeled using Simscape™ Multibody™, and represents a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The agent can apply a force to the cart and its training goal is to swing-up and balance the pole upright using minimal control effort. The example plots performance metrics such as the total training time and the total reward for each trained agent.

The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Simscape Cart-Pole Model Simulink Environment

The reinforcement learning environment for this example is a pole attached to an unactuated joint on a cart, which moves along a frictionless track. The training goal is to make the pole stand upright using minimal control effort.

Open the model.

```
mdl = "rlCartPoleSimscapeModel";  
open_system(mdl)
```


- The reward r_t , provided at every time step, is

$$r_t = -0.1(5\theta_t^2 + x_t^2 + 0.05u_{t-1}^2) - 100B$$

Here:

- θ_t is the angle of displacement from the upright position of the pole.
- x_t is the position displacement from the center position of the cart.
- u_{t-1} is the control effort from the previous time step.
- B is a flag (1 or 0) that indicates whether the cart is out of bounds.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the continuous cart-pole environment.

```
env = rlPredefinedEnv("CartPoleSimscapeModel-Continuous")
```

```
env =  
SimulinkEnvWithAgent with properties:
```

```
    Model : rlCartPoleSimscapeModel  
    AgentBlock : rlCartPoleSimscapeModel/RL Agent  
    ResetFcn : []  
    UseFastRestart : on
```

Obtain the observation and action information for later use when creating agents.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
rlNumericSpec with properties:
```

```
    LowerLimit: -Inf  
    UpperLimit: Inf  
    Name: "observations"  
    Description: [0x0 string]  
    Dimension: [5 1]  
    DataType: "double"
```

```
actInfo = getActionInfo(env)
```

```
actInfo =  
rlNumericSpec with properties:
```

```
    LowerLimit: -15  
    UpperLimit: 15  
    Name: "force"  
    Description: [0x0 string]  
    Dimension: [1 1]  
    DataType: "double"
```

The object has a continuous action space where the agent can apply torque values between -2 to 2 N·m to the pendulum.

Specify the agent sample time T_s and the simulation time T_f in seconds.

```
Ts = 0.02;  
Tf = 25;
```

Configure Training and Simulation Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of T_f/T_s (by default 1250) time steps.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.
- Stop the training when the average reward in the evaluation episodes is greater than -400. At this point, the agent can quickly swing up and balance the pole in the upright position using minimal control effort.

```
trainOpts = rlTrainingOptions(...  
    MaxEpisodes=5000, ...  
    MaxStepsPerEpisode=ceil(Tf/Ts), ...  
    StopTrainingCriteria="EvaluationStatistic",...  
    StopTrainingValue=-400);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for $\text{ceil}(T_f/T_s)$ steps.

```
simOptions = rlSimulationOptions(MaxSteps=ceil(Tf/Ts));
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rngState = rng(0,"twister")  
  
rngState = struct with fields:  
    Type: 'twister'  
    Seed: 0  
    State: [625x1 uint32]
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo,actInfo);
```


To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of one second, set the `SampleTime` property of `pgAgent`.

```
pgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

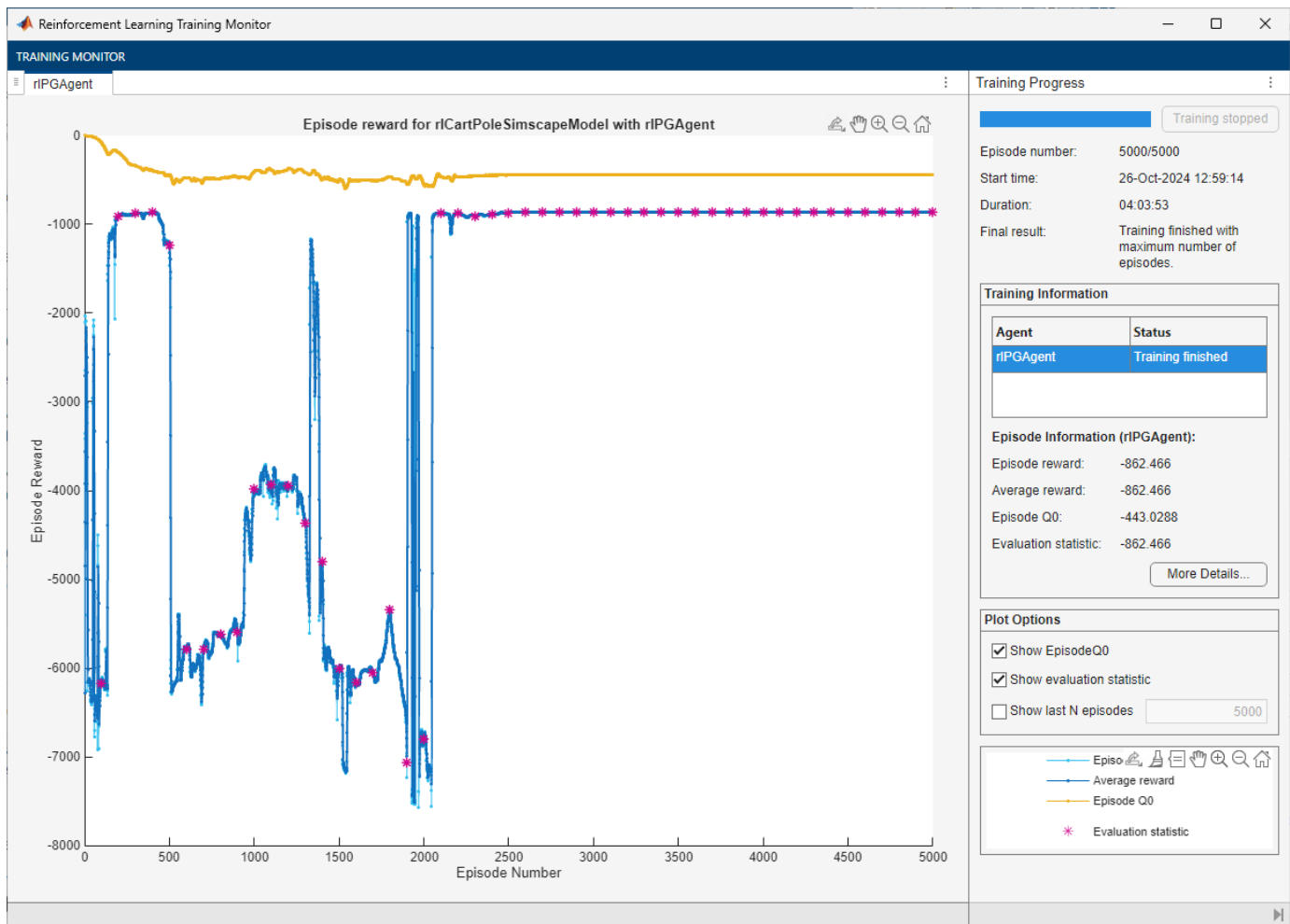
```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);
    pgTngTime = toc;
    % Extract number of training episodes and total steps.
    pgTngEps = pgTngRes.EpisodeIndex(end);
    pgTngSteps = sum(pgTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpsuBchPGAgent.mat", ...
    %     "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
else
    % Load the pretrained agent and results for the example.
    load("ccpsuBchPGAgent.mat", ...
        "pgAgent","pgTngEps","pgTngSteps","pgTngTime")
end
```



For the PG agent, the training does not converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, pgAgent, simOptions);
pgTotalRwd = sum(experience.Reward)
```

```
pgTotalRwd =
-862.4660
```

The trained PG agent is not able to swing up the pole.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rngState = rng(0,"twister")

rngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `acAgent`.

```
acAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

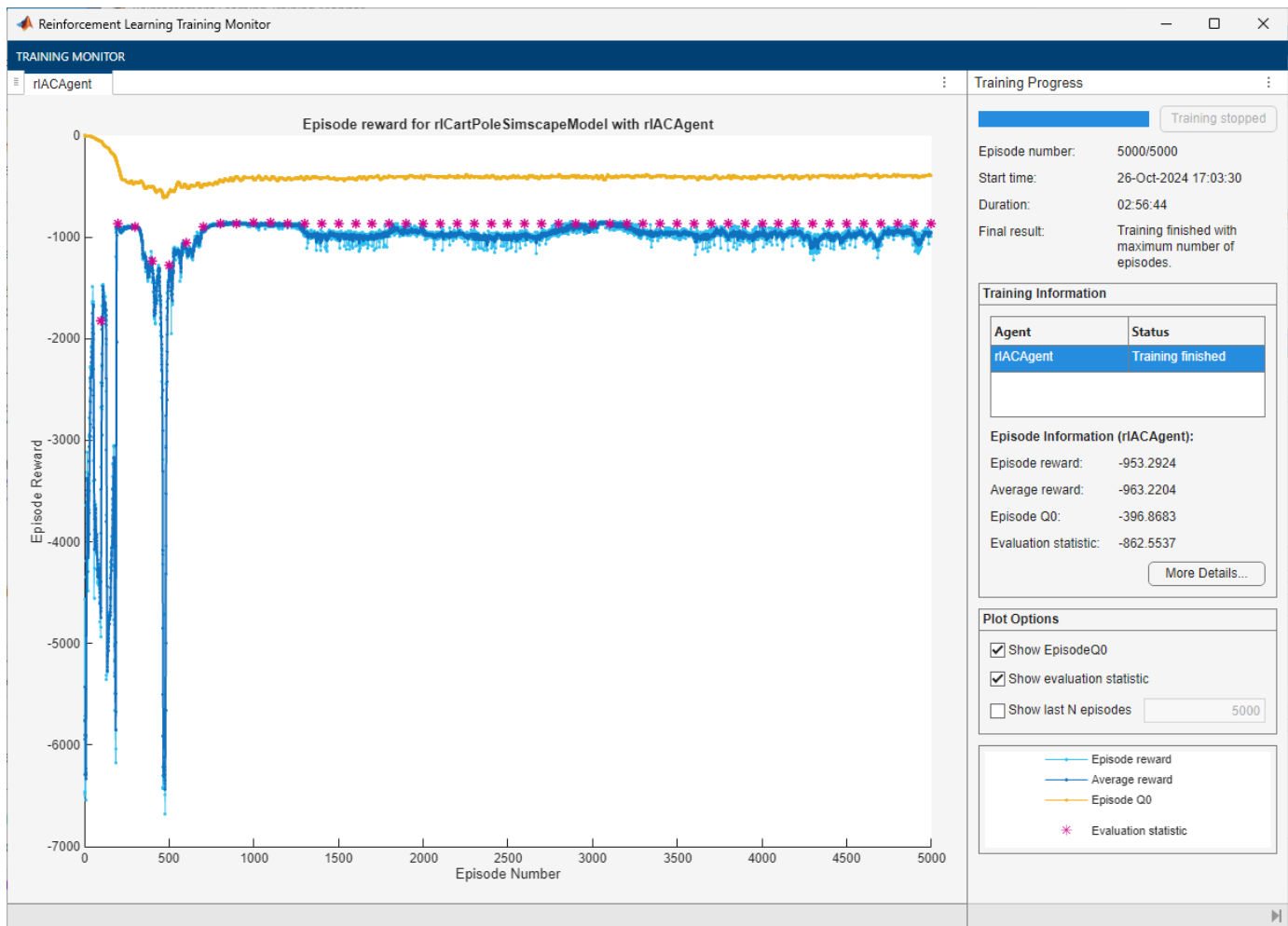
```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);
    acTngTime = toc;
    % Extract number of training episodes and total steps.
    acTngEps = acTngRes.EpisodeIndex(end);
    acTngSteps = sum(acTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpsuBchACAgent.mat", ...
    %     "acAgent","acTngEps","acTngSteps","acTngTime")
else
    % Load the pretrained agent and results for the example.
    load("ccpsuBchACAgent.mat", ...
        "acAgent","acTngEps","acTngSteps","acTngTime")
end
```



For the AC agent, the training does not converge to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, acAgent, simOptions);
acTotalRwd = sum(experience.Reward)
```

```
acTotalRwd =
-958.1490
```

The trained AC agent is not able to swing up the pole.

Create, Train, and Simulate a PPO Agent

For the PPO Agent, the system goes unstable during training, which causes an error. Therefore, for this example, do not train a PPO agent.

Create, Train, and Simulate a DDPG Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rngState = rng(0,"twister")

rngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rlDDPGAgent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `ddpgAgent`.

```
ddpgAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
ddpgAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

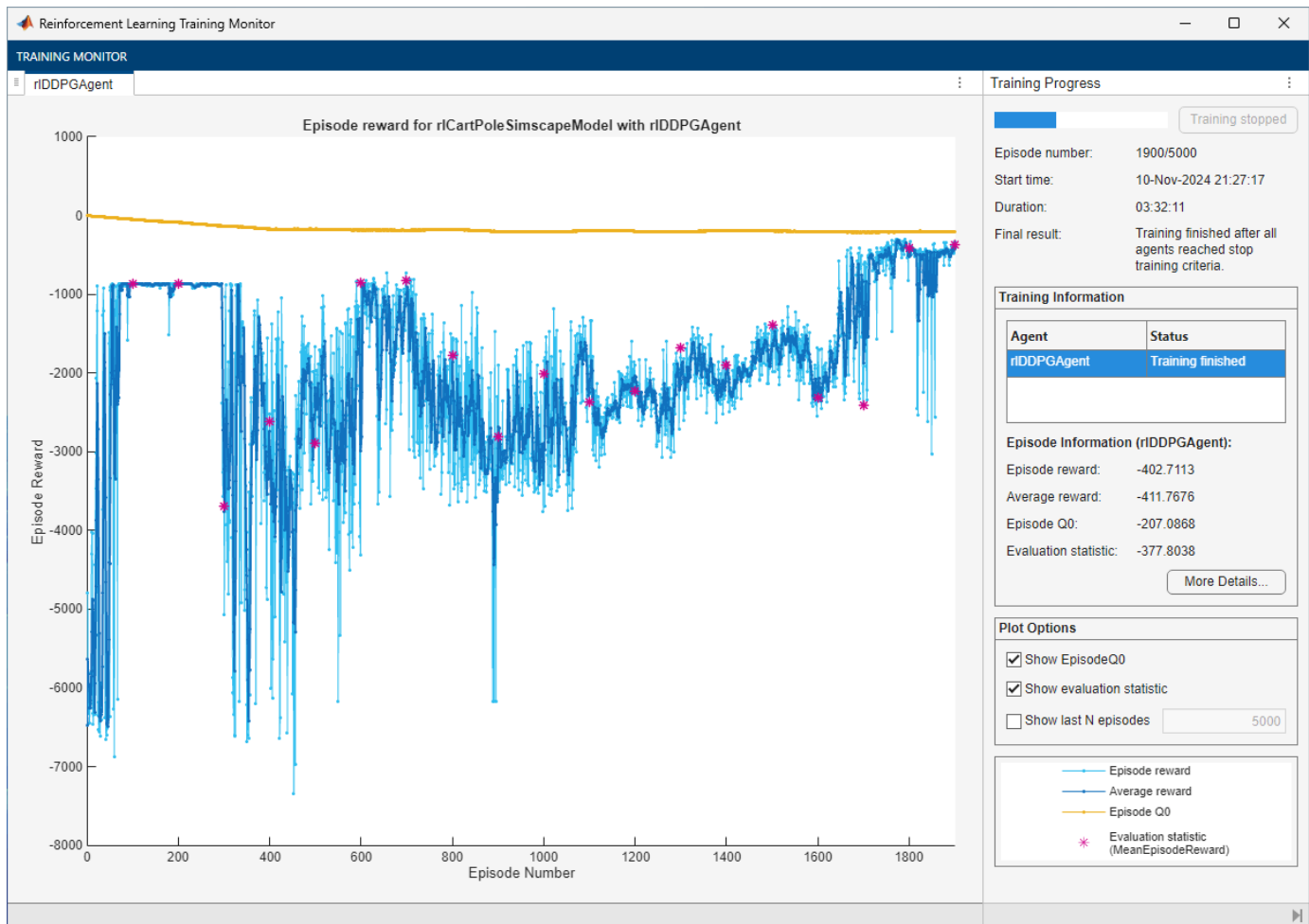
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    ddpdTngRes = train(ddpgAgent,env,trainOpts,Evaluator=evl);
    ddpdTngTime = toc;
    % Extract number of training episodes and total steps.
    ddpdTngEps = ddpdTngRes.EpisodeIndex(end);
    ddpdTngSteps = sum(ddpdTngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
    % save("ccpsuBchDDPGAgent.mat", ...
    %     "ddpgAgent","ddpdTngEps","ddpdTngSteps","ddpdTngTime")
```

```

else
    % Load the pretrained agent and results for the example.
    load("ccpsuBchDDPGAgent.mat", ...
        "ddpgAgent", "ddpgTngEps", "ddpgTngSteps", "ddpgTngTime")
end

```



For the DDPG Agent, the training converges to a solution after 1900 episodes. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```

experience = sim(env,ddpgAgent,simOptions);
ddpgTotalRwd = sum(experience.Reward)

```

```
ddpgTotalRwd =
-377.8038
```

The trained DDPG agent swings up the pole and maintains it upright.

Create, Train, and Simulate a TD3 Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rngState = rng(0,"twister")

rngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
td3Agent = rlTD3Agent(obsInfo,actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `td3Agent`.

```
td3Agent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
td3Agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
td3Agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
td3Agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
td3Agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
td3Agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
td3Agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
td3Agent.AgentOptions.ExperienceBufferLength = 1e6;
```

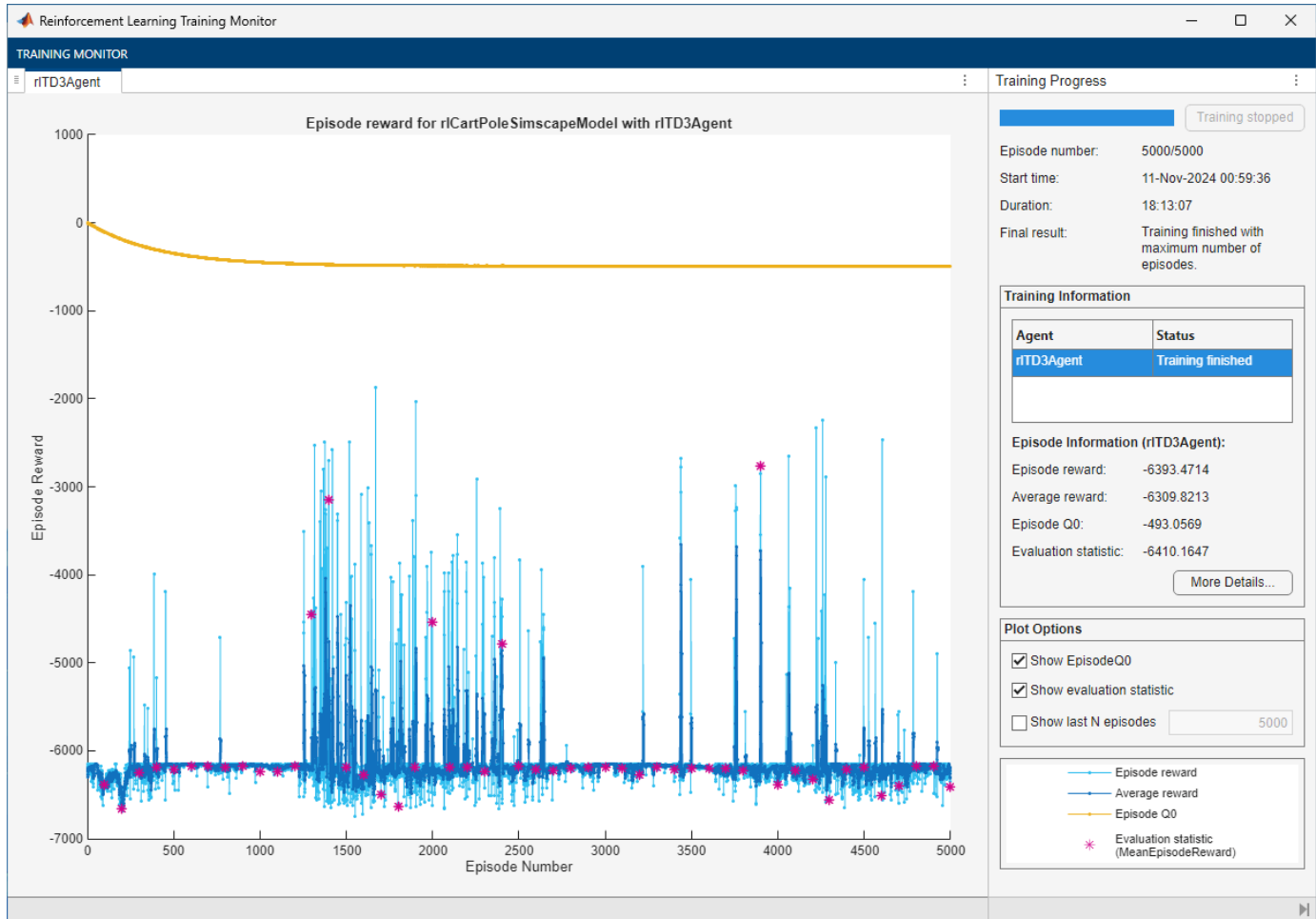
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    td3TngRes = train(td3Agent,env,trainOpts,Evaluator=evl);
    td3TngTime = toc;
    % Extract number of training episodes and total steps.
    td3TngEps = td3TngRes.EpisodeIndex(end);
    td3TngSteps = sum(td3TngRes.TotalAgentSteps);
    % Uncomment to save the trained agent and the training metrics.
```

```

% save("ccpsuBchTD3Agent.mat", ...
%     "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
else
% Load the pretrained agent and results for the example.
load("ccpsuBchTD3Agent.mat", ...
     "td3Agent","td3TngEps","td3TngSteps","td3TngTime")
end

```



For the TD3 Agent, the training converges to a solution. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env, td3Agent, simOptions);
td3TotalRwd = sum(experience.Reward)
```



```
td3TotalRwd =
-6.4102e+03
```

The trained TD3 agent is not able to swing up the pole.

Create, Train, and Simulate a SAC Agent

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rngState = rng(0, "twister")

rngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]
```

First, create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

To ensure that the RL Agent block in the environment executes every `Ts` seconds instead of the default setting of one second, set the `SampleTime` property of `sacAgent`.

```
sacAgent.AgentOptions.SampleTime = Ts;
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

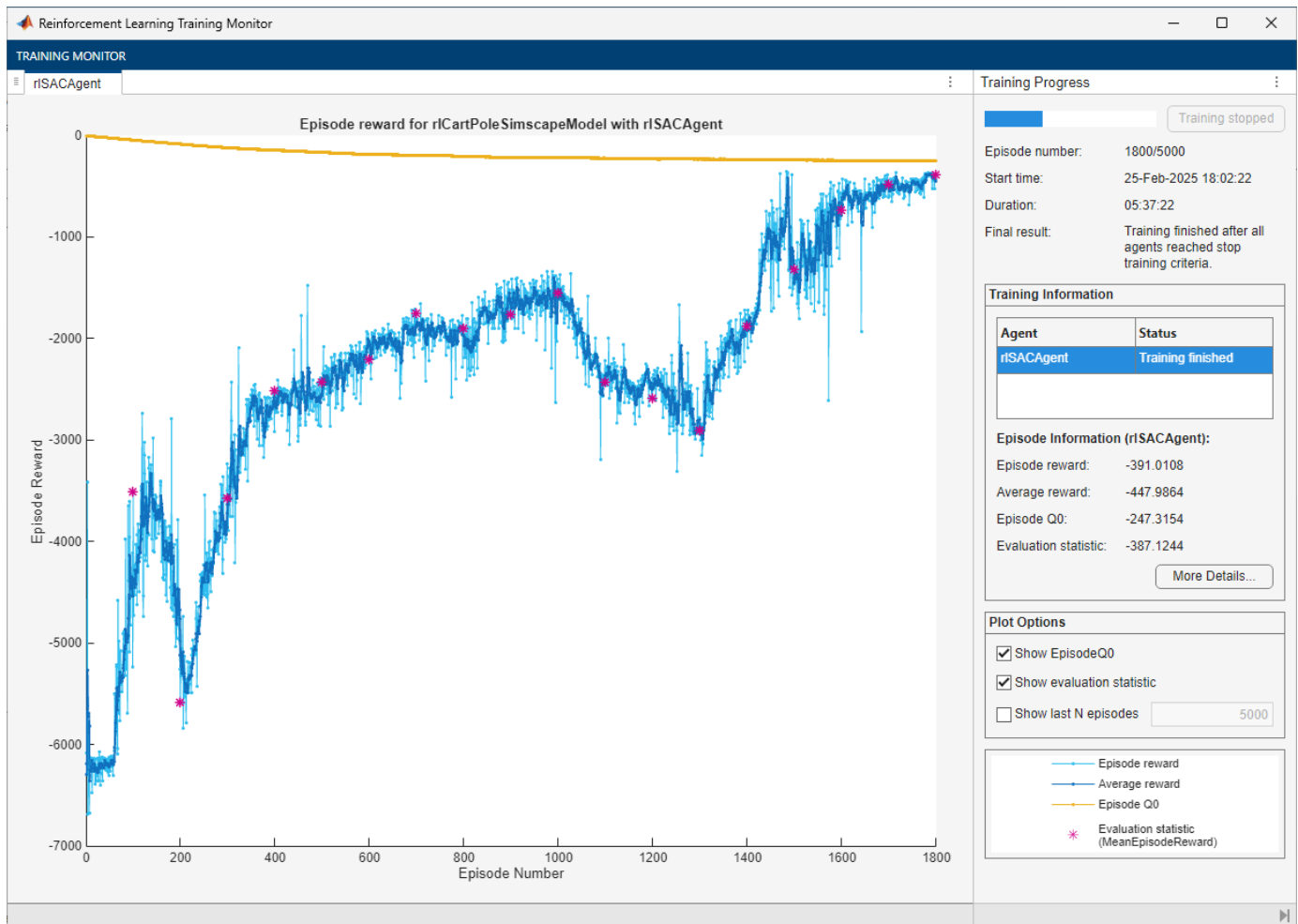
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    tic
    sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
```

```

sacTngTime = toc;
% Extract number of training episodes and total steps.
sacTngEps = sacTngRes.EpisodeIndex(end);
sacTngSteps = sum(sacTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("ccpsuBchSACAgent.mat", ...
%      "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
else
% Load the pretrained agent and results for the example.
load("ccpsuBchSACAgent.mat", ...
     "sacAgent","sacTngEps","sacTngSteps","sacTngTime")
end

```



For the SAC Agent, the training converges to a solution after 1800 episodes. You can check the trained agent within the cart-pole swing-up environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for `ceil(Tf/Ts)` steps. For more information on agent simulation, see `sim`.

```
experience = sim(env,sacAgent,simOptions);
sacTotalRwd = sum(experience.Reward)
```

```
sacTotalRwd =
-391.3531
```

The trained DDPG agent swings up the pole and maintains it upright.

Plot Training and Simulation Metrics

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor. Because the PPO agent training goes unstable, set its metrics to `NaN`.

```
simReward = [
    pgTotalRwd
    acTotalRwd
    NaN
    ddpgTotalRwd
    td3TotalRwd
    sacTotalRwd
];
```

```
tngEpisodes = [
    pgTngEps
    acTngEps
    NaN
    ddpgTngEps
    td3TngEps
    sacTngEps
];
```

```
tngSteps = [
    pgTngSteps
    acTngSteps
    NaN
    ddpgTngSteps
    td3TngSteps
    sacTngSteps
];
```

```
tngTime = [
    pgTngTime
    acTngTime
    NaN
    ddpgTngTime
    td3TngTime
    sacTngTime
];
```

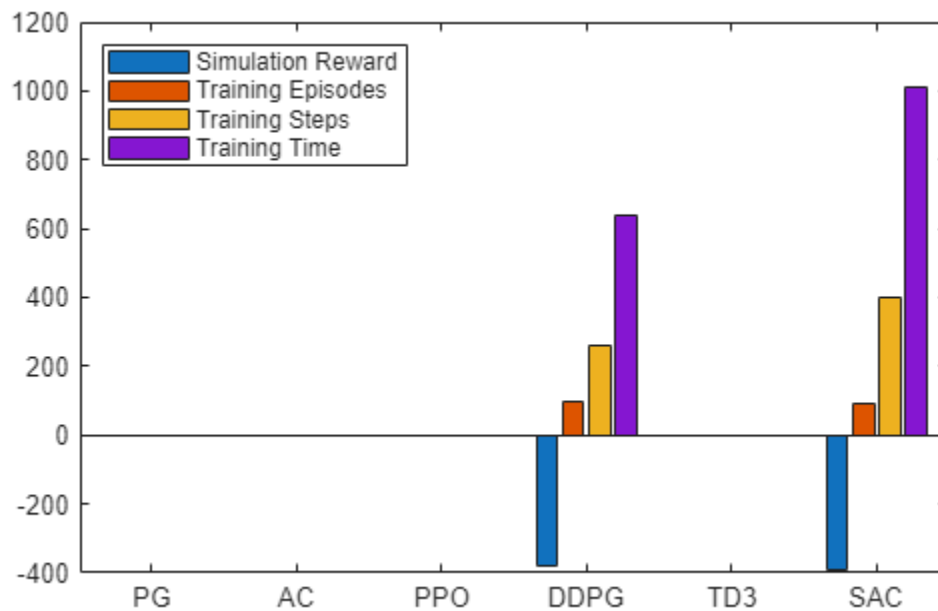
Because the training for the PG, AC, and TD3 agents does not converge, to avoid visualizing their metrics, set them to `NaN`.

```
simReward([1 2 5]) = NaN;
tngEpisodes([1 2 5]) = NaN;
```

```
tngSteps([1 2 5]) = NaN;
tngTime([1 2 5]) = NaN;
```

Plot the simulation reward, number of training episodes, number of training steps and training time. Scale the data by the factor [1 20 5e6 20] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 20 5e6 20])
xticklabels(["PG" "AC" "PPO" "DDPG" "TD3" "SAC"])
legend(["Simulation Reward","Training Episodes","Training Steps","Training Time"], ...
        "Location","northwest")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, only the DDPG and SAC agents stabilize the pole on the cart, with SAC taking a longer time (due to its more complex structure and the consequent need to calculate more gradients). With a different random seed, the initial agent networks would be different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables
% save ccpsuAllVars.mat
```

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

See Also

Functions

rlPredefinedEnv | getActionInfo | getObservationInfo | train | sim

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` |
`rlSimulationOptions`

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare Agents on Continuous Pendulum Swing-Up with Image

This example shows how to create and train frequently used default agents on a continuous action space pendulum swing-up environment. This environment represents a simple frictionless pendulum that initially hangs in a downward position. The agent, which senses the pendulum position from an image, can apply a control torque on the pendulum, and its goal is to make the pendulum stand upright using minimal control effort.

The example plots performance metrics such as the total training time and the total reward for each trained agent. The results that the agents obtain in this environment, with the selected initial conditions and random number generator seed, do not necessarily imply that specific agents are in general better than others. Also, note that the training times depend on the computer and operating system you use to run the example, and on other processes running in the background. Your training times might differ substantially from the training times shown in the example.

Due to the large observation space, the consequently large size of the default neural networks used in the agents, and the fact that the maximum number of training episodes is reached for many agents (because training often does not converge), the example might take days to run, even when using a GPU. To run the complete example, set `doTraining` to true. Otherwise, to execute only the result section, keep `doTraining` to false.

```
doTraining =  ;
if doTraining
```

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Simple Pendulum with Image MATLAB Environment

The reinforcement learning environment for this example is a simple frictionless pendulum that initially hangs in a downward position. The training goal is to make the pendulum stand upright using minimal control effort.

For this environment:

- The balanced, upright pendulum position is zero radians, and the downward hanging pendulum position is π radians.
- The torque action signal from the agent to the environment is from -2 to 2 N·m.

- The observations from the environment are an image indicating the location of the pendulum mass and the pendulum angular velocity.
- The reward r_t , provided at every time step, is

$$r_t = -\left(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.001u_{t-1}^2\right)$$

Here:

- θ_t is the angle of displacement from the upright position.
- $\dot{\theta}_t$ is the derivative of the displacement angle.
- u_{t-1} is the control effort from the previous time step.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

Create Environment Object

Create a predefined environment object for the continuous pendulum environment.

```
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous")
```

The environment reset function initializes and returns the environment state (linear and angular positions and velocities).

```
reset(env)
```

You can visualize the pendulum system using the `plot` function during training or simulation.

```
plot(env)
```

Obtain the observation and action information for later use when creating agents.

```
obsInfo = getObservationInfo(env)
actInfo = getActionInfo(env)
```

Configure Training Options for All Agents

Set up an evaluator object to evaluate the agent 10 times without exploration every 100 training episodes.

```
evl = rlEvaluator(NumEpisodes=10,EvaluationFrequency=100);
```

Create a training options object. For this example, use the following options.

- Run the training for a maximum of 5000 episodes, with each episode lasting 500 time steps.
- Stop the training when the agent receives an average cumulative reward greater than -1000 over the consecutive evaluation episodes. At this point, the agent can quickly balance the pendulum upright using minimal control effort.
- To have a better insight on the agent's behavior during training, plot the training progress (default option). If you want to achieve faster training times, set the `Plots` option to `none`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=500, ...
```

```
StopTrainingCriteria="EvaluationStatistic", ...  
StopTrainingValue=-1000);
```

For more information on training options, see `rlTrainingOptions`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 500 steps.

```
simOptions = rlSimulationOptions(MaxSteps=500);
```

For more information on simulation options, see `rlSimulationOptions`.

Create, Train, and Simulate a PG Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPGAgent` object using the environment specification objects.

```
pgAgent = rlPGAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
pgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
pgAgent.AgentOptions.EntropyLossWeight = 0.005;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpu rng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0,"threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(pgAgent);  
    actor.UseDevice = "gpu";  
    setActor(pgAgent,actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(pgAgent);  
    critic.UseDevice = "gpu";  
    setCritic(pgAgent,critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

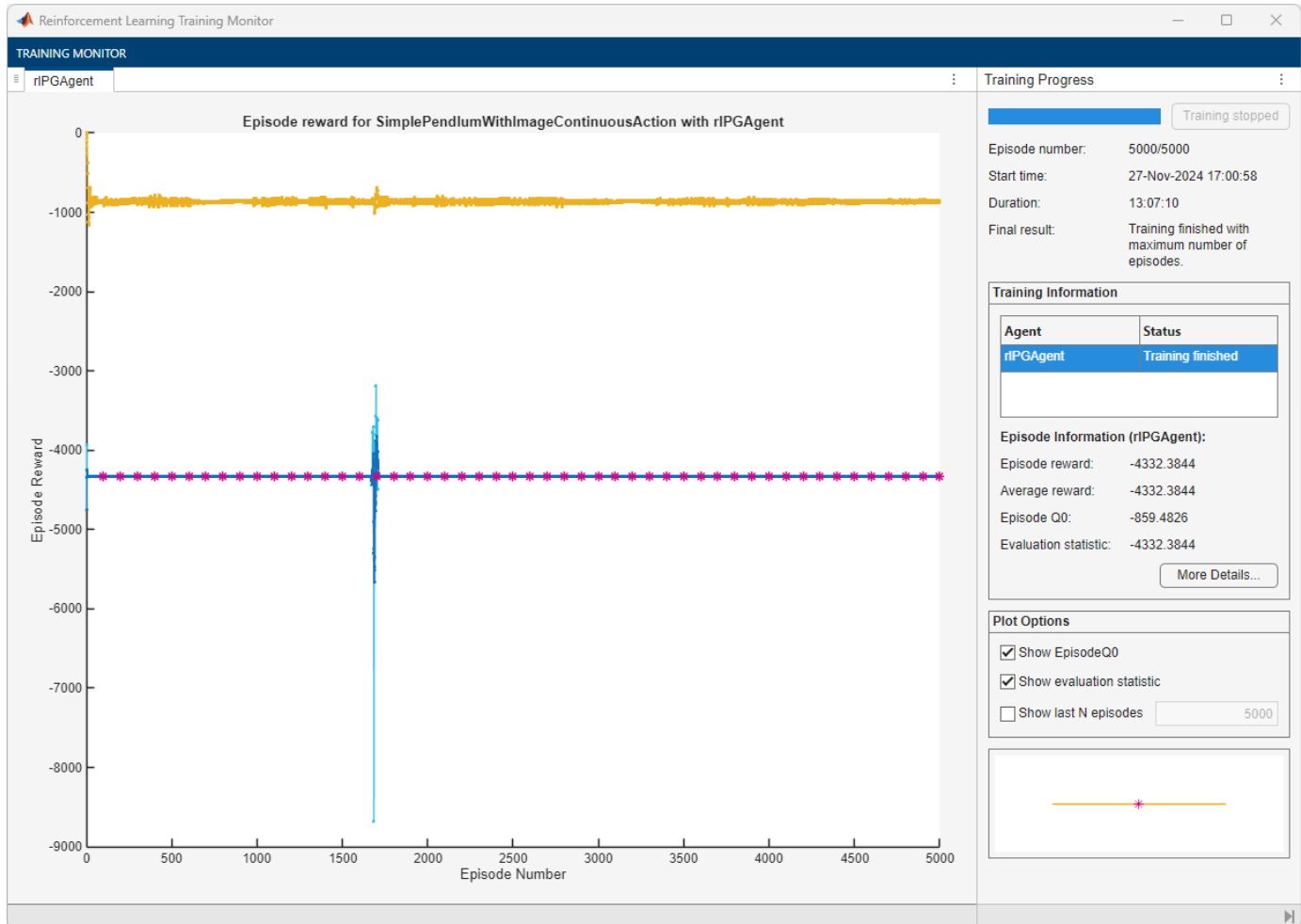
```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");  
% Train the agent. Record the training time.
```



```

tic
pgTngRes = train(pgAgent,env,trainOpts,Evaluator=evl);
pgTngTime = toc;
% Extract number of training episodes and total steps.
pgTngEps = pgTngRes.EpisodeIndex(end);
pgTngSteps = sum(pgTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cpsuImgBchPGData.mat", ...
%      "pgAgent","pgTngEps","pgTngSteps","pgTngTime")

```



For the PG agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0,"twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,pgAgent,simOptions);  
pgTotalRwd = sum(experience.Reward)
```

The trained PG agent is not able to swing up the pendulum.

Create, Train, and Simulate an AC Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlACAgent` object using the environment specification objects.

```
acAgent = rlACAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
acAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
acAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
acAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the entropy loss weight to increase exploration.

```
acAgent.AgentOptions.EntropyLossWeight = 0.005;
```

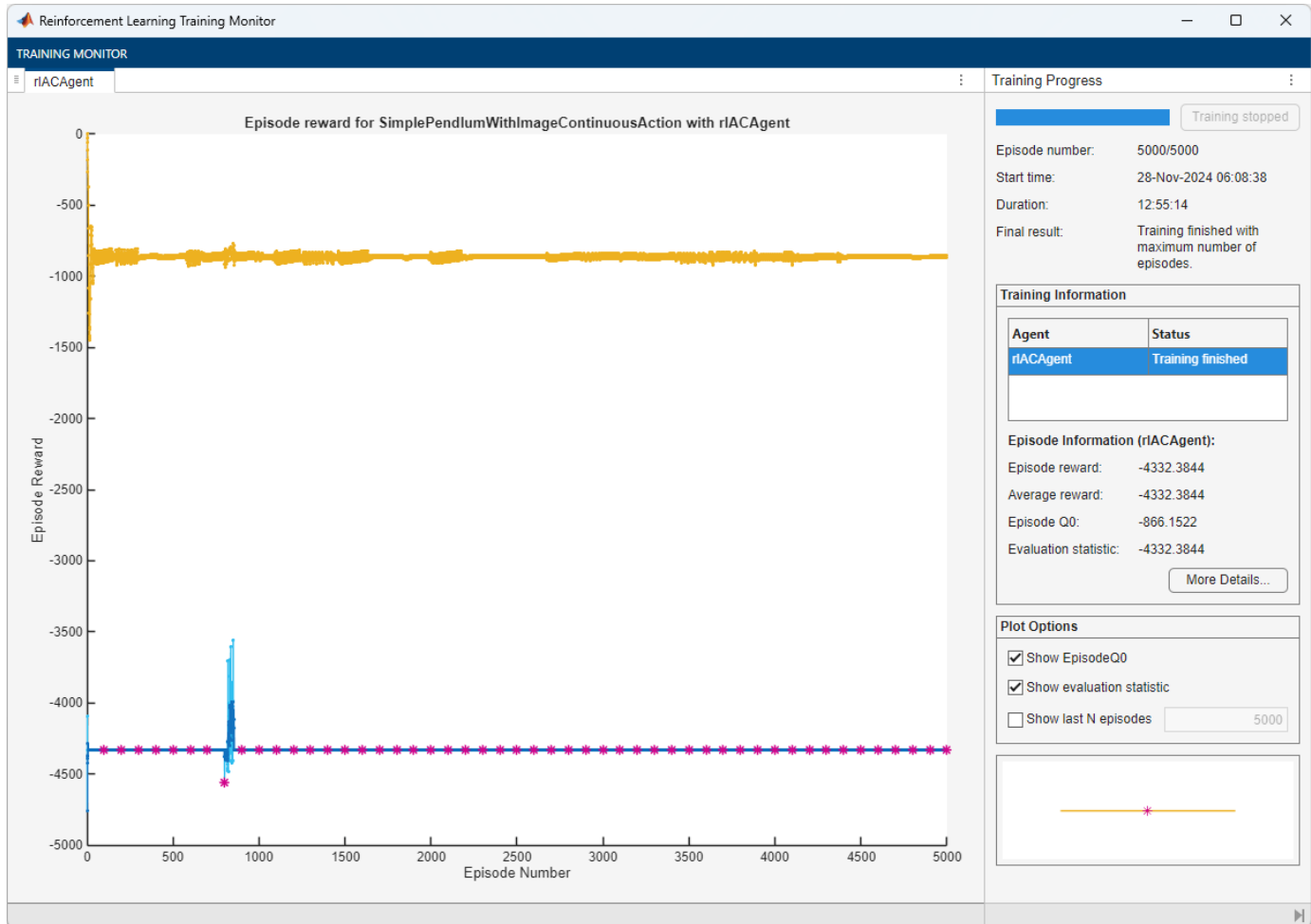
If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0,"threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(acAgent);  
    actor.UseDevice = "gpu";  
    setActor(acAgent,actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(acAgent);  
    critic.UseDevice = "gpu";  
    setCritic(acAgent,critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");  
% Train the agent. Record the training time.  
tic  
acTngRes = train(acAgent,env,trainOpts,Evaluator=evl);  
acTngTime = toc;  
% Extract number of training episodes and total steps.
```

```
acTngEps = acTngRes.EpisodeIndex(end);
acTngSteps = sum(acTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cpsuImgBchACData.mat", ...
%      "acAgent", "acTngEps", "acTngSteps", "acTngTime")
```



For the AC agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,acAgent,simOptions);
acTotalRwd = sum(experience.Reward)
```

The trained AC agent is not able to swing up the pendulum.

Create, Train, and Simulate a PPO Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0,"twister")
```

First, create a default `rlPPOAgent` object using the environment specification objects.

```
ppoAgent = rlPPOAgent(obsInfo,actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
ppoAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
ppoAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
ppoAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU
    % Fix random seed
    gpurng(0,"threefry")
    % Use GPU for actor gradient calculations
    actor = getActor(ppoAgent);
    actor.UseDevice = "gpu";
    setActor(ppoAgent,actor);
    % Use GPU for critic gradient calculations
    critic = getCritic(ppoAgent);
    critic.UseDevice = "gpu";
    setCritic(ppoAgent,critic);
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");
% Train the agent. Record the training time.
tic
ppoTngRes = train(ppoAgent,env,trainOpts,Evaluator=evl);
ppoTngTime = toc;
% Extract number of training episodes and total steps.
ppoTngEps = ppoTngRes.EpisodeIndex(end);
ppoTngSteps = sum(ppoTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cpsuImgBchPPOData.mat", ...
%     "ppoAgent","ppoTngEps","ppoTngSteps","ppoTngTime")
```



For the PPO Agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ppoAgent, simOptions);
ppoTotalRwd = sum(experience.Reward)
```

The trained PPO agent does not swing up the pendulum.

Create, Train, and Simulate a DDPG Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rlDDPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

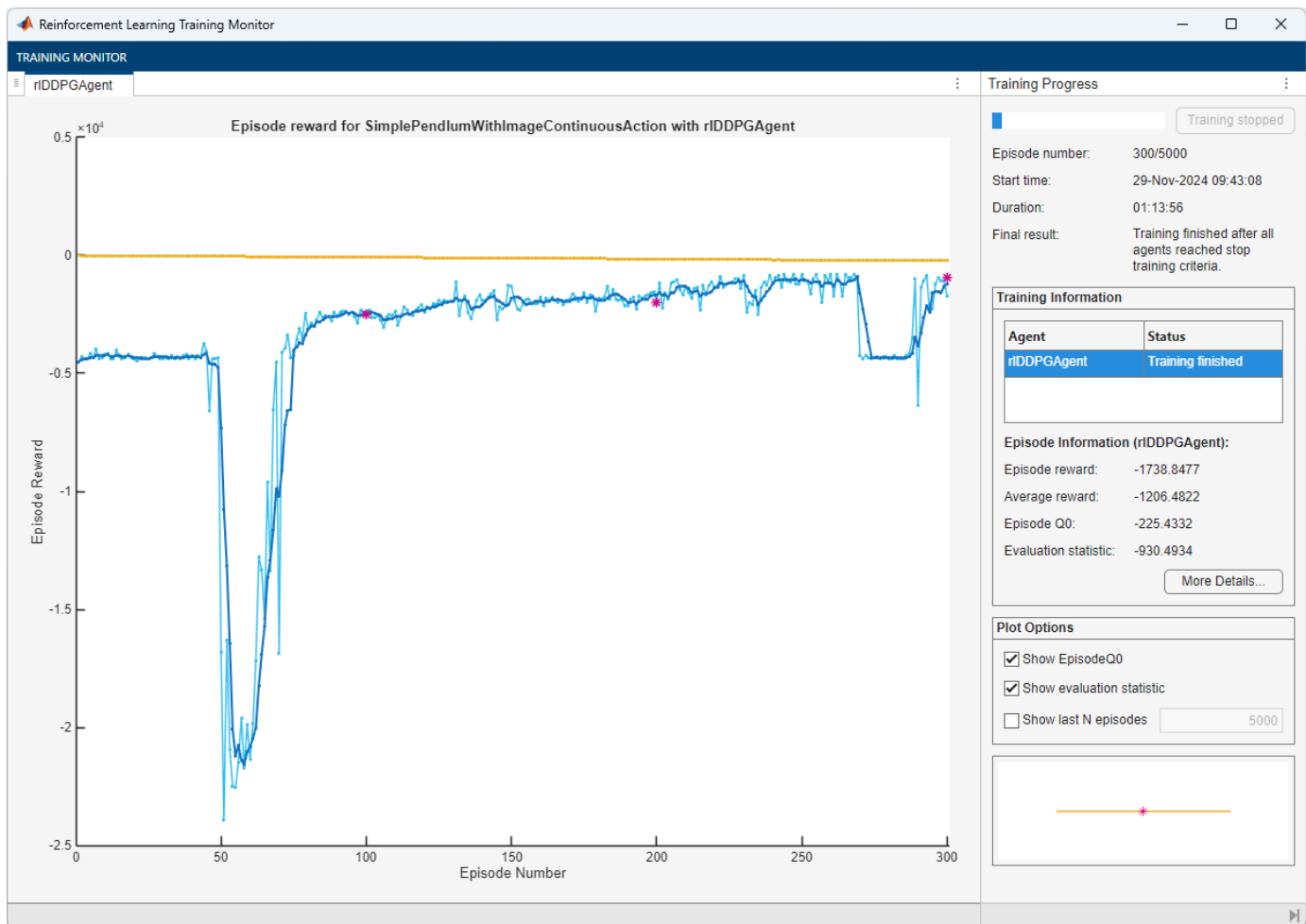
```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0, "threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(ddpgAgent);  
    actor.UseDevice = "gpu";  
    setActor(ddpgAgent, actor);  
    % Use GPU for critic gradient calculations  
    critic = getCritic(ddpgAgent);  
    critic.UseDevice = "gpu";  
    setCritic(ddpgAgent, critic);  
end
```

Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");  
% Train the agent. Record the training time.  
tic  
ddpgTngRes = train(ddpgAgent, env, trainOpts, Evaluator=evl);  
ddpgTngTime = toc;  
% Extract number of training episodes and total steps.  
ddpgTngEps = ddpTngRes.EpisodeIndex(end);  
ddpgTngSteps = sum(ddpgTngRes.TotalAgentSteps);  
% Uncomment to save the trained agent and the training metrics.  
% save("cpsuImgBchDDPGData.mat", ...  
%     "ddpgAgent", "ddpgTngEps", "ddpgTngSteps", "ddpgTngTime")
```



For the DDPG Agent, the training converges to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, ddpGAgent, simOptions);
ddpgTotalRwd = sum(experience.Reward)
```

The agent does not swing up the pendulum.

Create, Train, and Simulate a TD3 Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlDDPGAgent` object using the environment specification objects.

```
td3Agent = rlTD3Agent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
td3Agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
td3Agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
td3Agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
td3Agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
td3Agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
td3Agent.AgentOptions.ExperienceBufferLength = 1e6;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0, "threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(td3Agent);  
    actor.UseDevice = "gpu";  
    setActor(td3Agent, actor);  
    % Use GPU for critic gradient calculations  
    critics = getCritic(td3Agent);  
    critics(1).UseDevice = "gpu";  
    critics(2).UseDevice = "gpu";  
    setCritic(td3Agent, critics);  
end
```

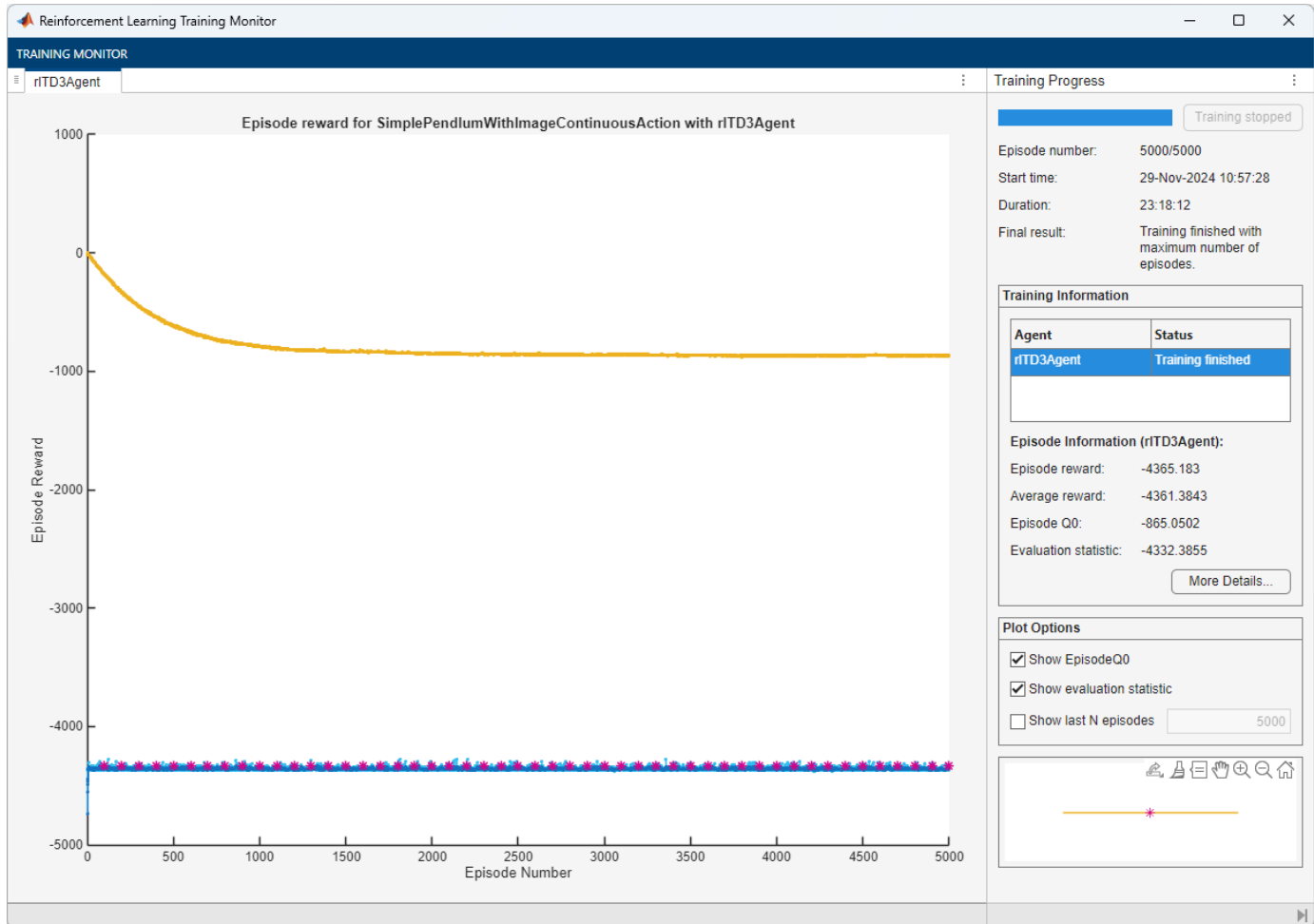
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");  
% Train the agent. Record the training time.  
tic  
td3TngRes = train(td3Agent, env, trainOpts, Evaluator=evl);  
td3TngTime = toc;  
% Extract number of training episodes and total steps.  
td3TngEps = td3TngRes.EpisodeIndex(end);  
td3TngSteps = sum(td3TngRes.TotalAgentSteps);  
% Uncomment to save the trained agent and the training metrics.
```



```
% save("cpsuImgBchTD3Data.mat", ...
%      "td3Agent", "td3TngEps", "td3TngSteps", "td3TngTime")
```

For the TD3 Agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.



Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env, td3Agent, simOptions);
td3TotalRwd = sum(experience.Reward)
```

The trained TD3 agent does not swing up the pendulum.

Create, Train, and Simulate a SAC Agent

The actor and critic networks are initialized stochastically. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

First, create a default `rlSACAgent` object using the environment specification objects.

```
sacAgent = rlSACAgent(obsInfo, actInfo);
```

Set a lower learning rate and a lower gradient threshold to promote a smoother (though possibly slower) training.

```
sacAgent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;  
sacAgent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;  
sacAgent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
```

```
sacAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
sacAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Set the initial entropy weight and target entropy to increase exploration.

```
sacAgent.AgentOptions.EntropyWeightOptions.EntropyWeight = 5e-3;  
sacAgent.AgentOptions.EntropyWeightOptions.TargetEntropy = 5e-1;
```

Use a larger experience buffer to store more experiences, therefore decreasing the likelihood of catastrophic forgetting.

```
sacAgent.AgentOptions.ExperienceBufferLength = 1e6;
```

If the computer can use the GPU, fix the random generator seed used on the GPU for reproducibility, and set the actor and critic to use the GPU for gradient calculation. For more information, see `canUseGPU` and `gpurng` (Parallel Computing Toolbox).

```
if canUseGPU  
    % Fix random seed  
    gpurng(0, "threefry")  
    % Use GPU for actor gradient calculations  
    actor = getActor(sacAgent);  
    actor.UseDevice = "gpu";  
    setActor(sacAgent, actor);  
    % Use GPU for critic gradient calculations  
    critics = getCritic(sacAgent);  
    critics(1).UseDevice = "gpu";  
    critics(2).UseDevice = "gpu";  
    setCritic(sacAgent, critics);  
end
```

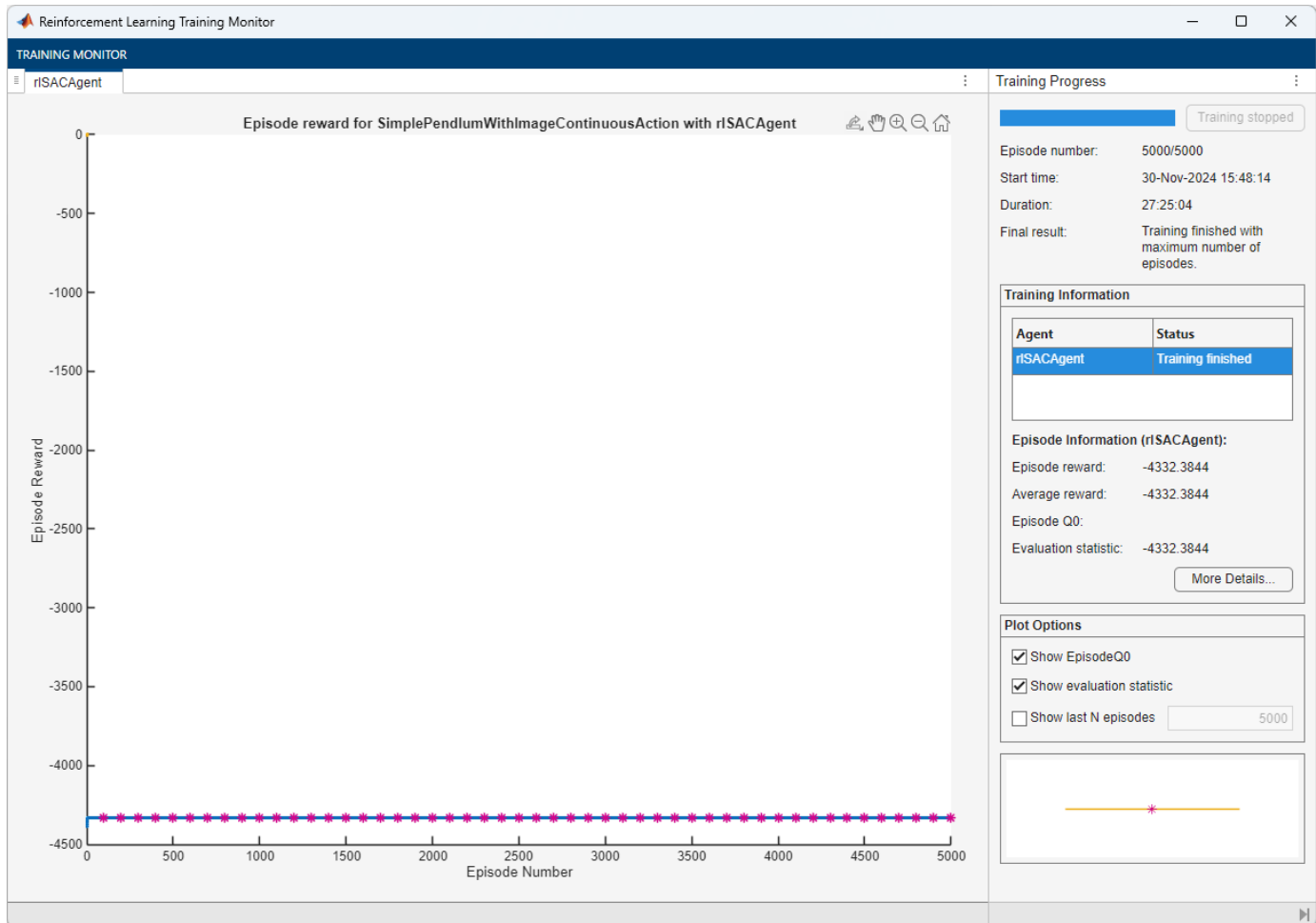
Train the agent, passing the agent, the environment, and the previously defined training options and evaluator objects to `train`.

```
% Recreate the environment so it does not plot during training.  
env = rlPredefinedEnv("SimplePendulumWithImage-Continuous");  
% Train the agent. Record the training time.  
tic  
sacTngRes = train(sacAgent, env, trainOpts, Evaluator=evl);
```

```

sacTngTime = toc;
% Extract number of training episodes and total steps.
sacTngEps = sacTngRes.EpisodeIndex(end);
sacTngSteps = sum(sacTngRes.TotalAgentSteps);
% Uncomment to save the trained agent and the training metrics.
% save("cpsuImgBchSACData.mat", ...
%      "sacAgent", "sacTngEps", "sacTngSteps", "sacTngTime")

```



For the SAC Agent, the training does not converge to a solution. You can check the trained agent within the pendulum environment.

Ensure reproducibility of the simulation by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Visualize the environment.

```
plot(env)
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the environment with the trained agent for 500 steps and display the total reward. For more information on agent simulation, see `sim`.

```
experience = sim(env,sacAgent,simOptions);  
sacTotalRwd = sum(experience.Reward)
```

The trained SAC agent is not able to swing up the pendulum.

Terminate All Agents Training and Simulation Part

Save all the variables created in this example, including the training results, for later use.

```
% Uncomment to save all the workspace variables  
% save cpsuImgAllVars.mat
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

End the training and simulation part of the example.

```
end
```

Plot Training and Simulation Metrics

If the training and simulation part of the example executes, collect all the agent metrics from the existing workspace variables.

```
if doTraining
```

For each agent, collect the total reward from the final simulation episode, the number of training episodes, the total number of agent steps, and the total training time as shown in the Reinforcement Learning Training Monitor.

```
simReward = [  
    pgTotalRwd  
    acTotalRwd  
    ppoTotalRwd  
    ddpGTotalRwd  
    td3TotalRwd  
    sacTotalRwd  
];
```

```
tngEpisodes = [  
    pgTngEps  
    acTngEps  
    ppoTngEps  
    ddpGtngEps  
    td3TngEps  
    sacTngEps  
];
```

```
tngSteps = [  
    pgTngSteps  
    acTngSteps  
    ppoTngSteps  
    ddpGtngSteps  
    td3TngSteps  
    sacTngSteps  
];
```

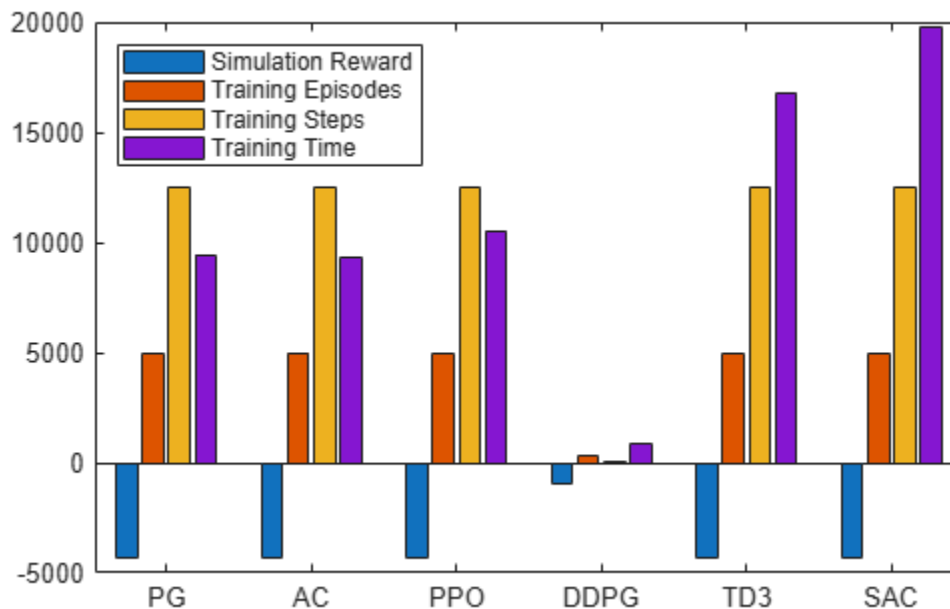
```
tngTime = [
    pgTngTime
    acTngTime
    ppoTngTime
    ddpdTngTime
    td3TngTime
    sacTngTime
];
```

If the training and simulation part does not execute, load the metrics from the supporting file.

```
else
load("cpsuImgBchResults.mat")
end
```

Plot the simulation reward, number of training episodes, number of training steps and training time. Scale the data by the factor [1 1 5e5 5] for better visualization.

```
bar([simReward,tngEpisodes,tngSteps,tngTime]./[1 1 5e5 5])
xticklabels(["PG" "AC" "PPO" "DDPG" "TD3" "SAC"])
legend(["Simulation Reward","Training Episodes", ...
        "Training Steps","Training Time"], ...
        "Location","northwest")
```



The plot shows that, for this environment, and with the used random number generator seed and initial conditions, only the DDPG converges to a solution and is able to swing up the pendulum. As expected, TD3 and SAC take a longer time due to their more complex algorithms, in which more gradients need to be calculated. With a different random seed, the initial agent networks would be

different, and therefore, convergence results might be different. For more information on the relative strengths and weaknesses of each agent, see “Reinforcement Learning Agents” on page 3-2.

See Also

Functions

`rlPredefinedEnv` | `getActionInfo` | `getObservationInfo` | `train` | `sim`

Objects

`rlEvaluator` | `rlAgentInitializationOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Train MBPO Agent to Balance Continuous Cart-Pole System” on page 5-356

More About

- “Use Predefined Control System Environments” on page 2-42
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Compare DDPG Agent to LQR Controller

This example shows how to train a deep deterministic policy gradient (DDPG) agent to control a second-order linear dynamic system modeled in MATLAB®. The example also compares a DDPG agent with a custom quadratic approximation model to an LQR controller.

For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67. For an example showing how to train a DDPG agent in a Simulink® environment, see “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78 and “Control Water Level in a Tank Using a DDPG Agent” on page 8-2.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

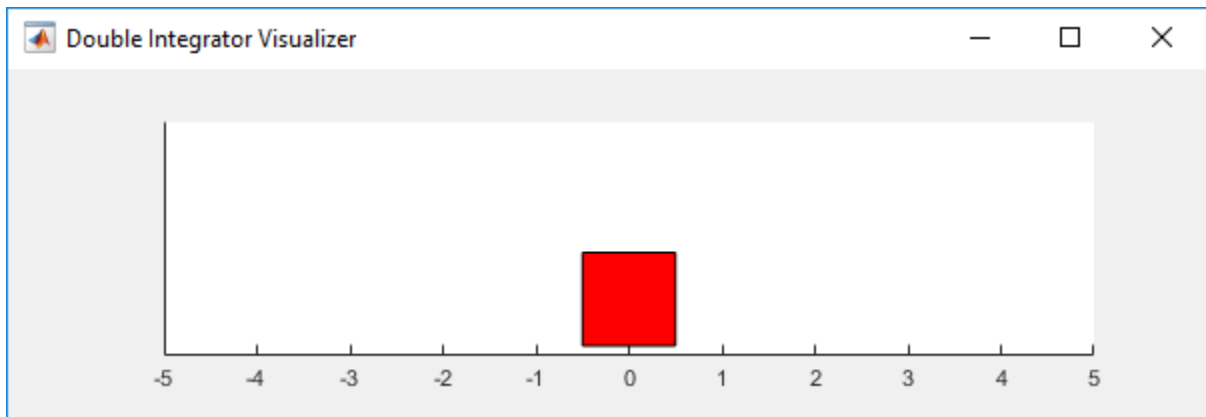
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Continuous Action Space Double-Integrator MATLAB Environment

The reinforcement learning environment for this example is a second-order double-integrator system with a gain. The training goal is to control the position of a mass in the second-order system by applying a force input.



For this environment:

- The mass starts at an initial position of either -4 or 4 m.
- The observations from the environment are the position and velocity of the mass.
- The episode terminates if the mass moves more than 5 m from the original position or if $|x| < 0.01$.

- The reward r_t , provided at every time step, is a discretization of $r(t)$:
$$r(t) = -(\dot{x}(t)' Q \dot{x}(t) + u(t)' R u(t))$$

Here:

- $\dot{x}$ is the state vector of the mass.
- u is the force applied to the mass.
- Q is the matrix of weight on the state deviation from zero; $Q = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$.
- R is the weight on the control effort; $R = 0.01$.

For more information on this model, see “Use Predefined Control System Environments” on page 2-42.

For this example the environment is a linear dynamic system, the environment state is observed directly, and the reward is a quadratic function of the observation and action. Therefore the problem of finding the sequence of actions that minimizes the cumulative long-term reward is a discrete-time linear-quadratic optimal control problem, for which the optimal action is known to be a linear function of the system states. This problem can also be solved using Linear-Quadratic Regulator (LQR) design, and in the last part of the example you can compare the agent to an LQR controller.

Create Environment Object

Create a predefined environment object for the continuous action space double-integrator system.

```
env = rlPredefinedEnv("DoubleIntegrator-Continuous")
```

```
env =  
    DoubleIntegratorContinuousAction with properties:
```

```
        Gain: 1  
        Ts: 0.1000  
    MaxDistance: 5  
    GoalThreshold: 0.0100  
        Q: [2x2 double]  
        R: 0.0100  
    MaxForce: Inf  
    State: [2x1 double]
```

The agent can apply force values from `-Inf` to `Inf` to the mass. The sample time is stored in `env.Ts`, while the continuous time cost function matrices are stored in `env.Q` and `env.R` respectively.

Obtain the observation and action specifications from the environment.

```
obsInfo = getObservationInfo(env)
```

```
obsInfo =  
    rlNumericSpec with properties:
```

```
    LowerLimit: -Inf  
    UpperLimit: Inf  
        Name: "states"  
    Description: "x, dx"  
    Dimension: [2 1]  
    DataType: "double"
```



```
actInfo = getActionInfo(env)

actInfo =
    rlNumericSpec with properties:

        LowerLimit: -Inf
        UpperLimit: Inf
        Name: "force"
        Description: [0x0 string]
        Dimension: [1 1]
        DataType: "double"
```

Reset the environment and get its initial state.

```
x0 = reset(env)

x0 = 2x1

    4
    0
```

Create a DDPG Agent

A DDPG agent approximates the discounted cumulative long-term reward using a Q-value function critic. A Q-value function critic must accept an observation and an action as inputs and return a scalar (the estimated discounted cumulative long-term reward) as output. To approximate the Q-value function within the critic, use a neural network. Because the value function of the optimal policy is known to be quadratic, use a network with a quadratic layer (which outputs a vector of quadratic monomials, as described in `quadraticLayer`) and a fully connected layer (which provides a linear combination of its inputs).

Define each network path as an array of layer objects, getting the dimension of the observation and action spaces from the environment specification objects. Assign names to the network input layers, so you can connect them to the output path and later explicitly associate them with the appropriate environment channel. Because there is no need for a bias term, set the bias term to zero (`Bias=0`) and prevent it from changing (`BiasLearnRateFactor=0`).

For more information on creating value function approximators, see “Create Actors, Critics, and Policy Objects” on page 4-2.

```
% Observation and action paths
obsPath = featureInputLayer(obsInfo.Dimension(1),Name="obsInLyr");
actPath = featureInputLayer(actInfo.Dimension(1),Name="actInLyr");

% Common path
commonPath = [
    concatenationLayer(1,2,Name="concat")
    quadraticLayer
    fullyConnectedLayer(1,Name="value", ...
        BiasLearnRateFactor=0,Bias=0)
];
```

Create `dlnetwork` object and add layers.

```
criticNet = dlnetwork;
criticNet = addLayers(criticNet,obsPath);
```

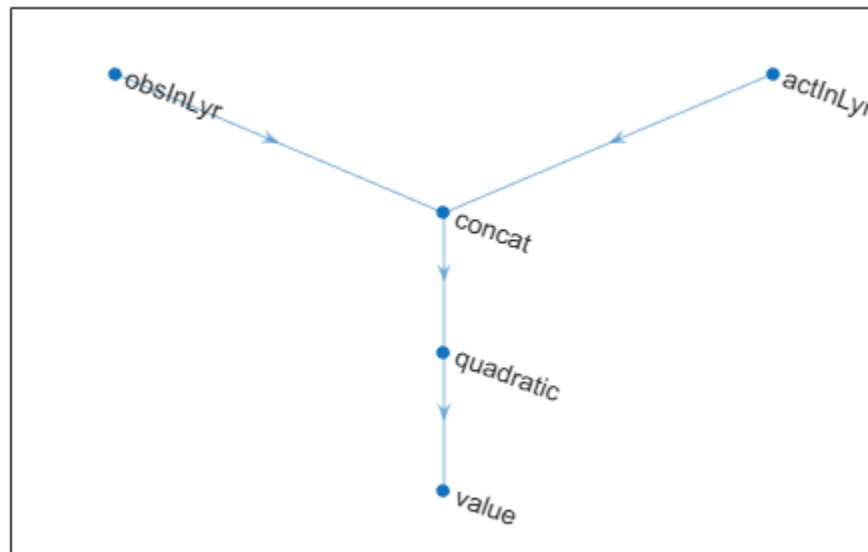
```
criticNet = addLayers(criticNet,actPath);
criticNet = addLayers(criticNet,commonPath);
```

Connect layers.

```
criticNet = connectLayers(criticNet,"obsInLyr","concat/in1");
criticNet = connectLayers(criticNet,"actInLyr","concat/in2");
```

View the critic network configuration.

```
plot(criticNet)
```



Initialize network and display the number of weights.

```
criticNet = initialize(criticNet);
summary(criticNet)
```

```
Initialized: true
```

```
Number of learnables: 7
```

```
Inputs:
```

```
  1  'obsInLyr'   2 features
  2  'actInLyr'   1 features
```

Create the critic object using `criticNet`, the environment observation and action specifications, and the names of the network input layers to be connected with the environment observation and action channels, respectively. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(criticNet, ...
    obsInfo,actInfo, ...
    ObservationInputNames="obsInLyr", ...
    ActionInputNames="actInLyr");
```

Check the critic with a random observation and action input.

```
getValue(critic,{rand(obsInfo.Dimension)},{rand(actInfo.Dimension)})
```

```
ans = single
```

```
-0.3762
```

DDPG agents use a parameterized continuous deterministic policy, which is learned by a continuous deterministic actor. This actor must accept an observation as input and return an action as output. To approximate the policy function within the actor, use a neural network. Because for this example the optimal policy is known to be linear in the state, use a shallow network with a fully connected layer to provide a linear combination of the two elements of the observation (position and velocity of the mass).

Define the network as an array of layer objects, and get the dimension of the observation and action spaces from the environment specification objects. Because there is no need for a bias term, as done for the critic, set the bias term to zero (**Bias=0**) and prevent it from changing (**BiasLearnRateFactor=0**). For more information on actors, see “Create Actors, Critics, and Policy Objects” on page 4-2.

```
actorNet = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(actInfo.Dimension(1), ...
        BiasLearnRateFactor=0,Bias=0)
];
```

Convert to `dlnetwork`, initialize network, and display the number of weights.

```
actorNet = dlnetwork(actorNet);
actorNet = initialize(actorNet);
summary(actorNet)
```

```
Initialized: true
```

```
Number of learnables: 3
```

```
Inputs:
    1 'input'    2 features
```

Create the actor using `actorNet` and the observation and action specifications. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNet,obsInfo,actInfo);
```

Check the actor with a random observation input.

```
getAction(actor,{rand(obsInfo.Dimension)})
```

```
ans = 1x1 cell array
    {-0.2809}
```

Create the DDPG agent using the actor and critic. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic);
```

Specify options for the agent, including training options for the critic, using dot notation. Alternatively, you can use `rlDDPGAgentOptions`, and `rlOptimizerOptions` objects before creating the agent.

Set the agent sample time. This ensures that the time interval between consecutive elements in the returned experience is T_s . It also ensures that if you use the agent in an RL Agent block, the block executes at intervals of T_s seconds.

```
agent.AgentOptions.SampleTime = env.Ts;
```

Specify a capacity of $1e6$ for the experience buffer to store a diverse set of experiences. Also specify a mini-batch size of 32 for learning. Smaller mini-batches are computationally efficient though they might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.

```
agent.AgentOptions.ExperienceBufferLength = 1e5;  
agent.AgentOptions.MiniBatchSize = 32;
```

Specify a decay rate of $1e-5$ for the noise standard deviation to gradually decay during training. This promotes exploration toward the beginning when the agent does not have a good policy, and exploitation toward the end when the agent has learned the optimal policy.

```
agent.AgentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-5;
```

Specify a learning rate of $1e-3$ for the agent actor and critic optimizer algorithms. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point. Also specify a gradient threshold value of 1.0 to clip the computed gradients to enhance the stability of the learning algorithm.

```
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 5e-3;  
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

Initialize Agent Parameters

Extract Parameters from Agent

Extract the actor and critic parameters from the agent.

```
par = getLearnableParameters(agent);
```

Actor Parameters

The policy implemented by the actor is $u = K_1x_1 + K_2x_2 = Kx$, where the feedback gains K_1 and K_2 are the two weights of the actor network. It can be shown that the closed loop system is stable if these gains are negative, therefore, initializing them to negative values can speed up convergence.

Set the gains K_1 and K_2 , to -1.

```
K = [-1 -1];
```

Set the actor parameters to K .

```
par.Actor{1} = single(K);
```

Critic Parameters

Because the critic uses `quadraticLayer` followed by a `fullyConnectedLayer`, the Q-value function has the following structure:

$$Q(x, u) = W_1 x_1^2 + W_2 x_1 x_2 + W_3 x_2^2 + W_4 x_1 u + W_5 x_2 u + W_6 u^2$$

where W_i are the weights of the fully connected layer. Alternatively, in matrix form:

$$Q(x, u) = [x_1 \ x_2 \ u] \begin{bmatrix} W_1 & \frac{W_2}{2} & \frac{W_4}{2} \\ \frac{W_2}{2} & W_3 & \frac{W_5}{2} \\ \frac{W_4}{2} & \frac{W_5}{2} & W_6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ u \end{bmatrix} = [x^T \ u] W_Q \begin{bmatrix} x \\ u \end{bmatrix}.$$

For a fixed policy $u = Kx$, the cumulative long-term reward (that is, the value of the policy) becomes:

$$V(x) = Q(x, Kx) = x^T [I \ K^T] W_Q \begin{bmatrix} I \\ K \end{bmatrix} x = x^T P x.$$

Because the rewards are always negative, to properly approximate the cumulative reward, both matrices P and W_Q must be negative definite. Therefore, to speed up convergence, initialize the critic network weights W_i so that the matrix W_Q is negative definite. Set the critic parameter vector W to $[-1.1, -0.1, -1.1, -0.1, -0.1, -1.1]$.

$W = [-1, -0.1, -1, -0.1, -0.1, -1.1];$

Build the matrix W_Q .

$WQ = [W(1) \ W(2)/2 \ W(4)/2; \ W(2)/2 \ W(3) \ W(5)/2; \ W(4)/2 \ W(5)/2 \ W(6)];$

Build the matrix P .

$P = [\text{eye}(2) \ K'] * WQ * [\text{eye}(2); K];$

Check that both matrices have negative eigenvalues:

`eig(WQ)`

`ans = 3×1`

```
-1.1500
-1.0000
-0.9500
```

`eig(P)`

`ans = 2×1`

```
-3.0500
-0.9500
```

Set the critic parameters to the vector W .

```
par.Critic{1} = single(W);
```

Set Parameters back in the Agent and Check Agent

Set the parameters of both actor and critic in the agent.

```
setLearnableParameters(agent,par);
```

Check the agent with a random observation input.

```
getAction(agent,{rand(obsInfo.Dimension)}))
```

```
ans = 1x1 cell array  
      {[-0.9422]}
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

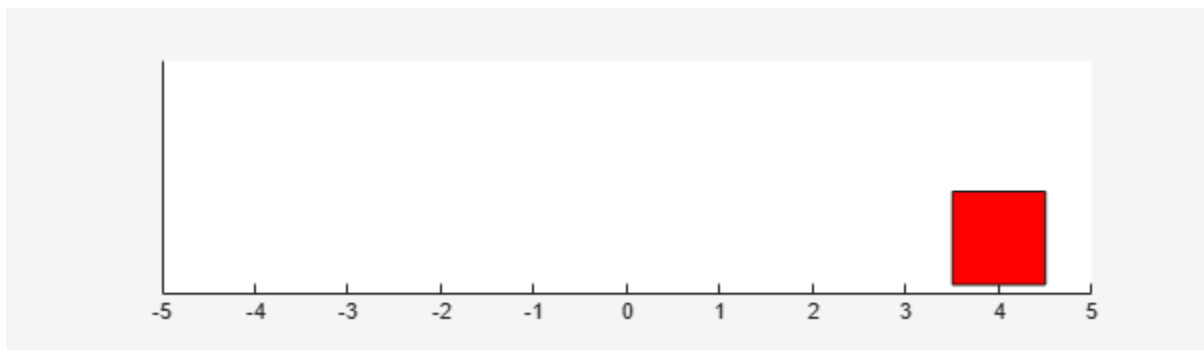
- Run a maximum of 5000 episodes in the training session, with each episode lasting a maximum of 200 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (`Verbose` option).
- Stop the training when the agent receives a moving average cumulative reward (over the default value of 5 episodes) greater than -66 . At this point, the agent can control the position of the mass using minimal control effort.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=5000, ...  
    MaxStepsPerEpisode=200, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="AverageReward", ...  
    StopTrainingValue=-66);
```

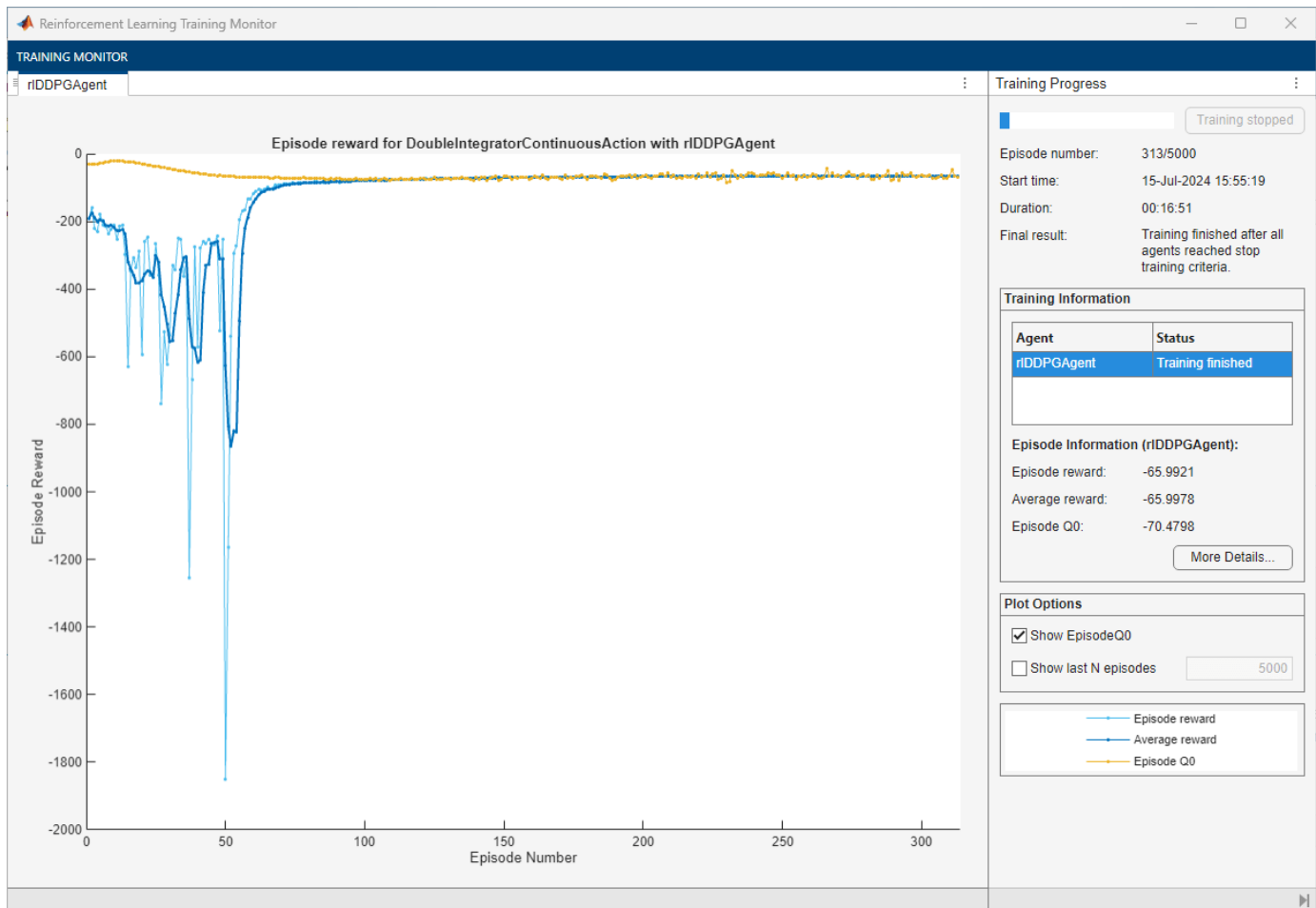
You can visualize the double-integrator environment by using the `plot` function during training or simulation.

```
plot(env)
```



Train the agent using `train`. Training this agent is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

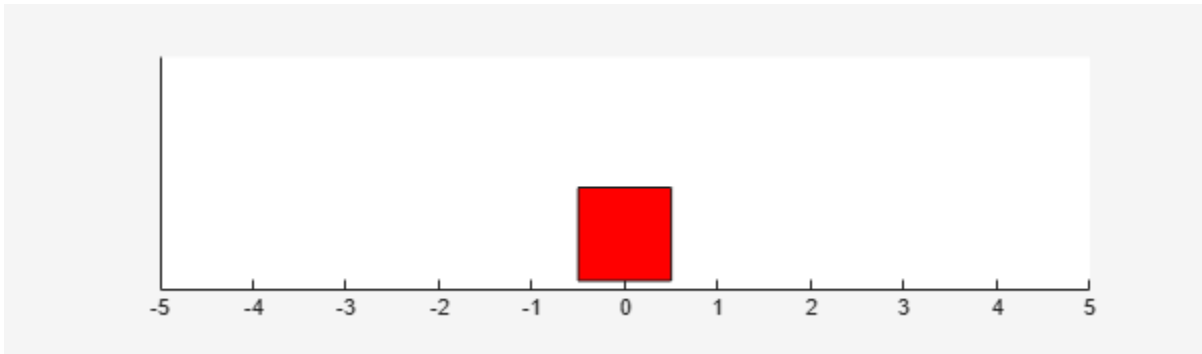
```
doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts);
else
    % Load the pretrained agent for the example.
    load("DoubleIntegDDPG.mat","agent");
end
```



Simulate Trained Agent

To validate the performance of the trained agent, simulate it within the double-integrator environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=500);
experience = sim(env,agent,simOptions);
```



```
totalReward = sum(experience.Reward)
```

```
totalReward =  
-65.9806
```

Solve LQR Problem

The function `lqrd` (Control System Toolbox) solves a discretized LQR problem, like the one presented in this example. This function calculates the optimal discrete-time gain matrix K_{lqr} , together with the solution of the Riccati equation P_{lqr} . When K_{lqr} is connected via negative state feedback to the plant input (force), the discrete-time equivalent of the cost function specified by `env.Q` and `env.R` is minimized going forward. Furthermore, the cumulative cost from initial time to infinity, starting from an initial state x_0 , is equal to $x_0' * P_{lqr} * x_0$.

Use `lqrd` to solve the discretized LQR problem.

```
[Klqr,Plqr] = lqrd([0 1;0 0],[0;env.Gain],env.Q,env.R,env.Ts);
```

Here, $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ \text{env.Gain} \end{bmatrix}$ are the continuous-time transition and input gain matrices, respectively, of the double-integrator system.

If Control System Toolbox™ is not installed, use the solution for the default example values.

```
Klqr = [17.8756 8.2283];  
Plqr = [4.1031 0.3376; 0.3376 0.1351];
```

If the actor policy $u = Kx$ successfully approximates the optimal policy, then the resulting K must be close to $-K_{lqr}$ (the minus sign is due to the fact that K_{lqr} is calculated assuming a negative feedback connection).

If the critic learns a good approximation of the optimal value function, then the resulting P , as defined before, must be close to $-P_{lqr}$ (the minus sign is due to the fact that the reward is defined as the negative of the cost).

Compare DDPG Agent to Optimal Controller

Extract the parameters (weights) of the actor and critic within the agent.

```
par = getLearnableParameters(agent);
```

Compare Actor Parameters to LQR Gains

Obtain and display the actor weights.


```
K = par.Actor{1}
```

```
K = 1x2 single row vector
```

```
-15.4663    -7.2746
```

Check whether the gains are close to the ones of the optimal solution -Klqr:

```
-Klqr
```

```
ans = 1x2
```

```
-17.8756    -8.2283
```

The two matrices K and -Klqr are very similar.

Compare Critic Value Functions

Obtain and display the critic weights.

```
W = par.Critic{1}
```

```
W = 1x6 single row vector
```

```
-5.0017    -1.5513    -0.3424    -0.1116    -0.0506    -0.0047
```

Recreate the matrices W_Q and P .

```
WQ = [W(1) W(2)/2 W(4)/2; W(2)/2 W(3) W(5)/2; W(4)/2 W(5)/2 W(6)];
```

```
P = [eye(2) K']*WQ*[eye(2);K] ...
```

```
P = 2x2 single matrix
```

```
-4.4050    -0.5099  
-0.5099    -0.2244
```

Check whether P is close to the solution of the Riccati equation -Plqr.

```
-Plqr
```

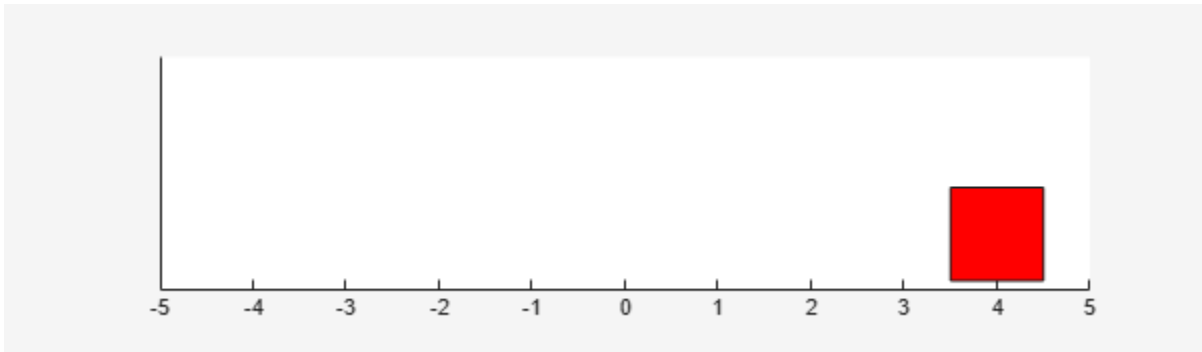
```
ans = 2x2
```

```
-4.1031    -0.3376  
-0.3376    -0.1351
```

The two matrices P and -Plqr are very similar.

Get the environment initial state.

```
x0=reset(env);
```



The value function is the estimate of future cumulative long-term reward when using the policy enacted by the actor. Calculate the value function at the initial state, according to the critic weights. This is the same value displayed in the training window as **Episode Q0**.

```
q0 = x0'*P*x0
```

```
q0 = single
```

```
-70.4798
```

Calculate the value of the initial state, following the true optimal policy enacted by the LQR controller.

```
-x0'*Plqr*x0
```

```
ans =
```

```
-65.6494
```

The two approximations of the value of the initial state are close to each other, and are also close to the total cumulative reward obtained in the validation simulation, `totalReward`, suggesting that the critic learns a good approximation of the value function for the policy enacted by the actor.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Apps

Reinforcement Learning Designer

Functions

`train` | `sim` | `lqr`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlTrainingOptions`

Related Examples

- “Train PG Agent with Custom Actor and Baseline Networks to Control Discrete Double Integrator” on page 5-115
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78

- “Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning” on page 8-284
- “Create and Train Custom LQR Agent” on page 5-345

More About

- “Use Predefined Control System Environments” on page 2-42
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Application Examples

- “Control Water Level in a Tank Using a DDPG Agent” on page 8-2
- “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14
- “Train DDPG Agent to Control Two-Thruster Sliding Vehicle” on page 8-22
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31
- “Train Multiple Agents for Area Coverage” on page 8-44
- “Train Multiple Agents for Path Following Control” on page 8-57
- “Train DDPG Agent for Adaptive Cruise Control” on page 8-68
- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Train PPO Agent for Automatic Parking Valet” on page 8-85
- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Train Hybrid SAC Agent for Path-Following Control” on page 8-109
- “Train Hybrid-Action PPO Agent for Path-Following Control” on page 8-120
- “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Quadruped Robot Locomotion Using DDPG Agent” on page 8-158
- “Add Safety Constraint to Simulate Two-Link Robot with SAC Agent” on page 8-168
- “Train SAC Agent for Ball Balance Control” on page 8-181
- “Automatic Parking Valet with Unreal Engine Simulation” on page 8-197
- “Generate Reward Function from a Model Predictive Controller for a Servomotor” on page 8-214
- “Generate Reward Function from a Model Verification Block for a Water Tank System” on page 8-227
- “Train TD3 Agent for PMSM Control” on page 8-242
- “Imitate MPC Controller for Lane Keeping Assist” on page 8-256
- “Train DDPG Agent with Pretrained Actor Network” on page 8-265
- “Imitate Nonlinear MPC Controller for Sliding Robot” on page 8-275
- “Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning” on page 8-284
- “Train DQN Agent with LSTM Network to Control House Heating System” on page 8-299
- “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313
- “Why Solving Regression Using Reinforcement Learning is Not Recommended” on page 8-327
- “Why Solving Classification Using Reinforcement Learning Is Not Recommended” on page 8-337
- “Train Agent to Play Turn-Based Game” on page 8-348
- “Water Distribution System Scheduling Using Reinforcement Learning” on page 8-361
- “Optimizing Queue Selection Strategies Using Reinforcement Learning” on page 8-372
- “Identify Vulnerabilities in DC Microgrids” on page 8-383

Control Water Level in a Tank Using a DDPG Agent

This example shows how to convert the PI controller in the `rlwatertank` Simulink® model to a reinforcement learning deep deterministic policy gradient (DDPG) agent. For an example that trains a DDPG agent in MATLAB®, see “Compare DDPG Agent to LQR Controller” on page 7-285.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

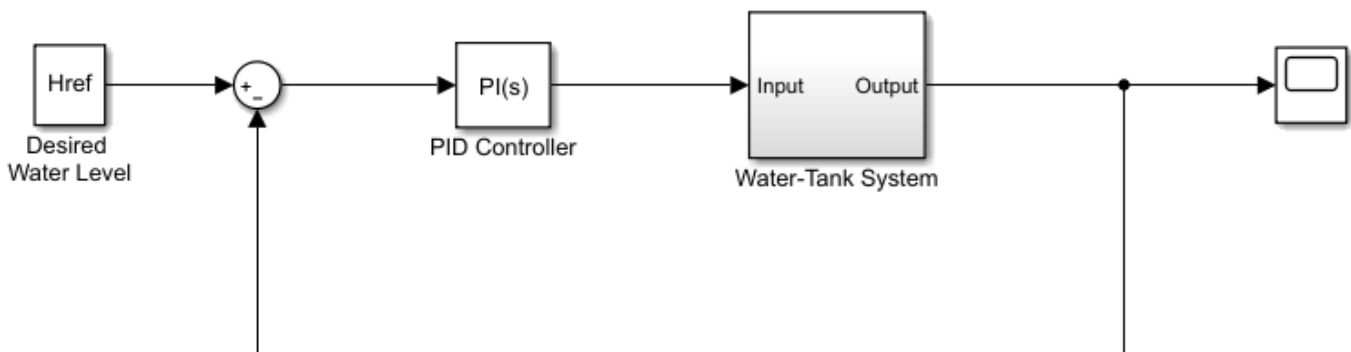
Fix the random number stream with seed zero and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Water Tank Model

The original model for this example is the water tank model. The goal is to control the level of the water in the tank. For more information about the water tank model, see “watertank Simulink Model” (Simulink Control Design).

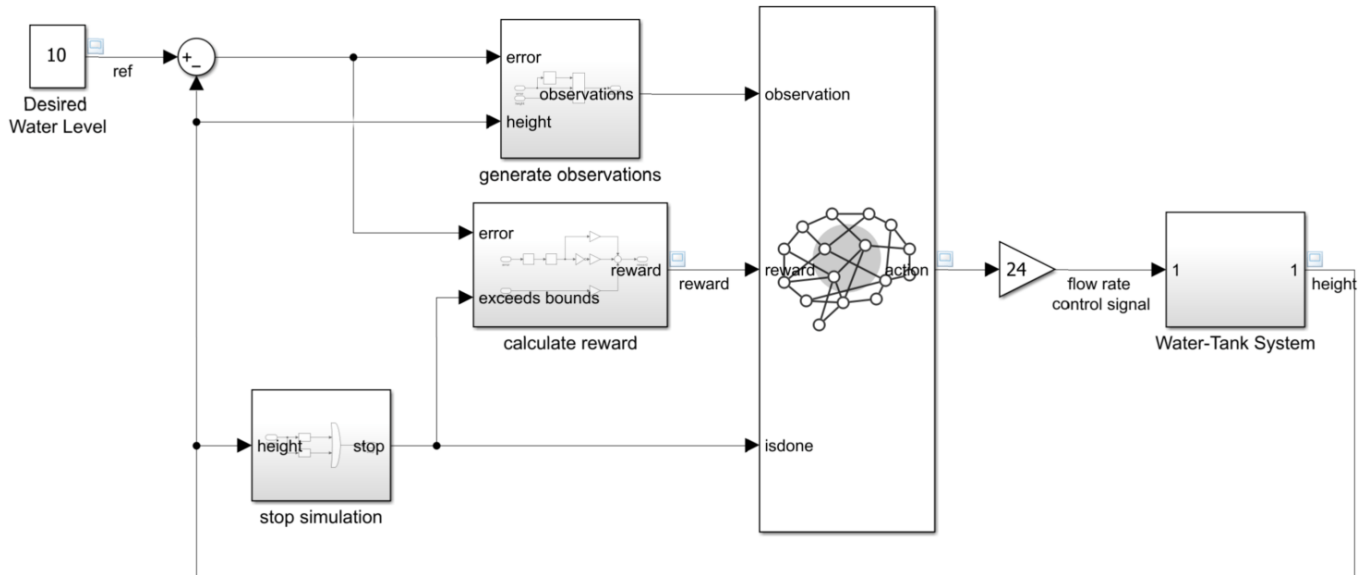


Modify the original model by making the following changes:

- 1 Delete the PID Controller.
- 2 Insert the RL Agent block.
- 3 Connect the observation vector $[f e \text{ dt } e \ h]^T$, where h is the height of the water in the tank, $e = r - h$, and r is the reference height.
- 4 Set up the reward $\text{reward} = 10(|e| < 0.1) - 1(|e| \geq 0.1) - 100(h \leq 0 || h \geq 20)$.
- 5 Configure the termination signal such that the simulation stops if $h \leq 0$ or $h \geq 20$.

The resulting model is `rlwatertank.slx`. For more information on this model and these changes, see “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132.

```
open_system("rlwatertank")
```



Copyright 2018-2025 The MathWorks, Inc.

For more information about custom Simulink environments, see “Create Custom Simulink Environments” on page 2-110.

Create the Environment

Creating an environment model includes defining the following:

- Action and observation signals that the agent uses to interact with the environment. For more information, see `rlNumericSpec` and `rlFiniteSetSpec`.
- Reward signal that the agent uses to measure its success. For more information, see “Define Observation and Reward Signals in Custom Environments” on page 2-127.

Define the observation specification `obsInfo` and action specification `actInfo`.

```
% Observation info
obsInfo = rlNumericSpec([3 1], ...
    LowerLimit=[-inf -inf 0]', ...
    UpperLimit=[ inf  inf  inf]');

% Name and description are optional and not used
% by the software
obsInfo.Name = "observations";
obsInfo.Description = "integrated error, error," + ...
    " and measured height";

% Action info
actInfo = rlNumericSpec([1 1], LowerLimit=0, UpperLimit=1);
actInfo.Name = "flow rate control signal";
```

Create the environment object.

```
env = rlSimulinkEnv("rlwatertank","rlwatertank/RL Agent", ...  
    obsInfo,actInfo);
```

Set a custom reset function that randomizes the reference values for the model. The `localResetFcn` function is provided at the end of the example.

```
env.ResetFcn = @(in)localResetFcn(in);
```

Specify the agent sample time T_s and the simulation time T_f in seconds.

```
Ts = 1.0;  
Tf = 100;
```

Create the Critic

DDPG agents use a parameterized Q-value function approximator to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output. The output is the estimated value of the policy, that is the estimated discounted cumulative long-term reward when the agent starts from the state corresponding to the current observation, takes the action, and follows the policy thereafter.

To model the parameterized Q-value function within the critic, use a neural network with two input layers (one for the observation channel, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value).

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel. Obtain the dimension of the observation and action spaces from the `obsInfo` and `actInfo` specifications.

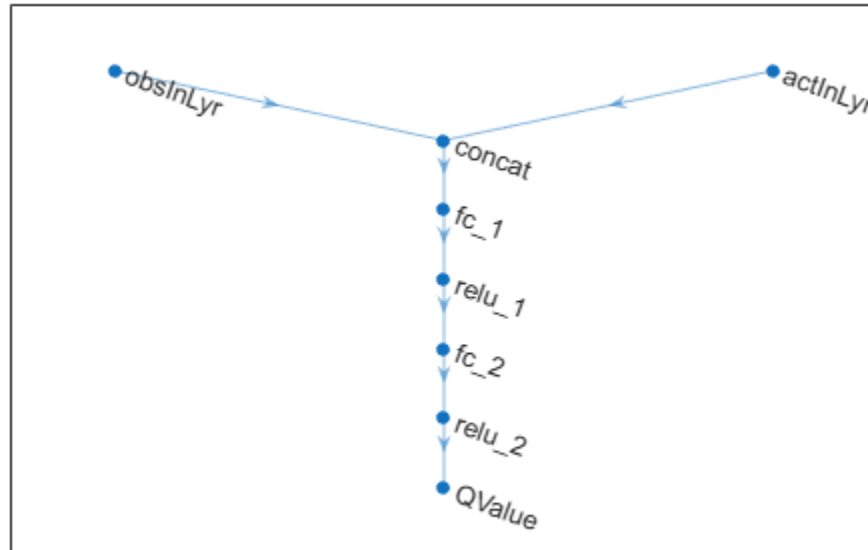
```
% Observation path  
obsPath = featureInputLayer(obsInfo.Dimension(1), ...  
    Name="obsInLyr");  
  
% Action path  
actPath = featureInputLayer(actInfo.Dimension(1), ...  
    Name="actInLyr");  
  
% Common path  
commonPath = [  
    concatenationLayer(1,2,Name="concat")  
    fullyConnectedLayer(25)  
    reluLayer()  
    fullyConnectedLayer(25)  
    reluLayer()  
    fullyConnectedLayer(1,Name="QValue")  
];  
  
% Create the network object and add the layers  
criticNet = dlnetwork();  
criticNet = addLayers(criticNet,obsPath);  
criticNet = addLayers(criticNet,actPath);  
criticNet = addLayers(criticNet,commonPath);  
  
% Connect the layers
```



```
criticNet = connectLayers(criticNet, ...
  "obsInLyr", "concat/in1");
criticNet = connectLayers(criticNet, ...
  "actInLyr", "concat/in2");
```

View the critic network configuration.

```
plot(criticNet)
```



Initialize the `dlnetwork` object and summarize its properties. The initial parameters of the critic network are initialized with random values. Fix the random number stream so that the network is always initialized with the same parameter values.

```
rng(0, "twister");
criticNet = initialize(criticNet);
summary(criticNet)
```

```
Initialized: true
```

```
Number of learnables: 801
```

```
Inputs:
```

```
1  'obsInLyr'   3 features
2  'actInLyr'   1 features
```

Create the critic approximator object using the specified deep neural network, the environment specification objects, and the names of the network inputs to be associated with the observation and action channels.

```
critic = rlQValueFunction(criticNet, ...
  obsInfo, actInfo, ...
  ObservationInputNames="obsInLyr", ...
  ActionInputNames="actInLyr");
```

For more information on Q-value function objects, see `rlQValueFunction`.

Check the critic with a random input observation and action.

```
getValue(critic, ...  
        {rand(obsInfo.Dimension)}, ...  
        {rand(actInfo.Dimension)})  
  
ans = single  
  
-0.4320
```

For more information on creating critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Create the Actor

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor.

A continuous deterministic actor implements a parameterized deterministic policy for a continuous action space. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`).

Define the network as an array of layer objects.

```
actorNet = [  
    featureInputLayer(obsInfo.Dimension(1))  
    fullyConnectedLayer(25)  
    reluLayer()  
    fullyConnectedLayer(25)  
    reluLayer()  
    fullyConnectedLayer(actInfo.Dimension(1))  
    tanhLayer()  
    scalingLayer("Scale",0.5,"Offset",0.5)  
];
```

Convert the network to a `dlnetwork` object and summarize its properties. The initial parameters of the actor network are initialized with random values. Fix the random number stream so that the network is always initialized with the same parameter values.

```
rng(0,"twister");  
actorNet = dlnetwork(actorNet);  
summary(actorNet)
```

```
Initialized: true
```

```
Number of learnables: 776
```

```
Inputs:  
 1 'input' 3 features
```

View the actor network configuration.

```
plot(actorNet)
```



Create the actor approximator object using the specified deep neural network, the environment specification objects, and the name of the network input to be associated with the observation channel.

```
actor = rlContinuousDeterministicActor(actorNet,obsInfo,actInfo);
```

For more information, see `rlContinuousDeterministicActor`.

Check the actor with a random input observation.

```
getAction(actor,{rand(obsInfo.Dimension)}))
```

```
ans = 1x1 cell array
      {[0.4500]}
```

For more information on creating critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Create the DDPG Agent

Create the DDPG agent using the specified actor and critic approximator objects.

```
agent = rlDDPGAgent(actor,critic);
```

For more information, see `rlDDPGAgent`.

Specify options for the agent, the actor, and the critic using dot notation. For this training:

- Use an experience buffer with capacity `1e6` to store experiences. A larger experience buffer can store a diverse set of experiences.
- Specify the actor and critic learning rates to be `1e-4` and `1e-3` respectively. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.

- Use a gradient threshold of 1 to clip the gradients. Clipping the gradients can improve training stability.
- Use mini-batches of 400 experiences. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Use a discount factor of 0.99 to enhance the importance of long term rewards.
- Use a sample time of one second is assigned to the agent.

```
agent.AgentOptions.SampleTime = Ts;
agent.AgentOptions.DiscountFactor = 0.99;
agent.AgentOptions.MinibatchSize = 400;
agent.AgentOptions.ExperienceBufferLength = 1e6;

actorOpts = rlOptimizerOptions( ...
    LearnRate=1e-4, ...
    GradientThreshold=1);
criticOpts = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1);
agent.AgentOptions.ActorOptimizerOptions = actorOpts;
agent.AgentOptions.CriticOptimizerOptions = criticOpts;
```

The DDPG agent in this example uses an Ornstein-Uhlenbeck (OU) noise model for exploration. For the noise model:

- Specify a standard deviation value of 0.03 to improve exploration during training.
- Specify a decay rate of 5e-5 for the standard deviation. The decay causes exploration to reduce gradually as the training progresses.

```
agent.AgentOptions.NoiseOptions.StandardDeviation = 0.03;
agent.AgentOptions.NoiseOptions.StandardDeviationMin = 0.001;
agent.AgentOptions.NoiseOptions.StandardDeviationDecayRate = 5e-5;
```

Alternatively, you can specify the agent options using the `rlDDPGAgentOptions` object.

Check the agent with a random input observation.

```
getAction(agent,{rand(obsInfo.Dimension)}))

ans = 1×1 cell array
    {[0.5506]}
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run each training for a maximum of 5000 episodes. Specify that each episode lasts for a maximum of `ceil(Tf/Ts)` (that is, 100) time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the performance of the greedy policy every 10 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score reaches 900. At this point, the agent can control the level of water in the tank.

For more information on training options, see `rlTrainingOptions`.

```
% training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    Plots="training-progress", ...
    Verbose=false, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=900);

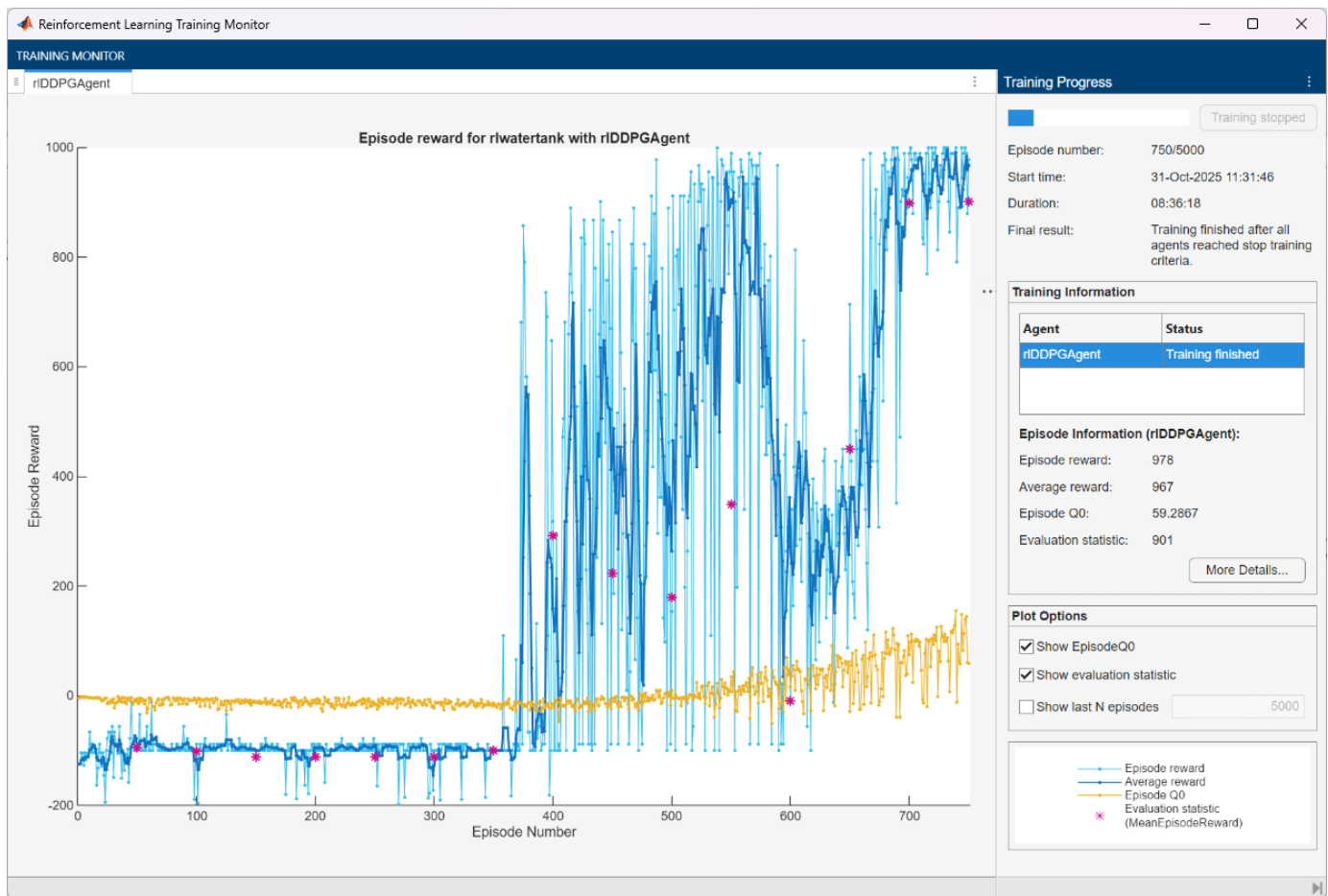
% agent evaluator
evl = rlEvaluator(EvaluationFrequency=50, NumEpisodes=5, RandomSeeds=1:5);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent.
    trainingStats = train(agent, env, trainOpts, Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("WaterTankDDPG.mat", "agent")
end
```



Validate Trained Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

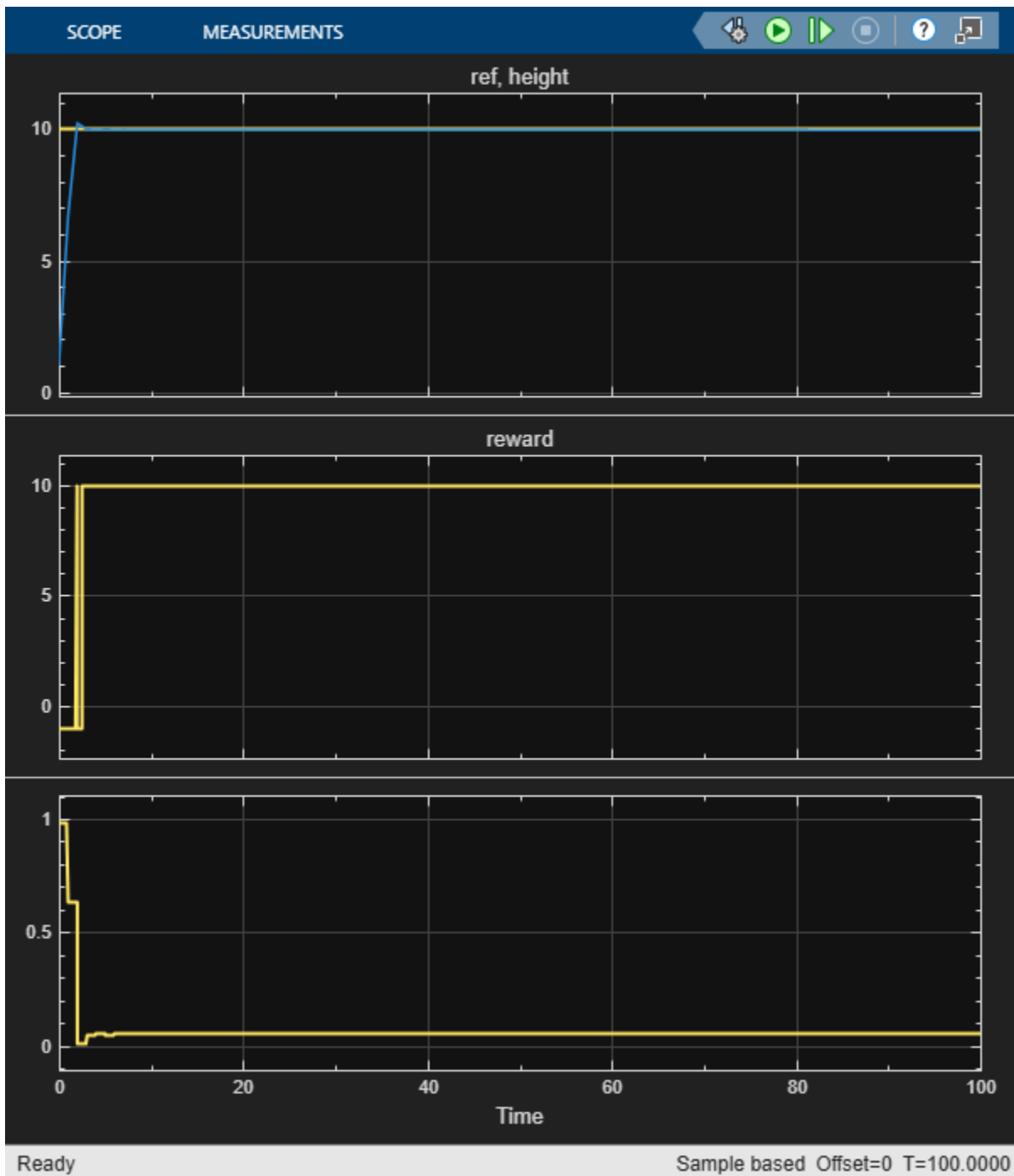
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To use a specific initial condition, set a custom reset function, `localValidationResetFcn`. This function is provided at the end of the example.

```
env.ResetFcn = @(in)localValidationResetFcn(in);
```

Simulate the agent within the environment, and return the experience as output. To use a specific initial conditions, set reset function for the environment.

```
simOpts = rlSimulationOptions( ...
    MaxSteps=ceil(Tf/Ts), ...
    StopOnError="on");
experience = sim(env, agent, simOpts);
```



Display the total reward that the agent collects in the simulation.

```
sum(experience.Reward)
```

```
ans =  
978
```

View the performance of the closed loop system using the Scope block in the Simulink model. The agent tracks the reference signal within 5 seconds.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Reset Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `reset` function uses `setBlockParameter` (Simulink) to specify random values of the reference signal and the initial water height. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)

    % Randomize reference signal
    blk = sprintf("rlwatertank/Desired \nWater Level");
    h = 3*randn + 10;
    while h <= 0 || h >= 20
        h = 3*randn + 10;
    end
    in = setBlockParameter(in,blk,Value=num2str(h));

    % Randomize initial water height
    h = 3*randn + 10;
    while h <= 0 || h >= 20
        h = 3*randn + 10;
    end
    blk = "rlwatertank/Water-Tank System/H";
    in = setBlockParameter(in,blk,InitialCondition=num2str(h));
end

function in = localValidationResetFcn(in)
    % Set reference signal
    blk = sprintf("rlwatertank/Desired \nWater Level");
    h = 10;
    in = setBlockParameter(in,blk,Value=num2str(h));

    % Set initial height
    h = 1;
    blk = "rlwatertank/Water-Tank System/H";
    in = setBlockParameter(in,blk,InitialCondition=num2str(h));
end
```

See Also

Functions

`train` | `sim`

Objects

`rlFiniteSetSpec` | `rlNumericSpec` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlDDPGAgent` | `rlDDPGAgentOptions` |
`rlTrainingOptions` | `rlSimulationOptions`

Related Examples

- “Compare DDPG Agent to LQR Controller” on page 7-285
- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning” on page 8-284
- “Train Default TD3 Agent to Control Quanser QUBE Pendulum” on page 8-14

More About

- “watertank Simulink Model” (Simulink Control Design)
- “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132
- “Reinforcement Learning for Control Systems Applications” on page 1-7
- “Create Custom Simulink Environments” on page 2-110
- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Reinforcement Learning Agents” on page 3-2
- “Train Reinforcement Learning Agents” on page 5-3

Train Default TD3 Agent to Control Quanser QUBE Pendulum

This example trains a default TD3 agent using parallel computing to swing up and control a Quanser QUBE™-Servo 2 inverted pendulum system.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

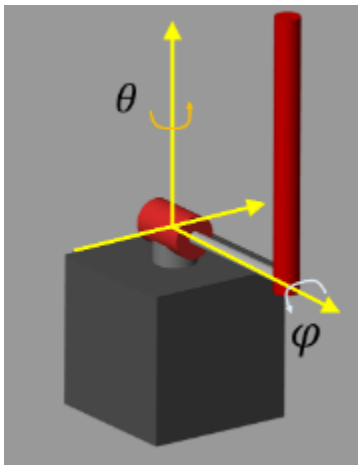
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Inverted Pendulum Model

The Quanser QUBE-Servo 2 pendulum system is a rotational inverted pendulum with two degrees of freedom. It is non-linear, underactuated, non-minimum phase, and it is modeled in Simulink® using Simscape™ Electrical™ and Simscape Multibody™. For a detailed description of the dynamics, see [1].

The pendulum is attached to the motor arm through a free revolute joint. The arm is actuated by a DC motor. The environment has the following properties:

- The angles and angular velocities of the motor arm ($\theta, \dot{\theta}$) and pendulum ($\phi, \dot{\phi}$) are measurable.
- The motor is constrained to $|\theta| \leq 5\frac{\pi}{8}$ rad and $|\dot{\theta}| \leq 30 \frac{\text{rad}}{\text{sec}}$.
- The pendulum is upright when $\phi = 0$.
- The motor input is constrained to $\pm 12\text{V}$.
- The agent action ($|u| \leq 1$) is scaled to the motor voltage in the environment.



Open the Simulink model.

```
mdl = "rlQubeServo";
open_system(mdl)
```

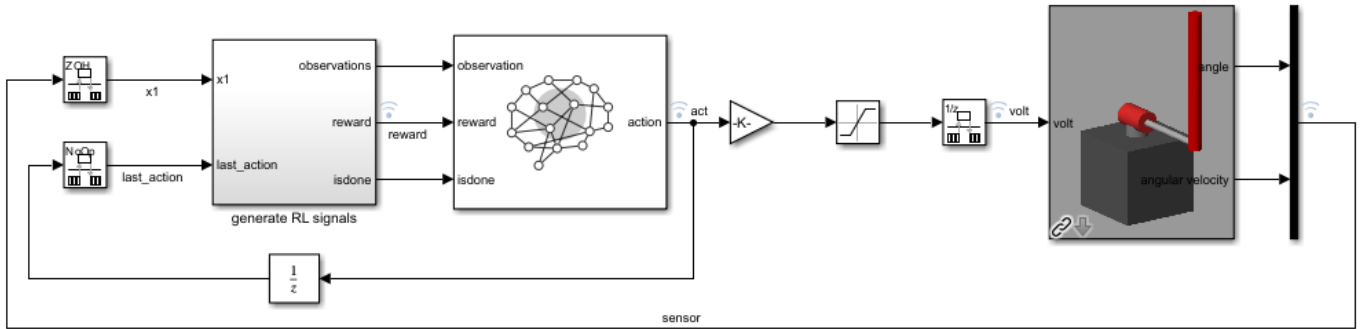
Define the limit for θ (radians), $\dot{\theta}$ (radians/second), voltage limit (volts), as well as the agent's sample time (seconds).

```
theta_limit = 5*pi/8;
dtheta_limit = 30;
volt_limit = 12;
Ts = 0.005;
```

Define the initial conditions for $\theta, \dot{\theta}, \phi, \dot{\phi}$.

```
theta0 = 0;
phi0 = 0;
dtheta0 = 0;
dphi0 = 0;
```

Qube Servo Swing Up with Reinforcement Learning



Copyright 2021-2024, The MathWorks Inc.

Create Environment Object

For this environment:

- The pendulum system modeled in Simscape Multibody.
- The observation is the vector $s_k = [\sin\theta_k, \cos\theta_k, \dot{\theta}_k, \sin\phi_k, \cos\phi_k, \dot{\phi}_k, u_{k-1}]$. Using the sine and cosine of the measured angles can facilitate training by representing the otherwise discontinuous angular measurements by a continuous two-dimensional parameterization.
- The action is the normalized input voltage command to the servo motor.
- The reward signal is defined as follows:

$$r(s_k, u_{k-1}) = F_k - 0.1(\theta_k^2 + \phi_k^2 + \dot{\theta}_k^2 + \dot{\phi}_k^2 + u_{k-1}^2 + 0.3(u_{k-1} - u_{k-2})^2)$$

$$F_k = \begin{cases} 1 & \theta_k \in \pm 5\frac{\pi}{8} \text{ rad and } \dot{\theta}_k \in \pm 30 \frac{\text{rad}}{\text{sec}} \\ 0 & \text{otherwise} \end{cases}$$

The above reward function penalizes six different terms:

- Deviations from the forward position of the motor arm ($\theta_k = 0$).
- Deviations for the inverted position of the pendulum ($\varphi_k = 0$).
- The angular speed of the motor arm $\dot{\theta}_k$.
- The angular speed of the pendulum $\dot{\phi}_k$.
- The control action u_k .
- Changes to the control action ($u_k - 1 - u_k - 2$).

The agent is rewarded while the system constraints are satisfied (that is, $F_k = 1$).

Create the input and output specifications for the agent. Set the action upper and lower limits to constrain the actions selected by the agent.

```
obsInfo = rlNumericSpec([7 1]);  
actInfo = rlNumericSpec([1 1],UpperLimit=1,LowerLimit=-1);
```

Create the environment object. Specify a reset function, defined at the end of the example, that sets random initial conditions.

```
agentBlk = mdl + "/RL Agent";  
env = rlSimulinkEnv(mdl,agentBlk,obsInfo,actInfo);  
env.ResetFcn = @localResetFcn;
```

Create TD3 Agent

The agent used in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents use two parameterized Q-value function approximators to estimate the value (that is, the expected cumulative long-term reward) of the policy.

To model the parameterized Q-value function within both critics, use a neural network with two inputs (the observation and action) and one output (the value of the policy when taking a given action from the state corresponding to a given observation). For more information on TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

Create an agent initialization object to initialize the critic network with the hidden layer size 64.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=64);
```

Specify the agent options for training using `rlTD3AgentOptions` and `rlOptimizerOptions` objects. For this training:

- Use an experience buffer with capacity 1e6 to collect experiences. A larger capacity maintains a diverse set of experiences in the buffer.
- Use mini-batches of 1024 experiences. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Use a total of 10 learning epochs to train the agent networks.
- Use the stochastic gradient descent with momentum (SGDM) algorithm to update the actor and critic neural networks.
- Specify the learning rate of 2e-3 for the actor and 5e-3 for the critics, and a gradient threshold of 1 for both the actor and the critics.

- Specify the initial standard deviation value of 0.5 for the gaussian exploration model. The standard deviation decays at the rate of 1e-6 which facilitates exploration toward the beginning and exploitation toward the end of training.

```
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=1024, ...
    NumEpoch=10);

agentOpts.ActorOptimizerOptions.Algorithm = "sgdm";
agentOpts.ActorOptimizerOptions.LearnRate = 2e-3;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
for i = 1:2
    agentOpts.CriticOptimizerOptions(i).Algorithm = "sgdm";
    agentOpts.CriticOptimizerOptions(i).LearnRate = 5e-3;
    agentOpts.CriticOptimizerOptions(i).GradientThreshold = 1;
end

agentOpts.ExplorationModel.StandardDeviation = 0.5;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-6;
```

Create the TD3 agent using the observation and action input specifications, initialization options and agent options. When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
agent = rlTD3Agent(obsInfo, actInfo, initOpts, agentOpts);
```

Train TD3 Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run the training for 2000 episodes, with each episode lasting a maximum of `maxSteps` time steps.
- Specify an averaging window of 10 for the episode rewards.

```
Tf = 5;
maxSteps = ceil(Tf/Ts);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=2000, ...
    MaxStepsPerEpisode=maxSteps, ...
    ScoreAveragingWindowLength=10, ...
    StopTrainingCriteria="none");
```

To train the agent in parallel, specify the following training options. Training in parallel requires Parallel Computing Toolbox™ software. If you do not have Parallel Computing Toolbox software installed, set `UseParallel` to `false`.

- Set the `UseParallel` option to `true`.
- Train the agent using asynchronous parallel workers.

```
trainOpts.UseParallel =  ;
trainOpts.ParallelizationOptions.Mode =  ;
```

For more information, see `rlTrainingOptions`.

In parallel training, workers simulate the agent's policy with the environment and store experiences in the replay memory. When workers operate asynchronously the order of stored experiences might not be deterministic and can ultimately make the training results different. To maximize the reproducibility likelihood:

- Initialize the parallel pool with the same number of parallel workers every time you run the code. For information on specifying the pool size see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox).
- Use synchronous parallel training by setting `trainOpts.ParallelizationOptions.Mode` to “sync”.
- Assign a random seed to each parallel worker using `trainOpts.ParallelizationOptions.WorkerRandomSeeds`. The default value of -1 assigns a unique random seed to each parallel worker.

Create an evaluator object to evaluate the performance of the greedy policy every 20 training episodes, averaging the cumulative reward of 5 simulations.

```
evl = rlEvaluator(EvaluationFrequency=20, NumEpisodes=5);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;  
if doTraining  
    % Train the agent.  
    trainingStats = train(agent, env, trainOpts, ...  
        Evaluator=evl);  
else  
    % Load the pretrained agent for the example.  
    load("rlQubeServoAgents.mat", "agent")  
end
```

A snapshot of the training progress is shown below. You might expect different results due to randomness in training.



Simulate TD3 Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Define simulation options.

```
simOpts = rlSimulationOptions(MaxSteps=maxSteps);
```

Simulate the trained agent and display its total reward.

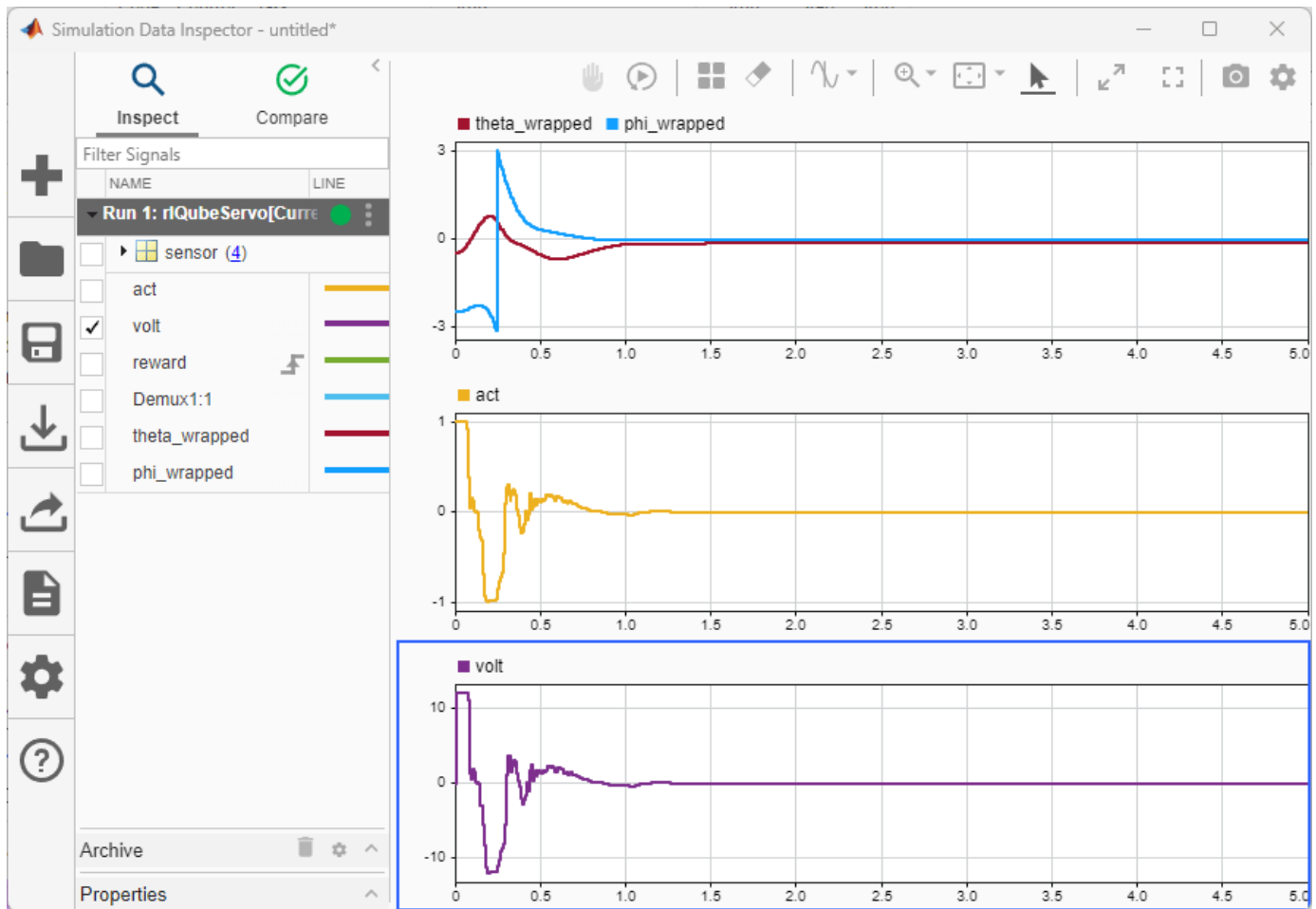
```
experience = sim(agent, env, simOpts);
sum(experience.Reward)
```

```
ans =
935.0247
```

View the performance of the agents in the Simulation Data Inspector. To open the Simulation Data Inspector, in the Simulink model window, on the **Simulation** tab, in the **Review Results** gallery, click **Data Inspector**.

In the plots:

- The measured values for θ (theta_wrapped) and φ (phi_wrapped) are stabilized at approximately 0 radians. The pendulum is stabilized at the upright equilibrium position.
- The act signal shows the actions of the agent.
- The control input is shown by the volt signal.



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (`Simulink`) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (`Simulink`) function to reset the initial angles to a random number and the initial speeds to 0. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
    theta0 = -pi/4+rand*pi/2;
    phi0 = pi-pi/4+rand*pi/2;
```



```
in = setVariable(in,"theta0",theta0);  
in = setVariable(in,"phi0",phi0);  
in = setVariable(in,"dtheta0",0);  
in = setVariable(in,"dphi0",0);  
end
```

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlSACAgent | rlSACAgentOptions | rlPPOAgent | rlPPOAgentOptions |
rlAgentInitializationOptions

Blocks

RL Agent

Related Examples

- “Train Reinforcement Learning Agent Offline to Control Quanser QUBE Pendulum” on page 5-184
- “Train Default DQN Agent to Swing Up and Balance Discrete Pendulum” on page 5-70
- “Train Default DDPG Agent to Swing Up and Balance Continuous Pendulum” on page 5-78
- “Run SIL and PIL Verification for Reinforcement Learning” on page 6-17
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31

More About

- “Create Custom Simulink Environments” on page 2-110
- “Proximal Policy Optimization (PPO) Agent” on page 3-50
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Train Reinforcement Learning Agents” on page 5-3

Train DDPG Agent to Control Two-Thruster Sliding Vehicle

This example shows how to train a deep deterministic policy gradient (DDPG) agent to generate trajectories for a vehicle with two thrusters that slides without friction over a 2-D plane. The vehicle dynamics is modeled in Simulink®. For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

For an example that shows how to use nonlinear model predictive control to solve the trajectory planning and control problems for a sliding vehicle, see “Trajectory Optimization and Control of Sliding Robot Using Nonlinear MPC” (Model Predictive Control Toolbox). For an example that shows how to train, validate, and test a deep neural network (DNN) that imitates the behavior of a nonlinear model predictive controller for a sliding vehicle, see “Imitate Nonlinear MPC Controller for Sliding Robot” on page 8-275.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Sliding Vehicle Model

The reinforcement learning environment for this example is a sliding vehicle, implemented in Simulink®, with its initial condition randomized around a ring having a radius of 15 m. The orientation of the vehicle is also randomized. The vehicle has two thrusters mounted on the side of the body that are used to propel and steer the vehicle. The training goal is to drive the vehicle from its initial condition to the origin facing east.

Open the model.

```
mdl = "rlSlidingRobotEnv";  
open_system(mdl)
```

Set the initial model state variables.

```
theta0 = 0;  
x0 = -15;  
y0 = 0;
```

Define the sample time T_s and the simulation duration T_f .

```
Ts = 0.4;  
Tf = 30;
```

In this model:

- The goal orientation is θ rad (vehicle facing east).
- The thrust from each actuator is bounded from -1 to 1 N
- The observations from the environment are the position, orientation (sine and cosine of orientation), velocity, and angular velocity of the vehicle.
- The reward r_t provided at every time step is

$$r_1 = 10((x_t^2 + y_t^2 + \theta_t^2) < 0.5)$$

$$r_2 = -100(|x_t| \geq 20 \parallel |y_t| \geq 20)$$

$$r_3 = -(0.2(R_{t-1} + L_{t-1})^2 + 0.3(R_{t-1} - L_{t-1})^2 + 0.03x_t^2 + 0.03y_t^2 + 0.02\theta_t^2)$$

$$r_t = r_1 + r_2 + r_3$$

where:

- x_t is the position of the vehicle along the x-axis.
- y_t is the position of the vehicle along the y-axis.
- θ_t is the orientation of the vehicle.
- L_{t-1} is the control effort from the left thruster.
- R_{t-1} is the control effort from the right thruster.
- r_1 is the reward when the vehicle is close to the goal.
- r_2 is the penalty when the vehicle drives beyond 20 m in either the x or y direction. The simulation is terminated when $r_2 < 0$.
- r_3 is a QR penalty that penalizes distance from the goal and control effort.

For more information on this sliding vehicle model, see [1 on page 8-29].

Create Closed Loop Simulink Model

To train an agent for the `SlidingRobotEnv` model, use the `createIntegratedEnv` function to automatically generate a Simulink model containing an RL Agent block that is ready for training.

```
integratedMdl = "IntegratedSlidingRobot";
[~,agentBlk,obsInfo,actInfo] = ...
    createIntegratedEnv(mdl,integratedMdl);
```

Actions and Observations

Before creating the environment object, specify names for the observation and action specifications, and bound the thrust actions between -1 and 1.

The observation vector for this environment is $[x \ y \ \dot{x} \ \dot{y} \ \sin(\theta) \ \cos(\theta) \ \dot{\theta}]^T$. Assign a name to the environment observation channel.

```
obsInfo.Name = "observations";
```

The action vector for this environment is $[T_R \ T_L]^T$. Assign a name, as well as upper and lower limits, to the environment action channel.

```
actInfo.Name = "thrusters";  
actInfo.LowerLimit = -ones(prod(actInfo.Dimension),1);  
actInfo.UpperLimit = ones(prod(actInfo.Dimension),1);
```

Note that `prod(obsInfo.Dimension)` and `prod(actInfo.Dimension)` return the number of dimensions of the observation and action spaces, respectively, regardless of whether they are arranged as row vectors, column vectors, or matrices.

Create Environment Object

Create an environment object using the integrated Simulink model.

```
env = rlSimulinkEnv( ...  
    integratedMdl, ...  
    agentBlk, ...  
    obsInfo, ...  
    actInfo);
```

Reset Function

Create a custom reset function that randomizes the initial position of the vehicle along a ring of radius 15 m and the initial orientation. The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For details on this reset function, see `slidingRobotResetFcn`. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in) slidingRobotResetFcn(in);
```

Create DDPG Agent

DDPG agents use a parameterized Q-value function approximator to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated discounted cumulative long-term reward given the action from the state corresponding to the current observation, and following the policy thereafter).

When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

To model the parameterized Q-value function within the critic, use a neural network with two input layers (one for the observation channel, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value).

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
% Specify the number of outputs for the hidden layers.  
hiddenLayerSize = 128;  
  
% Define observation path layers  
observationPath = [  
    featureInputLayer(prod(obsInfo.Dimension),Name="obsInLyr")
```

```

concatenationLayer(1,2,Name="cat")
fullyConnectedLayer(hiddenLayerSize)
reluLayer
fullyConnectedLayer(hiddenLayerSize)
reluLayer
fullyConnectedLayer(hiddenLayerSize)
reluLayer
fullyConnectedLayer(1,Name="fc4")
];

% Define action path layers
actionPath = featureInputLayer( ...
    prod(actInfo.Dimension), ...
    Name="actInLyr");

Assemble a dlnetwork object.

criticNetwork = dlnetwork();
criticNetwork = addLayers(criticNetwork,observationPath);
criticNetwork = addLayers(criticNetwork,actionPath);

Connect actionPath to observationPath.

criticNetwork = connectLayers(criticNetwork,"actInLyr","cat/in2");

Initialize network and display the number of parameters.

criticNetwork = initialize(criticNetwork);
summary(criticNetwork)

    Initialized: true

    Number of learnables: 34.4k

    Inputs:
         1  'obsInLyr'    7 features
         2  'actInLyr'    2 features

```

Create the critic using `criticNetwork`, the environment specifications, and the names of the network input layers to be connected to the observation and action channels. For more information, see `rlQValueFunction`.

```

critic = rlQValueFunction(criticNetwork,obsInfo,actInfo, ...
    ObservationInputNames="obsInLyr",ActionInputNames="actInLyr");

```

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`).

Define the network as an array of layer objects.

```

actorNetwork = [
    featureInputLayer(prod(obsInfo.Dimension))
    fullyConnectedLayer(hiddenLayerSize)
]

```

```
reluLayer
fullyConnectedLayer(hiddenLayerSize)
reluLayer
fullyConnectedLayer(hiddenLayerSize)
reluLayer
fullyConnectedLayer(prod(actInfo.Dimension))
tanhLayer
];
```

Convert the array of layer object to a `dlnetwork` object, initialize the network, and display the number of parameters.

```
actorNetwork = dlnetwork(actorNetwork);
actorNetwork = initialize(actorNetwork);
summary(actorNetwork)
```

```
Initialized: true
```

```
Number of learnables: 34.3k
```

```
Inputs:
  1  'input'   7 features
```

Define the actor using `actorNetwork`, and the specifications for the action and observation channels. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNetwork,obsInfo,actInfo);
```

Specify options for the critic and the actor using `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-03, ...
    GradientThreshold=5);
```

```
actorOptions = rlOptimizerOptions( ...
    LearnRate=1e-03, ...
    GradientThreshold=5);
```

Specify the DDPG agent options using `rlDDPGAgentOptions`, include the training options for the actor and critic.

```
agentOptions = rlDDPGAgentOptions( ...
    SampleTime=Ts, ...
    ActorOptimizerOptions=actorOptions, ...
    CriticOptimizerOptions=criticOptions, ...
    ExperienceBufferLength=1e6 , ...
    TargetSmoothFactor=1e-3, ...
    MiniBatchSize=512, ...
    MaxMiniBatchPerEpoch=50);
```

```
% Gaussian noise
```

```
agentOptions.NoiseOptions.MeanAttractionConstant = 1/Ts;
agentOptions.NoiseOptions.StandardDeviation = 1e-1;
agentOptions.NoiseOptions.StandardDeviationMin = 1e-2;
agentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-6;
```

Then, create the agent using the actor, the critic and the agent options. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic,agentOptions);
```

Alternatively, you can create the agent first, and then access its option object and modify the options using dot notation.

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run each training for a maximum of 10000 episodes, with each episode lasting a maximum of $\text{ceil}(T_f/T_s)$ time steps.
- Display the training progress in the Reinforcement Learning Training Monitor (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Evaluate the trained greedy policy every 50 training episodes by averaging the cumulative reward over 5 simulations.
- Stop the training the agent when the average cumulative reward obtained during the evaluation episodes equals or exceeds 425. At this point, the agent can drive the sliding vehicle to the goal position.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 10000;
maxsteps = ceil(Tf/Ts);
trainingOptions = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    StopOnError="on", ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=425, ...
    ScoreAveragingWindowLength=10);

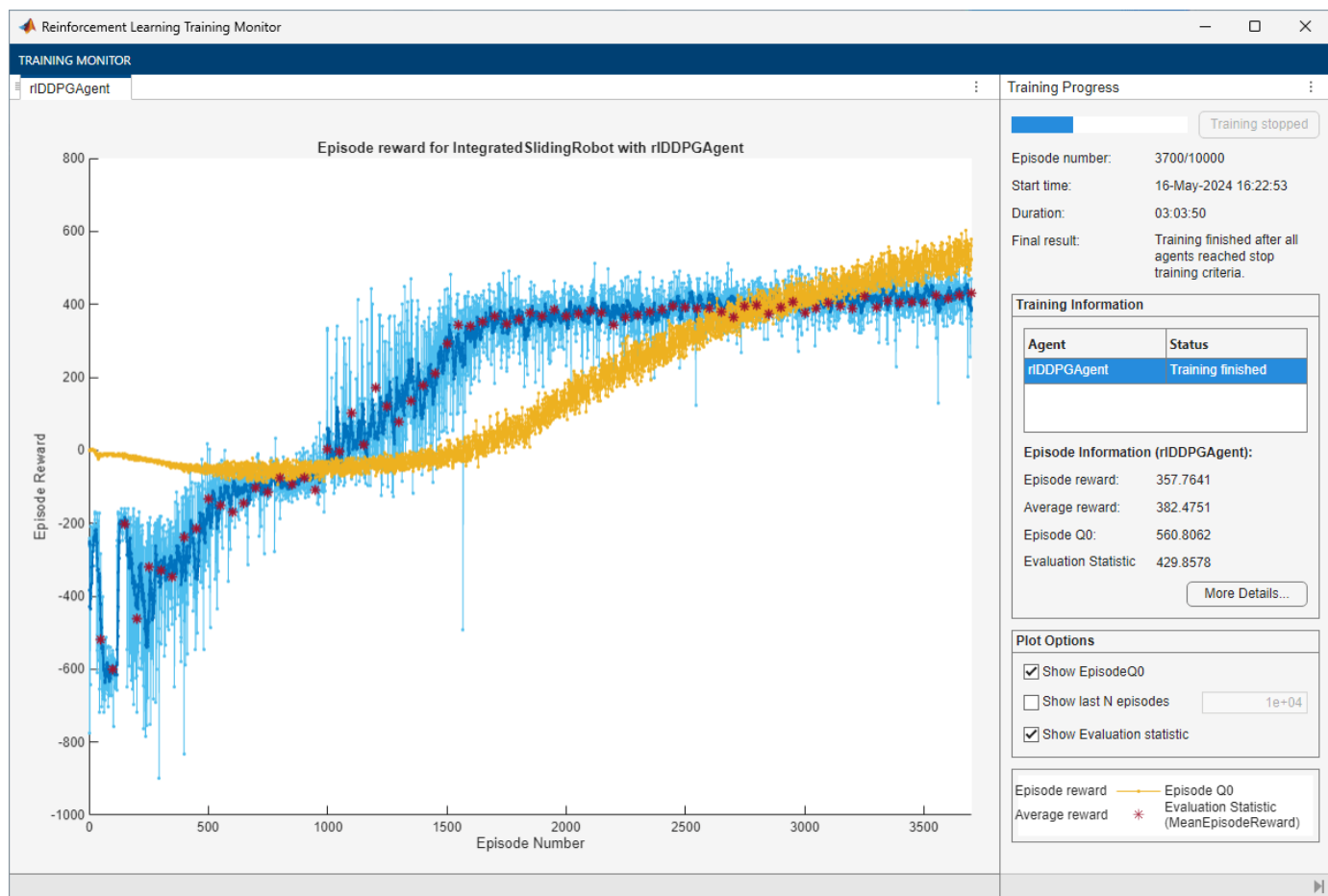
% agent evaluator
evl = rlEvaluator(EvaluationFrequency=50,NumEpisodes=5);
```

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainingOptions,Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("SlidingRobotDDPG.mat","agent")
end
```



Simulate DDPG Agent

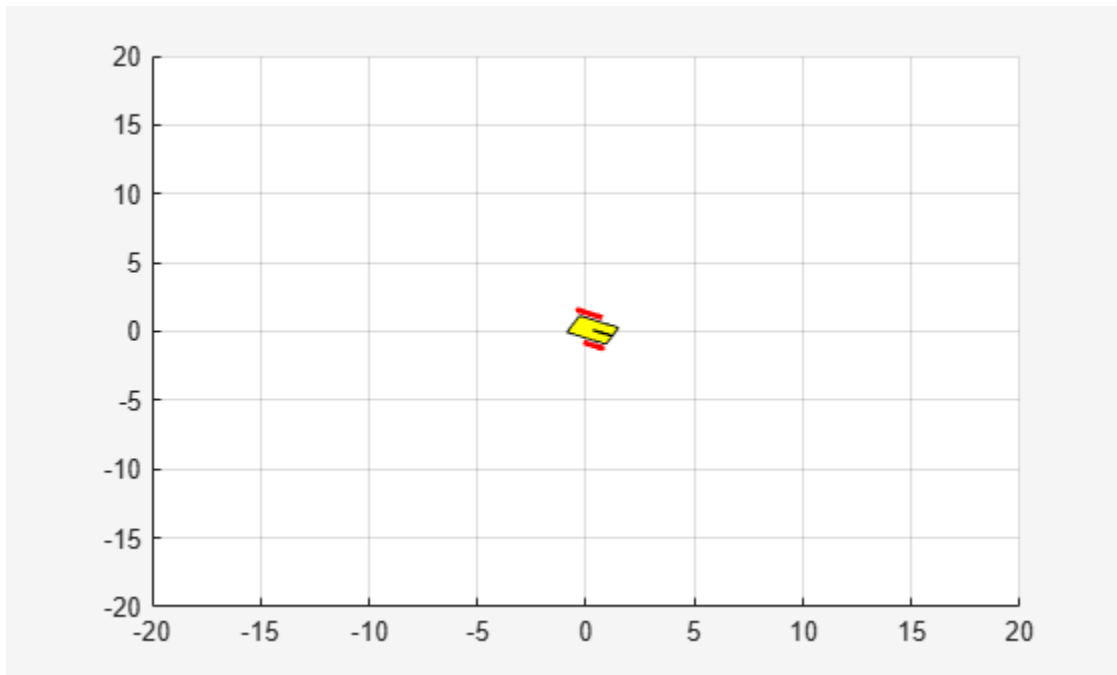
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To validate the performance of the trained agent, simulate the agent within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=maxsteps);
experience = sim(env, agent, simOptions);
```

Display the total reward.

```
totalRwd = sum(experience.Reward)
```

```
totalRwd =  
449.7492
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

[1] Y. Sakawa. "Trajectory planning of a free-flying robot by using the optimal control.", *Optimal Control Applications and Methods*, Vol. 20, 1999, pp. 235-248.

See Also

Functions

`train` | `sim` | `createIntegratedEnv` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Default DDPG Agent to Swing Up and Balance Continuous Cart-Pole” on page 5-85
- “Trajectory Optimization and Control of Sliding Robot Using Nonlinear MPC” (Model Predictive Control Toolbox)
- “Imitate Nonlinear MPC Controller for Sliding Robot” on page 8-275
- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93

More About

- “Create Custom Simulink Environments” on page 2-110
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train Multiple Agents to Perform Collaborative Task

This example shows how to set up a multiagent decentralized or centralized training with a Simulink® environment. In the example, you train two agents to collaboratively perform the task of moving an object.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

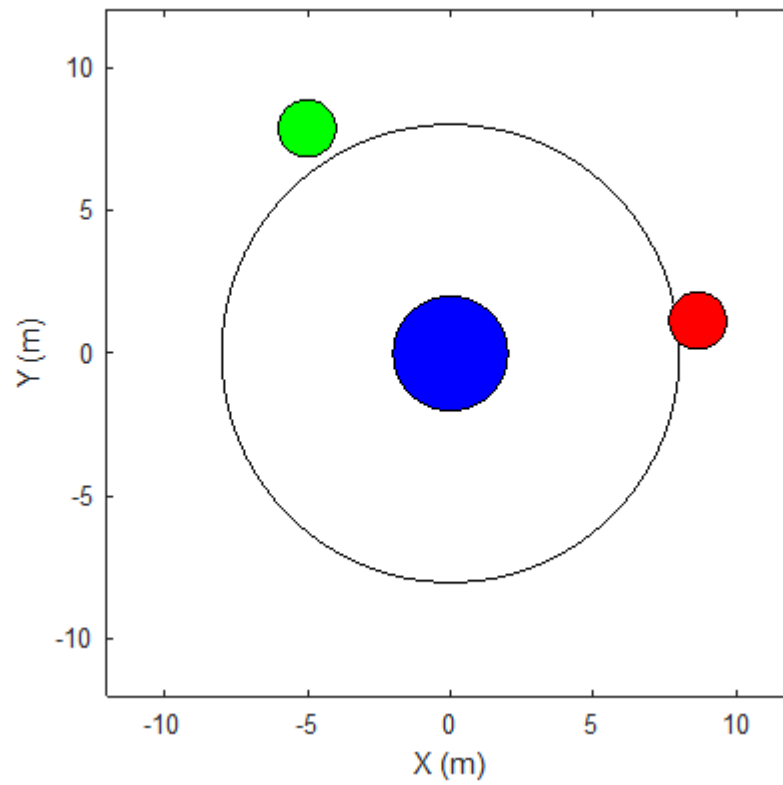
Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Environment Description

The environment in this example is a frictionless two dimensional surface containing elements represented by circles. A target object C is represented by the blue circle with a radius of 2 m, and robots A (red) and B (green) are represented by smaller circles with radii of 1 m each. The robots attempt to move object C outside a circular ring of a radius 8 m by applying forces through collision. All elements within the environment have mass and obey Newton's laws of motion. In addition, contact forces between the elements and the environment boundaries are modeled as spring and mass damper systems. The elements can move on the surface through the application of externally applied forces in the X and Y directions. The elements do not move in the third dimension and the total energy of the system is conserved.

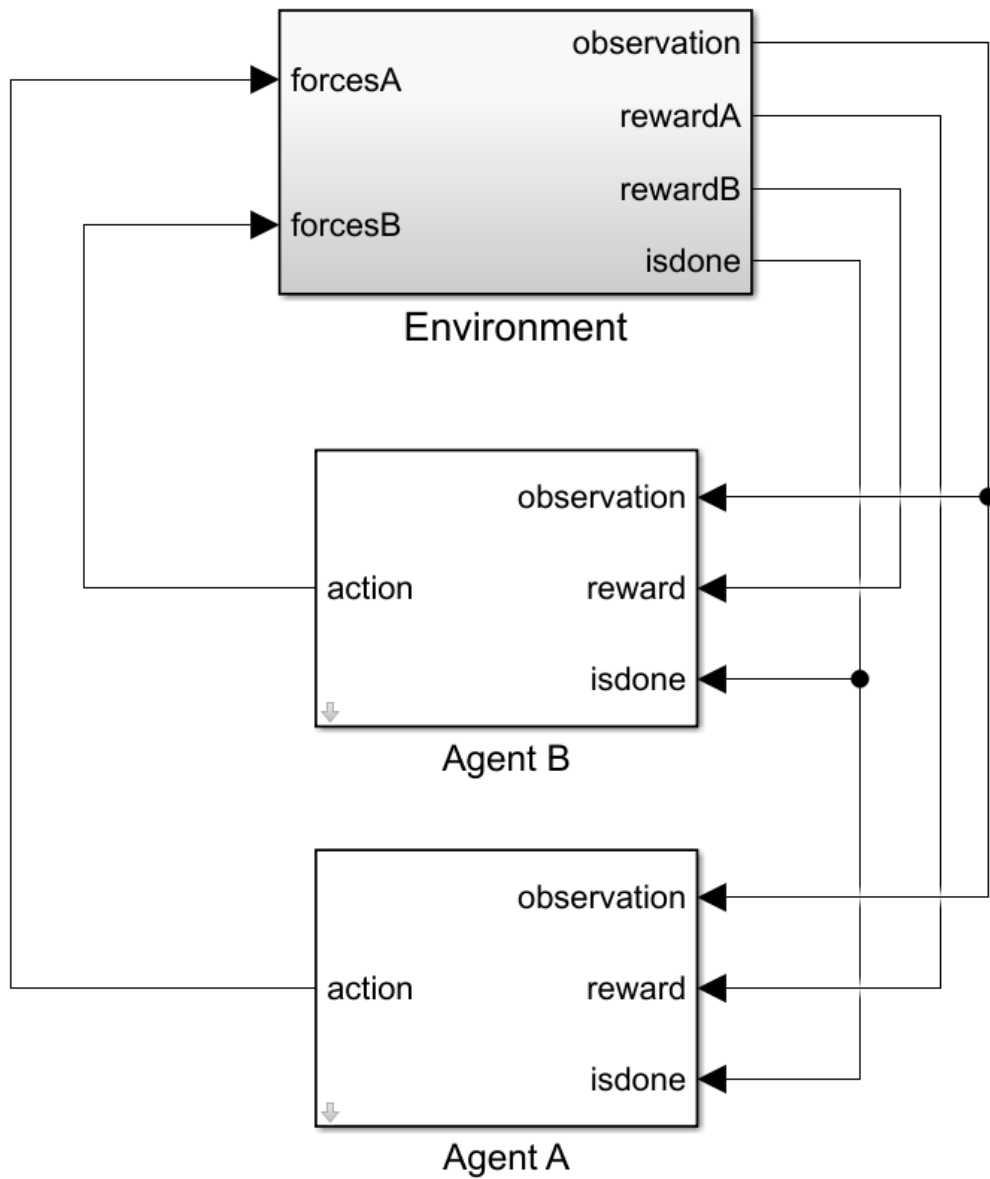


Create the variables required for this example using the `rlCollaborativeTaskParams` Script.

```
rlCollaborativeTaskParams
```

Open the Simulink model.

```
mdl = "rlCollaborativeTask";  
open_system(mdl)
```



For this environment:

- The 2-dimensional space is bounded from -12 m to 12 m in both the X and Y directions.
- The contact spring stiffness and damping values are 100 N/m and 0.1 N/m/s, respectively.
- The agents share the same observations for positions, velocities of A, B, and C and the action values from the last time step.
- The simulation terminates when object C moves outside the circular ring.
- At each time step, the agents receive the following reward:

$$\begin{aligned}r_A &= r_{\text{global}} + r_{\text{local},A} \\r_B &= r_{\text{global}} + r_{\text{local},B} \\r_{\text{global}} &= 0.001d_C \\r_{\text{local},A} &= -0.005d_{AC} - 0.008u_A^2 \\r_{\text{local},B} &= -0.005d_{BC} - 0.008u_B^2\end{aligned}$$

Here:

- r_A and r_B are the rewards received by agents A and B, respectively.
- r_{global} is a team reward that is received by both agents as object C moves closer toward the boundary of the ring.
- $r_{\text{local},A}$ and $r_{\text{local},B}$ are local penalties received by agents A and B based on their distances from object C and the magnitude of the action from the last time step.
- d_C is the distance of object C from the center of the ring.
- d_{AC} and d_{BC} are the distances between agent A and object C and agent B and object C, respectively.
- The agents apply external forces on the robots that result in motion. u_A and u_B are the action values of the two agents A and B from the last time step. The range of action values is between -1 and 1.

Create Environment Object

To create a multiagent environment, specify the block paths of the agents using a string array. Also, specify the observation and action specification objects using cell arrays. The order of the specification objects in the cell array must match the order specified in the block path array. When agents are available in the MATLAB® workspace at the time of environment creation, the observation and action specification arrays are optional. For more information on creating multiagent environments, see `rlSimulinkEnv`.

Create the I/O specifications for the environment. In this example, the agents are homogeneous and have the same I/O specifications.

```
% Number of observations
numObs = 16;

% Number of actions
numAct = 2;

% Maximum value of externally applied force (N)
maxF = 1.0;

% I/O specifications for each agent
oinfo = rlNumericSpec([numObs,1]);
ainfo = rlNumericSpec([numAct,1], ...
    UpperLimit= maxF, ...
    LowerLimit= -maxF);
oinfo.Name = "observations";
ainfo.Name = "forces";
```

Create the Simulink environment object.

```
blks = ["rlCollaborativeTask/Agent A", "rlCollaborativeTask/Agent B"];
obsInfos = {oinfo,oinfo};
actInfos = {ainfo,ainfo};
env = rlSimulinkEnv mdl,blks,obsInfos,actInfos);
```

Specify a reset function for the environment. The function `resetRobots` on page 8-40 ensures that the robots start from random initial positions at the beginning of each episode. The function is provided at the end of this example.

```
env.ResetFcn = @(in) resetRobots(in,RA,RB,RC,boundaryR);
```

Create Default PPO Agents

This example uses four proximal policy optimization (PPO) agents with continuous action spaces (specifically, two agents are trained in a decentralized way and two are trained in a centralized way). To learn more about PPO agents, see “Proximal Policy Optimization (PPO) Agent” on page 3-50.

The agents apply external forces on the robots, which results in their motion.

The agents train from collected trajectories with a mini-batch size of 300 and experience horizon of 600. An objective function clip factor of 0.2 is used to improve training stability and a discount factor of 0.99 is used to encourage long-term rewards.

Specify the agent options for this example.

```
agentOptions = rlPPOAgentOptions( ...
    ExperienceHorizon=600, ...
    ClipFactor=0.2, ...
    EntropyLossWeight=0.01, ...
    MiniBatchSize=300, ...
    NumEpoch=4, ...
    SampleTime=Ts, ...
    DiscountFactor=0.99);
```

Set the learning rate for the actor and critic to $1e-4$.

```
agentOptions.ActorOptimizerOptions.LearnRate = 1e-4;
agentOptions.CriticOptimizerOptions.LearnRate = 1e-4;
```

When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0,"twister");
```

Create the agents using the default agent creation syntax. For more information, see `rlPPOAgent`.

```
dcAgentA = rlPPOAgent(oinfo, ainfo, ...
    rlAgentInitializationOptions(NumHiddenUnit= 200), ...
    agentOptions);
dcAgentB = rlPPOAgent(oinfo, ainfo, ...
    rlAgentInitializationOptions(NumHiddenUnit= 200), ...
    agentOptions);
cnAgentA = rlPPOAgent(oinfo, ainfo, ...
    rlAgentInitializationOptions(NumHiddenUnit= 200), ...
    agentOptions);
cnAgentB = rlPPOAgent(oinfo, ainfo, ...
```

```
rlAgentInitializationOptions(NumHiddenUnit= 200), ...  
agentOptions);
```

Train Agents

To train multiple agents, you can pass an array of agents to the `train` function. The order of agents in the array must match the order of agent block paths specified during environment creation. Doing so ensures that the agent objects are linked to their appropriate I/O interfaces in the environment.

You can train multiple agents in a decentralized or centralized manner. In decentralized training, agents collect their own set of experiences during the episodes and learn independently from those experiences. In centralized training, the agents share the collected experiences and learn from them together. The actor and critic functions are synchronized between the agents after trajectory completion.

To configure a multiagent training, you can create agent groups and specify a learning strategy for each group through the `rlMultiAgentTrainingOptions` object. Each agent group might contain unique agent indices, and the learning strategy can be "centralized" or "decentralized". For example, you can use the following command to configure training for three agent groups with different learning strategies. The agents with indices [1,2] and [3,4] learn in a centralized manner while agent 4 learns in a decentralized manner.

```
opts = rlMultiAgentTrainingOptions(AgentGroups= {[1,2], 4, [3,5]},  
LearningStrategy= ["centralized","decentralized","centralized"])
```

For more information, see `rlMultiAgentTrainingOptions`.

1. Decentralized Training

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

To configure decentralized multiagent training for this example:

- Automatically assign agent groups using the `AgentGroups=auto` option. This allocates each agent in a separate group.
- Specify the "decentralized" learning strategy.
- Run the training for 500 episodes, with each episode lasting a maximum of 600 time steps.

```
dcOpts = rlMultiAgentTrainingOptions( ...  
    AgentGroups="auto", ...  
    LearningStrategy="decentralized", ...  
    MaxEpisodes=500, ...  
    MaxStepsPerEpisode=600, ...  
    ScoreAveragingWindowLength=30, ...  
    StopTrainingCriteria="none");
```

Train the agents using the `train` function. Training can take several hours to complete depending on the available computational power. To save time, load the MAT file `decentralizedAgents.mat` which contains a set of pretrained agents. To train the agents yourself, set `dcTraining` to true.

```
dcTraining = ;  
if dcTraining  
    dcResults = train([dcAgentA,dcAgentB], env, dcOpts);
```

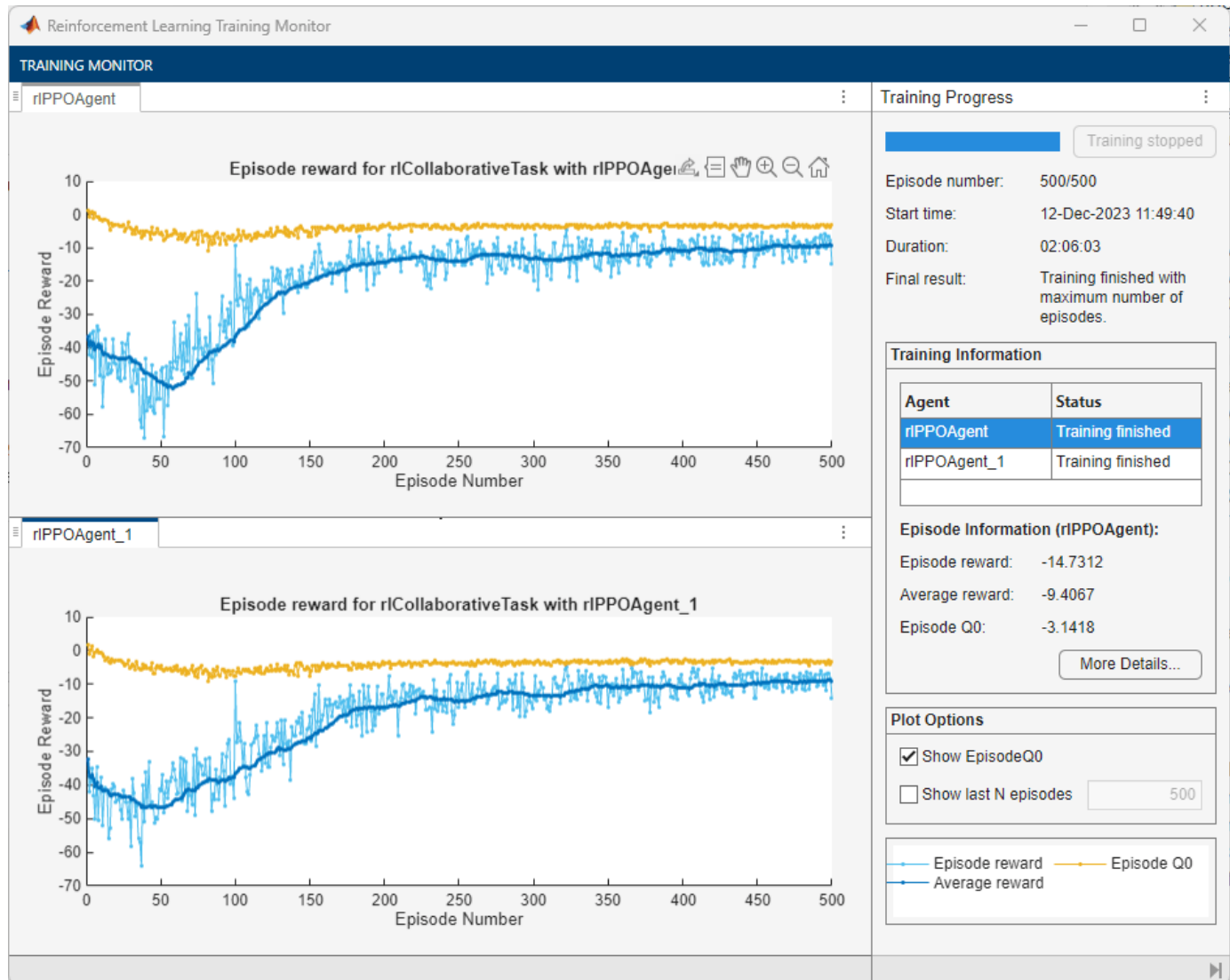


```

else
    load("decentralizedAgents.mat");
end

```

The following figure shows a snapshot of decentralized training progress. You can expect different results due to randomness in the training process.



2. Centralized Training

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To configure centralized multiagent training for this example:

- Allocate both agents (with indices 1 and 2) in a single group. You can do this by specifying the agent indices in the "AgentGroups" option.

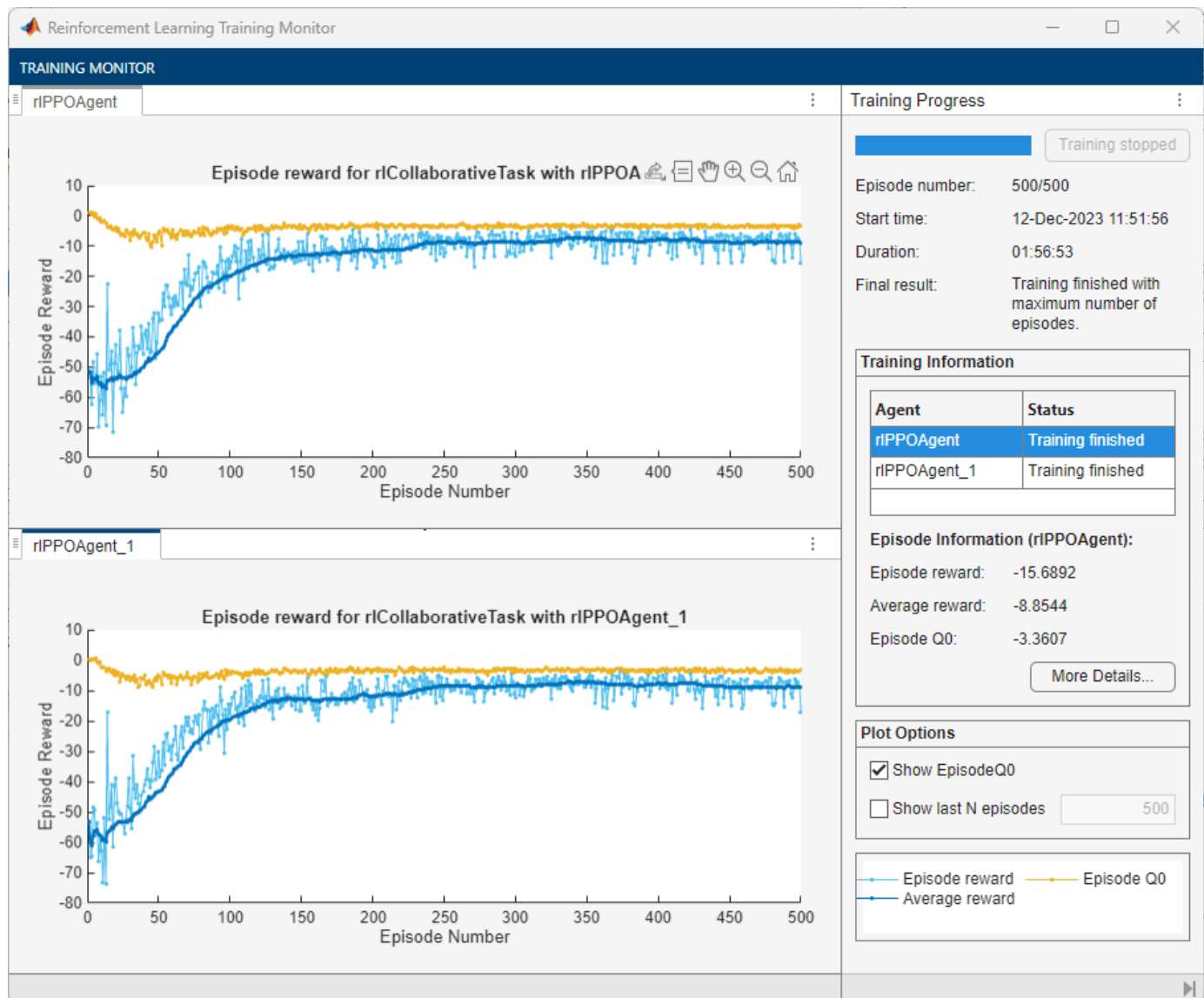
- Specify the "centralized" learning strategy.
- Run the training for 500 episodes, with each episode lasting a maximum of 600 time steps.

```
cnOpts = rlMultiAgentTrainingOptions( ...  
    AgentGroups={1,2}, ...  
    LearningStrategy="centralized", ...  
    MaxEpisodes=500, ...  
    MaxStepsPerEpisode=600, ...  
    ScoreAveragingWindowLength=30, ...  
    StopTrainingCriteria="none");
```

Train the agents using the `train` function. Training can take several hours to complete depending on the available computational power. To save time, load the MAT file `centralizedAgents.mat` which contains a set of pretrained agents. To train the agents yourself, set `cnTraining` to `true`.

```
cnTraining = ;  
if cnTraining  
    cnResults = train([cnAgentA,cnAgentB], env, cnOpts);  
else  
    load("centralizedAgents.mat");  
end
```

The following figure shows a snapshot of centralized training progress. You can expect different results due to randomness in the training process.



Simulate Environment

Fix the random stream for reproducibility.

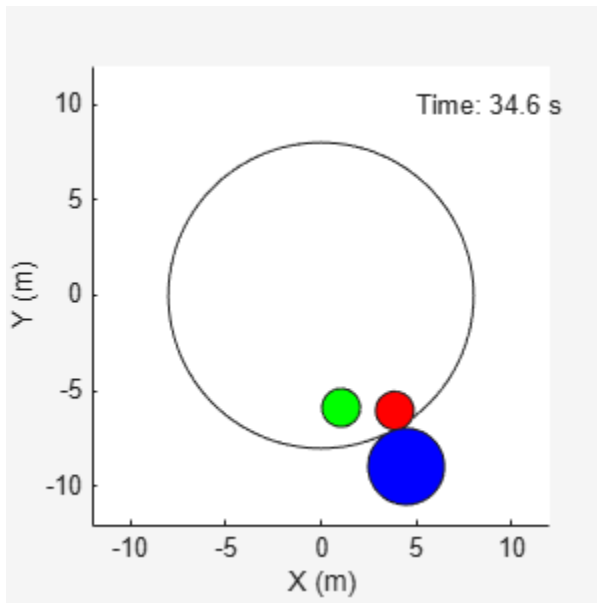
```
rng(0, "twister");
```

Create a simulation options object to run the simulation with a maximum of 600 environment steps.

```
simOptions = rlSimulationOptions(MaxSteps=600);
```

Simulate the trained (decentralized) agents with the environment.

```
simulationChoice = decentralized ;
if strcmpi(simulationChoice, "decentralized")
    dcExp = sim(env, [dcAgentA, dcAgentB], simOptions);
end
```



To simulate the trained centralized agents set `simulationChoice` to `centralized`.

```
if strcmpi(simulationChoice,"centralized")
    cnExp = sim(env,[cnAgentA,cnAgentB],simOptions);
end
```

The agents are able to push the object outside the ring.

For more information on agent simulation, see `rlSimulationOptions` and `sim`.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (`Simulink`) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the you use `setVariable` (`Simulink`) to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = resetRobots(in,RA,RB,RC,boundaryR)
% Reset the environment.

% Random initial positions
valid = false;
while ~valid
    xA0 = -10 + 20*rand;
    yA0 = -10 + 20*rand;
    xB0 = -10 + 20*rand;
    yB0 = -10 + 20*rand;
    r = 0.5*boundaryR*rand;
```

```

    th = -pi + 2*pi*rand;
    xC0 = r*cos(th);
    yC0 = r*sin(th);
    dAB = norm([xA0-xB0;yA0-yB0]);
    dBC = norm([xB0-xC0;yB0-yC0]);
    dAC = norm([xA0-xC0;yA0-yC0]);
    dA = norm([xA0;yA0]);
    dB = norm([xB0;yB0]);
    valid = dA > boundaryR && ...
        dB > boundaryR && ...
        dAB > (RA+RB) && ...
        dBC > (RB+RC) && ...
        dAC > (RA+RC);
end

% Set the variables in the simulation input object
in = setVariable(in,'xA0',xA0);
in = setVariable(in,'xB0',xB0);
in = setVariable(in,'yA0',yA0);
in = setVariable(in,'yB0',yB0);

Main helper function to plot the environment.

% Set a post sim function for visualization
in = setPostSimFcn(in,@(out) localPostSim(out,RA,RB,RC,boundaryR));
end

function out = localPostSim(out,RA,RB,RC,boundaryR)
tsqA = localGetState(out,'qA');
tsqB = localGetState(out,'qB');
tsqC = localGetState(out,'qC');
xA = tsqA.Data(:,1);
xB = tsqB.Data(:,1);
xC = tsqC.Data(:,1);
yA = tsqA.Data(:,2);
yB = tsqB.Data(:,2);
yC = tsqC.Data(:,2);
t = tsqA.Time;
for i = 1:numel(xA)
    plotEnvironment( ...
        [xA(i);xB(i);xC(i)], ...
        [yA(i);yB(i);yC(i)], ...
        [RA;RB;RC], boundaryR, t(i));
    drawnow();
end
end

function ts = localGetState(out,name)
stateObj = out.logout.get(name);
ts = stateObj.Values;
end

```

Secondary helper function to plot the environment (called from the setPostSimFcn).

```

function plotEnvironment(x,y,R,boundaryR,t)
% Plot the environment.

persistent f;

```

```
if isempty(f) || ~isvalid(f)
    f = figure( ...
        NumberTitle="off", ...
        Name="Multi Agent Collaborative Task", ...
        MenuBar="none", ...
        Position=[500,500,300,300], ...
        Visible="on");
    ha = gca(f);
    axis(ha,"equal");
    ha.XLim = [-12 12];
    ha.YLim = [-12 12];
    xlabel(ha,"X (m)");
    ylabel(ha,"Y (m)");

    D = [-1 -1 2 2]*boundaryR;
    rectangle(ha,Position=D,Curvature=[1 1]);

    hold(ha,'on');
end
% plot elements
ha = gca(f);
N = numel(R);
colors = ["r","g","b"];
for i = 1:N
    D = [(x(i)-R(i)) (y(i)-R(i)) 2*R(i) 2*R(i)];
    tagstr = sprintf("rect_%u",i);
    r = findobj(ha,Tag=tagstr);
    if isempty(r)
        r = rectangle(ha,Position=D,Curvature=[1 1],Tag=tagstr);
        r.FaceColor = colors(i);
    else
        r.Position = D;
    end
end
% Display time string
timestr = sprintf("Time: %0.1f s",t);
time = findobj(ha,Tag="time");
if isempty(time)
    text(ha,5,10,timestr,Tag="time");
else
    time.String = timestr;
end
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlPP0Agent` | `rlPP0AgentOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93
- “Train Multiple Agents for Area Coverage” on page 8-44
- “Train Multiple Agents for Path Following Control” on page 8-57

More About

- “Proximal Policy Optimization (PPO) Agent” on page 3-50
- “Train Reinforcement Learning Agents” on page 5-3

Train Multiple Agents for Area Coverage

This example demonstrates a multiagent collaborative task in which you train three proximal policy optimization (PPO) agents to achieve full coverage of a grid-world environment. As shown in this example, if you define your environment behavior using MATLAB® code, you can also incorporate it into a Simulink® environment using MATLAB Function blocks.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

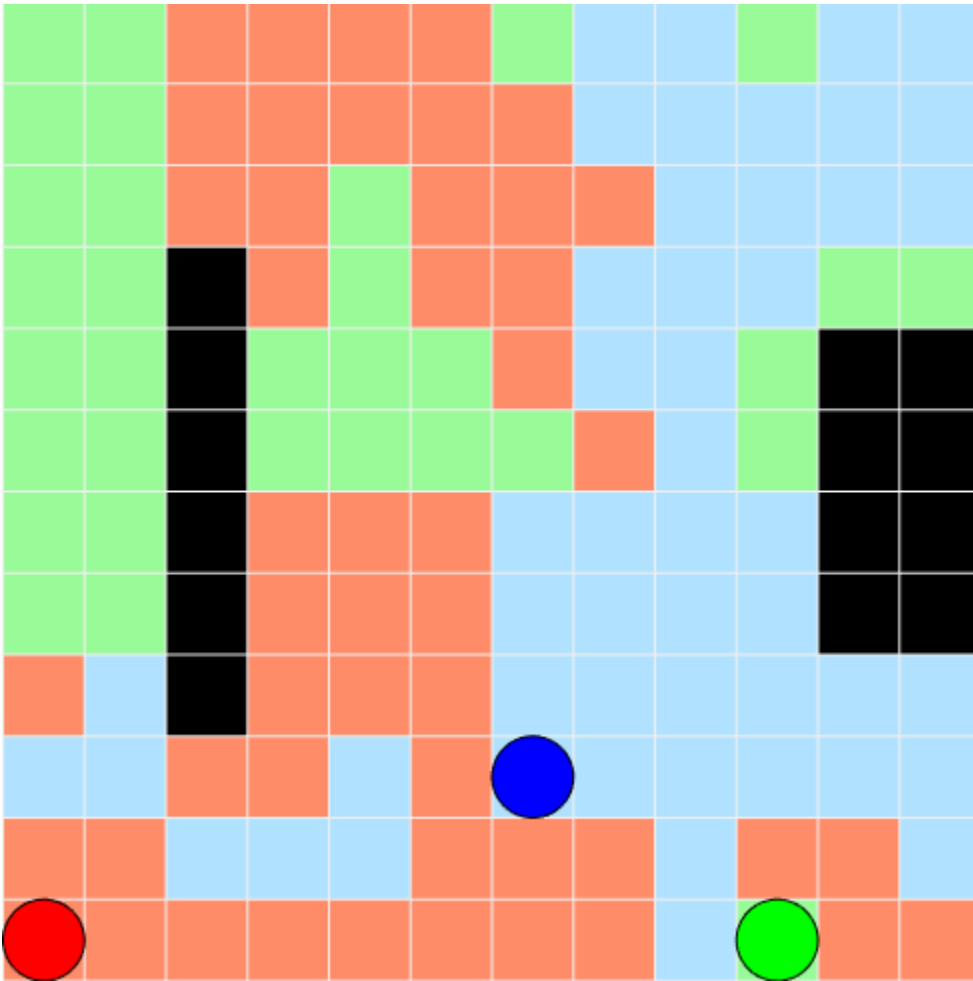
Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Environment Description

The environment in this example is a 12×12 grid world containing obstacles, with unexplored cells marked in white and obstacles marked in black. The environment contains three robots, represented by the red, green, and blue circles. Three proximal policy optimization agents with discrete action spaces control the robots. To learn more about PPO agents, see “Proximal Policy Optimization (PPO) Agent” on page 3-50.



The agents provide one of five possible movement actions (WAIT, UP, DOWN, LEFT, or RIGHT) to their respective robots. The robots decide whether an action is legal or illegal. For example, an action of moving LEFT when the robot is located next to the left boundary of the environment is deemed illegal. Similarly, actions for colliding against obstacles and other agents in the environment are illegal actions and draw penalties. The overall goal is to explore all cells within a fixed time frame.

At each time step, an agent observes the environment using a $12 \times 12 \times 4$ image. Each of the four channels in the image captures the following information in order:

- Cells with obstacles.
- Current position of the robot (self) being controlled.
- Position of other (friend) robots.
- Cells that have been explored during the episode.

The following figure shows an example of what the agent controlling the green robot observes for a given time step.



For the grid world environment:

- The search area is a 12×12 grid with obstacles.
- The observation for each agent is a 12×12×4 image.
- The discrete action set contains five actions (WAIT=0, UP=1, DOWN=2, LEFT=3, RIGHT=4).
- The simulation terminates when the grid is fully explored or the maximum number of steps is reached.

At each time step, agents receive the following rewards:

- +2 for moving to a previously unexplored cell (white).
- -1 for an illegal action (attempt to move outside the boundary or collide against other robots and obstacles).
- -1 for an action that results in no motion (lazy penalty).
- -0.5 for moving to an explored cell.
- +100 if the grid is fully explored.

Create Environment Object

Define the locations of obstacles within the grid using a matrix of indices. The first column contains the row indices, and the second column contains the column indices.

```
obsMat = [ ...
    4 3; ...
    5 3; ...
    6 3; ...
    7 3; ...
    8 3; ...
    9 3; ...
    5 11; ...
    6 11; ...
    7 11; ...
    8 11; ...
    5 12; ...
    6 12; ...
    7 12; ...
    8 12 ];
```

Initialize the robot positions.

```
sA0 = [2 2];
sB0 = [11 4];
sC0 = [3 12];
s0 = [sA0; sB0; sC0];
```

Configure the initial condition of the grid.

```
% initial grid
g0 = zeros(12,12);

% obstacle cells have the value 1.0
for idx = 1:size(obsMat,1)
    r = obsMat(idx,1);
    c = obsMat(idx,2);
    g0(r,c) = 1.0;
end

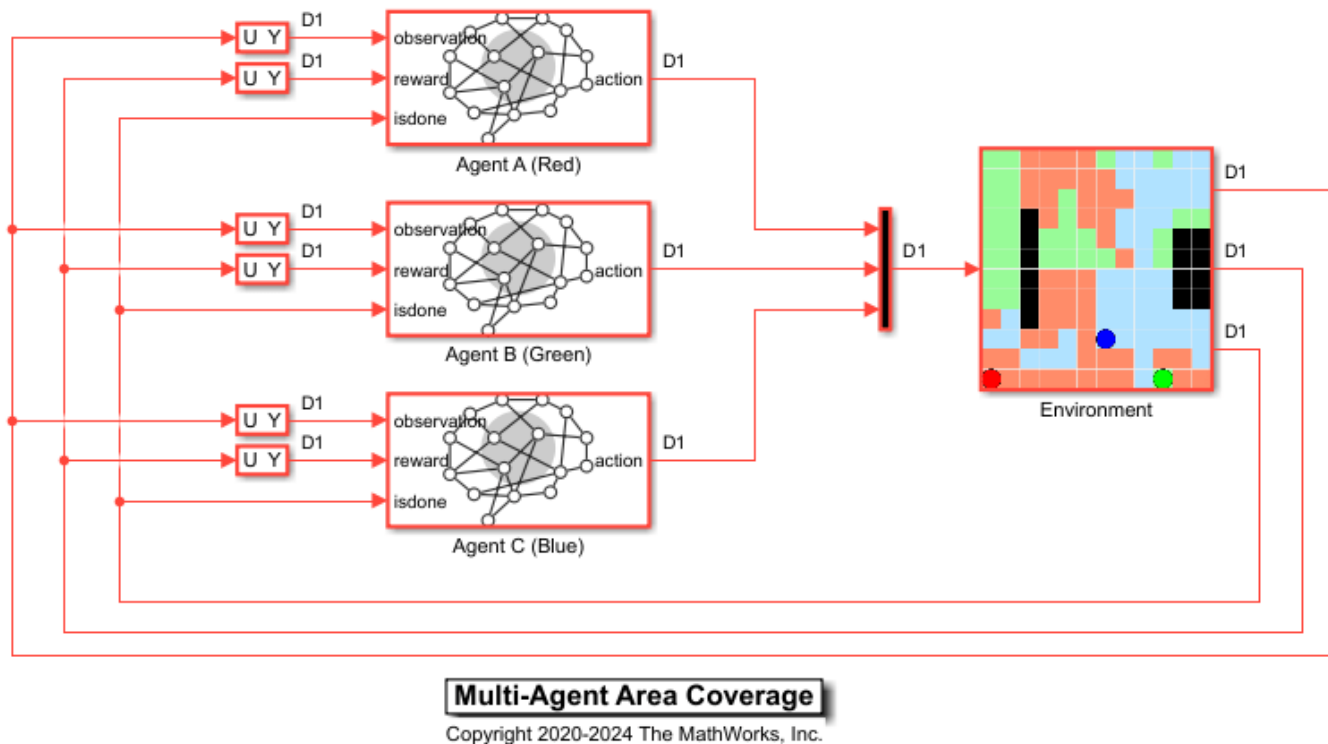
% robot cells have the values:
% 0.25 (red),
% 0.50 (green),
% and 0.75 (blue)
g0(sA0(1),sA0(2)) = 0.25;
g0(sB0(1),sB0(2)) = 0.50;
g0(sC0(1),sC0(2)) = 0.75;
```

Specify the sample time, simulation time, and maximum number of steps per episode.

```
Ts = 0.1;
Tf = 100;
maxsteps = ceil(Tf/Ts);
```

Open the model. There are three RL Agent blocks and a subsystem block that models the area coverage environment.

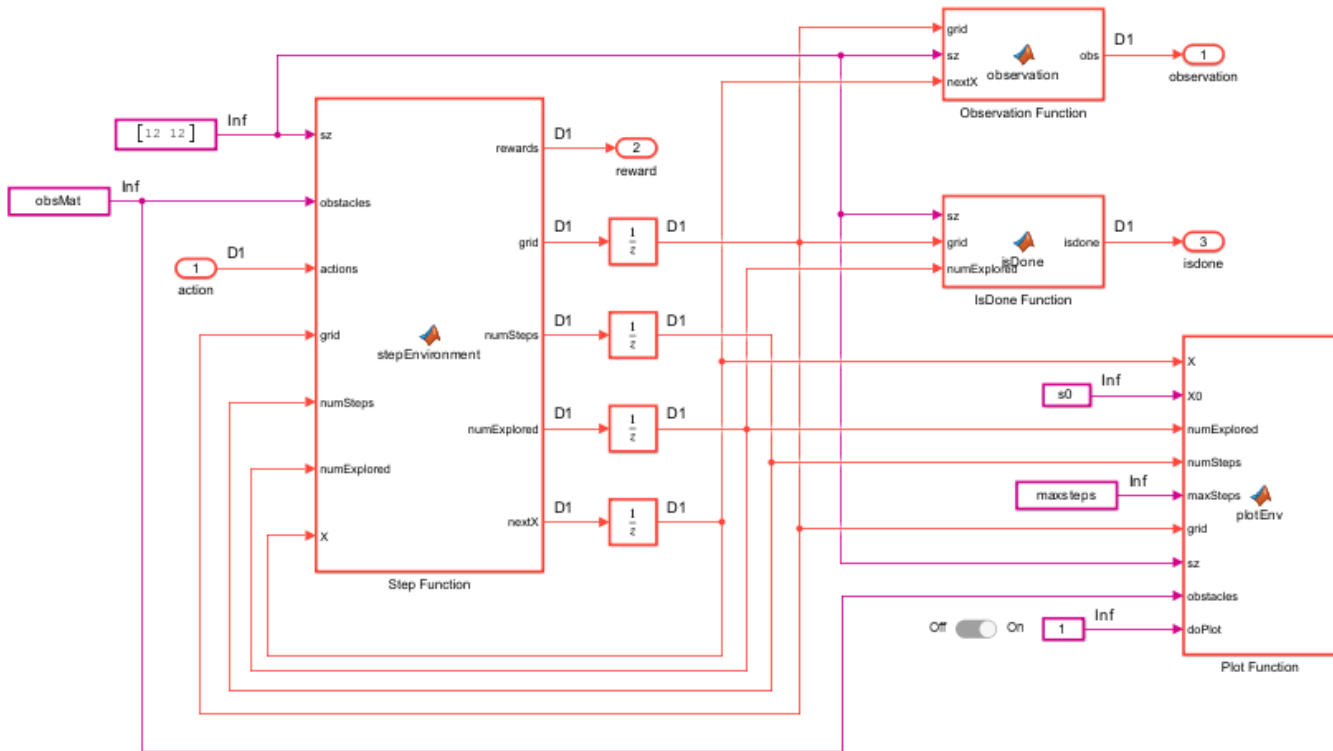
```
mdl = "rlAreaCoverage";
open_system(mdl)
```



The environment dynamics is implemented using MATLAB Function blocks. Navigate to the Environment subsystem to view the implementation.

For a related example on how to call environment functions from Simulink, see “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114.

```
open_system(mdl+"/Environment")
```



Note that the states of the environment (grid, numSteps, numExplored and nextX signals) are updated iteratively in the feedback loop. Initial values for those signals are specified in the unit delay blocks to ensure the environment is correctly initialized.

In this example, the agents are homogeneous and have the same observation and action specifications. Create the observation and action specifications for the environment. For more information, see `rlNumericSpec` and `rlFiniteSetSpec`.

```
% Define observation specifications.
obsSize = [12 12 4];
oinfo = rlNumericSpec(obsSize);
oinfo.Name = "observations";

% Define action specifications.
numAct = 5;
actionSpace = {0,1,2,3,4};
ainfo = rlFiniteSetSpec(actionSpace);
ainfo.Name = "actions";
```

Specify the block paths for the agents.

```
blks = mdl + ...
    ["/Agent A (Red)", "/Agent B (Green)", "/Agent C (Blue)"];
```

Create the environment object, specifying the same observation and action specifications for all three agents.

```
obsInfos = {oinfo,oinfo,oinfo};  
actInfos = {ainfo,ainfo,ainfo};  
env = rlSimulinkEnv mdl,blks,obsInfos,actInfos);
```

Specify a reset function for the environment. The function `resetMap` (provided at the end of this example) ensures that the robots start from random initial positions at the beginning of each episode. The random initialization makes the agents robust to different starting positions and can improve training performance.

```
env.ResetFcn = @(in) resetMap(in, obsMat);
```

Create PPO Agents with Custom Networks

The PPO agents in this example operate on a discrete action space and rely on actor and critic functions to learn the optimal policies. The agents maintain deep neural network-based function approximators for the actors and critics with similar network structures (a combination of convolution and fully connected layers). The critic outputs a scalar value representing the state value $V(s)$. The actor outputs the probabilities $\pi(a|s)$ of taking each of the five actions WAIT, UP, DOWN, LEFT, or RIGHT.

Create the actor and critic functions using the following steps.

- 1 Create the actor and critic deep neural networks.
- 2 Create the actor function objects using the `rlDiscreteCategoricalActor` command.
- 3 Create the critic function objects using the `rlValueFunction` command.

When you create the agent networks, the initial parameters are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0,"twister");
```

Use the same network structure for all three agents.

```
actorNetwork = [  
    imageInputLayer(obsSize)  
    convolution2dLayer(3,16,Padding="same", ...  
        PaddingValue=-1,Stride=2)  
    reluLayer  
    maxPooling2dLayer(3,Padding="same",Stride=2)  
    fullyConnectedLayer(256)  
    reluLayer  
    fullyConnectedLayer(256)  
    reluLayer  
    fullyConnectedLayer(numAct)  
    softmaxLayer  
];  
actorNetwork = dlnetwork(actorNetwork);  
  
criticNetwork = [  
    imageInputLayer(obsSize)  
    convolution2dLayer(3,16,Padding="same", ...  
        PaddingValue=-1,Stride=2)  
    reluLayer  
    maxPooling2dLayer(3,Padding="same",Stride=2)  
    fullyConnectedLayer(256)  
    reluLayer
```

```

        fullyConnectedLayer(256)
        reluLayer
        fullyConnectedLayer(1)
    ];
criticNetwork = dlnetwork(criticNetwork);

```

View the summary of the actor and critic networks.

```

summary(actorNetwork)

    Initialized: true

    Number of learnables: 104.8k

    Inputs:
        1 'imageinput' 12×12×4 images

```

```

summary(criticNetwork)

    Initialized: true

    Number of learnables: 103.8k

    Inputs:
        1 'imageinput' 12×12×4 images

```

Plot the actor and critic networks.

```
plot(actorNetwork)
```



```
plot(criticNetwork)
```



Create the actor and critic objects.

```

for idx = 1:3
    actor(idx) = rlDiscreteCategoricalActor(actorNetwork,oinfo,ainfo);
    critic(idx) = rlValueFunction(criticNetwork,oinfo);
end

```

Specify training options for the critic and the actor using `rlOptimizerOptions`.

- The agents learn using the learning rate `1e-3` for the actors and critics. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Use a gradient threshold of `1.0` to clip the gradients. Clipping the gradients can improve training stability.
- Use the L2 regularization factor `1e-4` to stabilize the training.

```

actorOpts = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);
criticOpts = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);

```

Specify the agent options using `rlPP0AgentOptions`, include the training options for the actor and critic. Use the same options for all three agents.

For this training:

- Use the experience horizon value of `500`. A large experience horizon can improve the stability of the training.

- Train with mini-batches of length 200. Smaller mini-batches are computationally efficient but might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.
- Use the discount factor 0.9 to favor long term rewards.
- Specify the sample time $T_s=0.1$ second for the agents.

```
agentOpts = rlPP0AgentOptions( ...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=criticOpts, ...
    ExperienceHorizon=500, ...
    MiniBatchSize=200, ...
    SampleTime=Ts, ...
    DiscountFactor=0.9);
```

Create the agents using the defined actors, critics, and agent options.

```
agentA = rlPP0Agent(actor(1),critic(1),agentOpts);
agentB = rlPP0Agent(actor(2),critic(2),agentOpts);
agentC = rlPP0Agent(actor(3),critic(3),agentOpts);
```

Alternatively, you can create the agents first, and then access their option object to modify the options using dot notation.

Train Agents

In this example, the agents are trained in centralized manner. Specify the following options for training:

- Specify all agents under one agent group.
- Specify the "centralized" learning strategy.
- Run the training for a maximum of 500 episodes, with each episode lasting a maximum of maxsteps (1000) time steps.
- Stop the training an agent when its average reward over 20 consecutive episodes is 200 or more.
- Do not store simulation data in memory to improve performance. Alternatively, you can specify SimulationStorageType as "file" to store simulation data to disk.

```
trainOpts = rlMultiAgentTrainingOptions( ...
    AgentGroups={1,2,3}, ...
    LearningStrategy="centralized", ...
    MaxEpisodes=500, ...
    MaxStepsPerEpisode=maxsteps, ...
    SimulationStorageType="none", ...
    ScoreAveragingWindowLength=20, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=200);
```

For more information, see `rlMultiAgentTrainingOptions`.

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

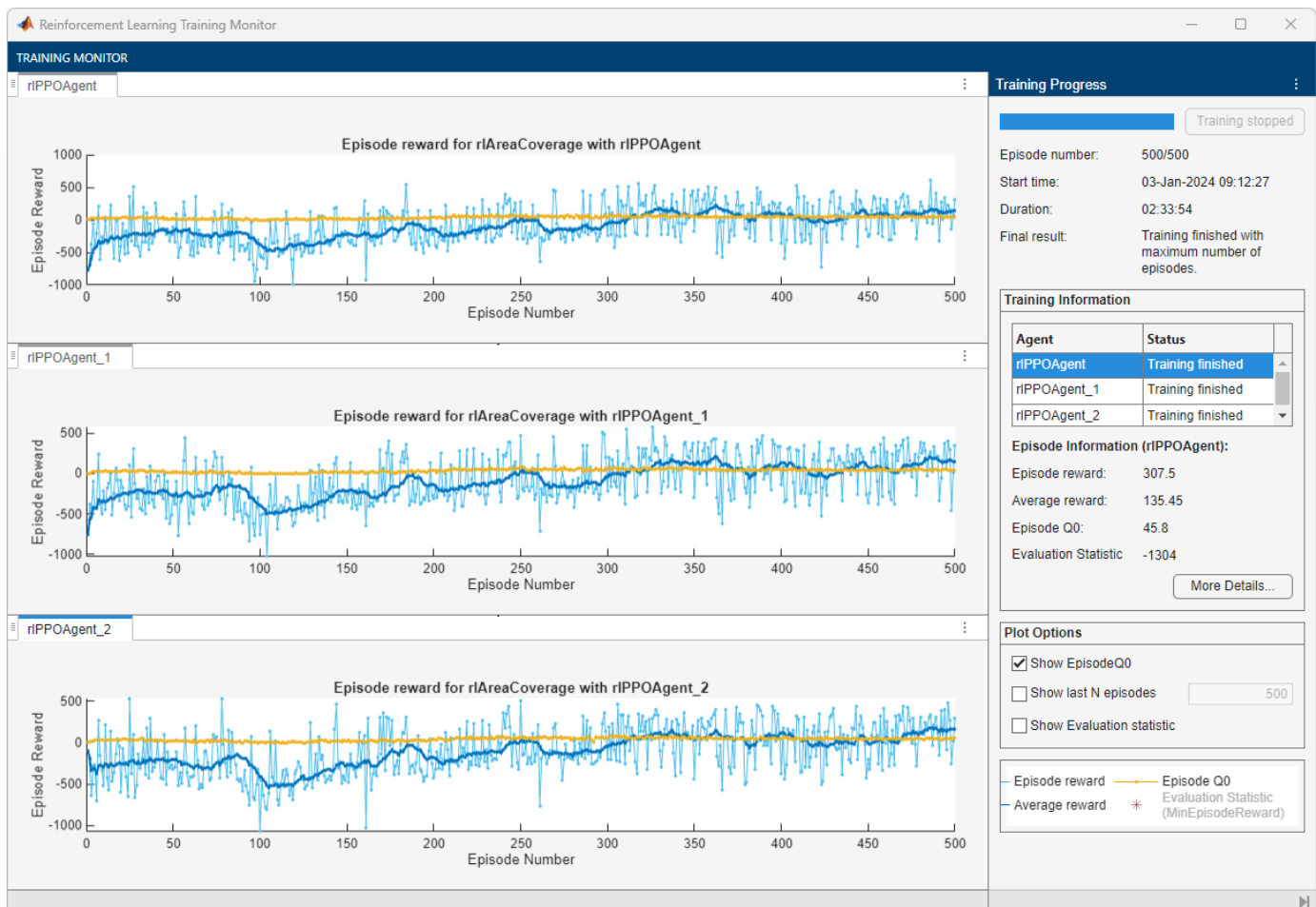
To train the agents, specify an array of agents to the `train` function. The order of the agents in the array must match the order of agent block paths specified during environment creation. Doing so

ensures that the agent objects are linked to the appropriate action and observation specifications in the environment.

Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load pretrained agent parameters by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    result = train([agentA,agentB,agentC],env,trainOpts);
else
    load("rlAreaCoverageAgents.mat","params");
    % The saved agents were trained in centralized manner
    % with synchronized model parameters.
    % Set the trained parameters on all agents.
    setLearnableParameters(agentA, params);
    setLearnableParameters(agentB, params);
    setLearnableParameters(agentC, params);
end
```

The following figure shows a snapshot of the training progress. You can expect different results due to randomness in the training process.



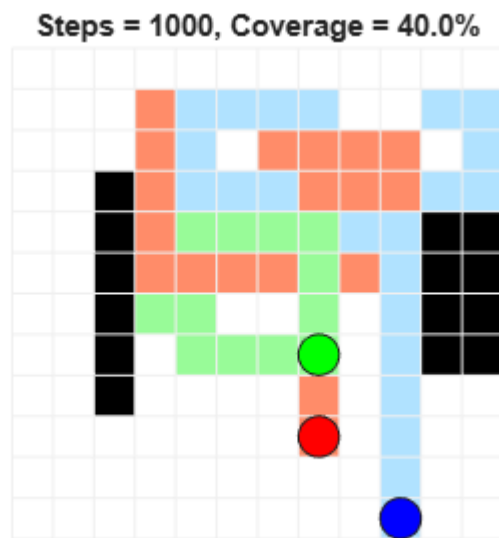
Simulate Agents

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Simulate the trained agents within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOpts = rlSimulationOptions(MaxSteps=maxsteps);
experience = sim(env,[agentA,agentB,agentC],simOpts);
```



The agents are able to achieve full coverage of the grid world.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `resetMap` uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = resetMap(in,obsMat)
% Reset function for multi agent area coverage example

gridSize = [12 12];
invalid = false;
while ~invalid
    rows = randperm(gridSize(1),3);
    cols = randperm(gridSize(2),3);
```

```
s0 = [rows' cols'];
if all(~all(s0(1,:) == obsMat,2)) && ...
    all(~all(s0(2,:) == obsMat,2)) && ...
    all(~all(s0(3,:) == obsMat,2))
    invalid = true;
end
end

% initial grid
g0 = zeros(12,12);

% obstacle cells
for idx = 1:size(obsMat,1)
    r = obsMat(idx,1);
    c = obsMat(idx,2);
    g0(r,c) = 1.0;
end

% robot cells
g0(s0(1,1),s0(1,2)) = 0.25;
g0(s0(2,1),s0(2,2)) = 0.50;
g0(s0(3,1),s0(3,2)) = 0.75;

in = setVariable(in, 's0', s0);
in = setVariable(in, 'g0', g0);
end
```

See Also

Functions

[train](#) | [sim](#) | [rlSimulinkEnv](#) | [rlNumericSpec](#) | [rlFiniteSetSpec](#)

Objects

[rlPPOAgent](#) | [rlPPOAgentOptions](#) | [rlTrainingOptions](#) | [rlSimulationOptions](#) | [rlOptimizerOptions](#)

Blocks

[RL Agent](#)

Related Examples

- “Train Default PPO Agent for Discrete Lander Vehicle” on page 5-93
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31
- “Train Multiple Agents for Path Following Control” on page 8-57
- “Create and Simulate Same Environment in Both MATLAB and Simulink” on page 2-114

More About

- “Proximal Policy Optimization (PPO) Agent” on page 3-50
- “Train Reinforcement Learning Agents” on page 5-3

Train Multiple Agents for Path Following Control

This example shows how to train multiple agents to collaboratively perform path-following control (PFC) for a vehicle. The goal of PFC is to make the ego vehicle travel at a set velocity while maintaining a safe distance from a lead car by controlling longitudinal acceleration and braking, and also while keeping the vehicle traveling along the center line of its lane by controlling the front steering angle. For more information on PFC, see Path Following Control System (Model Predictive Control Toolbox).

Overview

An example that trains a reinforcement learning agent to perform PFC is shown in “Train DDPG Agent for Path-Following Control” on page 8-97. In that example, a single deep deterministic policy gradient (DDPG) agent is trained to control both the longitudinal speed and lateral steering of the ego vehicle. In this example, you train two reinforcement learning agents — a DDPG agent provides continuous acceleration values for the longitudinal control loop and a deep Q-network (DQN) agent provides discrete steering angle values for the lateral control loop.

The trained agents perform PFC through cooperative behavior and achieve satisfactory results.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

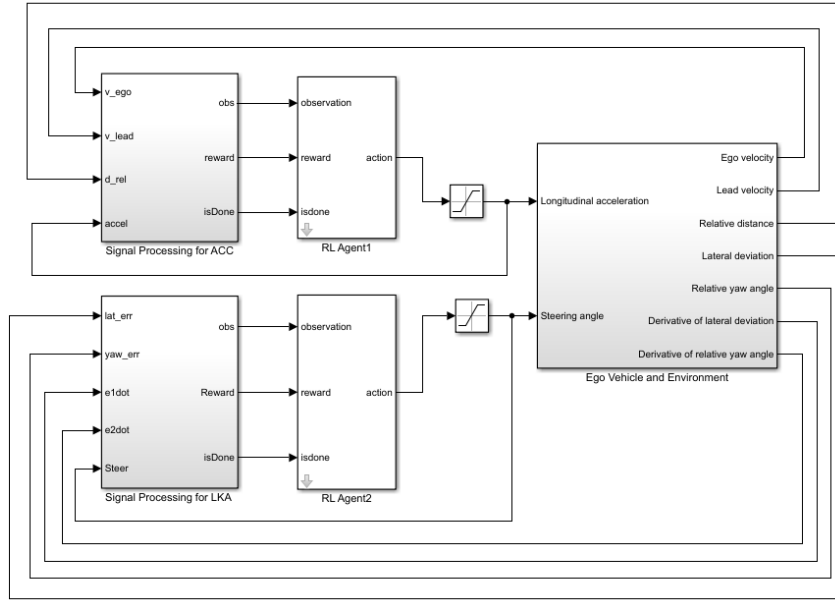
The environment for this example includes a simple bicycle model for the ego car and a simple longitudinal model for the lead car. The training goal is to make the ego car travel at a set velocity while maintaining a safe distance from lead car by controlling longitudinal acceleration and braking, while also keeping the ego car traveling along the centerline of its lane by controlling the front steering angle.

Load the environment parameters.

```
multiAgentPFCParams
```

Open the Simulink® model.

```
mdl = "rlMultiAgentPFC";  
open_system(mdl)
```



In this model, the two reinforcement learning agents (RL Agent1 and RL Agent2) provide longitudinal acceleration and steering angle signals, respectively.

The simulation terminates when any of the following conditions occur.

- $|e_1| > 1$ (magnitude of the lateral deviation exceeds 1)
- $V_{ego} < 0.5$ (longitudinal velocity of the ego car drops below 0.5).
- $D_{rel} < 0$ (distance between the ego and lead car is below zero)

For the longitudinal controller (RL Agent1):

- The reference velocity for the ego car V_{ref} is defined as follows. If the relative distance is less than the safe distance, the ego car tracks the minimum of the lead car velocity and driver-set velocity. In this manner, the ego car maintains some distance from the lead car. If the relative distance is greater than the safe distance, the ego car tracks the driver-set velocity. In this example, the safe distance is defined as a linear function of the current ego car longitudinal velocity V , that is, $t_{gap} * V + D_{default}$. The safe distance determines the tracking velocity for the ego car.
- The observations from the environment contain the longitudinal measurements: the velocity error $e_V = V_{ref} - V$, its integral $\int e$, and the ego car longitudinal velocity V .
- The action signal consists of continuous acceleration values between -3 and 2 m/s².
- The reward r_t , provided at every time step t , is

$$r_t = -(10e_V^2 + 100a_{t-1}^2) \times 0.001 - 10F_t + M_t$$

Here, a_{t-1} is the acceleration input from the previous time step, and:

- $F_t = 1$ if the simulation is terminated, otherwise $F_t = 0$.
- $M_t = 1$ if $e_V^2 < 1$, otherwise $M_t = 0$.

For the lateral controller (RL Agent2):

- The observations from the environment contain the lateral measurements: the lateral deviation e_1 , relative yaw angle e_2 , their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.
- The action signal consists of discrete steering angle actions which take values from -15 degrees (-0.2618 rad) to 15 degrees (0.2618 rad) in steps of 1 degree (0.0175 rad).
- The reward r_t , provided at every time step t , is

$$r_t = -(100e_1^2 + 500u_{t-1}^2) \times 0.001 - 10F_t + 2H_t$$

Here, u_{t-1} is the steering input from the previous time step, a_{t-1} is the acceleration input from the previous time step, and:

- $F_t = 1$ if the simulation is terminated, otherwise $F_t = 0$.
- $H_t = 1$ if $e_1^2 < 0.01$, otherwise $H_t = 0$.

The logical terms in the reward functions (F_t , M_t , and H_t) penalize the agents if the simulation terminates early, while encouraging the agents to make both the lateral error and velocity error small.

Create the observation and action specifications for longitudinal control loop.

```
obsInfo1 = rlNumericSpec([3 1]);
actInfo1 = rlNumericSpec([1 1],LowerLimit=-3,UpperLimit=2);
```

Create the observation and action specifications for lateral control loop.

```
obsInfo2 = rlNumericSpec([6 1]);
actInfo2 = rlFiniteSetSpec((-15:15)*pi/180);
```

Combine the observation and action specifications as a cell array.

```
obsInfo = {obsInfo1,obsInfo2};
actInfo = {actInfo1,actInfo2};
```

Create a Simulink environment object, specifying the block paths for both agent blocks. The order of the block paths must match the order of the observation and action specification cell arrays.

```
blks = mdl + ["/RL Agent1", "/RL Agent2"];
env = rlSimulinkEnv(mdl,blks,obsInfo,actInfo);
```

Specify a reset function for the environment using the `ResetFcn` property. The function `pfcResetFcn` on page 8-65 (defined at the end of this example) sets the initial conditions of the lead and ego vehicles at the beginning of every episode during training.

```
env.ResetFcn = @pfcResetFcn;
```

Create Agents

For this example you create two reinforcement learning agents. Both agents operate at the same sample time in this example. Set the sample time value (in seconds).

```
Ts = 0.1;
```

When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Longitudinal Control

The agent for the longitudinal control loop is a DDPG agent. A DDPG agent approximates the long-term reward given observations and actions using a critic value function representation and selects actions using an actor policy representation. For more information on creating deep neural network value function and policy representations, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Use the `createCCAgent` on page 8-65 function provided at the end of this example to create a DDPG agent for longitudinal control. The structure of this agent is similar to the “Train DDPG Agent for Adaptive Cruise Control” on page 8-68 example.

```
agent1 = createCCAgent(obsInfo1, actInfo1, Ts);
```

Lateral Control

The agent for the lateral control loop is a DQN agent. A DQN agent approximates the long-term reward given observations and actions using a critic value function representation.

Use the `createLKAAGENT` on page 8-66 function provided at the end of this example to create a DQN agent for lateral control. The structure of this agent is similar to the “Train DQN Agent for Lane Keeping Assist” on page 8-76 example.

```
agent2 = createLKAAGENT(obsInfo2, actInfo2, Ts);
```

Train Agents

Specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of `maxsteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Verbose` and `Plots` options).
- Stop the training the DDPG and DQN agents when they receive an average reward greater than 480 and 1195, respectively. When one agent reaches its stop criteria, it simulates its own policy without learning while the other agent continues training.
- Do not save simulation data during training to improve performance. To save simulation data set `SimulationStorageType` to “file” or “memory”.

```
Tf = 60; % simulation time
maxepisodes = 500;
maxsteps = ceil(Tf/Ts);
trainingOpts = rlMultiAgentTrainingOptions( ...
    AgentGroups={1,2}, ...
    LearningStrategy="decentralized", ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=[480,1195], ...
    SimulationStorageType="none");
```


Fix the random stream for reproducibility.

```
rng(0,"twister");
```

Train the agents using the `train` function. Training these agents is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train([agent1,agent2],env,trainingOpts);
else
    % Load pretrained agents for the example.
    load("rlPFCAgents.mat")
end
```

The following figure shows a snapshot of the training progress for the two agents.



Simulate Agents

Fix the random stream for reproducibility.

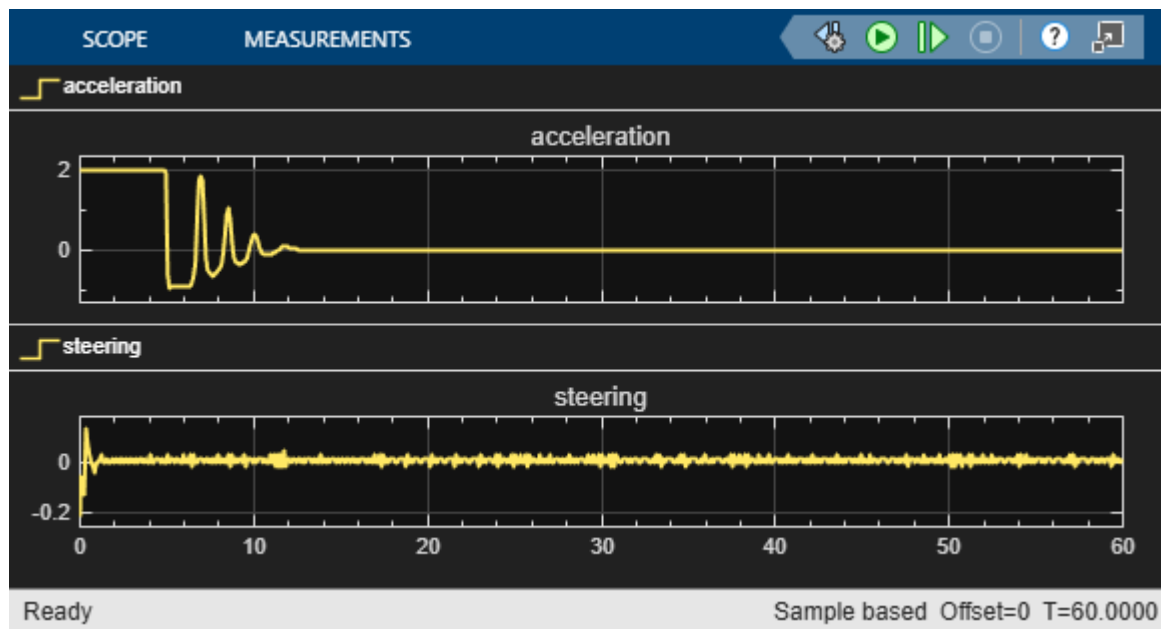
```
rng(0, "twister");
```

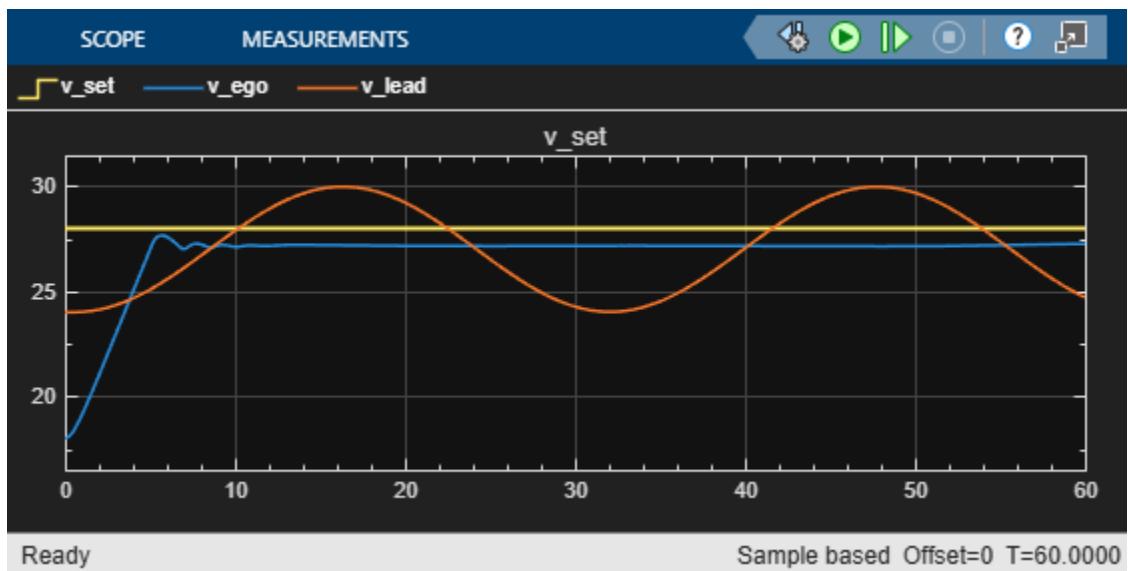
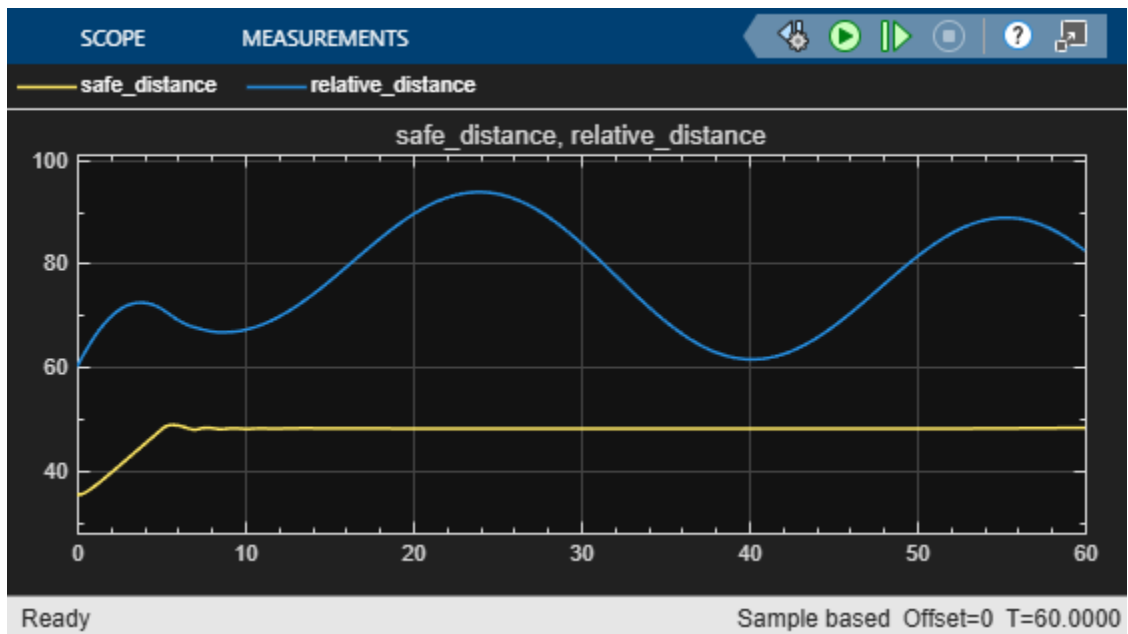
To validate the performance of the trained agents, simulate the agents within the Simulink environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

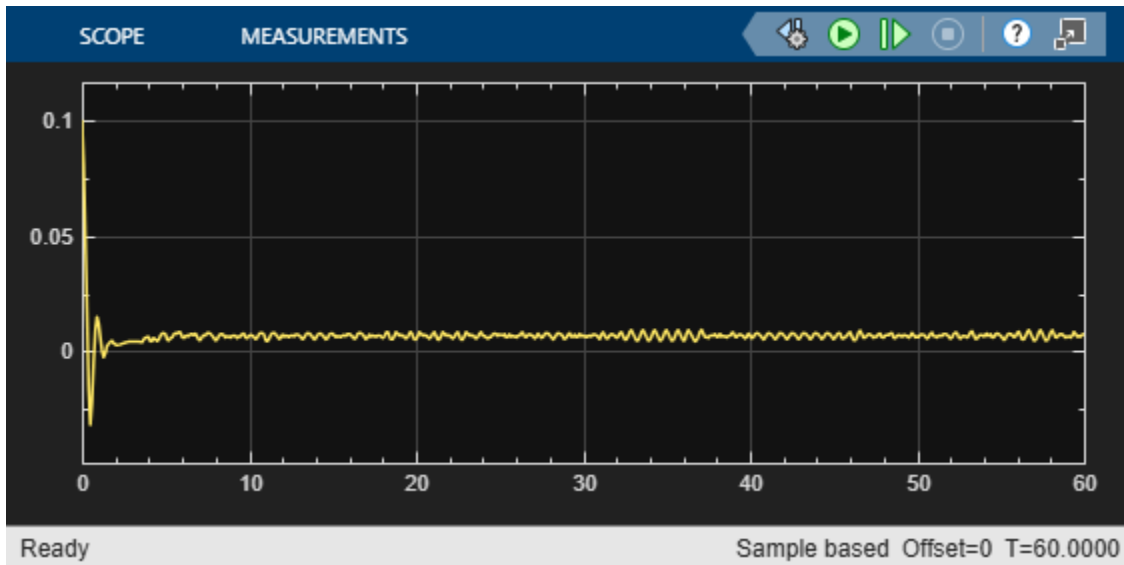
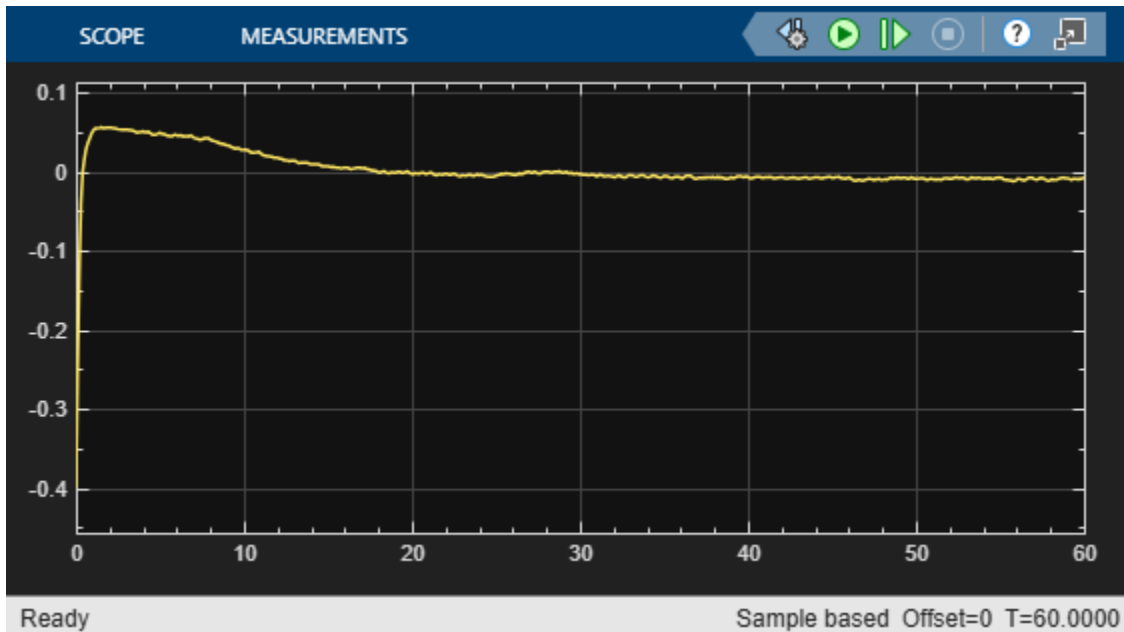
```
simOptions = rlSimulationOptions(MaxSteps=maxsteps);  
experience = sim(env,[agent1, agent2],simOptions);
```

To demonstrate the trained agent using deterministic initial conditions, simulate the model in Simulink.

```
e1_initial = -0.4;  
e2_initial = 0.1;  
x0_lead = 70;  
sim mdl
```







These plots show the results when the lead car is 70 m ahead of the ego car at the beginning of simulation.

- The lead car changes speed from 24 m/s to 30 m/s periodically (third plot). The ego car maintains a safe distance throughout the simulation (second plot).
- From 0 to 30 seconds, the ego car tracks the set velocity (third plot) and experiences some acceleration (first plot). After that, the acceleration is reduced to 0.
- The fourth plot shows the lateral deviation, which greatly decreases within one second and remains less than 0.1 m.
- The last plot shows the yaw angle error, which also greatly decreases within one second and remains at very low values.

Local Functions

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function uses the `setVariable` (Simulink) function to set variable values in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = pfcResetFcn(in)

% random value for initial position of lead car
in = setVariable(in, 'x0_lead', 40+randi(60,1,1));

% random value for lateral deviation
in = setVariable(in, 'el_initial', 0.5*(-1+2*rand));

% random value for relative yaw angle
in = setVariable(in, 'e2_initial', 0.1*(-1+2*rand));

end
```

Helper function for creating a DDPG agent.

```
function agent = createACCAgent(observationInfo, actionInfo, Ts)

L = 48; % number of neurons
statePath = [
    featureInputLayer(observationInfo.Dimension(1), Name="obsInLyr")
    fullyConnectedLayer(L)
    reluLayer()
    fullyConnectedLayer(L)
    additionLayer(2, Name="add")
    reluLayer()
    fullyConnectedLayer(L)
    reluLayer()
    fullyConnectedLayer(1)];
actionPath = [
    featureInputLayer(actionInfo.Dimension(1), Name="actInLyr")
    fullyConnectedLayer(L, Name="fc5")];
criticNetwork = layerGraph(statePath);
criticNetwork = addLayers(criticNetwork, actionPath);
criticNetwork = connectLayers(criticNetwork, "fc5", "add/in2");
criticNetwork = dlnetwork(criticNetwork);
critic = rlQValueFunction(criticNetwork, observationInfo, actionInfo, ...
    ObservationInputNames="obsInLyr", ...
    ActionInputNames="actInLyr");

actorNetwork = [
    featureInputLayer(observationInfo.Dimension(1), Name="obsInLyr")
    fullyConnectedLayer(L)
    reluLayer()
    fullyConnectedLayer(L)
    reluLayer()]
```

```
        fullyConnectedLayer(L)
        reluLayer()
        fullyConnectedLayer(1)
        tanhLayer()
        scalingLayer(Name="ActorScaling1",Scale=2.5,Offset=-0.5)];
actorNetwork = dlnetwork(actorNetwork);
actor = rlContinuousDeterministicActor( ...
    actorNetwork, ...
    observationInfo, ...
    actionInfo);

actorOptions = rlOptimizerOptions( ...
    LearnRate=1e-4, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);

criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);

agentOptions = rlDDPGAgentOptions( ...
    SampleTime=Ts, ...
    CriticOptimizerOptions=criticOptions, ...
    ActorOptimizerOptions=actorOptions, ...
    MiniBatchSize=128, ...
    ExperienceBufferLength=1e6);
agentOptions.NoiseOptions.StandardDeviation = 0.6;
agentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-5;

agent = rlDDPGAgent(actor,critic,agentOptions);
end
```

Helper function for creating a DQN agent.

```
function agent = createLKAAGent(observationInfo,actionInfo,Ts)

L = 24; % number of neurons
statePath = [
    featureInputLayer(observationInfo.Dimension(1),Name="obsInLyr")
    fullyConnectedLayer(L)
    reluLayer()
    fullyConnectedLayer(L)
    additionLayer(2,Name="add")
    reluLayer()
    fullyConnectedLayer(L)
    reluLayer()
    fullyConnectedLayer(1)];
actionPath = [
    featureInputLayer(actionInfo.Dimension(1),Name="actInLyr")
    fullyConnectedLayer(L,Name="fc5")];
criticNetwork = layerGraph(statePath);
criticNetwork = addLayers(criticNetwork, actionPath);
criticNetwork = connectLayers(criticNetwork,"fc5","add/in2");
criticNetwork = dlnetwork(criticNetwork);
critic = rlQValueFunction(criticNetwork, ...
    observationInfo, ...
```

```

        actionInfo, ...
        ObservationInputNames="obsInLyr", ...
        ActionInputNames="actInLyr");

criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-4, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);

agentOptions = rlDQNAgentOptions( ...
    SampleTime=Ts, ...
    UseDoubleDQN=true, ...
    CriticOptimizerOptions=criticOptions, ...
    MiniBatchSize=64, ...
    ExperienceBufferLength=1e6);

agent = rlDQNAgent(critic,agentOptions);
end

```

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlDQNAgent | rlDQNAgentOptions | rlDDPGAgent | rlDDPGAgentOptions |
rlTrainingOptions | rlSimulationOptions

Blocks

RL Agent

Related Examples

- “Train Multiple Agents for Area Coverage” on page 8-44
- “Train Multiple Agents to Perform Collaborative Task” on page 8-31
- “Train DDPG Agent for Adaptive Cruise Control” on page 8-68
- “Train Hybrid SAC Agent for Path-Following Control” on page 8-109
- “Adaptive Cruise Control System Using Model Predictive Control” (Model Predictive Control Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train DDPG Agent for Adaptive Cruise Control

This example shows how to train a deep deterministic policy gradient (DDPG) agent for adaptive cruise control (ACC) in Simulink®. For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

Fix Random Seed Generator to Improve Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Adaptive Cruise Control Simulink Model

The reinforcement learning environment for this example is the simple longitudinal dynamics for an ego car and lead car. The training goal is to make the ego car travel at a set velocity while maintaining a safe distance from lead car by controlling longitudinal acceleration and braking. This example uses the same vehicle model as the “Adaptive Cruise Control System Using Model Predictive Control” (Model Predictive Control Toolbox) example.

Specify the initial position and velocity for the two vehicles.

```
x0_lead = 50;    % initial position for lead car (m)
v0_lead = 25;    % initial velocity for lead car (m/s)
x0_ego = 10;     % initial position for ego car (m)
v0_ego = 20;     % initial velocity for ego car (m/s)
```

Specify standstill default spacing (m), time gap (s) and driver-set velocity (m/s).

```
D_default = 10;
t_gap = 1.4;
v_set = 30;
```

To simulate the physical limitations of the vehicle dynamics, constraint the acceleration to the range $[-3, 2]$ m/s².

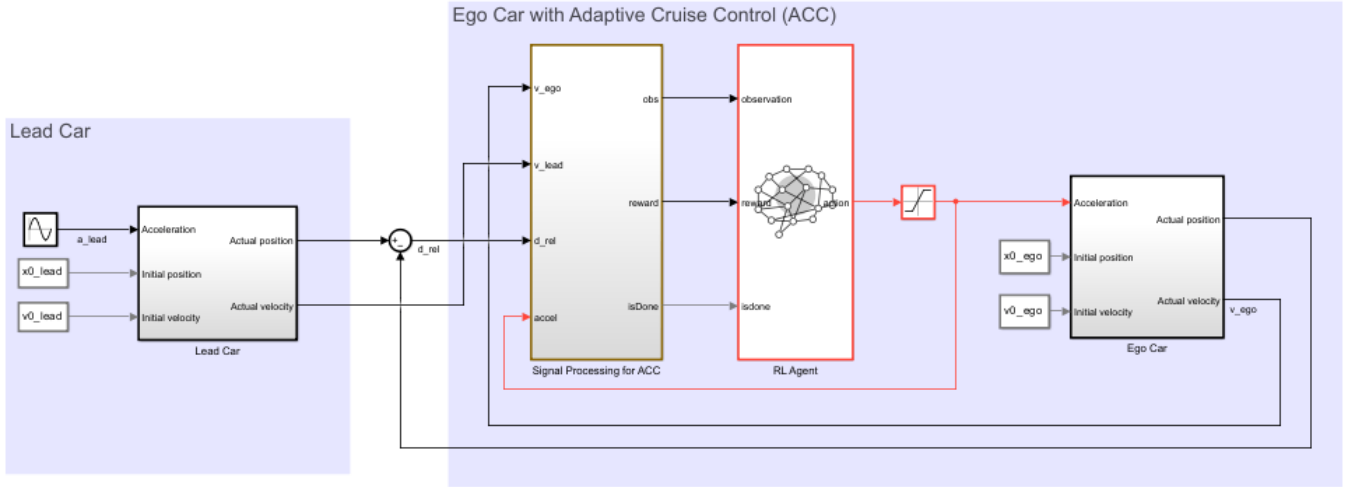
```
amin_ego = -3;
amax_ego = 2;
```

Define the sample time T_s and simulation duration T_f in seconds.

```
Ts = 0.1;
Tf = 60;
```

Open the model.


```
mdl = "rLACCMdl";
open_system(mdl)
agentblk = mdl + "/RL Agent";
```



Copyright 2018-2024 The MathWorks, Inc.

In this model:

- The acceleration action signal from the agent to the environment is from -3 to 2 m/s^2 .
- The reference velocity for the ego car V_{ref} is defined as follows. If the relative distance is less than the safe distance, the ego car tracks the minimum of the lead car velocity and driver-set velocity. In this manner, the ego car maintains some distance from the lead car. If the relative distance is greater than the safe distance, the ego car tracks the driver-set velocity. In this example, the safe distance is defined as a linear function of the ego car longitudinal velocity V ; that is, $t_{gap} * V + D_{default}$. The safe distance determines the reference tracking velocity for the ego car.
- The observations from the environment are the velocity error $e = V_{ref} - V_{ego}$, its integral $\int e$, and the ego car longitudinal velocity V .
- The simulation is terminated when longitudinal velocity of the ego car is less than 0, or the relative distance between the lead car and ego car becomes less than 0.
- The reward r_t , provided at every time step t , is

$$r_t = -(0.1e_t^2 + u_{t-1}^2) + M_t$$

where u_{t-1} is the control input from the previous time step. The logical value $M_t = 1$ if velocity error $e_t^2 < 0.25$; otherwise, $M_t = 0$.

Create Environment Object

Create a reinforcement learning environment object for the model.

Create the observation specification.

```
obsInfo = rlNumericSpec([3 1], ...  
    LowerLimit=-inf*ones(3,1), ...  
    UpperLimit=inf*ones(3,1));  
obsInfo.Name = "observations";  
obsInfo.Description = "velocity error and ego velocity";
```

Create the action specification.

```
actInfo = rlNumericSpec([1 1], ...  
    LowerLimit=-3,UpperLimit=2);  
actInfo.Name = "acceleration";
```

Create the environment object.

```
env = rlSimulinkEnv mdl,agentblk,obsInfo,actInfo);
```

To define the initial condition for the position of the lead car, specify the reset function `localResetFcn`, which is defined at the end of the example.

```
env.ResetFcn = @localResetFcn;
```

Create Default DDPG Agent

You will create and train a DDPG agent in this example. The agent uses:

- A Q-value function critic to estimate the value of the policy. The critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated discounted cumulative long-term reward given the action from the state corresponding to the current observation, and following the policy thereafter).
- A deterministic policy over a continuous action space, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

The actor and critic functions are approximated using neural network representations. Create an agent initialization object to initialize the networks with the hidden layer size 200.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=200);
```

Configure the hyperparameters for training using `rlDDPGAgentOptions`.

- Specify a mini-batch size of 50. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Specify the actor and critic learning rates as $3e-4$ and $1e-2$. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Specify the sample time as T_s .

Specify the actor and critic optimizer options.

```
actorOptions = rlOptimizerOptions( ...  
    LearnRate           = 3e-4, ...  
    GradientThreshold   = 1, ...  
    L2RegularizationFactor = 1e-4);
```

```
criticOptions = rlOptimizerOptions( ...
    LearnRate          = 1e-2, ...
    GradientThreshold  = 1, ...
    L2RegularizationFactor = 1e-4);
```

Create the agent options object. For more information, see `rlDDPGAgentOptions`.

```
agentOptions = rlDDPGAgentOptions( ...
    SampleTime          = Ts, ...
    MiniBatchSize       = 50, ...
    ActorOptimizerOptions = actorOptions, ...
    CriticOptimizerOptions = criticOptions, ...
    ExperienceBufferLength = 1e6);
```

The DDPG agent in this example uses an Ornstein-Uhlenbeck (OU) noise model for exploration. For the noise model:

- Specify a mean attraction value of 0.7. A larger mean attraction value keeps the noise values close to the mean.
- Specify a standard deviation value of 0.4 to improve exploration during training.
- Specify a decay rate of 1e-5 for the standard deviation. The decay causes exploration to reduce gradually as the training progresses.

```
agentOptions.NoiseOptions.MeanAttractionConstant = 0.7;
agentOptions.NoiseOptions.StandardDeviation      = 0.4;
agentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-5;
```

Create the DDPG agent using the observation and action input specifications, initialization options and agent options. When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
agent = rlDDPGAgent(obsInfo, actInfo, initOpts, agentOptions);
```

For more information, see `rlDDPGAgent`.

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run the training episode for 500 episodes, with each episode lasting a maximum of `maxsteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor window.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 500;
maxsteps = ceil(Tf/Ts);
trainingOpts = rlTrainingOptions( ...
    MaxEpisodes          = maxepisodes, ...
    MaxStepsPerEpisode    = maxsteps, ...
    Plots                 = "training-progress", ...
    StopTrainingCriteria = "none");
```

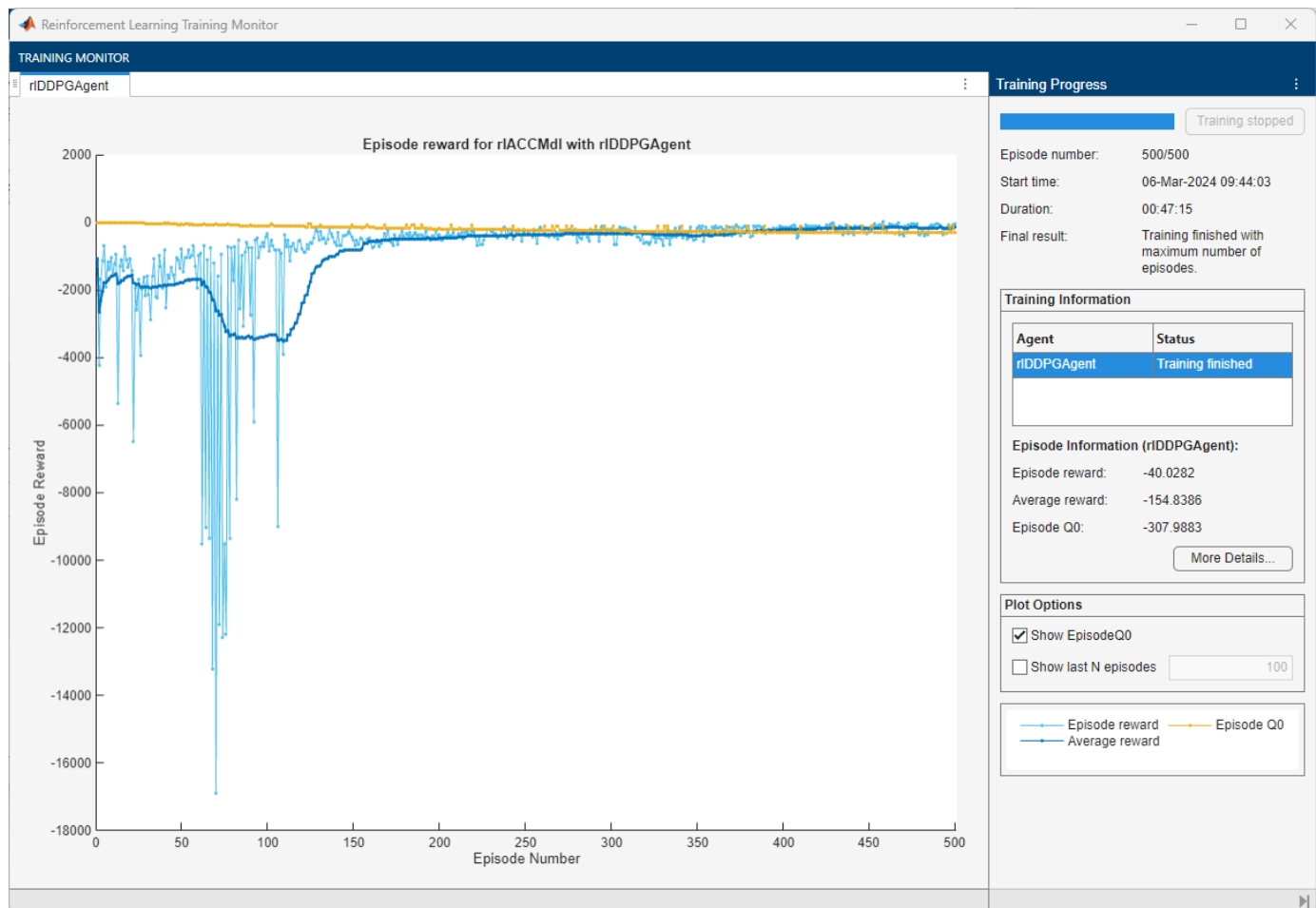
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent.
    trainResult = train(agent, env, trainingOpts);
else
    % Load a pretrained agent for the example.
    load("SimulinkACDDPG.mat", "agent")
end
```

An example of the training is shown below. The actual results might vary because of randomness in the training process.



Simulate Agent

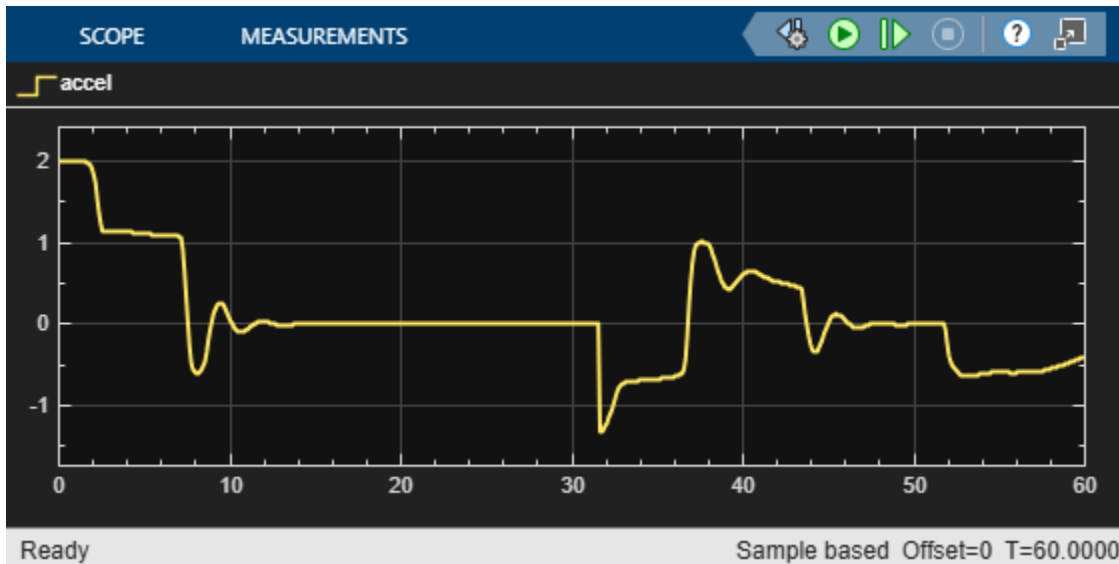
Fix the random stream for reproducibility.

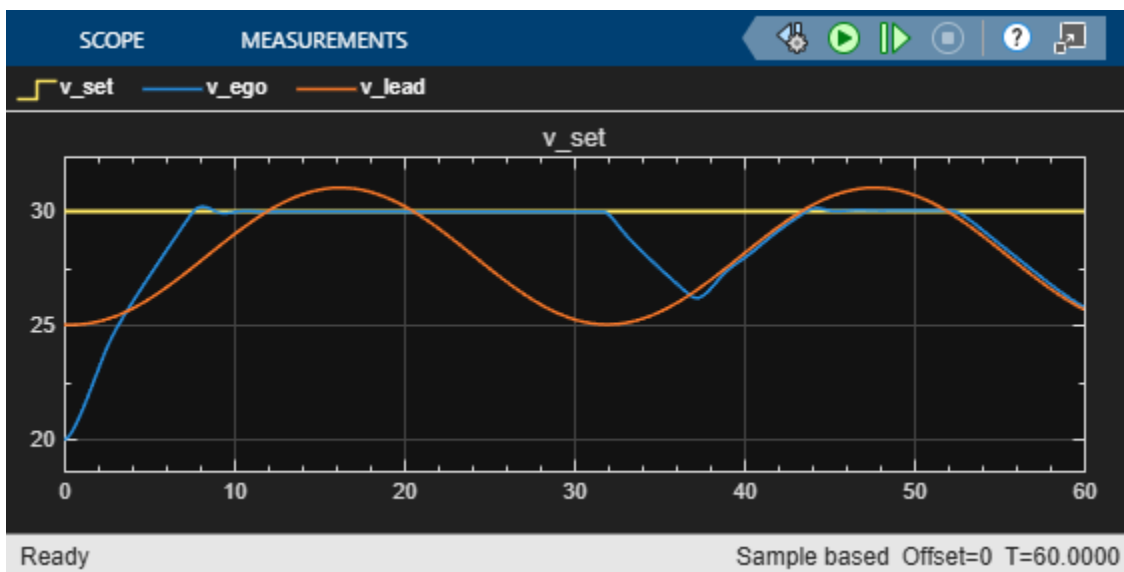
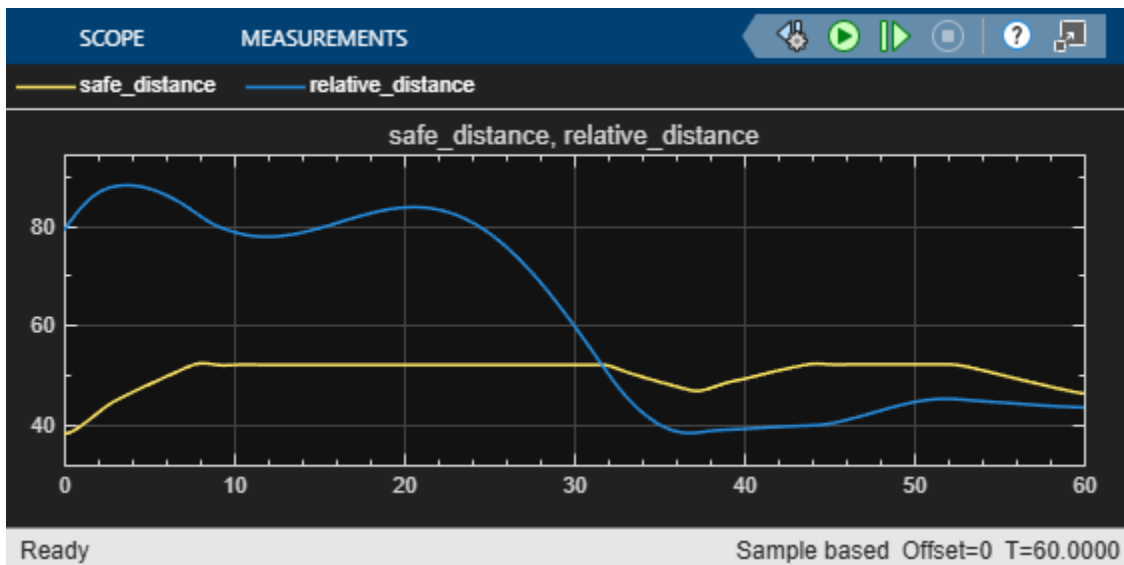
```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To validate the performance of the trained agent deterministically, set the initial position of the lead car to 80 (m) and simulate the model. To validate the performance for a randomly chosen initial position, set `useRandomInitialConditions` to `true`.

```
useRandomInitialConditions = ;
if useRandomInitialConditions
    % The simulink sim function uses the workspace value of x0_lead
    x0_lead = 80;
    sim mdl;
else
    % The reinforcement learning sim function
    % randomizes the value of x0_lead
    simOptions = rlSimulationOptions(MaxSteps=maxsteps);
    experience = sim(env,agent,simOptions);
end
```





For more information on agent simulation, see `rlSimulationOptions` and `sim`.

These plots show deterministic simulation results when lead car is initially 80 (m) ahead of the ego car.

- In the first 30 seconds, the relative distance is greater than the safe distance (middle plot), so the ego car tracks the set velocity (bottom plot). To speed up and reach the set velocity, acceleration is initially positive (top plot).
- At approximately 8 seconds, the ego car is able to track the set velocity and acceleration is reduced to zero.
- After 30 seconds, the relative distance is less than the safe distance (middle plot), so the ego car tracks the minimum of the lead velocity and set velocity. To slow down and track the lead car velocity, the acceleration turns negative (top plot).

- Throughout the simulation the ego car adjusts its acceleration to keep on tracking either the minimum between the lead velocity and the set velocity, or the set velocity, depending on whether the relative distance is deemed to be safe or not.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (Simulink) function to randomly set the variable `x0_lead` in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
% Reset the initial position of the lead car.
in = setVariable(in,"x0_lead",40+randi(60,1,1));
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlQValueFunction` |
`rlContinuousDeterministicActor` | `rlTrainingOptions` | `rlSimulationOptions` |
`rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train Multiple Agents for Path Following Control” on page 8-57
- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Adaptive Cruise Control System Using Model Predictive Control” (Model Predictive Control Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Train DQN Agent for Lane Keeping Assist

This example shows how to train a deep Q-learning network (DQN) agent for lane keeping assist (LKA) in Simulink®. For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Simulink Model for the Ego Car

The reinforcement learning environment for this example is a simple bicycle model for the ego vehicle dynamics. The training goal is to keep the ego vehicle traveling along the centerline of the lanes by adjusting the front steering angle. This example uses the same vehicle model as in “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox). The ego car dynamics is specified by the following parameters.

```
m = 1575;    % total vehicle mass (kg)
Iz = 2875;   % yaw moment of inertia (mNs^2)
lf = 1.2;    % longitudinal distance from CG to front tires (m)
lr = 1.6;    % longitudinal distance from CG to rear tires (m)
Cf = 19000;  % cornering stiffness of front tires (N/rad)
Cr = 33000;  % cornering stiffness of rear tires (N/rad)
Vx = 15;     % longitudinal velocity (m/s)
```

Define the sample time T_s and simulation duration T_f in seconds.

```
Ts = 0.1;
T = 15;
```

The output of the LKA system is the front steering angle of the ego car. To simulate the physical limitations of the ego car, constrain the steering angle to the range $[-0.5, 0.5]$ rad.

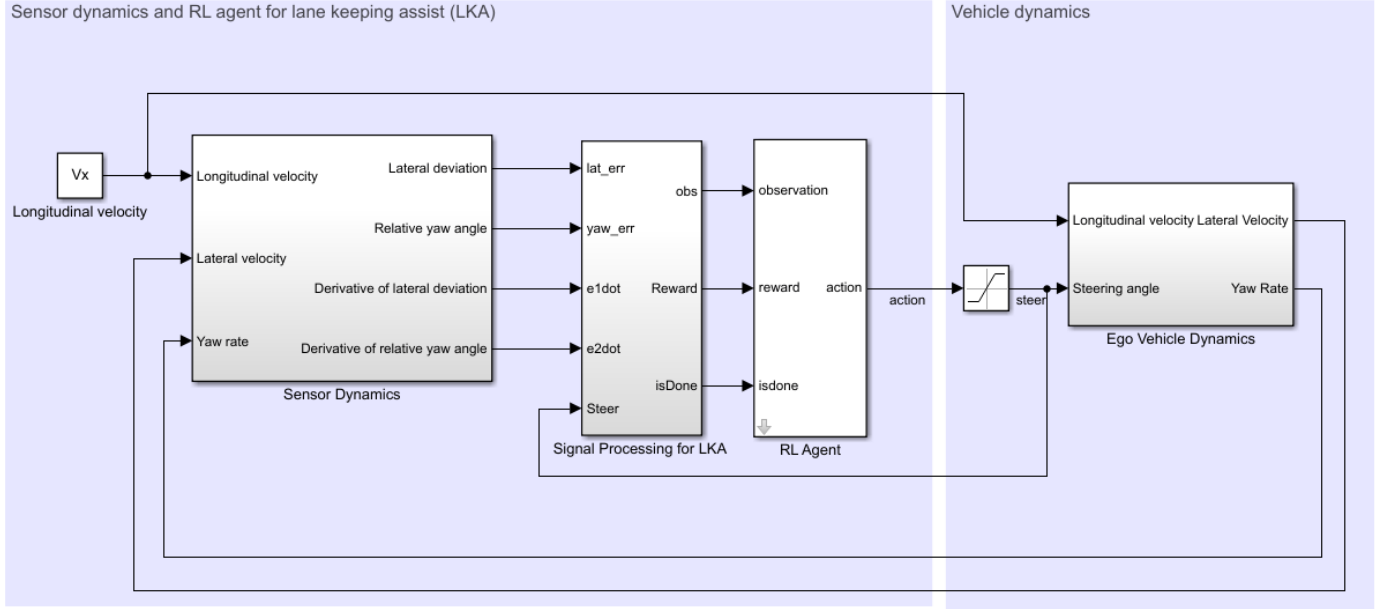
```
u_min = -0.5;
u_max = 0.5;
```

The curvature of the road is defined by a constant $0.001 \text{ (m}^{-1}\text{)}$. The initial value for the lateral deviation is 0.2 m and the initial value for the relative yaw angle is -0.1 rad.

```
rho = 0.001;
e1_initial = 0.2;
e2_initial = -0.1;
```

Open the model.


```
mdl = "rLKAMdl";
open_system(mdl);
agentblk = mdl + "/RL Agent";
```



Copyright 2018 The MathWorks, Inc.

In this model:

- The steering-angle action signal from the agent to the environment is from -15 degrees to 15 degrees.
- The observations from the environment are the lateral deviation e_1 , the relative yaw angle e_2 , their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.
- The simulation is terminated when the lateral deviation $|e_1| > 1$.
- The reward r_t , provided at every time step t , is

$$r_t = -(10e_1^2 + 5e_2^2 + 2u^2 + 5\dot{e}_1^2 + 5\dot{e}_2^2)$$

where u is the control input from the previous time step $t - 1$.

Create Environment Interface

Create a reinforcement learning environment object for the ego vehicle. To do so, first create the observation and action specifications.

```
observationInfo = rlNumericSpec([6 1], ...
    LowerLimit=-inf*ones(6,1), ...
    UpperLimit=inf*ones(6,1));
observationInfo.Name = "observations";
observationInfo.Description = ...
    "Lateral deviation and relative yaw angle";
actionInfo = rlFiniteSetSpec((-15:15)*pi/180);
actionInfo.Name = "steering";
```

Then, create the environment object.

```
env = rlSimulinkEnv mdl, agentblk, observationInfo, actionInfo);
```

The object has a discrete action space where the agent can apply one of 31 possible steering angles from -15 degrees to 15 degrees. The observation is the six-dimensional vector containing lateral deviation, relative yaw angle, as well as their derivatives and integrals with respect to time.

To define the initial condition for lateral deviation and relative yaw angle, specify an environment reset function using an anonymous function handle. This reset function, defined at the end of the example, randomizes the initial values for the lateral deviation and relative yaw angle.

```
env.ResetFcn = @(in) localResetFcn(in);
```

Create DQN Agent

DQN agents use a parameterized Q-value function approximator to estimate the value of the policy.

Because DQN agents have a discrete action space, you have the option to create a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic.

A discrete vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible discrete actions. The value of each output element represents the expected discounted cumulative long-term reward for taking the action corresponding to the element number, from the state corresponding to the current observation, and following the policy afterwards.

To model the parameterized Q-value function within the critic, use a deep neural network with one input (the six-dimensional observed state) and one output vector with 31 elements (evenly spaced steering angles from -15 to 15 degrees).

```
% Define number of inputs, neurons, and outputs
nI = observationInfo.Dimension(1); % number of inputs (6)
nL = 24; % number of neurons
nO = numel(actionInfo.Elements); % number of outputs (31)
```

```
% Define network as array of layer objects
```

```
dnn = [
    featureInputLayer(nI)
    fullyConnectedLayer(nL)
    reluLayer
    fullyConnectedLayer(nL)
    reluLayer
    fullyConnectedLayer(nO)
];
```

```
% Convert to dlnetwork object
```

```
dnn = dlnetwork(dnn);
```

Display the number of parameters and view the network configuration.

```
summary(dnn)
```

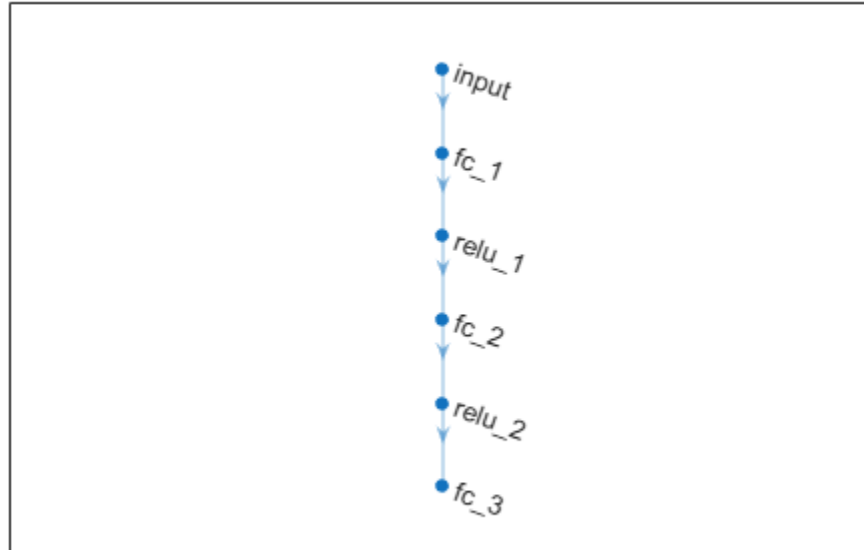
```
    Initialized: true
```

```
    Number of learnables: 1.5k
```

```

Inputs:
  1  'input'  6 features
plot(dnn)

```



Create the critic using `dnn` and the observation and action specifications. For more information, see `rlQValueFunction`.

```
critic = rlVectorQValueFunction(dnn,observationInfo,actionInfo);
```

Specify training options for the critic using `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);
```

Specify the DQN agent options using `rlDQNAgentOptions`, include the training options for the critic.

```
agentOptions = rlDQNAgentOptions( ...
    SampleTime=Ts, ...
    UseDoubleDQN=true, ...
    CriticOptimizerOptions=criticOptions, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=64);
agentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-4;
```

Then, create the DQN agent using the specified critic representation and agent options. For more information, see `rlDQNAgent`.

```
agent = rlDQNAgent(critic,agentOptions);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of `ceil(T/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option to `training-progress`) and disable the command line display (set the `Verbose` option to `false`).
- Stop the training when the episode reward reaches `-1`.
- Save a copy of the agent for each episode where the cumulative reward is greater than `-2.5`.

For more information on training options, see `rlTrainingOptions`.

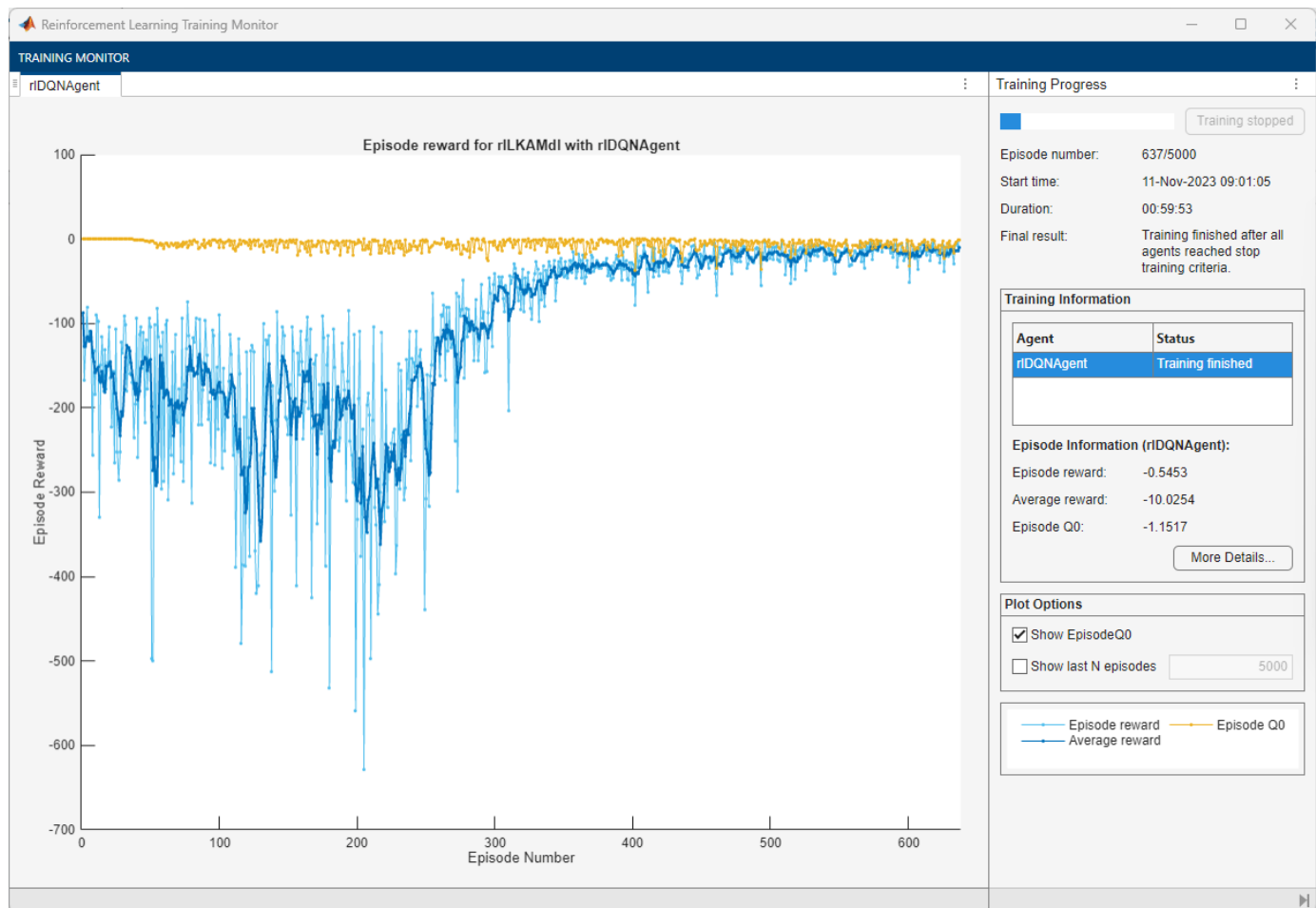
```
maxepisodes = 5000;
maxsteps = ceil(T/Ts);
trainingOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EpisodeReward", ...
    StopTrainingValue=-1, ...
    SaveAgentCriteria="EpisodeReward", ...
    SaveAgentValue=-2.5);
```

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainingOpts);
else
    % Load the pretrained agent for the example.
    load("SimulinkLKADQNMulti.mat","agent");
end
```



Simulate DQN Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

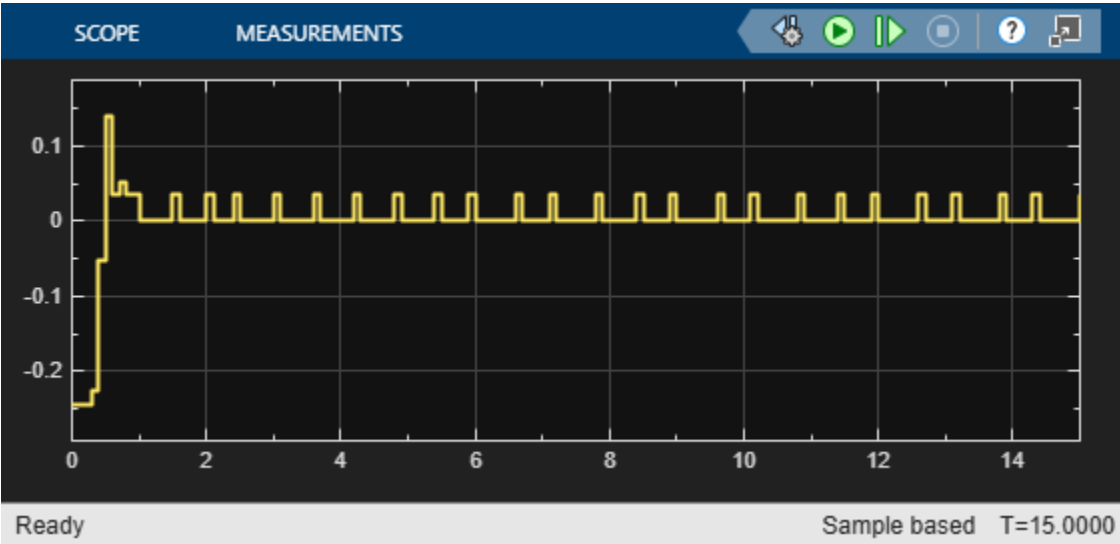
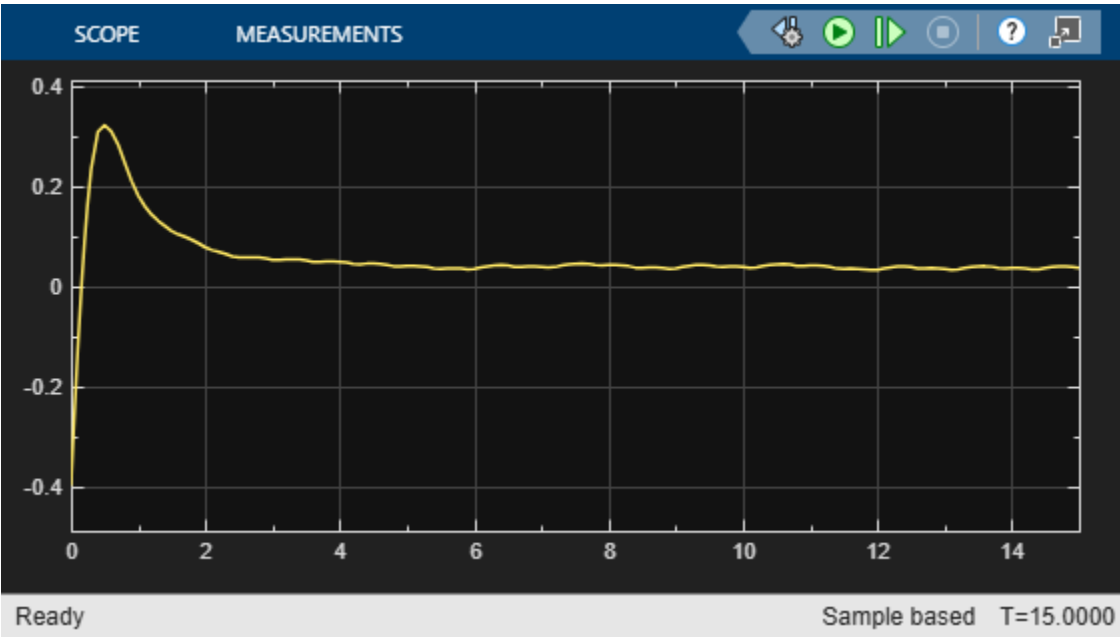
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

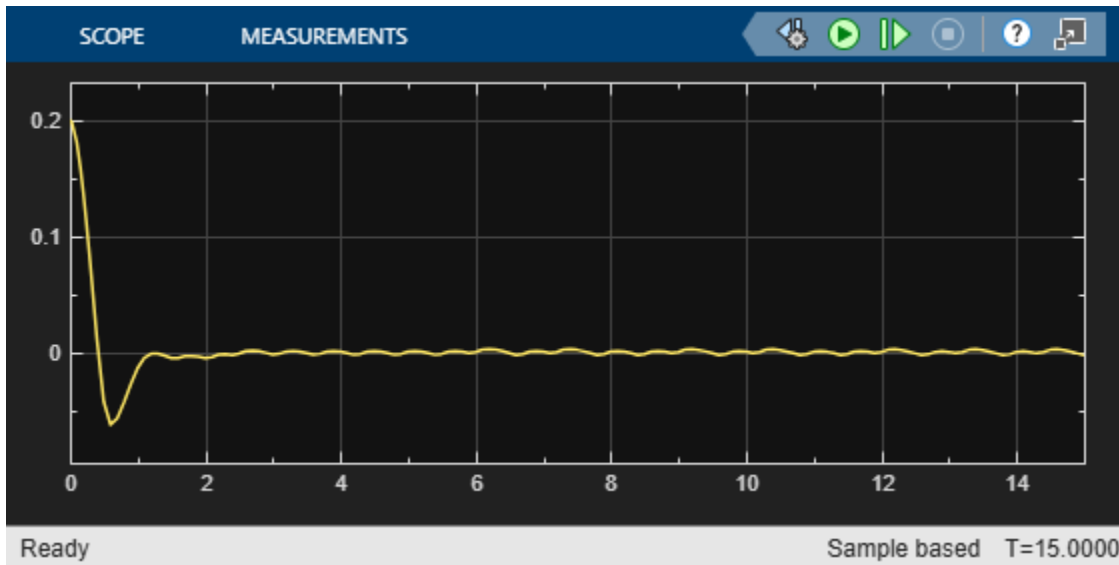
To validate the performance of the trained agent, uncomment the following two lines and simulate the agent within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
% simOptions = rlSimulationOptions(MaxSteps=maxsteps);
% experience = sim(env,agent,simOptions);
```

To demonstrate the trained agent on deterministic initial conditions, simulate the model in Simulink.

```
e1_initial = -0.4;
e2_initial = 0.2;
sim mdl
```





As the plots show, the lateral error (top plot) and relative yaw angle (bottom plot) are both driven close to zero. The vehicle starts from off the centerline (-0.4 m) and with a nonzero yaw angle error (0.2 rad). The lane keeping assist makes the ego car travel along the centerline after about 2.5 seconds. The steering angle (middle plot) shows that the controller reaches steady state after about 2 seconds.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (`Simulink`) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (`Simulink`) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
    % reset lateral deviation and relative yaw angle to random values
    in = setVariable(in,"e1_initial", 0.5*(-1+2*rand));
    in = setVariable(in,"e2_initial", 0.1*(-1+2*rand));
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlTrainingOptions` | `rlOptimizerOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134
- “Train PPO Agent with Curriculum Learning for a Lane Keeping Application” on page 5-164
- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3

Train PPO Agent for Automatic Parking Valet

This example demonstrates the design of a hybrid controller for an automatic search and parking task. The hybrid controller uses model predictive control (MPC) to follow a reference path in a parking lot and a trained reinforcement learning (RL) agent to perform the parking maneuver.

The automatic parking algorithm in this example executes a series of maneuvers while simultaneously sensing and avoiding obstacles in tight spaces. It switches between an adaptive MPC controller and an RL agent to complete the parking maneuver. The MPC controller moves the vehicle at a constant speed along a reference path while an algorithm searches for an empty parking spot. When a spot is found, the RL Agent takes over and executes a pretrained parking maneuver. Prior knowledge of the environment (the parking lot) including the locations of the empty spots and parked vehicles is available to the controllers.

Fix Random Seed Generator to Improve Reproducibility

The example code might involve computation of random numbers at several stages such as initialization of the agent, creation of the actor and critic, resetting the environment during simulations, generating observations for stochastic environments, generating exploration actions, and sampling mini-batches of experiences for learning. Fixing the random number stream preserves the sequence of the random numbers every time you run the code and improves reproducibility of results. You will fix the random number stream at several locations in the example.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

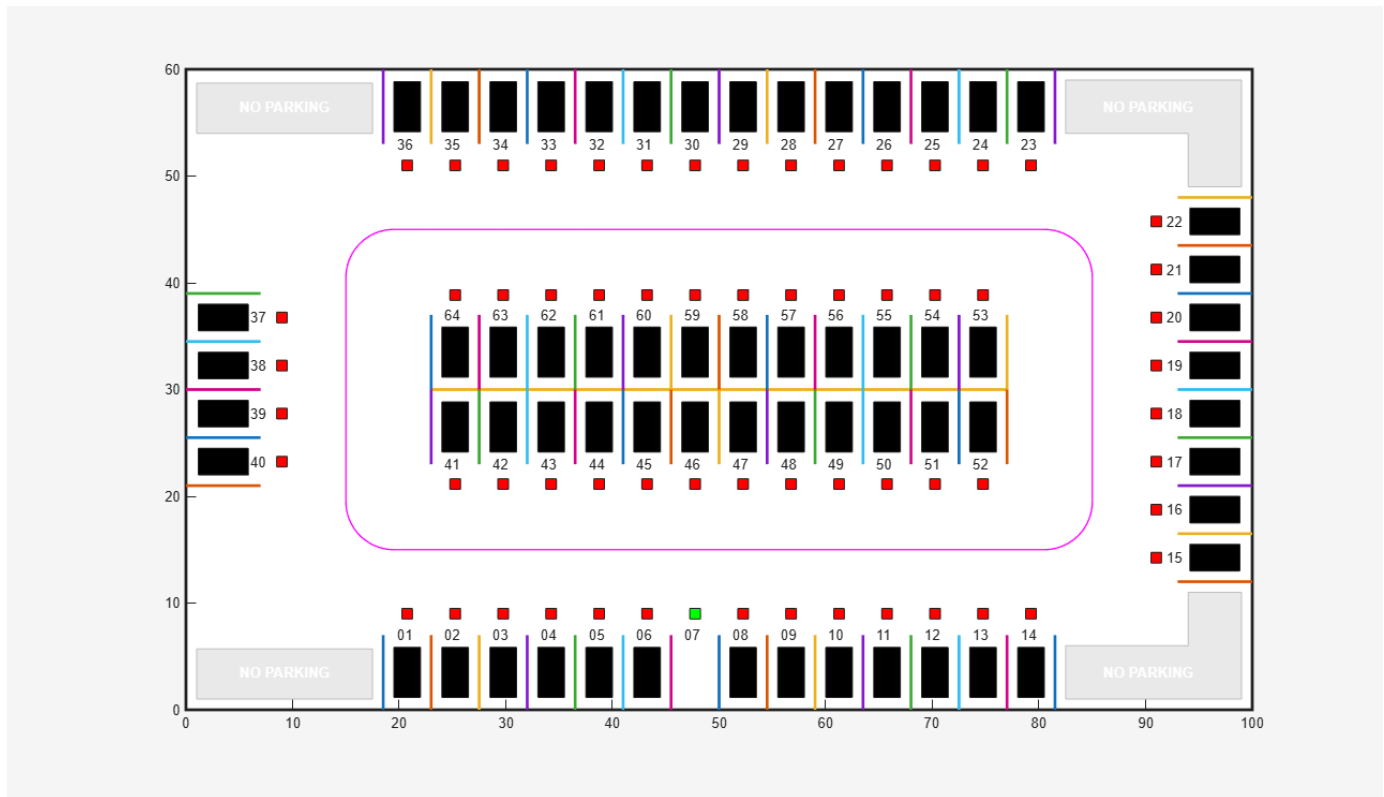
The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Parking Lot

The parking lot is represented by the `ParkingLot` class (provided in the example folder), which stores information about the ego vehicle, empty parking spots, and static obstacles (parked cars). Each parking spot has a unique index number and an indicator light that is either green (free) or red (occupied). Parked vehicles are represented in black.

Create a `ParkingLot` object with a free spot at location 7.

```
freeSpotIdx = 7;
map = ParkingLot(freeSpotIdx);
```



Specify an initial pose (X_0, Y_0, θ_0) for the ego vehicle. The target pose is determined based on the first available free spot as the vehicle navigates the parking lot.

```
egoInitialPose = [20, 15, 0];
```

Compute the target pose for the vehicle using the `createTargetPose` function. The target pose corresponds to the location in `freeSpotIdx`.

```
egoTargetPose = createTargetPose(map, freeSpotIdx)
```

```
egoTargetPose = 1x3
```

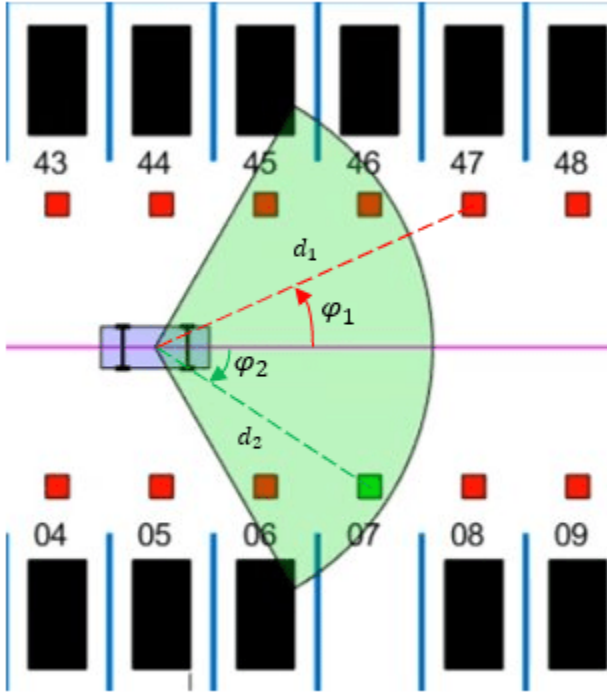
```
47.7500    4.9000   -1.5708
```

Sensor Modules

The parking algorithm uses camera and lidar sensors to gather information from the environment.

Camera

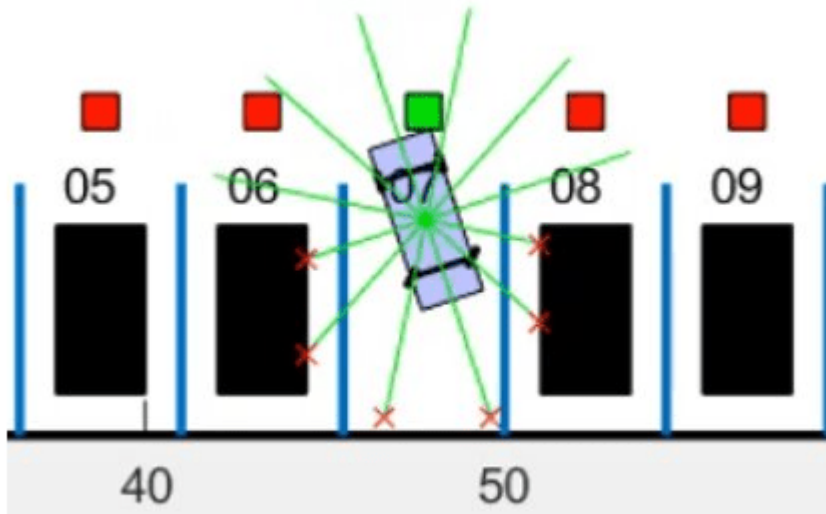
The field of view of a camera mounted on the ego vehicle is represented by the area shaded in green in the following figure. The camera has a field of view φ bounded by ± 60 degrees and a maximum measurement depth $d_{\max}$ of 10 m.



As the ego vehicle moves forward, the camera module senses the parking spots that fall within the field of view and determines whether a spot is free or occupied. For simplicity, this action is implemented using geometrical relationships between the spot locations and the current vehicle pose. A parking spot is within the camera range if $d_i \leq d_{\max}$ and $\varphi_{\min} \leq \varphi_i \leq \varphi_{\max}$, where d_i is the distance to the parking spot and φ_i is the angle to the parking spot.

Lidar

The reinforcement learning agent uses lidar sensor readings to determine the proximity of the ego vehicle to other vehicles in the environment. The lidar sensor in this example is also modeled using geometrical relationships. Lidar distances are measured along 12 line segments that radially emerge from the center of the ego vehicle. When a lidar line intersects an obstacle, it returns the distance of the obstacle from the vehicle. The maximum measurable lidar distance along any line segment is 6 m.



Auto Parking Valet Model

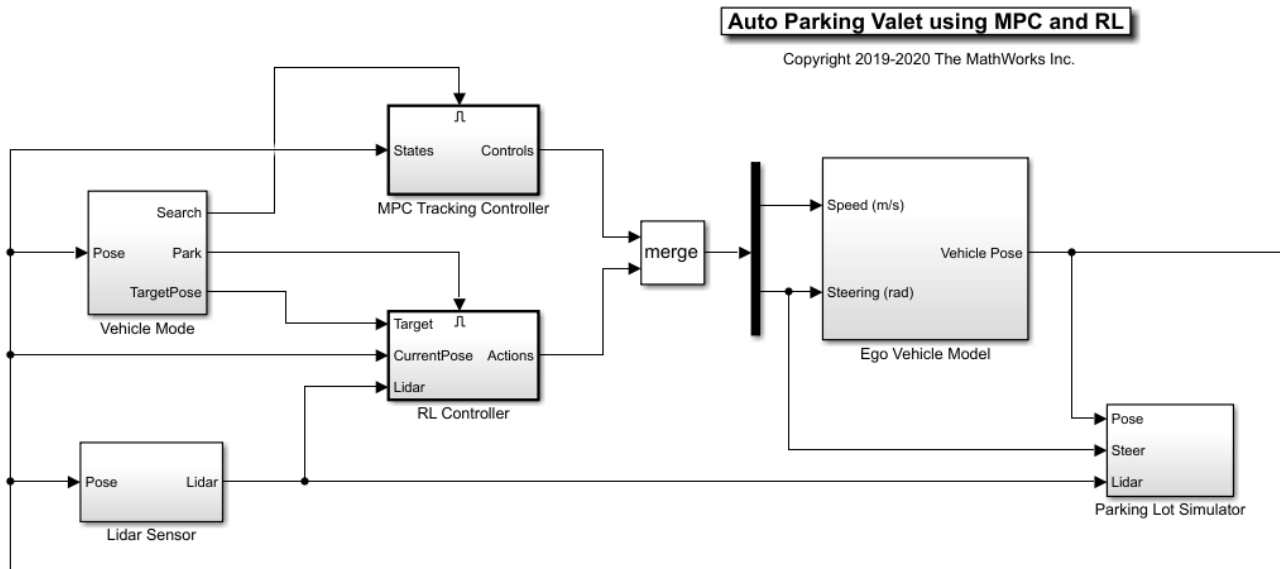
The parking valet model, including the controllers, ego vehicle, sensors, and parking lot, is implemented in Simulink®.

Load the auto parking valet parameters into the MATLAB® workspace.

```
autoParkingValetParams
```

Open the Simulink model.

```
mdl = "rlAutoParkingValet";  
open_system(mdl)
```



The ego vehicle dynamics in this model are represented by a single-track bicycle model with two inputs: vehicle speed v (m/s) and steering angle δ (radians). The MPC and RL controllers are placed within Enabled Subsystem blocks that are activated by signals representing whether the vehicle has to search for an empty spot or execute a parking maneuver. The enable signals are determined by the Camera algorithm within the Vehicle Mode subsystem. Initially, the vehicle is in *search* mode and the MPC controller tracks the reference path. When a free spot is found, *park* mode is activated and the RL agent executes the parking maneuver.

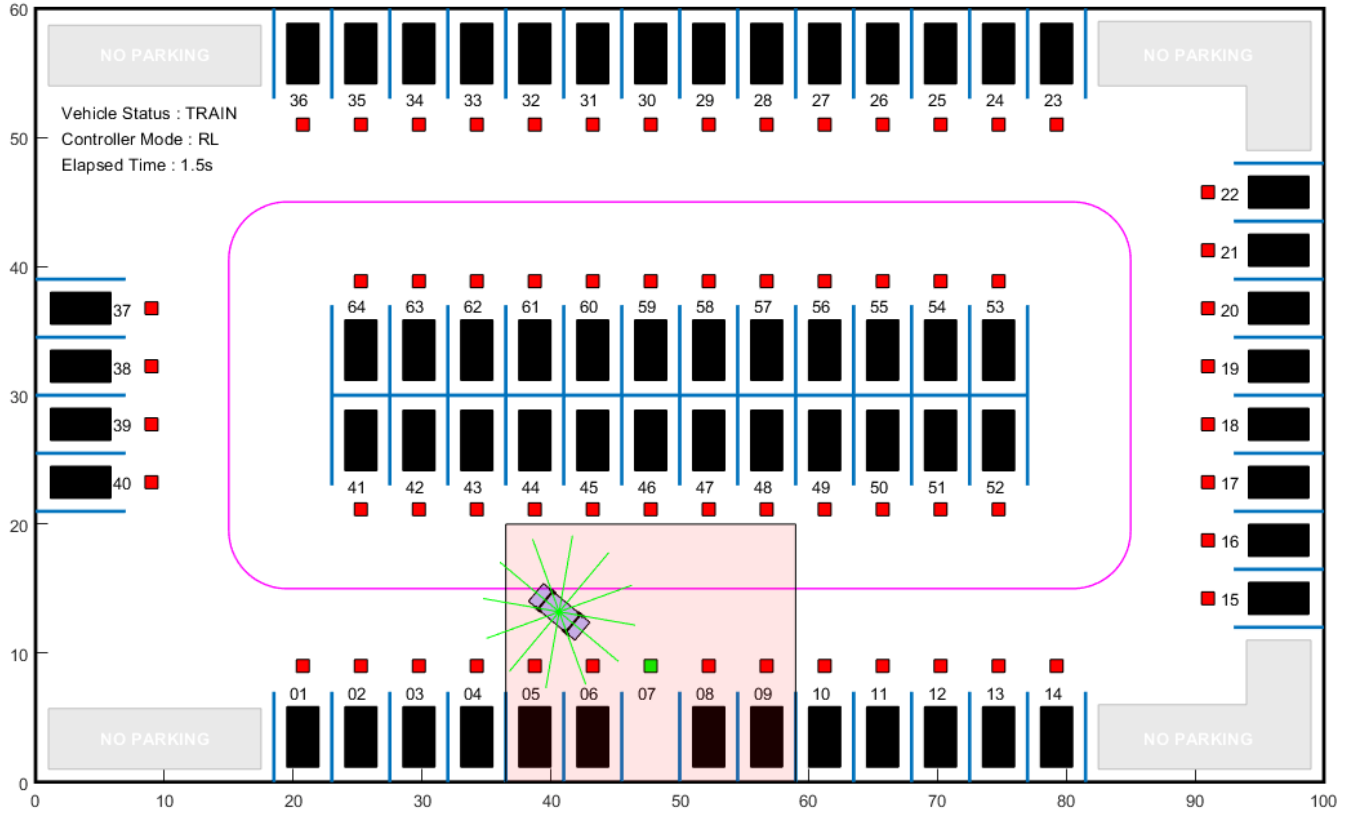
Adaptive Model Predictive Controller

Create the adaptive MPC controller object for reference trajectory tracking using the `createMPCForParking` script (provided in the example folder). For more information, see “Adaptive and Time-Varying MPC” (Model Predictive Control Toolbox).

```
createMPCForParking
```

Reinforcement Learning Environment

The environment for training the RL agent is the region shaded in red in the following figure. Due to symmetry in the parking lot, training within this region is sufficient for the policy to adjust to other regions after applying appropriate coordinate transformations to the observations (these transformations are performed in the Observation and Reward group inside the RL Controller block). Using this smaller training region significantly reduces training time compared to training over the entire parking lot.



For this environment:

- The training region is a 22.5 m x 20 m space with the target spot at its horizontal center.
- The observations are the position errors X_e and Y_e of the ego vehicle with respect to the target pose, the sine and cosine of the true heading angle θ , and the lidar sensor readings.
- The vehicle speed during parking is a constant 2 m/s.
- The action signals are discrete steering angles that range between ± 45 degrees in steps of 15 degrees.
- The vehicle is considered parked if the errors with respect to target pose are within specified tolerances of ± 0.75 m (position) and ± 10 degrees (orientation).
- The episode terminates if the ego vehicle goes out of the bounds of the training region, collides with an obstacle, or parks successfully.
- The reward r_t provided at time t , is:

$$r_t = 2e^{-(0.05X_e^2 + 0.04Y_e^2)} + 0.5e^{-40\theta_e^2} - 0.05\delta^2 + 100f_t - 50g_t$$

Here, X_e , Y_e , and θ_e are the position and heading angle errors of the ego vehicle from the target pose, and δ is the steering angle. f_t (0 or 1) indicates whether the vehicle has parked and g_t (0 or 1) indicates if the vehicle has collided with an obstacle at time t .

The coordinate transformations on vehicle pose (X, Y, θ) observations for different parking spot locations are as follows:

- 1-14: no transformation
- 15-22: $\bar{X} = Y, \bar{Y} = -X, \bar{\theta} = \theta - \pi/2$
- 23-36: $\bar{X} = 100 - X, \bar{Y} = 60 - Y, \bar{\theta} = \theta - \pi$
- 37-40: $\bar{X} = 60 - Y, \bar{Y} = X, \bar{\theta} = \theta - 3\pi/2$
- 41-52: $\bar{X} = 100 - X, \bar{Y} = 30 - Y, \bar{\theta} = \theta + \pi$
- 53-64: $\bar{X} = X, \bar{Y} = Y - 28, \bar{\theta} = \theta$

Create the observation and action specifications for the environment.

```
numObservations = 16;
obsInfo = rlNumericSpec([numObservations 1]);
obsInfo.Name = "observations";

steerMax = pi/4;
discreteSteerAngles = -steerMax : deg2rad(15) : steerMax;
actInfo = rlFiniteSetSpec(num2cell(discreteSteerAngles));
actInfo.Name = "actions";
numActions = numel(actInfo.Elements);
```

Create the Simulink environment object, specifying the path to the RL Agent block.

```
blk = mdl + "/RL Controller/RL Agent";
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo);
```

Specify a reset function for training. The `autoParkingValetResetFcn` function (provided at the end of this example) resets the initial pose of the ego vehicle to random values at the start of each episode.

```
env.ResetFcn = @autoParkingValetResetFcn;
```

For more information on creating Simulink environments, see `rlSimulinkEnv`.

Create PPO Agent

You will create and train a PPO agent in this example. The agent uses:

- A value function critic to estimate the value of the policy. The critic takes the current observation as input and returns a single scalar as output (the estimated discounted cumulative long-term reward for following the policy from the state corresponding to the current observation).
- A stochastic actor for computing actions. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

Create an agent initialization object to initialize the networks with the hidden layer size 128.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=128);
```

Specify hyperparameters for training using the `rlPPOAgentOptions` object:

- Specify an experience horizon of 230 steps. A large experience horizon can improve the stability of the training.
- Train with 40 epochs and mini-batches of length 100. Smaller mini-batches are computationally efficient but might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.

- Specify a learning rate of $4e-4$ for the actor and $3e-5$ for the critic. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Specify an objective function clip factor of 0.2 for improving training stability.
- Specify a discount factor value of 0.99 to promote long term rewards.
- Specify an entropy loss weight factor of 0.01 to enhance exploration during training.

```
% actor and critic optimizer options
actorOpts = rlOptimizerOptions(LearnRate=4e-4, ...
    GradientThreshold=1);
criticOpts = rlOptimizerOptions(LearnRate=3e-5, ...
    GradientThreshold=1);

% agent options
agentOpts = rlPPOAgentOptions( ...
    ExperienceHorizon      = 230, ...
    ActorOptimizerOptions = actorOpts, ...
    CriticOptimizerOptions = criticOpts, ...
    MiniBatchSize         = 100, ...
    NumEpoch              = 40, ...
    SampleTime             = Ts, ...
    DiscountFactor         = 0.99, ...
    ClipFactor             = 0.2, ...
    EntropyLossWeight      = 0.01);
```

Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Create the agent using the input specifications, initialization options, and the agent options objects.

```
agent = rlPPOAgent(obsInfo, actInfo, initOpts, agentOpts);
```

Train Agent

To train the PPO agent, specify the following training options.

- Run the training for a maximum of 10000 episodes, with each episode lasting a maximum of 200 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor window (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Specify an averaging window length of 200 for the episode reward.
- Evaluate the performance of the greedy policy every 20 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score reaches 100.

```
% training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=10000, ...
    MaxStepsPerEpisode=200, ...
    ScoreAveragingWindowLength=200, ...
    Plots="training-progress", ...
    Verbose=false, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=100);
```



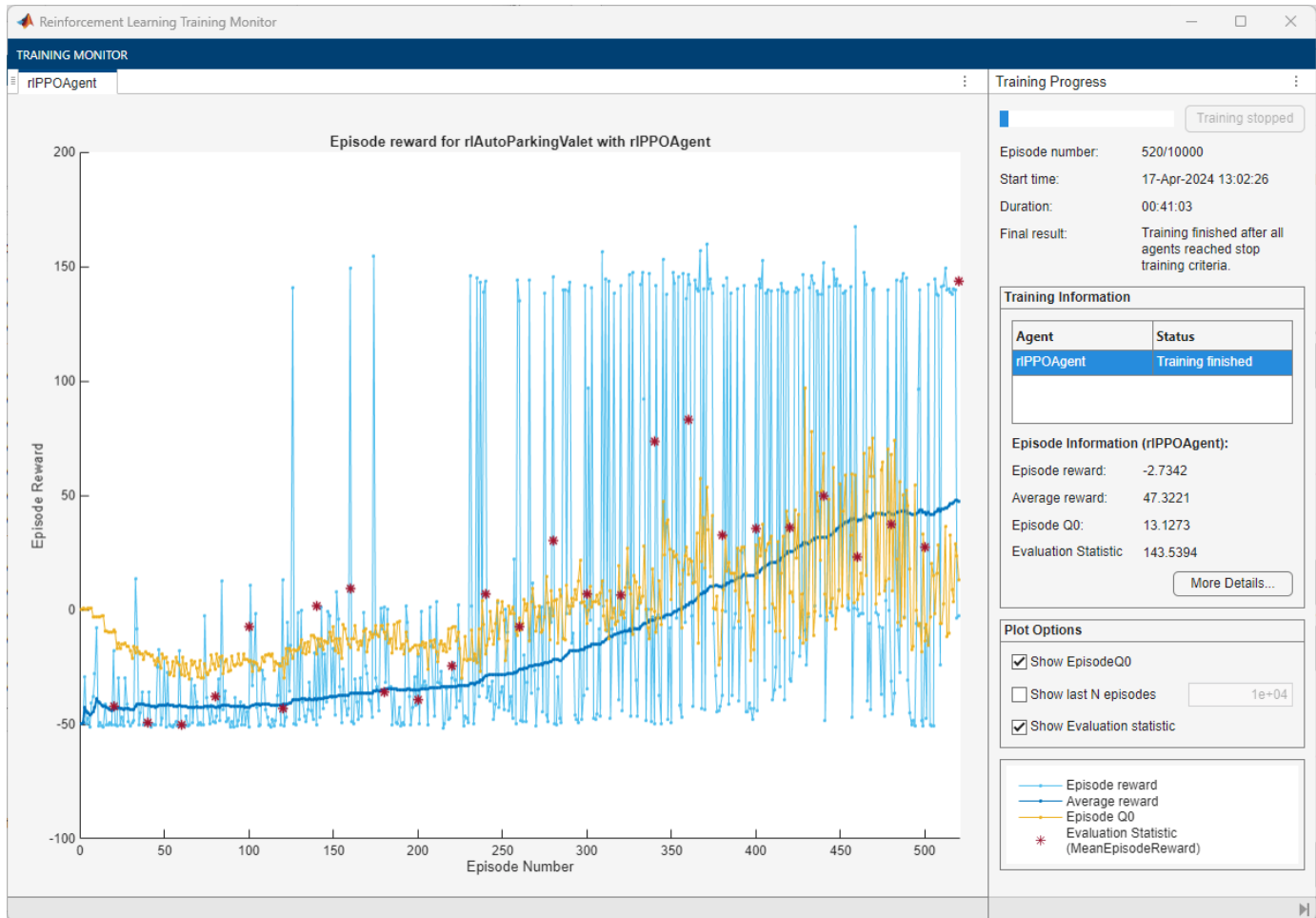
```
% agent evaluation
evl = rlEvaluator(EvaluationFrequency=20, NumEpisodes=5);
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    trainingStats = train(agent, env, trainOpts, Evaluator=evl);
else
    load('rlAutoParkingValetAgent.mat', 'agent');
end
```



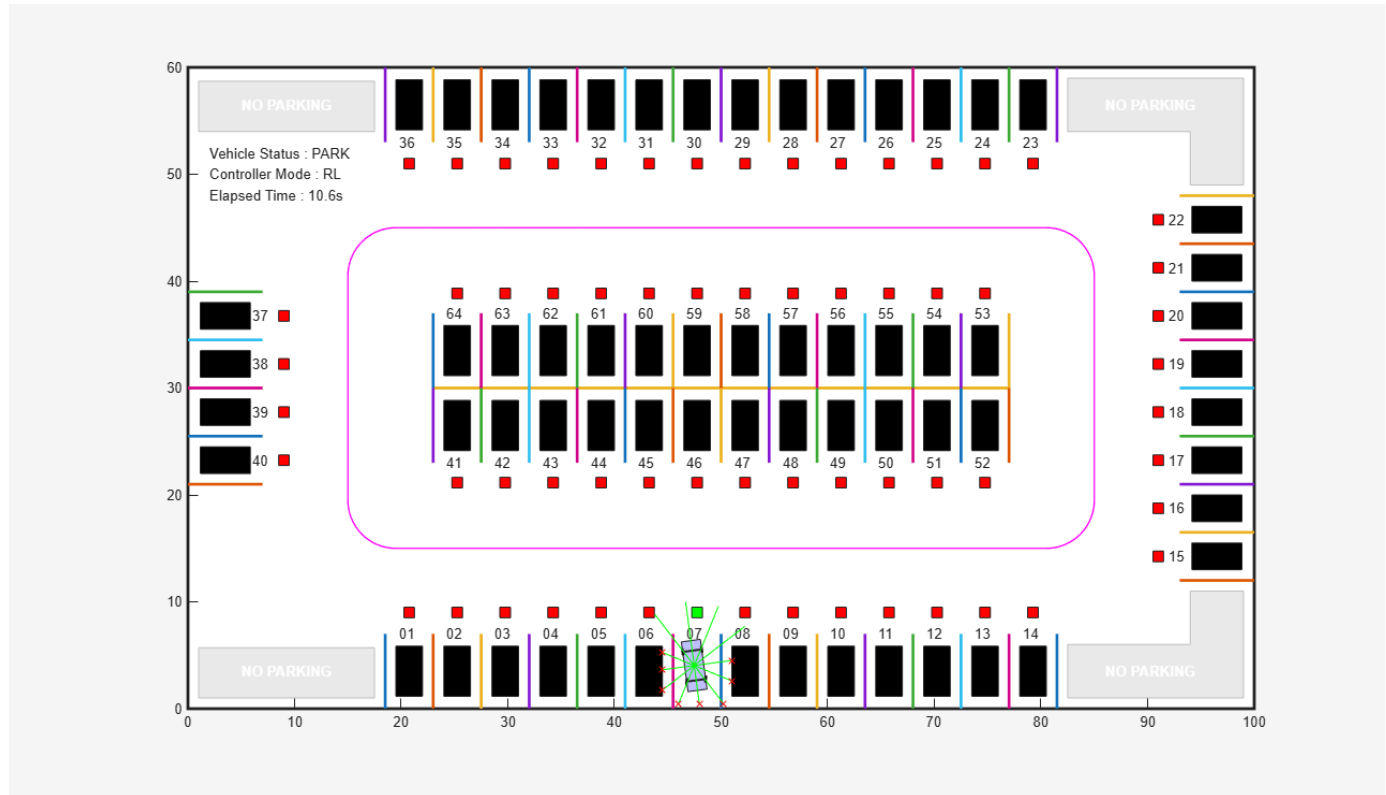
Simulate Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Simulate the model to park the vehicle in the parking spot specified by `freeSpotIdx` (1 to 64).

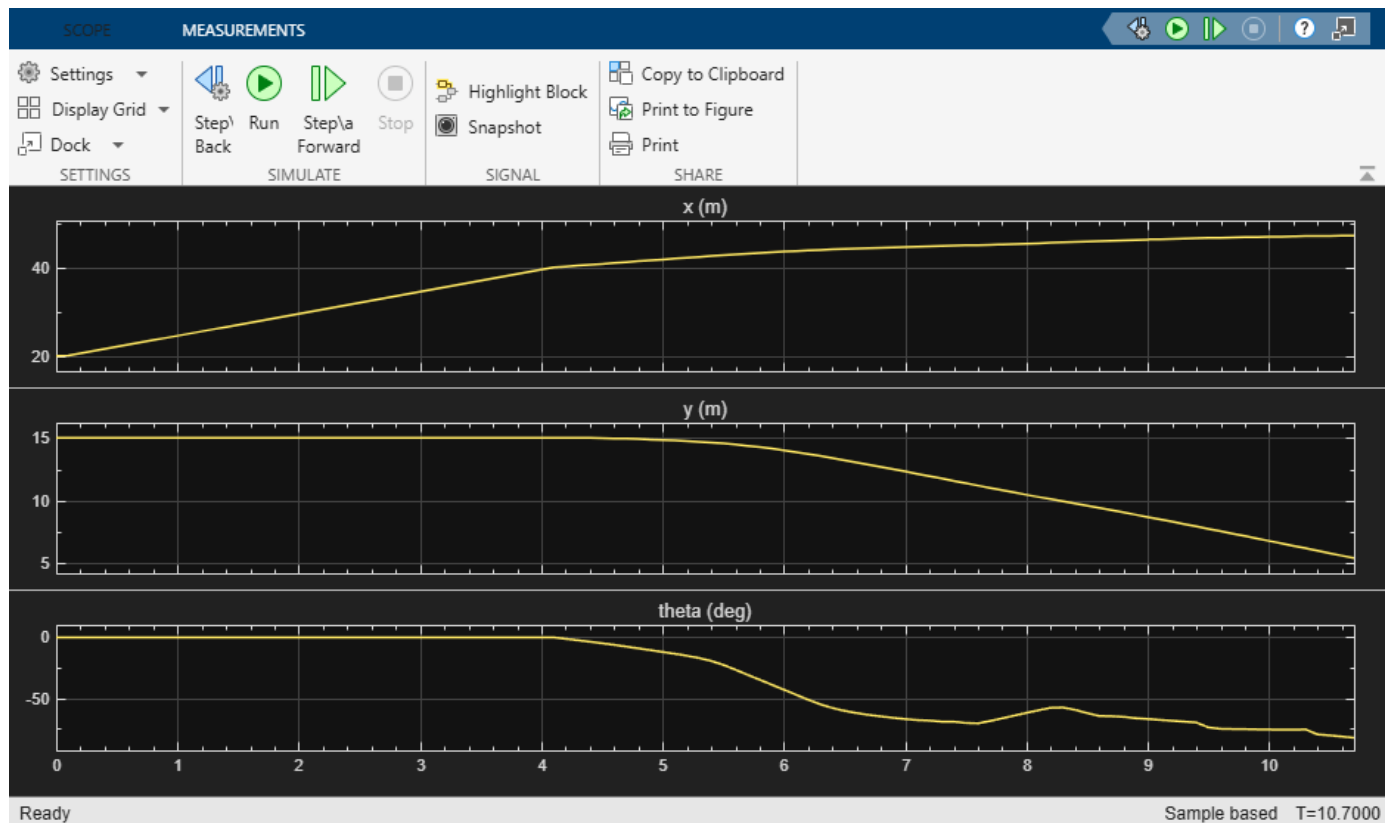
```
freeSpotIdx = ;  
sim mdl;
```



For the parking spot location 7, the vehicle reaches the target pose within the specified error tolerances of ± 0.75 m (position) and ± 10 degrees (orientation).

To view the ego vehicle position and orientation, open the Ego Vehicle Pose scope.

```
open_system(mdl + "/Ego Vehicle Model/Ego Vehicle Pose");
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Environment Reset Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `reset` function uses the `setVariable` (Simulink) function to specify random values for the initial vehicle pose and speed. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = autoParkingValetResetFcn(in)
% Reset function for auto parking valet example
choice = rand;
if choice <= 0.35
    x = 37;
    y = 16;
    t = deg2rad(-45 + 2*45*rand);
elseif choice <= 0.70
    x = 58;
    y = 16;
    t = deg2rad(-225 + 2*45*rand);
else
    zone = randperm(3,1);
    switch zone
```

```
case 1
    x = 36.5 + (45.5-36.5)*rand;
    t = deg2rad(-45 + 2*45*rand);
case 2
    x = 45.5 + (50-45.5)*rand;
    t = deg2rad(-135 + 2*45*rand);
case 3
    x = 50 + (59-50)*rand;
    t = deg2rad(-225 + 2*45*rand);
    if t <= -pi
        t = 2*pi + t;
    end
end
y = 10 + (20-10)*rand;
end
pose = [x,y,t];
speed = 5 * rand;
in = setVariable(in,'egoInitialPose',pose);
in = setVariable(in,'egoInitialSpeed',speed);
end
```

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlPPOAgent | rlPPOAgentOptions | rlOptimizerOptions | rlTrainingOptions |
rlSimulationOptions

Blocks

RL Agent

Related Examples

- “Automatic Parking Valet with Unreal Engine Simulation” on page 8-197
- “Parking Valet Using Multistage Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Parallel Parking Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Parallel Parking Using RRT Planner and MPC Tracking Controller” (Model Predictive Control Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Train Reinforcement Learning Agents” on page 5-3

Train DDPG Agent for Path-Following Control

This example shows how to train a deep deterministic policy gradient (DDPG) agent for path-following control (PFC) in Simulink®. For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

Fix Random Seed Generator to Improve Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Simulink Model

The reinforcement learning environment for this example consists in a simple bicycle model for the ego car together with a simple longitudinal model for the lead car. The training goal is to make the ego car travel at a set velocity while maintaining a safe distance from lead car by controlling longitudinal acceleration and braking, and also while keeping the ego car travelling along the centerline of its lane by controlling the front steering angle. For more information on PFC, see Path Following Control System (Model Predictive Control Toolbox). The ego car dynamics are specified by the following parameters.

```
m = 1600;    % total vehicle mass (kg)
Iz = 2875;   % yaw moment of inertia (mNs^2)
lf = 1.4;    % long. distance from center of gravity to front tires (m)
lr = 1.6;    % long. distance from center of gravity to rear tires (m)
Cf = 19000;  % cornering stiffness of front tires (N/rad)
Cr = 33000;  % cornering stiffness of rear tires (N/rad)
tau = 0.5;   % longitudinal time constant
```

Specify the initial position and velocity for the two vehicles.

```
x0_lead = 50;    % initial position for lead car (m)
v0_lead = 24;    % initial velocity for lead car (m/s)
x0_ego = 10;     % initial position for ego car (m)
v0_ego = 18;     % initial velocity for ego car (m/s)
```

Specify the standstill default spacing (m), time gap (s), and driver-set velocity (m/s).

```
D_default = 10;
t_gap = 1.4;
v_set = 28;
```

To simulate the physical limitations of the vehicle dynamics, constrain the acceleration to the range $[-3, 2]$ (m/s²), and the steering angle to the range $[-0.2618, 0.2618]$ (rad), that is, -15 and 15 degrees.

```

amin_ego = -3;
amax_ego = 2;
umin_ego = -0.2618; % +15 deg
umax_ego = 0.2618; % -15 deg

```

The curvature of the road is defined by a constant $0.001 \text{ (m}^{-1}\text{)}$. The initial value for lateral deviation is 0.2 m and the initial value for the relative yaw angle is -0.1 rad.

```

rho = 0.001;
e1_initial = 0.2;
e2_initial = -0.1;

```

Define the sample time T_s and simulation duration T_f in seconds.

```

Ts = 0.1;
Tf = 60;

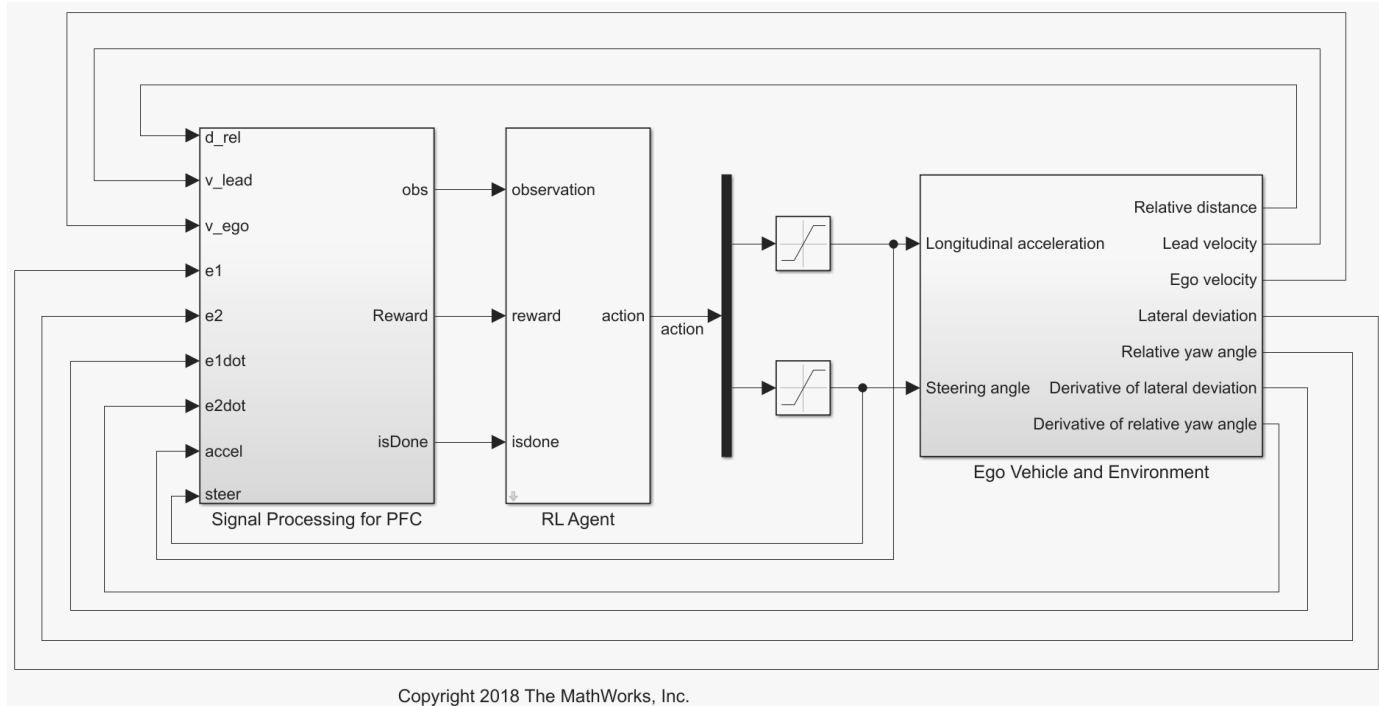
```

Open the model.

```

mdl = "rLPFCmdl";
open_system(mdl)
agentblk = mdl + "/RL Agent";

```



Copyright 2018 The MathWorks, Inc.

In this model:

- The action signal consists of acceleration and steering angle actions. The acceleration action signal takes value between -3 and 2 (m/s^2). The steering action signal takes a value between -15 degrees (-0.2618 rad) and 15 degrees (0.2618 rad).
- The reference velocity for the ego car V_{ref} is defined as follows. If the relative distance is less than the safe distance, the ego car tracks the minimum of the lead car velocity and driver-set velocity. In this manner, the ego car maintains some distance from the lead car. If the relative distance is

greater than the safe distance, the ego car tracks the driver-set velocity. In this example, the safe distance is defined as a linear function of the ego car longitudinal velocity V , that is, $t_{gap} * V + D_{default}$. The safe distance determines the tracking velocity for the ego car.

- The observations from the environment contain the longitudinal measurements: the velocity error $e_V = V_{ref} - V_{ego}$, its integral $\int e$, and the ego car longitudinal velocity V . In addition, the observations contain the lateral measurements: the lateral deviation e_1 , relative yaw angle e_2 , their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.
- The simulation terminates when the lateral deviation $|e_1| > 1$, when the longitudinal velocity $V_{ego} < 0.5$, or when the relative distance between the lead car and ego car $D_{rel} < 0$.
- The reward r_t , provided at every time step t , is

$$r_t = -(100e_1^2 + 500u_{t-1}^2 + 10e_V^2 + 100a_{t-1}^2) \times 1e^{-3} - 10F_t + 2H_t + M_t$$

where u_{t-1} is the steering input from the previous time step $t-1$, a_{t-1} is the acceleration input from the previous time step. The three logical values are as follows.

- $F_t = 1$ if simulation is terminated, otherwise $F_t = 0$
- $H_t = 1$ if lateral error $e_1^2 < 0.01$, otherwise $H_t = 0$
- $M_t = 1$ if velocity error $e_V^2 < 1$, otherwise $M_t = 0$

The three logical terms in the reward encourage the agent to make both lateral error and velocity error small, and in the meantime, penalize the agent if the simulation is terminated early.

Create Environment Object

Create an environment object for the Simulink model.

Create the observation specification.

```
obsInfo = rlNumericSpec([9 1], ...
    LowerLimit=-inf*ones(9,1), ...
    UpperLimit=inf*ones(9,1));
obsInfo.Name = "observations";
```

Create the action specification.

```
actInfo = rlNumericSpec([2 1], ...
    LowerLimit=[-3;-0.2618], ...
    UpperLimit=[2;0.2618]);
actInfo.Name = "accel;steer";
```

Create the environment object.

```
env = rlSimulinkEnv mdl, agentblk, obsInfo, actInfo;
```

To define the initial conditions, specify an environment reset function using an anonymous function handle. The reset function `localResetFcn`, which is defined at the end of the example, randomizes the initial position of the lead car, the lateral deviation, and the relative yaw angle.

```
env.ResetFcn = @localResetFcn;
```

Create DDPG Agent with Custom Networks

DDPG agents use a parameterized Q-value function approximator to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output (the estimated discounted cumulative long-term reward given the action from the state corresponding to the current observation, and following the policy thereafter).

To model the parameterized Q-value function within the critic, use a neural network with two input layers (one for the observation channel, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value). Note that `prod(obsInfo.Dimension)` and `prod(actInfo.Dimension)` return the number of dimensions of the observation and action spaces, respectively, regardless of whether they are arranged as row vectors, column vectors, or matrices.

When you create the networks, the initial parameters of the networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Define each network path as an array of layer objects, and assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
% Number of neurons
```

```
L = 100;
```

```
% Main path
```

```
mainPath = [  
    featureInputLayer(prod(obsInfo.Dimension), Name="obsInLyr")  
    fullyConnectedLayer(L)  
    reluLayer  
    fullyConnectedLayer(L)  
    additionLayer(2, Name="add")  
    reluLayer  
    fullyConnectedLayer(L)  
    reluLayer  
    fullyConnectedLayer(1, Name="QValLyr")  
];
```

```
% Action path
```

```
actionPath = [  
    featureInputLayer(prod(actInfo.Dimension), Name="actInLyr")  
    fullyConnectedLayer(L, Name="actOutLyr")  
];
```

Assemble `dlnetwork` object.

```
criticNet = dlnetwork();  
criticNet = addLayers(criticNet, mainPath);  
criticNet = addLayers(criticNet, actionPath);  
criticNet = connectLayers(criticNet, "actOutLyr", "add/in2");
```

Initialize `dlnetwork` object and display the number of weights.

```
criticNet = initialize(criticNet);  
summary(criticNet)
```



```
Initialized: true
```

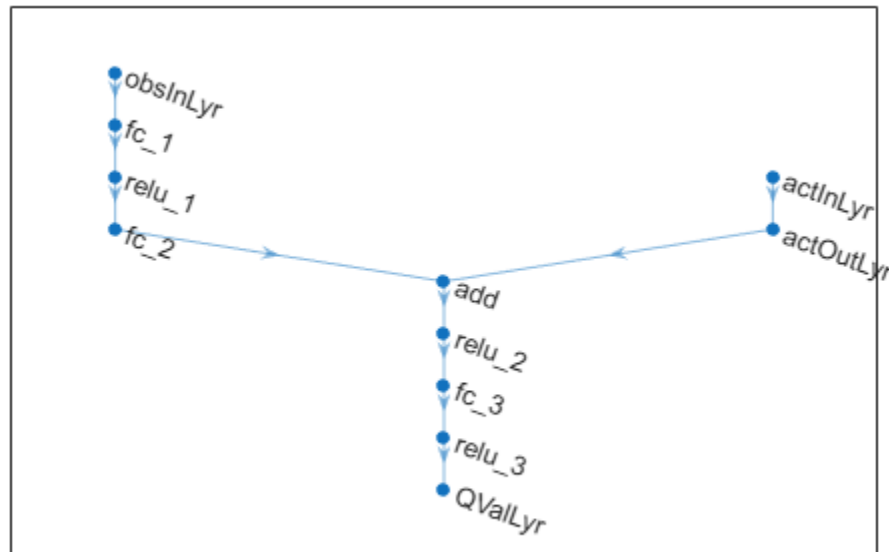
```
Number of learnables: 21.6k
```

```
Inputs:
```

```
  1 'obsInLyr'   9 features
  2 'actInLyr'   2 features
```

View the critic network configuration.

```
plot(criticNet)
```



Create the critic using the specified neural network and the environment action and observation specifications. Pass as additional arguments also the names of the network layers to be connected with the observation and action channels. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(criticNet,obsInfo,actInfo, ...
    ObservationInputNames="obsInLyr",ActionInputNames="actInLyr");
```

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`).

Define the network as an array of layer objects.

```
actorNet = [
    featureInputLayer(prod(obsInfo.Dimension))
    fullyConnectedLayer(L)
    reluLayer
```

```
fullyConnectedLayer(L)
reluLayer
fullyConnectedLayer(L)
reluLayer
fullyConnectedLayer(2)
tanhLayer
scalingLayer(Scale=[2.5;0.2618],Offset=[-0.5;0])
];
```

Convert to `dlnetwork` object, initialize the network, and display the number of weights.

```
actorNet = dlnetwork(actorNet);
actorNet = initialize(actorNet);
summary(actorNet)
```

```
Initialized: true
```

```
Number of learnables: 21.4k
```

```
Inputs:
  1  'input'   9 features
```

Construct the actor similarly to the critic. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNet,obsInfo,actInfo);
```

Specify training options for the critic and the actor using `rlOptimizerOptions`.

- Use learning rates `1e-4` and `1e-3` for the actor and critic respectively. A smaller learning rate can stabilize the training but at the cost of increased training time.
- Clip gradients using the gradient threshold value of `1`. This can prevent instability in learning due to large gradient values.
- Use the L2 regularization factor of `1e-4` to further stabilize the learning process.

```
criticOptions = rlOptimizerOptions( ...
    LearnRate=1e-3, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);
actorOptions = rlOptimizerOptions( ...
    LearnRate=1e-4, ...
    GradientThreshold=1, ...
    L2RegularizationFactor=1e-4);
```

Specify the DDPG agent options using `rlDDPGAgentOptions`.

- Specify the experience buffer length `1e6`. A larger buffer can store a diverse set of experiences.
- Specify the initial standard deviation of `[0.6;0.1]` for the two actions, and a decay rate of `1e-5` for the standard deviation. The decay facilitates exploration toward the beginning and exploitation toward the end of the training.
- Specify the same time `Ts` for the agent.

```
agentOptions = rlDDPGAgentOptions( ...
    SampleTime=Ts, ...
    ActorOptimizerOptions=actorOptions, ...
    CriticOptimizerOptions=criticOptions, ...
```

```

ExperienceBufferLength=1e6);
agentOptions.NoiseOptions.StandardDeviation = [0.6;0.1];
agentOptions.NoiseOptions.StandardDeviationDecayRate = 1e-5;

```

Create the DDPG agent using the actor, the critic, and the agent options. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic,agentOptions);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run each training episode for a maximum of 10000 episodes, with each episode lasting a maximum of `maxsteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Verbose` and `Plots` options).
- Evaluate the performance of the greedy policy every 50 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score 1600.

For more information on training options, see `rlTrainingOptions`.

```

maxepisodes = 1e4;
maxsteps = ceil(Tf/Ts);
trainingOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=1600);

% agent evaluator
evl = rlEvaluator(EvaluationFrequency=50, NumEpisodes=5);

```

Fix the random stream for reproducibility.

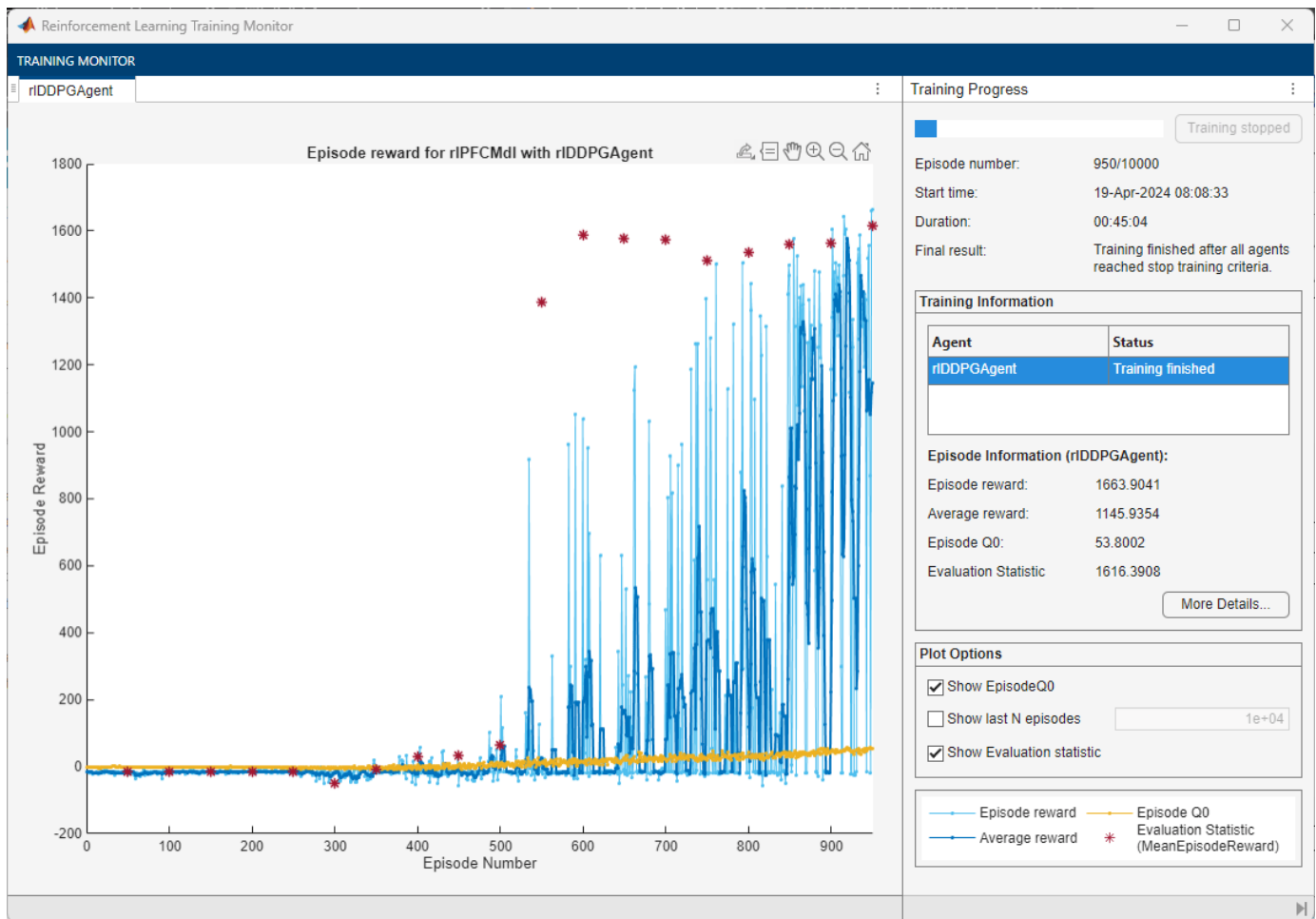
```
rng(0,"twister");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = ;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainingOpts,Evaluator=evl);
else
    % Load a pretrained agent for the example.
    load("SimulinkPFCDDPG.mat","agent")
end

```



Simulate DDPG Agent

Fix the random stream for reproducibility.

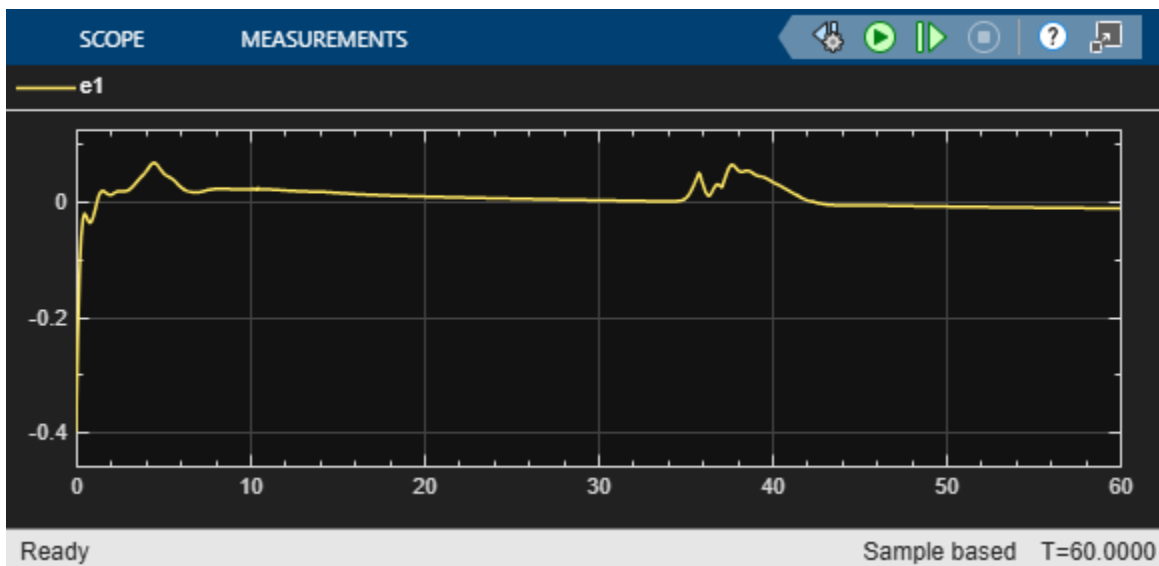
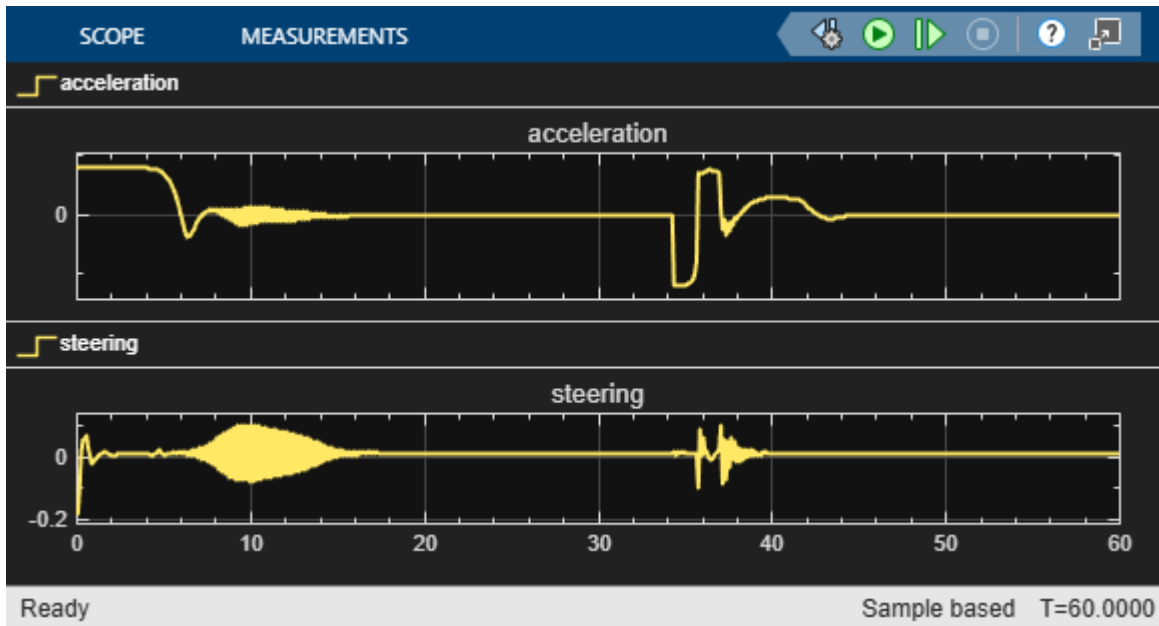
```
rng(0, "twister");
```

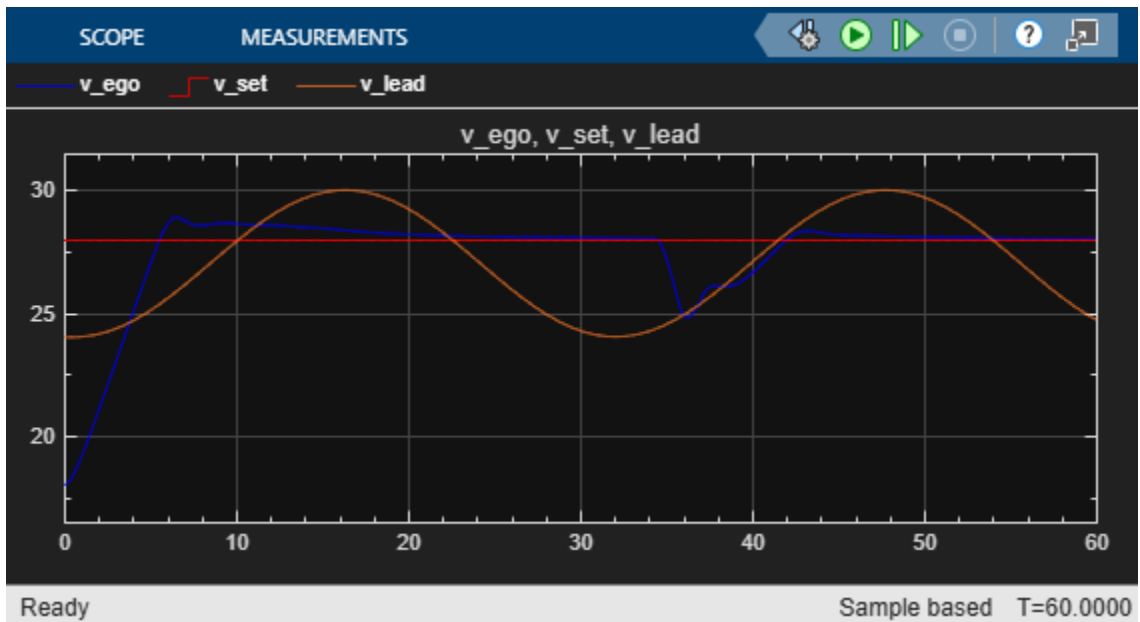
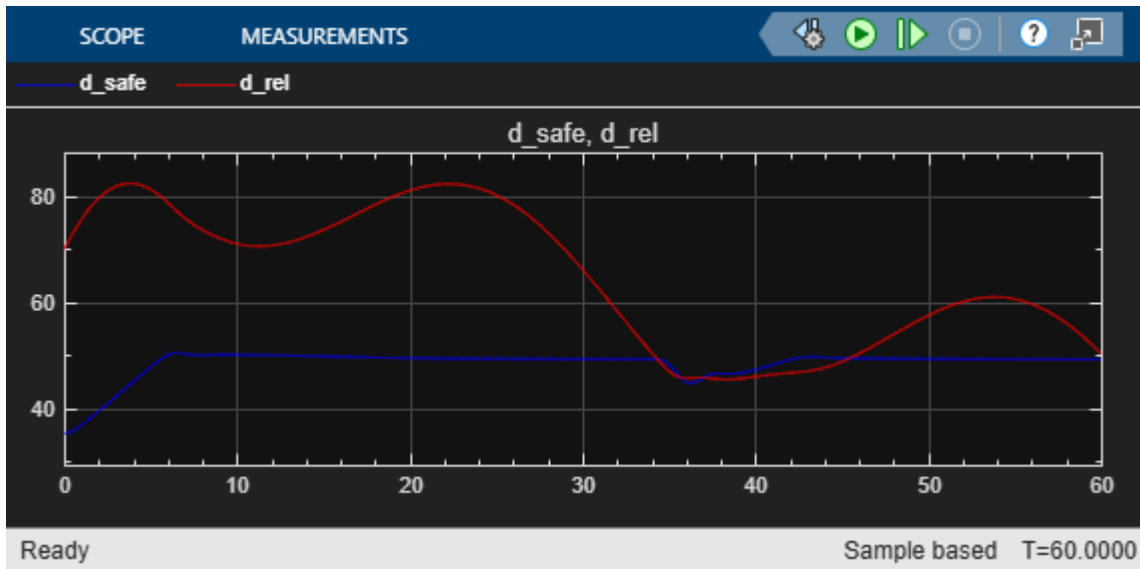
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

To demonstrate the trained agent using deterministic initial conditions, simulate the model in Simulink. To validate the performance of the trained agent with random initial conditions set userSpecifiedConditions to false.

```
userSpecifiedConditions = true;
if userSpecifiedConditions
    % The simulink sim function uses these workspace variables
    e1_initial = -0.4;
    e2_initial = 0.1;
    x0_lead = 80;
    sim mdl;
else
    % The reinforcement learning sim function randomizes the variables
```

```
simOptions = rlSimulationOptions(MaxSteps=maxsteps);  
experience = sim(env,agent,simOptions);  
end
```





For more information on agent simulation, see `rlSimulationOptions` and `sim`.

These plots show the simulation results when the lead car is 70 (m) ahead of the ego car.

- In the first 35 seconds, the relative distance is greater than the safe distance (bottom-right plot), so the ego car tracks the set velocity (top-right plot). To speed up and reach the set velocity, the acceleration is mostly nonnegative (top-left plot).
- From 35 to 42 seconds, the relative distance is mostly less than the safe distance (bottom-right plot), so the ego car tracks the minimum of the lead velocity and set velocity. Because the lead velocity is less than the set velocity (top-right plot), to track the lead velocity, the acceleration becomes nonzero (top-left plot).
- From 42 to 58 seconds, the ego car tracks the set velocity (top-right plot) and the acceleration remains zero (top-left plot).

- From 58 to 60 seconds, the relative distance becomes less than the safe distance (bottom-right plot), so the ego car slows down and tracks the lead velocity.
- The bottom-left plot shows the lateral deviation. As shown in the plot, the lateral deviation is greatly decreased within 1 second. The lateral deviation remains less than 0.05 m.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Environment Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (Simulink) function to randomly set the variables `x0_lead`, `e1_initial`, and `e2_initial` in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
% random value for initial position of lead car
in = setVariable(in,"x0_lead",40+randi(60,1,1));
% random value for lateral deviation
in = setVariable(in,"e1_initial", 0.5*(-1+2*rand));
% random value for relative yaw angle
in = setVariable(in,"e2_initial", 0.1*(-1+2*rand));
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions` | `rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Train Multiple Agents for Path Following Control” on page 8-57
- “Train Hybrid SAC Agent for Path-Following Control” on page 8-109
- “Lane Following Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Lane Following Control with Sensor Fusion and Lane Detection” (Automated Driving Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67

- “Train Reinforcement Learning Agents” on page 5-3

Train Hybrid SAC Agent for Path-Following Control

This example shows how to train a hybrid soft actor-critic (SAC) agent to perform path-following control (PFC) for a vehicle. A hybrid agent is an agent that has an action space that is partly discrete and partly continuous.

Overview

The goals of the PFC system are to make the ego vehicle travel at a set velocity, maintain a safe distance from a lead car by controlling longitudinal acceleration and braking, and travel along the centerline of its lane by controlling the front steering angle. An example that trains two RL agents (one with discrete action space, the other with continuous action space) to perform PFC is shown in “Train Multiple Agents for Path Following Control” on page 8-57. In that example, a DDPG agent provides continuous acceleration values for the longitudinal control loop while a DQN agent provides discrete steering angle values for the lateral control loop. For a PFC example that uses a single continuous action space for both longitudinal speed and lateral steering, see “Train DDPG Agent for Path-Following Control” on page 8-97. For more information on PFC systems, see Path Following Control System (Model Predictive Control Toolbox).

In this example, a single hybrid SAC agent is trained to control both the lateral steering (discrete action) the longitudinal speed (continuous action) of the ego vehicle. For more information about SAC agents, see “Soft Actor-Critic (SAC) Agent” on page 3-78.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

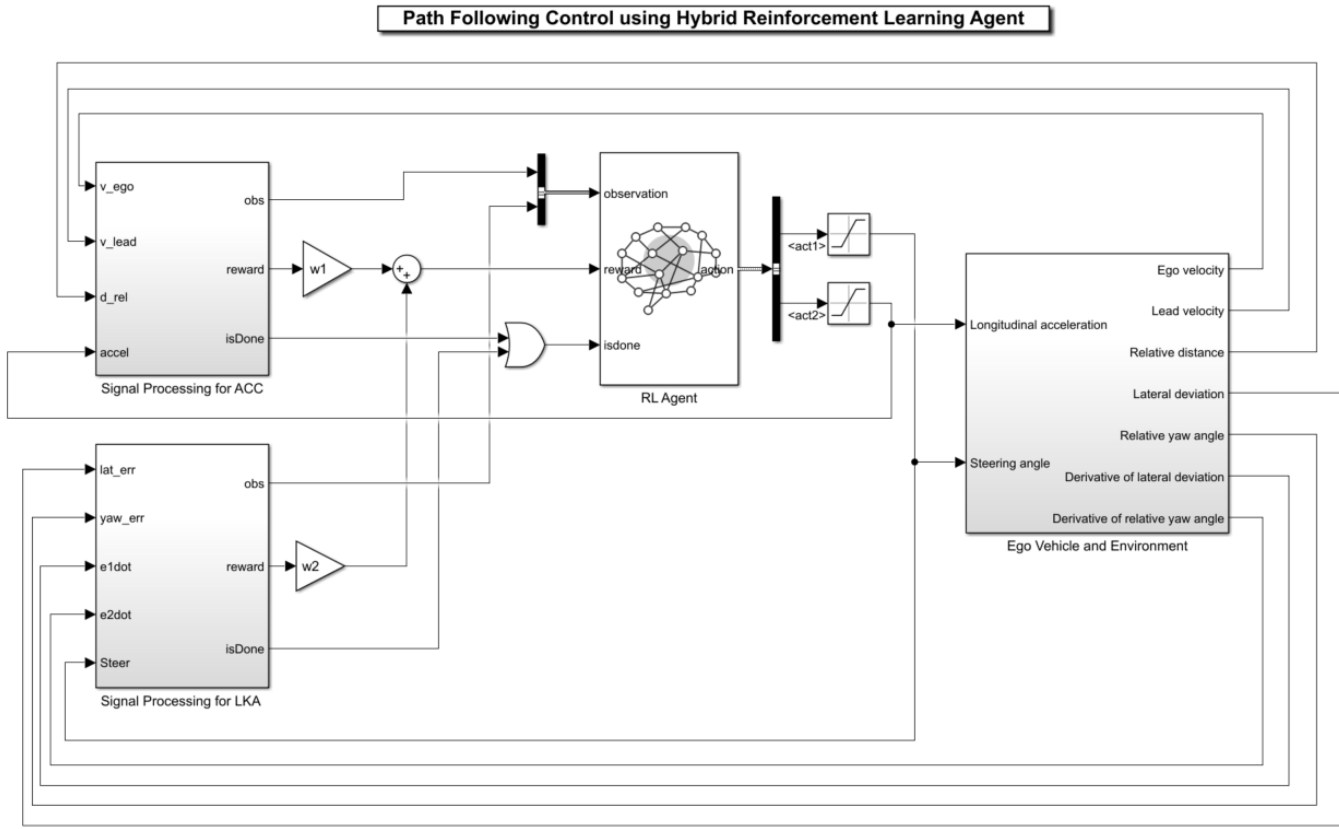
The environment for this example includes a simple bicycle model for the ego car and a simple longitudinal model for the lead car. The agent controls the ego car's longitudinal acceleration, braking, and the front steering angle.

Load the environment parameters.

```
hybridAgentPFParams
```

Open the Simulink® model.

```
mdl = "rlHybridAgentPFC";  
open_system(mdl)
```



Copyright 2024 The MathWorks, Inc.

The simulation terminates if any of the following conditions occur.

- $|e_1| > 1$ — magnitude of the lateral deviation is greater than 1.
- $V_{ego} < 0.5$ — longitudinal velocity of the ego car is less than 0.5.
- $D_{rel} < 0$ — distance between the ego and the lead car is less than zero.

To determine the ego car's reference velocity V_{ref} :

- The safe distance is a linear function of the ego car's longitudinal velocity V , that is, $t_{gap} * V + D_{default}$.
- If the relative distance is less than the safe distance, the ego car tracks the minimum value between the velocity of the lead car and the desired velocity set by the driver. Setting the reference velocity in this way allows the ego car to maintain a safe distance from the lead car. If the relative distance is greater than the safe distance, the ego car uses the desired velocity set by the driver as the reference velocity.

Observation:

- The first observation channel contains the longitudinal measurements — the velocity error $e_V = V_{ref} - V$, its integral $\int e$, and the ego car longitudinal velocity V .

- The second observation channel contains the lateral measurements — the lateral deviation e_1 , relative yaw angle e_2 (the yaw angle error with respect to the lane centerline), their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.

Action:

- The discrete action a_t — The action signal consists of discrete steering angle actions which take values from -15 degrees (-0.2618 rad) to 15 degrees (0.2618 rad) in steps of 1 degree (0.0175 rad).
- The continuous action u_t — The action signal consists of continuous acceleration values between -3 and 2 m/s^2 .

Reward:

- The reward r_t , provided at every time step t , is the weighted sum of the reward $r_{lateral}$ for the lateral control and the reward $r_{longitudinal}$ for the longitudinal control.

$$r_t = w_1 r_{lateral} + w_2 r_{longitudinal}$$

$$r_{lateral} = -(100e_1^2 + 500u_{t-1}^2) \times 0.001 - 10F_t + 2H_t$$

$$r_{longitudinal} = -(10e_V^2 + 100a_{t-1}^2) \times 0.001 - 10F_t + M_t$$

$$w_1 = 1/12$$

$$w_2 = 1/5$$

In these equations, u_{t-1} is the steering input from the previous time step, a_{t-1} is the acceleration input from the previous time step, and:

- $F_t = 1$ if the simulation terminates, otherwise $F_t = 0$.
- $M_t = 1$ if $e_V^2 < 1$, otherwise $M_t = 0$.
- $H_t = 1$ if $e_1^2 < 0.01$, otherwise $H_t = 0$.

The logical terms in the reward functions (F_t , M_t , and H_t) penalize the agent if the simulation terminates early, while encouraging the agent to make both the lateral error and velocity error small.

Create the observation specification. Because an observation contains multiple channels in this Simulink environment, you must use the `bus2RLSpec` function to create the specification.

Create a bus object.

```
obsBus = Simulink.Bus();
```

Add the first bus element.

```
obsBus.Elements(1) = Simulink.BusElement;
obsBus.Elements(1).Name = "signal1";
obsBus.Elements(1).Dimensions = [3,1];
```

Add the second bus element.

```
obsBus.Elements(2) = Simulink.BusElement;  
obsBus.Elements(2).Name = "signal2";  
obsBus.Elements(2).Dimensions = [6,1];
```

Create the observation specification.

```
obsInfo= bus2RLSpec("obsBus");
```

Create the action specification. For the hybrid SAC agent, you must have two action channels. The first action channel must be for the discrete part of the action, and the second must be for the continuous part of the action. Use the `bus2RLSpec` function to create the specification, as for the observation specification case.

Create a bus object.

```
actBus = Simulink.Bus();
```

Add the first bus element for the discrete action.

```
actBus.Elements(1) = Simulink.BusElement;  
actBus.Elements(1).Name = "act1";
```

Add the second bus element for the continuous action.

```
actBus.Elements(2) = Simulink.BusElement;  
actBus.Elements(2).Name = "act2";  
actBus.Elements(2).Dimensions = [1,1];  
actInfo = bus2RLSpec("actBus","DiscreteElements", ...  
    {"act1",(-15:15)*pi/180});
```

Define the limits of continuous actions.

```
actInfo(2).LowerLimit = -3;  
actInfo(2).UpperLimit = 2;
```

Create a Simulink environment object, specifying the block paths for the agent block. For more information, see `rlSimulinkEnv`.

```
blks = mdl + "/RL Agent";  
env = rlSimulinkEnv(mdl,blks,obsInfo,actInfo);
```

Specify a reset function for the environment using the `ResetFcn` property. The function `pfcResetFcn` on page 8-118 (provided at the end of the example) sets the initial conditions of the lead and ego vehicles at the beginning of every episode during training.

```
env.ResetFcn = @pfcResetFcn;
```

Create Hybrid SAC Agent

Fix the random number stream.

```
rng(0, "twister");
```

Set the sample time, in seconds, for the Simulink model and the RL agent object.

```
Ts = 0.1;
```

Create a default hybrid SAC agent. When the action specification defines a hybrid action space (that is, contains both a discrete and a continuous action channel), `rlSACAgent` creates a hybrid SAC agent.

```
agent = rlSACAgent(obsInfo, actInfo);
```

Specify the agent options:

- Set the mini-batch size to 128 to make the training more stable (at the expense of larger memory requirement).
- Use a maximum of 200 mini-batches to update the agent at the end of each episode. The default `LearningFrequency` is -1, meaning that the agent updates at the end of each episode.
- Set learning rate to $1e-3$ for the actor and the critics.
- Set gradient thresholds to 1 to limit the gradient values.
- Use an experience buffer that can store 1 million experiences. A large experience buffer can contain diverse experiences.
- Set the number of steps for the Q-value estimate (`NumStepsToLookAhead`) to 3 to accurately estimate the Q-value.
- Set the initial entropy weights for discrete actions and continuous actions to 0.1 to achieve a better balance between exploitation and exploration early in training.

```
agent.SampleTime = Ts;
agent.AgentOptions.MinibatchSize = 128;
agent.AgentOptions.MaxMiniBatchPerEpoch = 200;
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
agent.AgentOptions.CriticOptimizerOptions(1).LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions(1).GradientThreshold = 1;
agent.AgentOptions.CriticOptimizerOptions(2).LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions(2).GradientThreshold = 1;
agent.AgentOptions.ExperienceBufferLength = 1e6;
agent.AgentOptions.NumStepsToLookAhead = 3;
```

Set the initial entropy weight for the discrete and continuous actions.

```
agent.AgentOptions.EntropyWeightOptions(1).EntropyWeight = 0.1;
agent.AgentOptions.EntropyWeightOptions(2).EntropyWeight = 0.1;
```

Train Hybrid SAC Agent

Specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 5000 episodes, with each episode lasting a maximum of `maxsteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box.
- Stop the training when the agent receives an average evaluation episode reward greater than 196.
- To improve performance, do not save the simulation data during training. To save the simulation data during training instead, set `SimulationStorageType` to "file" or "memory".

```
Tf = 60; % Simulation time
maxepisodes = 5000;
maxsteps = ceil(Tf/Ts);
```

```
trainingOpts = rlTrainingOptions( ...  
    MaxEpisodes=maxepisodes, ...  
    MaxStepsPerEpisode=maxsteps, ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue= 196, ...  
    SimulationStorageType="none");
```

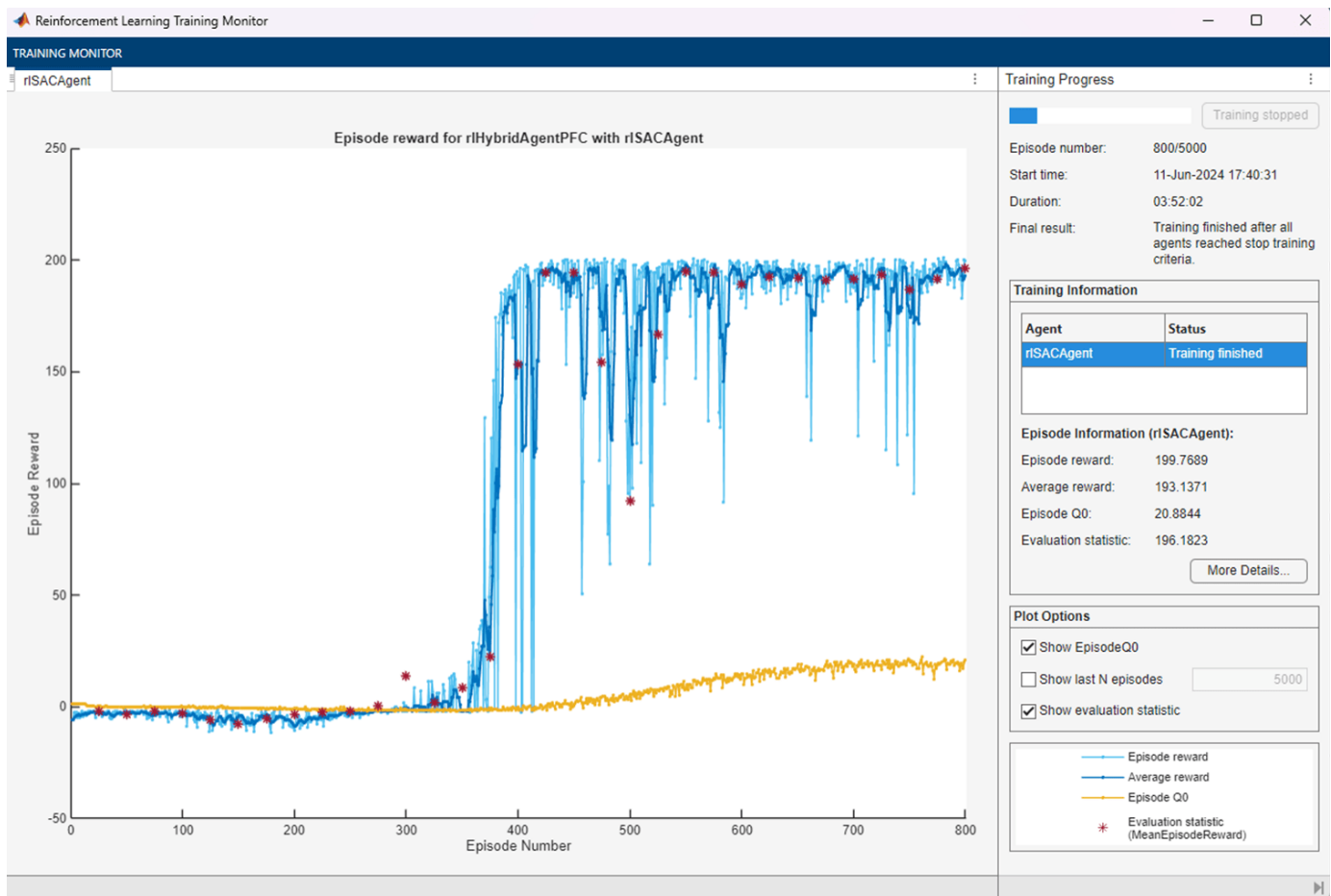
Fix the random number stream.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training the agent is a computationally intensive process that takes a couple of hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Evaluate the agent after every 25 training episodes.  
    % The evaluation statistic is the mean value of  
    % the statistic over five evaluation episodes.  
    evaluator = rlEvaluator(EvaluationFrequency=25, ...  
        NumEpisodes=5, RandomSeeds=[101:105]);  
  
    % Train the agent.  
    trainingStats = train(agent,env,trainingOpts,Evaluator=evaluator);  
else  
    % Load the pretrained agent for the example.  
    load("rlHybridPFCagent.mat")  
end
```

The figure shows a snapshot of the training progress.



Simulate the Trained SAC Agent

Fix the random number stream.

```
rng(0, "twister");
```

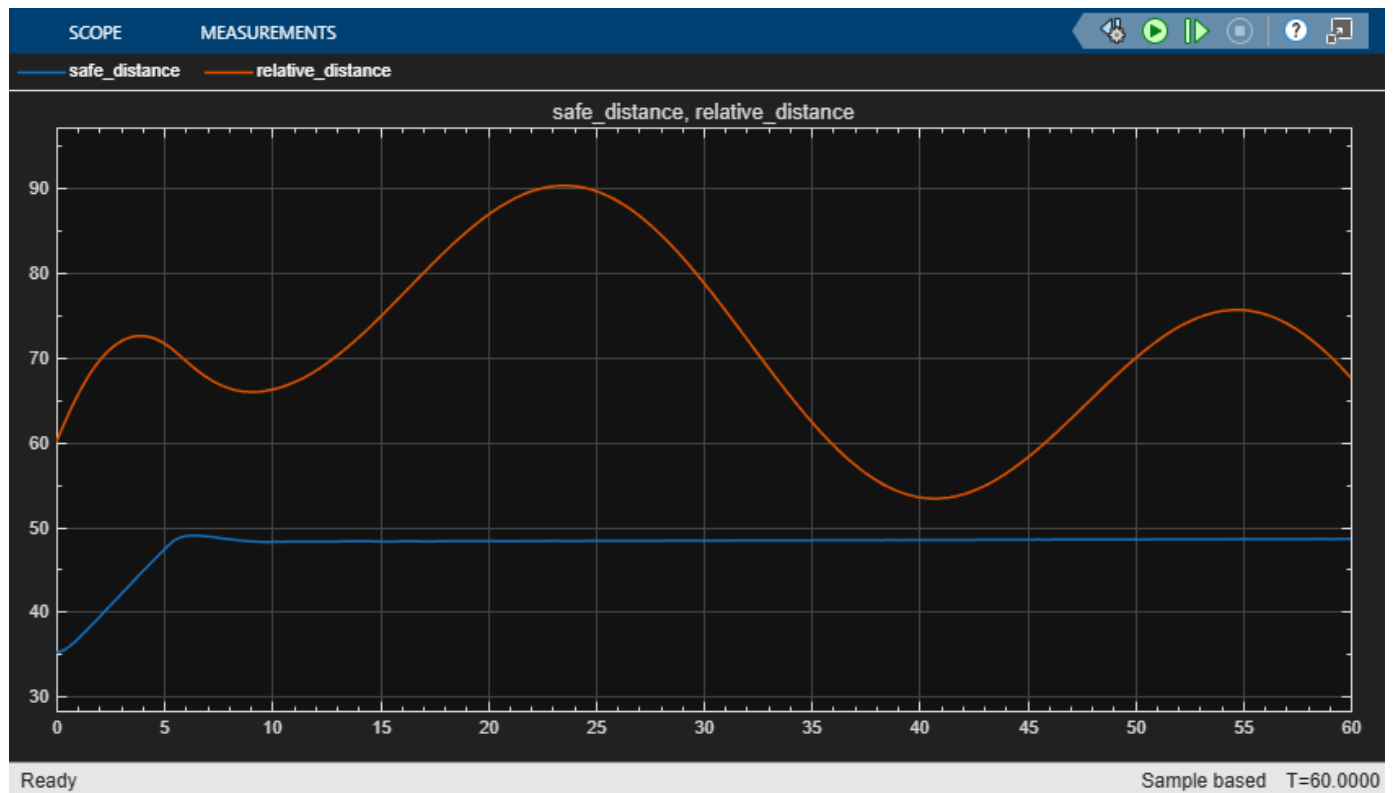
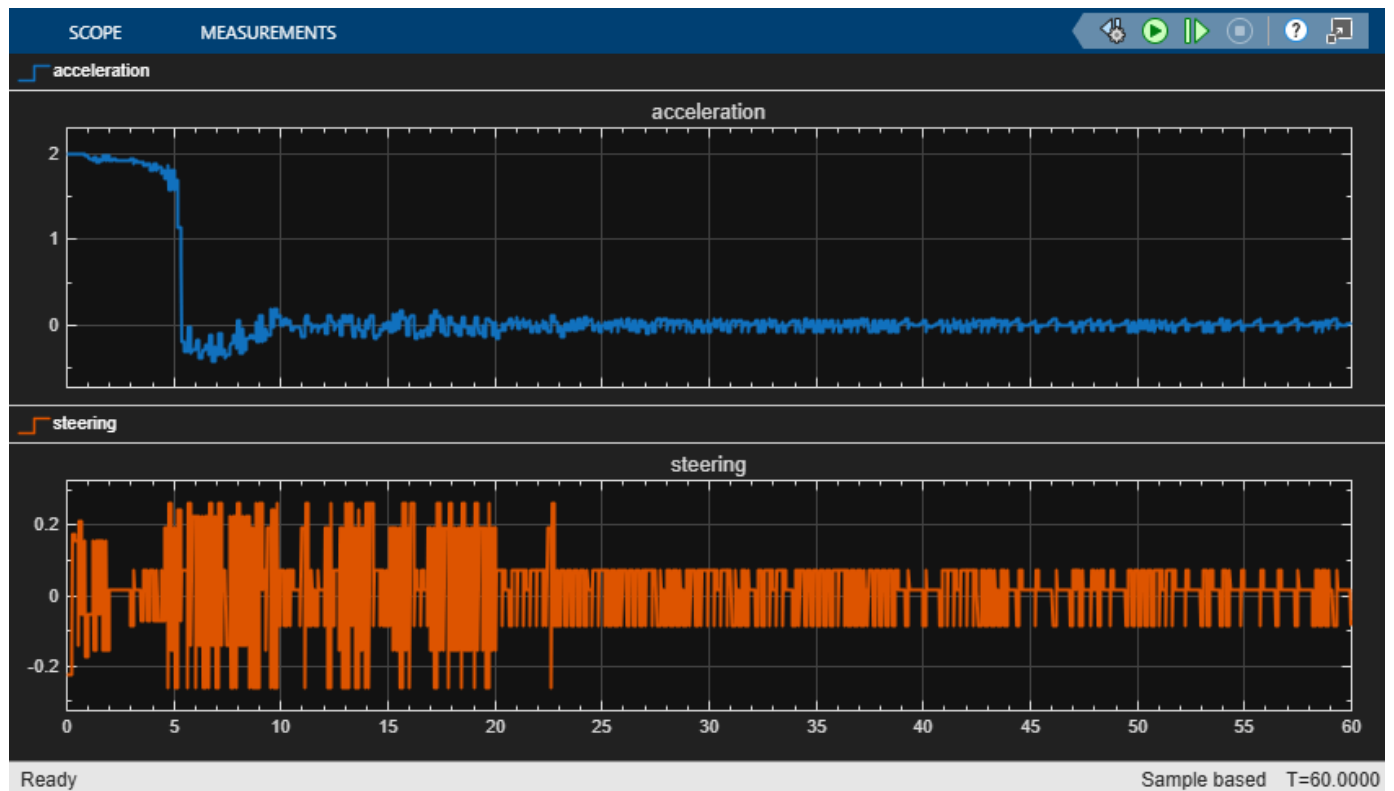
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

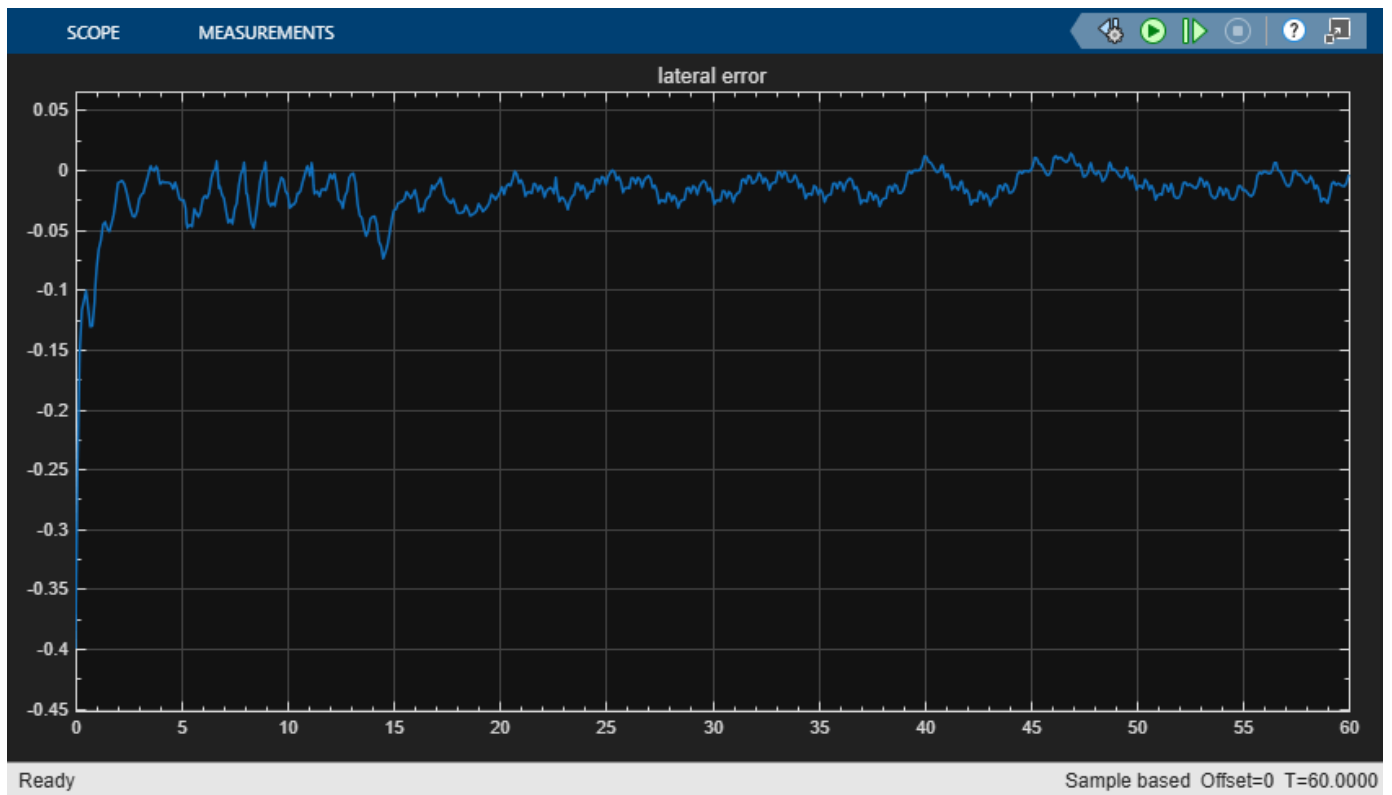
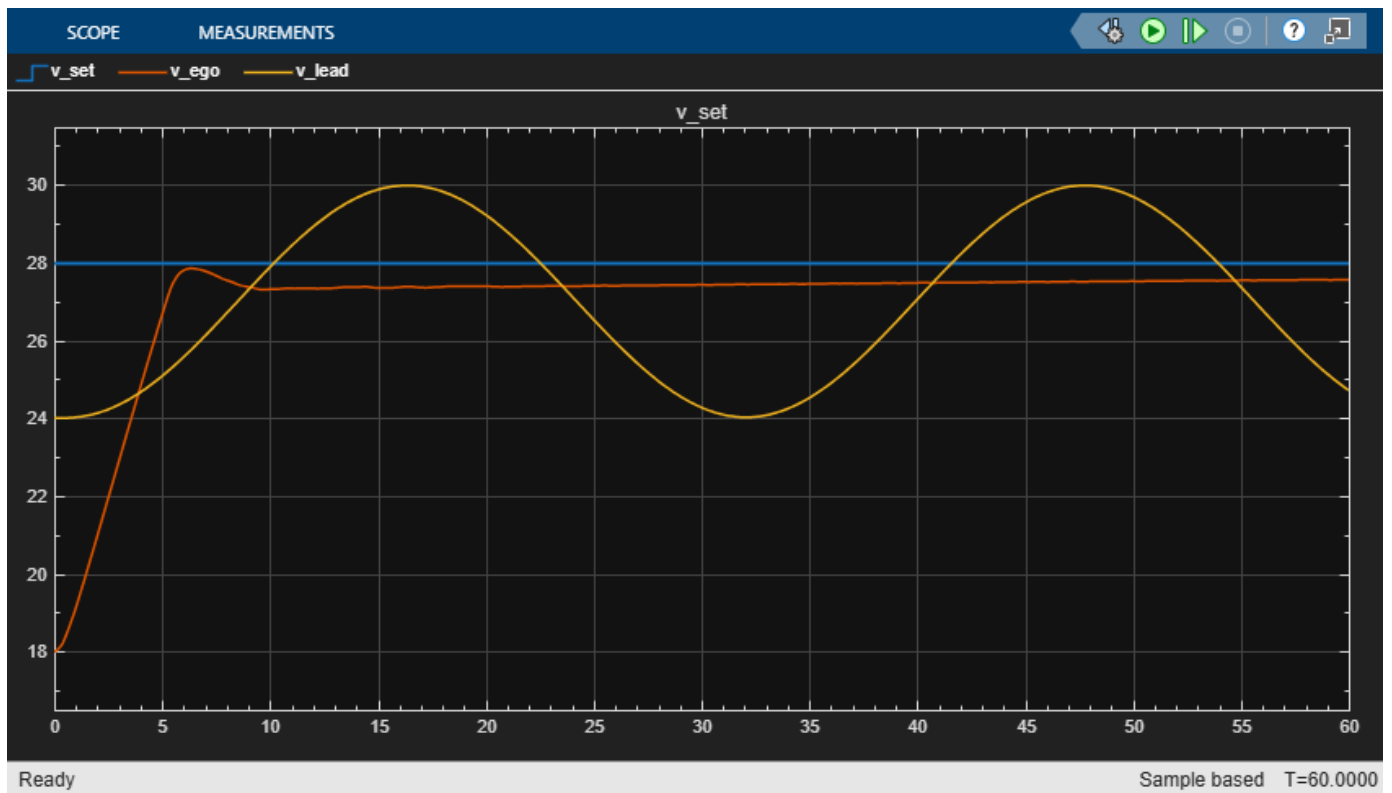
To validate the performance of the trained agent, simulate the agent within the Simulink environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

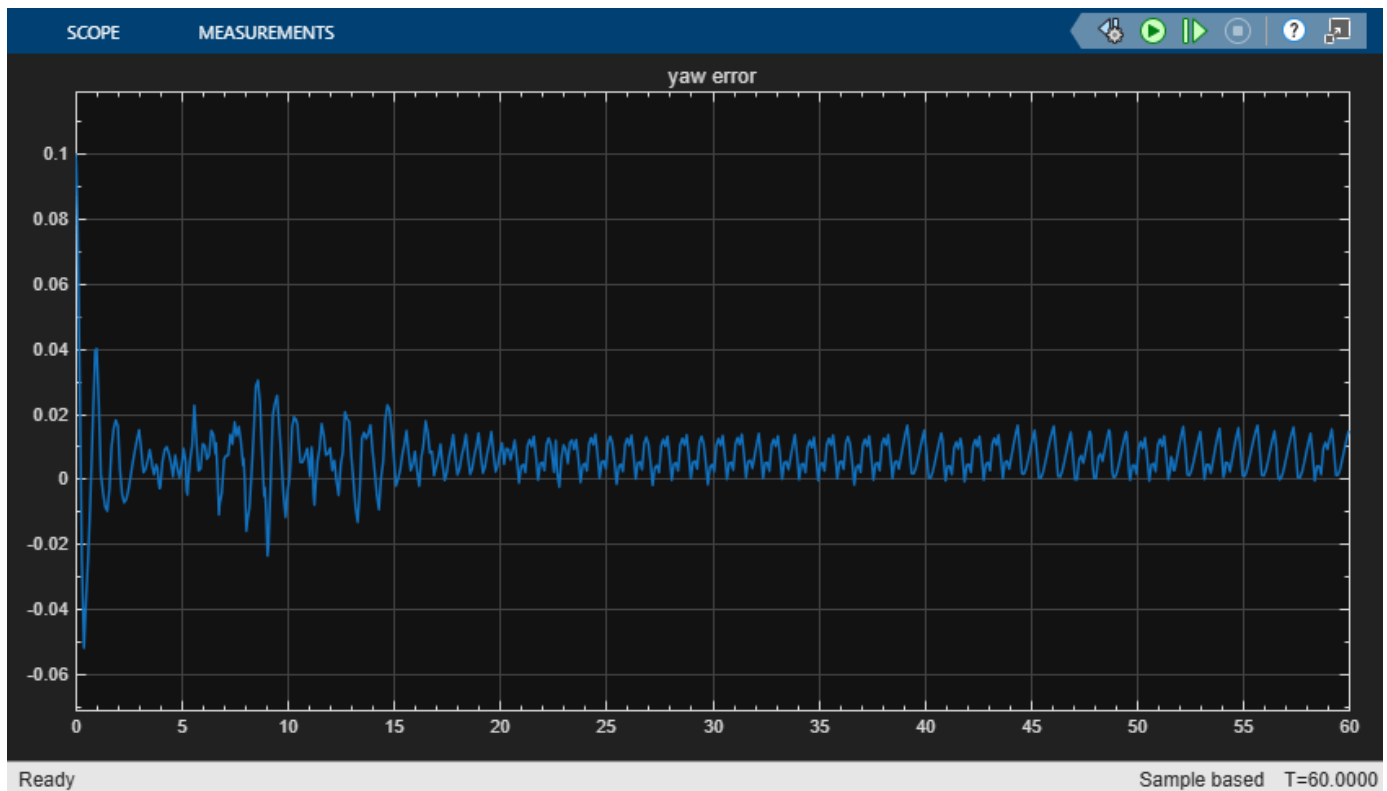
```
simOptions = rlSimulationOptions(MaxSteps=maxsteps);
experience = sim(env,agent,simOptions);
```

To demonstrate the trained agent using deterministic initial conditions, simulate the model in Simulink.

```
e1_initial = -0.4;
e2_initial = 0.1;
x0_lead = 70;
sim mdl)
```







The plots show the results when the lead car is 70 m ahead of the ego car at the beginning of the simulation.

- The lead car changes speed from 24 m/s to 30 m/s periodically (see the "velocity" plot).
- From 0 to 10 seconds, the ego car tracks the set velocity (see the "velocity" plot) and experiences some acceleration ("accel and steer" plot). After 10 seconds, the acceleration is close to 0.
- The lateral deviation decreases greatly within 1 second and remains less than 0.1 m (see the "lateral error" plot).
- The ego car maintains a safe distance throughout the simulation (see "distance" plot).

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState)
```

Environment Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the reset function uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see "Reset Function for Simulink Environments" on page 2-112.

```
function in = pfcResetFcn(in)
```

```

% random value for initial position of lead car
in = setVariable(in,'x0_lead',40+randi(60,1,1));

% random value for lateral deviation
in = setVariable(in,'e1_initial', 0.5*(-1+2*rand));

% random value for relative yaw angle
in = setVariable(in,'e2_initial', 0.1*(-1+2*rand));

end

```

See Also

Functions

bus2RLSpec | train | sim | rlSimulinkEnv

Objects

rlSACAgent | rlSACAgentOptions | rlTrainingOptions | rlSimulationOptions | rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Train Hybrid-Action PPO Agent for Path-Following Control” on page 8-120
- “Train Multiple Agents for Path Following Control” on page 8-57
- “Lane Following Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Lane Following Control with Sensor Fusion and Lane Detection” (Automated Driving Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Train Reinforcement Learning Agents” on page 5-3

Train Hybrid-Action PPO Agent for Path-Following Control

This example demonstrates how to train a hybrid-action Proximal Policy Optimization (PPO) agent to perform path-following control (PFC) for a vehicle. A hybrid action agent is an Reinforcement Learning (RL) agent that has an action space consisting of both discrete and continuous actions. For an example showing how to use a hybrid-action SAC agent, see “Train Hybrid SAC Agent for Path-Following Control” on page 8-109. For an example that shows how to use two RL agents (one with discrete action space, the other with continuous action space), see “Train Multiple Agents for Path Following Control” on page 8-57. In that example, a DDPG agent provides continuous acceleration values for the longitudinal control loop while a DQN agent provides discrete steering angle values for the lateral control loop.

Overview

A PFC system controls the vehicle under consideration (also referred as "ego vehicle") such that it:

- Maintains a given traveling speed
- Maintains a safe distance from a vehicle in front of it, also called lead vehicle, by controlling longitudinal acceleration and braking
- Travels along the centerline of its lane by controlling the front steering angle

For more information, see Path Following Control System (Model Predictive Control Toolbox).

In this example, you train a single hybrid-action PPO agent to control both the lateral steering (discrete action) and the longitudinal speed (continuous action) of the ego vehicle.

The actor approximation model has three outputs:

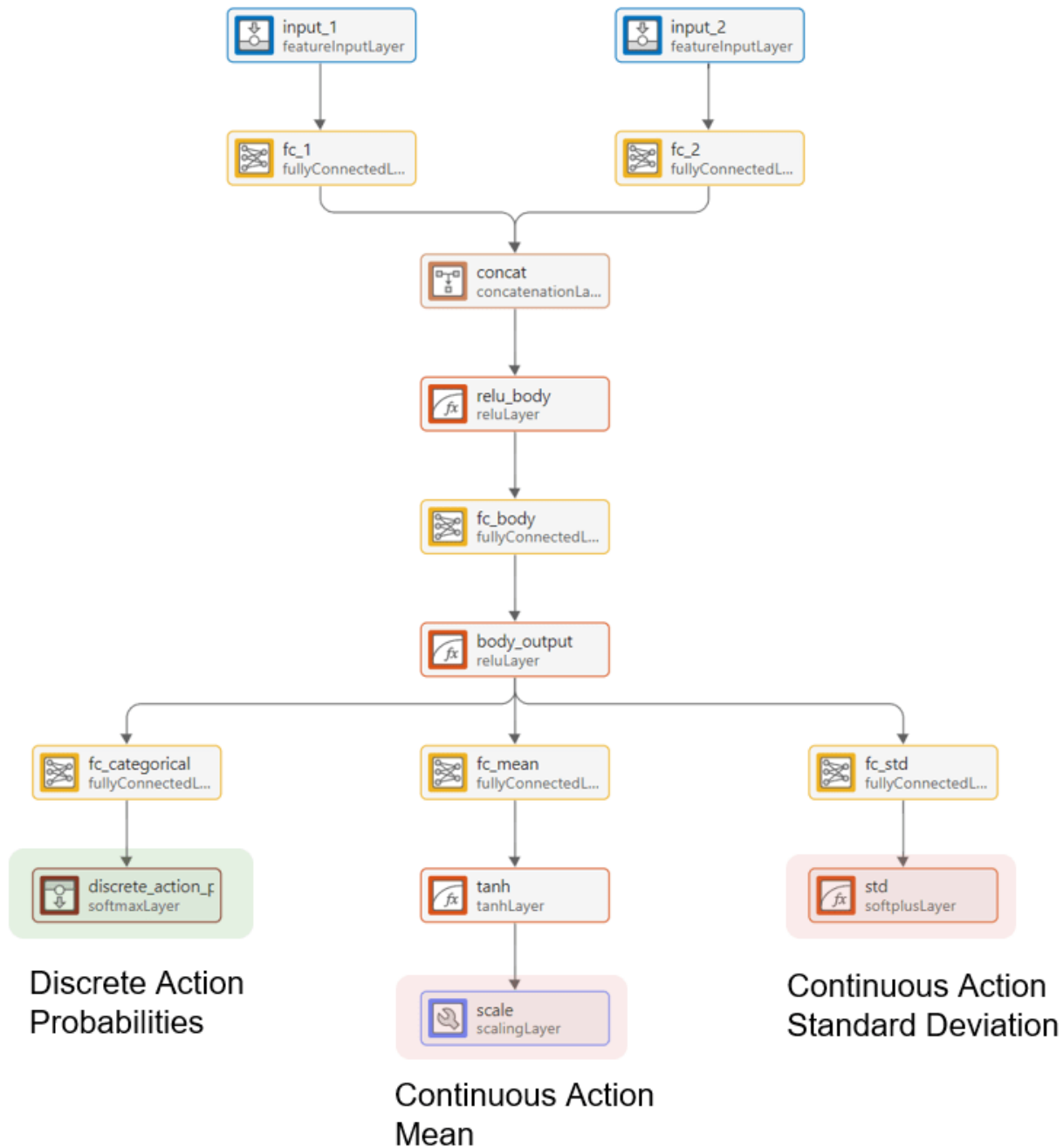
- 1 The categorical distribution output — A vector of probabilities for a discrete action.
- 2 The mean values output — A vector of mean values of Gaussian distributions, each for each continuous action dimension.
- 3 The standard deviation values output — A vector of standard deviations of those Gaussian distributions.

The actor samples actions as follows:

- 1 Discrete part of the action: A discrete value sampled among the possible values, according to the probabilities expressed by the categorical distribution output of the approximation model.
- 2 Continuous part of the action: Continuous values sampled according to the Gaussian distributions expressed by the mean and standard deviation outputs of the approximation model.

The image below shows the actor model used in this example.

Actor model



For more information on the hybrid-action PPO algorithm, see “Proximal Policy Optimization (PPO) Agent” on page 3-50.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Create Environment Object

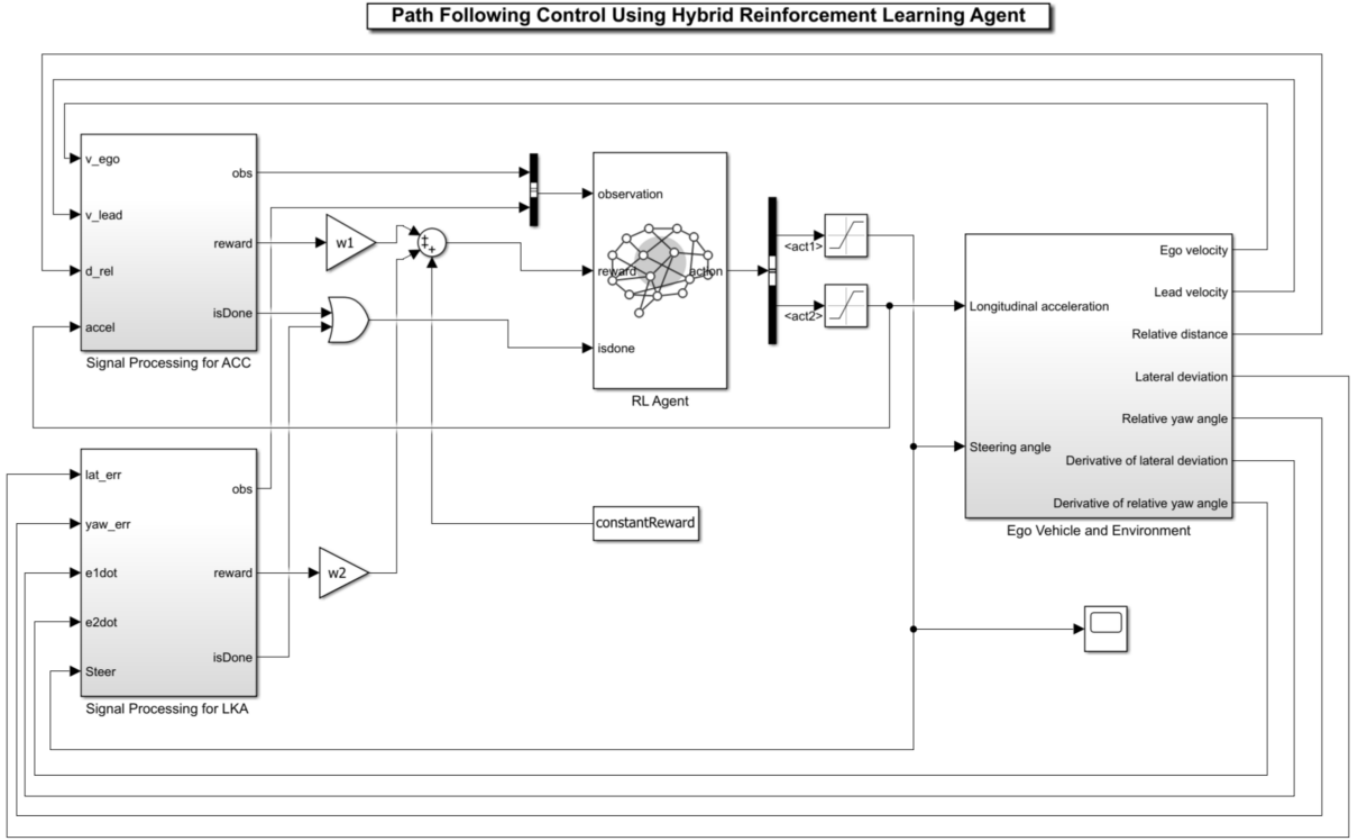
The environment for this example includes a simple bicycle model for the ego vehicle and a simple longitudinal model for the lead vehicle. The agent controls the longitudinal acceleration, braking, and the front steering angle of the ego vehicle.

Load the environment parameters.

```
HPP0PFCParams
```

Open the Simulink® model.

```
mdl = "HPP0PFC";  
open_system(mdl)
```



The simulation terminates if any of the following conditions occur.

- $|e_1| > 1$ — The magnitude of the lateral deviation is greater than 1.
- $V_{ego} < 0.5$ — The longitudinal velocity of the ego vehicle is less than 0.5.
- $D_{rel} < 0$ — The distance between the ego and the lead vehicle is less than zero.

To determine the ego vehicle's reference velocity V_{ref} :

- The safe distance is a linear function of the ego vehicle's longitudinal velocity V . That is, $t_{gap} * V + D_{default}$.
- If the relative distance is less than the safe distance, the ego vehicle tracks the minimum value between the velocity of the lead vehicle and the desired velocity set by the driver. Setting the reference velocity in this way allows the ego vehicle to maintain a safe distance from the lead vehicle. If the relative distance is greater than the safe distance, the ego vehicle uses the desired velocity set by the driver as the reference velocity.

Observation:

- The first observation channel contains the longitudinal measurements. These are the velocity error $e_V = V_{ref} - V$, its integral $\int e$, and the ego vehicle longitudinal velocity V .
- The second observation channel contains the lateral measurements. These are the lateral deviation e_1 , the relative yaw angle e_2 (the yaw angle error with respect to the lane centerline), their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.

Action:

- The discrete action a_t — The action signal consists of discrete steering angle actions which take values from -15 degrees (-0.2618 rad) to 15 degrees (0.2618 rad) in steps of 1 degree (0.0175 rad).
- The continuous action u_t — The action signal consists of continuous acceleration values between -3 and 2 m/s^2 .

Reward:

- The reward r_t , provided at every time step t , is the weighted sum of the reward $r_{lateral}$ for the lateral control, the reward $r_{longitudinal}$ for the longitudinal control, and the constant reward $r_{everystep}$.

$$r_t = w_1 r_{lateral} + w_2 r_{longitudinal} + r_{everystep}$$

$$r_{lateral} = -(0.1e_1^2 + 0.5u_{t-1}^2) - 100F_t + 2H_t$$

$$r_{longitudinal} = -(0.01e_V^2 + 0.1a_{t-1}^2) - 100F_t + M_t$$

$$w_1 = 1/50$$

$$w_2 = 1/120$$

$$r_{everystep} = 1/10$$

In these equations, u_{t-1} is the steering input from the previous time step, a_{t-1} is the acceleration input from the previous time step, and:

- $F_t = 1$ if the simulation terminates, otherwise $F_t = 0$.
- $M_t = 1$ if $e_V^2 < 1$, otherwise $M_t = 0$.
- $H_t = 1$ if $e_1^2 < 0.01$, otherwise $H_t = 0$.

The logical terms in the reward functions (F_t , M_t , and H_t) penalize the agent if the simulation terminates early, while encouraging the agent to make both the lateral error and velocity error small.

Create the observation specification. Because an observation contains multiple channels in this Simulink environment, you must use the `bus2RLSpec` function to create the specification. For more information about Simulink environments, see “Create Custom Simulink Environments” on page 2-110.

Create a bus object.

```
obsBus = Simulink.Bus();
```

Add the first bus element.

```
obsBus.Elements(1) = Simulink.BusElement;
obsBus.Elements(1).Name = "signal1";
obsBus.Elements(1).Dimensions = [3,1];
```

Add the second bus element.


```
obsBus.Elements(2) = Simulink.BusElement;
obsBus.Elements(2).Name = "signal2";
obsBus.Elements(2).Dimensions = [6,1];
```

Create the observation specification.

```
obsInfo= bus2RLSpec("obsBus");
```

Create the action specification. For the hybrid-action PPO agent, you must have two action channels. The first action channel must be for the discrete part of the action, and the second must be for the continuous part of the action. Use the `bus2RLSpec` function to create the specification, as for the observation specification case.

Create a bus object.

```
actBus = Simulink.Bus();
```

Add the first bus element for the discrete part of the action. The discrete part of the action must be the first action channel.

```
actBus.Elements(1) = Simulink.BusElement;
actBus.Elements(1).Name = "act1";
```

Add the second bus element for the continuous part of the action.

```
actBus.Elements(2) = Simulink.BusElement;
actBus.Elements(2).Name = "act2";
actBus.Elements(2).Dimensions = [1,1];
actInfo = bus2RLSpec("actBus","DiscreteElements", ...
    {"act1", (-15:15)*pi/180});
```

Define the limits of continuous actions.

```
actInfo(2).LowerLimit = -3;
actInfo(2).UpperLimit = 2;
```

Create a Simulink environment object, specifying the block paths for the agent block. For more information, see `rlSimulinkEnv`.

```
blks = mdl + "/RL Agent";
env = rlSimulinkEnv(mdl,blks,obsInfo,actInfo);
```

Specify a reset function for the environment by using its `ResetFcn` property. The function `pfcResetFcn` on page 8-133 (provided at the end of the example) sets the initial conditions of the lead and ego vehicles at the beginning of every episode during training.

```
env.ResetFcn = @pfcResetFcn;
```

Create Hybrid-Action PPO Agent

Fix the random number stream.

```
rng(0, "twister");
```

Set the sample time, in seconds, for the Simulink model and the RL agent object.

```
Ts = 0.1;
```

Set the simulation time, in seconds.

```
Tf = 60;
```

Create a default hybrid-action PPO agent. When the action specification defines a hybrid action space (that is, an action space that contains both a discrete and a continuous action channel), `rlPPOAgent` creates a hybrid-action PPO agent. The hybrid-action PPO agent uses `rlHybridStochasticActor` as the actor and `rlValueFunction` as the critic.

```
agent = rlPPOAgent(obsInfo, actInfo);
```

Specify the agent options:

- Set the mini-batch size to 512 to make the training more stable.
- The default `LearningFrequency` is -1, meaning that the PPO agent collects at least `10*MiniBatchSize` experiences and updates the agent at the end of the episode.
- Set learning rate to `1e-3` for the actor and the critics.
- Set gradient thresholds to 1 to limit the gradient values.
- Set the normalized advantage method to "current" for more stable training.

```
agent.SampleTime = Ts;  
agent.AgentOptions.MiniBatchSize = 512;  
agent.AgentOptions.ExperienceHorizon = Tf/Ts;  
agent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
agent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
agent.AgentOptions.NormalizedAdvantageMethod = "current";
```

Set the entropy loss weights for the discrete and continuous actions. The first element is for discrete actions and the second element is for continuous actions.

```
agent.AgentOptions.EntropyLossWeight= [0.001, 0.001];
```

Train Hybrid-Action PPO Agent

Specify the training options. For this example, use the following options.

- Run each training episode for a maximum of 15000 episodes, with each episode lasting a maximum of `maxsteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box.
- Stop the training when the agent receives an average evaluation episode reward greater than 79.6.
- Use parallel training for more efficient training when parallel computing resources are available.
- To improve performance, do not save the simulation data during training. To save the simulation data during training instead, set `SimulationStorageType` to "file" or "memory".

```
maxepisodes = 15000;  
maxsteps = ceil(Tf/Ts);  
trainingOpts = rlTrainingOptions( ...  
    MaxEpisodes=maxepisodes, ...  
    MaxStepsPerEpisode=maxsteps, ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=79.6, ...  
    UseParallel=canUseParallelPool, ...  
    SimulationStorageType="none");
```

Fix the random number stream.

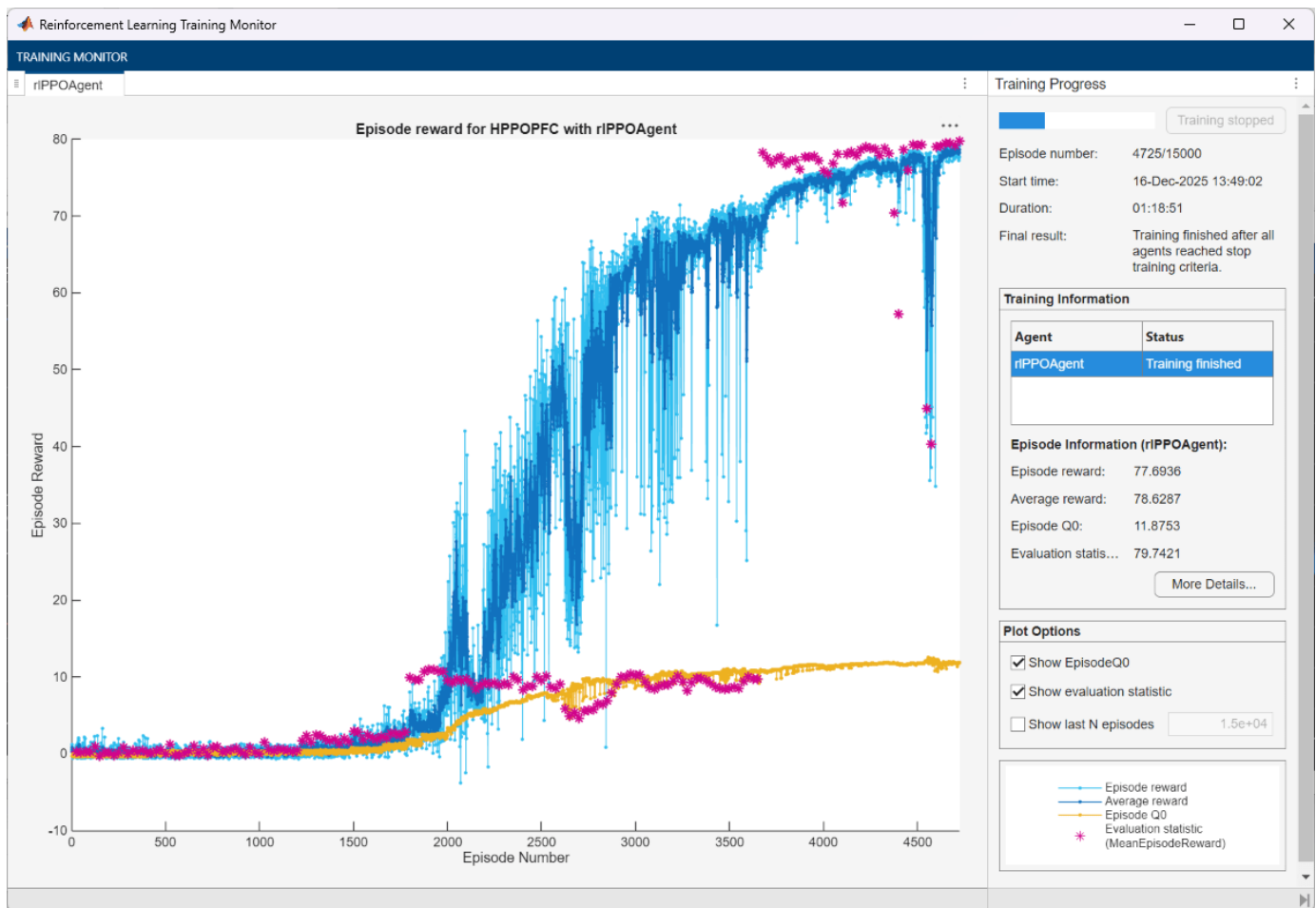
```
rng(0, "twister");
```

Train the agent by using the `train` function. Training the agent is a computationally intensive process. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Evaluate the agent after every 25 training episodes.
    % The evaluation statistic is the mean value of
    % the statistic over five evaluation episodes.
    evaluator = rlEvaluator(EvaluationFrequency=25, ...
        NumEpisodes=5, RandomSeeds=101:105);

    % Train the agent.
    trainingStats = train(agent,env,trainingOpts,Evaluator=evaluator);
else
    % Load the pretrained agent for the example.
    load("rlHybridPPOPFCAgent.mat")
end
```

This figure shows a snapshot of the training progress.



The training converges before 5000 episodes.

Simulate the Trained PPO Agent

Fix the random number stream.

```
rng(0, "twister");
```

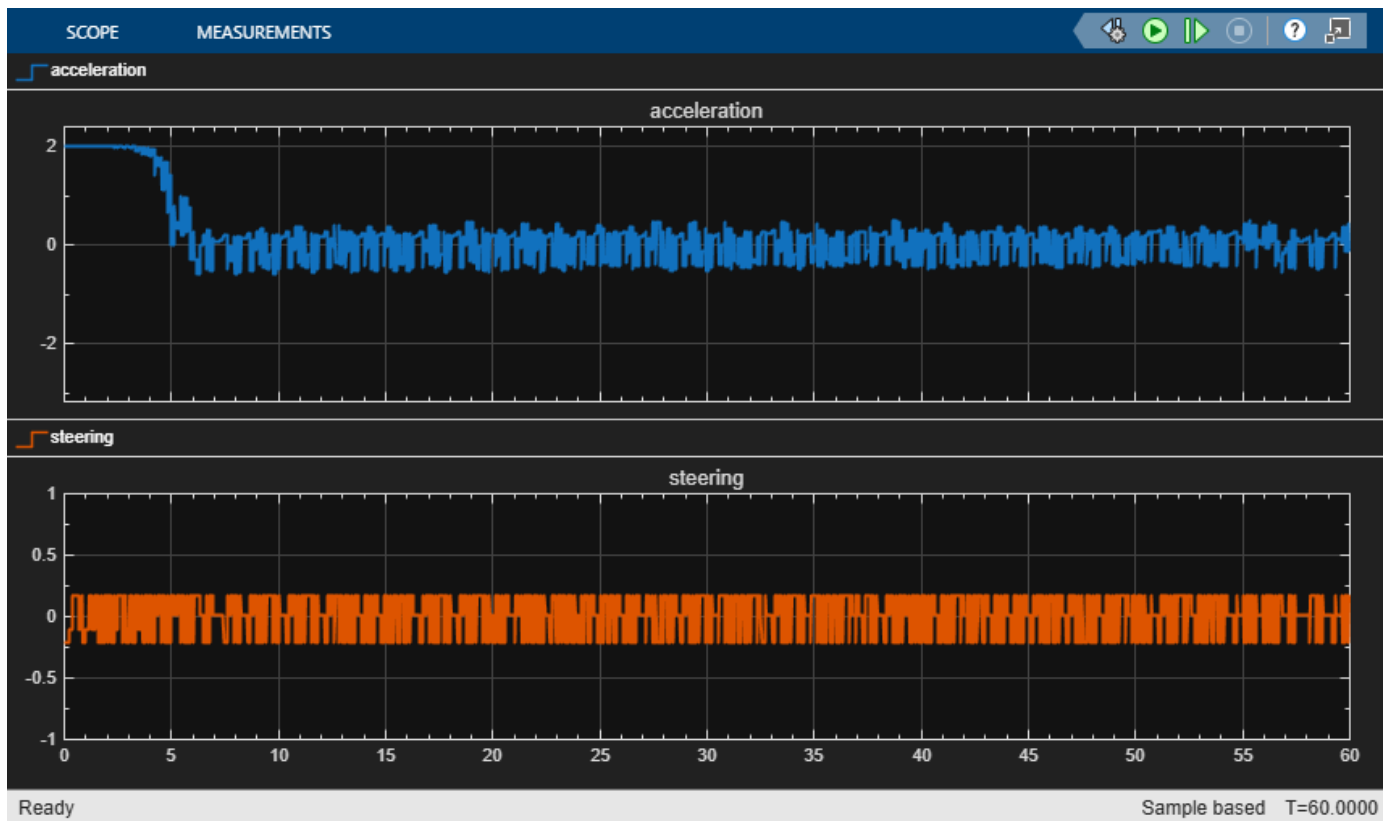
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

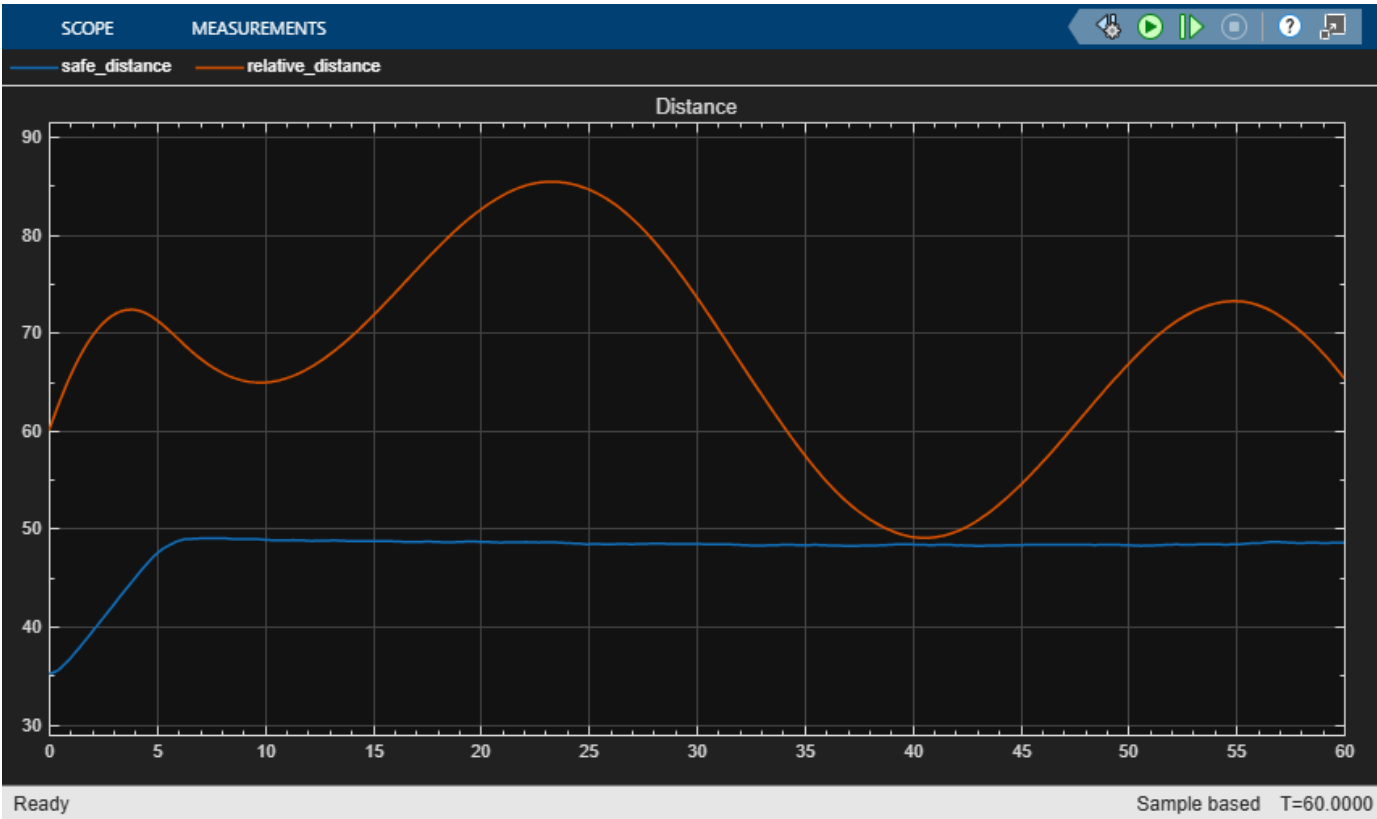
To validate the performance of the trained agent, simulate the agent within the Simulink environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

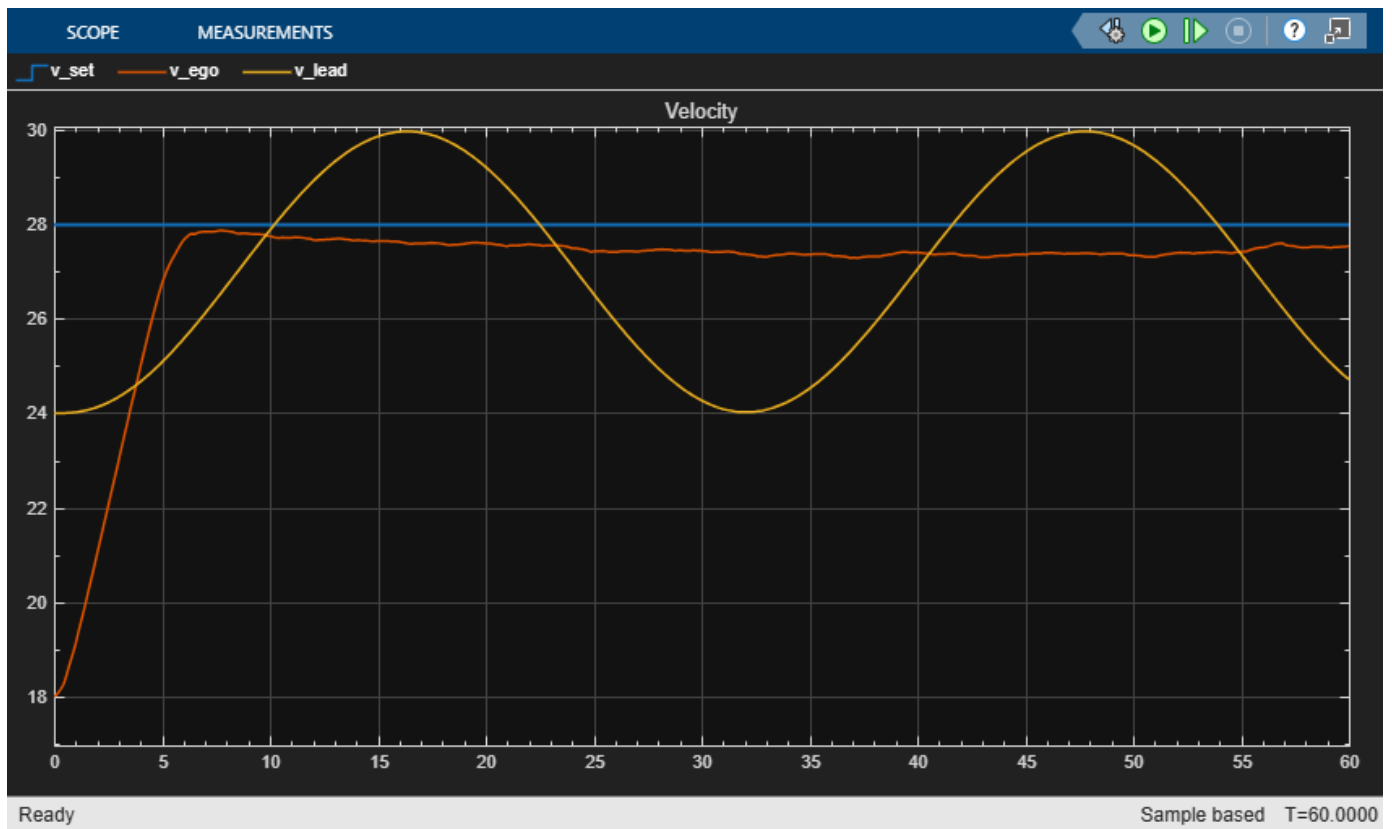
```
simOptions = rlSimulationOptions(MaxSteps=maxsteps);
experience = sim(env,agent,simOptions);
```

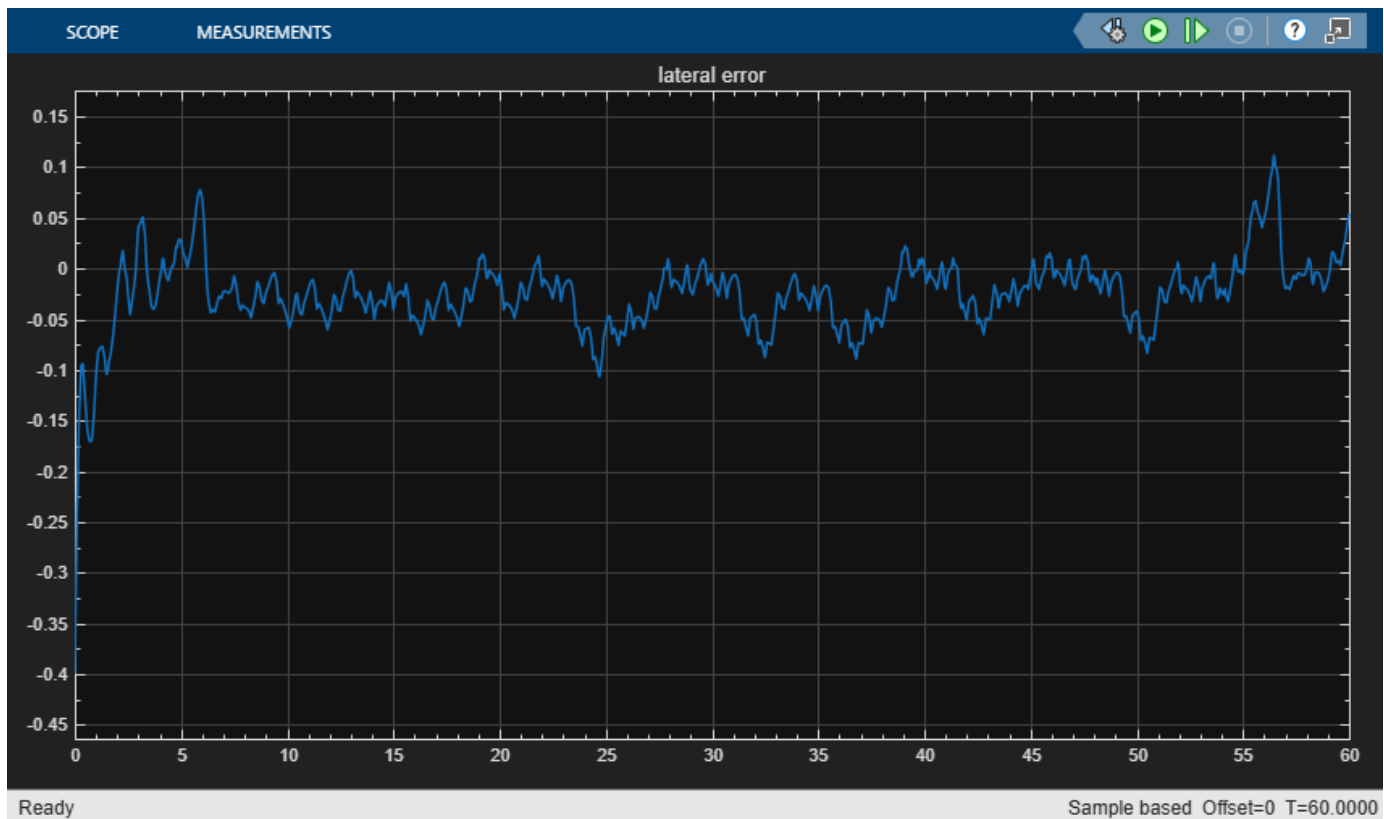
To validate the trained agent using deterministic initial conditions, simulate the model in Simulink. In this example, the lead vehicle is 70 m ahead of the ego vehicle at the beginning of the simulation.

```
e1_initial = -0.4;
e2_initial = 0.1;
x0_lead = 70;
sim mdl)
```









The plots show the results:

- The lead vehicle changes speed from 24 m/s to 30 m/s periodically (see the velocity plot).
- From 0 to 6 seconds, the ego vehicle tracks the set velocity (see the velocity plot) and experiences significant acceleration (see the acceleration and steering plot). After that, the acceleration becomes small.
- The lateral deviation decreases greatly within 1 second and remains less than 0.1 m (see the lateral error plot).
- The ego vehicle maintains a safe distance throughout the simulation (see the distance plot).

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState)
```

Environment Reset Function

The reset function `pfcResetFcn` sets the initial conditions of the lead and ego vehicles at the beginning of every episode during training. The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to the model, which are then discarded when the simulation or training completes.

For this example, the reset function uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.


```
function in = pfcResetFcn(in)

    % random value for initial position of lead vehicle
    in = setVariable(in,'x0_lead',40+randi(60,1,1));

    % random value for lateral deviation
    in = setVariable(in,'e1_initial', 0.5*(-1+2*rand));

    % random value for relative yaw angle
    in = setVariable(in,'e2_initial', 0.1*(-1+2*rand));

end
```

See Also

Functions

bus2RLSpec | train | sim | rlSimulinkEnv

Objects

rlPPOAgent | rlPPOAgentOptions | rlTrainingOptions | rlSimulationOptions | rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Train DDPG Agent for Path-Following Control” on page 8-97
- “Train Hybrid SAC Agent for Path-Following Control” on page 8-109
- “Train Multiple Agents for Path Following Control” on page 8-57
- “Lane Following Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Lane Following Control with Sensor Fusion and Lane Detection” (Automated Driving Toolbox)

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Train Reinforcement Learning Agents” on page 5-3

Train DQN Agent for Lane Keeping Assist Using Parallel Computing

This example shows how to train a deep Q-learning network (DQN) agent for lane keeping assist (LKA) in Simulink® using parallel training. For an example that shows how to train the agent without using parallel training, see “Train DQN Agent for Lane Keeping Assist” on page 8-76.

For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32. For an example that trains a DQN agent in MATLAB®, see “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64.

DQN Parallel Training Overview

DQN agents use experience-based parallelization, in which the environment simulation is done by the workers and the gradient computation is done by the client. Specifically, each worker generates new experiences from its copy of agent and environment and sends experience data back to the client. The client agent updates its parameters as follows.

- For asynchronous training, the client agent calculates gradients and updates agent parameters from the received experiences, without waiting to receive experiences from all the workers. The client then sends the updated parameters back to the worker that provided the experiences. Then, the worker updates its copy of the agent and continues to generate experiences using its copy of the environment. To specify asynchronous training, the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function must be set to `async`.
- For synchronous training, the client agent waits to receive experiences from all of the workers and then calculates the gradients from all these experiences. The client updates the agent parameters, and sends the updated parameters to all the workers at the same time. Then, all workers use a single updated agent copy, together with their copy of the environment, to generate experiences. To specify synchronous training, the `Mode` property of the `rlTrainingOptions` object that you pass to the `train` function must be set to `sync`.

For more information on experience-based parallelization, see “Train Agents Using Parallel Computing and GPUs” on page 5-151.

Simulink Model for Ego Car

The reinforcement learning environment for this example is a simple bicycle model for the ego vehicle dynamics. The training goal is to keep the ego vehicle traveling along the centerline of the lanes by adjusting the front steering angle. This example uses the same vehicle model as “Train DQN Agent for Lane Keeping Assist” on page 8-76.

```
m = 1575;    % total vehicle mass (kg)
Iz = 2875;   % yaw moment of inertia (mNs^2)
lf = 1.2;    % longitudinal distance from CG to front tires (m)
lr = 1.6;    % longitudinal distance from CG to rear tires (m)
Cf = 19000;  % cornering stiffness of front tires (N/rad)
Cr = 33000;  % cornering stiffness of rear tires (N/rad)
Vx = 15;     % longitudinal velocity (m/s)
```

Define the sample time `Ts` and simulation duration `T` in seconds.

```
Ts = 0.1;
T = 15;
```

The output of the LKA system is the front steering angle of the ego car. To simulate the physical steering limits of the ego car, constrain the steering angle to the range $[-0.5, 0.5]$ rad.

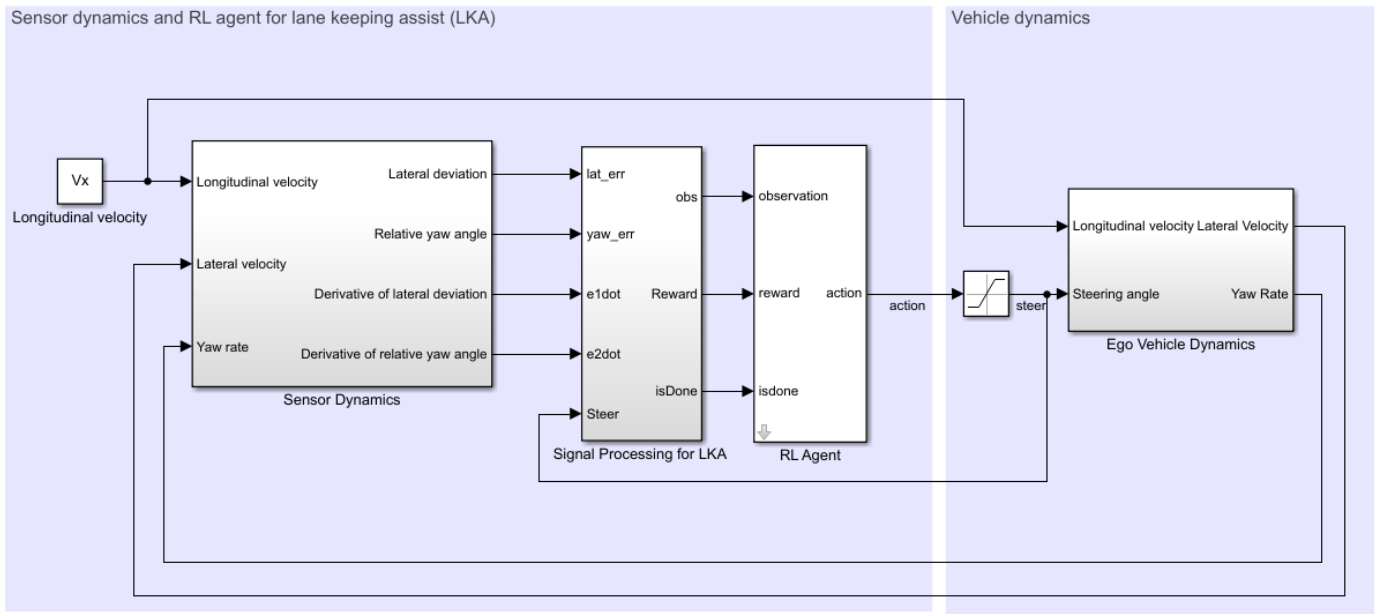
```
u_min = -0.5;
u_max = 0.5;
```

The curvature of the road is defined by a constant $0.001 \text{ (m}^{-1}\text{)}$. The initial value for the lateral deviation is 0.2 m and the initial value for the relative yaw angle is -0.1 rad .

```
rho = 0.001;
e1_initial = 0.2;
e2_initial = -0.1;
```

Open the model.

```
mdl = "rLLKAMdl";
open_system(mdl)
agentblk = mdl + "/RL Agent";
```



Copyright 2018 The MathWorks, Inc.

In this model:

- The steering-angle action signal from the agent to the environment is from -15 degrees to 15 degrees.
- The observations from the environment are the lateral deviation e_1 , relative yaw angle e_2 , their derivatives $\dot{e}_1$ and $\dot{e}_2$, and their integrals $\int e_1$ and $\int e_2$.
- The simulation is terminated when the lateral deviation $|e_1| > 1$.
- The reward r_t , provided at every time step t , is

$$r_t = -(10e_1^2 + 5e_2^2 + 2u^2 + 5\dot{e}_1^2 + 5\dot{e}_2^2)$$

where u is the control input from the previous time step $t - 1$.

Create Environment Interface

Create a reinforcement learning environment object for the ego vehicle.

Define the observation information.

```
obsInfo = rlNumericSpec([6 1], ...  
    LowerLimit=-inf*ones(6,1), ...  
    UpperLimit=inf*ones(6,1));  
  
obsInfo.Name = "observations";  
obsInfo.Description = ...  
    "lateral deviation and relative yaw angle";
```

Define the action information.

```
actInfo = rlFiniteSetSpec((-15:15)*pi/180);  
actInfo.Name = "steering";
```

Create the environment object.

```
env = rlSimulinkEnv mdl, agentblk, obsInfo, actInfo);
```

The object has a discrete action space where the agent can apply one of 31 possible steering angles from -15 degrees to 15 degrees. The observation is the six-dimensional vector containing lateral deviation, relative yaw angle, as well as their derivatives and integrals with respect to time.

To define the initial condition for the lateral deviation and relative yaw angle, specify an environment reset function using an anonymous function handle. `localResetFcn`, which is defined at the end of this example, randomizes the initial lateral deviation and relative yaw angle.

```
env.ResetFcn = @(in)localResetFcn(in);
```

Create DQN Agent

DQN agents use a parameterized Q-value function approximator to estimate the value of the policy. Because DQN agents have a discrete action space, you have the option to create a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic.

A vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible actions. The value of each output element represents the expected discounted cumulative long-term reward when an agent starts from the state corresponding to the given observation and executes the action corresponding to the element number (and follows a given policy afterwards).

To model the parameterized Q-value function within the critic, use a neural network with one input (the six-dimensional observed state) and one output vector with 31 elements (evenly spaced steering angles from -15 to 15 degrees). Get the number of dimensions of the observation space and the number of elements of the discrete action space from the environment specifications.

```
nI = obsInfo.Dimension(1);    % number of inputs (6)  
nL = 120;                    % number of neurons  
nO = numel(actInfo.Elements); % number of outputs (31)
```

Define the network as an array of layer objects.

```
dnn = [
    featureInputLayer(nI)
    fullyConnectedLayer(nL)
    reluLayer
    fullyConnectedLayer(nL)
    reluLayer
    fullyConnectedLayer(nO)
];
```

The critic network is initialized randomly. Ensure reproducibility of the section by fixing the seed of the random generator.

```
rng(0)
```

Convert to a `dlnetwork` object and display the number of parameters.

```
dnn = dlnetwork(dnn);
summary(dnn)

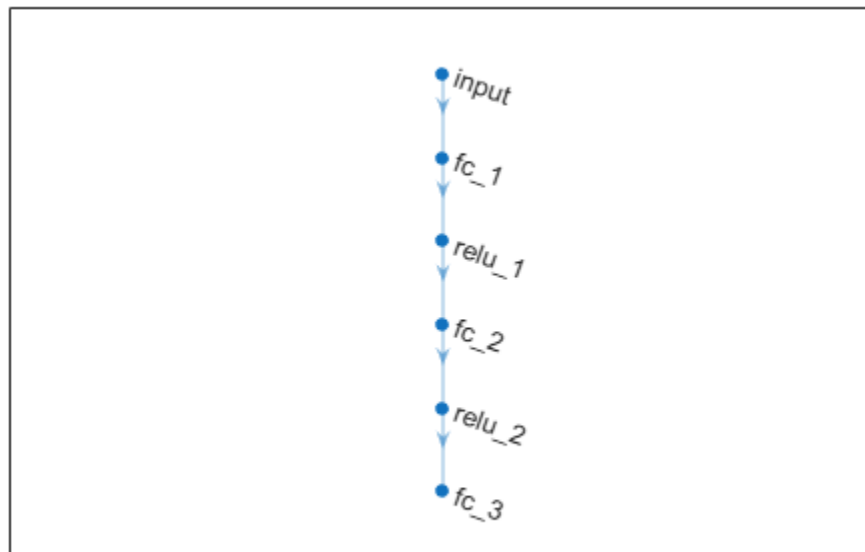
Initialized: true

Number of learnables: 19.1k

Inputs:
    1 'input' 6 features
```

View the network configuration.

```
plot(dnn)
```



Create the critic using `dnn` and the environment specifications. For more information on vector Q-value function approximators, see `rlVectorQValueFunction`.

```
critic = rlVectorQValueFunction(dnn,obsInfo,actInfo);
```

Specify training options for the critic using `rlOptimizerOptions`.

```
criticOptions = rlOptimizerOptions( ...  
    LearnRate=1e-4, ...  
    GradientThreshold=1, ...  
    L2RegularizationFactor=1e-4);
```

Specify the DQN agent options using `rlDQNAgentOptions`, include the critic options object.

```
agentOpts = rlDQNAgentOptions( ...  
    SampleTime=Ts, ...  
    UseDoubleDQN=true, ...  
    CriticOptimizerOptions=criticOptions, ...  
    ExperienceBufferLength=1e6, ...  
    MiniBatchSize=256);
```

You can also set or modify the agent options using dot notation.

```
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-4;
```

Alternatively, you can create the agent first, and then access its option object and modify the options using dot notation.

Create the DQN agent using the specified critic and the agent options. For more information, see `rlDQNAgent`.

```
agent = rlDQNAgent(critic,agentOpts);
```

Training Options

To train the agent, first specify the training options. For this example, use the following options.

- Run each training for a maximum of 10000 episodes, with each episode lasting a maximum of `ceil(T/Ts)` time steps.
- Display the training progress in the Episode Manager dialog box only (set the `Plots` and `Verbose` options accordingly).
- Stop the training when the episode reward reaches -1.
- Save a copy of the agent for each episode where the cumulative reward is greater than 100.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 10000;  
maxsteps = ceil(T/Ts);  
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=maxepisodes, ...  
    MaxStepsPerEpisode=maxsteps, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EpisodeReward", ...  
    StopTrainingValue= -1, ...  
    SaveAgentCriteria="EpisodeReward", ...  
    SaveAgentValue=100);
```

Parallel Training Options

To train the agent in parallel, specify the following training options.

- Set the `UseParallel` option to `true`.
- Train agent in parallel asynchronously by setting the `ParallelizationOptions.Mode` option to `"async"`.

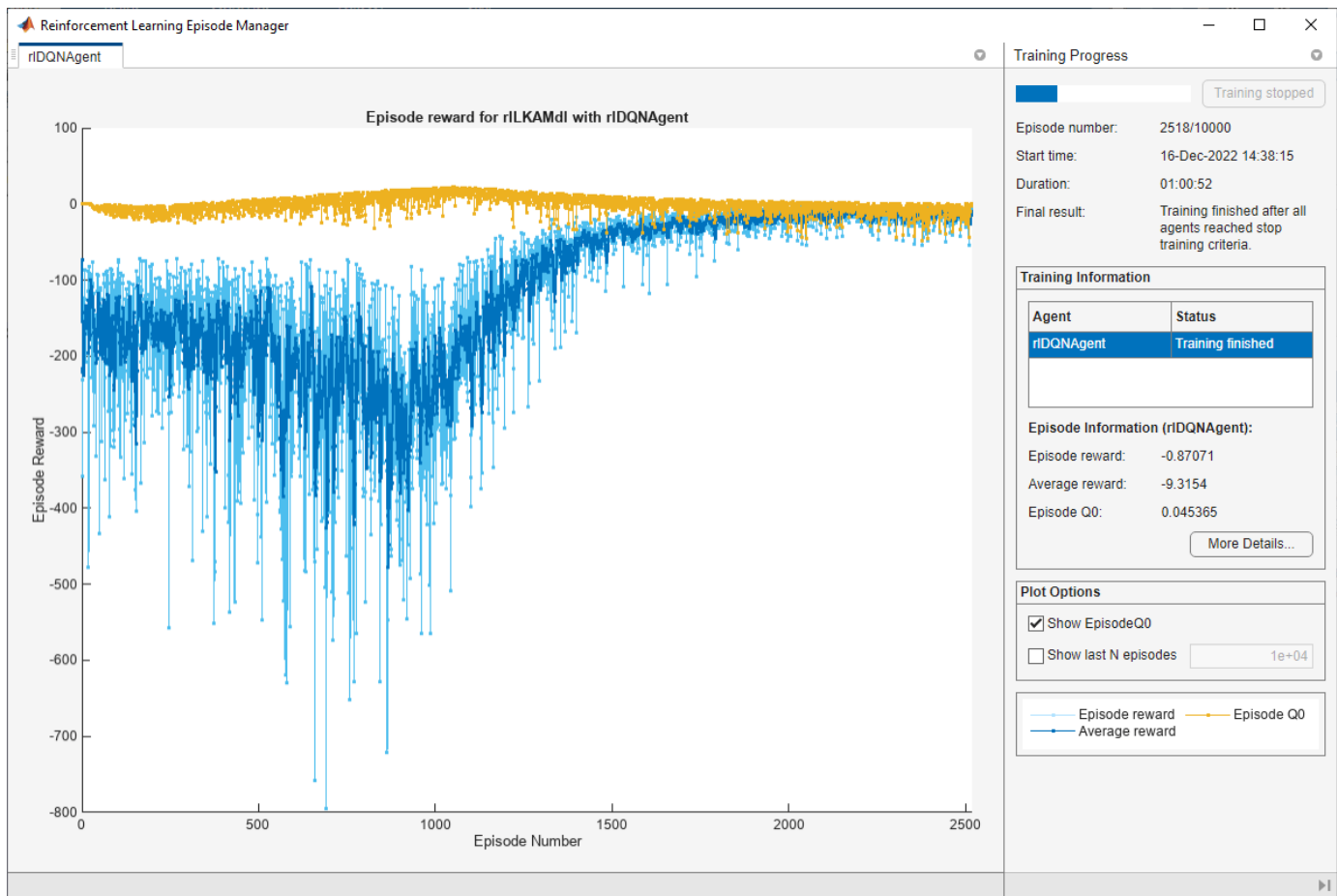
```
trainOpts.UseParallel = true;  
trainOpts.ParallelizationOptions.Mode = "async";
```

For more information on training options, see `rlTrainingOptions`.

Train Agent

Train the agent using the `train` function. Training the agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`. Due to randomness of the parallel training, you can expect different training results from the plot below. The plot shows the result of training with four workers.

```
doTraining = false;  
  
if doTraining  
    % Train the agent.  
    trainingStats = train(agent,env,trainOpts);  
else  
    % Load pretrained agent for the example.  
    load("SimulinkLKADQNParallel.mat","agent")  
end
```



Simulate the Agent

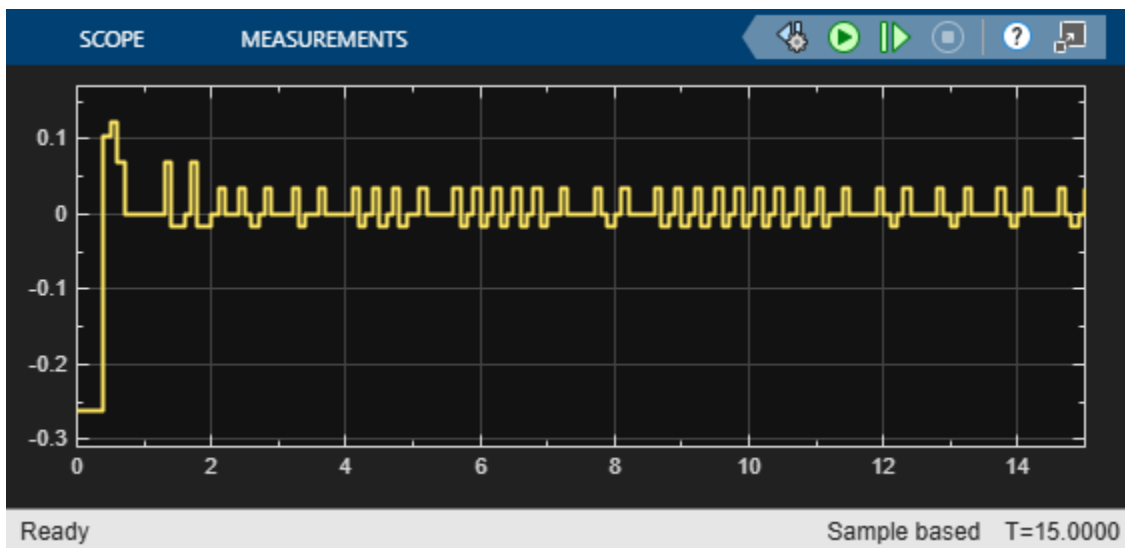
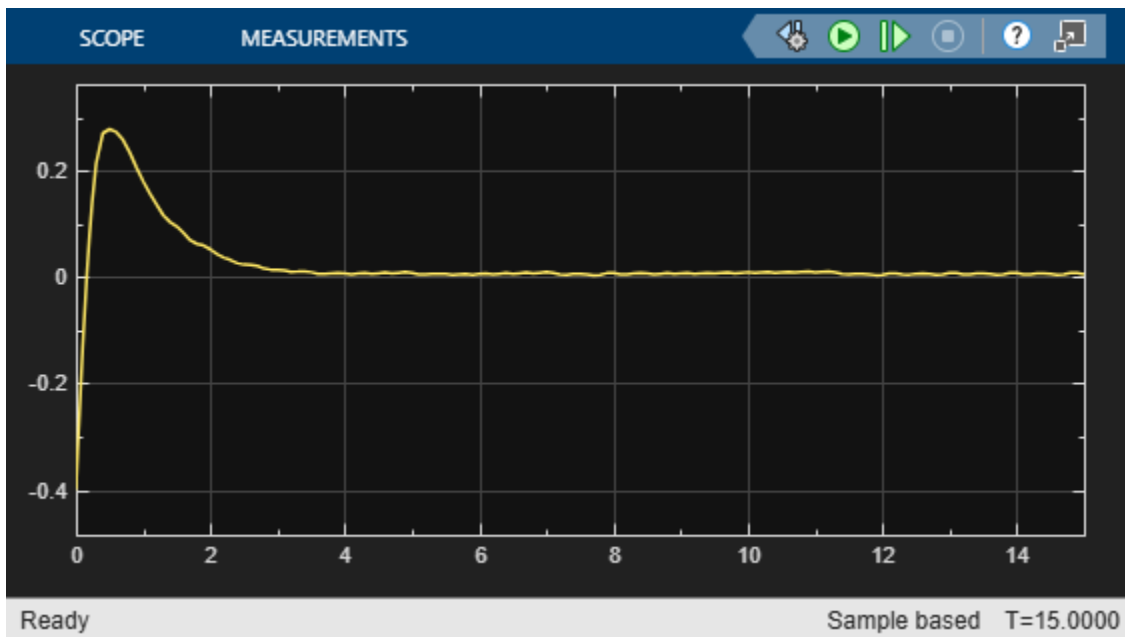
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

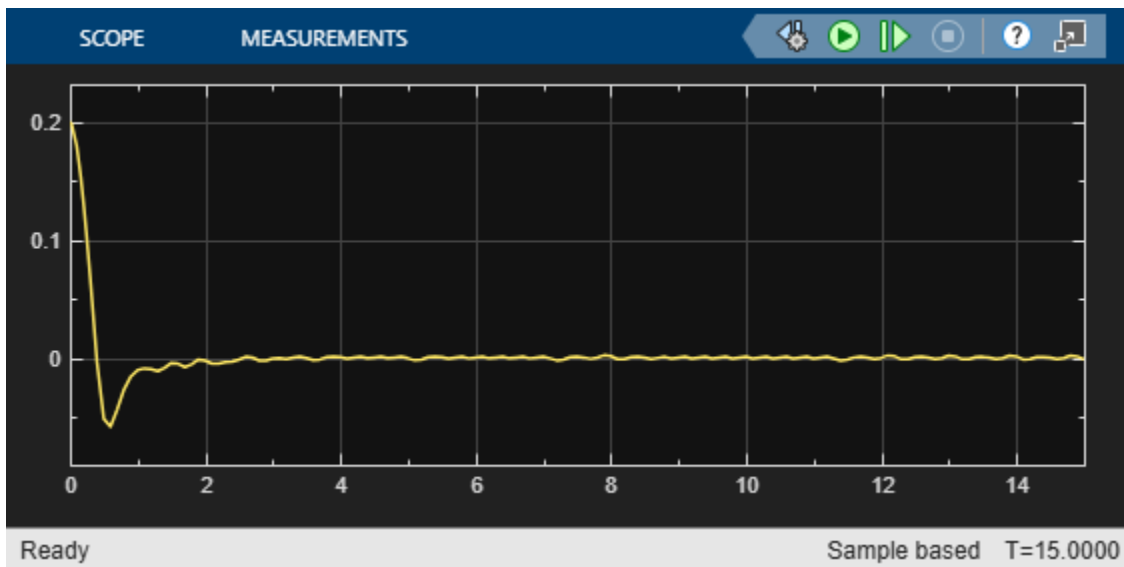
To validate the performance of the trained agent, uncomment the following two lines and simulate the agent within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
% simOptions = rlSimulationOptions(MaxSteps=maxsteps);
% experience = sim(env,agent,simOptions);
```

To demonstrate the trained agent using deterministic initial conditions, simulate the model in Simulink.

```
e1_initial = -0.4;
e2_initial = 0.2;
sim mdl)
```



As shown in these plots, the lateral error (top plot) and relative yaw angle (bottom plot) are both driven to zero. The vehicle starts from off centerline (-0.4 m) and nonzero yaw angle error (0.2 rad). The LKA enables the ego car to travel along the centerline after 2.5 seconds. The steering angle (middle plot) shows that the controller reaches steady state after 2 seconds.

Local Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
% set initial lateral deviation and relative yaw angle to random values
in = setVariable(in,"e1_initial", 0.5*(-1+2*rand));
in = setVariable(in,"e2_initial", 0.1*(-1+2*rand));
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlOptimizerOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Train PPO Agent with Curriculum Learning for a Lane Keeping Application” on page 5-164
- “Lane Keeping Assist with Lane Detection” (Automated Driving Toolbox)
- “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox)
- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Train Biped Robot to Walk Using Reinforcement Learning Agents

This example shows how to train a biped robot to walk using either a deep deterministic policy gradient (DDPG) agent or a twin-delayed deep deterministic policy gradient (TD3) agent. In the example, you also compare the performance of these trained agents. The robot in this example is modeled in Simscape™ Multibody™. For more information on these agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67 and “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

For the purpose of comparison in this example, this example trains both agents on the biped robot environment with the same model parameters. The example also configures the agents to have the following settings in common.

- Initial condition strategy of the biped robot.
- Network structure of actor and critic, inspired by [1].
- Options for actor and critic objects.
- Training options (sample time, discount factor, mini-batch size, experience buffer length, exploration noise).

Fix Random Seed Generator to Improve Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

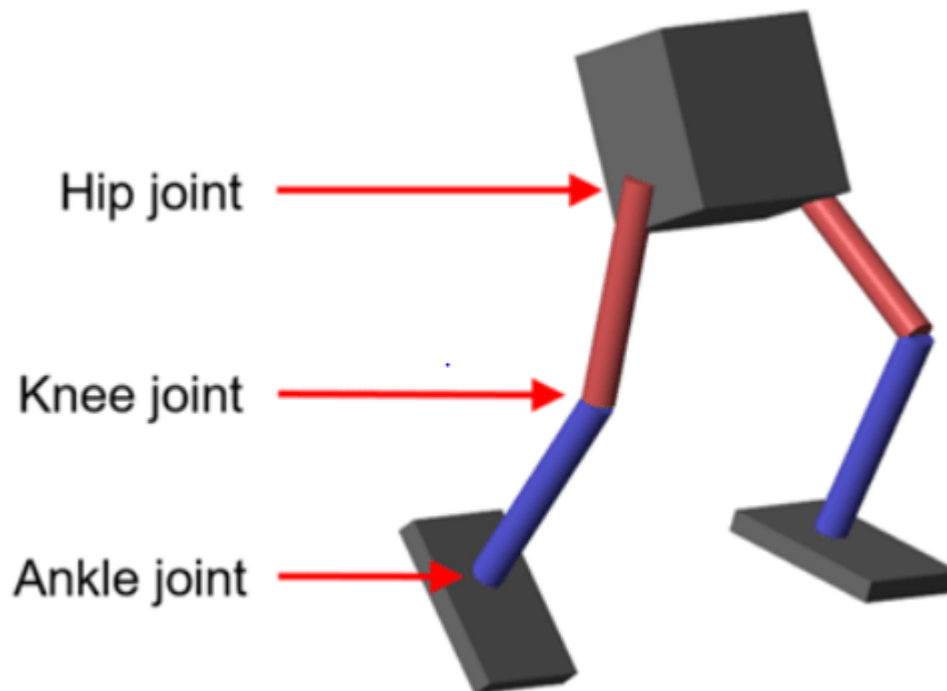
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Biped Robot Model

The reinforcement learning environment for this example is a biped robot. The training goal is to make the robot walk in a straight line using minimal control effort.



Load the parameters of the model into the MATLAB® workspace using the `robotParametersRL` Script provided in the example folder.

```
robotParametersRL
```

Open the Simulink® model.

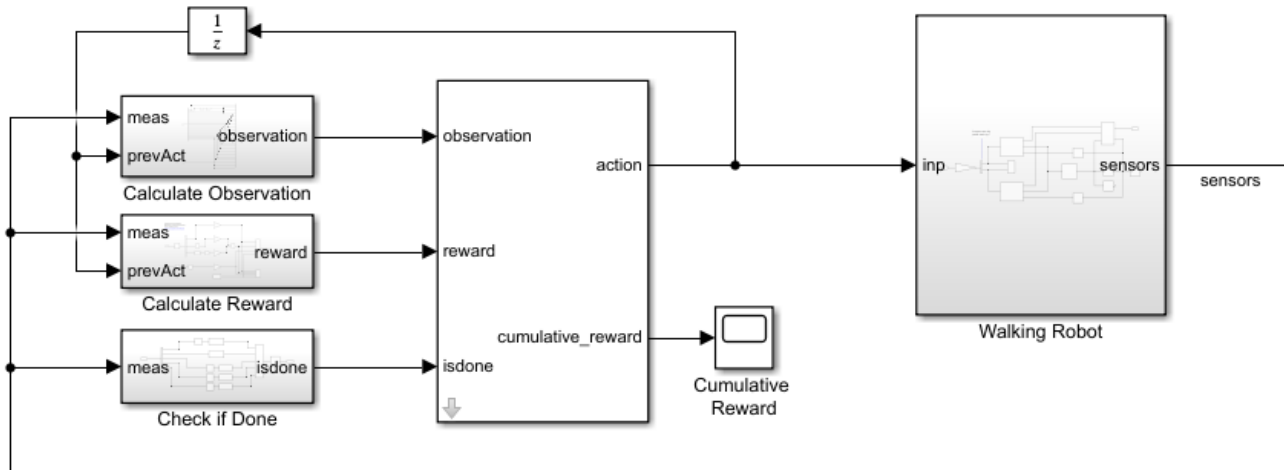
```
mdl = "rlWalkingBipedRobot";  
open_system(mdl)
```

Walking Robot: Reinforcement Learning (2-D)

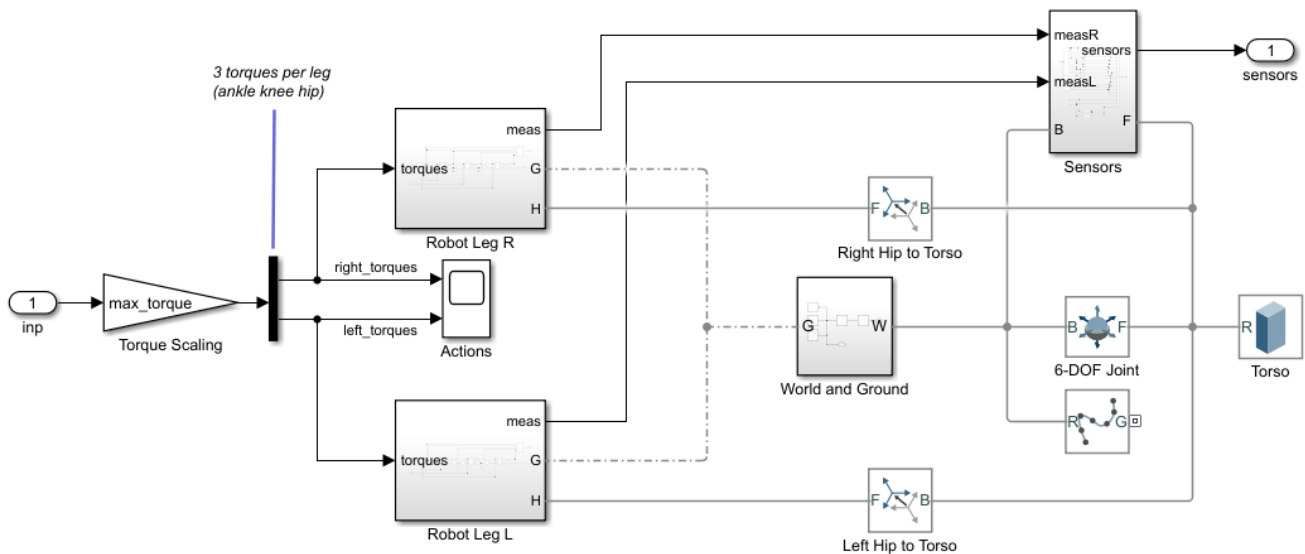
Copyright 2020 The MathWorks, Inc.

[Enable animation](#)

[Disable animation](#)



The robot is modeled using Simscape™ Multibody™.



In this model:

- In the neutral 0 rad position, both of the legs are straight and the ankles are flat.
- The foot contact is modeled using the Spatial Contact Force (Simscape Multibody) block.
- The agent can control 3 individual joints (ankle, knee, and hip) on both legs of the robot by applying torque signals from -3 to 3 N·m. The actual computed action signals are normalized between -1 and 1.

The environment provides the following 29 observations to the agent.

- Y (lateral) and Z (vertical) translations of the torso center of mass. The translation in the Z direction is normalized to a similar range as the other observations.
- X (forward), Y (lateral), and Z (vertical) translation velocities.
- Yaw, pitch, and roll angles of the torso.
- Yaw, pitch, and roll angular velocities of the torso.
- Angular positions and velocities of the three joints (ankle, knee, hip) on both legs.
- Action values from the previous time step.

The episode terminates if either of the following conditions occur.

- The robot torso center of mass is less than 0.1 m in the Z direction (the robot falls) or more than 1 m in the either Y direction (the robot moves too far to the side).
- The absolute value of either the roll, pitch, or yaw is greater than 0.7854 rad.

The following reward function r_t , which is provided at every time step is inspired by [2].

$$r_t = v_x - 3y^2 - 50\hat{z}^2 + 25\frac{T_s}{T_f} - 0.02\sum_i u_{t-1}^i{}^2$$

Here:

- v_x is the translation velocity in the X direction (forward toward goal) of the robot.
- y is the lateral translation displacement of the robot from the target straight line trajectory.
- $\hat{z}$ is the normalized vertical translation displacement of the robot center of mass.
- u_{t-1}^i is the torque from joint i from the previous time step.
- T_s is the sample time of the environment.
- T_f is the final simulation time of the environment.

This reward function encourages the agent to move forward by providing a positive reward for positive forward velocity. It also encourages the agent to avoid episode termination by providing a constant reward ($25\frac{T_s}{T_f}$) at every time step. The other terms in the reward function are penalties for substantial changes in lateral and vertical translations, and for the use of excess control effort.

Create Environment Object

Create the observation specification.

```
numObs = 29;
obsInfo = rlNumericSpec([numObs 1]);
obsInfo.Name = "observations";
```

Create the action specification.

```
numAct = 6;
actInfo = rlNumericSpec([numAct 1], LowerLimit=-1, UpperLimit=1);
actInfo.Name = "foot_torque";
```

Create the environment object for the walking robot model.

```
blk = mdl + "/RL Agent";
```

Specify the helper function `walkerResetFcn` as the reset function for the environment. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo);  
env.ResetFcn = @(in) walkerResetFcn(in, ...  
    upper_leg_length/100, ...  
    lower_leg_length/100, ...  
    h/100);
```

Select and Create Agent for Training

When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0,"twister");
```

This example provides the option to train the robot using either a DDPG or TD3 agent. To simulate the robot with the agent of your choice, set the `AgentSelection` flag accordingly.

```
AgentSelection = ;  
switch AgentSelection  
    case "DDPG"  
        agent = createDDPGAgent(numObs,obsInfo,numAct,actInfo,Ts);  
    case "TD3"  
        agent = createTD3Agent(numObs,obsInfo,numAct,actInfo,Ts);  
    otherwise  
        disp("Assign AgentSelection to DDPG or TD3")  
end
```

The `createDDPGAgent` and `createTD3Agent` helper functions (provided in the example folder) perform the following actions.

- Create actor and critic networks.
- Specify options for actors and critics.
- Create actors and critics using the networks and options previously created.
- Configure agent specific options.
- Create the agent.

DDPG Agent

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor, and a parameterized Q-value function approximator to estimate the value of the policy. Use neural networks to model both the policy and the Q-value function. The actor and critic networks for this example are inspired by [1].

For details on how the DDPG agent is created, see the `createDDPGAgent` helper function. For information on configuring DDPG agent options, see `rlDDPGAgentOptions`.

For more information on creating a deep neural network value function representation, see “Create Actors, Critics, and Policy Objects” on page 4-2. For an example that creates neural networks for DDPG agents, see “Compare DDPG Agent to LQR Controller” on page 7-285.

TD3 Agent

The critic of a DDPG agent can overestimate the Q value. Because the agent uses the Q value to update its policy (actor), the resultant policy can be suboptimal and can accumulate training errors that can lead to divergent behavior. The TD3 algorithm is an extension of DDPG with improvements that make it more robust by preventing overestimation of Q values [3].

- Two critic networks — TD3 agents learn two critic networks independently and use the minimum value function estimate to update the actor (policy). Doing so prevents accumulation of error in subsequent steps and overestimation of Q values.
- Addition of target policy noise — Adding clipped noise to value functions smooths out Q function values over similar actions. Doing so prevents learning an incorrect sharp peak of noisy value estimate.
- Delayed policy and target updates — For a TD3 agent, delaying the actor network update allows more time for the Q function to reduce error (get closer to the required target) before updating the policy. Doing so prevents variance in value estimates and results in a higher quality policy update. This example sets the policy and target frequency options to 1 as to better compare the impact of two critic networks and target policy noise between the DDPG and TD3 agents.

The structure of the actor and critic networks used for this agent are the same as the ones used for DDPG agent. For details on the creation of the TD3 agent, see the `createTD3Agent` helper function. For information on configuring TD3 agent options, see `rlTD3AgentOptions`.

Get the actor and critic neural networks from the agent.

```
actor = getActor(agent);
critic = getCritic(agent);
actorNet = getModel(actor);
criticNet = getModel(critic(1));
```

View a summary and plot of the actor neural network.

```
summary(actorNet);

    Initialized: true

    Number of learnables: 134.1k

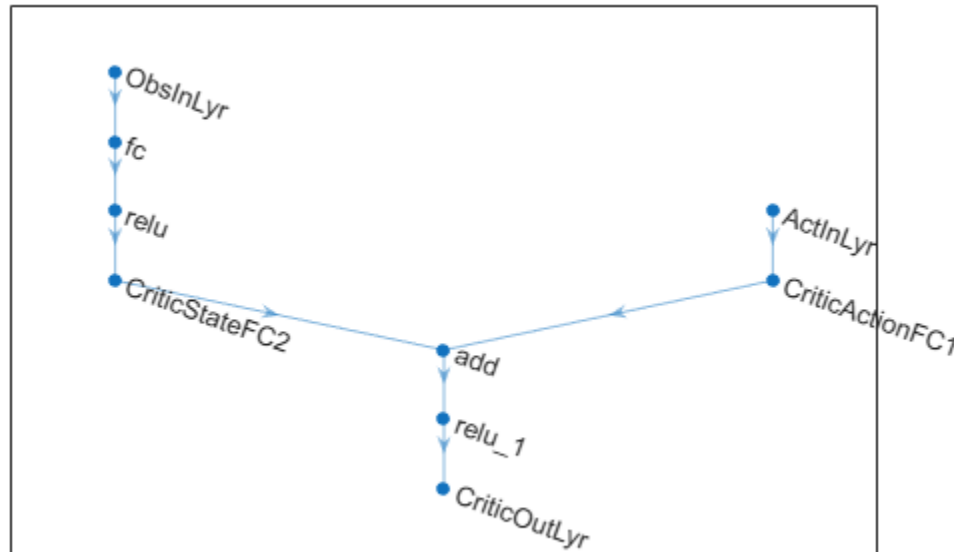
    Inputs:
      1   'input'   29 features

plot(actorNet);
```



View a summary and plot of the critic neural network.

```
summary(criticNet);  
  Initialized: true  
  Number of learnables: 134.7k  
  Inputs:  
    1  'ObsInLyr'  29 features  
    2  'ActInLyr'   6 features  
plot(criticNet);
```



Specify Training Options and Train Agent

For this example, the training options for the DDPG and TD3 agents are the same.

- Run each training session for 3000 episodes with each episode lasting at most `maxSteps` time steps.
- Terminate the training only when it reaches the maximum number of episodes (`maxEpisodes`) by setting `StopTrainingCriteria` to "none". Doing so allows the comparison of the learning curves for multiple agents over the entire training session.

For more information and additional options, see `rlTrainingOptions`.

```

maxEpisodes = 3000;
maxSteps = floor(Tf/Ts);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxEpisodes, ...
    MaxStepsPerEpisode=maxSteps, ...
    ScoreAveragingWindowLength=250, ...
    StopTrainingCriteria="none", ...
    SimulationStorageType="file", ...
    SaveSimulationDirectory=AgentSelection+"Sims");
  
```

To train the agent in parallel, specify the following training options. Training in parallel requires Parallel Computing Toolbox™. If you do not have Parallel Computing Toolbox software installed, set `UseParallel` to `false`.

- Set the `UseParallel` option to `true`.
- Train the agent in parallel asynchronously.

```

trainOpts.UseParallel = true;
trainOpts.ParallelizationOptions.Mode = async;
  
```

In parallel training, workers simulate the agent's policy with the environment and store experiences in the replay memory. When workers operate asynchronously the order of stored experiences might not be deterministic and can ultimately make the training results different. To maximize the reproducibility likelihood:

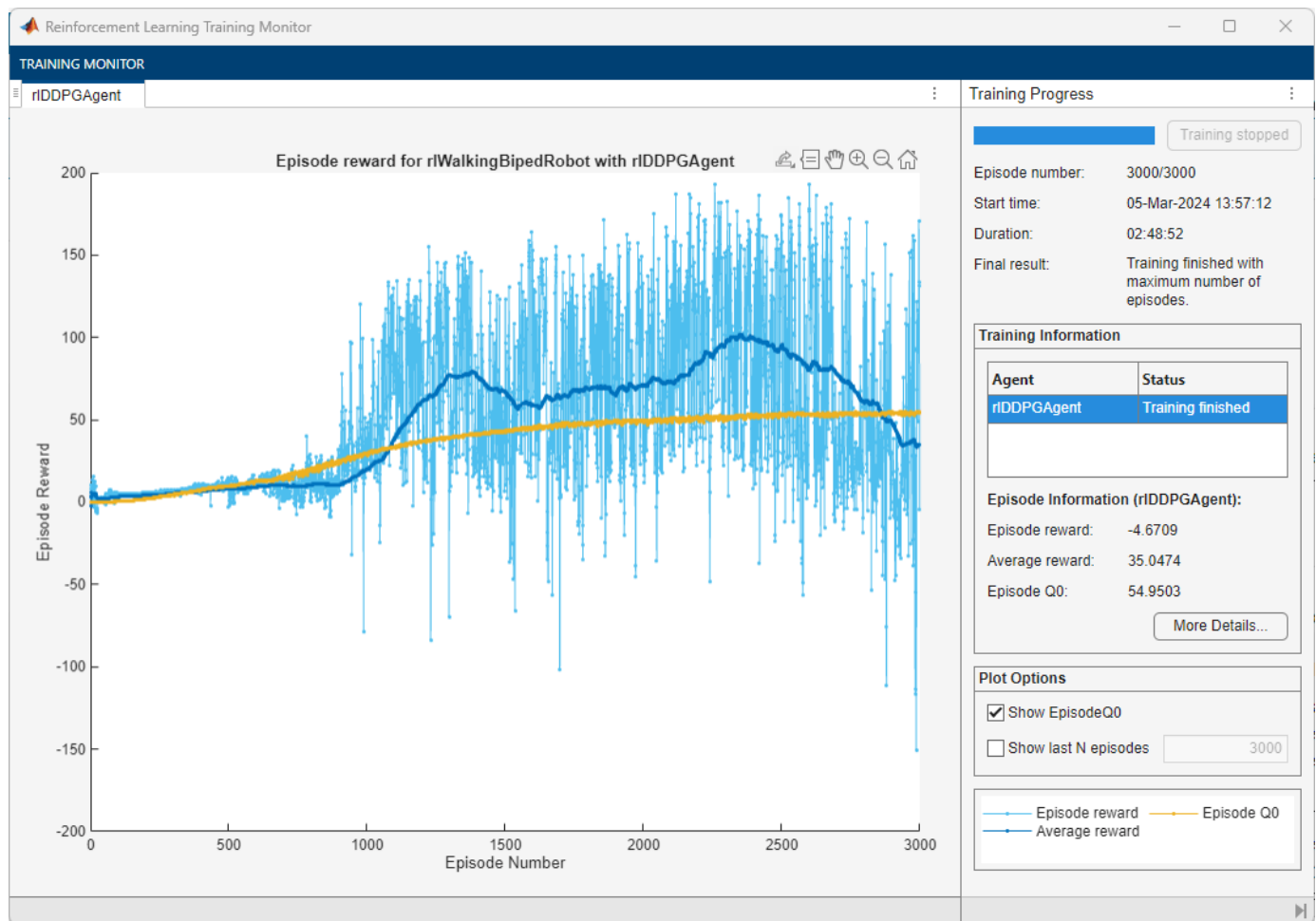
- Initialize the parallel pool with the same number of parallel workers every time you run the code. For information on specifying the pool size see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox).
- Use synchronous parallel training by setting `trainOpts.ParallelizationOptions.Mode` to “sync”.
- Assign a random seed to each parallel worker using `trainOpts.ParallelizationOptions.WorkerRandomSeeds`. The default value of -1 assigns a unique random seed to each parallel worker.

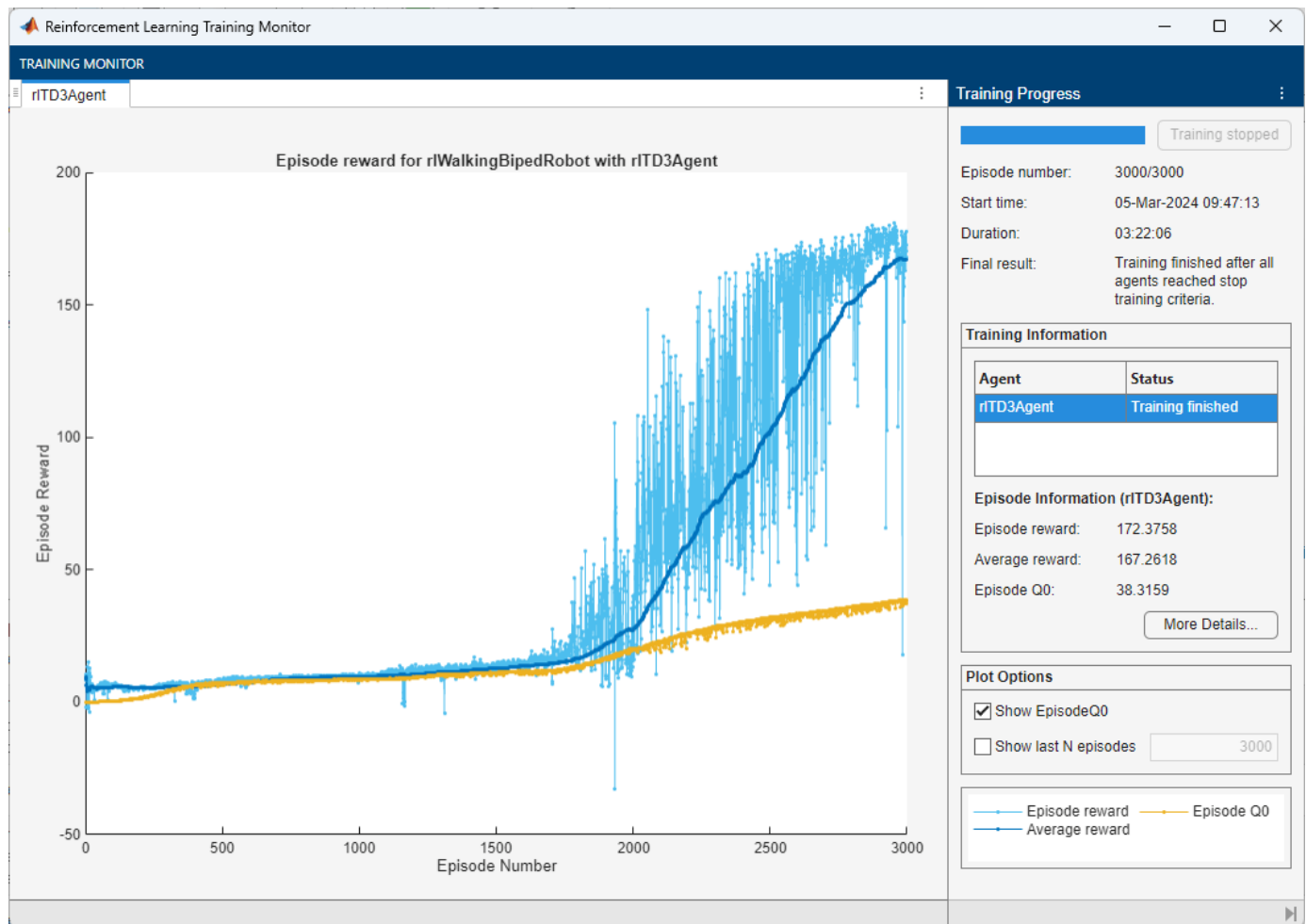
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. This process is computationally intensive and takes several hours to complete for each agent. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`. Due to randomness in the parallel training, you can expect different training results from the plots that follow. The pretrained agents were trained in parallel using four workers.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainingStats = train(agent, env, trainOpts);  
else  
    % Load a pretrained agent for the selected agent type.  
    if strcmp(AgentSelection, "DDPG")  
        load(fullfile("DDPGAgent", "run1.mat"), "agent")  
    else  
        load(fullfile("TD3Agent", "run1.mat"), "agent")  
    end  
end
```





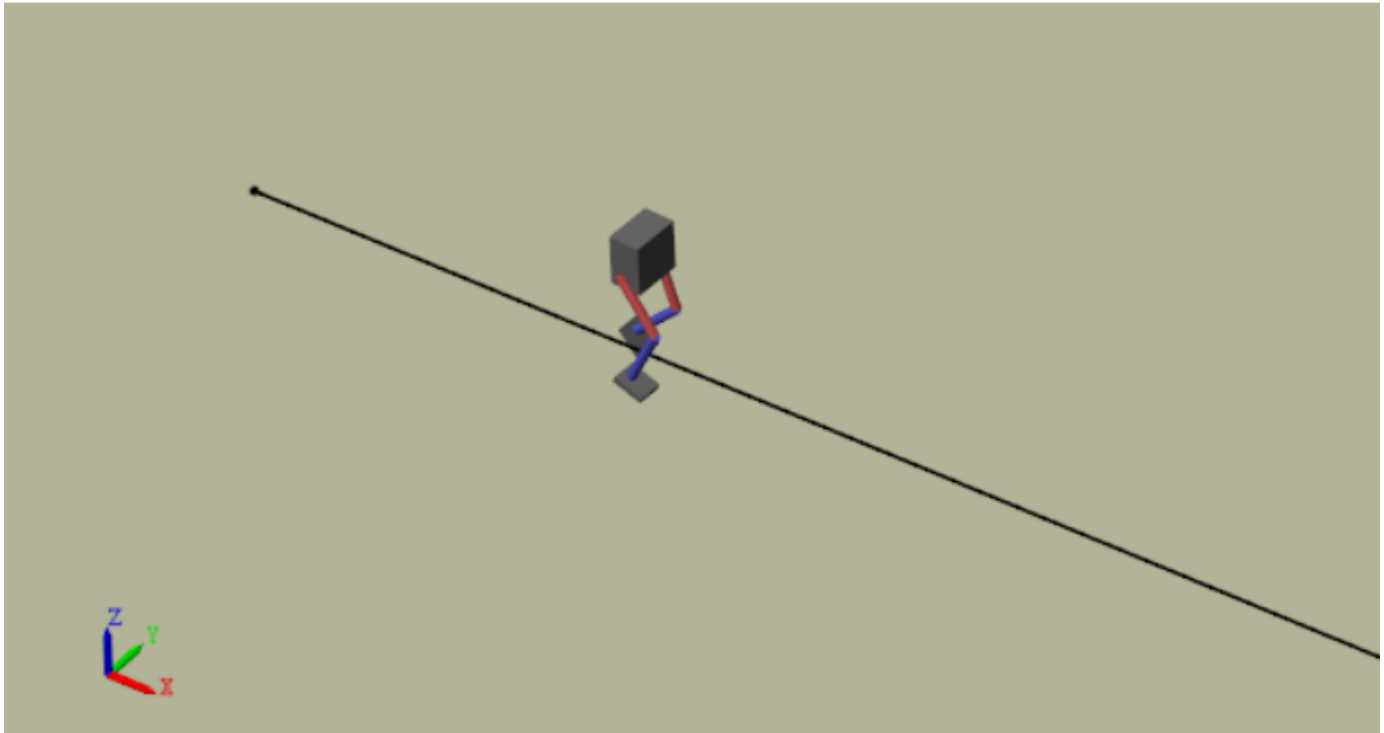
Simulate Trained Agents

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To validate the performance of the trained agent, simulate it within the biped robot environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=maxSteps);
experience = sim(env, agent, simOptions);
```



Compare Agent Performance

For the following agent comparison, each agent was trained five times using a different random seed each time. Due to the random exploration noise and the randomness in the parallel training, the learning curve for each run is different. Because the training of agents for multiple runs can take several days to complete, this comparison uses pretrained agents.

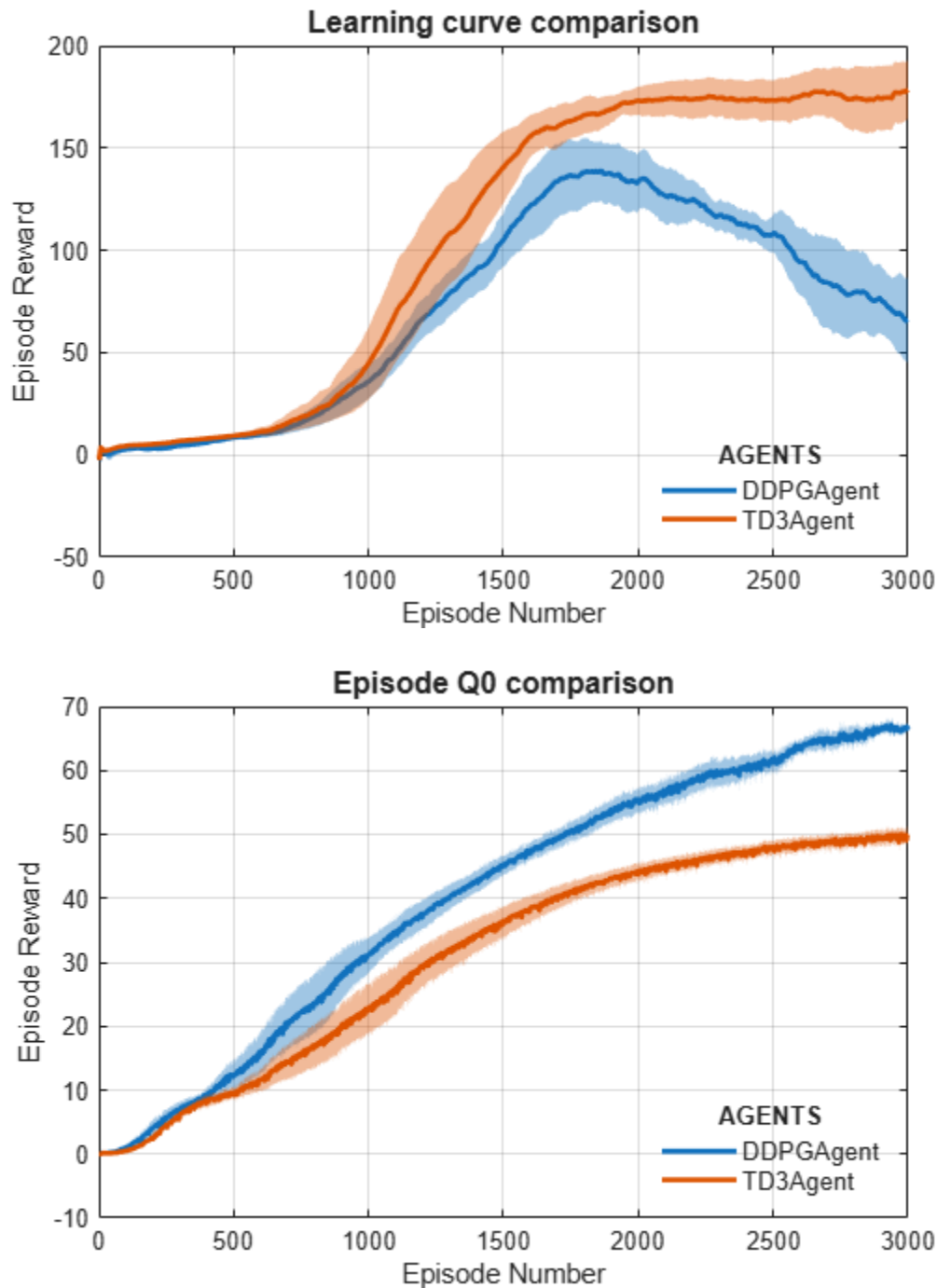
For the DDPG and TD3 agents, plot the average and standard deviation of the episode reward (top plot) and the episode Q0 value (bottom plot). The episode Q0 value is the critic estimate of the discounted long-term reward at the start of each episode given the initial observation of the environment. For a well-designed critic, the episode Q0 value approaches the true discounted long-term reward.

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Compare the performance of the pretrained agents using data stored in the `DDPGAgent` and `TD3Agent` folders. To plot the performance metrics, specify the folder names as input arguments to the `comparePerformance` function. The comparison data and the function are found in the example directory.

```
comparePerformance("DDPGAgent", "TD3Agent")
```



Based on the Learning curve comparison plot:

- The DDPG agent can pick up learning faster than the TD3 agent in some cases, however the DDPG agent tends to have inconsistent long-term training performance.
- The TD3 agent consistently learns satisfactory walking policies for the environment and tends to show better convergence properties compared to DDPG.

Based on the Episode Q0 comparison plot:

- The TD3 agent more conservatively estimates the discounted long-term reward compared to DDPG. This is because TD3 estimates the discounted long-term reward as the minimum of the values approximated from the two critics.

For another example on how to train a humanoid robot to walk using a DDPG agent, see “Train Humanoid Walker” (Simscape Multibody). For an example on how to train a quadruped robot to walk using a DDPG agent, see “Quadruped Robot Locomotion Using DDPG Agent” on page 8-158.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

References

- [1] Lillicrap, Timothy P, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous Control with Deep Reinforcement Learning." Preprint, submitted July 5, 2019. <https://arxiv.org/abs/1509.02971>.
- [2] Heess, Nicolas, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, et al. "Emergence of Locomotion Behaviours in Rich Environments." Preprint, submitted July 10, 2017. <https://arxiv.org/abs/1707.02286>.
- [3] Fujimoto, Scott, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods." Preprint, submitted October 22, 2018. <https://arxiv.org/abs/1802.09477>.

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDDPGAgent` | `rlDDPGAgentOptions` | `rlTD3Agent` | `rlTD3AgentOptions` | `rlQValueFunction` | `rlContinuousDeterministicActor` | `rlOptimizerOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

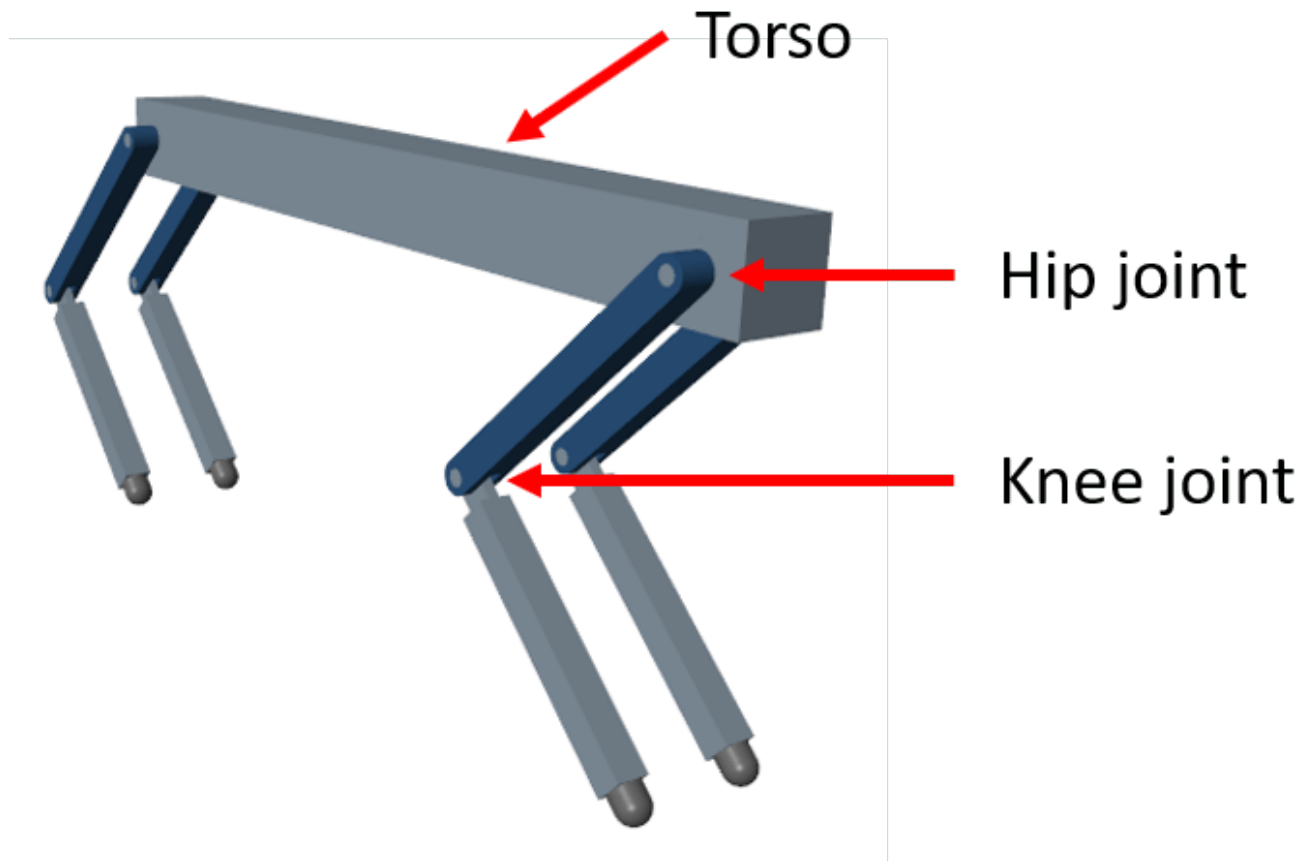
- “Train AC Agent to Balance Discrete Cart-Pole Using Parallel Computing” on page 5-157
- “Train DQN Agent for Lane Keeping Assist Using Parallel Computing” on page 8-134
- “Quadruped Robot Locomotion Using DDPG Agent” on page 8-158
- “Add Safety Constraint to Simulate Two-Link Robot with SAC Agent” on page 8-168

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Agents Using Parallel Computing and GPUs” on page 5-151

Quadruped Robot Locomotion Using DDPG Agent

This example shows how to train a quadruped robot to walk using a deep deterministic policy gradient (DDPG) agent. The robot in this example is modeled using Simscape™ Multibody™. For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.



Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Load the necessary parameters into the base workspace in MATLAB® using the Script `initializeRobotParameters` provided with the example.

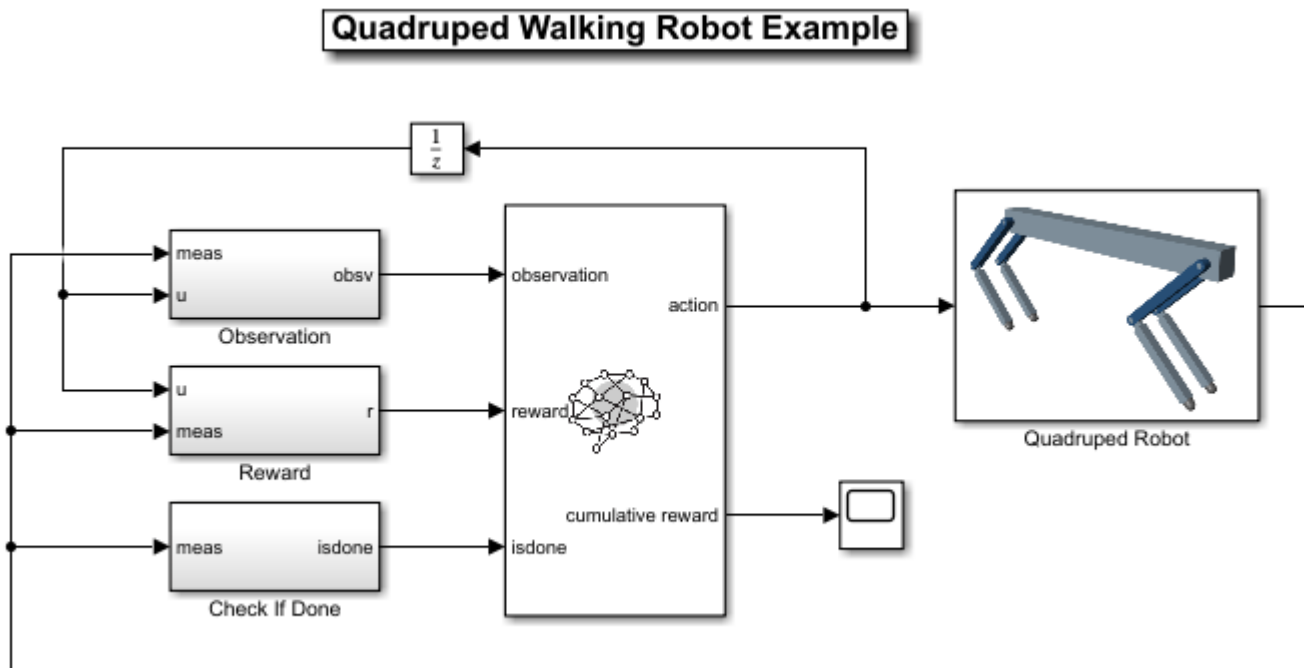
```
initializeRobotParameters;
```

Quadruped Robot Model

The environment for this example is a quadruped robot, and the training goal is to make the robot walk in a straight line using minimal control effort.

Open the model.

```
mdl = "rlQuadrupedRobot";  
open_system(mdl)
```



The robot is modeled using Simscape™ Multibody™ with its main structural components consisting of four legs and a torso. The legs are connected to the torso through revolute joints that enable rotation of the legs with respect to the torso. The joints are actuated by torque signals provided by the RL Agent.

Observations

The robot environment provides 44 observations to the agent, each normalized between -1 and 1. These observations are:

- Y (vertical) and Y (lateral) position of the torso center of mass
- Quaternion representing the orientation of the torso
- X (forward), Y (vertical), and Z (lateral) velocities of the torso at the center of mass
- Roll, pitch, and yaw rates of the torso

- Angular positions and velocities of the hip and knee joints for each leg
- Normal and friction force due to ground contact for each leg
- Action values (torque for each joint) from the previous time step

For all four legs, the initial values for the hip and knee joint angles are set to -0.8234 and 1.6468 radians, respectively. The neutral positions of the joints are set at 0 radian. The legs are in neutral position when they are stretched to their maximum and are aligned perpendicularly to the ground.

Actions

The agent generates eight actions normalized between -1 and 1 . After multiplying with a scaling factor, these correspond to the eight joint torque signals for the revolute joints. The overall joint torque bounds are ± 10 N·m for each joint.

Reward

The following reward is provided to the agent at each time step during training. This reward function encourages the agent to move forward by providing a positive reward for positive forward velocity. It also encourages the agent to avoid early termination by providing a constant reward ($25T_s/T_f$) at each time step. The remaining terms in the reward function are penalties that discourage unwanted states, such as large deviations from the desired height and orientation or the use of excessive joint torques.

$$r_t = v_x + 25 \frac{T_s}{T_f} - 50\hat{y}^2 - 20\theta^2 - 0.02 \sum_i u_{t-1}^i{}^2$$

where

- v_x is the velocity of the torso's center of mass in the x-direction.
- T_s and T_f are the sample time and final simulation time of the environment, respectively.
- $\hat{y}$ is the scaled height error of the torso's center of mass from the desired height of 0.75 m.
- θ is the pitch angle of the torso.
- u_{t-1}^i is the action value for joint i from the previous time step.

Episode Termination

During training or simulation, the episode terminates if any of the following situations occur.

- The height of the torso center of mass from the ground is below 0.5 m (fallen).
- The head or tail of the torso is below the ground.
- Any knee joint is below the ground.
- Roll, pitch, or yaw angles are outside bounds (± 0.1745 , ± 0.1745 , and ± 0.3491 radians, respectively).

Create Environment Object

Specify the parameters for the observation set.

```
numObs = 44;
obsInfo = rlNumericSpec([numObs 1]);
obsInfo.Name = "observations";
```

Specify the parameters for the action set.

```
numAct = 8;
actInfo = rlNumericSpec([numAct 1],LowerLimit=-1,UpperLimit=1);
actInfo.Name = "torque";
```

Create the environment using the reinforcement learning model.

```
blk = mdl + "/RL Agent";
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo);
```

During training, the function `quadrapedResetFcn` on page 8-165 provided at the end of the example introduces random deviations into the initial joint angles and angular velocities.

```
env.ResetFcn = @quadrapedResetFcn;
```

Create DDPG Agent Object

DDPG agents use a parameterized Q-value function approximator as a critic, to estimate the value of the policy. They also use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor.

For more information on creating a deep neural network value function, see “Create Actors, Critics, and Policy Objects” on page 4-2. For an example that creates neural networks for DDPG agents, see “Compare DDPG Agent to LQR Controller” on page 7-285.

First specify the following options for training the agent. For more information, see `rlDDPGAgentOptions`.

- Specify a capacity of 1e6 for the experience buffer to store a diverse set of experiences.
- Specify the `MaxMiniBatchPerEpoch` value to perform a maximum of 200 gradient step updates for each learning iteration.
- Specify a mini-batch size of 256 for learning. Smaller mini-batches are computationally efficient but might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.
- Specify a learning rate of 1e-3 for the agent's actor and critic optimizer algorithms. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Specify a gradient threshold value of 1.0 to clip the computed gradients to enhance the stability of the learning algorithm.
- Specify a standard deviation of 0.1 and mean attraction coefficient of 1.0 to improve exploration of the agent's action space.
- Specify a sample time `Ts=0.025s`.

```
% create the agent options object
agentOptions = rlDDPGAgentOptions();

% specify the options
agentOptions.SampleTime = Ts;
agentOptions.MiniBatchSize = 256;
agentOptions.ExperienceBufferLength = 1e6;
agentOptions.MaxMiniBatchPerEpoch = 200;

% optimizer options
```

```
agentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
agentOptions.ActorOptimizerOptions.GradientThreshold = 1;  
agentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
agentOptions.CriticOptimizerOptions.GradientThreshold = 1;
```

```
% exploration options
```

```
agentOptions.NoiseOptions.StandardDeviation = 0.1;  
agentOptions.NoiseOptions.MeanAttractionConstant = 1.0;
```

When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Create the `rlDDPGAgent` object for the agent with 256 hidden units per layer.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=256);  
agent = rlDDPGAgent(obsInfo, actInfo, initOpts, agentOptions);
```

Specify Training Options

To train the agent, first specify the following training options:

- Run each training episode for a maximum of 10,000 episodes, with each episode lasting a maximum of `maxSteps` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option).
- Stop the training when the greedy policy evaluation exceeds 300.

```
trainOpts = rlTrainingOptions( ...  
    MaxEpisodes=10000, ...  
    MaxStepsPerEpisode=floor(Tf/Ts), ...  
    ScoreAveragingWindowLength=250, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=300);
```

To train the agent in parallel, specify the following training options. Training in parallel requires Parallel Computing Toolbox™ software. If you do not have Parallel Computing Toolbox™ software installed, set `UseParallel` to `false`.

- Set the `UseParallel` option to `true`.
- Train the agent using asynchronous parallel workers.

```
trainOpts.UseParallel =  ;  
trainOpts.ParallelizationOptions.Mode =  ;
```

For more information, see `rlTrainingOptions`.

In parallel training, workers simulate the agent's policy with the environment and store experiences in the replay memory. When workers operate asynchronously the order of stored experiences might not be deterministic and can ultimately make the training results different. To maximize the reproducibility likelihood:

- Initialize the parallel pool with the same number of parallel workers every time you run the code. For information on specifying the pool size see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox).
- Use synchronous parallel training by setting `trainOpts.ParallelizationOptions.Mode` to “sync”.
- Assign a random seed to each parallel worker using `trainOpts.ParallelizationOptions.WorkerRandomSeeds`. The default value of -1 assigns a unique random seed to each parallel worker.

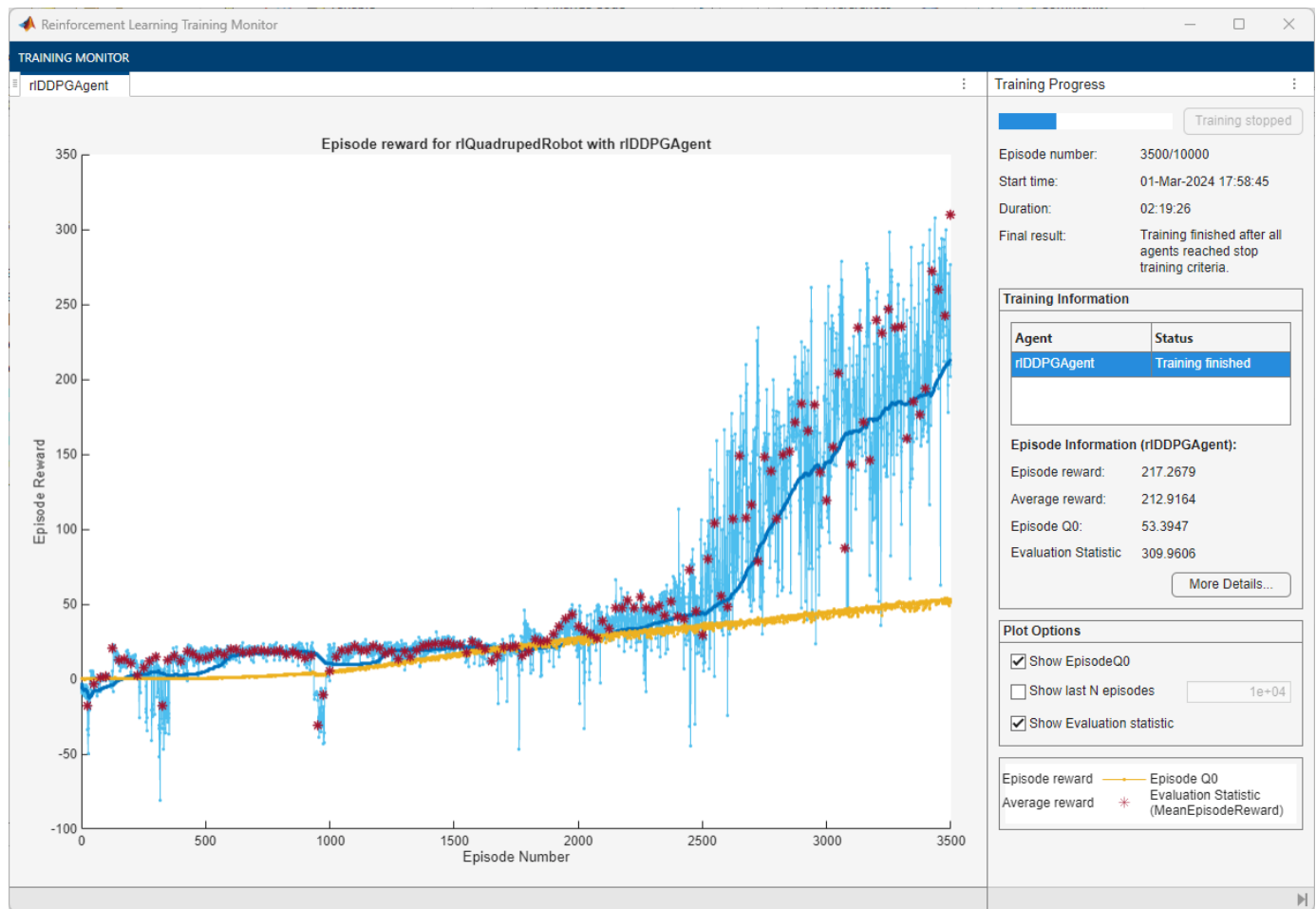
Train Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Due to the complexity of the robot model, this process is computationally intensive and takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true.

```
doTraining =  ;
if doTraining
    % Evaluate the greedy policy performance by taking the cumulative
    % reward mean over 5 simulations every 25 training episodes.
    evaluator = rlEvaluater(NumEpisodes=5, EvaluationFrequency=25);
    % Train the agent.
    result = train(agent, env, trainOpts, Evaluator=evaluator);
else
    % Load pretrained agent parameters for the example.
    load("rlQuadrapedAgentParams.mat", "params")
    setLearnableParameters(agent, params);
end
```



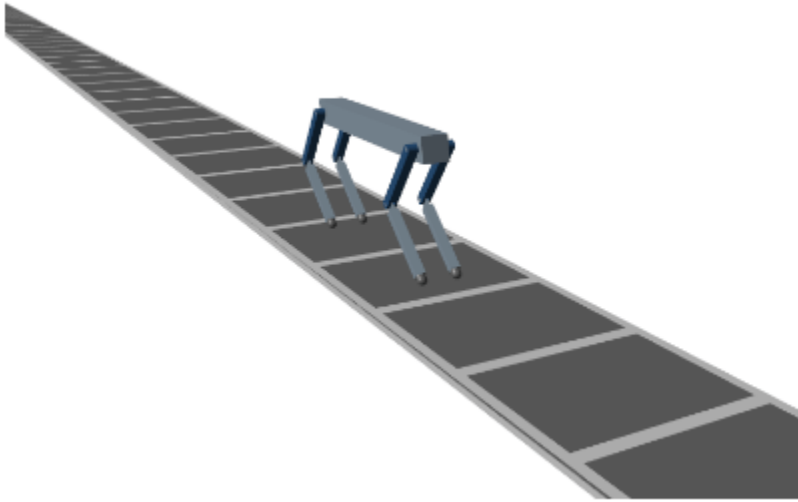
Simulate Trained Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To validate the performance of the trained agent, simulate it within the robot environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions(MaxSteps=floor(Tf/Ts));
experience = sim(env,agent,simOptions);
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Reset Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `reset` function uses the `setVariable` (Simulink) function to specify values in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = quadrupedResetFcn(in)
% Randomize initial conditions.

l1 = evalin('base','l1');
l2 = evalin('base','l2');

max_foot_disp_x = 0.1;
min_body_height = 0.7;
max_body_height = 0.8;
max_speed_x = 0.05;
max_speed_y = 0.025;

if rand < 0.5
    % Start from random initial conditions

    % Randomize height
    b = min_body_height + (max_body_height - min_body_height) * rand;

    % Randomize x-displacement of foot from hip joint
    a = -max_foot_disp_x + 2 * max_foot_disp_x * rand(1,4);

    % Calculate joint angles
    d2r = pi/180;
```

```
th_FL = d2r * quadrupedInverseKinematics(a(1),-b,l1,l2);
th_FR = d2r * quadrupedInverseKinematics(a(2),-b,l1,l2);
th_RL = d2r * quadrupedInverseKinematics(a(3),-b,l1,l2);
th_RR = d2r * quadrupedInverseKinematics(a(4),-b,l1,l2);

% Adjust for foot height
foot_height = ...
    0.05*l2*(1-sin(2*pi-(3*pi/2+sum([th_FL;th_FR;th_RL;th_RR],2))));
y_body = max(b) + max(foot_height);

% Randomize body velocities
vx = 2 * max_speed_x * (rand-0.5);
vy = 2 * max_speed_y * (rand-0.5);

else
    % Start from fixed initial conditions

    y_body = 0.7588;
    th_FL = [-0.8234 1.6468];
    th_FR = th_FL;
    th_RL = th_FL;
    th_RR = th_FL;
    vx = 0;
    vy = 0;
end

in = setVariable(in,'y_init',y_body);
in = setVariable(in,'init_ang_FL',th_FL);
in = setVariable(in,'init_ang_FR',th_FR);
in = setVariable(in,'init_ang_RL',th_RL);
in = setVariable(in,'init_ang_RR',th_RR);
in = setVariable(in,'vx_init',vx);
in = setVariable(in,'vy_init',vy);
end
```

References

[1] Heess, Nicolas, Dhruva TB, Srinivasan Sriram, Jay Lemmon, Josh Merel, Greg Wayne, Yuval Tassa, et al. 'Emergence of Locomotion Behaviours in Rich Environments'. *ArXiv:1707.02286 [Cs]*, 10 July 2017. <https://arxiv.org/abs/1707.02286>.

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlDDPGAgent | rlDDPGAgentOptions | rlQValueFunction |
rlContinuousDeterministicActor | rlTrainingOptions | rlSimulationOptions |
rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Add Safety Constraint to Simulate Two-Link Robot with SAC Agent” on page 8-168

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

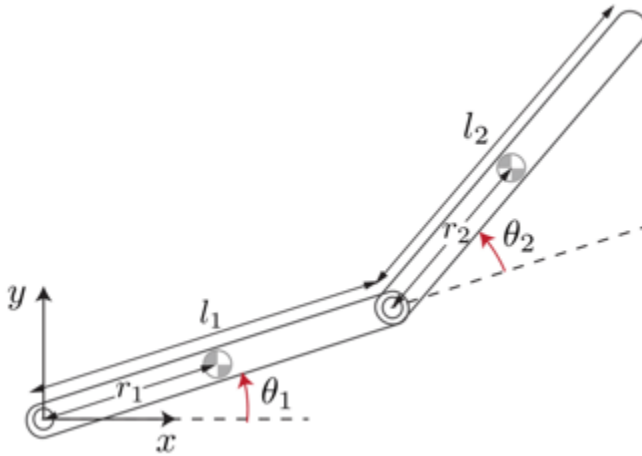
Add Safety Constraint to Simulate Two-Link Robot with SAC Agent

This example shows how to add safety constraints using control barrier functions to a “Soft Actor-Critic (SAC) Agent” on page 3-78 agent trained to control a two-link robotic arm. The two-Link robot nonlinear dynamics are based on [1] on page 8-180 and [2] on page 8-180. The two-link system is modeled in Simulink®.

Nonlinear Model of the Two-Link Robot

This example presents a controller design for a two link robot system. The control objective is to move the tip of the two-link robot from a starting point to a defined end point in the planar 2-D $\mathbf{x}\text{-}\mathbf{y}$ space. In this example, you train a reinforcement learning agent to control the two-link robot. You then impose state-space constraints on the second-arm tip position, and simulate the deployed reinforcement learning policy using these constraints.

This figure gives the schematics of the two-link planar robot.



The robot configuration is described by the joint angles θ_1 , θ_2 . The robot actuators apply the torques τ_1 and τ_2 at the joints to control the position of the tip. The state vector is

$$\zeta(t) = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1(t) \\ \dot{\theta}_2(t) \end{bmatrix}$$

The following equations describe the robot dynamics.

$$\begin{bmatrix} \alpha + 2\beta\cos(\theta_2) & \delta + \beta\cos(\theta_2) \\ \delta + \beta\cos(\theta_2) & \delta \end{bmatrix} \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} + \begin{bmatrix} -\beta\sin(\theta_2)\dot{\theta}_2 & \beta\sin(\theta_2)(\dot{\theta}_1 + \dot{\theta}_2) \\ \beta\sin(\theta_2)\dot{\theta}_1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

Here:

$$\alpha = I_{z1} + I_{z2} + m_1 r_1^2 + m_2 (l_1^2 + r_2^2)$$

$$\beta = m_2 l_1 r_2$$

$$\delta = I_{z2} + m_2 r_2^2$$

Initialize the variables used by the Simulink model.

Define the physical parameters.

```
m1 = 3;           % Mass of link 1, kg
m2 = 2;           % Mass of link 2, kg
L1 = 0.3;         % Length of link 1, meters
L2 = 0.3;         % Length of link 2, meters
```

Calculate the inertia terms.

```
r1 = L1/2;        % Length to centroid of link 1, meters.
r2 = L2/2;        % Length to centroid of link 2, meters.
Iz1 = m1*L1^2/12; % Moment of inertia of link 1, kg*m^2
Iz2 = m2*L2^2/12; % Moment of inertia of link 2, kg*m^2
```

Calculate other constants.

```
beta = m2*L1*r2;
delta = Iz2 + m2*r2^2;
alpha = Iz1 + Iz2 + m1*r1^2 + m2*(L1^2 + r2^2);
```

The X-Y coordinates of the end points are functions of the link angles $[\theta_1, \theta_2]$.

Link-1:

$$x_1 = l_1 \cos(\theta_1)$$

$$y_1 = l_1 \sin(\theta_1)$$

Link-2:

$$x_2 = x_1 + l_2 \cos(\theta_1 + \theta_2)$$

$$y_2 = y_1 + l_2 \sin(\theta_1 + \theta_2)$$

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at various stages. Fixing the random number stream at the beginning of various sections in the example code preserves the random number sequence in the section every time you run it, and increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

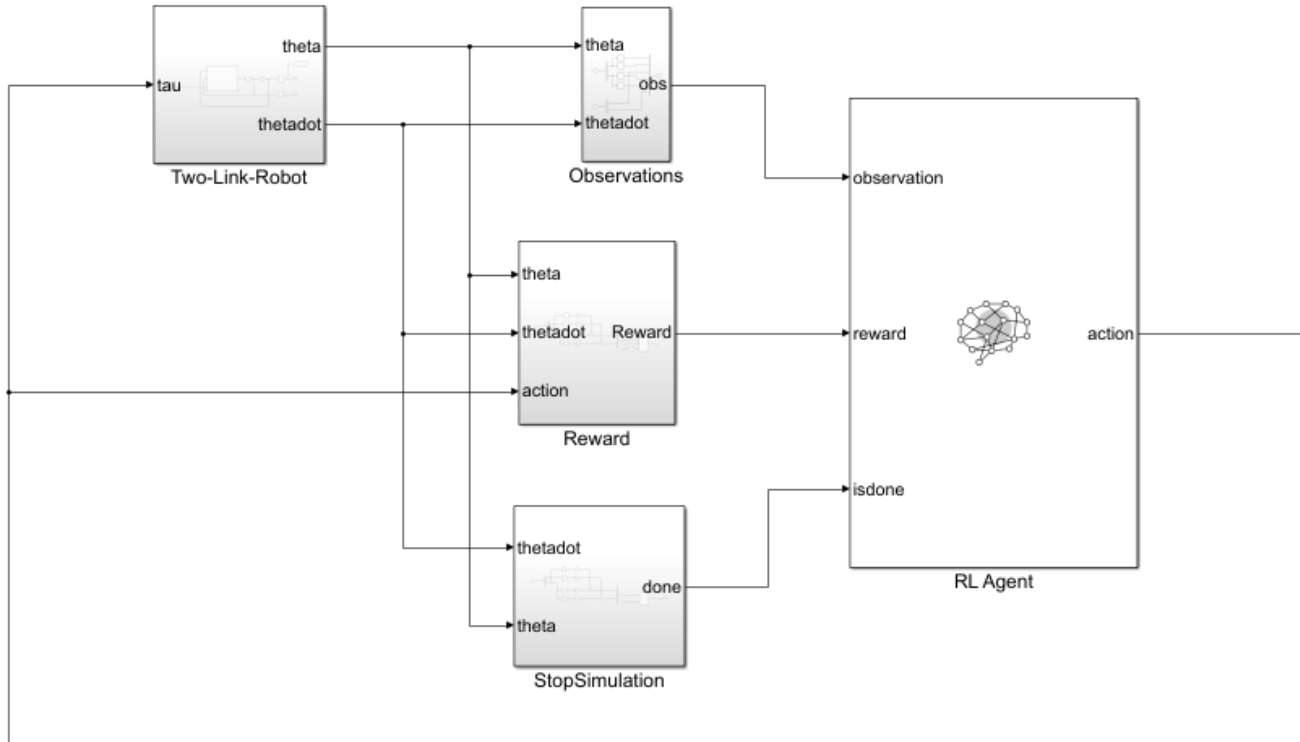
```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Simulink Model of the Two-Link Robot with RL Agent Block

Open the model.

```
mdl = 'TwoLinkRobot';
open_system(mdl);
```



In this model:

- The initial position of the tip of the two-link robot is $[-0.5 \ 0]$ and the final position to reach is $[0.5 \ 0.25]$.
- The agent actions, namely, the torques for each link, are bounded between -1 to 1 N·m.
- The observation from the environment is a vector containing the cosine, sine of the link angles, and their angular rates as follows $[\cos(\theta_1) \ \sin(\theta_1) \ \dot{\theta}_1 \ \cos(\theta_2) \ \sin(\theta_2) \ \dot{\theta}_2]$.
- The reward r_t , provided at each simulation time step, is designed as follows:

$$r_t = \begin{cases} \frac{1}{\|e_t\|^2} - 0.001\|\dot{\theta}_t\|^2 - 0.005\|u_{t-1}\|^2 & \text{if } \|e_t\|^2 > 0.05 \\ \frac{1}{\|e_t\|^2} - 0.001\|\dot{\theta}_t\|^2 - 0.005\|u_{t-1}\|^2 + 100 & \text{if } \|e_t\|^2 \leq 0.05 \end{cases}$$

where

- e_t is the position error of the tip of the two-link arms $e_t = X^* - X(t)$ and $X = [x_2(t), y_2(t)]$ and $X^* = [x_2^*, y_2^*]$.
- $\dot{\theta}_t$ is the vector containing the angular rate the two links.
- u_{t-1} is the control action from the previous time step.

Create Environment Object

Create the observation and action specifications for the environment.

The environment has a continuous state space of dimension [6,1].

```
ObservationInfo = rlNumericSpec([6 1]);
ObservationInfo.Name = 'Robot States';
ObservationInfo.Description = 'theta1,dtheta1,theta2,dtheta2';
```

The environment has a continuous action space of dimension [2,1] corresponding to torques applied to each link by the agent. The actions are bounded in the [-1, 1] range.

```
% Initialize Action settings
ActionInfo = rlNumericSpec([2 1]);
ActionInfo.LowerLimit = -1;
ActionInfo.UpperLimit = 1;
ActionInfo.Name = 'Link Torques';
```

Create the environment object.

```
env = rlSimulinkEnv("TwoLinkRobot", "TwoLinkRobot/RL Agent", ...
    ObservationInfo, ActionInfo);
```

Specify the agent sample time T_s and the simulation time T_f in seconds.

```
Ts = 0.01;
Tf = 5;
```

Create Soft Actor Critic Agent

When you create the agent, the actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create the agent options object.

```
agentOpts = rlSACAgentOptions;
```

To ensure that the RL Agent block in the environment executes every T_s seconds instead of the default setting of 1 second, set the `SampleTime` property of the agent.

```
agentOpts.SampleTime = Ts;
```

Set a lower learning rate to promote a smoother (though possibly slower) training.

```
agentOpts.ActorOptimizerOptions = rlOptimizerOptions(LearnRate= 1e-3);
agentOpts.CriticOptimizerOptions = rlOptimizerOptions(LearnRate= 1e-3);
```

Create a default `rlSACAgent` object using the environment specification and agent options object.

```
agent = rlSACAgent(ObservationInfo, ActionInfo,agentOpts);
```

Train SAC Agent

To train the agent, first specify the training options. For this example, use the following options.

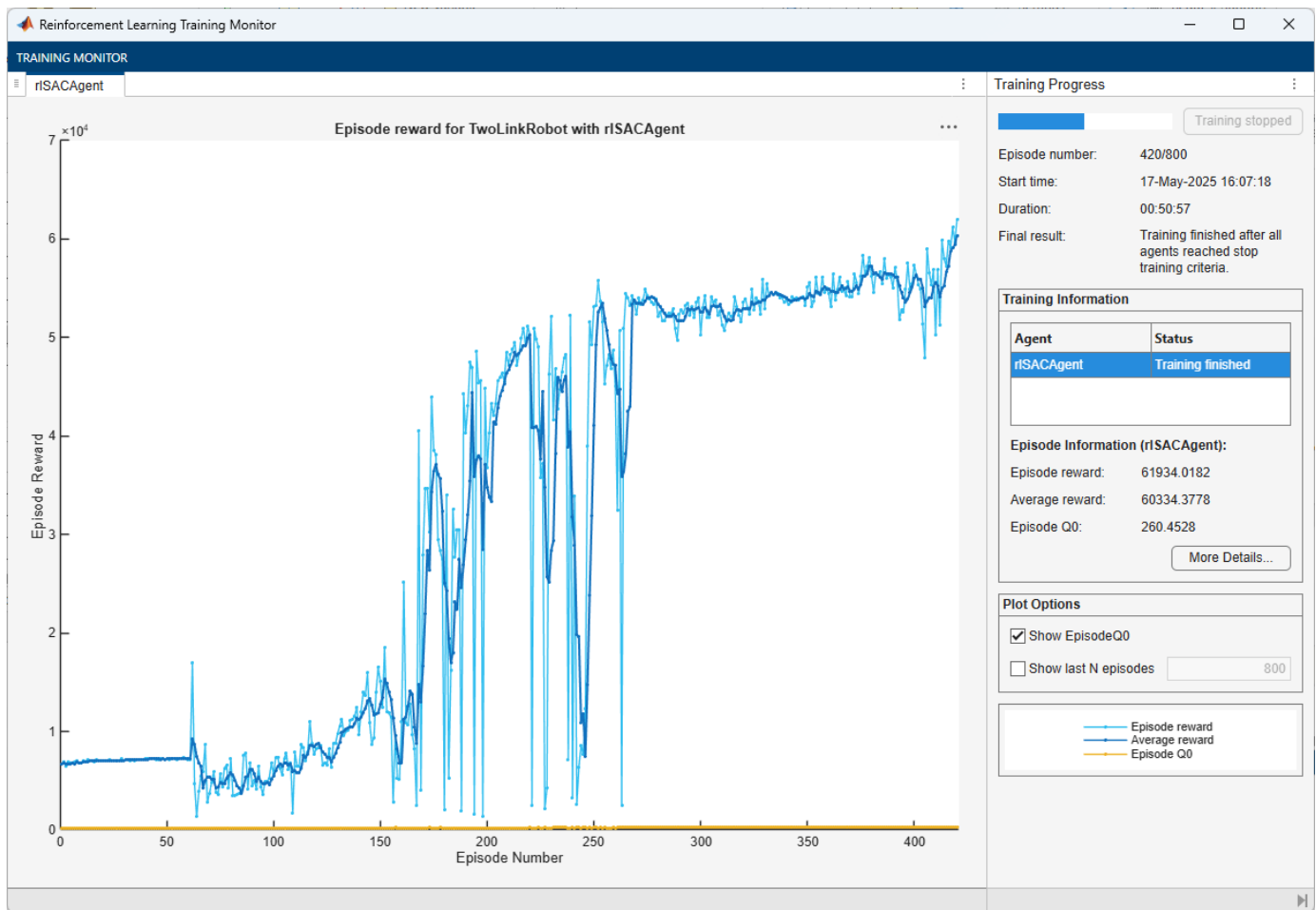
- Run training for a maximum of 5000 episodes, with each episode lasting at most `ceil(Tf/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Stop training when the agent receives an average cumulative reward greater than `6e4`. At this point, the agent is able to reach its goal using minimal control effort.
- Save a copy of the agent for each episode where the cumulative reward is greater than `6e4`.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 5000;  
maxsteps = ceil(Tf/Ts);  
trainOpts = rlTrainingOptions(...  
    MaxEpisodes=maxepisodes, ...  
    MaxStepsPerEpisode=maxsteps, ...  
    ScoreAveragingWindowLength=5, ...  
    Verbose=false, ...  
    Plots="training-progress",...  
    StopTrainingCriteria="AverageReward", ...  
    StopTrainingValue=6e4, ...  
    SaveAgentCriteria="AverageReward", ...  
    SaveAgentValue=6e4);
```

Train the agent using the `train` function. Training an agent is a computationally intensive process and can take a long time. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;  
if doTraining  
    % Train the agent.  
    trainingResults = train(agent,env,trainOpts);  
else  
    % Load the pretrained agent for the example.  
    load("TwoLinkRobotSAC.mat","agent")  
end
```

The training completes successfully after 420 episodes.

Simulate SAC Agent

For reproducibility, fix the seed for random number generation.

```
rng(0, "twister");
```

To validate the performance of the trained agent, simulate the two-link robot using the trained policy.

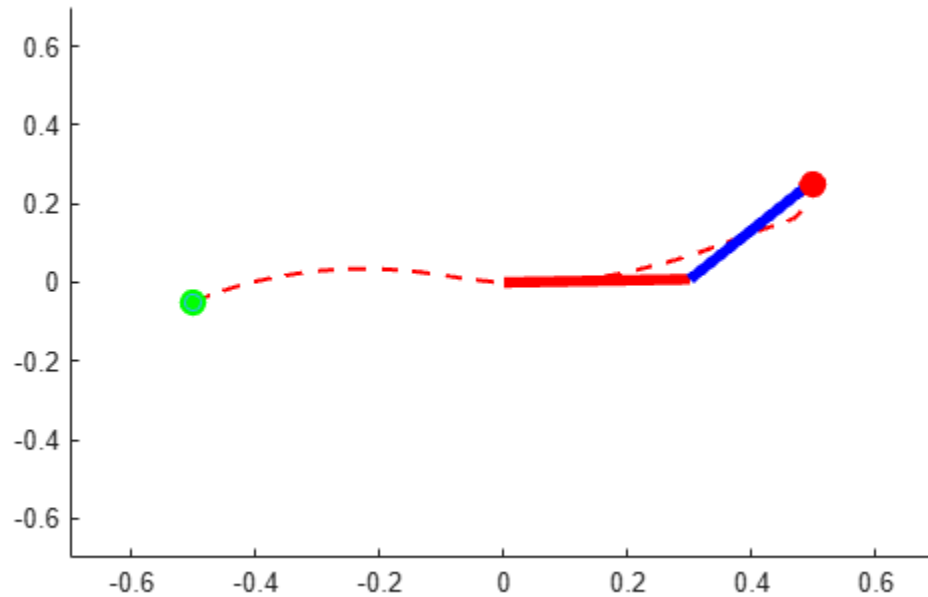
```
out = sim mdl;
```

Animate the two-link robot. The helper function `PlotTwoLinkArm` plots the robot configuration.

```
theta = squeeze(out.simout.Data);
blockVisible = false;
```

In the following plot, the green and red circles represent the initial and desired final position of the tip, respectively, while the dashed line represents the tip trajectory.

```
PlotTwoLinkArm(theta, 0.5, 0.25, L1, L2, blockVisible)
```

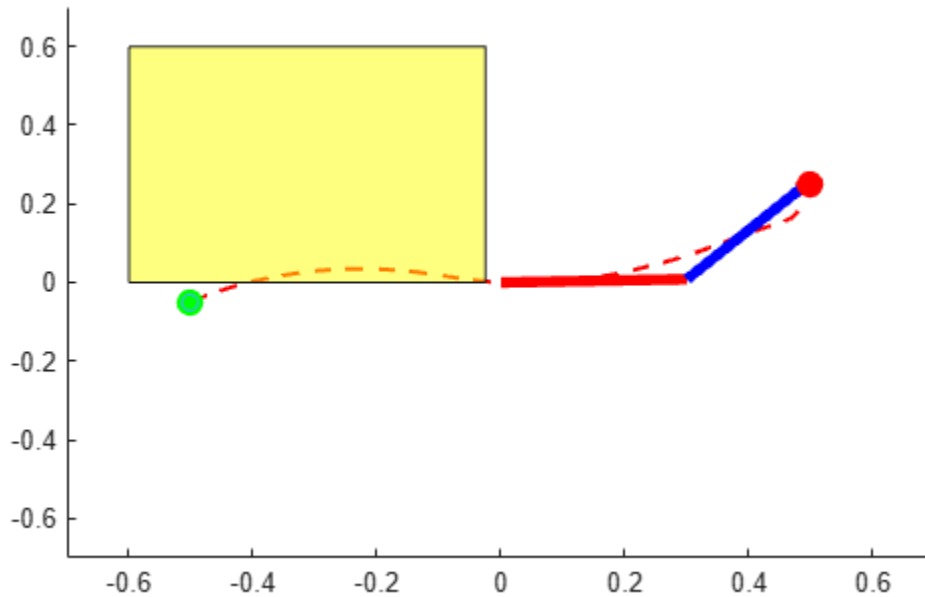


The figure shows that the agent successfully moves the robot so that the tip reaches the desired position.

Constrain Environment

The baseline SAC policy is trained in an unconstrained environment. However, here you add an obstacle in the workspace of the two-link robot. In the following picture, the yellow square represents the obstacle. As you can see, the baseline policy is not safe and the arm tip collides with the obstacle.

```
blockVisible = true;  
PlotTwoLinkArm(theta,0.5,0.25,L1,L2,blockVisible)
```



To solve this problem you need to impose a safety constraint so that the tip of second link stays clear of the obstacle while going from the start to the end position.

This can be done in two ways:

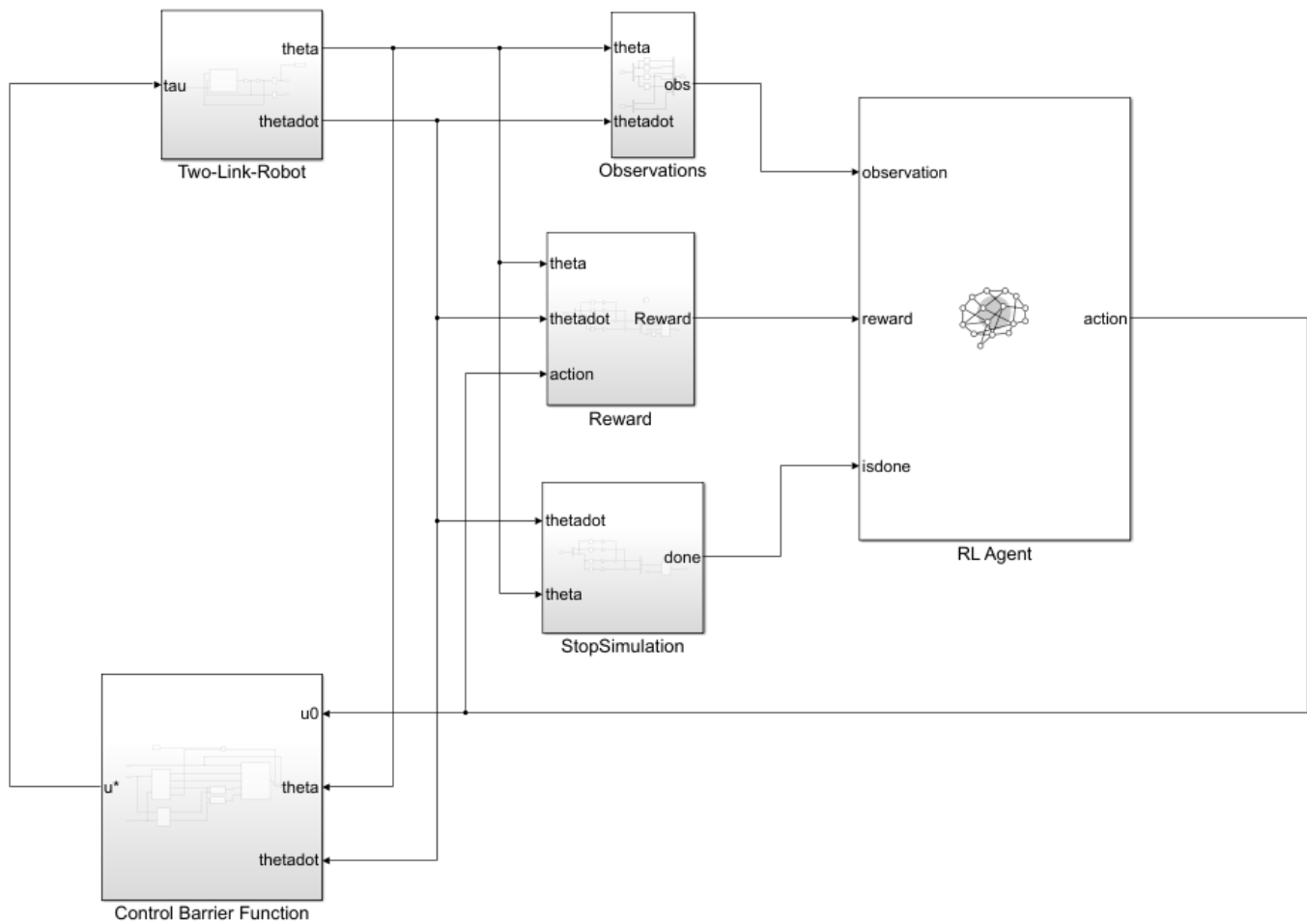
- 1 Modify the environment to provide a large penalty upon constraint violation and/or add a constraint enforcement. Then train a new agent within the new environment. This approach could produce a near-optimal policy, but it requires you to modify the environment, and retrain the agent which is computationally expensive. Furthermore, with stochastic agents, there is always a non zero probability that constraints can be violated even with a trained policy. Finally, it is hard to train an agent for changing constraints.
- 2 Add an external safety constraints like an high-order control barrier function (CBF) over the policy trained within the unconstrained environment. This approach is not general as requires plant information and it only applies to control-affine plants. Furthermore, it often results in a sub-optimal solution. However, it does not allow for constraint violation, it is computationally inexpensive, and can be easily modified for different or changing constraints.

In this example you use the second approach.

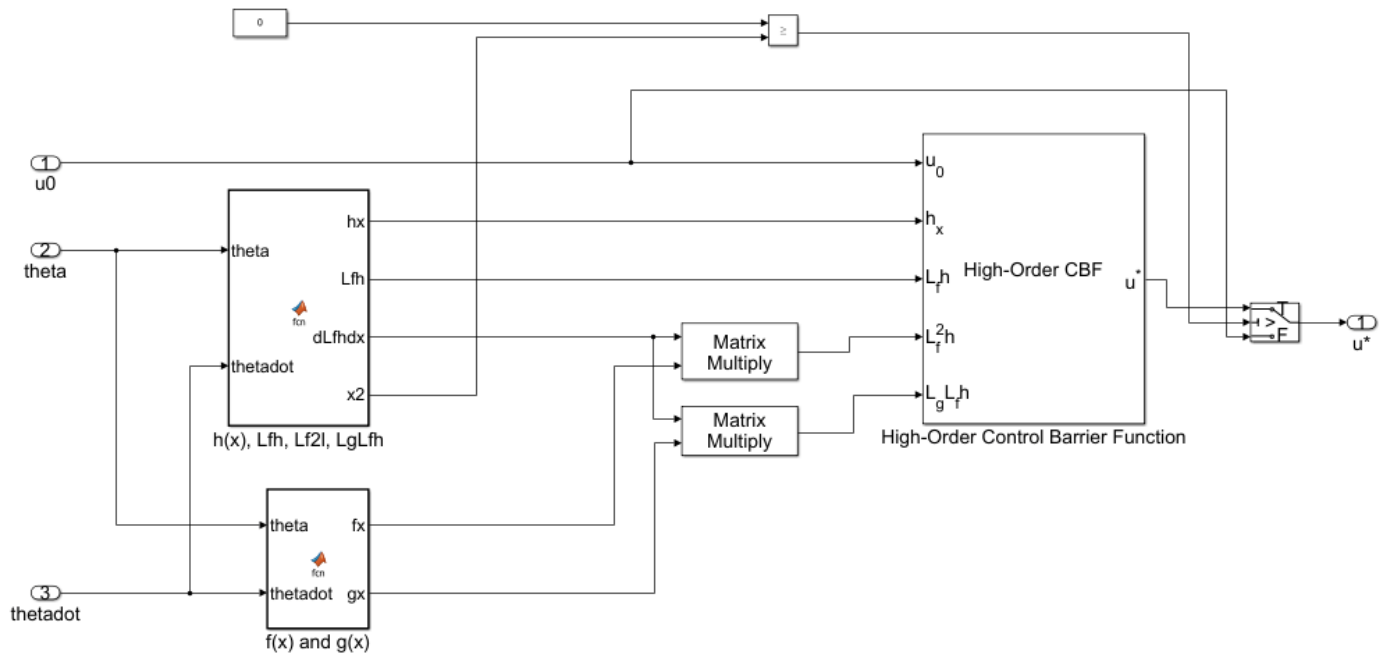
Simulink Model of the Two-Link Robot with RL Agent and CBF Subsystem

The Simulink model `TwoLinkRobot_HOCBF_SAC` contains the RL Agent block, the Two-Link robot dynamics, and the high-order CBF subsystem.

```
mdl = 'TwoLinkRobot_HOCBF_SAC';
open_system(mdl);
```



To view the CBF implementation, open the Control Barrier Function subsystem. Here, two MATLAB Function (Simulink) blocks calculate the barrier function and the several derivatives, while the High-Order Control Barrier Function (Simulink Control Design) block enforces the constraint on the control action.



The algorithm behind the implementation of the Control Barrier Function subsystem is explained in the Algorithm: Guarantee Safety of Two-Link Robot using Control Barrier Function on page 8-177 section at the end of the example.

Simulate Closed Loop System with High-Order CBF Constraint

For reproducibility, fix the seed of the random number generator.

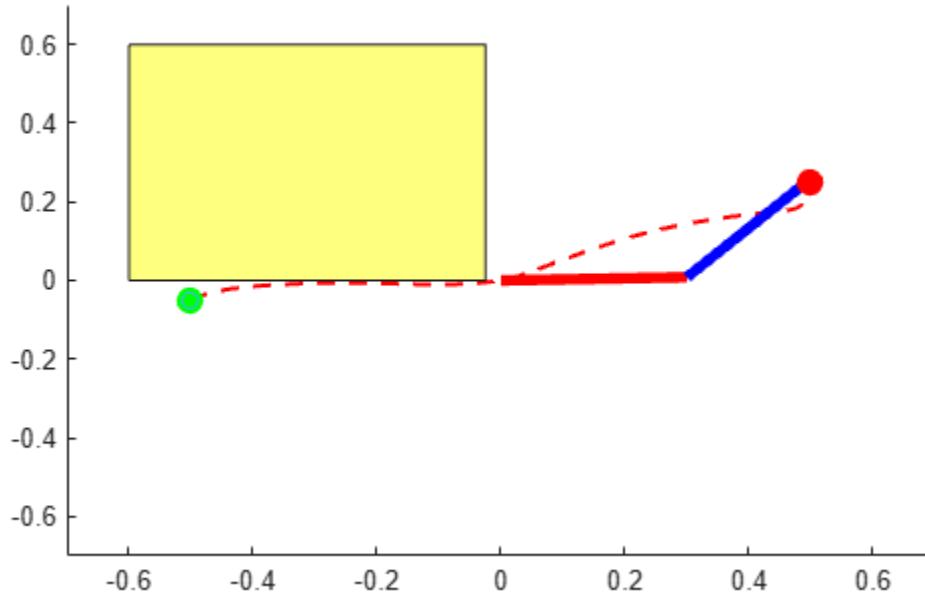
```
rng(0, "twister");
```

To validate the performance of the trained agent with safety constraints, use `sim` (Simulink) to simulate the trained agent in the loop with the two-link robot and the control barrier function.

```
out = sim mdl;
```

Show the robot trajectory.

```
theta = squeeze(out.simout.Data);
blockVisible = true;
PlotTwoLinkArm(theta, 0.5, 0.25, L1, L2, blockVisible)
```



The High-Order CBF block successfully constrains the tip of the two-link robot to avoid the obstacle.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Algorithm: Guarantee Safety of Two-Link Robot using Control Barrier Function

A control barrier function (CBF) can guarantee that the state of a system remains within a specified safe region. Therefore, you can use a CBF to guarantee the safety of a controlled system.

The CBF works by imposing constraints on the control input at each time step, thereby preventing the system from entering an unsafe region [3]. Specifically, at each time step, the algorithm solves a quadratic program to constrain the control input so that the derivative of the barrier function along the system trajectory does not fall below a given threshold. For more information, see “Enforce Safety Constraints with Control Barrier Functions” (Simulink Control Design).

For the problem in this example, the tip of second link must not hit the obstacle.

You can mathematically model this safety constraint as the following control barrier function:

$$h(x) = \begin{cases} -y_2(t) & x_2(t) \leq 0 \\ \text{no constraint} & x_2(t) > 0 \end{cases}$$

Using the Cartesian coordinates of the end point of second link, the CBF can be written as:

$$h(\zeta) = -l_1 \sin(\theta_1) - l_2 \sin(\theta_1 + \theta_2)$$

where $\zeta = [\theta_1 \ \theta_2]$

Using the control barrier function theory described in “Enforce Safety Constraints with Control Barrier Functions” (Simulink Control Design), you can write the constraint equation for safe control as:

$$\dot{h}(\zeta, u) \geq -\gamma h(\zeta)$$

Taking the first Lie derivative of the barrier function with respect to the states of the two-link robot yields:

$$\begin{aligned} \dot{h}(\zeta, u) &= \frac{\partial h}{\partial \zeta} \dot{\zeta} \\ \dot{h}(\zeta, u) &= - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \end{aligned}$$

Because the control term $u(t)$ does not appear in the first Lie derivative, take the second Lie derivative with respect to $\begin{bmatrix} \dot{\theta}_1, \dot{\theta}_2 \end{bmatrix}$:

$$\begin{aligned} \ddot{h}(\zeta, u) &= \frac{\partial h}{\partial \theta_1} \ddot{\theta}_1 + \frac{\partial h}{\partial \theta_2} \ddot{\theta}_2 \\ \ddot{h}(\zeta, u) &= - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} \end{aligned}$$

The second Lie derivative involves the term $\begin{bmatrix} \ddot{\theta}_1, \ddot{\theta}_2 \end{bmatrix}$. Hence the control term appears through system dynamics as follows:

$$\begin{aligned} \ddot{h}(\zeta, u) &= L_f^2 h + L_g L_f h u = - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) f(\zeta) \\ &\quad - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) g(\zeta) u \end{aligned}$$

where $L_f^2 h = - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) f(\zeta)$ and $L_g L_f h = - \left[l_1 \cos(\theta_1) + l_2 \cos(\theta_1 + \theta_2) \right] l_2 \cos(\theta_1 + \theta_2) g(\zeta)$

and the plant model equations $f(\zeta)$ and $g(\zeta)$ are:

$$\begin{aligned} f(\zeta) &= \begin{bmatrix} \alpha + 2\beta \cos(\theta_2) & \delta + \beta \cos(\theta_2) \\ \delta + \beta \cos(\theta_2) & \delta \end{bmatrix}^{-1} \begin{bmatrix} -\beta \sin(\theta_2) \dot{\theta}_2 & \beta \sin(\theta_2) (\dot{\theta}_1 + \dot{\theta}_2) \\ \beta \sin(\theta_2) \dot{\theta}_1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}, \\ g(\zeta) &= \begin{bmatrix} \alpha + 2\beta \cos(\theta_2) & \delta + \beta \cos(\theta_2) \\ \delta + \beta \cos(\theta_2) & \delta \end{bmatrix}^{-1} \text{ and } u = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \end{aligned}$$

You can then write the QP problem formulation for the two-link robot as follows:

$$\begin{aligned} u^* &= \min_u \frac{1}{2} \|u - u_{\text{pid}}\|^2 \\ \text{s.t. } & L_f^2 h + L_g L_f h u + \gamma_1 h + \gamma_2 L_f h \geq 0 \end{aligned}$$

And implement it using the High-Order Control Barrier Function (Simulink Control Design) block as done in the Control Barrier Function subsystem in the TwoLinkRobot_HOCBF_SAC model.

References

[1] Murray, Richard M., Zexiang Li, and Shankar Sastry. *A Mathematical Introduction to Robotic Manipulation*. Boca Raton: CRC Press, 1994.

[2] Seiler, Peter, Robert M. Moore, Chris Meissen, Murat Arcak, and Andrew Packard. "Finite Horizon Robustness Analysis of LTV Systems Using Integral Quadratic Constraints." *Automatica* 100 (February 2019): 135-43. <https://doi.org/10.1016/j.automatica.2018.11.009>.

[3] W. Xiao and C. Belta, "High-Order Control Barrier Functions," in *IEEE Transactions on Automatic Control*, vol. 67, no. 7, pp. 3655-3662, July 2022, doi: 10.1109/TAC.2021.3105491. <https://doi.org/10.1109/TAC.2006.882372>.

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlSACAgent` | `rlSACAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions` | `rlOptimizerOptions`

Blocks

RL Agent

Related Examples

- "Train Biped Robot to Walk Using Reinforcement Learning Agents" on page 8-144

More About

- "Define Observation and Reward Signals in Custom Environments" on page 2-127
- "Deep Deterministic Policy Gradient (DDPG) Agent" on page 3-67
- "Train Reinforcement Learning Agents" on page 5-3

Train SAC Agent for Ball Balance Control

This example shows how to train a soft actor-critic (SAC) reinforcement learning agent to control a robot arm for a ball-balancing task.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

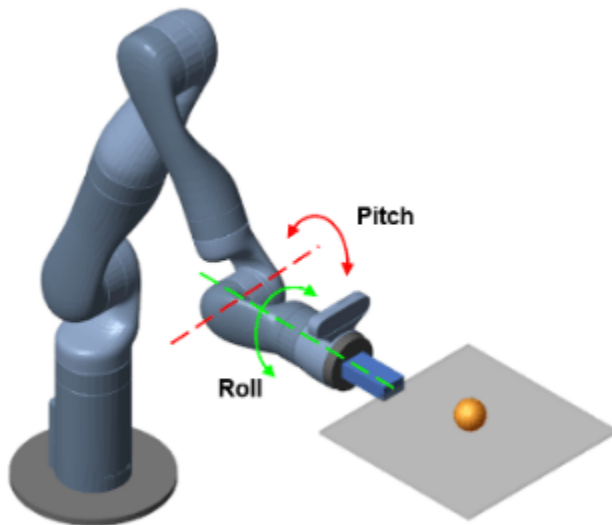
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

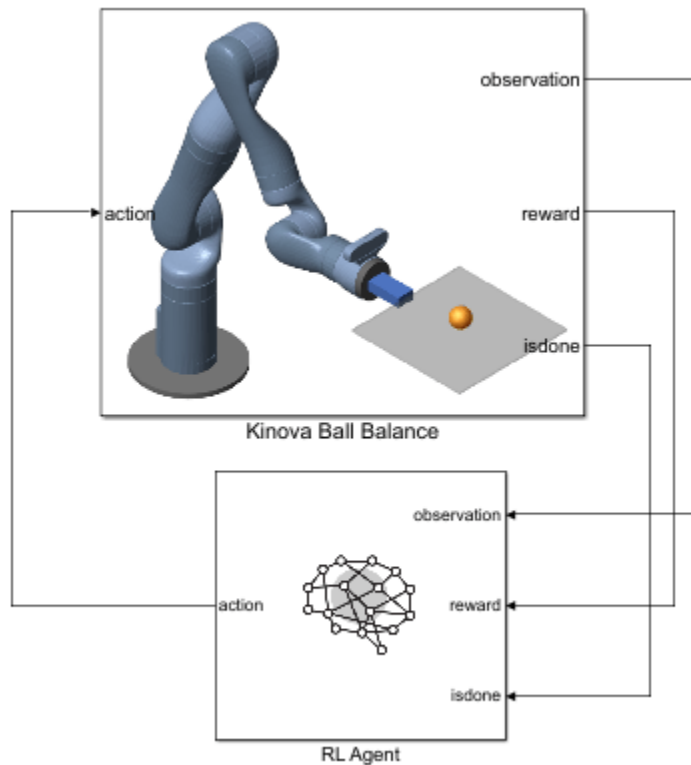
Environment Overview

The robot arm in this example is a Kinova® Gen3 robot, which is a seven degree-of-freedom (DOF) manipulator. The arm is tasked to balance a ping pong ball at the center of a flat surface (plate) attached to the robot gripper. Only the final two joints are actuated and contribute to motion in the pitch and roll axes as shown in the following figure. The remaining joints are fixed and do not contribute to motion.



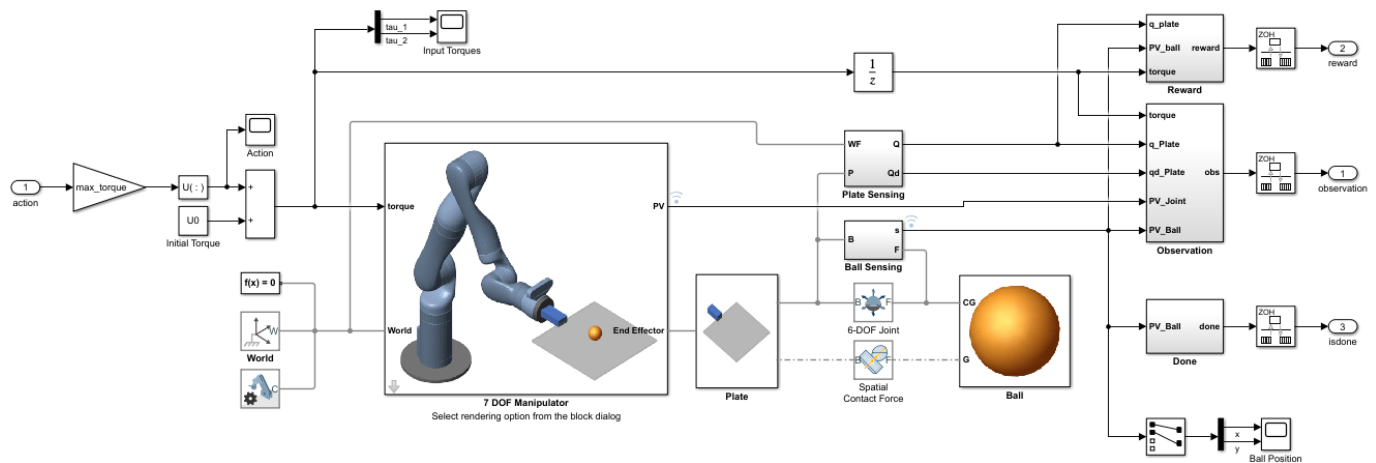
Open the Simulink® model to view the system. The model contains a Kinova Ball Balance subsystem connected to an RL Agent block. The agent applies an action to the robot subsystem and receives the resulting observation, reward, and is-done signals.

```
open_system("rlKinovaBallBalance")
```



View to the Kinova Ball Balance subsystem.

```
open_system("rlKinovaBallBalance/Kinova Ball Balance")
```



In this model:

- The physical components of the system (manipulator, ball, and plate) are modeled using Simscape™ Multibody™ components.

- The plate is constrained to the end effector of the manipulator.
- The ball has six degrees of freedom and can move freely in space.
- Contact forces between the ball and plate are modeled using the Spatial Contact Force block.
- Control inputs to the manipulator are the torque signals for the actuated joints.

Create the parameters for the example by running the `kinova_params` script included with this example.

```
kinova_params;
```

You can view a 3-D animation of the manipulator in the Multibody Explorer with the Robotics System Toolbox Robot Library Data support package. When you have the support package installed, the Script also adds the necessary 3D mesh files to the MATLAB® path. To download and install the support package, use the Add-On Explorer. For more information, see “Get and Manage Add-Ons”.

Create Environment Object

To train a reinforcement learning agent, you must define the environment with which it will interact. For the ball balancing environment:

- The observations are represented by a 22 element vector that contains information about the positions (sine and cosine of joint angles) and velocities (joint angle derivatives) of the two actuated joints, positions (x and y distances from plate center) and velocities (x and y derivatives) of the ball, orientation (quaternions) and velocities (quaternion derivatives) of the plate, joint torques from the last time step, ball radius, and mass.
- The actions are normalized joint torque values.
- The sample time is $T_s = 0.01s$, and the simulation time is $T_f = 10s$.
- The simulation terminates when the ball falls off the plate.
- The reward r_t at time step t is given by:

$$\begin{aligned} r_t &= r_{\text{ball}} + r_{\text{plate}} + r_{\text{action}} \\ r_{\text{ball}} &= e^{-0.001(x^2 + y^2)} \\ r_{\text{plate}} &= -0.1(\phi^2 + \theta^2 + \psi^2) \\ r_{\text{action}} &= -0.05(\tau_1^2 + \tau_2^2) \end{aligned}$$

Here, r_{ball} is a reward for the ball moving closer to the center of the plate, r_{plate} is a penalty for plate orientation, and r_{action} is a penalty for control effort. ϕ , θ , and ψ are the respective roll, pitch, and yaw angles of the plate in radians. τ_1 and τ_2 are the joint torques.

Create the observation and action input specifications for the environment.

```
nObs = 22; % Number of dimension of the observation space
nAct = 2;  % Number of dimension of the action space

obsInfo = rlNumericSpec([nObs 1]);

actInfo = rlNumericSpec([nAct 1]);
actInfo.LowerLimit = -1;
actInfo.UpperLimit = 1;
```

Create the Simulink environment object using the observation and action specifications. For more information on creating Simulink environments, see `rlSimulinkEnv`.

```
mdl = "rlKinovaBallBalance";  
blk = mdl + "/RL Agent";  
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo);
```

Specify a reset function for the environment using the `ResetFcn` parameter. The `kinovaResetFcn` on page 8-194 function is defined at the end of the example.

```
env.ResetFcn = @kinovaResetFcn;
```

This `kinovaResetFcn` function (provided at the end of this example) randomly initializes the initial x and y positions of the ball with respect to the center of the plate. For robust training, you can randomize other parameters inside the reset function, such as the mass and radius of the ball.

Specify the sample time T_s and simulation time T_f .

```
Ts = 0.01;  
Tf = 10;
```

Create Soft Actor-Critic Agent with Custom Networks

The agent in this example is a soft actor-critic (SAC) agent. SAC agents use one or two parameterized Q-value function approximators to estimate the value of the policy. A Q-value function critic takes the current observation and an action as inputs and returns a single scalar as output. The output is the estimated value of the policy, that is the estimated discounted cumulative long-term reward when the agent starts from the state corresponding to the current observation, takes the action, and follows the policy thereafter. For more information on SAC agents, see “Soft Actor-Critic (SAC) Agent” on page 3-78.

Create Critic

The SAC agent in this example uses two critics. To model the parameterized Q-value functions within the critics, use a neural network with two input layers (one for the observation channel, as specified by `obsInfo`, and the other for the action channel, as specified by `actInfo`) and one output layer (which returns the scalar value). For more information on creating deep neural networks for reinforcement learning agents, see “Create Actors, Critics, and Policy Objects” on page 4-2.

The initial parameters of the critic network are initialized with random values. Fix the random number stream so that the critic is always initialized with the same parameter values.

```
rng(0,"twister");
```

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
% Define the network paths.  
observationPath = [  
    featureInputLayer(nObs,Name="observation")  
    concatenationLayer(1,2,Name="concat")  
    fullyConnectedLayer(128)  
    reluLayer  
    fullyConnectedLayer(64)  
    reluLayer  
    fullyConnectedLayer(32)
```

```

        reluLayer
        fullyConnectedLayer(1,Name="QValueOutLyr")
    ];
    actionPath = featureInputLayer(nAct,Name="action");

```

Assemble dlnetwork object.

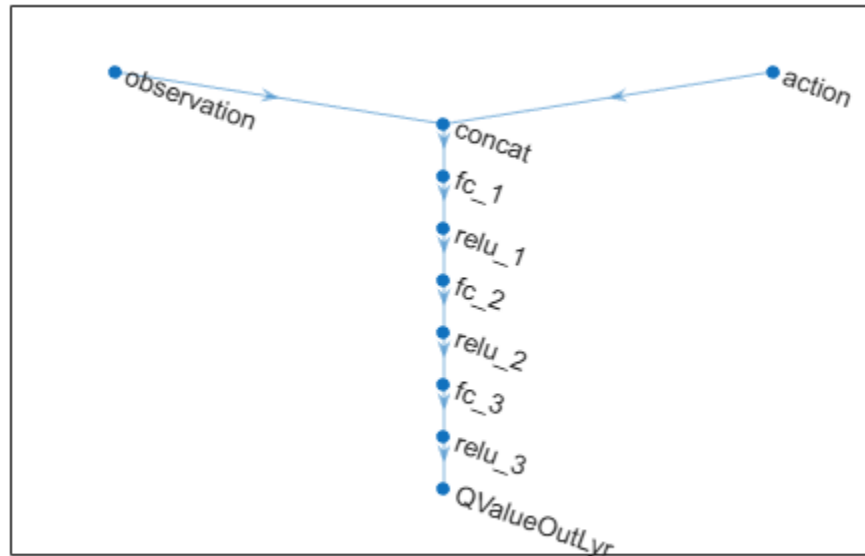
```

criticNet = dlnetwork;
criticNet = addLayers(criticNet, observationPath);
criticNet = addLayers(criticNet, actionPath);
criticNet = connectLayers(criticNet,"action","concat/in2");

```

View the critic neural network.

```
plot(criticNet)
```



Display the number of weights.

```

summary(initialize(criticNet))

Initialized: true

Number of learnables: 13.6k

Inputs:
  1  'observation'  22 features
  2  'action'      2 features

```

Create the critic approximators using `rlQValueFunction`. When using two critics, a SAC agent requires them to have different initial parameters. Initialize the networks separately to make sure that their initial weights are different.

```

critic1 = rlQValueFunction(initialize(criticNet), ...
    obsInfo,actInfo, ...

```

```
    ObservationInputNames="observation");
critic2 = rlQValueFunction(initialize(criticNet), ...
    obsInfo,actInfo, ...
    ObservationInputNames="observation");
```

Create Actor

Soft Actor-critic agents use a parameterized stochastic policy over a continuous action space, which is implemented by a continuous Gaussian actor.

This actor takes an observation as input and returns as output a random action sampled from a Gaussian probability distribution.

To approximate the mean values and standard deviations of the Gaussian distribution, you must use a neural network with two output layers, each having as many elements as the dimension of the action space. One output layer must return a vector containing the mean values for each action dimension. The other must return a vector containing the standard deviation for each action dimension.

Because standard deviations must be nonnegative, use a softplus or ReLU layer to enforce nonnegativity. The SAC agent automatically reads the action range from the `UpperLimit` and `LowerLimit` properties of `actInfo` (which is used to create the actor), and then internally scales the distribution and bounds the action. Therefore, do not add a `tanhLayer` as the last nonlinear layer in the mean output path.

The initial parameters of the actor network are initialized with random values. Fix the random number stream so that the actor is always initialized with the same parameter values.

```
rng(0,"twister");
```

Define each network path as an array of layer objects, and assign names to the input and output layers of each path.

```
% Create the actor network layers.
commonPath = [
    featureInputLayer(nObs,Name="observation")
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer(Name="commonPath")
];
meanPath = [
    fullyConnectedLayer(32,Name="meanFC")
    reluLayer
    fullyConnectedLayer(nAct,Name="actionMean")
];
stdPath = [
    fullyConnectedLayer(nAct,Name="stdFC")
    reluLayer
    softplusLayer(Name="actionStd")
];
```

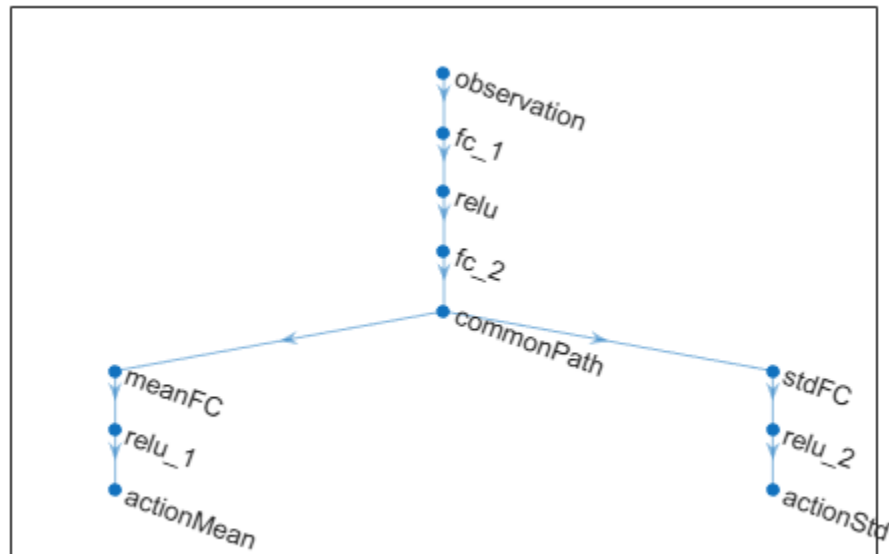
Assemble `dlnetwork` object.

```
actorNet = dlnetwork;
actorNet = addLayers(actorNet,commonPath);
actorNet = addLayers(actorNet,meanPath);
actorNet = addLayers(actorNet,stdPath);
```

```
actorNet = connectLayers(actorNet, "commonPath", "meanFC/in");
actorNet = connectLayers(actorNet, "commonPath", "stdFC/in");
```

View the actor neural network.

```
plot(actorNet)
```



Initialize network and display the number of weights.

```
actorNet = initialize(actorNet);
summary(actorNet)

Initialized: true

Number of learnables: 13.5k

Inputs:
  1  'observation'  22 features
```

Create the actor function using `rlContinuousGaussianActor`.

```
actor = rlContinuousGaussianActor(actorNet, obsInfo, actInfo, ...
    ObservationInputNames="observation", ...
    ActionMeanOutputNames="actionMean", ...
    ActionStandardDeviationOutputNames="actionStd");
```

Create Agent Object

The SAC agent in this example trains from an experience buffer of maximum capacity 1e6 by randomly selecting mini-batches of size 256. The fact that the discount factor is 0.99, that is, very close to 1, means that the agent keeps more into account long term rewards (a discount factor closer to 0 would instead place a heavier discount on future rewards, therefore encouraging short term ones). For a full list of SAC hyperparameters and their descriptions, see `rlSACAgentOptions`.

Specify the agent hyperparameters for training. For this example:

- The actor and critic neural networks are updated using the Adam algorithm with a learn rate of $1e-4$ and $5e-4$ respectively.
- A gradient threshold value of 1 is used to clip the gradients and improve stability of learning.
- The agent learns with mini-batches of 300 experiences, after a warm start duration of 1000 steps.
- The experience buffer capacity is $1e6$. A large capacity enables storing a diverse set of experiences.
- The agent sample time is $T_s=0.01$ second.

```
agentOpts = rlSACAgentOptions( ...
    SampleTime           = Ts, ...
    ExperienceBufferLength = 1e6, ...
    NumWarmStartSteps    = 1e3, ...
    MiniBatchSize        = 300);

agentOpts.ActorOptimizerOptions.Algorithm = "adam";
agentOpts.ActorOptimizerOptions.LearnRate = 1e-4;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;

for ct = 1:2
    agentOpts.CriticOptimizerOptions(ct).Algorithm = "adam";
    agentOpts.CriticOptimizerOptions(ct).LearnRate = 5e-4;
    agentOpts.CriticOptimizerOptions(ct).GradientThreshold = 1;
end
```

Fix the random number stream for reproducibility.

```
rng(0, "twister");
```

Create the SAC agent using the actor, critic and agent options objects.

```
agent = rlSACAgent(actor,[critic1,critic2],agentOpts);
```

Train Agent

To train the agent, first specify the training options using `rlTrainingOptions`. For this example, use the following options:

- Run each training for a maximum of 6000 episodes, with each episode lasting at most $\text{floor}(T_f/T_s)$ time steps.
- Evaluate the performance of the greedy policy every 100 training episodes, averaging the cumulative reward of 5 simulations.
- Stop the training when the evaluation score reaches 700. At this point, the robot is able to balance the ball at the center of the plate.
- Do not store the simulation data since the training can be memory intensive. Alternatively you can save simulation data to disk by setting `SimulationStorageType` to "file".

```
% training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=6000, ...
    MaxStepsPerEpisode=floor(Tf/Ts), ...
    ScoreAveragingWindowLength=100, ...
    StopTrainingCriteria="EvaluationStatistic", ...
```



```
StopTrainingValue=700, ...
SimulationStorageType="none");
```

```
% agent evaluation
```

```
evl = rlEvaluator(EvaluationFrequency=100, NumEpisodes=5);
```

To train the agent in parallel, specify the following training options. Training in parallel requires Parallel Computing Toolbox™ software. If you do not have Parallel Computing Toolbox software installed, set `UseParallel` to `false`.

- Set the `UseParallel` option to `true`.
- Train the agent using asynchronous parallel workers.

```
trainOpts.UseParallel =  ;
trainOpts.ParallelizationOptions.Mode =  ;
```

For more information, see `rlTrainingOptions`.

In parallel training, workers simulate the agent's policy with the environment and store experiences in the replay memory. When workers operate asynchronously the order of stored experiences might not be deterministic and can ultimately make the training results different. To maximize the reproducibility likelihood:

- Initialize the parallel pool with the same number of parallel workers every time you run the code. For information on specifying the pool size see “Discover Clusters and Use Cluster Profiles” (Parallel Computing Toolbox).
- Use synchronous parallel training by setting `trainOpts.ParallelizationOptions.Mode` to `"sync"`.
- Assign a random seed to each parallel worker using `trainOpts.ParallelizationOptions.WorkerRandomSeeds`. The default value of -1 assigns a unique random seed to each parallel worker.

You can log training data using the `rlDataLogger` object. For this example, log the actor and critic training losses using the `logAgentLearnData` on page 8-195 function provided at the end of this example.

```
logger = rlDataLogger();
logger.AgentLearnFinishedFcn = @logAgentLearnData;
```

For more information, see “Log Training Data to Disk” on page 5-231.

Fix the random number stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

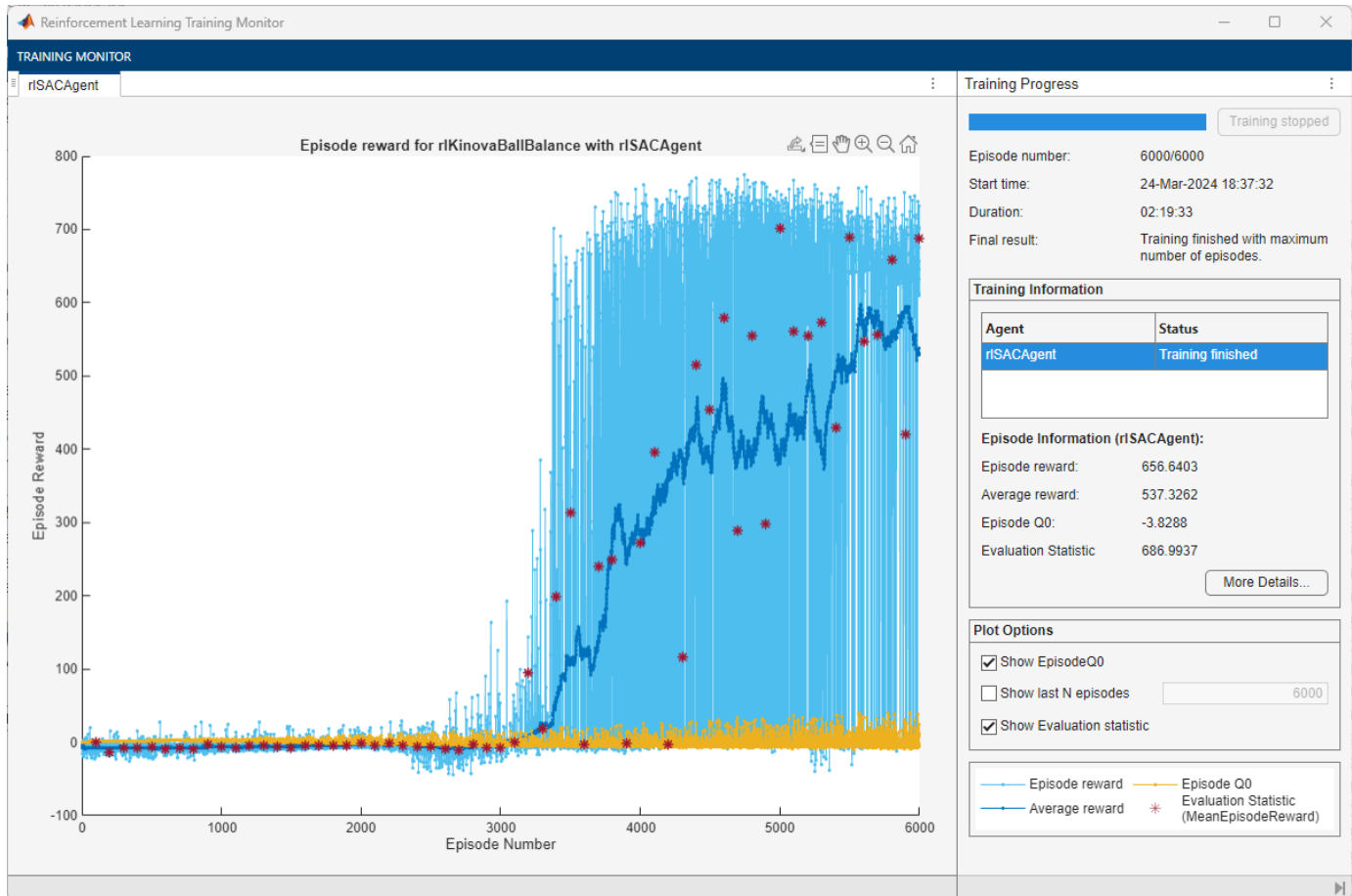
```
doTraining =  ;
if doTraining
    % train
```

```

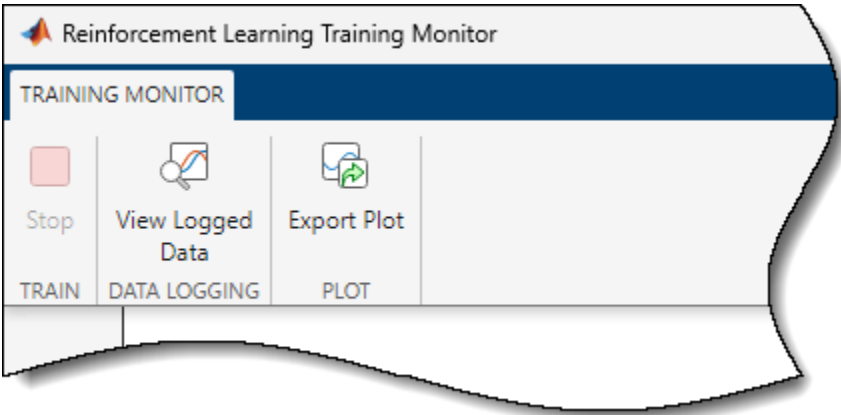
trainResult = train(agent, env, ...
    trainOpts, ...
    Logger = logger, ...
    Evaluator = evl);
else
    load("kinovaBallBalanceAgent.mat")
end

```

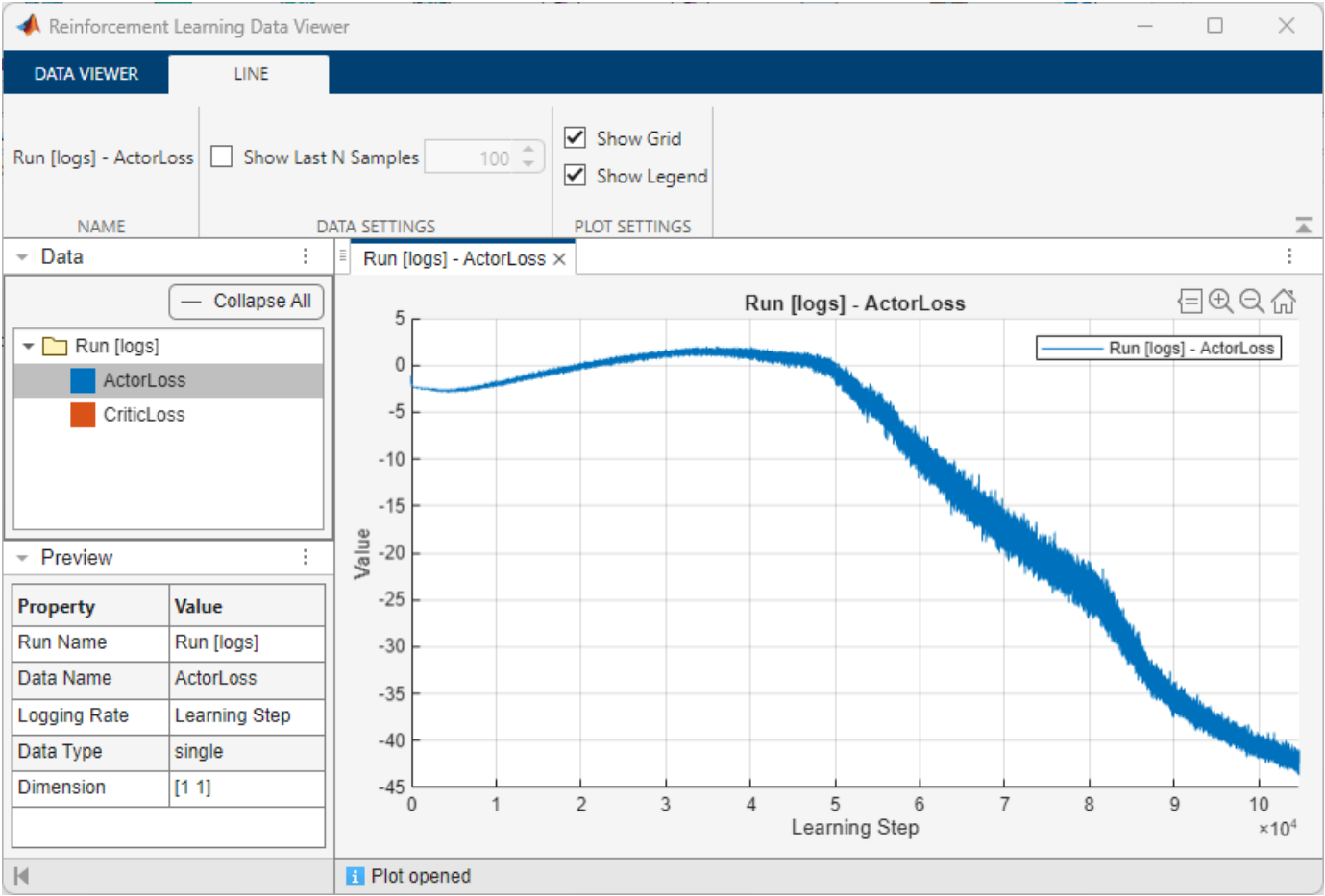
A snapshot of training progress is shown in the following figure. You can expect different results due to randomness in the training process.

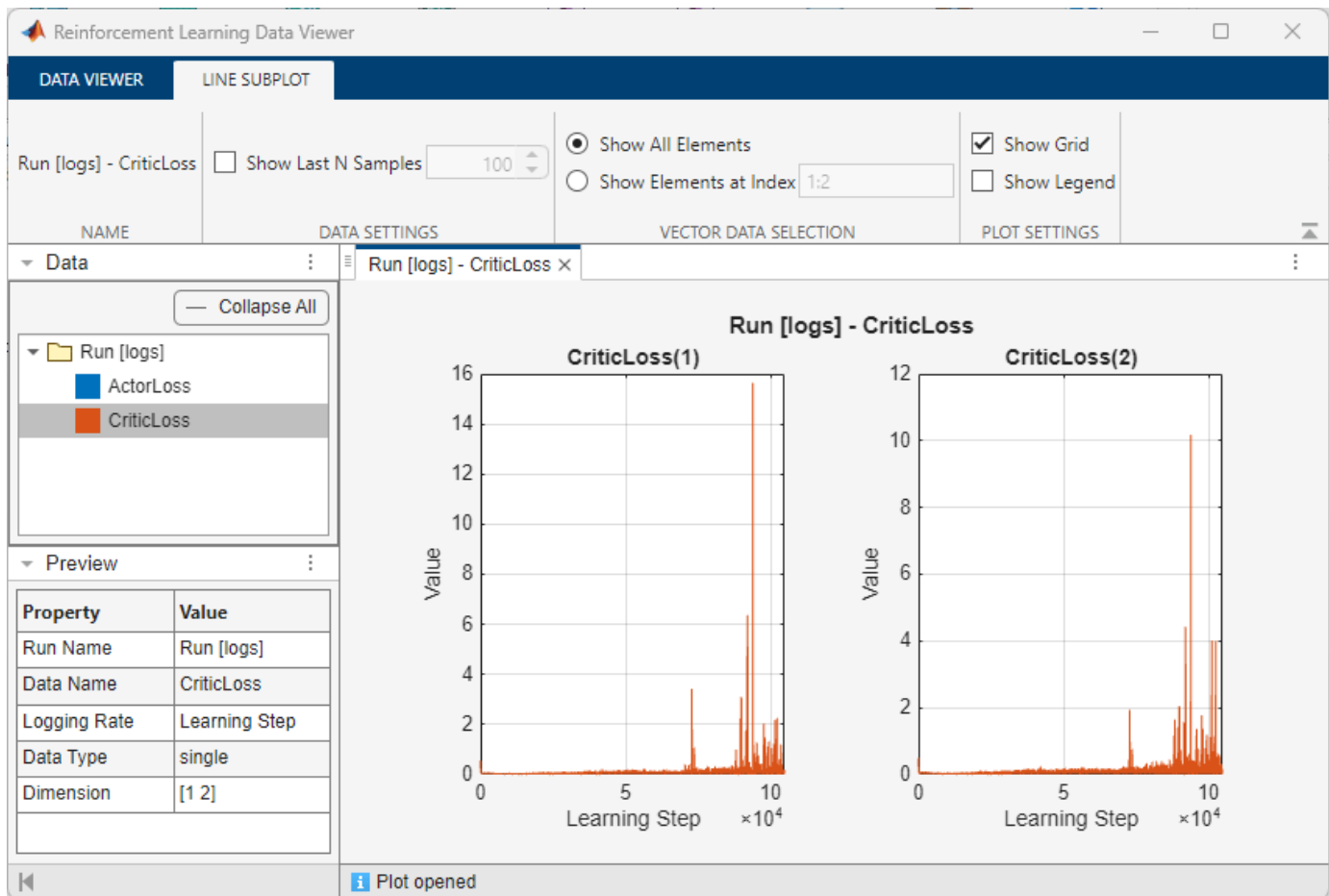


You can visualize data logged to disk using the interactive **Reinforcement Learning Data Viewer** graphical user interface. To open the visualization, click **View Logged Data** in the **Reinforcement Learning Training Monitor** window.



To create plots in the **Reinforcement Learning Data Viewer**, select a data from the **Data** panel and a choice of plot from the toolstrip. The following images show the actor and critic losses logged during training.





Simulate Trained Agent

Fix the random number stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Specify an initial position for the ball with respect to the plate center. To randomize the initial ball position during simulation, set the `userSpecifiedConditions` flag to `false`.

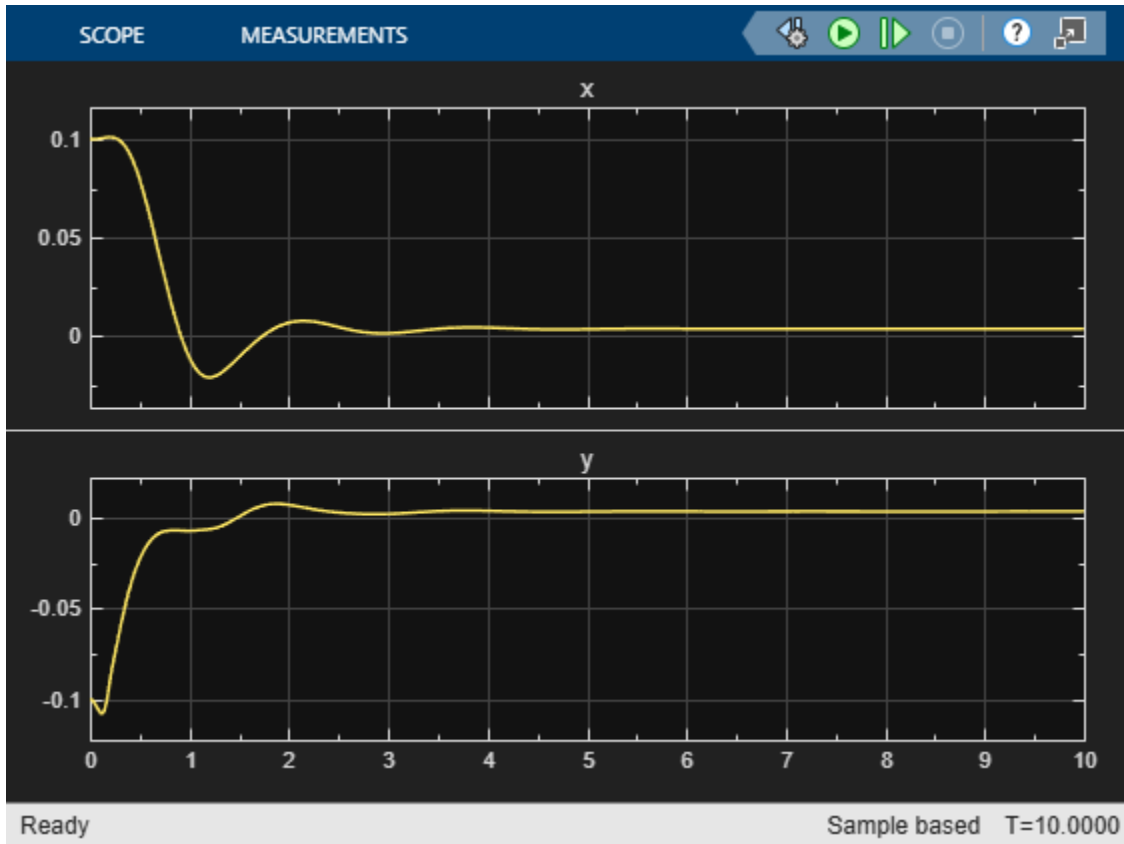
```
userSpecifiedConditions = true;
if userSpecifiedConditions
    ball.x0 = 0.1;
    ball.y0 = -0.1;
    env.ResetFcn = @(in) setPostSimFcn(in, @animatedPath);
else
    env.ResetFcn = @kinovaResetFcn;
end
```

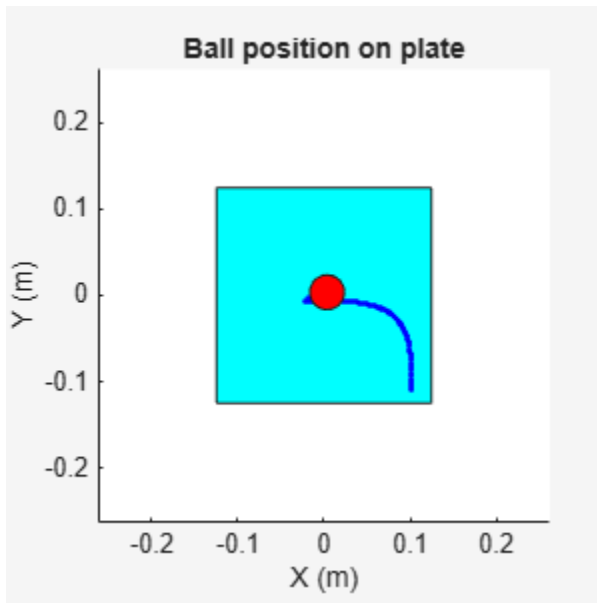
Create a simulation options object for configuring the simulation. The agent will be simulated for a maximum of `floor(Tf/Ts)` steps per simulation episode.

```
sim0pts = rlSimulationOptions(MaxSteps=floor(Tf/Ts));
```

Simulate the agent using the greedy policy. View the trajectory of the ball using the Ball Position scope block.

```
agent.UseExplorationPolicy = false;  
experiences = sim(agent,env,sim0pts);
```





Display the total reward.

```
totalRwd = sum(experiences.Reward)
```

```
totalRwd =  
674.3079
```

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Environment Reset Function

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `reset` function uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = kinovaResetFcn(in)
    % KinovaResetFcn is used to randomize
    % the initial joint angles R6_q0 and R7_q0 as well as
    % the initial wrist and hand torque values.

    % Ball parameters
    ball.radius = 0.02;      % m
    ball.mass   = 0.0027;    % kg
    ball.shell  = 0.0002;    % m

    % Calculate ball moment of inertia.
    ball.moi = calcMOI(ball.radius,ball.shell,ball.mass);

    % Initial conditions. +z is vertically upward.
```

```

% Randomize the x and y initial distances (in m) within plate.
ball.x0 = -0.1 + 0.2*rand; % x distance from plate center
ball.y0 = -0.1 + 0.2*rand; % y distance from plate center
ball.z0 = ball.radius; % z height from plate surface

ball.dx0 = 0; % m/s, ball initial x velocity
ball.dy0 = 0; % m/s, ball initial y velocity
ball.dz0 = 0; % m/s, ball initial z velocity

% Contact friction parameters
ball.staticfriction = 0.5;
ball.dynamicfriction = 0.3;
ball.criticalvelocity = 1e-3;

% Convert coefficient of restitution to spring-damper parameters.
coeff_restitution = 0.89;
[k, c, w] = cor2SpringDamperParams(coeff_restitution,ball.mass);
ball.stiffness = k;
ball.damping = c;
ball.transitionwidth = w;

in = setVariable(in,"ball",ball);

% Randomize joint angles within a range of +/- 5 deg from the
% starting positions of the joints.
R6_q0 = deg2rad(-65) + deg2rad(-5+10*rand);
R7_q0 = deg2rad(-90) + deg2rad(-5+10*rand);
in = setVariable(in,"R6_q0",R6_q0);
in = setVariable(in,"R7_q0",R7_q0);

% Compute approximate initial joint torques that hold the ball,
% plate and arm at their initial configuration
g = 9.80665;
wrist_torque_0 = ...
    (-1.882 + ball.x0*ball.mass*g)*cos(deg2rad(-65)-R6_q0);
hand_torque_0 = ...
    (0.0002349 - ball.y0*ball.mass*g)*cos(deg2rad(-90)-R7_q0);
U0 = [wrist_torque_0 hand_torque_0];
in = setVariable(in,"U0",U0);

% specify function to be executed after simulation
in = setPostSimFcn(in, @animatedPath); % visualization
end

```

Data Logging Functions

```

function dataToLog = logAgentLearnData(data)
% This function is executed after completion
% of the agent's learning subroutine.
dataToLog.ActorLoss = data.ActorLoss;

```

```
dataToLog.CriticLoss = data.CriticLoss;  
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlSACAgent` | `rlSACAgentOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Control Robot Manipulator Using Passivity-Based Nonlinear MPC” (Model Predictive Control Toolbox)

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Soft Actor-Critic (SAC) Agent” on page 3-78
- “Train Reinforcement Learning Agents” on page 5-3

Automatic Parking Valet with Unreal Engine Simulation

This example demonstrates the design of a hybrid controller for an automatic searching and parking task. You will learn how to combine adaptive model predictive control (MPC) with reinforcement learning (RL) to perform a parking maneuver. Unreal Engine® is used for environment perception and visualization.

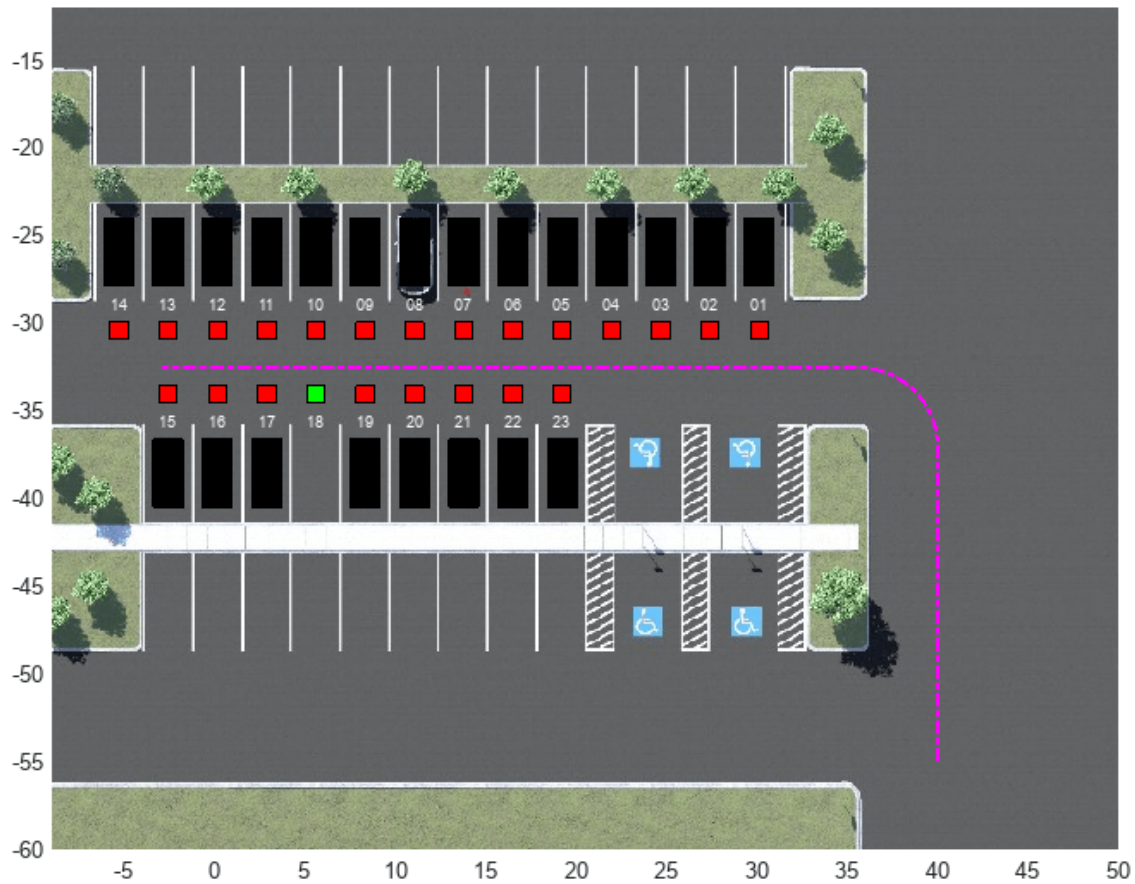


The control objective is to park the vehicle in an empty spot after starting from an initial pose. The control algorithm executes a series of maneuvers while sensing and avoiding obstacles in tight spaces. It switches between an adaptive MPC controller and a reinforcement learning agent to complete the parking maneuver.

The adaptive MPC controller moves the vehicle at a constant speed along a reference path while searching for an empty parking spot. When a spot is found, the reinforcement learning agent takes over and executes a parking maneuver according to its pretrained policy. Prior knowledge of the environment (the parking lot) including the locations of the empty spots and parked vehicles is available to both controllers.

Parking Lot

The parking lot environment used in this example is a subsection of the Large Parking Lot (Automated Driving Toolbox) scene. The `ParkingLotManager` object stores information on the ego vehicle, empty parking spots, and static obstacles (parked cars and boundaries), while the `ParkingLotVisualizer` object creates a 2-D visualization of the scene. In the 2-D visualization, each parking spot has a unique index number and an indicator light that is either green (free) or red (occupied). Parked vehicles are represented in black.



Specify a sample time T_s (seconds) for the controllers and a simulation time T_f (seconds).

```
Ts = 0.1;
Tf = 50;
```

Create a reference path for the ego vehicle using the `createReferenceTrajectory` method of the `ParkingLotManager` object. The reference path starts from the southeast corner of the parking lot and ends at the western edge as displayed with the dashed pink line.

```
parkingLot = ParkingLotManager;
xRef = parkingLot.createReferenceTrajectory(Ts,Tf);
info = parkingLot.getInfo();
```

Set the parking space at index 18 as the free spot and fill all the other spots with static vehicle actors.

```
freeSpotIndex = 18;
mdl = "rlAutoParkingValet3D";
setupActorVehicles(mdl,freeSpotIndex);
```

Create a `ParkingLotVisualizer` object with a free spot at index 18 and the specified reference path `xRef`. Enable the visualizer figure's `Visible` property so that it opens outside the live script.

```
visualizer = ParkingLotVisualizer(freeSpotIndex,Ts,Tf);
set(gcf,Visible="on");
```



Specify an initial pose (X_0, Y_0, θ_0) for the ego vehicle. The units of X_0 and Y_0 are meters and θ_0 is in radians.

```
egoInitialPose = [40 -55 pi/2];
```

Compute the target pose for the vehicle using the `findGoalPose` method of the `ParkingLotManager` object. The goal pose corresponds to the location in `freeSpotIdx`.

```
vehiclePose = [10 -32.5 pi];
egoTargetPose = parkingLot.findGoalPose(vehiclePose,freeSpotIndex);
```

Sensor Modules

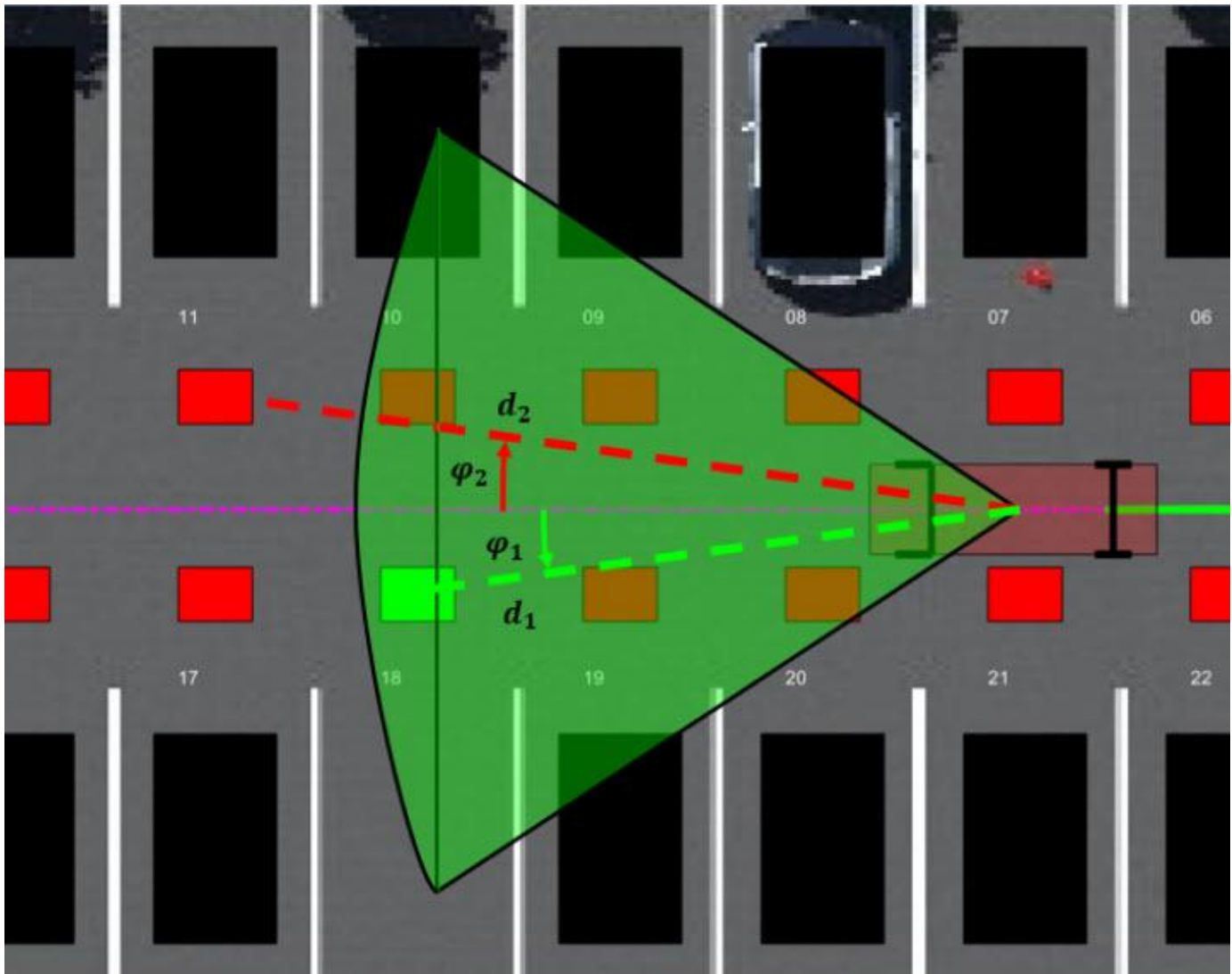
The parking algorithm uses a geometric "mode selector" object to scan for available parking spots and a lidar sensor to generate detections from the environment.

Mode Selector

The mode selector system scans a conical area in front of the vehicle for free parking spots. The scanning area is represented by the area shaded in green in the following figure. The camera has a field of view φ bounded by ± 60 degrees and a maximum measurement depth $d_{\max}$ of 10 m (equal to searchDist).

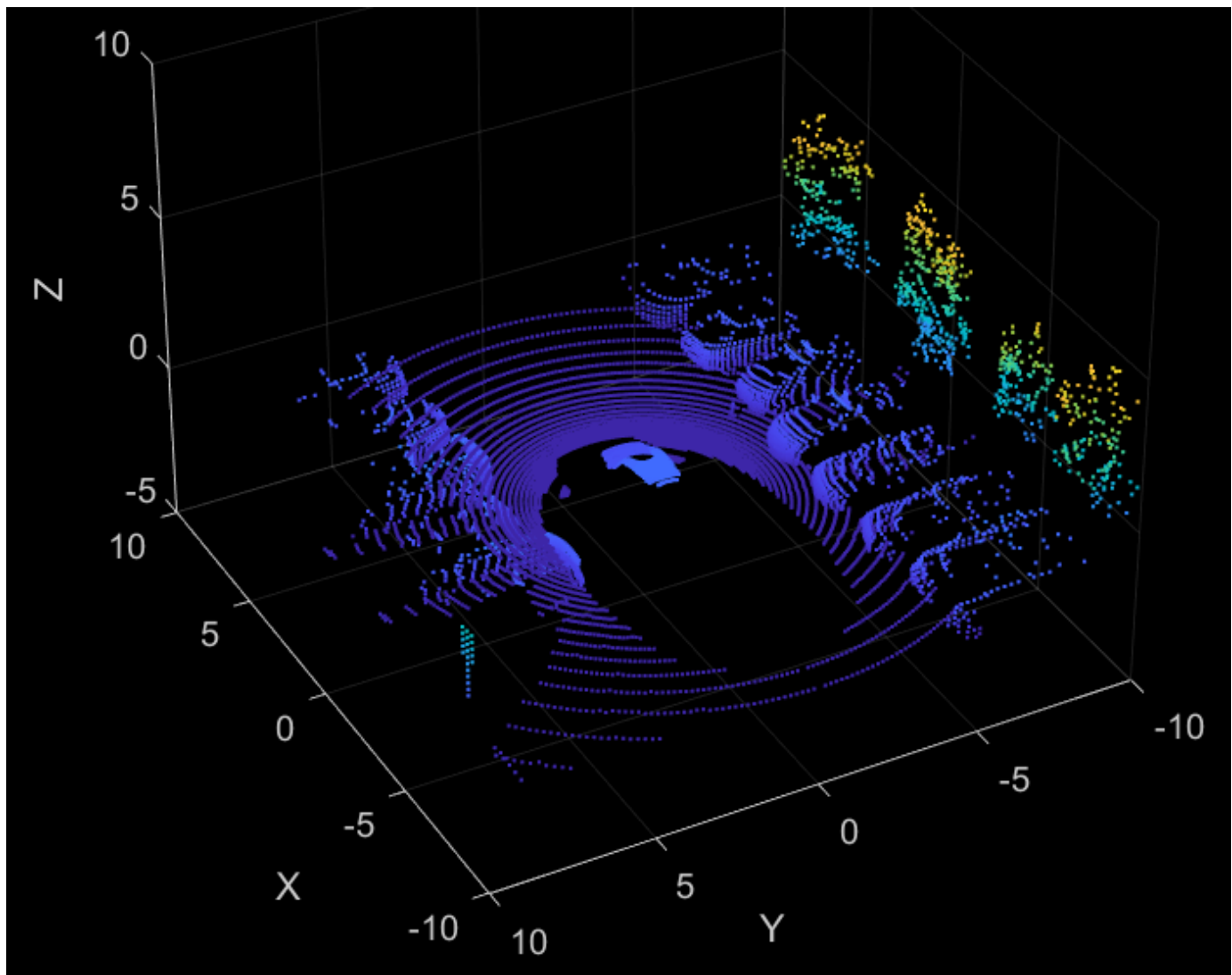
```
searchDist = 10;
```

As the ego vehicle moves forward, the mode selector system senses the parking spots within the field of view and determines whether a spot is free or occupied. For simplicity, this action is implemented using geometrical relationships between the spot locations and the current vehicle pose. A parking spot is within the scanning range if $d_i \leq d_{\max}$ and $\varphi_{\min} \leq \varphi_i \leq \varphi_{\max}$, where d_i is the distance to the parking spot and φ_i is the angle to the parking spot. Once a free parking spot is identified within the scanning area, the vehicle switches from the path-following adaptive MPC controller to the RL controller used to execute the parking maneuver.



Lidar

A Simulation 3D Lidar (Automated Driving Toolbox) block is used to generate a point cloud of detections from the environment. The lidar is mounted to the roof center and has a horizontal field-of-view (FOV) of 360 degrees and a vertical FOV of 40 degrees. The point cloud generated by the lidar contains five channels. The first three channels contain the X, Y, and Z position, in meters, of each detection relative to the sensor. In the lidar sensor's coordinate system, the X-axis extends forward from the vehicle, the Y-axis extends to the left of the vehicle (as viewed when looking in the forward direction of the vehicle), and the Z-axis points up. The fourth channel of the point cloud contains the straight-line distance to each detection in meters. The fifth channel contains the reflectivity of each detection, normalized in the range [0,1]. The reinforcement learning agent uses this point cloud data to understand its environment.



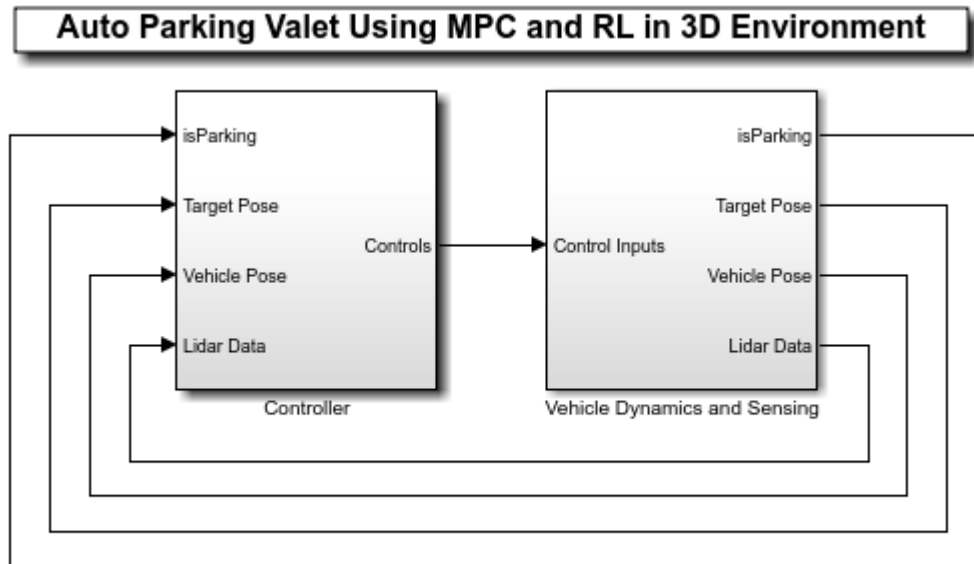
Auto Parking Valet Model

Load the automatic parking valet parameters.

```
autoParkingValetParams3D
```

The parking valet model, including the controllers, ego vehicle, sensors, and parking lot, is implemented as a Simulink® model. Open the model.

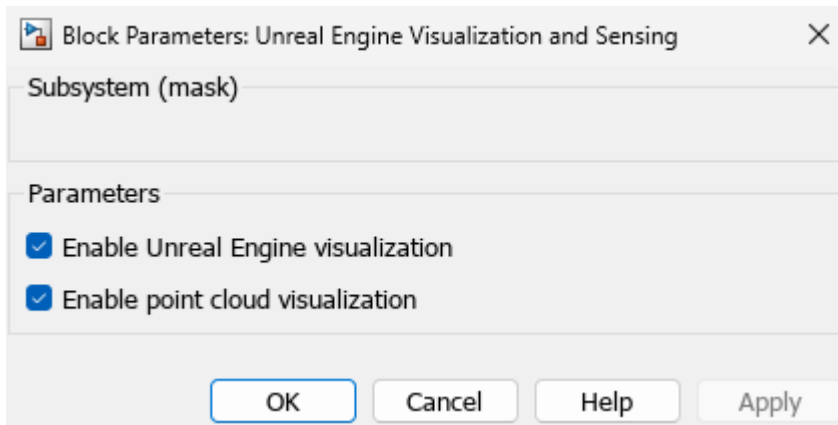
```
open_system mdl
```



Copyright 2021-2023 The MathWorks Inc.

In this model:

- The vehicle is modeled in the Vehicle Dynamics and Sensing subsystem. The vehicle kinematics are modeled using a single-track bicycle kinematics model with two input signals, vehicle speed v (m/s) and steering angle δ (radians).
- The adaptive MPC and RL agent blocks are found in the MPC Tracking Controller and RL Controller subsystems, respectively.
- Mode switching between the controllers is handled by the Vehicle Mode Selector subsystem, which outputs an "isParking" signal indicating whether the vehicle is actively making a parking maneuver. Initially, the vehicle is in search mode and the adaptive MPC controller tracks the reference path. When a free spot is found, the "isParking" signal activates the RL Agent to perform the parking maneuver.
- The Unreal Engine Visualization and Sensing subsystem handles animation of the environment. You can choose to enable Unreal Engine visualization and/or 3-D point cloud visualization. Double-click this subsystem to specify the visualization options.



Unreal Engine visualization can slow down the simulation, so the `Enable Unreal Engine visualization` parameter is automatically disabled when training the agent.

Adaptive Model Predictive Controller Design

Create the adaptive MPC controller object for reference trajectory tracking using the `createMPCForParking3D` script. For more information on adaptive MPC, see “Adaptive and Time-Varying MPC” (Model Predictive Control Toolbox).

```
createMPCForParking3D;
```

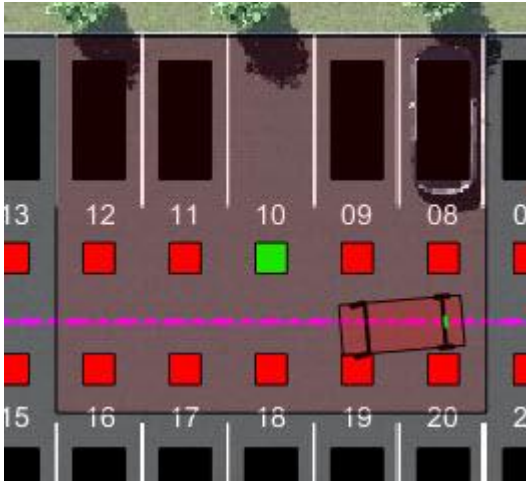
Reinforcement Learning Controller Design

To train the reinforcement learning agent, you must create an environment object and an agent object.

Create Environment

The environment for training is defined as a rectangular area surrounding the free parking spot, visualized by the region shaded in red in the following figure. Constraining the training to this region significantly reduces training duration when compared to training over the entire parking lot area.

The free spot selected for each training episode is randomly selected from one of four spot locations, with two on each side of the isle. This randomization is done to make the agent more robust to different vehicle shapes, sizes, colors, and lighting.



For this environment:

- The training region is a 13.625 m x 12.34 m space with the target spot at its horizontal center.
- The observations are the position errors X_e and Y_e of the ego vehicle with respect to the target pose, the heading angle error θ_e , and the five-channel lidar point cloud. The position and heading error observations are scaled to the range $[-1, 1]$ and the lidar data is scaled to the range $[0, 1]$.
- The vehicle speed during training varies randomly between 1 and 4 m/s each episode and is held constant for the entire episode.
- The action signal is a continuous steering angle that ranges between $\pm \pi/4$ radians.
- The vehicle is considered parked if the errors with respect to target pose are within specified tolerances of ± 0.75 m (position) and ± 10 degrees (heading).
- The episode is terminated if the ego vehicle parks successfully, leaves the training region, or collides with an obstacle.

The reward r_t provided at time t , is:

$$r_t = 2e^{-(0.05X_e^2 + 0.04Y_e^2)} + 0.5e^{-40\theta_e^2} - 0.05\delta^2 + 100f_t - 50g_t$$

Here, X_e , Y_e , and θ_e are the position and heading angle errors of the ego vehicle from the target pose, while δ is the steering angle. f_t (0 or 1) indicates whether the vehicle has parked and g_t (0 or 1) indicates if the vehicle has collided with an obstacle or left the training region at time t .

Create the observation specifications for the environment using the Simulink bus object stored in `ObservationBus.mat`.

```
load("ObservationBus.mat", "ObservationBus")
obsInfo = bus2RLSpec("ObservationBus");
obsNames = [obsInfo(1).Name, obsInfo(2).Name];
```

Create the action specifications for the environment.

```
nAct = 1;
actInfo = rlNumericSpec([nAct 1], LowerLimit=-1, UpperLimit=1);
actNames = "actionInput";
```

Specify the path to the RL Agent block and create the Simulink environment object.


```
agentBlock = mdl + "/Controller/RL Controller/RL Agent";
env = rlSimulinkEnv( ...
    mdl, ...
    agentBlock, ...
    obsInfo, ...
    actInfo, ...
    UseFastRestart="off");
```

Specify a reset function for training. The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `autoParkingValetResetFcn3D` function randomly resets the initial pose of the ego vehicle at the start of each episode. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @autoParkingValetResetFcn3D;
```

For more information on creating Simulink environments, see `rlSimulinkEnv`.

Create Agent

The agent in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents rely on actor and critic approximator objects to learn the optimal policy. To learn more about TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed of the random generator.

```
rng(0)
```

Create Critics

TD3 agents use two parameterized Q-value function approximators to estimate the value of the policy. The output of the critic network is the state-action value function for a taking a given action from a given observation.

Define each network path as an array of layer objects and assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
% Lidar path
lidarPath = [
    imageInputLayer( ...
        obsInfo(2).Dimension, ...
        Name="lidarData", ...
        Normalization="none")
    convolution2dLayer([5 5],8,Padding="same")
    reluLayer
    maxPooling2dLayer([3 3],Padding="same",Stride=[3 3])
    convolution2dLayer([5 5],8,Padding="same")
    reluLayer
    maxPooling2dLayer([3 3],Padding="same",Stride=[3 3])
    flattenLayer
    tanhLayer
    fullyConnectedLayer(512)
    reluLayer
```

```
fullyConnectedLayer(256,Name="lidar_fc"]];

% Pose path
posePath = [
    featureInputLayer(obsInfo(1).Dimension(1),Name="poseInfo")
    fullyConnectedLayer(64,Name="pose_fc"]];

% Action path
actionInput = [
    featureInputLayer(actInfo(1).Dimension(1),Name="actionInput")
    fullyConnectedLayer(32,Name="action_fc"]];

% Concatenated path
mainPath = [
    concatenationLayer(1,3,Name="concat")
    reluLayer
    fullyConnectedLayer(256)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(1,Name="criticOutLyr"]];

% Convert to layerGraph object and connect layers
criticNet = layerGraph(lidarPath);
criticNet = addLayers(criticNet,posePath);
criticNet = addLayers(criticNet,actionInput);
criticNet = addLayers(criticNet,mainPath);

criticNet = connectLayers(criticNet,"lidar_fc","concat/in1");
criticNet = connectLayers(criticNet,"pose_fc","concat/in2");
criticNet = connectLayers(criticNet,"action_fc","concat/in3");
```

Convert to a `dlnetwork` object and display information about the network including the number of learnable parameters and information on the network inputs.

```
criticdlnet = dlnetwork(criticNet);
summary(criticdlnet)
```

```
Initialized: true
```

```
Number of learnables: 585.8k
```

```
Inputs:
```

```
1  'lidarData'      16×360×5 images
2  'poseInfo'       3 features
3  'actionInput'    1 features
```

Create the critics using `criticdlnet`, the environment observation and action specifications, and the names of the network input layers to be connected with the environment observation and action channels. For more information, see `rlQValueFunction`.

```
critic = rlQValueFunction(criticdlnet,obsInfo,actInfo, ...
    ObservationInputNames=obsNames, ActionInputNames=actNames);
```

Create Actors

Create the neural network for the actor. The output of the actor network is the steering angle.

```

% Lidar path
lidarPath = [
    imageInputLayer( ...
        obsInfo(2).Dimension, ...
        Name="lidarData", ...
        Normalization="none")
    convolution2dLayer([5 5],8,Padding="same")
    reluLayer
    maxPooling2dLayer([3 3],Padding="same",Stride=[3 3])
    convolution2dLayer([5 5],8,Padding="same")
    reluLayer
    maxPooling2dLayer([3 3],Padding="same",Stride=[3 3])
    flattenLayer
    tanhLayer
    fullyConnectedLayer(512)
    reluLayer
    fullyConnectedLayer(256,Name="lidar_fc")];

% Pose path
posePath = [
    featureInputLayer(obsInfo(1).Dimension(1),Name="poseInfo")
    fullyConnectedLayer(64,Name="pose_fc")];

% Concatenated path
mainPath = [
    concatenationLayer(1,2,Name="concat")
    reluLayer
    fullyConnectedLayer(256)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(actInfo(1).Dimension(1),Name="criticOutLyr")
    tanhLayer];

% Convert to layerGraph object and connect layers
actorNet = layerGraph(lidarPath);
actorNet = addLayers(actorNet,posePath);
actorNet = addLayers(actorNet,mainPath);

actorNet = connectLayers(actorNet,"lidar_fc","concat/in1");
actorNet = connectLayers(actorNet,"pose_fc","concat/in2");

Convert to a dlnetwork object and display information about the network.
actordlnet = dlnetwork(actorNet);
summary(actordlnet)

    Initialized: true

    Number of learnables: 577.5k

    Inputs:
         1  'lidarData'    16×360×5 images
         2  'poseInfo'    3 features

Create the actor object for the TD3 agent. For more information, see
rlContinuousDeterministicActor.

actor = rlContinuousDeterministicActor(actordlnet,obsInfo,actInfo);

```

Specify the agent options and create the TD3 agent. For more information on TD3 agent options, see `rlTD3AgentOptions`.

```
agentOpts = rlTD3AgentOptions(SampleTime=Ts, ...
    DiscountFactor=0.99, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=256);
```

Set the noise options for exploration.

```
agentOpts.ExplorationModel.StandardDeviation = 0.1;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-4;
agentOpts.ExplorationModel.StandardDeviationMin = 0.01;
```

For this example, set the actor and critic learn rates to $1e-3$, respectively. Set a gradient threshold factor of 1 to limit the gradients during training. For more information, see `rlOptimizerOptions`.

Specify training options for the actor.

```
agentOpts.ActorOptimizerOptions.LearnRate = 1e-3;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
agentOpts.ActorOptimizerOptions.L2RegularizationFactor = 1e-3;
```

Specify training options for the critic.

```
agentOpts.CriticOptimizerOptions(1).LearnRate = 1e-3;
agentOpts.CriticOptimizerOptions(1).GradientThreshold = 1;
```

Create the agent using the actor, the critics, and the agent options objects. For more information, see `rlTD3Agent`.

```
agent = rlTD3Agent(actor,critic,agentOpts);
```

Train Agent

To train the agent, first specify the training options.

The agent is trained for a maximum of 10000 episodes with each episode lasting a maximum of 200 time steps. The training terminates when the maximum number of episodes is reached or the average reward over 200 episodes reaches a value of 116 or greater. Specify the options for training using the `rlTrainingOptions` function.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=10000, ...
    MaxStepsPerEpisode=200, ...
    ScoreAveragingWindowLength=200, ...
    Plots="training-progress", ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=116, ...
    UseParallel=false);
```

Train the agent using the `train` function. Fully training this agent is a computationally intensive process that might take several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true.

```
doTraining = false;
if doTraining
```

```

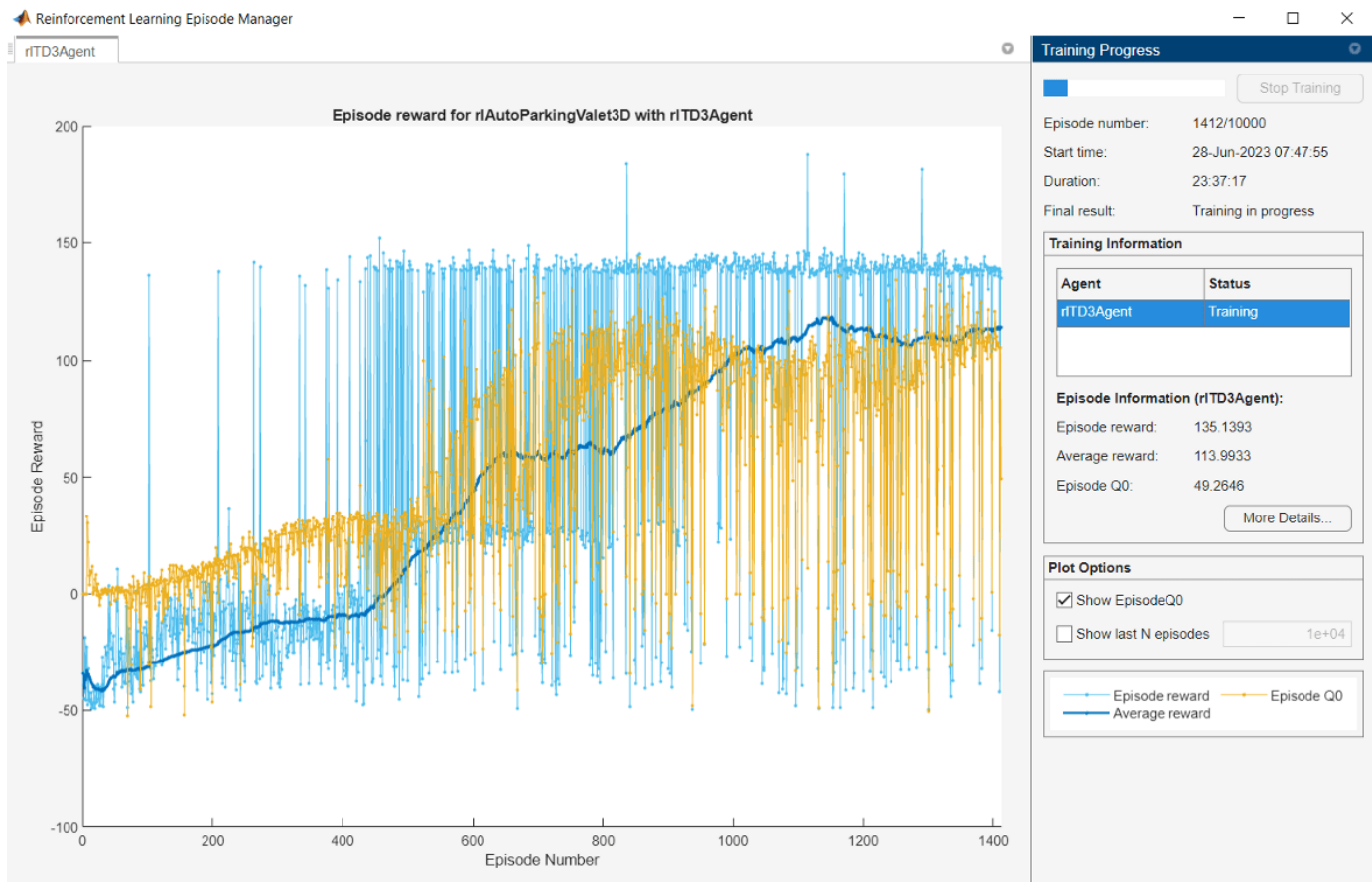
% Disable UE and point cloud visualization
setVisualizationOptions(UEViz="off",PCViz="off");

% If a GPU device is available, enable it for agent training
agent = setDeviceForAgentTraining(agent);

% Start training agent
trainingResult = train(agent,env,trainOpts);

else
    load("ParkingValetAgentTrained.mat","agent");
end

```



Simulate Parking Task

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To validate the trained agent, simulate the model and observe the parking maneuver. Enable both Unreal Engine and point cloud visualization to get a more complete view of the vehicle's behavior.

```

setVisualizationOptions(UEViz="on",PCViz="on");
sim mdl;

```

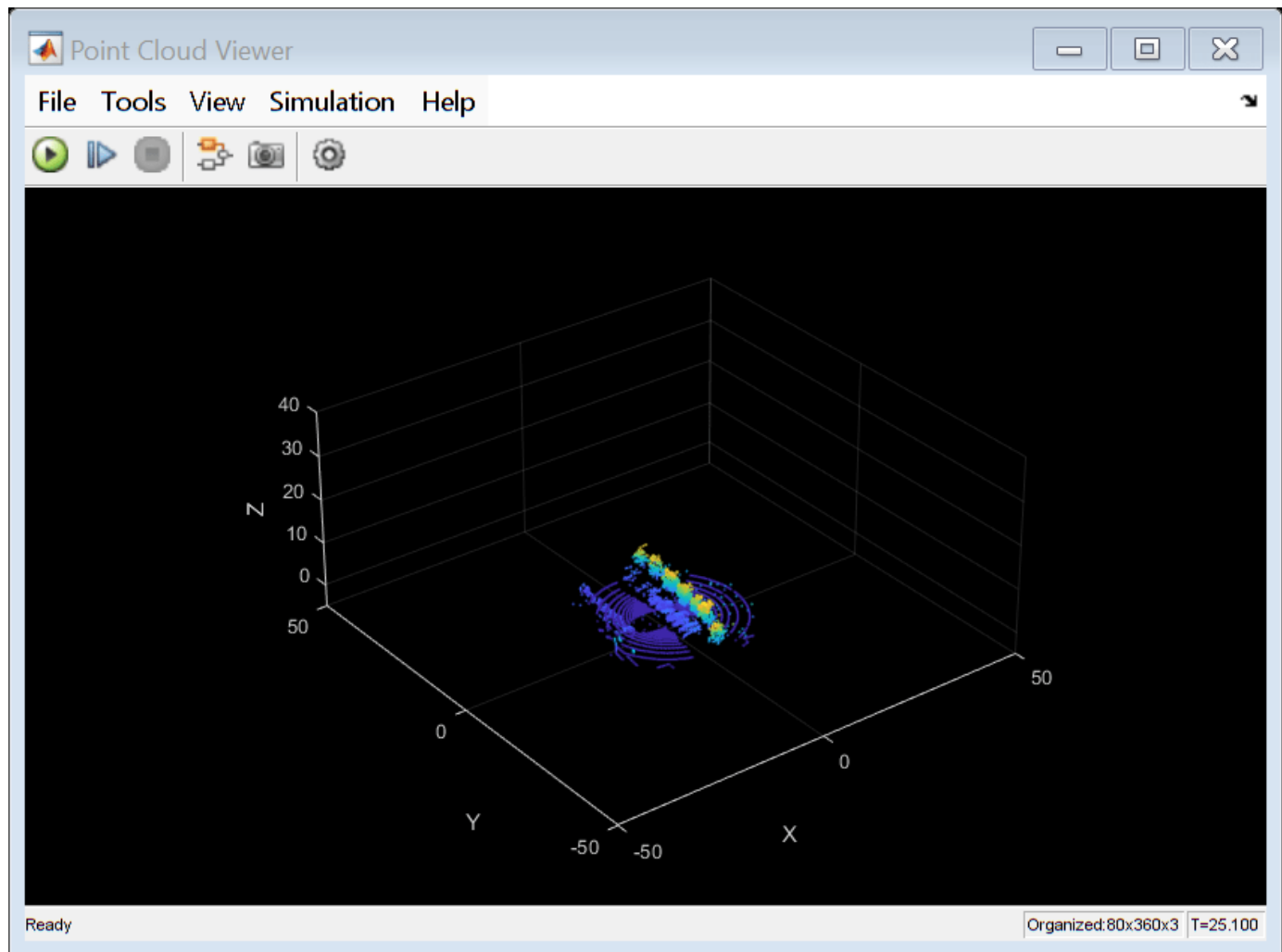
The vehicle tracks the reference path using the MPC controller before switching to the RL controller when the target spot is detected. The vehicle then completes the parking maneuver.

Change the free spot to the other side of the aisle and park the vehicle again.

```
freeSpotIndex = 6;  
setupActorVehicles mdl, freeSpotIndex);  
sim(mdl);
```

To view the trajectory graphically, open the Ego Vehicle Pose scope.

```
open_system(mdl + "/Vehicle Dynamics and Sensing/" + ...  
            "Ego Vehicle Model/Ego Vehicle Pose")
```





Local Functions

```
function agent = setDeviceForAgentTraining(agent)
% Determine whether a GPU device is available and
% enable it for training if so.

    if canUseGPU()
        agent.UseGPUForLearning = true;
        disp('GPU device found, using GPU for agent training.')
    else
        disp('No GPU device found, using CPU for agent training.')
    end
end

function setVisualizationOptions(options)
% This function takes two inputs, UEviz and PCviz.
% UEviz and PCviz are both Boolean scalars that enable or
% disable Unreal Engine visualization
% and point cloud visualization, respectively.

arguments
```

```
options.UEViz {mustBeTextScalar} = "on"
options.PCViz {mustBeTextScalar} = "on"
end

if options.UEViz == "on"
    set_param(strcat(bdroot, ...
        "/Vehicle Dynamics and Sensing/" + ...
        "Unreal Engine Visualization and Sensing" ), ...
        enableUEViz="on");
else
    set_param(strcat(bdroot, ...
        "/Vehicle Dynamics and Sensing/" + ...
        "Unreal Engine Visualization and Sensing" ), ...
        enableUEViz="off");
end

if options.PCViz == "on"
    set_param(strcat(bdroot, ...
        "/Vehicle Dynamics and Sensing/" + ...
        "Unreal Engine Visualization and Sensing" ), ...
        enablePCViz="on");
else
    set_param(strcat(bdroot, ...
        "/Vehicle Dynamics and Sensing/" + ...
        "Unreal Engine Visualization and Sensing" ), ...
        enablePCViz="off");
end
end
```

See Also

Functions

train | rlSimulinkEnv

Objects

rlTD3Agent | rlTD3AgentOptions | rlTrainingOptions | rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Train PPO Agent for Automatic Parking Valet” on page 8-85
- “Parking Valet Using Multistage Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Parallel Parking Using Nonlinear Model Predictive Control” (Model Predictive Control Toolbox)
- “Parallel Parking Using RRT Planner and MPC Tracking Controller” (Model Predictive Control Toolbox)

More About

- “Adaptive and Time-Varying MPC” (Model Predictive Control Toolbox)

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Reinforcement Learning Agents” on page 5-3

Generate Reward Function from a Model Predictive Controller for a Servomotor

This example shows how to automatically generate a reward function from cost and constraint specifications defined in a model predictive controller object. You then use the generated reward function to train a reinforcement learning agent.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

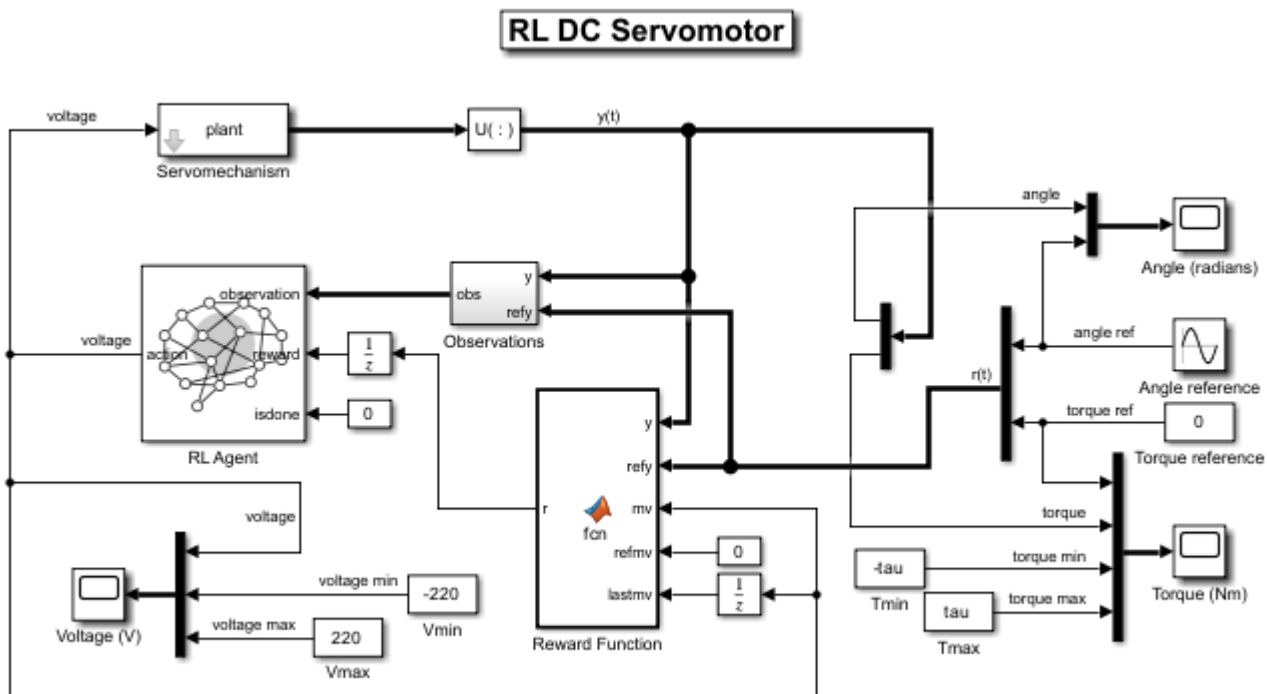
Introduction

You can use the `generateRewardFunction` to generate a reward function for reinforcement learning, starting from cost and constraints specified in a model predictive controller. The resulting reward signal is a sum of costs (as defined by an objective function) and constraint violation penalties depending on the current state of the environment.

This example is based on the “DC Servomotor with Constraint on Unmeasured Output” (Model Predictive Control Toolbox) example, in which you design a model predictive controller for a DC servomechanism under voltage and shaft torque constraints. Here, you will convert the cost and constraints specifications defined in the `mpc` object into a reward function and use it to train an agent to control the servomotor.

Open the Simulink® model for this example which is based on the above MPC example but has been modified for reinforcement learning.

```
mdl = "rl_motor";  
open_system(mdl);
```



Copyright 2021-2023 The MathWorks, Inc.

Create Model Predictive Controller

Create the open-loop dynamic model of the motor, defined in `plant` and the maximum admissible torque `tau` using a helper function.

```
[plant,tau] = mpcmotormodel;
```

Specify input and output signal types for the MPC controller. The shaft angular position is measured as a first output. The second output, torque, is unmeasurable.

```
plant = setmpcsignals(plant, MV=1, MO=1, UO=2);
```

Specify constraints on the manipulated variable, and define a scale factor.

```
MV = struct(Min=-220, Max=220, ScaleFactor=440);
```

Impose torque constraints during the first three prediction steps, and specify scale factor for both shaft position and torque.

```
OV = struct(Min={-Inf, [-tau;-tau;-tau;-Inf]}, ...
            Max={Inf, [tau;tau;tau;Inf]}, ...
            ScaleFactor={2*pi, 2*tau});
```

Specify weights for the quadratic cost function to achieve angular position tracking. Set to zero the weight for the torque, thereby allowing it to float within its constraint.

```
Weights = struct(MV=0, MVRate=0.1, OV=[0.1 0]);
```

Create an MPC controller for the `plant` model with a sample time of 0.1 s, a prediction horizon 10 steps, and a control horizon of 2 steps, using the previously defined structures for the weights, manipulated variables, and output variables.

```
mpcobj = mpc(plant, 0.1, 10, 2, Weights, MV, OV);
```

Display the controller specifications.

```
mpcobj
```

```
MPC object (created on 20-Apr-2026 10:51:41):
```

```
-----
Sampling time:      0.1 (seconds)
Prediction Horizon: 10
Control Horizon:    2
```

```
Plant Model:
```

```
-----
1 manipulated variable(s) --> 4 states |
0 measured disturbance(s) --> 1 inputs  | --> 1 measured output(s)
0 unmeasured disturbance(s) --> 2 outputs | --> 1 unmeasured output(s)
-----
```

```
Indices:
```

```
(input vector)    Manipulated variables: [1 ]
(output vector)   Measured outputs: [1 ]
                  Unmeasured outputs: [2 ]
```

```
Disturbance and Noise Models:
```

```
Output disturbance model: default (type "getoutdist(mpcobj)" for details)
Measurement noise model: default (unity gain after scaling)
```

```
Weights:
```

```
ManipulatedVariables: 0
ManipulatedVariablesRate: 0.1000
OutputVariables: [0.1000 0]
ECR: 10000
```

```
State Estimation: Default Kalman Filter (type "getEstimator(mpcobj)" for details)
```

```
Constraints:
```

```
-220 <= MV1 (V) <= 220, MV1/rate (V) is unconstrained,      M01 (rad) is unconstrained
-78.54 <= U01 (Nm)(t+1) <= 78.54
-78.54 <= U01 (Nm)(t+2) <= 78.54
-78.54 <= U01 (Nm)(t+3) <= 78.54
U01 (Nm)(t+4) is unconstrained
```

```
Use built-in "active-set" QP solver with MaxIterations of 120.
```

The controller operates on a plant with 4 states, 1 input (voltage) and 2 output signals (angle and torque) and has the following specifications:

- The cost function weights for the manipulated variable, manipulated variable rate and output variables are 0, 0.1 and [0.1 0] respectively.
- The manipulated variable is constrained between -220V and 220V.

- The manipulated variable rate is unconstrained.
- The first output variable (angle) is unconstrained but the second (torque) is constrained between -78.54 Nm and 78.54 Nm in the first three prediction time steps and unconstrained in the fourth step.

Note that for reinforcement learning only the constraints specification from the first prediction time step will be used since the reward is computed for a single time step.

Generate the Reward Function

Generate the reward function code from specifications in the `mpc` object using `generateRewardFunction`. The code is displayed in the MATLAB Editor.

```
generateRewardFunction(mpcobj)
```

The generated reward function is a starting point for reward design. You can modify the function with different penalty function choices and tune the weights. For this example, make the following change to the generated code:

- Scale the original cost weights `Qy` and `Qmvrate` by a factor of 50 and 10 respectively.
- Scale the penalty weights `Wy`, `Wmv` and `Wmvrate` by a factor of `1e-2`, `1e-3` and `1e-3` respectively.
- The default exterior penalty function method is `step`. Change the method to `quadratic`.

After you make changes, the cost and penalty specifications should be as follows:

```
Qy      = 50 * [0.1 0];
Qmv     = 0;
Qmvrate = 10 * 0.1;
Wy      = 1e-2 * [1 1];
Wmv     = 1e-3;
Wmvrate = 1e-3;
Py      = Wy      * exteriorPenalty(y,ymin,ymax,'quadratic');
Pmv     = Wmv     * exteriorPenalty(mv,mvmin,mvmax,'quadratic');
Pmvrate = Wmvrate * exteriorPenalty(mv-lastmv,mvratemin,mvratemax,'quadratic');
```

For this example, the modified code has been saved in the MATLAB® function file `rewardFunctionMpc.m`. Display the generated reward function.

```
type rewardFunctionMpc.m
```

```
function reward = rewardFunctionMpc(y,refy,mv,refmv,lastmv)
% REWARDFUNCTIONMPC generates rewards from MPC specifications.
%
% Description of input arguments:
%
% y : Output variable from plant at step k+1
% refy : Reference output variable at step k+1
% mv : Manipulated variable at step k
% refmv : Reference manipulated variable at step k
% lastmv : Manipulated variable at step k-1
%
% Limitations (MPC and NLMPC):
% - Reward computed based on first step in prediction horizon.
%   Therefore, signal previewing and control horizon settings are ignored.
% - Online cost and constraint update is not supported.
% - Custom cost and constraint specifications are not considered.
```

```
% - Time varying cost weights and constraints are not supported.
% - Mixed constraint specifications are not considered (for the MPC case).

% Reinforcement Learning Toolbox
% 02-Jun-2021 16:05:41

%#codegen

%% Specifications from MPC object
% Standard linear bounds as specified in 'States', 'OutputVariables', and
% 'ManipulatedVariables' properties
ymin = [-Inf -78.5398163397448];
ymax = [Inf 78.5398163397448];
mvmin = -220;
mvmax = 220;
mvratemin = -Inf;
mvratemax = Inf;

% Scale factors as specified in 'States', 'OutputVariables', and
% 'ManipulatedVariables' properties
Sy = [6.28318530717959 157.07963267949];
Smv = 440;

% Standard cost weights as specified in 'Weights' property
Qy = 50 * [0.1 0];
Qmv = 0;
Qmvrate = 10 * 0.1;

%% Compute cost
dy = (refy(:)-y(:)) ./ Sy';
dmv = (refmv(:)-mv(:)) ./ Smv';
dmvrate = (mv(:)-lastmv(:)) ./ Smv';
Jy = dy' * diag(Qy.^2) * dy;
Jmv = dmv' * diag(Qmv.^2) * dmv;
Jmvrate = dmvrate' * diag(Qmvrate.^2) * dmvrate;
Cost = Jy + Jmv + Jmvrate;

%% Penalty function weight (specify nonnegative)
Wy = 1e-2 * [1 1];
Wmv = 1e-3;
Wmvrate = 1e-3;

%% Compute penalty
% Penalty is computed for violation of linear bound constraints.
%
% To compute exterior bound penalty, use the exteriorPenalty function and
% specify the penalty method as 'step' or 'quadratic'.
%
% Alternatively, use the hyperbolicPenalty or barrierPenalty function for
% computing hyperbolic and barrier penalties.
%
% For more information, see help for these functions.
%
% Set Pmv value to 0 if the RL agent action specification has
% appropriate 'LowerLimit' and 'UpperLimit' values.
Py = Wy * exteriorPenalty(y,ymin,ymax,'quadratic');
Pmv = Wmv * exteriorPenalty(mv,mvmin,mvmax,'quadratic');
Pmvrate = Wmvrate * exteriorPenalty(mv-lastmv,mvratemin,mvratemax,'quadratic');
```

```
Penalty = Py + Pmv + Pmvrate;

%% Compute reward
reward = -(Cost + Penalty);
end
```

To integrate this reward function, open the MATLAB Function block in the Simulink model.

```
open_system("rl_motor/Reward Function")
```

Append the function with the following line of code and save the model.

```
r = rewardFunctionMpc(y,refy,mv,refmv,lastmv);
```

The MATLAB Function block will now execute `rewardFunctionMpc.m` during simulation.

For this example, the MATLAB Function block has already been modified and saved.

Create a Reinforcement Learning Environment

The environment dynamics are modeled in the **Servomechanism** subsystem. For this environment,

- The observations are the reference signals (angle and torque), output variables (angle and torque), and their integrals from the last 3 time steps. The angle and torque signals are normalized by multiplying with the gain $[0.1 \ 1/\tau]$.
- The action is the voltage V applied to the servomotor. The action values are limited between -220 and 220.
- The sample time is $T_s = 0.1$ s.
- The total simulation time is $T_f = 20$ s.

Specify the total simulation time and sample time.

```
Tf = 20;
Ts = 0.1;
```

Create observation and action specifications for the environment.

```
numObs = 24;
numAct = 1;
oinfo = rlNumericSpec([numObs 1]);
ainfo = rlNumericSpec([numAct 1], ...
    LowerLimit=-220, ...
    UpperLimit=220);
```

Create the reinforcement learning environment using the `rlSimulinkEnv` function.

```
blk = "rl_motor/RL Agent";
env = rlSimulinkEnv mdl, blk, oinfo, ainfo);
```

Create a Reinforcement Learning Agent

The agent used in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents use two parameterized Q-value function approximators to estimate the value (that is, the expected cumulative long-term reward) of the policy. To model the parameterized Q-value function within both critics, use a neural network with two inputs (the observation and action) and one output (the value of the policy when taking a given action from the state corresponding to a given

observation). For more information on TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

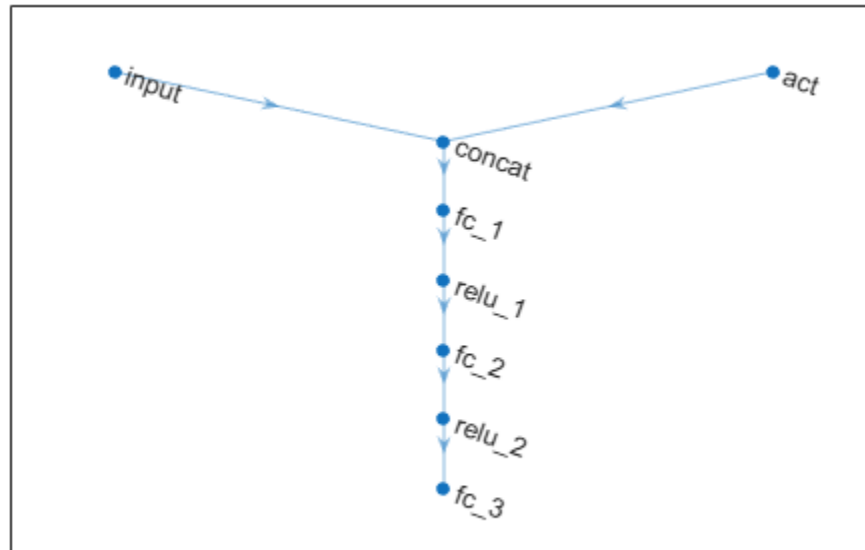
```
% Define paths.
mainPath = [
    featureInputLayer(numObs)
    concatenationLayer(1, 2, Name="concat")
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(1)
];
actionPath = [
    featureInputLayer(numAct, Name="act")
];

% Create dlnetwork object and add layers.
criticNet = dlnetwork;
criticNet = addLayers(criticNet, mainPath);
criticNet = addLayers(criticNet, actionPath);

% Connect Layers.
criticNet = connectLayers(criticNet, "act", "concat/in2");
```

Plot the critic network structure.

```
plot(criticNet);
```

Create the critic function objects using `rlQValueFunction`. The critic function object encapsulates the critic by wrapping around the critic deep neural network. To make sure the critics have different initial weights, explicitly initialize each network before using them to create the critics.

```
% Create the two critic functions for the TD3 agent
critic1 = rlQValueFunction(initialize(criticNet), oinfo, ainfo);
critic2 = rlQValueFunction(initialize(criticNet), oinfo, ainfo);
```

TD3 agents learn a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `ainfo`).

Define the network as an array of layer objects.

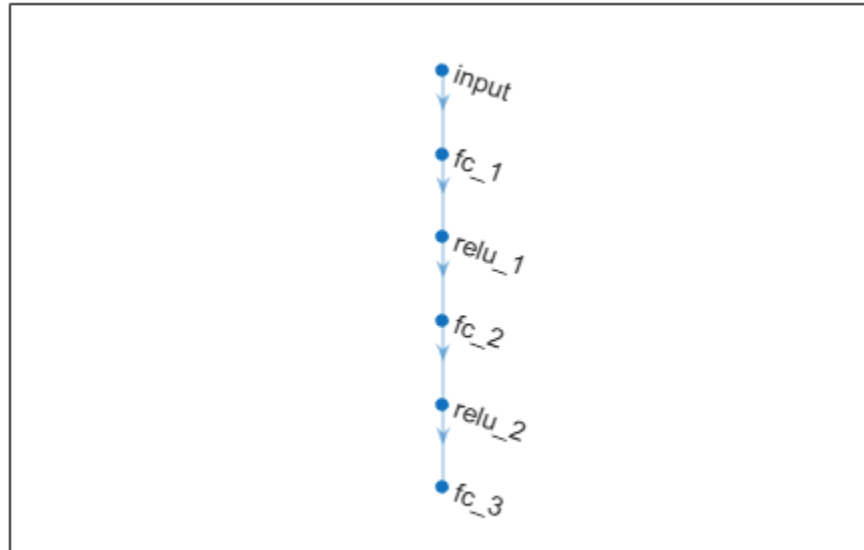
```
actorNet = [
    featureInputLayer(numObs)
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(64)
    reluLayer
    fullyConnectedLayer(numAct)
];
```

Convert to `dlnetwork` object.

```
actorNet = dlnetwork(actorNet);
```

Plot the actor network.

```
plot(actorNet);
```



Create a deterministic actor function that is responsible for modeling the policy of the agent. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor(actorNet, oinfo, ainfo);
```

Specify the following agent options using `rlTD3AgentOptions`:

- Specify the discount factor of `0.995` to favor long-term rewards.
- Store experiences in an experience buffer of maximum capacity `1e6`.
- Specify a mini-batch size of `1024` for learning. Smaller mini-batches are computationally efficient but might introduce variance in training. Contrarily, larger batch sizes can make the training stable but require higher memory.
- Collect experiences up to `1024` steps (use the `NumWarmStartSteps` option) before first executing the learning algorithm.
- Update the actor and critic networks using `10` learning epochs.
- Specify the actor and critic learning rates to be `2e-3` and `5e-3` respectively. A large learning rate causes drastic updates which might lead to divergent behaviors, while a low value might require many updates before reaching the optimal point.
- Use a gradient threshold of `1` to clip the gradients. Clipping the gradients can improve training stability.

```
agentOpts = rlTD3AgentOptions(SampleTime=Ts, ...
    DiscountFactor=0.995, ...
    ExperienceBufferLength=1e6, ...
    NumWarmStartSteps=1024, ...
    NumEpoch=10, ...
    MiniBatchSize=1024);
```

```

% Critic optimizer options
for idx = 1:2
    agentOpts.CriticOptimizerOptions(idx).LearnRate = 5e-3;
    agentOpts.CriticOptimizerOptions(idx).GradientThreshold = 1;
end

% Actor optimizer options
agentOpts.ActorOptimizerOptions.LearnRate = 2e-3;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;

```

During training, the agent explores the action space using a Gaussian action noise model. Set the standard deviation and decay rate of the noise using the `ExplorationModel` property. The noise has an initial standard deviation of 50 which exponentially decays at the rate of $1e-4$ until it reaches a minimum value of 10. This favors exploration toward the beginning of training and exploitation in later stages.

```

agentOpts.ExplorationModel.StandardDeviation = 50;
agentOpts.ExplorationModel.StandardDeviationMin = 10;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-4;

```

Create the TD3 agent using the actor and critic representations. For more information on TD3 agents, see `rlTD3Agent`.

```
agent = rlTD3Agent(actor, [critic1,critic2], agentOpts);
```

Train the Agent

To train the agent, first specify the training options using `rlTrainingOptions`. For this example, use the following options:

- Run each training for a maximum of 5000 episodes, with each episode lasting at most $\text{ceil}(T_f/T_s)$ time steps.
- Stop the training when the statistic returned by the evaluator object used with train equals or exceeds the specified value of -12. At this point, the agent can track the reference signal.

```

trainOpts = rlTrainingOptions( ...
    MaxEpisodes=5000, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-12, ...
    ScoreAveragingWindowLength=20);

```

To evaluate the agent during training, create an `rlEvaluator` object. Configure the evaluator to run 1 evaluation episode every 50 training episodes.

```

evl = rlEvaluator( ...
    NumEpisodes=1, ...
    EvaluationFrequency=50);

```

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

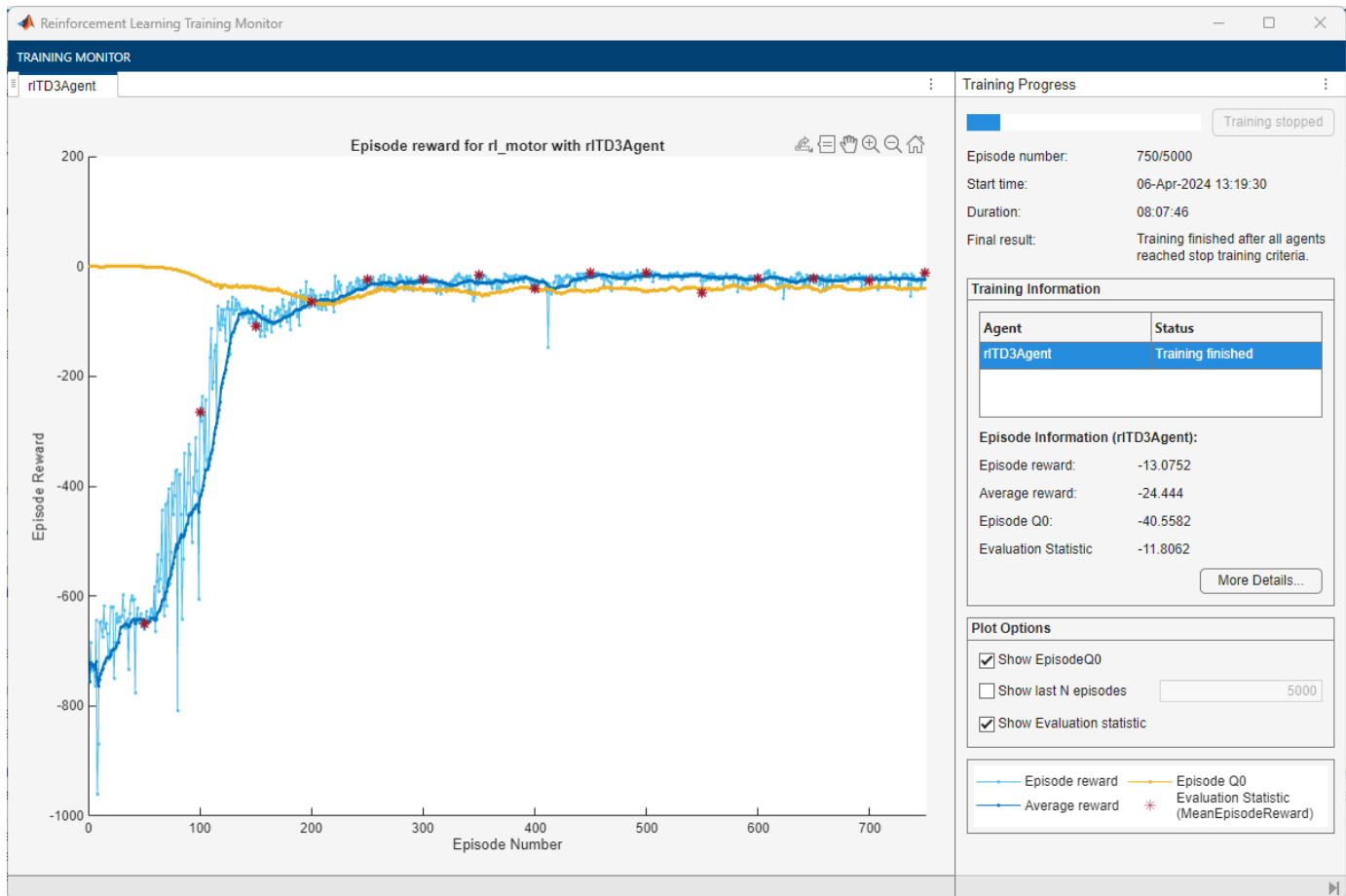
Train the agent using the `train` function. Training this agent is a computationally intensive process that might take several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true.

```

doTraining = ;
if doTraining
    trainResult = train(agent, env, trainOpts, Evaluator=evl);
else
    load('rLDCServomotorTD3Agent.mat')
end

```

A snapshot of the training progress is shown in the following figure. You can expect different results due to inherent randomness in the training process.



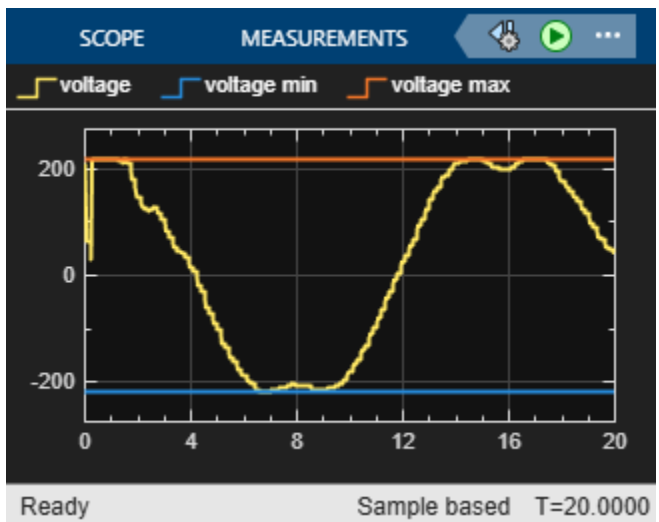
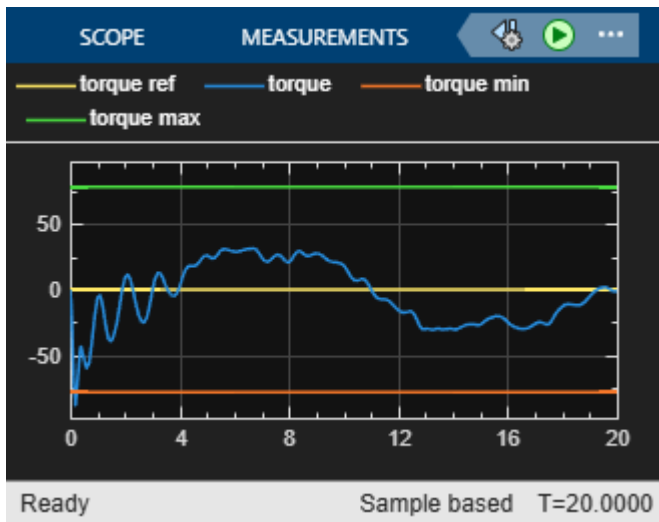
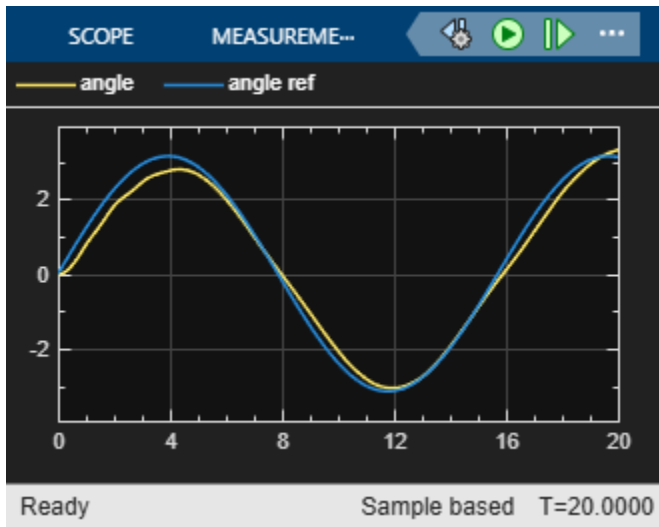
Validate Controller Response

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To validate the performance of the trained agent, simulate the model and view the response in the Scope blocks. The reinforcement learning agent is able to track the reference angle while satisfying the constraints on torque and voltage.

```
sim mdl;
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Copyright 2021-2024 The MathWorks, Inc..

See Also

Functions

`generateRewardFunction` | `train` | `rlSimulinkEnv`

Objects

`rlTD3Agent` | `rlTD3AgentOptions` | `rlTrainingOptions`

Blocks

RL Agent

Related Examples

- “DC Servomotor with Constraint on Unmeasured Output” (Model Predictive Control Toolbox)
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Generate Reward Function from a Model Verification Block for a Water Tank System” on page 8-227

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Reinforcement Learning Agents” on page 5-3
- “What Is Model Predictive Control?” (Model Predictive Control Toolbox)

Generate Reward Function from a Model Verification Block for a Water Tank System

This example shows how to automatically generate a reward function from performance requirement defined in a Simulink® Design Optimization™ model verification block. You then use the generated reward function to train a reinforcement learning agent.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Introduction

You can use the `generateRewardFunction` to generate a reward function for reinforcement learning, starting from performance constraints specified in a Simulink Design Optimization model verification block. The resulting reward signal is a sum of weighted penalties on constraint violations by the current state of the environment.

In this example, you will convert the cost and constraint specifications defined in a **Check Step Response Characteristics** block for a water tank system into a reward function. You then use the reward function and use it to train an agent to control the water tank.

Specify parameters for this example.

```
% Water tank parameters
a = 2;
b = 5;
A = 20;

% Initial and final height
h0 = 1;
hf = 2;

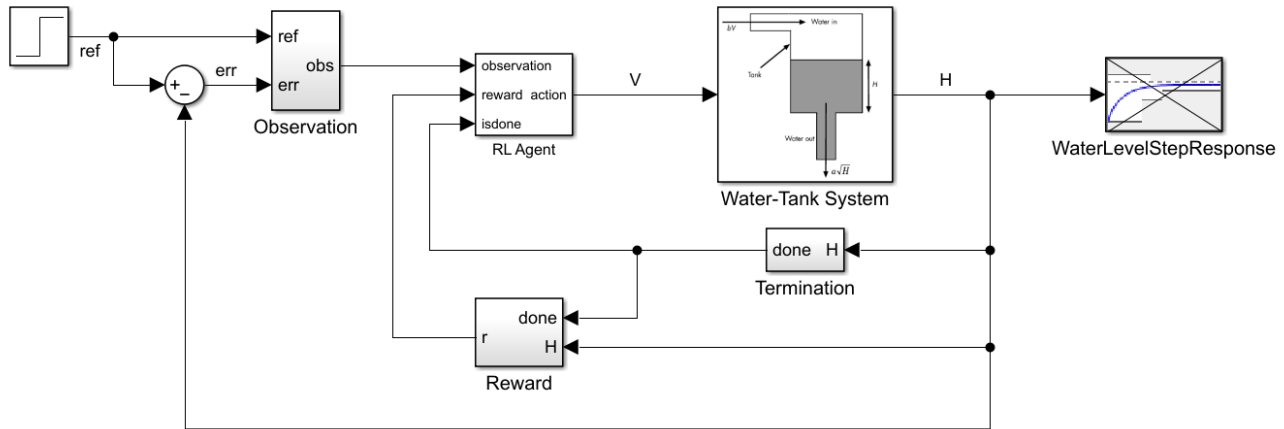
% Simulation and sample times
Tf = 10;
Ts = 0.1;
```

For more information on the original model for this example, see “watertank Simulink Model” (Simulink Control Design).

Open the model.

```
mdl = "rlWatertankStepInput";
open_system(mdl)
```

RL Watertank Step Input

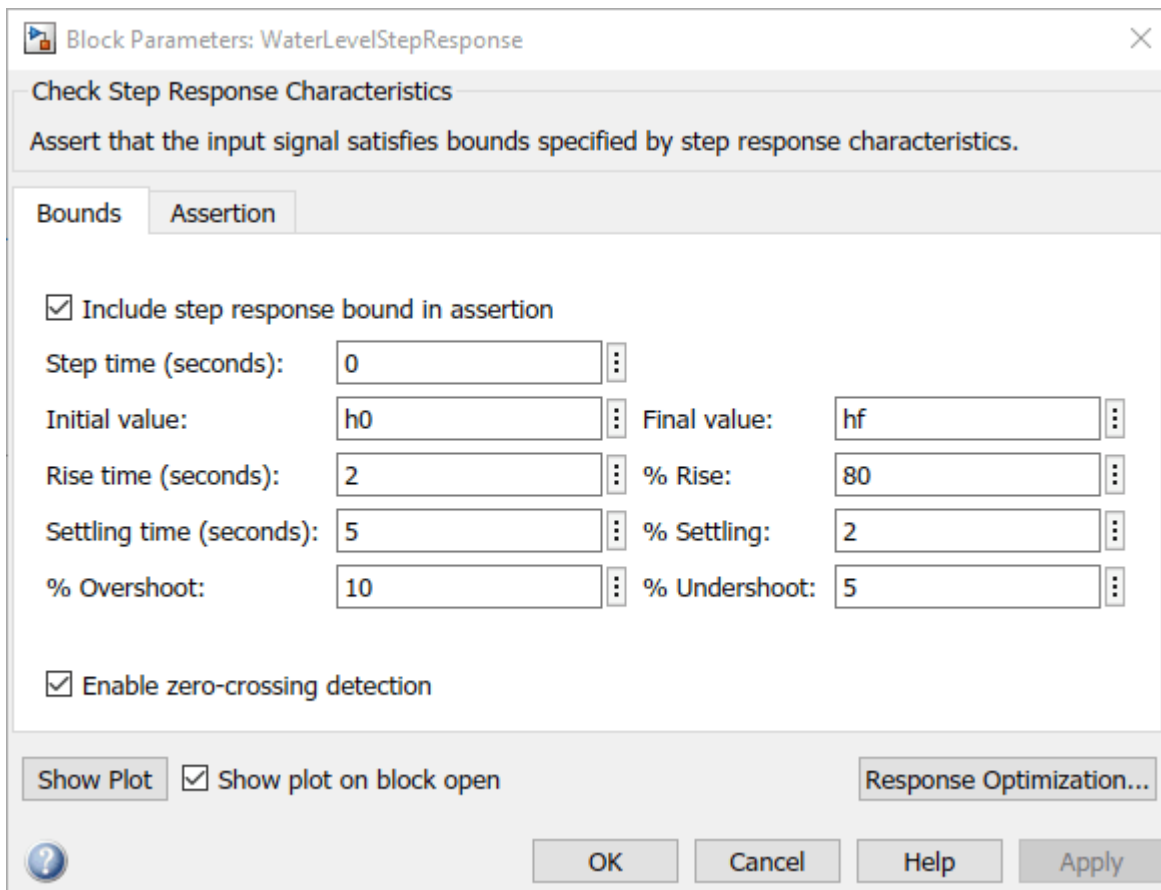


Copyright 2021 The MathWorks, Inc.

The model in this example has been modified for reinforcement learning. The goal is to control the level of the water in the tank using a reinforcement learning agent, while satisfying the response characteristics defined in the **Check Step Response Characteristics** block. For more information, see [Check Step Response Characteristics \(Simulink Design Optimization\)](#).

Open the block to view the desired step response specifications.

```
verifBlk = mdl + "/WaterLevelStepResponse";
open_system(verifBlk)
```

Block Parameters: WaterLevelStepResponse

Check Step Response Characteristics

Assert that the input signal satisfies bounds specified by step response characteristics.

Bounds **Assertion**

☒ Include step response bound in assertion

Step time (seconds):

Initial value: Final value:

Rise time (seconds): % Rise:

Settling time (seconds): % Settling:

% Overshoot: % Undershoot:

☒ Enable zero-crossing detection

 ☒ Show plot on block open

Generate a Reward Function

Generate the reward function code from specifications in the **WaterLevelStepResponse** block using `generateRewardFunction`. The code is displayed in the MATLAB Editor.

```
generateRewardFunction(blk)
```

The generated reward function is a starting point for reward design. You might modify the function by choosing different penalty functions and tuning the penalty weights. For this example, make the following change to the generated code:

- The default penalty weight is 1. Set weight to 10.
- The default exterior penalty function method is step. Change the method to **quadratic**.

After you make changes, the weight and penalty specifications should be as follows:

```
Weight = 10;
Penalty = sum(exteriorPenalty(x,Block1_xmin,Block1_xmax,'quadratic'));
```

For this example, the modified code has been saved in the MATLAB® function file `rewardFunctionVfb.m`. Display the generated reward function.

```
type rewardFunctionVfb.m

function reward = rewardFunctionVfb(x,t)
% REWARDFUNCTION generates rewards
```

```
% from Simulink block specifications.
%
% x : Input of
% watertank_stepinput_rl/WaterLevelStepResponse
% t : Simulation time (s)

% Reinforcement Learning Toolbox
% 26-Apr-2021 13:05:16

%#codegen

%% Specifications from
% watertank_stepinput_rl/WaterLevelStepResponse
Block1_InitialValue = 1;
Block1_FinalValue = 2;
Block1_StepTime = 0;
Block1_StepRange = Block1_FinalValue - ...
    Block1_InitialValue;

Block1_MinRise = Block1_InitialValue + ...
    Block1_StepRange * 80/100;

Block1_MaxSettling = Block1_InitialValue + ...
    Block1_StepRange * (1+2/100);

Block1_MinSettling = Block1_InitialValue + ...
    Block1_StepRange * (1-2/100);

Block1_MaxOvershoot = Block1_InitialValue + ...
    Block1_StepRange * (1+10/100);

Block1_MinUndershoot = Block1_InitialValue - ...
    Block1_StepRange * 5/100;

if t >= Block1_StepTime
    if Block1_InitialValue <= Block1_FinalValue

        Block1_UpperBoundTimes = [0,5; 5,max(5+1,t+1)];

        Block1_UpperBoundAmplitudes = [Block1_MaxOvershoot, ...
            Block1_MaxOvershoot; ...
            Block1_MaxSettling, ...
            Block1_MaxSettling];

        Block1_LowerBoundTimes = [0,2; 2,5; 5,max(5+1,t+1)];

        Block1_LowerBoundAmplitudes = [Block1_MinUndershoot, ...
            Block1_MinUndershoot; ...
            Block1_MinRise, ...
            Block1_MinRise; ...
            Block1_MinSettling, ...
            Block1_MinSettling];

    else

        Block1_UpperBoundTimes = [0,2; 2,5; 5,max(5+1,t+1)];

        Block1_UpperBoundAmplitudes = [Block1_MinUndershoot, ...
```

```

        Block1_MinUndershoot; ...
        Block1_MinRise, ...
        Block1_MinRise; ...
        Block1_MinSettling, ...
        Block1_MinSettling];

    Block1_LowerBoundTimes = [0,5; 5,max(5+1,t+1)];

    Block1_LowerBoundAmplitudes = [Block1_MaxOvershoot, ...
        Block1_MaxOvershoot; ...
        Block1_MaxSettling, ...
        Block1_MaxSettling];

end

Block1_xmax = zeros(1,size(Block1_UpperBoundTimes,1));

for idx = 1:numel(Block1_xmax)
    tseg = Block1_UpperBoundTimes(idx,:);
    xseg = Block1_UpperBoundAmplitudes(idx,:);
    Block1_xmax(idx) = interp1(tseg,xseg,t,'linear',NaN);
end

if all(isnan(Block1_xmax))
    Block1_xmax = Inf;
else
    Block1_xmax = max(Block1_xmax,[],'omitnan');
end

Block1_xmin = zeros(1,size(Block1_LowerBoundTimes,1));
for idx = 1:numel(Block1_xmin)
    tseg = Block1_LowerBoundTimes(idx,:);
    xseg = Block1_LowerBoundAmplitudes(idx,:);
    Block1_xmin(idx) = interp1(tseg,xseg,t,'linear',NaN);
end

if all(isnan(Block1_xmin))
    Block1_xmin = -Inf;
else
    Block1_xmin = max(Block1_xmin,[],'omitnan');
end

else
    Block1_xmin = -Inf;
    Block1_xmax = Inf;
end

%% Penalty function weight (specify nonnegative)
Weight = 10;

%% Compute penalty
% Penalty is computed for violation
% of linear bound constraints.
%
% To compute exterior bound penalty,
% use the exteriorPenalty function and
% specify the penalty method
% as 'step' or 'quadratic'.
%
```

```
% Alternately, use the hyperbolicPenalty
% or barrierPenalty function for
% computing hyperbolic and barrier penalties.
%
% For more information, see help for these functions.
Penalty = sum(exteriorPenalty( ...
    x,Block1_xmin,Block1_xmax, ...
    'quadratic'));

%% Compute reward
reward = -Weight * Penalty;
end
```

To integrate this reward function in the water tank model, open the MATLAB Function block under the Reward Subsystem.

```
open_system mdl + "/Reward/Reward Function")
```

Append the function with the following line of code and save the model.

```
r = rewardFunctionVfb(x,t);
```

The MATLAB Function block will now execute `rewardFunctionVfb.m` for computing rewards.

For this example, the MATLAB Function block has already been modified and saved.

Create a Reinforcement Learning Environment

The environment dynamics are modeled in the Water-Tank Subsystem. For this environment,

- The observations are the reference height `ref` from the last 5 time steps, and the height error is `err = ref - H`.
- The action is the voltage `V` applied to the pump.
- The sample time `Ts` is 0.1 s.

Create observation and action specifications for the environment.

```
numObs = 6;
numAct = 1;
oinfo = rlNumericSpec([numObs 1]);
ainfo = rlNumericSpec([numAct 1]);
```

Create the reinforcement learning environment using the `rlSimulinkEnv` function.

```
agentBlk = mdl + "/RL Agent";
env = rlSimulinkEnv(mdl, agentBlk, oinfo, ainfo);
```

Create a Reinforcement Learning Agent

The agent used in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents use two parameterized Q-value function approximators to estimate the value (that is, the expected cumulative long-term reward) of the policy. To model the parameterized Q-value function within both critics, use a neural network with two inputs (the observation and action) and one output (the value of the policy when taking a given action from the state corresponding to a given observation). For more information on TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

When you create the agent, the initial parameters of the actor and critic networks are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
mainPath = [
    featureInputLayer(numObs)
    fullyConnectedLayer(128)
    concatenationLayer(1, 2, Name="concat")
    reluLayer()
    fullyConnectedLayer(128)
    reluLayer()
    fullyConnectedLayer(1)];
actionPath = [
    featureInputLayer(numAct)
    fullyConnectedLayer(8, Name="fc_act")];
```

Create a dlnetwork object and add layers.

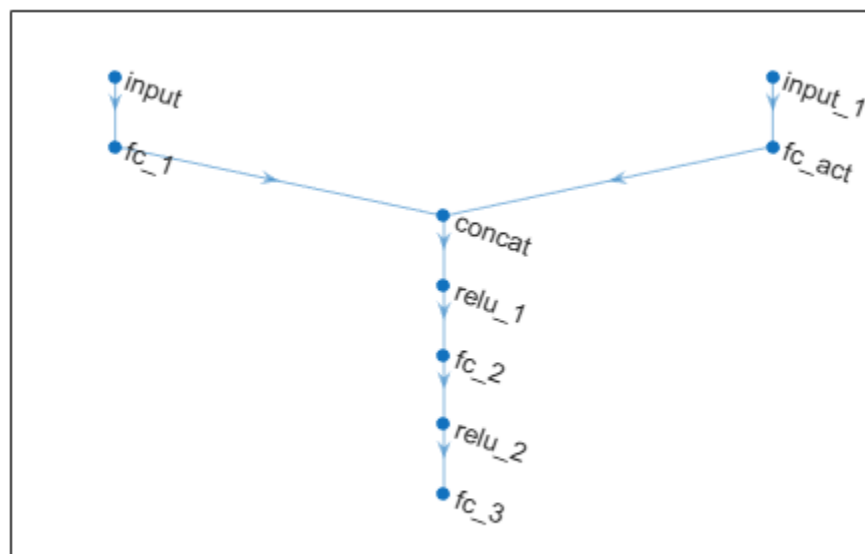
```
criticNet = dlnetwork();
criticNet = addLayers(criticNet, mainPath);
criticNet = addLayers(criticNet, actionPath);
```

Connect the layers.

```
criticNet = connectLayers(criticNet, "fc_act", "concat/in2");
```

Plot the critic network structure.

```
plot(criticNet);
```



Display the number of weights.

```
summary(initialize(criticNet));

Initialized: true

Number of learnables: 18.6k

Inputs:
  1  'input'      6 features
  2  'input_1'    1 features
```

Create the critic function objects using `rlQValueFunction`. The critic function object encapsulates the critic by wrapping around the critic deep neural network. To make sure the critics have different initial weights, explicitly initialize each network before using them to create the critics.

```
critic1 = rlQValueFunction(initialize(criticNet), oinfo, ainfo);
critic2 = rlQValueFunction(initialize(criticNet), oinfo, ainfo);
```

TD3 agents learn a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `ainfo`).

Define the network as an array of layer objects.

```
actorNet = [
    featureInputLayer(numObs)
    fullyConnectedLayer(128)
    reluLayer()
    fullyConnectedLayer(128)
    reluLayer()
    fullyConnectedLayer(numAct)
];
```

Convert to `dlnetwork` object, initialize network and display the number of weights.

```
actorNet = dlnetwork(actorNet);
actorNet = initialize(actorNet);
summary(actorNet)
```

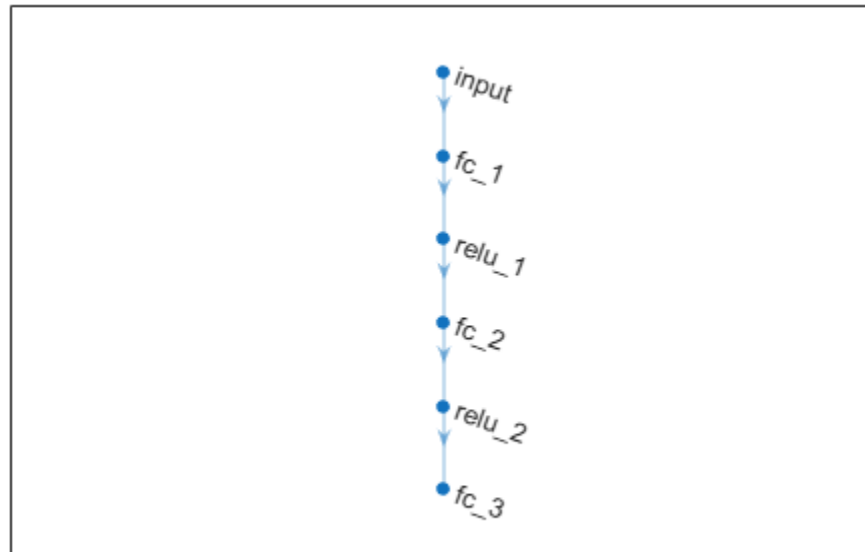
```
Initialized: true

Number of learnables: 17.5k

Inputs:
  1  'input'      6 features
```

Plot the actor network.

```
plot(actorNet);
```



Create a deterministic actor function that is responsible for modeling the policy of the agent. For more information, see `rlContinuousDeterministicActor`.

```
actordlNet = initialize(actorNet);
actor = rlContinuousDeterministicActor(actordlNet, oinfo, ainfo);
```

Specify the agent options using `rlTD3AgentOptions`.

- Use the experience buffer length of 1e6 for storing experiences. A larger experience buffer can store a diverse set of experiences.
- Use mini-batches of 512 experiences for learning. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Use a discount factor of 0.99 that favors long term rewards.
- Use 1e3 warm start steps to initially delay the learning process.

```
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts, ...
    DiscountFactor=0.99, ...
    ExperienceBufferLength=1e6, ...
    NumWarmStartSteps=1e3, ...
    MiniBatchSize=512);
```

The exploration model in this TD3 agent is a Gaussian noise model. During training the agent explores by sampling from this noise distribution.

- Set the initial standard deviation of the noise to 0.3.
- Set a decay rate of 1e-5 for the standard deviation parameter. This promotes exploration toward the beginning when the agent does not have a good policy, and exploitation toward the end when the agent has learned the optimal policy.

```
agentOpts.ExplorationModel.StandardDeviation = 0.3;  
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-5;
```

Specify the learn rate of 1e-3 for the actor, and 2e-3 for the critic models of the agent. Use a gradient threshold value of 10 for both actor and critics to stabilize gradients during learning.

```
agentOpts.ActorOptimizerOptions.LearnRate = 1e-3;  
agentOpts.ActorOptimizerOptions.GradientThreshold = 10;  
for i = 1:2  
    agentOpts.CriticOptimizerOptions(i).LearnRate = 2e-3;  
    agentOpts.CriticOptimizerOptions(i).GradientThreshold = 10;  
end
```

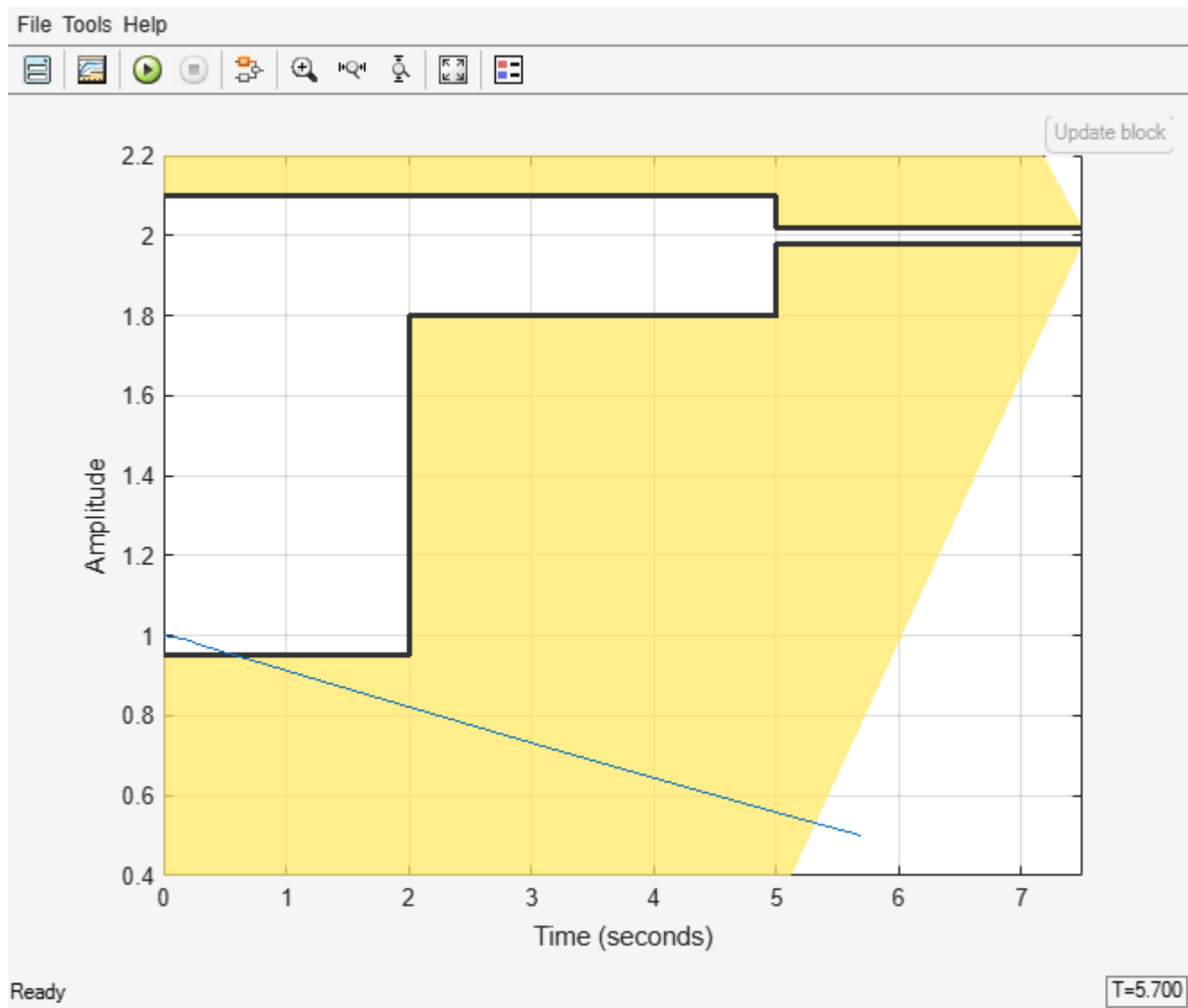
Create the TD3 agent using the actor and critic representations. For more information on TD3 agents, see `rlTD3Agent`.

```
agent = rlTD3Agent(actor, [critic1, critic2], agentOpts);  
agent.ExperienceBuffer = rlPrioritizedReplayMemory(oinfo,ainfo);
```

Closed Loop Response

Simulate the model to view the closed loop step response of the untrained agent. The response does not satisfy the performance bounds.

```
sim mdl;
```

Train the Agent

You will train the agent to satisfy the performance constraints. First specify the training options using `rlTrainingOptions`. For this example, use the following options:

- Run training for a maximum of 500 episodes, with each episode lasting at most $\text{ceil}(T_f/T_s)$ time steps, where the total simulation time T_f is 10 s.
- Evaluate the performance of the greedy policy by running 1 evaluation episode every 10 training episodes.
- Stop the training when the evaluation score reaches 0. At this point the agent is able to satisfy the performance constraints.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=500, ...
    MaxStepsPerEpisode=ceil(Tf/Ts), ...
    StopTrainingCriteria="EvaluationStatistic", ...
```

```
StopTrainingValue=0, ...  
ScoreAveragingWindowLength=20);
```

```
evl = rlEvaluator( ...  
    NumEpisodes=1, ...  
    EvaluationFrequency=10);
```

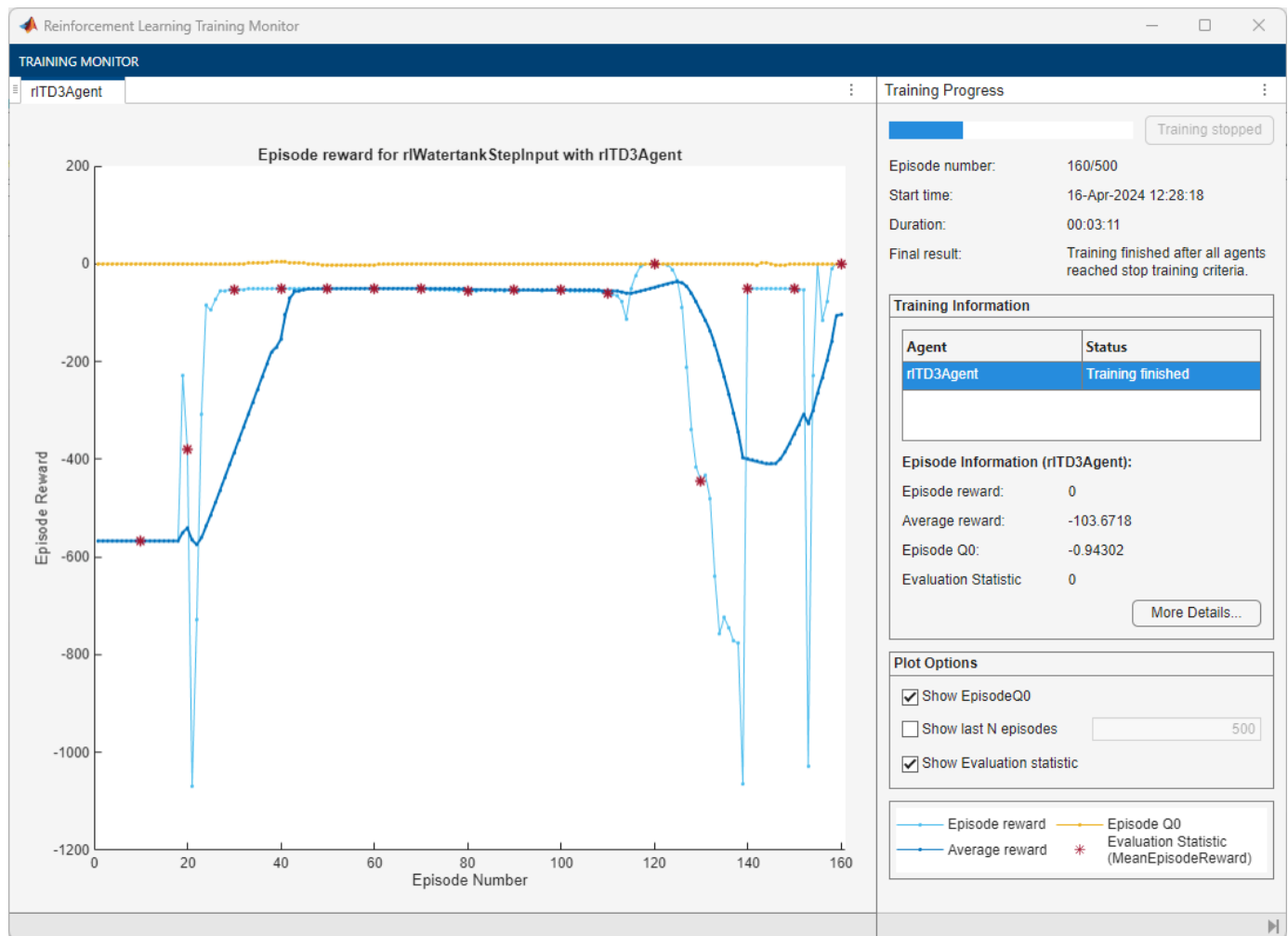
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that might take several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    trainingStats = train(agent, env, trainOpts, Evaluator=evl);  
else  
    load("rlWatertankTD3Agent.mat")  
end
```

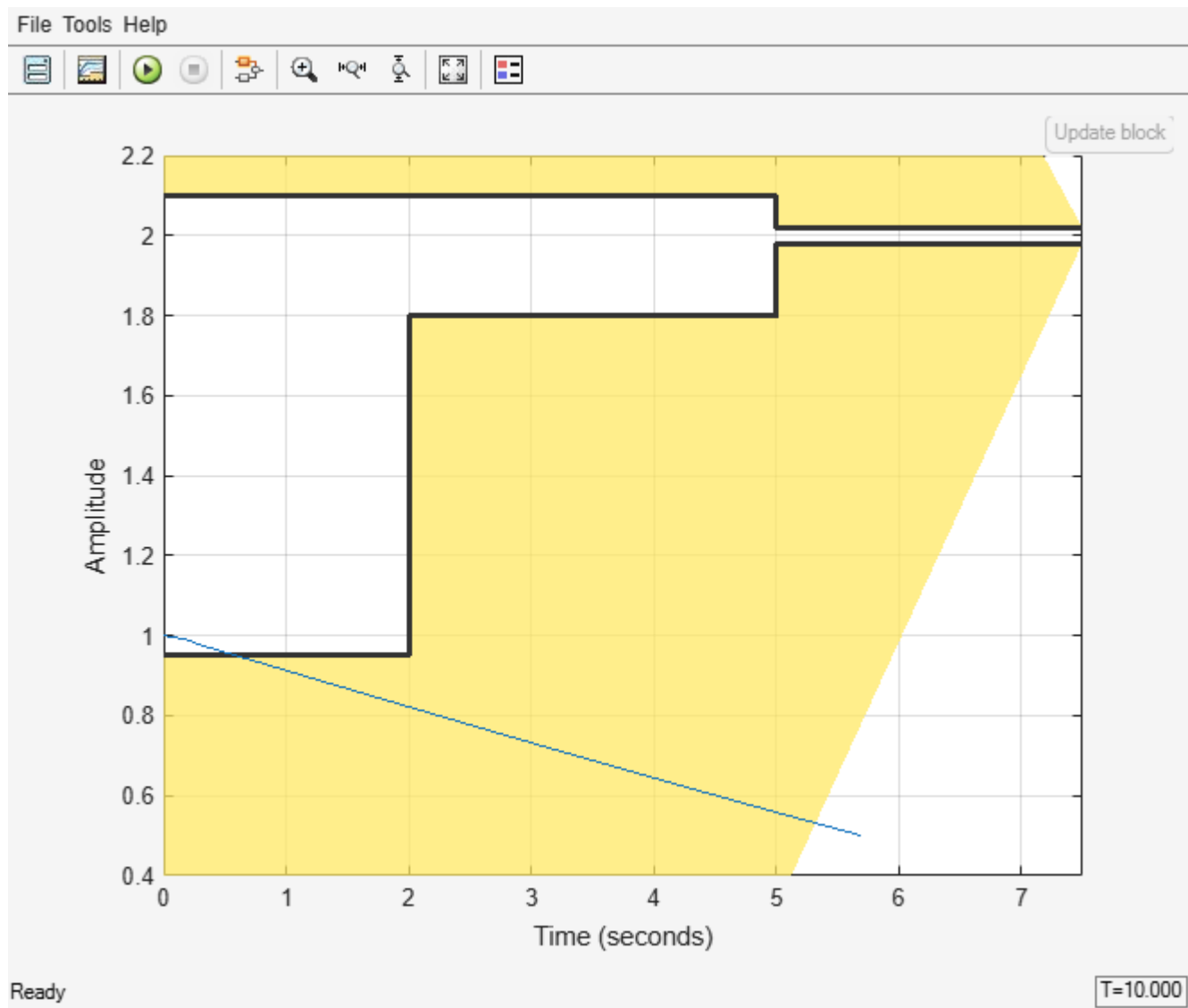
A snapshot of the training progress is shown in the following figure. You can expect different results due to inherent randomness in the training process.



Closed Loop Response of Trained Agent

Simulate the model to view the closed loop step response. The reinforcement learning agent is able to track the reference height while satisfying the step response constraints.

```
sim mdl;
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

See Also

Functions

`generateRewardFunction` | `train` | `sim` | `rlSimulinkEnv`

Objects

`rlTD3Agent` | `rlTD3AgentOptions` | `rlTrainingOptions`

Blocks

RL Agent

Related Examples

- “Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning” on page 8-284
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144
- “Generate Reward Function from a Model Predictive Controller for a Servomotor” on page 8-214

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Water Tank Custom Simulink Environment for Reinforcement Learning” on page 2-132
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Reinforcement Learning Agents” on page 5-3

Train TD3 Agent for PMSM Control

This example demonstrates speed control of a permanent magnet synchronous motor (PMSM) using a twin delayed deep deterministic policy gradient (TD3) agent.

The goal of this example is to show that you can use reinforcement learning as an alternative to linear controllers, such as PID controllers, to control the speed of PMSM systems. Linear controllers often do not produce good tracking performance outside the region in which the plant can be approximated with a linear system. In such cases, reinforcement learning provides a nonlinear control alternative.

Load the parameters for this example.

`sim_data`

```
### The Lq is observed to be lower than Ld. ###
### Using the lower of these two for the Ld (internal variable) ###
### and higher of these two for the Lq (internal variable) for computations. ###
### The Lq is observed to be lower than Ld. ###
### Using the lower of these two for the Ld (internal variable) ###
### and higher of these two for the Lq (internal variable) for computations. ###
```

```
    model: 'Maxon-645106'
      sn: '2295588'
      p: 7
      Rs: 0.2930
      Ld: 8.7678e-05
      Lq: 7.7724e-05
      Ke: 5.7835
      J: 8.3500e-05
      B: 7.0095e-05
    I_rated: 7.2600
    QEPslits: 4096
    N_base: 3476
    N_max: 4300
    FluxPM: 0.0046
    T_rated: 0.3471
    PositionOffset: 0.1650

    model: 'BoostXL-DRV8305'
      sn: 'INV_XXXX'
      V_dc: 24
      I_trip: 10
      Rds_on: 0.0020
      Rshunt: 0.0070
    CtSensAOffset: 2295
    CtSensBOffset: 2286
    CtSensCOffset: 2295
    ADcGain: 1
    EnableLogic: 1
    invertingAmp: 1
    ISenseVref: 3.3000
    ISenseVoltPerAmp: 0.0700
    ISenseMax: 21.4286
    R_board: 0.0043
    CtSensOffsetMax: 2500
    CtSensOffsetMin: 1500
```

```

model: 'LAUNCHXL-F28379D'
sn: '123456'
CPU_frequency: 200000000
PWM_frequency: 5000
PWM_Counter_Period: 20000
ADC_Vref: 3
ADC_MaxCount: 4095
SCI_baud_rate: 12000000

V_base: 13.8564
I_base: 21.4286
N_base: 3476
P_base: 445.3845
T_base: 1.0249

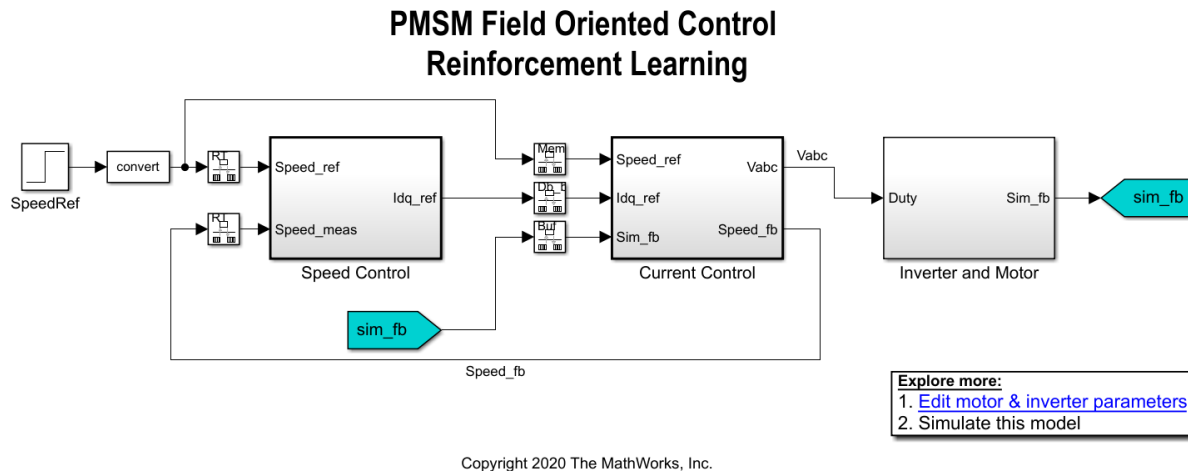
```

Open the Simulink® model.

```

mdl = "mcb_pmsm_foc_sim_RL";
open_system(mdl)

```



In a linear control version of this example, you can use PI controllers in both the speed and current control loops. An outer-loop PI controller can control the speed while two inner-loop PI controllers control the d-axis and q-axis currents. The overall goal is to track the reference speed in the Speed_Ref signal. This example uses a reinforcement learning agent to control the currents in the inner control loop while a PI controller controls the outer loop.

Create Environment Object

The environment in this example consists of the PMSM system, excluding the inner-loop current controller, which is the reinforcement learning agent. To view the interface between the reinforcement learning agent and the environment, open the Closed Loop Control subsystem.

```

open_system(mdl + ...
    "/Current Control/Control_System/Closed Loop Control")

```



The Reinforcement Learning subsystem contains an RL Agent block, the creation of the observation vector, and the reward calculation.

For this environment:

- The observations are the outer-loop reference speed Speed_ref , speed feedback Speed_fb , d-axis and q-axis currents and errors (i_d , i_q , i_{d_error} and i_{q_error}), and the error integrals.
- The actions from the agent are the voltages v_{d_rl} and v_{q_rl} .
- The sample time of the agent is $2e-4$ seconds. The inner-loop control occurs at a different sample time than the outer loop control.
- The simulation runs for 5000 time steps unless it is terminated early when the i_{q_ref} signal is saturated at 1.
- The reward at each time step is:

$$r_t = -\left(Q_1 * \text{id}_{\text{error}}^2 + Q_2 * \text{id}_{\text{error}}^2 + R * \sum_j u_{j,t-1}^2\right) - 100d.$$

Here, $Q_1 = Q_2 = 5$, and $R = 0.1$ are constants, id_{error} is the d-axis current error, iq_{error} is the q-axis current error, $u_{j_{t-1}}^j$ are the actions from the previous time step, and d is a flag that is equal to 1 when the simulation is terminated early.

Create the observation and action specifications for the environment. For information on creating continuous specifications, see `rlNumericSpec`.

```
% Create observation specifications.
numObs = 8;
obsInfo = rlNumericSpec( ...
    [numObs 1], ...
    DataType=dataType);
obsInfo.Name = "observations";
obsInfo.Description = "Error and reference signal";
```



```
% Create action specifications.
numAct = 2;
actInfo = rlNumericSpec([numAct 1], "DataType", dataType);
actInfo.Name = "vqdRef";
```

Create the Simulink environment object using the observation and action specifications. For more information on creating Simulink environments, see `rlSimulinkEnv`.

```
agentblk = "mcb_pmsm_foc_sim_RL/Current Control/" + ...
    "Control_System/Closed Loop Control/" + ...
    "Reinforcement Learning/RL Agent";

env = rlSimulinkEnv mdl, agentblk, obsInfo, actInfo);
```

Provide a reset function for this environment using the `ResetFcn` environment property. The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the `resetPMSM` function randomly initializes the final value of the reference speed in the `SpeedRef` block to 695.4 rpm (0.2 pu), 1390.8 rpm (0.4 pu), 2086.2 rpm (0.6 pu), or 2781.6 rpm (0.8 pu), and their corresponding negative values. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @resetPMSM;
```

Create Agent

The agent used in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents use two parameterized Q-value function approximators to estimate the value (that is, the expected cumulative long-term reward) of the policy.

To model the parameterized Q-value function within both critics, use a neural network with two inputs (the observation and action) and one output (the value of the policy when taking a given action from the state corresponding to a given observation). For more information on TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

The actor and critic networks are initialized randomly. Ensure reproducibility of the section by fixing the seed of the random generator.

```
rng(0)
```

Define each network path as an array of layer objects. Assign names to the input and output layers of each path. These names allow you to connect the paths and then later explicitly associate the network input and output layers with the appropriate environment channel.

```
% State input path
statePath = [
    featureInputLayer(numObs, Name="StateInLyr")
    fullyConnectedLayer(128, Name="fc1")
];

% Action input path
actionPath = [
    featureInputLayer(numAct, Name="ActionInLyr")
    fullyConnectedLayer(128, Name="fc2")
];
```

```

% Common output path
commonPath = [
    additionLayer(2, Name="add")
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(1, Name="QValueOutLyr")
];

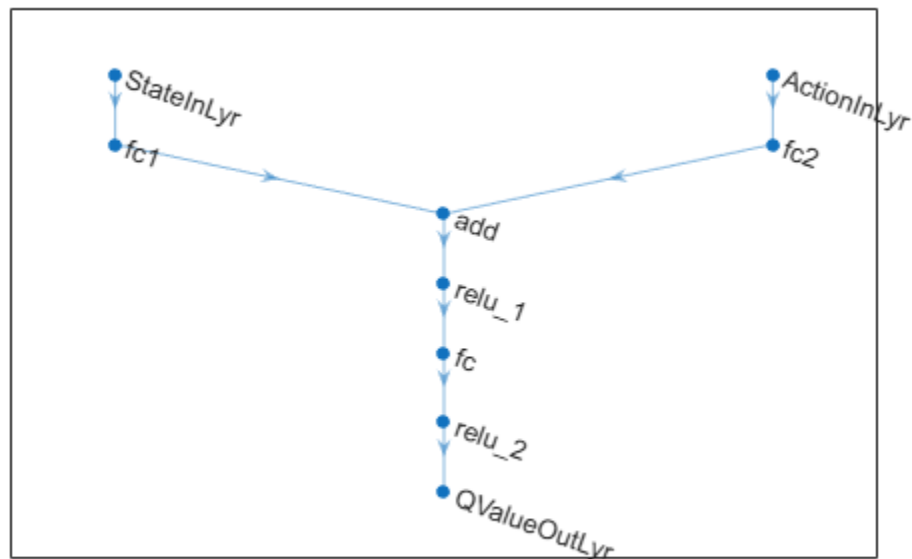
% Create dlnetwork object and add layers.
criticNet = dlnetwork();
criticNet = addLayers(criticNet, statePath);
criticNet = addLayers(criticNet, actionPath);
criticNet = addLayers(criticNet, commonPath);

% Connect layers.
criticNet = connectLayers(criticNet, "fc1", "add/in1");
criticNet = connectLayers(criticNet, "fc2", "add/in2");

```

Plot the critic network structure.

```
plot(criticNet);
```



Display the number of weights.

```
summary(initialize(criticNet));
```

```
Initialized: true
```

```
Number of learnables: 18.2k
```

```
Inputs:
```

```

1  'StateInLyr'    8 features
2  'ActionInLyr'  2 features

```

Create the critic objects using `rlQValueFunction`. The critics are function approximator objects that use the deep neural network as approximation model. To make sure the critics have different initial weights, explicitly initialize each network before using them to create a critic.

```

critic1 = rlQValueFunction(initialize(criticNet),obsInfo,actInfo);
critic2 = rlQValueFunction(initialize(criticNet),obsInfo,actInfo);

```

TD3 agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor. This actor takes the current observation as input and returns as output an action that is a deterministic function of the observation.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer (which returns the action to the environment action channel, as specified by `actInfo`). For more information on creating approximator objects such as actors and critics, see “Create Actors, Critics, and Policy Objects” on page 4-2.

Define the network as an array of layer objects.

```

actorNet = [
    featureInputLayer(numObs, Name="StateInLyr")
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(128)
    reluLayer
    fullyConnectedLayer(numAct)
    tanhLayer(Name="ActionOutLyr")
];

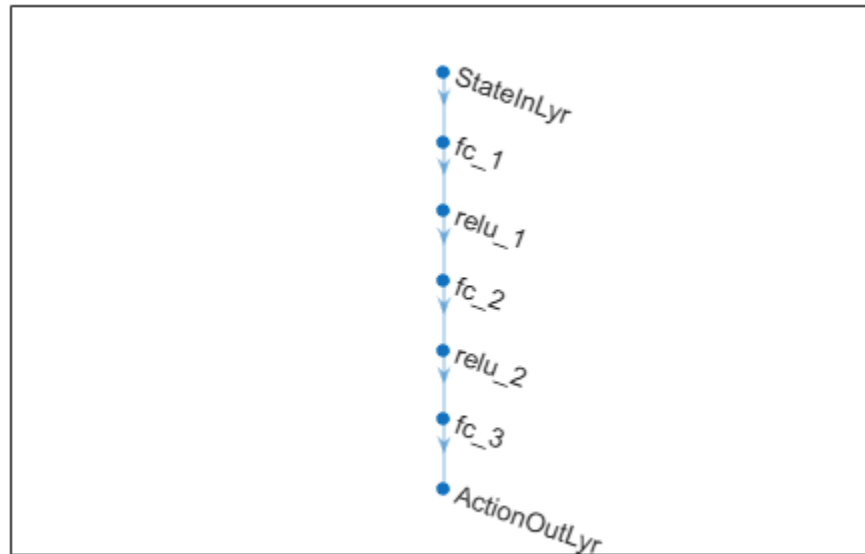
```

Convert to `dlnetwork` object.

```
actorNet = dlnetwork(actorNet);
```

Plot the actor network.

```
plot(actorNet);
```



Initialize network and display the number of weights.

```

actorNet = initialize(actorNet);
summary(actorNet)

Initialized: true

Number of learnables: 17.9k

Inputs:
  1  'StateInLyr'   8 features
  
```

Create the actor using `rlContinuousDeterministicActor`, passing the actor network and the environment specifications as input arguments.

```
actor = rlContinuousDeterministicActor(actorNet, obsInfo, actInfo);
```

To create the TD3 agent, first specify the agent options using an `rlTD3AgentOptions` object. Alternatively, you can create the agent first, and then access its option object and modify all the options using dot notation.

The agent trains from an experience buffer of maximum capacity 2e6 by randomly selecting mini-batches of size 512. Use a discount factor of 0.995 to favor long-term rewards. TD3 agents maintain time-delayed copies of the actor and critics known as the *target actor and critics*.

```

Ts_agent = Ts;
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts_agent, ...
    DiscountFactor=0.995, ...
    ExperienceBufferLength=2e6, ...
    MiniBatchSize=512);
  
```

Specify training options for the critics and the actor using `rlOptimizerOptions`. For this example, set a learning rate of 1e-3 for the actor and critics respectively.

```

% Critic optimizer options
for idx = 1:2
    agentOpts.CriticOptimizerOptions(idx).LearnRate = 1e-3;
    agentOpts.CriticOptimizerOptions(idx).GradientThreshold = 1;
    agentOpts.CriticOptimizerOptions(idx).L2RegularizationFactor = 1e-3;
end

% Actor optimizer options
agentOpts.ActorOptimizerOptions.LearnRate = 1e-3;
agentOpts.ActorOptimizerOptions.GradientThreshold = 1;
agentOpts.ActorOptimizerOptions.L2RegularizationFactor = 1e-3;

```

During training, the agent explores the action space using a Gaussian action noise model. Set the noise standard deviation using the `ExplorationModel` property. For more information on the noise model, see `rlTD3AgentOptions`.

```

agentOpts.ExplorationModel.StandardDeviationMin = 0.05;
agentOpts.ExplorationModel.StandardDeviation = 0.05;

```

The agent also uses a Gaussian action noise model for smoothing the target policy updates. Specify the noise standard deviation for this model using the `TargetPolicySmoothModel` property.

```

agentOpts.TargetPolicySmoothModel.StandardDeviation = 0.1;

```

The `LearningFrequency` is set to 100 and `MaxMiniBatchPerEpoch` is set to 10, where `LearningFrequency` controls how often the agent updates from collected samples and `MaxMiniBatchPerEpoch` limits the number of gradient steps per epoch to balance training time and sample collection.

```

agentOpts.LearningFrequency = 100;
agentOpts.MaxMiniBatchPerEpoch = 10;

```

Create the agent using the specified actor, critics, and options.

```

agent = rlTD3Agent(actor, [critic1,critic2], agentOpts);

```

Train Agent

To train the agent, first specify the training options using `rlTrainingOptions`. For this example, use the following options.

- Run each training for a maximum of 1000 episodes, with each episode lasting at most `ceil(T/Ts_agent)` time steps.
- Stop the training when the agent evaluation receives a mean cumulative reward greater than -125. At this point, the agent can track the reference speeds.

```

T = 0.5;
maxepisodes = 1000;
maxsteps = ceil(T/Ts_agent);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-125, ...
    ScoreAveragingWindowLength=10);

```

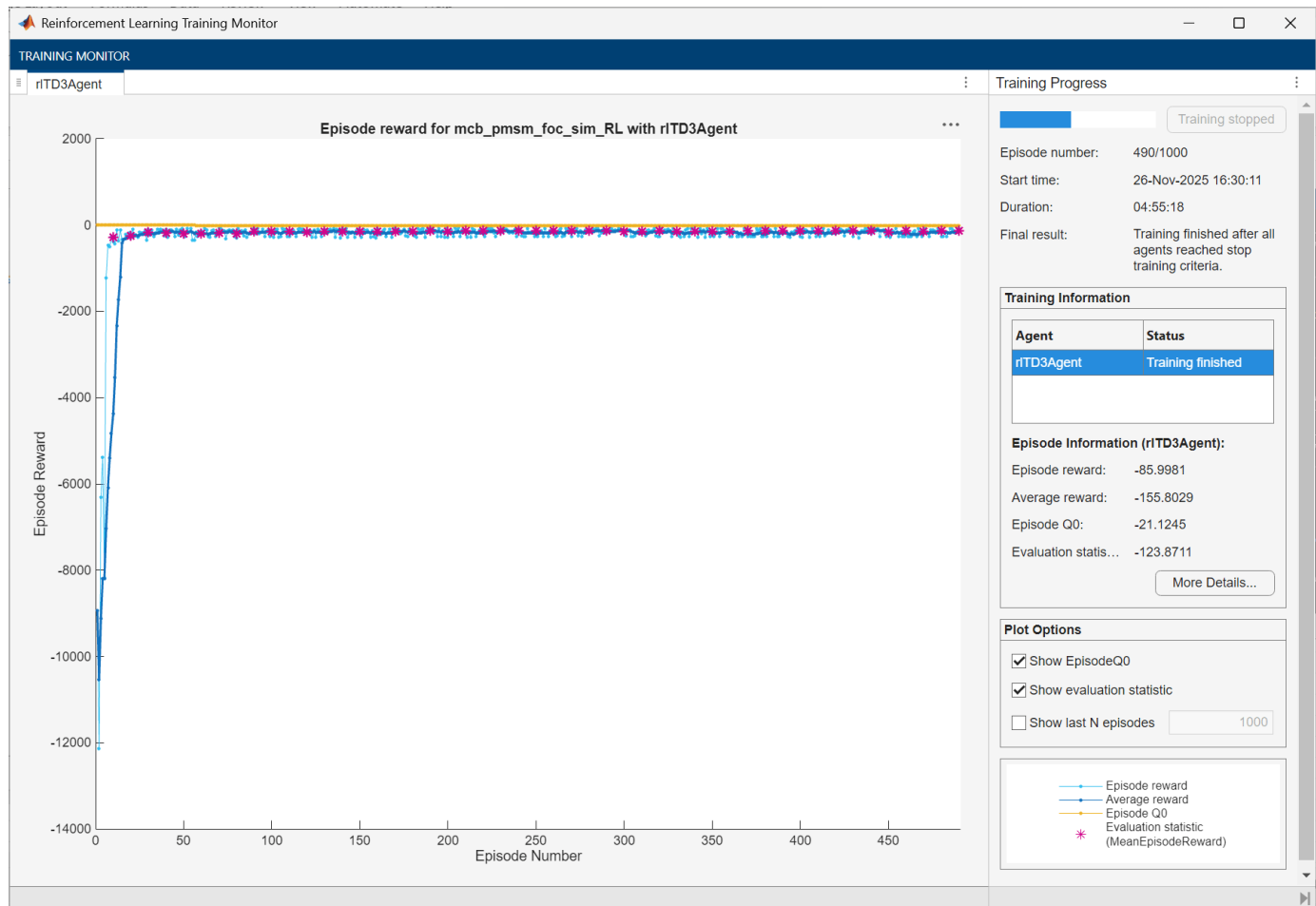
To evaluate the agent during training, create an `rlEvaluator` object. Configure the evaluator to run 3 consecutive evaluation episodes every 10 training episodes.

```
evaluator = rlEvaluater( ...
    NumEpisodes=3, ...
    EvaluationFrequency=10, ...
    RandomSeeds=[1 2 3]);
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false;
if doTraining
    trainResult = train(agent, env, trainOpts, Evaluator=evaluator);
else
    load("rlPMSMAgent.mat", "agent")
end
```

A snapshot of the training progress is shown in the following figure. You can expect different results due to randomness in the training process.



Simulate Agent

Fix the random number stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

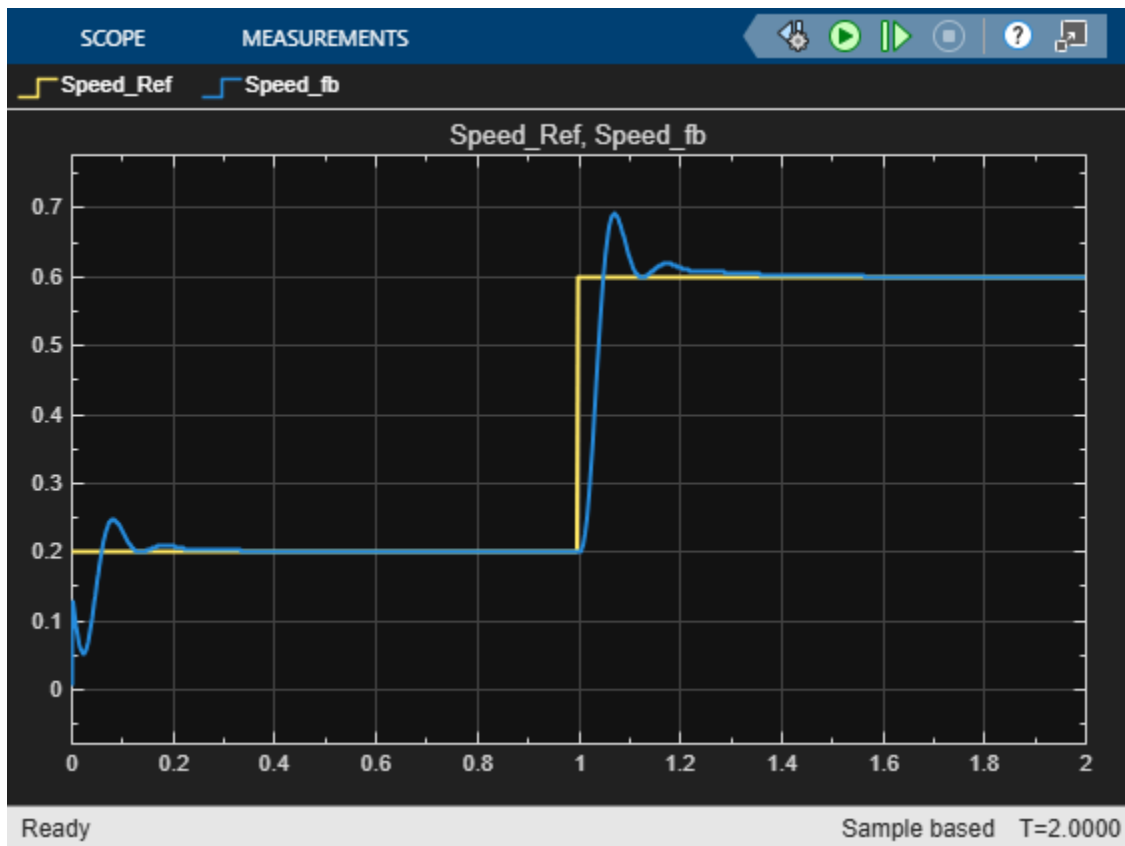
To validate the performance of the trained agent, simulate the model and view the closed-loop performance through the Speed Tracking Scope block. Use a `Simulink.SimulationInput` (Simulink) object to temporarily modify block parameters and simulate the model.

```
in = Simulink.SimulationInput mdl;
sim(in);
```

You can also simulate the model at different reference speeds. Set the reference speed in the `SpeedRef` block to a different value between 0.2 and 1.0 per-unit and simulate the model again.

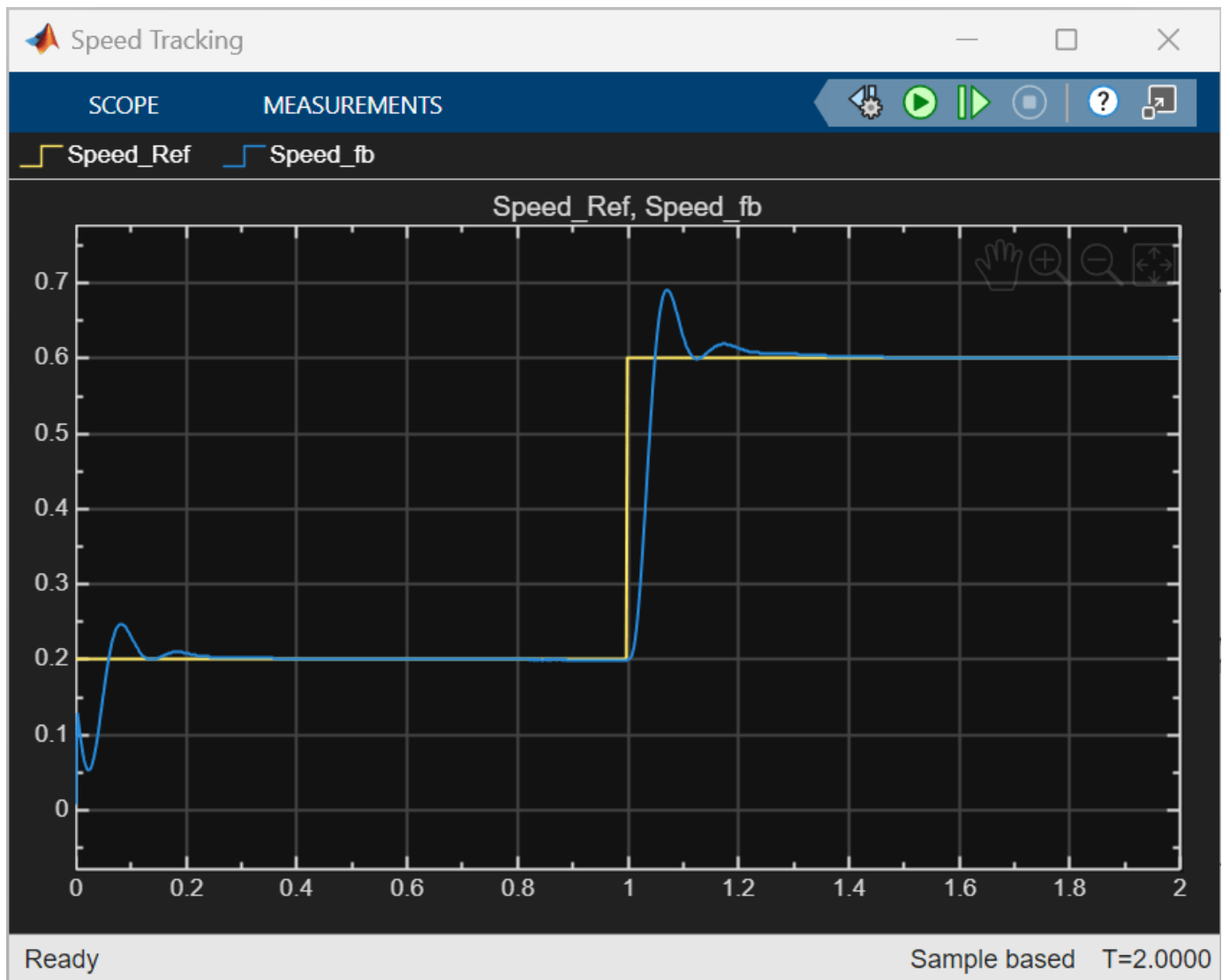
```
in = setBlockParameter(in, "mcb_pmsm_foc_sim_RL/SpeedRef", "After", "0.6");
sim(in);
```





The following figure shows an example of closed-loop tracking performance. In this simulation, the reference speed steps through values of 695.4 rpm (0.2 per-unit) and 1738.5 rpm (0.5 pu). The PI and reinforcement learning controllers track the reference signal changes within 0.5 seconds.

Although the agent was trained to track the reference speed of 0.2 per-unit and not 0.5 per-unit, it was able to generalize well.



The following figure shows the corresponding current tracking performance. The agent was able to track the i_d and i_q current references with small steady-state error.



See Also

Functions

`train` | `rlSimulinkEnv`

Objects

`rlTD3Agent` | `rlTD3AgentOptions` | `rlOptimizerOptions` | `rlTrainingOptions`

Blocks

RL Agent

Related Examples

- "Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning" on page 8-284

- “Field-Oriented Control of PMSM Using Reinforcement Learning” (Motor Control Blockset)
- “Train Biped Robot to Walk Using Reinforcement Learning Agents” on page 8-144

More About

- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Reinforcement Learning Agents” on page 5-3

Imitate MPC Controller for Lane Keeping Assist

This example shows how to train, validate, and test a deep neural network that imitates the behavior of a model predictive controller for an automotive lane keeping assist system. In the example, you also compare the behavior of the deep neural network with that of the original controller.

Model predictive control (MPC) solves a constrained quadratic-programming (QP) optimization problem in real time based on the current state of the plant (for more information, see “What Is Model Predictive Control?” (Model Predictive Control Toolbox)). Because MPC solves its optimization problem in an open-loop fashion, you can potentially replace the controller with a deep neural network. Evaluating a deep neural network can sometimes be more computationally efficient than solving a QP problem in real time (while this is not the case for this example). For an example that replaces a nonlinear MPC controller with a neural network, see “Imitate Nonlinear MPC Controller for Sliding Robot” on page 8-275.

If the training of the network sufficiently traverses the state-space for the application, you can create a reasonable approximation of the controller behavior. You can then deploy the network for your control application. You can also use the network as a warm starting point for training the actor network of a reinforcement learning agent. For an example, see “Train DDPG Agent with Pretrained Actor Network” on page 8-265.

Design MPC Controller

Design an MPC controller for lane keeping assist. To do so, first create a dynamic model for the vehicle.

```
[sys,Vx] = createModelForMPCImLKA;
```

Create and design the MPC controller object `mpcobj`. Also, create an `mpcstate` object for setting the initial controller state. For details on the controller design, type `edit createMPCobjImLKA`.

```
[mpcobj,initialState] = createMPCobjImLKA(sys);
```

For more information on designing model predictive controllers for lane keeping assist applications, see “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox) and “Lane Keeping Assist with Lane Detection” (Automated Driving Toolbox).

Prepare Input Data

The data in `InputDataFileImLKA.mat` was created by computing the MPC control actions for randomly generated states, previous control actions, and measured disturbances. To generate your own training data, use the `collectDataImLKA` function, provided in the example folder.

For this example, load the input data from `InputDataFileImLKA.mat`.

```
dataStruct = load("InputDataFileImLKA.mat");  
data = dataStruct.Data;
```

The columns of the data set are ordered as follows:

- 1 Lateral velocity V_y
- 2 Yaw angle rate r
- 3 Lateral deviation e_1

- 4 Relative yaw angle e_2
- 5 Previous steering angle (control variable) u
- 6 Measured disturbance (road yaw rate: longitudinal velocity * curvature (ρ))
- 7 Cost function value
- 8 MPC iterations
- 9 Steering angle computed by MPC controller: u^*

Divide the input data into training, validation, and testing data. First, determine the number of validation data rows based on a given percentage.

```
totalRows = size(data,1);
validationSplitPercent = 0.1;
numValidationDataRows = floor(validationSplitPercent*totalRows);
```

Determine the number of test data rows based on a given percentage.

```
testSplitPercent = 0.05;
numTestDataRows = floor(testSplitPercent*totalRows);
```

Randomly extract validation and testing data from the input data set. To do so, first randomly extract enough rows for both data sets.

```
randomIdx = randperm(totalRows,numValidationDataRows+numTestDataRows);
randomData = data(randomIdx,:);
```

Divide the random data into validation and testing data.

```
validationData = randomData(1:numValidationDataRows,:);
testData = randomData(numValidationDataRows + 1:end,:);
```

Extract the remaining rows as training data.

```
trainDataIdx = setdiff(1:totalRows,randomIdx);
trainData = data(trainDataIdx,:);
```

Randomize the training data.

```
numTrainDataRows = size(trainData,1);
shuffleIdx = randperm(numTrainDataRows);
shuffledTrainData = trainData(shuffleIdx,:);
```

Reshape the training and validation data into 4-D matrices for use with `trainNetwork`.

```
numObs = 6;
numActions = 1;

trainInput = shuffledTrainData(:,1:6);
trainOutput = shuffledTrainData(:,9);

validationInput = validationData(:,1:6);
validationOutput = validationData(:,9);
validationCellArray = {validationInput,validationOutput};
```

Reshape the testing data for use with `predict`.

```
testDataInput = testData(:,1:6);
testDataOutput = testData(:,9);
```

Create Deep Neural Network

The deep neural network architecture uses the following layers.

- `imageInputLayer` is the input layer of the neural network.
- `fullyConnectedLayer` multiplies the input by a weight matrix and then adds a bias vector.
- `reluLayer` is the activation function of the neural network.
- `tanhLayer` constrains the value to the range to $[-1,1]$.
- `scalingLayer` scales the value to the range to $[-1.04,1.04]$, this constrains the steering angle to the range $[-60,60]$.

Fix the random seed for reproducibility.

```
rng(0);
```

Create the deep neural network that will imitate the MPC controller after training.

```
imitateMPCLayers = [  
    featureInputLayer(numObs)  
    fullyConnectedLayer(45)  
    reluLayer  
    fullyConnectedLayer(45)  
    reluLayer  
    fullyConnectedLayer(45)  
    reluLayer  
    fullyConnectedLayer(numActions)  
    tanhLayer  
    scalingLayer(Scale=1.04)  
];
```

Plot the network.

```
plot(dlnetwork(imitateMPCLayers))
```



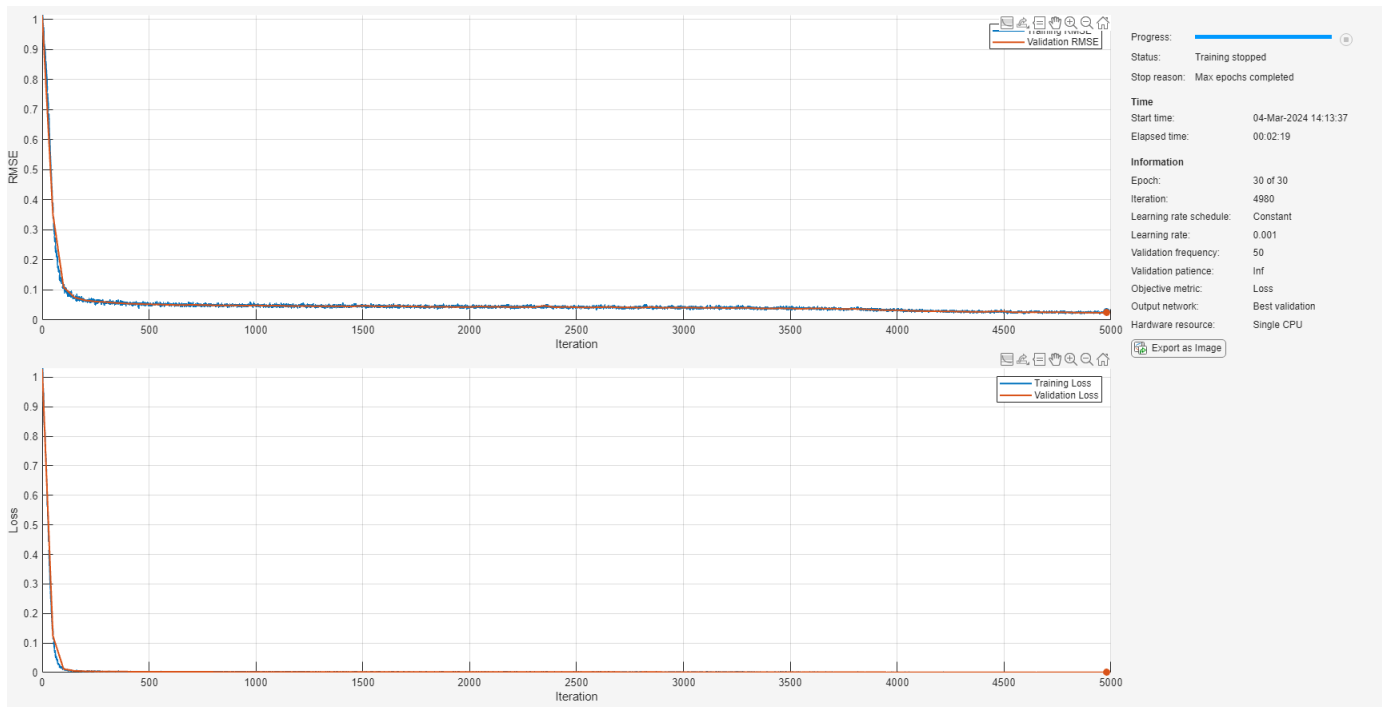
Train Deep Neural Network

Specify training options.

```
options = trainingOptions("adam", ...
    Verbose=false, ...
    Plots="training-progress", ...
    Metrics="rmse", ...
    Shuffle="every-epoch", ...
    MaxEpochs=30, ...
    MiniBatchSize=512, ...
    ValidationData=validationCellArray, ...
    InitialLearnRate=1e-3, ...
    GradientThresholdMethod="absolute-value", ...
    ExecutionEnvironment="cpu", ...
    GradientThreshold=10, ...
    Epsilon=1e-8);
```

Train the deep neural network. To view detailed training information in the Command Window, set the Verbose training option to true.

```
imitateMPCNetwork = trainnet( ...
    trainInput, ...
    trainOutput, ...
    imitateMPCLayers, ...
    "mse", ...
    options);
```



Training of the deep neural network stops after the final iteration.

The training and validation loss are nearly the same for each mini-batch, which indicates that the trained network does not overfit.

Test Trained Network

Check that the trained deep neural network returns steering angles similar to the MPC controller control actions given the test input data. Compute the network output using the `predict` function.

```
predictedTestDataOutput = predict(imitateMPCNetwork,testDataInput);
```

Calculate the root mean squared error (RMSE) between the network output and the testing data.

```
testRMSE = sqrt(mean((testDataOutput - predictedTestDataOutput).^2));
fprintf("Test Data RMSE = %d\n", testRMSE);
```

```
Test Data RMSE = 2.468181e-02
```

The small RMSE value indicates that the network outputs closely reproduce the MPC controller outputs.

Compare Trained Network with MPC Controller

To compare the performance of the MPC controller and the trained deep neural network, run closed-loop simulations using the vehicle plant model.

Generate random initial conditions for the vehicle that are not part of the original input data set, with values selected from the following ranges:

- 1 Lateral velocity V_y — Range (-2,2) m/s

- 2 Yaw angle rate r — Range (-1.04,1.04) rad/s
- 3 Lateral deviation e_1 — Range (-1,1) m
- 4 Relative yaw angle e_2 — Range (-0.8,0.8) rad
- 5 Last steering angle (control variable) u — Range (-1.04,1.04) rad
- 6 Measured disturbance (road yaw rate, defined as longitudinal velocity * curvature (ρ)) — Range (-0.01,0.01) with a minimum road radius of 100 m

```
rng(5e7)
[x0,u0,rho] = generateRandomDataImLKA(data);
```

Set the initial plant state and control action in the `mpcstate` object.

```
initialState.Plant = x0;
initialState.LastMove = u0;
```

Extract the sample time from the MPC controller. Also, set the number of simulation steps.

```
Ts = mpcobj.Ts;
Tsteps = 30;
```

Obtain the A and B state-space matrices for the vehicle model.

```
A = sys.A;
B = sys.B;
```

Initialize the state and input trajectories for the MPC controller simulation.

```
xHistoryMPC = repmat(x0',Tsteps+1,1);
uHistoryMPC = repmat(u0',Tsteps,1);
```

Run a closed-loop simulation of the MPC controller and the plant using the `mpcmove` function.

```
for k = 1:Tsteps

    % Obtain plant outputs
    xk = xHistoryMPC(k,:)';

    % Compute control action using the MPC controller
    uk = mpcmove(mpcobj,initialState,xk,zeros(1,4),Vx*rho);

    % Store the control action
    uHistoryMPC(k,:) = uk;

    % Update plant state using the control action
    xHistoryMPC(k+1,:) = (A*xk + B*[uk;Vx*rho])';
end
```

Initialize the state and input trajectories for the deep neural network simulation.

```
xHistoryDNN = repmat(x0',Tsteps+1,1);
uHistoryDNN = repmat(u0',Tsteps,1);
lastMV = u0;
```

Run a closed-loop simulation of the trained network and the plant. The `neuralnetLKAmove` function, provided in the example folder, computes the deep neural network output using the `predict` function.

```
for k = 1:Tsteps

    % Obtain plant outputs
    xk = xHistoryDNN(k,:)';

    % Predict the next move using trained network
    uk = neuralnetLKAmove(imitateMPCNetwork,xk,lastMV,rho);

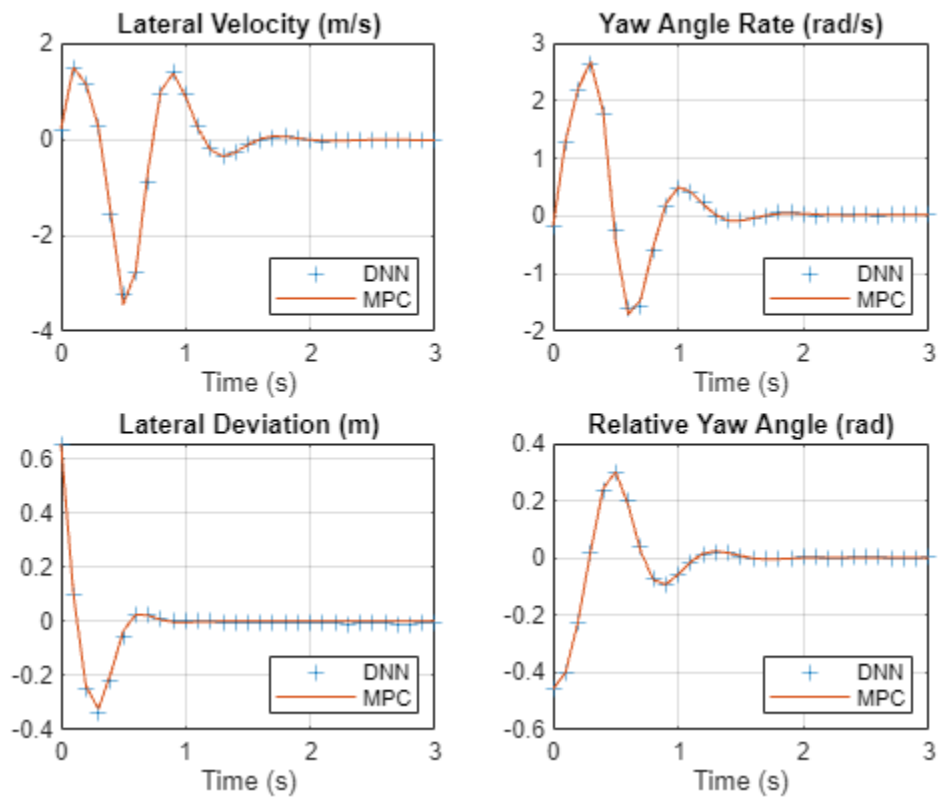
    % Store control action
    uHistoryDNN(k,:) = uk;

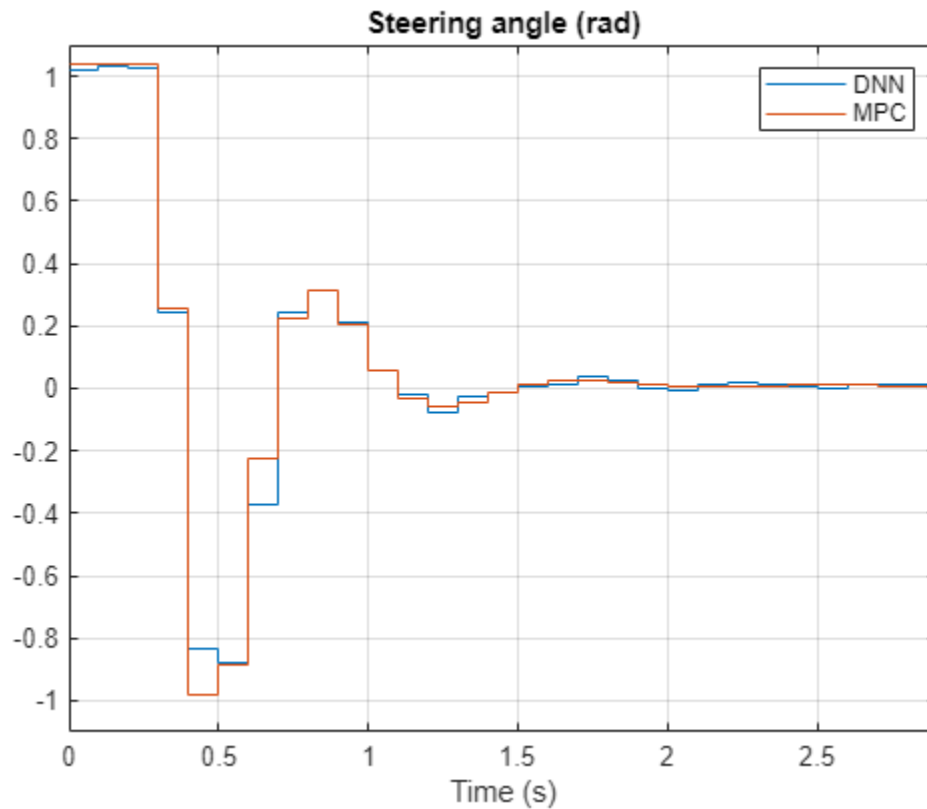
    % Update the last MV for the next step
    lastMV = uk;

    % Update plant state using the control action
    xHistoryDNN(k+1,:) = (A*xk + B*[uk;Vx*rho])';
end
```

Plot the results to compare the MPC controller and trained deep neural network (DNN) trajectories.

```
plotValidationResultsImLKA(Ts, ...
    xHistoryDNN,uHistoryDNN, ...
    xHistoryMPC,uHistoryMPC);
```





The deep neural network successfully imitates the behavior of the MPC controller. The vehicle state and control action trajectories for the controller and the deep neural network closely align.

See Also

Functions

`trainNetwork` | `predict` | `mpcmove`

Objects

`SeriesNetwork` | `mpc`

Related Examples

- “Train DDPG Agent with Pretrained Actor Network” on page 8-265
- “Imitate Nonlinear MPC Controller for Sliding Robot” on page 8-275
- “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox)
- “Lane Keeping Assist with Lane Detection” (Automated Driving Toolbox)

More About

- “What Is Model Predictive Control?” (Model Predictive Control Toolbox)

Train DDPG Agent with Pretrained Actor Network

This example shows how to train a deep deterministic policy gradient (DDPG) agent for lane keeping assist (LKA) in Simulink®. To make training more efficient, the actor of the DDPG agent is initialized with a deep neural network that was previously trained using supervised learning. This actor is trained in the “Imitate MPC Controller for Lane Keeping Assist” on page 8-256 example.

For more information on DDPG agents, see “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Simulink Model

The training goal for the lane-keeping application is to keep the ego vehicle traveling along the centerline of the lane by adjusting the front steering angle. This example uses the same ego vehicle dynamics and sensor dynamics as the “Train DQN Agent for Lane Keeping Assist” on page 8-76 example.

```
m = 1575;    % total vehicle mass (kg)
Iz = 2875;   % yaw moment of inertia (mNs^2)
lf = 1.2;    % long. distance from center of gravity to front tires (m)
lr = 1.6;    % long. distance from center of gravity to rear tires (m)
Cf = 19000;  % cornering stiffness of front tires (N/rad)
Cr = 33000;  % cornering stiffness of rear tires (N/rad)
Vx = 15;     % longitudinal velocity (m/s)
```

Define the sample time, T_s , and simulation duration, T , in seconds.

```
Ts = 0.1;
T = 15;
```

The output of the LKA system is the front steering angle of the ego vehicle. Considering the physical limitations of the ego vehicle, constrain its steering angle to the range $[-60, 60]$ degrees. Specify the constraints in radians.

```
u_min = -1.04;
u_max = 1.04;
```

Define the curvature of the road as a constant $0.001(m^{-1})$.

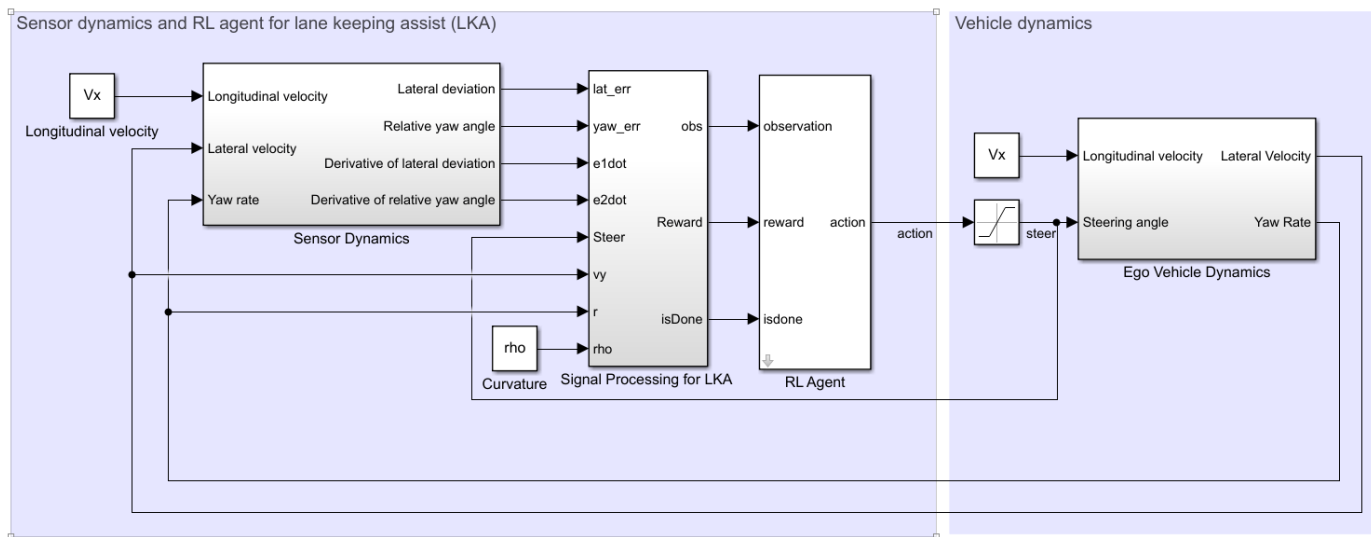
```
rho = 0.001;
```

Set initial values for the lateral deviation (`e1_initial`) and the relative yaw angle (`e2_initial`). During training, these initial conditions are set to random values for each training episode.

```
e1_initial = 0.2;
e2_initial = -0.1;
```

Open the model.

```
mdl = "rlActorLKAMdl";
open_system(mdl)
```



Copyright 2018 The MathWorks, Inc.

Define the path to the RL Agent block within the model.

```
agentblk = mdl + "/RL Agent";
```

Create Environment

Create a reinforcement learning environment object for the ego vehicle. To do so, first define the observation and action specifications. These observations and actions are the same as the features for supervised learning used in “Imitate MPC Controller for Lane Keeping Assist” on page 8-256.

The six observations for the environment are the lateral velocity v_y , yaw rate $\dot{\psi}$, lateral deviation e_1 , relative yaw angle e_2 , steering angle at previous step u_0 , and curvature ρ .

```
obsInfo = rlNumericSpec([6 1], ...
    LowerLimit=-inf*ones(6,1), ...
    UpperLimit= inf*ones(6,1))

obsInfo =
    rlNumericSpec with properties:
        LowerLimit: [6x1 double]
        UpperLimit: [6x1 double]
        Name: [0x0 string]
        Description: [0x0 string]
        Dimension: [6 1]
```

```
DataType: "double"
```

```
obsInfo.Name = "observations";
```

The action for the environment is the front steering angle. Specify the steering angle constraints when creating the action specification object.

```
actInfo = rlNumericSpec([1 1], ...
    LowerLimit=u_min, ...
    UpperLimit=u_max);
actInfo.Name = "steering";
```

In the model, the Signal Processing for LKA block creates the observation vector signal, computes the reward function, and calculates the stop signal.

The reward r_t , provided at every time step t , is as follows, where u is the control input from the previous time step $t - 1$.

$$r_t = -(10e_1^2 + 5e_2^2 + 2u^2 + 5\dot{e}_1^2 + 5\dot{e}_2^2)$$

The simulation stops when $|e_1| > 1$.

Create the reinforcement learning environment.

```
env = rlSimulinkEnv mdl, agentblk, obsInfo, actInfo;
```

To define the initial condition for lateral deviation and relative yaw angle, specify an environment reset function using an anonymous function handle. The `localResetFcn` function, which is defined at the end of the example, sets the initial lateral deviation and relative yaw angle to random values.

```
env.ResetFcn = @(in)localResetFcn(in);
```

Create DDPG Agent

DDPG agents use a parameterized deterministic policy over continuous action spaces, which is learned by a continuous deterministic actor, and a parameterized Q-value function approximator to estimate the value of the policy. Use neural networks to model both the parameterized policy within the actor and the Q-value function within the critic. For this example, use the helper functions `createLaneKeepingCritic` and `createLaneKeepingActor` to create the critic and the actor, along with their training options sets.

```
[critic,criticOpts] = createLaneKeepingCritic(obsInfo,actInfo);
[actor,actorOpts] = createLaneKeepingActor(obsInfo,actInfo);
```

These initial actor and critic networks have random initial parameter values.

Specify the DDPG agent options using `rlDDPGAgentOptions`, include the training options for the actor and critic.

```
agentOptions = rlDDPGAgentOptions( ...
    SampleTime=Ts, ...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=criticOpts, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=256, ...
    TargetSmoothFactor=1e-3);
```

You can also set or modify the agent options using dot notation.

```
agentOptions.NoiseOptions.StandardDeviation = 0.3;  
% set the MeanAttractionConstant to 1/Ts for Gaussian action noise.  
agentOptions.NoiseOptions.MeanAttractionConstant = 1/Ts;
```

Create the DDPG agent using the specified actor and critic objects and the agent options. For more information, see `rlDDPGAgent`.

```
agent = rlDDPGAgent(actor,critic,agentOptions);
```

Alternatively, you can also create the agent first, and then access its option object, and modify the options, using dot notation.

Train Agent

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

As a baseline, train the agent with an actor that has random initial parameters. To train the agent, first specify the training options. For this example, use the following options.

- Run training for a maximum of 10,000 episodes, with each episode lasting a maximum of 150 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box.
- Stop the training when the evaluation statistic exceeds -12.
- Save a copy of the agent when the evaluation statistic exceeds -15.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 10000;  
maxsteps = T/Ts;  
trainingOpts = rlTrainingOptions( ...  
    MaxEpisodes=maxepisodes, ...  
    MaxStepsPerEpisode=maxsteps, ...  
    Verbose=false, ...  
    Plots="training-progress", ...  
    StopTrainingCriteria="EvaluationStatistic", ...  
    StopTrainingValue=-12, ...  
    SaveAgentCriteria="EvaluationStatistic", ...  
    SaveAgentValue=-15);
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Every 10 episodes evaluate performance of the greedy policy by  
    % running 5 evaluation episodes.  
    evaluator = rlEvaluator( ...  
        EvaluationFrequency=10, ...  
        NumEpisodes=5);  
    % Train the agent.
```



```

    trainingStats = train(agent,env,trainingOpts,Evaluator=evaluator);
else
    % Load pretrained agent for the example.
    load("ddpgFromScratch.mat");
end

```



Train Agent with Pretrained Actor

You can set the actor network of your agent to a deep neural network that has been previously trained. For this example, use the deep neural network from the “Imitate MPC Controller for Lane Keeping Assist” on page 8-256 example. This network was trained to imitate a model predictive controller using supervised learning.

Load the pretrained actor network.

```
load("imitateMPCNetActorObj.mat","imitateMPCNetObj");
```

Create an actor representation using the pretrained actor.

```

supervisedActor = rlContinuousDeterministicActor( ...
    imitateMPCNetObj, ...
    obsInfo, ...
    actInfo);

```

Check that the network used by `supervisedActor` is the same one that was loaded. To do so, evaluate both the network and the agent using the same random input observation.

```
testData = rand(6,1);
```

Evaluate the deep neural network.

```
predictImNN = predict(imitateMPCNetObj,testData');
```

Evaluate the actor.

```
evaluateRLRep = getAction(supervisedActor,{testData});
```

Compare the results.

```
error = evaluateRLRep{:} - predictImNN
```

```
error = single
```

```
0
```

Create a DDPG agent using the pretrained actor.

```
agent = rlDDPGAgent(supervisedActor,critic,agentOptions);
```

Reduce the maximum number of training episodes and train the agent using the `train` function. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
trainingOpts.MaxEpisodes = 5000;
```

```
doTraining =  ;
```

```
if doTraining
```

```
    % Every 10 episodes evaluate performance of the greedy policy by  
    % running 5 evaluation episodes.
```

```
    evaluator = rlEvaluater( ...  
        EvaluationFrequency=10, ...  
        NumEpisodes=5);
```

```
    % Train the agent.
```

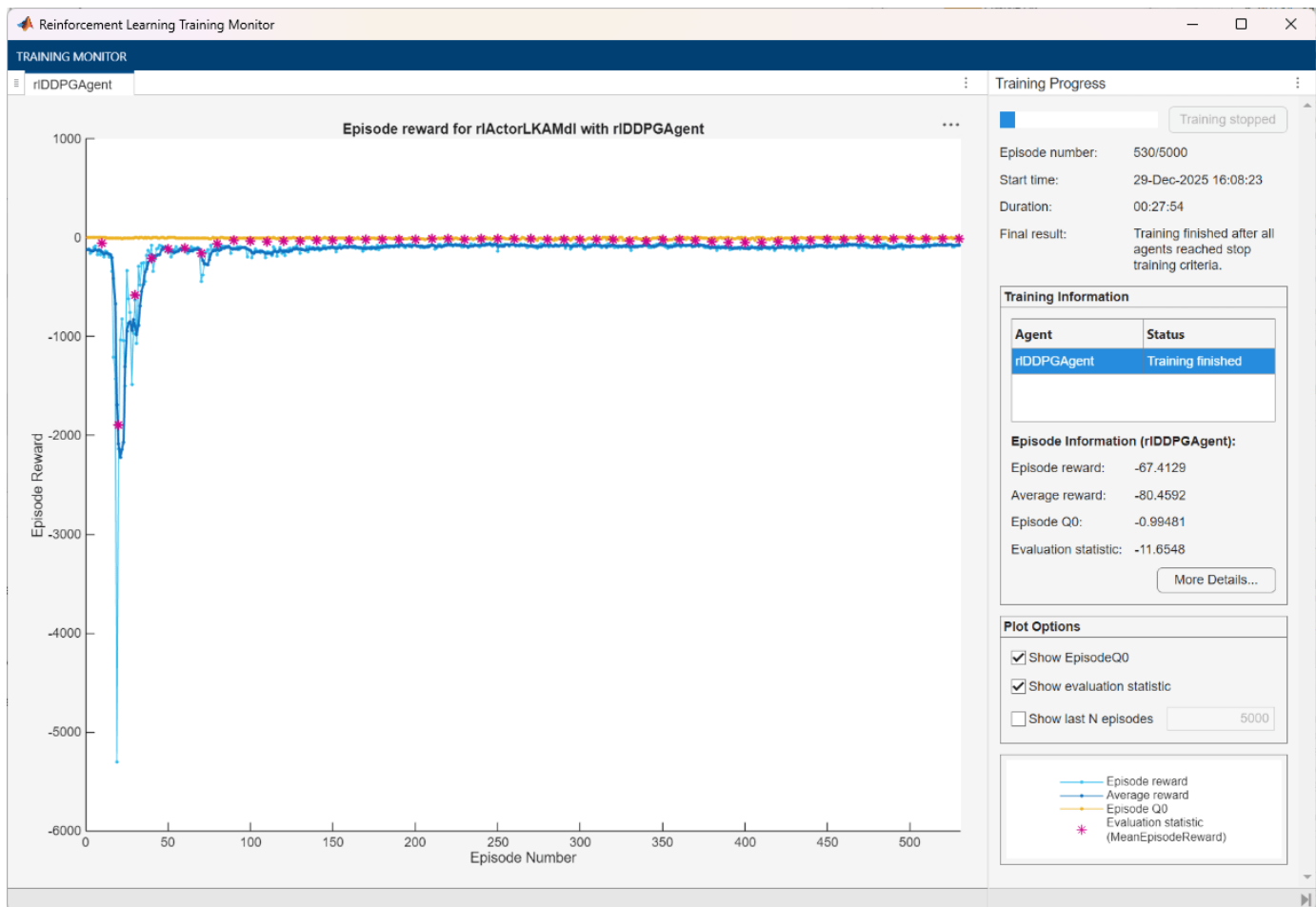
```
    trainingStats = train(agent,env,trainingOpts,Evaluator=evaluator);
```

```
else
```

```
    % Load pretrained agent for the example.
```

```
    load("ddpgFromPretrained.mat");
```

```
end
```



By using the pretrained actor network, the training of the DDPG agent converges to a satisfactory policy in fewer episodes.

Simulate the Trained DDPG Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

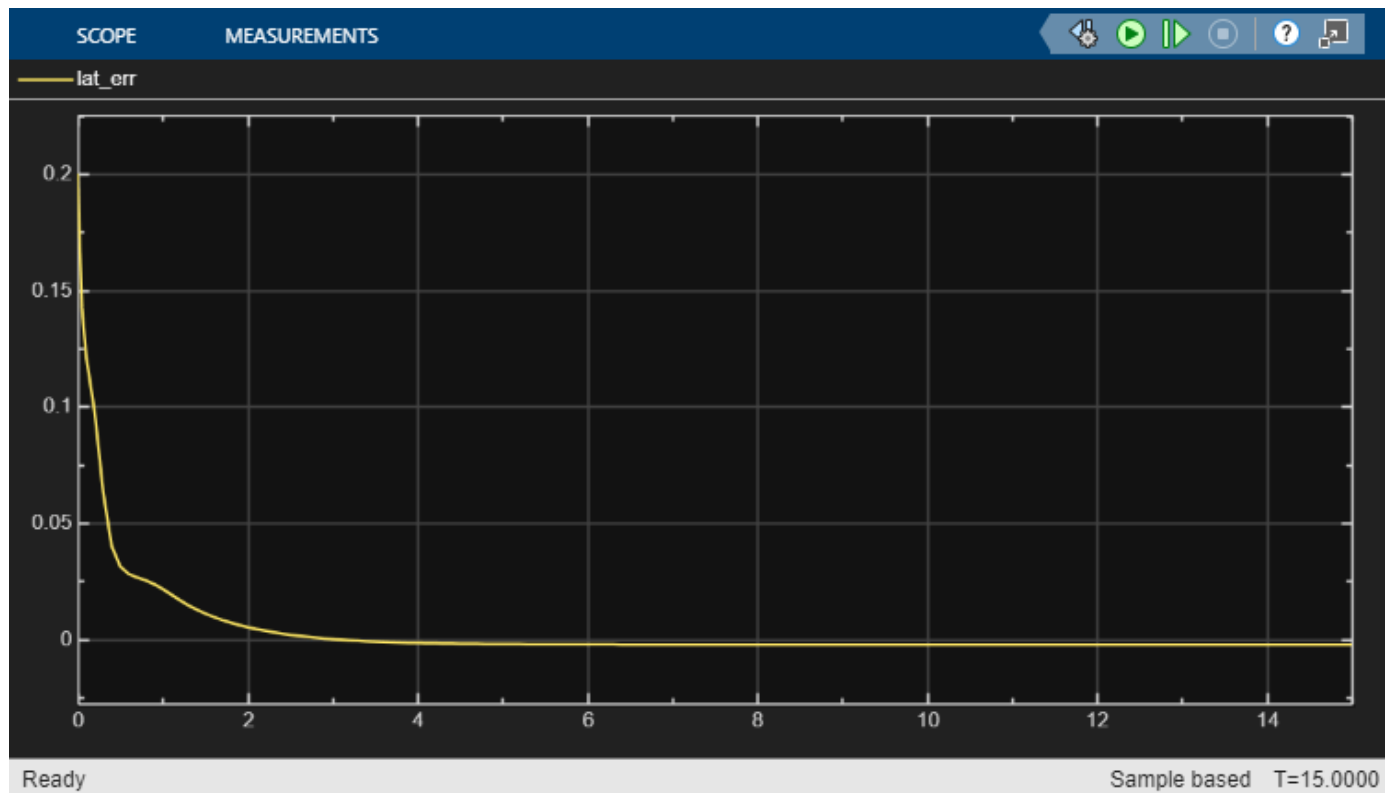
By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the UseExplorationPolicy agent property to true.

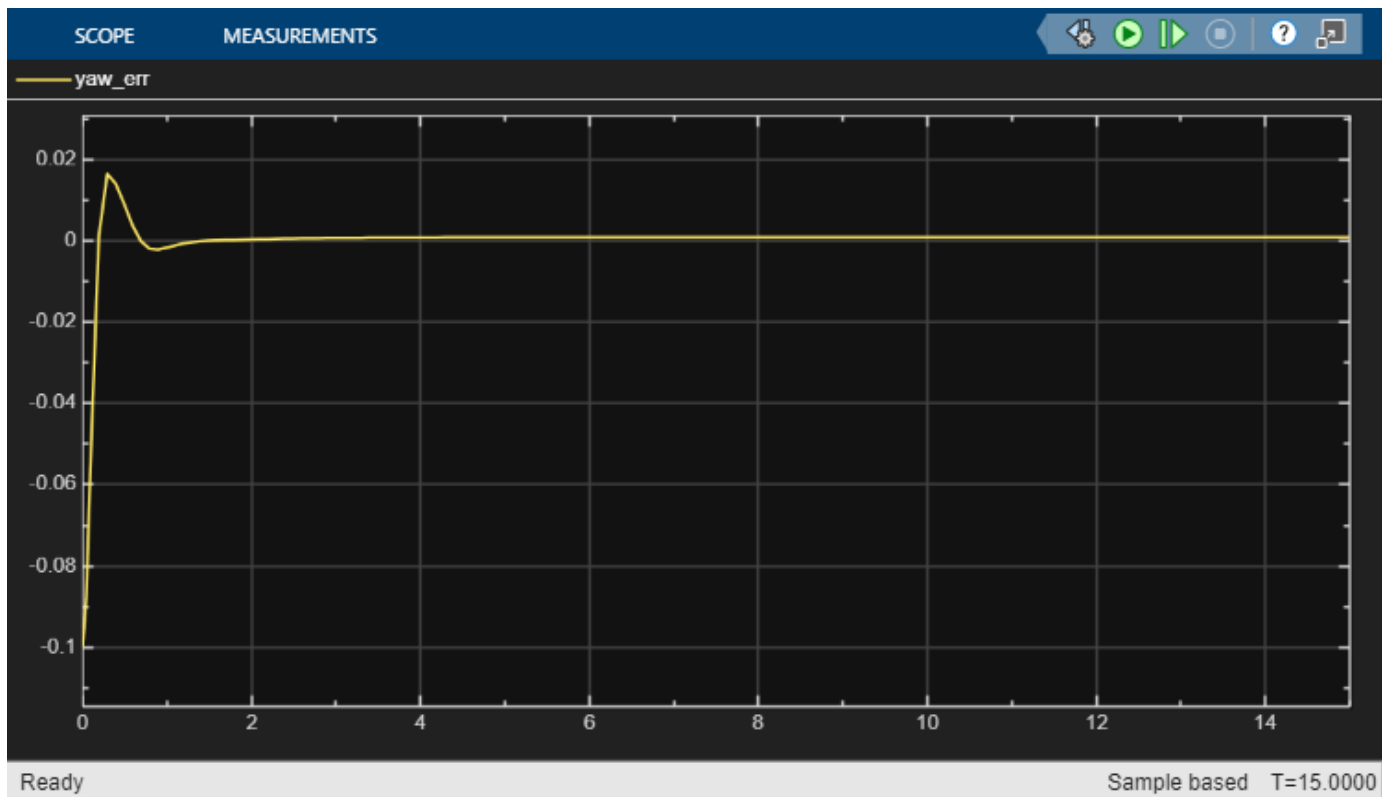
To validate the performance of the trained agent, uncomment the following two lines and simulate it within the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
% simOptions = rlSimulationOptions(MaxSteps=maxsteps);
% experience = sim(env,agent,simOptions);
```

To check the performance of the trained agent within the Simulink model, simulate the model using the previously defined initial conditions (`e1_initial = 0.2` and `e2_initial = -0.1`).

```
sim mdl)
```





As shown above, the lateral error (top plot) and relative yaw angle (bottom plot) are both driven to zero. The vehicle starts with a lateral deviation from the centerline (0.2 m) and a nonzero yaw angle error (-0.1 rad). The lane-keeping controller makes the ego vehicle travel along the centerline after around two seconds. The steering angle (middle plot) shows that the controller reaches steady state after about two seconds.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

The `sim` function calls the `reset` function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The `reset` function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the function `localResetFcn` uses the `setVariable` (Simulink) function to set variables in the model workspace. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)
% Set random value for lateral deviation.
in = setVariable(in,"el_initial", 0.5*(-1+2*rand));
% Set random value for relative yaw angle.
```

```
in = setVariable(in,"e2_initial", 0.1*(-1+2*rand));  
end
```

See Also

Functions

train | sim | rlSimulinkEnv

Objects

rlDDPGAgent | rlDDPGAgentOptions | rlQValueFunction |
rlContinuousDeterministicActor | rlTrainingOptions | rlSimulationOptions |
rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Imitate MPC Controller for Lane Keeping Assist” on page 8-256
- “Train DQN Agent for Lane Keeping Assist” on page 8-76
- “Compare DDPG Agent to LQR Controller” on page 7-285
- “Lane Keeping Assist System Using Model Predictive Control” (Model Predictive Control Toolbox)

More About

- “Define Observation and Reward Signals in Custom Environments” on page 2-127
- “Deep Deterministic Policy Gradient (DDPG) Agent” on page 3-67
- “Train Reinforcement Learning Agents” on page 5-3

Imitate Nonlinear MPC Controller for Sliding Robot

This example shows how to train, validate, and test a deep neural network (DNN) that imitates the behavior of a nonlinear model predictive controller for a robot sliding over a 2-D frictionless plane, modeled in Simulink®. It then compares the behavior of the deep neural network with that of the original controller. To train the deep neural network, this example uses the data aggregation (DAgger) approach described in [1].

Nonlinear model predictive control (NLMPC) solves a constrained nonlinear optimization problem in real time based on the current state of the plant. For more information on NLMPC, see “Nonlinear MPC” (Model Predictive Control Toolbox). For an example that uses two nonlinear MPC controllers to respectively generate and track an optimal trajectory for a sliding robot model, see “Trajectory Optimization and Control of Sliding Robot Using Nonlinear MPC” (Model Predictive Control Toolbox). For an example that instead uses a reinforcement learning agent to solve the control problem for a sliding robot, see “Train DDPG Agent to Control Two-Thruster Sliding Vehicle” on page 8-22.

Replacing an NLMPC controller with a trained DNN can be an appealing option because evaluating a DNN can be more computationally efficient than solving a nonlinear optimization problem in real-time. For an example that replaces a classical MPC controller with a neural network, see “Imitate MPC Controller for Lane Keeping Assist” on page 8-256.

If the trained DNN reasonably approximates the controller behavior, you can then deploy the network for your control application. You can also use the network as a warm starting point for training the actor network of a reinforcement learning agent. For an example that does so with a DNN trained for an MPC application, see “Train DDPG Agent with Pretrained Actor Network” on page 8-265.

Design Nonlinear MPC Controller

Design a nonlinear MPC controller for a sliding robot. The dynamics for the sliding robot are the same as in “Trajectory Optimization and Control of Sliding Robot Using Nonlinear MPC” (Model Predictive Control Toolbox) example, and are described in [2 on page 8-282]. First, define the limit for the control variables, which are the robot thrust levels.

```
umax = 3;
```

Create the nonlinear MPC controller object `nlobj`. To reduce command-window output, disable the MPC update messages. For more information, see `nlimpc` (Model Predictive Control Toolbox).

```
mpcverbosity off;
nlobj = createMPCobjImSlidingRobot(umax);
```

Prepare Input Data

Load the input data from `DAggerInputDataFileImSlidingRobot.mat`. The columns of the data set contain:

- 1 x is the position of the robot along the x-axis.
- 2 y is the position of the robot along the y-axis.
- 3 θ is the orientation of the robot.
- 4 $\dot{x}$ is the velocity of the robot along the x-axis.
- 5 $\dot{y}$ is the velocity of the robot along the y-axis.

- 6 $\dot{\theta}$ is the angular velocity of the robot.
- 7 u_l is the thrust on the left side of the sliding robot
- 8 u_r is the thrust on the right side of the sliding robot
- 9 u_l^* is the thrust on the left side computed by NLMPC
- 10 u_r^* is the thrust on the right side computed by NLMPC

The data in `DaggerInputDataFileImSlidingRobot.mat` is created by computing the NLMPC control action for randomly generated states $(x, y, \theta, \dot{x}, \dot{y}, \dot{\theta})$, and previous control actions (u_l, u_r) . To generate your own training data, use the `collectDataImSlidingRobot` function.

Load the input data.

```
fileName = "DaggerInputDataFileImSlidingRobot.mat";
DaggerData = load(fileName);
data = DaggerData.data;
existingData = data;
numCol = size(data,2);
```

Create Deep Neural Network

Create the deep neural network that will imitate the NLMPC controller after training. The network architecture uses the following types of layers.

- `imageInputLayer` is the input layer of the neural network.
- `fullyConnectedLayer` multiplies the input by a weight matrix and then adds a bias vector.
- `reluLayer` is the activation function of the neural network.
- `tanhLayer` constrains the value to the range to $[-1,1]$.
- `scalingLayer` scales the value to the range to $[-3,3]$.

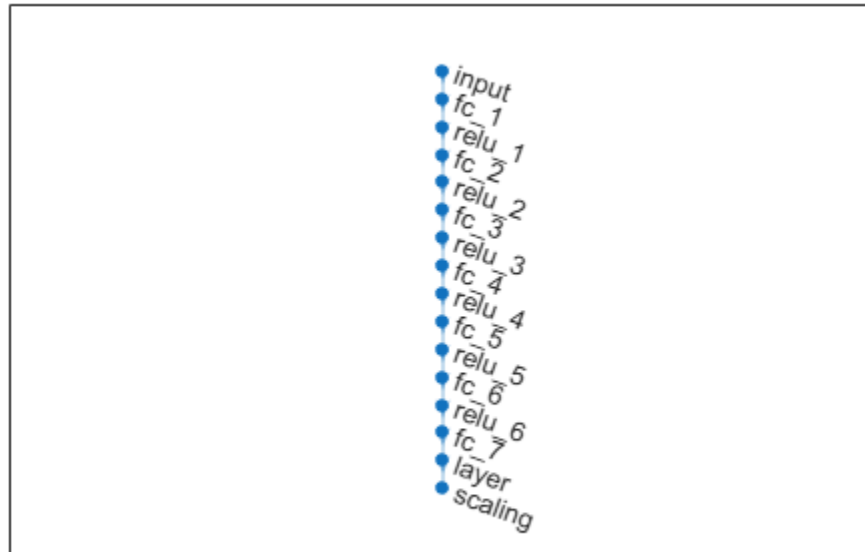
Define the network as an array of layer objects.

```
numObs = numCol-2;
numAct = 2;
hiddenLayerSize = 256;

imitateMPCNetwork = [
    featureInputLayer(numObs)
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(hiddenLayerSize)
    reluLayer
    fullyConnectedLayer(numAct)
    tanhLayer
    scalingLayer(Scale=umax)
];
```


Plot the network.

```
plot(dlnetwork(imitateMPCNetwork))
```



Behavior Cloning Approach

One approach to learning an expert policy using supervised learning is the behavior cloning method. This method divides the expert demonstrations (NLMPC control actions in response to observations) into state-action pairs and applies supervised learning to train the network.

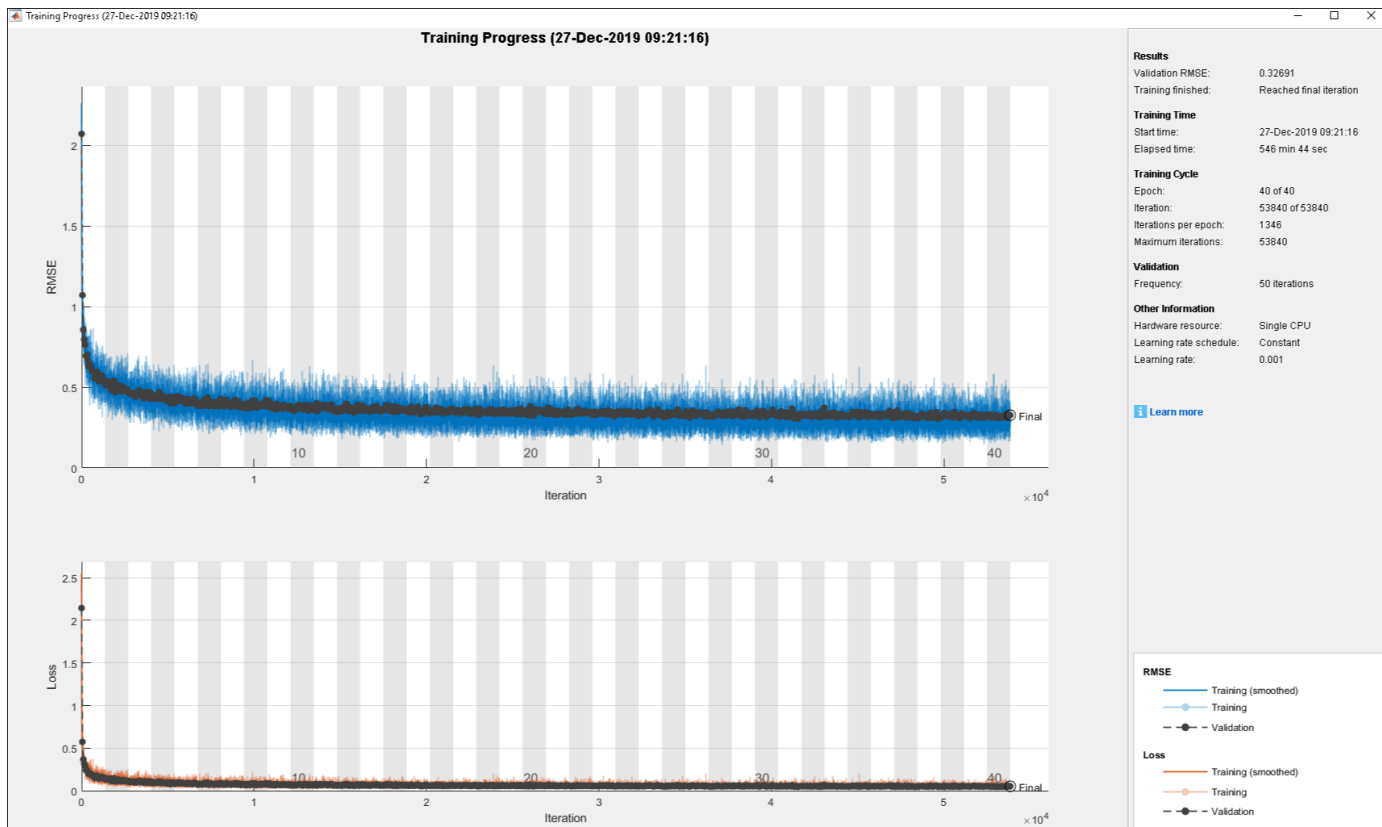
Specify training options.

```
% Initialize validation cell array
validationCellArray = {0,0};

options = trainingOptions("adam", ...
    Metrics="rmse", ...
    Shuffle="every-epoch", ...
    MiniBatchSize=512, ...
    ValidationData=validationCellArray, ...
    InitialLearnRate=1e-3, ...
    ExecutionEnvironment="cpu", ...
    GradientThreshold=10, ...
    MaxEpochs=40 ...
);
```

You can train the behavior cloning neural network by following below steps:

- 1 Collect data using the `collectDataImSlidingRobot` function.
- 2 Train the behavior cloning network using the `behaviorCloningTrainNetwork` function.



The training of the DNN using behavior cloning reduces the gap between the DNN and NLMPC performance. However, the behavior cloning neural network fails to imitate the behavior of the NLMPC controller correctly on some randomly generated data.

Training a DNN is a computationally intensive process. For this example, to save time, load a pretrained neural network object.

```
load("behaviorCloningMPCImDNNObject.mat");
```

Data Aggregation Approach

To improve the performance of the DNN, you can learn the policy using an interactive demonstrator method. DAgger is an iterative method where the DNN is run in the closed-loop environment. The expert, in this case the NLMPC controller, outputs actions based on the states visited by the DNN. In this manner, more training data is aggregated and the DNN is retrained for improved performance. For more information, see [1].

Train the deep neural network using the `DAggerTrainNetwork` function. It creates `DAggerImSlidingRobotDNNObj.mat` file that contains the following information.

- `DatasetPath`: path where the dataset corresponding to each iteration is stored
- `policyObjs`: policies that were trained in each iteration
- `finalData`: total training data collected till final iteration
- `finalPolicy`: best policy among all the collected policies

First, create and initialize the parameters for training. Use the network trained using behavior cloning (`behaviorCloningNNObj.imitateMPCNetObj`) as the starting point for the DAgger training.

```
[dataStruct,nlmpcStruct,tuningParamsStruct,neuralNetStruct] = ...
    loadDaggerParameters(existingData,numCol,nlobj,umax, ...
        options, behaviorCloningNNObj.imitateMPCNetObj);
```

To save time, load a pretrained neural network by setting `doTraining` to `false`. To train the DAgger yourself, set `doTraining` to `true`.

```
doTraining = false;

if doTraining
    DAgger = DAggerTrainNetwork( ...
        nlmpcStruct, ...
        dataStruct, ...
        neuralNetStruct, ...
        tuningParamsStruct);
else
    load("DAggerImSlidingRobotDNNObj.mat");
end
DNN = DAgger.finalPolicy;
```

As an alternative, you can train the neural network with a modified policy update rule using the `DAggerModifiedTrainNetwork` function. In this function, after every 20 training iterations, the DNN is set to the most optimal configuration from the previous 20 iterations. To run this example with a neural network object with the modified DAgger approach, use the `DAggerModifiedImSlidingRobotDNNObj.mat` file.

Compare Trained DAgger Network with NLMPC Controller

To compare the performance of the NLMPC controller and the trained DNN, run closed-loop simulations with the sliding robot model.

Set initial condition for the states of the sliding robot ($x, y, \theta, \dot{x}, \dot{y}, \dot{\theta}$) and the control variables of sliding robot (u_l, u_r).

```
x0 = [-1.8200 0.5300 -2.3500 1.1700 -1.0400 0.3100]';
u0 = [-2.1800 -2.6200]';
```

Define simulation duration, sample time and number of simulation steps.

```
% Duration
Tf = 15;

% Sample time
Ts = nlobj.Ts;

% Simulation steps
Tsteps = Tf/Ts+1;
```

Run a closed-loop simulation of the NLMPC controller.

```
tic
[xHistoryMPC,uHistoryMPC] = ...
    simModelMPCImSlidingRobot(x0,u0,nlobj,Tf);
toc
```

Elapsed time is 13.169980 seconds.

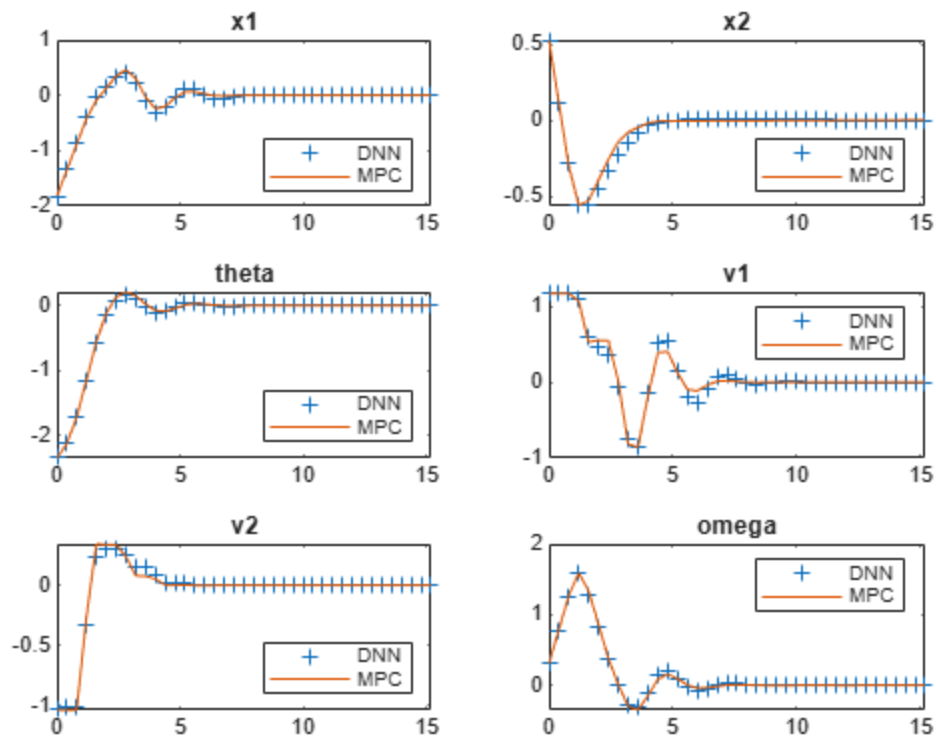
Run a closed-loop simulation of the trained DAGger network.

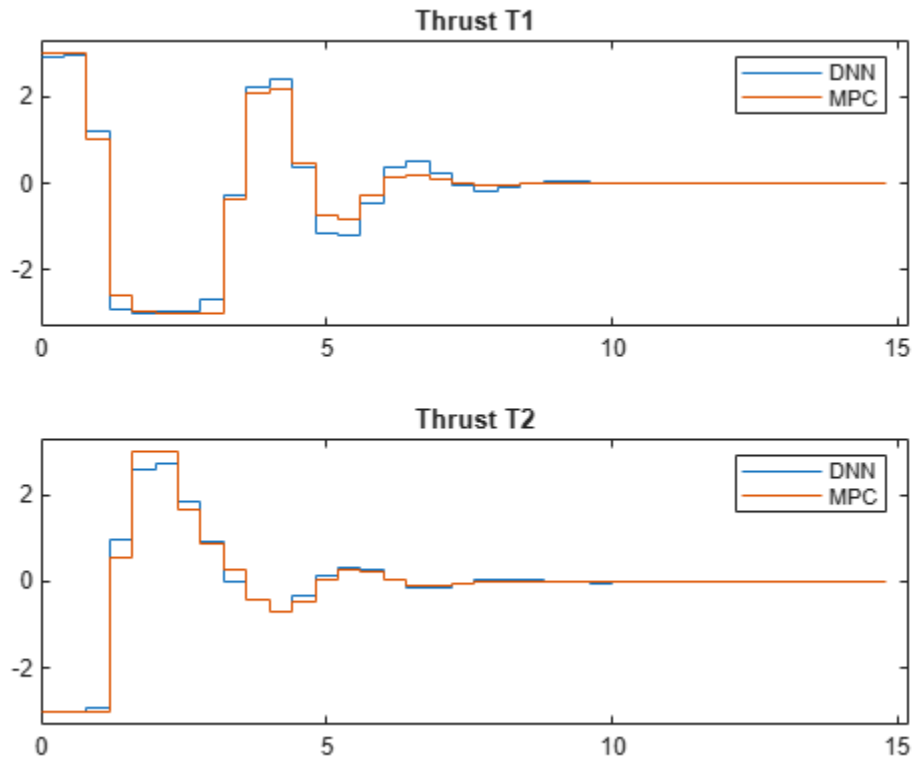
```
tic
[xHistoryDNN,uHistoryDNN] = ...
    simModelDaggerImSlidingRobot(x0,u0,DNN,Ts,Tf);
toc
```

Elapsed time is 1.050496 seconds.

Plot the results, and compare the NLMPC and trained DNN trajectories.

```
plotSimResultsImSlidingRobot(nlobj, ...
    xHistoryMPC,uHistoryMPC,xHistoryDNN,uHistoryDNN,umax,Tf);
```



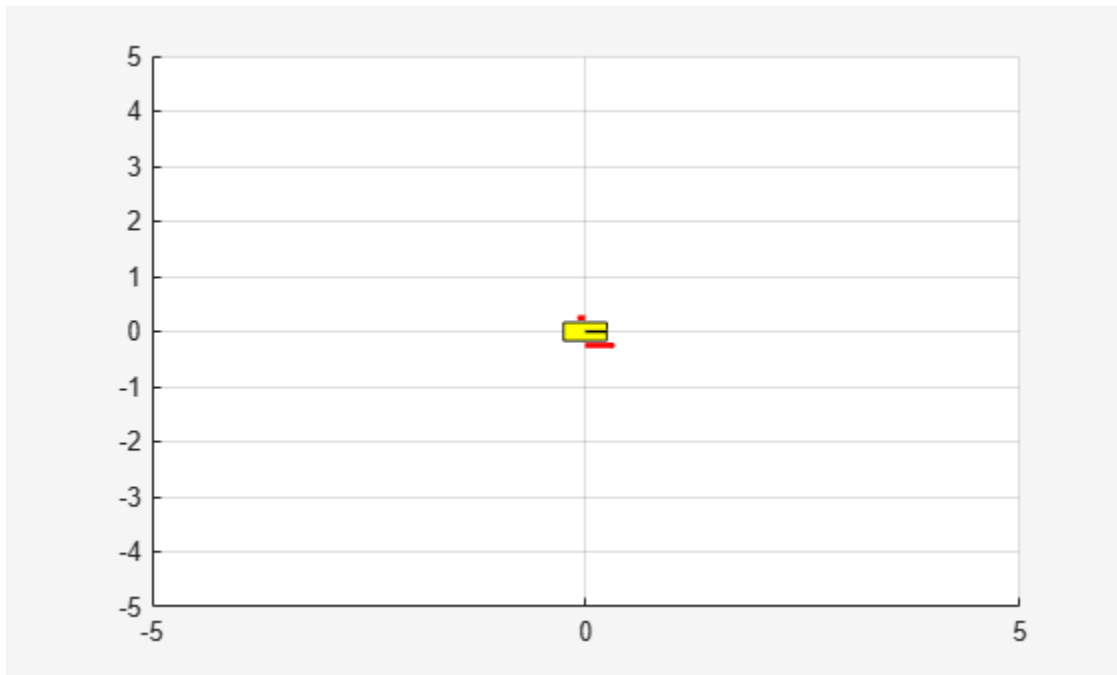


The Dagger neural network successfully imitates the behavior of the NLMPC controller. The sliding robot states and control action trajectories for the controller and the DAGger deep neural network closely align. The closed-loop simulation time for the DNN is significantly less than that of the NLMPC controller.

Animate the Sliding Robot with Trained DAGger Network

To validate the performance of the trained DNN, animate the sliding robot with data from the DNN closed-loop simulation. The sliding robot lands at the origin successfully.

```
Lx = 5;
Ly = 5;
for ct = 1:Tsteps
    x = xHistoryDNN(ct,1);
    y = xHistoryDNN(ct,2);
    theta = xHistoryDNN(ct,3);
    tL = uHistoryDNN(ct,1);
    tR = uHistoryDNN(ct,2);
    plotSlidingRobot(x,y,theta,tL,tR,Lx,Ly);
    pause(0.05);
end
```



```
% Turn on MPC messages  
mpcverbosity on;
```

References

[1] Osa, Takayuki, Joni Pajarinen, Gerhard Neumann, J. Andrew Bagnell, Pieter Abbeel, and Jan Peters. 'An Algorithmic Perspective on Imitation Learning'. *Foundations and Trends in Robotics* 7, no. 1-2 (2018): 1-179. <https://doi.org/10.1561/23000000053>.

[2] Y. Sakawa. "Trajectory planning of a free-flying robot by using the optimal control.", *Optimal Control Applications and Methods*, Vol. 20, 1999, pp. 235-248.

See Also

Functions

`trainNetwork` | `predict` | `nlmpcmove`

Objects

`SeriesNetwork` | `nlmpc`

Related Examples

- "Imitate MPC Controller for Lane Keeping Assist" on page 8-256
- "Train DDPG Agent to Control Two-Thruster Sliding Vehicle" on page 8-22
- "Trajectory Optimization and Control of Sliding Robot Using Nonlinear MPC" (Model Predictive Control Toolbox)
- "Train DDPG Agent with Pretrained Actor Network" on page 8-265

More About

- “Nonlinear MPC” (Model Predictive Control Toolbox)

Tune Single PI Controller Gains For Multiple Operating Points Using Reinforcement Learning

This example demonstrates how to tune the parameters of a controller using a reinforcement learning (RL) algorithm. Specifically, this example shows the first of three common approaches to using RL for proportional-integral-derivative (PID) parameter tuning:

1) Single fixed (PI) gains across multiple operating points (this example)

2) Per-simulation gain scheduling

3) Online adaptive PI gain adjustment

Tuning PI Gains Using RL: Three Approaches

For relatively simple control tasks with a small number of tunable parameters, model-based tuning techniques can get good results with a faster tuning process compared to model-free RL-based methods. See “Control System Design and Tuning” (Control System Toolbox) and “Get Started with Simulink Design Optimization” (Simulink Design Optimization) for more information. However, RL methods are more suitable for highly nonlinear systems or adaptive controller tuning. While this example focuses on PI controller tuning for a simple system, you can extend it to general parameter tuning or calibration in more complex control systems.

1. Single set of gains across multiple operating points (this example)



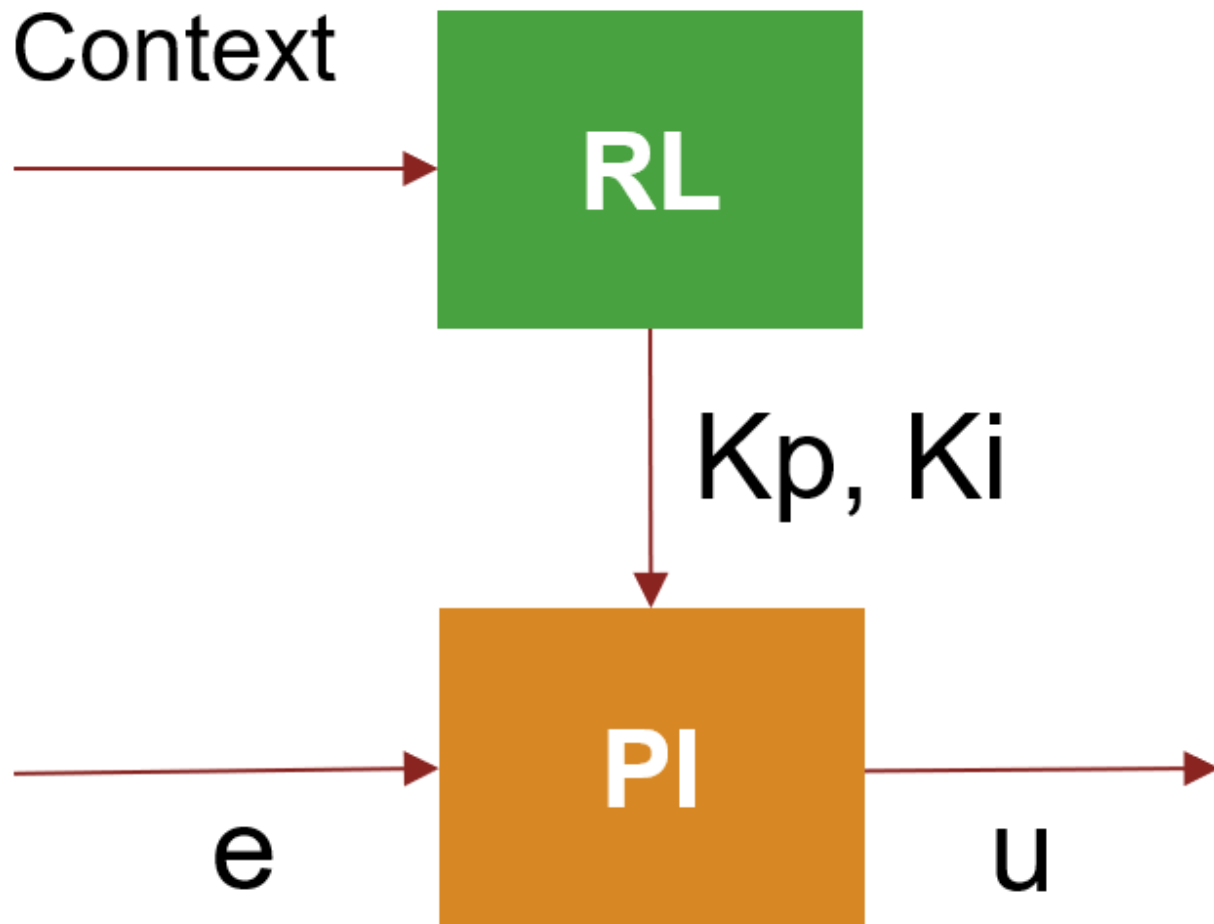
- **Goal:** Find one set of gains that performs well across different contexts (that is, across different operating conditions). This is also the preferred approach when you know the plant operating conditions and you know that they do not change substantially.

- **Approach:** Treat the PI controller as a linear RL policy where the weights directly map to PI gains. Learning the policy weights is equivalent to tuning the PI gains. The policy outputs the control command from the error signal. You can also extend this approach for parameter tuning or calibration applications.

- **Alternatives:** For many PI tuning problems, model-based tuning algorithm (see “Tune a Control System Using Control System Tuner” (Simulink Control Design)), swarm particle optimization (see

“Field-Oriented Control of PMSM Using Fuzzy PI Controller” (Fuzzy Logic Toolbox)), or Bayesian optimization can be simpler and more sample-efficient.

2. Per-simulation gain scheduling

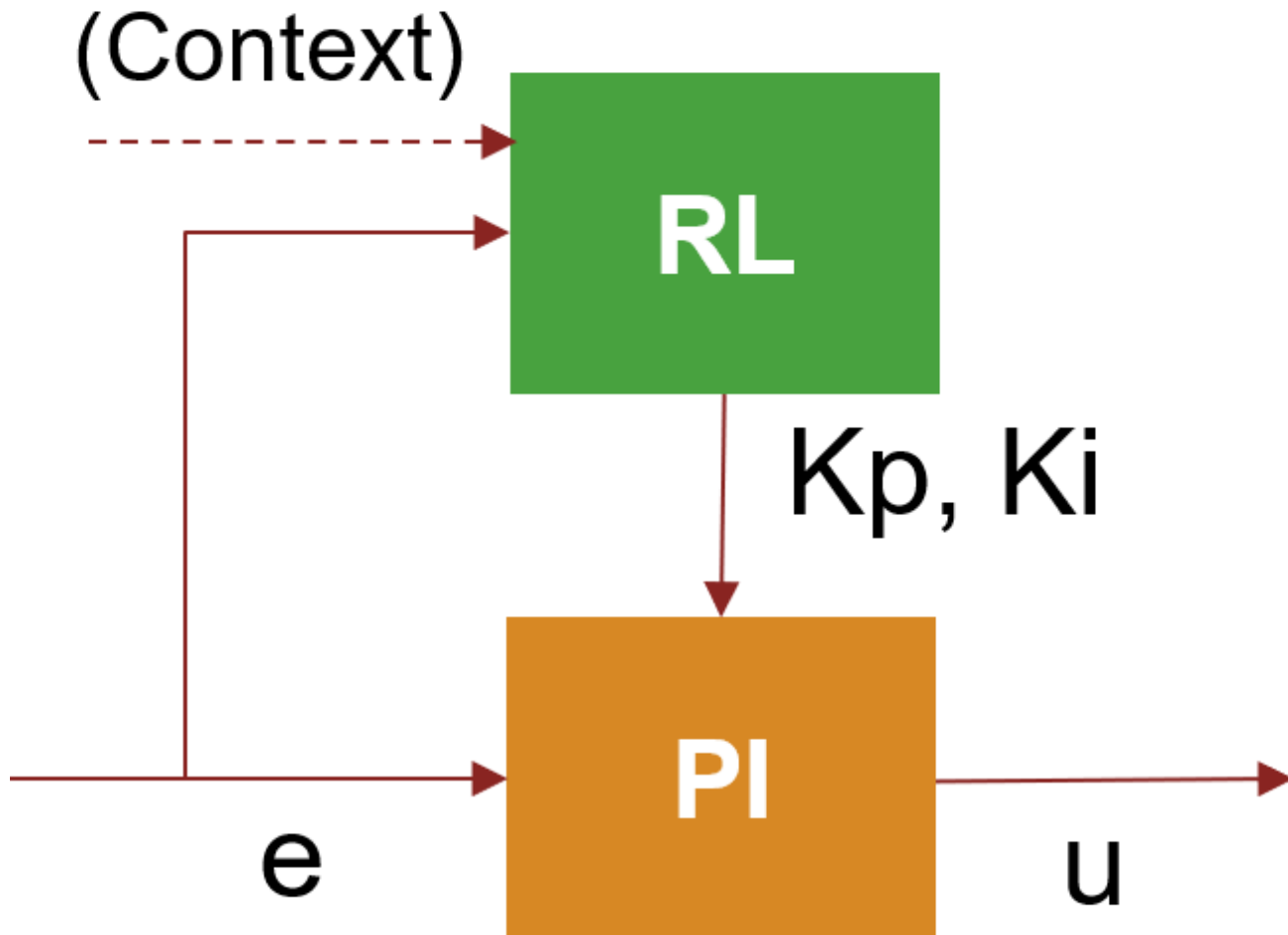


- **Goal:** Learn a mapping from a possible operating condition (hereby also referred to as context) to its PI gains, assuming that the context does not change while the plant is operating. Here, the RL policy associates a given operating condition to a specific set of PI gains.

- **Approach:** Use a contextual bandit approach (also referred to as single-step RL) to select the PI gains based on the current operating condition. Each RL environment step corresponds to a single simulation run, during which the PI gains remain fixed. Specifying the range of the actions (PI gains) is recommended for effective training. For an example on contextual bandits, see “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313.

- **Alternatives:** If your context space is small or you do not know the range of the PI gains, preparing multiple policies using the first approach or traditional approaches (see `sys tune` (Simulink Control Design) for example) can be a better choice. If the context changes during an episode, try the third approach (online adaptive PI gain adjustment).

3. Online adaptive PI gain adjustment



- **Goal:** Continuously adapt gains to improve performance. This is a more general approach than the previous ones, as the operating conditions and the PI gains are allowed to change while the plant is operating.

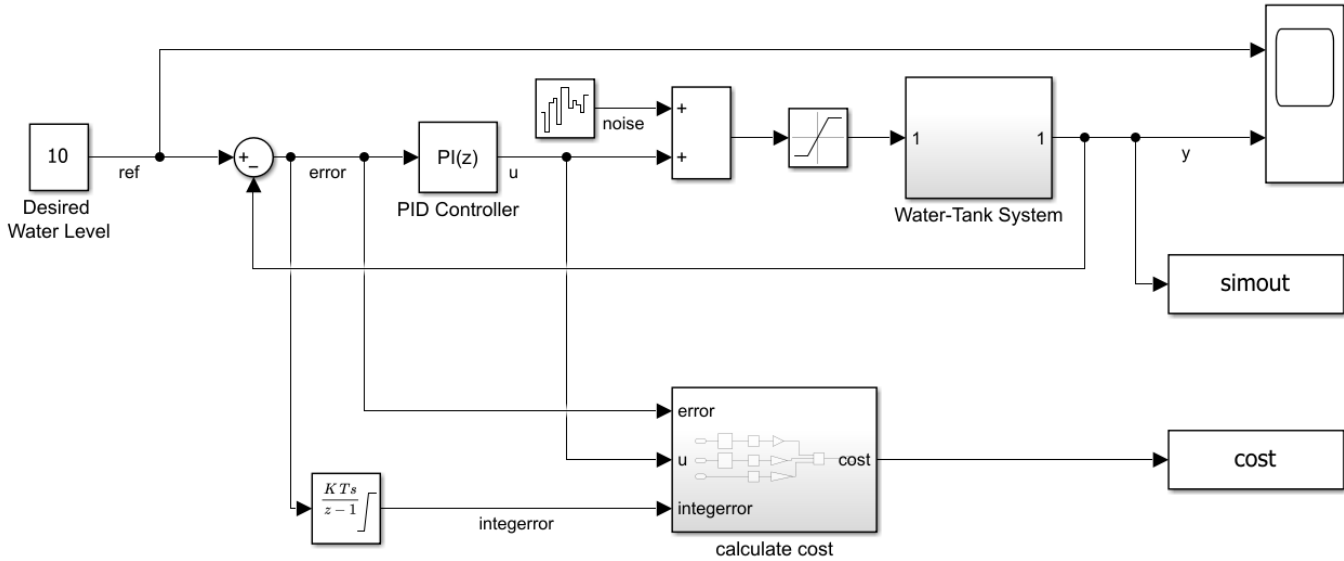
- **Approach:** Use an RL policy that updates the PI gains at each time step based on the current error. The context can also be used as additional observation.

- **Alternatives:** Using an RL based controller, in which the output of the RL agent is u instead of PI gains, or “Gain Scheduling” (Control System Toolbox) can be a better choice for plants that have a dynamics which is reasonably well known in advance. For an example on how to implement an RL-based controller, see “Control Water Level in a Tank Using a DDPG Agent” on page 8-2.

Environment Model

The environment model in this example is a water tank. The goal of this control system is to maintain the water level in the tank at a reference value.

```
WaterTankModel = 'watertankLQG';
open_system(WaterTankModel)
```



The model includes process noise with variance $E(n^2(t)) = 1$. The PID controller block used in the model is a discrete-time PID controller. `simout` is a To Workspace block that stores the water levels, and `cost` is a To Workspace block that stores the costs recorded during the simulation. The model includes process noise with variance $E(n^2(t)) = 0.01$.

To maintain the water level y while minimizing control effort u , the controller in this example uses the following a linear quadratic Gaussian (LQG) criterion for the discrete-time PID controller.

$$J_{LQG} = \lim_{T \rightarrow \infty} E \left(\frac{1}{T} \sum_{t=0}^T \left((H_{ref} - y(t))^2 + 0.01 u^2(t) + 0.0001 \left(\sum_{k=0}^t T_s (H_{ref} - y(k)) \right)^2 \right) \right)$$

Here, H_{ref} is the desired water level, T is the number of time steps, and T_s is the sample time. This LQG goal includes the integration error term to keep the steady-state error close to zero. To simulate the controller in this model, you must specify the simulation time T_f and the controller sample time T_s in seconds.

```
Ts = 1;
Tf = 100;
```

For more information about the water tank model, see “watertank Simulink Model” (Simulink Control Design).

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with the seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

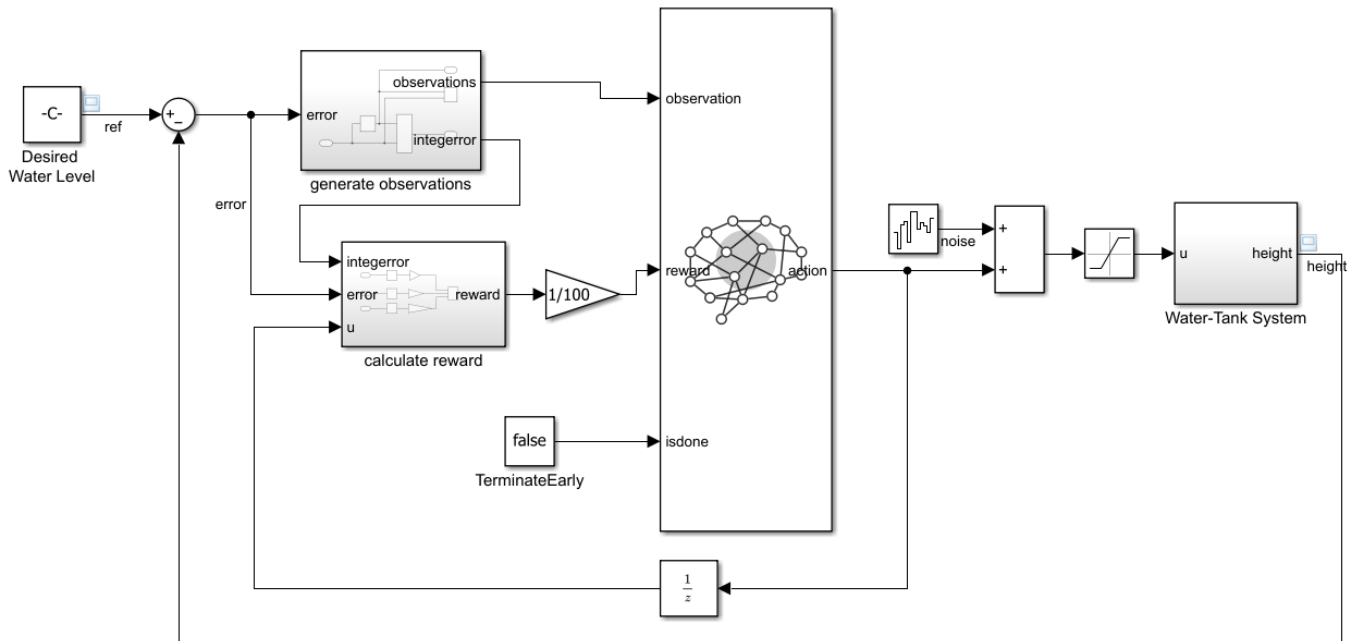
Create Environment Object

In this example, you implement the PI controller as a reinforcement learning policy that is linear in the observations. Over a training session consisting of 3000 episodes, where each episode is a simulation of the Simulink closed loop model lasting 100 time steps. During the simulation the reinforcement learning agent acts on the pump, observes the water level and its corresponding reward, and uses these observations to tune the weights of its actor model (which are the PI gains) with the objective to maximize the expected cumulative reward.

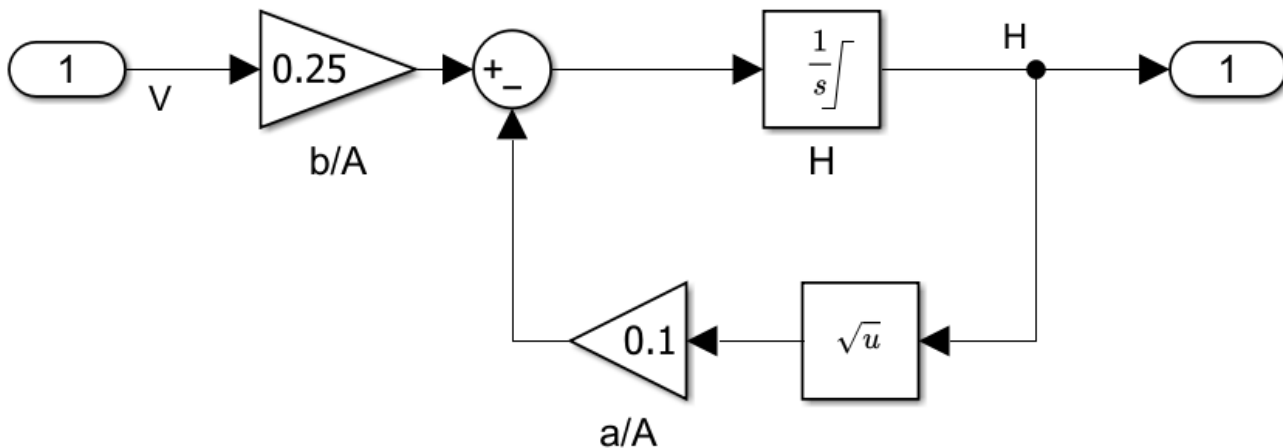
To define the model for training the RL agent, modify the water tank model using the following steps.

- 1 Delete the PID Controller.
- 2 Insert an RL Agent block.
- 3 Create the observation vector $[fe \ dt \ e]^T$, where $e = Href - h$, h is the level of the water in the tank, and $Href$ is the reference water level. Connect the observation signal to the RL Agent block.
- 4 Define the reward function for the RL agent as the **negative** of the scaled LQG cost, that is,
$$\text{Reward} = -0.01 \left((H_{ref} - h(t))^2 + 0.01 u^2(t) + 0.0001 \left(\sum_{k=0}^t T_s (H_{ref} - y(k)) \right)^2 \right).$$
 The RL agent maximizes this reward, thus minimizing the LQG cost. A scaling factor 0.01 is applied to each step's reward so that its expected range is approximately between -1 and 0. This results in a better optimization in general. The integral error term in the LQG cost is not required for reinforcement learning; however, it is included in this example for consistency with the Control System Tuner approach presented later for comparison.

The resulting model is `rlWatertankPITuningUseCase1.slx`.



The Water-Tank System in `rlWatertankPITuningUseCase1` is shown below:



Here, V is the voltage applied to the pump, H is the water level in the tank. For more information about the water tank model, see “watertank Simulink Model” (Simulink Control Design).

Define the observation specification `obsInfo` and the action specification `actInfo`.

```
obsInfo = rlNumericSpec([2 1]);
obsInfo.Name = 'observations';
obsInfo.Description = 'integrated error and error';

actInfo = rlNumericSpec([1 1]);
```

```
actInfo.Name = 'PID output';  
actInfo.LowerLimit = 0;  
actInfo.UpperLimit = 12;
```

Create the water tank RL environment object using `rlSimulinkEnv`.

```
mdl = 'rlWatertankPITuningUseCase1';  
env = rlSimulinkEnv(mdl,[mdl '/RL Agent'],obsInfo,actInfo);
```

Set a custom reset function that randomizes the reference value and the initial water level for the model. To set the reset function, use `localResetFcn` on page 8-297, which is defined at the end of this example.

```
env.ResetFcn = @(in)localResetFcn(in,mdl);
```

Extract the observation and action dimensions for this environment. Use `prod(obsInfo.Dimension)` and `prod(actInfo.Dimension)` to obtain the number of dimensions of the observation and action spaces, respectively, regardless of whether they are arranged as row vectors, column vectors, or matrices.

```
numObs = prod(obsInfo.Dimension);  
numAct = prod(actInfo.Dimension);
```

Create RL Agent

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

You can model a PI controller as an actor. The actor uses both the error integral and the error as inputs, and outputs the control signal.

$$u = \left[\int e \, dt \, e \right] * [K_i \, K_p]^T$$

Here:

- u is the output of the actor.
- K_i and K_p are the weights of the actor.
- $e = H_{ref} - h(t)$, where $h(t)$ is the water level of the water in the tank, and H_{ref} is the reference water level.

To create the actor, use `rlContinuousDeterministicActor` with a basis function `myBasisFcn`, which is provided as a separate file in the example folder.

Display the file implementing the basis function.

```
type("myBasisFcn.m")  
  
function feature = myBasisFcn(obs)  
    % A basis function for Tune PI Controller Using Reinforcement Learning example  
  
    % Copyright 2024 The MathWorks Inc.  
  
    feature = obs;  
end
```

Define the initial controller gains (K_i and K_p , respectively).

```
initialGain = single([1e-3 2]);
```

Create the actor. For more information, see `rlContinuousDeterministicActor`.

```
actor = rlContinuousDeterministicActor( ...
    {@myBasisFcn,initialGain'},obsInfo, actInfo);
```

The agent in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents rely on actor and critic approximator objects to learn the optimal policy. A TD3 agent approximates the long-term reward, given its observations and actions, by using two critic value-function representations.

Create the critic network using the function `createCriticNet`, which is defined in the following code section. Use the same network structure for both critic representations.

```
obsInputName = 'stateInLyr';
actInputName = 'actionInLyr';
criticNet1 = createCriticNet(numObs,numAct,obsInputName,actInputName);
criticNet2 = createCriticNet(numObs,numAct,obsInputName,actInputName);

function criticNet ...
    = createCriticNet(numObs,numAct,obsInputName,actInputName)
    statePath = [
        featureInputLayer(numObs,Name=obsInputName)
        fullyConnectedLayer(64,Name='fc1')
    ];

    actionPath = [
        featureInputLayer(numAct,Name=actInputName)
        fullyConnectedLayer(64,Name='fc2')
    ];

    commonPath = [
        concatenationLayer(1,2,Name='concat')
        reluLayer
        fullyConnectedLayer(64)
        reluLayer
        fullyConnectedLayer(1,Name='qvalOutLyr')
    ];

    criticNet = dlnetwork();
    criticNet = addLayers(criticNet,statePath);
    criticNet = addLayers(criticNet,actionPath);
    criticNet = addLayers(criticNet,commonPath);

    criticNet = connectLayers(criticNet,'fc1','concat/in1');
    criticNet = connectLayers(criticNet,'fc2','concat/in2');
end
```

Create the critic objects using the specified neural network, the environment action and observation specifications, and the names of the network layers to be connected with the observation and action channels.

```
critic1 = rlQValueFunction(criticNet1, ...
    obsInfo,actInfo, ...
    ObservationInputNames=obsInputName, ...
    ActionInputNames=actInputName);
```

```
critic2 = rlQValueFunction(criticNet2, ...
    obsInfo,actInfo, ...
    ObservationInputNames=obsInputName,...
    ActionInputNames=actInputName);
```

Define the vector of critic objects.

```
critic = [critic1 critic2];
```

Specify the training options for the actor and critic.

```
actorOpts = rlOptimizerOptions( ...
    LearnRate=5e-4, ...
    GradientThreshold=0.5);
```

```
criticOpts = rlOptimizerOptions( ...
    LearnRate=5e-4, ...
    GradientThreshold=0.5);
```

Specify the TD3 agent options using `rlTD3AgentOptions`. Include the training options for the actor and critic.

- Set the agent to use the controller sample time `Ts`.
- Set the mini-batch size to 256 experience samples.
- Set the experience buffer length to `1e6`.
- Set the learning frequency to -1 to update the agent at the end of each episode (default value).
- Set the max number of minibatches per epoch to 20.

```
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts, ...
    MiniBatchSize=256, ...
    DiscountFactor=0.99,...
    ExperienceBufferLength = 1e6, ...
    MaxMiniBatchPerEpoch = 20,...
    LearningFrequency = -1,...
    ActorOptimizerOptions=actorOpts, ...
    CriticOptimizerOptions=criticOpts);
```

To modify the exploration model, use dot notation.

```
agentOpts.ExplorationModel.StandardDeviation = 0.5;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 5e-5;
agentOpts.TargetPolicySmoothModel.StandardDeviation = sqrt(0.1);
```

Create the TD3 agent using the specified actor representation, critic representation, and agent options. For more information, see `rlTD3AgentOptions`.

```
agent1 = rlTD3Agent(actor,critic,agentOpts);
```

Train RL Agent

Fix the random number stream for reproducibility.

```
rng(0,"twister");
```


Create an evaluator object to evaluate the performance of the agent over 10 simulations every 50 episodes, using the random seeds 1 to 10. The different random seed allows different random initial conditions. The evaluation score is the average cumulative reward over the 10 evaluation episodes.

```
evaluatorObj = rlEvaluator("NumEpisodes",10,...
    "EvaluationFrequency",50,...
    "RandomSeeds",1:10);
```

To train the agent, first specify the following training options.

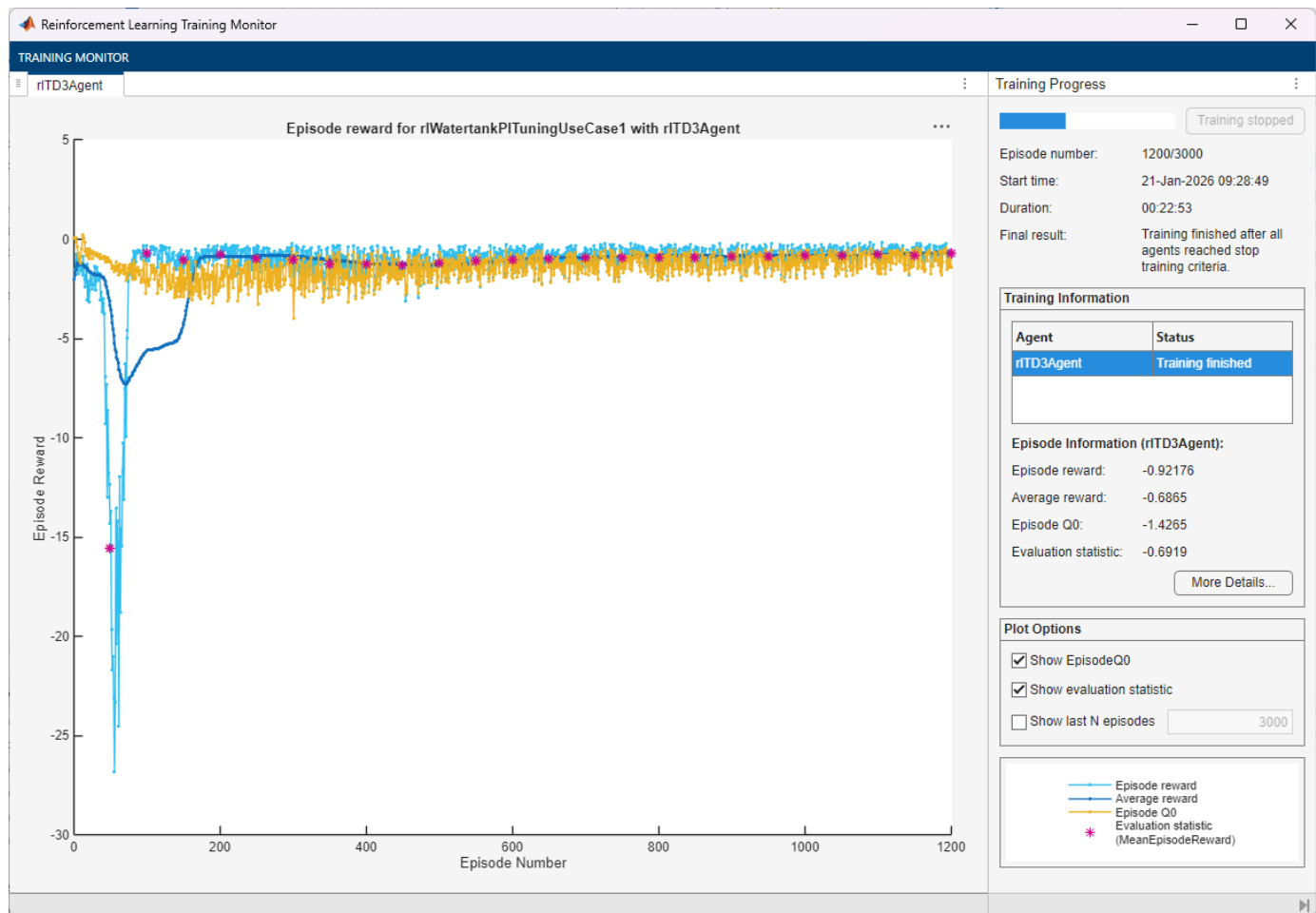
- Run each training for a maximum of 3000 episodes, with each episode lasting a maximum of 100 time steps.
- Display the training progress in the Reinforcement Learning Training Monitor (set the `Plots` option as "training-progress").
- Disable the command-line display (set the `Verbose` option as `true`).
- Stop the training when the evaluation score reaches -0.7. At this point, the agent can control the level of water in the tank.

For more information on training options, see `rlTrainingOptions`.

```
maxepisodes = 3000;
maxsteps = ceil(Tf/Ts);
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=maxepisodes, ...
    MaxStepsPerEpisode=maxsteps, ...
    ScoreAveragingWindowLength=100, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=-0.7);
```

Train the agent using the `train` function. Training this agent is a computationally intensive process. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent.
    trainingStats = train(agent1,env,trainOpts,Evaluator=evaluatorObj);
    save("WaterTankPITuningTD3AgentUseCase1.mat","agent1")
else
    % Load pretrained agent for the example.
    load("WaterTankPITuningTD3AgentUseCase1.mat","agent1")
end
```



Validate Trained Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to true. To validate the trained agent, simulate it within the environment `maxsteps` steps. For more information on agent simulation, see `sim`.

```
simOpts = rlSimulationOptions(MaxSteps=maxsteps);
experiences = sim(env, agent1, simOpts);
```

Compare Performance of RL and Control System Tuner Gains

You can tune a controller in Simulink using Control System Tuner (CST). To do so, you must specify the controller block as a tuned block and define the goals for the tuning process, and then on the **Tuning** tab, click **Tune**. For more information on using **Control System Tuner**, see “Tune a Control System Using Control System Tuner” (Simulink Control Design).

The values of the PI gains obtained using CST are as follows:

```
Kp_CST = 3.43726595858363;
Ki_CST = 0.0357784909005128;
```

The PI gains of the RL controller are the weights of the actor approximation model. To obtain the weights, first extract the learnable parameters from the actor.

```
actor = getActor(agent1);
parameters = getLearnableParameters(actor);
```

Obtain the controller gains.

```
Ki = parameters{1}(1)
```

```
Ki = single
```

```
0.1130
```

```
Kp = parameters{1}(2)
```

```
Kp = single
```

```
3.8090
```

Apply the gains obtained from the trained RL agent to the original PI controller block and run a step-response simulation.

```
open_system(WaterTankModel);
set_param(WaterTankModel,"FastRestart","off")
```

Use the Simulink.SimulationInput (Simulink) object `simIn` to temporarily set the random seed for the noise, reference, and initial water levels in the respective blocks.

```
refWaterLevel = 10;
initialWaterLevel = 1;
randomSeed = 1;
simIn = Simulink.SimulationInput(WaterTankModel);
noiseBlk = sprintf([WaterTankModel '/Band-Limited\nWhite Noise/']);
simIn = setBlockParameter(simIn,noiseBlk,'Seed',num2str(randomSeed));
refBlk = sprintf([WaterTankModel '/Desired \nWater Level/']);
simIn = setBlockParameter(simIn,refBlk,'Value',num2str(refWaterLevel));
initBlk = [WaterTankModel '/Water-Tank System/H/'];
simIn = setBlockParameter(simIn,initBlk,'InitialCondition',num2str(initialWaterLevel));
```

Set the PI gains obtained from the trained RL agent in the PID controller block.

```
PIBlk = [WaterTankModel '/PID Controller/'];
set_param([WaterTankModel '/PID Controller/'],'P',num2str(Kp))
set_param([WaterTankModel '/PID Controller/'],'I',num2str(Ki))
```

Use the Simulink `sim` function to simulate the water tank model with the applied temporary changes. Collect the water level and cost recorded during simulation in `simResult`. These values were obtained using To Workspace blocks (`simout` and `cost`) in the model.

```
simResults = sim(simIn);
rlStep1 = simResults.simout;
rlCost1 = simResults.cost;
```

Calculate the stability margin for the open-loop system. To compute the stability margin, use the `localStabilityAnalysis` on page 8-298 function defined at the end of this example. The function

returns a structure containing several stability margins for a snapshot of the system at a given time. Here, a time of 50 seconds is selected because it represents the midpoint of the 100-second simulation, when the system is expected to be near or at steady state.

```
blockIn = [WaterTankModel '/Sum1'];
blockOut = [WaterTankModel '/Water-Tank System'];
stabilityAnalysisTime = 50;
rlStabilityMargin = localStabilityAnalysis(WaterTankModel,...
                                           blockIn,...
                                           blockOut,...
                                           stabilityAnalysisTime);
```

Apply the gains obtained using **Control System Tuner** to the original PI controller block and run a step-response simulation.

```
set_param([WaterTankModel '/PID Controller'],'P',num2str(Kp_CST))
set_param([WaterTankModel '/PID Controller'],'I',num2str(Ki_CST))
```

Use the Simulink `sim` function to simulate the water tank model with the applied temporary changes. Collect the simulation results in `simResult`.

```
simResults = sim(simIn);
cstStep = simResults.simout;
cstCost = simResults.cost;
```

Calculate the stability margin for the open-loop system using `localStabilityAnalysis` on page 8-298. Use the same `stabilityAnalysisTime` value of 50 seconds as for the RL controller.

```
cstStabilityMargin = localStabilityAnalysis(WaterTankModel,...
                                           blockIn,...
                                           blockOut,...
                                           stabilityAnalysisTime);
```

To analyze the step response for both simulations, use the `stepinfo` (Control System Toolbox) function.

```
rlStepInfo = stepinfo(rlStep1.Data,rlStep1.Time);
cstStepInfo = stepinfo(cstStep.Data,cstStep.Time);
```

Convert the structure returned from `stepinfo` (Control System Toolbox) into a table variable. For more information about table data type, see “Tables”.

```
stepInfoTable = struct2table([cstStepInfo rlStepInfo]);
stepInfoTable = removevars(stepInfoTable,{'SettlingMin', ...
    'TransientTime','SettlingMax','Undershoot','PeakTime'});
stepInfoTable.Properties.RowNames = {'CST','RL1'};
```

Display the table.

`stepInfoTable`

`stepInfoTable=2×4 table`

	RiseTime	SettlingTime	Overshoot	Peak
CST	2.8482	3.896	0.75978	9.9904
RL1	2.8737	12.582	2.7717	10.269

Convert the stability margin structures to table variables.

```
stabilityMarginTable = struct2table( ...
    [cstStabilityMargin rlStabilityMargin]);
```

Remove unneeded variables, and add row names.

```
stabilityMarginTable = removevars(stabilityMarginTable,{...
    'GMFrequency','PMFrequency','DelayMargin','DMFrequency'});
stabilityMarginTable.Properties.RowNames = {'CST','RL1'};
```

Display the stability margin table.

```
stabilityMarginTable

stabilityMarginTable=2x3 table
      GainMargin  PhaseMargin  Stable
      _____  _____  _____
      CST         2.3401       67.134   true
      RL1         2.1324       63.126   true
```

Both controllers produce stable responses.

Compute the cumulative LQG costs for the two controllers.

```
rlCumulativeCost = sum(rlCost1.Data)

rlCumulativeCost =
-155.3710

cstCumulativeCost = sum(cstCost.Data)

cstCumulativeCost =
-155.7214
```

The step response, stability margin, and LQG cost results are comparable between the two tuning approaches.

Local Functions

The `localResetFcn` function randomizes the reference signal and the initial water level.

```
function in = localResetFcn(in,mdl)
    % Randomize disturbance signal.
    randomSeed = randi(10000);
    noiseBlk = sprintf([mdl '/Band-Limited\nWhite Noise/']);
    in = setBlockParameter(in,noiseBlk,'Seed',num2str(randomSeed));

    % Randomize reference signal.
    hRef = 10 + 4*(rand-0.5);
    hRefBlk = sprintf([mdl '/Desired \nWater Level/']);
    in = setBlockParameter(in,hRefBlk,'Value',num2str(hRef));

    % Randomize initial water level.
    hInit = 2*rand;
```

```
hInitBlk = [mdl '/Water-Tank System/H'];  
in = setBlockParameter(in,hInitBlk,'InitialCondition',num2str(hInit));  
end
```

The `localStabilityAnalysis` function computes the stability margin by linearizing the closed-loop model at a specified time using loop opening analysis points

```
function margin = localStabilityAnalysis(mdl,blockIn,blockOut,time)  
    set_param(mdl,"FastRestart","off")  
    io(1) = linio(blockIn,1,'input');  
    io(2) = linio(blockOut,1,'openoutput');  
    op = operpoint(mdl);  
    op.Time = time;  
    linsys = linearize(mdl,io,op);  
    margin = allmargin(linsys);  
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlTD3Agent` | `rlTD3AgentOptions` | `rlTrainingOptions`

Related Examples

- “Compare DDPG Agent to LQR Controller” on page 7-285
- “Train TD3 Agent for PMSM Control” on page 8-242
- “Create and Train Custom LQR Agent” on page 5-345

More About

- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Define Custom Deep Learning Layers”
- “Train Reinforcement Learning Agents” on page 5-3

Train DQN Agent with LSTM Network to Control House Heating System

This example shows how to train a deep Q-learning network (DQN) agent with a Long Short-Term Memory (LSTM) network to control a house heating system modeled in Simscape™. For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32.

Fix Random Number Seed to Improve Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

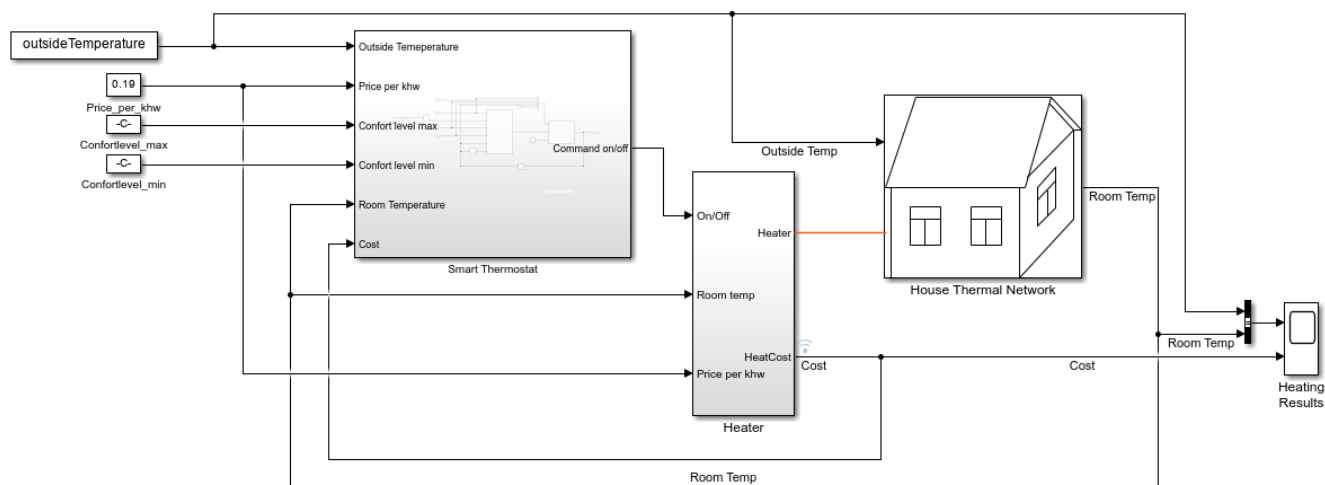
Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

House Heating Simulink Model

The reinforcement learning (RL) environment for this example uses the Simulink® model described in the “House Heating System” (Simscape) example. The model contains a heater block, a thermostat block (which contains an RL agent block and a reward block) and the House Thermal Network block (modeled in Simscape). Heat moves between the interior of the home and the outside environment through the walls, windows, and roof. You use weather station data between March 21 and April 15, 2022, to simulate the outside temperature. For more information about the data acquisition, see “Compare Temperature Data from Three Different Days” (ThingSpeak).



The training goal for the agent is to minimize the energy cost and maximize the amount of time the house is in the comfortable temperature range by turning on/off the heater. The house is comfortable when its interior temperature T_{room} is between $T_{\text{comfortMin}}$ and $T_{\text{comfortMax}}$.

The reinforcement learning problem is defined as follows:

- The observation is a 6-dimensional column vector that consists of the room temperature ($^{\circ}\text{C}$), outside temperature ($^{\circ}\text{C}$), maximum comfort temperature ($^{\circ}\text{C}$), minimum comfort temperature ($^{\circ}\text{C}$), last action, and price per kWh (USD).
- The action is discrete. Specifically, the action 0 represents turning the heater off, and the action 1 represents turning it on.
- The reward block calculates the total reward R as the sum of the comfort reward R_c and a penalty for switching P_s , from which an energy cost C_e is subtracted: $R = R_c + P_s - C_e$. These three components are defined as follows.

$$R_c = \begin{cases} 0.1 & \text{if } T_{\text{comfortMin}} \leq T_{\text{room}} \leq T_{\text{comfortMax}} \\ -w|T_{\text{room}} - T_{\text{comfortMin}}| & \text{if } T_{\text{room}} < T_{\text{comfortMin}} \\ -w|T_{\text{room}} - T_{\text{comfortMax}}| & \text{if } T_{\text{room}} > T_{\text{comfortMax}} \end{cases}$$

where $w = 0.1$, $T_{\text{comfortMin}} = 18$, and $T_{\text{comfortMax}} = 23$.

$$P_s = \begin{cases} -0.01 & \text{if } a_t \neq a_{t-1} \text{ where } a_t \text{ is the current action and } a_{t-1} \text{ is the previous action} \\ 0 & \text{otherwise} \end{cases}$$

$$C_e = \text{CostPerStep} = \text{PricePerKwh} * \text{ElectricityUsed}$$

The reward function is inspired by [1].

- The Is-Done signal supplied to the RL Agent block is always 0, indicating that there is no early termination condition.

Define Simulink Environment

Open the model.

```
mdl = "rlHouseHeatingSystem";
open_system(mdl)
```

Define the sample time and the maximum number of steps per episode. You need these variables for the script and reset functions.

```
sampleTime = 120; % seconds
maxStepsPerEpisode = 1000;
```

Assign the agent block path information to the `agentBlk` variable, for later use.

```
agentBlk = mdl + "/Smart Thermostat/RL Agent";
```

Load the outside temperature data to simulate the environment temperature.

```
data = load('temperatureMar21toApr15_2022.mat');
temperatureData = data.temperatureData;
```

Extract validation data.


```
temperatureMarch21 = temperatureData(1:60*24,:);
temperatureApril15 = temperatureData(end-60*24+1:end,:);
```

Extract training data.

```
temperatureData = temperatureData(60*24+1:end-60*24,:);
```

The Simulink model loads the outside temperature history, the maximum comfort temperature, and the minimum comfort temperature. These variables are part of the observation vector.

```
outsideTemperature = temperatureData;
comfortMax = 23;
comfortMin = 18;
```

Define the observation and action specifications.

```
obsInfo = rlNumericSpec([6,1]);
actInfo = rlFiniteSetSpec([0,1]); % (0=off,1=on)
```

Create an environment object for the house heating environment.

```
env = rlSimulinkEnv mdl, agentBlk, obsInfo, actInfo);
```

Use `hRLHeatingSystemResetFcn` to reset the environment at the beginning of each episode. This function randomly selects a time between March 22 and April 14 such that at least `maxStepsPerEpisode` samples are available. The environment uses this time as the initial time for the outside temperatures. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in) hRLHeatingSystemResetFcn(in);
```

Create DQN Agent with LSTM Network

DQN agents use a parameterized Q-value function approximator to estimate the value of the policy. Because DQN agents have a discrete action space, you have the option to create a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic.

A vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible actions. The value of each output element represents the expected discounted cumulative long-term reward when an agent starts from the state corresponding to the given observation and executes the action corresponding to the element number (and follows a given policy afterwards).

Fix the random number stream.

```
rng(0, "twister");
```

Specify training options for the critic and the actor using `rlOptimizerOptions`.

```
criticOpts = rlOptimizerOptions( ...
    LearnRate=0.001, ...
    GradientThreshold=1);
```

Specify the DQN agent options using `rlDQNAgentOptions`, include the training options for the critic.

```
agentOpts = rlDQNAgentOptions( ...
    UseDoubleDQN = false, ...
```

```
TargetSmoothFactor = 1, ...
TargetUpdateFrequency = 4, ...
ExperienceBufferLength = 1e6, ...
CriticOptimizerOptions = criticOpts, ...
MiniBatchSize = 64);
```

You can also set or modify the agent options using dot notation.

```
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 0.0001;
```

To model the parameterized Q-value function within the critic, use a recurrent neural network, which can capture the effect of previous observations. By setting the `UseRNN` option in `rlAgentInitializationOptions`, you can create a default DQN agent with an LSTM network. Alternatively, you can manually configure the LSTM network. See “Water Distribution System Scheduling Using Reinforcement Learning” on page 8-361 to create an LSTM network for the DQN agent manually.

```
useRNN = ;
initOpts = rlAgentInitializationOptions( ...
    UseRNN=useRNN, ...
    NumHiddenUnit=64);
```

When using a recurrent neural network, you must set `SequenceLength` to be greater than 1 in `rlDQNAgentOptions`. This option is used in training to determine the length of the mini-batch used to calculate the gradient. For more information about LSTM layers, see “Long Short-Term Memory Neural Networks”.

```
if useRNN
    agentOpts.SequenceLength = 20;
end
```

Create a DQN agent with default critic network using `rlDQNAgent`, the agent initialization options object (which specify that a recurrent network is needed), and the agent options object.

```
agent = rlDQNAgent(obsInfo, actInfo, initOpts, agentOpts);
```

You can also modify the agent options, including the critic options, using dot notation. For example, specify the agent sample time.

```
agent.SampleTime = sampleTime;
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run training for a maximum of 500 episodes, with each episode lasting 1000 time steps.
- Set the `Plots` option to “training-progress”, which displays training progress in the Reinforcement Learning Training Monitor
- Set the `Verbose` option to `false` to disable the command line display.
- Stop the training when the agent receives a mean evaluation episode reward greater than 85.

For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes = 500, ...
```

```

MaxStepsPerEpisode = maxStepsPerEpisode, ...
ScoreAveragingWindowLength = 5, ...
Verbose = false, ...
Plots = "training-progress", ...
StopTrainingCriteria = "EvaluationStatistic", ...
StopTrainingValue = 85);

```

Fix the random number stream.

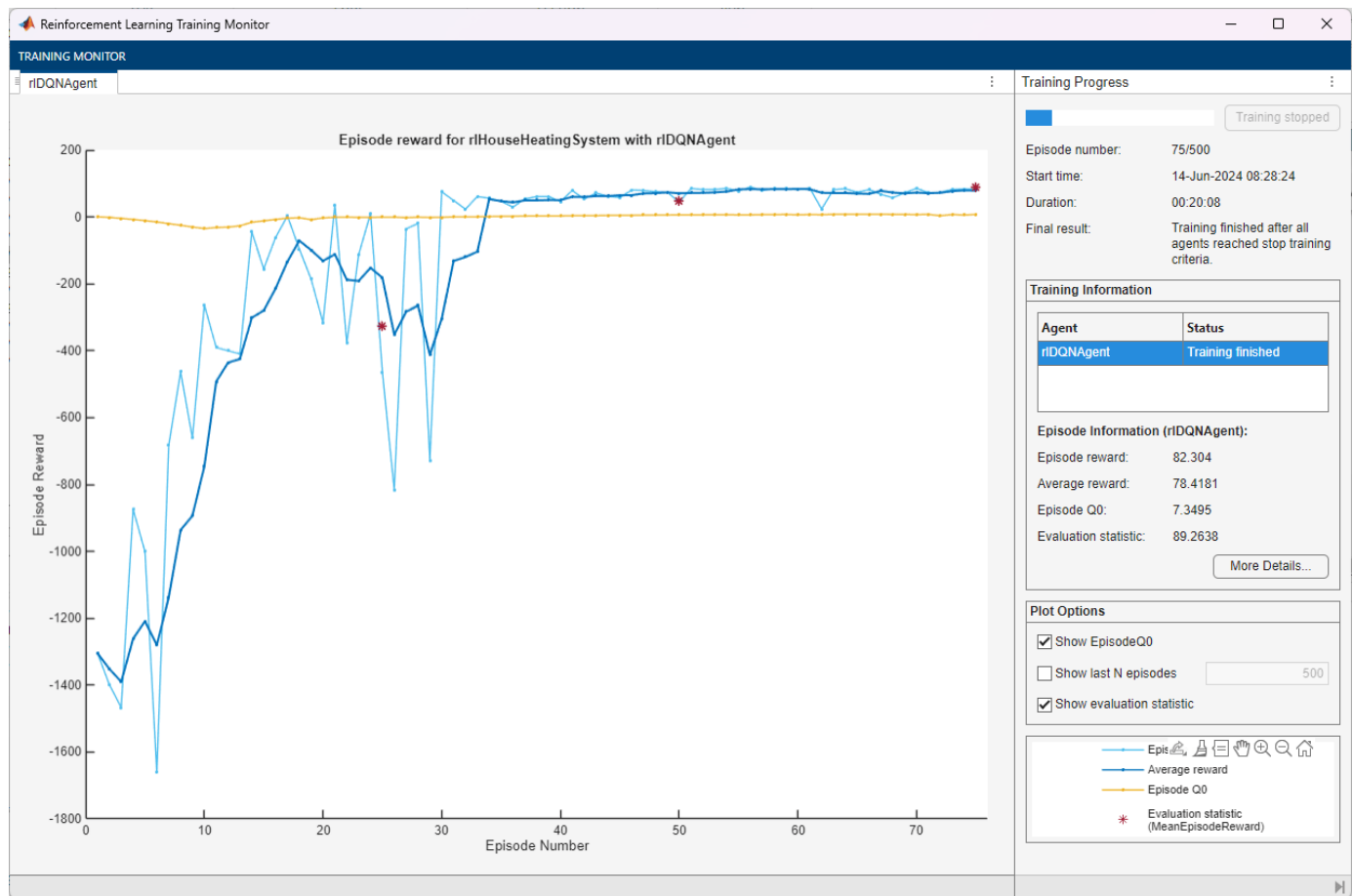
```
rng(0, "twister");
```

Train the agent using the `train` function. Training this agent is a computationally-intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```

doTraining = ;
if doTraining
    % Evaluate the agent every 25 training episodes.
    evl = rlEvaluator( ...
        EvaluationFrequency=25, ...
        NumEpisodes=3, ...
        RandomSeeds=[1:3]);
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=evl);
else
    % Load the pretrained agent for the example.
    load("HeatControlDQNAgent.mat","agent")
end

```



Simulate DQN Agent

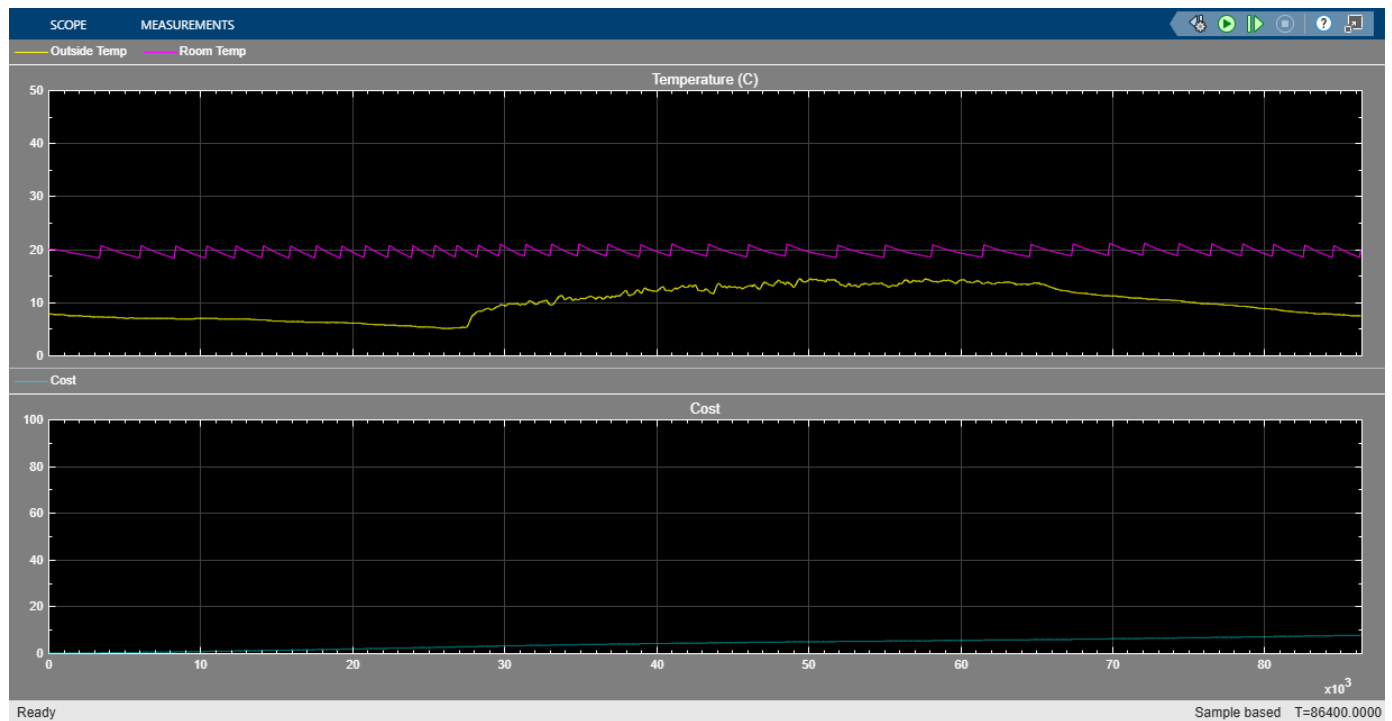
To validate the performance of the trained agent, simulate it within the house heating system. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

Fix the random number stream.

```
rng(0, "twister");
```

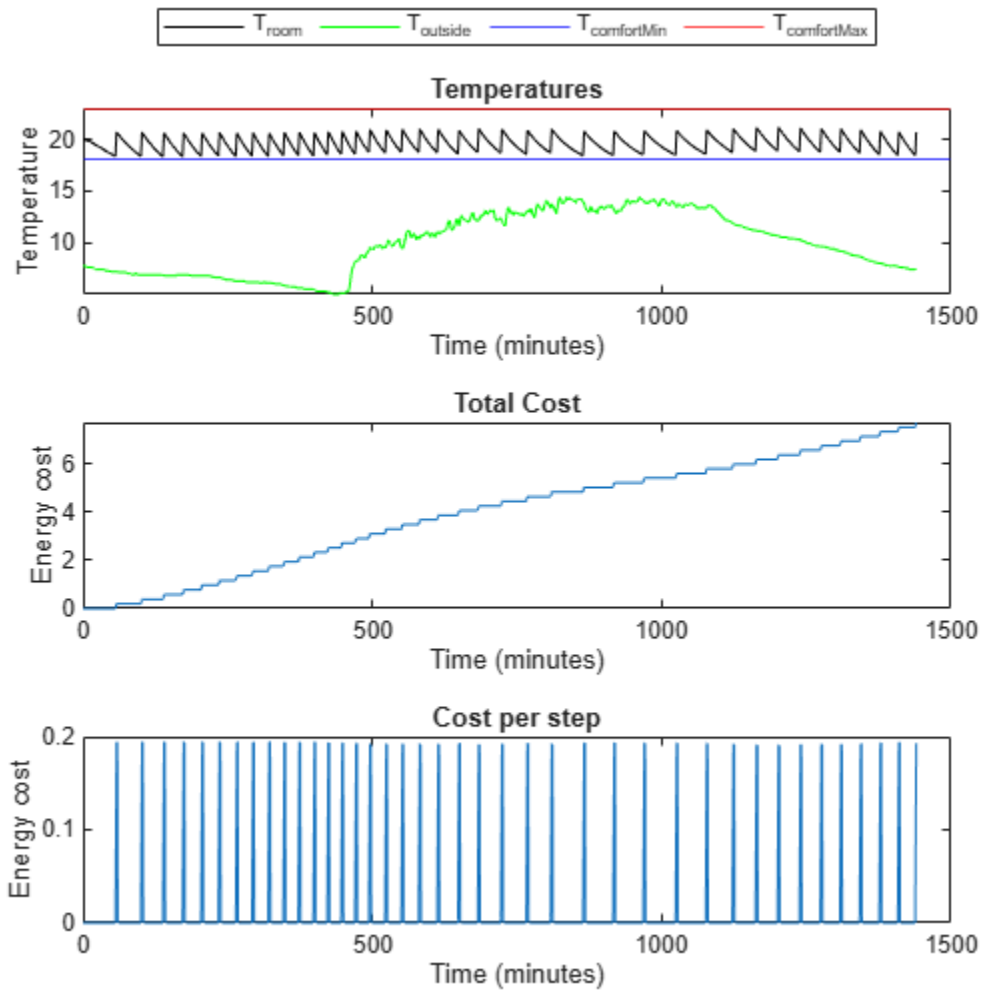
First, evaluate the agent's performance using the temperature data from March 21st, 2022. The agent did not use this temperature data for training.

```
maxSteps= 720;
validationTemperature = temperatureMarch21;
env.ResetFcn = @(in) hRLHeatingSystemValidateResetFcn(in);
simOptions = rlSimulationOptions(MaxSteps = maxSteps);
experience1 = sim(env,agent,simOptions);
```



Use the `localPlotResults` function, provided at the end of the example, to analyze the performance.

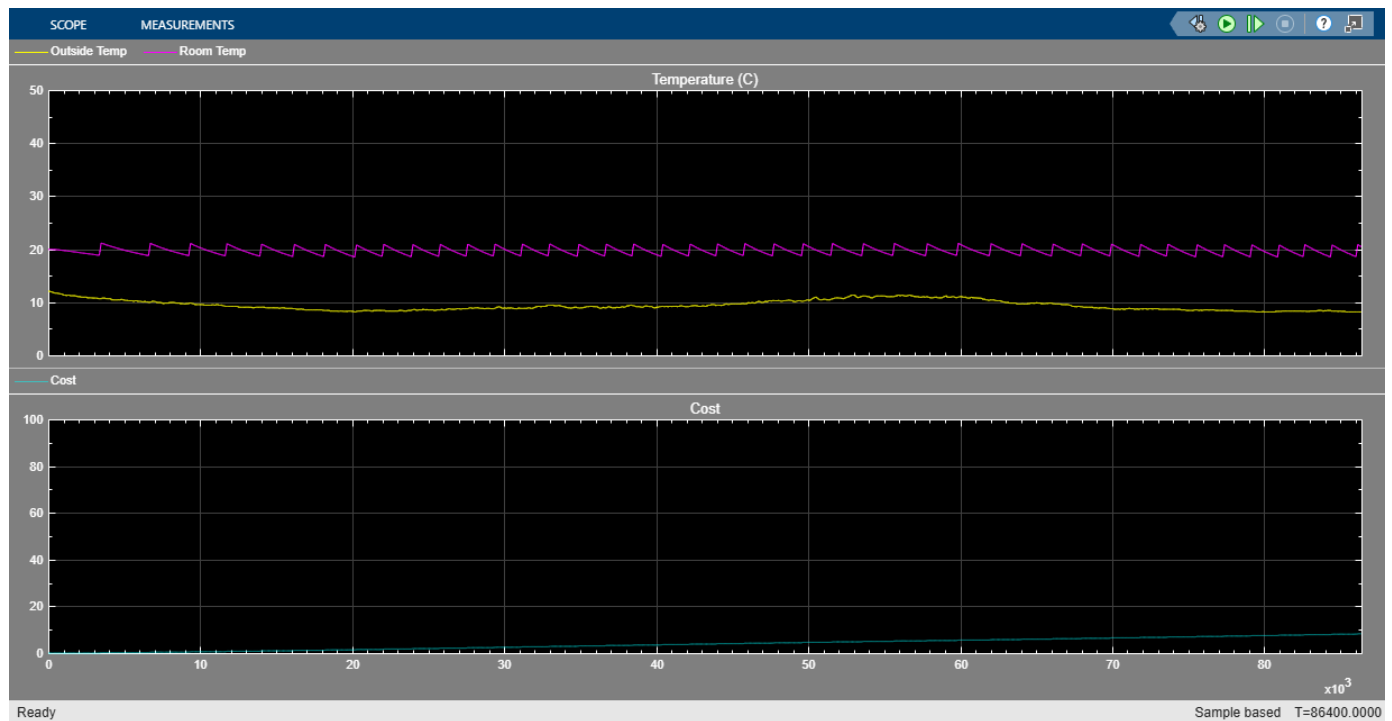
```
localPlotResults(experience1, maxSteps, ...
    comfortMax, comfortMin, sampleTime,1)
```



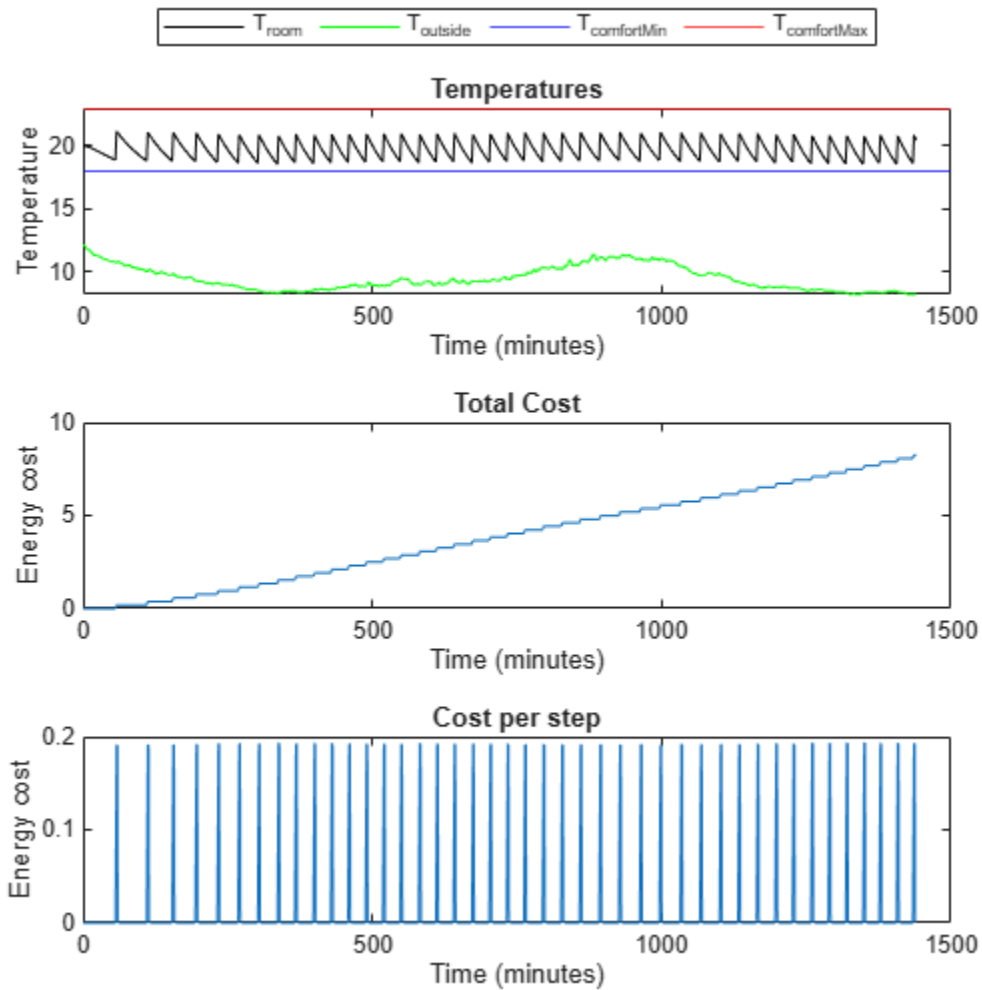
Comfort Temperature violation: 0/1440 minutes, cost: 7.717194 dollars

Next, evaluate the agent's performance using the temperature data from April 15th, 2022. The agent did not use this temperature data during training either.

```
% Validate agent using the data from April 15
validationTemperature = temperatureApril15;
experience2 = sim(env,agent,simOptions);
```



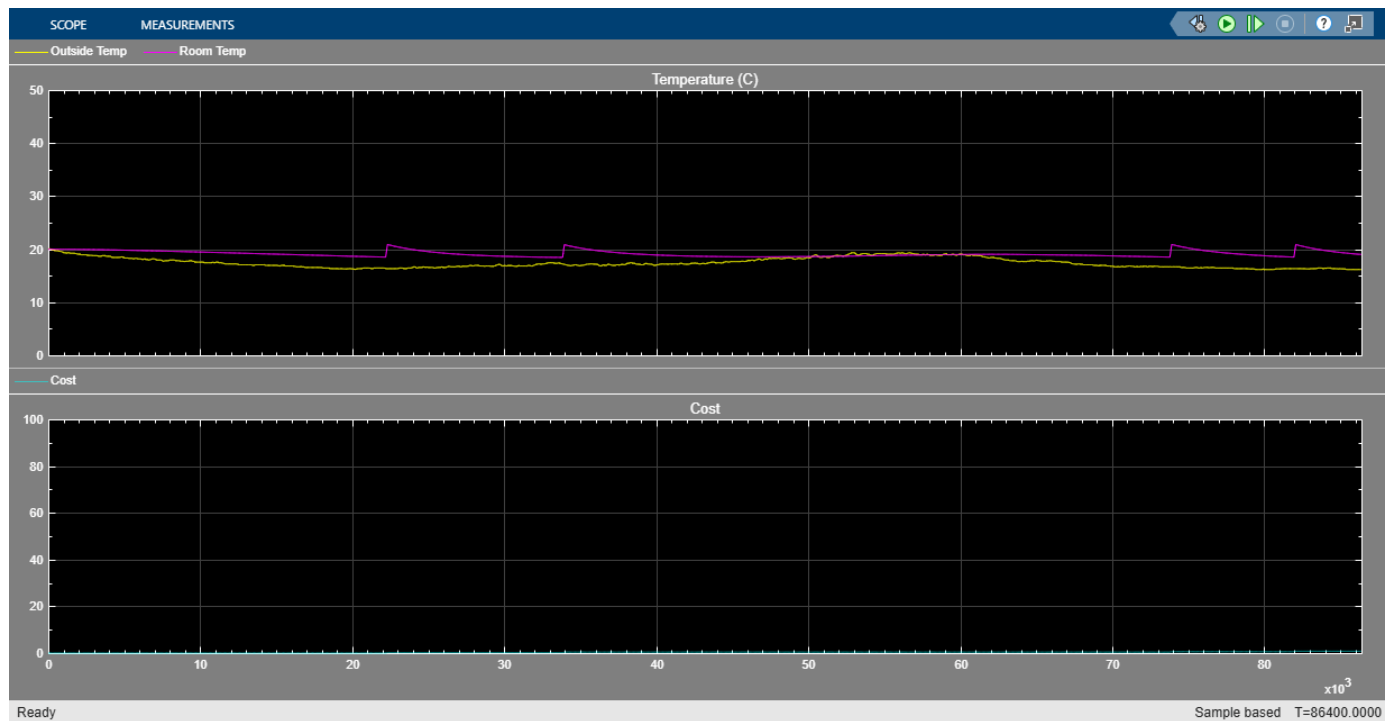
```
localPlotResults( ...  
    experience2, ...  
    maxSteps, ...  
    comfortMax, ...  
    comfortMin, ...  
    sampleTime,2)
```



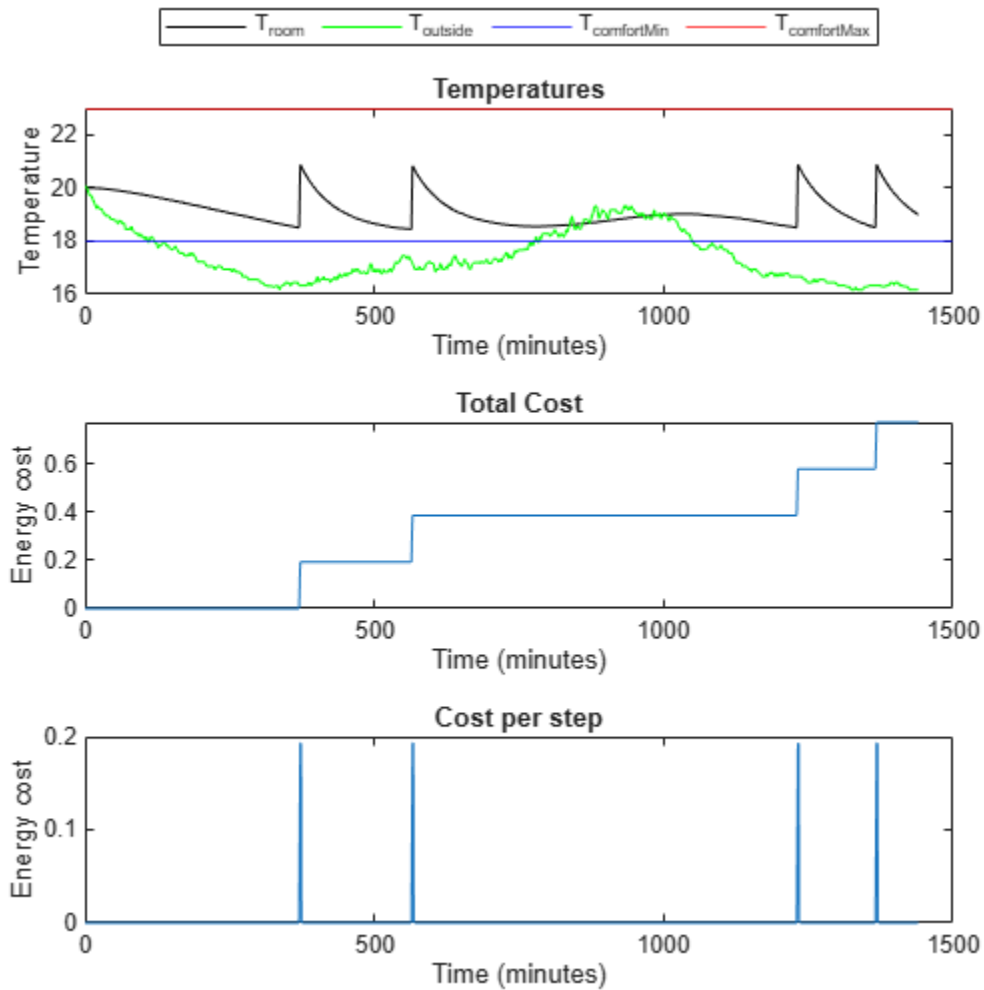
Comfort Temperature violation: 0/1440 minutes, cost: 8.259467 dollars

Evaluate the agent's performance when the temperature is mild. Add eight degrees to the temperature from April 15th to create data for mild temperatures.

```
% Validate agent using the data from April 15 + 8 degrees
validationTemperature = temperatureApril15;
validationTemperature(:,2) = validationTemperature(:,2) + 8;
experience3 = sim(env,agent,simOptions);
```

```
localPlotResults(experience3, ...  
    maxSteps, ...  
    comfortMax, ...  
    comfortMin, ...  
    sampleTime, ...  
    3)
```



Comfort Temperature violation: 0/1440 minutes, cost: 0.771909 dollars

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState)
```

Local Function

```
function localPlotResults( ...
    experience, maxSteps, comfortMax, comfortMin, sampleTime, figNum)
% localPlotResults plots results of validation

% Compute comfort temperature violation
obsData = experience.Observation.obs1.Data(1,:,1:maxSteps);
minutesViolateComfort = ...
    sum(obsData < comfortMin) ...
    + sum(obsData > comfortMax);

% Cost of energy
totalCosts = ...
```

```

        experience.SimulationInfo(1).househeat_output{1}.Values;
totalCosts.Time = totalCosts.Time/60;
totalCosts.TimeInfo.Units="minutes";
totalCosts.Name = "Total Energy Cost";
finalCost = totalCosts.Data(end);

% Cost of energy per step
costPerStep = ...
    experience.SimulationInfo(1).househeat_output{2}.Values;
costPerStep.Time = costPerStep.Time/60;
costPerStep.TimeInfo.Units="minutes";
costPerStep.Name = "Energy Cost per Step";
minutes = (0:maxSteps)*sampleTime/60;

% Plot results

fig = figure(figNum);
% Change the size of the figure
fig.Position = fig.Position + [0, 0, 0, 200];

% Temperatures
layoutResult = tiledlayout(3,1);
nexttile
plot(minutes, ...
    reshape(experience.Observation.obs1.Data(1,:,:), ...
    [1,length(experience.Observation.obs1.Data)]), "k")
hold on
plot(minutes, ...
    reshape(experience.Observation.obs1.Data(2,:,:), ...
    [1,length(experience.Observation.obs1.Data)]), "g")
yline(comfortMin, 'b')
yline(comfortMax, 'r')
lgd = legend("T_{room}", "T_{outside}", "T_{comfortMin}", ...
    "T_{comfortMax}", "location", "northoutside");
lgd.NumColumns = 4;
title("Temperatures")
ylabel("Temperature")
xlabel("Time (minutes)")
hold off

% Total cost
nexttile
plot(totalCosts)
title("Total Cost")
ylabel("Energy cost")

% Cost per step
nexttile
plot(costPerStep)
title("Cost per step")
ylabel("Energy cost")
fprintf("Comfort Temperature violation:" + ...
    " %d/1440 minutes, cost: %f dollars\n", ...
    minutesViolateComfort, finalCost);
end

```

Reference

[1] Du, Yan, Fangxing Li, Kuldeep Kurte, Jeffrey Munk, and Helia Zandi. “Demonstration of Intelligent HVAC Load Management With Deep Reinforcement Learning: Real-World Experience of Machine Learning in Demand Control.” *IEEE Power and Energy Magazine* 20, no. 3 (May 2022): 42-53. <https://doi.org/10.1109/MPE.2022.3150825>.

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlTrainingOptions`

Related Examples

- “House Heating System” (Simscape)
- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64
- “Water Distribution System Scheduling Using Reinforcement Learning” on page 8-361
- “Compare Temperature Data from Three Different Days” (ThingSpeak)

More About

- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3
- “Long Short-Term Memory Neural Networks”

Train Reinforcement Learning Agent for Simple Contextual Bandit Problem

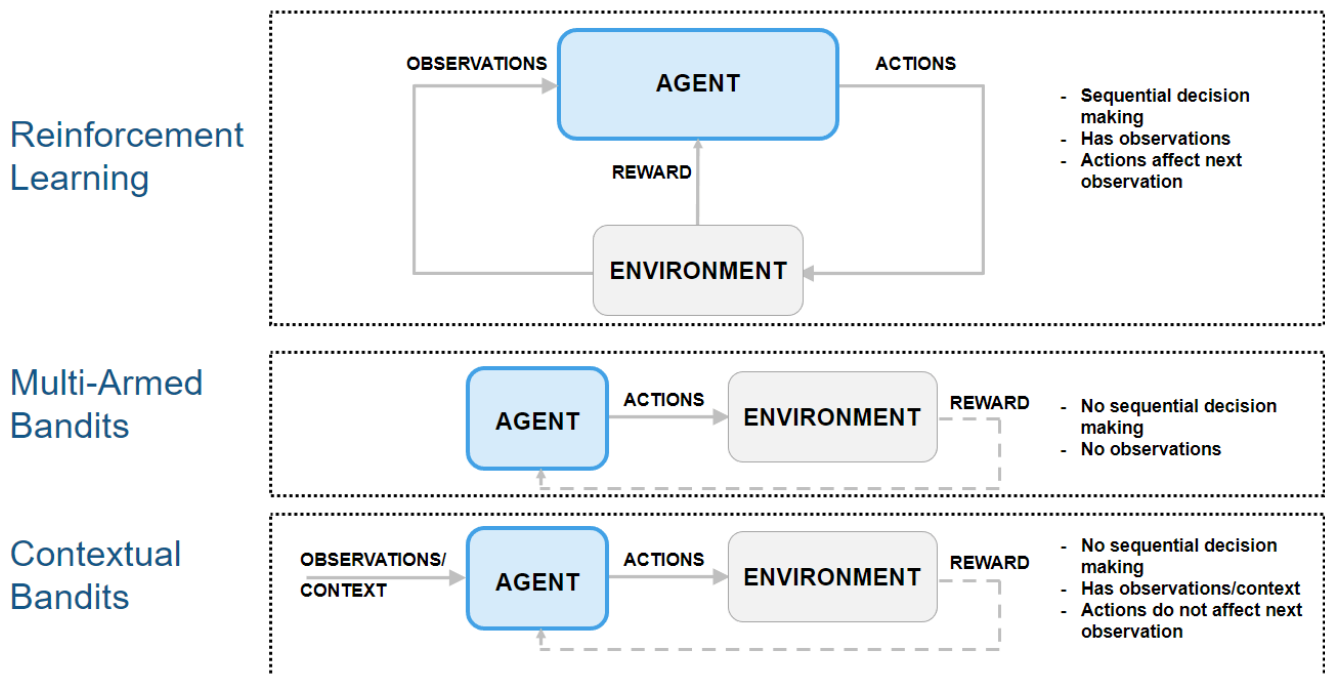
This example shows how to solve a contextual bandit problem [1 on page 8-325] using reinforcement learning by training DQN and Q agents. For more information on these agents, see “Deep Q-Network (DQN) Agent” on page 3-32 and “Q-Learning Agent” on page 3-22.

In bandit problems, the environment has no dynamics (that is, the state is constant and there are no state transitions), so the reward is only influenced by the current action and (for contextual bandits) the (constant) observation. In these problems the observation is also referred to as context. An alternative way to express the previous statements is that in contextual bandit problems an agent selects an action given the initial observation (context), it receives a reward, and the episode terminates.

Because neither rewards nor observations are influenced by previous actions or observations, the environment does not evolve along the time dimension, and there is no sequential decision making. The problem then becomes one of finding the action that maximizes the resulting immediate reward (given a context, if present). Single-armed bandit problems are just special cases of multi-armed bandit problems in which the action is a scalar instead of a vector.

The following figure shows how multi-armed bandits and contextual bandits are special cases of reinforcement learning problems.

Reinforcement Learning vs Multi-Armed Bandits vs Contextual Bandits



Contextual bandits can be used for several applications such as hyperparameter tuning, recommender systems, medical treatment, and 5G communication.

Supervised learning problems can be also be recast as contextual bandit problems. For example, a classification problem can be recast as a contextual bandit problem in which the observation x is an element that needs to be classified as belonging to a specific class, the action a is the agent's guess of the class to which x belongs, and the corresponding reward indicates whether the agent's guess is correct or not.

Similarly, a regression problem in which a function $y = f(x)$ needs to be approximated by a function $a = \hat{f}(x, \theta)$ of the parameter vector θ , can be recast as contextual bandit problem in which the observation x is an element of the feasible domain of $f(x)$, a is the action (the agent's guess of the value y), and the corresponding reward indicates how close a is to y , (for example, the reward could be $-(y - a)^2$).

Since the reward (which can only be a scalar) intrinsically contains less information than the true class of x (for the classification case) or than $y = f(x)$ (for the regression case), you can generally expect training time to be considerably longer for the reinforcement learning case than for the corresponding supervised learning case. For examples discussing classification and regression as contextual bandit problems, see “Why Solving Classification Using Reinforcement Learning Is Not Recommended” on page 8-337 and “Why Solving Regression Using Reinforcement Learning is Not Recommended” on page 8-327, respectively.

For more information on the relationship between reinforcement learning and supervised learning, see [2 on page 8-325]. For a comprehensive introduction to bandit problems, see [3 on page 8-325].

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Environment

The contextual bandit environment in this example is defined as follows:

- **Observation** (discrete): {1, 2}

The context (initial observation) is sampled randomly.

$$\Pr(s = 1) = 0.5$$

$$\Pr(s = 2) = 0.5$$

- **Action** (discrete): {1, 2, 3}
- **Reward:**

Rewards in this environment are stochastic. The probability of each observation and action pair is defined as follows.

```

1.  $s = 1, a = 1$ 
 $\Pr(r = 5 \mid s = 1, a = 1) = 0.3$ 
 $\Pr(r = 2 \mid s = 1, a = 1) = 0.7$ 

2.  $s = 1, a = 2$ 
 $\Pr(r = 10 \mid s = 1, a = 2) = 0.1$ 
 $\Pr(r = 1 \mid s = 1, a = 2) = 0.9$ 

3.  $s = 1, a = 3$ 
 $\Pr(r = 3.5 \mid s = 1, a = 3) = 1$ 

4.  $s = 2, a = 1$ 
 $\Pr(r = 10 \mid s = 2, a = 1) = 0.2$ 
 $\Pr(r = 2 \mid s = 2, a = 1) = 0.8$ 

5.  $s = 2, a = 2$ 
 $\Pr(r = 3 \mid s = 2, a = 2) = 1$ 

6.  $s = 2, a = 3$ 
 $\Pr(r = 5 \mid s = 2, a = 3) = 0.5$ 
 $\Pr(r = 0.5 \mid s = 2, a = 3) = 0.5$ 

```

Note that the agent does not know these distributions.

- **Is-Done signal:** Because this is a contextual bandit problem, each episode has only one step. Hence, the Is-Done signal is always 1.

Create Environment Object

The contextual bandit environment is implemented in the file `ToyContextualBanditEnvironment`, located in this example folder. For more information on how to implement a custom environment using the class template, see “Create Custom Environment from Class Template” on page 2-103.

Display the environment class. Note how the rewards are calculated in the environment `step` function, and how the observation (context) remains constant at its initial value.

```

type("ToyContextualBanditEnvironment.m")

classdef ToyContextualBanditEnvironment < rl.env.MATLABEnvironment

    %% Properties (set properties' attributes accordingly)
    properties
        % Initialize state
        State = zeros(1,1)
    end

    properties(Access = protected)
        % Initialize internal flag to indicate episode termination.
        IsDone = false
    end

    %% Necessary Methods
    methods
        % Constructor method creates an instance of the environment.
    end
end

```

```
% Change class name and constructor name accordingly.
function this = ToyContextualBanditEnvironment()
    % Initialize Observation settings
    % Observation = {s1, s2}, discrete
    obsInfo = rlFiniteSetSpec([1 2]);

    % Initialize Action settings
    % Action = {a1, a2, a3}, discrete
    actInfo = rlFiniteSetSpec([1 2 3]);

    % Implement built-in functions of RL env
    this = this@rl.env.MATLABEnvironment(obsInfo,actInfo);
end

% Apply system dynamics and simulate the environment
% with the given action for one step.
function [Observation,Reward,IsDone,aux] = step(this,Action)
    aux = [];

    % The action doesn't affect the next state
    % in a contextual bandit problem.
    Observation = this.State;

    % Get reward
    if this.State == 1
        if Action == 1
            % E(reward) = 2.9
            if rand < 0.3
                Reward = 5;
            else
                Reward = 2;
            end
        elseif Action == 2
            % E(reward) = 1.9
            if rand < 0.1
                Reward = 10;
            else
                Reward = 1;
            end
        elseif Action == 3
            % E(reward) = 3.5
            Reward = 3.5;
        end
    elseif this.State == 2
        if Action == 1
            % E(reward) = 3.6
            if rand < 0.2
                Reward = 10;
            else
                Reward = 2;
            end
        elseif Action == 2
            % E(reward) = 3.0
            Reward = 3.0;
        elseif Action == 3
            % E(reward) = 3
            if rand < 0.5
                Reward = 5;
            end
        end
    end
end
```



```

        else
            Reward = 0.5;
        end
    end
end

% Get IsDone.
IsDone = true;
this.IsDone = IsDone;
% (Optional) Use notifyEnvUpdated to signal
% that the environment has been updated
% (e.g. to update visualization).
notifyEnvUpdated(this);
end

% Reset environment to initial state
% and output initial observation.
function InitialObservation = reset(this)
    % Pr(s1) = 0.5, Pr(s2) = 0.5
    InitialObservation = randi(2);
    this.State = InitialObservation;

    % (Optional) Use notifyEnvUpdated to signal
    % that the environment has been updated
    % (e.g. to update visualization).
    notifyEnvUpdated(this);
end
end
%% Optional Methods (set methods' attributes accordingly)
methods
    % (Optional) Visualization method
    function plot(this)
        % Update the visualization
        envUpdatedCallback(this)
    end

    % (Optional) Properties validation through set methods
    function set.State(this,state)
        mustBeMember(state,[1,2])
        this.State = state;
        notifyEnvUpdated(this);
    end
end
end
end

```

Create the environment object.

```
env = ToyContextualBanditEnvironment;
```

Get observation and action specification objects.

```
obsInfo = getObservationInfo(env);
actInfo = getActionInfo(env);
```

Create a DQN Agent

Create a DQN agent option object. For more information, see `rLDQNAgentOptions`.

```
agentOpts = rLDQNAgentOptions( ...
    UseDoubleDQN = false, ...
    TargetSmoothFactor = 1, ...
    TargetUpdateFrequency = 4, ...
    MiniBatchSize = 64, ...
    MaxMiniBatchPerEpoch = 2);
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 0.0005;
```

To create an agent with default network structure, in which each hidden layer has 16 neurons, use `rlAgentInitializationOptions`.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit = 16);
```

Create a DQN agent. For more information, see `rLDQNAgent`.

```
dqnAgent = rLDQNAgent(obsInfo, actInfo, initOpts, agentOpts);
```

Train the DQN Agent

To train the agent, first specify the training options. For this example, use the following options:

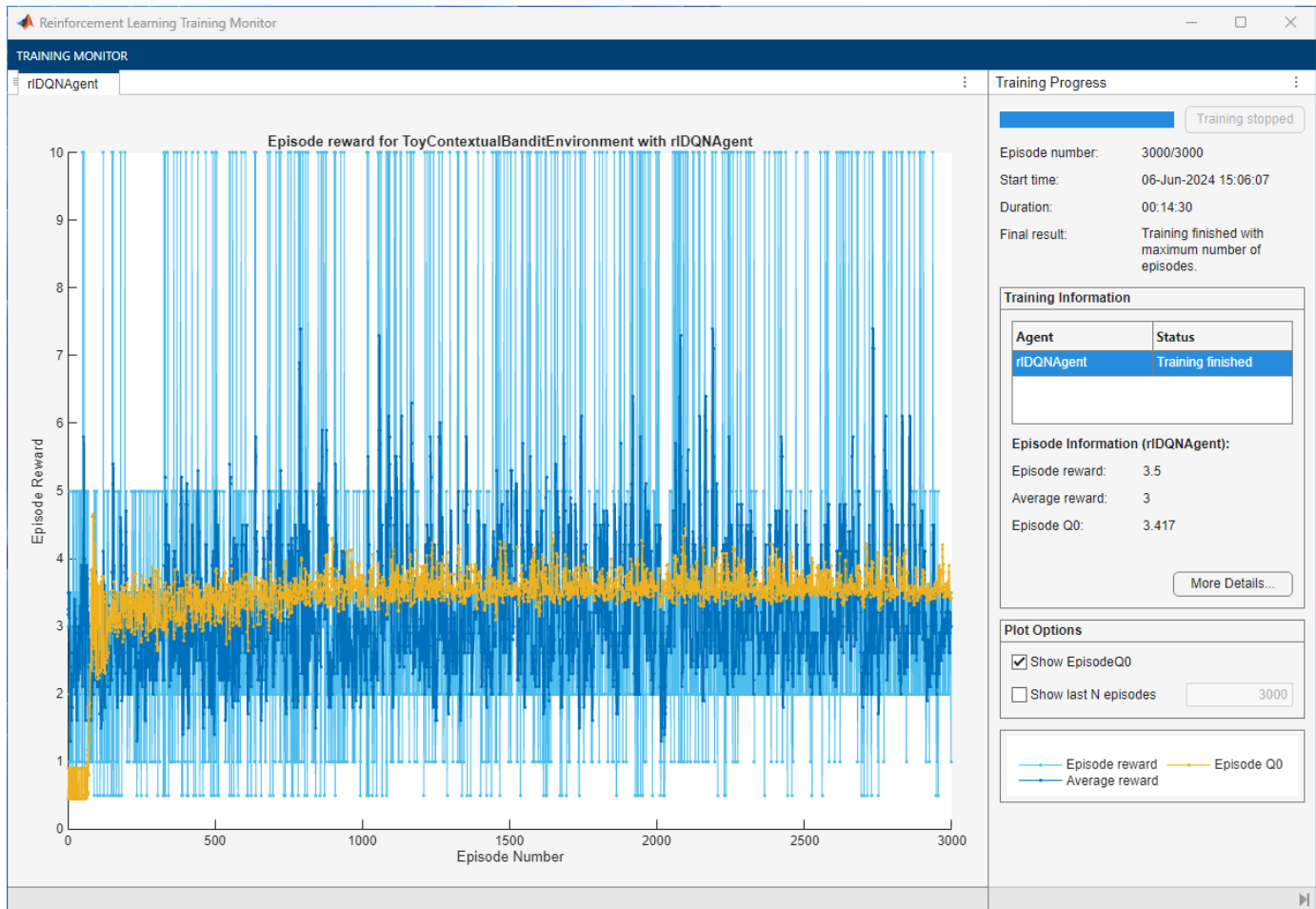
- Train for 3000 episodes.
- Because this is a contextual bandit problem, and each episode has only one step, set `MaxStepsPerEpisode` to 1.

For more information, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=3000, ...
    MaxStepsPerEpisode=1, ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="None", ...
    StopTrainingValue="None");
```

Train the agent using the `train` function. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % Train the agent
    trainingStats = train(dqnAgent,env,trainOpts);
else
    % Load the pretrained agent for the example
    load("ToyContextualBanditDQNAgent.mat","dqnAgent")
end
```



Validate the DQN Agent

For this example, you know the distribution of the rewards, and you can compute the optimal actions. Validate the agent's performance by comparing these optimal actions with the actions selected by the agent.

First, compute the true expected rewards using their true probability distribution.

1. The expected reward of each action at $s=1$ is as follows.

If $a = 1$

$$E[R] = 0.3 * 5 + 0.7 * 2 = 2.9$$

If $a = 2$

$$E[R] = 0.1 * 10 + 0.9 * 1 = 1.9$$

If $a = 3$

$$E[R] = 3.5$$

Hence, the optimal action is 3 when $s=1$.

2. The expected reward of each action at $s=2$ is as follows.

If $a = 1$

$$E[R] = 0.2 * 10 + 0.8 * 2 = 3.6$$

If $a = 2$

$$E[R] = 3.0$$

If $a = 3$

$$E[R] = 0.5 * 5 + 0.5 * 0.5 = 2.75$$

Hence, the optimal action is 1 when $s=2$.

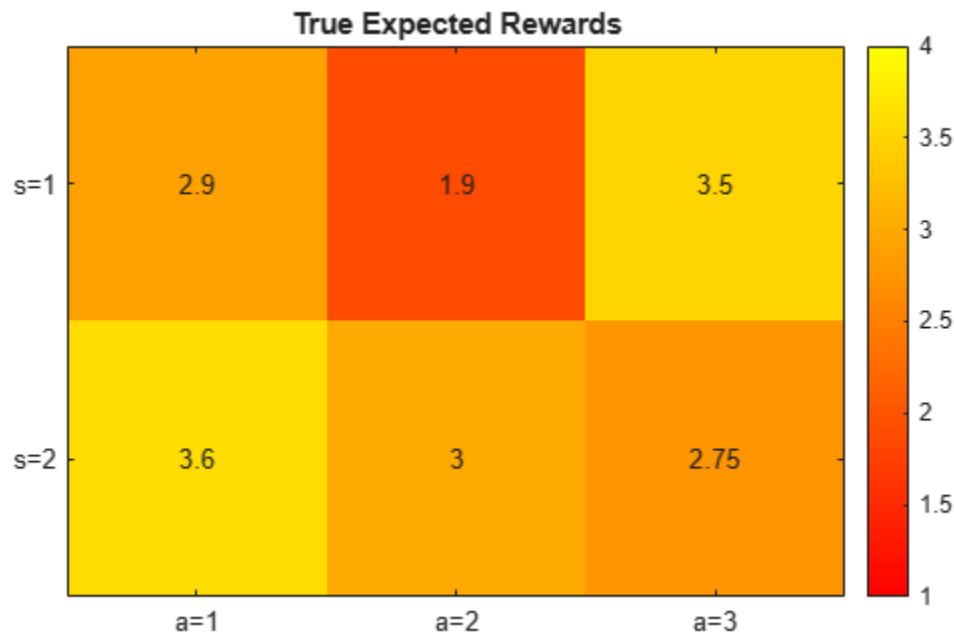
With enough sampling, the Q-value estimates of the trained agent should be closer to the true expected reward.

Collect the true expected rewards in the ExpectedRewards variable.

```
ExpectedRewards = zeros(2,3);
ExpectedRewards(1,1) = 0.3*5 + 0.7*2;
ExpectedRewards(1,2) = 0.1*10 + 0.9*1;
ExpectedRewards(1,3) = 3.5;
ExpectedRewards(2,1) = 0.2*10 + 0.8*2;
ExpectedRewards(2,2) = 3.0;
ExpectedRewards(2,3) = 0.5*5 + 0.5*0.5;
```

Visualize the true expected rewards using the function `localPlotQvalues` on page 8-324 defined at the end of the example.

```
localPlotQvalues(ExpectedRewards, "True Expected Rewards")
```



Now, validate whether the DQN agent learns the optimal behavior. Use `getActionInfo` to return the agent action given an input observation.

If the state is 1, the optimal action is 3.

```

observation = 1;
getAction(dqnAgent, observation)

ans = 1x1 cell array
      {[3]}

```

The agent selects the optimal action.

If the state is 2, the optimal action is 1.

```

observation = 2;
getAction(dqnAgent, observation)

ans = 1x1 cell array
      {[1]}

```

The agent selects the optimal action. Thus, the DQN agent has learned the optimal behavior.

Next, compare the Q-Value function to the true expected reward when selecting the optimal action.

Use `getCritic` to extract the critic from the trained agent, and `getValue` to return the value of an observation (using the learned value function).

```

dqnCritic = getCritic(dqnAgent);
qValues = zeros(2,3);
for s = 1:2
    qValues(s,:) = getValue(dqnCritic, {s});
end

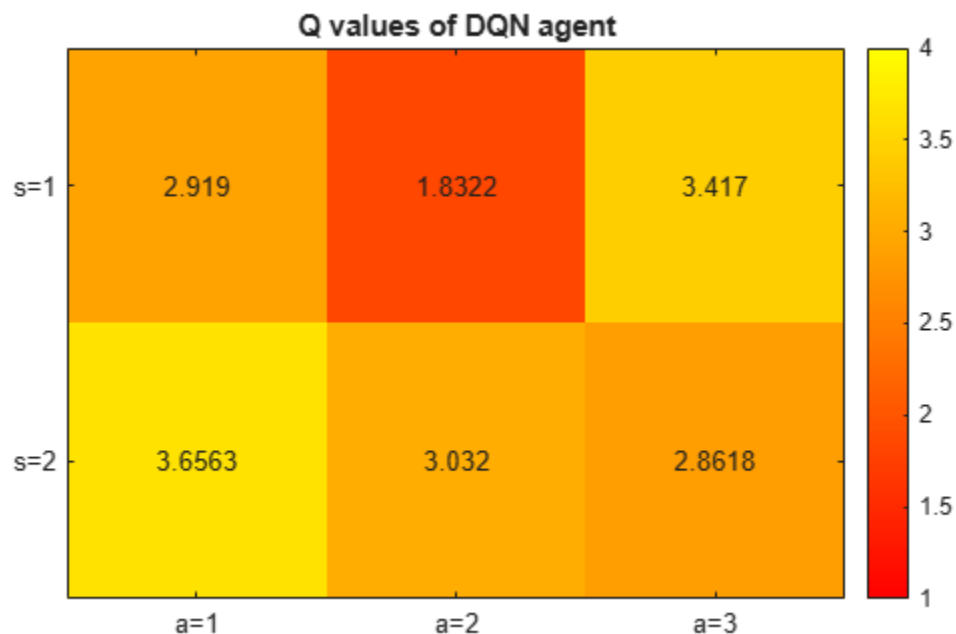
```

Visualize the Q values for the DQN agent.

```

figure(1)
localPlotQvalues(qValues, "Q values of DQN agent")

```



The learned Q-values are close to the true expected rewards computed above.

Create a Q-Learning Agent

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

For this example, use a table as approximation model for the critic.

Create a table using the observation and action specifications from the environment.

```
qTable = rlTable(obsInfo, actInfo);
```

Create an `rlQValueFunction` critic.

```
critic = rlQValueFunction(qTable, obsInfo, actInfo);
```

To set the agent exploration options, create an `rlQAgentOptions` object.

```
opt = rlQAgentOptions;  
opt.EpsilonGreedyExploration.Epsilon = 1;  
opt.EpsilonGreedyExploration.EpsilonDecay = 0.0005;
```

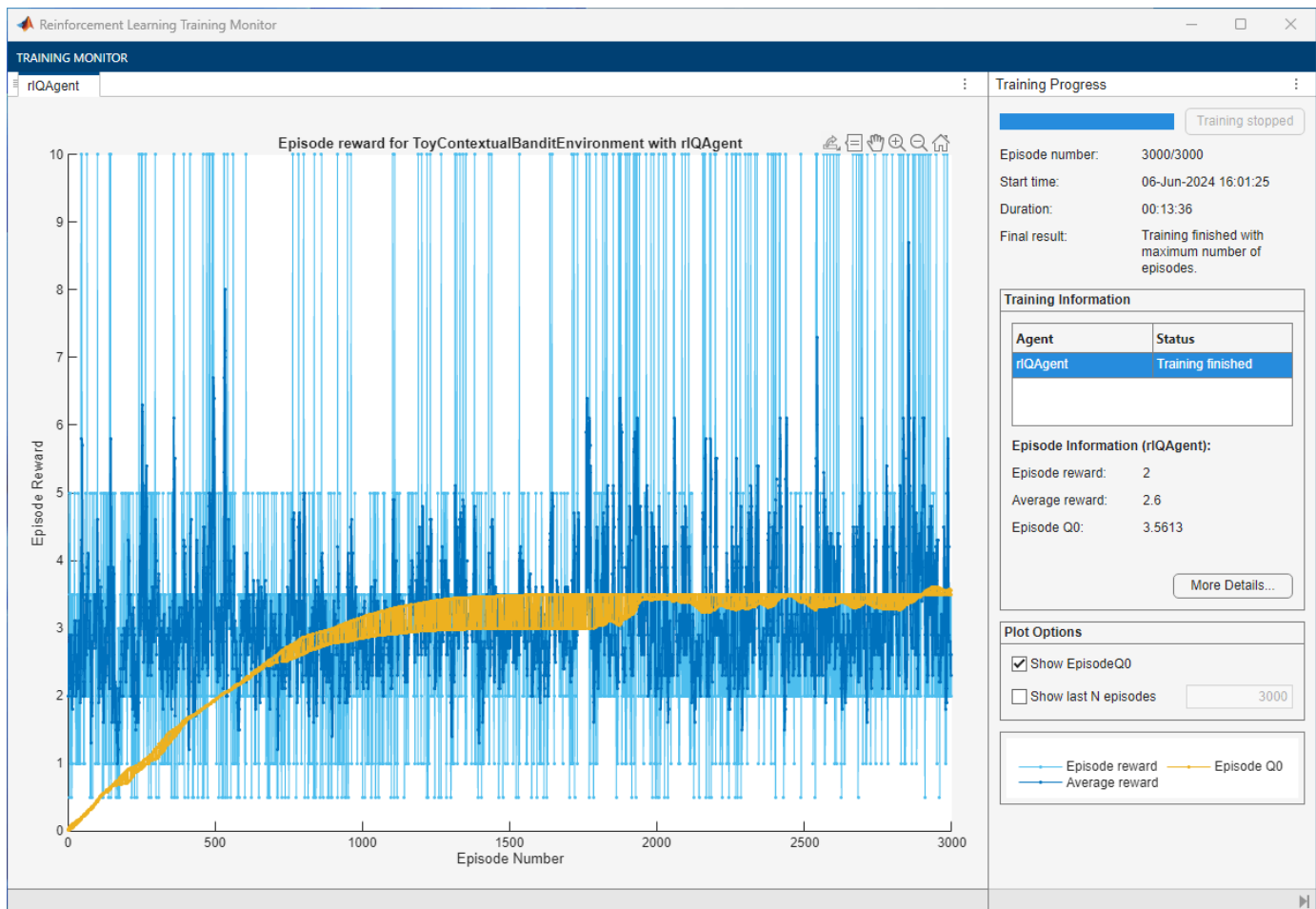
Create a Q agent. For more information, see `rlQAgent`.

```
qAgent = rlQAgent(critic,opt);
```

Train the Q-Learning Agent

To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainingStats = train(qAgent,env,trainOpts);  
else  
    % Load the pretrained agent for the example.  
    load("ToyContextualBanditQAgent.mat","qAgent")  
end
```



Validate the Q-Learning Agent

When the state is 1, the optimal action is 3.

```
observation = 1;
getAction(qAgent, observation)
```

```
ans = 1x1 cell array
      {3}
```

The agent selects the optimal action.

When the state is 2, the optimal action is 1.

```
observation = 2;
getAction(qAgent, observation)
```

```
ans = 1x1 cell array
      {1}
```

The agent selects the optimal action.

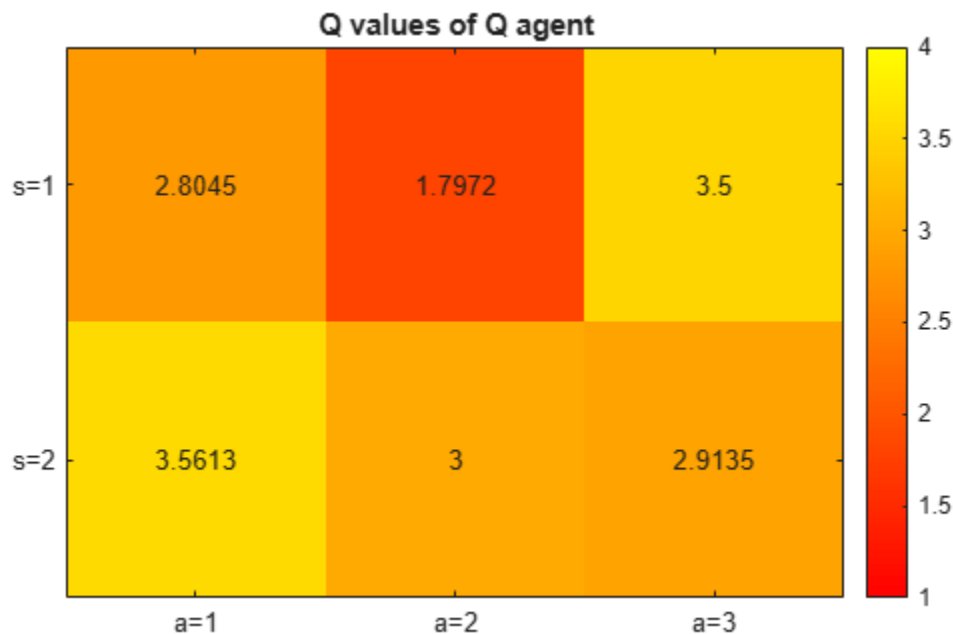
Next, compare the Q-Value function to the true expected reward when selecting the optimal action.

Use `getCritic` to extract the critic from the trained agent, and `getValue` to return the value of an observation (using the learned value function).

```
figure(2)
qCritic = getCritic(qAgent);
qValues = zeros(2,3);
for s = 1:2
    for a = 1:3
        qValues(s,a) = getValue(qCritic, {s}, {a});
    end
end
```

Visualize the Q values for the DQN agent.

```
localPlotQvalues(qValues, "Q values of Q agent")
```



The learned Q-values are close to the true expected rewards. The Q-values for deterministic rewards, $Q(s=1, a=3)$ and $Q(s=2, a=2)$, are the same as the true expected rewards.

Note that the corresponding Q-values learned by the DQN network, while close, are not identical to the true values. This happens because the DQN uses a neural network instead of a table as function approximation model.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Function

```
function localPlotQvalues(QValues, titleText)
    % Visualize Q values
    figure;
    imagesc(QValues,[1,4])
```



```

colormap("autumn")
title(titleText)
colorbar
set(gca,"Xtick",1:3,"XTickLabel",{ "a=1", "a=2", "a=3" })
set(gca,"Ytick",1:2,"YTickLabel",{ "s=1", "s=2" })

% Plot values on the image
x = repmat(1:size(QValues,2), size(QValues,1), 1);
y = repmat(1:size(QValues,1), size(QValues,2), 1)';
QValuesStr = num2cell(QValues);
QValuesStr = cellfun(@num2str, QValuesStr, UniformOutput=false);
text(x(:), y(:), QValuesStr, HorizontalAlignment = "Center")
end

```

Reference

[1] Sutton, Richard S., and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second edition. Adaptive Computation and Machine Learning Series. Cambridge, Massachusetts: The MIT Press, 2018.

[2] Barto, Andrew G., and Thomas G. Dietterich. *Reinforcement Learning and its Relationship to Supervised Learning*. https://all.cs.umass.edu/pubs/2004/barto_d_04.pdf.

[3] Slivkins, Aleksandrs. "Introduction to Multi-Armed Bandits." arXiv, April 3, 2024. <https://doi.org/10.48550/arXiv.1904.07272>.

See Also

Apps

Reinforcement Learning Designer

Functions

getActionInfo | train | sim

Objects

rlQAgent | rlQAgentOptions | rlDQNAgent | rlDQNAgentOptions | rlTrainingOptions

Related Examples

- "Train Default DQN Agent to Balance Discrete Cart-Pole" on page 5-64
- "Why Solving Regression Using Reinforcement Learning is Not Recommended" on page 8-327
- "Why Solving Classification Using Reinforcement Learning Is Not Recommended" on page 8-337

More About

- "Deep Q-Network (DQN) Agent" on page 3-32
- "Use Predefined Grid World Environments" on page 2-9
- "Train Reinforcement Learning Agents" on page 5-3

- “Create Actors, Critics, and Policy Objects” on page 4-2

Why Solving Regression Using Reinforcement Learning is Not Recommended

This example shows how to solve a regression problem using both a supervised learning approach and a reinforcement learning approach, illustrating the differences between these methods. This example also illustrates why reinforcement learning is not the right tool to solve a supervised learning problem. For more information on the relationship between reinforcement and supervised learning, see [1 on page 8-336].

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0, "twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Function Approximation Problem

In this example, you solve a regression problem consisting in approximating the following nonlinear function:

$$f(x) = 2/30 * (x_1^2 + x_2 + x_3 + x_1 * x_4) - 1/30$$

Over the domain $x_1 = [0, 1]$, $x_2 = [-5, 0]$, $x_3 = [-6, 6]$ and $x_4 = [-2, 7]$.

To train a neural network in a supervised learning fashion you supply to the network both x and the correct $f(x)$ over the training dataset, and the algorithm adjusts the network weights so that the network output $n(x)$ approximates $f(x)$. You then use unseen inputs in a validation dataset to check how well the network is able to predict the correct output.

Supervised Learning Solution

Ensure reproducibility of the section by fixing the seed used for random number generation.

```
rng(0, "twister")
```

Create a training data set with 5000 data points in the function domain.

```
x = ([1 0 6 7] - [0 -5 -6 -2]).*rand(5e3,4) + [0 -5 -6 -2];
y = 2/30*(x(:,1).^2 + x(:,2) + x(:,3) + x(:,1).*x(:,4)) - 1/30;
```

Define a neural network to approximate the function. Create the network as a vector of layer objects.

```
layers = [
    featureInputLayer(4)
    fullyConnectedLayer(16)
```

```
reluLayer
fullyConnectedLayer(1)
];
```

Create a `dlnetwork` object and display the number of learnable parameters.

```
net = dlnetwork(layers);
summary(net)
```

```
Initialized: true

Number of learnables: 97

Inputs:
    1 'input'    4 features
```

Define the training options.

```
opts = trainingOptions("adam",...
    InitialLearnRate=0.005,...
    LearnRateSchedule="piecewise",...
    LearnRateDropPeriod=100,...
    Plots="none",...
    Verbose=true, ...
    VerboseFrequency=500, ...
    LearnRateDropFactor=0.97,...
    GradientThreshold=1,...
    MaxEpochs=150);
```

Train the network using the `trainnet` function.

```
net = trainnet(x,y,net,"mse",opts);
```

Iteration	Epoch	TimeElapsed	LearnRate	TrainingLoss
1	1	00:00:02	0.005	4.6273
500	13	00:00:08	0.005	0.0011827
1000	26	00:00:13	0.005	0.00060517
1500	39	00:00:17	0.005	0.00043892
2000	52	00:00:21	0.005	0.00039223
2500	65	00:00:24	0.005	0.00024674
3000	77	00:00:28	0.005	0.00017821
3500	90	00:00:31	0.005	0.000121
4000	103	00:00:34	0.00485	6.7063e-05
4500	116	00:00:38	0.00485	9.0661e-05
5000	129	00:00:41	0.00485	6.9279e-05
5500	142	00:00:44	0.00485	6.3177e-05
5850	150	00:00:47	0.00485	6.5419e-05

Training stopped: Max epochs completed

The training converges in less than 30 seconds.

Create a validation dataset with 50 points in the function domain.

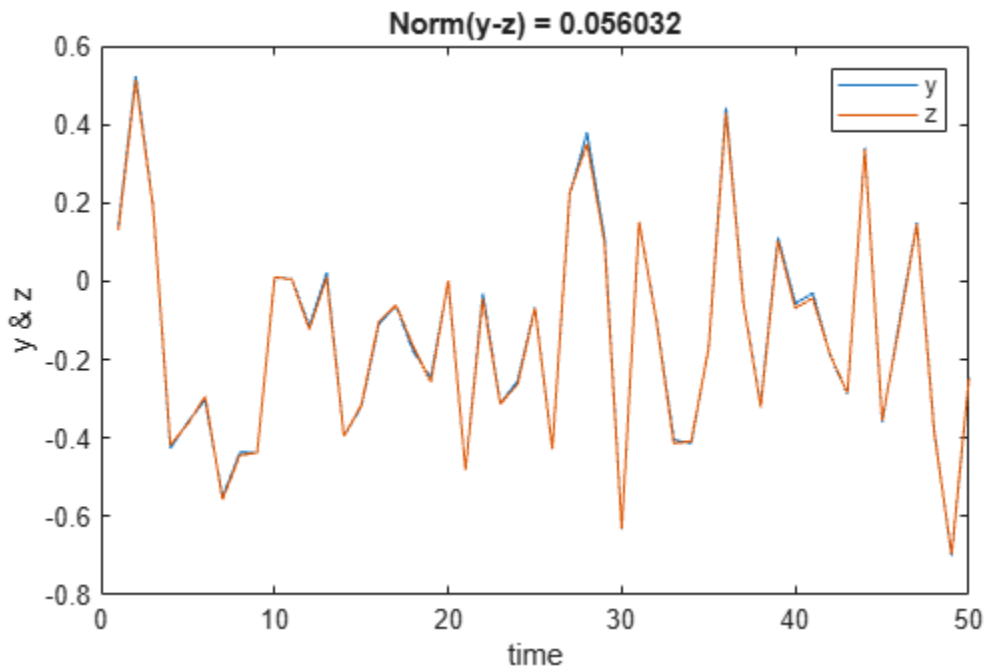
```
x = ([1 0 6 7]-[0 -5 -6 -2]).*rand(50,4) + [0 -5 -6 -2];
y = 2/30*(x(:,1).^2 + x(:,2) + x(:,3) + x(:,1).*x(:,4)) - 1/30;
```

Calculate the neural network predictions using the `predict` function.

```
z = predict(net,x);
```

Plot the true function value against the approximation, displaying the norm of the difference.

```
plot(1:50,[y z]); legend("y","z")
xlabel("time"); ylabel("y & z");
title(['Norm(y-z) = ' num2str(norm(y-z))]);
```



The trained network successfully approximates the function.

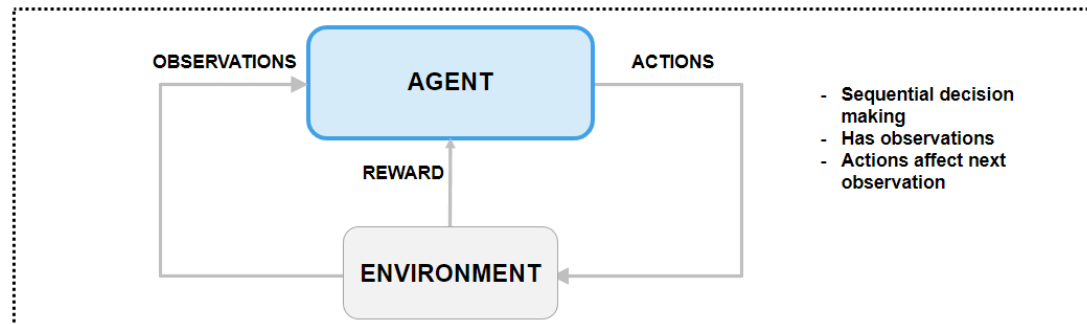
Reinforcement Learning Solution: Contextual Bandit Framework

You can express this function approximation problem as a reinforcement learning problem. During reinforcement learning training, your environment supplies to an agent both x (the observation) and a scalar reward that is higher the closer the agent output $a(x)$ (the action) approximates $f(x)$. Note that the reward only captures the information of how close $a(x)$ is to $f(x)$ but the full knowledge of $f(x)$ is not available to the agent. By contrast, this information is fully used in the supervised learning setting, (which is why training is generally much faster for supervised learning).

Because the next observation is independent of the action, this type of reinforcement learning problem can be classified as a "contextual bandit" problem. In contextual bandits problems, the environment has no state dynamics, so the reward is only influenced by the current observation and action.

The following figure shows how contextual bandit problems are special cases of reinforcement learning problems.

Reinforcement Learning



- Sequential decision making
- Has observations
- Actions affect next observation

Contextual Bandits



- No sequential decision making
- Has observations/context
- Actions do not affect next observation

For an example that shows how to use an agent to solve a contextual bandit problem, see “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313. For an example discussing classification as a contextual bandit problems, see “Why Solving Classification Using Reinforcement Learning Is Not Recommended” on page 8-337. For a comprehensive introduction to bandit problems, see [2 on page 8-336].

Create Custom Continuous-Action-Space Environment Object

Create the environment specifications. The observation (state) is the current value of x , and the action is the estimated value of the function, $a(x)$.

```

obsInfo = rlNumericSpec([4 1], ...
    LowerLimit=[0 -5 -6 -2]', ...
    UpperLimit=[1 0 6 7]');
actInfo = rlNumericSpec([1 1], ...
    LowerLimit=-1, ...
    UpperLimit=1);
  
```

At the beginning of each training or simulation episode, a reset function resets the environment to an initial state and returns both the initial state (needed for the next environment step) and the resulting initial observation value (for the agent to see).

Display the custom reset function, which is provided in the file `rgsnfResetFcn.m`.

```

type("rgsnResetFcn.m")

function [obs0,x0] = rgsnResetFcn()
% Reset function to set environment in random initial state.

% Return initial observation and state.
x0 = ([1 0 6 7]-[0 -5 -6 -2]).'*rand(4,1) + [0 -5 -6 -2]';
obs0 = x0;

end
  
```

This function takes no input argument, calculates a random initial point in the domain, and returns it both as initial observation and state.

A step function is called by `train` (or `sim`) at each step of the training (or simulation) episode to advance the environment to the next step.

Display the custom step function, which is provided in the file `rgsnStepFcn.m`.

```
type("rgsnStepFcn.m")

function [nextObs,r,isdn,xp] = rgsnStepFcn(a,x)
% Custom step function to advance custom environment by one step

% Initialize is-done.
isdn = 0;

% Make sure action is finite; otherwise, terminate.
if ~allfinite(a)
    isdn = 1;
    a(~isfinite(a)) = rand;
end

% Calculate true function value.
f_star = 2/30*(x(1)^2 + x(2) + x(3) + x(1)*x(4)) - 1/30;

% Calculate reward (reward for a).
r = 0.1-norm(a-f_star);

% Make sure reward value is finite; otherwise, terminate.
if ~allfinite(r)
    isdn = 1;
    r(~isfinite(r)) = -1e20;
end

% Calculate next observation and state.

% Calculate random action-independent feasible point.
p = ([1 0 6 7]-[0 -5 -6 -2]).*rand(4,1) + [0 -5 -6 -2]';
% Move current state toward p, and set next obs = next state.
xp = x + 0.2*(p-x);
nextObs = xp;

end
```

This function takes the action from the agent and the current environment state as input arguments. It then uses the current state to calculate the true function value, `f_star`, which is in turn used to calculate the reward that the agent receives for the action `a`. The reward decreases proportionally to the Euclidean distance between the action and the true function value. If the distance is less than 0.1, the reward is positive.

The step function then calculates the next state by moving the current state toward a random point in the domain, setting the next observation to the next state.

If the received action or calculated reward is non-finite, the is-done flag is set to one, which causes the episode to terminate.

Create the custom function environment object, passing the observation and action specifications and the handles of the custom step and reset functions to the `rlFunctionEnv` function.

```
env = rlFunctionEnv(obsInfo,actInfo, @rgsnStepFcn, @rgsnResetFcn);
```

For a more detailed example on custom step and reset functions, see “Create Custom Environment Using Step and Reset Functions” on page 2-94.

Create DDPG Agent and Specify Agent Options

The actor and critic networks are initialized randomly. Specify random seed and algorithm for reproducibility.

```
rng(0, "twister")
```

Create a default `rlDDPGAgent` object using the environment specification objects.

```
ddpgAgent = rlDDPGAgent(obsInfo, actInfo);
```

Set a lower learning rate and gradient thresholds to avoid instability.

```
ddpgAgent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
ddpgAgent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;  
ddpgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Because future reward is independent on the action, set a discount factor of zero.

```
ddpgAgent.AgentOptions.DiscountFactor = 0;
```

Configure Training Options

Create an evaluator object to evaluate the agent five times without exploration every 100 episodes.

```
evl = rlEvaluator(NumEpisodes=5, EvaluationFrequency=100);
```

Create a training options object. For more information, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions;
```

Specify training options. For this example:

- Set the maximum number of steps per episode to 500 (note that the environment has no terminal state).
- Stop the training when the average reward collected over the evaluation episodes reaches 48 (in this example, the maximum reward over an episode is $0.1 \times 500 = 50$).
- In any case, stop after 1000 episodes.

```
trainOpts.MaxEpisodes = 1000;  
trainOpts.MaxStepsPerEpisode = 500;  
trainOpts.StopTrainingCriteria = "EvaluationStatistic";  
trainOpts.StopTrainingValue = 48;
```

Alternatively, because the next state is independent of the action, you can set the maximum number of steps per episode to 1 (and consequently allow for a larger number of episodes). Adopting this view, each episode is a stand-alone event in which the agent associates a single observation to a single action, that is, each episode represents an instance of a bandit problem with a different context (a different input value). You must use this approach if your environment does not allow multiple steps per episode.

However, if you use this alternative approach to train agents like PG agents, which calculate network gradients at the end of the episode and use only the steps available in the episode, then the network

gradients are calculated using only a single observation and reward. This can lead to inefficient or unstable learning.

For an example using this alternative approach, see “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313.

Train the DDPG Agent

To reproduce the results of this section, set random seed and algorithm.

```
rng(0, "twister")
```

To train the agent, pass the agent, the environment, the training options, and the evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    pgTrainingRes = train(ddpgAgent, env, trainOpts, Evaluator=evl);
    % Uncomment to save the trained agent and results.
    % save("rgsnDDPGAgent.mat", "ddpgAgent", "pgTrainingRes")
else
    % Load the pretrained agent and results for the example.
    load("rgsnDDPGAgent.mat", "ddpgAgent", "pgTrainingRes")
end
```



The trained agent achieves a reward of 48 after 200 episodes. With a different random seed, the initial agent networks would be different, so convergence results might be different.

In general, you might achieve higher accuracy by training the agent for longer. Note that, while training times depend on many factors, for this example, training takes more than ten minutes. By contrast, training the network in a supervised learning setting, as shown in the related previous section, takes less than 30 seconds. This difference is mostly due to the fact that a supervised learning algorithm can fully use all the available information during training (specifically, the information about the correct value of the function output associated with the observation), as previously mentioned. This performance difference can increase with the number dimensions of the function output. For these reasons, converting a supervised learning problem to a reinforcement learning problem is not recommended.

Validate Trained DDPG Agent

To reproduce the results of this section, set random seed and algorithm.

```
rng(1, "twister")
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 50 steps. For more information, see `rlSimulationOptions`.

```
simOptions = rlSimulationOptions(MaxSteps=50);
```

Simulate the environment with the trained agent and display the total reward. For more information on agent simulation, see `sim`.

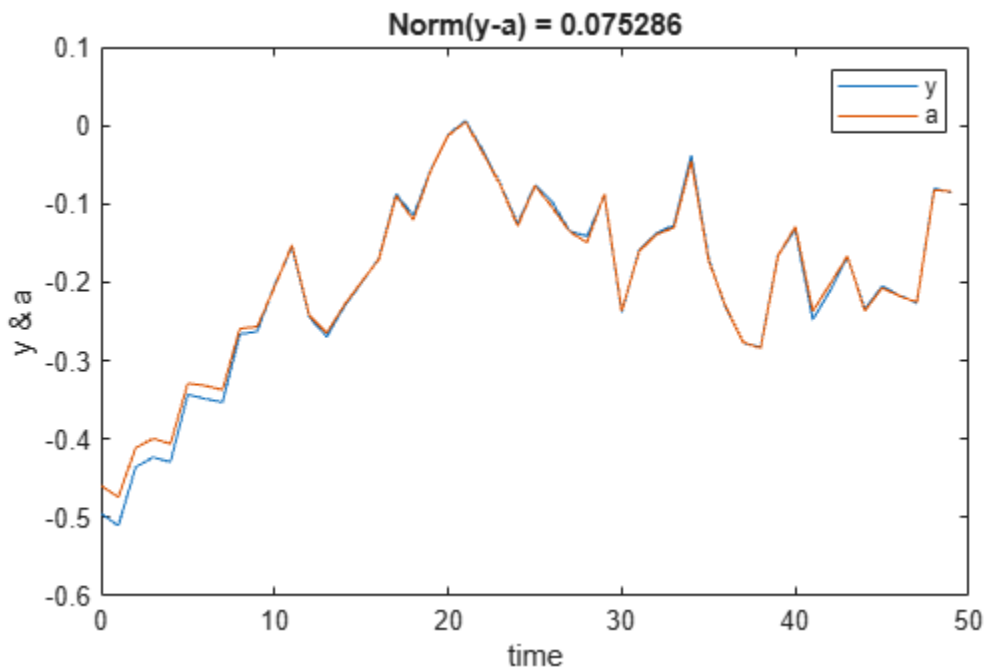
```
experience = sim(env,ddpgAgent,simOptions);
totalReward = sum(experience.Reward)
```

```
totalReward =
4.6846
```

This total reward value indicates that the trained agent is able to approximate the function with adequate accuracy.

Plot the true function value against the approximation, displaying the norm of the difference.

```
x = zeros(50,4);
for i=1:4
    x(:,i) = squeeze(experience.Observation.obs1.Data(i,1,1:end-1));
end
y = 2/30*(x(:,1).^2 + x(:,2) + x(:,3) + x(:,1).*x(:,4)) - 1/30;
t = experience.Action.act1.Time;
a = squeeze(experience.Action.act1.Data(1,1,:));
plot(t,[y a]); legend("y","a")
xlabel("time"); ylabel("y & a");
title(['Norm(y-a) = ' num2str(norm(y-a))]);
```



The figure confirms that the agent is able to approximate the function at least in this region of the observation space.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

This example has shown that you can convert a function approximation problem into a contextual bandit problem, which you can then solve using a reinforcement learning agent. This process illustrates the main differences between the two paradigms, and why converting a supervised learning problem into a reinforcement learning problem is not recommended.

Reference

[1] Barto, Andrew G., and Thomas G. Dietterich. "Reinforcement Learning and its Relationship to Supervised Learning". 2004. https://all.cs.umass.edu/pubs/2004/barto_d_04.pdf.

[2] Slivkins, Aleksandrs. "Introduction to Multi-Armed Bandits." arXiv, April 3, 2024. <https://doi.org/10.48550/arXiv.1904.07272>.

See Also

Functions

`trainnet` | `predict` | `train` | `sim`

Objects

`dlnetwork` | `rlDDPGAgent` | `rlDDPGAgentOptions` | `rlTrainingOptions`

Related Examples

- "Train Reinforcement Learning Agent for Simple Contextual Bandit Problem" on page 8-313
- "Why Solving Classification Using Reinforcement Learning Is Not Recommended" on page 8-337

More About

- "Deep Deterministic Policy Gradient (DDPG) Agent" on page 3-67
- "Train Reinforcement Learning Agents" on page 5-3
- "Create Actors, Critics, and Policy Objects" on page 4-2

Why Solving Classification Using Reinforcement Learning Is Not Recommended

This example shows how to solve a regression problem using both a supervised learning approach and a reinforcement learning approach, illustrating the differences between these methods. This example also illustrates why reinforcement learning is not the right tool to solve a supervised learning problem. For more information on the relationship between reinforcement and supervised learning, see [1 on page 8-346].

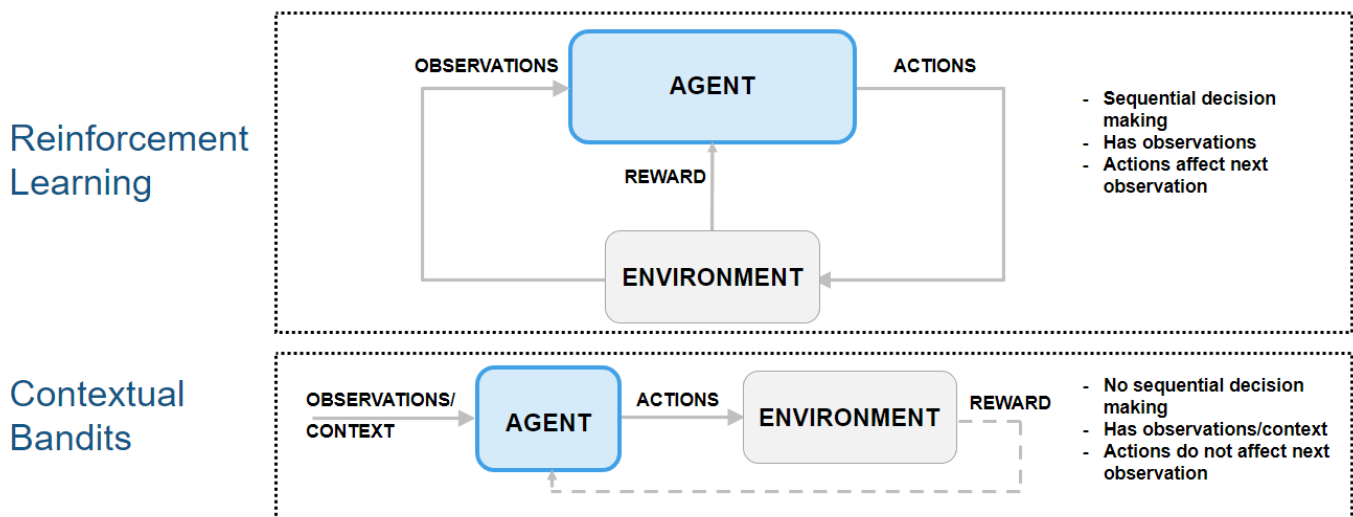
Introduction

The problem considered in this example is the classification of images containing handwritten digits. In a supervised learning setting, you solve the problem by training a network to predict the known correct digit associated with each image in a training dataset. You then use images in a validation dataset to check how well the network is able to predict the correct digit given an unseen input image. For an example, see "Create Simple Image Classification Network".

Similarly, in a reinforcement learning setting, you train an agent to predict the correct digit associated with each image in the training dataset. Specifically, the agent's observation input is an image in the dataset, while the agent's action output is the predicted digit for the image. The environment then supplies to the agent a scalar reward that represents the quality of the agent prediction. Note that the reward only captures the information of whether the prediction was correct, but does not capture the full information of which one is the right digit for the presented image. By contrast, the right digit information is fully used in the supervised learning setting, (which is why training is generally much faster for supervised learning).

Because the next observation is independent on the action, this type of reinforcement learning problem can be classified as a "contextual bandit" problem. In these problems, the environment has no state dynamics, so the reward is only influenced by the current observation and action.

The following figure shows how contextual bandit problems are special cases of reinforcement learning problems.



For an example on how to use an agent to solve a contextual bandit problem, see “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313. For an example discussing regression as a contextual bandit problem, see “Why Solving Regression Using Reinforcement Learning is Not Recommended” on page 8-327. For a comprehensive introduction to bandit problems, see [2 on page 8-346].

Preserve Random Number Sequence for Reproducibility

Some sections in this example require random number computations. Specify the seed and random number generator algorithm at the beginning of a section to use the same random number sequence each time you run it. Preserving the random number sequence allows you to reproduce the results of the section. For more information, see “Results Reproducibility” on page 5-10.

Set the random number seed to 0 and the algorithm to Mersenne Twister. For more information, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the sequence. You will restore the state at the end of the example.

Extract Image Data and Create Training and Validation Sets

For this example, use the same digit data file in used in “Create Simple Image Classification Network”.

Unzip the digit sample data, unless the data folder already exists.

```
if ~exist("DigitsData","dir")
    unzip("DigitsData.zip")
end
```

Create an image datastore. The `imageDatastore` function automatically labels the images based on folder names. For more information about datastores, see “Getting Started with Datastore”.

```
imds = imageDatastore("DigitsData", ...
    IncludeSubfolders=true, ...
    LabelSource="foldernames");
```

Divide the data into training and validation data sets, so that each category in the training set contains 750 images and the validation set contains the remaining images from each label. `splitEachLabel` splits the image datastore into two new datastores for training and validation. Because there are 10 categories (one for each digit) the training datastore contains a total of $750 \times 10 = 7500$ available images.

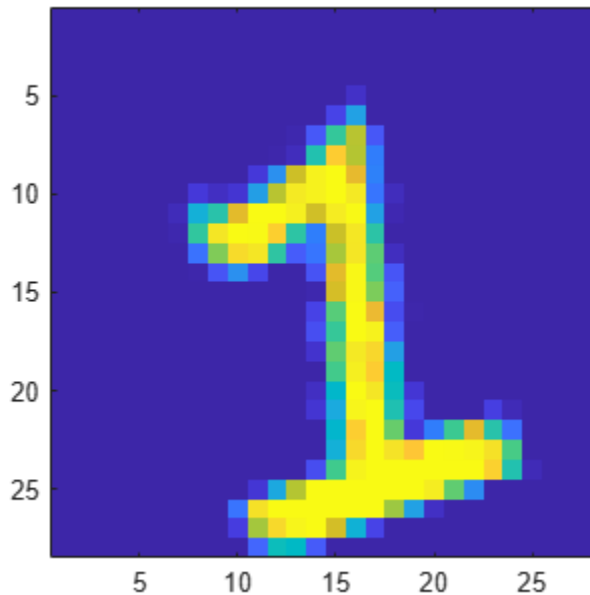
```
numTrainFiles = 750;
[imdsTrain,imdsValidation] = ....
    splitEachLabel(imds,numTrainFiles,"randomized");
```

To view the class names, run the following command.

```
classNames = categories(imdsTrain.Labels)
```

Display a random image from the training datastore.

```
image(readimage(imdsTrain,randi(numTrainFiles*10)));
axis("square")
```



Create Custom Discrete-Action-Space Environment Object

Create the environment specifications. The (continuous) observation is a gray scale 28-by-28 pixels image, with each element normalized between 0 and 1, and the (discrete) action is the predicted digit (from 0 to 9) associated with the input image.

```
obsInfo = rlNumericSpec([28 28], ...
    LowerLimit=0, ...
    UpperLimit=1);
actInfo = rlFiniteSetSpec(0:9);
```

At the beginning of each training or simulation episode, a reset function resets the environment to an initial state and returns both the initial state (needed for the next environment step) and the resulting initial observation value (for the agent to see).

Display the custom reset function, which is provided in the file `clsfResetFcn.m`.

```
type("clsfResetFcn.m")

function [obs,x] = clsfResetFcn(imds,nmax)
% Reset function to set environment in random initial state.

% Read a random image from dataset, the image is the observation.
[obs,info] = readimage(imds,randi(nmax));

% Normalize image.
obs = double(obs)./255;

% State is the true label (used to calculate the reward).
x = grp2idx(info.Label)-1;

end
```

This function takes as input argument an image datastore (it could be either the training or validation set) and the total number of images in the dataset. The function then reads a random image, converts the image to a double, and normalizes it. The output arguments are the initial observation for the agent, (which is the normalized image), and the initial state of the environment, which is the correct label (0 to 9) associated with the image.

A step function is called by `train` (or `sim`) at each step of the training (or simulation) episode to advance the environment to the next step.

Display the custom step function, which is provided in the file `clsfStepFcn.m`.

```
type("clsfStepFcn.m")

function [nextObs,r,isdn,xp] = clsfStepFcn(a,x,imds,nmax)
% Custom step function to advance custom environment by one step.

% Calculate reward (reward for a).
r = double(a==x);

% Calculate next observation and state.

% Read a random image from dataset, the image is the next observation.
[nextObs,info] = readimage(imds,randi(nmax));

% Normalize image.
nextObs = double(nextObs)./255;

% Store new label (for use at the next step).
xp = grp2idx(info.Label)-1;

% Episode does not end because of an action.
isdn = 0;

end
```

This function takes as input arguments the action from the agent, the current environment state, an image datastore and the total number of images in the dataset.

First, the function calculates a reward for the agent, which is equal to one if the action is equal to the current state (this means the agent was able to correctly identify the image), and zero otherwise. Note that, differently from a supervised learning setting, when the guess is not correct, the environment does not convey to the agent any information about which label is the correct one.

The step function then reads a new random image, converts the image to a double, and normalizes it. The output arguments are the next observation for the agent, (which is the new normalized image), the reward associated with the agent action, an is-done flag that indicates if the episode is finished and the updated state of the environment, which is the correct label (0 to 9) associated with the new image.

Create the custom function environment object for training, using the training datastore.

Because the custom reset and step functions that you must pass to `rlFunctionEnv` must have exactly zero and two arguments, respectively, define the actual reset and step functions as anonymous functions (with zero and two arguments, respectively). In turn, these anonymous functions, call `clsfResetFcn` and `clsfStepFcn`, passing the additional `imds` and `nmax` arguments which are stored in the workspace of the anonymous function at definition time.


```
envTrain = rlFunctionEnv(obsInfo,actInfo, ...
    @(a,x) clsfStepFcn(a,x,imdsTrain,numTrainFiles*10), ...
    @() clsfResetFcn(imdsTrain,numTrainFiles*10));
```

For a more detailed example on custom step and reset functions, see “Create Custom Environment Using Step and Reset Functions” on page 2-94.

Create Discrete Categorical Actor from Custom Network

Policy gradient agents use a parameterized stochastic policy, which for discrete action spaces is implemented by a discrete categorical actor. This actor takes an observation as input and returns as output a random action sampled (among the finite number of possible actions) from a categorical probability distribution.

To model the parameterized policy within the actor, use a neural network with one input layer (which receives the content of the environment observation channel, as specified by `obsInfo`) and one output layer. The output layer must return a vector of probabilities for each possible action, as specified by `actInfo`.

Define the network as an array of layer objects. This is the same network used in the example “Create Simple Image Classification Network”.

```
layers = [
    imageInputLayer([28 28 1])
    convolution2dLayer(5,20)
    reluLayer
    fullyConnectedLayer(10)
    softmaxLayer
];
```

For more information about deep learning layers, see “List of Deep Learning Layers”.

The network is initialized randomly. Specify random seed and algorithm for reproducibility.

```
rng(0, "twister")
```

Create `dlnetwork` object.

```
net = dlnetwork(layers);
```

Display the number of learnable parameters.

```
summary(net)

    Initialized: true

    Number of learnables: 115.7k

    Inputs:
        1 'imageinput' 28×28×1 images
```

Create a discrete categorical actor, for more information, see `rlDiscreteCategoricalActor`.

```
actor = rlDiscreteCategoricalActor(net,obsInfo,actInfo);
```

To check your actor, use the `getAction` function to return a batch of 5 actions from a batch of 5 random observations, given the current network weights.

```
obs = rand([obsInfo.Dimension 5]);  
act = getAction(actor,{obs});
```

Return the third element of the batch.

```
act{1}(1,1,3)
```

```
ans =  
3
```

To return the probability distribution of the actions, given an observation, use `evaluate`.

```
prb = evaluate(actor,{obs});  
prb{1}(:,3)
```

```
ans = 10×1 single column vector
```

```
0.1217  
0.0974  
0.0738  
0.1123  
0.1247  
0.0926  
0.1036  
0.1188  
0.0734  
0.0817
```

Create a PG Agent and Specify Agent Options

Create a `rlPGAgent` object using the previously created actor.

```
pgAgent = rlPGAgent(actor);
```

Set a lower learning rate and gradient thresholds to avoid instability.

```
pgAgent.AgentOptions.ActorOptimizerOptions.LearnRate = 1e-3;  
pgAgent.AgentOptions.ActorOptimizerOptions.GradientThreshold = 1;
```

Because future reward is independent on the action, set a discount factor of zero.

```
pgAgent.AgentOptions.DiscountFactor = 0;
```

Configure Training Options

Create an evaluator object to evaluate the agent five times without exploration every 100 episodes.

```
evl = rlEvaluator(NumEpisodes=5,EvaluationFrequency=100);
```

Create a training options object. For more information on training options, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions;
```

- Set the maximum number of steps per episode to 100 (note that the environment has no terminal state).
- Stop the training when the average reward collected over the evaluation episodes reaches 90. This means the agent can classify images with a 90% accuracy.

- In any case, stop after 5000 episodes (which is enough for the agent to see the same image approximately $100 \times 5000 / 7500 = 66.6$ times).

```
trainOpts.MaxEpisodes = 5000;
trainOpts.MaxStepsPerEpisode = 100;
trainOpts.StopTrainingCriteria = "EvaluationStatistic";
trainOpts.StopTrainingValue = 90;
```

Alternatively, because the next state is independent of the action, you can set the maximum number of steps per episode to 1 (and consequently allow for a larger number of episodes). Adopting this view, each episode is a stand-alone event in which the agent associates a single observation to a single action, that is, each episode represents an instance of a bandit problem with a different context (a different input value). You must use this approach if your environment does not allow multiple steps per episode.

However, if you use this alternative approach to train agents like PG agents, which calculate network gradients at the end of the episode and use only the steps available in the episode, then the network gradients are calculated using only a single observation and reward. This can lead to inefficient or unstable learning.

For an example using this alternative approach, see “Train Reinforcement Learning Agent for Simple Contextual Bandit Problem” on page 8-313.

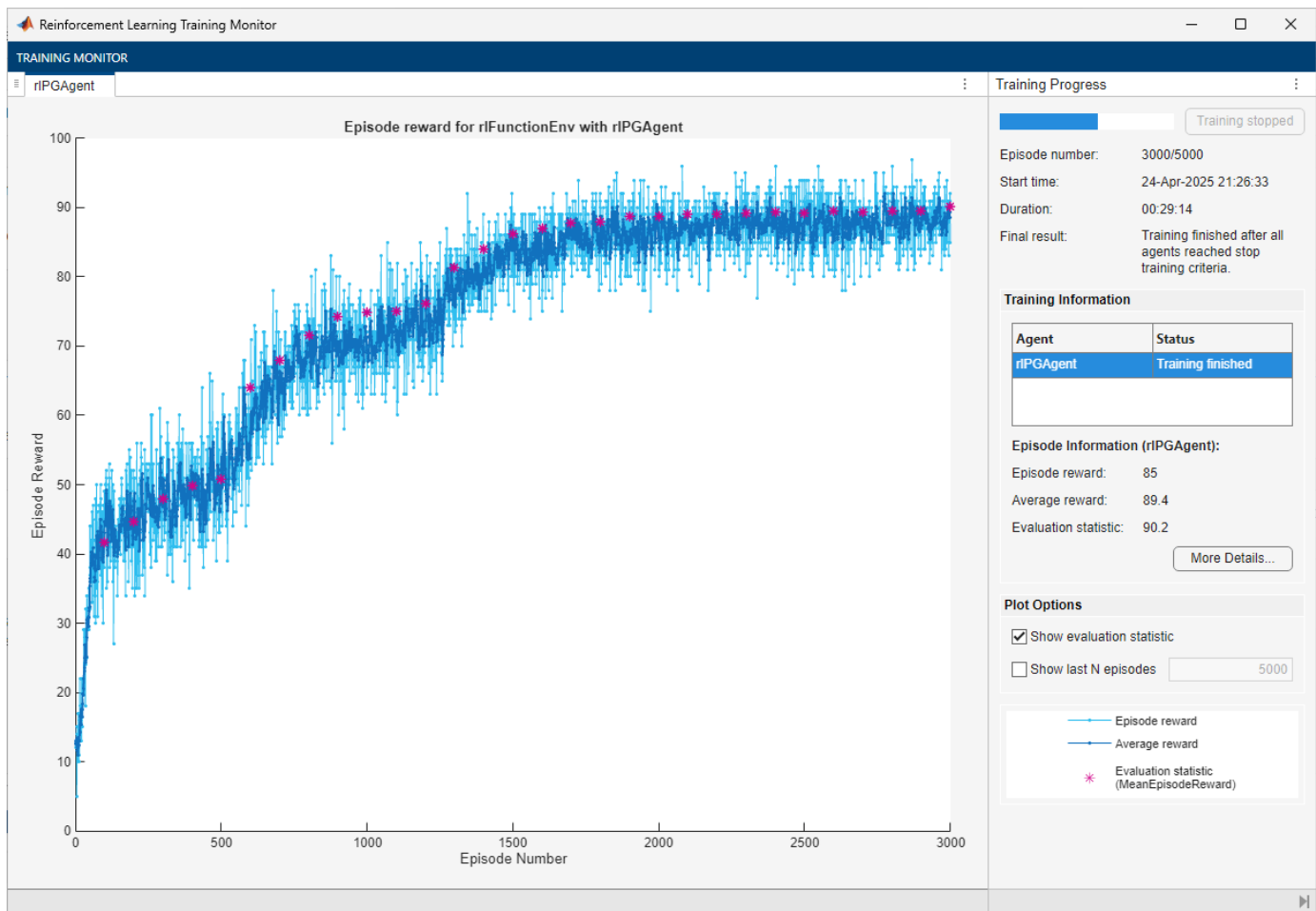
Train the PG Agent

To reproduce the results of this section, set random seed and algorithm.

```
rng(0, "twister")
```

To train the agent, pass the agent, the training environment, the training options, and the evaluator objects to `train`. Training is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining =  ;
if doTraining
    % Train the agent. Save the final agent and training results.
    pgTrainingRes = train(pgAgent, envTrain, trainOpts, Evaluator=evl);
    % Uncomment to save the trained agent and results.
    % save("clsfPGAgent.mat", "pgAgent", "pgTrainingRes")
else
    % Load the pretrained agent and results for the example.
    load("clsfPGAgent.mat", "pgAgent", "pgTrainingRes")
end
```



The trained agent achieves a 90.2% accuracy after 3000 episodes. With a different random seed, the initial agent network would be different, and therefore, convergence results might be different.

In general, you can achieve higher accuracy by training the agent for longer. Note that, while training times depend on many factors, for this example, training takes around half an hour. By contrast, training the network in a supervised learning setting, as shown in “Create Simple Image Classification Network”, takes less than one minute. This difference is mostly due to the fact that a supervised learning algorithm can fully use all the available information during training, (specifically, the information about the correct digit associated with the input image), as previously mentioned. This performance difference can increase with the number of classes to be identified. Furthermore, the cross-entropy loss function typically used for classification problems in supervised learning settings is specifically tailored to classification tasks. For these reasons, converting a supervised learning problem to a reinforcement learning problem is not recommended.

Validate Trained PG Agent

To reproduce the results of this section, set random seed and algorithm.

```
rng(0, "twister")
```

Calculate the number of images in the validation datastore.

```
nVal = sum(imdsValidation.countEachLabel.(2))
```

```
nVal =
2500
```

Create the environment object for validation, using the validation datastore.

```
envVal = rlFunctionEnv(obsInfo,actInfo, ...
    @(a,x) clsfStepFcn(a,x,imdsValidation,nVal), ...
    @() clsfResetFcn(imdsValidation,nVal));
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

To simulate the trained agent, create a simulation options object and configure it to simulate for 100 steps. For more information, see `rlSimulationOptions`.

```
simOptions = rlSimulationOptions(MaxSteps=100);
```

Simulate the validation environment with the trained agent and display the total reward. For more information on agent simulation, see `sim`.

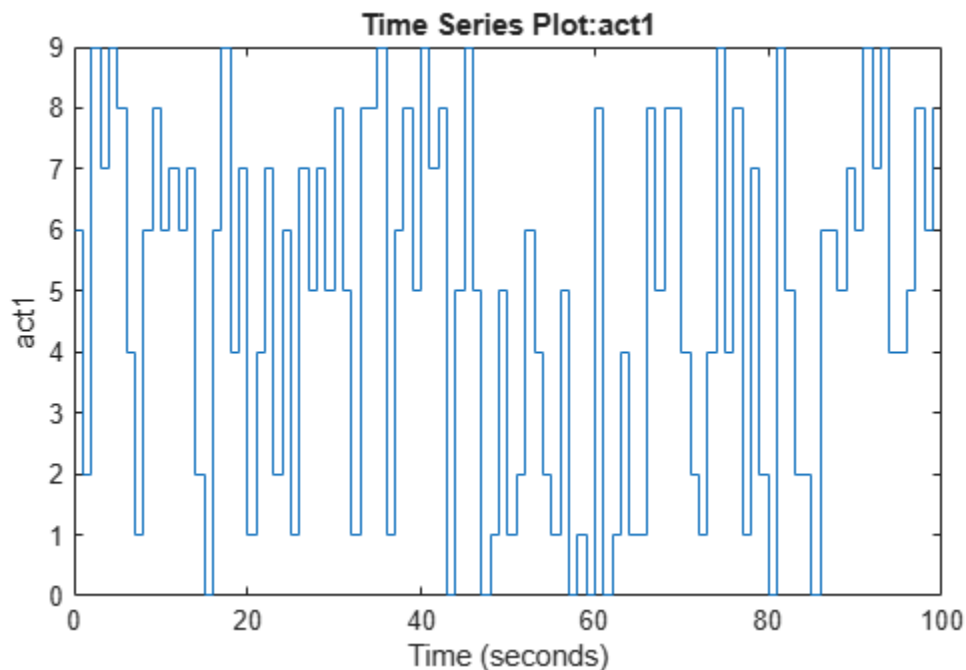
```
experience = sim(envVal,pgAgent,simOptions);
totalReward = sum(experience.Reward)
```

```
totalReward =
88
```

This total reward value indicates that the trained agent is able to successfully classify unseen images with an accuracy of 89%.

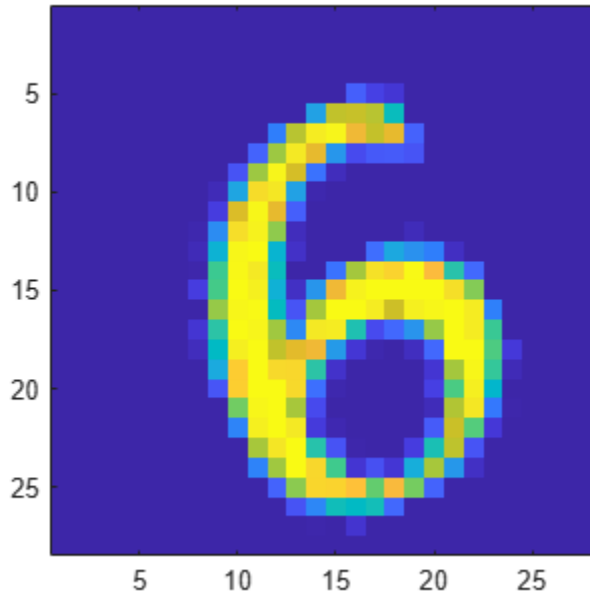
Plot the image label with time in the simulation episode.

```
plot(experience.Action.act1)
```



Display the image and the resulting action at step number 25.

```
sn = 25;  
image(uint8(255*experience.Observation.obs1.Data(:,:,sn)))  
axis("square")
```



```
experience.Action.act1.Data(:,:,sn)
```

```
ans =  
6
```

The agent has classified the image correctly.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

This example has shown that you can convert an image classification problem into a contextual bandit problem, which you can then solve using a reinforcement learning agent. This process illustrates the main differences between the two paradigms, and why converting a supervised learning problem into a reinforcement learning problem is not recommended.

Reference

[1] Barto, Andrew G., and Thomas G. Dietterich. "Reinforcement Learning and its Relationship to Supervised Learning". 2004. https://all.cs.umass.edu/pubs/2004/barto_d_04.pdf.

[2] Slivkins, Aleksandrs. "Introduction to Multi-Armed Bandits." arXiv, April 3, 2024. <https://doi.org/10.48550/arXiv.1904.07272>.

See Also

Functions

`train` | `sim` | `trainnet`

Objects

`dlnetwork` | `rlDiscreteCategoricalActor` | `rlPGAgent` | `rlPGAgentOptions` | `rlTrainingOptions`

Related Examples

- "Create Simple Image Classification Network"
- "Train Reinforcement Learning Agent for Simple Contextual Bandit Problem" on page 8-313
- "Why Solving Regression Using Reinforcement Learning is Not Recommended" on page 8-327

More About

- "Getting Started with Datastore"
- "REINFORCE Policy Gradient (PG) Agent" on page 3-37
- "Train Reinforcement Learning Agents" on page 5-3
- "Create Actors, Critics, and Policy Objects" on page 4-2

Train Agent to Play Turn-Based Game

This example shows you how to train a deep Q-network (DQN) reinforcement learning agent to play a turn-based multi-player game.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10. Fix the random number stream with seed 0 and random number algorithm Mersenne twister. For more information on controlling the seed used for random number generation, see `rng`.

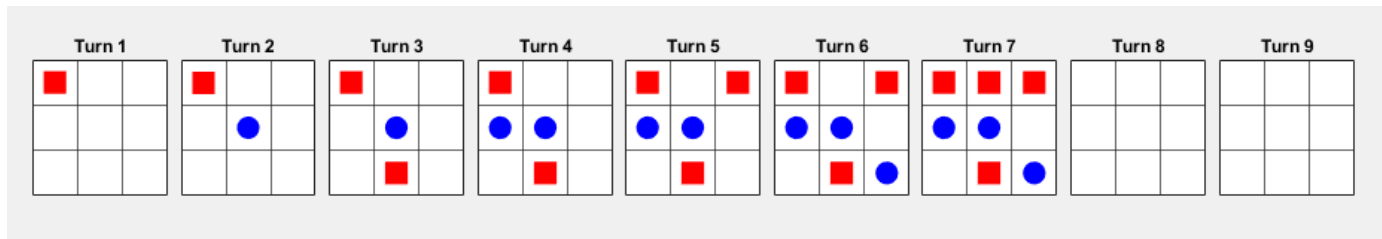
```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Overview

Environments in which agent execution is controlled by a turn-based logic are known as turn-based environments. Turns are analogous to environment steps, and one or more agents might take actions at a given turn. For example, in a two-player game such as chess, each turn requires one player (or agent) to take action.

In this example, you create a turn-based multiagent environment using MATLAB® functions and train an agent to play against a policy that takes random actions.



The environment in this example is a simple turn-based two-player game. For this environment:

- The game contains a 3-by-3 grid with nine cells.
- Two adversarial players (agents) play the game. At each turn, a player marks a location in the grid. Player one has red squares and player two has blue circles.
- The observation of each agent is a vector obtained from the 3-by-3 grid by flattening it in column-major fashion. The vector contains zeros in unmarked locations, ones in locations marked by the agent, and negative ones in locations marked by the opponent agent.
- The action of an agent is the grid location to be marked (an integer between one and nine, inclusive).
- The game is won if an agent has marked three consecutive cells (a horizontal row, vertical column, or any diagonal).

The reward function for the active agent is described below.

- A positive reward 10 if the agent wins the game in the current turn.
- A positive reward of 1 if the agent has marked two consecutive grid cells with the third cell in line unmarked, setting up a possible win in the next turn.
- A positive reward of 10 if the agent marks a cell that prevents the opponent from winning in the next turn.
- A negative reward of -10 if the agent fails to mark a cell that enables the opponent to win in the next turn.
- A negative reward of -10 if the agent takes an illegal move like marking an occupied cell. The simulation is also terminated in this case.

The inactive agent receives the reward 0 always.

Create Environment Object

First, create the observation specification for each agent. The observation is of a vector of 9 elements.

```
oinfo = rlNumericSpec([9,1]);
```

Next, create the action specification for each agent, which is a set of discrete cell indices.

```
ainfo = rlFiniteSetSpec({1;2;3;4;5;6;7;8;9});
```

The environment's observation and action specifications are cell arrays. Create the arrays.

```
obsInfo = {oinfo, oinfo};
actInfo = {ainfo, ainfo};
```

Create a turn-based environment using `rlTurnBasedFunctionEnv`. The command takes as input arguments the observation and action specifications of the agent and function handles for the step and reset operations.

Create the environment object. The functions `stepGame` and `resetGame` are provided at the end of this example.

```
env = rlTurnBasedFunctionEnv(obsInfo, actInfo, @stepGame, @resetGame)
```

```
env =
    rlTurnBasedFunctionEnv with properties:
```

```
    StepFcn: @stepGame
    ResetFcn: @resetGame
    Info: [1x1 struct]
```

Add a listener for changes to the `Info` property of the environment. When the `Info` property is updated during simulation, the function `hPlotGame` (provided at the end of this example) plots the updated environment. Turn off the visualization to improve the training performance. To turn on visualization, set `doPlot` to `true`.

```
doPlot = ;
if doPlot
    addlistener(env,"Info","PostSet",@(~,~) hPlotGame(env));
end
```

Create Agents

Two adversarial agents play the game.

- An agent generating random allowed actions controls the blue circles in the game. This agent is not trained.
- A deep Q-network (DQN) agent controls the red squares in the game. You train this agent to win against the random policy.

Create a custom agent object using the `AgentWithRandomActions` class. For more information, see “Create Custom Reinforcement Learning Agents” on page 3-92.

```
randomAgent = AgentWithRandomActions(oinfo,ainfo);
```

View the implementation of `AgentWithRandomActions`.

```
type("AgentWithRandomActions.m")

classdef AgentWithRandomActions < rl.agent.CustomAgent
    % AgentWithRandomActions models a custom agent with random actions.

    % Copyright 2023 The MathWorks, Inc.

    properties (Access = private)
        SampleTime_
    end

    methods
        function this = AgentWithRandomActions(obsinfo, actinfo)
            setObservationInfo_(this, obsinfo);
            setActionInfo_(this,actinfo);
            this.SampleTime = 1;
        end
    end

    methods (Access = protected)
        function learnImpl(~,~)
            % no op because the agent does not learn
        end

        function action = getActionWithExplorationImpl(this, obs)
            action = getActionImpl(this, obs);
        end

        function action = getActionImpl(~, obs)
            % Generate random actions.
            x = obs{1};
            % legal moves are positions which are unmarked (value is 0)
            legalmoves = find(x==0);
            % choose a random legal move
            randidx = randperm(numel(legalmoves),1);
            action = legalmoves(randidx);
        end

        function ts = getSampleTime_(this)
            ts = this.SampleTime_;
        end
    end
end
```

```

        function this = setSampleTime_(this,ts)
            this.SampleTime_ = ts;
        end
    end
end
end

```

Next, specify options for training the DQN agent. The agent learns using the double deep Q-network algorithm (by setting `UseDoubleDQN` to `true`) with a mini-batch size of 64, target smoothing factor of 0.01, and a discount factor of 0.99 which favors long-term rewards. For more information, see `rLDQNAgentOptions`.

```

agentOpts = rLDQNAgentOptions( ...
    UseDoubleDQN=true, ...
    MiniBatchSize=64, ...
    TargetSmoothFactor=1e-2, ...
    DiscountFactor=0.99);

```

Specify options for the critic optimization algorithm. The critic learns using the adam algorithm with a learning rate of $1e-3$.

```

agentOpts.CriticOptimizerOptions.Algorithm = "adam";
agentOpts.CriticOptimizerOptions.LearnRate = 1e-3;

```

Specify exploration options for training. The agent uses an epsilon-greedy exploration strategy with an initial Epsilon value of 0.9, which exponentially decays at the rate of $1e-4$ until it reaches the value of 0.01.

```

agentOpts.EpsilonGreedyExploration.Epsilon = 0.9;
agentOpts.EpsilonGreedyExploration.EpsilonDecay = 1e-4;
agentOpts.EpsilonGreedyExploration.EpsilonMin = 0.01;

```

Set the length of the experience buffer as $1e6$. This ensures that the agent learns from a large set of experiences.

```

agentOpts.ExperienceBufferLength = 1e6;

```

Create initialization options for the agent. The agent's critic uses a neural network model with a hidden layer size of 64.

```

initOpts = rlAgentInitializationOptions(NumHiddenUnit=64);

```

Create a default DQN agent object. When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values. For more information, see `rLDQNAgentOptions`.

```

rng(0,"twister");
agent = rLDQNAgent(oinfo, ainfo, initOpts, agentOpts);

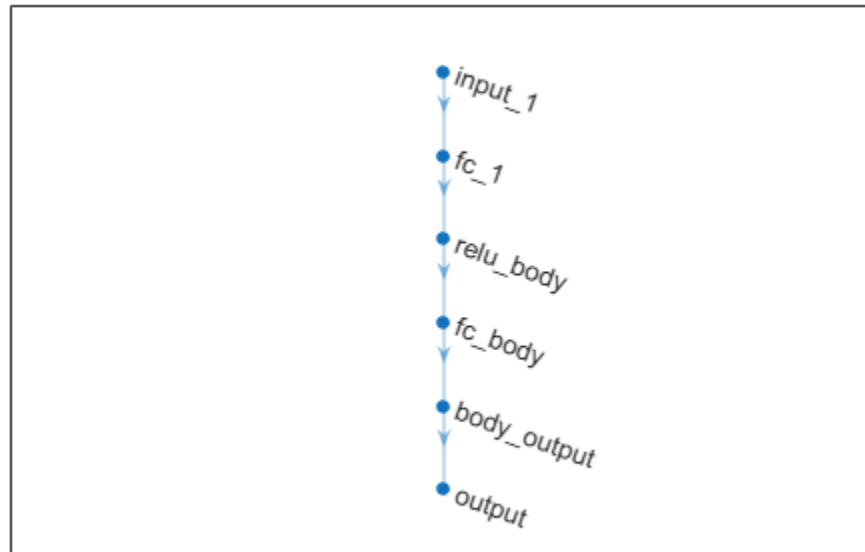
```

View the critic network of the agent.

```

critic = getCritic(agent);
criticNet = getModel(critic);
plot(criticNet);

```



View a summary of the critic network.

```
summary(criticNet);

Initialized: true

Number of learnables: 5.4k

Inputs:
  1  'input_1'   9 features
```

Training

In this example, only the agent controlling the red squares is trained.

To evaluate the simulation behavior of the agent during training, configure an evaluator object that periodically computes an evaluation score. In this example the evaluator object runs 25 evaluation simulations every 100 training episodes with different random seeds (0 to 24) and computes the minimum episode reward as the evaluation score. The evaluation score might be used as a criterion to save agents during training. Although the agent evaluations are run with different random seeds, the random stream of the training process is not affected.

For more information on evaluation objects, see `rlEvaluator`.

```
evaluator = rlEvaluator( ...
    EvaluationFrequency=100, ...
    EvaluationStatisticType="MinEpisodeReward", ...
    NumEpisodes=25, ...
    RandomSeeds=0:24);
```

For this training session:

- Run the training for 5000 episodes.

- Specify `AgentGroups` as a cell array of agent indices. Omit the index of the agent that is not trained. Note that the index must correspond to the order of agents specified in the `train` function.
- Compute the average episodic rewards using a moving window of size 20.
- Save the agents from the episodes where the evaluation score is 9 or higher. This score is close to the reward received (10) when the game is won and might indicate that the policy has learned sufficiently.
- Turn off visualization during training (set the `Plots` option) and display the training progress in the command window using the `Verbose` option.
- View the training progress plot after the training is finished using the `show` command.

```
trainOpts = rlMultiAgentTrainingOptions( ...
    AgentGroups={1}, ...
    MaxEpisodes=5000, ...
    ScoreAveragingWindowLength=20, ...
    StopTrainingCriteria="none", ...
    SaveAgentCriteria="EvaluationStatistic", ...
    SaveAgentValue=9, ...
    Plots="none", ...
    Verbose=true);
```

For more information, see `rlMultiAgentTrainingOptions`.

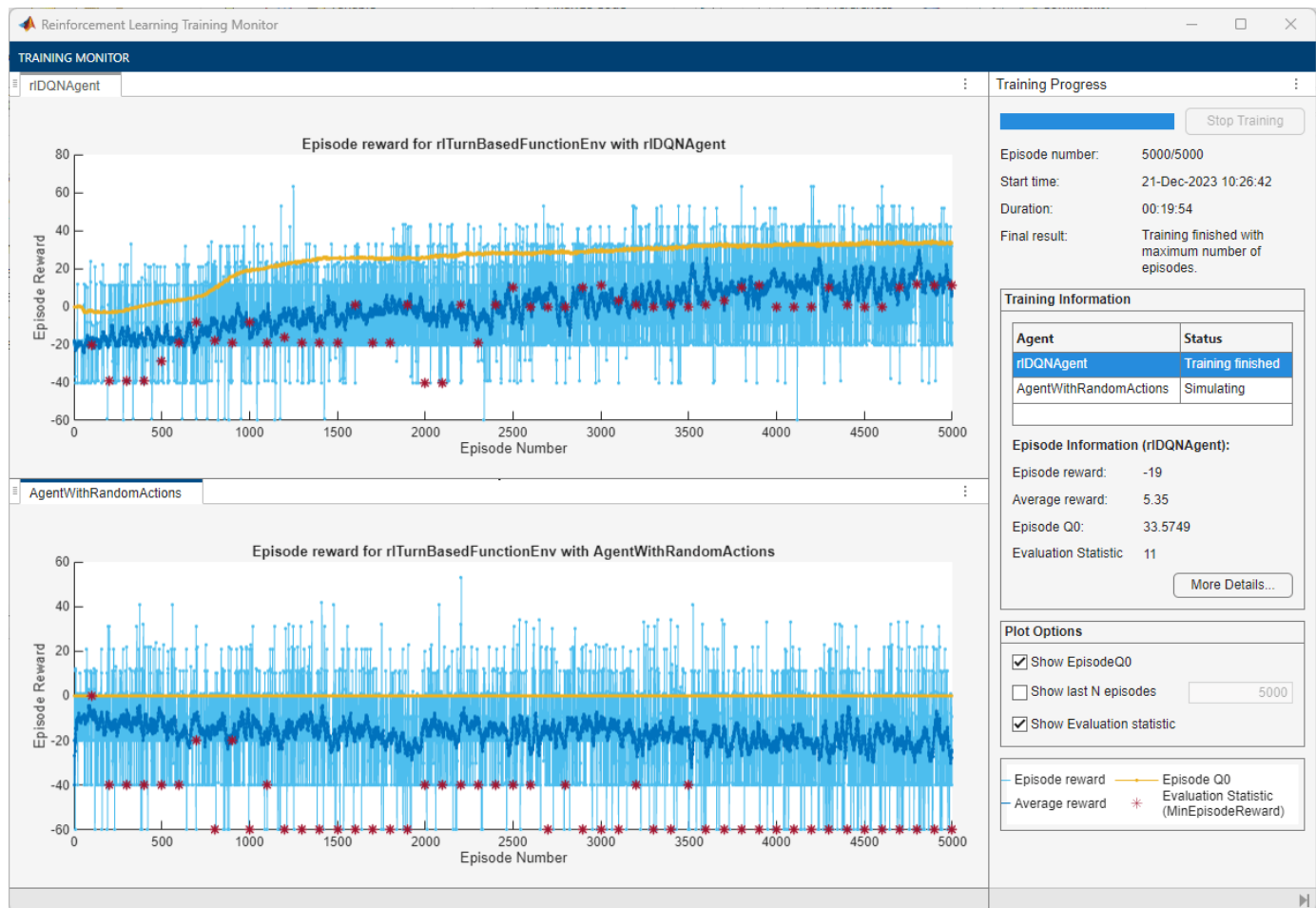
Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Train the agents using the `train` function. Training can take several hours to complete depending on the available computational power. To save time, load the MAT file `twoPlayerGameAgent.mat`, which contains a set of pretrained agents. To train the agents yourself, set `doTraining` to `true`.

```
doTraining = ;
if doTraining
    % train
    trainResults = train([agent, randomAgent], env, ...
        trainOpts, Evaluator=evaluator);
    % show the training progress visualization
    show(trainResults);
else
    load("twoPlayerGameAgent.mat");
end
```

A snapshot of the training progress is shown in the following figure. You might see different results from your training process.



Simulation

Fix the random stream for reproducibility.

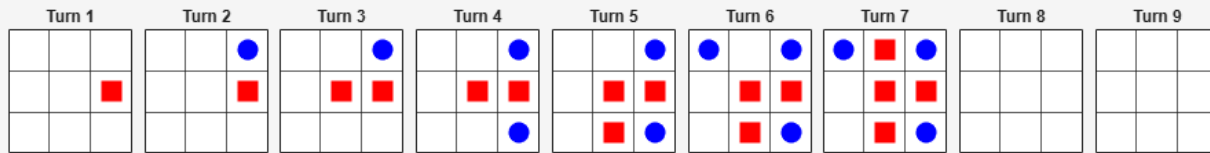
```
rng(0, "twister");
```

Specify the function `hPlotGame` for environment visualization.

```
addlistener(env, "Info", "PostSet", @(~,~) hPlotGame(env));
```

Simulate the trained agent with the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
simOptions = rlSimulationOptions();
experience = sim(env, [agent, randomAgent], simOptions);
```



The visualization shows the sequence of turns for the game. The trained agent has learned the actions to win the game.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Local Functions

Custom reset function for the two-player game. The reset function initializes the environment, randomly selects an agent for the first turn, and returns the initial observations of the agents along with the `info` variable that passes information between simulation steps.

```
function [initialObs, info] = resetGame()
% Reset the two-player game.

% The state is a 3x3 matrix with 0s in unmarked cells,
% -1s in cells marked with squares and
% 1s in cells marked with circles.
info.State = zeros(3,3);

% The initial turn is randomly selected.
info.ActiveAgentIndex = 1; %randperm(2,1);

% The current environment step count.
info.StepCount = 0;

% Flag to keep track of invalid action.
info.IsInvalidAction = false;

% Get the active agent for the current turn.
selfIdx      = info.ActiveAgentIndex;
opponentIdx  = mod(selfIdx,2) + 1;
playerVals   = [-1,1];
selfVal      = playerVals(selfIdx);
opponentVal  = playerVals(opponentIdx);

% Initial observation is the state of the environment
% as seen by each agent:
% self cells 1, opponent cells -1, unmarked 0
selfObs = ...
    double(info.State==selfVal) - double(info.State==opponentVal);
opponentObs = ...
    -double(info.State==selfVal) + double(info.State==opponentVal);
```

```
initialObs{selfIdx} = selfObs(:);  
initialObs{opponentIdx} = opponentObs(:);  
end
```

Custom step function for the two-player game. This function steps the environment dynamics to the next state.

```
function [nextobs, reward, isdone, info] = stepGame(action, info)
```

```
% Get the active agent for the current turn.  
selfIdx = info.ActiveAgentIndex;
```

```
% Compute the next agent index.  
opponentIdx = mod(selfIdx,2) + 1;
```

```
% Player cell values:  
% Player 1 (red square) is identified by the value -1.  
% Player 2 (blue circle) is identified by the value 1.  
playerVals = [-1,1];
```

```
% Get current player value.  
selfVal = playerVals(selfIdx);  
opponentVal = playerVals(opponentIdx);
```

```
% Index of cell for the current player.  
% action{1} is the move of the current player,  
% that is, the index (between 1 and 9)  
% of new grid location to be marked.  
playerCell = action{1};
```

```
% Advance environment to the next state.  
state = info.State;  
if state(playerCell) == 0  
    % Move to next state.  
    state(playerCell) = selfVal;  
    % Compute reward and terminal condition.  
    [rwd, isdone, iswin] = hComputeRewardAndIsDone( ...  
        state, playerCell, selfVal, opponentVal);
```

```
else  
    % For an illegal action,  
    % terminate with a large penalty.  
    rwd = -20;  
    isdone = true;  
    iswin = false;  
    info.IsInvalidAction = true;  
end
```

```
% The active agent receives the reward rwd,  
% and the other agent receives the reward 0.  
reward = [0,0];  
reward(selfIdx) = rwd;
```

```
% Optional: To make this a zero sum game,  
% uncomment the following line.  
% reward(opponentIdx) = -rwd;
```

```
% Next observation.  
% self cells 1
```



```

% opponent cells -1
% unmarked 0
selfObs = ...
    double(state==selfVal) - double(state==opponentVal);
opponentObs = ...
    -double(state==selfVal) + double(state==opponentVal);

nextObs{selfIdx} = selfObs(:);
nextObs{opponentIdx} = opponentObs(:);

% Update the info structure with:
% 1. The state for the next step.
% 2. The agent turn for the next step.
% 3. The step count.
info.State = state;
info.ActiveAgentIndex = opponentIdx;
info.StepCount = info.StepCount + 1;
info.IsWin = iswin;
end

```

Helper function to compute reward and terminal conditions.

```

function [reward, isover, iswin] = hComputeRewardAndIsDone( ...
    state, index, player, opponent)
% Advance to the next state of the game by marking a cell.
%
% Reward for active agent:
%
% +10 if the agent wins the game in the current turn.
%
% +1 if the agent has marked two consecutive grid cells
% with the third cell in line unmarked.
%
% +10 if the agent marks a cell that prevents
% the opponent from winning in the next turn.
%
% -10 if the agent fails to mark a cell
% that enables the opponent to win in the next turn.
%
% -10 if the agent takes an illegal move
% like marking an occupied cell.
%
% 0 otherwise.

sz = size(state);
reward = 0;
isover = false;
iswin = 0;

% Current row and column of active agent.
[r,c] = ind2sub(sz, index);

function nestedComputeReward_(arr)
    % arr is a row or column or a diagonal.
    if all(arr==player)
        % Row/column/diagonal complete, game over.
        reward = reward + 10;
        isover = true;
    end
end

```

```

        iswin = true;
    elseif nnz(arr==player)==2 && any(arr==0)
        % Player marks adjacent cells,
        % possible win next turn.
        reward = reward + 1;
    elseif nnz(arr==opponent)==2
        % Player blocks opponent win.
        reward = reward + 10;
    end
end

% Horizontal cells.
hcells = state(r,:);
nestedComputeReward_(hcells);

% Vertical cells.
vcells = state(:,c);
nestedComputeReward_(vcells);

% Check the two diagonals:
% r==c is the main diagonal,
% r+c==4 is the other diagonal.
if r==c || r+c==4
    % Main diagonal cells.
    mdcells = [state(1,1) state(2,2) state(3,3)];
    nestedComputeReward_(mdcells);
    % Other diagonal cells.
    odcells = [state(3,1) state(2,2) state(1,3)];
    nestedComputeReward_(odcells);
end

% If all cells are marked the game is over.
if ~any(state==0)
    isover = true;
end

% penalty if opponent is winning next turn
if ~isover
    ishcellWin = any(cellfun(@(x) nnz(x==opponent)==2 && any(x==0), ...
        {state(1,:),state(2,:),state(3,:)}));
    isvcellWin = any(cellfun(@(x) nnz(x==opponent)==2 && any(x==0), ...
        {state(:,1),state(:,2),state(:,3)}));
    isdiag1Win = any(cellfun(@(x) nnz(x==opponent)==2 && any(x==0), ...
        {[state(1,1),state(2,2),state(3,3)]}));
    isdiag2Win = any(cellfun(@(x) nnz(x==opponent)==2 && any(x==0), ...
        {[state(3,1),state(2,2),state(1,3)]}));
    if ishcellWin || isvcellWin || isdiag1Win || isdiag2Win
        reward = -20;
    end
end
end

Helper function for visualizing the environment.

function hPlotGame(env)
% Plot the Simple Two Player Game environment.

persistent f ax hrect

```

```

state = env.Info.State;
stepCt = env.Info.StepCount;
isdone = env.Info.IsInvalidAction;
if isempty(f) || ~isvalid(f)
    f = figure( ...
        Toolbar="auto", ...
        NumberTitle="off", ...
        Name="Simple Two Player Game", ...
        MenuBar="none", ...
        Visible="on");
    f.Position(3:4) = [1000,250];

    t = tiledlayout( ...
        f, ...
        "horizontal", ...
        TileSpacing="compact", ...
        Padding="compact");

    for ct = 1:9
        hax = nexttile(t);
        axis(hax,"equal");
        title(hax,"Turn "+ct);
        hax.XLim = [-15 15];
        hax.YLim = [-15 15];
        hax.XTick = [];
        hax.YTick = [];
        hax.Box = "on";
        hold(hax,"on");
        xline(hax,-5);
        xline(hax,5);
        yline(hax,-5);
        yline(hax,5);
        ax{ct} = hax;
    end
end
if stepCt<=1
    delete(hrect);
    hrect = [];
end
if ~isdone
    for ct = 1:9
        if ismember(ct,[1,2,3])
            x = -10;
        elseif ismember(ct,[4,5,6])
            x = 0;
        else
            x = 10;
        end
        if ismember(ct,[1,4,7])
            y = 10;
        elseif ismember(ct,[2,5,8])
            y = 0;
        else
            y = -10;
        end
        if state(ct)==-1
            hr = rectangle( ...
                ax{stepCt}, ...

```

```
        Position=[x-2.5,y-2.5,5,5], ...
        FaceColor="r", ...
        EdgeColor="none");
    hrect = [hrect; hr];
elseif state(ct)==1
    hr = rectangle( ...
        ax{stepCt}, ...
        Position=[x-2.5,y-2.5,5,5], ...
        Curvature=[1 1], ...
        FaceColor="b", ...
        EdgeColor="none");
    hrect = [hrect; hr];
end
end
end
drawnow();
end
```

See Also

Functions

`validateEnvironment` | `getObservationInfo` | `getActionInfo`

Objects

`rlTurnBasedFunctionEnv` | `rlMultiAgentFunctionEnv` | `rlFunctionEnv` | `rlNumericSpec` | `rlFiniteSetSpec`

Related Examples

- “Create Custom Environment Using Step and Reset Functions” on page 2-94

More About

- “Reinforcement Learning Environments” on page 2-2

Water Distribution System Scheduling Using Reinforcement Learning

This example shows how to learn an optimal pump scheduling policy for a water distribution system using reinforcement learning (RL).

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

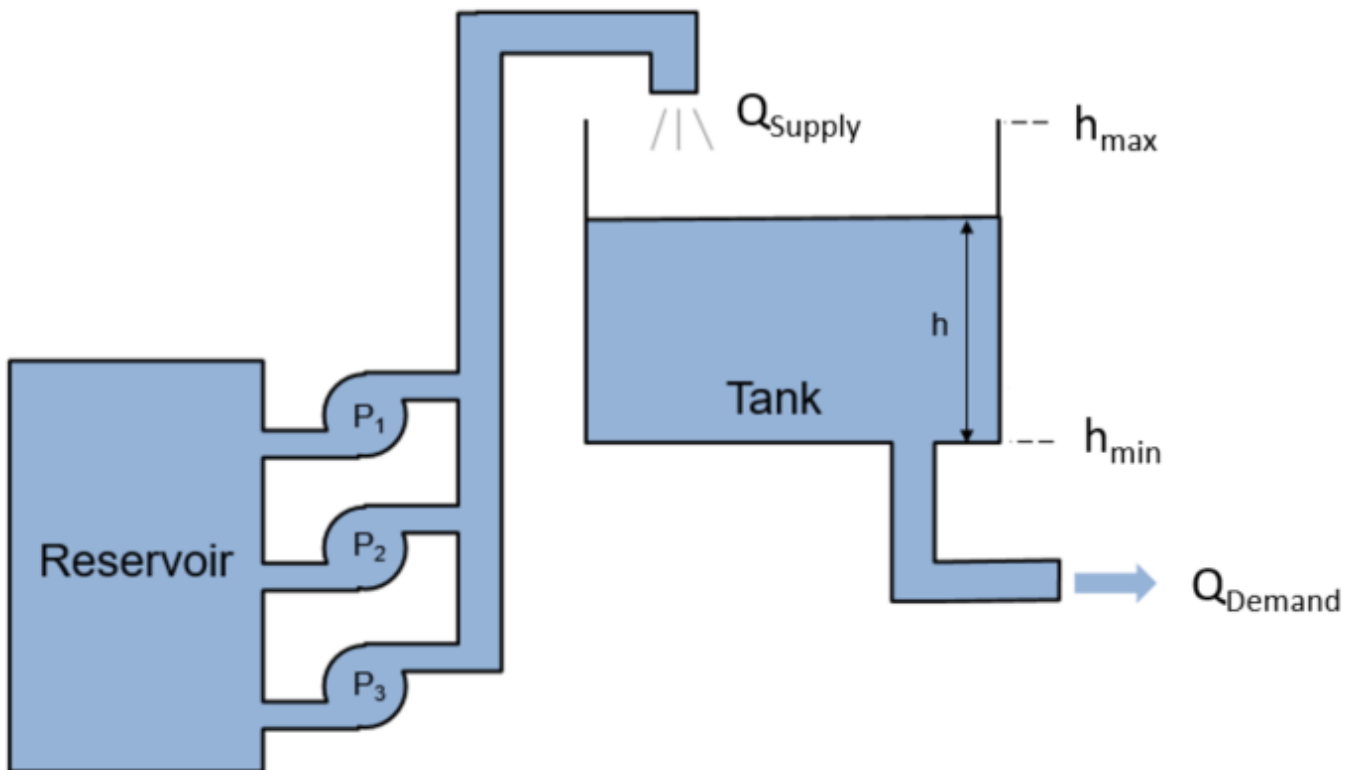
Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Water Distribution System

The following figure shows a water distribution system.



Here:

- Q_{Supply} is the amount of water supplied to the water tank from a reservoir.
- Q_{Demand} is the amount of water flowing out of the tank to satisfy usage demand.

The objective of the reinforcement learning agent is to schedule the number of pumps running to both minimize the energy usage of the system and satisfy the usage demand ($h > 0$). The dynamics of the tank system are governed by the following equation.

$$A \frac{dh}{dt} = Q_{\text{Supply}}(t) - Q_{\text{Demand}}(t)$$

Here, $A = 40 \text{ m}^2$ and $h_{\text{max}} = 7\text{m}$. The demand over a 24 hour period is a function of time given as

$$Q_{\text{Demand}}(t) = \mu(t) + \eta(t)$$

where $\mu(t)$ is the expected demand and $\eta(t)$ represents the demand uncertainty, which is sampled from a uniform random distribution.

The supply is determined by the number of pumps running, $a \in \{0, 1, 2, 3\}$ according to the following mapping.

$$Q_{\text{Supply}}(t) = Q(a) = \begin{cases} 0 & a = 0 \\ 164 & a = 1 \\ 279 & a = 2 \\ 344 & a = 3 \end{cases} \frac{\text{cm}}{h}$$

To simplify the problem, power consumption is defined as the number of pumps running, a .

The following function is the reward for this environment. To avoid overflowing or emptying the tank, an additional cost is added if the water height is close to the maximum or minimum water levels, h_{max} or h_{min} , respectively.

$$r(h, a) = -10(h \geq (h_{\text{max}} - 0.1)) - 10(h \leq 0.1) - a$$

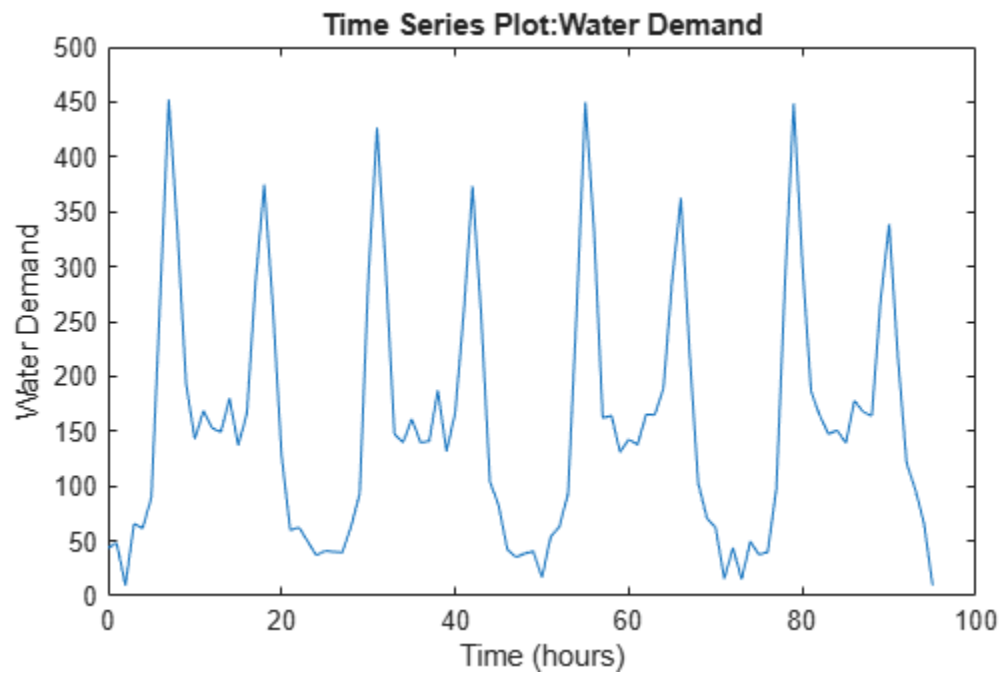
Generate Demand Profile

To generate and a water demand profile based on the number of days considered, use `generateWaterDemand` on page 8-369 defined at the end of this example.

```
num_days = 4; % Number of days
[WaterDemand, T_max] = generateWaterDemand(num_days);
```

View the demand profile.

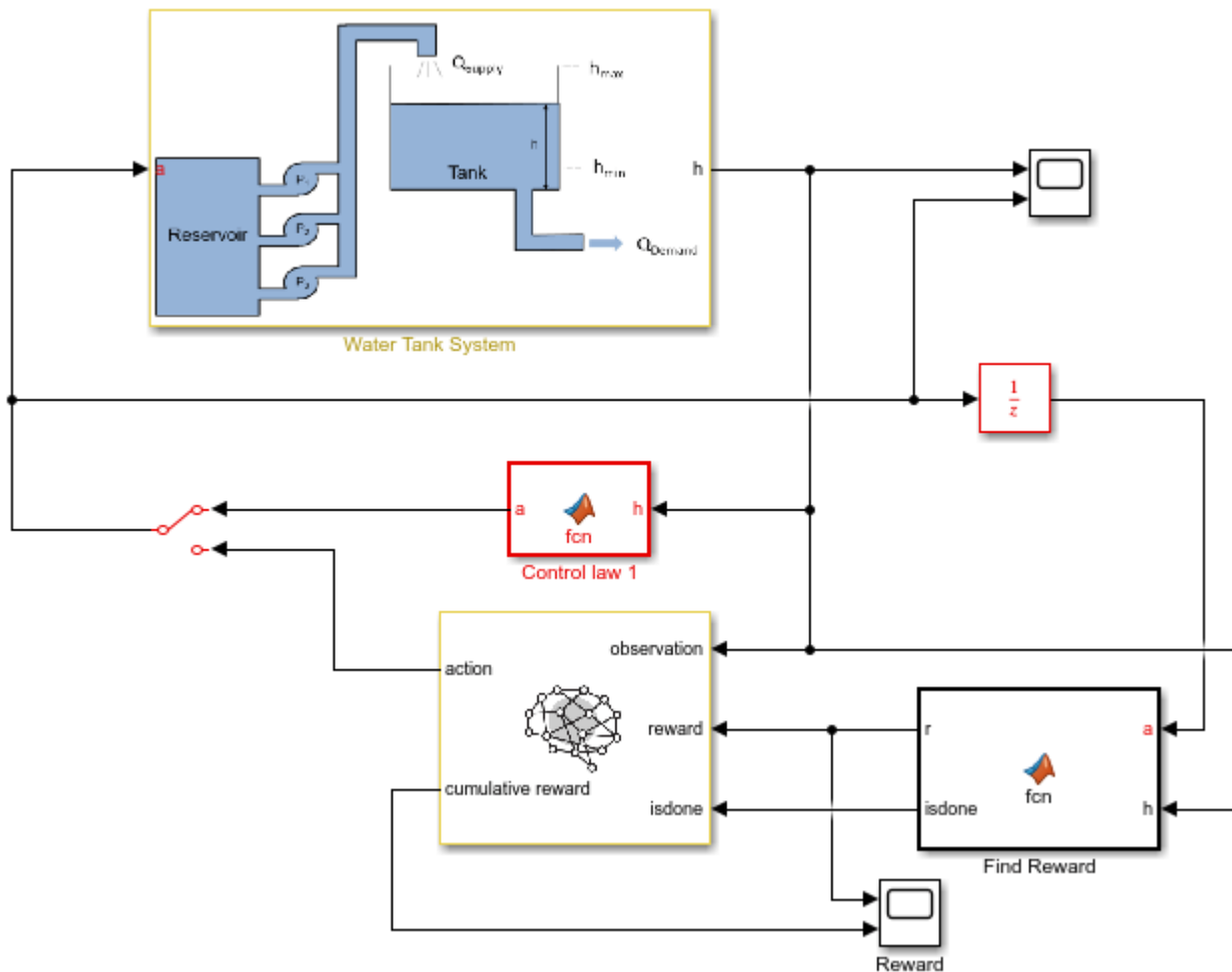
```
plot(WaterDemand)
```



Open and Configure Model

Open the distribution system Simulink® model.

```
mdl = "watertankscheduling";  
open_system(mdl)
```



In addition to the reinforcement learning agent, a simple baseline controller is defined in the Control law MATLAB Function block. This controller activates a certain number of pumps depending on the water level.

Specify the initial water height.

```
h0 = 3; % m
```

Specify the model parameters.

```
SampleTime = 0.2;
H_max = 7; % Max tank height (m)
A_tank = 40; % Area of tank (m^2)
```

Create Environment Object

To create an environment object for the Simulink model, first define the action and observation specifications, `actInfo` and `obsInfo`, respectively. The agent action is the selected number of pumps. The agent observation is the water height, which is measured as a continuous-time signal.


```
actInfo = rlFiniteSetSpec([0,1,2,3]);
obsInfo = rlNumericSpec([1,1]);
```

Create the environment object.

```
env = rlSimulinkEnv mdl, mdl+ "/RL Agent", obsInfo, actInfo);
```

Initialize the random seed for each simulation.

```
simulationSeed(0);
```

Specify the custom reset function `localResetFcn` on page 8-370, which is defined at the end of this example, that randomizes the initial water height and the water demand. Doing so allows the agent to be trained on different initial water levels and water demand functions for each episode.

```
env.ResetFcn = @localResetFcn;
```

Create DQN Agent

DQN agents use a parameterized Q-value function approximator to estimate the value of the policy. Because DQN agents have a discrete action space, you have the option to create a vector (that is, multi-output) Q-value function critic, which is generally more efficient than a comparable single-output critic.

A vector Q-value function takes only the observation as input and returns as output a single vector with as many elements as the number of possible actions. The value of each output element represents the expected discounted cumulative long-term reward when an agent starts from the state corresponding to the given observation and executes the action corresponding to the element number (and follows a given policy afterwards).

The network parameters are initialized randomly. Ensure reproducibility of the section by fixing the seed of the random generator.

```
rng(0, "twister");
```

To model the parameterized Q-value function within the critic, use a non-recurrent neural network.

```
layers = [
    featureInputLayer(obsInfo.Dimension(1))
    fullyConnectedLayer(32)
    reluLayer
    fullyConnectedLayer(32)
    reluLayer
    fullyConnectedLayer(32)
    reluLayer
    fullyConnectedLayer(numel(actInfo.Elements))
];
```

Create to a `dlnetwork` object and display the number of parameters.

```
dnn = dlnetwork(layers);
summary(dnn)
```

```
    Initialized: true
```

```
    Number of learnables: 2.3k
```

```
Inputs:
  1   'input'   1 features
```

Create a critic using `rlVectorQValueFunction` using the defined deep neural network and the environment specifications.

```
critic = rlVectorQValueFunction(dnn,obsInfo,actInfo);
```

Create DQN Agent with Custom Network

Specify training options for the critic and the actor using `rlOptimizerOptions`.

```
criticOpts = rlOptimizerOptions(LearnRate=1e-2,GradientThreshold=10);
```

Specify the agent options using `rlDQNAgentOptions`. If you are using an LSTM network, set the sequence length to 20.

```
opt = rlDQNAgentOptions(SampleTime=SampleTime);
opt.DiscountFactor = 0.995;
opt.ExperienceBufferLength = 1e6;
opt.EpsilonGreedyExploration.EpsilonDecay = 1e-5;
opt.EpsilonGreedyExploration.EpsilonMin = .02;
opt.MinibatchSize = 128;
opt.TargetSmoothFactor = 1e-2;
```

Include the training options for the critic.

```
opt.CriticOptimizerOptions = criticOpts;
```

Create the agent using the critic and the options object.

```
agent = rlDQNAgent(critic,opt);
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options.

- Run training for 500 episodes, with each episode lasting at `ceil(T_max/Ts)` time steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option).
- Specify the averaging window length of 100 for the episode rewards.

Specify the options for training using an `rlTrainingOptions` object.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=500, ...
    MaxStepsPerEpisode=ceil(T_max/SampleTime), ...
    Verbose=false, ...
    Plots="training-progress", ...
    StopTrainingCriteria="none", ...
    ScoreAveragingWindowLength=100);
```

Fix the random stream for reproducibility.

```
rng(0,"twister");
```

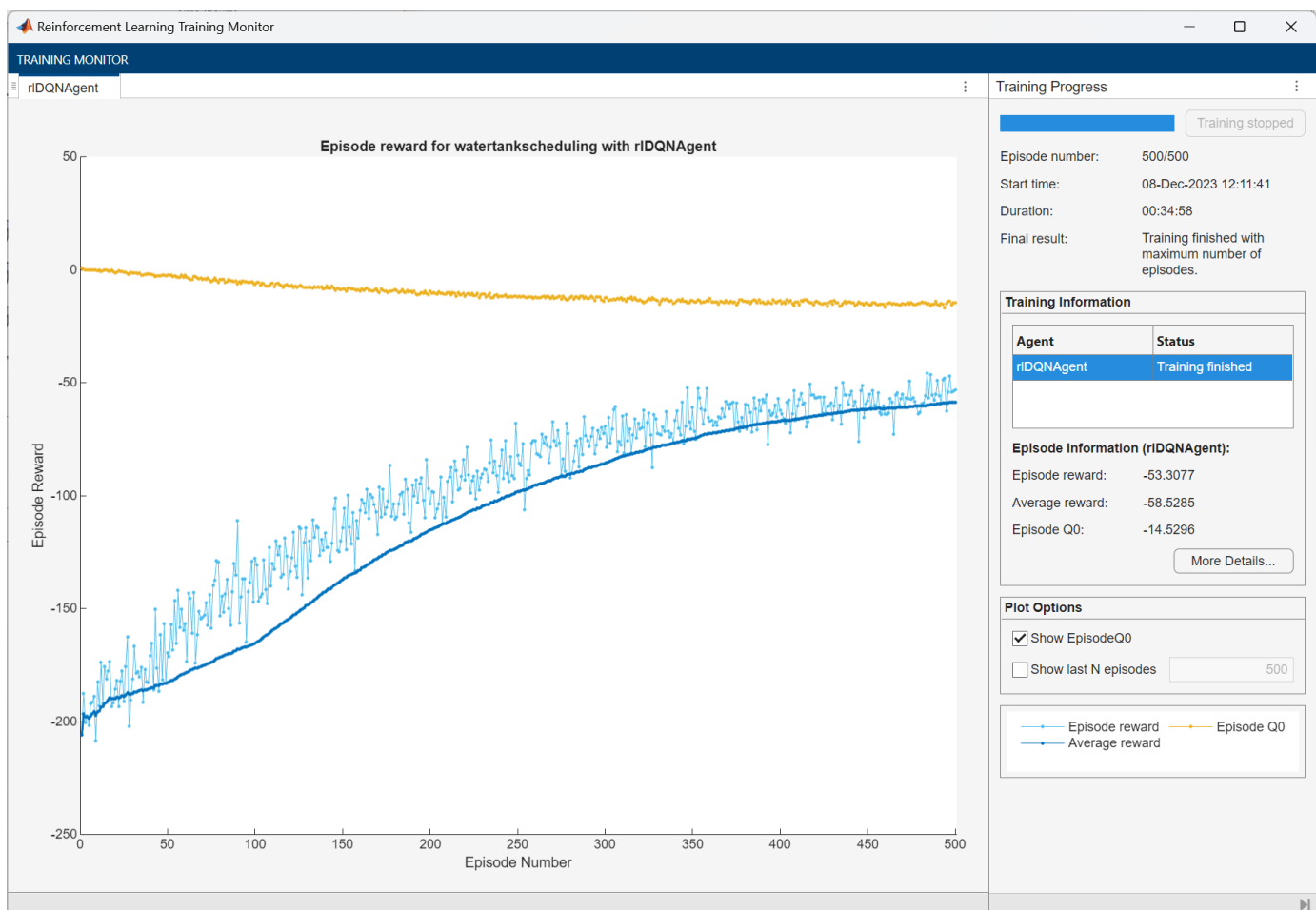
Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several hours to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = false ;
if doTraining

    % Connect the RL Agent block by toggling
    % the Manual Switch block
    set_param mdl+"/Manual Switch","sw","0");

    % Train the agent.
    trainingStats = train(agent,env,trainOpts);
else
    % Load the pretrained agent for the example.
    load("SimulinkWaterDistributionDQN.mat","agent")
end
```

The following figure shows the training progress.



Simulate DQN Agent

To validate the performance of the trained agent, simulate it in the water-tank environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

To simulate the agent performance, connect the RL Agent block by toggling the manual switch block.

```
set_param mdl+"/Manual Switch","sw","0");
```

Set the maximum number of steps for each simulation and the number of simulations. For this example, run 30 simulations. The environment reset function sets a different initial water height and water demand are different in each simulation.

```
NumSimulations = 30;  
simOptions = rlSimulationOptions( ...  
    MaxSteps=T_max/SampleTime, ...  
    NumSimulations=NumSimulations);
```

To compare the agent with the baseline controller under the same conditions, reset the initial random seed used in the environment reset function.

```
simulationSeed(0);
```

By default, the agent uses a greedy (hence deterministic) policy in simulation. To use the exploratory policy instead, set the `UseExplorationPolicy` agent property to `true`.

Simulate the agent within the environment.

```
experienceDQN = sim(env,agent,simOptions);
```

Simulate Baseline Controller

To compare DQN agent with the baseline controller, you must simulate the baseline controller using the same simulation options and initial random seed for the reset function.

Enable the baseline controller.

```
set_param mdl+"/Manual Switch","sw","1");
```

To compare the agent with the baseline controller under the same conditions, reset the random seed used in the environment reset function.

```
simulationSeed(0);
```

Simulate the baseline controller against the environment.

```
experienceBaseline = sim(env,agent,simOptions);
```

Compare DQN Agent with Baseline Controller

Initialize cumulative reward result vectors for both the agent and baseline controller.

```
resultVectorDQN = zeros(NumSimulations, 1);  
resultVectorBaseline = zeros(NumSimulations,1);
```

Calculate cumulative rewards for both the agent and the baseline controller.

```
for ct = 1:NumSimulations  
    resultVectorDQN(ct) = sum(experienceDQN(ct).Reward);
```

```

    resultVectorBaseline(ct) = sum(experienceBaseline(ct).Reward);
end

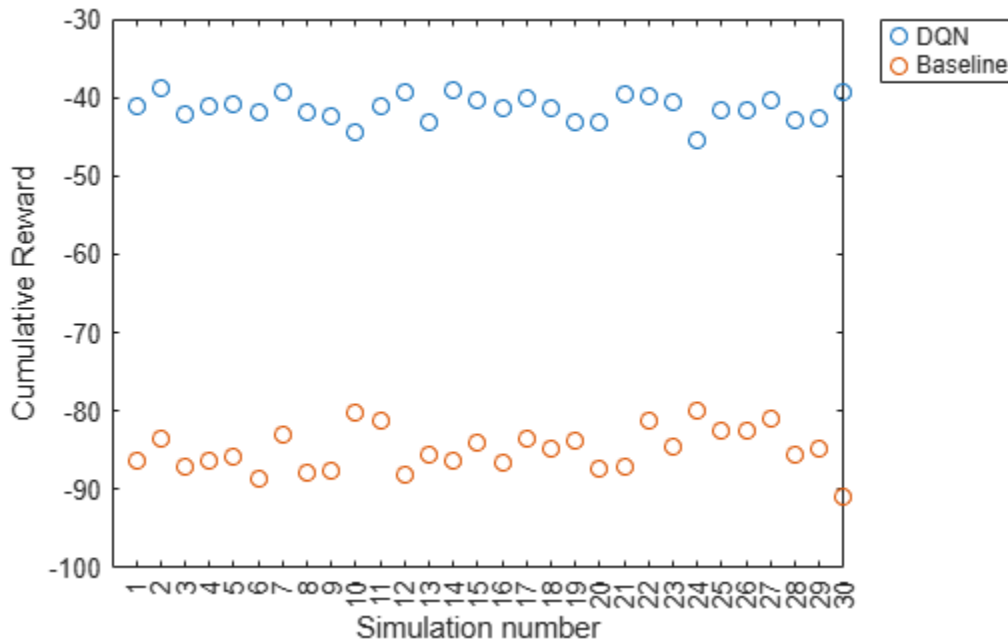
```

Plot the cumulative rewards.

```

plot([resultVectorDQN resultVectorBaseline],"o")
set(gca,"xtick",1:NumSimulations)
xlabel("Simulation number")
ylabel("Cumulative Reward")
legend("DQN","Baseline","Location","NorthEastOutside")

```



The cumulative reward obtained by the agent is consistently around -40. This value is much greater than the average reward obtained by the baseline controller. Therefore, the DQN agent consistently outperforms the baseline controller in terms of energy savings.

Restore the random number stream using the information stored in previousRngState.

```
rng(previousRngState);
```

Local Functions

Water Demand Function

```

function [WaterDemand,T_max] = generateWaterDemand(num_days)

    t = 0:(num_days*24)-1; % hr
    T_max = t(end);

    Demand_mean = [28, 28, 28, 45, 55, 110, ...
                    280, 450, 310, 170, 160, 145, ...
                    130, 150, 165, 155, 170, 265, ...
                    360, 240, 120, 83, 45, 28 ]'; % m^3/hr

```

```
Demand = repmat(Demand_mean,1,num_days);
Demand = Demand(:);

% Add noise to demand
a = -25; % m^3/hr
b = 25; % m^3/hr
Demand_noise = a + (b-a).*rand(numel(Demand),1);

WaterDemand = timeseries(Demand + Demand_noise,t);
WaterDemand.Name = "Water Demand";
WaterDemand.TimeInfo.Units = "hours";
end
```

Reset Function

The `sim` function calls the reset function at the start of each simulation episode, and the `train` function calls it at the start of each training episode. The reset function takes as input, and returns as output, a `Simulink.SimulationInput` (Simulink) object. The output object specifies temporary changes applied to model, which are then discarded when the simulation or training completes. For this example, the reset function uses `setBlockParameter` (Simulink) to specify random values of the reference signal and the initial water height. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
function in = localResetFcn(in)

% Use a persistent random seed value to evaluate the agent
% and the baseline controller under the same conditions.

randomSeed = simulationSeed(simulationSeed() + 1);
rng(randomSeed);

% Randomize water demand.
num_days = 4;
H_max = 7;
[WaterDemand,~] = generateWaterDemand(num_days);
assignin("base","WaterDemand",WaterDemand)

% Randomize initial height.
h0 = 3*randn;
while h0 <= 0 || h0 >= H_max
    h0 = 3*randn;
end

blk = "watertankscheduling/Water Tank System/Initial Water Height";
in = setBlockParameter(in,blk,"Value",num2str(h0));

end
function s = simulationSeed(u)
    persistent x;
    if isempty(x)
        x = 0;
    end
    if nargin
        x = u;
    end
end
```

```
        s = x;  
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlOptimizerOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3
- “Optimizing Queue Selection Strategies Using Reinforcement Learning” on page 8-372

Optimizing Queue Selection Strategies Using Reinforcement Learning

In this example, you train a deep Q-learning network (DQN) reinforcement learning agent to optimize a queue selection strategy. Specifically, the agent minimizes customer waiting time by effectively routing and scheduling customers through a multi-queue checkout system. You then compare the agent performance to a baseline heuristic. The discrete-event system is modeled in Simulink® using SimEvents®.

For more information on DQN agents, see “Deep Q-Network (DQN) Agent” on page 3-32. For an example that shows how queuing systems can be modeled in SimEvents and a comparison of different queuing strategies, see “Comparing Queuing Strategies” (SimEvents).

Fix Random Number Stream for Reproducibility

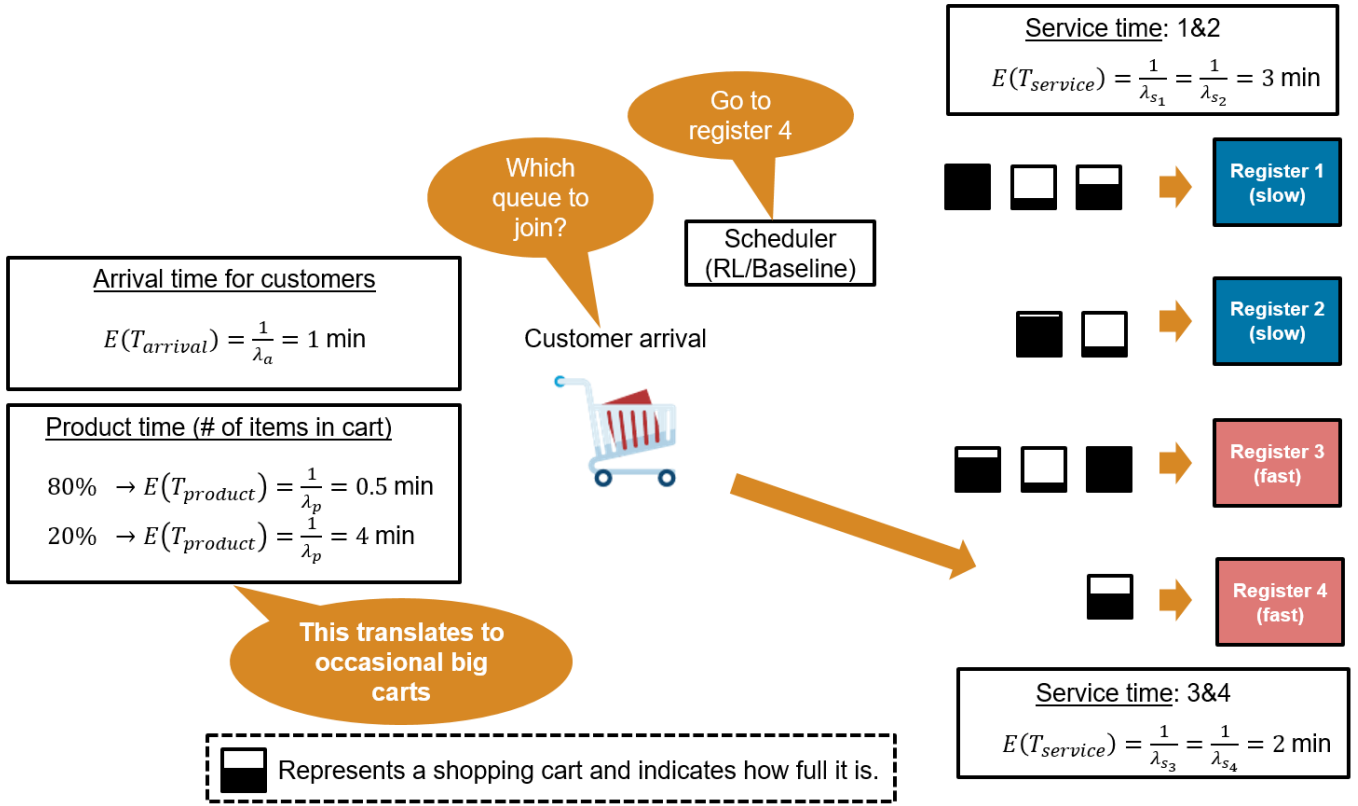
The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing the results. For more information, see “Results Reproducibility” on page 5-10.

Fix the random number stream with seed 0 and random number algorithm Mersenne Twister. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister");
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

Environment Setup



The system consists of four checkout registers, and the goal is to minimize customer waiting time by selecting the best queue for each arriving customer. The environment is as follows:

- **Customer Arrival:** Customers arrive at the checkout area with exponentially distributed inter-arrival times. The mean arrival time is 1 minute. Because the arrival rate parameter λ_a is 1 customer per minute, the mean customer arrival time is $= \frac{1}{\lambda_a} = 1 \text{ minute}$:

$$E(T_{arrival}) = \frac{1}{\lambda_a} = 1 \text{ minute}$$

- **Product Time (Cart Size):** At generation, each customer is assigned a cart product time that represents the time needed to scan their items. The product time is drawn from an exponential distribution with two possible means: 80% of customers have a small cart with a mean of 0.5 min, and 20% have a large cart with a mean product time of 4 min.

$$E(T_{product}) = \frac{1}{\lambda_{ps}} = 0.5 \text{ minutes (small carts, 80% of the time)}$$

$$E(T_{product}) = \frac{1}{\lambda_{pb}} = 4 \text{ minutes (big carts, 20% of the time)}$$

- **Service Time (Registers):** The checkout area consists of four registers, each with its own dedicated queue. Registers 1 and 2 are classified as slow, with exponentially distributed service times averaging 3 min, while registers 3 and 4 are fast, with service times averaging 2 min.

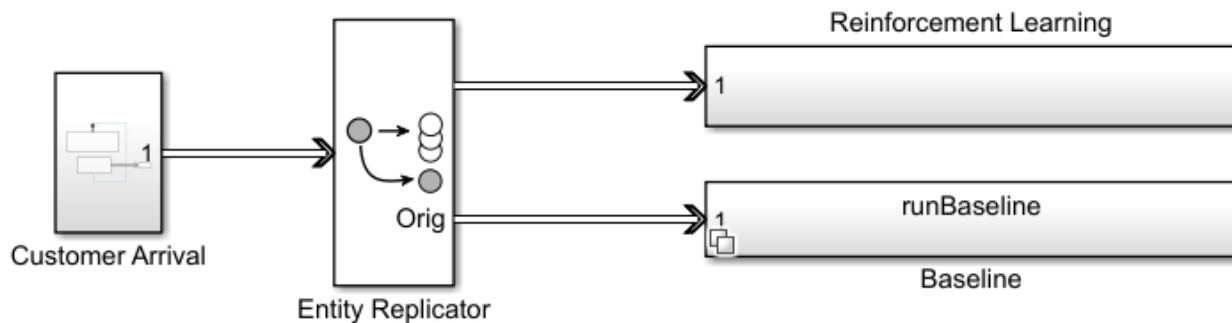
$$E(T_{\text{service}}) = \frac{1}{\lambda_{s1}} = \frac{1}{\lambda_{s2}} = 3 \text{ minutes (slow lanes)}$$

$$E(T_{\text{service}}) = \frac{1}{\lambda_{s3}} = \frac{1}{\lambda_{s4}} = 2 \text{ minutes (fast lanes)}$$

The total time a customer spends at a register is equal to the service time (which is register specific) plus the product time (which is cart specific). This means that a slow register with a big cart will take much longer than a fast register with a small cart.

Open the model.

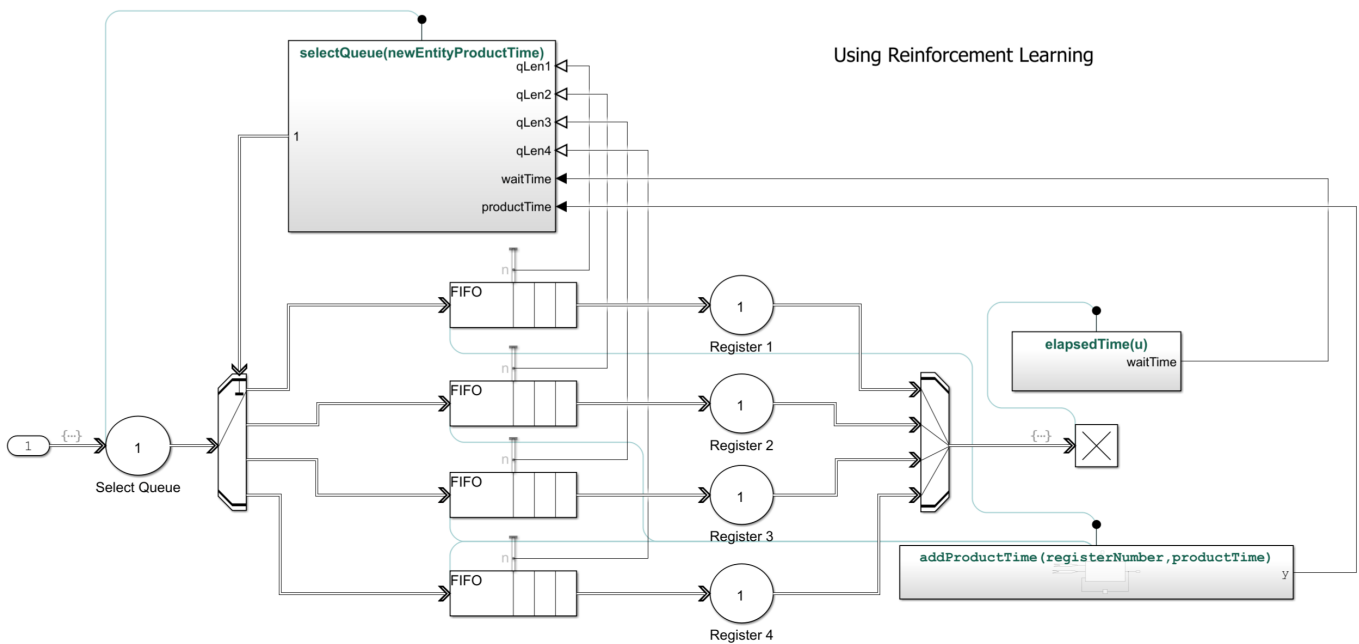
```
mdl = "rlQueuingStrategiesModel";
open_system(mdl)
```



In this example, you model customer arrivals using the SimEvents Entity Generator block; see Entity Generator (SimEvents) for more information. The Entity Generator block assigns each customer a product time (cart size) at generation. To enable comparison between the reinforcement learning and the baseline strategies, the Entity Replicator block clones each generated customer, and then sends the same customer through both the RL and baseline strategies block. Each of the two strategy block routes the customer to one of the four queues. When a customer joins a queue, the corresponding queue block assigns the appropriate service time distribution based on the register type.

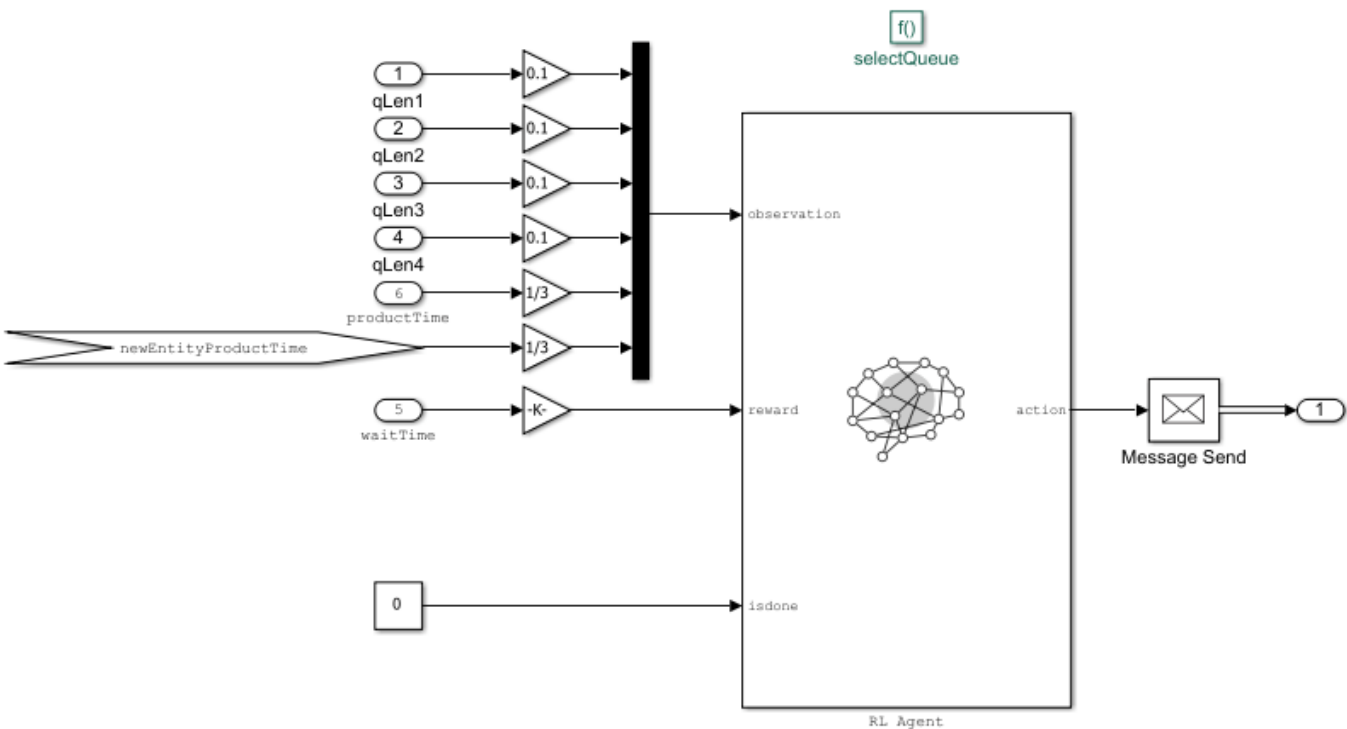
Open the reinforcement learning subsystem.

```
open_system(mdl+"/Reinforcement Learning")
```



Open the Queue Selector block inside the Reinforcement Learning subsystem.

```
open_system mdl+"/Reinforcement Learning/Queue Selector")
```



In this Simulink model:

- The RL Agent block is inside the Queue Selector block.
- The environment provides the following 9 observations to the RL Agent block: queue lengths for all 4 registers, total product time in each queue (proxy for number of items in carts), current customer's product time (cart size). Note that the agent does not have any prior knowledge of which registers are fast and which are slow.
- The agent has a discrete action space consisting of the elements 1,2,3 and 4 corresponding to the which queue the arriving customer should join.
- The reward is computed as the negative of the waiting time. The waiting time is measured from entity generation until the customer completes checkout at the register.

Baseline strategy: To evaluate performance, the RL agent is compared against a baseline heuristic strategy. The baseline aims to minimize estimated future workload by selecting the queue with the smallest product between the number of people waiting in line and the cumulative product time. It is implemented as a variant subsystem that remains disabled during training and is enabled during the simulation phase for comparison. You enable this strategy using the `runBaseline` workspace variable.

Create Environment Object

Define the observation specification `obsInfo` and the action specification `actInfo`.

```
obsInfo = rlNumericSpec([9 1]);  
actInfo = rlFiniteSetSpec(1:4);
```

Use `rlSimulinkEnv` to create the environment object from the Simulink model, passing to it the path of the RL Agent block and the observation and action specification.

```
env = rlSimulinkEnv mdl, mdl + "/Reinforcement Learning/Queue Selector/RL Agent", obsInfo, actInfo)
```

For more information about Simulink environments, see “Create Custom Simulink Environments” on page 2-110.

Create DQN Agent

When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0, "twister");
```

Create an agent initialization object to initialize the critic the network with a hidden layer size of 128.

```
initOptions = rlAgentInitializationOptions("NumHiddenUnit", 128);
```

Create the DQN agent using the observation and action specifications and initialization options that you defined.

```
agent = rlDQNAgent(obsInfo, actInfo, initOptions);
```

Specify the agent options for training:

- Specify the sample time as -1. Since this is an event-based simulation, you place the RL Agent block inside a conditionally executed subsystem. A `SampleTime` of -1 causes the block to inherit the sample time of its parent subsystem. For more information, see “Conditionally Executed Subsystems Overview” (Simulink).

- Specify the discount factor of 0.9995. In this problem, the reward (negative customer waiting time) is delayed until service completes, so the agent must consider a longer sequence of actions before seeing the outcome. A high discount factor of 0.9995 ensures that the agent puts larger weight on future outcomes, enabling it to learn long-term strategies rather than making short-sighted decisions.
- Specify the critic learning rate as 1e-3. A large learning rate may lead to divergent behaviors, while a low value may require learning rate might cause training to take longer.
- Use a gradient threshold of 1 to clip the gradients. Clipping the gradients can improve training stability.
- Use mini-batches of 128 experiences. Smaller mini-batches are computationally efficient, but might introduce variance in training. Larger batch sizes can make the training more stable but require more memory.
- Specify the experience buffer length as 100 and number of epochs as 5. At each learning step, the agent samples up to `MaxMiniBatchPerEpoch` unique mini-batches per epoch.

```
agent.AgentOptions.SampleTime = -1;
agent.AgentOptions.DiscountFactor = 0.9995;
agent.AgentOptions.CriticOptimizerOptions.LearnRate = 1e-3;
agent.AgentOptions.CriticOptimizerOptions.GradientThreshold = 1;
agent.AgentOptions.MinibatchSize = 128;
agent.AgentOptions.ExperienceBufferLength = 1e6;
agent.AgentOptions.NumEpoch = 5;
```

A DQN agent uses the epsilon-greedy algorithm to explore the action space during training. Specify a decay rate of 1e-4 for the epsilon value to gradually decay during training. The epsilon value is the probability that the agent will choose a random action instead of exploiting its current knowledge to maximize the reward. The gradual decay of the epsilon value during training promotes exploration toward the beginning of training when the agent does not have a good policy, and exploitation toward the end of training when the agent has learned the optimal policy.

```
agent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-4;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonMin = 0.01;
```

For more information, see `rldQNAgentOptions`.

Train DQN Agent

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Create an agent evaluator to evaluate the performance of the greedy policy every 25 training episodes.

```
rEval = rlEvaluator("EvaluationFrequency",25,"NumEpisodes",1,"RandomSeeds",0);
```

To train the agent, first specify the training options. For this example, use the following options.

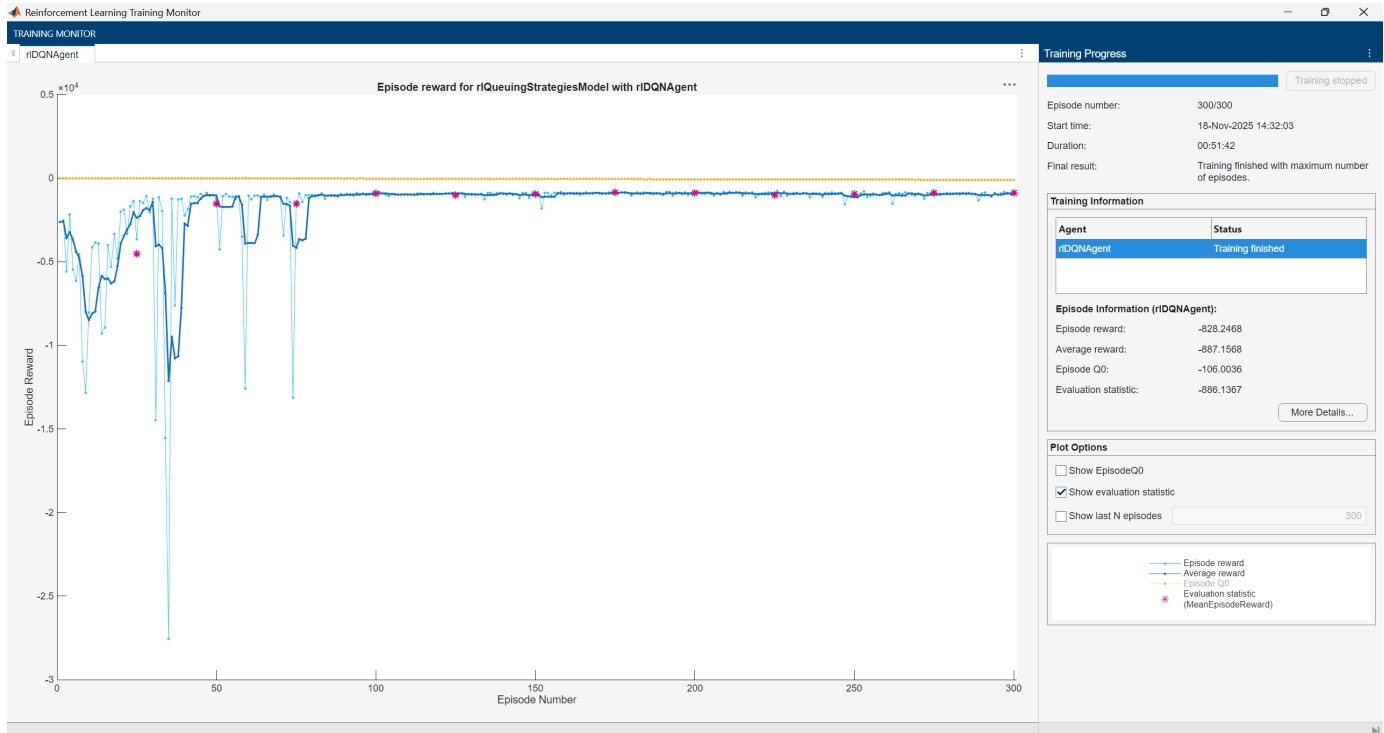
- Run each training for a maximum of 300 episodes, with each episode lasting 1000 steps.
- Display the training progress in the Reinforcement Learning Training Monitor dialog box (set the `Plots` option) and disable the command line display (set the `Verbose` option to `false`).
- Set `StopTrainingCriteria` to "None" so that the training stops only when the number of episodes reaches `MaxEpisodes`.

For more information on training options, see `rlTrainingOptions`.

```
% Training options
trainOpts = rlTrainingOptions(...
    MaxEpisodes=300, ...
    MaxStepsPerEpisode=1000, ...
    Verbose=false, ...
    Plots="training-progress",...
    StopTrainingCriteria="EvaluationStatistic", ...
    StopTrainingValue=0);
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, you can load a pretrained agent by setting `doTraining` to false. To train the agent yourself, set `doTraining` to true. The baseline controller is not needed during training, so set `runBaseline` to false.

```
doTraining = false;
runBaseline = false;
if doTraining
    % Train the agent.
    trainingStats = train(agent,env,trainOpts,Evaluator=rlEval);
else
    % Load the pretrained agent for the example.
    load("trainedAgentQueueingStrategies.mat","agent");
end
```



The training converges after 100 episodes.

Simulate DQN Agent

Fix the random stream for reproducibility.

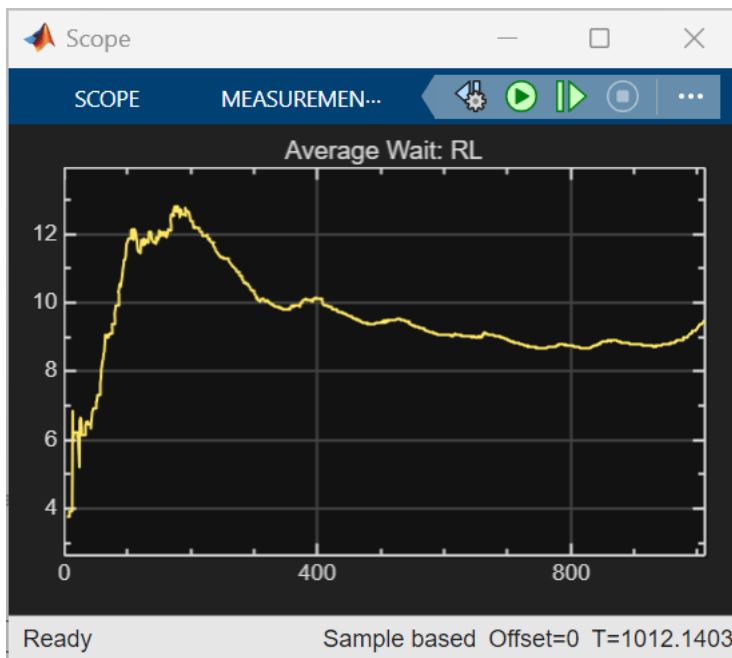
```
rng(0, "twister");
```

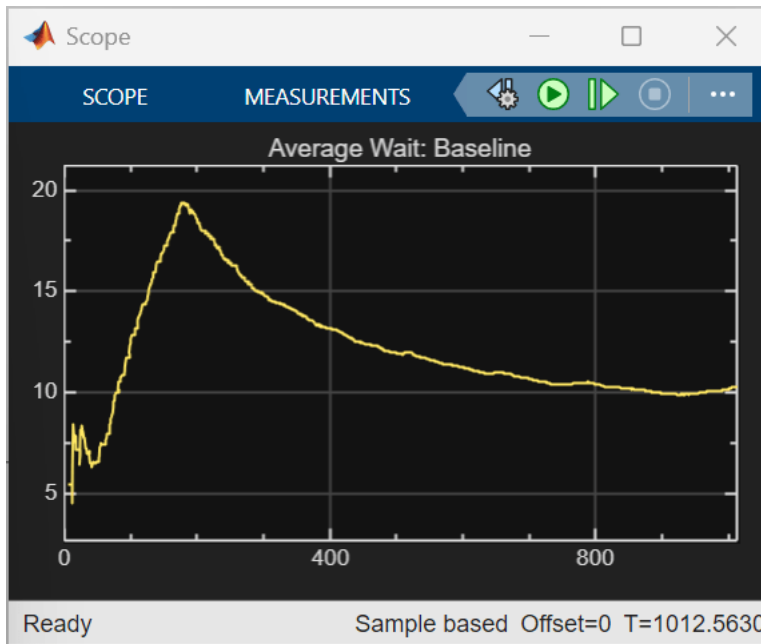
Set `runBaseline` to `true` to enable the baseline controller and compare its performance against the RL agent.

```
runBaseline =  ;
```

To validate the performance of the trained agent, simulate the trained agent with the environment. For more information on agent simulation, see `rlSimulationOptions` and `sim`.

```
agent.UseExplorationPolicy = false;
simOptions = rlSimulationOptions(MaxSteps=1000);
experience = sim(env, agent, simOptions);
```





The trained RL agent achieves a lower total average waiting time of 9468 minutes, compared to 12150 minutes for the baseline strategy, showing better overall efficiency. RL also converges faster and maintains a lower average wait after the initial peak, while the baseline strategy exhibits higher peaks and slower recovery. These results reflect the current reward design. Alternative reward functions could combine minimizing average waiting time with constraints on maximum waiting time or queue length.

Understanding the Learned Policy

By comparing the agent's decision with an intuitive heuristic (different from the baseline strategy), you can learn about its priorities, such as whether it minimizes immediate waiting time, protects fast lanes from large carts, or balances long-term efficiency. This is not an exhaustive validation, but a way to understand the strategies the policy has internalized.

To interpret what the RL agent has learned, examine its choices in representative scenarios. Each scenario is described by nine observations:

- Number of people in each of the four queues
- Total product time in each queue
- Product time of the newly arrived customer

Define observations related to various scenarios.

```
obsScenarios = [
    0, 0, 0, 0, 0, 0, 0, 0, 1;
    0, 0, 0, 0, 0, 0, 0, 0, 8;
    2, 1, 0, 0, 15, 10, 0, 0, 8;
    1, 0, 0, 1, 5, 0, 0, 6, 0.5;
    1, 1, 2, 2, 5, 5, 1.5, 0.75, 0.5;
    1, 1, 2, 2, 4, 4, 8, 8, 0.5 ]';
```

Compute the action the trained agent takes in each of six different scenarios. In the Simulink model, observations are scaled before use. Therefore, scale the observations using the local function

`scaleObservations` on page 8-382 (defined at the end of the example) before feeding them to the trained agent to calculate the corresponding action.

```
actScenarios = getAction(agent,{scaleObservations(obsScenarios)});
```

Display the actions related to the six observations.

```
squeeze(actScenarios{:})'
```

```
ans = 1×6
```

```
     3     2     1     3     4     2
```

For each scenario, observations and actions are captured in the following table.

Scenario	Number of People in Each Queue	Total Product Time in Each Queue	New Customer's Product Time (Cart Size)	Intent/What to Look For	Expected Preference
1	0, 0, 0, 0	0, 0, 0, 0	1	Small cart, all queues empty	Send to fast lane (3 or 4)
2	0, 0, 0, 0	0, 0, 0, 0	8	Big cart, all queues empty	Send to slow lane (1 or 2)
3	2, 1, 0, 0	15, 10, 0, 0	8	Big cart, slow lanes are occupied and fast lanes are empty	Prefer slow lane (1 or 2) to avoid clogging fast lanes
4	1, 0, 0, 1	5, 0, 0, 6	0.5	Small cart, lane 4 has work	Send to lane 3 (fast lane)
5	1, 1, 2, 2	5, 5, 1.5, 0.75	0.5	All queues have work, small cart	Prefer fast lane with less work (3 or 4)
6	1, 1, 2, 2	4, 4, 8, 8	0.5	Small cart, fast lanes heavily loaded	Prefer slow lane (1 or 2) to minimize wait

Insights

- **Scenario 1:** The agent sends a small cart to a fast lane when all queues are empty, which suggests that it values speed when there is no congestion.
- **Scenario 2:** For a large cart with all queues empty, the agent chooses a slow lane, which might mean that the agent tries to keep fast lanes available for smaller carts.
- **Scenario 3:** When slow lanes already have work and a large cart arrives, the agent still picks a slow lane, which again might mean that the it tries to protect fast lanes from heavy loads.
- **Scenario 4:** With a small cart and lane 4 busy, the agent selects lane 3, which suggests that it considers both lane type and current load.
- **Scenario 5:** When all queues have work and a small cart arrives, the agent prefers a fast lane, which could mean that it prioritizes minimizing expected waiting time even under congestion.
- **Scenario 6:** When all queues have work and a small cart arrives, the agent chooses a slow lane instead of a fast lane. This suggests that the policy considers overall queue congestion and may route a small cart to a slower register if the fast lanes are heavily loaded, demonstrating a balance between lane speed and current load.

This analysis reveals that the policy has learned sensible strategies, such as sending small carts to fast lanes, large carts to slow lanes, and balancing queue load when possible. The agent also seems to have identified which registers are fast and which are slow from experience, without being explicitly told this information.

Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Scale Observations Function

The `scaleObservations` function scales the observations.

```
function scaledObs = scaleObservations(obsMatrix)
    % obsMatrix: 9xN matrix where each column is [q1 q2 q3 q4 p1 p2 p3 p4 pc]
    % First 4 rows scaled by 0.1
    % Next 5 rows scaled by 1/3

    [rows,~] = size(obsMatrix);
    if rows ~= 9
        error('Input must have 9 rows.');
```

```
    end

    scaledObs = obsMatrix; % copy original
    scaledObs(1:4,:) = obsMatrix(1:4,:)*0.1; % scale first 4 rows
    scaledObs(5:9,:) = obsMatrix(5:9,:)*(1/3); % scale next 5 rows
end
```

See Also

Functions

`train` | `sim` | `rlSimulinkEnv`

Objects

`rlDQNAgent` | `rlDQNAgentOptions` | `rlOptimizerOptions` | `rlTrainingOptions` | `rlSimulationOptions`

Blocks

RL Agent

Related Examples

- “Train Default DQN Agent to Balance Discrete Cart-Pole” on page 5-64

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Deep Q-Network (DQN) Agent” on page 3-32
- “Train Reinforcement Learning Agents” on page 5-3
- “Water Distribution System Scheduling Using Reinforcement Learning” on page 8-361

Identify Vulnerabilities in DC Microgrids

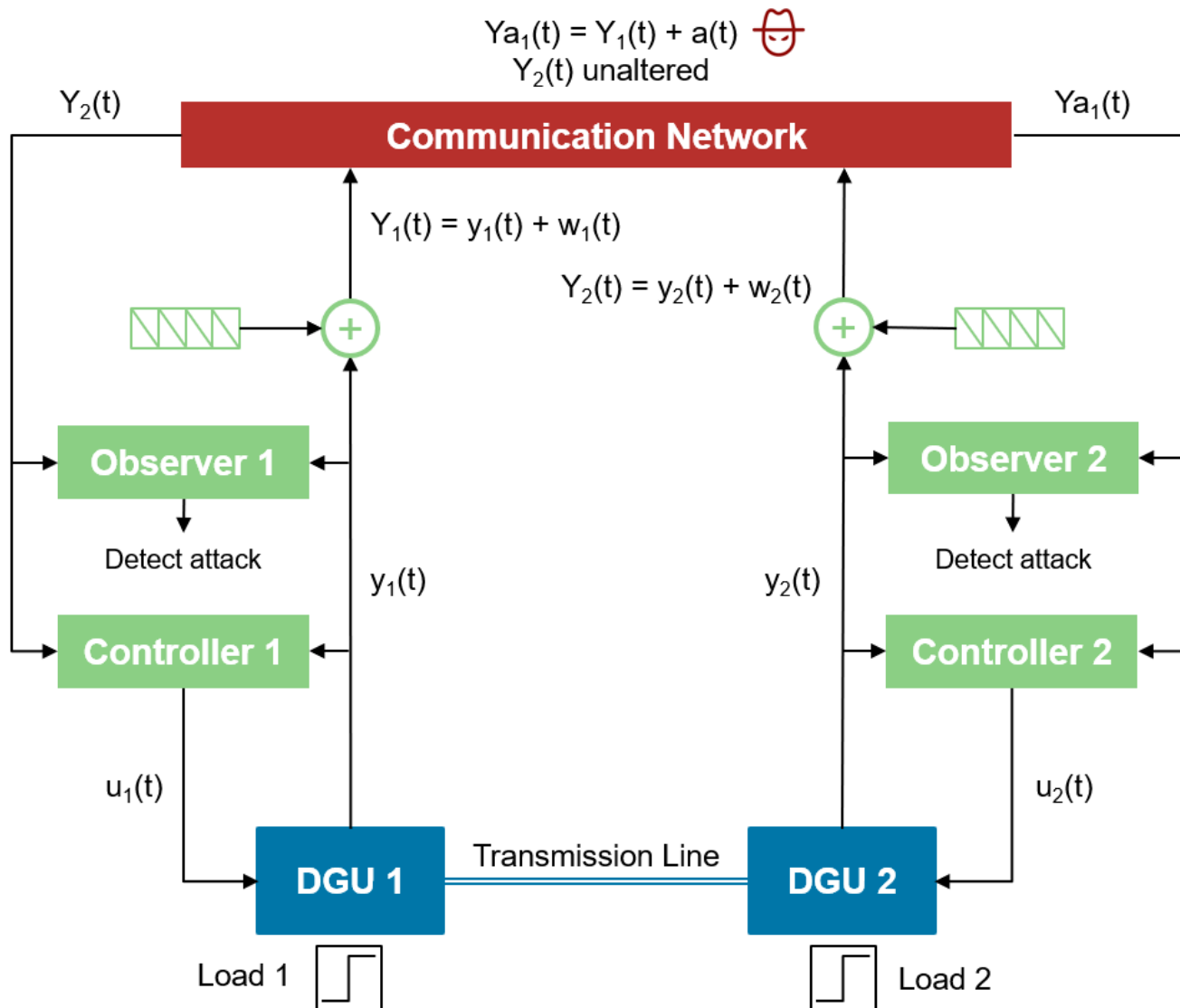
This example demonstrates how to use reinforcement learning to identify vulnerabilities in cyber-physical systems (CPS) such as a DC microgrid. A DC microgrid (DCmG) is an electric power system that distributes direct current (DC) power within a small geographic area through an interconnected system of distributed generation units (DGU).

Introduction

Cyber-physical systems can have multiple units transmitting or receiving data over a communication network. Cyber attacks on the system might falsify the transmitted data to disrupt the physical system or to financially benefit the attacker.

System operators can build security mechanisms to protect against cyber-attacks, as demonstrated in the example “Detect Replay Attacks in DC Microgrids Using Distributed Watermarking” (Control System Toolbox). In that example, the two DGUs regulate their voltages and currents against load profiles based on a current consensus rule. The units transmit their sensor measurements to each other over a communication network for the consensus control. An attacker who has gained access to the network might inject a signal $a(t)$ into the sensor readings to falsify the data. The system is secured by a technique known as watermarking, which adds a periodic signal $w_i(t)$ (unknown to the attacker) into the true sensor readings $y_i(t)$. The local observer monitors the presence (or absence) of the watermark in the received signals (specifically, the observer removes the watermark signal). Because the watermark is known only to the system operators, any manipulation by an attacker alters the expected pattern of the watermark in the system's output, and operators can detect the cyber-attack.

The diagram below shows the overall architecture of the system.



Identification of System Vulnerabilities

When designing a security mechanism, it might be useful to identify and mitigate any vulnerabilities in the design that can be exploited by attackers. Reinforcement learning provides a testing mechanism for identifying such attacks. Specifically, you can train a policy to attack the system and bypass the existing security measures. If the policy successfully performs the attack without triggering the security measures, analyzing the policy actions can lead to key insights and further improvements of the security measures.

Fix Random Number Stream for Reproducibility

The example code might involve computation of random numbers at several stages. Fixing the random number stream at the beginning of some sections in the example code preserves the random number sequence in the section every time you run it, which increases the likelihood of reproducing

the results. For more information, see “Results Reproducibility” on page 5-10. For more information on controlling the seed used for random number generation, see `rng`.

```
previousRngState = rng(0,"twister")

previousRngState = struct with fields:
    Type: 'twister'
    Seed: 0
    State: [625x1 uint32]
```

The output `previousRngState` is a structure that contains information about the previous state of the stream. You will restore the state at the end of the example.

DC Microgrid System

The DC microgrid system for this example is modeled in Simulink® and is based on the example “Detect Replay Attacks in DC Microgrids Using Distributed Watermarking” (Control System Toolbox). First load the system parameters using the `dcmgParameters` helper Script provided in the example folder.

```
dcmgParameters
```

Set the variables that enable (1) or disable (0) the different modes of the system, starting with all modes enabled.

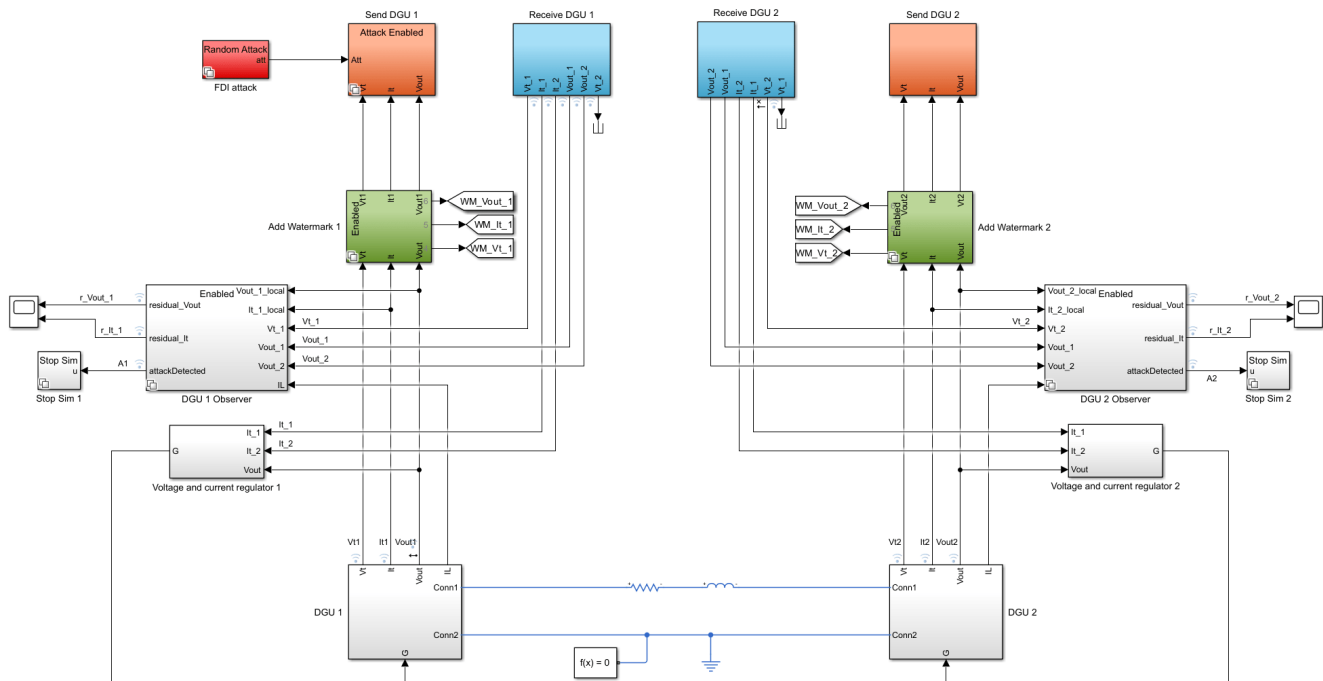
```
enableAttack = 1;
gaussianAttack = 1;
enableObserver = 1;
enableWatermarking = 1;
```

Specify the residual and attack limits $r_{\max}$ and $a_{\max}$.

```
rmax = 2.0;
amax = 5.0;
attackUpper = amax;
attackLower = -amax;
```

Open the Simulink model.

```
mdl = "rldcmicrogrid";
open_system(mdl);
```



Simulate Under Nominal Conditions

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To simulate the system under no attack, set enableAttack to 0.

```
enableAttack = 0;
```

Simulate the model.

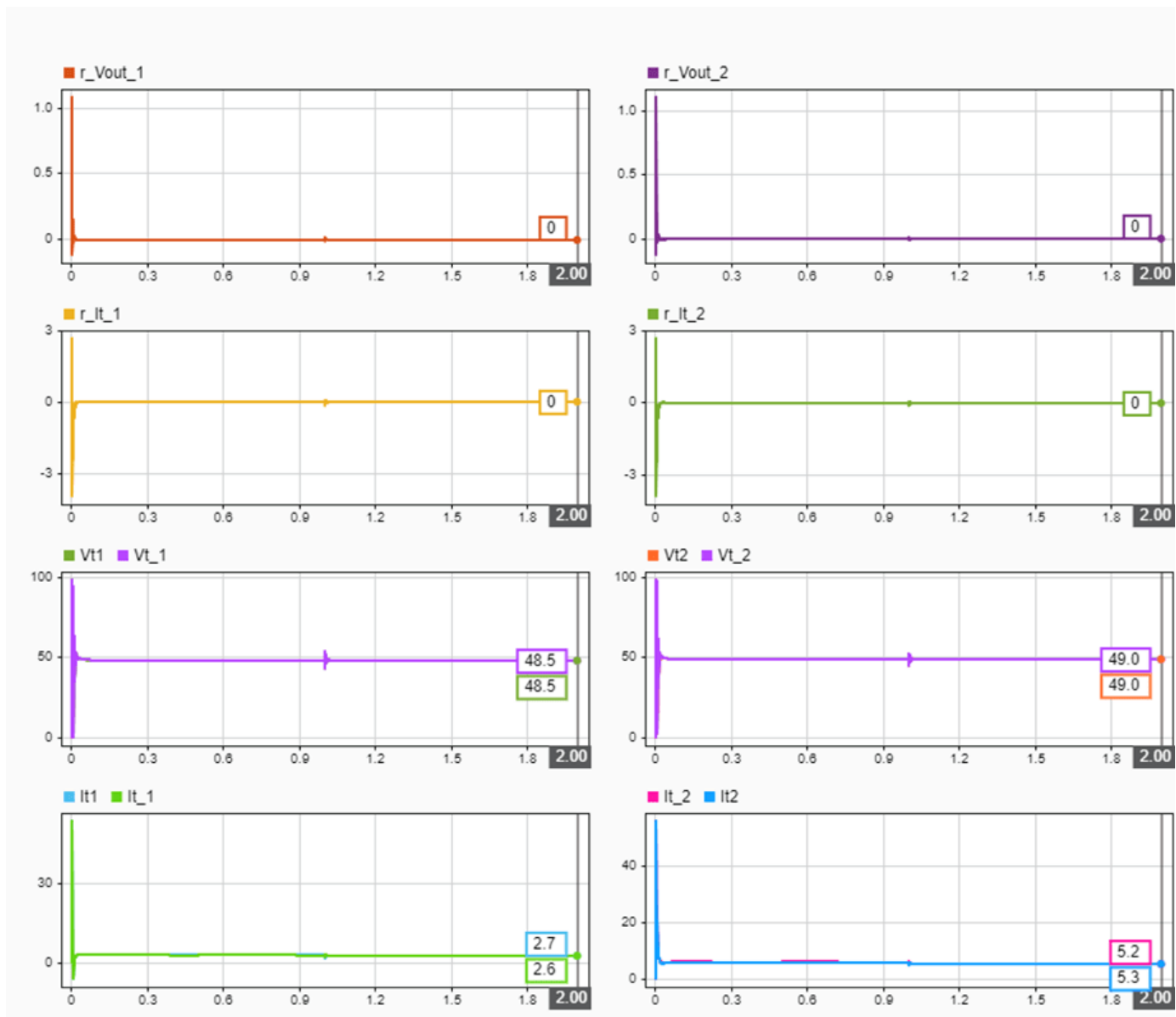
```
nominalOut = sim mdl;
```

Visualize the simulation data in the Simulation Data Inspector using the `plotInSDI` on page 8-398 helper function.

```
plotInSDI(enableAttack)
```

The plots below show the simulation of the several states of the system.

- `r_Vout_1` and `r_Vout_2` are the voltage residual values for the two DGUs as received over the communication network.
- `r_It_1` and `r_It_2` are the current residual values as received over the communication network.
- `Vt_1`, `Vt_2`, `It_1`, and `It_2` are the voltage and current values as received over the communication network.
- `Vt1`, `Vt2`, `It1`, and `It2` are the true voltage and current values.



The system operates as expected under the no attack condition. The voltages and currents satisfy the control requirements and the residual values are zero.

Simulate Gaussian Attack

Simulate the system with an attack model that injects a Gaussian noise into the sensor measurements (Vt_1 , It_1 and $Vout_1$) and observe the responses.

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

To simulate the Gaussian attack, set `enableAttack` and `gaussianAttack` to 1. The attack is saturated within the limits $\pm a_{\max}$.

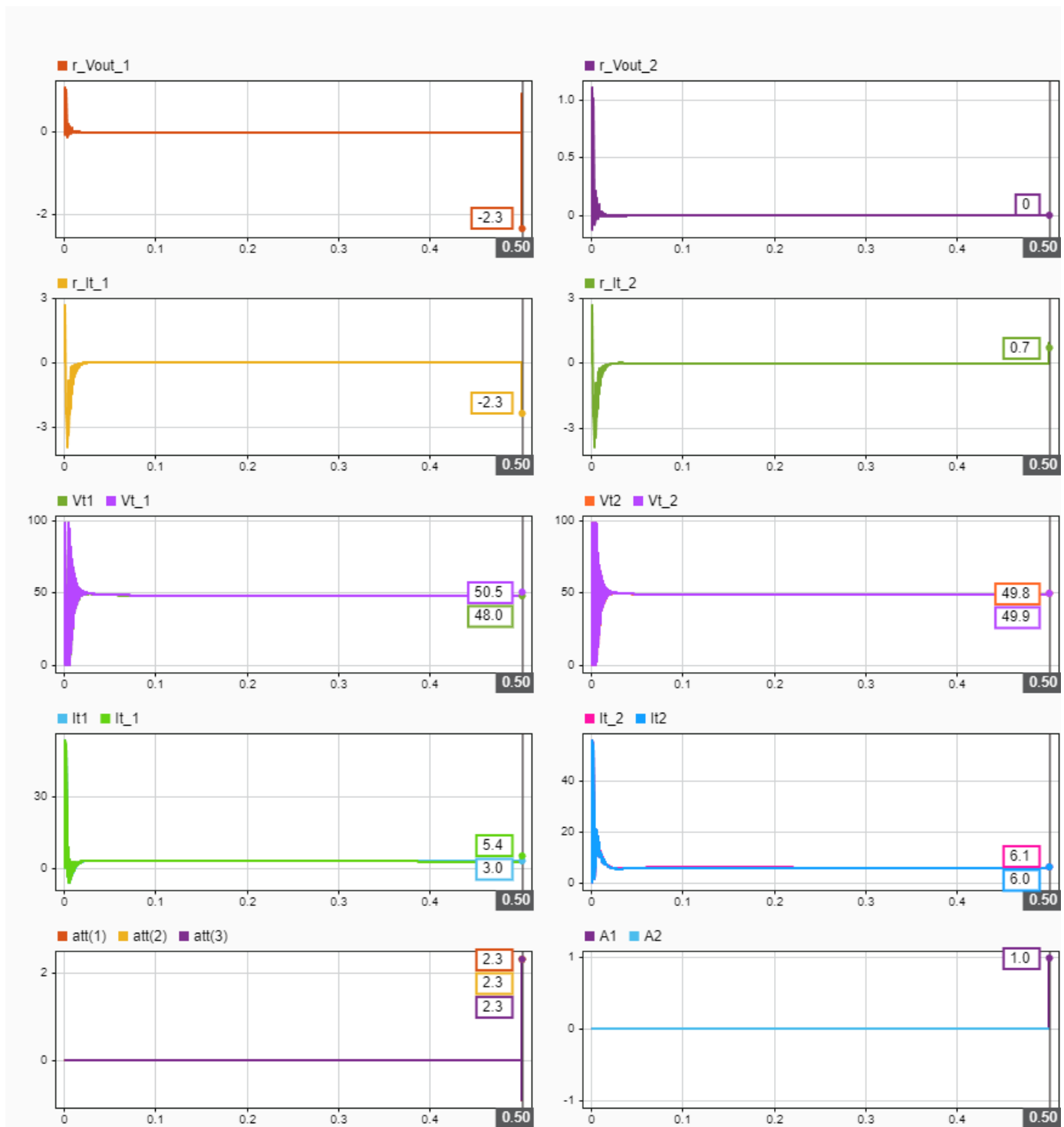
```
enableAttack = 1;  
gaussianAttack = 1;
```

Simulate the model.

```
gaussianOut = sim mdl;
```

Visualize the simulation data.

```
plotInSDI(enableAttack)
```

As shown in the plot, the observer for DGU 1 detects an attack at $t = 0.5$ seconds for the attack vector $a(t) = [3.9, 2.3, 2.3]$. At this time, the detection signals $A1$ and $A2$ are nonzero because the residual values r_Vout_1 and r_It_1 are greater than r_{max} .

Optimal Attack Problem

The optimal attack bypasses the security of the system while satisfying a set of constraints. The attack model represents the following scenario:

- The attack is performed on the data transmitted from the DGU 1 unit, while DGU 2 data remains unaffected (see the diagram of the system architecture earlier in this example).
- The attacker adds a disturbance to the true signals V_{t1} , I_{t1} and V_{out1} . This type of attack is known as **false data injection (FDI)**.
- The attack policy aims to find an attack sequence $S = [a_0, a_1, \dots, a_T]$ for some time horizon T that minimizes the line current I_{t1} while being undetected by the observers ($r_k \leq r_{\max}$). Minimizing the current is one of many possible attack objectives.
- The magnitude of the attack at any time step is kept small ($a_k \leq a_{\max}$) to avoid detection.

This scenario leads to the constrained optimization problem

$$\min_S \sum_{k=0}^T I_{t1,k}$$

$$s.t. r_k \leq r_{\max} \text{ and } a_k \leq a_{\max}$$

The corresponding unconstrained problem is

$$\max_S \sum_{k=0}^T -I_{t1,k} - W * D_k(r_k, a_k, r_{\max}, a_{\max})$$

where r_k, a_k are the observer residual and attack vectors at time k , and $W * D_k$ is a soft penalty for deviating from the allowable residual and attack limits. You solve this unconstrained problem using reinforcement learning.

Create Environment Object

The first step in solving the unconstrained problem is to create the environment object. The environment is the DC microgrid system under attack. For this environment:

- The observation vector $0.1 * [V_{t1}, I_{t1}, V_{out1}, V_{t2}, I_{t2}, V_{out2}, a_{last}]$ contains the true voltage and current values from the two DGU units, as well as the attack vector from the last time step scaled by the factor 0.1.

The action is a three-element vector representing the cyber-attack. The attacker adds the action vector to the $[V_{t1}, I_{t1}, V_{out1}]$ measurements during the transmission from the DGU 1 unit to the DGU 2 unit.

- The reward at time k is given by:

$R_k = -0.1 I_{t1,k} - 0.0001 D_k$, where D_k is a quadratic exterior penalty computed using the `exteriorPenalty` function.

Disable the Gaussian attack model.

```
gaussianAttack = 0;
```

Create the observation and action input specifications.

```
obsInfo = rlNumericSpec([9,1]);
obsInfo.Name = "observation";
```

```
actInfo = rlNumericSpec([3,1]);
actInfo.Name = "action";
```

Specify the action range $[-1, 1]$. The action is scaled up using `attackGain` in the Simulink model, and saturated within `[attackLower, attackUpper]`.

```
actInfo.UpperLimit = 1;
actInfo.LowerLimit = -1;
attackGain = [0.1;10;0.1];
```

Create the Simulink environment object.

```
blk = mdl + "/FDI attack/RL Attack/RL/RL Agent";
env = rlSimulinkEnv(mdl,blk,obsInfo,actInfo)
```

```
env =
SimulinkEnvWithAgent with properties:
```

```
    Model : rldcmicrogrid
  AgentBlock : rldcmicrogrid/FDI attack/RL Attack/RL/RL Agent
    ResetFcn : []
UseFastRestart : on
```

Specify a reset function to set the time of attack to 0 at the beginning of the training episodes, so that the agent starts learning when the attack is going on. For more information, see “Reset Function for Simulink Environments” on page 2-112.

```
env.ResetFcn = @(in) setVariable(in,"timeOfAttack",0);
```

Create TD3 Agent

The agent in this example is a twin-delayed deep deterministic policy gradient (TD3) agent. TD3 agents use two parameterized Q-value function approximators (that is, two critics) to estimate the value (that is, the expected cumulative long-term reward) of the policy. For more information on TD3 agents, see “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72.

Create an agent initialization object to initialize the default critic network with the hidden layer size 300.

```
initOpts = rlAgentInitializationOptions(NumHiddenUnit=300);
```

Specify the agent options for training using an `rlTD3AgentOptions` object. For this agent:

- Use an experience buffer with capacity $1e6$ to collect experiences. A larger capacity maintains a diverse set of experiences in the buffer.
- Use mini-batches of 300 experiences. Smaller mini-batches are computationally efficient but might introduce variance in training. By contrast, larger batch sizes can make the training stable but require higher memory.
- Specify the learning rate of $1e-5$ for the actor and critics. A smaller learning rate can improve the stability of training.

- Specify the initial standard deviation value of 0.2 for the agent's exploration model. The standard deviation decays at the rate of $1e-6$ until it reaches the minimum value of 0.01. This decay rate facilitates exploration toward the beginning and exploitation toward the end of training.
- The sample time is $T_s = 5e - 5$ seconds.

```
agentOpts = rlTD3AgentOptions( ...
    SampleTime=Ts, ...
    ExperienceBufferLength=1e6, ...
    MiniBatchSize=300);

agentOpts.ActorOptimizerOptions.LearnRate = 1e-5;
for i = 1:2
    agentOpts.CriticOptimizerOptions(i).LearnRate = 1e-5;
end

agentOpts.ExplorationModel.StandardDeviation = 0.2;
agentOpts.ExplorationModel.StandardDeviationMin = 0.01;
agentOpts.ExplorationModel.StandardDeviationDecayRate = 1e-6;
```

Create the TD3 agent using the observation and action input specifications, initialization options, and agent options. When you create the agent, the initial parameters of the critic network are initialized with random values. Fix the random number stream so that the agent is always initialized with the same parameter values.

```
rng(0,"twister");
agent = rlTD3Agent(obsInfo,actInfo,initOpts,agentOpts)
```

```
agent =
    rlTD3Agent with properties:

        ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]
        AgentOptions: [1x1 rl.option.rlTD3AgentOptions]
        UseExplorationPolicy: 0
        ObservationInfo: [1x1 rl.util.rlNumericSpec]
        ActionInfo: [1x1 rl.util.rlNumericSpec]
        SampleTime: 5.0000e-05
        UseGPUForLearning: 0
```

Train Agent

To train the agent, first specify the training options. For this example, use the following options:

- Run the training for 50 episodes, with each episode lasting a maximum of $\text{ceil}(0.1/T_s)$ time steps.
- Specify an averaging window length of 10 for the episode rewards.

For more information, see `rlTrainingOptions`.

```
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=50, ...
    MaxStepsPerEpisode=floor(0.1/Ts), ...
    ScoreAveragingWindowLength=10, ...
    StopTrainingCriteria="none");
```

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

Enable the attack, and disable the Gaussian attack model.

```
enableAttack = 1;  
gaussianAttack = 0;
```

Train the agent using the `train` function. Training this agent is a computationally intensive process that takes several minutes to complete. To save time while running this example, load a pretrained agent by setting `doTraining` to `false`. To train the agent yourself, set `doTraining` to `true`.

```
doTraining = ;  
if doTraining  
    % Train the agent.  
    trainResult = train(agent, env, trainOpts);  
else  
    % Load the pretrained agent for the example.  
    load("DCMicrogridAgent.mat", "agent");  
end
```

A snapshot of the training progress is shown below. You might expect different results due to randomness in training.



Simulate Learned Attack

Fix the random stream for reproducibility.

```
rng(0, "twister");
```

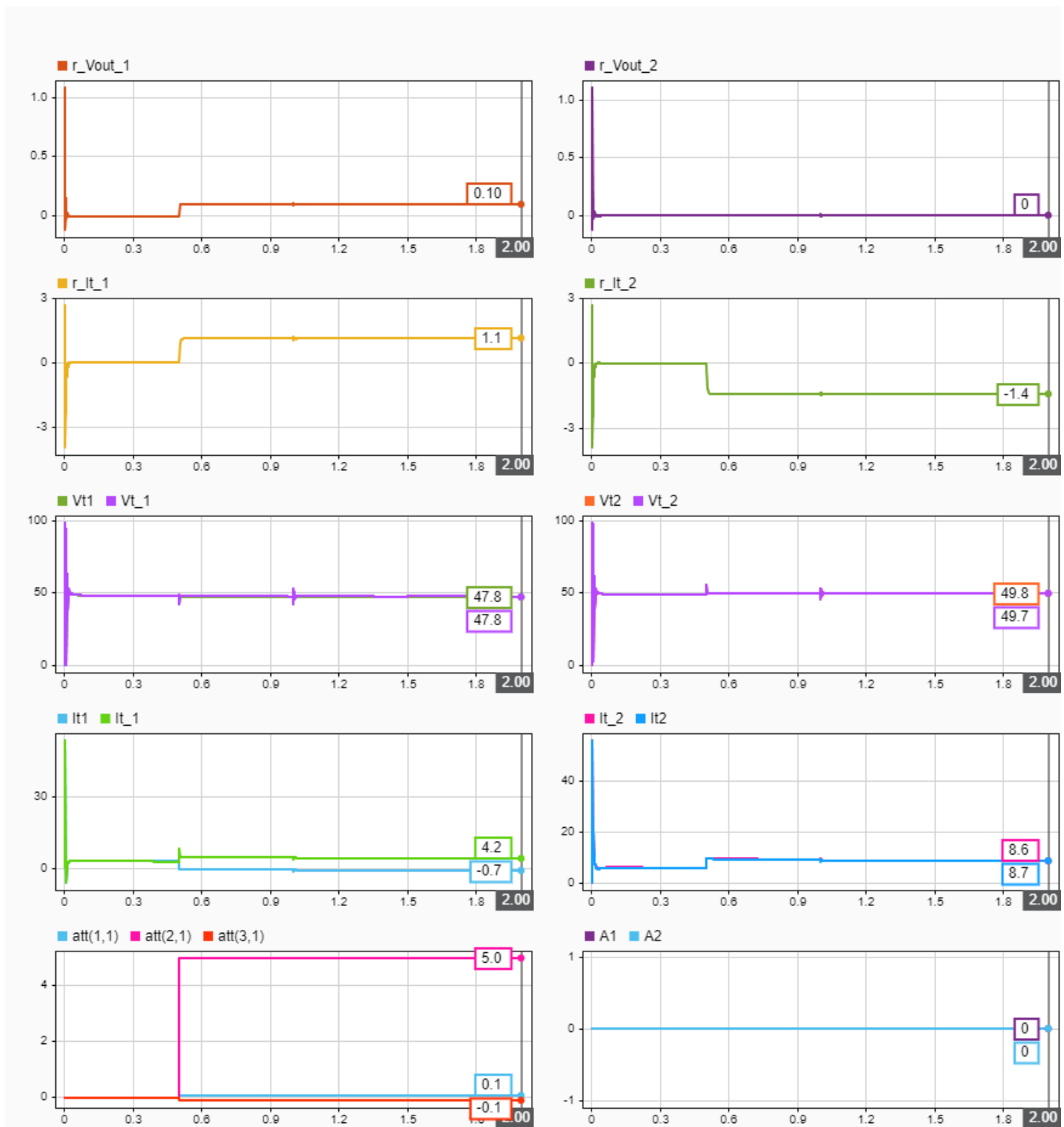
Simulate the model with the trained reinforcement learning agent.

```
enableAttack = 1;
gaussianAttack = 0;
optimalOut = sim mdl;
```

Visualize the simulation data.

```
plotInSDI(enableAttack)
```

The optimal attack signal is able to keep the residual values within the allowable range $\pm r_{\max}$, while creating impact on the current regulation. At 2.0 seconds, as a result of the attack $a(t) = [0.1, 5.0, -0.1]$, a false current (I_{t_1}) of 4.2 A is transmitted over the network while the true value (I_{t1}) is -0.7 A. To satisfy the current consensus rule, the DGU 2 unit compensates by generating a higher current (I_{t_2}) than required, creating an advantage for the DGU 1 unit over time.



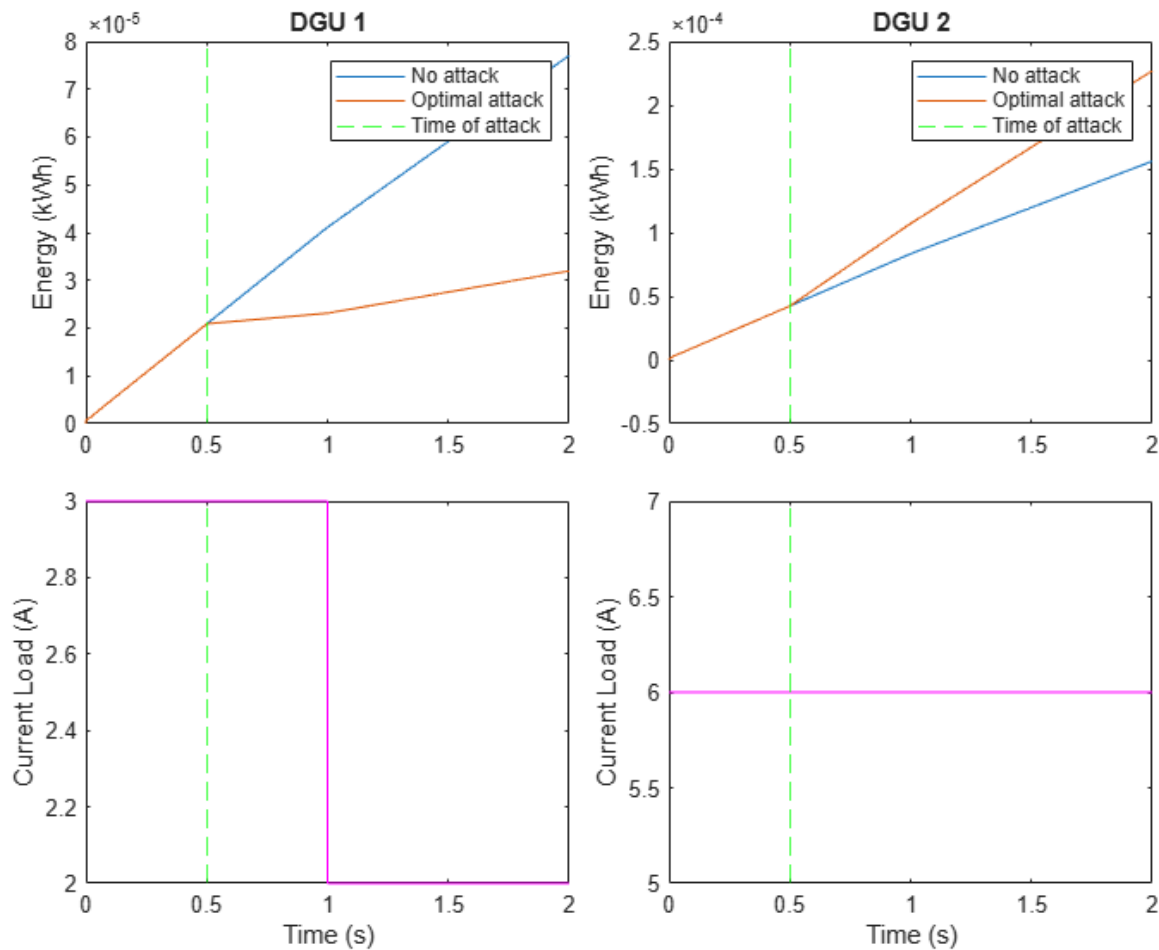
Analysis and Conclusion

Compare the energy generation of the DGUs for the nominal and optimal attack cases. First, extract the logged data from the simulations.

```
eNominal(1) = getElement(nominalOut.logout, "E1");
eNominal(2) = getElement(nominalOut.logout, "E2");
eOptimal(1) = getElement(optimalOut.logout, "E1");
eOptimal(2) = getElement(optimalOut.logout, "E2");
attOptimal = getElement(optimalOut.logout, "att");
refLoad(1) = getElement(optimalOut.logout, "IL1");
refLoad(2) = getElement(optimalOut.logout, "IL2");
```

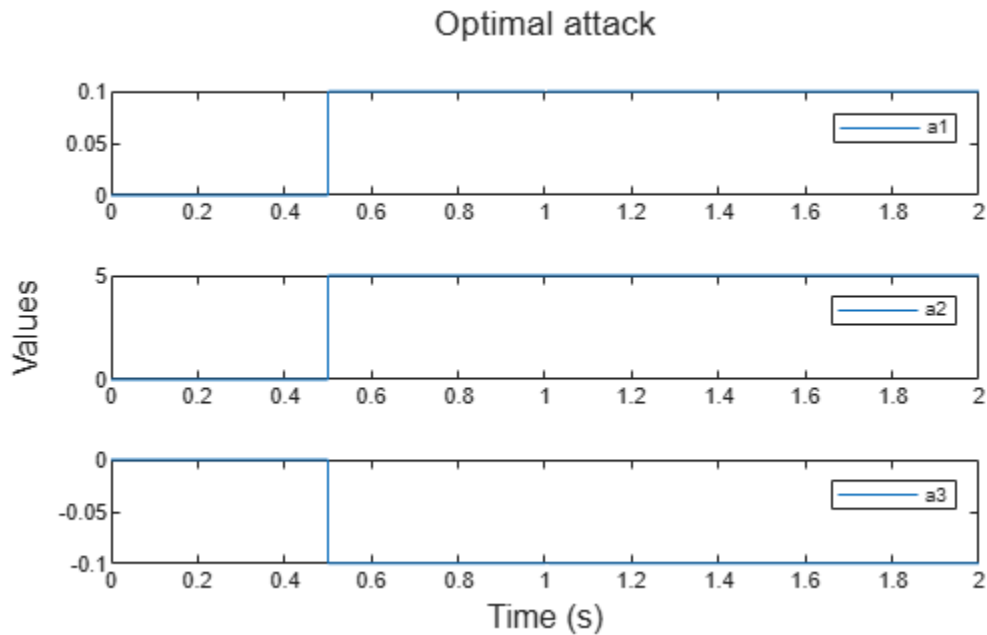
Plot the energy consumption in the two units. At the time of attack (0.5 seconds), the optimal attack policy reduces the energy generation in DGU 1 while simultaneously increasing the generation in DGU 2. The effect continues even after the load change at 1 second.

```
f = figure();
t = tiledlayout(f,2,2,TileSpacing="compact");
for i = 1:2
    ax = nexttile(t);
    plot(ax,eNominal(i).Values.Time,eNominal(i).Values.Data);
    hold(ax,"on");
    plot(ax,eOptimal(i).Values.Time,eOptimal(i).Values.Data);
    xline(ax,timeOfAttack,"g--");
    ylabel(ax,"Energy (kWh)");
    legend(ax,["No attack","Optimal attack","Time of attack"]);
    title(ax,sprintf("DGU %d",i));
end
for i = 1:2
    ax = nexttile(t);
    plot(ax,refLoad(i).Values.Time,refLoad(i).Values.Data(:),"m");
    hold(ax,"on");
    xline(ax,timeOfAttack,"g--");
    xlabel(ax,"Time (s)");
    ylabel(ax,"Current Load (A)");
end
f.Position(3:4) = [800,600];
```

Plot the optimal attack vector over time. The magnitude of attack on the V_{t1} and V_{out1} signals (a_1 and a_3) is low, while the attack on the I_{t1} signal (a_2) is large. The attacker bypasses observer-based security by freely modifying the I_{t1} signal. This observation can be used as a starting point to investigate and further improve the system. For instance, the observer design can be improved to consider any difference between the actual and transmitted values of the I_{ti} signal (out of the scope for this example).

```
h = figure();
t = tiledlayout(h,3,1,TileSpacing="compact");
for i = 1:3
    ax = nexttile(t);
    plot(ax,attOptimal.Values.Time, ...
        squeeze(attOptimal.Values.Data(i,:,:)));
    legend(ax,"a"+i);
end
title(t,"Optimal attack");
xlabel(t,"Time (s)");
ylabel(t,"Values");
```



Restore the random number stream using the information stored in `previousRngState`.

```
rng(previousRngState);
```

Helper Functions

```
function plotInSDI(enableAttack)
% Open and configure the SDI
Simulink.sdi.view;
Simulink.sdi.clearAllSubPlots;
% Generate subplots
if enableAttack
    Simulink.sdi.setSubPlotLayout(5,2);
else
    Simulink.sdi.setSubPlotLayout(4,2);
end
% Get the current run
run = Simulink.sdi.getCurrentSimulationRun("rldcmicrogrid");
% Subplot 1
plotOnSubPlot(getSignalsByName(run,"r_Vout_1"),1,1,true);
% Subplot 2
plotOnSubPlot(getSignalsByName(run,"r_Vout_2"),1,2,true);
% Subplot 3
plotOnSubPlot(getSignalsByName(run,"r_It_1"),2,1,true);
% Subplot 4
plotOnSubPlot(getSignalsByName(run,"r_It_2"),2,2,true);
% Subplot 5
plotOnSubPlot(getSignalsByName(run,"Vt1"),3,1,true);
plotOnSubPlot(getSignalsByName(run,"Vt_1"),3,1,true);
% Subplot 6
plotOnSubPlot(getSignalsByName(run,"Vt2"),3,2,true);
plotOnSubPlot(getSignalsByName(run,"Vt_2"),3,2,true);
% Subplot 7
plotOnSubPlot(getSignalsByName(run,"It1"),4,1,true);
```

```

plotOnSubPlot(getSignalsByName(run,"It_1"),4,1,true);
% Subplot 8
plotOnSubPlot(getSignalsByName(run,"It2"),4,2,true);
plotOnSubPlot(getSignalsByName(run,"It_2"),4,2,true);
% Subplots 9 and 10
if enableAttack
    % Subplot 9
    att = getSignalsByName(run,"att");
    plotOnSubPlot(att.Children(1),5,1,true);
    plotOnSubPlot(att.Children(2),5,1,true);
    plotOnSubPlot(att.Children(3),5,1,true);
    % Subplot 10
    plotOnSubPlot(getSignalsByName(run,"A1"),5,2,true);
    plotOnSubPlot(getSignalsByName(run,"A2"),5,2,true);
end
end

```

See Also

Functions

train | rlSimulinkEnv

Objects

rlTD3Agent | rlTD3AgentOptions | rlTrainingOptions | rlOptimizerOptions

Blocks

RL Agent

Related Examples

- “Detect Attack in Cyber-Physical Systems Using Dynamic Watermarking” (Control System Toolbox)
- “Detect Replay Attacks in DC Microgrids Using Distributed Watermarking” (Control System Toolbox)
- “Train TD3 Agent for PMSM Control” on page 8-242

More About

- “Create Actors, Critics, and Policy Objects” on page 4-2
- “Twin-Delayed Deep Deterministic (TD3) Policy Gradient Agent” on page 3-72
- “Train Reinforcement Learning Agents” on page 5-3

